The Recommender Systems Design Atlas

Production recommender architectures from across the industry

01. The Canonical Multi-Stage Recommender Architecture

Overview
1. Why a funnel at all
2. Netflix: the online / nearline / offline blueprint (2013)
3. YouTube: the two-stage deep network (2016)
4. Meta: Instagram Explore and Facebook Feed
5. X / Twitter: the open-sourced For You pipeline
6. Pinterest: candidate generators, a light-weight scorer and a blender
7. LinkedIn: first-pass rankers to a unified LLM-era stack
8. Alibaba, Taobao and Kuaishou: the explicit pre-ranking stage
9. The time axis: offline, nearline, online in modern stacks
10. Request flow, end to end
11. How each company draws it: comparison
12. Scale numbers at a glance
13. Recurring design tricks
Key takeaways for system designers
References

02. Candidate Generation and Retrieval: Two-Tower Models, Negative Sampling, ANN Indexes and Multi-Source Retrieval

Overview
1. Foundations: from extreme multiclass to two towers
2. Meta: Instagram Explore and Andromeda
3. Pinterest: learned retrieval, multi-embedding, graph retrievers and Manas
4. LinkedIn: GPU retrieval and LLM-based dual encoders
5. Twitter/X: a portfolio of graph and embedding retrievers
6. Snap: EBR for friend recommendation
7. Marketplaces: Uber Eats, Etsy, Airbnb and Amazon
8. Beyond dot products: multi-interest, tree, path and generative retrieval
9. ANN index families
10. Multi-source retrieval and merging
11. System-level comparison of production retrieval designs
Key takeaways for system designers
References

03. The Evolution of Ranking and CTR Models: From Logistic Regression to Scaling-Law Architectures

Overview
1. The Linear Era: Logistic Regression and FTRL-Proximal (Google, 2013)
2. Automating Feature Crosses: GBDT + LR at Facebook (2014)
3. Factorized Interactions: FM and FFM
4. Hybrid Deep Models: Memorization plus Generalization
5. User-Behavior Modeling: The Alibaba Line (DIN, DIEN, SIM) and Kuaishou TWIN
6. Scaling Era: Ensembles, Stacked FMs, and Foundation Rankers
7. Comparison Table
8. Feature-Interaction Design Choices
Key takeaways for system designers
References

04. Multi-Task and Multi-Objective Ranking: From Shared-Bottom to Value Models

Overview
1. Multi-task learning architectures
2. The value-model layer: how industry blends objectives
3. Calibration: the contract between the model and the value model
4. Training-time balancing: loss weights and gradient surgery
5. Pareto tuning and objective blending in practice

Key takeaways for system designers
References

05. Sequential and User-History Modeling: From GRU4Rec to Lifelong Sequences and Foundation User Models

Overview
1. Academic foundations: next-item prediction
2. Target attention in industrial rankers: DIN, DIEN and BST (Alibaba)
3. Long and lifelong sequences in ranking
4. Pinterest: real-time, batch and lifelong user sequences
5. Kuaishou: sequence models for retrieval
6. Generative recommenders and foundation user models
7. Comparison table
8. Recurring design patterns
9. Key takeaways for system designers
References

06. Generative Recommenders (2023-2026): HSTU, Semantic IDs, OneRec, MTGR and the Foundation-Model Turn

1. The conceptual shift: from DLRM to generative recommenders
2. Meta HSTU: "Actions Speak Louder than Words"
3. Semantic IDs and TIGER (Google)
4. Kuaishou OneRec family
5. Meituan MTGR (and MTGRBoost, DFGR, EGA)
6. Other generative ranking systems worth knowing
7. Netflix: Foundation Model for Personalized Recommendation
8. LinkedIn 360Brew
9. Pinterest PinFM (and PinRec, UniPinRec)
10. X (Twitter) For You: Phoenix transformer ranker
11. Scaling laws for recommendation
12. Comparison table
Key takeaways for system designers
References

07. Graph-Based Recommendation in Production: Random Walks, Graph Embeddings and GNNs at Web Scale

1. A taxonomy of graph recommenders in production
2. Pinterest Pixie: real-time random walks on a 3-billion-node graph
3. Pinterest PinSage: GraphSAGE at 3 billion nodes
4. Twitter/X: a full graph recommendation stack
5. LinkedIn LiGNN: production GNNs on a temporal, heterogeneous graph
6. Uber Eats: GraphSAGE for dish and restaurant recommendations
7. Alibaba: session graphs, side information and GNN platforms
8. Meta PyTorch-BigGraph: the shallow-embedding workhorse
9. Cross-system comparison
10. Recurring design patterns
11. Key takeaways for system designers
References

08. Feature Engineering and Feature Stores: Point-in-Time Correctness, Streaming Features, and Serving-Time Logging

1. Why recommenders need a feature platform
2. Core concepts and vocabulary
3. Twitter: the aggregation framework (batch + real-time counters)
4. Uber Michelangelo and Palette
5. Airbnb Zipline to Chronon
6. LinkedIn: Feathr (and Venice as the serving substrate)
7. Feast: the open-source reference design

8. Tecton: managed real-time feature platform
9. Hopsworks: FTI pipelines and a transformation taxonomy
10. Featureform: the virtual feature store
11. Netflix, DoorDash and others (secondary pointers)
12. Cross-cutting design patterns
13. Comparison of public feature platforms
Key takeaways for system designers
References

09. Training Infrastructure at Scale: Sparse Embeddings, Sharding, Parameter Servers and Online Learning

Overview
1. The Shape of the Problem: DLRM as Reference Workload
2. Meta: Neo, ZionEX, TorchRec and FBGEMM
3. ByteDance Monolith: Collisionless Tables and Real-Time Training
4. Parameter-Server Lineage: XDL, DeepRec, PERSIA, HET, OpenEmbedding, TFRA
5. NVIDIA HugeCTR, Merlin and HierarchicalKV
6. Continual / Incremental Training and Checkpointing
7. Hashing Tricks and Embedding Compression
8. Hardware Direction: Accelerating Sparse Lookups
9. Cross-System Comparison
10. Key Takeaways for System Designers
References

10. Online Serving and Inference: Latency Budgets, Model Servers, Caches and Cascades

10.1 The shape of the problem: funnels and latency budgets
10.2 X / Twitter: an open-source reference serving stack
10.3 Meta: Instagram Explore and the Andromeda ads retrieval engine
10.4 GPU inference for ranking: Pinterest and the general recipe
10.5 Search-engine-based serving: Vespa and Elasticsearch
10.6 Generative recommenders at serving time: HSTU and M-FALCON
10.7 Embedding serving and caching
10.8 Feature fetching fan-out
10.9 Making models cheaper: cascades, early exit, distillation, quantization
10.10 Comparison of serving designs
10.11 Key takeaways for system designers
References

11. Training Data and Labeling: Impression Logs, Label Joins, Delayed Feedback, Negatives and Pipelines

Overview
1. Impression logging and the label join
2. Delayed feedback and conversion attribution
3. Negative sampling strategies
4. Label definitions and implicit feedback
5. Sample weighting
6. Dataset freshness, retraining cadence and leakage
7. Data pipelines at scale
8. Key takeaways for system designers
References

12. Evaluation and Experimentation: Offline Metrics, Off-Policy Evaluation, A/B Platforms, Interleaving and Long-Term Measurement

Overview
12.1 Offline metrics
12.2 Counterfactual and off-policy evaluation
12.3 Interleaving

12.4 A/B testing platforms
12.5 Long-term metrics, surrogates and holdouts
12.6 Putting it together: a reference evaluation architecture
12.7 Case summaries at a glance
Key takeaways for system designers
References

13. Exploration, Bandits and Reinforcement Learning in Recommendation

1. The problem: closed loops and missing counterfactuals
2. Yahoo! Today Module: LinUCB and replay evaluation (2010-2011)
3. Thompson sampling: Yahoo display ads, news, and Bing adPredictor
4. Microsoft Decision Service and MSN (2016-2017)
5. Netflix: artwork personalization with contextual bandits (2016-2017)
6. Spotify: BaRT and the home page (2018) and impatient bandits (2023)
7. Deezer carousels and Amazon page layouts: slate bandits in practice
8. YouTube: top-K off-policy corrected REINFORCE (2019)
9. SlateQ: tractable value-based RL for slates (Google, 2019)
10. Long-term value, retention and notifications
11. Off-policy evaluation: the shared foundation
12. Comparison of production designs
13. Recurring design patterns
14. Key takeaways for system designers
References

14. Debiasing, Diversity, Fairness and Cold Start: The Corrective Layers of Production Recommenders

Overview
14.1 A Taxonomy of Bias in Recommender Logs
14.2 Position Bias
14.3 Popularity, Exposure and Sampling Bias
14.4 Diversity: From MMR to Learned Listwise Rankers
14.5 Calibrated Recommendations (Netflix)
14.6 Cold Start
14.7 Fairness for Creators, Sellers and Users
14.8 Putting It Together: A Reference Corrective Stack
Key Takeaways for System Designers
References

15. Re-ranking, Blending and Whole-Page Optimization

1. Why a separate re-ranking stage exists
2. Rule-based and heuristic re-ranking
3. Listwise contextual re-rankers
4. Generative and generator-evaluator re-ranking
5. Slate RL and long-horizon reranking
6. Heterogeneous feed blending
7. Whole-page optimization
8. The business-rules and constraints layer
9. Comparison of re-ranking designs
10. Evaluating re-rankers
11. Key takeaways for system designers
References

16. Ads Recommendation Systems: pCTR/pCVR Prediction, Auctions, Pacing, Calibration and the Meta, Google, Pinterest, Snap and Twitter Ads Stacks

Overview
1. Foundations: pCTR, pCVR and the expected-value score
2. Google: the linear era and the plumbing that made it work
3. Facebook/Meta: from GBDT+LR to foundation models

4. Conversion modeling: delayed and biased labels
5. Calibration: the contract with the auction
6. Pinterest, Snap, Twitter/X, LinkedIn and others
7. Auctions: allocation and pricing
8. Budgets, pacing and auto-bidding
9. Ad load and allocation against organic content
10. The 2025-2026 scaling wave in ads ranking
Key takeaways for system designers
References

17. Google & YouTube Case Studies: From Co-Visitation Graphs to Semantic IDs and LLM-Native Recommenders

Overview
17.1 Google News personalization (2007, 2010)
17.2 The YouTube video recommendation system (2010)
17.3 Wide & Deep Learning for Google Play (2016)
17.4 Deep Neural Networks for YouTube Recommendations (2016)
17.5 Two-tower retrieval with sampling-bias correction (2019)
17.6 Recommending What Video to Watch Next: the multitask ranker (2019)
17.7 Reinforcement learning at YouTube: REINFORCE and SlateQ (2019)
17.8 Diversity, fairness and the "valued watch time" objective
17.9 TIGER: generative retrieval with semantic IDs (NeurIPS 2023)
17.10 The LLM / Gemini era: PLUM and user-journey modeling (2023-2026)
17.11 Infrastructure threads
17.12 Comparison table
Key takeaways for system designers
References

18. Netflix Case Study: From the Netflix Prize to a Foundation Model for Personalization

1. The Netflix Prize (2006-2009) and what Netflix actually took from it
2. The Netflix recommender as a set of algorithms (TMIS 2015)
3. System architecture: online, nearline, offline (2013)
4. Page generation and row selection
5. Artwork personalization
6. Search
7. Evaluation: offline, interleaving, A/B, and member-level metrics
8. Consolidation: from many models to few
9. Foundation model for personalization (2025)
10. ML infrastructure: Metaflow and Maestro
11. Comparison of Netflix recommender designs across eras
12. Key takeaways for system designers
References

19. Meta Case Study: From News Feed Ranking to Andromeda, Lattice, GEM and Generative Recommenders

1. Facebook News Feed ranking
2. Instagram Explore
3. DLRM and the infrastructure it created
4. Ads ranking: from GBDT+LR to DHEN, Wukong, InterFormer and Lattice
5. Andromeda: deep ads retrieval on GPUs (December 2024)
6. GEM: the Generative Ads Recommendation Model (2025-2026)
7. HSTU and generative recommenders (2024-2026)
8. Comparing Meta's systems
9. Surfaces and systems not verified in this run
10. Key takeaways for system designers
References

20. TikTok/ByteDance and Kuaishou: Real-Time Short-Video Recommendation, from Monolith to OneRec

20.1 The short-video recommendation problem
20.2 ByteDance / TikTok: what the company has disclosed
20.3 Monolith: real-time training with collisionless embeddings (ByteDance, 2022)
20.4 ByteDance retrieval: Deep Retrieval and LLM-based item/user models
20.5 ByteDance ranking: long sequences and hardware-efficient scaling
20.6 Kuaishou: the cascade, quantified
20.7 Kuaishou lifelong user modelling: TWIN and TWIN V2
20.8 KuaiFormer: Transformer retrieval (2024)
20.9 OneRec: collapsing the cascade into one generative model
20.10 Comparison
20.11 Cross-cutting design patterns
Key takeaways for system designers
References

21. Twitter/X Case Study: From WTF and GraphJet to the 2023 the-algorithm and the 2026 Phoenix Transformer

21.1 Pre-history: Who-to-Follow, Cassovary and RealGraph
21.2 GraphJet: Real-Time Collaborative Filtering on a Single Server
21.3 Representation Layer: SimClusters and TwHIN
21.4 The 2023 For You Pipeline (the-algorithm)
21.5 Why X rebuilt it: limits of the 2023 design
21.6 The 2026 xAI Rebuild: x-algorithm and Phoenix
21.7 Visibility and Trust & Safety in 2026
21.8 Side-by-side: 2013 to 2026
21.9 Key takeaways for system designers
References

22. Pinterest Case Study: From Pixie Random Walks to PinFM Foundation Models

1. The home feed funnel at a glance
2. Pixie: real-time random walks on a pruned bipartite graph (2017-2018)
3. PinSage: graph convolution at 3 billion nodes (2018)
4. Multi-graph and multi-entity successors
5. PinnerFormer: a daily batch user embedding trained for the long term (KDD 2022)
6. Learned retrieval and the Manas ANN platform (2022-2026)
7. Pre-ranking: the Light-Weight Scorer
8. Ranking: Pinnability and its sequence modules
9. Utility blending and long-term objectives
10. GPU serving for ranking
11. Pinterest ads ranking
12. Related Pins
13. System-level comparison
Key takeaways for system designers
References

23. LinkedIn Case Study: Feed, PYMK, Jobs, LiRank, LiGNN, 360Brew and the ML Platform Beneath Them

23.1 The shape of LinkedIn's recommendation problems
23.2 The statistical era: GLMix, Photon ML and GDMix
23.3 Feature, storage and search substrate: Feathr, Venice, Galene
23.4 Feed ranking: from federated FPR/SPR to multi-objective autotuning
23.5 LiRank: industrial large-scale ranking (KDD 2024)
23.6 Retrieval and graph learning: LiNR and LiGNN
23.7 People You May Know (PYMK)
23.8 Job recommendations and the Jobs marketplace
23.9 360Brew: a decoder-only foundation model for ranking (2025)

23.10 Experimentation and responsible ranking
23.11 Comparison of LinkedIn's published recommender systems
23.12 Key takeaways for system designers
References

24. Amazon and E-commerce: From Item-to-Item Collaborative Filtering to Query-Aware Retrieval

1. Amazon item-to-item collaborative filtering (1998-2003)
2. Amazon's deep-learning recommendation stack
3. Amazon product search as a recommender
4. Amazon Personalize: Amazon's recommender as a managed service
5. Embedding lineage for product recommendations
6. eBay: recommendations without a stable catalog
7. Etsy: unified embedding-based personalized retrieval
8. Comparison of the major e-commerce designs
9. Other retailers: Wayfair, Shopify, Walmart, Target, Instacart
10. A reference architecture for e-commerce recommendation
11. Key takeaways for system designers
References

25. Spotify and Audio Recommendation: Discover Weekly, BaRT, Podcasts, AI DJ, LLM Playlists, Deezer Flow, Pandora, and the Music Streaming Stack

1. Spotify's foundation layer: implicit collaborative filtering and embeddings
2. Spotify Home: BaRT and explainable bandit recommendations
3. User representations and sequence models at Spotify
4. Podcasts and audiobooks: cold-start domains with graph models
5. The LLM era: AI DJ, AI Playlist and generative retrieval
6. Deezer: Flow, cold-start onboarding and carousel bandits
7. Pandora: the Music Genome Project and thumbs-based radio
8. YouTube Music, Apple Music and SoundCloud
9. Cross-company comparison
10. Recurring architectural patterns in audio recommendation
Key takeaways for system designers
References

26. Marketplace and Travel Recommenders: Airbnb, Booking.com, Uber Eats, DoorDash and Others

26.1 What makes marketplace recommenders different
26.2 Airbnb search ranking
26.3 Booking.com: 150 models and 6 lessons
26.4 Expedia and the hotel-ranking problem
26.5 Uber Eats
26.6 DoorDash
26.7 Lyft
26.8 Zillow, Yelp and Instacart: patterns only
26.9 Cross-company comparison
26.10 Recurring design patterns
26.11 Key takeaways for system designers
References

27. The Chinese Ecosystem: Alibaba, Tencent, Meituan, Baidu, Huawei, JD, Xiaohongshu and the Architecture of Hyperscale Commerce and Content

Overview
1. Alibaba: the Alimama and Taobao line
2. Tencent: WeChat, Tencent Video and Tencent Ads
3. Meituan: local services, food delivery and ads
4. Huawei Noah's Ark: feature interactions, bias and benchmarks
5. Baidu, JD.com, Xiaohongshu, Weibo and Pinduoduo

6. Comparison of the major designs
7. Architecture patterns characteristic of Chinese hyperscale
Key takeaways for system designers
References

28. News, Social and Content Platforms: From Click Co-visitation to Open Feed Protocols

Overview
28.1 Google News: Scalable Online Collaborative Filtering (2007) and Click-Behavior Profiles (2010)
28.2 Yahoo! JAPAN: Embedding-Based News Recommendation for Millions of Users (KDD 2017)
28.3 Microsoft News and the MIND Ecosystem
28.4 The New York Times: Collaborative Topic Modeling
28.5 Reddit: Closed-Form Ranking at the Core
28.6 Bluesky: Recommendation as an Open Protocol Service
28.7 Mastodon: Chronological Fan-Out and Trend Detection
28.8 Snapchat: Embedding-Based Retrieval, GNNs and Generative Recommendation
28.9 Streaming and Long-Tail Content Platforms: Hulu, Disney+, Roku, Twitch
28.10 Quora, Medium, Substack and Discord
28.11 Cross-Platform Comparison
Key Takeaways for System Designers
References

29. LLMs in Recommender Systems (2023-2026): Rankers, Features, Agents, Distillation and Production Deployments

29.1 A taxonomy: where and how the LLM is used
29.2 LLM as the recommender: the 2022-2024 academic foundations
29.3 LLM-generated features, knowledge and embeddings (the most deployed pattern)
29.4 LLM-native generative retrieval: semantic IDs meet pre-trained LLMs
29.5 LLM as the ranker in production: LinkedIn 360Brew (2025)
29.6 Conversational and agentic recommenders
29.7 Knowledge distillation from LLMs
29.8 Industry deployments: what is publicly documented
29.9 Deployments we could not verify in this session
29.10 Cost and latency trade-offs
29.11 Key takeaways for system designers
References

30. Scaling Laws and Hardware for Recommenders: Compute, Embeddings, Accelerators and Cost

Overview
1. Why recommender compute is different
2. Scaling laws for classic DLRMs: data, parameters, compute
3. Dense-compute scaling laws (2024-2026)
4. Embedding compute versus dense compute: systems consequences
5. Google TPU SparseCore: embeddings in silicon
6. Meta MTIA: a recommendation-first accelerator
7. GPU versus CPU (and accelerator) trade-offs
8. Power, cost and datacenter footprint
9. Putting it together: a reference scaling stack (2026)
Key takeaways for system designers
References

31. Search Meets Recommendation: Learning-to-Rank, Personalized and Semantic Search, Hybrid Retrieval and Unified Models

31.1 The common skeleton
31.2 Learning-to-rank: RankNet, LambdaRank, LambdaMART
31.3 Query understanding and semantic matching
31.4 Embedding-based retrieval in production search

31.5 Hybrid lexical plus vector retrieval
31.6 Personalized search
31.7 Unified search and recommendation
31.8 Reference architecture: a unified discovery stack
31.9 Key takeaways for system designers
References

32. Notification, Push and Email Recommendation: Volume, Fatigue and Timing
Overview
32.1 The canonical notification recommendation pipeline
32.2 X / Twitter: `pushservice` (open-sourced 2023)
32.3 Twitter: "Should I send this notification?" (modeling the future)
32.4 LinkedIn
32.5 Pinterest: notification volume control and optimization
32.6 Duolingo: sleeping, recovering bandits for practice reminders
32.7 Meta: Facebook and Instagram notifications
32.8 Fatigue modeling: patterns across systems
32.9 Send-time optimization (STO)
32.10 Email recommendation specifics
32.11 Comparison of published designs
Key takeaways for system designers
References

33. Content Understanding and Integrity in Recommender Systems: Multimodal Item Embeddings, Integrity Filters, Borderline Demotion, Spam and the Policy Layer
33.1 Where content understanding and integrity sit in the stack
33.2 Multimodal item embeddings
33.3 Integrity filters inside the funnel
33.4 Borderline content demotion
33.5 Harmful-content classifiers that serve recommendation
33.6 Spam, fraud and manipulation in recommendations
33.7 Responsible recommendation and the policy layer
33.8 Side-by-side: integrity mechanisms by company
Key takeaways for system designers
References

34. Open-Source Frameworks, Tooling, Benchmarks and Public Datasets
34.1 Industrial-scale training frameworks
34.2 Research and prototyping libraries
34.3 Retrieval indexes: ANN libraries and vector databases
34.4 Feature stores and ML platforms
34.5 Public datasets
34.6 Benchmarking practice
34.7 Key takeaways for system designers
References

35. The Recommender System Design Interview Playbook: Feeds, PYMK, Ads, E-commerce and Notifications
Overview
35.1 The answer framework
35.2 Back-of-envelope numbers
35.3 Design 1: YouTube / TikTok video recommendation feed
35.4 Design 2: Social news feed (Facebook, LinkedIn, X)
35.5 Design 3: People You May Know (friend / follow / connection recommendation)
35.6 Design 4: Ad click prediction (pCTR) and ads ranking
35.7 Design 5: E-commerce recommendations
35.8 Design 6: Notification and push recommendation
35.9 Cross-design comparison

35.10 Common follow-up questions and strong answers
35.11 Anti-patterns interviewers flag
Key takeaways for system designers
References

36. Classic Algorithms Foundations: Neighborhoods, Factorization, Linear Autoencoders, Embeddings, and Why Baselines Still Win

Overview
1. Neighborhood collaborative filtering
2. Matrix factorization for explicit feedback: Funk-SVD, SVD++, timeSVD++
3. Implicit-feedback MF: confidence-weighted ALS (Hu, Koren, Volinsky)
4. Pairwise learning to rank: BPR and WARP
5. Sparse and dense linear item-item models: SLIM and EASE
6. Word2vec-style item embeddings: item2vec, prod2vec, Airbnb
7. Content-based and hybrid recommenders
8. The reproducibility crisis: why classic baselines still matter
9. Comparison: classic algorithms as a system designer sees them
10. Reference architecture: classic models in a modern pipeline
Key takeaways for system designers
References

37. Real-Time and Session-Based Recommendation: Streaming Updates, Online Learning, Nearline Pipelines and In-Session Intent

37.1 The landscape: where freshness lives
37.2 Event backbone: a canonical action stream
37.3 Streaming features: counters, windows and tiles
37.4 Real-time candidate generation
37.5 Real-time user intent inside the ranker
37.6 Session-based models: the academic line
37.7 Online learning: streaming labels and continual training
37.8 TikTok and the short-video case
37.9 Freshness versus stability
37.10 Comparison of public real-time designs
37.11 Key takeaways for system designers
References

01. The Canonical Multi-Stage Recommender Architecture

Overview

Almost every large recommender system that has been publicly described, including YouTube, Instagram, Facebook, X/Twitter, Pinterest, LinkedIn, Taobao, Kuaishou and Netflix, has the same basic shape. It is a funnel of progressively more expensive models, each applied to a smaller candidate set, sitting on top of a split between offline, nearline and online computation. The funnel exists because of one constraint. You cannot run your best model over a corpus of millions to billions of items for every request within a latency budget of tens to a few hundred milliseconds. The layering exists because of a second constraint. Some computation, such as training on months of logs, cannot happen at request time, and some, such as reacting to the click a user made two seconds ago, cannot wait for a nightly batch job.

This chapter documents the canonical architecture as practitioners have drawn it, from Netflix's 2013 "online / nearline / offline" blueprint [1] and YouTube's 2016 two-stage deep network [3] through Instagram's four-stage Explore funnel [7][8][9], X's open-sourced For You pipeline [11][12], Alibaba's pre-ranking work [19][20] and Kuaishou's multi-stage cascade [22][23]. It ends with the 2024-2026 push to collapse the funnel into one generative model (Kuaishou OneRec [25][26], LinkedIn's LLM retrieval and generative ranker [18], Meta's GEM [28], Netflix's foundation model [29]).

The canonical stages, using the vocabulary that most companies converge on:

1. Retrieval / candidate generation / matching / recall: many heterogeneous sources reduce a corpus of 10^6 to 10^10 items to roughly 10^3 to 10^4 candidates.
2. Pre-ranking / first-stage ranking / light ranking / early-stage ranking (ESR) / coarse ranking: a cheap learned model, often a two-tower network or a distilled one, cuts thousands down to hundreds.
3. Ranking / second-stage ranking / heavy ranking / late-stage ranking (LSR) / fine ranking: a large multi-task neural network with rich user-item cross features predicts several engagement probabilities for each surviving candidate.
4. Re-ranking / blending / ordering / final pass: business rules, integrity filters, diversity, freshness, ad insertion, de-duplication and listwise optimization produce the final page.

1. Why a funnel at all

1.1 The cost argument

Google's public recommendation-systems course gives the canonical three-box version: candidate generation, scoring and re-ranking [4]. Its description is the clearest one-line

rationale available. The candidate generator "reduces billions of videos down to hundreds or thousands" for YouTube, so it must be fast. The scoring model then ranks a smaller pool and picks the set to show, "on the order of 10 items," and can afford to be more precise [4]. The course also makes a point that is often forgotten in system-design interviews. When a system has multiple candidate generators, their scores are not comparable, so a separate scoring model is required rather than simply merging retrieval scores [4].

Eugene Yan's widely cited essay "System Design for Recommendations and Search" draws the same structure as a 2x2 grid. One axis is offline versus online, and the other is candidate retrieval versus ranking [5]. Offline, you train models, compute item embeddings and build approximate nearest neighbour (ANN) indices. Online, you embed the user or query, retrieve candidates from the index, then rank the few hundred survivors with a heavier model that can use richer features [5][32].

```
              RETRIEVAL (recall, fast, coarse)    RANKING (precision, slow, rich)
            +-----------------------------------+-----------------------------------+
OFFLINE     | train embedding / two-tower       | train ranking model on logged     |
(batch)     | compute item vectors              | impressions + labels              |
            | build ANN / i2i / graph indices   | build feature tables / store      |
            +-----------------------------------+-----------------------------------+
ONLINE      | embed user/query, ANN lookup,     | hydrate features for ~10^2-10^3   |
(request)   | graph walks, rules, merge         | candidates, score, re-rank, serve |
            +-----------------------------------+-----------------------------------+
                      (after Yan, 2021 [5])
```

NVIDIA's Merlin team, citing Even Oldridge's article "Recommender Systems, Not Just Recommender Models," formalises a four-stage pattern: retrieval, filtering, scoring and ordering [6]. Retrieval is "fast but coarse-grained," with efficiency taking priority over precision. Filtering removes items the model would struggle to exclude, such as already-consumed or out-of-stock items. Scoring estimates interest. Ordering aligns the output with business needs and constraints [6]. The explicit filtering stage reflects a practical lesson. Hard constraints belong in deterministic code, not in a model you hope learned them.

1.2 The recall-precision hand-off

Every stage hands off to the next under a different objective. Early stages are judged on recall, meaning whether the eventual winner survived. Late stages are judged on precision and calibrated prediction. Pinterest's Related Pins paper describes this shift explicitly: once machine-learned ranking was introduced, candidate generation moved from optimising precision to optimising recall (summarised in [14]). Uber Eats likewise states that its first-pass ranker must achieve low latency "while maintaining a high recall metric" [30].

The hand-off creates the main pathology of cascades, which is stage inconsistency. Each stage is trained on a different sample distribution. The ranker is usually trained only on exposed items, but it scores items selected by a pre-ranker, which in turn was trained on whatever the ranker let through. Kuaishou's RecFlow dataset exists precisely to expose this. It logs not only exposures but also unexposed items filtered at each stage: 1.9 billion stage samples from 9.3 million online requests over 37 days, spanning 6 stages [22].

2. Netflix: the online / nearline / offline blueprint (2013)

2.1 Problem

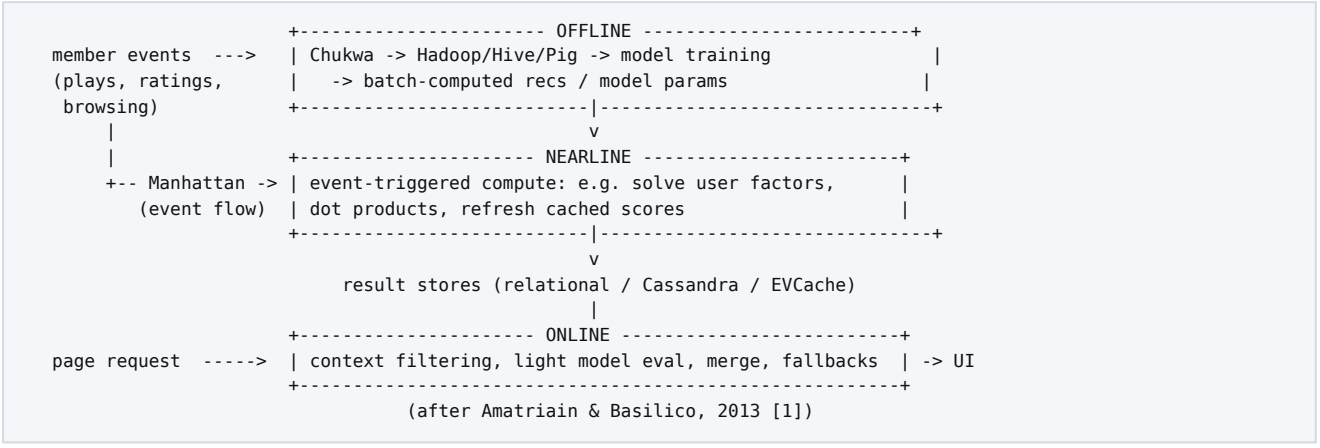
In March 2013, Xavier Amatriain and Justin Basilico published "System Architectures for Personalization and Recommendation" on the Netflix Tech Blog [1]. This was the first widely read reference architecture that separated where computation happens from what is computed. Netflix had to combine large batch-trained models (matrix factorisation and later ensembles, see also [34]) with responsiveness to what a member just played, under strict page-render latency.

2.2 Architecture

The post defines three computation modes [1]:

- Offline computation has few limits on data volume and algorithmic complexity because it runs in batch. Its results go stale between updates and it cannot react to the current context. Netflix ran model training and batch pre-computation of results here, using Hadoop and query tools such as Hive and Pig.
- Online computation responds to the current request and context but must meet a strict SLA. The post discusses fallbacks for when the online computation cannot complete in time, such as reverting to a pre-computed result.
- Nearline computation is "an intermediate compromise between these two modes in which we can perform online-like computations, but do not require them to be served in real-time" [1]. It is event-driven. A user action triggers computation, and the result is stored for later serving.

Event and data distribution was handled by an internal framework called Manhattan, which the post calls "a distributed computation system that is central to our algorithmic architecture for recommendation" and compares to Twitter's Storm. Logging flowed through Chukwa into Hadoop, and Hermes provided publish-subscribe delivery [1]. Results were persisted in different stores depending on access pattern. My recollection of the post is that it names MySQL, Cassandra and EVCache for this purpose; treat the exact mapping as approximate.



Netflix slide decks from the same era give a concrete example of the nearline layer. On a play event, a matrix-factorisation model solves the user's factors, computes user-video dot products and stores the scores in a cache, so that the online layer only applies

presentation-context filtering and serves the result (as summarised in search coverage of the decks around [1]).

2.3 Key ideas and lessons

- Learning, feature computation and model evaluation can each be placed in any layer. The architecture is a set of placement choices, not a fixed pipeline [1].
- Distinguish events (small, time-sensitive units such as a play) from data (larger, less latency-sensitive aggregates). They take different paths [1].
- Always keep a pre-computed fallback for when online computation misses its deadline.

In 2023 Amatriain revisited the post in "Blueprints for recommender system architectures: 10th anniversary edition" [2]. It summarises four existing blueprints, including the Netflix one, and proposes a generalised blueprint built around a scoring-and-ranking model trained on data from a feedback loop [2]. The online/nearline/offline split has survived a decade essentially unchanged. What changed is that "nearline" grew into streaming feature platforms and near-real-time training (Section 9).

3. YouTube: the two-stage deep network (2016)

3.1 Problem

Covington, Adams and Sargin's RecSys 2016 paper "Deep Neural Networks for YouTube Recommendations" describes what it calls "one of the largest scale and most sophisticated industrial recommendation systems in existence" [3]. The challenges it names are scale, freshness (new uploads every second) and noisy implicit feedback.

3.2 Architecture

The paper follows the "classic two-stage information retrieval dichotomy" [3]:

```
video corpus           candidate generation          ranking             top-N
(millions)   ------->  (deep net over user history ->  (deep net,       -> (dozens)
                        extreme multiclass softmax;     rich features,
other candidate ---->   ANN search at serving)          watch-time
sources                 => hundreds                     objective)
                                  (after Covington et al., 2016 [3][15])
```

- Candidate generation treats recommendation as extreme multiclass classification. Given the user's watch and search history (averaged embeddings), demographics and context, the network predicts which video will be watched next, trained with a sampled softmax. At serving time the softmax output layer is replaced by nearest-neighbour search over video embeddings with the user vector as the query, so selecting the top N classes reduces to ANN lookup [3][15]. It outputs a small subset, in the hundreds, from the corpus [3][15].
- Ranking uses a separate deep network with many more features, including impression-level and user-video interaction features. Its objective is a weighted logistic regression that predicts expected watch time rather than click probability, which counteracts clickbait [15].

- Other candidate sources, described as "earlier work," are blended in before ranking. Ranking's job is partly to make scores from heterogeneous sources comparable [3].

Reported scale: approximately one billion parameters, trained on hundreds of billions of examples (approximate, from a secondary summary of the paper [15]). The paper also introduced the "example age" feature to model freshness. To my recollection it frames the funnel as millions to hundreds to dozens under a serving budget of tens of milliseconds; these figures could not be re-verified for this chapter and should be treated as approximate.

3.3 Lessons

- Separating retrieval from ranking lets each be optimised for its own metric and cost profile.
- Training retrieval as classification plus ANN serving became the template for every later two-tower system.
- Ranking on watch time rather than clicks is an early example of moving the objective closer to long-term value.

4. Meta: Instagram Explore and Facebook Feed

4.1 Instagram Explore, 2019: the three-pass distillation funnel

Instagram's 2019 post "Powered by AI: Instagram's Explore recommender system" describes a three-part ranking funnel that "extracts 65 billion features and makes 90 million model predictions every second" [7]. Candidate sourcing relies on ig2vec, a word2vec-like framework that treats the sequence of account IDs a user interacts with as words in a sentence to learn account embeddings [7]. Given seed accounts, nearest-neighbour lookup yields candidate accounts and from those candidate media.

The ranking funnel then progressively distils:

```
ig2vec account KNN + other sources
          |
          v
 ~500 candidates
          |   pass 1: distillation model (minimal features) mimics later stages
          v
 ~150 candidates
          |   pass 2: lightweight neural network
          v
 ~50 candidates
          |   pass 3: full deep neural network (all dense features)
          v
 ~25 shown first (after integrity/diversity rules)
                              (after Instagram, 2019 [7]; counts approximate)
```

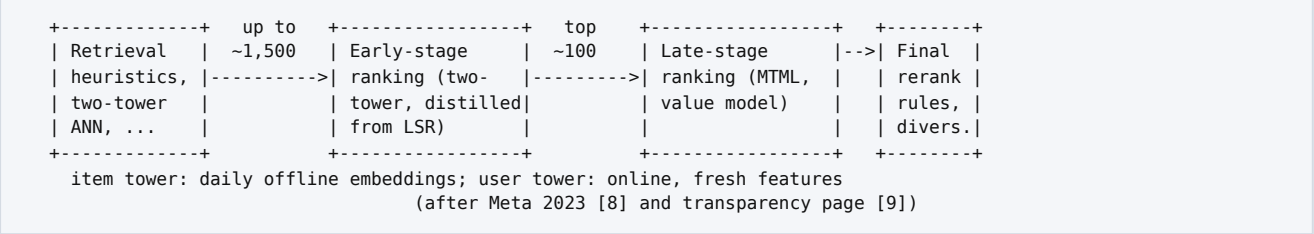
The 500, 150, 50 and 25 figures come from the post and its coverage [7]. The key trick is that the first-pass model is trained to imitate the combined output of the heavier stages using a fraction of the features. Distillation is how the funnel keeps its stages aligned.

4.2 Instagram Explore, 2023: four stages, caching everywhere

Meta's August 2023 engineering post "Scaling the Instagram Explore recommendations system" redraws the funnel as four stages: retrieval, first-stage ranking, second-stage ranking and final re-ranking. It explicitly says the "leitmotif" is "clever use of caching and pre-computation in different ranking stages," which allows heavier models at every stage and less reliance on heuristics [8].

- Retrieval combines heuristic sources (for example trending or followed-author content) with ML sources, chiefly two-tower networks. Because the user and item towers are independent after training, item embeddings are precomputed offline, typically daily. At request time a fresh user embedding is computed and an ANN search returns similar items [8].
- First-stage ranking reuses the two-tower architecture "because of its cacheability property." It is trained to predict the output of the second-stage ranker, a form of knowledge distillation [8].
- Second-stage ranking is a heavier multi-task multi-label (MTML) network with user-item interaction features. It predicts probabilities of several events (click, like, save, "see less" and so on). These are combined by a value model, a weighted sum such as w_click * P(click) + w_like * P(like) - w_seeless * P(see less) + ... [8].
- Final re-ranking applies integrity, diversity and business rules [8].

Meta's public transparency page for Explore gives concrete funnel sizes. Retrieval fetches up to 1,500 media. Early-stage ranking, a two-tower network, passes the top 100 onward. Late-stage ranking produces the final list [9]. The page cautions that "these models and their input signals ... change frequently" [9].



4.3 Facebook News Feed (2021)

Meta's January 2021 post "How machine learning powers Facebook's News Feed ranking algorithm" describes feed ranking as multiple passes over an inventory of more than 1,000 candidate posts per user, evaluated in parallel on machines called predictors [10]. Integrity checks run first. A lightweight model then narrows the pool to roughly 500 of the most relevant posts, and heavier neural networks in later passes produce the main per-story score that orders the feed [10]. The final ordering also enforces contextual rules such as content-type diversity. The structure is the same canonical shape: eligibility and integrity filtering, a light pass, a heavy pass and contextual re-ranking.

4.4 Meta ads: Andromeda and GEM (2024-2025)

Ads retrieval is the extreme case of the funnel. Secondary coverage of Meta's December 2024 Andromeda post reports that retrieval narrows tens of millions of eligible ads to a few thousand candidates per impression before ranking and auction. Andromeda replaced a stack of smaller models and heuristics with a deep network co-designed with NVIDIA Grace

Hopper hardware and Meta's MTIA, with a reported 10,000x increase in model capacity, +6% retrieval recall and +8% ad-quality gains on selected segments [27]. All of these figures are secondhand and should be treated as approximate.

In November 2025 Meta described GEM, the "Generative Ads Recommendation Model," as "the largest foundation model for recommendation systems (RecSys) in the industry, trained at the scale of large language models" [28]. Architecturally, GEM is not served in the request path for every candidate. It acts as a central teacher whose learnings are propagated "across the entire ads model fleet" through post-training techniques (distillation and representation transfer) into the vertical models that actually run in each funnel stage. Meta credits it with a 5% increase in ad conversions on Instagram and 3% on Facebook Feed in Q2 2025 [28]. This is a new layer in the canonical diagram: an offline foundation-model tier that feeds every online stage.

5. X / Twitter: the open-sourced For You pipeline

5.1 2023 architecture

On 31 March 2023 Twitter published "Twitter's Recommendation Algorithm" together with the source repository twitter/the-algorithm [11][12]. The blog frames the problem as distilling the roughly 500 million tweets posted daily down to a handful for each user. It describes three stages: candidate sourcing, ranking with a machine learning model, and heuristics and filters [11]. To my recollection the post states that the pipeline runs about 5 billion times per day and completes in under 1.5 seconds on average; this could not be re-verified for this chapter and should be treated as approximate.

Components [11][12]:

- Home Mixer, the service that assembles For You, is built on Product Mixer, a Scala framework of pipelines. Product pipelines select mixer or recommendation pipelines, which in turn run multiple candidate pipelines.
- Candidate sources together produce roughly 1,500 candidates (approximate; reported in coverage of the release). They split roughly 50/50 between in-network and out-of-network, though this varies per user [11].
    - In-network: the Earlybird search index with a light ranker (logistic regression), with author selection heavily driven by RealGraph, a model of the probability that one user engages with another.
    - Out-of-network: social-graph traversal through UTEG (user-tweet-entity-graph, an in-memory interaction graph on GraphJet) and embedding spaces (SimClusters community embeddings and TwHIN knowledge-graph embeddings). Embedding approaches became the larger out-of-network source.
- Heavy Ranker: a multi-task neural network of approximately 48M parameters (reported by secondary sources) that predicts several engagement probabilities. These are combined with weights into a single score. Candidates are scored in batch without cross-candidate attention.
- Heuristics and filters: visibility filtering (blocked or muted accounts, NSFW settings), author diversity, content balance, feedback fatigue, de-duplication and social proof, then mixing with ads and Who-to-Follow modules.

```
request -> Home Mixer (Product Mixer framework)
             |
             +-- in-network:  Earlybird index + light ranker (RealGraph) --+
             +-- out-of-net:  UTEG/GraphJet, SimClusters ANN, TwHIN ANN ---+--> ~1,500
             |                                                              |
             +--> feature hydration --> Heavy Ranker (~48M param MTL NN) <-+
             |
             +--> visibility filtering, author diversity, dedup, fatigue
             +--> mix ads / WTF modules --> For You timeline
                                  (after Twitter 2023 [11][12])
```

5.2 2026: the Grok-era x-algorithm

In January 2026 xAI released a new x-algorithm repository and updated it substantially on 15 May 2026 [13]. Secondary coverage describes the composition as Thunder for in-network candidates, Phoenix retrieval for out-of-network, Home Mixer for orchestration, and a generic candidate pipeline built from reusable Rust traits named Source, Hydrator, Filter, Scorer, Selector and SideEffect [13]. A single phoenix/run_pipeline.py now runs retrieval and ranking end to end from exported checkpoints, and a roughly 3 GB "mini Phoenix" model is downloadable. Production weights and training code remain closed [13]. The trait list is a useful canonical vocabulary in its own right: every stage of every funnel in this chapter is some composition of source, hydrate, filter, score and select.

6. Pinterest: candidate generators, a light-weight scorer and a blender

Pinterest's WWW 2022 slides draw the home feed as a funnel [14]:

- Candidate generators: token-based indexes, HNSW embedding indexes, graph-based random walks (Pixie) and explore/exploit sources.
- A Light-Weight Scorer (LWS) acting as pre-ranking.
- A Ranking Service containing a Utility Prediction Scorer (the multi-task ranker) and a Multi-Objective Blender.

Pixie, a real-time random-walk system over the pin-board bipartite graph, was reported to serve 3+ billion items to 200+ million users in real time. Systems backed by Pixie contributed more than 80% of user engagement at the time [15b]. An older Pinterest engineering post describes the same split in services. Polaris retrieves, filters and applies lightweight scoring, then passes a list to the Feed Generator, which sends it with real-time user signals to Scorpion for "a second pass of full scoring" [16]. Historically, each content type was ranked by its own specialised model and then "blended with a ratio-based round robin" to create the final home feed [33]. Later diversification work at retrieval and ranking (round-robin, DPP, bucketised ANN) is covered in a later chapter.

```
Pixie walks | HNSW embeds | token idx | explore  --> merge+dedup --> LWS (light)
                                                                      |
                                    Utility Prediction Scorer (MTL) <-+
                                                  |
                               Multi-Objective Blender --> home feed
                                   (after Pinterest WWW 2022 [14], [16])
```

7. LinkedIn: first-pass rankers to a unified LLM-era stack

LinkedIn's 2019 feed architecture is a federated variant of the funnel. Multiple first-pass rankers (FPRs) preselect candidates from their own inventories, such as network updates, jobs and recommended content, and a second-pass ranker (SPR) merges their outputs into one list. FollowFeed, which serves updates from a member's network, is the dominant FPR: more than 80% of feed updates came from it [17]. The federated design matches the organisation (each inventory team owns its FPR) but makes global calibration the SPR's problem.

By 2025-2026 LinkedIn had published work on replacing multi-source retrieval with a single LLM-based dual encoder fine-tuned on engagement data [18]. Secondary coverage of a March 2026 LinkedIn post describes a unified LLM retrieval system plus a transformer-based sequential "Generative Recommender" ranker. These details are from secondary sources and are approximate. The direction is the same as at Meta and Kuaishou: fewer, larger, shared models with fewer hand-maintained sources.

8. Alibaba, Taobao and Kuaishou: the explicit pre-ranking stage

8.1 Why Chinese e-commerce and short-video stacks name "pre-ranking"

Chinese industrial papers almost always draw the cascade as matching, pre-ranking, ranking and re-ranking. Pre-ranking has become a research area in its own right because it handles orders of magnitude more items than the ranker under similar latency. A Taobao Search paper ("Rethinking the Role of Pre-ranking in Large-scale E-Commerce Searching System") describes billions of items in the corpus, with pre-ranking taking in hundreds of thousands of candidates and passing roughly the top thousands to ranking. Its input set is "tens or even hundreds of times larger" than the ranking stage's [20]. The paper argues that pre-ranking should be judged by whether it preserves the items the whole system would ultimately select, not by its standalone AUC [20].

8.2 COLD (Alibaba, 2020)

COLD ("Computing power cost-aware Online and Lightweight Deep pre-ranking system") rejected the then-standard two-tower pre-ranker, which cannot use user-item cross features. It replaced it with a small fully connected network with cross features, co-designed with the serving system [19]. The model and its computing cost are optimised together, so the trade-off between quality and compute is controllable. The design adds feature selection under a compute budget, inference acceleration and online learning to handle distribution shift. COLD was deployed from 2019 in almost all products with a pre-ranking module in Alibaba display advertising [19]. Its lesson generalises: the architecture of a stage should be chosen by its share of the latency and compute budget, not by tradition.

8.3 Kuaishou: a six-stage cascade, quantified

Kuaishou's public papers give the most complete stage-level numbers. RecFlow logs six stages of the production short-video funnel [22]. A 2026 paper on cross-stage fusion describes candidate sizes in that cascade shrinking from about 8,000 down to 6, with roughly 500 candidates selected at pre-ranking and about 10 items finally exposed [23]. KuaiFormer describes retrieval feeding a pre-rank stage, then a cascading rank stage, then a fine-grained full rank stage [24].

```
corpus (10^8+)
   | retrieval: many channels (i2i, u2i, transformer retrieval e.g. KuaiFormer)
   v
~8,000 merged candidates
   | pre-ranking / coarse ranking
   v
~500
   | cascade ranking -> full (fine) ranking (multi-task)
   v
~tens
   | re-ranking (listwise, diversity, business)
   v
~6-10 exposed
                          (after RecFlow [22], UniRec [23]; approximate)
```

8.4 Collapsing the cascade: OneRec

Kuaishou's OneRec (February 2025) and its June 2025 technical report argue that the cascade itself is the problem. Each stage is optimised for a different objective, compute is fragmented, and earlier stages cap the quality of later ones [25][26]. OneRec replaces retrieval and ranking with a single encoder-decoder generative model (later decoder-only variants). It uses sparse Mixture-of-Experts for capacity and session-wise list generation, and it is aligned to the final objective with preference optimisation [25][26]. The reported motivation includes better engagement and lower operating cost than the legacy multi-stage pipeline [26]. In practice, generative systems so far keep a thin rule and filter layer and a re-ranking stage. What collapses is the learned middle of the funnel.

9. The time axis: offline, nearline, online in modern stacks

The Netflix three-layer split [1] maps onto modern components as follows:

Layer	Netflix 2013 [1]	Modern equivalent	Examples in this chapter
Offline	Hadoop, Hive, Pig; model training; batch recs	Data lake + distributed training; daily item-embedding jobs; ANN index builds; foundation models	Item tower daily refresh [8]; GEM teacher [28]; Netflix FM [29]
Nearline	Manhattan event processing; cached scores	Stream processing, real-time feature store, streaming/online training, incremental index updates	Monolith online training [31]; COLD online learning [19]
Online	Request-time filtering, light models, fallbacks	Retrieval fan-out, feature hydration, pre-rank, rank, re-rank under a p99 SLA	Home Mixer [12]; Explore [8]; Feed [10]

ByteDance's Monolith (2022) is the clearest published nearline-training design. It argues that standard frameworks keep batch training and serving "completely separated," which prevents models from reacting to feedback in real time. Monolith uses a collisionless hash

embedding table with expirable embeddings and frequency filtering, plus an online training architecture that synchronises updated parameters from training to serving in near real time [31]. Netflix's 2025 foundation model shows the opposite trend at the offline end. A single transformer, trained autoregressively on hundreds of billions of interactions from over 300 million members, centralises member preference learning, and downstream models consume it through fine-tuning, embeddings or direct prediction [29]. Both Meta [28] and Netflix [29] thus add an explicit foundation-model tier beneath the classic offline layer.

10. Request flow, end to end

Putting the pieces together, a generic request in a modern feed looks like this (synthesised from [4][5][8][10][11][12][14]):

```
client --> API/gateway --> mixer/orchestrator (per-surface pipeline)
   1. fetch user context: profile, recent actions (real-time feature store)
   2. fan out to N candidate sources in parallel, each with its own timeout
        two-tower ANN | i2i / graph walk | follow graph | trending | rules
   3. merge, dedup, eligibility & integrity filter          (~10^3-10^4)
   4. pre-rank: cached / distilled / two-tower or small DNN (-> ~10^2-10^3)
   5. hydrate rich features (batch RPC to feature store, embeddings)
   6. rank: multi-task NN -> P(event_i); value model sum w_i * P_i (-> ~10^1-10^2)
   7. re-rank: diversity, freshness, fatigue, policy, ads/module insertion
   8. log impressions + features served ("log-and-wait") for training
   9. respond; async side effects (cache next page, update seen-sets)
```

Three implementation details recur across sources:

- Per-source timeouts and graceful degradation. Candidate sources run in parallel, and slow ones are dropped rather than allowed to block the page. This echoes Netflix's insistence on fallbacks [1].
- Hydration is often the real latency hotspot. Uber Eats reported that retrieval was fetching tens of thousands of candidates that were then fully hydrated before ranking discarded most of them. Removing low-yield retrieval paths saved about 120 ms, and splitting hydration into ranking and presentation phases saved over 100 ms more [30].
- Logging the served features at request time (rather than recomputing them later) avoids training-serving skew. This is a theme in Yan's survey of discovery systems [5].

11. How each company draws it: comparison

Company / system	Stages as named	Funnel sizes (public, approx.)	Retrieval	Light / pre-rank	Heavy rank	Final pass
Netflix 2013 [1]	offline / nearline / online	n/a	batch MF, precomputed	nearline cached scores	online light eval	context filter, fallbacks
YouTube 2016 [3]	candidate gen, ranking	millions -> hundreds -> dozens	softmax-trained DNN + ANN	none (2 stages)	DNN, weighted LR on watch time	not detailed

Company / system	Stages as named	Funnel sizes (public, approx.)	Retrieval	Light / pre-rank	Heavy rank	Final pass
Google course [4]	candidate gen, scoring, re-ranking	billions -> 100s-1000s -> ~10	multiple generators	none	scoring model	freshness, diversity, fairness
Instagram 2019 [7]	sourcing + 3-pass funnel	500 -> 150 -> 50 -> 25	ig2vec account KNN	distilled model, light NN	full DNN	integrity, diversity
Instagram 2023 [8][9]	retrieval, ESR, LSR, rerank	up to 1,500 -> 100 -> final	heuristics + two-tower ANN	cached two-tower, distilled	MTML + value model	rules, diversity
Facebook Feed 2021 [10]	multi-pass ranking	1,000+ -> ~500 -> ordered	inventory of eligible posts	lightweight model	neural nets on predictors	contextual rules
Twitter 2023 [11] [12]	sourcing, rank, heuristics	~500M/day -> ~1,500 -> feed	Earlybird, UTEG, SimClusters, TwHIN	Earlybird light ranker	~48M-param MTL NN	visibility, diversity, mixing
Pinterest [14][16]	CG, LWS, ranking, blender	not public	Pixie, HNSW, token idx	Light-Weight Scorer	utility prediction scorer	multi-objective blender
LinkedIn 2019 [17]	FPRs -> SPR	FollowFeed >80% of updates	per-inventory FPRs	FPR scoring	second-pass ranker	not detailed
Taobao search [20]	matching, pre-rank, rank	billions -> 10^5 -> 10^3	multi-channel	pre-ranking	ranking	not detailed
Alibaba ads [19]	matching, pre-rank (COLD), rank	not public	multi-channel	COLD cross-feature DNN	ranking DNN	not detailed
Kuaishou [22][23][24]	6 stages	~8,000 -> ~500 -> ... -> 6-10	KuaiFormer and others	pre-rank / cascade rank	full rank	re-rank
Kuaishou OneRec [25][26]	single generative model	session list generated	merged into generator	merged	merged	thin rule layer
Meta Ads [27][28]	retrieval (Andromeda), rank, auction	10^7 ads -> few thousand	Andromeda DNN on GPUs	(not public)	ranking models distilled from GEM	auction

12. Scale numbers at a glance

Metric	Value	Source
Instagram Explore features extracted	65 billion per second	[7]
Instagram Explore model predictions	90 million per second	[7]
Instagram Explore retrieval output	up to 1,500 media; top 100 after ESR	[9]
Facebook Feed candidates per user	1,000+, pruned to ~500	[10]
Twitter daily tweets considered	~500 million	[11]
Twitter candidates per request	~1,500 (approx.)	[11][12]

Metric	Value	Source
Twitter heavy ranker size	~48M parameters (approx.)	[12]
YouTube 2016 model size	~1B parameters (approx.)	[15]
Kuaishou RecFlow	9.3M requests, 1.9B stage samples, 6 stages	[22]
Pixie	3+B items, 200+M users, real time	[15b]
Uber Eats hydration fix	~120 ms and 100+ ms latency savings	[30]
Meta Andromeda	~10,000x capacity; +6% recall (secondhand)	[27]
Meta GEM	+5% IG / +3% FB Feed ad conversions (Q2 2025)	[28]

13. Recurring design tricks

1. Distil downward. Train each cheaper stage to imitate the next stage, not just the label: Instagram's first-pass distillation [7] and its two-tower first-stage ranker trained on second-stage outputs [8]. Meta GEM pushes the same idea fleet-wide [28].
2. Cache whatever is user-independent. Item towers are computed offline, and only the user tower and its dot product run online [8].
3. Make early stages expressive only when the budget allows. COLD shows that cross features in pre-ranking pay off when co-designed with serving [19].
4. Federate sources, centralise scoring. Many independent candidate sources feed one scorer, because raw retrieval scores are not comparable [4][11][17].
5. Separate hard rules from learned preferences. Filtering and ordering are explicit stages [6], and integrity checks run before expensive models [10].
6. Optimise a value model, not a single label. Weighted multi-task outputs produce one score [8][11], and objectives move towards long-term value (watch time [3]).
7. Measure the funnel, not the stage. Log what each stage dropped [22] and evaluate pre-ranking against end-to-end selection [20].
8. Push freshness into nearline. Event-triggered recomputation [1] and online training [31] are cheaper than making the online path heavier.
9. Collapse stages when one model can afford it. Generative recommenders [25][26] and unified LLM retrieval [18] remove learned stages, but keep rules and filtering.

Key takeaways for system designers

- Start from the latency budget and the corpus size, then derive the funnel. Each stage should cut candidates by one to two orders of magnitude while its per-item cost rises by a similar factor. Public funnels cluster around 10^3-10^4 after retrieval, 10^2-10^3 after pre-ranking, and 10^1-10^2 after ranking [8][9][10][22][23].
- Treat online, nearline and offline as placement decisions for every component (features, models, results), as Netflix did in 2013 [1]. Most production wins come from moving work out of the online path through caching, pre-computation and nearline updates [8].
- Judge each stage by what it does for the final output. Recall at the next stage's cut-off matters more than standalone AUC [20]. Log stage-wise drops so that you can debug selection bias [22].

- Keep candidate sources heterogeneous but cheap. Graph walks, embeddings, social graph and trending each capture different intent, and the ranker reconciles them [4][11][14].
- Use distillation and shared towers to keep the stages consistent [7][8]. Inconsistent stages are the main source of wasted compute in cascades.
- Put integrity, eligibility and business logic in explicit filter and re-rank stages rather than hoping a model learns them [6][10][11].
- Feature hydration and fan-out are frequent latency hotspots. Prune low-yield sources and split hydration by need [30].
- Expect the 2025-2026 trend to continue. Foundation models trained offline [28][29] feed smaller serving models, and generative recommenders [25][26] and LLM-based retrievers [18] collapse the middle of the funnel. Filters, policy and listwise re-ranking survive in every published design.

References

1. Xavier Amatriain and Justin Basilico (Netflix). "System Architectures for Personalization and Recommendation." Netflix Technology Blog, March 2013. https://netflixtechblog.com/system-architectures-for-personalization-and-recommendation-e081aa94b5d8
2. Xavier Amatriain. "Blueprints for recommender system architectures: 10th anniversary edition." Machine Learning Times (Predictive Analytics World), 2023. https://www.predictiveanalyticsworld.com/machinelearningtimes/blueprints-for-recommender-system-architectures-10th-anniversary-edition/12879/
3. Paul Covington, Jay Adams and Emre Sargin (Google). "Deep Neural Networks for YouTube Recommendations." RecSys 2016. https://research.google/pubs/pub45530 (PDF: https://cseweb.ucsd.edu/classes/fa17/cse291-b/reading/p191-covington.pdf)
4. Google for Developers. "Recommendation systems overview" (Machine Learning: Recommendation Systems course). https://developers.google.com/machine-learning/recommendation/overview/types
5. Eugene Yan. "System Design for Recommendations and Search." eugeneyan.com, 2021. https://eugeneyan.com/writing/system-design-for-discovery/
6. NVIDIA (Merlin team, citing Even Oldridge's "Recommender Systems, Not Just Recommender Models"). "Offline to Online: Feature Storage for Real-time Recommendation Systems with NVIDIA Merlin." NVIDIA Technical Blog, 2022-2023. https://developer.nvidia.com/blog/offline-to-online-feature-storage-for-real-time-recommendation-systems-with-nvidia-merlin
7. Ivan Medvedev, Haotian Wu and Taylor Gordon (Instagram/Facebook AI). "Powered by AI: Instagram's Explore recommender system." Instagram Engineering, 2019. https://instagram-engineering.com/powered-by-ai-instagrams-explore-recommender-system-7ca901d2a882
8. Vladislav Vorotilov and Ilnur Shugaepov (Meta). "Scaling the Instagram Explore recommendations system." Engineering at Meta, 9 August 2023. https://engineering.fb.com/2023/08/09/ml-applications/scaling-instagram-explore-recommendations-system/
9. Meta Transparency Center. "Instagram Explore AI system." Accessed 2026. https://transparency.meta.com/features/explaining-ranking/ig-explore/

10. Akos Lada, Meihong Wang and Tak Yan (Facebook). "How machine learning powers Facebook's News Feed ranking algorithm." Engineering at Meta, 26 January 2021. https://engineering.fb.com/2021/01/26/ml-applications/news-feed-ranking/
11. Twitter Engineering. "Twitter's Recommendation Algorithm." Twitter (X) Engineering Blog, 31 March 2023. https://blog.x.com/engineering/en_us/topics/open-source/2023/twitter-recommendation-algorithm
12. Twitter. "the-algorithm: Source code for the X Recommendation Algorithm" (including home-mixer README). GitHub, 2023. https://github.com/twitter/the-algorithm
13. Pasquale Pillitteri. "X Algorithm Open Source: May 15, 2026 Major Update Brings End-to-End Pipeline and Downloadable Phoenix Model." 2026 (secondary coverage of xAI's x-algorithm repository). https://pasqualepillitteri.it/en/news/2594/x-algorithm-open-source-may-2026
14. Pinterest. "Pinterest at WWW 2022" (sponsor slides on home feed recommendation funnel). 2022. https://archives.iw3c2.org/www2022/wp-content/uploads/Sponsors/WWW2022-Pinterest.pdf
15. Adrian Colyer. "Deep neural networks for YouTube recommendations" (paper summary). The Morning Paper, 19 September 2016. https://blog.acolyer.org/2016/09/19/deep-neural-networks-for-youtube-recommendations/ 15b. Adrian Colyer. "Pixie: a system for recommending 3+ billion items to 200+ million users in real-time" (summary of Eksombatchai et al., WWW 2018). The Morning Paper, 2018. https://blog.acolyer.org/2018/05/23/pixie-a-system-for-recommending-3-billion-items-to-200-million-users-in-real-time/
16. Pinterest Engineering. "Building a dynamic and responsive Pinterest." Pinterest Engineering Blog (Medium). https://medium.com/@Pinterest_Engineering/building-a-dynamic-and-responsive-pinterest-7d410e99f0a9
17. LinkedIn Engineering. "Community-focused Feed optimization." LinkedIn Engineering Blog, June 2019. https://engineering.linkedin.com/blog/2019/06/community-focused-feed-optimization
18. LinkedIn. "Large Scale Retrieval for the LinkedIn Feed using Causal Language Models." arXiv 2510.14223, 2025. https://arxiv.org/html/2510.14223v1
19. Zhe Wang, Liqin Zhao, Biye Jiang et al. (Alibaba). "COLD: Towards the Next Generation of Pre-Ranking System." DLP-KDD 2020, arXiv 2007.16122. https://arxiv.org/abs/2007.16122
20. Alibaba (Taobao Search). "Rethinking the Role of Pre-ranking in Large-scale E-Commerce Searching System." arXiv 2305.13647, 2023. https://arxiv.org/abs/2305.13647
21. "A Comprehensive Survey on Retrieval Methods in Recommender Systems." arXiv 2407.21022, 2024. https://arxiv.org/abs/2407.21022
22. Kuaishou. "RecFlow: An Industrial Full Flow Recommendation Dataset." ICLR 2025, arXiv 2410.20868. https://arxiv.org/html/2410.20868v1
23. "UniRec: Cross-stage Multi-Task Fusion with Preference Alignment for Cascaded Recommender Systems." arXiv 2609.11052, 2026. https://arxiv.org/pdf/2609.11052
24. Kuaishou. "KuaiFormer: Transformer-Based Retrieval at Kuaishou." arXiv 2411.10057, 2024. https://arxiv.org/pdf/2411.10057
25. Jiaxin Deng et al. (Kuaishou). "OneRec: Unifying Retrieve and Rank with Generative Recommender and Iterative Preference Alignment." arXiv 2502.18965, February 2025. https://arxiv.org/abs/2502.18965

26. Kuaishou OneRec Team. "OneRec Technical Report." arXiv 2506.13695, June 2025. https://arxiv.org/html/2506.13695v1
27. Jon Loomer. "Meta Andromeda: What It Means for Your Ad Strategy" (secondary summary of Meta's December 2024 Andromeda engineering post). 2025. https://www.jonloomer.com/meta-andromeda/
28. Meta. "Meta's Generative Ads Model (GEM): The central brain accelerating ads recommendation AI innovation." Engineering at Meta, 10 November 2025. https://engineering.fb.com/2025/11/10/ml-applications/metas-generative-ads-model-gem-the-central-brain-accelerating-ads-recommendation-ai-innovation/
29. ZenML LLMOps Database. "Foundation Model for Personalized Recommendation at Scale" (summary of Netflix Tech Blog, March 2025). https://www.zenml.io/llmops-database/foundation-model-for-personalized-recommendation-at-scale
30. Uber Engineering. "Uber Eats search pipeline" (retrieval, hydration and ranking latency work). Uber Blog, 2026. https://www.uber.com/us/en/blog/uber-eats-search-pipeline/
31. Zhuoran Liu et al. (ByteDance). "Monolith: Real Time Recommendation System With Collisionless Embedding Table." arXiv 2209.07663, 2022. https://arxiv.org/abs/2209.07663
32. Eugene Yan. "System Design for Discovery (RecSys and Search)." ApplyingML. https://applyingml.com/resources/discovery-system-design/
33. MLconf. Pinterest Homefeed talk abstract (ratio-based round-robin blending), 2016. https://mlconf.com/?p=8909
34. Carlos A. Gomez-Uribe and Neil Hunt. "The Netflix Recommender System: Algorithms, Business Value, and Innovation." ACM Transactions on Management Information Systems 6(4), 2015. https://dl.acm.org/doi/10.1145/2843948

02. Candidate Generation and Retrieval: Two-Tower Models, Negative Sampling, ANN Indexes and Multi-Source Retrieval

Overview

Every large recommender is a funnel. A corpus of millions to billions of items (videos, Pins, posts, ads, listings, stores, users) must be narrowed to a few hundred or a few thousand candidates within a budget of tens of milliseconds. Only then can a heavy ranking model with rich cross features score them. The first stage of that funnel is called candidate generation, retrieval, recall or matching. It is the subject of this chapter.

Retrieval has three properties that set it apart from ranking:

1. **It must be sub-linear in corpus size.** You cannot score a billion items per request with a deep model. Most production designs therefore factor the score into a user (query) vector and an item vector, precompute and index the item side, and use approximate nearest neighbor (ANN) search. Alternatives replace the vector index with a learned tree or path structure (TDM, Deep Retrieval, Andromeda) or with graph walks (Pixie, GraphJet).
2. **It optimizes recall, not precision.** Its job is to make sure good items reach the ranker. The ranker decides their order.
3. **It is almost always multi-source.** Production systems run a portfolio of retrievers, such as learned embeddings, item-to-item co-occurrence, graph walks, social-graph expansion and heuristic "trending" sources. They merge the outputs with quotas, weights or round-robin interleaving.

The chapter has six parts:

- the foundational modeling work: the YouTube DNN, sampled softmax, in-batch negatives and the logQ correction;
- company-by-company production designs: Facebook Search, Instagram, Meta Ads Andromeda, Pinterest, LinkedIn, Twitter/X, Snap, Uber, Etsy, Airbnb and Alibaba/ByteDance;
- the ANN index families: IVF-PQ in FAISS, ScaNN, HNSW and DiskANN;
- multi-source merging, graph retrievers and heuristic retrievers;
- a comparison table;
- takeaways for system designers.

```
                  THE CANONICAL RETRIEVAL FUNNEL
corpus: 1e6 .. 1e10 items
  |
  |  [Retrieval / candidate generation]   many sources in parallel
  |    - two-tower EBR (ANN over item embeddings)
  |    - item-to-item CF / co-engagement tables
  |    - graph walks (Pixie, GraphJet/UTEG, PPR)
  |    - social graph (friends-of-friends, in-network)
  |    - heuristic (trending, fresh, location)
  v
~1e3 .. 1e4 candidates (dedup + merge + filter)
  |
  |  [Pre-rank / lightweight scoring]  often a two-tower too
  v
~1e2 .. 1e3
  |
  |  [Heavy ranking]  multi-task DNN with cross features
  v
~1e1 .. 1e2 shown, after re-ranking for diversity, policy, ads
```

---

1. Foundations: from extreme multiclass to two towers

1.1 YouTube DNN candidate generation (Google, 2016)

Problem. Recommend videos from a corpus of millions to a very large user base. The model must narrow those millions down to a few hundred candidates per user [1].

Architecture. Covington, Adams and Sargin split the system into two neural networks, following the classic information-retrieval two-stage dichotomy. A candidate-generation network reads the user's activity history and context and produces a user vector. A separate ranking network then scores the few hundred survivors [1]. Candidate generation is framed as **extreme multiclass classification**: predict which video, out of millions of classes, the user will watch next.

Training trick. A full softmax over millions of classes is too expensive. The model is trained with **sampled softmax**, which draws a few thousand negatives per example and corrects for the sampling distribution. Secondary summaries of the paper report a speedup of more than 100x over the full softmax [1].

Serving. The learned output-layer video vectors become the item embeddings. At serving time the softmax is replaced by a nearest-neighbor search over those vectors in dot-product space [1]. This "train as a classifier, serve as a nearest-neighbor lookup" pattern is the template for almost everything that follows.

```
YouTube DNN candidate generation (2016)
user history + context features --> MLP (ReLU stack) --> user vector u
                                                            |
training:  softmax over all videos (approximated by sampled softmax)
serving :  top-N by dot(u, v_j) via nearest-neighbor index over v_j
```

Lesson. Phrase retrieval as next-item classification and keep the scoring function a dot product. That turns serving into maximum-inner-product search (MIPS), which can be solved approximately at very large scale.

1.2 The two-tower model with in-batch negatives and logQ correction (YouTube, 2019)

Problem. The 2016 output layer had one learned vector per video ID. That makes cold-start and content features awkward. Google's RecSys 2019 paper "Sampling-Bias-Corrected Neural Modeling for Large Corpus Item Recommendations" (Yi et al.) moved to a **two-tower** design. A query tower encodes user and context features, and a candidate tower encodes item features. Both produce embeddings that are compared by inner product [2].

In-batch negatives. Instead of sampling negatives from a fixed vocabulary, every other positive item in the mini-batch serves as a negative for each example. This is cheap, because the item embeddings are already computed. It is also streaming-friendly, because no global vocabulary is needed. It has a bias, though. An item's chance of appearing as a negative is proportional to its frequency in the training stream. Popular items get over-penalized, and under a power-law item distribution this "can severely restrict model performance" [2].

logQ correction. The fix subtracts log(p_j) from each logit s(x, y_j), where p_j is the estimated probability that item j appears in a batch. The corrected softmax then approximates the full softmax [2][5]. The paper's other contribution is a **streaming frequency estimator** that needs no fixed vocabulary. A secondary write-up describes it as two hashed arrays. One array stores the last global step at which each item was seen. The other stores a running average of the gap between occurrences, updated like SGD with a fixed learning rate. Both arrays live on parameter servers during distributed training [2]. The estimated probability is roughly 1 / (average gap).

Deployment. The model was deployed as "Neural Deep Retrieval" (NDR) for YouTube, retrieving personalized suggestions from a corpus of tens of millions of videos. Live A/B tests showed improved recommendation quality [2].

```
Two-tower training with in-batch softmax + logQ   (batch of B pairs)

query tower  q_i = f(user_i, context_i)          i = 1..B
item tower   v_j = g(item_j features)            j = 1..B
logits       s_ij = <q_i, v_j> / temperature  -  log p_j     (logQ)
loss         -sum_i log softmax_j(s_ij)[j = i]
p_j          streaming estimate from hashed "last-seen step" arrays
```

1.3 Mixed Negative Sampling (Google, 2020)

In-batch negatives have a second problem: **selection bias**. Items that never received engagement never show up as positives, so they are never seen as negatives either. A fresh or long-tail item can therefore drift into a region of embedding space where it scores high for everyone. Yang et al.'s Mixed Negative Sampling (MNS) adds a second stream of negatives sampled **uniformly from the whole corpus** (an "index data stream"). These are shared by every batch and concatenated with the in-batch negatives. The method was evaluated on Google Play app recommendations and outperformed batch-only sampling [3]. A later paper that compared against MNS used 128 in-batch negatives plus a large shared set of uniform negatives [3]. That gives a sense of the typical ratio.

1.4 Cross-batch negatives and memory banks (Huawei, 2021)

The quality of in-batch sampling depends on batch size, and very large batches are expensive. Cross-Batch Negative Sampling (CBNS, SIGIR 2021) keeps a FIFO memory bank of item embeddings from recent mini-batches and reuses them as extra negatives [4]. The justification is empirical. After warm-up, encoders produce fairly stable embeddings for the same input, so stale embeddings remain useful negatives. The same "slow drift" observation underlies Cross-Batch Memory in metric learning [4]. The paper reports many more negatives at a training cost close to plain in-batch sampling [4].

1.5 Revisiting the logQ correction (Yandex, RecSys 2025)

Khrylchenko et al. point out that the standard logQ derivation assumes positives are drawn from the sampling distribution. In practice the positive item is in its own batch with probability 1 [5]. Their corrected formulation does two things:

- It applies importance weighting only to the negatives and removes the positive from the corrected denominator.
- It adds a confidence-based weight that shrinks the loss when the model already scores the positive highly.

The authors report consistent gains over the standard logQ correction on public and proprietary datasets [5]. The practical takeaway is that sampled-softmax retrieval training is still being refined. Small correctness details in the loss can move recall.

1.6 Facebook Search EBR: triplet loss, hard negatives and ANN tuning (KDD 2020)

Problem. Facebook Search historically relied on Boolean term matching in **Unicorn**, the inverted-index retrieval engine behind most Facebook search products. Social search also needs personalization from the searcher's own graph, which text matching alone cannot provide [6].

Architecture. A **unified embedding** two-tower model encodes the query side (query text plus searcher context, such as location and social connections) and the document side (text plus document context). Each document's vector is quantized with FAISS and stored inside the existing inverted index:

- the coarse cluster ID becomes a **term**;
- the product-quantized residual becomes the **payload**.

This lets a single Unicorn query mix Boolean constraints with nearest-neighbor matching [6].

```
Facebook EBR inside Unicorn (2020)

query + searcher context --> query tower --> q (e.g. 128-d)
                                                |
          Unicorn query:  (nn  q :radius ...)  AND  (term filters)
                                                |
inverted index:  term = coarse cluster id (IVF centroid)
                 payload = PQ code of residual
                                                |
           candidates --> later-stage ranking (embedding as a feature)
```

Loss and sampling details:

- *Triplet loss with a margin.* The best margin varied across tasks, and different margins moved KNN recall by 5-10% [6].
- *Number of negatives.* The authors argue that sampling n random negatives per positive approximately optimizes recall at the top N/n of a serving pool of size N [6].
- *Impressed-but-not-clicked items are bad negatives.* Using non-click impressions as negatives caused about a 55% absolute recall regression for the people embedding model compared with random negatives. Those negatives are too hard and not representative of the retrieval task [6].
- *Online hard negative mining.* Within the batch, the other positives with the highest similarity to the query are used as hard negatives. This gave +8.38% recall for people search, +7% for groups and +5.33% for events [6]. A reading note also describes an offline variant that takes documents ranked roughly 101-500 by the current model as hard negatives (approximate; from a secondary source).
- *Unified vs. text embedding.* Unified embeddings beat text-only embeddings by +18% recall for events search and +16% for groups search [6].

ANN tuning lessons:

- The authors measure 1-Recall@10, the fraction of exact-KNN top-1 results that appear in the ANN top-10.
- They suggest sizing PQ bytes at about d/4, with diminishing returns beyond that.
- In one 128-d experiment, PCA+PQ16 reached 51.62% 1-Recall@10 versus 74.81% for the flat index at a similar fraction of the index scanned. Quantization is not free.
- The number of coarse clusters and nprobe trade off performance against recall.
- Re-tune ANN parameters whenever the model changes significantly.
- Use an offline tuning pipeline to shortlist settings, then pick the final configuration with online experiments [6].

---

2. Meta: Instagram Explore and Andromeda

2.1 Instagram Explore retrieval (2023)

Explore is a four-stage funnel: **retrieval, first-stage ranking, second-stage ranking and final re-ranking** [7]. Retrieval tries to predict what the later stages would rank highly, starting from the general media distribution. Meta's design relies on **caching and precomputation** combined with **Two Towers networks**. The post's figure shows two-tower inference with caching on both the user side and the item side [7].

Sources. Explore runs many retrieval sources at once. They include heuristics, such as trending posts, and ML strategies that capture both real-time and pre-generated interests. Sources are mixed with tunable weights, so different user groups get different blends [7]. Meta's transparency documentation names concrete source types [8]:

- author-based sources: media from accounts you engaged with;
- media-based sources: media similar to media you engaged with;
- item collaborative filtering;

- personalized PageRank;
- two-tower sparse-network sourcing.

Each source selects hundreds of items from a pool of billions [8]. Meta notes that an earlier generation used Word2Vec-style ID embeddings for users, media and authors.

Counts. Up to **1,500** media survive retrieval. A two-tower early-stage ranker then passes the top **100** to the next stage [8]. The early-stage two-tower model scores user features against media features. Its objective is reportedly trained to predict how the second-stage ranker would rank the item. That is a distillation-style alignment between stages, reported by a secondary source [7].

```
Instagram Explore (2023, per Meta)
 sources (weighted mix): heuristic | item-CF | PPR | two-tower | ...
      |  hundreds each, from billions
      v
 retrieval output: <= ~1500 media
      |  first-stage ranker = two-tower (cached user + item embeddings)
      v
 top 100 --> second-stage ranker (heavy MTML) --> final re-rank
```

2.2 Meta Andromeda: a learned hierarchical index for ads (2024)

Problem. Generative-AI ad tools sharply increased the number of eligible ad creatives. Meta said more than a million advertisers created more than 15 million ads in one month with its GenAI tools. Two-tower plus ANN retrieval caps how much user-ad interaction a model can express [9].

Architecture. Andromeda is a personalized ads retrieval engine. It is a highly customized deep neural network with **sub-linear inference cost** that runs with heavy parallelism on NVIDIA Grace Hopper Superchips [9]. Ads are organized into a **hierarchical index with multiple layers**, so inference only expands the most relevant nodes. The **index and the retrieval model are trained jointly**, which Meta says gives better precision and recall than two-tower or ANN approaches [9].

Results. Deployed on Facebook and Instagram, Andromeda gave **+6% recall** to the retrieval system and **+8% ads quality** on selected segments [9]. Third-party explainers cite a "10,000x" increase in model capacity and a narrowing from tens of millions of ads to a few thousand. These figures could not be verified against the excerpts of Meta's post and should be treated as approximate.

Lesson. At the top end of scale the industry is moving beyond dot-product-only retrieval. The options are learned trees or hierarchies (Andromeda, TDM, Deep Retrieval) and GPU brute force with richer scoring (LiNR). In both cases the retrieval model can capture interactions that a factorized dot product cannot.

---

3. Pinterest: learned retrieval, multi-embedding, graph retrievers and Manas

Pinterest has one of the best-documented retrieval stacks. It has graph retrievers (Pixie), graph neural network embeddings (PinSage), learned two-tower retrieval and its own ANN platform (Manas).

3.1 Pixie: real-time random walks (2017-2018)

Problem. Recommend from 3+ billion items to 200+ million users in real time [18].

Design. Pixie loads a pruned bipartite **Pin-board graph** into the RAM of a single machine. The graph has 3 billion nodes and 17 billion edges and takes about 120 GB of memory. The unpruned graph had 7 billion nodes and more than 100 billion edges. Every walk therefore stays on one machine, and the service scales by replicating machines [18]. A query is a set of Pins with weights. Pixie runs many short random walks with restarts from those Pins, counts visits, and returns the most-visited Pins. The walk is biased toward the user's interests and boosts smaller boards so that high-degree nodes do not dominate [18].

Scale.

- The deployment used AWS r3.8xlarge machines with 244 GB of RAM.
- 99th-percentile latency was under 60 ms.
- One server handled about 1,200 requests per second.
- The cluster served about 100,000 requests per second.
- Pixie gave up to 50% higher user engagement than the previous Hadoop-based system [18].

```
Pixie (graph retriever)
 query pins (weighted) --> N short random walks w/ restart on Pin<->Board graph
                            (single-machine RAM, ~120 GB pruned graph)
                       --> visit counts (early stop when top-k stabilizes)
                       --> top-k pins as candidates   (p99 < 60 ms)
```

3.2 PinSage: GCN embeddings at web scale (2018)

PinSage is a graph convolutional network trained on a graph of **3 billion nodes and 18 billion edges**, using 7.5 billion training examples [17]. Its key tricks are:

- **Random-walk neighborhoods.** Short random walks select the top-T most-visited neighbors instead of using the full k-hop neighborhood. This bounds memory per node.
- **Importance pooling.** Neighbors are weighted by visit counts. The paper credits this with a 46% gain in offline metrics.
- **Curriculum training.** Harder negatives are introduced progressively, for a 12% gain [17].

The resulting Pin embeddings are served with nearest-neighbor search for related-Pin and homefeed candidate generation. This is a good example of graph structure being "compiled" into vectors so that standard ANN serving can be used.

3.3 Learned two-tower retrieval for Homefeed

Problem. Pinterest's older candidate generators were heuristic and graph-based. A learned retriever lets the model optimize the engagement objective directly [10].

Architecture.

- A standard two-tower model with no explicit user-Pin interaction features.
- The **Pin tower** encodes visual, text and statistical features. Its embeddings are precomputed offline and pushed to Pinterest's in-house ANN system, **Manas**.
- The **user tower** encodes engagement sequences and context. It runs once per Homefeed request [10][11].
- Training uses in-batch negatives with a popularity-bias (logQ-style) correction [10]. The logQ detail is reported by a secondary summary.

Operations. Each two-tower model is split into **two artifacts deployed to separate services**: the user-embedding service and the index or Pin-embedding pipeline. Auto-retraining must keep the model versions of both services in sync. Otherwise user vectors from model version k get compared with Pin vectors from version k-1, which silently breaks retrieval [10].

Results. Secondary summaries say the learned retriever became one of the top-performing candidate generators, with the **highest user coverage** and a top-three save rate. It was deployed for Homefeed and notifications [10].

3.4 Advancements: feature crossing, ID embeddings and multi-embedding retrieval

The 2025 "Advancements in EBR at Pinterest Homefeed" post describes several upgrades [11]:

- **Feature crossing inside each tower.** MaskNet and DHEN blocks recover some of the expressiveness lost by the dot-product bottleneck. DHEN reportedly gave +0.1-0.2% engagement and more than 1% more homefeed saves (approximate; from a secondary summary).
- **Pre-trained ID embeddings.** User and Pin ID embeddings are pre-trained contrastively on a cross-surface dataset to reduce overfitting.
- **Multi-embedding retrieval.** Several user embeddings are generated per request. Implicit interests come from a differentiable clustering module, and explicit interests come from conditional retrieval [12].
- **Merging.** Each embedding's ANN results are merged by **round-robin with de-duplication**, which balances interest clusters. In the companion paper's experiments, conditional retrieval with post-filtering performed best. Serving p90 latency rose from 150 ms to 205 ms, which stayed within budget [12].
- **Reported gains.** Engagement rose 0.1% to 1.2% across metrics such as sessions, saves and clicks (approximate; secondary summary) [11].

3.5 Conditional learned retrieval

Conditional retrieval feeds an explicit condition, such as a topic or interest, into the user tower. One user can then issue several conditioned queries that still share the same overall behavior representation [13]. Pinterest's paper on bootstrapping conditional retrieval

reports that a topic-based notification feed built this way gave **+0.26% weekly active users** and beat two-tower models with filters [19]. A 2026 post on scaling conditional learned retrieval for Home Feed describes redesigning the serving infrastructure to support many conditions per request [13].

3.6 Manas: Pinterest's ANN platform

Manas started as an HNSW-based serving system:

- **Real-time HNSW.** Pins can be inserted into the HNSW graph incrementally for real-time EBR. Filtering was initially done by over-fetching K neighbors and then post-filtering [16].
- **Streaming filters.** The inverted index was reused as the filter store. Skiplist posting lists were replaced with bitmap and array-backed posting lists for faster lookups [15].
- **Quantization and SSD serving.** The 2026 "Evolving Pinterest's Embedding Retrieval Platform" post benchmarks quantization on a 100-million-embedding GraphSage dataset [14]:
  - baseline HNSW: 121 GB at 93.72% Recall@100;
  - HNSW with scalar quantization (SQ): 50 GB at 92.92%;
  - baseline IVF: 97 GB at 91.69%;
  - IVF with SQ: 25 GB at 95.71%.

  PQ compressed more but lost much more recall. Manas also implemented **SPANN** for SSD-resident indexes, with an in-memory HNSW over the cluster centroids [14].

Lesson. Scalar quantization is often the best first compression step for recommendation embeddings. Production ANN platforms tend to move from all-in-RAM HNSW toward IVF or SPANN-style designs with quantization once cost and update rates matter.

---

4. LinkedIn: GPU retrieval and LLM-based dual encoders

4.1 LiNR: model-based neural retrieval on GPUs (2024)

Problem. LinkedIn needed out-of-network post recommendations for the Feed, meaning posts from people the member does not follow. Standard KNN with post-filtering lost quality when attribute filters, such as language or freshness, removed many results [20].

Architecture. LiNR stores the **item vectors and the model weights in the same model binary**, so index construction becomes part of model training and export. It supports a **billion-sized index on GPUs** [20]. Key features:

- **Exhaustive GPU search with attribute-based pre-filtering.** Filters run before scoring, which avoids the recall loss of post-filtering. The authors note that PyTorch and TensorFlow lacked efficient pre-filtering, so they built custom indexing and filtering ops.
- **Multi-embedding retrieval** and cold-start strategies.

- **Quantization for larger indexes.** A secondary review describes 1-bit Sign-OPORP random-projection compression.
- **Near-real-time updates.** Per a secondary summary, a change-data-capture stream updates the GPU-resident index without hurting inference latency [20].

Results. About a **3% relative increase in professional daily active users** [20]. The authors present LiNR as a step toward merging retrieval and ranking into one GPU model.

```
LiNR (GPU "retrieval as a model")
 model binary = { user/query tower weights , item matrix [N x d] , attrs }
 request:  q = tower(member)          (GPU)
           mask = prefilter(attrs)    (GPU, exact)
           scores = q . items^T  over masked rows  (dense matmul / quantized)
           top-k --> ranking
```

A later LinkedIn paper (2026) calls this pattern **"GPU Retrieval as Ranking"**: the entire candidate set sits in GPU memory and every candidate is scored with dense matrix operations. It also mentions **SilverTorch**, which implements ANN search and filtering as tensor operations, using a fused int8 IVF kernel and a GPU bloom-filter index [22].

4.2 LLM dual-encoder retrieval for the Feed (2025)

LinkedIn fine-tuned **LLaMA 3**, a decoder-only causal LM, as a dual encoder. Member and post texts are encoded separately and pooled into embeddings [21]. The scale is about **2,000 candidates from a pool of hundreds of millions**, within a latency budget of a few milliseconds, at several thousand queries per second. Two findings stand out:

- Numerical features written into the prompt work better when they are **quantized** into tokens. This improved alignment between retrieval and ranking.
- The largest online gains were for **newer members** with sparse networks. This is the cold-start regime, where content-based LLM embeddings beat ID or graph signals [21].

---

5. Twitter/X: a portfolio of graph and embedding retrievers

Twitter's 2023 open-source release documents a canonical multi-source retrieval design. The For You timeline extracts about **1,500 candidate Tweets from a pool of hundreds of millions** per request. On average it is split **50% in-network and 50% out-of-network** [23].

- **In-network.** Tweets from accounts the user follows, ranked by **RealGraph**, a model of the likelihood of engagement between two users. A logistic-regression model is used for relevance [23].
- **Out-of-network:**
  - **GraphJet / UTEG.** An in-memory user-Tweet interaction graph that generates candidates through graph traversals, such as "Tweets engaged with by people you follow" [23][24].

- **SimClusters.** Sparse, interpretable embeddings over overlapping communities, discovered by a custom matrix-factorization and community-detection step over the producer-producer similarity graph [25]. A secondary source quoting Twitter puts the count at about 145k communities, updated every three weeks (approximate).
- **TwHIN.** Dense knowledge-graph embeddings for users and Tweets, served with ANN [24].

```
Twitter "For You" candidate sourcing (2023)
 in-network  : follow graph --> RealGraph-weighted (LR) ---------\
 out-network : UTEG/GraphJet traversal ---------------------------+--> ~1500
               SimClusters sparse community vectors -------------|     merged
               TwHIN dense KG embeddings (ANN) ------------------/     to ranker
```

Lesson. The interpretable sparse community vectors in SimClusters serve several products at once: Tweets, accounts and Communities [25]. One shared representation layer can feed many retrievers.

---

6. Snap: EBR for friend recommendation

Friend recommendation traditionally uses **friends-of-friends (FoF)** graph expansion. That expansion reaches little in cold-start settings and becomes expensive beyond one or two hops [26]. Snap added an **EBR source** built on GNN embeddings for billions of users. It retrieves candidates beyond two hops and also provides a natural way to rank FoF candidates. Online A/B tests showed significant gains in friendships made, in both low-density and high-density network markets [26].

Snap's GiGL paper reports that a **sampled-softmax retrieval loss** suited the friend-recommendation EBR use case. It was combined with feature-reconstruction and whitening-decorrelation losses in multi-task training [28]. A follow-up added **ANN query expansion**, which issues several queries per user. It improved recall and gave an 11% relative gain in friend reciprocated-communication metrics, serving more than 800 million monthly active users [27].

---

7. Marketplaces: Uber Eats, Etsy, Airbnb and Amazon

7.1 Uber Eats Two-Tower Embeddings (TTE)

Problem. The earlier Deep MF model was trained per city for the first-pass ranking stage. It needed thousands of weekly Spark jobs and did not scale as Uber expanded to more cities [29].

Design. A single two-tower model. Store embeddings are computed offline and indexed in Uber's search platform (SIA). The eater (query) tower runs in real time per Homefeed request, followed by ANN search over store embeddings. This cuts cost from O(q x M) real-time inference to O(M) offline plus O(q) online, where q is queries and M is stores [29]. A Feature Store Summit talk gives the latency target: retrieve the best stores out of

thousands in about **50 ms**. It also notes that about 95% of users start their journey on the Homefeed [30].

7.2 Etsy Unified Embedding

Etsy's two-tower model has a **product encoder** and a **joint query-user encoder**, scored with cosine similarity, so products can be indexed offline for ANN [31]. A notable trick is **ANN-based boosting**. Quality signals, such as price or seller quality, are appended to the vectors as extra dimensions, so the inner product adds a tuned quality boost inside the ANN search. The weights are tuned with black-box optimization. Live A/B tests showed **+5.58% search purchase rate** and **+2.63% site-wide conversion rate** [31].

7.3 Airbnb

- **2018 listing embeddings.** Airbnb learned 32-dimensional skip-gram listing embeddings for about 4.5 million active listings, from more than 800 million search click sessions. Sessions were split at 30-minute gaps, and the objective added market-level negatives and the booked listing as global context. The embeddings power similar-listing recommendations and real-time personalization features. Similar-listing carousel CTR rose 21% [32] (figures from secondary summaries of the KDD 2018 paper).
- **2025-2026 EBR for search.** A two-tower model with a listing tower computed offline daily and a query tower online. It was trained with **session-based hard negatives** rather than random negatives [33][34]. HNSW scored slightly better on recall, but Airbnb chose **IVF**. Frequent price and availability updates caused memory overhead with HNSW. With IVF, cluster assignments become an ordinary search filter in the existing search system [33]. The paper reports significant booking gains [34].

7.4 Amazon item-to-item collaborative filtering (2003)

This is the classic heuristic retriever. An offline job builds a **similar-items table** from co-purchase statistics. Item B is related to item A when buyers of A are unusually likely to buy B. Online retrieval is just a lookup over the items the user interacted with [35][36]. The 2003 paper was later chosen by IEEE Internet Computing as the paper that best withstood the "test of time" [36]. Co-engagement tables like this remain a standard source in most retrieval portfolios today. Instagram's item-CF source and Pinterest's related-Pins candidates are direct descendants.

---

8. Beyond dot products: multi-interest, tree, path and generative retrieval

Design	Org / year	Structure	Retrieval op	Key claim
MIND [37]	Alibaba / 2019	K user vectors via capsule routing	K ANN queries	Multi-interest; Tmall homepage
TDM [38]	Alibaba / 2018	Max-heap-like item tree + DNN	Layer-wise beam search	Log-cost with deep models

Design	Org / year	Structure	Retrieval op	Key claim
Deep Retrieval [39]	ByteDance / 2020	D x K discrete path codes	Beam search on paths	Near brute-force accuracy
TIGER [40]	Google / 2023	RQ-VAE semantic IDs	Autoregressive decoding	Beats random IDs; cold start
Andromeda [9]	Meta / 2024	Learned hierarchical index	Hierarchical expansion	+6% recall vs prior

- **MIND** (Multi-Interest Network with Dynamic Routing) uses capsule routing to group a user's behaviors into several interest vectors, trained with label-aware attention. Each vector issues its own ANN query. It was deployed for major traffic on the Mobile Tmall homepage [37]. Pinterest's multi-embedding work follows the same pattern [12].
- **TDM** (Tree-based Deep Model) attaches items to the leaves of a tree and trains a deep model to score tree nodes, using a max-heap-like formulation. Retrieval is a layer-wise top-down beam search, so even a model that is not an inner product achieves cost logarithmic in corpus size. The follow-up JTM learns the tree jointly with the model [38].
- **Deep Retrieval** (ByteDance) maps each item to discrete latent paths. The path codes are learned as model parameters with the same objective, and training uses EM. Beam search over paths retrieves candidates. With sub-linear cost it matched brute-force accuracy on public data and beat a well-tuned ANN baseline in production engagement [39].
- **TIGER** (generative retrieval) quantizes content embeddings, such as Sentence-T5 vectors, into hierarchical **semantic IDs** with an RQ-VAE. A seq2seq Transformer then decodes the next item's ID tokens [40]. Because similar items share code prefixes, this helps cold start. Generative retrieval is covered in more depth in a later chapter.

---

9. ANN index families

Every embedding retriever above needs a MIPS or cosine nearest-neighbor index. The main families follow.

9.1 Inverted file plus product quantization (IVF-PQ, FAISS)

Product quantization (Jegou, Douze and Schmid, TPAMI 2011) splits a d-dimensional vector into m sub-vectors and quantizes each with its own small codebook. A vector becomes m bytes, and **asymmetric distance computation** compares an uncompressed query against the compressed codes using lookup tables. The method was validated on a two-billion-vector dataset [41]. **IVF-ADC** adds a coarse quantizer (k-means centroids). Each vector goes into the inverted list of its nearest centroid, with PQ encoding the residual. A query scans only the `nprobe` nearest lists [41][42].

FAISS (Meta, open source since 2017) is the reference implementation [44][43]. The GPU paper by Johnson, Douze and Jegou (2017) introduced a k-selection algorithm reaching about 55% of theoretical peak and an 8.5x speedup over the prior GPU state of the art. It built a k-NN graph over 95 million YFCC100M images in 35 minutes, and over 1 billion

vectors in under 12 hours on 4 Maxwell Titan X GPUs [42]. The 2024 "The Faiss library" paper documents the design space: flat, IVF, PQ, OPQ, HNSW and their compositions [43]. Facebook EBR and LiNR's successors both use IVF with quantized codes [6][22].

9.2 ScaNN and SOAR (Google)

ScaNN introduced **anisotropic vector quantization**. Quantization error parallel to the data point is penalized more than orthogonal error, because parallel error distorts the inner products that matter for top-k MIPS. On ann-benchmarks glove-100-angular at launch, it handled about twice the QPS of the next-best library at equal recall [45]. **SOAR** (NeurIPS 2023) adds **spilling**: each vector is assigned to multiple clusters, and a modified loss encourages the secondary assignment's residual to be orthogonal to the primary one. Google reports the best speed-memory-recall trade-off on glove-100. Libraries close in query speed needed more than 10x the memory and 50x the indexing time [46].

9.3 HNSW

Hierarchical Navigable Small World graphs (Malkov and Yashunin, 2016/TPAMI 2018) build a multi-layer proximity graph. Each element's top layer is drawn from an exponentially decaying distribution. Search starts greedily at the sparse top layer and descends layer by layer, giving roughly logarithmic scaling [47]. Later work notes that the logarithmic behavior is hard to achieve consistently in high dimensions [47].

HNSW gives excellent recall and latency in RAM. Its costs are memory overhead, slow builds and awkward deletes and updates. Pinterest's Manas started on HNSW [16], and Airbnb rejected it because of its update-memory overhead [33]. **Spotify's Voyager** (2023), built on hnswlib, replaced Spotify's tree-based Annoy. Spotify reports up to 10x the speed of Annoy at similar recall, or up to 50% higher accuracy at the same speed. It has Python and Java bindings and is queried hundreds of millions of times per day [48].

9.4 DiskANN and SSD-resident indexes

DiskANN (Microsoft, NeurIPS 2019) builds a **Vamana** graph with a smaller search radius than HNSW or NSG, which keeps SSD reads per query low. Compressed PQ vectors stay in RAM, and full-precision vectors and the graph live on SSD. On SIFT1B, a single 64 GB-RAM, 16-core machine served more than 5,000 QPS at under 3 ms mean latency with at least 95% 1-recall@1. It indexes 5-10x more points per node than in-memory graphs in the high-recall regime [49]. **FreshDiskANN** adds concurrent real-time inserts, deletes and searches, with thousands of each per second, on a billion-point index. It reports a 5-10x lower cost of maintaining freshness [50]. Pinterest's adoption of SPANN, which keeps an in-memory centroid index and SSD posting lists, follows the same economics [14].

9.5 Comparison of ANN families

Family	Example impl.	Memory	Updates	Filtering	Strength	Weakness
Flat / brute force	FAISS Flat, LiNR GPU	Full vectors	Trivial	Exact pre-filter	Exact; GPU-friendly	O(N) per query

Family	Example impl.	Memory	Updates	Filtering	Strength	Weakness
IVF-Flat / IVF-SQ	FAISS, Airbnb, Manas	Medium	Easy (append)	Cluster ID as filter	Simple; index-friendly	Recall depends on nprobe
IVF-PQ	FAISS, FB EBR	Very low (m B/vec)	Easy	Terms + payloads	Billions in RAM	Quantization recall loss
Anisotropic VQ + spill	ScaNN / SOAR	Low	Rebuild-oriented	Limited	Top MIPS benchmarks	Google-centric tooling
HNSW	hnswlib, Voyager, Manas	High (graph + vectors)	Inserts OK, deletes hard	Post-filter typical	Best in-RAM recall/latency	Memory; churn
Vamana / DiskANN	DiskANN, FreshDiskANN	RAM: PQ; SSD: graph	Fresh variant	Filtered variant	Billions per node on SSD	SSD I/O tuning
SPANN	Manas (2026)	RAM centroids + SSD	Moderate	Via postings	Cheap at scale	Long-tail latency

---

10. Multi-source retrieval and merging

Almost every system above combines several retrievers. The recurring patterns are:

1. **Quota or weight mixing.** Instagram Explore mixes heuristic and ML sources with tunable weights per user group [7]. Twitter targets roughly a 50/50 in-network/out-of-network split [23].
2. **Round-robin with de-duplication.** Pinterest's multi-embedding retrieval interleaves results from each interest embedding and de-duplicates them, to balance interest coverage [12].
3. **A learned pre-ranker as the merger.** Instead of hand-tuned quotas, all candidates go through a cheap shared scorer. Instagram's first-stage two-tower ranker cuts about 1,500 candidates to 100 [8]. Pinterest has also written about a unified lightweight two-tower scoring stage for Home Feed.
4. **Source-specific strengths:**
    - graph walks give freshness and serendipity (Pixie, GraphJet);
    - item-to-item CF gives precise, explainable "more like this" (Amazon, Instagram);
    - EBR gives generalization and long-tail coverage (Facebook, Pinterest);
    - social-graph expansion gives explicit relationships (Twitter in-network, Snap FoF);
    - heuristics give guarantees for trending, fresh or local content (Instagram trending).
5. **Pre-filter vs. post-filter.** Post-filtering ANN results, such as by language, availability or policy, wastes recall when filters are selective. Systems respond in different ways:
    - pushing filters into the index (Facebook's Unicorn terms; Airbnb's IVF cluster as a filter);
    - streaming bitmap filters (Manas);
    - over-fetching (early Manas);
    - exact pre-filtering under GPU brute force (LiNR) [6][15][20][33].

```
Generic multi-source retrieval service
request --> fan-out (parallel, per-source timeout)
   |-- EBR source(s): user tower -> ANN (k per embedding) -----\
   |-- item-CF: recent items -> co-engagement table lookups ---|
   |-- graph walk: seeds -> random walks / traversals ---------+--> union
   |-- social: follow / FoF expansion --------------------------|   dedup
   |-- heuristic: trending / fresh / local ---------------------/   filter
                                                                     |
                    quotas / weights / round-robin / pre-ranker ------+
                                                                      v
                                                          1e2..1e3 to ranking
```

---

11. System-level comparison of production retrieval designs

System	Retrieval type	Corpus scale	Output size	Index / engine	Notable trick
YouTube DNN 2016 [1]	Softmax-trained embeddings	Millions of videos	Hundreds	NN search	Sampled softmax
YouTube NDR 2019 [2]	Two-tower	Tens of millions	n/a	ANN	In-batch + streaming logQ
Facebook Search EBR [6]	Two-tower, triplet loss	Billions of docs	n/a	FAISS IVF-PQ in Unicorn	Hard negatives; ANN in inverted index
Instagram Explore [7][8]	Multi-source + two-tower	Billions of media	<= ~1,500 -> 100	Cached towers	Weighted source mix
Meta Andromeda [9]	Learned hierarchical index	Very large ad pool	Few thousand (approx.)	Grace Hopper GPUs	Joint index + model training
Pinterest Pixie [18]	Random walk	3B nodes / 17B edges	Top-k pins	Single-machine RAM	p99 < 60 ms, ~100k QPS cluster
Pinterest learned retrieval [10][11]	Two-tower, multi-embedding	Billions of Pins	n/a	Manas (HNSW, IVF, SPANN)	Round-robin merge; version sync
LinkedIn LiNR [20]	GPU exhaustive + pre-filter	~1B items	n/a	Model binary on GPU	Pre-filtering; CDC updates
LinkedIn LLM retrieval [21]	LLaMA 3 dual encoder	Hundreds of millions	~2,000	ANN	Quantized numeric tokens
Twitter/X For You [23]	Graph + embeddings	Hundreds of millions	~1,500	GraphJet, SimClusters, TwHIN	50/50 in/out-of-network
Snap friend EBR [26][27]	GNN embeddings	Billions of users	n/a	ANN	Query expansion
Uber Eats TTE [29]	Two-tower	Thousands per city	n/a	SIA ANN	One model replaces per-city models
Etsy UEPPR [31]	Two-tower	Large catalog	n/a	ANN	Quality boost as extra dims
Airbnb EBR [33]	Two-tower	Millions of listings	n/a	IVF as filter	Session hard negatives

---

Key takeaways for system designers

1. **Factor for retrieval, interact for ranking.** Keep the retrieval score decomposable, such as a dot product, a tree or a path code. Push feature interactions into the towers (MaskNet or DHEN crossing at Pinterest [11]) or into the next stage. Learned indexes (Andromeda [9], TDM [38], Deep Retrieval [39]) and GPU brute force (LiNR [20]) are the escape routes when the dot product becomes the bottleneck.
2. **Negative sampling is the model.**
    - In-batch negatives are cheap but popularity-biased. Always apply a logQ-style correction [2], and check how it treats the positive [5].
    - Add uniform corpus negatives so unseen items are not left unconstrained [3].
    - Use memory banks to get more negatives cheaply [4].
    - Mine hard negatives carefully. Shown-but-not-clicked items can destroy recall when used as retrieval negatives [6].
3. **Measure the ANN gap separately from the model gap.** Track exact-KNN recall and ANN recall such as 1-Recall@10 [6]. Re-tune nprobe, cluster count and quantization whenever the model changes. Scalar quantization is often almost free, and PQ often is not [14].
4. **Choose the index for your update pattern, not only for benchmark recall.** HNSW wins in-RAM benchmarks but struggles with heavy churn, which is why Airbnb chose IVF [33]. SSD-resident designs (DiskANN, SPANN) cut cost for billion-scale corpora [49][14]. Pushing filters into the index beats post-filtering for selective constraints [6][20].
5. **Version both towers atomically.** User-tower serving and item-index builds must use the same model version. Make that a deployment invariant [10].
6. **Run a portfolio.** Combine EBR with item-CF, graph walks, social expansion and heuristics, then merge them with quotas, round-robin or a learned pre-ranker [7][12] [23]. Each source covers a different failure mode: cold start, freshness, diversity or explicit relationships.
7. **One user vector is not enough for diverse users.** Multi-interest embeddings (MIND [37], Pinterest [12]) and query expansion (Snap [27]) raise recall at a moderate latency cost. Pinterest reported p90 going from 150 to 205 ms [12].
8. **Content and LLM encoders fix cold start.** Text-based LLM dual encoders helped LinkedIn's newest members most [21]. Semantic IDs give generative retrieval a shared structure for new items [40].
9. **Precompute and cache aggressively.** Item embeddings are computed offline. User embeddings are computed once per request or cached (Instagram caches both sides [7]). Retrieval latency budgets are tens of milliseconds, for example Uber's 50 ms [30].
10. **Align retrieval with ranking.** Train early stages to predict what later stages would rank highly (Instagram [7]), and feed retrieval embeddings into rankers as features (Facebook [6], Airbnb [32]). Retrieval that returns items the ranker never picks is wasted capacity.

References

1. Covington, P., Adams, J., Sargin, E. "Deep Neural Networks for YouTube Recommendations." Google, RecSys 2016. https://research.google/pubs/pub45530
2. Yi, X., et al. "Sampling-Bias-Corrected Neural Modeling for Large Corpus Item Recommendations." Google, RecSys 2019. https://research.google/pubs/sampling-bias-corrected-neural-modeling-for-large-corpus-item-recommendations/
3. Yang, J., Yi, X., et al. "Mixed Negative Sampling for Learning Two-tower Neural Networks in Recommendations." Google, WWW 2020 Companion. https://research.google/pubs/mixed-negative-sampling-for-learning-two-tower-neural-networks-in-recommendations/
4. Wang, J., Zhu, J., He, X. "Cross-Batch Negative Sampling for Training Two-Tower Recommenders." Huawei Noah's Ark Lab, SIGIR 2021. https://arxiv.org/abs/2110.15154v1
5. Khrylchenko, K., et al. "Correcting the LogQ Correction: Revisiting Sampled Softmax for Large-Scale Retrieval." Yandex, RecSys 2025. https://arxiv.org/pdf/2507.09331
6. Huang, J.-T., et al. "Embedding-based Retrieval in Facebook Search." Facebook, KDD 2020. https://arxiv.org/pdf/2006.11632
7. Meta Engineering. "Scaling the Instagram Explore recommendations system." Meta, 2023. https://engineering.fb.com/2023/08/09/ml-applications/scaling-instagram-explore-recommendations-system/
8. Meta Transparency Center. "Instagram Explore AI system." Meta, accessed 2026. https://transparency.meta.com/features/explaining-ranking/ig-explore/
9. Meta Engineering. "Meta Andromeda: Supercharging Advantage+ automation with the next-gen personalized ads retrieval engine." Meta, 2024. https://engineering.fb.com/2024/12/02/production-engineering/meta-andromeda-advantage-automation-next-gen-personalized-ads-retrieval-engine/
10. Pinterest Engineering. "Establishing a Large Scale Learned Retrieval System at Pinterest." Pinterest, 2025. https://medium.com/pinterest-engineering/establishing-a-large-scale-learned-retrieval-system-at-pinterest-eb0eaf7b92c5
11. Pinterest Engineering. "Advancements in Embedding-Based Retrieval at Pinterest Homefeed." Pinterest, 2025. https://medium.com/pinterest-engineering/advancements-in-embedding-based-retrieval-at-pinterest-homefeed-d7d7971a409e
12. Pinterest. "Synergizing Implicit and Explicit User Interests: A Multi-Embedding Retrieval Framework at Pinterest." KDD 2025. https://arxiv.org/pdf/2506.23060
13. Pinterest Engineering. "Scaling Conditional Learned Retrieval for Pinterest Home Feed." Pinterest, 2026. https://medium.com/pinterest-engineering/scaling-conditional-learned-retrieval-for-pinterest-home-feed-ecfba7e5a426
14. Pinterest Engineering. "Evolving Pinterest's Embedding Retrieval Platform." Pinterest, 2026 (summarized by InfoQ, https://www.infoq.com/news/2026/09/pinterest-search/). https://medium.com/pinterest-engineering/evolving-pinterests-embedding-retrieval-platform-aede4e831e01
15. Koh, T. "Manas HNSW Streaming Filters." Pinterest Engineering, 2022. https://medium.com/pinterest-engineering/manas-hnsw-streaming-filters-351adf9ac1c4
16. Pinterest Engineering. "Manas HNSW Realtime: Powering Realtime Embedding-Based Retrieval." Pinterest. https://medium.com/pinterest-engineering/manas-hnsw-realtime-powering-realtime-embedding-based-retrieval-dc71dfd6afdd
17. Ying, R., et al. "Graph Convolutional Neural Networks for Web-Scale Recommender Systems" (PinSage). Pinterest/Stanford, KDD 2018. https://arxiv.org/pdf/1806.01973 ;

blog: https://medium.com/pinterest-engineering/pinsage-a-new-graph-convolutional-neural-network-for-web-scale-recommender-systems-88795a107f48
18. Eksombatchai, C., et al. "Pixie: A System for Recommending 3+ Billion Items to 200+ Million Users in Real-Time." Pinterest/Stanford, WWW 2018. https://arxiv.org/pdf/1711.07601
19. Pinterest. "Bootstrapping Conditional Retrieval for User-to-Item Recommendations." 2024/2025. https://arxiv.org/html/2508.16793
20. Borisyuk, F., et al. "LiNR: Model Based Neural Retrieval on GPUs at LinkedIn." LinkedIn, 2024. https://arxiv.org/pdf/2407.13218
21. LinkedIn. "Large Scale Retrieval for the LinkedIn Feed using Causal Language Models." 2025 (AAAI 2026). https://arxiv.org/abs/2510.14223
22. LinkedIn. "Connected Content Retriever: Dense Graph Edge Features Powering Pre-Ranking at LinkedIn." 2026. https://arxiv.org/pdf/2609.22441
23. Twitter Engineering. "Twitter's Recommendation Algorithm." Twitter, 2023. https://blog.x.com/engineering/en_us/topics/open-source/2023/twitter-recommendation-algorithm
24. Twitter/X. "the-algorithm" source repository. 2023. https://github.com/twitter/the-algorithm
25. X Help Center. "Communities Recommendations" (SimClusters). X Corp. https://help.x.com/en/resources/recommender-systems/communities-recommendations
26. Snap Research. "Embedding Based Retrieval in Friend Recommendation." Snap, SIGIR 2023. https://research.snap.com/publications/embedding-based-retrieval-in-friend-recommendation.html
27. Snap Research. "Improving Embedding-Based Retrieval in Friend Recommendation with ANN Query Expansion." Snap. https://research.snap.com/publications/improving-embedding-based-retrieval-in-friend-recommendation-with-ann-query-expansion.html
28. Snap. "GiGL: Large-Scale Graph Neural Networks at Snapchat." 2025. https://arxiv.org/pdf/2502.15054
29. Uber Engineering. "Innovative Recommendation Applications Using Two Tower Embeddings at Uber." Uber. https://eng.uber.com/blog/innovative-recommendation-applications-using-two-tower-embeddings/
30. Kurra, D. D. "Uber - Embeddings" (Feature Store Summit 2024 talk). Uber, 2024. https://content.hopsworks.ai/hubfs/Feature%20Store%20Summit%202024/FS%20Summit%2024%20-%20Uber%20-%20Embeddings.pdf
31. Jha, R., et al. "Unified Embedding Based Personalized Retrieval in Etsy Search." Etsy, 2023/2024. https://arxiv.org/abs/2306.04833v2
32. BestPractice.AI case study of Grbovic and Cheng, "Real-time Personalization using Embeddings for Search Ranking at Airbnb" (KDD 2018), secondary summary. https://bestpractice.ai/ai-use-cases/case-studies/consumer-retail/airbnb-increased-similar-property-listing-recommendations-click-through-rates-by-21-with-machine-learning-embeddings
33. Airbnb Engineering. "Embedding-Based Retrieval for Airbnb Search." Airbnb, 2025. https://airbnb.tech/ai-ml/embedding-based-retrieval-for-airbnb-search/
34. Airbnb. "Applying Embedding-Based Retrieval to Airbnb Search." 2026. https://arxiv.org/pdf/2601.06873
35. Linden, G., Smith, B., York, J. "Amazon.com Recommendations: Item-to-Item Collaborative Filtering." IEEE Internet Computing, 2003. https://www.cs.umd.edu/%7Esamir/498/Amazon-Recommendations.pdf

36. Amazon Science. "The history of Amazon's recommendation algorithm." Amazon. https://www.amazon.science/the-history-of-amazons-recommendation-algorithm
37. Li, C., et al. "Multi-Interest Network with Dynamic Routing for Recommendation at Tmall" (MIND). Alibaba, 2019. https://arxiv.org/abs/1904.08030v1
38. Zhu, H., et al. "Learning Tree-based Deep Model for Recommender Systems" (TDM). Alibaba, KDD 2018. https://arxiv.org/pdf/1801.02294 ; follow-up JTM: https://arxiv.org/pdf/1902.07565
39. Gao, W., et al. "Deep Retrieval: Learning A Retrievable Structure for Large-Scale Recommendations." ByteDance, 2020/2021. https://arxiv.org/abs/2007.07203v1
40. Rajput, S., et al. "Recommender Systems with Generative Retrieval" (TIGER). Google, NeurIPS 2023. https://ar5iv.labs.arxiv.org/html/2305.05065
41. Jegou, H., Douze, M., Schmid, C. "Product Quantization for Nearest Neighbor Search." INRIA, IEEE TPAMI 2011. https://hal-univ-rennes1.archives-ouvertes.fr/inria-00514462
42. Johnson, J., Douze, M., Jegou, H. "Billion-scale similarity search with GPUs." Facebook AI Research, 2017. https://arxiv.org/pdf/1702.08734
43. Douze, M., et al. "The Faiss library." Meta FAIR, 2024. https://arxiv.org/abs/2401.08281
44. Meta Engineering. "Faiss: A library for efficient similarity search." Facebook, 2017. https://engineering.fb.com/2017/03/29/data-infrastructure/faiss-a-library-for-efficient-similarity-search/
45. Google Research. "Announcing ScaNN: Efficient Vector Similarity Search." Google, 2020. https://research.google/blog/announcing-scann-efficient-vector-similarity-search/
46. Google Research. "SOAR: New algorithms for even faster vector search with ScaNN." Google, 2024. https://research.google/blog/soar-new-algorithms-for-even-faster-vector-search-with-scann/
47. Malkov, Y. A., Yashunin, D. A. "Efficient and robust approximate nearest neighbor search using Hierarchical Navigable Small World graphs." arXiv 2016 / TPAMI 2018. https://arxiv.org/pdf/1603.09320
48. Spotify Engineering. "Introducing Voyager: Spotify's New Nearest-Neighbor Search Library." Spotify, 2023. https://engineering.atspotify.com/introducing-voyager-spotifys-new-nearest-neighbor-search-library
49. Subramanya, S. J., et al. "DiskANN: Fast Accurate Billion-point Nearest Neighbor Search on a Single Node." Microsoft Research, NeurIPS 2019. https://papers.neurips.cc/paper/9527-rand-nsg-fast-accurate-billion-point-nearest-neighbor-search-on-a-single-node.pdf
50. Singh, A., et al. "FreshDiskANN: A Fast and Accurate Graph-Based ANN Index for Streaming Similarity Search." Microsoft Research, 2021. https://www.microsoft.com/en-us/research/?p=905277

03. The Evolution of Ranking and CTR Models: From Logistic Regression to Scaling-Law Architectures

Overview

The ranking stage is where most of a recommender's modeling budget is spent. Retrieval narrows millions of items to hundreds or a few thousand candidates; the ranker then scores every candidate with a rich model that predicts click-through rate (CTR), conversion rate (CVR), or a vector of engagement probabilities. Because ranking quality converts almost directly into revenue in ads and into engagement in feeds, this is the most published-on component of industrial recommendation.

The history of ranking models is mostly a history of one design question: how should a model learn interactions between sparse, high-cardinality categorical features (user id, item id, category, query, context) and a smaller set of dense features? The answers fall into roughly five eras:

1. Linear models with hand-crafted crosses, trained online at huge scale (Google FTRL-Proximal, 2013 [1]).
2. Automated non-linear feature transforms and factorized interactions (Facebook GBDT+LR, 2014 [3]; Factorization Machines [4]; FFM [6]).
3. Hybrid "explicit interaction + deep MLP" networks (Wide & Deep [8], DeepFM [9], DCN [10], xDeepFM [13], AutoInt [14], DLRM [15]).
4. User-behavior-sequence modeling with target attention (Alibaba DIN [17], DIEN [18], SIM [19], Kuaishou TWIN [20]).
5. Scaling-oriented architectures that treat ranking like a foundation model (DCN-V2 [11], DHEN [21], Wukong [22], HSTU [23], InterFormer [24], RankMixer [30], Meta GEM [25]).

This chapter walks through each design with its problem statement, architecture, scale numbers where public, key tricks, and lessons, then closes with a comparison table and takeaways.

```
Evolution of CTR / ranking architectures (simplified)

2010  FM (Rendle) ---------------------------------------+
2013  LR + FTRL-Proximal (Google ads)                    |
2014  GBDT -> LR (Facebook ads)      FFM (Kaggle/Criteo) |
2016  Wide & Deep (Google Play)                          |
2017  DeepFM (Huawei)  DCN (Google)  DIN (Alibaba) <-----+ factorized / attention
2018  xDeepFM (CIN)  AutoInt (self-attn)  DIEN (AUGRU)
2019  DLRM (Facebook, open source)
2020  DCN-V2 (Google)  SIM (Alibaba, 54k-length sequences)
2021  MaskNet (Weibo) -> Twitter heavy ranker (2023)
2022  DHEN (Meta)
2023  TWIN (Kuaishou)
2024  Wukong (Meta)  HSTU (Meta)  LiRank (LinkedIn)  InterFormer (Meta)
2025  RankMixer (ByteDance)  GEM (Meta ads foundation model)
2026  TokenMixer-Large (ByteDance)  ML-DCN (Pinterest)  GEM training at LLM scale
```

1. The Linear Era: Logistic Regression and FTRL-Proximal (Google, 2013)

Problem

Google search ads needed a CTR model that could be trained online on an enormous stream of impressions, with billions of sparse features, while staying small enough in memory to serve at very high QPS. Calibration matters as much as ranking, because ad auctions multiply predicted CTR by bids.

Architecture and algorithm

McMahan et al.'s KDD 2013 paper "Ad Click Prediction: a View from the Trenches" [1] describes a logistic-regression model trained with the FTRL-Proximal online learning algorithm, chosen for its combination of sparsity (via L1 regularization that produces exact zeros) and convergence properties, plus per-coordinate learning rates. Per-coordinate learning rates require tracking the sum of gradients and the sum of squared gradients for each feature, which improves accuracy at the cost of memory [2].

```
impression stream --> feature extraction --> hashed / sparse feature ids
                                                |
                                                v
                        +-----------------------------------------+
                        |  FTRL-Proximal logistic regression      |
                        |  per-coordinate state: z_i, n_i         |
                        |  L1 => exact zeros => sparse model      |
                        +-----------------------------------------+
                                  |                     |
                       weights (q2.13, 16-bit)   calibration layer
                                  |                     |
                                  v                     v
                             serving replicas  -->  pCTR to auction
```

Memory tricks (the most copied part of the paper)

- Probabilistic feature inclusion. Rare features dominate the feature space but carry little signal. Instead of admitting every feature, the paper tests Poisson inclusion (admit a new feature with probability p) and Bloom-filter inclusion (admit after the feature has been seen n times, using a rolling set of counting Bloom filters). Reported results: Poisson p = 0.03 saves 60% RAM at 0.020% AucLoss detriment; p = 0.1 saves

40% at 0.006%; Bloom with n = 2 saves 66% at 0.008%, so Bloom filtering gives the better trade-off [1][2].
- Reduced-precision weights. Coefficients are stored in a 16-bit fixed-point "q2.13" encoding with randomized rounding; the authors report no measurable loss versus 64-bit floats, and summaries attribute roughly 75% RAM savings for coefficient storage to the combination of tricks [1][2]. The same group's companion paper "Large-Scale Learning with Less RAM via Randomization" formalizes the randomized-rounding idea [39].
- Training many model variants simultaneously, sharing feature-index structures, and subsampling training data (keeping all clicks but only a fraction of non-click queries, with importance weights) are also described [1].

Negative results

A distinctive contribution is the list of ideas that did not pay off in production despite good literature results. Feature hashing, for instance, needed too large a table to avoid damaging collisions [2]. This "what did not work" reporting style became a model for later industrial papers.

Lessons

- A linear model can be extremely strong when the feature engineering (crosses) is rich and the data fresh; its weakness is that humans must design the crosses.
- Memory, not FLOPs, was the binding constraint. Feature admission and low-precision storage remain standard in every modern embedding system (they reappear as embedding admission policies, frequency thresholds, and FP16/INT8 embedding tables).
- Calibration is a first-class output, not an afterthought.

2. Automating Feature Crosses: GBDT + LR at Facebook (2014)

Problem

Facebook ads needed better CTR accuracy than a linear model with hand-built crosses, while still training online on fresh data.

Architecture

He et al. ("Practical Lessons from Predicting Clicks on Ads at Facebook", ADKDD 2014) [3] combine gradient-boosted decision trees with logistic regression. Each boosted tree acts as a learned, non-linear feature transform: the index of the leaf an example lands in becomes a one-hot categorical feature, and the concatenation of leaf indices across all trees is fed to a linear classifier.

```
raw features (dense + categorical)
         |
         v
+----------------------------+     trained in batch (e.g. daily)
|  Boosted trees T1 ... TK   |
+----------------------------+
  | leaf id   | leaf id    | leaf id
  v           v            v
[one-hot(T1) | one-hot(T2) | ... | one-hot(TK)]  + other sparse features
         |
         v
+----------------------------+     trained online, near real time
| Logistic regression (SGD)  |
+----------------------------+
         |
         v
    pCTR  --> recalibrate for negative downsampling --> auction
```

Key findings

- The hybrid beats either trees or LR alone by over 3% in normalized entropy (NE) [3]; summaries report LR + trees at about 96.6% of the trees-only NE and a 3.4% NE improvement from the tree transform [3][40].
- Normalized entropy (log loss divided by the entropy of the background CTR) is introduced as the primary metric because raw log loss looks deceptively good when base CTR is near 0 or 1 [40].
- Feature choice matters most: historical features describing the user's or ad's past behavior dominate contextual features. Data freshness, learning-rate schedule, and data sampling help only modestly by comparison [3].
- Data freshness: shorter delay between training and serving lowers NE, which motivates training the LR part online while retraining trees less frequently [40].
- Negative downsampling speeds training but requires re-calibration: with negative sampling rate w and in-sample prediction p, the live-space estimate is q = p / (p + (1 - p) / w) [3][40].
- Subsampling to 10% of the data reportedly costs about 1% NE (secondary summary) [40].

Lessons

GBDT+LR was the first widely-copied "learned feature crossing" design and the template for splitting a model into a slow-trained feature transformer and a fast-trained head. That split reappears later as frozen embeddings/towers plus online-updated top layers.

3. Factorized Interactions: FM and FFM

Factorization Machines (Rendle, ICDM 2010)

FM [4] models every pairwise interaction between features i and j as the dot product of two learned latent vectors, <v_i, v_j>, instead of a free weight w_ij. This lets the model estimate interactions between feature pairs that never co-occurred in training, which is essential under extreme sparsity, and it generalizes matrix factorization and several specialized factorization models through choice of input encoding [4]. The pairwise term looks quadratic in the number of features n, but can be rewritten as one half of the sum over

latent dimensions of (square of sum minus sum of squares), giving O(k*n) cost where k is the latent dimension [5][41].

Field-aware FM (Juan et al., RecSys 2016)

FFM [6] gives each feature a separate latent vector per field it interacts with: the interaction between feature i (field F_i) and feature j (field F_j) uses <v_{i,F_j}, v_{j,F_i}>. This increases parameters by roughly the number of fields but lets the model represent that, for example, "advertiser" interacts differently with "publisher" than with "gender". FFM-based solutions won two Kaggle CTR competitions in 2014, including the Criteo display-ad challenge [6]. Criteo later reported running FFM in a production display-ads system for CTR and conversion prediction, adding a seeding algorithm and a distributed learning mechanism to cut training time [7].

Lessons

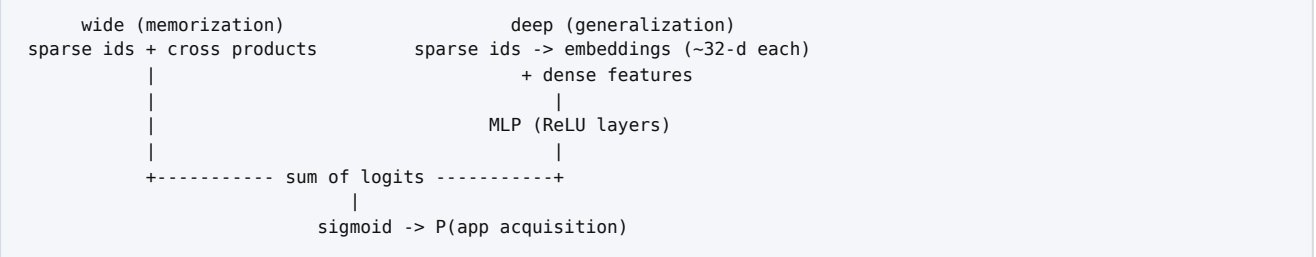
- FM introduced the idea that would dominate the next decade: every feature gets an embedding, and interactions are computed from embeddings rather than memorized as cross weights.
- FFM showed that interaction structure should respect field semantics, at a large parameter cost. Later designs (FwFM, DCN-V2's full-rank matrix, field-specific token mixing in RankMixer) are all attempts to get field awareness more cheaply.

4. Hybrid Deep Models: Memorization plus Generalization

4.1 Wide & Deep (Google Play, 2016)

Problem: Google Play app recommendations, with over one billion active users and over one million apps [8]. Linear models with cross-product features memorize well but generalize poorly to unseen combinations; pure embedding models generalize but over-generalize for niche users.

Architecture: a jointly trained wide linear part (raw sparse features plus cross-product transformations such as AND(user_installed_app, impression_app)) and a deep part (categorical features mapped to dense embeddings, concatenated with dense features, passed through a ReLU MLP). Their logits are summed and passed through a sigmoid. The wide part is trained with FTRL with L1 regularization; the deep part with AdaGrad [8].



Scale and serving: models were trained on over 500 billion examples, with warm-starting from the previous model so that each retrain does not start from scratch. Scoring all candidates in a single-threaded batch took 31 ms; splitting into smaller batches across

threads brought client-side latency to 14 ms (batch 50, 4 threads), against a serving target on the order of 10 ms [8][42]. A three-week live A/B test showed +3.9% app acquisitions over a wide-only control and about +1% over deep-only [8][42].

Lesson: joint training of a memorization path and a generalization path became the default template. Its weakness is that the wide part still requires hand-designed crosses.

4.2 DeepFM (Huawei, 2017)

DeepFM [9] replaces the hand-crafted wide part with an FM component, and crucially shares the same embedding layer between the FM and the DNN. Because both parts take the same raw input, no feature engineering beyond raw features is needed. It reported AUC and logloss improvements over Wide & Deep variants on Criteo and on a commercial (Huawei) app-store dataset, with training efficiency comparable to the fastest deep baselines [9].

4.3 Deep & Cross Network (Google, 2017)

DCN [10] introduces a cross network in which each layer explicitly multiplies the original input x0 with the current layer's output, so the maximum polynomial degree of interaction grows by one with each layer. In the original formulation, each cross layer used a weight vector, making it very cheap (parameters linear in input width). The paper reports lower logloss than a plain DNN with far fewer parameters and no manual feature engineering [10].

4.4 xDeepFM and the Compressed Interaction Network (Microsoft, 2018)

xDeepFM [13] argues that DCN's crosses operate "bit-wise" (on individual embedding elements) and proposes the Compressed Interaction Network (CIN), which generates interactions "vector-wise" (treating each field embedding as a unit). Each CIN layer takes outer products between the previous hidden feature maps and the base field embeddings, then compresses them with convolution-like filters, so interaction degree grows with depth. xDeepFM combines a linear part, CIN, and a DNN. A commonly noted downside is CIN's time complexity, about an order of magnitude higher than a plain DNN (secondary analysis) [13][43], which limited its adoption in latency-constrained production systems.

4.5 AutoInt (2018)

AutoInt [14] maps all numerical and categorical features into a shared low-dimensional space and applies stacked multi-head self-attention with residual connections over the field embeddings. Each attention layer models interactions among fields; stacking layers increases the order. A side benefit is interpretability: attention weights show which field pairs interact [14]. AutoInt was an early indicator of the later move toward Transformer-style token mixing over feature "tokens".

4.6 DLRM (Facebook/Meta, 2019)

Problem: provide a reference architecture and benchmark that reflects production personalization models, which are dominated by huge embedding tables rather than dense

compute.

Architecture [15][16]:

```
dense features --> bottom MLP --> d-dim vector ----+
                                                   |
sparse feat 1 --> embedding table 1 --> d-dim -----+
sparse feat 2 --> embedding table 2 --> d-dim -----+--> pairwise dot products
...                                                |    (all pairs of vectors)
sparse feat N --> embedding table N --> d-dim -----+          |
                                                              v
                                concat(dot products, bottom-MLP output)
                                                              |
                                                         top MLP
                                                              |
                                                          sigmoid
```

- The bottom MLP projects dense features to the same dimension as embeddings so that all vectors can be dotted together [16][44].
- Interactions are deliberately limited to second order (dot products), in the spirit of FM, which keeps the interaction cheap and dimensionality small; the authors contrast this with DCN-style higher-order crossing [16].
- Parallelism: embedding tables are model-parallel (sharded across devices because they do not fit on one), while the MLPs are data-parallel. Moving from the model-parallel embedding lookup to the data-parallel interaction and top MLP requires a personalized all-to-all communication [16][44].
- Released open source in July 2019 with PyTorch and Caffe2 implementations [15]; the repository later added MLPerf benchmark support and TorchRec integration [15].

Lesson: DLRM made explicit that industrial recommenders are memory- and communication-bound, and established the "embeddings + bottom MLP + interaction + top MLP" skeleton that DHEN, Wukong and many internal models extend.

4.7 DCN-V2 (Google, 2020)

Problem: the original DCN's vector-parameterized cross layer was too weak to learn predictive crosses at web scale [11].

Architecture: the cross layer becomes x_{l+1} = x0 * (W_l x_l + b_l) + x_l, where * is element-wise product and W_l is a full matrix. This makes the cross network much more expressive while retaining bounded-degree, explicit crosses. Cross and deep networks can be combined either stacked (cross output feeds the DNN) or in parallel (outputs concatenated) [11][12].

Efficiency trick: the authors observed that learned W matrices are effectively low-rank, so they factor W = U V^T with rank r much smaller than d, reducing per-layer cost from O(d^2) to roughly O(2dr), and further use a mixture of low-rank experts with an input-dependent gate (often called DCN-Mix) [11][45]. The paper reports state-of-the-art results on public benchmarks and significant offline accuracy and online business-metric gains across many web-scale learning-to-rank systems at Google; it also stresses that production must respect extremely high QPS and strict latency [11].

Industrial echoes:

- TensorFlow Recommenders ships DCN as a first-class layer with a tutorial [12].
- LinkedIn's LiRank (KDD 2024) extends DCNv2 with attention and residual connections ("Residual DCN"), uses low-rank matrices, and reports nearly 30% parameter reduction; it also introduces an isotonic calibration layer co-trained inside the deep model, used across Feed, Jobs and Ads CTR [29].
- Pinterest's ads models used DCNv2 as the production baseline when adding Transformer user-sequence features, and reported DCNv2 cross-layer overflow issues under float16 training [32]. In 2026 Pinterest reported that naively scaling DCNv2 and MaskNet gave rapidly diminishing offline gains, and proposed ML-DCN (an instance-conditioned mask on a low-rank crossing layer), deployed with neutral serving cost [34].

4.8 MaskNet and the Twitter heavy ranker

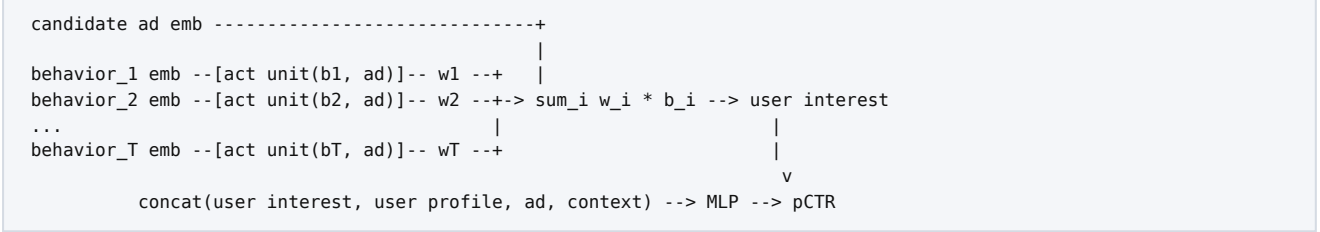
MaskNet (Sina Weibo, 2021) [27] argues plain feed-forward layers are inefficient at learning multiplicative crosses and introduces the instance-guided mask: an element-wise product applied to embeddings and hidden layers, with mask values computed from the input instance. A MaskBlock combines layer normalization, the mask, and a feed-forward layer; analysis on Criteo shows over half of mask values near zero, acting like bit-wise attention [27]. When Twitter open-sourced its recommendation algorithm in 2023, its heavy ranker was a roughly 48M-parameter neural network, continuously trained, that predicts about ten engagement probabilities (likes, retweets, replies, and so on) on about 1,500 candidates per request [28]; secondary analyses of the released code identify it as a parallel MaskNet [28].

5. User-Behavior Modeling: The Alibaba Line (DIN, DIEN, SIM) and Kuaishou TWIN

Feature-interaction networks treat a user's history as a bag of features pooled into a fixed vector. In e-commerce display ads, that loses information: a user's interest in running shoes is irrelevant when scoring a kitchen appliance. Alibaba's line of work makes the user representation a function of the candidate item.

5.1 DIN: Deep Interest Network (KDD 2018)

Architecture: a local activation unit computes, for each historical behavior, a relevance weight with respect to the candidate ad, then takes a weighted sum pooling of behavior embeddings. Behaviors related to the candidate dominate the representation [17]. The weights are not normalized by softmax, so the sum also reflects the intensity of interest (a point emphasized in the paper's design discussion [17]).



Training tricks [17]:

- Mini-batch aware regularization: L2 over billions of parameters every mini-batch is infeasible, so regularization is applied only to the parameters of features present in the mini-batch. It added 0.0031 absolute AUC over dropout on the Alibaba dataset.
- Dice activation: a data-adaptive generalization of PReLU whose rectification point depends on input distribution; +0.0015 AUC over PReLU.

Scale and results: the production dataset has over 2 billion samples; an online A/B test (May to June 2017) showed up to +10.0% CTR and +3.8% RPM over the base model; DIN was deployed to serve the main traffic of Alibaba display advertising [17].

5.2 DIEN: Deep Interest Evolution Network (AAAI 2019)

DIEN [18] adds time. An interest-extractor layer (GRU) turns the behavior sequence into latent interest states, with an auxiliary loss that supervises each hidden state with the next actual behavior so states carry real interest signal. An interest-evolving layer uses AUGRU, a GRU whose update gate is scaled by attention to the target item, so relevant interests evolve while irrelevant ones (interest drift) are suppressed [18].

Results and serving: deployed in Taobao display advertising with a reported 20.7% CTR improvement. Serving a GRU per candidate is expensive; the paper reports reducing latency from 38.2 ms to 6.6 ms using element-parallel GRU with kernel fusion, batching adjacent requests from different users on GPU, and model compression, with the system serving more than one million users per second at peak [18][46].

5.3 SIM: Search-based Interest Model (CIKM 2020)

Problem: Alibaba's earlier MIMN could handle about 1,000 behaviors, but accuracy degraded as sequences grew 10x or more [19]. Lifelong histories carry valuable long-term signal but cannot be attended over in full within latency.

Architecture: a two-stage cascade [19][47]:

```
lifelong behaviors (up to ~54,000)
         |
         v
+----------------------------------------+
| General Search Unit (GSU)              |
|   hard search: behaviors with same     |
|     category as candidate (index lookup) |
|   soft search: embedding MIPS          |
+----------------------------------------+
         | top-K relevant behaviors (K ~ hundreds)
         v
+----------------------------------------+
| Exact Search Unit (ESU)                |
|   multi-head target attention          |
+----------------------------------------+
         |
         v
  concat with short-term DIN/DIEN features --> MLP --> pCTR
```

- Sequence length up to 54,000, about 54x the previous state of the art [19][47].
- Hard search is non-parametric (category match) and was the variant deployed; soft search was only slightly better offline. Behaviors are stored in a two-level user

behavior tree keyed by user and category; a secondary summary reports this index at about 22 TB and under 5 ms of added serving latency (approximate, unverified against the paper) [47].
- Deployed in Alibaba display advertising since 2019 with 7.1% CTR and 4.4% RPM lift [19].

5.4 TWIN (Kuaishou, KDD 2023)

TWIN [20] identifies a flaw in GSU/ESU cascades: the cheap GSU relevance metric differs from the ESU's target attention, so the GSU retrieves behaviors the ESU would rank low and misses ones it would rank high. TWIN's Consistency-Preserved GSU uses the same target-attention relevance metric as the ESU (hence "twins"). To make this affordable over tens of thousands of behaviors, item features are split into item-only (cacheable, precomputed) and user-item cross parts, which the authors say cut the computational bottleneck by 99.3%. The GSU scans tens of thousands of behaviors and passes the top 100 to the ESU; offline evaluation used a 46-billion-sample production dataset, and the model is deployed on Kuaishou's main traffic [20]. TWIN-V2 later compresses lifelong sequences with hierarchical clustering [20].

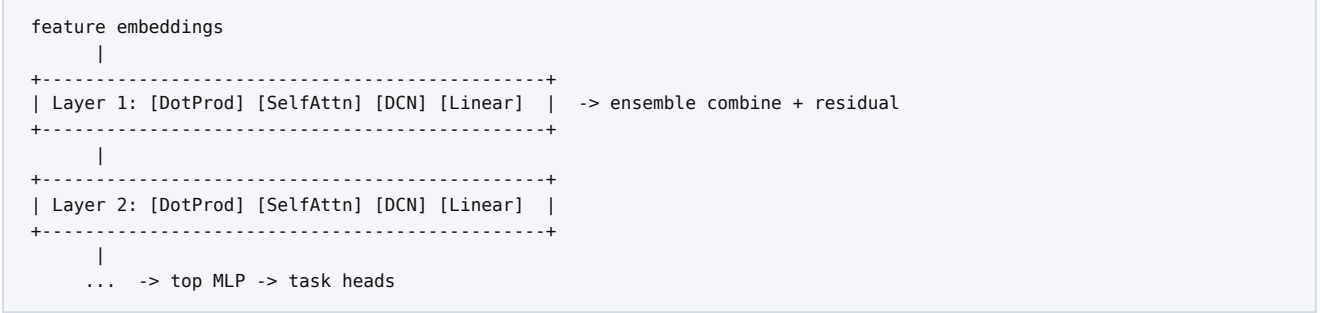
Lessons from the sequence line

- Target attention ("represent the user relative to the item") is the single most durable idea from this line; nearly every modern ranker has a DIN-style attention block.
- Long sequences are a retrieval problem inside the ranker; the GSU/ESU split mirrors the retrieval/ranking cascade of the whole system.
- Serving engineering (kernel fusion, cross-user batching, precomputed item-side projections) is as important as architecture.

6. Scaling Era: Ensembles, Stacked FMs, and Foundation Rankers

6.1 DHEN: Deep and Hierarchical Ensemble Network (Meta, 2022)

Observation: interaction modules claiming the same interaction order (e.g., DCN cross, self-attention, FM-style dot products) perform differently on different datasets, so no single module is best [21]. DHEN stacks layers, each of which is an ensemble of heterogeneous interaction modules (such as AdvancedDLRM-style dot products, self-attention, convolution, linear, DCN), with outputs combined and fed to the next layer. Stacking yields a hierarchy of interactions of increasing order [21].



Systems co-design: the deeper model is harder to train, so the paper introduces Hybrid Sharded Data Parallel (HSDP), reported at up to 1.2x throughput over fully sharded data parallel. On a large-scale CTR dataset DHEN improved normalized entropy by 0.27% over an AdvancedDLRM baseline [21]. Meta later applied DHEN to ad conversion-rate prediction [38] and built AutoML over DHEN-style module search for its ranking models [37].

6.2 Wukong (Meta, ICML 2024)

Problem: unlike LLMs, ranking models had not shown predictable quality gains from scaling dense compute; upscaling DLRM-like or DCN-like models saturates [22].

Architecture: a stack of identical Wukong layers. Each layer has two parallel blocks whose outputs are concatenated: a Factorization Machine Block (FMB: an FM-style pairwise interaction over embeddings, followed by an MLP that emits new embeddings) and a Linear Compress Block (LCB: linear recombination of embeddings without raising interaction order). Because each layer interacts the outputs of the previous one, layer i covers interaction orders from 1 to 2^i, analogous to binary exponentiation [22]. Low-rank FM reduces cost from O(n^2 d) to O(n k d) (secondary analysis) [22].

Scaling knobs are number of layers, number of embeddings emitted by FMB and LCB, FM rank, and MLP width. Wukong beat state-of-the-art models on six public datasets, and on an internal dataset held a scaling law across two orders of magnitude of model complexity, beyond 100 GFLOP per example, where prior architectures stopped improving [22].

6.3 HSTU and Generative Recommenders (Meta, ICML 2024)

HSTU [23] reframes ranking and retrieval as sequential transduction over user action sequences, using a modified attention block (Hierarchical Sequential Transduction Unit). It reports 5.3x to 15.2x speedups over FlashAttention2-based Transformers on 8,192-length sequences, power-law quality scaling over three orders of magnitude of training compute, and 1.5-trillion-parameter models that improved online A/B metrics by 12.4% on surfaces of a platform with billions of users [23]. HSTU is covered in depth in the generative recommendation chapter; here the point is that it merges the feature-interaction line and the sequence line into one scalable architecture.

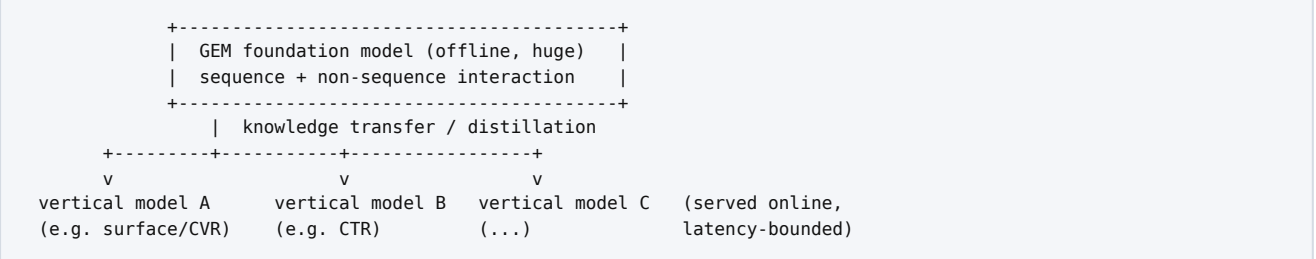
6.4 InterFormer (Meta, 2024-2025)

InterFormer [24] targets heterogeneous inputs (non-sequential features plus behavior sequences). It identifies two weaknesses of prior models: one-directional information flow between modes, and early, aggressive summarization of sequences. It interleaves an Interaction Arch (behavior-aware feature interaction) and a Sequence Arch (context-aware sequence modeling) with bidirectional flow, plus a Cross Arch for selective summarization. It reports up to 0.14% AUC gain on public benchmarks, 0.15% NE and 24% QPS gain on Meta internal data, and a 0.6% topline gain in pilot ads launches [24].

6.5 Meta GEM (2025-2026)

Meta's Generative Ads Recommendation Model (GEM), announced November 2025, is described as the largest foundation model for recommendation in the industry, trained

across thousands of GPUs on an LLM-inspired paradigm [25]. Its role is not a single online ranker: Meta describes three pillars (scaling the model architecture, post-training knowledge transfer to downstream ads models, and training scalability), with GEM's learnings transferred to the many production ranking models across the ads stack. Reported launch results were +5% ad conversions on Instagram and +3% on Facebook Feed in Q2 2025 [25]. A follow-up post (August 2026) reports doubling end-to-end training efficiency to 20-25% model FLOPs utilization while scaling training FLOPs 4x in twelve months via co-design of kernels, precision, parallelism, networking and memory [26]. Secondary coverage links GEM's architecture to InterFormer and Wukong [48].



Design lesson: when the best model is too expensive to serve, the industry pattern is "teacher foundation model + many distilled students", the same pattern LLM serving uses.

6.6 RankMixer and TokenMixer-Large (ByteDance, 2025-2026)

Problem: industrial rankers had accumulated many handcrafted interaction modules with poor hardware efficiency. RankMixer [30] reports that such models ran at about 4.5% model FLOPs utilization (MFU).

Architecture: features are grouped into tokens; each RankMixer block applies multi-head token mixing (split each token into heads and recombine across tokens, a parameter-free substitute for quadratic self-attention over heterogeneous feature spaces) and a per-token FFN with separate parameters per token (feature subspace), optionally extended into a sparse mixture-of-experts [30].

Results: MFU rose from 4.5% to 45%, ranking-model parameters scaled about 100x at roughly constant inference latency, and full-traffic deployment in Douyin feed ranking gave +0.2% active days and +0.5% app duration [30]. The successor TokenMixer-Large (2026) reports offline scaling to 7B and 15B parameters for Douyin ads and e-commerce [31].

6.7 A cautionary thread: embedding collapse and benchmark reality

- Guo et al. (Tsinghua and Tencent, ICML 2024) show that scaling embedding size often fails because embedding matrices collapse into low-dimensional subspaces; feature interaction both worsens collapse and limits overfitting. Their multi-embedding design (several independent embedding sets with set-specific interaction modules) keeps improving up to 10x embedding size [35].
- The BARS/FuxiCTR open benchmark re-ran 24 CTR models with over 7,000 experiments and 12,000+ GPU hours, finding that with adequate tuning many deep models differ far less than their papers claim [36].
- Pinterest notes that benchmark-leading interaction modules face industry obstacles: latency, GPU memory limits, reproducibility [33].

7. Comparison Table

Model (org, year)	Interaction mechanism	Order	Hand-crafted crosses?	Production evidence (public)
LR + FTRL (Google, 2013) [1]	Linear over hashed/sparse crosses	Manual	Yes	Google search ads
GBDT + LR (Facebook, 2014) [3]	Tree leaves as features, linear head	Tree depth	Partly	FB ads; >3% NE gain vs either alone
FM (2010) [4]	Dot product of latent vectors	2	No	Basis of later models
FFM (2016) [6][7]	Field-aware latent vectors	2	No	Kaggle wins; Criteo prod
Wide & Deep (Google, 2016) [8]	Linear crosses + MLP	Manual + implicit	Wide part	Play: +3.9% acquisitions
DeepFM (Huawei, 2017) [9]	FM + MLP, shared embeddings	2 + implicit	No	Huawei app store data
DCN (Google, 2017) [10]	Vector cross layers + MLP	Bounded, grows w/ depth	No	Criteo
xDeepFM (2018) [13]	CIN (vector-wise outer products)	Bounded	No	Costly (approx 10x DNN)
AutoInt (2018) [14]	Multi-head self-attention over fields	Grows w/ depth	No	Public datasets
DLRM (Meta, 2019) [16]	Pairwise dot products	2	No	Reference / MLPerf
DIN (Alibaba, 2017-18) [17]	Target attention over behaviors	n/a (sequence)	No	+10.0% CTR, +3.8% RPM
DIEN (Alibaba, 2019) [18]	GRU + AUGRU	n/a	No	+20.7% CTR; 6.6 ms
DCN-V2 (Google, 2020) [11]	Matrix cross, low-rank MoE	Bounded	No	Many Google LTR systems
SIM (Alibaba, 2020) [19]	GSU search + ESU attention	n/a	No	54k behaviors; +7.1% CTR
MaskNet (Weibo, 2021) [27]	Instance-guided multiplicative mask	Implicit	No	Twitter heavy ranker [28]
DHEN (Meta, 2022) [21]	Hierarchical ensemble of modules	Grows	No	+0.27% NE
TWIN (Kuaishou, 2023) [20]	Consistent GSU/ESU attention	n/a	No	Main traffic
Wukong (Meta, 2024) [22]	Stacked FM + linear compress	Up to 2^L	No	Scaling >100 GFLOP/ex
HSTU (Meta, 2024) [23]	Sequential transduction	n/a	No	1.5T params; +12.4%
RankMixer (ByteDance, 2025) [30]	Token mixing + per-token FFN/MoE	Implicit	No	MFU 4.5% -> 45%

8. Feature-Interaction Design Choices

Pulling the threads together, the main design axes are:

1. Explicit vs implicit interactions. MLPs can in principle learn crosses but are inefficient at multiplicative structure [27]; explicit modules (FM, cross layers, CIN, dot products) add inductive bias. Nearly all production models combine both (Wide & Deep, DeepFM, DCN, DLRM).
2. Bit-wise vs vector-wise. DCN crosses individual elements; CIN, FM and DLRM treat a field embedding as a unit [13]. RankMixer's per-token parameters are a modern answer that keeps field identity while mixing across tokens [30].
3. Bounded order vs depth-growing order. DLRM stops at order 2 for cost reasons [16]; DCN/DCN-V2 add one order per layer [10][11]; Wukong doubles order per layer [22].
4. Parameter budget for interactions. Full matrices (DCN-V2) are expressive but O(d^2); low-rank and mixture-of-low-rank factorization recovers most quality [11]; LinkedIn and Pinterest both rely on low-rank variants in production [29][34].
5. Static features vs behavior sequences. Interaction networks over static features and target-attention over sequences evolved separately and are now being merged (InterFormer, HSTU, GEM) [23][24][25].
6. Hardware efficiency. Modules that look cheap in FLOPs may run at very low MFU; ByteDance reported 4.5% MFU before RankMixer [30], and Meta reports 20-25% MFU for GEM training after co-design [26]. Architecture choices increasingly follow GPU-friendliness (dense matmuls, uniform tokens) rather than theoretical elegance.
7. Calibration and online freshness. From FTRL [1] through GBDT+LR [3] to LiRank's co-trained isotonic layer [29], calibrated probabilities and fresh models remain non-negotiable for auctions and multi-objective blending.

Key takeaways for system designers

- Start from the DLRM skeleton (embeddings, bottom MLP, interaction, top MLP); it is the shared ancestor of most modern rankers and maps cleanly onto model-parallel embeddings plus data-parallel dense layers.
- Use an explicit interaction module, and prefer low-rank DCN-V2-style crosses as a strong, cheap default; consider MaskNet-style multiplicative gating as an alternative.
- Make the user representation target-aware: a DIN-style attention block over recent behaviors is usually the highest-ROI addition after the base model.
- Treat long histories as an in-model retrieval problem (SIM, TWIN), keep the GSU and ESU relevance metrics consistent, and precompute item-side projections.
- Budget memory before FLOPs: feature admission (Bloom filters, frequency thresholds) and low-precision storage saved Google 60-75% RAM a decade ago and still matter for embedding tables.
- Expect benchmark gains to shrink in production: tuned baselines close most gaps [36], and latency, GPU memory and reproducibility constrain choices [33].
- If you want to scale, pick architectures with demonstrated scaling behavior (Wukong, HSTU, RankMixer) and measure MFU; watch for embedding collapse when scaling embedding width [35].
- When the best model cannot be served, train a large foundation ranker and transfer its knowledge into smaller serving models (Meta GEM).
- Never ship uncalibrated probabilities; correct for negative downsampling and consider a calibration layer trained jointly with the model.

References

1. McMahan, H. B., Holt, G., Sculley, D., et al. "Ad Click Prediction: a View from the Trenches." Google, KDD 2013. https://research.google/pubs/ad-click-prediction-a-view-from-the-trenches/
2. Qun Le. "Paper Notes: Ad Click Prediction: a View from the Trenches." Blog. https://le.qun.ch/en/blog/paper-notes-ad-click-prediction-a-view-from-the-trenches/
3. He, X., Pan, J., Jin, O., et al. "Practical Lessons from Predicting Clicks on Ads at Facebook." Facebook, ADKDD 2014. https://quinonero.net/Publications/predicting-clicks-facebook.pdf (also https://ai.meta.com/research/publications/practical-lessons-from-predicting-clicks-on-ads-at-facebook/)
4. Rendle, S. "Factorization Machines." IEEE ICDM 2010. https://doi.org/10.1109/ICDM.2010.127
5. Dive into Deep Learning. "Factorization Machines" chapter. d2l.ai. https://d2l.ai/chapter_recommender-systems/fm.html
6. Juan, Y., Zhuang, Y., Chin, W.-S., Lin, C.-J. "Field-aware Factorization Machines for CTR Prediction." National Taiwan University, RecSys 2016. https://www.csie.ntu.edu.tw/~cjlin/papers/ffm.pdf
7. Juan, Y., Lefortier, D., Chapelle, O. "Field-aware Factorization Machines in a Real-world Online Advertising System." Criteo, WWW 2017 Companion. https://arxiv.org/abs/1701.04099
8. Cheng, H.-T., et al. "Wide & Deep Learning for Recommender Systems." Google, DLRS 2016. https://arxiv.org/abs/1606.07792
9. Guo, H., Tang, R., Ye, Y., Li, Z., He, X. "DeepFM: A Factorization-Machine based Neural Network for CTR Prediction." Huawei Noah's Ark, IJCAI 2017. https://arxiv.org/abs/1703.04247
10. Wang, R., Fu, B., Fu, G., Wang, M. "Deep & Cross Network for Ad Click Predictions." Google/Stanford, ADKDD 2017. https://arxiv.org/abs/1708.05123
11. Wang, R., et al. "DCN V2: Improved Deep & Cross Network and Practical Lessons for Web-scale Learning to Rank Systems." Google, WWW 2021. https://arxiv.org/abs/2008.13535
12. TensorFlow Recommenders. "Deep & Cross Network (DCN)" tutorial. Google. https://www.tensorflow.org/recommenders/examples/dcn
13. Lian, J., et al. "xDeepFM: Combining Explicit and Implicit Feature Interactions for Recommender Systems." Microsoft/USTC, KDD 2018. https://arxiv.org/abs/1803.05170
14. Song, W., et al. "AutoInt: Automatic Feature Interaction Learning via Self-Attentive Neural Networks." Peking University/Mila, CIKM 2019. https://arxiv.org/abs/1810.11921
15. Meta AI. "DLRM: An advanced, open source deep learning recommendation model." Blog, July 2019. https://ai.meta.com/blog/dlrm-an-advanced-open-source-deep-learning-recommendation-model/
16. Naumov, M., et al. "Deep Learning Recommendation Model for Personalization and Recommendation Systems." Facebook, 2019. https://arxiv.org/abs/1906.00091
17. Zhou, G., et al. "Deep Interest Network for Click-Through Rate Prediction." Alibaba, KDD 2018. https://arxiv.org/abs/1706.06978

18. Zhou, G., Mou, N., et al. "Deep Interest Evolution Network for Click-Through Rate Prediction." Alibaba, AAAI 2019. https://arxiv.org/abs/1809.03672
19. Pi, Q., et al. "Search-based User Interest Modeling with Lifelong Sequential Behavior Data for Click-Through Rate Prediction." Alibaba, CIKM 2020. https://arxiv.org/abs/2006.05639
20. Chang, J., et al. "TWIN: TWo-stage Interest Network for Lifelong User Behavior Modeling in CTR Prediction at Kuaishou." Kuaishou, KDD 2023. https://arxiv.org/abs/2302.02352
21. Zhang, B., Luo, L., Liu, X., et al. "DHEN: A Deep and Hierarchical Ensemble Network for Large-Scale Click-Through Rate Prediction." Meta, 2022. https://arxiv.org/abs/2203.11014
22. Zhang, B., Luo, L., Chen, Y., et al. "Wukong: Towards a Scaling Law for Large-Scale Recommendation." Meta, ICML 2024. https://arxiv.org/abs/2403.02545
23. Zhai, J., et al. "Actions Speak Louder than Words: Trillion-Parameter Sequential Transducers for Generative Recommendations." Meta, ICML 2024. https://arxiv.org/abs/2402.17152
24. Zeng, Z., et al. "InterFormer: Effective Heterogeneous Interaction Learning for Click-Through Rate Prediction." Meta/UIUC, 2024-2025. https://arxiv.org/abs/2411.09852
25. Engineering at Meta. "Meta's Generative Ads Model (GEM): The Central Brain Accelerating Ads Recommendation AI Innovation." November 2025. https://engineering.fb.com/2025/11/10/ml-applications/metas-generative-ads-model-gem-the-central-brain-accelerating-ads-recommendation-ai-innovation/
26. Engineering at Meta. "GEM Training: How Meta Doubled the Efficiency of Its LLM-Scale Ads Foundation Model." August 2026. https://engineering.fb.com/2026/08/03/ml-applications/training-gem-at-llm-scale-meta-ads-recommendation-foundation-model/
27. Wang, Z., She, Q., Zhang, J. "MaskNet: Introducing Feature-Wise Multiplication to CTR Ranking Models by Instance-Guided Mask." Sina Weibo, 2021. https://arxiv.org/abs/2102.07619
28. Twitter Engineering. "Twitter's Recommendation Algorithm." March 2023. https://blog.x.com/engineering/en_us/topics/open-source/2023/twitter-recommendation-algorithm (secondary analysis: https://blog.reachsumit.com/posts/2023/04/the-twitter-ml-algo/)
29. Borisyuk, F., et al. "LiRank: Industrial Large Scale Ranking Models at LinkedIn." LinkedIn, KDD 2024. https://arxiv.org/abs/2402.06859
30. ByteDance. "RankMixer: Scaling Up Ranking Models in Industrial Recommenders." 2025. https://arxiv.org/abs/2507.15551
31. ByteDance. "TokenMixer-Large: Scaling Up Large Ranking Models in Industrial Recommenders." 2026. https://arxiv.org/abs/2602.06563
32. Pinterest Engineering. "User Action Sequence Modeling for Pinterest Ads Engagement Modeling." Medium. https://medium.com/pinterest-engineering/user-action-sequence-modeling-for-pinterest-ads-engagement-modeling-21139cab8f4e
33. Pinterest. "Improving feature interactions at Pinterest under industry constraints." CIKM 2024 workshop. https://arxiv.org/abs/2412.01985
34. Li, J., et al. "ML-DCN: Masked Low-Rank Deep Crossing Network Towards Scalable Ads Click-through Rate Prediction at Pinterest." Pinterest, 2026. https://arxiv.org/abs/2602.09194
35. Guo, X., et al. "On the Embedding Collapse When Scaling up Recommendation Models." Tsinghua/Tencent, ICML 2024. https://arxiv.org/abs/2310.04400

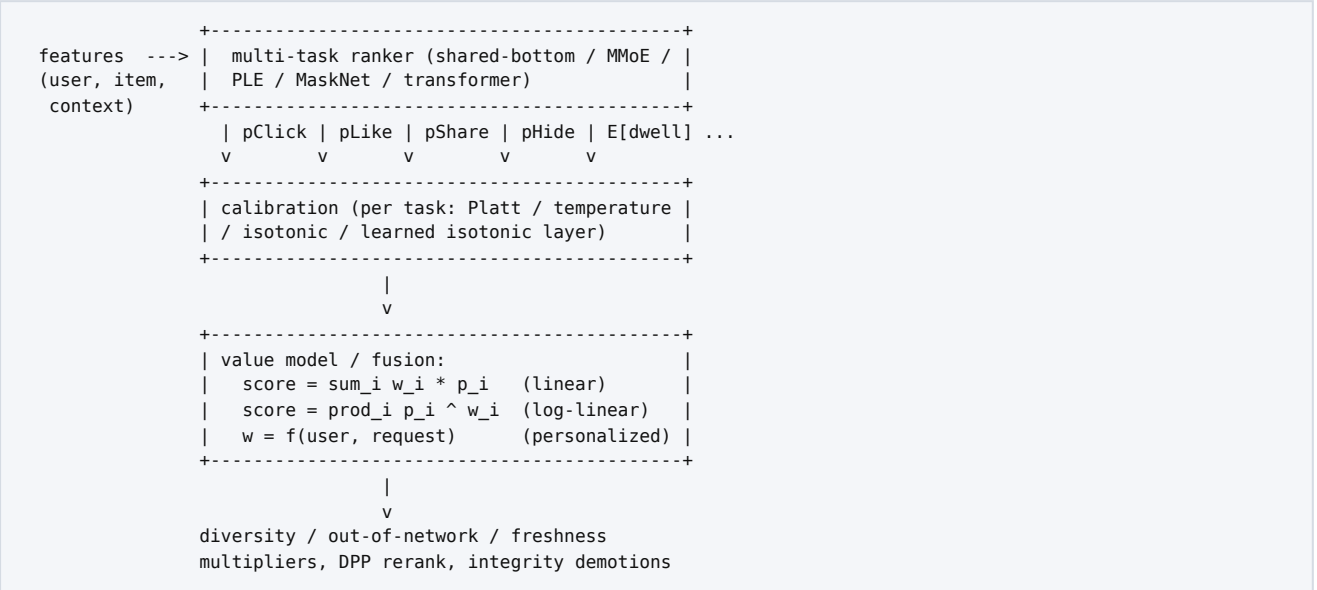
36. Zhu, J., Liu, J., Yang, S., Zhang, Q., He, X. "BARS-CTR: Open Benchmarking for Click-Through Rate Prediction." Huawei, CIKM 2021. https://arxiv.org/abs/2009.05794
37. Meta. "AutoML for Large Capacity Modeling of Meta's Ranking Systems." 2023. https://arxiv.org/abs/2311.07870
38. Meta. "On the Practice of Deep Hierarchical Ensemble Network for Ad Conversion Rate Prediction." 2025. https://arxiv.org/abs/2504.08169
39. Golovin, D., Sculley, D., McMahan, H. B., Young, M. "Large-Scale Learning with Less RAM via Randomization." Google, ICML 2013. https://arxiv.org/abs/1303.4664
40. Sodhani, S. "Practical Lessons from Predicting Clicks on Ads at Facebook" (paper notes); and E. Melt notes. https://shagunsodhani.com/papers-I-read/Practical-Lessons-from-Predicting-Clicks-on-Ads-at-Facebook and https://notes.elimelt.com/ml/recommender-systems/predicting-clicks-on-ads-at-facebook
41. NextRoll Tech Blog. "Factorization Machines." 2015. https://tech.nextroll.com/blog/data-science/2015/08/25/factorization-machines.html
42. Lacuna summary of "Wide & Deep Learning for Recommender Systems." https://lacuna.tiptreesystems.com/work/wide-deep-learning-for-recommender-systems/wrk_808f4b394e1c85f393b84b3964d18fd7
43. "xDeepFM: Combining Explicit and Implicit Feature Interactions" (review). Substack. https://happystrongcoder.substack.com/p/xdeepfm-combining-explicit-and-implicit
44. Emergent Mind. "DLRM: Deep Learning Recommendation Model" (topic summary). https://www.emergentmind.com/topics/deep-learning-recommendation-model-dlrm
45. Salamone, L. "Summary: Deep & Cross Net v2." Blog. https://blog.lukesalamone.com/posts/deep-cross-net-v2/
46. Zhou, G., et al. DIEN paper PDF (serving section). https://arxiv.org/pdf/1809.03672
47. Emergent Mind. "Search-based Interest Model (SIM)" (topic summary). https://www.emergentmind.com/topics/search-based-interest-model-sim
48. ByteByteGo. "How Meta Built a New AI-Powered Ads Model for 5% Better Conversions." https://blog.bytebytego.com/p/how-meta-built-a-new-ai-powered-ads

04. Multi-Task and Multi-Objective Ranking: From Shared-Bottom to Value Models

Overview

Almost every large recommender ranks by more than one thing. A feed item can be clicked, liked, shared, replied to, watched to the end, hidden, or reported. A product can be clicked, carted, bought, or returned. An ad can be clicked, can convert, and can earn revenue. Production systems split this into two separate problems, and it helps to keep them apart:

1. **Multi-task learning (MTL), the modeling problem.** How to train one network, or a few, that predicts many behaviors (pClick, pLike, pShare, pHide, expected watch time, ...) well. Data volumes differ across tasks, tasks conflict, and some labels are only seen after another event happens. Shared-bottom, MMoE, PLE, ESMM and their successors live here.
2. **Multi-objective optimization (MOO), the decision problem.** How to turn those predictions into one ordering that serves the business and the user over the long run. This is the "value model" or "ensemble sort" or "utility" layer: a weighted sum or product of calibrated probabilities whose weights get tuned by hand, by Bayesian optimization, or more and more by reinforcement learning.



This chapter goes through the canonical MTL architectures, then the value-model layer as it is publicly documented at X/Twitter, Meta (Facebook Feed, Instagram Explore, ads via Lattice), YouTube, LinkedIn, Pinterest and Kuaishou. It ends with calibration, loss balancing, Pareto tuning and lessons for designers.

---

1. Multi-task learning architectures

1.1 Shared-bottom (hard parameter sharing)

Design. All tasks share one embedding layer and a bottom MLP. Each task then gets its own small "tower" with a sigmoid or regression head. The loss is a weighted sum of per-task losses.

```
input embeddings -> [shared MLP] -> h
                                  |-> tower_click -> pClick
                                  |-> tower_like  -> pLike
                                  |-> tower_share -> pShare
```

Strengths. It is the cheapest option, and sparse tasks get regularized by dense ones. It is still the default in many production rankers. Pinterest describes its Homefeed ranker (Pinnability) as a point-wise multi-task model: a wide-and-deep stack feeds feature-interaction layers and MLPs that produce per-action scores such as click, repin and hide [33].

Weakness. When tasks are weakly correlated or in conflict, the shared representation causes *negative transfer*: improving one head hurts another. The MMoE paper used synthetic tasks with controlled Pearson correlation and showed that shared-bottom quality drops as task correlation falls [1].

1.2 MMoE: Multi-gate Mixture-of-Experts (Google, KDD 2018)

Problem. Learn task relationships from data instead of fixing how much tasks share.

Architecture. The single shared bottom is replaced by *n* expert networks shared by all tasks. Each task k has its own gating network g_k(x), a linear transformation followed by a softmax over experts. Task k's tower input is the sum over experts of g_k(x)_i * expert_i(x) [1].

```
                  +----------+  +----------+       +----------+
x ------------->  | expert 1 |  | expert 2 |  ...  | expert n |
|                 +----------+  +----------+       +----------+
|                       \            |            /
|-> gate_A (softmax) ---> weighted sum_A --> tower_A --> y_A
|-> gate_B (softmax) ---> weighted sum_B --> tower_B --> y_B
```

Findings.

- On the synthetic data, a one-gate MoE (OMoE) and MMoE perform about the same when tasks are highly correlated. As correlation drops, MMoE is consistently better [1].
- The authors report that the model was trained on hundreds of billions of examples in a large-scale Google content recommendation system and improved online metrics in live experiments [1].

Known pathology: gate polarization / expert collapse. Later industrial work reports that MMoE gates can collapse onto a few experts. A Kuaishou paper notes that under ReLU

some experts output mostly zeros, so gates cannot weight experts fairly. It also notes that "shared" experts can degrade into serving only one task [9].

1.3 YouTube "What Video to Watch Next" (RecSys 2019)

Problem. YouTube ranks several hundred candidates against *multiple competing objectives* and has to deal with *implicit selection bias*, especially position bias, in the feedback it logs [2].

Objectives. The paper groups them in two families [2][3]:

- *Engagement objectives*: clicks and the degree of engagement (for example watch time).
- *Satisfaction objectives*: liking a video, leaving a rating on the recommendation, and so on.

Architecture. The paper extends Wide & Deep:

- **Main tower (deep, MMoE).** Soft parameter sharing through MMoE, so each objective can choose which experts to share [2]. The paper compares shared-bottom with MMoE at 4 and 8 experts and reports that, at the same model complexity, MMoE significantly improves both engagement and satisfaction metrics (per secondary summaries of the paper's tables).
- **Shallow tower (wide).** A small model that takes bias-related inputs such as the position where the item was shown and the device. It outputs a bias logit that is *added* to the main model's logit during training. At serving time the position input is dropped, so the main tower learns position-debiased utility [2][3]. The KDnuggets write-up reports that the position feature gets 10% dropout during training, so the model does not lean on it too heavily [3]. Treat that number as secondary.

```
user/video/context features --> [MMoE: experts + per-objective gates]
                                      |               |
                                engagement towers  satisfaction towers
                                      |               |
position, device -> [shallow tower] --+ (bias logit added at train only)
                                      v
            per-objective predictions --> weighted combination --> rank
```

Fusion. Each objective's prediction is combined into one ranking score by a weighted combination whose weights are *manually tuned* for the best online engagement/satisfaction trade-off. That description comes from secondary summaries of the paper [3]. This "MTL model plus hand-tuned fusion" pattern shows up again and again below.

Lessons. (a) Debiasing belongs in the model structure: a separate additive tower absorbs position effects. (b) MMoE gains held at equal compute, so the gain comes from better sharing, not more parameters.

1.4 PLE: Progressive Layered Extraction (Tencent, RecSys 2020 Best Long Paper)

Problem. Tencent's video recommender saw a *seesaw phenomenon*: MMoE and similar models improved one task at the expense of another, especially when tasks had complex or

competing correlations [4].

Architecture.

- **CGC (Customized Gate Control).** Each task gets its own *task-specific experts* plus a pool of *shared experts*. Task k's gate mixes only its own experts and the shared ones. The paper removes connections from a task's tower to other tasks' specific experts [4].
- **PLE.** Several CGC layers are stacked. Intermediate layers also have a *shared gate* that mixes all experts to feed the next level's shared experts. Shared and task-specific knowledge are separated step by step [4].

```
Level 1:  [A-experts] [shared-experts] [B-experts]
            gate_A      gate_shared      gate_B
              |              |              |
Level 2:  [A-experts] [shared-experts] [B-experts]
            gate_A                       gate_B
              |                            |
           tower_A -> VTR (AUC)          tower_B -> VCR (MSE)
```

Tasks and data. The main experiments use a Tencent video dataset of about 1 billion samples with two tasks: VTR, a binary "effective view" classification measured by AUC, and VCR, a view-completion-ratio regression measured by MSE [4].

Results. MMoE beat single-task models, but its VCR gain was tiny (about +0.0001 MSE improvement). CGC improved VTR more than VCR, and PLE improved both significantly over MMoE [4]. Online, PLE raised view count by 2.23% and watch time by 1.84% over the earlier SOTA MTL model, and it was deployed in Tencent's video recommender [4].

Lesson. Explicit task-specific capacity guards against negative transfer. Shared capacity still allows positive transfer, and stacking the levels lets the separation happen gradually.

1.5 HoME: Hierarchy of Multi-Gate Experts (Kuaishou, KDD 2025)

Problem. In production at Kuaishou, MMoE-style models showed three anomalies [9]:

- *Expert collapse*: experts whose ReLU outputs are mostly zero. Gates cannot balance them.
- *Expert degradation*: shared experts that end up serving a single task.
- *Expert underfitting*: sparse tasks lean on shared experts, which dense tasks' gradients dominate, and neglect their own experts.

Architecture. Experts are arranged in a hierarchy: a global shared level, then task-group levels (for example passive watch-time tasks vs. interaction tasks), then task-specific experts, with cascaded gates. The paper also adds normalization to expert outputs and feature-gating to fight collapse [9][44].

Scale. The paper reports deployment on Kuaishou short-video services serving about 400 million daily users [9].

Lesson. At very large scale, getting MoE-style MTL to work well needs normalization and grouping by task semantics. Gates alone are not enough.

1.6 STEM: Shared and Task-specific EMbeddings (Tencent, AAAI 2024)

Problem. Negative transfer is worst on samples where users give *comparable* positive feedback across tasks. The STEM authors trace this to the single shared embedding table, which gives every user and item one universal embedding [10].

Design. STEM-Net keeps one shared embedding table *plus* a separate embedding table per task. Experts are built on shared or task-specific embeddings. An "All Forward Task-specific Backward" gate lets every expert contribute forward, while stop-gradient keeps one task's gradients out of other tasks' embeddings [10].

Lesson. Negative transfer is not only a tower/expert problem. In ID-heavy recommenders the embedding table is where most parameters sit, and it is also where most interference happens.

1.7 Sequential-dependency models: ESMM and successors

ESMM: Entire Space Multi-Task Model (Alibaba, SIGIR 2018)

Problem. Conventional CVR (post-click conversion) models train only on clicked impressions but score *all* impressions, which is *sample selection bias*. Conversions are also very sparse. In Taobao data the CVR task had only about 4% as many samples as the CTR task [5].

Key identity. Over the impression space:

```
pCTCVR = P(click=1, conv=1 | x) = pCTR(x) * pCVR(x)
```

Architecture. Two towers, CTR and CVR, share the embedding layer. Supervision goes on pCTR (click labels on all impressions) and on pCTCVR = pCTR * pCVR (click-and-convert labels on all impressions). The CVR tower is never directly supervised, so it is trained implicitly over the entire space. The model uses the product form instead of dividing pCTCVR by pCTR, because division by small CTR values is numerically unstable [5].

```
impression features
       |
 [shared embeddings]
   /            \
[CTR tower]   [CVR tower]
   |               |
 pCTR ----(x)---- pCVR
   |         |
loss(click)  pCTCVR -> loss(click & conversion)
```

Scale and results. The full Taobao dataset has 8.9 billion samples with sequential click and conversion labels. A 1% sample was released publicly as the first such dataset. On the production data, ESMM beat the baseline by roughly 2.18% AUC on CVR and 2.32% on CTCVR (approximate: secondary sources quote slightly different numbers) [5].

Successors

- **ESM2 (Alibaba, 2019).** Breaks the click-to-purchase path into intermediate post-click actions such as add-to-cart and wishlist, so the sparse purchase label gets denser supervision [6].
- **ESCM2 (SIGIR 2022).** Shows that ESMM's CVR estimates are themselves biased: they over-estimate and ignore the causal click-to-conversion link. It adds a counterfactual risk-minimization regularizer to the entire-space framework [7].
- **AITM (Meituan, KDD 2021).** For multi-step conversion funnels in credit-card customer acquisition (impression -> click -> application -> approval -> activation), an Adaptive Information Transfer module learns what and how much information to pass from earlier steps to later ones. A Behavioral Expectation Calibrator in the loss keeps later-step probabilities consistent with earlier ones. It was deployed in the Meituan app [8].

Lesson. When tasks form a funnel, encode the funnel in the model: either as probability chain rules (ESMM) or as learned information transfer between task towers (AITM). Do not train later stages only on survivors of earlier stages.

1.8 Comparison of MTL architectures

Architecture	Sharing mechanism	Handles conflict	Handles funnel / SSB	Public production use	Typical failure mode
Shared-bottom	Hard: one bottom MLP	Poorly	No	Pinterest Homefeed (MTL heads) [33], many others	Negative transfer, seesaw
MMoE [1]	Shared experts, per-task softmax gates	Moderately	No	Google, YouTube [2]; Snap ads [41]	Gate polarization, expert collapse [9]
PLE / CGC [4]	Task-specific + shared experts, stacked	Well	No	Tencent video [4]; Snap ads [41]	More params, more tuning
HoME [9]	Hierarchical expert groups, normalized	Well	No	Kuaishou, ~400M DAU [9]	Complexity, group design
STEM [10]	Shared + per-task embedding tables	Well (embedding level)	No	Tencent (paper)	Embedding memory multiplies
ESMM [5]	Shared embeddings, product of towers	N/A	Yes (CTR x CVR)	Taobao / Alibaba ads	CVR bias (fixed by ESCM2) [7]
AITM [8]	Learned transfer between step towers	N/A	Yes (multi-step)	Meituan [8]	Transfer is one-directional
Parallel MaskNet [11]	Shared trunk, per-engagement heads	N/A	No	Twitter heavy ranker [11]	Calibration across heads

---

2. The value-model layer: how industry blends objectives

The MTL model outputs a vector of probabilities. The *value model* (Meta's term), *scoring weights* (X), *utility* (Pinterest), or *ensemble sort* (Kuaishou) collapses that vector into one score. Most of the real product policy lives here.

2.1 Twitter (2023): heavy ranker plus a public weight table

Pipeline. Twitter's 2023 open-source release describes the For You timeline as: candidate sourcing (search-index in-network, which supplies roughly 50% of posts; tweet-mixer out-of-network; UTEG graph traversal on GraphJet; Follow Recommendation Service), then a light ranker, then the *heavy ranker*, then filters/heuristics and mixing in Home Mixer [12][13]. Home Mixer hydrates about 6,000 features per candidate for ranking [13]. Its heuristics include author diversity, in-/out-of-network content balance, feedback fatigue, deduplication, and visibility filtering [13].

Model. The heavy ranker is a *parallel MaskNet* that outputs a probability in [0, 1] for each engagement type [11].

Fusion. The final score is a plain weighted sum:

```
score = sum_i  weight_i * P(engagement_i)
```

The weights published in the README (as of April 5, 2023) [11]:

Predicted engagement	Weight
Favorite (like)	0.5
Retweet	1.0
Reply	13.5
Good profile click (open profile, then like or reply)	12.0
Video playback 50%	0.005
Reply engaged by author	75.0
Good click (open conversation, then reply or like)	11.0
Good click v2 (open conversation, stay 2+ minutes)	10.0
Negative feedback v2 (show less, block, mute)	-74.0
Report	-369.0

How the weights were set. The README says the weights were first chosen so that *each weighted engagement contributes roughly equally on average* to the score. They were then adjusted periodically to optimize platform metrics, and they live in a config file that can be changed at any time [11]. That initialization is a useful heuristic: a weight scaled by the inverse of the base rate. A rare action like a reply or report gets a large weight because its probability is tiny.

2.2 X / xAI (2026): Phoenix transformer and the `xai-value-model`

The 2026 `xai-org/x-algorithm` repository documents the successor system in more detail [14][15][16].

Model. *Phoenix*, a transformer (JAX training, Rust serving), handles both embedding-based retrieval and ranking. The main input is the viewer's recent engagement history. It predicts probabilities for many actions instead of a single relevance score. Hash-based embeddings mean there is no vocabulary to maintain. *Candidate isolation* stops candidates from

attending to each other: they attend only to viewer context, so a post's score does not depend on what else is in the batch, and scores can be cached [14].

Predicted heads in five groups [14]:

- Engagement: favorite, reply, repost, quote, share, share via DM, share via copy link.
- Clicks: post, profile, link, photo expand, video open, quoted post.
- Attention: video quality view, dwell, dwell time, click dwell time, video continuation seconds, profile visit seconds.
- Author: follow author.
- Negative: not interested, mute author, block author, report, not dwelled.

Weights. Default weights mirrored from feature switches in `home-mixer/params/param.rs` (live values may differ) [15]:

Head	Weight	Head	Weight
Favorite	0.5	Share via copy link	20.0
Reply	5.0	Share via DM	5.0
Bidirectional-follow reply boost	15.0	Share	2.0
Retweet	1.0	Quote	5.0
Follow author	4.0	Click	0.3
Open link	0.2	Video open	0.07
Photo expand	0.05	Dwell	0.05
Continuous click dwell time	0.4	Continuous dwell time	0.004
Not interested	-47.52	Block author	-31.2
Mute author	-58.8	Report	-234.0
Not dwelled	-0.02	Post unexplored (OON)	0.015

The source comments make two points worth repeating to anyone who reads these tables [15]:

- Weights multiply *predicted probabilities or continuous values*, not raw counts. "One report cancels 468 likes" is a misreading.
- The report weight is large because the report base rate is "more than 1000x lower than a Like", which is the same base-rate normalization logic as Twitter 2023.

Scoring arithmetic (`xai-value-model/scoring.rs`) [16]:

1. Weighted sum of heads. Two-factor heads (video continuation, profile visit time) use p * seconds. Eligibility gates zero out some weights, for example video-quality-view for ineligible posts. Reply and dwell weights can vary per candidate.
2. **Negative-score offset.** Let T be the total weight sum and N the magnitude of the negative weights. Negative raw scores are squashed into [0, OFFSET) as (s + N) / T * OFFSET. Non-negative scores become s + OFFSET. This keeps scores positive and order-preserving, so the later multiplicative adjustments behave.
3. **Exploration bonus** added in unoffset space.
4. **Author diversity multiplier** (1 - floor) * decay^k + floor, where k is the number of higher-ranked posts already shown from the same author.

5. **Out-of-network multiplier** below 1 for posts from accounts not followed (optionally also for in-network replies and reposts).
6. **Cold-start lift.** A new-author candidate is raised to at least a target rank, never lowered.
7. A separate `vm-ranker` then reorders with a **determinantal point process (DPP)** over embeddings, giving up a little score for less similarity between neighbors [14].

```
Phoenix (transformer, candidate-isolated) -> ~25 heads
      -> weighted sum (param.rs weights) -> offset negatives
      -> + exploration bonus -> x author-diversity decay
      -> x out-of-network factor -> cold-start lift
      -> vm-ranker DPP rerank -> filters -> timeline
```

Lesson. The value function is a small, versioned, auditable piece of code that sits apart from the model. Policy changes (more replies, fewer reports) ship as config changes with no retraining.

2.3 Meta / Facebook News Feed: MSI and survey-calibrated value

Scale. Feed ranking must score candidates for more than 2 billion people in real time. On average there are more than 1,000 candidate posts per user per day. A lightweight first pass narrows them to about 500 for the heavier multitask neural models [17][20].

Model and fusion. Each post gets several predicted actions (like, comment, share, ...) from *multitask neural networks*. These "all need to somehow aggregate up to a single value" [17]. Meta's 2021 post uses a running example (a user named Juan). Several prediction models can disagree about which post he will respond to most. The disagreement is resolved by weighting predictions by how *meaningful* the corresponding interactions are, using survey data on what people say they value [17].

MSI (Meaningful Social Interactions, 2018). Internal documents reported by CNN in 2021 describe MSI as a point system. Likes, comments, re-shares, reactions and even event RSVPs had different point values, adjusted for the relationship between the people involved. Weights were set using surveys (one cited survey covered over 69,000 people in five countries) together with analysis of user data. A 2019 internal note said MSI was "beneficial for most metrics" in 2018, including likes and comments between users, daily users, revenue, and time on Feed [18]. The same reporting describes how the weighting backfired: heavy weights on comments, reshares and reaction emojis also amplified outrage-provoking content [18].

Survey-based objectives. From 2019 Meta added "Is this post worth your time?" surveys. In 2021 it gave more weight to survey-derived predictions about how meaningful friends, Pages and Groups are to a person. Classifiers trained to predict survey answers let survey signals generalize to posts without surveys [19].

Lesson. Engagement-derived weights optimize what can be measured. Survey-predicted objectives give a value model a counterweight that is closer to what users say they want. Weight choices also have large second-order effects on the content ecosystem.

2.4 Instagram Explore: log-linear vs. linear value models

2020 design (constrained exploration). Instagram's multi-label model predicts engagement (like, save) and negative feedback (see less / not interested). Point-wise heads are trained with cross-entropy, alongside GBDTs and list-wise LambdaRank models. The value model is *log-linear* [21]:

```
Value(post) = P(like) ^ w_like  *  (1 - P(not interested)) ^ w_see_less
```

The weights are tuned with (a) *offline replay over user sessions* and (b) *online Bayesian optimization*, and they are retuned often as the system changes [21]. A multiplicative form has a useful property: a high predicted negative probability can veto an item however likely a like is.

2023 design (scaling Explore). Four stages: retrieval, first-stage ranking (lightweight), second-stage ranking with a *multi-task multi-label (MTML)* neural network, and final reranking. The value model became an explicitly *linear* weighted sum [22]:

```
Expected Value = W_click * P(click) + W_like * P(like)
                 - W_see_less * P(see less) + ...
```

Meta describes the value model as approximating how much value each media item brings to the user. Tuning its weights lets the team "explore different tradeoffs between online engagement metrics" [22]. The second-stage MTML model is much heavier than two-tower retrieval, so Meta precomputes some users' recommendations off-peak. Business rules in the value model handle diversity, such as penalizing repeated posts from the same author or seed account [22]. For tuning, the post contrasts Bayesian optimization, where engineers specify only the parameters, the goal metric and regression thresholds on guardrail metrics, with offline tuning [22].

Lesson. Linear and log-linear value models make different trade-offs. Linear is easy to interpret and lets each term add its own contribution. Log-linear lets a single term dominate (a veto). In both cases the weights are search parameters for online BO, not something learned by gradient descent.

2.5 Meta ads: Lattice, multi-domain multi-task consolidation

Problem. Ads ranking had grown into many models across surfaces and objectives. That slowed iteration and wasted compute [23].

Design. Lattice consolidates models into a *multi-domain, multi-task* architecture with sparse activation. It learns from many data sources and handles delayed feedback ranging from seconds to days. Meta's 2023 description cites *trillions of parameters* trained on *hundreds of billions of examples* [23]. Balancing thousands of domains and tens of objectives by hand is impossible, so Meta uses Pareto-optimality-based methods, including Pareto-front feature selection and MetaBalance [23][25]. The 2025 paper describes cross-domain knowledge sharing, data consolidation, model unification, and *distillation* of a large consolidated model into lean user-facing models. It reports +10% on revenue-driving top-line metrics, +11.5% user satisfaction, +6% conversion rate, and 20% capacity saving (Meta's own figures) [24].

MetaBalance. At each iteration, and per part of the network, the method rescales each *auxiliary* task's gradient so its magnitude is closer to the *target* task's gradient. Auxiliary tasks then neither dominate nor vanish. The paper reports +8.34% NDCG@10 over the strongest baseline on two datasets [25].

2.6 LinkedIn Feed: constrained objectives and autotuning

Formulation. LinkedIn treats feed ranking as multi-objective optimization. Separate models predict actions such as viral actions (react, comment, share), contributions, and downstream effects on creators. A linear combination of per-objective probabilities ranks items [26][27][28]. A 2019 post describes objectives that include a member's likelihood to view updates, to join conversations, and the value of timely feedback to content creators [27].

Autotuning (2018-2020). The weights had been set by hand by ML engineers, which LinkedIn called very inefficient [26]. The automated approach [26][28]:

- Treat online metrics as black-box functions of the control parameters (for example the click weight). Model each metric with a *Gaussian process* from A/B data.
- Optimize a primary metric subject to constraints on others. LinkedIn observed that contributions *fall* as click weight rises while the constraint metric (normalized clicks) *rises*, so the optimum sits on the constraint boundary.
- Choose the next parameters by sampling functions from the GP posterior and computing each sample's argmax. That gives a probability that each parameter value is optimal, a Thompson-sampling-style procedure.
- Ramp volume and duration are limited in practice, so many experiments run in parallel at low traffic.

LiRank calibration (KDD 2024). LinkedIn's LiRank paper adds an *isotonic calibration layer trained jointly* inside the deep model instead of post-hoc Platt/isotonic regression, and reports significant gains in predictive accuracy [29]. The 2026 "Isotonic Layer" paper extends this to a single learnable component that handles calibration, position debiasing and multi-task bias correction, first deployed in LiRank [30].

2.7 Pinterest Homefeed: utility weights, from global to per-request RL

Model. The Homefeed ranker (Pinnability) predicts actions such as click, repin and hide, with hide as a negative signal. TransAct adds a real-time user-action sequence transformer to it [33].

Utility. A utility-weight vector defines each item's utility as a weighted sum of predicted objective scores. Historically the weights were set by offline analysis and online experiments, applied *globally* (one vector for all requests), and left in place for long periods [32].

PRL-PUTS (2026). Pinterest reframes weight selection as a *one-step, value-based RL* problem. For each request an agent picks the weight vector that maximizes expected request-level engagement reward. An inference-time scalarization parameter yields a family of policies and an *empirical Pareto frontier*, used as a governance artifact for choosing the operating point. It runs alongside ranking inference, adds no serving latency, and needs no

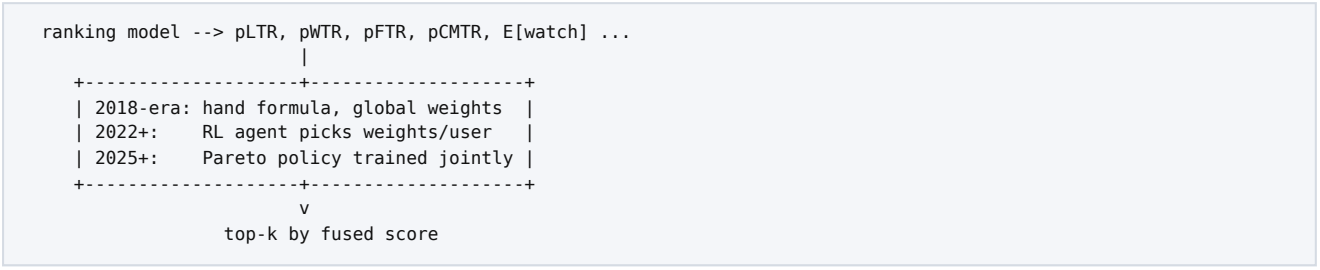
ranker change. Training uses unbiased exploration logs. Online it gave +0.13% successful sessions, with controllable Repin vs. P2P trade-offs [32]. Pinterest has also published on optimizing longer-horizon downstream rewards beyond immediate engagement on Homefeed [34] and on lifecycle-dependent utility weights (secondary reporting).

2.8 Kuaishou: from formula fusion to learned ensemble sort

Kuaishou's papers describe the industry-standard "ensemble sort" stage: the ranking model emits many pXTRs (pLTR, pWTR, pFTR, ...), and a manually tuned formula merges them into one score before the top dozen items are picked. The Pantheon authors call this formula-based adjustment the "wide-accepted industry" practice [35].

A line of Kuaishou and related work replaces the hand formula with learned, personalized fusion:

- **RL-based multi-task fusion (MTF).** Users are the environment, the recommender is the agent, and the action is the fusion-weight vector. Fusion is often a log-transformed weighted sum with a bias term. The reward is long-term user satisfaction [38][37].
- **xMTF.** A "formula-free" MTF model that drops the fixed functional form [37].
- **Pantheon (2025).** Personalized ensemble sort trained jointly with the real-time ranking model using Iterative Pareto Policy Optimization (IPPO). It searches objective weights automatically and is deployed at Kuaishou for about 400 million daily users [35].
- **GRADE (2025).** Critic-free, GRPO-style group-relative RL over candidate weight vectors with Dirichlet exploration. In Kuaishou marketplace search it reports +0.595% CTR, +1.193% CVR, +1.788% OPM and +1.568% order volume [36].
- **Rank-consistency ensembles (2026).** Moves the ensemble from score fusion toward rank consistency across objectives, using Kuaishou data [43]. UMRE proposes a unified monotonic transformation of pXTRs before fusion [46].



2.9 Other public data points

- **Snap ads** use MMoE and PLE to predict multiple conversion events (app installs, purchases, sign-ups) jointly [41].
- **Spotify audio ads** use multi-task learning and calibrate each task with *per-task temperature scaling* (one parameter per task, a simplified Platt scaling). They chose it over Platt or isotonic for simplicity, because miscalibration distorts auction ordering and pricing [31].
- **Netflix** reports consolidating several related search and recommendation models into one multi-task model to cut maintenance cost and improve quality [42].

- **Alibaba PE-LTR (RecSys 2019)** optimizes GMV and CTR together with a weighted aggregation. It derives a condition for Pareto efficiency and a two-step Pareto-efficient optimization algorithm that can also generate a Pareto frontier [39].

2.10 Value-model comparison

System	MTL model	Fusion form	Weight setting	Post-fusion adjustments
Twitter 2023 [11]	Parallel MaskNet	Linear sum, 10 heads	Base-rate equalization, then periodic hand tuning	Author diversity, content balance, fatigue [13]
X 2026 [14] [15][16]	Phoenix transformer, candidate-isolated	Linear sum, ~25 heads, negative offset	Feature-switch config, A/B	Author decay, OON factor, cold-start lift, DPP
Facebook Feed [17][18][19]	Multitask NNs	Weighted value; MSI points	Surveys + data analysis	Diversity, integrity demotions [20]
Instagram Explore 2020 [21]	Multi-label NN + GBDT + LambdaRank	Log-linear product	Offline replay + online BO	Exploration constraints
Instagram Explore 2023 [22]	MTML NN	Linear sum	BO with guardrail thresholds	Same-author penalty
YouTube 2019 [2][3]	MMoE + shallow bias tower	Weighted combination	Manual tuning	n/a (not public)
LinkedIn Feed [26][28]	Per-objective models (LiRank)	Linear combination	GP-based constrained BO	n/a
Pinterest Homefeed [32]	Pinnability MTL + TransAct	Linear utility	Global -> per-request RL, Pareto sweep	n/a
Kuaishou [35] [36]	MMoE/PLE/HoME	Formula -> learned ensemble	Hand -> RL (IPPO, GRPO)	n/a

---

3. Calibration: the contract between the model and the value model

A value model that adds w_i * p_i is only meaningful if each p_i means what it says. If pReply is systematically 2x too high, the effective reply weight is 2x too. Changes to training data, sampling or architecture then silently shift product policy. Practitioners describe calibration as giving the fusion layer "a stable contract about score scale." For pure ordering inside one objective calibration does not matter, but for ads auctions and feed blending it does [45][31].

Techniques seen in industry:

- **Platt / temperature scaling per task.** A cheap, global monotonic rescaling of logits. Spotify uses one temperature per task [31].
- **Isotonic regression.** Non-parametric and monotone, applied post hoc per task. Applied to ranking calibration since Menon et al. (2012) [45].

- **Field-level calibration.** Separate calibration per advertiser, placement or other field value, using posterior statistics (for example AdaCalib, per secondary sources) [45].
- **Learned isotonic layers inside the network.** LinkedIn trains the calibration jointly with the model. The 2026 follow-up argues that Platt (two parameters, one global transform) and per-task isotonic regression cannot capture interactions across task x position x platform. It replaces them with one learnable isotonic layer that also handles position debiasing [29][30].
- **Negative sampling correction.** If negatives are downsampled at rate r, predicted odds must be rescaled to bring probabilities back to the true base rate before fusion (standard practice; ESMM-style funnels need the same care at every step).

Rule of thumb. Recalibrate whenever a head's training distribution changes. Monitor predicted vs. observed rate per head, per surface, per position bucket. Treat a calibration drift as a value-model change.

---

4. Training-time balancing: loss weights and gradient surgery

Fusion weights act at *serving* time. MTL training also needs per-task *loss* weights, which are a separate set of hyperparameters:

- **Static loss weights.** Hand-tuned. Simple, but they interact with label sparsity.
- **Uncertainty weighting (Kendall et al., 2018).** Scale each task loss by 1/sigma_k^2 and add a log sigma_k regularizer, with sigma learned (homoscedastic uncertainty) [48]. The GradNorm authors argue these weights can change without constraint and push the effective learning rate up [40].
- **GradNorm (2018).** Learn per-task loss weights so that per-task gradient norms on shared parameters sit on a common scale, adjusted by relative training rates. It has one asymmetry hyperparameter alpha [40].
- **PCGrad (2020).** When two tasks' gradients conflict (negative cosine), project each onto the normal plane of the other [47].
- **MetaBalance (Meta, 2022).** Scale *auxiliary* gradients relative to the *target* task's gradient magnitude, per network part. Built for recommenders where one task is primary [25].
- **Structural decoupling.** Task-specific experts (PLE), task-specific embeddings with stop-gradient (STEM), and hierarchical expert groups (HoME) avoid much of the gradient conflict by construction [4][10][9].

Practical note. Loss weights change *what the model learns*. Fusion weights change *what gets shown*. Teams that confuse the two often try to fix a serving trade-off by retraining, or the reverse.

---

5. Pareto tuning and objective blending in practice

5.1 The workflow most companies converge on

```
1. Train calibrated MTL heads (one model, many heads).
2. Define the value function: score = sum w_i * p_i (+ multipliers).
3. Initialize w_i ~ 1 / mean(p_i) * importance_i  (equal-contribution
   heuristic, as in Twitter 2023 [11]).
4. Offline: replay logged sessions; sweep w; trace a Pareto front of
   proxy metrics (e.g. predicted likes vs predicted hides) [21][32].
5. Online: constrained Bayesian optimization or GP Thompson sampling
   over w, primary metric s.t. guardrails [22][26].
6. Personalize: learn w = f(user, request) with RL / policy
   optimization, keep a Pareto frontier for governance [32][35][36].
7. Re-check calibration on every model change; re-tune w on every
   product-goal change.
```

5.2 Why weighted sums persist

Even teams with RL-based fusion keep a weighted sum as the interface. It is interpretable ("what does a share buy us?"), it can be audited (X publishes its weights [15]), it can be changed without retraining, and it composes with business multipliers. Learned fusion usually outputs the weights, not the score, so the human-readable layer survives [32][35].

5.3 Pareto frontiers as governance artifacts

Pinterest's PRL-PUTS makes the empirical Pareto frontier an explicit artifact for choosing among Repin vs. P2P trade-offs [32]. Meta's Lattice uses Pareto-front methods to avoid hand-tuning thousands of domains [23]. Alibaba's PE-LTR gives a theoretical Pareto-efficiency condition for weighted aggregation [39]. The general pattern: compute the frontier, then make the operating point a product decision.

5.4 Multiplicative and offset tricks

- **Log-linear products** (Instagram 2020) let a negative-feedback term veto an item [21].
- **Negative-score offsets** (X 2026) map negative weighted sums into a small positive band, so later multiplicative adjustments (diversity decay, out-of-network discount) cannot accidentally *promote* a negatively scored item by scaling it toward zero [16].
- **Monotone transforms before fusion** (UMRE) align heads with very different scales and distributions [46].

5.5 Failure modes documented in public sources

- **Engagement weights amplifying harmful content.** Facebook MSI's heavy weights on comments, reshares and reactions are reported to have boosted divisive content [18].
- **Seesaw.** Improving one head degrades another (PLE motivation) [4].
- **Expert collapse and sparse-task underfitting** in MoE MTL [9].
- **Biased CVR from entire-space models** [7].
- **Global weights fitting no one.** Pinterest and Kuaishou both motivate personalized fusion by noting that one global weight vector is slow to adapt and ignores individual intent [32][36].
- **Manual tuning cost.** LinkedIn [26] and Kuaishou [35] both describe hand-tuning as a bottleneck.

Key takeaways for system designers

1. **Separate prediction from policy.** Train an MTL model that predicts calibrated probabilities of concrete actions. Keep the business trade-off in a small, versioned value function. X, Meta, Pinterest and LinkedIn all keep this split.
2. **Start with shared-bottom and move up only when you see the seesaw.** Use MMoE when tasks are moderately related. Use PLE/CGC (task-specific experts) when they conflict. Watch for expert collapse at scale, and use normalization or hierarchy as HoME does. Consider task-specific embeddings (STEM) when the conflict is in the ID embeddings.
3. **Encode funnels explicitly.** For impression -> click -> conversion, use entire-space modeling (ESMM: pCTCVR = pCTR * pCVR) or learned transfer (AITM). Add counterfactual corrections (ESCM2) when you need unbiased CVR.
4. **Model bias structurally.** An additive shallow tower for position and device (YouTube) or a learned isotonic layer (LinkedIn) keeps presentation bias out of the utility heads.
5. **Calibrate every head the value model consumes.** Without calibration, fusion weights silently drift with every retrain. Monitor predicted vs. observed rates per head.
6. **Initialize weights by base-rate equalization**, then tune. Twitter's "equal average contribution" rule and X's ">1000x rarer than a like" comment explain why rare actions (replies, reports) get large weights.
7. **Give negatives real teeth.** Large negative weights on report, block, mute and "not interested" (X), or a multiplicative veto term (Instagram), are the cheapest safety mechanisms in the stack.
8. **Tune online with constrained BO.** Optimize one primary metric subject to guardrail thresholds (Meta, LinkedIn). Expect optima on the constraint boundary.
9. **Personalize the weights, not just the predictions.** Per-request RL weight selection (Pinterest PRL-PUTS, Kuaishou Pantheon/GRADE) gives small but real online gains with no extra serving latency, and a Pareto frontier for governance.
10. **Include survey and long-term objectives.** Pure engagement weights can amplify harmful content (MSI). Survey-predicted value (Facebook) and downstream or long-horizon rewards (Pinterest) give a counterweight.
11. **Apply diversity and network-mix multipliers after fusion**, and protect against score-sign bugs (X's negative-score offset).
12. **Consolidate models when you can afford one big MTL model plus distillation** (Meta Lattice, Netflix). Fewer models means faster iteration and less compute, provided loss balancing (MetaBalance, Pareto methods) is handled.

References

1. Ma, J., Zhao, Z., Yi, X., Chen, J., Hong, L., Chi, E. H. "Modeling Task Relationships in Multi-task Learning with Multi-gate Mixture-of-Experts." Google, KDD 2018. https://research.google/pubs/modeling-task-relationships-in-multi-task-learning-with-multi-gate-mixture-of-experts/

2. Zhao, Z., et al. "Recommending What Video to Watch Next: A Multitask Ranking System." Google/YouTube, RecSys 2019. https://research.google/pubs/recommending-what-video-to-watch-next-a-multitask-ranking-system/ (ACM: https://dl.acm.org/doi/10.1145/3298689.3346997)
3. "How YouTube is Recommending Your Next Video." KDnuggets, 2019. https://www.kdnuggets.com/2019/10/youtube-recommending-next-video.html
4. Tang, H., Liu, J., Zhao, M., Gong, X. "Progressive Layered Extraction (PLE): A Novel Multi-Task Learning (MTL) Model for Personalized Recommendations." Tencent, RecSys 2020. https://dl.acm.org/doi/fullHtml/10.1145/3383313.3412236
5. Ma, X., Zhao, L., et al. "Entire Space Multi-Task Model: An Effective Approach for Estimating Post-Click Conversion Rate." Alibaba, SIGIR 2018. https://arxiv.org/abs/1804.07931
6. Wen, H., et al. "Entire Space Multi-Task Modeling via Post-Click Behavior Decomposition for Conversion Rate Prediction (ESM2)." Alibaba, 2019. https://arxiv.org/abs/1910.07099
7. Wang, H., et al. "ESCM2: Entire Space Counterfactual Multi-Task Model for Post-Click Conversion Rate Estimation." SIGIR 2022. https://arxiv.org/abs/2204.05125
8. Xi, D., Chen, Z., et al. "Modeling the Sequential Dependence among Audience Multi-step Conversions with Multi-task Learning in Targeted Display Advertising (AITM)." Meituan, KDD 2021. https://arxiv.org/abs/2105.08489
9. Wang, X., et al. "HoME: Hierarchy of Multi-Gate Experts for Multi-Task Learning at Kuaishou." Kuaishou, 2024 (KDD 2025). https://arxiv.org/abs/2408.05430
10. Su, L., Pan, J., et al. "STEM: Unleashing the Power of Embeddings for Multi-task Recommendation." Tsinghua/Tencent, AAAI 2024. https://arxiv.org/abs/2308.13537
11. Twitter. "Heavy Ranker README (projects/home/recap)." the-algorithm-ml, 2023. https://github.com/twitter/the-algorithm-ml/blob/main/projects/home/recap/README.md
12. Twitter. "the-algorithm README." 2023. https://github.com/twitter/the-algorithm
13. Twitter. "Home Mixer README." 2023. https://github.com/twitter/the-algorithm/blob/main/home-mixer/README.md
14. xAI / X. "x-algorithm README." 2026. https://github.com/xai-org/x-algorithm
15. xAI / X. "home-mixer/params/param.rs (scoring weights)." 2026. https://github.com/xai-org/x-algorithm/blob/main/home-mixer/params/param.rs
16. xAI / X. "xai-value-model/scoring.rs." 2026. https://github.com/xai-org/x-algorithm/blob/main/xai-value-model/scoring.rs
17. Lada, A., Wang, M., Yan, T. "How machine learning powers Facebook's News Feed ranking algorithm." Engineering at Meta, 2021. https://engineering.fb.com/2021/01/26/ml-applications/news-feed-ranking/
18. CNN Business (syndicated). "Likes, anger emojis and RSVPs: the math behind Facebook's News Feed and how it backfired." 2021. https://abc17news.com/news/2021/10/27/likes-anger-emojis-and-rsvps-the-math-behind-facebooks-news-feed-and-how-it-backfired/
19. Meta. "Incorporating More Feedback Into News Feed Ranking." 2021. https://about.fb.com/news/2021/04/incorporating-more-feedback-into-news-feed-ranking/
20. Meta Transparency Center. "Facebook Feed AI system." https://transparency.meta.com/features/explaining-ranking/fb-feed
21. Meta / Instagram. "How Instagram suggests new content (Designing a constrained exploration system)." Engineering at Meta, 2020. https://engineering.fb.com/2020/12/10/web/how-instagram-suggests-new-content/

22. Meta / Instagram. "Scaling the Instagram Explore recommendations system." Engineering at Meta, 2023. https://engineering.fb.com/2023/08/09/ml-applications/scaling-instagram-explore-recommendations-system/
23. Meta AI. "Meta Lattice: AI ads performance and efficiency." 2023. https://ai.meta.com/blog/ai-ads-performance-efficiency-meta-lattice/
24. Meta. "Meta Lattice: Model Space Redesign for Cost-Effective Industry-Scale Ads Recommendations." 2025. https://arxiv.org/abs/2512.09200
25. He, Y., Feng, X., Cheng, C., Ji, G., Guo, Y., Caverlee, J. "MetaBalance: Improving Multi-Task Recommendations via Adapting Gradient Magnitudes of Auxiliary Tasks." Meta AI / TAMU, WWW 2022. https://arxiv.org/abs/2203.06801
26. LinkedIn (via KDnuggets). "Autotuning for Multi-Objective Optimization on LinkedIn's Feed Ranking." 2020. https://www.kdnuggets.com/2020/08/autotuning-multi-objective-optimization-linkedin-feed-ranking.html
27. LinkedIn Engineering. "Community-focused Feed optimization." 2019. https://engineering.linkedin.com/blog/2019/06/community-focused-feed-optimization
28. Colyer, A. "Online parameter selection for web-based ranking problems." The Morning Paper, 2018. https://blog.acolyer.org/2018/10/08/online-parameter-selection-for-web-based-ranking-problems/
29. Borisyuk, F., et al. "LiRank: Industrial Large Scale Ranking Models at LinkedIn." LinkedIn, KDD 2024. https://arxiv.org/abs/2402.06859
30. "Isotonic Layer: A Unified Framework for Recommendation Calibration and Debiasing." LinkedIn, 2026. https://arxiv.org/abs/2603.06589
31. "An Audio-centric Multi-task Learning Framework for Streaming Ads Targeting on Spotify." Spotify, 2025. https://arxiv.org/abs/2506.18735
32. "A Production-Ready RL Framework for Personalized Utility Tuning with Pareto Sweeping in Pinterest Recommender Systems (PRL-PUTS)." Pinterest, 2026. https://arxiv.org/abs/2605.16344
33. Xia, X., et al. "TransAct: Transformer-based Realtime User Action Model for Recommendation at Pinterest." Pinterest, KDD 2023. https://arxiv.org/abs/2306.00248
34. "Long-term User Engagement Optimization through Model-agnostic Downstream Rewards Learning." Pinterest, 2026. https://arxiv.org/abs/2607.14192
35. Cao, J., et al. "Pantheon: Personalized Multi-objective Ensemble Sort via Iterative Pareto Policy Optimization." Kuaishou, 2025. https://arxiv.org/abs/2505.13894
36. "GRADE: Personalized Multi-Task Fusion via Group-relative Reinforcement Learning with Adaptive Dirichlet Exploration." Kuaishou, 2025. https://arxiv.org/abs/2510.07919
37. "xMTF: A Formula-Free Model for Reinforcement-Learning-Based Multi-Task Fusion in Recommender Systems." 2025. https://arxiv.org/abs/2504.05669
38. "Multi-Task Fusion via Reinforcement Learning for Long-Term User Satisfaction in Recommender Systems." 2022. https://arxiv.org/abs/2208.04560
39. Lin, X., et al. "A Pareto-Efficient Algorithm for Multiple Objective Optimization in E-Commerce Recommendation (PE-LTR)." Alibaba, RecSys 2019. https://www.doi.org/10.1145/3298689.3346998
40. Chen, Z., Badrinarayanan, V., Lee, C.-Y., Rabinovich, A. "GradNorm: Gradient Normalization for Adaptive Loss Balancing in Deep Multitask Networks." ICML 2018. https://arxiv.org/abs/1711.02257
41. InfoQ. "Snap ad ranking (MMoE and PLE for multiple conversion events)." 2022. https://www.infoq.com/news/2022/10/snap-ad-ranking/

42. Netflix Technology Blog. "Lessons learnt from consolidating ML models in a large scale recommendation system." 2023. https://netflixtechblog.medium.com/lessons-learnt-from-consolidating-ml-models-in-a-large-scale-recommendation-system-870c5ea5eb4a
43. "Rethinking Multi-objective Ranking Ensemble in Recommender System: From Score Fusion to Rank Consistency." 2026. https://arxiv.org/abs/2601.02955
44. "Advances and Challenges of Multi-task Learning Method in Recommender System: A Survey." 2023. https://arxiv.org/abs/2305.13843
45. "Ranking model calibration in recommender systems." RecSys ML Substack. https://recsysml.substack.com/p/ranking-model-calibration-in-recommender (and parametric ranking calibration, https://arxiv.org/abs/2112.07428)
46. "UMRE: A Unified Monotonic Transformation for Ranking Ensemble in Recommender Systems." 2025. https://arxiv.org/abs/2508.07613
47. Yu, T., et al. "Gradient Surgery for Multi-Task Learning (PCGrad)." NeurIPS 2020. https://arxiv.org/abs/2001.06782
48. Kendall, A., Gal, Y., Cipolla, R. "Multi-Task Learning Using Uncertainty to Weigh Losses for Scene Geometry and Semantics." CVPR 2018. https://arxiv.org/abs/1705.07115

05. Sequential and User-History Modeling: From GRU4Rec to Lifelong Sequences and Foundation User Models

Overview

Most modern recommender systems are, at heart, systems for turning a user's history into a prediction. A user's ordered stream of actions (clicks, views, saves, purchases, hides, searches) is the richest signal a platform owns. Over the last decade the industry has moved through four broad generations of models for that stream:

1. **Session RNNs (2015 to 2017).** GRU4Rec showed that a recurrent network over a session could beat item-to-item similarity for next-click prediction [1], although later benchmarks showed tuned nearest-neighbour baselines remained strong [2].
2. **Self-attention for next-item prediction (2018 to 2023).** SASRec [3] and BERT4Rec [4] brought Transformers to sequential recommendation. A long replicability debate followed [5][6].
3. **Target-attention and Transformers inside industrial CTR rankers (2018 onward).** Alibaba's DIN/DIEN line [8], BST [7], and the long-sequence family MIMN [9], SIM [10], ETA [11], SDIM [13], TWIN [14] and TWIN V2 [15] put user histories of 10^2 to 10^5 behaviours into the ranking stage under tight latency budgets. Pinterest's TransAct [20] and TransAct V2 [21] do the same for real-time and lifelong action sequences.
4. **User representations and foundation user models (2020 onward).** PinnerSage [18] and PinnerFormer [19] produce reusable user embeddings. PinFM [23], Meta's HSTU generative recommenders [24], Netflix's foundation model [25][26] and ByteDance's LONGER [17] pretrain large sequence models on billions of interactions and share them across many surfaces.

This chapter walks through each design: what problem it solved, how it is built, the public scale numbers, the key tricks, and what the industry learned. The last sections give a comparison table and design takeaways.

A note on sources: every number below comes from the cited paper or blog. Figures that only appear in secondary summaries are marked "reported" or "approximately".

```
                  The user-history design space
+-------------------------------------------------------------------------+
|                 short (10^1-10^2)    long (10^3-10^4)    lifelong (10^5+) |
| retrieval       GRU4Rec, SASRec,     PinnerFormer,       PinFM, HSTU,     |
| (user tower)    BERT4Rec, KuaiFormer Netflix FM          Netflix FM       |
| ranking         DIN, BST, TransAct   MIMN, LONGER,       SIM, ETA, SDIM,  |
| (target-aware)                       TransAct V2         TWIN, TWIN V2    |
+-------------------------------------------------------------------------+
   cost grows with length x candidates x model depth -> most tricks attack this
```

1. Academic foundations: next-item prediction

1.1 GRU4Rec (Hidasi et al., 2015)

Problem. In e-commerce and media sites, many sessions are anonymous and short. The standard approach before 2015 was item-to-item similarity (for example "people who viewed X also viewed Y"), which uses only the last item and ignores the rest of the session [1].

Architecture. GRU4Rec feeds the session's item sequence, one item per step, into a GRU. The hidden state after each step is scored against all items to predict the next click. The paper made RNNs practical for this task with several changes [1]:

- **Session-parallel mini-batches.** Sessions have very different lengths, so the batch is built from many sessions in parallel. When one session ends, the next one takes its slot and the hidden state is reset.
- **Sampled output.** Scoring every item is too expensive, so the loss uses other items in the mini-batch as negatives (an early form of in-batch negatives).
- **Ranking losses.** The paper introduced pairwise ranking losses (BPR and TOP1) in place of plain cross-entropy.

Lessons. Jannach and Ludewig (RecSys 2017) showed that a simple session-based nearest-neighbour method beat GRU4Rec in most of their tested configurations, and the best results often came from combining the two [2]. Hidasi and Karatzoglou later published a follow-up ("Recurrent Neural Networks with Top-k Gains", 2017) on loss functions and sampling for RNN session recommenders. The lasting lesson: sequence models need careful loss and negative-sampling choices, and strong simple baselines should always be reported.

1.2 SASRec (Kang and McAuley, 2018)

Problem. Markov-chain models use only the last few actions and work well on sparse data. RNNs can use the full history but are slow and need dense data. SASRec aimed to get both: capture long-range dependencies, but make predictions that depend mostly on a few relevant actions [3].

Architecture. A causal (left-to-right) Transformer encoder over item embeddings plus learned position embeddings. At each position the output is matched by dot product to the next item's embedding. The original training used binary cross-entropy with one sampled negative per positive.

Results. The authors report SASRec beat MC, CNN and RNN baselines on both sparse and dense datasets, and was about an order of magnitude more efficient than comparable CNN/RNN models [3]. SASRec became the default academic baseline and, later, the conceptual ancestor of most industrial "user tower" Transformers.

1.3 BERT4Rec (Sun et al., 2019)

Problem. Left-to-right models only use past context. BERT4Rec argued that this unidirectional design limits how well hidden representations capture the history [4].

Architecture. A bidirectional Transformer encoder trained with a Cloze (masked item) task: a share of items in each sequence is replaced by a [mask] token and predicted from both left and right context. At inference a [mask] token is appended to the end of the sequence and its final hidden state is used to predict the next item [4]. The paper notes that masking k items in a sequence of length n gives C(n, k) possible training samples, versus n for next-item training.

1.4 The replicability debate and gSASRec

Petrov and Macdonald (RecSys 2022) reviewed published BERT4Rec versus SASRec comparisons [5]. They found:

- BERT4Rec was not consistently better than SASRec across the literature.
- They could not reproduce the original BERT4Rec results with the default configuration.
- The original results could be reproduced if training ran much longer, up to about 30x.
- Their Hugging Face Transformers based implementation reproduced the results on 3 of 4 datasets with up to 95% less training time.

Their follow-up, gSASRec (RecSys 2023), pins down the cause [6]. Negative sampling raises the share of positives the model sees during training, so the model becomes **overconfident**: it overestimates the probability of positive interactions. gSASRec keeps the SASRec architecture but uses more negatives and a generalized BCE (gBCE) loss that corrects for the sampling rate. They report gSASRec can beat BERT4Rec (for example +9.47% NDCG on MovieLens-1M) while training about 73% faster, and that it scales to catalogues with over 1 million items, where BERT4Rec's full softmax does not [6].

Lesson for system designers. The training objective and negative-sampling scheme often matter more than uni- versus bidirectional attention. This is why most industrial user towers use a causal model with sampled softmax plus logQ correction (see PinnerFormer, KuaiFormer and PinFM below).

2. Target attention in industrial rankers: DIN, DIEN and BST (Alibaba)

2.1 DIN and DIEN

The Alibaba advertising team introduced **target attention** for CTR models. In DIN, a local activation unit weights each historical behaviour by its relevance to the candidate ad, so

the user representation changes with each candidate (as summarised in the DIEN paper [8]). DIN does not model relationships between items within the sequence.

DIEN (AAAI 2019) adds two layers [8]:

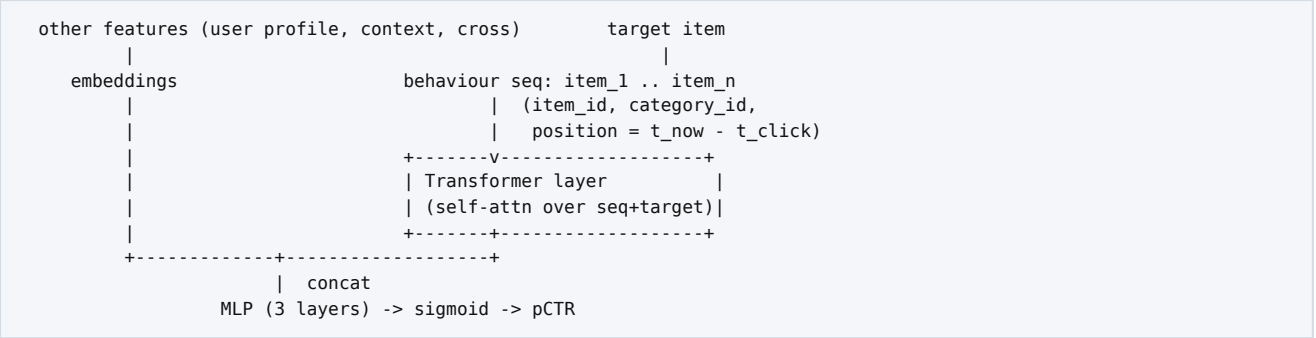
- An **interest extractor** GRU over the behaviour sequence, with an auxiliary loss that supervises each hidden state with the next behaviour.
- An **interest evolving** layer, AUGRU, which scales the GRU update gate by the attention score against the target item, so behaviours unrelated to the target have less effect on the hidden state.

The DIEN paper reports deployment in Taobao display advertising with a 20.7% CTR improvement [8]. However, sequential GRUs are slow to serve, which limited DIEN in production to about 50 behaviours according to the MIMN paper [9].

2.2 BST: Behavior Sequence Transformer (Taobao, 2019)

Problem. Embedding-and-MLP models such as WDL and DIN mostly concatenate or pool features and ignore the order of user behaviours [7].

Architecture.



Key choices reported for BST [7]:

- Each sequence token is built from item_id and category_id embeddings plus a **position feature defined as the time gap** between the recommendation time and the click time, not sinusoidal positions.
- The target item is appended to the sequence so self-attention can mix candidate and history.
- It runs in the **ranking stage** of Taobao recommendation.

Results. In the paper's table BST reached offline AUC 0.7894 versus 0.7734 for WDL and 0.7866 for DIN, and an online CTR gain of +7.57% versus +4.55% for DIN [7]. Secondary summaries report online latency rising from about 13 ms to 20 ms (approximate). The paper found that a single Transformer block worked best on its data; stacking more blocks did not help.

Lesson. BST is the template that TransAct and many later rankers follow: a short recent sequence, a light Transformer, candidate-aware fusion, and the result concatenated into a standard DNN ranker.

3. Long and lifelong sequences in ranking

The obvious next step is to use much longer histories. Lifelong histories on Taobao, Kuaishou or Pinterest contain 10^4 to 10^5 behaviours. Full target attention costs O(L x d) per candidate, and a ranker scores hundreds to thousands of candidates per request, so naive attention is not affordable. Each design below cuts this cost in a different way.

3.1 MIMN and the User Interest Center (Alibaba, KDD 2019)

Problem. Production DIEN used about 50 behaviours due to serving pressure, while longer histories clearly helped offline [9].

Design. A **system and algorithm co-design**:

- **UIC (User Interest Center)** is a separate serving module that keeps per-user interest state and updates it **incrementally** whenever a behaviour event arrives, not at request time.
- **MIMN (Multi-channel user Interest Memory Network)** is a memory-network model (based on Neural Turing Machine style memory with multiple channels) whose memory is exactly what UIC stores and updates.

At ranking time the model reads the precomputed memory, so latency is constant regardless of history length. The authors claim the design could in principle handle unlimited length, and it handled sequences in the thousands [9].

Results. On an Alibaba dataset MIMN improved AUC over DIEN by about 0.01. In an online A/B test from 2019-03-30 to 2019-05-10, MIMN with UIC gave +7.5% CTR and +6% RPM over DIEN [9].

Lesson. The memory is user-only (not candidate-aware), so it compresses away target-specific detail. The SIM paper later reported that MIMN accuracy drops when sequences grow 10x or more [10].

3.2 SIM: Search-based Interest Model (Alibaba, CIKM 2020)

Problem. Model lifelong histories (up to 54,000 behaviours) in CTR prediction while staying candidate-aware [10].

Architecture. Two cascaded search units:

```
lifelong behaviours (10^4 - 10^5)
         |
+--------v------------------------------+
| GSU: General Search Unit               |
|  hard search: behaviours with the same  |
|   category as the candidate             |   offline: User Behavior Tree
|  soft search: MIPS on embeddings        |<--  key1 = user_id
+--------+------------------------------+     key2 = category_id
         | top-K sub-sequence (SBS)            value = behaviour items
+--------v------------------------------+     (distributed, ~22 TB)
| ESU: Exact Search Unit                 |
|  multi-head target attention on SBS     |
|  + time-interval embeddings             |
+--------+------------------------------+
         v
  concat with short-term DIN features -> MLP -> pCTR
```

- **Soft search** embeds behaviours and uses maximum inner product search to get the top-K.
- **Hard search** simply keeps behaviours in the same category as the candidate.
- The authors found top-K lists from the two methods were very similar, and hard search was much cheaper. They deployed **hard search** online [10].
- The hard-search index is a two-level **User Behavior Tree (UBT)**: first key user ID, second key category ID, value the behaviour items. It is built offline, is distributed, and reaches about 22 TB [10].

Results. Deployed in Alibaba display advertising since 2019, with +7.1% CTR and +4.4% RPM, and a maximum modelled length of 54,000, about 54x MIMN [10]. Secondary write-ups report the added serving latency as only a few milliseconds (approximate) [28].

Lesson. SIM established the **two-stage GSU/ESU pattern** that nearly every later lifelong-sequence model adopts. Its weakness is that the GSU relevance metric (category match or a separately trained embedding) differs from the ESU's attention metric.

3.3 ETA and SDIM: hashing the GSU

ETA (Alibaba, 2021). ETA replaces the separate GSU with **locality-sensitive hashing** inside the model so retrieval is end-to-end trainable [11]. Each item embedding is turned into a binary signature with SimHash (random projections, keep the sign). The top-k behaviours are chosen by Hamming distance to the candidate's signature, which needs only cheap bit operations. Multi-head target attention then runs on those k items. Because the hash uses the same embeddings the attention uses, the GSU and ESU stay roughly consistent. The paper reports a 3.1% GMV gain [11]; the extended ETA-Net version reports +1.8% CTR and +3.1% GMV on Taobao over two-stage methods [12].

SDIM (Meituan, CIKM 2022). SDIM goes further and drops the attention step [13]. It hashes the candidate and every behaviour with several hash functions, then gathers (sums) the behaviours that share the candidate's signature. The authors prove this sampling approximates target attention's pattern. Behaviour hashing is moved into a separate **Behavior Sequence Encoding (BSE)** service, so the CTR model only does a lookup. Meituan reports +2.98% CTR and +2.69% VBR in its search system [13].

3.4 TWIN: TWo-stage Interest Network (Kuaishou, KDD 2023)

Problem. In SIM-like systems, the GSU's relevance metric differs from the ESU's target attention. The GSU therefore misses behaviours the ESU would rate highly and keeps ones the ESU would ignore [14].

Key idea: consistency. TWIN makes the GSU and ESU "twins". The **Consistency-Preserved GSU (CP-GSU)** uses the same target-attention relevance as the ESU, with the same architecture and parameters [14]. CP-GSU scores all ~10^4 long-term behaviours and keeps the **top 100**. The ESU then does weighted pooling (multi-head target attention) over those 100 finalists [14].

Making full attention affordable. Running real target attention on 10^4 to 10^5 behaviours per candidate would be too slow. TWIN **splits behaviour features**:

- **Inherent item features** (video ID, author, tags, duration and so on) do not depend on the user. Their linear projections (the keys) can be **precomputed and cached** per item, and shared across all users.
- **User-item cross features** (for example when the user watched it and how long they played it) are user-specific. TWIN compresses them to a cheap scalar bias added to the attention logit, avoiding a full projection.

```
per behaviour j:  K_j = W * inherent_j     (cached offline per item)
                  b_j = w . cross_j         (cheap, per user)
per candidate c:  score_j = (Q(c) . K_j) / sqrt(d) + b_j
CP-GSU: top-100 by score_j  ->  ESU: multi-head attention over top-100
```

Scale. Offline experiments use a production dataset of about 46 billion samples. With optimized online infrastructure the authors report a **99.3% reduction** in the compute bottleneck, which allowed deployment on Kuaishou's main traffic [14]. Secondary summaries report watch-time gains of about 1.3% to 6.2% across the Featured-Video, Discovery and Slide tabs (approximate; check the paper's tables) [29].

3.5 TWIN V2 (Kuaishou, CIKM 2024)

Problem. Users' full life-cycle histories now exceed 10^5 to 10^6 behaviours, beyond what even TWIN can scan online [15].

Design: divide and conquer.

1. **Offline hierarchical clustering.** Each user's behaviours are grouped into clusters of similar items. Secondary descriptions say this is recursive k-means that splits clusters until each leaf has at most a set number of items (gamma) [29]. Limiting cluster size compresses sequences well beyond 10^5 to a length manageable online [15].
2. **Cluster representation.** Each cluster becomes one virtual behaviour (aggregated features).
3. **Cluster-aware target attention.** The GSU/ESU run on clusters, and attention scores are adjusted by cluster size so large clusters (strong interests) carry their weight [15].

Results. Deployed on primary traffic serving hundreds of millions of daily active users. The paper's Table 5 lists watch-time gains of +0.672%, +0.800% and +0.728% on the Featured-Video, Discovery and Slide tabs; the authors note that a 0.1% gain is significant at Kuaishou [15].

3.6 LONGER (ByteDance, RecSys 2025)

Problem. Two-stage retrieval (SIM, TWIN) is lossy. ByteDance wanted **end-to-end** Transformer modelling of sequences up to 10,000 items in ranking [17].

Design [17][33]:

- **Global tokens** (for example the target item and user tokens) that attend across the whole context to stabilise attention over long sequences.
- **Token merge**: adjacent behaviours are merged into fewer tokens, with lightweight **InnerTransformers** that keep detail inside each merged group.
- **Hybrid attention**: cross-attention from a small set of query tokens over the long sequence, followed by self-attention layers on the short output, to cut quadratic cost.
- Engineering: mixed-precision training, activation recomputation, and **KV-cache serving** so the user-only part is computed once per request and shared across candidates.

Results. Consistent offline and online gains in advertising and e-commerce. The v2 paper says LONGER is fully deployed in dozens of ByteDance scenarios serving billions of users [17].

4. Pinterest: real-time, batch and lifelong user sequences

Pinterest has published one of the most complete public lines of user-history work, so it is worth following in order.

4.1 PinnerSage (KDD 2020): multi-embedding users

Problem. One user vector cannot represent users with many unrelated interests (for example recipes, tattoos and home decor) [18].

Design. Take the user's action pins from the last 90 days, cluster their PinSage embeddings with Ward hierarchical clustering, and represent each cluster by its **medoid** (a real pin from the user's history, which keeps clusters coherent and robust to outliers). Clusters get an importance score; at retrieval time the system samples a few medoids and uses each as an ANN query [18]. A/B tests showed it beat single-embedding methods.

4.2 PinnerFormer (KDD 2022): a batch user embedding trained for the long term

Problem. Pinterest wanted a single user embedding, computed in a **daily batch job**, that works nearly as well as a real-time embedding, and that predicts long-term engagement rather than only the next action [19].

Architecture.

```
last N actions (pin, action type, surface, time features)
     |  each pin: 256-d PinSage embedding (frozen) + learned feature embeddings
+----v-----------------------------+
| causal Transformer encoder         |
+----+-----------------------------+
     v
user embedding (L2-normalised), computed daily for every user
     |
     v  ANN retrieval / ranking feature, matched against pin embeddings
```

Dense all-action loss. Instead of predicting only the next action (SASRec style), each position's embedding is trained to predict **all positive engagements (repins, close-ups, clicks) in a future window of up to 28 days** [19]. Training uses sampled softmax with a mix of in-batch and random negatives and logQ correction [19].

Why it matters. The objective makes the embedding robust to staleness. Moving from real-time to daily batch inference cut Recall@10 by 13.9% with the SASRec-style next-action objective, but only by 8.3% with the dense all-action objective [19]. A/B tests showed better engagement and retention than the previous user representation, and it has been in production since fall 2021 [19]. A secondary summary reports the team did not scale beyond about 256 actions (approximate).

Lesson. For an embedding that will be computed offline and reused, train it to forecast a window of future behaviour, not one next click.

4.3 TransAct (KDD 2023): real-time actions inside the ranker

Problem. PinnerFormer is updated daily, so Homefeed ranking could not react to what a user did seconds ago. A real-time sequence model is needed, but running a full user model on every action is expensive [20].

Hybrid design. TransAct is a module inside Pinterest's Homefeed ranking model (Pinnability) that encodes the most recent actions in real time, while PinnerFormer and other batch features provide long-term context [20].

```
real-time action stream (Kafka-like)                  candidate pin
       |                                                    |
latest ~100 actions: PinSage emb + action type + time       |
       |                                                    |
early fusion: concat each action with candidate embedding <-+
       |
Transformer encoder (small: ~2 layers, reported)
       |
output compression: first K tokens + max pooling -> flatten
       |
concat with PinnerFormer user emb + other features -> DCN-style ranker
       |
multi-head outputs (repin, click, hide, ...) -> utility score
```

Key tricks reported in the paper [20]:

- **Early fusion** of the candidate with every sequence item (concatenation), which made the Transformer candidate-aware.
- **Action type embeddings** were important; **positional encodings added little**.
- **Random time-window masking** during training: hide actions from a random recent window so the model does not over-rely on the last few actions and recommendations stay diverse.

- Serving on GPUs, since the per-candidate Transformer is expensive.

Results and challenges. A third-party summary of the paper reports +11% Homefeed repin volume and -10% hides in online A/B tests (approximate). TransAct was deployed in Homefeed, Related Pins, Notifications and Search [20]. The paper discusses three production problems: high serving complexity, reduced diversity, and **engagement decay** (gains shrink if the model is not retrained often), which led to frequent retraining.

4.4 TransAct V2 (2025): lifelong sequences in ranking

Problem. TransAct used only recent actions (about 100). TransAct V2 extends Homefeed ranking to lifelong histories of about 16,000 actions, roughly 160x more [21][22][31].

Architecture.

```
lifelong seq (~16k, int8 PinSage, offline store)  real-time seq   impression seq
        \                                            |             /
         +------- nearest-neighbour selection by candidate -------+
                 (dot product in PinSage space; always keep
                  the most recent real-time actions)
                            |
                   short selected sequence
                            |
              Transformer (fused single-kernel)
                            |
            features -> ranker heads  +  Next Action Loss (aux)
```

- Each token holds timestamp, a multi-hot action type, surface, and a **32-dimensional PinSage embedding**; lifelong actions are stored losslessly with **int8 quantization** [21][22].
- **Candidate-anchored nearest-neighbour (NN) selection** picks a short subsequence from the lifelong, real-time and impression sequences. It is a SIM-style GSU, with the most recent actions always kept [21].
- **Next Action Loss**: an auxiliary loss that asks the model to predict what the user does next, not only engagement probability [21][31].
- Serving: a fused Triton kernel combines dequantization to float16, normalisation and NN search, cutting model latency by 20% versus native PyTorch. A single-kernel Transformer (SKUT) is reported 6.6x faster than PyTorch in the forward pass. A broadcast-free NN search reportedly cut PCIe transfers for sequences by 8x. End-to-end inference latency improved by 103x (p50), 338x (p90) and 250x (p99) versus the unoptimised baseline [21].

Results. Pinterest reports +6.35% repins, +1.41% time spent, a 12.8% drop in hides [31], and a 13.31% increase in top-3 repin hit rate [22].

4.5 PinFM (RecSys 2025): a foundation model for user activity

Problem. Many Pinterest surfaces each trained their own sequence modules. Pinterest wanted one large pretrained model of user activity to share across surfaces [23].

Design.

- **Pretraining.** A Transformer with more than 20 billion parameters (most in ID embedding tables) is pretrained on about two years of user actions, with contrastive

next-token, multi-token and future-token losses (in the PinnerFormer tradition) [23].

- **Fine-tuning.** The pretrained Transformer and embedding tables are plugged into ranking models for each application and fine-tuned with the ranking losses; the pretraining losses can optionally be kept [23]. An ablation found that switching to bidirectional attention in fine-tuning cost 1.4% in evaluation metrics, so causal attention is kept [23].
- **DCAT (Deduplicated Cross-Attention Transformer).** At serving, there are about 1,000 candidates per unique user sequence (about 10 in training). DCAT splits the model into a **context part** that runs the Transformer once on the user history and caches its KV, and a **crossing part** that scores each candidate against the cached history [23]. It raised throughput by 600% on internal data. Embeddings are int4-quantized [23].
- **Cold start.** A conference talk describes ID noise, age-dependent dropout and a class token for fresh pins (secondary).

Results. +20% engagement on new items, deployed to Home Feed and Related Items for more than half a billion users at neutral cost and marginal latency increase [23].

4.6 PinRec (2025): generative retrieval from the same sequences

PinRec uses a user-sequence Transformer as a generative retriever, with outcome conditioning (for example generating for saves versus clicks) and multi-token generation [32]. It shows the same history encoder moving from ranking into retrieval.

5. Kuaishou: sequence models for retrieval

KuaiFormer (2024)

Problem. Single-vector two-tower retrieval squeezes all user interests into one vector and handles long sequences poorly [16].

Design [16]:

- **Multiple learnable query tokens** (like BERT's [CLS]) appended to the sequence, each producing one interest vector for multi-interest ANN retrieval.
- **Adaptive sequence compression.** Older behaviours are merged in groups of 64, middle behaviours in groups of 16, each group aggregated by a single-layer bidirectional Transformer; recent items are kept as-is. For example, 256 behaviours are fed as 55 tokens.
- A causal Transformer over the compressed sequence, trained with a smoothed in-batch softmax with logQ correction.

Scale. In production since May 2024 in Kuaishou's short-video recommendation, serving more than 400 million daily active users [16]. Kuaishou has since published generative recommenders for long sequences, such as GEMs (2026) with a multi-stream decoder [34].

6. Generative recommenders and foundation user models

6.1 Meta HSTU and Generative Recommenders (ICML 2024)

Problem. Classic deep learning recommenders (DLRMs) did not keep improving with more compute. Meta asked whether a sequence model of user actions, scaled like an LLM, would [24].

Reformulation. Meta rewrote ranking and retrieval as **sequential transduction**. The user's history becomes one chronological stream mixing content and actions. Retrieval predicts the next content item; ranking predicts the action on an inserted candidate. Most hand-built features are replaced by the sequence itself [24].

HSTU (Hierarchical Sequential Transduction Unit) [24][30]:

- **Pointwise aggregated attention.** Scores go through SiLU instead of row-wise softmax and are scaled by sequence length. This keeps the intensity of user preferences (how many related actions there are) and suits non-stationary, streaming vocabularies.
- **Relative attention bias** from both position and time gaps.
- A fused, simplified layer (fewer linear layers than a standard Transformer block) that uses less memory, enabling deeper stacks.
- Efficiency: HSTU is 5.3x to 15.2x faster than FlashAttention2 Transformers at length 8,192 [24]. The paper also describes stochastic length (sampling sub-sequences of long histories) to cut cost. At inference, **M-FALCON** batches many candidates into one pass with attention masks so the user history cost is shared.

Results. Up to 65.8% better NDCG than baselines on public and synthetic data; a **1.5-trillion-parameter** generative recommender improved online A/B metrics by 12.4% across multiple surfaces of a platform with billions of users. Model quality followed a power law of training compute across three orders of magnitude, reaching GPT-3/LLaMa-2 scale [24]. Attention dominates inference time (over 90% in one characterisation study), which drove fused kernels such as NVIDIA's cuDNN HSTU attention [30].

6.2 Netflix foundation model for personalization (2025)

Problem. Netflix ran many specialised models ("Continue Watching", "Today's Top Picks" and others), each trained separately on similar data. Maintenance was expensive and improvements did not transfer between models [25][35].

Design (from the Netflix tech blog [25] and secondary summaries [35]):

- An LLM-style, autoregressive Transformer trained on **hundreds of billions of interactions from more than 300 million users**, roughly the token scale of an LLM.
- **Interaction tokenization.** Raw events are merged into meaningful tokens (for example consecutive related events grouped), balancing sequence length and detail, similar to BPE for text.
- Very long histories are handled with **sparse attention and sliding-window sampling** in training (Netflix published a separate sliding-window training paper [27]), plus KV caching at inference.

- **Objectives.** Next-item prediction, with auxiliary targets (for example genre or original language) and **multi-token prediction** of several future items, which also reduces myopia.
- **Cold start.** Item embeddings combine learnable ID embeddings with metadata embeddings, so new titles get useful vectors before anyone watches them. Training supports incremental updates with unseen entities.
- Reported scaling from millions to billions of parameters, with gains following scaling laws (secondary).

Integration [26][35]. Netflix describes three ways downstream teams consume the model:

Pattern	How it works	Trade-off
Embeddings	Precomputed user/item embeddings published to an Embedding Store	Cheap and easy; can be stale
Subgraph	FM layers embedded in the downstream model and trained with it	Fresh, adaptable; downstream must build the FM's inputs
Fine-tuning	FM fine-tuned directly for a task	Best fit; most compute and ops cost

6.3 LinkedIn and others

LinkedIn has publicly described a large foundation ranking model (360Brew, reported at about 150B parameters) that reads member history as part of its input, but public details on its user-history component come mainly from secondary sources and are not covered here.

7. Comparison table

System (org, year)	Stage	History length	Selection / compression	Model	Public results
GRU4Rec (academic, 2015) [1]	Next-item	Session	None	GRU, in-batch negatives	Beat item-kNN in paper; tuned kNN competitive [2]
SASRec (UCSD, 2018) [3]	Next-item	~10^2	None	Causal Transformer	~10x faster than CNN/RNN
BERT4Rec (Alibaba, 2019) [4]	Next-item	~10^2	None	Bidirectional, Cloze	Strong only with long training [5]
gSASRec (Glasgow, 2023) [6]	Next-item	~10^2	None	SASRec + gBCE	+9.47% NDCG vs BERT4Rec (ML-1M)
DIEN (Alibaba, 2019) [8]	Ranking	~50 online	None	GRU + AUGRU	+20.7% CTR
BST (Alibaba, 2019) [7]	Ranking	Short (recent clicks)	None	1 Transformer block	+7.57% CTR, AUC 0.7894
MIMN + UIC (Alibaba, 2019) [9]	Ranking	~10^3	Incremental memory	Memory network	+7.5% CTR, +6% RPM
SIM (Alibaba, 2020) [10]	Ranking	Up to 54,000	Category hard search (UBT)	GSU + ESU attention	+7.1% CTR, +4.4% RPM

System (org, year)	Stage	History length	Selection / compression	Model	Public results
ETA (Alibaba, 2021) [11]	Ranking	~10^4	SimHash + Hamming top-k	End-to-end attention	+3.1% GMV
SDIM (Meituan, 2022) [13]	Ranking	~10^4	Multi-hash sampling	No attention	+2.98% CTR
TWIN (Kuaishou, 2023) [14]	Ranking	10^4 - 10^5	CP-GSU, top-100	Shared GSU/ESU attention	99.3% less compute bottleneck
TWIN V2 (Kuaishou, 2024) [15]	Ranking	10^5+	Hierarchical clustering	Cluster-aware attention	+0.67% to +0.80% watch time
LONGER (ByteDance, 2025) [17]	Ranking	Up to 10,000	Token merge, global tokens	End-to-end Transformer	Dozens of scenarios
PinnerSage (Pinterest, 2020) [18]	Retrieval	90 days	Ward clustering, medoids	Non-parametric	Beat single embedding
PinnerFormer (Pinterest, 2022) [19]	Retrieval/feature	~256 (approx.)	None, daily batch	Causal Transformer, dense loss	Batch drop 8.3% vs 13.9%
TransAct (Pinterest, 2023) [20]	Ranking	~100 real-time	Random time-window mask	Small Transformer, early fusion	~+11% repins (reported)
TransAct V2 (Pinterest, 2025) [21]	Ranking	~16,000	Candidate NN search	Transformer + next-action loss	+6.35% repins, -12.8% hides
KuaiFormer (Kuaishou, 2024) [16]	Retrieval	256 -> 55 tokens	Group merging (64/16)	Causal + multi-query	400M+ DAU
PinFM (Pinterest, 2025) [23]	Ranking (FM)	Lifelong, 2 yrs	DCAT KV reuse	20B+ param pretrained	+20% new-item engagement
HSTU GR (Meta, 2024) [24]	Retrieval + ranking	Up to 8,192+	Stochastic length, M-FALCON	HSTU, 1.5T params	+12.4% online
Netflix FM (2025) [25]	Shared FM	Lifetime	Tokenization, sliding window	Autoregressive Transformer	Replaces many models

8. Recurring design patterns

8.1 Split the user part from the candidate part

The single most important serving trick is to compute the **user-only part once per request** and reuse it across all candidates:

```
request -> [ user history encoder ]  ---- once per request (or cached / batch)
                  |   KV cache / embedding
        +---------+----------+----------+
        v         v          v          v
     cand_1    cand_2  ...  cand_1000      cheap cross-attention or dot product
```

Examples: MIMN/UIC (precompute memory), SDIM's BSE service, TWIN's cached item keys, LONGER's KV-cache serving, PinFM's DCAT, HSTU's M-FALCON, PinnerFormer's daily embeddings.

8.2 Retrieve, then attend

For histories past about 10^3, every production system first selects a short relevant subsequence and then runs exact attention: category search (SIM), LSH (ETA, SDIM), shared attention scores (TWIN), clustering (TWIN V2, PinnerSage), and embedding NN search (TransAct V2). TWIN's lesson is that the **selection metric should match the final model's metric**.

8.3 Choose the objective for how the representation is used

- Batch embeddings: forecast a future window (PinnerFormer's dense all-action loss).
- Real-time ranking: candidate-aware target attention with a multi-task ranking head (TransAct), plus an auxiliary next-action loss (TransAct V2).
- Foundation models: next-token, multi-token and future-token losses (PinFM, Netflix).
- With sampled negatives, correct for sampling (logQ, gBCE) or the model becomes overconfident [6].

8.4 Real-time versus batch

Pinterest's hybrid (daily PinnerFormer + real-time TransAct) is the clearest public example of splitting long-term and short-term interest across infrastructure tiers. Real-time models react within seconds but cost more and can reduce diversity (hence random time-window masks); batch models are cheap but stale (hence objectives robust to staleness).

8.5 Scale like an LLM

HSTU, PinFM, Netflix's model and LONGER all report that larger sequence models trained on more interactions keep improving, which DLRM-style models did not [24][25]. The cost is dominated by attention on long sequences, so these systems invest in custom kernels (Triton, cuDNN HSTU attention), quantization (int8/int4 embeddings), and KV reuse.

9. Key takeaways for system designers

1. **History is the main feature.** Moving from ~50 to 10^4 or more behaviours has repeatedly given multi-percent online gains (MIMN, SIM, TWIN, TransAct V2). Plan storage and serving for lifelong histories from the start.
2. **Never pay O(L) per candidate if you can pay it once per request.** Split user-only computation from candidate crossing and cache it (KV cache, precomputed keys, embeddings).
3. **Use two stages for very long sequences, and keep them consistent.** A cheap GSU picks about 100 behaviours; an exact ESU attends. Share parameters or metrics between them (TWIN), or make retrieval differentiable (ETA).
4. **Compress the old, keep the recent exact.** Clustering (TWIN V2), group merging (KuaiFormer, LONGER) and always-keep-recent rules (TransAct V2) are cheap and

effective.

5. **Pick the training target for the consumer.** Daily embeddings should forecast a window of future behaviour; rankers should be candidate-aware and multi-task.
6. **Mind negative sampling.** In-batch and sampled negatives need logQ correction or a calibrated loss such as gBCE; otherwise models are overconfident and benchmarks mislead.
7. **Do not trust academic leaderboards blindly.** BERT4Rec versus SASRec showed that training time and loss choices can flip conclusions; also compare against strong kNN baselines.
8. **Real-time features cause decay and diversity problems.** Retrain often, use random time window masks or similar regularisation, and monitor hides and diversity, not just clicks.
9. **Positional encodings matter less than action types and timestamps.** BST uses time gaps; TransAct found action-type embeddings and early fusion mattered more than position.
10. **Foundation user models are now standard at the largest platforms.** Pretrain once on billions of interactions, then serve through embeddings, subgraphs or fine-tuning (Netflix) or as a module inside rankers (PinFM). Budget for custom kernels and quantization.
11. **Cold start needs explicit handling in ID-heavy sequence models.** Combine ID embeddings with content/metadata embeddings (Netflix) and regularise new IDs (PinFM).

References

1. Hidasi, B., Karatzoglou, A., Baltrunas, L., Tikk, D. "Session-based Recommendations with Recurrent Neural Networks." arXiv / ICLR 2016, 2015. https://arxiv.org/abs/1511.06939
2. Jannach, D., Ludewig, M. "When Recurrent Neural Networks meet the Neighborhood for Session-Based Recommendation." RecSys 2017. https://web-ainf.aau.at/pub/jannach/files/Conference_RecSys_2017.pdf
3. Kang, W.-C., McAuley, J. "Self-Attentive Sequential Recommendation." UC San Diego, ICDM 2018. https://arxiv.org/abs/1808.09781
4. Sun, F. et al. "BERT4Rec: Sequential Recommendation with Bidirectional Encoder Representations from Transformer." Alibaba, CIKM 2019. https://arxiv.org/abs/1904.06690
5. Petrov, A., Macdonald, C. "A Systematic Review and Replicability Study of BERT4Rec for Sequential Recommendation." University of Glasgow, RecSys 2022. https://arxiv.org/abs/2207.07483
6. Petrov, A., Macdonald, C. "gSASRec: Reducing Overconfidence in Sequential Recommendation Trained with Negative Sampling." University of Glasgow, RecSys 2023. https://arxiv.org/abs/2308.07192
7. Chen, Q., Zhao, H., Li, W., Huang, P., Ou, W. "Behavior Sequence Transformer for E-commerce Recommendation in Alibaba." Alibaba, DLP-KDD 2019. https://arxiv.org/abs/1905.06874
8. Zhou, G. et al. "Deep Interest Evolution Network for Click-Through Rate Prediction." Alibaba, AAAI 2019. https://arxiv.org/abs/1809.03672

9. Pi, Q., Bian, W., Zhou, G. et al. "Practice on Long Sequential User Behavior Modeling for Click-Through Rate Prediction." Alibaba, KDD 2019. https://arxiv.org/abs/1905.09248
10. Pi, Q., Zhu, X., Zhou, G. et al. "Search-based User Interest Modeling with Lifelong Sequential Behavior Data for Click-Through Rate Prediction." Alibaba, CIKM 2020. https://arxiv.org/abs/2006.05639
11. Chen, Q., Pei, C. et al. "End-to-End User Behavior Retrieval in Click-Through Rate Prediction Model." Alibaba, 2021. https://arxiv.org/abs/2108.04468
12. "Efficient Long Sequential User Data Modeling for Click-Through Rate Prediction" (ETA-Net). Alibaba, 2022. https://arxiv.org/abs/2209.12212
13. Cao, Y. et al. "Sampling Is All You Need on Modeling Long-Term User Behaviors for CTR Prediction." Meituan, CIKM 2022. https://arxiv.org/abs/2205.10249
14. Chang, J. et al. "TWIN: TWo-stage Interest Network for Lifelong User Behavior Modeling in CTR Prediction at Kuaishou." Kuaishou, KDD 2023. https://arxiv.org/abs/2302.02352
15. Si, Z. et al. "TWIN V2: Scaling Ultra-Long User Behavior Sequence Modeling for Enhanced CTR Prediction at Kuaishou." Kuaishou, CIKM 2024. https://arxiv.org/abs/2407.16357
16. "KuaiFormer: Transformer-Based Retrieval at Kuaishou." Kuaishou, 2024. https://arxiv.org/abs/2411.10057
17. "LONGER: Scaling Up Long Sequence Modeling in Industrial Recommenders." ByteDance, RecSys 2025. https://arxiv.org/abs/2505.04421
18. Pal, A., Eksombatchai, C. et al. "PinnerSage: Multi-Modal User Embedding Framework for Recommendations at Pinterest." Pinterest, KDD 2020. https://arxiv.org/abs/2007.03634
19. Pancha, N., Zhai, A., Leskovec, J., Rosenberg, C. "PinnerFormer: Sequence Modeling for User Representation at Pinterest." Pinterest, KDD 2022. https://arxiv.org/abs/2205.04507
20. Xia, X. et al. "TransAct: Transformer-based Realtime User Action Model for Recommendation at Pinterest." Pinterest, KDD 2023. https://arxiv.org/abs/2306.00248
21. "TransAct V2: Lifelong User Action Sequence Modeling on Pinterest Recommendation." Pinterest, 2025. https://arxiv.org/abs/2506.02267
22. Pinterest Engineering. "Next-Level Personalization: How 16k+ Lifelong User Actions Supercharge Pinterest's Recommendations." Medium, 2025. https://medium.com/pinterest-engineering/next-level-personalization-how-16k-lifelong-user-actions-supercharge-pinterests-recommendations-bd5989f8f5d3
23. "PinFM: Foundation Model for User Activity Sequences at a Billion-scale Visual Discovery Platform." Pinterest, RecSys 2025. https://arxiv.org/abs/2507.12704
24. Zhai, J. et al. "Actions Speak Louder than Words: Trillion-Parameter Sequential Transducers for Generative Recommendations." Meta, ICML 2024. https://arxiv.org/abs/2402.17152
25. Netflix Technology Blog. "Foundation Model for Personalized Recommendation." Netflix, 2025. https://netflixtechblog.com/foundation-model-for-personalized-recommendation-1a0bd8e02d39
26. Netflix Technology Blog. "Integrating Netflix's Foundation Model into Personalization Applications." Netflix, 2025. https://netflixtechblog.medium.com/integrating-netflixs-foundation-model-into-personalization-applications-cf176b5860eb

27. "Sliding Window Training: Utilizing Historical Recommender Systems Data for Foundation Models." Netflix, RecSys 2024. https://arxiv.org/abs/2409.14517
28. Machine Learning Frontiers (S. Bhattacharya). "Towards Life-Long User History Modeling in Recommender Systems." Substack, 2024. https://mlfrontiers.substack.com/p/towards-life-long-user-history-modeling
29. Machine Learning Frontiers. "Target Attention Is All You Need: Modeling Extremely Long User Action Sequences in Recommender Systems." Substack, 2024. https://mlfrontiers.substack.com/p/target-attention-is-all-you-need
30. NVIDIA. "HSTU Attention." cuDNN frontend documentation, 2025. https://docs.nvidia.com/cudnn/fe-oss-apis/hstu/hstu_attention
31. Social Media Today. "Pinterest Updates Ranking Model With More Data." 2025. https://www.socialmediatoday.com/news/pinterest-updates-ranking-model-with-more-data/750122/
32. "PinRec: Outcome-Conditioned, Multi-Token Generative Retrieval for Industry-Scale Recommendation Systems." Pinterest, 2025. https://arxiv.org/abs/2504.10507
33. Machine Learning at Scale. "[RecSys] Part 3: LONGER, scaling up long sequence modelling in industrial recommenders." Substack, 2025. https://machinelearningatscale.substack.com/p/recsys-part-3-longer-scaling-up-long
34. "GEMs: Long-sequence generative recommendation with a multi-stream decoder." Kuaishou, 2026. https://arxiv.org/abs/2602.13631
35. ZenML LLMOps Database. "Netflix: Foundation Model for Large-Scale Personalized Recommendation." 2025. https://www.zenml.io/llmops-database/foundation-model-for-large-scale-personalized-recommendation

06. Generative Recommenders (2023-2026): HSTU, Semantic IDs, OneRec, MTGR and the Foundation-Model Turn

Between 2023 and 2026 the industrial recommender stack went through its largest architectural change since deep learning recommendation models (DLRMs) replaced matrix factorization. Three ideas drove it:

1. **Recommendation as sequence transduction.** Meta's HSTU paper ("Actions Speak Louder than Words") recast retrieval and ranking as generative modeling over a user's chronological stream of (item, action) tokens. It reported power-law quality scaling with compute and production models of about 1.5 trillion parameters [1][2].
2. **Items as token sequences (semantic IDs).** Google's TIGER replaced atomic item IDs with short tuples of discrete codes learned by residual quantization (RQ-VAE). Retrieval then becomes autoregressive decoding of the next item's code tuple [4][5].
3. **One model instead of a cascade.** Kuaishou's OneRec, Meituan's MTGR and EGA, Tencent's GPR, and others replaced parts or all of the retrieve -> pre-rank -> rank -> re-rank cascade with a single generative model, often post-trained with reinforcement learning against a reward model [10][11][12][16][41].

At the same time, Netflix, LinkedIn and Pinterest published "foundation model" approaches. Each pre-trains one large user-sequence model and feeds many downstream recommenders from it [25][27][29]. X (formerly Twitter) open-sourced a For You pipeline whose main ranker, Phoenix, is a transformer over the viewer's engagement sequence with candidate-isolated attention [31].

This chapter covers each design: the problem it targets, its architecture and data flow, model and scale numbers where public, the key tricks, and the lessons learned. It ends with a comparison table and takeaways for system designers.

> Source note: this chapter was compiled when only some primary hosts could be fetched directly. Facts marked "(approx.)" or "as reported" come from abstracts or secondary technical notes, not from a full read of the paper. Check them against the cited primary source before you depend on an exact figure.

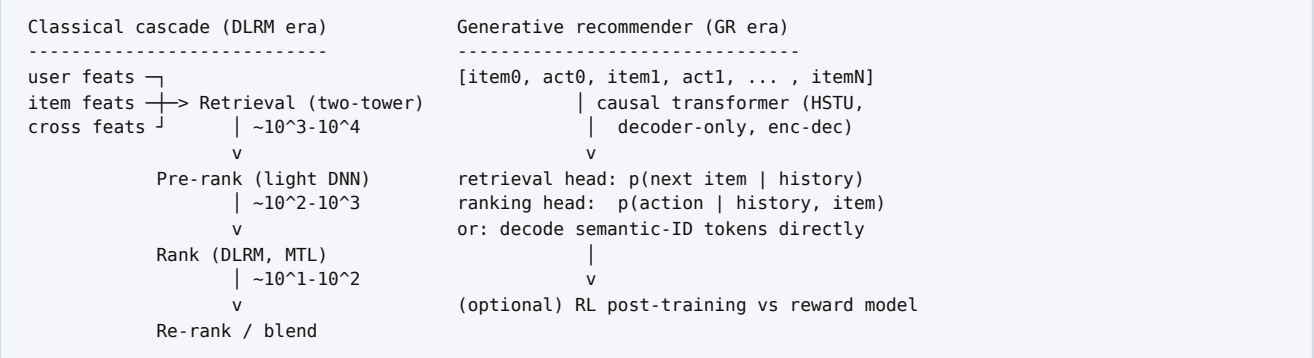
---

1. The conceptual shift: from DLRM to generative recommenders

A classical DLRM ranker scores each (user, candidate) pair independently. It uses hundreds of hand-built features, very large sparse embedding tables, and a feature-interaction network (DCN, DeepFM, and similar). Three structural problems limited further scaling:

- **Hardware inefficiency.** Embedding tables often hold more than 90% of DLRM parameters, and those parameters are sparse and memory-bound [22][21]. Secondary analyses put DLRM model FLOPs utilization (MFU) at roughly 4-5%, against 40-60% for LLM training [23].
- **Cascade information loss.** Items dropped by an early stage are never seen by later, stronger models. Each stage also optimizes a different objective [22].
- **Per-impression training cost.** Each impression is a separate training example, so a user with 500 interactions gets 500 separate forward passes over almost the same history [18].

Generative recommenders (GRs) address all three. They model the user as a single token sequence, run one causal forward pass that produces supervision at every position, and use dense, GEMM-heavy transformer compute that scales the way LLMs do.



---

2. Meta HSTU: "Actions Speak Louder than Words"

2.1 Problem

Meta wanted to know whether recommendation quality scales with compute the way language modeling does, and what architecture would allow it on high-cardinality, non-stationary streaming data with billions of users [1]. Standard transformers did not fit well. Softmax attention normalizes away the intensity of user interest, and the FFN-heavy blocks used too much activation memory for the very large batches that industrial training needs [18].

2.2 Problem formulation

HSTU interleaves content tokens (Φ) and action tokens (a) into one chronological sequence [Φ0, a0, Φ1, a1, ..., Φn-1, an-1] of length 2n_c. Other categorical features are merged onto this timeline by timestamp, and runs of identical values are compressed [18]. Both main tasks then become sequence transduction:

- **Ranking:** p(a_i | Φ0, a0, ..., Φi). This is target-aware, because the candidate item is in the context when the action is predicted.
- **Retrieval:** p(Φ_i | Φ0, a0, ..., a_{i-1}). This predicts the next item.

Under a causal mask, one forward pass yields n_c supervision signals. For a user with about 500 interactions, the secondary analysis estimates roughly a 500x reduction in training

forward passes compared with impression-level DLRM training [18].

2.3 The HSTU block

Each layer has three parts [1][18]:

1. **Pointwise projection:** U, V, Q, K = Split(SiLU(f1(X))).
2. **Pointwise aggregated attention:** A(X)V(X) = SiLU(Q K^T + rab_{p,t}) ⊙ V. There is no softmax, so the weights do not sum to 1 and the absolute strength of interest is preserved. rab_{p,t} is a relative attention bias over both position and time differences.
3. **Pointwise transformation:** Y = f2(Norm(A(X)V(X)) ⊙ U(X)). The gate U replaces the usual 4x-wide FFN.

```
X ──> Linear ──> SiLU ──> split ──> U, V, Q, K
                                     |
          SiLU(Q·K^T + rab(pos,time)) ⊙ V      <- no softmax; rel. pos+time bias
                                     |
                     LayerNorm(·) ⊙ U          <- gating replaces the FFN
                                     |
                          Linear ──> + X (residual)
```

The secondary analysis reports per-layer activation memory falling from about 33x hidden dimension (standard transformer) to about 14x [18]. This matters because industrial batch sizes run from tens of thousands to hundreds of thousands, and activation memory is usually the limit [18].

2.4 Efficiency tricks

- **Stochastic Length (SL).** Long user histories are sparsely subsampled during training, so attention cost grows sub-quadratically. The secondary notes report 80%+ sparsity with almost no loss in quality [18].
- **Efficient attention kernels.** The paper reports HSTU running 5.3x to 15.2x faster than FlashAttention2-based transformers on 8192-length sequences [1]. The open-source repo now includes Triton kernels, an HSTU attention kernel based on FlashAttention-3 for H100 GPUs, and row-wise FP8 GEMMs [2].
- **M-FALCON (inference).** To score m candidates against one history, the candidates are appended to the sequence with an attention mask so that candidates see only the history and not each other. They are processed in micro-batches, and the history's K/V cache is shared across micro-batches. The secondary analysis attributes speedups of tens to hundreds of times to the combination [18].

2.5 Scale and results

- Up to 65.8% NDCG improvement over baselines on public and synthetic datasets [1].
- 1.5-trillion-parameter models deployed on multiple surfaces of a billion-user platform, with a reported +12.4% on online A/B metrics [1].
- Quality scales as a power law of training compute across three orders of magnitude, up to roughly GPT-3/LLaMa-2 training compute [1]. Secondary notes give the fitted laws as HR@100 = 0.15 + 0.0195 ln C for retrieval and NE = 0.549 - 0.0053 ln C for

ranking. The sweep covered sequence lengths 512-8192, hidden dimensions 256-1024, and up to 24 layers [18].
- Public benchmark reproduction from the repo: on ML-20M, HSTU-large reaches HR@10 0.3556 against 0.2889 for SASRec (+23.1%). On Amazon Books it reaches NDCG@10 0.0262 against 0.0164 (+60.7%) [2].

2.6 Follow-ups

The repo also contains a "DLRM-v3" ranker built on HSTU with an MLCommons LoadGen inference harness, and "HSTU Ultra", which adds semi-local and target-aware attention and FP8 projections [2]. In 2026 Meta published "Bending the Scaling Law Curve in Large-Scale Recommendation Systems" (listed under the ULTRA-HSTU name), and an analysis of how well generative recommenders generalize [35][45].

Lessons. (1) Softmax is not sacred: for recommendation, unnormalized SiLU attention plus a relative time bias works better. (2) Most of the value comes from restructuring training (one sequence per user, causal supervision), not only from model size. (3) Inference needs its own algorithm (M-FALCON) to make target-aware ranking over many candidates affordable.

---

3. Semantic IDs and TIGER (Google)

3.1 Problem

Atomic item IDs carry no meaning: item 81723 and item 81724 are unrelated unless behavior links them. Very large ID vocabularies also make softmax retrieval expensive, and brand-new items have no trained embedding at all. TIGER ("Recommender Systems with Generative Retrieval", NeurIPS 2023) proposed describing every item with a short tuple of discrete semantic codes and training a sequence model to generate the next item's code tuple [4][5].

3.2 Pipeline

```
item text/metadata ──> frozen text encoder (e.g., Sentence-T5) ──> dense emb e
                                                                    |
                      RQ-VAE: encoder -> z
                      level 1: c1 = argmin ||z - C1[k]||, r1 = z - C1[c1]
                      level 2: c2 = argmin ||r1 - C2[k]||, r2 = r1 - C2[c2]
                      level 3: c3 = ...
                                                                    |
                  Semantic ID = (c1, c2, c3 [, dedup token])  e.g. (10, 5, 42)
                                                                    |
user history = [SID(item1), SID(item2), ...] ──> Transformer seq2seq (T5-style)
                                                  ──> beam-decode SID of next item
                                                  ──> map SID -> item (lookup)
```

- **RQ-VAE loss.** Reconstruction error, plus a codebook term and a commitment term (weight β) at each level. Gradients pass through the quantizer with the straight-through estimator [21].
- **Hierarchy.** On the Beauty dataset, the first codeword tracks broad categories and later codewords refine them. Items that share a prefix are semantically similar, so

knowledge transfers between related items [4].

- **Cold start.** A new item gets its semantic ID from its content embedding with no interaction history. The authors report better behavior on cold items [4].
- **Collisions.** When two items get the same code tuple, an extra disambiguation token is appended. Decoding can also be restricted to valid prefixes so that every output maps to a real item [6].
- Commonly reported TIGER configuration (approx.): a 768-d Sentence-T5 embedding, 3 RQ levels with 256 codes each, plus the collision token. The open-source reproduction uses smaller toy settings in its quick-start (codebook 16, 3 levels) [6].

3.3 Variations on semantic IDs

Variant	Idea	Where used
RQ-VAE	Learned encoder + residual codebooks, end to end	TIGER [4]; EGA (3 levels x 1024) [24]
RQ-KMeans	Residual k-means on frozen embeddings; decoupled from representation learning; new items assigned by nearest-centroid search	OneRec (3 levels, about 8K codes per level) [20][21]
RQ-KMeans+	Improved init/utilization; codebook utilization reported 65% -> 92%, reconstruction error -15%	Tencent GPR [24]
RQ + OPQ suffix	3 RQ layers (4096/1024/512) plus 2 OPQ codes (2 x 256) to separate near-duplicates	Kuaishou OneSearch [21] [43]
Constrained RQ-KMeans	Balancing and dedup layer for conflicting items	MiniOneRec [15]
Hybrid sparse + dense	Generate SID, then refine with a dense vector	Baidu COBRA [40], Meta LIGER [39]

Known pathologies and practical guidance:

- **The "hourglass" effect.** Intermediate RQ levels can collapse onto a few codes, which limits the ceiling of generative retrieval. A 2024 analysis proposes fixes [9].
- **Codebook collapse and imbalance** cause collisions. Uniform allocation, capacity limits, and k-means-based assignment all reduce it [21].
- Snap's GRID framework and its "Practitioner's Handbook" (CIKM 2025) benchmark RQ-VAE, RVQ and RQ-KMeans tokenizers inside a TIGER-style model on a common pipeline [8]. A companion study looks at how semantic-ID generative recommenders scale with model size [36].
- Google's PLUM (2025) adapts pre-trained LLMs to industrial generative recommendation with semantic IDs [38].

Lessons. The semantic-ID tokenizer is the "vocabulary design" of a generative recommender, and it is a first-class system component. Its collision rate, codebook utilization, refresh cadence, and how well it reflects collaborative signal, not only content, matter as much as the sequence model.

---

4. Kuaishou OneRec family

OneRec is the most complete public example of replacing an entire recommendation cascade with a single generative model in a short-video app serving hundreds of millions of users.

4.1 OneRec (V1) - February 2025 paper and June 2025 technical report

Problem. Remove the retrieve/pre-rank/rank cascade. Generate a session of recommended videos end to end, and align the generator with user preference through RL [10][11].

Tokenizer.

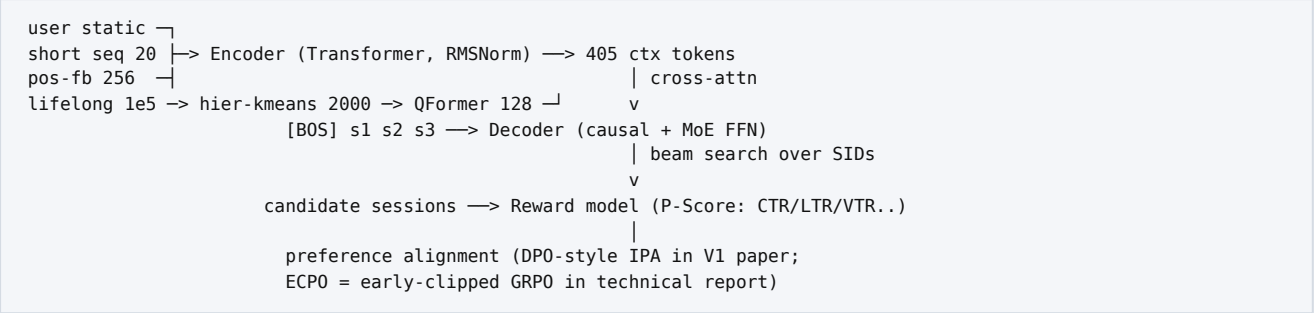
- Inputs (title, tags, ASR, OCR, cover, sampled frames) are compressed by a multimodal LLM (miniCPM-V-8B) to 1280 tokens, then by a QFormer to 4 query vectors.
- An item-pair contrastive loss injects collaborative signal, and title generation is used as an auxiliary task.
- RQ-KMeans then produces 3 code levels [20].

Encoder. Four pathways [20]:

- Static user features, through two dense layers.
- Short-term behavior: the last 20 interactions.
- Positive-feedback behavior: the last 256 high-engagement interactions.
- Lifelong history: up to 100,000 records. These are compressed by hierarchical k-means to 2,000 items, then by a QFormer to 128 tokens.

The encoder context is 1 + 20 + 256 + 128 = 405 tokens.

Decoder. A [BOS] token plus semantic-ID tokens, with causal self-attention, cross-attention to the encoder, and MoE feed-forward layers with top-k routing [20].



Alignment and RL.

- The original paper used Iterative Preference Alignment (IPA). A reward model scores sampled sessions, and chosen/rejected pairs drive DPO-style updates [10].
- The technical report moved to ECPO, an early-clipped variant of GRPO that prevents exploding gradients on negative-advantage samples [20].
- It also added format rewards so the model generates only legal semantic IDs.
- "SIR" industrial alignment down-weights viral content. It is reported to cut viral-content exposure by 9.59% while keeping core metrics stable [20].

Scale (as reported, approx.). The technical report claims:

- Training MFU of about 24% and inference MFU of about 29%.

- Operating cost of about 10% of the previous cascade.
- Deployment on the main Kuaishou and Kuaishou Lite feeds, handling about a quarter of their traffic.
- App stay-time gains of about 0.5-1.2%.

These figures come from the report's abstract and should be verified there [11].

4.2 OneRec-V2 (August 2025)

Problem. In V1 most compute went into the encoder. Only a small fraction trained the generator, and RL from a learned reward model risked reward hacking [20].

Lazy decoder-only architecture.

- A context processor computes key/value pairs once and shares them across decoder layers. There is no per-layer KV projection, and the shared KV uses GQA.
- The decoder input is only [BOS] plus the first two semantic-ID tokens.
- Each layer has lazy cross-attention, causal self-attention, and an FFN, with MoE in the deeper layers [20].

At 1B parameters, the encoder-decoder used 296.36 GFLOPs and reached a converged loss of 3.28. The lazy decoder-only model used 18.89 GFLOPs and reached 3.27. That is about 94% less compute and about 90% less training resource for the same quality [20]. Secondary notes put OneRec-V2 MFU at 20%+, against roughly 5-10% for encoder-decoder setups [22].

Scaling law.

- Dense models from 0.1B to 8B parameters fit $L(N) = E + A / N^{\alpha}$, with $E = 3.13$, $A = 3660$, $\alpha = 0.489$.
- Converged loss falls from 3.57 (0.1B) to 3.19 (8B).
- A 4B-total / 0.5B-active MoE reaches 3.22, better than a 2B dense model [20].

RL from real user feedback.

- Rewards come directly from logged watch time, which is log-bucketed. The top 25% within a bucket get advantage +1, explicit negative feedback gets -1, and the rest get 0.
- GBPO (Gradient-Bounded Policy Optimization) keeps gradients for all samples but bounds the RL gradient by the BCE gradient [20].

Reported online stay-time gains are again in the sub-1% range (approx.) [12].

4.3 OneRec-Think, OpenOneRec, MiniOneRec

- **OneRec-Think (Oct 2025).** Built on Qwen3 with a vocabulary extended by semantic-ID tokens. Training runs in three stages: itemic alignment (mapping item tokens into text space), reasoning activation (chain-of-thought SFT), and reasoning enhancement with a reward that allows multiple valid answers. It reports a further +0.159% app stay time online [13].
- **OpenOneRec (Dec 2025).** Kuaishou's open release.

    - Model checkpoints: OneRec-1.7B and 8B, trained on about 33B tokens, and "-Pro" variants trained on about 130B tokens, including industrial data. All use Qwen3 backbones.
    - Items are represented as three-level "itemic tokens".
    - Post-training pipeline: multi-task SFT, then on-policy distillation to restore general reasoning, then RL.
    - RecIF-Bench: 8 tasks over 100M interactions from 200K users in short video, ads, and products [14].
- **MiniOneRec (Oct 2025).** A community open-source reproduction. It builds text-embedding semantic IDs (RQ-VAE / RQ-KMeans / constrained RQ-KMeans) and SFT co-trained with language alignment. It then applies GRPO with a rank-aware reward and a KL penalty, and decodes with constrained beam search through a logits processor [15].

Kuaishou extended the same recipe to e-commerce search (OneSearch), query suggestion (OneSug), live streaming (OneLive), and local services (OneLoc) [43][44].

Lessons.

- (1) In a generative recommender the encoder is mostly a context compressor, and compute belongs in the generating decoder. This is V2's main lesson.
- (2) RL post-training needs guardrails: format rewards, clipping and bounding of policy gradients, and anti-virality shaping.
- (3) Lifelong histories of about 10^5 events are compressed (clustering, then QFormer) instead of being attended to directly.

---

5. Meituan MTGR (and MTGRBoost, DFGR, EGA)

5.1 Problem

Meituan's food-delivery ranking depends heavily on cross features: user x item statistics such as the user's historical CTR on a category, or their preference by hour of day. Pure HSTU-style generative ranking drops these features. Meituan found that this hurt so much that even the largest generative model without cross features performed worse than a mid-sized DLRM [19].

5.2 Architecture

MTGR is a hybrid: a transformer encoder with HSTU-style user-level sequence compression, trained with a discriminative (CTR/CTCVR) objective [16][19].

- **Token groups:**
    - User tokens: static attributes, one token per attribute.
    - Sequence tokens: long-term history, one per item.
    - Real-time tokens: recent events.
    - Candidate tokens: MLP(concat(item embedding, cross-feature embedding)), plus position and exposure time.

- **Cross features stay on candidate tokens** and never enter the history, which avoids causal leakage.
- **Group LayerNorm (GLN):** separate normalization statistics and affine parameters per token group.
- **Dynamic masking:** static tokens are visible to all. Real-time tokens are causal, and a candidate can see them only if their timestamp is before that candidate's exposure time. Candidates cannot see each other (a diagonal block), so all candidates for a user are scored in one pass.
- **Loss** is computed only at candidate positions. Encoding the history once and scoring m candidates in parallel costs O((n+m)^2), against O(m·n^2) when candidates are scored separately [19].

```
[U1..Uk | S1 ... Sn | R1 ... Rr | C1  C2  ...  Cm]
  static   long seq   real-time   candidates (item ⊕ cross feats)
mask: U,S -> visible to all; R causal & time-gated per candidate; Ci ⊥ Cj
heads: CTR / CTCVR on C positions only
```

Scale (as reported, approx.). The paper reports per-sample inference compute about 65x that of the DLRM baseline. It describes this as the largest offline and online gain in about two years, deployed on Meituan's delivery recommendation [16]. Training was supported by MTGRBoost, a companion system paper on large-scale generative-model training infrastructure [17].

Related Meituan work:

- **DFGR** ("Action is All You Need: Dual-Flow Generative Ranking Network") restructures action and item flows.
- **EGA-V1/V2** build fully generative advertising. EGA uses RQ-VAE semantic IDs (3 levels x 1024 codes) and decomposes generation into P(POI) x P(creative | POI). It puts auction mechanics into the model with token-level bids and a payment network that guarantees individual rationality (payment never exceeds bid). Incentive compatibility is enforced approximately through a Lagrangian on ex-post regret [24] [42].

Lessons. Generative architecture and discriminative objectives can coexist. Keep the scaling benefits of user-level sequence encoding and parallel candidate scoring, and keep the cross features that a pure next-token model throws away.

---

6. Other generative ranking systems worth knowing

- **Xiaohongshu GenRank ("Towards Large-scale Generative Ranking", 2025).**
  - Trained on hundreds of billions of exposure logs.
  - An action-oriented sequence makes actions the main tokens and folds items in as attributes. This halves the sequence (2n_c -> n_c), cuts attention compute by 75%, and speeds up training by 78.7%.
  - ALiBi-style parameter-free position and time biases raise the total speedup to 94.8%.

    - Loss at history positions hurt AUC, and so did removing the causal mask. Real-time features stayed important while most historical aggregate features lost value [22][32].
- **ByteDance RankMixer (2025).**
    - Not a generative model, but part of the same scaling trend.
    - Self-attention is replaced by parameter-free token mixing, plus per-token FFNs and a dense-train / sparse-infer MoE.
    - Raises ranking MFU from about 4-5% to about 45% [23][33].
    - Successors: OneTrans and TokenMixer-Large [44].
- **Tencent GPR (2025, WeChat Channels ads).**
    - Four token types (user, organic, environment, ad), RQ-KMeans+ semantic IDs, and a heterogeneous hierarchical decoder with "thinking tokens".
    - Value-guided trie beam search.
    - Reported results: latency from 200ms+ to about 80ms, and 5 models collapsed into 1 [24][41].
- **Alibaba LUM (2025).** A three-step paradigm: pre-train a large user model generatively, query it, then use its outputs in discriminative models. It aims to unlock scaling laws in industrial recommendation [37].

---

7. Netflix: Foundation Model for Personalized Recommendation

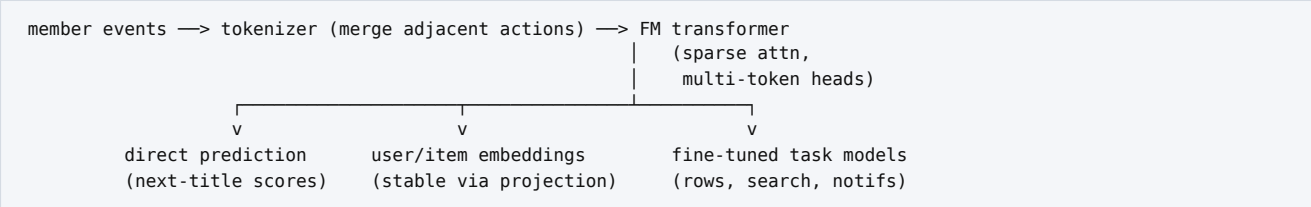
7.1 Problem

Netflix ran many specialized models (rows, ranking, search, notifications). Each was trained on overlapping data with its own feature engineering, which made improvements hard to share. In a March 2025 tech-blog post, Netflix described consolidating user-preference learning into one large, centrally trained foundation model in the spirit of LLMs. Downstream systems consume it [25].

7.2 Design elements (as described in the blog; summarized via a public reimplementation's mapping)

- **Interaction tokenization.** User interaction events are tokenized, and adjacent related actions are merged, much like BPE in language models. Sequences stay informative but shorter [25][26].
- **Long-context handling.** Sparse or local attention, plus sliding-window sampling over long histories during training [25][26].
- **Multi-token / multi-attribute prediction.** Heads predict several future items and attributes such as genre, not only the next item ID. This reduces myopia [25][26].
- **Cold start.** Metadata-based embeddings are mixed with learned ID embeddings, with a gate weighted by item "age". New titles lean on metadata, and the model is incrementally retrained to add new IDs [25][26].
- **Embedding stability.** A projection, described as an orthogonal transformation, keeps embedding spaces comparable across retrains, so downstream consumers do not break [25][26].

- **Downstream use.** Three modes: (a) the model directly as a predictive model, (b) exported user and item embeddings, and (c) fine-tuning for specific surfaces [25].



Netflix reported that performance kept improving as it scaled the model and data, consistent with scaling-law behavior (approx.; see [25]).

Lessons. For companies with many surfaces, the main gain is organizational: one representation learner, with stable embedding contracts, feeding many consumers. The engineering work is concentrated in tokenization, cold-start mixing, and keeping embeddings stable across retrains.

---

8. LinkedIn 360Brew

360Brew ("A Decoder-only Foundation Model for Personalized Ranking and Recommendation", 2025) takes the most radical "LLM-as-recommender" route of the systems here [27][28].

- **Problem.** Replace many task-specific ranking models and their hand-engineered features with one model that reasons over textual member profiles and histories.
- **Interface.** Each task is written as a natural-language prompt. For example: given the member's profile and past job interactions (applied, viewed, dismissed), predict whether they will apply to, view, or dismiss a given job. No manual feature engineering is needed [28].
- **Scaling observations.** Performance improved with (a) more training tokens across five releases over nine months, (b) longer context, meaning more interaction history, and (c) larger model size [28].
- **Scale (approx.).** The paper abstract describes a decoder-only model of about 150B parameters covering 30+ ranking tasks across LinkedIn surfaces, matching or beating production models without task-specific fine-tuning. Verify the figures in [27].

Lessons. Text-native foundation rankers trade very high serving cost for much less feature and model sprawl, plus zero-shot transfer to new surfaces. In practice they are often distilled or used for some surfaces rather than as the single high-QPS scorer.

---

9. Pinterest PinFM (and PinRec, UniPinRec)

PinFM ("Foundation Model for User Activity Sequences at a Billion-scale Visual Discovery Platform", 2025) pre-trains a large transformer on user activity sequences and then fine-tunes it inside multiple downstream ranking models [29].

- **Pre-train / fine-tune split.** Large-scale pre-training on lifelong activity sequences. The module is then fine-tuned inside each downstream ranker, for example home-feed or related-pins ranking, and not served as a separate system [29].
- **Serving efficiency (approx.).** The abstract describes a deduplicated cross-attention design (DCAT). It computes the user-sequence part once per request and shares it across all candidates, and is reported to improve throughput substantially (approx. 600%). It also reports gains in engagement with new items. Verify the figures in [29].
- **Model size (approx.).** The parameter count is reported in the tens of billions (approx. 20B+) [29].
- **Related work.** PinRec (2025) is an outcome-conditioned, multi-token generative retrieval model [30]. UniPinRec (2026) unifies generative retrieval and ranking at Pinterest scale [46].

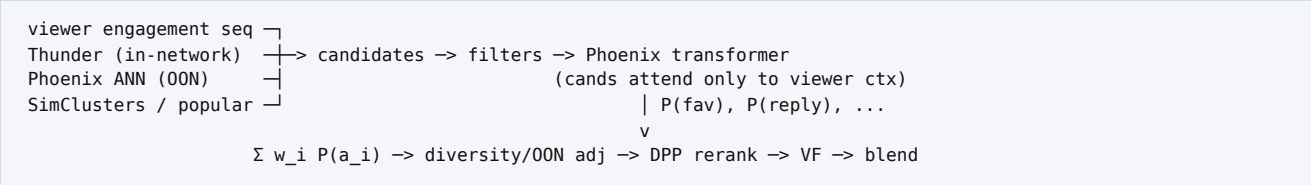
Lessons. The "user-sequence foundation model plus downstream fine-tune" pattern deploys incrementally. It slots into an existing ranker as a learned feature extractor. Dedup and KV-sharing tricks like DCAT (similar to M-FALCON and MTGR's masking) are what make it affordable.

---

10. X (Twitter) For You: Phoenix transformer ranker

X's open-sourced `x-algorithm` repository describes the 2025-2026 For You pipeline [31]. (Public commentary often calls Phoenix "Grok-based". The repository itself says only that it is a transformer. "Grox" is a separate set of LLM content classifiers [31].)

- **Home-mixer pipeline:** query hydration, then candidate sources, then hydration, then filters, then scoring, then selection, then post-selection visibility filtering, then blending.
- **Candidate sources:**
  - Thunder: an in-memory store of recent posts from followed accounts.
  - Phoenix retrieval: embedding nearest-neighbor search for out-of-network posts.
  - SimClusters.
  - Popular posts.
- **Phoenix ranker.** Its main input is the viewer's recent engagement sequence. It predicts per-action probabilities for engagement (favorite, reply, repost, quote, share), clicks, attention (dwell, video view), follow, and negative actions (not interested, mute, block, report). Training and serving are written in JAX with a Rust serving layer [31].
- **Candidate isolation.** "During transformer inference, candidates cannot attend to each other - only to the viewer context." Each post's score is therefore independent of how candidates are batched. This is the same masking idea as M-FALCON and MTGR [31].
- **Hash embeddings.** Retrieval and ranking both use multiple hash functions for embedding lookup, so there is no ID vocabulary to maintain [31].
- **Value model.** Final score = Σ w_i · P(action_i). It is followed by an author-diversity decay, an out-of-network discount, a new-author boost, and a DPP-based re-ranker (VMRanker) that trades score for diversity [31].



Lessons. A transformer over the engagement sequence can replace a hand-built feature ranker without becoming end-to-end generative. Multi-action prediction combined with an explicit, tunable value model keeps product control (weights, negative signals) outside the network.

---

11. Scaling laws for recommendation

Work	Model family	Scaling claim (as reported)
HSTU (Meta, 2024) [1][18]	Sequential transducer	Power law in compute over 3 orders of magnitude; HR@100 and NE linear in ln(compute)
Wukong (Meta, 2024) [34]	Stacked factorization machines (DLRM-style)	Scaling law for dense interaction stacks across two orders of magnitude in complexity (approx.)
OneRec-V2 (Kuaishou, 2025) [20]	Lazy decoder-only + MoE	L(N) = 3.13 + 3660 / N^0.489, 0.1B-8B
360Brew (LinkedIn, 2025) [28]	Text decoder LLM	Gains with tokens, context length, model size
LUM (Alibaba, 2025) [37]	Generative pre-trained user model	Scaling unlocked by three-step generative -> discriminative paradigm
SID scaling study (Snap, 2025) [36]	Semantic-ID GR	Studies model-size scaling limits of SID-based GR
ULTRA-HSTU (Meta, 2026) [35]	HSTU successor	"Bending" the scaling curve via architecture/efficiency

Common findings:

- (a) Gains are roughly logarithmic in compute, so each 10x buys a fixed increment. For HSTU this is about 4.5 points of HR@100 and about 1.2 points of NE per 10x, per secondary analysis [18].
- (b) Sequence length is a scaling axis on equal footing with depth and width.
- (c) MoE gives better loss per active FLOP [20].
- (d) Embedding-table size alone does not scale. Dense compute does.

---

12. Comparison table

System	Org / year	Paradigm	Item representation	Stage replaced	Notable scale / result (as reported)
HSTU / GR	Meta 2024	Causal sequential transducer	Atomic IDs + actions interleaved	Retrieval & ranking models	1.5T params; +12.4% online; 5.3-15.2x faster attention [1]
TIGER	Google 2023	Seq2seq generative retrieval	Semantic IDs (RQ-VAE)	Retrieval	SOTA on Amazon datasets; cold-start gains [4]

System	Org / year	Paradigm	Item representation	Stage replaced	Notable scale / result (as reported)
OneRec V1	Kuaishou 2025	Enc-dec + MoE + RL (IPA/ECPO)	3-level RQ-KMeans SIDs	Whole cascade	MFU about 24%/29%; OPEX about 10% (approx.) [11]
OneRec-V2	Kuaishou 2025	Lazy decoder-only + RL from real feedback	SIDs	Whole cascade	94% less compute at equal loss; 8B scaling [20]
MTGR	Meituan 2025	Hybrid generative-encoder, discriminative loss	Atomic IDs + cross features	Ranking	about 65x FLOPs/sample vs DLRM (approx.) [16]
GenRank	Xiaohongshu 2025	Action-oriented generative ranking	Actions as tokens	Ranking	94.8% training speedup [22]
GPR	Tencent 2025	Decoder + value-guided trie search	RQ-KMeans+ SIDs	Ads cascade	5 models -> 1; latency to about 80ms [24]
Netflix FM	Netflix 2025	Central foundation model	Merged interaction tokens + metadata	Feeds many models	Scaling with data/params (approx.) [25]
360Brew	LinkedIn 2025	Text decoder LLM ranker	Natural-language prompts	Many task rankers	about 150B params, 30+ tasks (approx.) [27]
PinFM	Pinterest 2025	Pre-trained user-seq FM, fine-tuned downstream	IDs + content	Feature extractor in rankers	about 20B+ params; DCAT about 600% throughput (approx.) [29]
Phoenix	X 2025-26	Transformer ranker + ANN retrieval	Hash embeddings	Main ranker	Candidate-isolated attention; multi-action heads [31]

Key takeaways for system designers

1. **Restructure training before you scale.** Moving from impression-level samples to one causal sequence per user (HSTU, GenRank, MTGR) gives order-of-magnitude efficiency gains before any parameters are added.
2. **Candidate-isolated attention is the universal serving trick.** M-FALCON (Meta), dynamic masking (Meituan), DCAT (Pinterest) and Phoenix (X) all encode the user context once, share its KV, and let candidates attend only to the context. This makes target-aware transformer ranking linear in the number of candidates.
3. **Treat the tokenizer as infrastructure.** Semantic IDs need a pipeline: embedding model, RQ-VAE or RQ-KMeans, collision handling, valid-prefix tries for constrained decoding, and a refresh policy. Monitor codebook utilization, collision rate, and the "hourglass" effect [9][21].
4. **Keep the features that carry signal.** MTGR shows that dropping cross features to be "purely generative" can lose more than scaling gains back. Hybrid designs are legitimate.
5. **Put compute in the generator.** OneRec-V2's lazy decoder-only design matched encoder-decoder quality with about 94% less compute. Compress long context (clustering, QFormer) instead of attending over 10^5 raw events.
6. **RL post-training needs stabilization.** Use format rewards, clipped or bounded policy gradients (ECPO, GBPO), rewards grounded in real user feedback, and anti-virality or diversity shaping. Learned reward models alone invite reward hacking.
7. **MFU is the new north-star infrastructure metric.** Going from about 5% (DLRM) to 20-45% (OneRec, RankMixer) changes what model sizes are affordable.

8. **Foundation models win organizationally.** Netflix, PinFM and 360Brew show that one user-sequence model feeding many consumers, with stable embeddings, reduces duplicated feature work. Plan for embedding-versioning contracts.
9. **Keep business control explicit.** Even end-to-end systems keep value models, auction or payment modules, diversity re-rankers and visibility filters outside or on top of the generator (X value model, EGA payment network).
10. **Expect logarithmic returns.** Budget for about 10x compute per fixed metric increment, and use MoE and sequence-length scaling to stretch the curve.

References

1. Zhai, J. et al. "Actions Speak Louder than Words: Trillion-Parameter Sequential Transducers for Generative Recommendations." Meta, ICML 2024. https://arxiv.org/abs/2402.17152
2. meta-recsys. "generative-recommenders" (HSTU, M-FALCON, DLRM-v3, HSTU Ultra code). Meta, 2024-2026. https://github.com/meta-recsys/generative-recommenders
3. Zhai, J. et al. ICML 2024 slides, "Actions Speak Louder than Words." Meta, 2024. https://icml.cc/media/icml-2024/Slides/32684.pdf
4. Rajput, S. et al. "Recommender Systems with Generative Retrieval" (TIGER). Google, NeurIPS 2023. https://arxiv.org/abs/2305.05065
5. Rajput, S. et al. TIGER, NeurIPS proceedings PDF. Google, 2023. https://proceedings.neurips.cc/paper_files/paper/2023/file/20dcab0f14046a5c6b02b61da9f13229-Paper-Conference.pdf
6. Botta, E. "RQ-VAE-Recommender" (PyTorch implementation of generative retrieval with semantic IDs). 2024-2025. https://github.com/EdoardoBotta/RQ-VAE-Recommender
7. Lee, D. et al. "Autoregressive Image Generation using Residual Quantization" (RQ-VAE). POSTECH/Kakao, 2022. https://arxiv.org/abs/2203.01941
8. Ju, C. et al. "Generative Recommendation with Semantic IDs: A Practitioner's Handbook" (GRID). Snap, CIKM 2025. https://arxiv.org/abs/2507.22224 ; code: https://github.com/snap-research/GRID
9. "The Hourglass phenomenon in residual-quantized semantic IDs for generative search" (arXiv 2407.21488). 2024. https://arxiv.org/html/2407.21488v1
10. Deng, J. et al. "OneRec: Unifying Retrieve and Rank with Generative Recommender and Iterative Preference Alignment." Kuaishou, 2025. https://arxiv.org/abs/2502.18965
11. Kuaishou OneRec Team. "OneRec Technical Report." Kuaishou, 2025. https://arxiv.org/abs/2506.13695
12. Kuaishou OneRec Team. "OneRec-V2 Technical Report." Kuaishou, 2025. https://arxiv.org/abs/2508.20900
13. Kuaishou OneRec Team. "OneRec-Think: In-Text Reasoning for Generative Recommendation." Kuaishou, 2025. https://arxiv.org/abs/2510.11639 ; code: https://github.com/wangshy31/OneRec-Think
14. Kuaishou OneRec Team. "OpenOneRec Technical Report" and repository. Kuaishou, 2025. https://arxiv.org/abs/2512.24762 ; https://github.com/Kuaishou-OneRec/OpenOneRec

15. Kong et al. "MiniOneRec" (open-source generative recommendation framework). 2025. https://arxiv.org/abs/2510.24431 ; https://github.com/AkaliKong/MiniOneRec
16. Han, R. et al. "MTGR: Industrial-Scale Generative Recommendation Framework in Meituan." Meituan, 2025. https://arxiv.org/abs/2505.18654
17. "MTGRBoost: Boosting Large-scale Generative Recommendation Models in Meituan." Meituan, 2025. https://arxiv.org/abs/2505.12663
18. Xing, H. "HSTU: The Cornerstone of Scaling Laws" (fun-rec-mdbook, technical notes). 2026. https://github.com/Haozhe-Xing/fun-rec-mdbook/blob/main/src/en/part7-scaling/hstu.md
19. Xing, H. "MTGR: A Hybrid Paradigm" (fun-rec-mdbook). 2026. https://github.com/Haozhe-Xing/fun-rec-mdbook/blob/main/src/en/part7-scaling/mtgr.md
20. Xing, H. "End-to-End Generative Recommendation" (OneRec / OneRec-V2 notes, fun-rec-mdbook). 2026. https://github.com/Haozhe-Xing/fun-rec-mdbook/blob/main/src/en/part8-e2e/e2e-recommendation.md
21. Xing, H. "Codebook Quantization & Semantic IDs" (fun-rec-mdbook). 2026. https://github.com/Haozhe-Xing/fun-rec-mdbook/blob/main/src/en/part6-gr-basic/codebook-quantization.md
22. Xing, H. "The Generative Ranking Paradigm" and "Architectural Foundations of Generative Recommendation" (fun-rec-mdbook). 2026. https://github.com/Haozhe-Xing/fun-rec-mdbook/blob/main/src/en/part7-scaling/generative-ranking.md ; https://github.com/Haozhe-Xing/fun-rec-mdbook/blob/main/src/en/part6-gr-basic/gr-architecture.md
23. Xing, H. "RankMixer: A Hardware-Efficient Architecture" (fun-rec-mdbook). 2026. https://github.com/Haozhe-Xing/fun-rec-mdbook/blob/main/src/en/part7-scaling/rankmixer.md
24. Xing, H. "End-to-End Generative Advertising" (EGA, GPR notes, fun-rec-mdbook). 2026. https://github.com/Haozhe-Xing/fun-rec-mdbook/blob/main/src/en/part8-e2e/e2e-advertising.md
25. Hsiao, K.-J., Feng, Y., Lamkhede, S. "Foundation Model for Personalized Recommendation." Netflix Technology Blog, 2025. https://netflixtechblog.com/foundation-model-for-personalized-recommendation-1a0bd8e02d39
26. Venkatesan, M. "Foundation Model for Personalized Recommendation (H&M Dataset)" (open reimplementation mapping the Netflix design). 2026. https://github.com/madanmohan-venkatesan/Recommender-systems-on-HNM-fashion-dataset
27. Firooz, H. et al. "360Brew: A Decoder-only Foundation Model for Personalized Ranking and Recommendation." LinkedIn, 2025. https://arxiv.org/abs/2501.16450
28. LLMPersonalization. "360Brew-V1" repository README. LinkedIn, 2025. https://github.com/LLMPersonalization/360Brew-V1
29. "PinFM: Foundation Model for User Activity Sequences at a Billion-scale Visual Discovery Platform." Pinterest, 2025. https://arxiv.org/abs/2507.12704
30. "PinRec: Outcome-Conditioned, Multi-Token Generative Retrieval for Industry-Scale Recommendation Systems." Pinterest, 2025. https://arxiv.org/abs/2504.10507
31. xAI / X. "x-algorithm" (For You feed: home-mixer, Thunder, Phoenix). 2025-2026. https://github.com/xai-org/x-algorithm
32. "Towards Large-scale Generative Ranking" (GenRank). Xiaohongshu, 2025. https://arxiv.org/abs/2505.04180
33. "RankMixer: Scaling Up Ranking Models in Industrial Recommenders." ByteDance, 2025. https://arxiv.org/abs/2507.15551

34. Zhang, B. et al. "Wukong: Towards a Scaling Law for Large-Scale Recommendation." Meta, ICML 2024. https://arxiv.org/abs/2403.02545
35. "Bending the Scaling Law Curve in Large-Scale Recommendation Systems" (ULTRA-HSTU). Meta, 2026. https://arxiv.org/abs/2602.16986
36. "Understanding Generative Recommendation with Semantic IDs from a Model-scaling View." Snap, 2025. https://arxiv.org/abs/2509.25522
37. "Unlocking Scaling Law in Industrial Recommendation Systems with a Three-step Paradigm based Large User Model" (LUM). Alibaba, 2025. https://arxiv.org/abs/2502.08309
38. "PLUM: Adapting Pre-trained Language Models for Industrial-scale Generative Recommendations." Google/YouTube, 2025. https://arxiv.org/abs/2510.07784
39. "Unifying Generative and Dense Retrieval for Sequential Recommendation" (LIGER). Meta, 2024. https://arxiv.org/abs/2411.18814
40. "Sparse Meets Dense: Unified Generative Recommendations with Cascaded Sparse-Dense Representations" (COBRA). Baidu, 2025. https://arxiv.org/abs/2503.02453
41. "GPR: Towards a Generative Pre-trained One-Model Paradigm for Large-Scale Advertising Recommendation." Tencent, 2025. https://arxiv.org/abs/2511.10138
42. "EGA-V2: An End-to-end Generative Framework for Industrial Advertising." Meituan, 2025. https://arxiv.org/abs/2505.17549
43. "OneSearch: A Preliminary Exploration of the Unified End-to-End Generative Framework for E-commerce Search." Kuaishou, 2025. https://arxiv.org/abs/2509.03236
44. Huang, Y. et al. "Awesome-Generative-Recommendation" (curated industrial paper list). 2025-2026. https://github.com/uestc-huangyw/Awesome-Generative-Recommendation
45. "How Well Does Generative Recommendation Generalize?" Meta, 2026. https://arxiv.org/abs/2603.19809
46. "UniPinRec: Unifying Generative Retrieval and Ranking at Pinterest Scale." Pinterest, 2026. https://arxiv.org/abs/2606.00422

07. Graph-Based Recommendation in Production: Random Walks, Graph Embeddings and GNNs at Web Scale

Graphs are the oldest structure in recommendation. "People who bought X also bought Y" is a two-hop walk on a user-item bipartite graph. "People you may know" is a triangle-closing walk on a social graph. Most of the large consumer platforms (Pinterest, Twitter/X, LinkedIn, Alibaba, Uber, Meta) have published at least one production system that treats recommendation as a graph problem. This chapter collects those designs and groups them into four families. Each family makes a different trade-off between freshness, expressiveness and serving cost:

1. **Online random-walk serving.** An in-memory graph answers each query with a short random walk or a SALSA-style traversal, all at request time. Examples: Pinterest Pixie, Twitter GraphJet (UTEG/UTG/UUG/UVG), Twitter's WTF/SALSA follow recommendations.
2. **Community or sparse factorisation representations.** Offline graph clustering produces sparse, interpretable vectors that streaming jobs keep fresh. Example: Twitter SimClusters and its SimClusters-ANN serving layer.
3. **Shallow (transductive) graph embeddings.** Large-scale node-embedding training (DeepWalk, TransE-style knowledge-graph embeddings) produces dense vectors for ANN retrieval and for use as ranking features. Examples: Alibaba EGES, Twitter TwHIN, Meta PyTorch-BigGraph.
4. **Inductive GNNs.** GraphSAGE-style aggregators compute embeddings from features plus sampled neighbourhoods, usually offline in batch and sometimes nearline. Examples: Pinterest PinSage (and PinnerSage built on it), Uber Eats GraphSAGE, LinkedIn LiGNN, Alibaba AliGraph/Graph-Learn and its Dynamic Graph Service.

A note on sourcing. Facts about Twitter/X systems (SimClusters, GraphJet services, TwHIN training code, Recos-Injector, the SALSA and two-hop candidate sources) and about Alibaba Graph-Learn/DGS were read directly from the open-source repositories and their documentation [1][2][3][4][5][6][7]. Numbers from the academic papers (Pixie, PinSage, PinnerSage, EGES, TwHIN, LiGNN, Uber Eats) are given as reported by their authors and should be treated as approximate. Check them against the cited paper before you quote them.

---

1. A taxonomy of graph recommenders in production

```
                  where is the graph computation done?
                request time                  offline / nearline
             +-----------------------+    +-----------------------------+
no learned   | Pixie, GraphJet,      |    | SimClusters (community      |
params       | SALSA/WTF, 2-hop walk |    | detection + sparse vectors), |
             | (walks over in-memory |    | Tweepcred PageRank,         |
             |  graph)               |    | RealGraph aggregates        |
             +-----------------------+    +-----------------------------+
learned      | AliGraph DGS (sample  |    | PinSage, LiGNN, Uber Eats   |
params       |  subgraph online, GNN |    | GraphSAGE (inductive GNN);  |
             |  forward in TF-Serve) |    | EGES, TwHIN, PBG (shallow   |
             |                       |    | embeddings) -> ANN index    |
             +-----------------------+    +-----------------------------+
```

Request-time walks give the freshest results, at the cost of a large RAM-resident graph per replica. Offline embeddings are cheap to serve because retrieval becomes ANN lookup, but they are stale between refreshes, and transductive models do not cover new nodes. Inductive GNNs plus nearline inference are the current compromise. Every large platform in this chapter runs more than one family at once, usually as several candidate sources feeding one ranker.

---

2. Pinterest Pixie: real-time random walks on a 3-billion-node graph

Problem

Pinterest needed to recommend pins (images) in real time for Homefeed, Related Pins and other surfaces. The recommendations had to be personalised to a user's recent actions, which are a handful of "query pins". The earlier pipeline was Hadoop-based and precomputed results offline, so it could not react within a session [8].

Architecture

Pixie keeps the whole pin-board bipartite graph in RAM on each server. Pins are saved into boards, so every save is an edge. The paper reports a graph of roughly 3 billion nodes and 17 billion edges, compacted to fit in the memory of a single large machine. Each server is a full replica, which removes any cross-machine communication from the walk [8].

```
user actions (saves, clicks)      offline graph build (Hadoop, daily)
        |                                  |
        v                                  v
 query pins Q={q1..qk}, weights w_q    pruned pin<->board graph
        |                             (CSR arrays, loaded into RAM)
        v                                  |
 +-------------------------- Pixie server --+-----------------+
 | for each q in Q: budget N_q ~ w_q * f(degree(q))            |
 |   repeat: walk q -> board -> pin -> board ... (biased       |
 |           by user language/topic), restart w.p. alpha       |
 |   count visits V_q[p]; early-stop when n_p pins have >= n_v |
 | combine: V[p] = (sum_q sqrt(V_q[p]))^2   (multi-hit boost)  |
 +-------------------------------------------+-----------------+
                                             v
                          top-K pins -> ranking model -> surface
```

Algorithm and key tricks

- **Random walk with restarts on the bipartite graph.** Each step goes pin to board to pin. Visit counts approximate personalised PageRank from the query pin [8].
- **Biased walks.** Edge selection is biased towards boards and pins that match user features such as language or topic. This personalises the walk without storing per-user graphs [8].
- **Multiple weighted query pins.** Each query pin gets its own step budget. The budget scales with the pin's weight (recency, action type) and is adjusted by the pin's degree, so popular pins do not take over the walk. Visit counts are then combined with a sum of square roots, squared, which rewards candidates that several query pins reach over candidates that one query pin reaches many times [8].
- **Early stopping.** A walk stops once a target number of pins have each been visited at least a minimum number of times. This cuts latency substantially with little loss in quality [8].
- **Graph pruning.** Boards that are too diverse, which the paper measures as high topic entropy over their pins, are removed. Very popular pins have their edges pruned down to the most relevant boards. The paper reports that pruning made the graph smaller and also improved recommendation quality, so it was not only a memory-saving measure [8].

Scale and results (as reported, approximate)

The paper reports that Pixie served on the order of a billion-plus users' worth of requests with per-query latencies in the tens of milliseconds at high percentiles. It also reports large engagement lifts over the previous Hadoop-based system, on the order of +50% in some surfaces [8]. Treat the exact figures as approximate.

Lessons

- If the graph fits in RAM on one machine, **replicate it rather than partition it**. Walks are dominated by pointer chasing, and network hops would ruin them.
- Pruning the graph is a modelling decision as much as a memory decision. Removing noisy hub nodes improves walk quality.
- Random walks give a candidate *distribution*. A learned ranker still has to sit on top.

---

3. Pinterest PinSage: GraphSAGE at 3 billion nodes

Problem

Pixie needs graph connectivity at query time and cannot use content features. New pins with few saves get poor recommendations. Pinterest wanted one embedding per pin that combined visual and text features with graph structure. The embedding had to be computed for billions of items and served through nearest-neighbour search [9][10].

Architecture

PinSage is a GraphSAGE-family inductive GCN [11] adapted to web scale. The paper reports a graph of about 3 billion nodes (pins and boards) and about 18 billion edges,

trained on about 7.5 billion examples [9].

```
          TRAINING (GPU, producer-consumer)
CPU producers: sample minibatch of (query, positive) pin pairs
               -> random-walk neighbourhoods (top-T by visit count)
               -> gather features (visual, text, ...)  ---+
                                                          v
GPU consumer:  K=2 conv layers; importance pooling (weights =
               normalised visit counts); max-margin loss with
               random + curriculum "hard" negatives
                                                          |
          INFERENCE (MapReduce, no repeated computation)  v
layer-1 embeddings for all nodes -> join -> layer-2 embeddings
                -> pin embeddings -> ANN index (related pins, homefeed)
```

Key tricks

1. **Random-walk-defined neighbourhoods ("importance pooling").** PinSage does not use the raw adjacency. It runs short random walks from each node and keeps the top-T nodes by visit count as the node's neighbourhood. The normalised counts become aggregation weights. This bounds fan-out (degree can be huge on Pinterest) and makes the neighbourhood relevance-weighted [9].
2. **Producer-consumer minibatching.** CPUs sample neighbourhoods and fetch features while GPUs run the convolutions, so the GPUs stay busy [9].
3. **Curriculum training with hard negatives.** Random negatives are too easy among billions of items. Later epochs add "hard" negatives: items ranked moderately high, but not top, by personalised PageRank from the query. The paper reports this was important for fine-grained relevance [9].
4. **MapReduce inference.** If the GCN ran per node with its sampled neighbourhood, overlapping neighbourhoods would be recomputed many times. PinSage instead computes each layer once for all nodes and joins the results, which removes the redundant work [9].
5. **Serving.** Embeddings go into an approximate nearest-neighbour index for candidate generation [9][10].

Results (as reported, approximate)

In offline evaluation the paper reports large gains in hit-rate and MRR over content-only and earlier graph baselines. User studies and A/B tests showed engagement improvements for related-pin and homefeed recommendations [9][10].

Descendant: PinnerSage (multi-modal user embeddings)

A single user vector averages over unrelated interests. PinnerSage (KDD 2020) represents each user as **several** PinSage-space embeddings. It clusters the user's recent action pins, the paper describes roughly 90 days of history, using Ward hierarchical clustering. Each cluster is represented by its **medoid**, an actual pin, not a centroid. Medoids are weighted by time-decayed frequency and a few are sampled per request for ANN retrieval [12]. Because PinSage embeddings stay fixed, PinnerSage only has to cluster the user's own actions. It never retrains the item model. Pinterest later moved part of this role to PinnerFormer, a transformer over action sequences [13]. That system is covered in the sequential-models chapter.

Lessons

- Use random walks for *neighbourhood selection*, not only for retrieval. This gives bounded fan-out and relevance weights in one step.
- Batch inference must avoid redundant computation. Layer-wise MapReduce costs O(layers x nodes) instead of O(nodes x fanout^layers).
- Item embeddings that combine content and graph give a stable "coordinate system" that many downstream systems can reuse (PinnerSage, ads, ranking features).

---

4. Twitter/X: a full graph recommendation stack

Twitter has published the most graph-based recommendation infrastructure of any company. Most of it can be read in the open-sourced `the-algorithm` repository and its launch blog post [1][23]. That repository says candidate sourcing narrows about 1 billion posts to a few thousand per request [2], and in-network search supplies about 50% of For You posts [1]. Out-of-network candidates come largely from graph systems: GraphJet services, SimClusters, TwHIN and the Follow Recommendations Service.

```
unified-user-actions / event bus (fav, RT, follow, reply, click ...)
       |                |                          |
       v                v                          v
 Recos-Injector     SimClusters Heron jobs     batch (Scalding/GCP)
 (filter, publish   (tweet embeddings updated   KnownFor, InterestedIn,
  per-service Kafka  on every fav)              producer/topic embeds,
  topics)               |                       TwHIN, RealGraph,
       |                v                       Tweepcred PageRank
       v           SimClusters-ANN                    |
 GraphJet services  (cluster->top tweets index)       v
 UTEG UTG UUG UVG        |                  Representation Manager
       |                 |                  / Representation Scorer
       +--------+--------+-------------------------+
                v
         CR-Mixer / tweet-mixer (fetch, mix, filter, light rank)
                v
         Home Mixer -> heavy ranker -> For You timeline
```

4.1 GraphJet: in-memory, real-time bipartite graph engine

Problem. Tweets are short-lived. Recommendations such as "X liked this tweet" have to reflect engagement from the past hours, not yesterday's batch.

Design. GraphJet is a Java library that "maintains a full graph index over a sliding time window in memory on a single server" [3]. Edges are ingested from streams (Kafka), and lookups are served concurrently. It uses compact edge encoding and dynamic memory allocation: the graph is held as a series of time-ordered segments, and the oldest segment is dropped as the window slides. Each server can ingest up to **one million edges per second** and, in steady state, compute up to **500 recommendations per second**, which amounts to several million edge reads per second [3][14].

Production services built on GraphJet (from the repo READMEs [1]):

Service	Graph	Window	Use
UTEG (UserTweetEntityGraph)	user -> tweet engagements (one-directional)	24-48 h	"X liked" out-of-network tweets on Home; input is a weighted follow list
UTG (UserTweetGraph)	user <-> tweet (bi-directional)	24-48 h	1-, 2-, 3+-hop CF and random walks from seed users/tweets; more memory than UTEG
UUG (UserUserGraph)	user -> user follows	~1 week	"who your network recently followed" account recommendations
UVG (UserVideoGraph)	user <-> video tweets	24-48 h	video recommendations from seed tweets

All four are stateful. They rebuild state from a dedicated Kafka topic that **Recos-Injector** produces. Recos-Injector is a streaming processor that consumes arbitrary event streams (favs, retweets, follows, client events), filters them, and publishes a cleaned stream to each GraphJet service [1].

Algorithms. UTEG's core is collaborative filtering by aggregation. It takes the requesting user's weighted follow list (weights from RealGraph), walks out to tweets those users recently engaged with, and scores each tweet by how many and which of those users engaged with it. The engagers are returned as social proof [1]. The VLDB paper describes SALSA-style random walks on the bipartite graph as the main personalised algorithm [14].

Lessons. A sliding window bounds memory and also acts as a freshness prior, because old engagement is forgotten automatically. Storing edges in only one direction (UTEG) halves memory but limits the traversals you can run (contrast with UTG).

4.2 Follow recommendations: SALSA, two-hop walks and graph features

The Follow Recommendations Service (FRS) collects many graph candidate sources [1]:

- **SALSA PYMK.** For each target user it builds a local bipartite graph. The left side holds the user plus top candidates from the forward and reverse address book, mutual follows, recent followings and recent followers, each source weighted differently. The right side holds those candidates plus the mutual follows of the left-side entries. It then runs a **personalised random walk with restarts** from the target user. This lineage goes back to Twitter's 2013 "WTF: Who to Follow" system [15].
- **Two-hop random walk.** Take the user's top K first-degree connections by RealGraph weight, expand each to its top L, drop first-degree nodes, score by weighted sum, and keep the top M.
- **Graph Feature Service.** Computes pairwise graph features for ranking, such as "how many of A's followings have favourited C" or "how many of A's followings follow C" [1].
- **RealGraph.** A gradient-boosted classifier, trained on daily-aggregated interaction graphs, that predicts whether user A will interact with user B. Its scores weight the edges for most of the walks above [1].
- **Tweepcred.** A PageRank-based user reputation score [1].

FRS ranks the union of candidates by a weighted sum of p(follow | rec) and p(positive engagement | follow) [1].

4.3 SimClusters: sparse community embeddings

Problem. Twitter wanted one representation that could cover users, tweets, topics and entities, could be updated in real time for tweets, and could be interpreted by people [16] [1].

Algorithm (from the repo documentation [1]):

1. Treat follows as a **bipartite graph** of consumers (followers) and producers (followees).
2. Compute **producer-producer cosine similarity** over follower sets, and drop edges below a threshold.
3. Run **Metropolis-Hastings sampling-based community detection** with k communities on the similarity graph. The output is **KnownFor** (V), a maximally sparse matrix in which each producer belongs to at most one community. In production it covers the **top 20M producers** with **k of about 145,000 communities**.
4. **InterestedIn** U = A x V (follow matrix times KnownFor), with noise removal. These are long-term consumer embeddings.
5. **Producer embeddings**: cosine similarity between each producer's follower set and each community's InterestedIn vector. This lets a producer belong to many communities.
6. **Tweet embeddings** start empty. On every favourite, the faver's InterestedIn vector is added to the tweet's vector. A Heron (Summingbird) streaming job does this in real time and persists from Memcache to Manhattan.
7. **Topic embeddings** come from time-decayed favourites on topic-annotated tweets, or (in the TFG variant) from the sum of the topic's followers' fav-based InterestedIn vectors.

Serving: SimClusters-ANN and approximate cosine. The streaming job maintains two indexes: **top 400 tweets per cluster** and **top 100 clusters per tweet**. Exact cosine was too expensive. One home-timeline candidate path (TWISTLY) scanned up to 15,000 candidates per request (6 source tweets x 25 clusters x 100 tweets) at more than 6,000 RPS. So SimClusters-ANN uses a two-stage approximation [1]:

```
source embedding SV (top N=50 clusters)
     |  fetch top M=400 tweets per cluster   (cluster->tweet index)
     v
sparse approx matrix R (M*N tweets x N clusters)
     |  score = R . SV^T  (dot product on truncated vectors)
     v
top X=200 tweets -> fetch full tweet embeddings (only ~6% of M*N)
     |  exact cosine vs SV, keep top Y above threshold Z
     v
candidates (cosine >= ~0.7 treated as highly related in prod)
```

The second-stage cosine filter is there because raw dot products favour popular tweets [1]. The same index pattern is built in BigQuery for other engagement types, for example push-open-based indexes for notifications and video-view-based indexes for Home video [1].

Lessons. Sparse, interpretable embeddings with roughly 145K dimensions can be updated *additively* in streaming, which dense embeddings cannot easily do. They also give an inverted index (cluster to items) at no extra cost. The cost is a separate rescoring pass to remove popularity bias.

4.4 TwHIN: dense heterogeneous-graph embeddings

Problem. SimClusters is derived from the follow graph alone. Twitter wanted dense embeddings that combine many relation types (follows, favs, retweets, replies, ad clicks) across many entity types, for use in candidate retrieval and as model features [2][17].

Model. TwHIN is a knowledge-graph embedding over a heterogeneous information network. It uses translation-style scoring (TransE-like): each relation has a learned translation vector, and an edge (lhs, rel, rhs) scores highly when lhs + r is close to rhs. The paper describes training at billions of nodes and on the order of 10^11 edges with PyTorch-BigGraph-style partitioned training [17][18]. The open-source version reimplements the training in **TorchRec**. It reads parquet edge lists with `lhs`, `rel` and `rhs` columns, defines one embedding table per entity type and a `translation` operator per relation (example relations: fav, reply, retweet, magic_recs), and uses **in-batch negatives** [4]. Subsampled, anonymised follow and fav graphs were released as public datasets [4].

Tricks reported in the paper (approximate):

- **Multi-modal user embeddings.** Users can have several embeddings, made by clustering the TwHIN embeddings of the accounts they follow. This mirrors PinnerSage [17].
- The same embeddings are reused across tasks: candidate generation (ANN), features in heavy rankers, ads, and harmful-content detection [17].

Twitter's retrieval-signals table shows TwHIN uses follows, favs, retweets, quotes and replies as features or labels, and notification opens as features [2].

4.5 How the Twitter graph pieces fit together

Component	Family	Freshness	Representation	Serving
UTEG/UTG/UVG	online walk / CF	seconds (Kafka)	none (raw graph)	in-memory GraphJet replicas
UUG, SALSA, 2-hop	online walk	seconds-days	none	GraphJet / FRS
SimClusters	community + sparse vectors	KnownFor batch; tweets real time	~145K-dim sparse	cluster->tweet inverted index
TwHIN	shallow KG embedding	batch	dense	ANN + ranking features
RealGraph	GBDT edge scorer	daily	edge weights	feature store

---

5. LinkedIn LiGNN: production GNNs on a temporal, heterogeneous graph

Problem

LinkedIn's graph links members, jobs, companies, posts, ads, groups and skills. It runs recommendation across Feed, Jobs, People You May Know (PYMK) and Ads. Static shallow

embeddings did not handle cold-start entities (new jobs, new posts), temporal drift, or the cost of training on very large graphs [19].

Architecture and techniques (as reported in [19])

```
heterogeneous member/job/post/ad/company graph (multi-edge-type, timestamps)
       |
       v
graph densification for cold start (add edges from low-degree nodes
       to similar/content-matched nodes)
       |
       v
GNN training: temporal neighbourhood sampling (multi-hop),
  learnable ID embeddings + content features, long-term losses,
  adaptive neighbour sampling, grouped/sliced batches,
  shared-memory sampling queue, local gradient optimisation
       |
       v
embeddings refreshed (batch + nearline) -> feature store / ANN
       |
       v
downstream rankers & retrievers: Feed, Jobs, PYMK, Ads
```

- **Temporal graph architectures with long-term losses.** Sampling and training use edge timestamps, so the model predicts future engagement from past neighbourhoods. This avoids leakage and captures drift.
- **Graph densification for cold start.** Low-degree nodes get extra edges, for example to content-similar entities, so the GNN has neighbours to aggregate.
- **ID embeddings plus multi-hop neighbour sampling.** This combines transductive capacity with inductive generalisation.
- **Training-systems work.** The paper reports about a **7x training speedup** from adaptive neighbour sampling, grouping and slicing of training batches, a specialised shared-memory queue, and local gradient optimisation [19].
- **Nearline inference** keeps embeddings fresh for new entities.

Reported online impact (approximate)

The paper reports A/B lifts that include, approximately: about +1% in job-application hearing-back rate, about +2% Ads CTR, about +0.5% feed engaged daily active users, about +0.2% sessions, and about +0.1% weekly active users from people recommendations [19]. The individual lifts are small, but LiGNN runs as an embedding platform, so they add up across many products.

Lessons

- In production, **temporal correctness** matters more than architecture novelty. Sampling neighbours from the future inflates offline metrics.
- Cold start is a graph-construction problem (densification) as much as a model problem.
- The biggest wins were in the **systems layer**: sampling throughput, data layout and queueing.

---

6. Uber Eats: GraphSAGE for dish and restaurant recommendations

Problem

Uber Eats wanted to recommend dishes and restaurants. Ordering is sparse and strongly local: a user orders from a small set of nearby restaurants [20].

Design (as reported in [20])

- **Graphs.** Bipartite user-dish and user-restaurant graphs, built **per city**, because the delivery radius makes cross-city edges almost meaningless. Edge weights encode order frequency.
- **Model.** GraphSAGE [11] with sampled neighbourhoods and node features (for example dish and restaurant descriptors).
- **Loss.** Max-margin ranking loss, extended so that heavily ordered (high-weight) positives are pushed above lightly ordered "low-rank positives", and those above negatives. The graph's edge weights are therefore used in the loss as well as in sampling.
- **Pipeline.** Daily offline training and embedding generation on Uber's ML platform. User-dish and user-restaurant similarity from the embeddings becomes **features** in the downstream recommendation ranking model, and also feeds candidate retrieval.
- **Results (approximate).** The blog reports a notable offline AUC improvement for the ranker when graph features were added, with the graph-based similarity among the most important features. Online tests in initial cities showed CTR uplift [20].

```
orders (per city) -> user-dish / user-restaurant weighted bipartite graphs
       -> GraphSAGE (2-hop sampled, max-margin + low-rank positives)
       -> user, dish, restaurant embeddings (daily)
       -> similarity features -> ranking model -> Eats homefeed carousels
```

Lessons

Partitioning the graph by geography gives scalability (each city is trained independently) and also encodes a hard constraint. A GNN does not have to be the recommender: GNN embeddings used as *features* in an existing ranker are often the lowest-risk way to ship.

---

7. Alibaba: session graphs, side information and GNN platforms

7.1 EGES: billion-scale commodity embedding (Taobao)

Problem. Taobao matches about a billion users to about two billion items in candidate generation. Item-item similarity from co-purchase CF misses long-tail and new items [21].

Pipeline (as reported in [21]):

1. **Build a weighted directed item graph from user sessions.** Behaviour sequences are split into sessions (the paper uses a time window of about one hour). Consecutive items in a session become a directed edge, weighted by co-occurrence count across all users. The paper also describes filtering noise such as very short clicks and over-active users.
2. **BGE (base graph embedding).** DeepWalk-style weighted random walks plus skip-gram.
3. **GES.** Adds side information (category, shop, brand, price level and similar) with separate embeddings that are averaged with the item ID embedding, which helps cold-start items.
4. **EGES.** Learns per-item **attention weights over side-information embeddings** (softmax of learned scalars), because the importance of brand or shop differs by item.

Deployment. Item embeddings drive item-to-item similarity in the matching stage. Each item's nearest neighbours by embedding become its candidate list. The paper reports CTR gains over the previous CF-based matching in online A/B tests [21].

Lesson. A session graph combined with random walks turns ordered co-occurrence into multi-hop transitive similarity, and side-information embeddings close the cold-start gap at almost no serving cost.

7.2 AliGraph / Graph-Learn and the Dynamic Graph Service

Alibaba open-sourced its GNN platform, **Graph-Learn (formerly AliGraph)**, which is used "in many scenarios within Alibaba, such as search recommendation, network security, and knowledge graph" [5][22]. It has two parts:

- **GraphLearn-Training.** Distributed sampling over batch graphs, a gremlin-like Graph Sampling Language, TensorFlow and PyTorch support, and offline or incremental GNN training. The documented production loop retrains incrementally every hour on a window of graph data and pushes the model to TF Serving [5].
- **Dynamic Graph Service (DGS).** Online inference on dynamic graphs [6]. DGS does not query a general graph store at request time. Instead, it **pre-samples the update stream according to a pre-registered sampling query** and caches only the sampled results, so each request becomes a local memory lookup.

```
graph updates (stream/bulk)
   -> Data Loader (build Vertex/EdgeUpdateRecord, partition)
   -> Sampling Workers (decompose n-hop query into 1-hop samplers,
      Sample Store + Subscription Table, Subgraph Router)
   -> Serving Workers (all n-hop samples for a vertex co-located;
      SubGraph Store)                      <- client request (Java GSL)
   -> sampled subgraph -> TF model service -> prediction
(all hops connected via fault-tolerant queues)
```

Documented performance: on one node (64 cores, 256 GB RAM), DGS completes a **2-hop sampling query within 20 ms P99**, at **20,000 QPS**, while ingesting **110 MB/s** of graph updates, and it scales out roughly linearly [6]. Alibaba also maintains **Euler**, a separate graph-learning framework from its advertising (Alimama) teams [7].

Lesson. The DGS design applies "materialise the query at write time" to GNN inference. If the sampling pattern is known in advance, keeping the sample current incrementally is

cheaper than running the traversal on every read.

---

8. Meta PyTorch-BigGraph: the shallow-embedding workhorse

PyTorch-BigGraph (PBG) is Meta's open-source system for learning embeddings on graphs "with up to billions of entities and trillions of edges" [18]. It supports TransE, RESCAL, DistMult and ComplEx with per-relation operators over shared entity embeddings. Its scaling techniques are:

- **Graph partitioning.** Entities are split into P partitions and edges into P x P buckets, so only two partitions need to be in memory at once.
- **Multi-threaded and distributed training** with a shared parameter server for relation parameters.
- **Batched negative sampling.** Negatives are reused across a chunk of positives, which turns scoring into a matrix multiply [18].

TwHIN's original training followed this pattern, and its TorchRec reimplementation keeps the same edge-list (lhs, rel, rhs) contract [4][17]. PBG is the reference design when you need *transductive* embeddings for every node of a huge multi-relational graph on commodity hardware.

---

9. Cross-system comparison

System (org, year)	Family	Graph	Scale (reported, approx.)	Serving path	Freshness
Pixie (Pinterest, 2018) [8]	online random walk	pin-board bipartite	~3B nodes, ~17B edges, RAM per server	walk per request, tens of ms	graph rebuilt periodically; query pins real time
PinSage (Pinterest, 2018) [9]	inductive GCN	pin-board + features	~3B nodes, ~18B edges, ~7.5B examples	MapReduce inference -> ANN	batch
PinnerSage (Pinterest, 2020) [12]	clustering on GNN embeds	user action history	per-user multi-medoid	medoid ANN (HNSW-style)	daily + real-time actions
GraphJet / UTEG (Twitter, 2016) [3][14]	online CF / SALSA	user-tweet bipartite, sliding window	1M edges/s ingest, 500 recs/s per server	in-memory per replica	seconds; 24-48 h window
SimClusters (Twitter, 2020) [1][16]	community detection, sparse vectors	follow bipartite	20M producers, ~145K clusters	cluster->top-400-tweets index; >6K RPS path	KnownFor batch; tweets streaming
TwHIN (Twitter, 2022) [4][17]	KG embedding (translation)	multi-relation HIN	billions of nodes, ~10^11 edges	ANN + features	batch
LiGNN (LinkedIn, 2024) [19]	temporal GNN	member/job/post/ad HIN	very large; 7x training speedup	features + retrieval, nearline	batch + nearline

System (org, year)	Family	Graph	Scale (reported, approx.)	Serving path	Freshness
Uber Eats GNN (Uber, 2019) [20]	GraphSAGE	user-dish/restaurant per city	per-city graphs	ranking features + retrieval	daily
EGES (Alibaba, 2018) [21]	random walk + skip-gram + side info	session item graph	~1B users, ~2B items	item-item ANN in matching	batch
Graph-Learn DGS (Alibaba) [5][6]	online GNN sampling	dynamic HIN	2-hop P99 20 ms, 20K QPS/node	pre-sampled cache -> TF Serving	streaming
PBG (Meta, 2019) [18]	KG embedding	multi-relation	billions of entities, trillions of edges	offline embeddings	batch

10. Recurring design patterns

10.1 Random walks show up at every layer

The same primitive does different jobs in different systems:

- As the **retriever** (Pixie, UTG, SALSA).
- As a **neighbourhood sampler** for GNNs (PinSage importance pooling).
- As the **training corpus** for skip-gram (EGES/DeepWalk).
- As a **negative-mining oracle**: PinSage uses PPR ranks to select hard negatives.

10.2 Bipartite thinking

Pinterest (pin-board), Twitter (consumer-producer, user-tweet) and Uber (user-dish) all project their data onto bipartite graphs. The reasons are practical:

- Two-hop walks give item-item similarity directly.
- SALSA's hub/authority split matches consumer and producer roles.
- Storage can be one-directional (UTEG) when only one traversal direction is needed.

10.3 Popularity control

Hubs dominate every graph method, so each system controls popularity explicitly:

- Pixie prunes edges on popular pins and scales walk budgets by degree.
- SimClusters re-scores by cosine rather than dot product.
- PinSage uses top-T neighbour truncation.
- EGES filters over-active users from sessions.

10.4 Multi-interest users

When an item embedding space is good, users are usually represented as *several* points in it. PinnerSage uses medoids of action clusters, and TwHIN builds multi-modal user embeddings from clusters of followed accounts.

10.5 Freshness tiers

Production systems almost always combine three tiers:

1. A slow batch backbone: KnownFor, TwHIN, PinSage.
2. A streaming update of item vectors: SimClusters tweet embeddings, LiGNN nearline.
3. An online traversal over a short window: GraphJet, Pixie.

10.6 Sampling infrastructure is the bottleneck

PinSage's producer-consumer loop, LiGNN's shared-memory queues and 7x speedup, and DGS's pre-sampling all make the same point. The engineering effort in a production GNN goes into **neighbourhood sampling and feature fetching**, not into the convolution.

---

11. Key takeaways for system designers

1. **Start with walks before GNNs.** A replicated in-memory bipartite graph with random walks with restart (Pixie) or SALSA (GraphJet) is a strong, explainable, real-time candidate source. It has no training loop, and it returns social proof ("liked by X") for free.
2. **Replicate the graph if it fits in RAM.** Pixie and GraphJet both keep the whole working graph per server. Distributed walks are latency poison. Use sliding windows (24-48 h in GraphJet services) or pruning to make the graph fit.
3. **Separate representation from serving.** Twitter's Representation Manager/Scorer and Pinterest's PinSage-as-coordinate-system let many products share one embedding space. Treat graph embeddings as a platform.
4. **Choose transductive or inductive deliberately.** Shallow embeddings (TwHIN, PBG, EGES) are cheap and strong for established nodes. Inductive GNNs (PinSage, LiGNN, Uber) cover cold-start items through features. EGES-style side-information embeddings are a cheap halfway point.
5. **Sparse, additive representations stream well.** The SimClusters approach (add the InterestedIn vector of each user who favourites a tweet) gives real-time tweet embeddings plus an inverted index. Plan a cosine rescoring pass to undo popularity bias.
6. **Use two-stage approximate scoring to save I/O.** SimClusters-ANN fetches only about 6% of full embeddings by first scoring truncated vectors. The same idea (coarse score, then exact rescoring of a short list) applies to any embedding-retrieval path.
7. **Be strict about time.** Use temporal neighbourhood sampling, label windows after feature windows (RealGraph labels the day after the candidate period [1]), and long-horizon losses (LiGNN).
8. **Engineer for hubs.** Use degree-aware walk budgets, neighbour truncation, entropy-based pruning and cosine normalisation. Without them, graph recommenders turn into popularity recommenders.
9. **Ship GNN output as features first.** Uber Eats and LinkedIn both report most of their wins from adding graph-derived embeddings or similarities to existing rankers and retrievers. This is lower risk than replacing the stack.

10. **Precompute the query when the pattern is fixed.** Alibaba DGS keeps the n-hop sample current incrementally as the graph changes, which reaches 20 ms P99 at 20K QPS per node. Prefer this design when every request runs the same sampling pattern.

References

1. Twitter/X, "the-algorithm" repository: README files for SimClusters v2, SimClusters ANN, user_tweet_entity_graph, user_tweet_graph, user_user_graph, user_video_graph, recos-injector, follow-recommendations-service (SALSA, two-hop random walk), graph-feature-service, interaction_graph (RealGraph), representation-manager, representation-scorer, cr-mixer. X Corp., 2023. https://github.com/twitter/the-algorithm
2. Twitter/X, "Signals for Candidate Sources" (RETREIVAL_SIGNALS.md), the-algorithm repository, 2023. https://github.com/twitter/the-algorithm/blob/main/RETREIVAL_SIGNALS.md
3. Twitter, "GraphJet" open-source repository README, 2016. https://github.com/twitter/GraphJet
4. Twitter/X, "TwHIN in torchrec" project README and configs, the-algorithm-ml repository, 2023. https://github.com/twitter/the-algorithm-ml/blob/main/projects/twhin/README.md
5. Alibaba, "Graph-Learn (formerly AliGraph)" repository README, 2020-2023. https://github.com/alibaba/graph-learn
6. Alibaba, "Dynamic Graph Service (DGS) Introduction", Graph-Learn documentation (docs/en/dgs/intro.md). https://github.com/alibaba/graph-learn/blob/master/docs/en/dgs/intro.md
7. Alibaba (Alimama), "Euler" graph learning framework repository. https://github.com/alibaba/euler
8. C. Eksombatchai, P. Jindal, J. Z. Liu, Y. Liu, R. Sharma, C. Sugnet, M. Ulrich, J. Leskovec, "Pixie: A System for Recommending 3+ Billion Items to 200+ Million Users in Real-Time", Pinterest / WWW 2018. https://arxiv.org/abs/1711.07601
9. R. Ying, R. He, K. Chen, P. Eksombatchai, W. L. Hamilton, J. Leskovec, "Graph Convolutional Neural Networks for Web-Scale Recommender Systems" (PinSage), Pinterest / KDD 2018. https://arxiv.org/abs/1806.01973
10. Pinterest Engineering, "PinSage: A new graph convolutional neural network for web-scale recommender systems", Pinterest Engineering Blog, 2018. https://medium.com/pinterest-engineering/pinsage-a-new-graph-convolutional-neural-network-for-web-scale-recommender-systems-88795a107f48
11. W. L. Hamilton, R. Ying, J. Leskovec, "Inductive Representation Learning on Large Graphs" (GraphSAGE), Stanford / NeurIPS 2017. https://arxiv.org/abs/1706.02216
12. A. Pal, C. Eksombatchai, Y. Zhou, B. Zhao, C. Rosenberg, J. Leskovec, "PinnerSage: Multi-Modal User Embedding Framework for Recommendations at Pinterest", Pinterest / KDD 2020. https://arxiv.org/abs/2007.03634
13. N. Pancha, A. Zhai, J. Leskovec, C. Rosenberg, "PinnerFormer: Sequence Modeling for User Representation at Pinterest", Pinterest / KDD 2022. https://arxiv.org/abs/2205.04507

14. A. Sharma, J. Jiang, P. Bommannavar, B. Larson, J. Lin, "GraphJet: Real-Time Content Recommendations at Twitter", Twitter / PVLDB 9(13), 2016. http://www.vldb.org/pvldb/vol9/p1281-sharma.pdf
15. P. Gupta, A. Goel, J. Lin, A. Sharma, D. Wang, R. Zadeh, "WTF: The Who to Follow Service at Twitter", Twitter / WWW 2013. https://web.stanford.edu/~rezab/papers/wtf_overview.pdf
16. V. Satuluri, Y. Wu, X. Zheng, Y. Qian, B. Wichers, Q. Dai, G. M. Tang, J. Jiang, J. Lin, "SimClusters: Community-Based Representations for Heterogeneous Recommendations at Twitter", Twitter / KDD 2020. https://www.kdd.org/kdd2020/accepted-papers/view/simclusters-community-based-representations-for-heterogeneous-recommendatio
17. A. El-Kishky, T. Markovich, S. Park, C. Verma, B. Kim, R. Eskander, Y. Malkov, F. Portman, S. Samaniego, Y. Xiao, A. Haghighi, "TwHIN: Embedding the Twitter Heterogeneous Information Network for Personalized Recommendation", Twitter / KDD 2022. https://arxiv.org/abs/2202.05387
18. A. Lerer, L. Wu, J. Shen, T. Lacroix, L. Wehrstedt, A. Bose, A. Peysakhovich, "PyTorch-BigGraph: A Large-scale Graph Embedding System", Facebook AI / SysML 2019 (paper and repository). https://mlsys.org/Conferences/2019/doc/2019/71.pdf and https://github.com/facebookresearch/PyTorch-BigGraph
19. F. Borisyuk et al., "LiGNN: Graph Neural Networks at LinkedIn", LinkedIn / KDD 2024. https://arxiv.org/abs/2402.11139
20. A. Jain, I. Liu, A. Sarda, P. Molino, "Food Discovery with Uber Eats: Using Graph Learning to Power Recommendations", Uber Engineering Blog, 2019. https://www.uber.com/blog/uber-eats-graph-learning/
21. J. Wang, P. Huang, H. Zhao, Z. Zhang, B. Zhao, D. L. Lee, "Billion-scale Commodity Embedding for E-commerce Recommendation in Alibaba" (EGES), Alibaba / KDD 2018. https://arxiv.org/abs/1803.02349
22. R. Zhu, K. Zhao, H. Yang, W. Lin, C. Zhou, B. Ai, Y. Li, J. Zhou, "AliGraph: A Comprehensive Graph Neural Network Platform", Alibaba / PVLDB 12(12), 2019. https://arxiv.org/abs/1902.08730
23. Twitter/X Engineering, "Twitter's Recommendation Algorithm", Twitter Engineering Blog, 2023. https://blog.x.com/engineering/en_us/topics/open-source/2023/twitter-recommendation-algorithm

08. Feature Engineering and Feature Stores: Point-in-Time Correctness, Streaming Features, and Serving-Time Logging

A ranking model is only as good as the features it sees, and in a production recommender the difficult part is rarely the feature formula. The difficult part is computing the same value twice: once offline for millions of historical (user, item, timestamp) training rows, and once online for a few thousand candidates inside a 50-200 ms request budget. Feature platforms exist to make those two computations agree. This chapter surveys the public designs (Twitter's aggregation framework, Uber Michelangelo and Palette, Airbnb Zipline/Chronon, LinkedIn Feathr and Venice, Feast, Tecton, Hopsworks and Featureform) and the techniques they share: declarative feature definitions, point-in-time-correct (PITC) backfills, streaming windowed aggregation with tiles, and logging features at serving time.

A note on sourcing. Most of this chapter is drawn from primary open-source documentation and code: the Chronon, Feast, Feathr, Venice, Hopsworks and Tecton example repositories and Twitter's `the-algorithm` repository [1-16]. Several corporate blogs could not be reached while this chapter was being researched: Uber, Medium-hosted Airbnb posts, DoorDash, Netflix and Google developer pages. Sections that rely on those sources say so explicitly, and their numbers should be treated as approximate.

---

1. Why recommenders need a feature platform

Recommender ranking models are fed by four families of features:

1. **Entity attributes**: slowly changing properties of a user or item (country, account age, item category, price), usually from database snapshots.
2. **Windowed aggregates (counters)**: "user's clicks on video in last 1h/24h/30d", "item CTR last 7d", "user x author engagement count with 50-day half-life". These carry most of the predictive power in classic ranking stacks and are the hardest to keep consistent.
3. **Embeddings**: precomputed user and item vectors from two-tower or graph models. They are stored as feature tables and combined at request time, for example as cosine similarity.
4. **Request/context features**: query text, device, time of day, page position, and the candidate's retrieval source. These are only known at request time.

Twitter's Home Mixer says that its For You pipeline does "Feature Hydration — Fetch the ~6000 features needed for ranking" between candidate generation and scoring [5]. Its follow-recommendation service builds a `DataRecord` per (user, candidate) pair after

hydration. `DataRecord` is "the Twitter-standard Machine Learning data format used to represent feature data, labels, and predictions when training or serving" [6]. Several thousand features times hundreds to thousands of candidates per request is the workload a feature store must serve.

1.1 The three failure modes

Every system in this chapter is built to avoid the same three problems:

- **Label leakage / time travel errors.** A training row for an impression at time t uses feature values computed with data from after t. Feathr's docs call this feature leakage: "Models should not use the future to predict the present... The model will (usually) perform better during training, but it will not perform as good during online serving" [10].
- **Training/serving skew.** The offline pipeline (SQL in a warehouse) and the online pipeline (Java/Scala in a service) implement "the same" feature differently. They may differ in null handling, time zones, window boundaries, dedup rules or freshness lag.
- **Duplicated, unshared pipelines.** LinkedIn's Feathr talk states the problem directly: "The cost of building and maintaining feature pipelines was borne redundantly across many teams... no common type system, no common feature namespace" [12].

1.2 Two naive strategies and their trade-offs

Chronon's documentation frames the design space around two approaches teams take without a platform [3]:

Approach	How it works	Pros	Cons
Log-and-wait	Compute features in the serving path; log them to the warehouse; train on the logs once enough data has accumulated	Train/serve consistency by construction; can use service-call and request-context features	Weeks to months before training data exists; large windowed aggregations hard in the app layer; cannot use historical warehouse data
Replicate offline-online	Train on warehouse SQL; re-implement the features online	Uses all historical data; warehouse is good at heavy aggregation	"Often very error prone, resulting in inconsistent data between training and serving"; complex infra; real-time windowed aggregations are hard

Every platform below is an attempt to get the benefits of both columns. You write one declarative definition. It compiles into a PITC backfill for training and an online materialization for serving, and serving-time logs are used to verify, or substitute for, the backfill.

---

2. Core concepts and vocabulary

Different systems use different names for the same ideas:

Concept	Chronon	Feast	Feathr	Hopsworks	Tecton	Twitter agg. framework
Unit of feature definition	`GroupBy`	`FeatureView` / `StreamFeatureView`	`FeatureAnchor`	Feature group	Batch/Stream FeatureView	`AggregateGroup`
Model-level feature set	`Join`	`FeatureService`	feature query (list of names)	Feature view	`FeatureService`	config of groups
Training "left side"	left source / query log	entity dataframe	observation data	spine / label FG	spine dataframe	training DataRecords
Request-time features	contextual / external parts	on-demand FV	(derived, UDF)	on-demand transformation	`realtime_feature_view`	(hydration in service)
Online store	pluggable KV store	Redis, DynamoDB, Bigtable, Cassandra, etc.	Redis, Cosmos DB, Aerospike	RonDB	managed	Manhattan (batch), memcached (real-time)

Sources: [1-4, 7-11, 13-16].

Point-in-time join. For each left-side row (key, ts), find for each feature the value that would have been observed online at ts. That means the most recent value with event time < ts, and for aggregates, the window ending at ts. Feast scans "backward in time from the entity dataframe timestamp up to a maximum of the TTL". The TTL is "relative to each timestamp within the entity dataframe", not to query time [7]. Hopsworks uses event_time columns "to determine the most recent (but not newer) feature values" [14].

Event time vs. availability time. This subtlety is often missed. Feast's default PIT join constrains only the event timestamp. A value that was "backfilled or corrected after an entity dataframe timestamp can still be returned for it" unless you pass `filter_by_created_timestamp=True`, which restricts retrieval to values already available at the row's timestamp [7]. In production, a daily batch feature for day D only lands in the online store some hours after midnight. A truly faithful backfill must model that landing lag, not just the event date.

---

3. Twitter: the aggregation framework (batch + real-time counters)

Twitter's open-sourced `the-algorithm` contains one of the clearest public descriptions of a production counter-feature system for recommendation [4, 17].

Problem. The Home Timeline heavy ranker, MagicRecs (push) and EmailRecs all need user, user x author and tweet engagement aggregates, conditional on content type and engagement label. The README notes that "these types of engineered aggregate features have proven to be highly impactful across different teams at Twitter" [4].

Abstraction: `AggregateGroup`. It behaves like a SQL GROUP BY with ML-specific knobs [17]:

- `keys`: grouping keys such as `USER_ID`, `(USER_ID, AUTHOR_ID)` or `(USER_ID, ENGAGER_ID)`. Sparse-binary keys get "flatMap" semantics. Grouping by `(USER_ID, AUTHOR_INTEREST_IDS)` emits one record per (user, interest) pair.
- `features`: the binary or continuous features to aggregate (for example `HAS_PHOTO`).
- `labels`: the engagement conditioning the count (for example `IS_FAVORITED`). The aggregate is a feature x label cross.
- `metrics`: `CountMetric`, `SumMetric` and others.
- `halfLives`: exponential decay instead of hard windows. The example group computes "the number of times each user has faved a tweet with a photo... decayed with a 50 day halflife" [17].

Decayed counters are a notable design choice. They are monoid-friendly: an old value plus elapsed time is enough to roll forward. Each key stores O(1) state instead of a window of events.

Architecture.



The same `AggregateGroup` definitions drive a daily summingbird-scalding batch job (outputs uploaded "to Manhattan for online hydration") and a summingbird-storm real-time job backed by memcached [4, 17]. The real-time job subscribes to client-event Kafka topics, converts events to `DataRecord`s and can enrich them mid-stream. MagicRecs left-joins each engagement with a user-history store before aggregating [17]. Offline, training data joins aggregates with `joinWithLarger((USER_ID, AUTHOR_ID), ...)`, and `AggregatesV2MostRecentFeatureSourceBeforeDate` reads "the most recent aggregates before a provided date". This is a day-granularity form of point-in-time correctness [17].

Feature identity by hash. Twitter predefines feature names and types so that "when features are being generated, scribed or used to score they can be identified with only a hash of their name" [18]. This keeps logged and served payloads compact. Feature logging ("scribing") is treated as a first-class path.

Event backbone. Unified User Actions (UUA) is "a centralized, real-time stream of user actions". It merges client- and server-side events into one Kafka stream, which is replicated to HDFS, Pub/Sub, GCS and BigQuery [19]. The User Signal Service standardizes explicit and implicit signals for retrieval and ranking features [20]. Graph Feature Service answers pairwise graph features such as "how many of A's followings have favorited C" [21].

Lessons. (1) One definition, two engines (batch Scalding and streaming Storm) is the oldest form of the "write once" idea. (2) Decayed counters avoid storing windows. (3) Day-

boundary PITC ("most recent before date") is cheap but gives up intra-day freshness. Chronon later made that trade-off configurable.

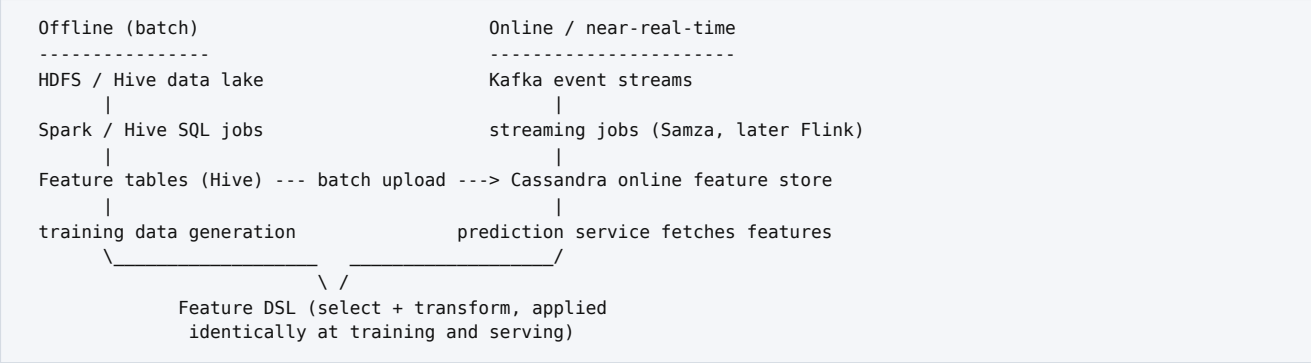
---

4. Uber Michelangelo and Palette

Primary pages for this section (Uber engineering blog and InfoQ) could not be fetched during research. The description follows the widely cited 2017 Michelangelo post [22] and the 2019 Palette talk [23]. Treat specifics as approximate.

Problem. Uber's 2017 Michelangelo post describes an end-to-end ML platform: manage data, train, evaluate, deploy, predict, monitor. Feature management was singled out as the hardest part to make reusable [22].

Architecture (as publicly described).



Key design elements.

- **Shared feature store.** Teams publish features to a central store and reference them by name in model configs. The 2017 post reported on the order of 10,000 features in the store (approximate).
- **Two pipelines.** A batch path precomputes features from the data lake and periodically bulk-loads them into an online KV store (Cassandra). A near-real-time path computes streaming metrics, for example a restaurant's recent average meal-prep time, from Kafka and writes them to the same store. The streaming path also logs values back to HDFS for training.
- **Feature DSL.** Feature selection and lightweight transformations (for example normalization or day-of-week extraction) are expressed in a restricted DSL. The DSL is part of the model configuration and is applied identically at training and prediction time. This is the main defense against transformation skew.
- **Palette (2019).** Palette is the evolution of the Michelangelo feature store into a feature engineering platform. It formalizes feature definitions over batch, near-real-time and RPC/external sources, and adds a catalog, ownership and orchestration [23].

Lessons. Michelangelo popularized two ideas that every later system copies: the dual offline/online store, and putting transformations inside the model artifact so they are applied the same way everywhere. Hopsworks later called these model-dependent transformations (Section 9). Tecton grew directly out of this lineage.

---

5. Airbnb Zipline to Chronon

Zipline was Airbnb's internal feature platform, presented publicly in 2018 and 2020 [24, 25]. It was rebuilt and open-sourced as **Chronon**, which the project says powers "all major ML applications within Airbnb, as well as major use cases at Stripe", jointly maintained by both companies [2]. Chronon's internal design docs still use the name "zipline" [1].

5.1 Data model

Chronon distinguishes sources by their temporal semantics [1, 26]:

- **EventSource**: an immutable log, for example purchases. It needs a date-partitioned Hive table, where each partition holds only that day's events, plus optionally a stream topic.
- **EntitySource**: a mutable table, for example users or listings. It needs a daily snapshot table, and for intra-day accuracy a mutation (CDC) table plus a mutation topic. In the docs' terms these are "fact/measure" sources versus "dimension" sources [1].
- Request context and external service calls ("contextual" and "external" parts of a `Join`), which cannot be backfilled natively [27].

A **GroupBy** aggregates a source by keys over windows (SUM, COUNT, AVERAGE, LAST_K, approx percentiles via KLL sketches, and others) [2, 28]. A **Join** assembles many GroupBys onto a left source. That left source is either the events that drive predictions (for example checkouts or searches) or the logged online queries. One definition produces three things: training backfills, serving endpoints and monitoring pipelines [2].

```
        v1 = GroupBy(
            sources=[Source(events=EventSource(table="data.purchases",
                                               topic="events/purchases",
                                               query=Query(selects=..., time_column="ts")))],
            keys=["user_id"], online=True,
            aggregations=[Aggregation(input_column="price", operation=Operation.SUM,
                                      windows=[Window(d, TimeUnit.DAYS) for d in [3, 14, 30]]),
                          Aggregation(input_column="price", operation=Operation.LAST_K(10))])
```

(abridged from [2])

5.2 Accuracy: SNAPSHOT vs TEMPORAL

`accuracy` is an explicit GroupBy toggle [28]:

- `SNAPSHOT`: values computed as of midnight and refreshed daily. The backfill for a row at ts uses the value as of the prior midnight. That is exactly what the online store would have served if updates land only in batch.
- `TEMPORAL`: real-time updates online, and millisecond-accurate point-in-time backfills offline. This is the default when a `topic` or `mutationTopic` is set.

The Join docs give the accuracy for each (left source type, right source type) combination. For example, left events with right batch-only events default to midnight-accurate unless TEMPORAL is requested [27]. Making the online freshness model explicit in the backfill

semantics is the key idea. The backfill does not compute the "true" value at ts. It computes the value the online system would have served at ts.

5.3 The backfill algorithm: from quadratic joins to sawtooth windows

Chronon's "Window Tiling Algorithm" doc is the most detailed public treatment of how to make PITC backfills scale to recommender training sets [1]:

1. **Naive SQL.** Join queries to events on key with `event.ts in [query.ts - W, query.ts)` and group by query. For skewed keys this is quadratic, and daily rollups only reduce "trillions" of operations to "billions".
2. **Sort-and-scan.** Two cursors give O(n log n) time, but only for subtractable aggregates such as sum and count, not min, max or last. It also requires holding all events in memory.
3. **Interval tree with broadcast join.** Labels (queries) are usually far fewer than events, "at least two orders of magnitude". Queries are broadcast to all workers, events are streamed through an interval tree over sorted query timestamps with O(log n) updates per event, and the results are reduced. Row orientation (one tree traversal updating hundreds of features) beats column orientation.
4. **But recommenders break the assumption.** "There is a class of very impactful models that this is not true for - search ranking & recommendation systems... the labelled training data can be as large as all the searches on the platform." Both sides are now too big for memory [1].
5. **Sawtooth windows.** Sliding windows are fresh but hard to reuse. Hopping windows are reusable but stale. Chronon combines them: the window is `[round(query_ts - W, hop), query_ts)`. The *tail* is hop-aligned and precomputed once per hop bucket, shared by every query in that 5-minute bucket. The *head* (from the last hop boundary to query_ts) is computed with a sorted merging scan. This avoids quadratic compute while keeping the head exact to the millisecond.

```
window W, hop h:
|<------------- tail: precomputed per hop bucket ------------->|<- head ->|
[round(ts - W, h) ............................. round(ts, h)) [.. ts)
  shared by all queries whose ts falls in the same h-bucket      exact, per query
```

The online store mirrors these semantics. Batch uploads hold hop-aligned partial aggregates as of the batch end. Streaming updates cover everything after that end. "Streaming fetches replay everything after the batch end date" [28].

5.4 Online architecture: untiled vs tiled (Flink)

The Tiled Architecture doc quantifies the serving problem [29]:

- **Untiled.** Spark Streaming or Flink writes individual events to the KV store, and the fetcher reads O(events) and aggregates at request time. With 10 events/sec for one key and a 12-hour window, a single request must fetch and fold 432,000 events.
- **Tiled.** A stateful Flink job pre-aggregates events into tiles of intermediate representations (IRs), for example (sum, count) for AVG, and writes the tiles. The fetcher reads O(tiles). The same request reads 12 or 13 one-hour tiles. Stripe reported a 33% reduction in feature-serving latency after the initial implementation. Tiling also

helps hot keys (for example big merchants) and stores without range scans. "If your hottest keys don't exceed a few thousand events per day, the untiled approach may still be sufficient" [29].

```
Kafka -> Flink: [Source] -> [Spark-SQL eval (CatalystUtil)] -> [Window/tiling op]
                                                               |  IRs per tile,
                                                               v  emitted per event
                                        [Avro encode] -> [async KV sink]
Fetcher: get(key, tiles in [now-W, now)) -> merge IRs -> finalize -> feature values
e.g. COUNT/LAST over 3h, 1h tiles: [2,"B"] + [9,"A"] + [3,"C"] => [14, "C"]
```

The Flink job evaluates the same Spark SQL selects as the batch engine by embedding Catalyst inside Flink [30]. This is how one expression language is guaranteed across batch and streaming.

5.5 Measuring online/offline consistency from serving logs

Chronon logs every fetch, as Avro-encoded key bytes, value bytes, ts and caller, to a `query_logs` table partitioned by ds and join name [31]. A daily Spark job then:

1. Decodes the logs into `<join>_logged`.
2. Re-runs the join offline using the logged table as the left side.
3. Compares logged values against backfilled values per feature. It computes mismatch, missing and extra rates; SMAPE and delta distributions for numerics; and Levenshtein edit distance for sequences such as LAST_K. Distributions are summarized with ReqSketch, which was chosen over t-digest because it is more accurate at all ranges and more than 2x faster to update and serialize [31].

Logging sampling is key-based: `hash(key_bytes) % 100 * 1000 <= sample_percent * 1000`. This captures all queries for a subset of keys rather than a random subset of queries, which keeps the join's bloom filter small [31]. The doc names the inherent sources of skew. Kafka plus KV write latency means streaming features lag by milliseconds to seconds. Batch features are inconsistent "centered around midnight until the bulk upload of new features to kv store finishes". Proposed future work is a per-GroupBy write-lag that shifts offline query timestamps to match [31].

5.6 Bootstrap: log stitching

Chronon's **Bootstrap** primitive lets a Join take precomputed values from tables, including serving logs, for any (row, column) they cover. It backfills only the rest [32]. Listed scenarios include "Log stitching: Use online serving log as the source for production features instead of continuously recomputing it offline to ensure online/offline consistency and reduce computation cost"; backfilling external or contextual features before they have been logged; sharing data between production and experimental Joins; and recovering from logging outages [32]. To make logs joinable, row IDs such as an `event_id` are registered as contextual features. They are then logged with the served features [27].

Chronon also supports **chaining**: the output of a Join can feed another GroupBy, for example "average price of the last 10 items that a user purchased" (LAST_K followed by AVERAGE), served in real time [33].

Lessons from Chronon. (1) Make the online freshness model part of the offline semantics (SNAPSHOT vs TEMPORAL). (2) Recommender-scale PITC needs specialized algorithms such as sawtooth windows, not generic SQL. (3) Pre-aggregate on the write path (tiles of IRs) when keys are hot. (4) Logging is part of the platform, used both to measure skew and to replace recomputation.

---

6. LinkedIn: Feathr (and Venice as the serving substrate)

History and scale. According to the Feathr community talk, LinkedIn began Feathr development in 2017, saw broad internal adoption in 2018, and had the "majority of LinkedIn ML applications onboarded" by 2020. It was open-sourced with Azure integration in 2022 and joined LF AI & Data [12]. At LinkedIn it supports "hundreds of models", "thousands of features", "many kinds of entities (economic graph)" at "petabyte scale" [12]. Reported impact: experimentation with new features went "from weeks to days", running time improved "as much as 50%" over custom pipelines, and features became shareable across teams [12]. The model deployment service uses Feathr to ensure "a model's feature dependencies are deployed, before deploying the model" [12]. Use cases named in the talk include LinkedIn Jobs hiring preferences, LinkedIn Learning recommendations, and near-real-time personalization [12].

Abstractions [9, 10]:

- **Observation data**: the training "left side" of keys and timestamps, for example a click stream. Other stores call this the entity dataframe.
- **Source**: batch (HDFS, Delta, Parquet, etc.) or streaming (Kafka, Event Hubs).
- **Anchor** (`FeatureAnchor`): a set of features defined on one source with row-level transforms (`"cast_float(trip_distance)>30"`) or sliding-window aggregations.
- **Derived feature**: a feature computed from other features. The docs' recommender example is `user_item_similarity` built from a user embedding and an item embedding owned by different teammates [9].
- **Registry**: named features shared across teams, "like a package manager for feature engineering". Model code requests `FeatureQuery(feature_list=[...], key=item_id)` [12].

Point-in-time join. For each observation row, Feathr looks up the feature value "closest from the past" for the same key. This includes repeated keys at different times, for example the same user with labels on 05/01 and 05/03 getting different feature snapshots [10]. Feathr's docs contrast this with simple "time travel" to a single global timestamp. The per-event join wastes no data and is what Feathr implements at scale, with built-in optimizations "such as bloom filters and salted joins" for billions of rows [8].

Serving substrate: Venice. LinkedIn's open-source Venice is a "derived data storage platform" with high-throughput asynchronous ingestion from batch (Hadoop) and streaming (Samza) sources, low-latency reads "via remote queries or in-process caching", active-active cross-region replication with CRDT-based conflict resolution, and multi-tenancy. The project describes it as "particularly suitable as the stateful component backing a Feature Store, such as Feathr" [13]. The pattern is a write-optimized bulk/stream ingestion path plus a

read-optimized, possibly in-process, store. It reflects the reality that recommendation feature stores are read-dominated. Every request fans out to thousands of candidate keys.

```
HDFS / Spark (offline features) ----bulk push----\
                                                  >---> Venice (derived data store)
Kafka -> Samza (nearline features) ---stream-----/        |  remote reads or
                                                           |  Da Vinci in-process cache
Feathr definitions (anchors, derived, registry)            v
      |-- offline: PIT join onto observation data --> training sets
      `-- online: feature fetch for inference <------ ranking services
```

---

7. Feast: the open-source reference design

Feast (**Fea**ture **St**ore) originated at Gojek with Google Cloud in 2019 [34]. Its stated purpose is to make "features consistently available for training and serving" through an offline store, an online store and a feature server, and to "avoid data leakage by generating point-in-time correct feature sets" [35].

Architecture choices [36, 37, 38]:

- **Push model.** Producers push feature values to Feast, which stores them in the online store. Feast does not pull from producers at request time, because that latency "would be at least as long as your slowest call". The trade-off is stated plainly: "strong consistency is not guaranteed out of the box" [36].
- **Write patterns.** Sync vs async writes crossed with precomputed vs on-demand vs hybrid transformations. Hybrid is recommended for "time since last" features [37].
- **Transformation engines.** The feature server, the offline store (Snowflake, BigQuery, Spark, DuckDB...) or a compute engine. The docs warn that "different feature transformation code may be used under different transformation engines". This is a candid admission that the multi-engine design can itself cause skew [38].
- **Language.** Python for the feature server, on the argument that serving should be "a lightweight database lookup" over precomputed values. Java and Go clients are also available [39].
- **Online store backends.** As of 2026 the list includes Redis, Valkey, Dragonfly, DynamoDB, Bigtable, Datastore, Cassandra, ScyllaDB, HBase, Aerospike, Postgres, MySQL, MongoDB, Couchbase, Hazelcast, SingleStore, Snowflake, Elasticsearch, and vector stores such as Milvus, Qdrant and FAISS. There is also a Chronon bridge in which Chronon owns materialization and serving and Feast routes `get_online_features` to Chronon's join or groupby endpoints [40, 41].

Point-in-time joins. `get_historical_features(entity_df, features)` scans backward from each entity row's timestamp up to the feature view's TTL. Rows older than the earliest feature or outside the TTL get nulls. `created_timestamp` dedups ties, and `filter_by_created_timestamp=True` enforces availability-time correctness [7].

Streaming and tiling. `StreamFeatureView` supports time-windowed aggregations over Kafka, Kinesis or push sources (Local and Spark compute engines; Flink is listed as not yet supported) [42]. Feast's tiling docs make the same IR argument Chronon does: avg(tile1, tile2) is not (avg1 + avg2)/2. In their example, tiles [10, 20, 30] and [100] give a correct average of 40 but a naive one of 60. So tiles must store (sum, count) for avg, and analogous

sufficient statistics for std and var. The docs claim reuse of "90%+ of tiles between updates" [43].

Serving-time logging and monitoring. A `FeatureService` can declare `logging_config=LoggingConfig(destination=..., sample_rate=...)`. Logged serving data can then feed data-quality monitoring (`feast monitor run --source-type log`) next to batch baselines [44].

Labels as first-class objects (2026). Recent Feast adds a `LabelView` that stores mutable "judgments" (rewards, human reviews, LLM-judge scores) separately from append-only feature observations. It supports multiple labelers and conflict policies, and is composed with feature views in a `FeatureService` for PIT-correct training retrieval [45]. This matters for recommenders whose labels arrive late or get corrected, for example delayed conversions or moderation outcomes.

Inference patterns. Feast's docs list four ways to serve predictions: online inference with online features; precomputed predictions served as a feature; online inference with cached predictions that are recomputed when producers write; and inference without features [46]. The third pattern resembles the "score on write" caches used for some recommendation surfaces.

---

8. Tecton: managed real-time feature platform

Tecton's own blog was unreachable during research. This section uses its public example repository [15].

Tecton grew out of the Michelangelo lineage described above. Its public examples show the declarative surface for recommender and ads features [15]:

- **Batch feature views**, for example `user_distinct_ad_count_7d`.
- **Stream feature views** with built-in windowed `Aggregates`, for example ad-click counts over 1h, 24h and 72h windows fed by a push (ingest API) source. Flags `online=True`, `offline=True` materialize to both stores.
- **Continuous stream processing mode** (`StreamProcessingMode.CONTINUOUS`) for low-latency counters such as 1-minute and 5-minute keyword-click counts, with `AggregationLeadingEdge.LATEST_EVENT_TIME` controlling how the window's leading edge is defined.
- **Realtime (on-demand) feature views** that combine a `RequestSource` with precomputed feature tables at request time. The example computes cosine similarity between a request-time query embedding and a stored user embedding. This is the standard way to add "user-query affinity" to a ranker without materializing every pair.
- **Feature tables** for embeddings (`user_embeddings`, `ad_embeddings`) written by external jobs.
- **Feature services** that bundle features per model, for example `ad_ctr_feature_service`.
- **Two compute options**: Spark/Databricks, or "Rift", a Python-native engine for batch, streaming and real-time features without a Spark cluster [15].

The design lesson matches Chronon and Feast. Aggregation semantics (window, function, leading edge) are declared, not coded. That lets the platform compute them identically in backfill, streaming and serving.

---

9. Hopsworks: FTI pipelines and a transformation taxonomy

Hopsworks contributes a useful conceptual frame [14, 16, 47, 48]:

- **FTI architecture.** Every ML system splits into **F**eature pipelines (data to feature data), **T**raining pipelines (features to model) and **I**nference pipelines (features plus model to predictions plus prediction logs). These connect through a feature store with a row-oriented online store, a columnar offline store and a vector index, plus a model registry [47].
- **Transformation taxonomy.**
  - *Model-independent* transformations (aggregations, windowed counts, binning) produce reusable features and belong in the feature store.
  - *Model-dependent* transformations (scaling, one-hot, tokenization) are parameterized by a training set. They should not be stored, because storing them causes write amplification (every new write would force re-encoding of existing rows). They must be applied identically at training and inference.
  - *On-demand* transformations need request-time inputs, such as `time_since_last_transaction`. They run online, and the same function can be used to backfill them.
  - The taxonomy is meant to locate exactly where skew can arise [48].
- **Write path for consistency.** The Stream API writes to Kafka first. Hopsworks then syncs to the online store (RonDB, latest values only, idempotent upserts) and the offline store (Hudi with dedup). This upgrades Kafka's at-least-once delivery to effectively exactly-once and keeps online and offline consistent [16].
- **PIT training data with spines.** Feature views join feature groups updated at different cadences (hourly vs daily) with a transparent PIT join keyed on event_time. "Spine groups" define precisely which entities and timestamps appear on the left, which avoids duplicate-entity bias [14].

---

10. Featureform: the virtual feature store

Featureform describes itself as a **virtual feature store**. It "sits atop your existing infrastructure and orchestrates it to work like a traditional feature store". It manages definitions, versions ("variants"), lineage and immutability, and leaves computation to existing engines and storage [49]. Its README cites a taxonomy of "three common architectures" for feature stores. In this chapter's terms, the space looks like this:

Archetype	Examples	Who computes features	Pros	Cons
Literal store	early Feast	external pipelines; store only holds values	simple, infra-agnostic	skew risk stays with users
Physical platform	Chronon, Tecton, Feathr, Hopsworks, Michelangelo/Palette	platform engines (Spark/Flink/Rift) from declarative defs	strongest consistency guarantees; PITC backfills built in	heavier infra; DSL limits
Virtual store	Featureform	existing infra, orchestrated	reuse existing warehouse and streams	consistency depends on underlying engines

---

11. Netflix, DoorDash and others (secondary pointers)

These primary posts could not be fetched during research. They are listed for completeness, without detailed claims.

- **Netflix: "Distributed Time Travel for Feature Generation" (2016)** [50]. A snapshot-based approach: periodically snapshot the online services' state for sampled contexts, then regenerate features "as of" those snapshots, so new features can be evaluated on historical contexts. It sits between log-and-wait and full replication.
- **DoorDash Fabricator (2022) and Riviera (2021)** [51, 52]. Declarative batch and real-time (Flink-based) feature frameworks.
- **DoorDash gigascale feature store with Redis (2020)** [53]. Covers Redis benchmarking, binary serialization and compression for very large online feature sets.
- **LinkedIn near-real-time features (2022)** [54]. Nearline feature computation for personalization, also cited in the Feathr talk [12].
- **Twitter's feature store talk (2019)** [55], Condé Nast [56], Monzo [57], QuintoAndar Butterfree [58] and Constructor.io [59] are further industrial case studies.

---

12. Cross-cutting design patterns

12.1 Serving-time feature logging ("log what you serve")

Logging the exact feature vector used at inference is the strongest skew defense available. Google's *Rules of ML* recommends saving the features used at serving time and piping them to training (Rule #29; source not re-fetched for this chapter) [60]. The public systems implement it in different ways:

- **Twitter** scribes `DataRecords` with hashed feature IDs [18].
- **Chronon** logs fetch requests and responses, uses them for consistency metrics, and can *bootstrap* training rows directly from logs [31, 32].
- **Feast** offers `LoggingConfig` per feature service for monitoring [44].
- **Hopsworks** puts "prediction logs" in the inference pipeline's output contract [47].

A practical design that combines these patterns:



Trade-offs of pure logging. You cannot add a new feature to past data, which needs a PITC backfill (hence Chronon's bootstrap mix of logged and backfilled columns). Logs for thousands of features times every scored candidate are enormous. Sampling is needed, and Chronon samples by key hash so that whole key histories are kept [31]. Logging also captures only what was scored, so retrieval-stage counterfactuals are missing.

12.2 Streaming feature computation

- **Ingestion backbone.** A unified, deduplicated user-action stream such as Twitter UUA [19] or Kafka at Airbnb, Stripe and LinkedIn. Feature jobs should consume a canonical event schema rather than raw client logs.
- **Engines.** Storm/Heron (Twitter, historical) [17], Samza (LinkedIn) [13], Spark Streaming and Flink (Chronon) [30], Spark (Feast's stream processor) [42], Rift (Tecton) [15].
- **State model.** Choose decayed counters (O(1) state, Twitter [17]), raw-event storage with read-side aggregation (simple, but O(events) per read), or tiles of IRs (write-side pre-aggregation; Chronon and Feast [29, 43]).
- **Mergeability.** Only algebraic aggregates (sum, count, min, max, last, and avg via sum and count) tile exactly. Holistic ones (percentiles, distinct counts) need sketches such as KLL or HLL. Chronon exposes a memory/accuracy knob for approximate aggregations [28].
- **Hot keys.** Popular items and large merchants dominate traffic. Tiling and write-side aggregation are the standard fix, and Chronon cites "very hot key entities" as a primary motivation [29].
- **Skew from lag.** Streaming lag (ms to seconds) and batch landing lag (hours) are inherent. Either model them in the backfill (Chronon's proposed write-lag shift [31], Feast's created-timestamp filter [7]) or log what was served.

12.3 Point-in-time correctness checklist

1. Every source carries an event time, and if possible an availability or ingestion time.
2. Backfills reproduce the online freshness model (midnight snapshot vs real-time).
3. Windows are defined identically in both engines, including inclusive/exclusive bounds. Chronon uses `[ts - W, ts)` [1].
4. Mutable entities use snapshots plus CDC mutations, not the current table state [1, 26].
5. TTL and staleness semantics match. Feast's TTL is relative to each row [7].

6. Labels are joined with an explicit attribution window, and label corrections are versioned (Feast `LabelView` [45]).
7. Spines define the training population explicitly so it is not inferred from label tables [14].

12.4 Online store engineering for recommenders

- Ranking is read-heavy fan-out: one request reads features for 1 user plus N candidates (N in the hundreds to thousands). Batch multi-get, co-locate features for the same entity, and cache item features in-process. Venice's Da Vinci in-process client is one example [13].
- Encode compactly: hashed feature IDs (Twitter [18]), Avro (Chronon [31]), binary serialization and compression (DoorDash [53]).
- Store embeddings as feature tables and compute pairwise similarities on demand (Tecton [15], Feathr derived features [9]) instead of materializing user x item.

---

13. Comparison of public feature platforms

System	Org / year public	Definition API	Batch engine	Streaming engine	Online store	PITC backfill	Serving-log support	Notable numbers (public)
Aggregation framework	Twitter, 2023 (open-sourced code)	Scala `AggregateGroup`	summingbird-scalding	summingbird-storm/Heron	Manhattan (batch), memcached (RT)	day-boundary ("most recent before date")	features scribed by hashed ID	~6000 features hydrated for For You ranking [5]
Michelangelo / Palette	Uber, 2017 / 2019	config + feature DSL	Spark/Hive	Kafka + Samza (later Flink)	Cassandra	yes (batch + logged streaming)	streaming features logged to HDFS	~10k features in store (2017, approx.) [22]
Zipline / Chronon	Airbnb (+Stripe), 2018 / OSS 2024	Python `GroupBy` / `Join`	Spark (sawtooth tiling)	Spark Streaming or Flink (tiled)	pluggable KV	yes, SNAPSHOT or TEMPORAL	query logs, consistency metrics, bootstrap	12h window: 432k events vs 12-13 tiles; Stripe -33% latency [29]
Feathr	LinkedIn, 2017 internal / OSS 2022	Python anchors + derived	Spark (bloom filters, salted joins)	Kafka/Event Hubs streaming	Redis, Cosmos DB, Aerospike; Venice at LinkedIn	yes (per-event "closest from past")	(not documented)	hundreds of models, thousands of features, PB scale; up to 50% runtime gain [12]
Feast	Gojek + Google Cloud 2019; OSS	Python `FeatureView`	offline store / Spark / DuckDB etc.	Spark stream processor; push sources	20+ backends	yes (TTL, created-ts filter)	`LoggingConfig` + monitoring	tile reuse "90%+" claim [43]
Tecton	commercial	Python decorators	Spark or Rift	Spark or Rift (continuous mode)	managed	yes	(not covered in sources)	--

System	Org / year public	Definition API	Batch engine	Streaming engine	Online store	PITC backfill	Serving-log support	Notable numbers (public)
Hopsworks	commercial / OSS core	Python feature groups / views	Spark, Python	Kafka + OnlineFS; Flink/Spark	RonDB	yes (event_time + spines)	prediction logs in FTI	--
Featureform	OSS	Python	orchestrates existing	orchestrates existing	existing	via underlying infra	--	--

---

Key takeaways for system designers

1. **Declare features once, compile everywhere.** Every successful platform (Twitter's AggregateGroup, Michelangelo's DSL, Chronon's GroupBy, Feathr's anchors, Feast and Tecton feature views) makes a single declarative definition the source of truth for backfill, streaming and serving. Hand-written duplicates are the main source of skew.
2. **Point-in-time correctness means "what would online have served at ts".** It does not mean "the true value at ts". Model batch landing lag and streaming lag explicitly, using Chronon's SNAPSHOT vs TEMPORAL and Feast's created-timestamp filter.
3. **Recommender training sets break generic PITC joins.** When labels are as numerous as impressions or searches, use specialized algorithms (sawtooth windows with hop-aligned shared tails and exact heads) and broadcast or bloom-filter tricks, not naive range joins.
4. **Pre-aggregate on the write path for hot keys.** Tiles of intermediate representations (sum and count, not avg; sketches for percentiles) turn O(events) reads into O(tiles). Decayed counters give O(1) state when hard windows are not required.
5. **Log what you serve.** Serving-time feature logs are the ground truth for consistency measurement and the cheapest source of training data for existing features. Use PITC backfills only for new features, and stitch the two together (Chronon bootstrap). Sample by key, not by request.
6. **Classify transformations.** Store model-independent features. Bundle model-dependent transforms with the model. Run on-demand features at request time with the same code used for backfill.
7. **Design the online store for fan-out reads.** Batch multi-gets, compact encodings (hashed IDs, Avro, compression), in-process caches for item features, and on-demand similarity over stored embeddings.
8. **Measure skew continuously.** Mismatch, missing and extra rates and value-distribution deltas per feature, computed daily from logs (Chronon's consistency job), catch silent breakages that offline AUC never will.

References

1. Airbnb / Chronon project, "Window Tiling Algorithm" (docs/source/window_tiling.md), Chronon repository, 2023-2024. https://github.com/airbnb/chronon/blob/main/docs/so

urce/window_tiling.md
2. Airbnb / Chronon project, "What is Chronon?" (Introduction) and README, Chronon repository, 2024. https://github.com/airbnb/chronon
3. Airbnb / Chronon project, "Benefits of Chronon over other approaches" (docs/source/getting_started/Benefits.md), 2024. https://github.com/airbnb/chronon/blob/main/docs/source/getting_started/Benefits.md
4. Twitter/X, "Aggregation framework" README (timelines/data_processing/ml_util/aggregation_framework), the-algorithm repository, 2023. https://github.com/twitter/the-algorithm/tree/main/timelines/data_processing/ml_util/aggregation_framework
5. Twitter/X, "Home Mixer" README, the-algorithm repository, 2023. https://github.com/twitter/the-algorithm/blob/main/home-mixer/README.md
6. Twitter/X, "Follow Recommendations Service" README, the-algorithm repository, 2023. https://github.com/twitter/the-algorithm/blob/main/follow-recommendations-service/README.md
7. Feast project, "Point-in-time joins" (docs/getting-started/concepts/point-in-time-joins.md), 2026. https://github.com/feast-dev/feast/blob/master/docs/getting-started/concepts/point-in-time-joins.md
8. Feathr project (LinkedIn / LF AI & Data), Feathr documentation README, 2022-2023. https://github.com/feathr-ai/feathr/blob/main/docs/README.md
9. Feathr project, "Concepts for Beginners" (docs/concepts/feathr-concepts-for-beginners.md), 2022. https://github.com/feathr-ai/feathr/blob/main/docs/concepts/feathr-concepts-for-beginners.md
10. Feathr project, "Point-in-time Correctness and Point-in-time Join" (docs/concepts/point-in-time-join.md), 2022. https://github.com/feathr-ai/feathr/blob/main/docs/concepts/point-in-time-join.md
11. Feathr project, "Comparison between Feature Stores" (docs/concepts/feature-store-comparison.md), 2022. https://github.com/feathr-ai/feathr/blob/main/docs/concepts/feature-store-comparison.md
12. Feathr project (LinkedIn and Microsoft), "Feathr Community Talk - An Enterprise-Grade High Performance Feature Store" and "Feathr Feature Store Talk" slide decks (docs/talks), 2022. https://github.com/feathr-ai/feathr/tree/main/docs/talks
13. LinkedIn, "Venice: Derived Data Platform for Planet-Scale Workloads" (docs/README.md), Venice repository, 2022-2026. https://github.com/linkedin/venice
14. Hopsworks (Logical Clocks), "Offline API" / point-in-time correct training data and spine groups (docs/concepts/fs/feature_view/offline_api.md), 2026. https://github.com/logicalclocks/logicalclocks.github.io/blob/main/docs/concepts/fs/feature_view/offline_api.md
15. Tecton, "Tecton Examples Repository" (rift/ads and rift/fraud feature definitions), 2024-2025. https://github.com/tecton-ai/tecton-sample-repo
16. Hopsworks, "Write APIs" (docs/concepts/fs/feature_group/write_apis.md) and "Features and Feature Groups" (fg_overview.md), 2026. https://github.com/logicalclocks/logicalclocks.github.io/blob/main/docs/concepts/fs/feature_group/write_apis.md
17. Twitter/X, Aggregation framework docs: aggregation.rst, batch.rst, real-time.rst, joining.rst, the-algorithm repository, 2023. https://github.com/twitter/the-algorithm/tree/main/timelines/data_processing/ml_util/aggregation_framework/docs
18. Twitter/X, "Prediction Features" README (src/scala/com/twitter/timelines/prediction/features), the-algorithm repository, 2023. h

ttps://github.com/twitter/the-algorithm/blob/main/src/scala/com/twitter/timelines/prediction/features/README.md
19. Twitter/X, "Unified User Actions (UUA)" README, the-algorithm repository, 2023. https://github.com/twitter/the-algorithm/blob/main/unified_user_actions/README.md
20. Twitter/X, "User Signal Service" README, the-algorithm repository, 2023. https://github.com/twitter/the-algorithm/blob/main/user-signal-service/README.md
21. Twitter/X, "Graph Feature Service" README, the-algorithm repository, 2023. https://github.com/twitter/the-algorithm/blob/main/graph-feature-service/README.md
22. J. Hermann, M. Del Balso, "Meet Michelangelo: Uber's Machine Learning Platform", Uber Engineering, 2017 (not re-fetched; content as widely cited). https://eng.uber.com/michelangelo-machine-learning-platform/
23. Uber, "Michelangelo Palette: A Feature Engineering Platform at Uber", InfoQ presentation, 2019 (not re-fetched). https://www.infoq.com/presentations/michelangelo-palette-uber/
24. Airbnb, "Zipline: Airbnb's Machine Learning Data Management Platform", talk (YouTube), 2018 (not re-fetched). https://www.youtube.com/watch?v=Tg5VEMEsC-0
25. Airbnb, "Zipline - A Declarative Feature Engineering Framework", talk (YouTube), 2020 (not re-fetched). https://www.youtube.com/watch?v=LjcKCm0G_OY
26. Airbnb / Chronon project, "Source" (docs/source/authoring_features/Source.md), 2024. https://github.com/airbnb/chronon/blob/main/docs/source/authoring_features/Source.md
27. Airbnb / Chronon project, "Join" (docs/source/authoring_features/Join.md), 2024. https://github.com/airbnb/chronon/blob/main/docs/source/authoring_features/Join.md
28. Airbnb / Chronon project, "GroupBy" (docs/source/authoring_features/GroupBy.md), 2024. https://github.com/airbnb/chronon/blob/main/docs/source/authoring_features/GroupBy.md
29. Airbnb / Stripe, Chronon project, "The Tiled Architecture" (docs/source/Tiled_Architecture.md), 2024-2025. https://github.com/airbnb/chronon/blob/main/docs/source/Tiled_Architecture.md
30. Airbnb / Chronon project, "Chronon on Flink" (docs/source/setup/Flink.md), 2024-2025. https://github.com/airbnb/chronon/blob/main/docs/source/setup/Flink.md
31. Airbnb / Chronon project, "Online Offline Consistency" (docs/source/test_deploy_serve/Online_Offline_Consistency.md), 2024. https://github.com/airbnb/chronon/blob/main/docs/source/test_deploy_serve/Online_Offline_Consistency.md
32. Airbnb / Chronon project, "Bootstrap" (docs/source/authoring_features/Bootstrap.md), 2024. https://github.com/airbnb/chronon/blob/main/docs/source/authoring_features/Bootstrap.md
33. Airbnb / Chronon project, "Chaining Features" (docs/source/authoring_features/ChainingFeatures.md), 2024. https://github.com/airbnb/chronon/blob/main/docs/source/authoring_features/ChainingFeatures.md
34. Gojek / Google Cloud, "Introducing Feast: An Open Source Feature Store for Machine Learning", Google Cloud Blog, 2019 (not re-fetched). https://cloud.google.com/blog/products/ai-machine-learning/introducing-feast-an-open-source-feature-store-for-machine-learning
35. Feast project, README, 2026. https://github.com/feast-dev/feast
36. Feast project, "Push vs Pull Model" (docs/getting-started/architecture/push-vs-pull-model.md), 2026. https://github.com/feast-dev/feast/blob/master/docs/getting-started/

architecture/push-vs-pull-model.md
37. Feast project, "Writing Data to Feast" (docs/getting-started/architecture/write-patterns.md), 2026. https://github.com/feast-dev/feast/blob/master/docs/getting-started/architecture/write-patterns.md
38. Feast project, "Feature Transformation" (docs/getting-started/architecture/feature-transformation.md), 2026. https://github.com/feast-dev/feast/blob/master/docs/getting-started/architecture/feature-transformation.md
39. Feast project, "Architecture Overview" (docs/getting-started/architecture/overview.md), 2026. https://github.com/feast-dev/feast/blob/master/docs/getting-started/architecture/overview.md
40. Feast project, online store reference index (docs/reference/online-stores), 2026. https://github.com/feast-dev/feast/tree/master/docs/reference/online-stores
41. Feast project, "Chronon online store (contrib)" (docs/reference/online-stores/chronon.md), 2026. https://github.com/feast-dev/feast/blob/master/docs/reference/online-stores/chronon.md
42. Feast project, "Stream Feature View" and "Stream Processor" docs, 2026. https://github.com/feast-dev/feast/blob/master/docs/getting-started/concepts/stream-feature-view.md
43. Feast project, "Tiling with Intermediate Representations in Feast" (docs/getting-started/concepts/tiling.md), 2026. https://github.com/feast-dev/feast/blob/master/docs/getting-started/concepts/tiling.md
44. Feast project, "Feature monitoring" how-to (feature service logging and log metrics), 2026. https://github.com/feast-dev/feast/blob/master/docs/how-to-guides/feature-monitoring.md
45. Feast project, "Label View" (docs/getting-started/concepts/label-view.md), 2026. https://github.com/feast-dev/feast/blob/master/docs/getting-started/concepts/label-view.md
46. Feast project, "Feature Serving and Model Inference" (docs/getting-started/architecture/model-inference.md), 2026. https://github.com/feast-dev/feast/blob/master/docs/getting-started/architecture/model-inference.md
47. Hopsworks, "FTI Pipeline Architecture" (docs/concepts/fti.md), 2026. https://github.com/logicalclocks/logicalclocks.github.io/blob/main/docs/concepts/fti.md
48. Hopsworks, "Data Transformations" taxonomy (docs/concepts/mlops/data_transformations.md), 2025-2026. https://github.com/logicalclocks/logicalclocks.github.io/blob/main/docs/concepts/mlops/data_transformations.md
49. Featureform, "What is Featureform?" README, 2023-2025. https://github.com/featureform/featureform
50. Netflix Technology Blog, "Distributed Time Travel for Feature Generation", 2016 (not re-fetched). https://netflixtechblog.com/distributed-time-travel-for-feature-generation-389cccdd3907
51. DoorDash Engineering, "Introducing Fabricator: A Declarative Feature Engineering Framework", 2022 (not re-fetched). https://doordash.engineering/2022/01/11/introducing-fabricator-a-declarative-feature-engineering-framework/
52. DoorDash Engineering, "Building Riviera: A Declarative Real-Time Feature Engineering Framework", 2021 (not re-fetched). https://doordash.engineering/2021/03/04/building-a-declarative-real-time-feature-engineering-framework/

53. DoorDash Engineering, "Building a Scalable ML Feature Store with Redis, Binary Serialization, and Compression", 2020 (not re-fetched). https://doordash.engineering/2020/11/19/building-a-gigascale-ml-feature-store-with-redis/
54. LinkedIn Engineering, "Near real-time features for near real-time personalization", 2022 (not re-fetched). https://engineering.linkedin.com/blog/2022/near-real-time-features-for-near-real-time-personalization
55. Twitter, "The Architecture That Powers Twitter's Feature Store", talk (YouTube), 2019 (not re-fetched). https://www.youtube.com/watch?v=UNailXoiIrY
56. Conde Nast Technology, "Accelerating Machine Learning with the Feature Store Service", 2019 (not re-fetched). https://technology.condenast.com/story/accelerating-machine-learning-with-the-feature-store-service
57. N. Lathia (Monzo), "Building a Feature Store", 2020 (not re-fetched). https://nlathia.github.io/2020/12/Building-a-feature-store.html
58. QuintoAndar, "Butterfree: A Spark-based Framework for Feature Store Building", 2020 (not re-fetched). https://medium.com/quintoandar-tech-blog/butterfree-a-spark-based-framework-for-feature-store-building-48c3640522c7
59. Constructor.io, "Feature Store Design at Constructor", 2023 (not re-fetched). https://medium.com/constructor-engineering/feature-store-design-at-constructor-330b65f64b18
60. M. Zinkevich (Google), "Rules of Machine Learning: Best Practices for ML Engineering", Google Developers (not re-fetched). https://developers.google.com/machine-learning/guides/rules-of-ml
61. E. Yan et al., "applied-ml: Papers and tech blogs by companies sharing their work on data science and machine learning in production" (Feature Stores section; used to locate and verify URLs for refs 22-25, 34, 50-59). https://github.com/eugeneyan/applied-ml

09. Training Infrastructure at Scale: Sparse Embeddings, Sharding, Parameter Servers and Online Learning

Overview

Industrial recommenders differ from most other deep learning workloads in one important way. Nearly all of their parameters sit in **sparse embedding tables** keyed by categorical IDs (users, items, creators, queries, ad IDs, crossed features). Only a small fraction sit in the dense neural network. In the DLRM family of models [1], each categorical feature looks up one or more rows in a table, pools them (sum or mean), and feeds the result into feature interaction and an MLP. Production tables have billions of rows, and their total size runs from hundreds of gigabytes to tens or hundreds of terabytes. Meta's ZionEX/Neo paper describes models of up to 12 trillion parameters [3]. Kuaishou's PERSIA reports a system that scales to 100 trillion parameters [4]. Alibaba's DeepRec claims support for "over ten trillion parameters" [10].

This shape of workload produces a set of system problems that look quite different from training a large language model:

- **Memory capacity, not FLOPs, is the binding constraint.** The tables do not fit on one accelerator, and often not on one host.
- **Embedding lookups are memory-bandwidth and network bound.** Each lookup is a small random read. Each step needs an all-to-all exchange of pooled embeddings.
- **The ID space is open and keeps changing.** New users, items and videos arrive every second, so a fixed-size table either collides (hashing) or has to grow (dynamic hash tables).
- **Freshness matters.** User interest and the item corpus drift within hours, which pushes systems toward online or continual training and toward frequent parameter sync to serving.
- **Data access is very skewed.** A small fraction of IDs accounts for most lookups, which makes caching, tiered storage and frequency-based admission effective.

This chapter covers the main publicly documented designs for these problems:

- Meta's DLRM, Neo/ZionEX, TorchRec and FBGEMM.
- ByteDance's Monolith.
- NVIDIA's HugeCTR, Merlin and HierarchicalKV.
- Parameter-server systems: Alibaba XDL and DeepRec, Kuaishou PERSIA, HET, and 4Paradigm OpenEmbedding.
- TensorFlow Recommenders-Addons, which Tencent and others contribute to.
- Checkpointing for continual training.
- Embedding compression: the hashing trick, hash embeddings, QR compositional embeddings, mixed-dimension embeddings, TT-Rec, ROBE-style and DHE-style

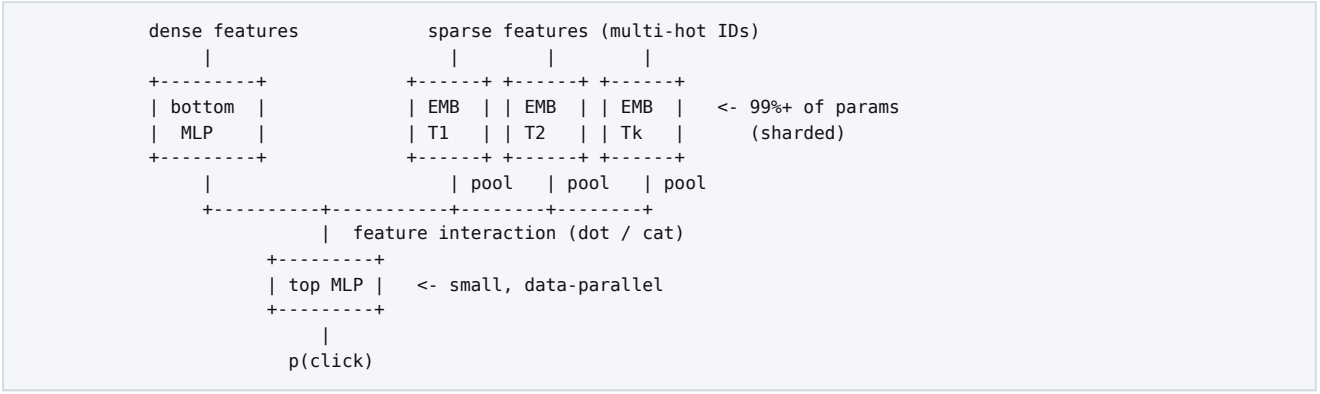
methods.
- Hardware directions such as Google's TPU SparseCore.

A note on sourcing: the research for this chapter hit a web-search budget limit and DNS failures on several sites (arxiv.org, pytorch.org, developer.nvidia.com and engineering.fb.com could not be fetched). The facts here come from the project repositories that could be read (GitHub READMEs and docs) and from the primary papers cited by ID. Where a number could not be re-verified against a fetched page, it is marked approximate.

1. The Shape of the Problem: DLRM as Reference Workload

1.1 Model anatomy

DLRM [1] is the reference model that most training-infrastructure papers assume. Dense features go through a bottom MLP. Each sparse feature indexes an embedding table. A pairwise interaction operator (dot product, or concatenation, or sum) combines the dense output with the pooled embeddings, and a top MLP predicts a click probability [11]. The open-source DLRM code supports synchronous distributed training over gloo, NCCL and MPI backends. It also includes MLPerf benchmark tooling with AUC-based early stopping [11].



1.2 Why it is hard to train efficiently

Meta's study "Understanding Training Efficiency of Deep Learning Recommendation Models at Scale" [12] characterized production DLRM training. Its main points:

- Embedding operators are memory-bound and dominated by communication.
- The large embedding tables and the varied sparse-feature profiles across models make one hardware configuration a poor fit for all models.
- In the CPU parameter-server era, trainers spent much of each step waiting on embedding pulls and pushes.

The paper argued for co-designing hardware and software around embedding traffic. That argument led to the Zion and ZionEX GPU platforms [3].

Two kinds of parallelism are the standard solution:

- **Model parallelism for embeddings.** Tables, or slices of them, are placed on different devices. Each device does the lookups for the IDs it owns.
- **Data parallelism for the dense MLPs.** The network is replicated and gradients are all-reduced.

The two worlds meet at an **all-to-all** collective. In the forward pass it moves pooled embeddings from the devices that own the tables to the devices that own the samples. In the backward pass it moves the gradients the other way.

```
forward:  sample-sharded batch -> [input dist: all-to-all of IDs]
          -> table-sharded lookup + pooling
          -> [output dist: all-to-all of pooled embeddings]
          -> data-parallel interaction + MLP on each GPU
backward: MLP grads -> all-reduce (dense)
          pooled-emb grads -> all-to-all back to owners
          -> fused sparse optimizer update in place on owner GPU
```

2. Meta: Neo, ZionEX, TorchRec and FBGEMM

2.1 Problem

Before 2020, Meta trained recommendation models asynchronously on CPU clusters with parameter servers. That approach scaled in capacity, but it had two drawbacks. Asynchrony hurt model quality and reproducibility. Throughput was also limited by network traffic to the parameter servers [12]. The goal of Neo/ZionEX [3] was synchronous, GPU-based training of models with trillions of parameters.

2.2 Architecture

ZionEX hardware [3]:

- Each node has 8 GPUs (A100-class in the paper) connected internally by NVLink.
- Each node also has a dedicated RDMA over Converged Ethernet (RoCE) **scale-out network**: several high-bandwidth NICs per node, so every GPU has its own path for inter-node all-to-all traffic.
- The paper's evaluation covers clusters of up to 16 nodes (128 GPUs).
- The emphasis is on all-to-all bandwidth, because embedding exchange dominates the step time.

Neo software [3] introduced **4D parallelism** for embeddings, chosen per table:

Sharding scheme	What is split	When it fits
Table-wise (TW)	Whole tables placed on different GPUs	Many small/medium tables; low comm
Row-wise (RW)	Rows of one table split across GPUs	Huge tables beyond one GPU's HBM
Column-wise (CW)	Embedding dimension split across GPUs	Wide tables; balances load per GPU
Data-parallel (DP)	Table replicated everywhere	Tiny tables; avoids all-to-all entirely

Neo also contributed the following [3]:

- **Hybrid kernel fusion.** All tables on a device are batched into one fused kernel, and the sparse optimizer update is fused into the backward pass, so no full gradient tensor

is ever materialized.
- **A sharding planner** that balances compute, memory and communication cost across devices.
- **Software-managed caching** of embedding rows across HBM, host DRAM and UVM.
- **Quantized collectives.** Embedding all-to-all traffic is sent in reduced precision (FP16/BF16) to save network bandwidth, while model quality is preserved.

The paper reports training models of up to **12 trillion parameters**. It also reports roughly order-of-magnitude (approximately up to 40x, per the paper's headline claim) time-to-solution speedups over Meta's earlier CPU-based asynchronous training [3].

2.3 TorchRec and FBGEMM_GPU

TorchRec [2][13] is the open-source productionization of these ideas as a PyTorch domain library. According to its README [13], it provides:

- Sharders for **data-parallel, table-wise, row-wise, table-wise-row-wise, column-wise and table-wise-column-wise** sharding.
- A **planner** that automatically produces an optimized sharding plan.
- RecSys kernels from **FBGEMM** [14].
- A **train pipeline** that overlaps data loading, host-to-device copies, input distribution (all-to-all of IDs) and forward/backward compute.
- Reduced-precision training and inference, with C++ inference export.

The project states that TorchRec "powers many production RecSys models at Meta." Its citation abstract describes how Meta moved from CPU asynchronous training to accelerator-based full-sync training with it [13]. The README also links later work on **"Toward 100TB model with Embedding Offloading"** [13], which keeps hot rows in HBM and offloads cold rows to host or SSD tiers.

FBGEMM_GPU supplies the kernels [14]. Its **table-batched embedding (TBE)** operator runs many embedding bags in one launch, and it also supports quantized embeddings and data-layout transforms. Two ideas carry over to any design here:

1. **Table batching and fused optimizers.** Thousands of small kernel launches become one. The optimizer state (for example rowwise Adagrad) is updated in the same pass as the backward lookup.
2. **Pipelining across stages.** The ID all-to-all for batch i+1 overlaps with the dense compute for batch i.

```
TorchRec TrainPipelineSparseDist (conceptual)
 time ->
 batch i   : [H2D copy][ID a2a][lookup][emb a2a][fwd/bwd MLP][emb grad a2a+update]
 batch i+1 :           [H2D copy][ID a2a]......(overlapped with batch i compute)
 batch i+2 :                     [H2D copy]....
```

2.4 Lessons

- **Synchronous training at scale is achievable.** It is preferable for reproducibility, but it needs dedicated all-to-all bandwidth [3].

- **Use a heterogeneous sharding plan.** No single scheme is best. The planner choosing per table (TW for most tables, RW for the huge ones, DP for the tiny ones) is where much of the performance comes from [3][13].
- **Kernel fusion and fused optimizers** avoid materializing dense gradients for sparse tables.

3. ByteDance Monolith: Collisionless Tables and Real-Time Training

3.1 Problem

ByteDance's feed products (Douyin/TikTok and others) have three properties that clash with standard TensorFlow training:

- The ID space is **non-stationary and unbounded**.
- Feature popularity is **long-tailed**.
- **Concept drift** happens within hours.

TensorFlow's built-in variables are fixed-size dense tensors. Fitting unbounded IDs into them means hashing, and hashing collisions degrade quality. Daily batch retraining is also too slow to keep up with drift [5][15].

3.2 Collisionless embedding table

Monolith [5] replaces fixed `tf.Variable` tables with a **key-value hash table on the parameter servers**, built on **Cuckoo hashing**. Cuckoo hashing uses two hash functions and two arrays. An insert that hits an occupied slot evicts the occupant to its other location, and this repeats until every key has a slot. The result is guaranteed O(1) lookup and no collisions between distinct IDs [5]. The repository describes the goal directly: "each ID feature gets its own unique representation" [15].

Unbounded growth is controlled with two mechanisms [5]:

- **Frequency filtering (admission).** An ID gets an embedding only after it has appeared some threshold number of times. A probabilistic filter keeps the memory needed to count rare IDs low. Long-tail IDs seen once or twice add memory cost without adding quality.
- **Expirable embeddings (eviction).** Each feature has an expiry time. IDs that go inactive, such as stale users or deleted videos, are evicted.

3.3 Real-time (online) training pipeline

```
user actions (clicks, plays, ...) ---> Kafka (action log)
serving features ------------------>  Kafka (feature log)
                                          |
                                   Flink online joiner
                    (join by request key, in-memory cache + on-disk KV
                     for late actions; negative sampling)
                                          |
                    +---------------------+------------------+
                    |                                        |
          Kafka (training examples)                  HDFS (batch dumps)
                    |                                        |
           online training workers  <--- same model --->  batch training
                    |
           training parameter servers (Cuckoo hash tables)
                    |  periodic sync: sparse incremental (minutes), dense (slower)
                    v
           serving parameter servers -> online serving
```

How the loop works, per the paper [5]:

1. **One framework, two modes.** Batch training runs on historical HDFS data to bootstrap a model. Online training then consumes the streaming Kafka example queue indefinitely.
2. **Online joiner.** A Flink job joins user actions with the features logged at serving time. Features are cached because the corresponding action can arrive minutes later. Negative sampling reduces the share of negatives, and a **log-odds correction** at serving time compensates for the changed label distribution.
3. **Parameter synchronization.** Moving terabytes of parameters to serving every minute is impractical. Monolith therefore makes the update asymmetric:
   - **Sparse parameters** are synced **incrementally**, sending only the IDs that changed since the last sync, at minute-level intervals.
   - **Dense parameters** change slowly under momentum-style optimizers, so they are synced much less often (the paper describes a daily-scale cadence).
4. **Fault tolerance.** The paper studies how snapshot frequency affects model quality. It concludes that **daily snapshots** of parameter servers lose very little quality on PS failure, because one shard holds only a small fraction of IDs and the model recovers quickly with online training. Infrequent snapshots save large amounts of compute and I/O [5].

3.4 Results and lessons

The Monolith paper reports [5]:

- **Collisionless** embeddings beat hashed (colliding) baselines on both MovieLens-style public data and internal production data.
- **Shorter parameter-sync intervals** (moving from hours toward ~30 minutes in their experiments) give consistently higher online AUC. Real-time training measurably beats batch training for ads.

Lessons:

- Freshness is a model-quality feature, not only an operations concern.
- The **asymmetric sync** of sparse and dense parameters is the key trick that makes minute-level updates affordable.
- Admission and eviction policies matter as much as the table data structure.

The Monolith GitHub repository is now archived and read-only. It is built on TensorFlow with Bazel [15]. The same ideas carried over to BytePlus's commercial recommendation offering.

4. Parameter-Server Lineage: XDL, DeepRec, PERSIA, HET, OpenEmbedding, TFRA

4.1 Alibaba XDL and DeepRec

XDL (X-DeepLearning) [16] is Alibaba Mama's framework for high-dimensional sparse data in advertising, recommendation and search. It came with:

- the **Blaze** serving engine,
- the **TDM** (tree-based deep matching) retrieval engine.

The XDL paper (DLP-KDD 2019) describes a "bridge" design. XDL handles the high-dimensional sparse part on a distributed parameter server, and the dense sub-network runs on a pluggable backend (TensorFlow/MXNet).

Version 1.2 added automatic global parameter allocation, request merging, and removal of PS compute, storage and communication hotspots. It reports 50–100% gains for large-batch, low-concurrency workloads [16]. Its **streaming-training features** are the same set that later frameworks standardized on [16]:

- **feature admission**,
- **feature eviction**,
- **incremental model export**,
- **feature frequency statistics**.

DeepRec [10] is Alibaba's successor: a heavily modified TensorFlow. Its features include:

- **EmbeddingVariable**, a dynamic hash-backed embedding, in dynamic-dimension, adaptive and multi-hash variants, plus group embedding.
- **Multi-tier hybrid embedding storage** across HBM, DRAM, PMem and SSD.
- **Delta (incremental) checkpoint** loading and export.
- Two training modes:
  - **Asynchronous PS training** over gRPC+Seastar, FuseRecv and StarServer.
  - **Synchronous collective training** via HybridBackend and NVIDIA's Sparse Operation Kit.
- Graph and runtime optimizations: SmartStage pipelining, AutoGraphFusion, MicroBatch, a graph-aware memory allocator, and CUDA Graph support.

DeepRec claims support for "trillion samples and over ten trillion parameters" [10].

4.2 Kuaishou PERSIA: hybrid synchronous/asynchronous training

Problem. Fully synchronous training is slow, because embedding communication is large. Fully asynchronous training hurts the accuracy of the dense network. PERSIA [4][17] (Kuaishou with ETH Zurich, KDD 2022) observes that the two parts behave differently:

- The **embedding layer** is huge, but each step touches very few rows. Staleness on sparse rows is therefore rare and harmless.
- The **dense network** is small, but every step updates all of it, so staleness there does hurt.

Design. PERSIA trains the embeddings **asynchronously** and the dense NN **synchronously** (all-reduce, via the Bagua communication library). The implementation is in Rust and Python. Its components:

```
data loaders --> embedding workers <--> embedding parameter servers (CPU, huge)
                       |   (async: get/put embeddings, stale-tolerant)
                       v
               NN workers (GPU) -- sync all-reduce of dense grads (Bagua)
```

Scale. PERSIA reports training models of up to **100 trillion parameters**, described at the time as the largest recommendation model reported [17]. It ran in production at Kuaishou for applications with roughly hundreds of millions of daily active users. The paper reports several-fold throughput gains (approximately up to 7x) over contemporary systems at similar accuracy [4]. The project README now notes that PERSIA is unmaintained after a reorganization [17].

Lesson. Matching the consistency model to each parameter's access pattern gives most of the speed of asynchrony without most of its accuracy cost. TFRA uses the same idea (section 4.4).

4.3 HET: cache-enabled embedding training

HET [18] (Peking University, VLDB 2022) targets the PS communication bottleneck by keeping an **embedding cache on each worker**. Because accesses are skewed, hot embeddings are cached locally. A **bounded-staleness consistency** protocol, with per-embedding clocks, decides when a cached copy must be refreshed or written back. This cuts PS traffic sharply while keeping convergence close to synchronous training. The design point to note is that caching combined with **explicit staleness bounds** is a principled middle ground between fully synchronous and fully asynchronous training.

4.4 TensorFlow Recommenders-Addons (TFRA)

TFRA [19] is the SIG-Recommenders project that adds a trainable **dynamic embedding** to TensorFlow. It is a key-value structure with no hash conflicts, and it works with native TF optimizers, initializers, checkpoints and SavedModel. Its storage backends are pluggable:

- **cuckoohash_map** (from CMU) on CPU,
- NVIDIA **HierarchicalKV** on GPU,
- **Redis**, for distributed inference.

TFRA also supports **half-synchronous training with Horovod**: synchronous for dense weights, asynchronous for sparse weights. Listed contributors and supporters include Tencent, Alibaba, VIP.com and BOSS Zhipin, plus the NVIDIA Merlin team [19].

4.5 4Paradigm OpenEmbedding

OpenEmbedding [20] keeps embedding layers on a parameter server and trains dense layers with Horovod. It exports standalone SavedModels for TF Serving. It also has an experimental **persistent-memory (PMem) PS**, where PMem acts as a lightweight checkpointing mechanism (see the associated ICDE 2023 paper). It reports 3–8x speedups over pure all-reduce (Horovod) on a single 8-GPU machine for sparse models [20].

5. NVIDIA HugeCTR, Merlin and HierarchicalKV

5.1 Problem

The goal is to keep the whole training loop on GPUs: data reading, embedding lookup, communication and dense compute. The system should still handle tables larger than aggregate HBM, and serve them with low latency [6][21].

5.2 Training architecture

HugeCTR [6][21] is a CUDA/C++ framework with a Python front end. Its embedding layers are model-parallel across GPUs and nodes, while the dense model is data-parallel. Inter-node communication uses NCCL with GPUDirect RDMA [21]. Its main embedding types:

- **LocalizedSlotSparseEmbeddingHash.** Each feature slot lives on one GPU, with no global reduction. This suits embeddings that fit on a single GPU [21].
- **DistributedSlotSparseEmbeddingHash.** Keys are spread across GPUs regardless of slot, so a single slot can span devices. This suits tables larger than one GPU at the cost of more communication [21].
- **Hybrid embedding** (described in the RecSys 2022 paper [6]). Hot categories are replicated data-parallel and cold categories are model-parallel. Frequent IDs then need no all-to-all, which was a key optimization in NVIDIA's MLPerf DLRM submissions.

Other features [21]:

- Mixed precision, with FP16 used on embeddings to save bandwidth and loss scaling to prevent underflow.
- Learning-rate warmup and decay schedules.
- HDFS model I/O.
- A `hugectr2onnx` converter.

Two nodes with 8x A100 80 GB each can train models of up to about **1 TB entirely in GPU memory** [21].

For TensorFlow users, NVIDIA ships the **Sparse Operation Kit (SOK)**, a model-parallel GPU embedding layer [21]. Alibaba DeepRec uses SOK for its synchronous mode [10].

5.3 Beyond HBM: ETC, HPS and HierarchicalKV

- **Embedding Training Cache (ETC)** [6] trains models larger than HBM by keeping a working set in GPU memory. The full table lives in host or SSD storage, and the working set is swapped in per data pass.

- **Hierarchical Parameter Server (HPS)** [6] is the inference counterpart:

```
request -> GPU embedding cache (hot, HBM)
        -> miss -> CPU-memory volatile DB (e.g., Redis cluster / hash maps)
        -> miss -> persistent DB (e.g., RocksDB on SSD)
online-training updates streamed (e.g., via Kafka) into the tiers
```

  HPS and the training cache are listed as **deprecated as of v25.03** in the current HugeCTR repository [21]. NVIDIA's investment has moved toward the general-purpose library below and toward framework plugins.

- **HierarchicalKV (HKV)** [22] is a header-only GPU hash table:
  - Keys always reside in HBM. Values can spill to host memory.
  - Buckets are locally ordered (128 slots by default), and the default maximum load factor is 0.5 before rehash.
  - Eviction happens only when a bucket is full, using a per-key 64-bit **score**. Built-in policies: LRU (device clock), LFU, EpochLRU, EpochLFU, or fully custom scores.

  HKV published benchmarks on a single A100 80 GB [22]:

Configuration	insert_or_assign (B KV/s)	find (B KV/s)
Pure HBM, dim 8, 128M keys, load 0.5	~1.09	~2.47
Pure HBM, dim 64, 64M keys, load 0.5	~0.83	~1.98
HBM+host, dim 64, 512M keys (32 GB HBM + 96 GB host)	~0.049	~0.069

  The pointer-returning `find*` variant stays at about 3.5–4 billion KV/s even in hybrid mode, because it avoids copying values [22]. HKV is the GPU backend for TFRA's dynamic embedding [19]. It is effectively the GPU counterpart of Monolith's CPU Cuckoo tables.

5.4 Lessons

- **Replicate the hot head, shard the long tail.** Hybrid embedding exploits frequency skew to remove most all-to-all traffic for the most common IDs [6].
- **Put score-based eviction inside the hash table** (HKV), so that admission and eviction policy is a pluggable function rather than a separate service [22].
- **Use a tiered store with a GPU cache** for inference. Skew makes high hit rates possible, but cache consistency with online training must be designed in from the start [6].

6. Continual / Incremental Training and Checkpointing

6.1 Patterns

Across Monolith, XDL, DeepRec, TFRA and HugeCTR the same continual-training pattern recurs:

```
stream (Kafka) -> joiner (Flink/Storm-like) -> online trainer
     |                                              |
     |                          incremental export (delta of touched IDs)
     v                                              v
offline log (HDFS) -> periodic full retrain   serving PS / embedding cache
                       (bootstrap / recovery)
```

Its components:

- **Warm start plus streaming.** A batch model trained on history is the starting point, and streaming updates follow.
- **Delta export.** Only rows touched since the last export are shipped. Dense weights are shipped less often [5][10][16].
- **Admission and eviction** keep table size bounded as the ID space churns [5][16][22].
- **Sampling corrections** (for example log-odds correction under negative downsampling) keep calibrated scores consistent across training and serving [5].

6.2 Check-N-Run: checkpointing trillion-parameter tables

Training jobs on terabyte-scale models must checkpoint often to survive failures, but full checkpoints are huge. Meta's **Check-N-Run** [9] (NSDI 2022) uses two techniques:

1. **Differential checkpointing.** In any interval only a small fraction of embedding rows change, so only the modified rows are written. This is tracked per row.
2. **Checkpoint quantization.** Rows are written in reduced precision (for example 4-bit or 8-bit with adaptive schemes). The paper shows this is acceptable for recovery when applied with care, for example by limiting how many consecutive quantized restores are allowed.

The paper reports roughly 6–17x reductions in checkpoint write bandwidth and roughly 2.5–8x reductions in storage capacity (approximate, from the paper's abstract) with no meaningful accuracy loss [9]. The idea is the same as Monolith's incremental sparse sync: **write only what changed, and write it compactly**.

7. Hashing Tricks and Embedding Compression

When a table cannot hold one row per ID, or when memory, bandwidth or serving cost matters more than the last fraction of a percent of quality, compression techniques apply. They form a spectrum from "fewer rows" to "no table at all".

7.1 The hashing trick

Weinberger et al. [7] introduced **feature hashing**: map a huge or unbounded feature space into a fixed number of buckets with a hash function, and use a sign hash to make inner products unbiased. Applied to embeddings, an ID is mapped to `row = h(id) mod M`. This is simple and stateless and handles unseen IDs, but distinct IDs collide. Monolith's motivation was precisely that collisions measurably hurt ranking quality at ByteDance scale [5].

Hash embeddings [23] (Svenstrup et al., NeurIPS 2017) reduce the damage from collisions:

- Each token picks k vectors from a shared pool using k hash functions.
- It combines them with **learned per-token importance weights**.
- Two colliding tokens are then unlikely to share the same combination.

7.2 Compositional embeddings: the QR trick

Shi et al. [8] (Meta, KDD 2020) proposed **compositional embeddings using complementary partitions**. In the quotient-remainder (QR) case, an ID x uses two smaller tables:

```
q = x // m      -> row in quotient table   (size ceil(|S|/m) x d)
r = x  % m      -> row in remainder table  (size m x d)
emb(x) = combine(Wq[q], Wr[r])     combine in {multiply, add, concat}
```

Every ID still gets a **unique** embedding, because the pair (q, r) is unique. Memory, however, drops from |S| x d to about (|S|/m + m) x d. With m near sqrt(|S|) this is O(sqrt(|S|)) rows. The paper generalizes the idea to multiple complementary partitions and to path-based composition. It shows large memory reductions on Criteo-style DLRM with small accuracy loss [8]. The open-source DLRM cites this work as the "reduce the number of vectors" approach [11].

7.3 Mixed-dimension embeddings

Ginart et al. [24] observe that popular IDs need more capacity than rare ones. They assign **embedding dimensions in proportion to frequency (popularity)**, with blocks of rows at different widths, and project each block to a common dimension. This is the "reduce vector dimensions" approach cited by DLRM [11]. In practice it composes well with QR, with low-precision storage, and with frequency admission.

7.4 TT-Rec: tensor-train decomposition

TT-Rec [25][26] (Meta, MLSys 2021) factorizes each large table into a sequence of small **Tensor-Train cores**. The table shape is factored as products of `p_shapes` x `q_shapes` with TT ranks. Only the requested rows are reconstructed on the fly. According to the FBTT-Embedding repository [26]:

- It is a drop-in replacement for `EmbeddingBag`, with a fused-backward kernel for SGD and Adagrad.
- It achieves **up to 100x smaller models** on Meta's open-source DLRM with the same model quality, and up to 10,000x lower runtime memory per table.
- A **32-way set-associative LFU software cache** keeps uncompressed hot rows, so the frequent IDs skip TT reconstruction.

This is the same skew-exploiting pattern as hybrid embedding and HKV: compress the long tail, and keep the head in full precision.

7.5 Hashing-based "shared memory" compression (ROBE-style)

Random-offset block embedding methods (ROBE, MLSys 2022) push hashing further. All tables share **one small parameter array**, and each embedding is read as a contiguous

block at a hashed offset. That keeps memory access cache-friendly while allowing extreme compression ratios. These methods are surveyed widely, but the primary paper could not be fetched for this chapter, so specific ratios are omitted here.

7.6 Deep Hash Embedding: no table at all

Google's **DHE** [27] (KDD 2021) removes the table entirely. Each ID is encoded by many (on the order of 1,000) **dense hash functions** into a real-valued vector. A **deep MLP** then decodes that vector into an embedding. The parameter count is independent of the vocabulary, every ID gets a unique code, and unseen IDs are handled naturally. The cost moves from memory to compute, which makes DHE attractive on compute-rich, memory-poor accelerators. The trade-off is that serving cost per lookup rises.

7.7 Comparison

Technique	Unique per ID?	Memory scaling	Extra compute	Handles new IDs	Typical use
Full table (dense array)	Yes (if pre-indexed)	O(N d)	None	Needs re-indexing	Small/stable vocabularies
Dynamic hash table (Monolith Cuckoo, HKV, TFRA, DeepRec EV)	Yes	O(active N d)	Hashing	Yes, with admission	Streaming, open vocab
Hashing trick [7]	No (collisions)	O(M d), M fixed	Hash	Yes	Cheap baseline, long tail
Hash embeddings [23]	Mostly	O(B d + N k)	k lookups + weights	Yes	NLP / long-tail IDs
QR compositional [8]	Yes	O((N/m + m) d)	2 lookups + op	Within range	Huge ID tables
Mixed dimension [24]	Yes	Sum over blocks of N_i d_i	Projection	Needs re-indexing	Skewed popularity
TT-Rec [25][26]	Yes	O(sum of core sizes)	TT reconstruct	Within shape	10–100x compression
ROBE-style shared array	No (shared)	O(M), one array	Hash + block read	Yes	Extreme compression
DHE [27]	Yes	O(MLP params)	Deep MLP per ID	Yes	Memory-bound accelerators
Low-precision rows (FP16/INT8/INT4)	Yes	O(N d bits)	Dequantize	n/a	Serving, checkpoints [9]

8. Hardware Direction: Accelerating Sparse Lookups

Google's TPU v4 [28] includes **SparseCores**. These are dataflow processors dedicated to embedding lookups and the related all-to-all traffic. The paper reports that they accelerate embedding-heavy models by about 5–7x while using only about 5% of die area and power (approximate figures from the TPU v4 paper's abstract) [28]. Together with ZionEX's dedicated RoCE fabric [3], this shows the industry treating embedding traffic as a first-class hardware concern rather than a software afterthought.

Newer **generative recommenders** shift the balance. Meta's HSTU work [29] (ICML 2024) reports sequential transduction models of up to **1.5 trillion parameters**, with much more dense compute per example (long user sequences and attention over jagged tensors). The embedding tables are still huge, but these models move ranking training closer to LLM training in compute intensity. The TorchRec and FBGEMM stack (jagged tensors, sharded embeddings, offloading [13]) therefore has to serve both regimes at once. See Chapter 06 for the modeling side.

9. Cross-System Comparison

System (org)	Sparse storage	Consistency	Hardware	Online/continual	Notable scale claim
Neo / ZionEX (Meta) [3]	Sharded HBM tables + caching	Fully sync	GPU, RoCE fabric	Batch-oriented	Up to 12T params
TorchRec + FBGEMM (Meta) [2][13][14]	TW/RW/CW/DP sharded, TBE kernels, offload	Sync	GPU	Via pipelines	100TB offloading work
Monolith (ByteDance) [5][15]	Cuckoo hash on PS, filter + expiry	Async PS (TF)	CPU PS + workers	Yes, minute-level sync	Production feeds/ads
PERSIA (Kuaishou/ETH) [4][17]	CPU embedding PS	Hybrid: async emb, sync dense	GPU NN + CPU PS	Supported	Up to 100T params
XDL (Alibaba) [16]	Distributed PS	Async PS	CPU/GPU	Admission, eviction, incremental export	Ads/search/rec
DeepRec (Alibaba) [10]	EmbeddingVariable, multi-tier	Async PS or sync collective	CPU/GPU	Delta checkpoints	">10T params"
HugeCTR / SOK (NVIDIA) [6][21]	GPU hash, hybrid embedding	Sync	Multi-node GPU	Kafka into HPS	~1 TB on 2x8 A100
HierarchicalKV (NVIDIA) [22]	GPU hash, HBM+host, scored eviction	Library	GPU	Eviction policies	~1–4 B KV/s on A100
TFRA (SIG-Rec) [19]	Dynamic embedding (cuckoo/HKV/Redis)	Half-sync	CPU/GPU	Yes	Tencent et al.
HET (PKU) [18]	Worker cache + PS	Bounded staleness	CPU/GPU	n/a	Comm reduction
OpenEmbedding (4Paradigm) [20]	PS (DRAM/PMem)	Sync dense + PS	GPU + PS	PMem checkpoint	3–8x vs Horovod

10. Key Takeaways for System Designers

1. **Design around the embeddings.** For DLRM-style models, the capacity, bandwidth and communication of embeddings set the cost. Size the cluster by table bytes and all-to-all bandwidth first, and by FLOPs second [3][12].
2. **Use model parallelism for sparse parameters and data parallelism for dense parameters.** Pick the sharding scheme per table: table-wise, row-wise, column-wise or replicated. Use a cost-model planner rather than hand placement [3][13].

3. **Exploit skew everywhere.** Replicate or cache the hot head (hybrid embedding, HET caches, TT-Rec LFU cache, HPS GPU cache) and shard, compress or offload the tail [6][18][26].
4. **Use dynamic hash tables when the vocabulary is open.** Collision-free embeddings with frequency admission and time- or score-based eviction (Monolith, HKV, TFRA, DeepRec) beat fixed hashing in quality for fast-churning ID spaces [5][19][22].
5. **Match consistency to access pattern.** Sparse rows tolerate staleness, and dense weights do not. Hybrid sync/async (PERSIA, TFRA half-sync) or bounded staleness (HET) captures most of the throughput of asynchrony [4][18][19].
6. **Freshness pays.** Streaming training plus incremental sync of sparse parameters at minute-level, with dense weights synced less often, gives measurable AUC gains in production [5].
7. **Make everything incremental.** Delta exports, delta checkpoints and quantized checkpoints keep trillion-parameter continual training affordable and recoverable [5][9][10].
8. **Fuse kernels and pipeline aggressively.** Table-batched lookups, fused sparse optimizers, and overlap of input all-to-all with compute are table stakes [13][14].
9. **Compression is a spectrum.** Choose by constraint:
    - QR or mixed dimensions for memory with near-zero quality loss [8][24],
    - TT-Rec for 10–100x [26],
    - DHE when memory is scarce and compute is cheap [27],
    - quantization for serving and checkpoints [9].
10. **Co-design hardware.** Dedicated scale-out networks (ZionEX) and on-chip sparse engines (TPU SparseCore) show that embedding traffic justifies specialized silicon and fabric [3][28].

References

1. Naumov, M. et al. "Deep Learning Recommendation Model for Personalization and Recommendation Systems." Meta (Facebook), 2019. https://arxiv.org/abs/1906.00091
2. Ivchenko, D. et al. "TorchRec: a PyTorch Domain Library for Recommendation Systems." Meta, RecSys 2022. https://doi.org/10.1145/3523227.3547387
3. Mudigere, D. et al. "Software-Hardware Co-design for Fast and Scalable Training of Deep Learning Recommendation Models" (Neo / ZionEX). Meta, ISCA 2022 (arXiv 2021). https://arxiv.org/abs/2104.05158
4. Lian, X., Yuan, B., Zhu, X. et al. "Persia: A Hybrid System Scaling Deep Learning Based Recommenders up to 100 Trillion Parameters." Kuaishou and ETH Zurich, 2021 (KDD 2022). https://arxiv.org/abs/2111.05897
5. Liu, Z. et al. "Monolith: Real Time Recommendation System With Collisionless Embedding Table." ByteDance, 2022. https://arxiv.org/abs/2209.07663
6. Wei, Y. et al. "Merlin HugeCTR: GPU-accelerated Recommender System Training and Inference." NVIDIA, RecSys 2022. https://arxiv.org/abs/2210.08803
7. Weinberger, K. et al. "Feature Hashing for Large Scale Multitask Learning." Yahoo! Research, ICML 2009. https://arxiv.org/abs/0902.2206
8. Shi, H.-J. M. et al. "Compositional Embeddings Using Complementary Partitions for Memory-Efficient Recommendation Systems." Meta, KDD 2020 (arXiv 2019). https://arxiv.org/abs/1909.02107

9. Eisenman, A. et al. "Check-N-Run: A Checkpointing System for Training Deep Learning Recommendation Models." Meta, NSDI 2022 (arXiv 2020). https://arxiv.org/abs/2010.08679
10. Alibaba. "DeepRec" (GitHub repository and README). Alibaba, accessed 2026. https://github.com/alibaba/DeepRec
11. Meta Research. "DLRM: Deep Learning Recommendation Model" (GitHub repository and README). Meta, accessed 2026. https://github.com/facebookresearch/dlrm
12. Acun, B. et al. "Understanding Training Efficiency of Deep Learning Recommendation Models at Scale." Meta, HPCA 2021 (arXiv 2020). https://arxiv.org/abs/2011.05497
13. PyTorch. "TorchRec" (GitHub repository and README). Meta / PyTorch, accessed 2026. https://github.com/pytorch/torchrec
14. PyTorch. "FBGEMM / FBGEMM_GPU" (GitHub repository and README). Meta / PyTorch, accessed 2026. https://github.com/pytorch/FBGEMM
15. ByteDance. "Monolith" (GitHub repository, archived). ByteDance, accessed 2026. https://github.com/bytedance/monolith
16. Alibaba. "X-DeepLearning (XDL)" (GitHub repository and README). Alibaba Mama, accessed 2026. https://github.com/alibaba/x-deeplearning
17. PersiaML. "PERSIA" (GitHub repository and README). Kuaishou and ETH Zurich, accessed 2026. https://github.com/PersiaML/PERSIA
18. Miao, X. et al. "HET: Scaling out Huge Embedding Model Training via Cache-enabled Distributed Framework." Peking University, VLDB 2022 (arXiv 2021). https://arxiv.org/abs/2112.07221
19. TensorFlow SIG Recommenders. "TensorFlow Recommenders-Addons (TFRA)" (GitHub repository and README). accessed 2026. https://github.com/tensorflow/recommenders-addons
20. 4Paradigm. "OpenEmbedding" (GitHub repository and README). 4Paradigm, accessed 2026. https://github.com/4paradigm/OpenEmbedding
21. NVIDIA. "HugeCTR Core Features" (repository documentation). NVIDIA Merlin, accessed 2026. https://github.com/NVIDIA-Merlin/HugeCTR/blob/main/docs/source/hugectr_core_features.md
22. NVIDIA. "HierarchicalKV" (GitHub repository and README, benchmarks). NVIDIA Merlin, accessed 2026. https://github.com/NVIDIA-Merlin/HierarchicalKV
23. Svenstrup, D., Hansen, J., Winther, O. "Hash Embeddings for Efficient Word Representations." NeurIPS 2017. https://arxiv.org/abs/1709.03933
24. Ginart, A. et al. "Mixed Dimension Embeddings with Application to Memory-Efficient Recommendation Systems." Meta / Stanford, 2019. https://arxiv.org/abs/1909.11810
25. Yin, C. et al. "TT-Rec: Tensor Train Compression for Deep Learning Recommendation Models." Meta, MLSys 2021. https://arxiv.org/abs/2101.11714
26. Meta Research. "FBTT-Embedding" (GitHub repository for TT-Rec, archived). Meta, accessed 2026. https://github.com/facebookresearch/FBTT-Embedding
27. Kang, W.-C. et al. "Learning to Embed Categorical Features without Embedding Tables for Recommendation" (DHE). Google, KDD 2021 (arXiv 2020). https://arxiv.org/abs/2010.10784
28. Jouppi, N. et al. "TPU v4: An Optically Reconfigurable Supercomputer for Machine Learning with Hardware Support for Embeddings." Google, ISCA 2023. https://arxiv.org/abs/2304.01433
29. Zhai, J. et al. "Actions Speak Louder than Words: Trillion-Parameter Sequential Transducers for Generative Recommendations" (HSTU). Meta, ICML 2024. https://arx

ive.org/abs/2402.17152

10. Online Serving and Inference: Latency Budgets, Model Servers, Caches and Cascades

A recommender is only as good as what it can compute inside one request. Every system in this atlas, from a two-tower retriever to a trillion-parameter generative recommender, eventually has to answer one question within a few hundred milliseconds: given this viewer, right now, which K items do we return? This chapter covers the serving side of that question. It looks at how production systems split a latency budget across stages, how they run model inference on CPUs, GPUs and custom accelerators, how they fan out feature and embedding fetches, what they cache, and how they shrink models through cascades, distillation, quantization and early exit.

The chapter builds on primary sources: open-source production code (X/Twitter's `the-algorithm` and xAI's `x-algorithm` repositories, Meta's `generative-recommenders`, TorchRec, FBGEMM), official documentation (Vespa, Triton Inference Server, NVIDIA Merlin HugeCTR), and engineering papers and posts (DeepRecSys, Earlybird, Instagram Explore, Meta Andromeda, Pinterest GPU inference).

> Source note. In this research pass, outbound access was limited to GitHub-hosted content. Facts drawn from repositories and documentation were read in full. Sections on Meta (Instagram Explore, Andromeda) and Pinterest summarize their published engineering posts. Those posts could not be re-fetched here, so their numbers are marked approximate and should be checked against the linked originals before anyone quotes them.

10.1 The shape of the problem: funnels and latency budgets

Almost every large recommender shares the same request-time skeleton, popularized by YouTube's candidate-generation-then-ranking design [33]. A cheap stage looks at a huge corpus. Then progressively more expensive stages look at progressively fewer items:

```
corpus (10^6 - 10^10 items)
  |
  |  RETRIEVAL / candidate generation   ANN, inverted index, graph walks, in-mem
  v                                     follow-graph stores; cost ~O(log N) per source
10^3 - 10^4 candidates
  |
  |  FEATURE HYDRATION                  fan-out to feature stores, embedding servers,
  v                                     counters; often the latency long pole
  |
  |  LIGHT / EARLY-STAGE RANKING        LR, GBDT, two-tower dot product, distilled nets
  v
10^2 - 10^3 candidates
  |
  |  HEAVY RANKING                      large MTML DNN / transformer, GPU or accelerator
  v
10^1 - 10^2 candidates
  |
  |  RE-RANKING / POLICY                diversity (DPP, author decay), filters, ads mix
  v
response (page of K items)
```

X's For You timeline makes the funnel explicit. Home Mixer fetches candidates from several sources, hydrates "the ~6000 features needed for ranking", scores them with an ML model, and then applies filters and heuristics such as author diversity, in-network/out-of-network balance, feedback fatigue, dedup and visibility filtering. Only after that does it mix in ads and who-to-follow modules [2]. xAI's 2026 release of the X algorithm keeps the same skeleton. Its parts are query hydration, parallel candidate sources (Thunder in-network, Phoenix retrieval, SimClusters, popular posts), candidate hydration, pre-scoring filters, Phoenix scoring, a weighted value model, DPP re-ranking (`vm-ranker`), top-K selection and post-selection visibility filters [11].

Why a cascade at all? The Vespa documentation puts the economics plainly: "A good quality ranking expression will for most applications consume too much CPU to be runnable on all retrieved or matched documents within the latency budget/SLA." The engineering job therefore becomes finding "a first-phase function, which correlates sufficiently well with the second-phase function" [13]. Each stage is a cheaper approximation of the next, and the rerank counts at the stage boundaries are the main knobs for latency and cost.

A typical budget breakdown, written here as a design heuristic rather than a number from any one source, looks like this:

Stage	Typical share of p99 budget	Dominant cost
Query/user hydration	5-15%	KV lookups, user sequence fetch
Candidate generation (parallel)	15-25%	ANN / index traversal, slowest source
Feature hydration	20-35%	fan-out RPCs, tail latency of slowest shard
Light ranking	5-10%	CPU, simple models over many items
Heavy ranking	20-35%	GPU/accelerator compute, batching delay
Re-rank, filters, mixing	5-10%	rule evaluation, DPP, ads auction

Two structural lessons follow from production systems:

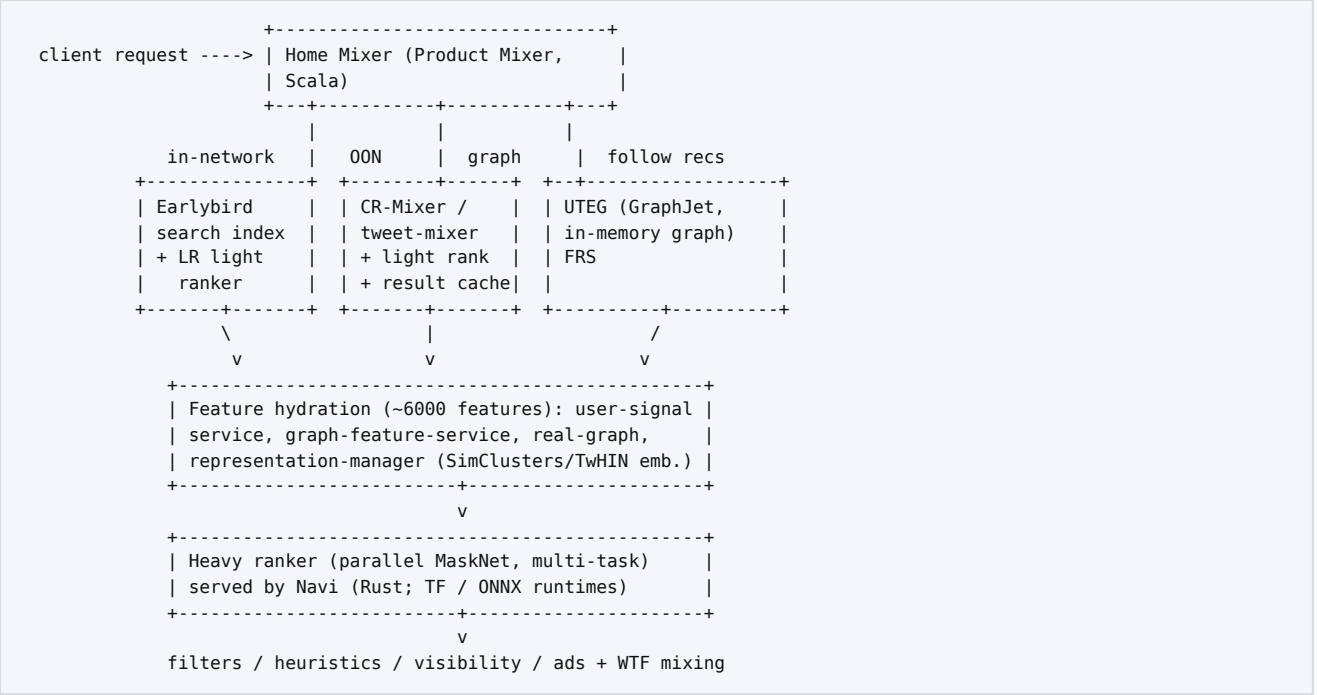
1. Parallelize sources and fetches, then bound the tail. In X's Product Mixer framework, Candidate Pipelines run under Mixer/Recommendation Pipelines, and stages run "in parallel where it can" [8][11]. The response time is set by the slowest parallel branch,

so each branch needs its own timeout and a fallback. Home Mixer keeps a reverse-chronological "backup ... pipeline in case Scored Tweets fails" [2].
2. Side effects belong off the critical path. xAI's Home Mixer records served posts, refreshes the post cache and logs client and ad events "after the response is sent" [11].

10.2 X / Twitter: an open-source reference serving stack

X has published more of its serving stack than any other large company [10]: the 2023 `the-algorithm` release and the 2025-2026 `x-algorithm` (xAI) release. That makes it the best public case study for end-to-end serving.

10.2.1 Architecture (2023 Twitter stack)



Candidate sources. Earlybird, a Lucene-based real-time search system, does in-network retrieval. It supplies roughly half of For You posts [1]. The index is split into a realtime cluster (about the last 7 days of public tweets), a protected cluster, and an archive cluster. Each cluster is partitioned into a sharded, replicated inverted index with a single-writer/multi-reader design. Earlybird "roots" fan requests out to partitions and merge the responses [6]. Out-of-network candidates come through CR-Mixer, which "fetches source signals externally ... calls external candidate generation services, and caches results", then applies filters and a light ranking step [7]. A second out-of-network source is the in-memory UTEG graph built on GraphJet [1].

Light ranking at the index. The Earlybird light ranker is a logistic regression model, "a simplified version of the heavy ranker which can run on a greater amount of tweets". It runs inside the search index itself and uses static features computed at ingestion (has URL, is reply, text-quality score, tweepcred) plus real-time engagement counters pushed by a signal ingester [5]. This is an example of compute pushed to the data, the same idea as

Vespa's content-node first phase (Section 10.5). The README also admits the model "was last trained several years ago" and was due to be replaced [5], a reminder that cascade stages tend to rot when nobody owns them.

Heavy ranking. The Recap heavy ranker is a parallel MaskNet that outputs about ten engagement probabilities (fav, retweet, reply, good click, negative feedback, report and others). They are combined by a configurable weighted sum. The April 2023 weights included reply-engaged-by-author 75.0, report -369.0 and fav 0.5. The weights were first set "so that, on average, each weighted engagement probability contributes a near-equal amount to the score" [9][28].

10.2.2 Navi: a minimalist Rust model server

Twitter replaced generic model serving for this path with Navi, "a high-performance, versatile machine learning serving server implemented in Rust". It speaks the TensorFlow Serving gRPC API and has pluggable runtimes: TensorFlow (most complete), ONNX Runtime (used for home recommendation with a proprietary `BatchPredictRequest` Thrift input) and experimental PyTorch [3]. Its CLI defaults show the core batching levers of any recsys model server [4]:

Navi flag	Default	Meaning
`max_batch_size`	16	max requests coalesced into one batch
`batch_time_out_millis`	2	max wait to accumulate a batch
`batch_drop_millis`	90	threshold to start dropping batches under stress
`model_check_interval_secs`	300	polling for new model versions
`onnx_gpu_ep`	(none)	choose CUDA or TensorRT execution provider
`max_warmup_records`	-	warm-up records before serving (TF)

Lessons from Navi. (a) A server that does fewer things can be much faster. Navi trades features for performance [3]. (b) Load shedding is a first-class setting. Dropping a batch that is already 90 ms late beats letting a queue build. (c) Feature transformation from the wire format (`dr_transform`, `segdense`) lives inside the server, so the client ships raw records once [3]. (d) Hot model reloading by version directory gives continuous deployment without restarts [3].

10.2.3 xAI x-algorithm (2026): transformer ranking built to be cacheable

The 2026 release replaces MaskNet with Phoenix, a transformer used both as a two-tower retriever and as the ranker. It is trained in JAX and served by "a Rust gRPC serving engine" [11][12]. Several serving-driven design decisions stand out:

- Candidate isolation. In the ranking transformer, "candidates cannot attend to each other", only to the user and history tokens. The attention mask is full bidirectional over user+history, and diagonal-only among candidates. The stated reason is serving: "the score for a post doesn't depend on which other posts are in the batch, making scores consistent and cacheable" [11][12]. It also means the user/history prefix can be computed once and shared by every candidate in the request. This is the same amortization idea as Meta's M-FALCON (Section 10.6).

- Batch-invariant serving. Together with a retrieval index baked "inside the checkpoint" (the trainer runs the candidate tower over the corpus at every checkpoint save), "a candidate's score depends only on the user and that candidate" [12]. Batch invariance makes caching, A/B debugging and replay possible.
- Hash-based embeddings plus semantic IDs. Multiple hash functions per entity mean "no dictionary service", so no ID-to-row lookup RPC sits on the serving path. Semantic IDs (residual-quantized codes, 6 levels x 256 codes) give new posts usable representations immediately [11][12].
- Multi-action outputs plus a separate value model. Phoenix outputs `[B, num_candidates, num_actions]` plus dwell regression. `RankingScorer` computes `sum(weight_i * P(action_i))`, then applies author-diversity decay, an out-of-network discount and a new-author boost. `vm-ranker` reorders by a determinantal point process over embeddings [11].
- Hardware. The open-source quickstart detects A100/H100/H200/GB200/GB300 and ships Hopper- and Blackwell-tuned attention kernels [12]. The ranker is a GPU workload.

10.3 Meta: Instagram Explore and the Andromeda ads retrieval engine

10.3.1 Instagram Explore (2023 post)

Meta's "Scaling the Instagram Explore recommendations system" (2023) describes a four-stage funnel: retrieval, first-stage (early) ranking, second-stage (late) ranking, and final re-ranking [31]. As summarized from the post (approximate; check the original):

```
sources: two-tower retrieval, user-interaction histories, heuristics
          |   (item embeddings precomputed offline, ANN at request time)
          v
first-stage ranking: lightweight two-tower-style model, trained partly to
          |          reproduce the late-stage ranker's output (distillation)
          v
second-stage ranking: multi-task multi-label (MTML) DNN predicting many
          |           engagement events
          v
value model: weighted combination of MTML outputs (weights tuned online)
          |
          v
final re-ranking: integrity filters, diversity / business rules
```

Serving-relevant points in the post:

- Two-tower retrieval lets item embeddings be computed and indexed offline, so only the user tower runs at request time.
- The first-stage ranker is cheap because it is a two-tower-style model whose item side is precomputed. It is trained to mimic the heavier second stage, a form of ranking distillation (Section 10.8).
- The final score is a value model: a formula over predicted event probabilities with tunable weights. Product trade-offs then become configuration changes, not retraining. The post describes tuning these weights online (Bayesian-optimization-style experimentation).

- Caching and precomputation are used heavily, because Explore serves a very large user base under tight latency.

10.3.2 Andromeda (2024): GPU-native ads retrieval

Meta's Andromeda post (December 2024) describes a personalized ads retrieval engine built for NVIDIA Grace Hopper Superchips and Meta's MTIA accelerator [32]. As reported in the post (numbers approximate):

- It replaces a retrieval stage that had been constrained to lightweight models with a deep neural retrieval model co-designed with the hardware. The post cites about a 10,000x increase in retrieval model complexity.
- A hierarchical index makes retrieval sub-linear in the number of ads. The index structure is trained jointly with the model rather than being a separate offline ANN build.
- Reported results include a recall improvement of several percent and ads-quality gains on selected segments. The post attributes this to applying deep models at a stage that previously used simple scoring.
- The key lesson is that co-designing model, index and hardware (high-bandwidth unified CPU-GPU memory on Grace Hopper) makes "heavy" computation affordable at the stage that sees the most items. That moves the cascade's quality bottleneck upstream.

10.3.3 The Meta inference software substrate

Meta's open-source stack shows what production recsys inference is built on:

- FBGEMM, "a low-precision, high-performance matrix-matrix multiplications and convolution library for server-side inference". It supports small batch sizes and "row-wise quantization and outlier-aware quantization", and it fuses operators to beat bandwidth limits [18]. FBGEMM_GPU adds recsys-focused GPU operators (embedding bags, table-batched embeddings). FBGEMM GenAI adds FP8 row-wise quantization [18].
- TorchRec Inference, a C++ library for GPU inference. It captures the model with `torch.fx` and runs it as TorchScript in a C++ gRPC server. The reference example serves a DLRM on one GPU [17]. The README calls it "a simple reference ... not a fully scaled out solution" [17]. Production adds sharding, request batching and feature pipelines on top.
- DeepRecSys (Harvard + Facebook, ISCA 2020) characterized eight industry-representative models (DLRM-RMC1/2/3, NCF, Wide&Deep, MT-WnD, DIN, DIEN). It built DeepRecSched, a scheduler that chooses query batch size and splits queries between CPU cores and accelerators based on query-size and arrival distributions. It reports roughly 2x latency-bounded throughput and "over 30% latency reduction" in an at-scale production datacenter [22]. The core insight is that recsys queries vary widely in size (number of candidates), so a fixed batch policy wastes either latency or throughput.

10.4 GPU inference for ranking: Pinterest and the general recipe

10.4.1 Pinterest (2022 post)

Pinterest's "GPU-accelerated ML Inference at Pinterest" (2022) describes moving ranking models from CPU to GPU serving [30]. In outline (approximate, per the original post):

- Motivation: CPU serving costs grew roughly linearly with model size, which blocked larger ranking models (bigger embeddings, transformer-style user-sequence modules).
- Result: GPU serving let Pinterest serve much larger models, on the order of 100x more compute per the post, at comparable or lower cost and latency.
- Techniques the post discusses: aggressive batching of candidates per request, CUDA Graphs to cut kernel-launch overhead, merging many small embedding-table lookups into fewer larger ops, mixed precision, and moving feature preprocessing onto the GPU so the host is not the bottleneck.

10.4.2 Why recsys GPU inference is different from vision/NLP inference

Aspect	Typical CV/NLP inference	Recsys ranking inference
Work per request	1 input	100s-1000s of candidates x same user
Parameter mix	dense weights	mostly sparse embeddings (GB-TB), small dense MLP
Bottleneck	FLOPs	memory bandwidth (embedding gathers), launch overhead
Shared computation	none	user tower / user sequence shared across candidates
Batching	across requests	within request (candidates) and across requests
Model freshness	days-weeks	minutes-hours (online / streaming updates)

Recipe distilled from the sources:

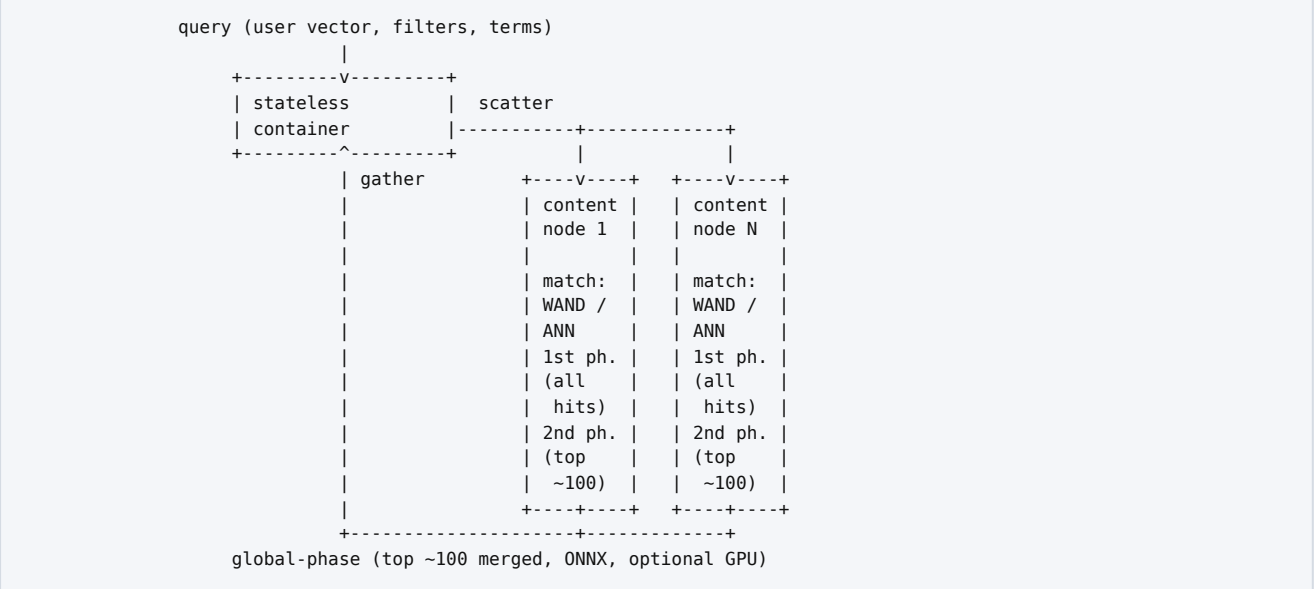
1. Batch inside the request first. Score all candidates of one request as one batch, so user-side work is computed once (Phoenix candidate isolation [12], M-FALCON [21]).
2. Batch across requests with a bounded delay. Triton's dynamic batcher holds requests up to `max_queue_delay_microseconds` (e.g. 100 us) to form preferred or maximum batch sizes. Its docs advise starting from defaults and raising the delay only "until the latency budget is exceeded" [16]. Navi's 2 ms timeout and 16-request batches play the same role [4].
3. Cut launch overhead with CUDA Graphs, fused kernels, Triton/CUTLASS custom kernels. The HSTU repository ships Triton kernels with roughly 2-2.3x speedups over PyTorch for the attention block on H100 [21].
4. Lower the precision. HSTU-Ultra FP8 projection GEMMs give 1.13-1.25x end-to-end forward speedups over BF16 on one H100 (e.g. 9.375 ms to 7.768 ms for a 12-layer, 1024-length stack) [21]. FBGEMM provides INT8/row-wise quantization for CPU paths [18].
5. Keep embeddings near compute. Use GPU embedding caches and hierarchical parameter servers (Section 10.7).

10.5 Search-engine-based serving: Vespa and Elasticsearch

Many companies serve recommendations from a search engine. Retrieval and first-phase ranking then run where the data lives, so candidate features never leave the shard.

10.5.1 Vespa phased ranking

Vespa splits the work into content nodes (stateful, hold documents and indexes) and a stateless container (scatter-gather, global logic) [13]:



- Retrieval. Top-k operators (`weakAnd`/WAND, nearest-neighbor) are sublinear. They expose only the top-k hits to ranking, and several retrievers can be combined in one query [13].
- First phase runs on every retrieved hit on each content node and must be cheap. `rank-score-drop-limit` can drop hits early [13].
- Second phase re-ranks, by default, the 100 best first-phase hits per content node. `total-rerank-count` caps the total across nodes. The documentation example uses `bm25 + freshness` for the first phase and an XGBoost model for the second [13].
- Global phase runs in the container on the merged global top hits (default 100, `rerank-count`), is "optimized for inference with ONNX models", and can use a GPU via `gpu-device`. It supports cross-hit normalization (`normalize_linear`, `reciprocal_rank`, `reciprocal_rank_fusion` with k = 60 by default) [13].
- Placement advice. Large GBDTs (XGBoost/LightGBM) and expressions that need lots of vector data should stay in the second phase on content nodes, because "moving vector data across the network is expensive". Expensive functions computed on content nodes can be passed up via `match-features`, so the global phase reuses them instead of recomputing [13].
- Observability. Mutable attributes can count how often each document survives each phase (`on-match`, `on-first-phase`, `on-second-phase`, `on-summary`). This makes cascade recall measurable per document [13].

Vespa's sample apps include an album-recommendation intro and a news recommendation tutorial that uses ANN search in a shared user/item embedding space for retrieval [14]. That is the canonical two-tower-in-a-search-engine pattern.

10.5.2 Elasticsearch / OpenSearch with Learning to Rank

The OpenSource Connections LTR plugin stores features as query templates, logs feature values for training, and stores linear, XGBoost or RankLib models that rank search results [15]. In recommender settings this works as a rescoring window. The engine's own scoring retrieves candidates, and the LTR model re-scores the top window, a two-stage cascade inside the engine. Its limits compared with Vespa are coarser control over per-shard phases and no native tensor/ONNX global phase. Teams often add an external re-ranking service for deep models.

10.5.3 Earlybird as a search-engine recommender

Twitter's Earlybird (Section 10.2) belongs here too. It is an inverted-index search system whose main recsys role is UserID-based retrieval (tweets from accounts you follow). It is ranked in-index by a light model, and features stay fresh through a separate Feature Update Service [5][6]. Twitter reported reducing search indexing latency to about one second (see the blog cited in [6]). Freshness of the index directly bounds the freshness of recommendations.

10.6 Generative recommenders at serving time: HSTU and M-FALCON

Meta's Generative Recommenders work (ICML 2024) restates ranking and retrieval as sequential transduction over long user action histories, with HSTU as the encoder. Serving cost is the obvious objection, since attention over thousands of history events per candidate is expensive. The paper's answer is M-FALCON (Microbatched-Fast Attention Leveraging Cacheable OperatioNs) [21]:

```
naive: for each candidate c_j:  encode([history ; c_j])     cost ~ m * O(n^2)

M-FALCON:
  encode history once ---------------> cached keys/values (n tokens)
  split m candidates into microbatches of size b
  for each microbatch: attend [b candidates] -> cached history,
                       candidates masked from each other
                                                              cost ~ O(n^2) + m*O(n)
```

- Candidates in a microbatch attend to the shared history but not to each other. The xAI Phoenix "candidate isolation" mask is the same contract (Section 10.2.3) [12].
- The repository describes HSTU and M-FALCON as accelerating training and inference "by 10x-1000x" and reports deployment in "billion-user scale recommendation systems" [21].
- The same repository includes a DLRM-v3 (HSTU-based) inference benchmark wired to MLPerf LoadGen. It reports H100 forward latencies of roughly 5-9 ms for 8-12-layer stacks at sequence lengths 512-1536 in BF16, and roughly 5-8 ms in FP8 [21]. The

README notes that cached incremental decoding was "not included yet" in the open-source release [21].

Design takeaway: once a ranker has a large shared user-side computation, the right serving unit is (user, many candidates), not (user, candidate). Caching user-side state across requests (for example a KV cache updated incrementally as new actions arrive) is the next step.

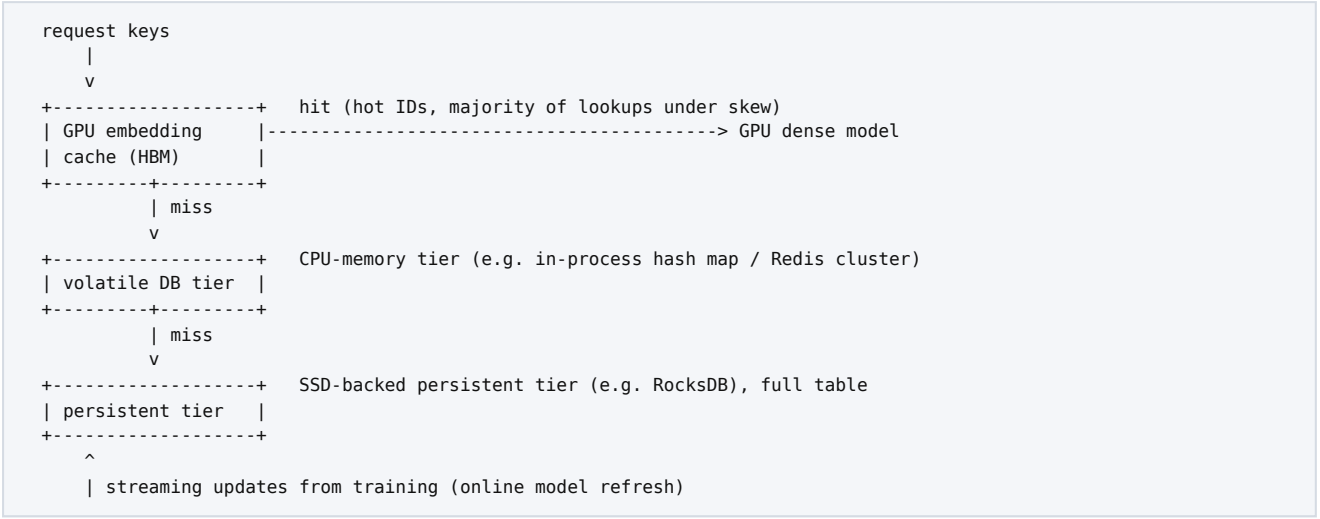
10.7 Embedding serving and caching

10.7.1 The embedding problem

DLRM-style models (DLRM, arXiv 1906.03109 [23]) put most parameters in sparse embedding tables. ByteDance's Monolith argues for collisionless embedding tables and real-time training, so that tables grow and change continuously [24]. Serving therefore has to handle tables that (a) exceed GPU memory, (b) are updated at minute granularity, and (c) are accessed with a heavily skewed (power-law) distribution. Skew is what makes caching work.

10.7.2 NVIDIA Merlin HugeCTR Hierarchical Parameter Server

HugeCTR's Hierarchical Parameter Server (HPS) was "a native C++ library that provides caching and hierarchical storage for embeddings". It combined a GPU embedding cache with a database backend, for embeddings "that exceed the capacity of GPU and host memory". It shipped as a TensorFlow plugin and as a Triton backend [19]. NVIDIA also blogged about scaling inference with it [20]. It was published in RecSys 2022 ("A GPU-specialized Inference Parameter Server for Large-Scale Deep Recommendation Models") [19]:



Note that HPS is marked deprecated since HugeCTR v25.03 [19]. The hierarchy itself (HBM cache, then DRAM, then SSD, with streaming updates) remains the standard design.

10.7.3 Other caching layers in production recsys

Cache	What is cached	Invalidation	Example
Result / candidate cache	candidate lists per source per user	TTL, session	CR-Mixer caches candidate generation results [7]
Served/seen cache	IDs already shown	session, rolling window	X `PreviouslyServedPostsFilter`, post cache refresh [11]
Item embedding index	precomputed item tower outputs	per checkpoint	Phoenix index baked into checkpoint [12]
Embedding cache	hot embedding rows	streaming updates	HPS GPU cache [19]
Score cache	(user, item) scores	model version, TTL	enabled by batch-invariant scoring [11] [12]
User-state cache	user tower / sequence KV	new user action	M-FALCON-style history encoding [21]
Feature cache	user/item features	TTL / write-through	feature stores (Ch. 8) [25]

Caching is only safe when the model's output is a function of a stable key. That is why xAI treats candidate isolation as a serving invariant, not a modeling detail: listwise models whose scores depend on the other candidates cannot reuse cached per-item scores.

10.8 Feature fetching fan-out

Feature hydration is often the latency long pole. X's Home Mixer hydrates about 6,000 features per request from many services (user-signal service, graph-feature-service, real-graph, representation-manager, engagement counters) [1][2]. Earlybird co-locates features with the index and updates them through a separate stream [5][6].

Common patterns:

- Co-locate or precompute. Push static and slow-moving item features into the index or the model server, as Earlybird and Vespa do [5][13]. Hash-based embeddings remove the ID-to-row dictionary lookup service entirely [11].
- Batch-get by entity. Fetch features per entity type in bulk (one call per store for all candidates), not per candidate.
- Online feature stores. Feast describes an offline store for training, "a low-latency online store" for real-time prediction, and a feature server, with Redis, DynamoDB, Bigtable and Dragonfly among online backends [25]. DoorDash's 2020 post on building a "gigascale" feature store on Redis is a widely cited case study [38]. It covers Redis hashes, hashed feature names and compressed embedding values to cut memory and latency (details approximate here; see the original). Lyft and Etsy describe similar feature-serving and recommendation-serving platforms [36][37], and Eugene Yan's applied-ml index collects many more such posts [29].
- Tail-latency control. With fan-out across F services, request p99 approaches the max of F service latencies. Hedged requests, per-dependency timeouts with default feature values, and degrade-to-cached-features keep the tail bounded. Navi's batch-drop threshold applies the same thinking at the model server [4].
- Train-serve consistency. Feature logging at serving time (log what you served, train on it) avoids skew. The Elasticsearch LTR plugin's feature-logging mode is a small

example [15].

10.9 Making models cheaper: cascades, early exit, distillation, quantization

10.9.1 Cascades and pre-ranking

- Rule-of-thumb cascade. A cheap model on everything, then an expensive model on the top slice: Vespa first/second/global phases [13], the Earlybird LR light ranker before the heavy ranker [5], the X pushservice light ranker that "bridges candidate generation and heavy ranking by pre-selecting highly-relevant candidates from the initial huge candidate pool" [1].
- Pre-ranking with compute awareness. Alibaba's COLD (Computing power cost-aware Online and Lightweight Deep pre-ranking system, 2020) argued that pre-ranking can be a real deep model with cross features if its cost is co-optimized with the serving system. Feature selection should account for compute cost, and engineering should cover low precision and parallelism [34].
- Stage alignment. A cascade stage should optimize agreement with the downstream stage, not only the click label. Otherwise the early stage discards items the late ranker would have liked. Vespa's per-document phase-survival counters are a practical way to measure this [13].

10.9.2 Early exit

Early exit means stopping computation for items or requests once the answer is clear. In cascades it shows up as:

- score thresholds that drop hits after the first phase (`rank-score-drop-limit` in Vespa [13]);
- top-k operators (WAND, ANN) that never fully score most documents [13];
- adaptive rerank counts per query (`ranking.globalPhase.rerankCount` can be set per query [13]) or per load level, which is graceful degradation under stress;
- load shedding at the server (Navi `batch_drop_millis` [4]).

10.9.3 Distillation

- Ranking distillation. Tang and Wang (KDD 2018) train a small student ranker on the teacher's top-K ranking as extra supervision, and report students at a fraction of the teacher's size with comparable ranking quality [35].
- Cross-stage distillation. Instagram Explore's first-stage ranker is trained to reproduce late-stage outputs [31]. That is distillation used to align a cascade, not only to compress it.
- Shared trunk. xAI's Phoenix retrieval user tower "uses the same transformer trunk and input machinery as the ranking model". The two models "differ in their heads" [12]. This shares representation learning across stages while keeping the retrieval-time item side precomputable.

10.9.4 Quantization and low precision

- Embedding tables: row-wise INT8/INT4 quantization (FBGEMM) cuts memory and bandwidth for the gather-heavy part of the model [18].
- Dense compute: FP8 row-wise GEMMs on H100 (HSTU-Ultra) give 1.13-1.25x forward speedups [21]. BF16 is the common baseline.
- ANN indexes: compressed codes (PQ and variants) let FAISS scale to "billions of vectors in main memory on a single server", with GPU implementations [26]. ScaNN uses anisotropic vector quantization for maximum inner product search [27]. Semantic IDs (residual quantization) are now also used as model inputs [12].

10.10 Comparison of serving designs

System	Retrieval serving	Ranking compute	Model server / runtime	Notable serving trick
X (2023)	Earlybird index, GraphJet UTEG, CR-Mixer	parallel MaskNet MTL	Navi (Rust; TF/ONNX)	LR light ranker in index; ~6k features; batch-drop
X/xAI (2026)	Thunder in-memory, Phoenix two-tower, SimClusters	Phoenix transformer	Rust gRPC engine, JAX-trained	candidate isolation; hash embeddings; index in checkpoint
Instagram Explore	two-tower + ANN, multiple sources	2-stage: light then MTML	Meta internal	distilled first stage; value model
Meta Andromeda (ads)	deep neural retrieval + hierarchical index	n/a (retrieval stage)	Grace Hopper, MTIA	model-index-hardware co-design
Meta GR (HSTU)	generative retrieval/ranking	HSTU transformer	GPU, Triton/CUDA kernels	M-FALCON microbatching + caching; FP8
Pinterest	(various)	DNN rankers on GPU	GPU serving	batching, CUDA graphs, fused embeddings
Vespa apps	WAND / ANN in content nodes	1st, 2nd phase on nodes; global on container	Vespa + ONNX (opt. GPU)	compute near data; rerank-count knobs
Elasticsearch LTR	BM25 / kNN	LTR rescoring window	ES plugin (linear/XGBoost)	in-engine 2-stage cascade
NVIDIA Merlin	ANN (FAISS)	DLRM-style	Triton + HPS backend	GPU embedding cache hierarchy

---

10.11 Key takeaways for system designers

1. Budget per stage, not per system. Write down p99 budgets per stage (Section 10.1) and enforce them with per-branch timeouts and fallbacks. X keeps a reverse-chron backup pipeline for when the scored pipeline fails [2].
2. The serving unit is (user, many candidates). Compute user-side work once per request: two-tower precomputation, M-FALCON microbatching, Phoenix candidate isolation [12][21][31].

3. Make scores batch-invariant if you want to cache them. Listwise cross-candidate attention costs you score caching and reproducibility [11][12].
4. Push cheap compute to the data. Index-side light rankers (Earlybird), Vespa first and second phases on content nodes, and `match-features` to avoid moving vectors over the network [5][13].
5. Batch with a bounded delay and shed load. Dynamic batching (Triton `max_queue_delay_microseconds`, Navi 2 ms / 16-request defaults) plus explicit drop thresholds [4][16].
6. Embeddings are a memory-hierarchy problem. GPU cache, then DRAM, then SSD, with streaming updates. Skew makes small caches effective [19][24].
7. Feature hydration is usually the long pole. Co-locate, batch-get, use defaults on timeout, and log served features for training [2][5][25].
8. Cascades need alignment. Train early stages to agree with late stages (distillation) and measure per-stage survival, or the cascade silently caps quality [13][31][35].
9. Spend the precision budget wisely. Low precision first for embeddings (INT8/INT4) and ANN (PQ), then FP8 for dense GEMMs on modern GPUs [18][21][26].
10. Co-design moves the frontier. Andromeda (hardware + index + model) and Pinterest GPU serving show that bigger models upstream can be cheaper than more stages downstream [30][32].
11. Keep a separate, explicit value model. Combining multi-task predictions with configurable weights lets product trade-offs ship without retraining [9][11][31].
12. Simpler servers win on the hot path. Navi's minimalist Rust design and TorchRec's TorchScript C++ path show that the critical path benefits from a small, purpose-built runtime [3][17].

References

1. "X's Recommendation Algorithm" (repository README), X Corp / Twitter, 2023. https://github.com/twitter/the-algorithm
2. "Home Mixer" README, Twitter, 2023. https://github.com/twitter/the-algorithm/blob/main/home-mixer/README.md
3. "Navi: High-Performance Machine Learning Serving Server in Rust", Twitter, 2023. https://github.com/twitter/the-algorithm/blob/main/navi/README.md
4. Navi `cli_args.rs` (batching defaults), Twitter, 2023. https://github.com/twitter/the-algorithm/blob/main/navi/navi/src/cli_args.rs
5. "Earlybird Light Ranker" README, Twitter, 2023. https://github.com/twitter/the-algorithm/blob/main/src/python/twitter/deepbird/projects/timelines/scripts/models/earlybird/README.md
6. "Tweet Search System (Earlybird)" README, Twitter, 2023 (cites "Earlybird: Real-Time Search at Twitter", http://notes.stephenholiday.com/Earlybird.pdf, and "Reducing search indexing latency to one second", https://blog.twitter.com/engineering/en_us/topics/infrastructure/2020/reducing-search-indexing-latency-to-one-second). https://github.com/twitter/the-algorithm/blob/main/src/java/com/twitter/search/README.md
7. "CR-Mixer" README, Twitter, 2023. https://github.com/twitter/the-algorithm/blob/main/cr-mixer/README.md

8. "Product Mixer" README, Twitter, 2023. https://github.com/twitter/the-algorithm/blob/main/product-mixer/README.md
9. "Heavy Ranker" (Recap) README, Twitter, 2023. https://github.com/twitter/the-algorithm-ml/blob/main/projects/home/recap/README.md
10. "Twitter's Recommendation Algorithm", Twitter Engineering Blog, 2023. https://blog.x.com/engineering/en_us/topics/open-source/2023/twitter-recommendation-algorithm
11. "X For You Feed Algorithm" (x-algorithm README), xAI / X, 2025-2026. https://github.com/xai-org/x-algorithm
12. "Phoenix" README (retrieval and ranking models, Rust serving engine), xAI, 2026. https://github.com/xai-org/x-algorithm/blob/main/phoenix/README.md
13. "Phased Ranking", Vespa.ai documentation, 2025-2026. https://docs.vespa.ai/en/ranking/phased-ranking.html (source: https://github.com/vespa-engine/documentation/blob/master/en/ranking/phased-ranking.html)
14. "Vespa sample applications" README, Vespa.ai. https://github.com/vespa-engine/sample-apps
15. "Elasticsearch Learning to Rank" plugin README, OpenSource Connections. https://github.com/o19s/elasticsearch-learning-to-rank
16. "Batchers" (dynamic batching), Triton Inference Server user guide, NVIDIA. https://github.com/triton-inference-server/server/blob/main/docs/user_guide/batcher.md
17. "TorchRec Inference Library", PyTorch / Meta. https://github.com/pytorch/torchrec/blob/main/torchrec/inference/README.md
18. "The FBGEMM Project", PyTorch / Meta. https://github.com/pytorch/FBGEMM
19. "Merlin HugeCTR" README and "Hierarchical Parameter Server" docs, NVIDIA; Wei et al., "A GPU-specialized Inference Parameter Server for Large-Scale Deep Recommendation Models", RecSys 2022. https://github.com/NVIDIA-Merlin/HugeCTR ; https://github.com/NVIDIA-Merlin/HugeCTR/blob/main/docs/source/hierarchical_parameter_server/index.md ; https://dl.acm.org/doi/10.1145/3523227.3546765
20. Verma et al., "Scaling Recommendation System Inference with Merlin Hierarchical Parameter Server", NVIDIA Developer Blog, 2022. https://developer.nvidia.com/zh-cn/blog/scaling-recommendation-system-inference-with-merlin-hierarchical-parameter-server/
21. Zhai et al., "Actions Speak Louder than Words: Trillion-Parameter Sequential Transducers for Generative Recommendations", ICML 2024, and the generative-recommenders repository (HSTU, M-FALCON, DLRM-v3 inference, FP8 benchmarks), Meta. https://proceedings.mlr.press/v235/zhai24a.html ; https://github.com/facebookresearch/generative-recommenders
22. Gupta et al., "DeepRecSys: A System for Optimizing End-To-End At-scale Neural Recommendation Inference", ISCA 2020, Harvard / Facebook. http://vlsiarch.eecs.harvard.edu/wp-content/uploads/2020/05/DeepRecSys_Gupta_ISCA2020.pdf ; https://github.com/harvard-acc/DeepRecSys
23. Naumov et al., "Deep Learning Recommendation Model for Personalization and Recommendation Systems", Facebook, 2019. https://arxiv.org/abs/1906.03109
24. Liu et al., "Monolith: Real Time Recommendation System With Collisionless Embedding Table", ByteDance, 2022. https://arxiv.org/abs/2209.07663 ; https://github.com/bytedance/monolith
25. "Feast: Feature Store for Machine Learning" README, Feast project. https://github.com/feast-dev/feast

26. Johnson, Douze, Jegou, "Billion-scale similarity search with GPUs", 2017, and the FAISS README, Meta FAIR. https://arxiv.org/abs/1702.08734 ; https://github.com/facebookresearch/faiss
27. Guo et al., "Accelerating Large-Scale Inference with Anisotropic Vector Quantization" (ScaNN), Google, ICML 2020. https://github.com/google-research/google-research/tree/master/scann
28. Wang, She, Zhang, "MaskNet: Introducing Feature-Wise Multiplication to CTR Ranking Models by Instance-Guided Mask", 2021. https://arxiv.org/abs/2102.07619
29. Eugene Yan, "applied-ml: Papers & tech blogs by companies sharing their work on data science & machine learning in production" (curated index used for source discovery). https://github.com/eugeneyan/applied-ml
30. Pinterest Engineering, "GPU-accelerated ML Inference at Pinterest", 2022. https://medium.com/@Pinterest_Engineering/gpu-accelerated-ml-inference-at-pinterest-ad1b6a03a16d
31. Meta Engineering, "Scaling the Instagram Explore recommendations system", 2023. https://engineering.fb.com/2023/08/09/ml-applications/scaling-instagram-explore-recommendations-system/
32. Meta Engineering, "Meta Andromeda: Supercharging Advantage+ automation with the next-gen personalized ads retrieval engine", 2024. https://engineering.fb.com/2024/12/02/production-engineering/meta-andromeda-advantage-automation-next-gen-personalized-ads-retrieval-engine/
33. Covington, Adams, Sargin, "Deep Neural Networks for YouTube Recommendations", Google, RecSys 2016 (canonical two-stage candidate-generation/ranking serving design). https://static.googleusercontent.com/media/research.google.com/en//pubs/archive/45530.pdf
34. Wang et al., "COLD: Towards the Next Generation of Pre-Ranking System", Alibaba, 2020. https://arxiv.org/abs/2007.16122
35. Tang, Wang, "Ranking Distillation: Learning Compact Ranking Models With High Performance for Recommender System", KDD 2018. https://arxiv.org/abs/1809.07428
36. Lyft Engineering, "ML Feature Serving Infrastructure at Lyft", 2021 (further reading). https://eng.lyft.com/ml-feature-serving-infrastructure-at-lyft-d30bf2d3c32a
37. Etsy Code as Craft, "Building a Platform for Serving Recommendations at Etsy", 2022 (further reading). https://www.etsy.com/codeascraft/building-a-platform-for-serving-recommendations-at-etsy
38. DoorDash Engineering, "Building a Gigascale ML Feature Store with Redis", 2020. https://doordash.engineering/2020/11/19/building-a-gigascale-ml-feature-store-with-redis/

11. Training Data and Labeling: Impression Logs, Label Joins, Delayed Feedback, Negatives and Pipelines

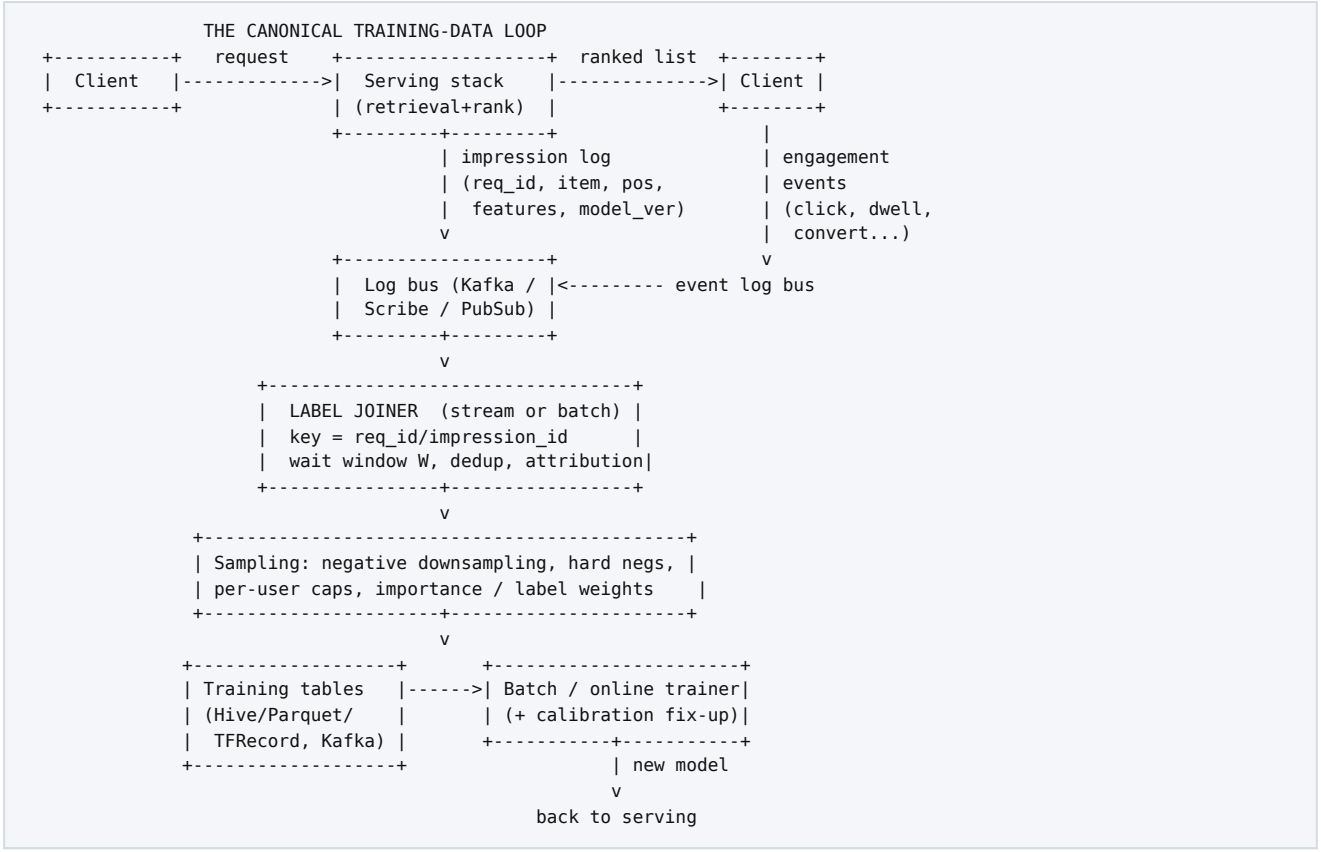
> Research note: when this chapter was compiled, live web search and page fetching were unavailable (search quota exhausted, egress blocked). Everything below comes from primary sources (papers, official engineering blogs, open-source READMEs) as the author already knew them, not from a fresh crawl. Numbers marked "approx." should be checked against the cited source before they are quoted.

Overview

People usually picture a recommender as a model, but in production most of the work and most of the bugs sit in the data that trains it. Every large public system covered in this atlas runs roughly the same loop. It serves a ranked list and logs what was shown (the impression) along with the features used at that moment. It then waits for user feedback such as clicks, watch time, purchases or hides, joins that feedback to the impression to make a labeled example, chooses or creates negatives, reweights, and feeds the result to a batch or streaming trainer. Each step forces a design decision with real consequences:

- **What counts as a positive?** A click, a "long click", watch time, a conversion within 30 days, or a survey answer.
- **When is a label final?** Conversions can arrive days after the click, and the model has to train before then.
- **What counts as a negative?** An impression that wasn't clicked, a random item, an in-batch item, or a hard negative.
- **Where do the features come from?** Logged at serving time, or reconstructed later ("time travel").
- **How fresh does the data have to be?** Daily batch, hourly, or minute-level streaming.

This chapter collects the public designs that answer these questions. It starts with impression logging and the label join (Facebook, Google Photon, ByteDance Monolith, Netflix). It then covers delayed feedback and conversion attribution (Criteo, Twitter, Alibaba, CyberAgent), negative sampling (YouTube, Google Play, Facebook Search, Pinterest, Airbnb), sample weighting and label definitions (YouTube, Twitter/X, Yahoo, Kuaishou, Pinterest), and the large data-ingestion pipelines behind them (Meta DSI, TFX/Beam, Spotify Scio). Feature stores are covered in Chapter 8 and training infrastructure in Chapter 9. Here the focus is the *example*: how it is defined, labeled, sampled, weighted and delivered.



---

1. Impression logging and the label join

1.1 Facebook Ads (2014): the Online Joiner

Problem. Click-through-rate (CTR) prediction for ads at Facebook's scale, at the time more than 750M daily active users and over 1M active advertisers. The model had to be retrained often because, as the paper reports, prediction quality decays measurably as the gap between the training data and the serving day grows [1].

Architecture. He et al. describe an *online data joiner*. Ad impressions go into one stream, keyed by a request ID, and clicks go into another. The joiner holds impressions in a window. If a click with the same key arrives within the **waiting window** the example is labeled positive, and otherwise it is emitted as a negative once the window closes. The labeled stream feeds an online trainer for the logistic-regression layer, which sits on top of boosted-decision-tree feature transforms, and the trained models are pushed back to the ranker [1].



Key tricks and lessons.

- **The waiting-window trade-off.** If the window is too short, real clicks arrive too late and are mislabeled as negatives, which biases CTR downward. If it is too long, examples reach the trainer late and the trainer needs more memory to buffer them. The paper treats this explicitly as a tuning knob and says the joiner needs to be protected against anomalies, for example by raising alarms and cutting the trainer off if the click stream stops [1].
- **Freshness matters.** The paper's experiments show normalized entropy getting worse as the training data falls days behind the evaluation day, which is the reason for daily or online retraining [1].
- **Negative downsampling plus re-calibration.** Negatives are kept at rate w, and predictions are then corrected with q = p / (p + (1 - p) / w), where p is the model's prediction in the downsampled space. Since then this formula has been standard in CTR systems [1].
- **Uniform subsampling** of all data: according to the paper, keeping about 10% of the data cost only about 1% in normalized entropy (approx.). That makes subsampling a cheap cost lever [1].

1.2 Google Photon (2013): exactly-once joins across datacenters

Problem. Google's ads system has to join a *query/impression* log with a *click* log continuously, across several datacenters, with exactly-once semantics, because each joined click is billed and also used for training [2].

Architecture. Photon joins each click to its originating query event by a unique query ID. Its core is the **IdRegistry**, a Paxos-replicated, geo-distributed key-value store holding the IDs of events that have already been joined. Every joiner checks it before emitting, so a click is never emitted twice even when several datacenters process the same click stream at once for fault tolerance. When a click arrives before its query, which is possible across datacenters, the click is retried until the query becomes visible. The paper reports throughput in the millions of joined events per minute, with end-to-end latency in the seconds range (approx.) [2].

Lessons.

- **Use stable join keys.** The serving system must mint a unique ID per request or impression and pass it through to every downstream event. This is the single most important logging decision.
- **Make the join idempotent.** Duplicate positives inflate CTR and corrupt calibration. Deduplication has to be part of the joiner's design, not a cleanup job added later.

1.3 ByteDance Monolith (2022): streaming joiner for online training

Problem. TikTok/Douyin-style recommenders need minute-level freshness. User interests shift within a session, and new videos have to be learnable right away [3].

Architecture. Monolith uses Kafka queues for (a) user actions and (b) the features logged at serving time. A **Flink online joiner** joins the two by request key and writes training examples to another Kafka queue, which the online trainer consumes. User actions can arrive long after the features, so the joiner keeps an **in-memory cache backed by an on-disk key-value store** for features still waiting for their actions. The same examples are

also written to HDFS for batch training, so a single code path produces both batch and streaming data [3].

```
serving --> [Kafka: features] --+
                                +--> Flink joiner --> [Kafka: examples] --> online
client  --> [Kafka: actions] ---+    (mem cache +                           trainer
                                      disk KV for                             |
                                     pending feats)                         | param sync
                                       |                                    v  (minutes)
                                       +--> HDFS (batch training)       serving PS
```

Key tricks.

- **Negative sampling with log-odds correction inside the joiner.** Negatives far outnumber positives, so the joiner samples them, and the bias this introduces is corrected at serving time so the prediction approximates the original distribution [3].
- **Parameter sync measured in minutes**, from the training parameter servers to the serving parameter servers. Only touched sparse IDs are pushed, and dense weights are synced less often. The paper reports that more frequent syncing measurably improves AUC [3].
- **Collisionless embedding tables** (cuckoo hashing), plus **ID frequency filtering** and **embedding expiry**. These keep the streaming tables bounded, which matters because the example stream keeps introducing new IDs [3].

1.4 Netflix: from "time travel" snapshots to a logged fact store

Problem. If features are computed offline when a new model is trained, they reflect *today's* state of the world and not the state at the moment the recommendation was made. That is label leakage plus training/serving skew.

DeLorean (2016). Netflix's first answer was to snapshot the online facts that features are computed from (viewing history, My List, and similar) at regular intervals. A service calls the online microservices, through the Prana sidecar, for a sampled set of contexts (profile, time, device) and stores the snapshots on S3 as Parquet. Feature generation then runs in Spark: for a given **context set** and **label set**, it "time travels" by loading the snapshot for that timestamp and running the *same* feature encoders that the online system uses [4].

Evolution (2022, Axion fact store). Snapshotting turned out to be expensive and covered only a sample of contexts. Netflix moved to logging the facts that were actually used *at compute time*. The online system logs the facts, Axion stores and indexes them, and offline feature generation reads the logged facts and reuses the shared feature-encoder library. The blog describes how the storage design evolved: Iceberg tables, and an EVCache layer for low-latency lookups during offline jobs [5].

Lesson. This is the clearest public account of the industry-wide move from **"backfill features offline" to "log and wait"**. Google's Rules of ML gives the same advice. Rule #29 says the best way to train like you serve is to save the features used at serving time and pipe them to a log for training, and notes that YouTube Home switched to serving-time feature logging and saw significant quality gains with less code complexity [6].

1.5 Comparison: label-join designs

System	Join type	Join key	Waiting / attribution	Freshness	Notable trick
Facebook Ads 2014 [1]	Streaming joiner	request ID	fixed waiting window W	online LR + daily trees	negative downsampling + recalibration
Google Photon [2]	Streaming, multi-DC	query ID	until matching query visible	seconds (approx.)	Paxos IdRegistry for exactly-once
ByteDance Monolith [3]	Flink streaming	request key	cache + disk KV for late actions	minutes	in-joiner neg sampling, PS sync
Netflix DeLorean [4]	Batch, Spark	context set + timestamp	n/a (offline labels)	daily	snapshot-based time travel
Netflix Axion [5]	Batch over logged facts	logged fact keys	n/a	daily	log facts at compute time
Twitter delayed fb [7]	Streaming, no wait	impression	none: negatives now, positives later	real time	fake negatives + loss correction
Alibaba ES-DFM / DEFER [8][9]	Streaming	click / impression	short window e + duplicate	near real time	elapsed-time sampling, real negatives

2. Delayed feedback and conversion attribution

Conversions (purchases, installs, subscriptions) can arrive minutes or weeks after the click. A trainer that waits for the full attribution window (often 7 to 30 days) gets the right labels but uses stale data. A trainer that does not wait sees **fake negatives**: clicks that will convert later but are labeled 0 for now. Four families of fixes appear in the public literature.

2.1 Criteo: joint conversion and delay model (Chapelle, KDD 2014)

Chapelle's paper started this line of work [10]. Each click x has a latent outcome C (whether it will ever convert). If C = 1, the conversion arrives after a delay D, modeled with an exponential distribution whose rate depends on x. At training time, a click that has not converted after elapsed time e is either a real negative or a positive whose delay exceeds e. The likelihood combines the two models:

- converted at delay d: P(C=1|x) * lambda(x) * exp(-lambda(x) d)
- not converted after elapsed e: 1 - P(C=1|x) + P(C=1|x) * exp(-lambda(x) e)

Both models are logistic/linear and are trained jointly, either by EM or by direct gradient optimization. This lets the system train on recent clicks without treating every unconverted click as a negative. The paper uses Criteo display-advertising data, and Criteo released a related conversion-log dataset for research [10].

2.2 Twitter: fake negatives with loss correction (Ktena et al., RecSys 2019)

Setting. Ads at Twitter, with continuous training of neural networks [7].

Design. Instead of waiting, each engagement-eligible impression is ingested **immediately as a negative**. When the positive engagement arrives, the example is ingested **again as a**

positive. This is the "fake negative" pipeline. It has no waiting window and the best possible freshness, but the training distribution is biased: positives appear both as negatives and as positives. The paper derives and compares several losses that undo the bias [7]:

- **Fake-negative weighted (FN-weighted):** importance-sample each example with weights derived from the model's own p(y=1|x), so the loss matches the true distribution in expectation.
- **Fake-negative calibration (FN-calibration):** train on the biased data and invert the bias at prediction time, p = b / (1 - b), where b is the biased prediction.
- **Positive-unlabeled (PU) loss** and a **delayed-feedback loss** in the style of Chapelle's model.

According to the paper, FN-weighted and FN-calibration gave the best results in offline experiments on Twitter data and in an online A/B test. They outperformed both waiting-window baselines and the delayed-feedback model [7].

```
t0 impression ------> emit (x, y=0)           <- fake negative, immediately
t1 click/convert ---> emit (x, y=1)           <- duplicate as positive
trainer: loss reweights each example by f(p_model(x)) to undo the duplication
```

2.3 Feedback-shift importance weighting (CyberAgent, WWW 2020)

Yasui et al. frame delayed conversions as a **feedback shift**: the label distribution at training time differs from the label distribution at test time. Their importance-weighting method (FSIW) trains an auxiliary model that estimates the probability that an observed negative is a real negative given its elapsed time. Each example is then reweighted with those estimates [11].

2.4 Alibaba: elapsed-time sampling (ES-DFM) and real negatives (DEFER)

ES-DFM (AAAI 2021). This approach sits between "wait for everything" and "wait for nothing". Yang et al. wait a short **elapsed window e**. A click that converts within e is emitted once as a positive. Otherwise it is emitted as a negative, and if it converts later it is re-emitted as a positive. Importance weights derived from the modeled elapsed-time distribution correct the resulting bias [8].

DEFER (KDD 2021). Gu et al. point out that in fake-negative schemes, real negatives never get their own corrected entry. DEFER therefore **re-ingests real negatives too** once the attribution window closes, so every sample appears with its final label in addition to the early copy, and an importance-sampling loss accounts for the duplication. The paper reports offline and online gains in Alibaba's display-advertising system, and gives variants of the design for different attribution-window lengths [9].

Later work such as DEFUSE (WWW 2022) refines unbiased estimation for the duplicated-sample streams. The common pattern is to emit early, emit again when the label changes, and correct with importance weights.

2.5 Sample selection bias in conversion: ESMM (Alibaba, SIGIR 2018)

A separate problem from delay is that conversion-rate (CVR) models are traditionally trained only on *clicked* impressions but are used to score *all* impressions. ESMM (Entire Space Multi-task Model) trains two towers over the **entire impression space**. One predicts pCTR and the other predicts pCVR, and the supervised outputs are pCTR and pCTCVR = pCTR * pCVR. In this way, CVR is learned implicitly from impression-level data and the selection bias is avoided. Alibaba released the Ali-CCP dataset with this paper [12].

2.6 Comparison of delayed-feedback strategies

Strategy	Freshness	Label noise	Extra storage	Correction	Source
Wait full window (7-30 d)	worst	none	buffer all clicks	none	baseline
Fixed short window W	good	fake negs after W	buffer W	none / calibration	[1]
Joint delay model	good	modeled	none	survival likelihood	[10]
Fake negatives (no wait)	best	high, corrected	duplicates positives	FN-weight / FN-calib	[7]
FSIW	good	modeled	none	aux elapsed-time model	[11]
Elapsed-time sampling	good	moderate	duplicates late positives	importance weights	[8]
Real-negative re-ingest	good	lowest	duplicates all	importance weights	[9]

---

3. Negative sampling strategies

The right negative-sampling strategy depends on the stage. **Ranking** models see real impressions, so the negatives are shown-but-not-engaged items, usually downsampled. **Retrieval** models (two-tower) score the whole corpus, so they need negatives drawn from the corpus, and impressions alone are a biased choice.

3.1 YouTube candidate generation (Covington et al., RecSys 2016)

- **Extreme multiclass formulation.** The model predicts the next watch out of millions of videos with a sampled softmax. According to the paper, it samples several thousand negatives and corrects with importance weighting [13].
- **Training examples come from all watches**, including watches of YouTube videos embedded on other sites, and not only from recommendation impressions. This keeps the model from just reinforcing the current recommender's exploitation loop [13].
- **Fixed number of examples per user.** This keeps a small group of very active users from dominating the loss [13].
- **Predict the *next* watch rather than a random held-out watch**, and withhold information that would leak the label (for example, the search query that immediately led to the watch). Episodic viewing makes "future" leakage very harmful [13].
- **"Example age" feature.** The model gets the age of each training example as a feature, set to zero (or slightly negative) at serving time. This removes the bias

toward older content that comes from training on past windows, and it shows directly how dataset freshness enters the model [13].

3.2 Sampling-bias-corrected in-batch softmax (YouTube, RecSys 2019)

Two-tower retrieval models commonly use **in-batch negatives**: the other items in the mini-batch serve as negatives. That is cheap, but popular items appear as negatives far more often, so the model over-penalizes them. Yi et al. apply the **logQ correction**: subtract log(p_j) from each logit, where p_j is the item's sampling probability. Because the item vocabulary changes in a streaming setting, they also propose a **streaming frequency estimator**. Two hashed arrays, A and B, record the last step at which an item was seen and a moving average of the gap between sightings, and p_j is approximately 1/B[h(j)]. The paper reports gains in offline recall and in live YouTube experiments [14].

3.3 Mixed negative sampling (Google Play, WWW 2020)

In-batch negatives contain only items that someone engaged with, so long-tail items rarely appear as negatives. **Mixed Negative Sampling (MNS)** adds a second set of negatives sampled **uniformly from the whole corpus** and combines them with the in-batch negatives in one softmax. This reduces the selection bias and improves retrieval of fresh and long-tail apps in Google Play experiments [15].

3.4 Facebook Search embedding-based retrieval (KDD 2020)

Huang et al. give some of the most quoted negative-sampling results [16]:

- **Random negatives beat "impressed but not clicked" negatives for retrieval.** The authors found that using non-click impressions as negatives clearly lowered recall compared with random negatives. Their explanation is that retrieval has to separate relevant items from the whole corpus, while impressed items are already near-relevant (the reported drop was large; see paper for exact figure).
- **Hard negative mining, online and offline.** Online: within a batch, use the highest-scoring non-positive items as hard negatives. Offline: retrieve with the current model and take results from a middle rank band (for example, around rank 101-500), because the very top ranks contain unlabeled positives.
- **Mix easy and hard negatives.** Hard negatives alone hurt the model. The paper found blends heavily weighted toward easy/random negatives worked best (on the order of 100:1 easy:hard, approx.).
- **Hard positives.** Positives mined from failed searches that the production system had not retrieved.

3.5 PinSage hard negatives and curriculum (Pinterest, KDD 2018)

PinSage trains with a max-margin loss on (query, positive) pairs taken from co-engagement. Negatives are mostly random, but **hard negatives** are added. These are items that are related to the query but not as related as the positive, drawn from roughly ranks 2000-5000 of personalized PageRank scores for the query. **Curriculum training** adds one more hard negative per epoch. The paper reports that the hard negatives roughly doubled the number of epochs needed to converge but improved quality [17].

3.6 Airbnb listing embeddings (KDD 2018)

Grbovic and Cheng train skip-gram embeddings on **click sessions** [18]:

- **Session definition.** Sessions are split when two clicks are more than 30 minutes apart. Only clicks where the user stayed on the listing page long enough (about 30 seconds, approx.) count, which removes accidental clicks. This is a dwell-based implicit-feedback filter.
- **Booked listing as global context.** The eventually booked listing is added to the context of every listing in the session, so the embeddings are pulled toward conversion.
- **Market-aware negatives.** Random negatives come from the whole world, so they are trivially easy: a Paris listing versus a Tokyo listing. Airbnb also samples negatives **from the same market** as the central listing, which forces the model to learn distinctions within a market.
- **Rejections as explicit negatives** for user-type and listing-type embeddings. Host rejections are used as negatives.

3.7 Ranking-stage downsampling and calibration

For ranking, negatives are the impressions the user did not engage with, and the main question is **how many to keep**. Facebook's uniform negative downsampling with the q = p/(p + (1-p)/w) correction [1] and Monolith's log-odds correction [3] are the standard public designs. Rule of thumb: downsampling negatives changes the *base rate*, so any consumer that uses probabilities directly (an auction, a weighted-sum value model, multi-task blending) must recalibrate. Ranking-only consumers do not need to.

3.8 Negative sampling summary

Negative type	Stage	Pros	Cons	Public examples
Non-engaged impressions	ranking	true counterfactual of exposure	position/selection bias; useless for retrieval	[1], [3]
Uniform random corpus	retrieval	covers long tail; unbiased	too easy alone	[15], [16]
In-batch	retrieval	free; scales	popularity bias, needs logQ	[14]
Mixed (in-batch + uniform)	retrieval	balances both	extra compute	[15]
Hard (model-mined)	retrieval	sharper boundaries	false negatives; slower convergence	[16], [17]
Same-market / contextual	retrieval / embeddings	in-context discrimination	domain-specific	[18]
Explicit (hide, reject, report)	both	strong signal	sparse	[18], [19]

---

4. Label definitions and implicit feedback

4.1 Clicks are not enough: dwell time (Yahoo, RecSys 2014)

Yi et al. ("Beyond Clicks") argue that clicks on news and content streams are noisy proxies for satisfaction, and propose **dwell time** as the training signal. Their practical findings [20]:

- Dwell time must be **computed from client-side events**: page focus and blur, scrolling and app backgrounding. Naive server-side timestamp differences are inflated by idle tabs.
- Raw dwell time differs systematically by **content type** (article, slideshow, video), **content length** and **device**. They normalize dwell time per context, for example by z-scoring the log of dwell time within each group, before using it as a label or weight.
- Models trained on normalized dwell did better than click-trained models on engagement metrics.

4.2 YouTube: from clicks to watch time to "valued" watch time

Weighted logistic regression (2016). YouTube's ranking model predicts *expected watch time* using weighted logistic regression. Positive (clicked) impressions are weighted by observed watch time and negatives have unit weight. The learned odds then approximate E[T] * (1 + P), which is roughly E[T] when CTR P is small, so serving uses exp(logit) as the expected watch time [13]. This is the standard example of **turning a continuous engagement signal into a sample weight**.

Satisfaction (2021). YouTube's public explanation of its recommender describes how the optimization target moved: first clicks (which encouraged clickbait), then watch time, then "valued watch time" measured by **user surveys** ("How satisfied were you with this video?"). Only a small fraction of views get a survey, so a model is trained to predict survey responses across all watches, and that prediction becomes a label or objective for the recommender [21].

4.3 Short-video watch time: duration bias (Kuaishou, KDD 2022)

In short-video feeds, longer videos mechanically collect more watch time, so raw watch time as a label favors long content. D2Q (Duration-Deconfounded Quantile) splits videos into **duration buckets** and trains the model to predict the *quantile* of watch time within each bucket. This removes the confounding effect of duration while keeping the useful signal [22]. Industry teams also often define binary labels such as "effective view" or "complete play" from watch ratio thresholds that depend on duration (thresholds are company-specific and seldom published).

4.4 Twitter/X: a weighted portfolio of engagement labels

The 2023 open-sourcing of Twitter's recommendation code made public a heavy ranker that predicts about ten engagement probabilities. The final score is a **weighted sum** of those probabilities, with weights published in the README [19][23]. Approximate values from the April 2023 README: like 0.5, retweet 1.0, reply 13.5, profile click then engagement 12.0, "good click" 11.0, reply that the author engages with 75.0, video watched at least 50% 0.005, negative feedback (show less, block, mute) -74.0, report -369.0 [19].

Two label definitions are worth noting:

- **"Good click"** is a click into the conversation followed by a reply or like, or by staying for at least a few minutes (about 2 minutes in one variant). This is a dwell-qualified click, built in the same spirit as Airbnb's and Yahoo's dwell filters.
- **Negative labels with large negative weights** let rare explicit negative feedback outweigh many positive engagements.

4.5 Facebook News Feed: multi-signal labels and survey-derived targets

Meta's description of News Feed ranking lists many predicted events: like, comment, share, hide, and "worth your time" survey responses. Each is a separate binary label from the joined impression stream, combined into a value score with tunable weights [24]. As at YouTube, survey responses are used to calibrate and validate engagement-based objectives against what users say they value.

4.6 Pinterest PinnerFormer: long-horizon labels

PinnerFormer learns a user embedding meant to predict the user's positive engagements over a **future window** (up to 28 days), not just the next action. The "dense all action" loss uses every positive action in the window as a target. Pinterest counts repins, close-ups and long clicks as positive actions (outbound clicks with long dwell, so short bounces are excluded). Training data is produced in daily batches, and the paper reports that a daily-refreshed embedding largely matched a real-time sequence model while being much cheaper to serve [25].

4.7 Position and selection bias in labels

Logged labels depend on what was *shown* and *where*. Two public lines of work are standard references:

- **Google personal search (Wang et al., SIGIR 2016):** estimate position propensities from result randomization and reweight clicks by inverse propensity [26].
- **Unbiased learning-to-rank (Joachims et al., WSDM 2017):** a counterfactual inverse-propensity-scoring framework for learning from biased click logs [27].

In practice, recommender pipelines handle this in one of three ways. They log **position** and other presentation features into the impression record, then set them to a constant at serving time (the same idea as YouTube's "example age"). They weight examples by inverse propensity. Or they keep a small **randomized exploration slice** of traffic to provide propensity estimates and unbiased evaluation data.

---

5. Sample weighting

Sample weights in recommender training data fall into five groups:

1. **Correction weights**, which undo a sampling choice. Examples are 1/w for downsampled negatives (or recalibration of the outputs instead) [1], logQ for in-batch sampling [14], importance weights for fake and duplicated delayed-feedback samples [7][8][9][11], and inverse propensity for position bias [26][27].

2. **Value weights**, which encode how much an outcome is worth. Examples are watch time as the weight on positives [13] and a value-weighted sum of predicted engagements [19][24].
3. **User-balance weights.** YouTube's fixed number of examples per user [13] can be implemented as truncation or as per-user weights. This keeps heavy users from dominating the loss and, in doing so, shapes whose preferences the model learns.
4. **Recency weights**, which up-weight recent examples in batch training to approximate freshness. These are often used as a cheap substitute for a streaming pipeline.
5. **Label-quality weights**, which down-weight noisy labels such as short-dwell clicks or bot traffic, or drop them entirely as Airbnb does with its dwell filter [18].

A practical warning that runs through all of these sources: **every weight changes the calibration of the output.** Any downstream system that uses scores as probabilities (ads auctions, multi-objective blending, thresholds) needs a calibration stage after weighting, and calibration needs to be monitored per segment.

---

6. Dataset freshness, retraining cadence and leakage

6.1 Evidence that freshness matters

- **Facebook Ads:** model quality degrades as the training data falls days behind, which supports daily or online training [1].
- **Monolith:** shorter parameter-sync intervals between training and serving give higher AUC, measured in production-like experiments [3].
- **YouTube:** the "example age" feature lets the model learn time-dependent popularity, especially for fresh uploads, so it does not average over the training window [13].
- **Pinterest PinnerFormer:** shows the opposite case. With a properly defined long-horizon objective, a *daily* batch embedding can come close to real-time quality [25]. How much freshness you need depends on the objective.

6.2 Training/serving skew and leakage

Rules of ML [6] gives the most compact list of failure modes:

- **Rule #29:** log features at serving time ("log and wait") to get exact training/serving parity.
- **Rule #31:** if training joins against a table that changes over time, the features seen in training will differ from those seen at serving.
- **Rule #32:** reuse code between the training and serving pipelines.
- **Rule #33:** test on data collected *after* the training window (temporal split), not on random splits.

Netflix's DeLorean and Axion [4][5], Feathr's point-in-time joins at LinkedIn [28], and Monolith's serving-time feature logging [3] are all ways of putting these rules into practice.

```
LEAKAGE TIMELINE (what a point-in-time join must enforce)

  feature_ts <= impression_ts  <  label_ts <= impression_ts + attribution_window
  |------- allowed features ---|-- label collection --|
                               ^ impression            ^ label final
  Using any feature value with ts > impression_ts = leakage.
```

---

7. Data pipelines at scale

7.1 Meta: Data Storage and Ingestion (DSI), ISCA 2022

Zhao et al. describe the end-to-end pipeline that feeds Meta's DLRM-style recommendation trainers, and the finding that the resources spent on **data storage and ingestion can be comparable to, or exceed, those spent on GPU training** [29].

Architecture [29]:

- **Logging.** Serving hosts log features and events through **Scribe**, Meta's distributed log transport.
- **ETL and label join.** Streaming and batch jobs join features with labels and write partitioned **Hive tables** (columnar DWRF, a variant of ORC) to **Tectonic**, Meta's exabyte-scale distributed filesystem.
- **DPP (Data PreProcessing service).** A disaggregated fleet of CPU workers reads columnar data, runs feature transforms (filtering, hashing, normalization) and streams tensors to trainers. This frees trainer hosts from the preprocessing bottleneck.
- **Feature-level reads.** Each model reads only a subset of the thousands of logged features, so column projection and file layout (feature flattening, coalesced reads) dominate I/O efficiency.

```
serving --> Scribe --> streaming/batch ETL (label join) --> Hive tables (DWRF)
                                                              on Tectonic
                                                                  |
                                  +-------------------------------+
                                  v
                      DPP workers (CPU fleet: read cols, decode,
                      transform, batch)  ==tensors==>  GPU trainers
```

Lessons [29]:

- Data volume grows faster than compute, and datasets are reported in the petabyte range per model family (approx.).
- Many trainings run on the same tables (exploration, hyperparameter sweeps, combo jobs), so storage and decoding efficiency become fleet-level costs.
- Preprocessing has to scale independently of training. Hence the disaggregated DPP.

Related Meta work deduplicates training samples within a session, since many impressions from one request share identical user features, to cut ingestion and training cost. This general idea of **request-level deduplication** works in any pipeline that logs per impression.

7.2 Google TFX and Apache Beam

TensorFlow Extended (TFX) builds the training-data pipeline from components (ExampleGen, StatisticsGen, SchemaGen, ExampleValidator, Transform). These run on **Apache Beam** for distributed processing, typically on Google Cloud Dataflow. The pattern provides: (a) schema inference and **data validation** that catch skew and drift before training; (b) **tf.Transform**, which computes full-pass statistics such as vocabularies and normalization constants in Beam and exports them as a TensorFlow graph that runs identically at serving time, closing one major source of skew [30].

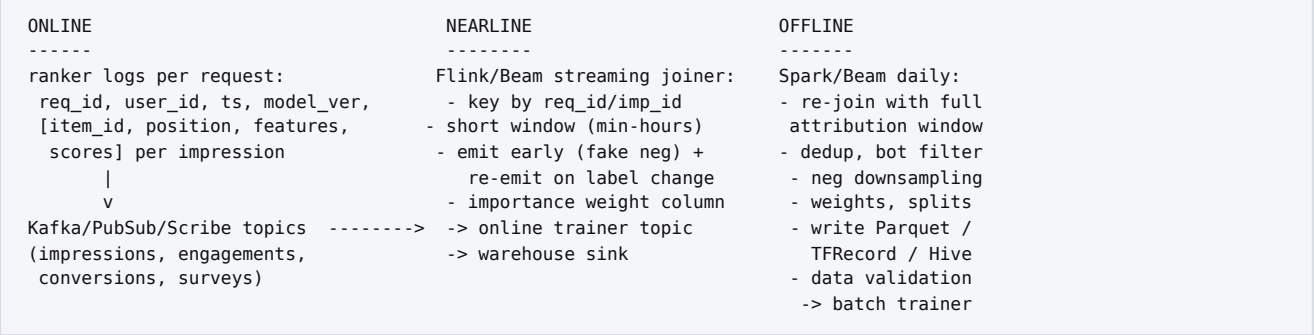
7.3 Spotify Scio

Spotify built and open-sourced **Scio**, a Scala API for Apache Beam that runs on Dataflow. It is the backbone of much of Spotify's batch and streaming data processing, including generating training data for personalization models [31]. Scio is one public example of a typed, functional Beam API used at scale for recommendation data.

7.4 Spark-based pipelines

Spark is the most common public substrate for batch label joins and feature generation. Netflix DeLorean [4] and Axion [5] run offline feature generation in Spark. LinkedIn's Feathr provides point-in-time-correct feature joins on Spark, used to build training datasets from observation data (impressions with timestamps) and timestamped feature sources [28]. In the typical Spark design, impressions are the "observation" table, labels are joined with a time-bounded window, and features are joined "as of" the impression time.

7.5 Reference pipeline (synthesis)



---

8. Key takeaways for system designers

1. **Mint an impression ID at serving time and pass it through every downstream event.** Photon, Facebook's online joiner and Monolith all depend on a stable join key [1][2][3]. If you get this wrong, the label join becomes guesswork based on timestamps.
2. **Log features at serving time ("log and wait")** instead of reconstructing them offline. Netflix moved from snapshot time travel to logged facts, and Google's Rules of ML explicitly recommends logging [4][5][6].
3. **Treat the label window as a first-class hyperparameter.** For clicks, a short waiting window works. For conversions, use a delayed-feedback method: fake

negatives with FN weighting or calibration [7], elapsed-time sampling [8], real-negative re-ingest [9], or a delay model [10].

4. **Choose negatives for the stage.** Ranking uses non-engaged impressions, downsampled and then recalibrated [1]. Retrieval uses corpus negatives: in-batch with logQ [14], plus uniform [15], plus a small dose of hard negatives [16][17]. Using impressions as retrieval negatives is a known way to lose recall [16].
5. **Define implicit labels with dwell or quality thresholds and normalize by context.** Raw clicks reward clickbait, and raw watch time rewards long videos [13][20][21][22]. Dwell-qualified clicks (Twitter's "good click", Airbnb's session filter, Pinterest's long clicks) are now standard [18][19][25].
6. **Keep explicit negative feedback as heavily weighted labels.** It is rare but highly informative (Twitter's -74 and -369 weights) [19].
7. **Every sampling and weighting choice moves calibration.** Recalibrate after downsampling and weighting [1][3], and monitor calibration per segment.
8. **Balance users and correct exposure bias.** Use per-user caps [13], log position and set it to a constant at serving time, and keep a small randomized slice for propensity estimates [26][27].
9. **Match freshness to the objective.** Minute-level streaming pays off for fast-moving feeds and ads [3][7], while long-horizon objectives can run on daily batches [25]. Measure the decay curve before building streaming infrastructure [1].
10. **Budget for data ingestion as much as for training.** At Meta's scale, storage, decoding and preprocessing are comparable to GPU training in resources. Use columnar storage, column projection, disaggregated preprocessing and request-level deduplication [29].
11. **Validate the data before training.** Schema checks, distribution-drift checks and transform code shared between training and serving (TFX/tf.Transform) catch silent pipeline breaks that offline AUC will not [30].

References

1. He, X., Pan, J., Jin, O., et al. "Practical Lessons from Predicting Clicks on Ads at Facebook." ADKDD 2014, Facebook. https://dl.acm.org/doi/10.1145/2648584.2648589
2. Ananthanarayanan, R., et al. "Photon: Fault-tolerant and Scalable Joining of Continuous Data Streams." SIGMOD 2013, Google. https://research.google/pubs/photon-fault-tolerant-and-scalable-joining-of-continuous-data-streams/
3. Liu, Z., et al. "Monolith: Real Time Recommendation System With Collisionless Embedding Table." 2022, ByteDance. https://arxiv.org/abs/2209.07663
4. Netflix Technology Blog. "Distributed Time Travel for Feature Generation." 2016, Netflix. https://netflixtechblog.com/distributed-time-travel-for-feature-generation-389ccdd3907
5. Netflix Technology Blog. "Evolution of ML Fact Store." 2022, Netflix. https://netflixtechblog.com/evolution-of-ml-fact-store-5941d3231762
6. Zinkevich, M. "Rules of Machine Learning: Best Practices for ML Engineering." Google. https://developers.google.com/machine-learning/guides/rules-of-ml
7. Ktena, S. I., Tejani, A., Theis, L., et al. "Addressing Delayed Feedback for Continuous Training with Neural Networks in CTR Prediction." RecSys 2019, Twitter. https://arxiv.

org/abs/1907.06558
8. Yang, J.-Q., et al. "Capturing Delayed Feedback in Conversion Rate Prediction via Elapsed-Time Sampling." AAAI 2021, Alibaba. https://arxiv.org/abs/2012.03245
9. Gu, S., et al. "Real Negatives Matter: Continuous Training with Real Negatives for Delayed Feedback Modeling." KDD 2021, Alibaba. https://arxiv.org/abs/2104.14121
10. Chapelle, O. "Modeling Delayed Feedback in Display Advertising." KDD 2014, Criteo. https://dl.acm.org/doi/10.1145/2623330.2623634
11. Yasui, S., Morishita, G., Fujita, K., Shibata, M. "A Feedback Shift Correction in Predicting Conversion Rates under Delayed Feedback." WWW 2020, CyberAgent. https://dl.acm.org/doi/10.1145/3366423.3380032
12. Ma, X., et al. "Entire Space Multi-Task Model: An Effective Approach for Estimating Post-Click Conversion Rate." SIGIR 2018, Alibaba. https://arxiv.org/abs/1804.07931
13. Covington, P., Adams, J., Sargin, E. "Deep Neural Networks for YouTube Recommendations." RecSys 2016, Google. https://research.google/pubs/deep-neural-networks-for-youtube-recommendations/
14. Yi, X., Yang, J., Hong, L., et al. "Sampling-Bias-Corrected Neural Modeling for Large Corpus Item Recommendations." RecSys 2019, Google. https://research.google/pubs/sampling-bias-corrected-neural-modeling-for-large-corpus-item-recommendations/
15. Yang, J., Yi, X., et al. "Mixed Negative Sampling for Learning Two-tower Neural Networks in Recommendations." WWW 2020 Companion, Google. https://research.google/pubs/mixed-negative-sampling-for-learning-two-tower-neural-networks-in-recommendations/
16. Huang, J.-T., et al. "Embedding-based Retrieval in Facebook Search." KDD 2020, Facebook. https://arxiv.org/abs/2006.11632
17. Ying, R., He, R., Chen, K., et al. "Graph Convolutional Neural Networks for Web-Scale Recommender Systems" (PinSage). KDD 2018, Pinterest. https://arxiv.org/abs/1806.01973
18. Grbovic, M., Cheng, H. "Real-time Personalization using Embeddings for Search Ranking at Airbnb." KDD 2018, Airbnb. https://dl.acm.org/doi/10.1145/3219819.3219885
19. Twitter. "the-algorithm-ml: Heavy Ranker README (projects/home/recap)." 2023, Twitter/X. https://github.com/twitter/the-algorithm-ml/blob/main/projects/home/recap/README.md
20. Yi, X., Hong, L., Zhong, E., Liu, N. N., Rajan, S. "Beyond Clicks: Dwell Time for Personalization." RecSys 2014, Yahoo. https://dl.acm.org/doi/10.1145/2645710.2645724
21. Goodrow, C. "On YouTube's recommendation system." 2021, YouTube Official Blog. https://blog.youtube/inside-youtube/on-youtubes-recommendation-system/
22. Zhan, R., et al. "Deconfounding Duration Bias in Watch-time Prediction for Video Recommendation." KDD 2022, Kuaishou. https://arxiv.org/abs/2206.06003
23. Twitter Engineering. "Twitter's Recommendation Algorithm." 2023, Twitter/X. https://blog.x.com/engineering/en_us/topics/open-source/2023/twitter-recommendation-algorithm
24. Lada, A., Wang, M., Yan, T. "How machine learning powers Facebook's News Feed ranking algorithm." 2021, Meta Engineering. https://engineering.fb.com/2021/01/26/ml-applications/news-feed-ranking/
25. Pancha, N., Zhai, A., Leskovec, J., Rosenberg, C. "PinnerFormer: Sequence Modeling for User Representation at Pinterest." KDD 2022, Pinterest. https://arxiv.org/abs/2205.

04507
26. Wang, X., Bendersky, M., Metzler, D., Najork, M. "Learning to Rank with Selection Bias in Personal Search." SIGIR 2016, Google. https://research.google/pubs/learning-to-rank-with-selection-bias-in-personal-search/
27. Joachims, T., Swaminathan, A., Schnabel, T. "Unbiased Learning-to-Rank with Biased Feedback." WSDM 2017, Cornell. https://arxiv.org/abs/1608.04468
28. LinkedIn / Feathr AI. "Feathr: A scalable, unified data and AI engineering platform" (point-in-time feature joins). 2022, LinkedIn. https://github.com/feathr-ai/feathr
29. Zhao, M., et al. "Understanding Data Storage and Ingestion for Large-Scale Deep Recommendation Model Training." ISCA 2022, Meta. https://arxiv.org/abs/2108.09373
30. TensorFlow. "TensorFlow Extended (TFX)" documentation (ExampleGen, Transform, data validation on Apache Beam). Google. https://www.tensorflow.org/tfx
31. Spotify. "Scio: A Scala API for Apache Beam and Google Cloud Dataflow." Spotify (open source). https://github.com/spotify/scio

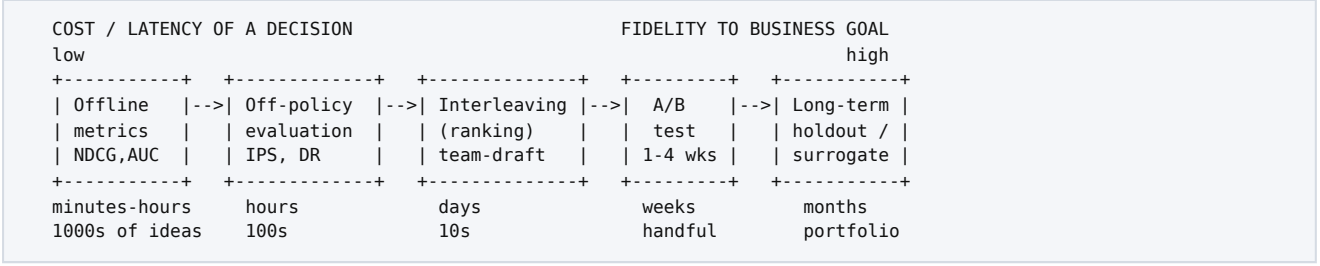
12. Evaluation and Experimentation: Offline Metrics, Off-Policy Evaluation, A/B Platforms, Interleaving and Long-Term Measurement

> Research note: live web search and page fetching were unavailable when this chapter was compiled. The search quota was used up and outbound DNS to the source sites failed. Everything below comes from primary sources (papers, official engineering blogs, books) as the author already knew them, not from a fresh crawl. Numbers marked "approx." should be checked against the cited source before anyone quotes them. URLs are listed only where the author was confident they are canonical.

Overview

The quality of a recommender is only as believable as the measurement behind it. Every mature recommendation organization covered in this atlas, including Netflix, Airbnb, LinkedIn, Spotify, Google/YouTube, Microsoft, Booking.com, Criteo and Uber, has ended up with the same layered evaluation stack:

1. **Offline metrics.** NDCG, recall@K, MAP, AUC, log loss / normalized entropy and calibration are computed on held-out logs. They are cheap and fast, but biased by the logging policy. Teams use them to screen ideas, not to make launch decisions.
2. **Counterfactual / off-policy evaluation (OPE).** IPS, self-normalized IPS, the direct method and doubly robust estimators reweight logged interactions to estimate how a *new* policy would have performed. They need logged propensities and enough exploration.
3. **Interleaving.** Two rankers are merged into one list for the same user, and attributed engagement shows which one the user prefers. It is roughly one to two orders of magnitude more sensitive than A/B testing for ranking changes, and teams use it to triage candidates quickly.
4. **A/B testing.** Randomized controlled experiments measure business metrics such as retention, streaming hours, bookings and sessions. This is the final arbiter, run on platforms like Netflix's ABlaze, Airbnb's ERF, LinkedIn's XLNT/T-REX, Spotify's Experimentation Platform/Confidence, Google's overlapping layers and Microsoft ExP.
5. **Long-term measurement.** Holdouts, surrogate indices and long-term proxy metrics address the fact that most launches are decided on 1 to 4 weeks of data, while the metrics that matter (retention, LTV) move over months.



The funnel is the main architectural idea of this chapter. Each stage is cheaper but less faithful than the next, so it is used to cut down the candidate set for the stage after it. The rest of the chapter goes through the stages in order, then the long-term layer, and ends with design lessons.

12.1 Offline metrics

12.1.1 Ranking metrics: NDCG, recall@K, MRR, MAP

NDCG (normalized discounted cumulative gain) comes from Järvelin and Kekäläinen [1]. DCG@K = sum over positions i <= K of (2^rel_i - 1) / log2(i + 1). That sum is then divided by the ideal DCG, which is the DCG of the best possible ordering. NDCG rewards putting highly relevant items near the top, and it is the default listwise metric for search and feed ranking. Airbnb, for example, reports NDCG of the booked listing over its search logs as the main offline signal for search ranking models [2].

Recall@K / HitRate@K measures what fraction of the held-out positives appear in the top K. It is the standard metric for *retrieval* stages, where the question is "did the candidate generator surface the item at all", not "is it in the right position". **MRR** (mean reciprocal rank) fits single-answer settings. **MAP** averages precision at each relevant position.

Pointwise metrics for rankers include AUC, log loss, and Facebook's *normalized entropy* (NE). NE is log loss divided by the entropy of the background CTR, which makes it comparable across datasets with different base rates [3]. Google's ads paper [4] relies on log loss and AUC and also tracks calibration slices. Both papers stress that tiny relative improvements in log loss/NE (fractions of a percent) can matter online.

12.1.2 Calibration

A ranking model can have excellent AUC and still be badly miscalibrated. That breaks every downstream consumer that treats scores as probabilities: auctions (bid x pCTR), multi-task value models that add pCTR, pLike and pWatch, and thresholds. Production teams track:

- **Global calibration ratio**: sum of predicted / sum of observed (Facebook reports "calibration" this way [3]).
- **Calibration by slice and by predicted-score bucket**: a reliability diagram, as in Google's ads paper, which lists calibration plots across many slices as a core monitoring tool [4].

Steck's **Calibrated Recommendations** [5] (Netflix, RecSys 2018) gives the word a second, distributional meaning. A recommendation list is *calibrated* if its genre distribution q(g) matches the user's historical distribution p(g). Steck measures miscalibration with KL

divergence and re-ranks greedily to maximize (1 - lambda) * utility - lambda * KL(p || q~), where q~ is a slightly smoothed q. The paper shows that accuracy-trained models tend to crowd out a user's minority interests. That is useful as an offline diagnostic, not only as a re-ranker.

12.1.3 Pitfalls of offline evaluation

Sampled metrics are inconsistent. Krichene and Rendle [6] (Google, KDD 2020) showed that the common shortcut of ranking the true item against ~100 random negatives and reporting HR@10/NDCG@10 is *inconsistent*: the ranking of algorithms under sampled metrics can disagree with the ranking under full-catalogue metrics. They propose corrections, but the main advice is to compute metrics over the full catalogue whenever you can.

Weak baselines and leakage. Ferrari Dacrema et al. [7] tried to reproduce 18 neural recommendation papers from top venues. They could reproduce only about 7, and 6 of those were beaten by well-tuned simple baselines (kNN, graph-based, linear). Rendle et al. [8] showed that a properly tuned dot-product MF beats the MLP-based NCF. The lesson for practitioners is to tune baselines as carefully as the new model, split by time rather than at random, and remove future information from features.

Logging-policy bias. Every logged interaction happened because the *production* policy chose to show that item. Offline metrics computed on these logs reward models that imitate the current policy and penalize models that would surface items users never saw. Schnabel et al. [9] cast this as missing-not-at-random data and propose propensity-weighted evaluation (see 12.2). Joachims et al. [10] do the same for position bias in learning-to-rank.

```
OFFLINE EVAL PIPELINE (typical)
impression+engagement logs ---> time-based split ----> train window | eval window
                                                                     |
   candidate model --(score full catalogue or logged slate)----------+
                                                                     v
        +----------------+--------------+---------------+------------------+
        | retrieval:     | ranking:     | calibration:  | beyond-accuracy: |
        | recall@K       | NDCG, AUC,   | sum(p)/sum(y) | coverage, KL     |
        | (full catalog) | NE/logloss   | by slice      | genre calib.,    |
        |                | per slice    | reliability   | diversity, nov.  |
        +----------------+--------------+---------------+------------------+
                  \_____________________ gate: "worth an online test?" ______/
```

12.1.4 The offline-online gap

The best-documented industrial evidence comes from three sources:

- **Booking.com** [11] reported from about 150 production models that offline model-performance gains are *not* a reliable predictor of online business gains. In their words, increased performance does not necessarily translate to increased value. They describe cases where better click models saturated, or optimized proxies that diverged from conversion. Their remedy is to A/B test everything and to treat offline metrics only as a health check.
- **Airbnb** [2] documented, in "Applying Deep Learning to Airbnb Search", a long series of model iterations where offline NDCG gains did not carry over to online bookings. One example was a model that learned to favour listings that looked attractive but did

not lead to bookings. The team learned to rely on online tests and to be suspicious of offline wins that rest on features which are effectively proxies for the label.
- **Netflix** [12]: Gomez-Uribe and Hunt describe offline metrics (ranking accuracy on held-out plays) as useful for screening only. Final decisions rest on A/B tests of member retention and streaming hours, because offline metrics cannot capture the counterfactual "what would the member have played otherwise".

The usual causes of the gap are (a) logging-policy bias, (b) mismatch between proxy and objective (clicks vs. satisfaction), (c) feedback loops and position effects that offline replay cannot simulate, (d) train/serve skew in features, and (e) metric saturation, where an already-good model has little room to improve on the measured axis. The rest of the chapter covers techniques that attack (a) through (c).

12.2 Counterfactual and off-policy evaluation

12.2.1 Problem setup

The logs contain tuples (x, a, r, p): context x, action (item or slate) a chosen by the logging policy pi_0 with probability p = pi_0(a|x), and reward r. We want V(pi) = E_x E_{a~pi(.|x)} [r] for a new target policy pi, *without deploying it*.

12.2.2 Estimators

Estimator	Formula (per-sample, averaged over logs)	Bias	Variance	Needs
Replay / rejection (Li et al. [13])	keep only events where pi picks the logged a; average r	unbiased if pi_0 uniform	high (discards data)	uniformly random logging bucket
Direct Method (DM)	r_hat(x, pi(x)) from a reward model	biased by model error	low	reward model
IPS	(pi(a|x) / p) * r	unbiased if p > 0 wherever pi > 0	high when p small	logged propensities
Clipped / capped IPS	min(w, M) * r	biased (downward)	bounded	propensities, cap M
Self-normalized IPS (SNIPS)	sum(w r) / sum(w)	small, consistent	lower than IPS	propensities
Doubly Robust (Dudik et al. [14])	r_hat(x, pi) + w * (r - r_hat(x, a))	unbiased if either p or r_hat is correct	lower than IPS	both
Pseudo-inverse for slates (Swaminathan et al. [15])	linear decomposition over slot-item pairs	unbiased under additivity	far lower than slate-level IPS	per-slot propensities

Li, Chu, Langford and Wang (Yahoo!, WSDM 2011) [13] introduced the replay method for news recommendation on the Yahoo! Today module. A slice of traffic was served *uniformly at random*. Any candidate bandit policy can be evaluated by streaming through those logs and keeping only events where it agrees with the random action. The paper showed that offline replay estimates closely matched the policies' online CTR. This set the template: **pay for a small exploration bucket, gain unbiased offline evaluation**.

Dudik, Langford and Li (ICML 2011) [14] formalized the doubly robust estimator, which combines a direct reward model with an IPS correction on the residual. It is unbiased if *either* the propensities *or* the reward model is correct, and usually has much lower variance than IPS. DR and its variants (switch-DR, DR with optimistic shrinkage, and others) are now the default in OPE libraries.

Swaminathan and Joachims [16] (JMLR 2015) developed counterfactual risk minimization (CRM) for *learning* from logged bandit feedback. Its key idea is to penalize the empirical variance of the IPS estimate (POEM). Related work introduced the self-normalized estimator to deal with "propensity overfitting".

12.2.3 Slates and rankings

Recommenders show slates. The naive propensity of a whole slate is the product of per-slot probabilities, which is vanishingly small, so IPS on slates explodes. Two industrial-grade answers:

- **Pseudo-inverse estimator (Microsoft Research / Cornell, NeurIPS 2017)** [15] assumes that slate reward decomposes additively over (slot, item) pairs. Under that assumption it gives an unbiased estimator whose variance scales with the number of slot-item pairs, not with the number of slates. It was validated on Microsoft search (MSLR/Bing-derived) data.
- **Criteo's "Offline A/B testing for Recommender Systems" (WSDM 2018)** [17] targets the top-K display ads setting. It proposes capped and normalized IPS variants that use only the top-K positions, trading a controlled bias for a large drop in variance. The estimators were evaluated against real online A/B outcomes at Criteo, and normalized capped importance sampling predicted the A/B *sign and ranking* of policies more reliably than standard offline metrics.

12.2.4 Off-policy correction for learning: YouTube REINFORCE

Chen et al. (Google/YouTube, WSDM 2019) [18] trained a REINFORCE candidate generator over an action space of *millions* of videos. Because training data came from earlier, different policies, they added (1) an importance weight pi_theta/beta, with the behaviour policy beta *estimated* by a separate softmax head because YouTube did not log propensities directly, and (2) a **top-K off-policy correction** multiplier lambda_K = K * (1 - pi)^(K-1). The second term accounts for the fact that K items are recommended at once. They also capped weights to control variance. They reported a statistically significant online gain in viewing time, which they described as one of the largest single launches in a couple of years at the time (exact effect sizes in [18]; treat as approx.). The point for designers is that **OPE is not only for evaluation**. The same weights correct training bias, and the missing ingredient in most stacks is *logging the propensity at serve time*.

12.2.5 Tooling and benchmarks

- **Open Bandit Pipeline / Open Bandit Dataset (ZOZO, Saito et al.)** [19] is a public dataset from a fashion e-commerce site. It was logged under *two* policies (uniform random and Bernoulli Thompson sampling), which makes it possible to validate OPE estimators against an on-policy ground truth. The paper benchmarks DM, IPS, SNIPS,

DR and variants and finds that estimator rankings depend heavily on the setting. No estimator wins everywhere.

- **Practical requirements** that recur in all the sources above:
  1. Log p(a|x) at serve time, including post-ranking randomization and re-ranker effects.
  2. Keep a small exploration bucket (epsilon-greedy, softmax sampling or random reshuffle of the top-N) so that the support condition holds.
  3. Monitor the effective sample size, (sum w)^2 / sum w^2. When it collapses, the target policy is too far from logging and only an A/B test will do.

```
OPE DATA FLOW
 request --> logging policy pi_0 (ranker + sampler) --> slate shown
                       |  logs: x, slate, per-slot p(a|x), model_version
                       v
               [ propensity-aware log store ]
                       |
   candidate pi ------>+-----> IPS / SNIPS / DR / PI estimators
   reward model r_hat->+            |
                                    +--> V_hat(pi), CI, ESS
                                    v
                gate: V_hat significantly > V_hat(pi_0) and ESS ok
                       --> interleaving / A/B
```

12.3 Interleaving

12.3.1 Method

Interleaving goes back to Joachims and was refined by Radlinski, Kurup and Joachims [20]. It shows each user a *single merged list* built from rankers A and B, then credits each engagement to the ranker that contributed the item. The variant used in industry is **team-draft interleaving (TDI)**: like picking teams in a schoolyard, a coin flip decides who picks first in each round, and each ranker adds its highest-ranked item not already in the list. Chapelle, Joachims, Radlinski and Yue [21] validated interleaving at scale on Yahoo! and Bing search. They found that interleaving agreed with the direction of offline relevance judgments and of A/B results, and needed one to two orders of magnitude less data than absolute-metric A/B tests to reach the same statistical power.

The sensitivity comes from a within-user comparison. Every user sees both rankers, so between-user variance (heavy vs. light users) cancels out.

```
TEAM-DRAFT INTERLEAVING
Ranker A: a1 a2 a3 a4 ...           Ranker B: b1 a1 b2 b3 ...
round 1: coin=A -> A picks a1 ; B picks b1
round 2: coin=B -> B picks b2 (a1 taken) ; A picks a2
...
merged list: [a1(A), b1(B), b2(B), a2(A), ...]
user engages with b1, b2  --> preference(B) for this user
aggregate over users: Delta = (#users prefer A - #users prefer B) / N  -> sign test
```

12.3.2 Netflix

Netflix's 2017 post "Innovating Faster on Personalization Algorithms at Netflix Using Interleaving" [22] describes a **two-stage online experimentation process**:

1. **Stage 1, interleaving.** Use interleaving to quickly prune a large set of candidate ranking algorithms down to the most promising few.
2. **Stage 2, A/B.** Run a traditional A/B test on the survivors to measure effects on core metrics (streaming hours, retention).

Key points from the post:

- Engagement is attributed by **hours streamed** from titles contributed by each ranker, not by clicks.
- Netflix applies interleaving to *rows* of personalized videos and uses team-draft with care for position. Each ranker must have an equal chance of filling each position, so they "pick" in randomized order to avoid position bias.
- **Sensitivity**: in Netflix's simulations, interleaving reached the same statistical power with a much smaller sample. The post reports that A/B testing needed on the order of 100x more subscribers than interleaving to detect the same preference reliably (approx.; see the sensitivity plot in [22]).
- **Agreement**: the interleaving preference metric was strongly correlated with the A/B test's core engagement metric across the experiments they compared. That justified using it as a pruning stage.
- **Limitations**: interleaving measures *relative preference between two rankings* on one surface. It cannot measure absolute effects on retention, page-level effects (for example, how the whole homepage composes), or non-ranking changes.

12.3.3 Airbnb

Airbnb's 2022 post "Beyond A/B Test: Speeding up Airbnb Search Ranking Experimentation through Interleaving" [23] adapts TDI to a two-sided marketplace with sparse, delayed conversions (bookings):

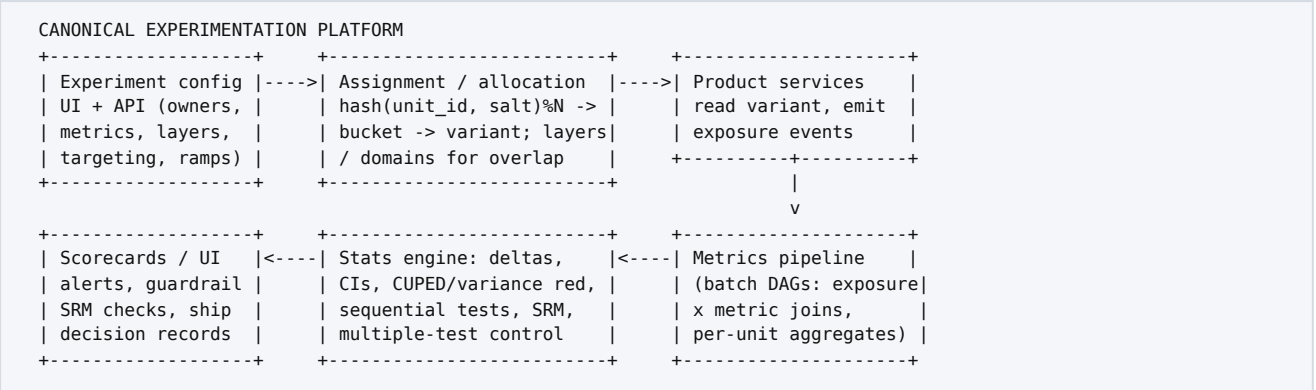
- **Competitive pairs.** Many interleaved positions hold the same listing from both rankers. Airbnb only uses "competitive pairs" for attribution, meaning the positions where A and B actually differ. Searches where the rankers agree contribute no signal and are filtered out, which raises sensitivity.
- **Attribution across a search journey.** A guest issues many searches before booking, so engagement (clicks, then bookings) is attributed back over the guest's search session. Listings are *deduplicated* so that a listing first contributed by A stays credited to A.
- **Metrics**: interleaving variants of click-through and booking metrics, with booking-based metrics as the primary decision signal.
- **Results**: Airbnb reports roughly **50x speed-up** in experiment sensitivity relative to A/B testing for search ranking. It also reports that interleaving results agreed directionally with the subsequent A/B tests in the large majority of paired comparisons (approx. 80+%; see [23]).
- **Usage**: interleaving runs on a small slice of traffic and is used to triage ranking candidates before A/B. That frees A/B capacity, which is scarce because marketplace experiments need large samples.

12.3.4 Design notes

- Interleaving needs **both rankers to be served live in the same request**, which roughly doubles the ranking compute on that traffic slice. Cache the candidate set and only re-score.
- It suits **ranking-only changes**. It does not fit retrieval changes that alter the candidate pool size, UI changes, or anything that affects the whole page.
- Debiasing matters. If one ranker systematically wins "first pick" positions or produces items with longer descriptions, the attribution is skewed. Netflix and Airbnb both describe position-balanced picking and filtering for this reason.

12.4 A/B testing platforms

All the large platforms converge on a common architecture:



12.4.1 Google: overlapping experiment infrastructure

Tang, Agarwal, O'Brien and Meyer (KDD 2010) [24] described Google search's **layers and domains**. Parameters are partitioned into *layers*. Each layer splits traffic independently, so one request can be in one experiment per layer. That made it possible to run many more concurrent experiments than with single-membership bucketing. *Domains* give exclusive traffic slices for experiments that conflict with everything else, and *launch layers* handle gradual rollouts. Tooling enforces data-file checks and pre-defined metrics. Nearly every later platform (LinkedIn, Spotify, Airbnb, Netflix) uses a variant of this layering idea.

12.4.2 Microsoft ExP and the "trustworthiness" canon

Kohavi et al. (KDD 2013) [25] reported that Bing ran hundreds of concurrent experiments, with each user in roughly 15 concurrent experiments (approx.). They catalogued pitfalls: carry-over effects, novelty/primacy, and the need for A/A tests. The book *Trustworthy Online Controlled Experiments* [26] by Kohavi, Tang and Xu (Microsoft, Google, LinkedIn) is the standard reference. It introduces the **Overall Evaluation Criterion (OEC)**, guardrail metrics, and **sample ratio mismatch (SRM)** checks. Fabijan et al. (KDD 2019) [27] taxonomize SRM causes (bot filtering, redirect timing, residual effects, telemetry loss) and report that SRMs occurred in a notable fraction of Microsoft experiments (on the order of 6%, approx.). That makes an automated SRM chi-square test a must-have platform feature.

CUPED (Deng, Xu, Kohavi, Walker, WSDM 2013) [28] reduces variance by using pre-experiment covariates: Y_cuped = Y - theta * (X - mean(X)), with theta = cov(X, Y)/var(X). Using the same metric in the pre-period as X typically cuts variance by around half for

engagement metrics at Bing (approx.). That is equivalent to doubling traffic. Nearly every platform below implements CUPED or a regression-adjustment generalization.

12.4.3 Netflix: ABlaze and the experimentation analysis rewrite

Netflix runs essentially every product change, including every recommendation algorithm, through A/B testing. Gomez-Uribe and Hunt [12] describe the core OEC as **member retention**, with **streaming hours** as a more sensitive proxy that correlates with it. Their paper also estimates that personalization and recommendations save Netflix more than USD 1B per year in reduced churn.

The 2016 post "It's All A/Bout Testing: The Netflix Experimentation Platform" [29] describes:

- **ABlaze**, the front-end for configuring tests, allocation and viewing results.
- **Allocation**: test cells, a choice between **batch allocation** (a pre-computed population of existing members meeting criteria) and **real-time allocation** (members allocated at the moment of a qualifying action, such as signup or first device use), plus a test-allocation service that clients call to get cell membership.
- **Metrics/analysis**: an analysis pipeline producing per-test results with statistical significance on retention, streaming and other metrics, and support for the many concurrent tests run across the product.

The 2019 post "Reimagining Experimentation Analysis at Netflix" [30] covers the move to a modular, open stack. Analyses are written in Python/R, and a shared library of **statistical "causal models"** (including quantile bootstrapping and regression-based variance reduction) is applied across metrics, so data scientists can add new methods without platform rewrites. Netflix's 2022 survey of causal inference applications [31] adds context on quasi-experiments, synthetic control and heterogeneous treatment effects, which are used where randomization is impossible.

12.4.4 Airbnb: from ERF to a scaled platform

Airbnb's early post "Experiments at Airbnb" (2014) [32] documents the pitfalls specific to a marketplace with long booking funnels:

- **Context dependence**: the same change can show opposite effects at different times or in different markets.
- **Stopping too early**: their search experiments showed p-values dipping below 0.05 transiently before settling. They argue for deciding the run time in advance and for dynamic p-value thresholds.
- **Assignment hygiene**: users must be assigned at the moment they experience the change, not earlier, or the signal is diluted.

The **Experiment Reporting Framework (ERF)** became Airbnb's central system. The 2017 post "Scaling Airbnb's Experimentation Platform" [33] describes how it was rebuilt to keep up with growth:

- Metric computation moved to **Airflow DAGs over Hive/Spark**, with assignments joined to a library of pre-defined metric "events" and per-subject aggregation.

- Scale: the post describes ERF supporting on the order of hundreds of concurrent experiments and thousands of metrics, computing results daily (approx.; see [33]).
- **Metric hierarchy**: core company metrics computed for every experiment, plus target metrics chosen by the team, plus certified metrics shared across teams.
- Pipeline optimizations reduced daily compute time by pre-aggregating assignment-metric joins and splitting the work into smaller, parallel DAG stages.

Airbnb search adds two further layers. **Interleaving** (12.3.3) handles fast triage, and offline NDCG [2] is the first screen.

12.4.5 LinkedIn: XLNT and T-REX

Xu, Chen, Fernandez, Sinno and Bhasin (LinkedIn, KDD 2015) [34] describe **XLNT**, LinkedIn's end-to-end A/B platform:

- **Lix** (LinkedIn Experiment) for targeting and assignment. Experiments are defined with a targeting DSL over member attributes (country, industry, activity level and so on) and ramped gradually (for example 1%, 5%, 50%).
- An offline analysis pipeline built on Hadoop that computes **many hundreds of metrics** for every experiment each day and produces reports with statistical significance.
- Scale at the time: about 200 experiments running concurrently, and well over a thousand metrics in the library (approx.; see [34]).
- Cultural mechanisms: an experimentation review process, metric "ownership", and A/B results as a gate for launches.
- Social-network challenges: **network effects/interference** (a treated member's posts reach control members), which motivated cluster-based and ego-cluster randomization later on.

LinkedIn's 2019 post "Making the LinkedIn experimentation engine 20x faster" [35] covers the next-generation **T-REX** online assignment engine. The Lix DSL was re-implemented so that evaluating which variant a member gets is a very low-latency, in-process computation. Moving from interpreted to compiled evaluation gave the 20x speed-up, at the scale of **trillions of variant evaluations per day** (approx.; see [35]). That speed matters because ranking services evaluate experiment membership per request, often several times per request.

12.4.6 Spotify: Experimentation Platform and Confidence

Spotify's 2020 post "Spotify's New Experimentation Platform (Part 1)" [36] describes a replacement for its earlier homegrown "ABBA" system. It has three parts: a **Remote Configuration** service for feature flags and variant properties, a **Metrics Catalog** for certified metrics computed in a shared pipeline, and an **Experiment Planner** UI for designing, sizing and launching experiments. Experiments are allocated via **"domains"** and exclusive/overlapping **layers**, with salted-hash bucketing. The post cites hundreds of teams running experiments on the platform and many experiments per year (exact figures in [36]).

Spotify's statistical work is unusually public:

- **Risk-aware decision rules with multiple metrics** (2024) [37] separates *success metrics* (shown to be better), *guardrail metrics* (shown to be non-inferior) and *deterioration* checks. It derives how to set alpha and beta across them so that the decision rule, not each metric separately, has the intended false-positive and false-negative rates. A notable result is that guardrail non-inferiority tests do not need multiple-testing correction on alpha, but power must be corrected for.
- Sequential testing (group sequential tests with spending functions) is used so that teams can check results without inflating false positives.
- **Confidence** (announced 2023) [38] is the platform offered as a commercial product to other companies. It keeps the same concepts: flags, metrics on the customer's warehouse, and Spotify's sequential and risk-aware statistics.

12.4.7 Uber XP

Uber's 2018 post "Under the Hood of Uber's Experimentation Platform" [39] describes a platform supporting A/B tests, multi-armed bandits, switchback experiments (for marketplace interference) and causal-inference methods. It runs on the order of 1,000+ concurrent experiments (approx.). Its statistics engine picks the test per metric type (proportions, continuous, ratio via the delta method), includes sequential testing for monitoring, and uses CUPED-style variance reduction.

12.4.8 Platform comparison

Platform	Org	Assignment model	Notable statistics	Scale (public, approx.)	Ref
Overlapping layers	Google	layers + domains, per-layer hash	pre-defined metrics, launch layers	many concurrent expts per request	[24]
ExP	Microsoft/Bing	concurrent overlapping	CUPED, SRM checks, OEC	~15 expts per user (approx.)	[25] [27] [28]
ABlaze + analysis lib	Netflix	batch + real-time allocation cells	retention OEC, bootstrap, regression adj.	essentially all product changes	[29] [30]
ERF	Airbnb	user/visitor hashing; search-level for interleave	metric hierarchy, dynamic p-value stopping	hundreds expts, thousands metrics	[32] [33]
XLNT / T-REX	LinkedIn	Lix DSL targeting + ramps	hundreds of metrics per test, network-effect designs	trillions evals/day (approx.)	[34] [35]
EP / Confidence	Spotify	domains + layers, remote config	sequential, risk-aware multi-metric rules	hundreds of teams	[36] [37] [38]
XP	Uber	A/B, switchback, bandits	delta method, sequential, CUPED	1000+ concurrent (approx.)	[39]

12.4.9 Recommender-specific A/B pitfalls

- **Shared-model interference / feedback loops.** If control and treatment both train on the pooled logs, the treatment's data leaks into the control model and vice versa. For ranking tests that change data collection (exploration, new surfaces), teams use *separate training data per arm* or "data-diverted" arms. Kohavi et al. [26] discuss leakage through shared resources in general.

- **Two-sided marketplace interference.** At Airbnb and Uber, treatment and control compete for the same inventory, so effects are biased. Remedies are switchbacks [39] or market-level/cluster randomization.
- **Novelty and primacy effects.** Short tests overestimate (novelty) or underestimate (learning) long-run effects. Google's long-term work [40] quantifies this (see 12.5).
- **Metric sensitivity.** Retention barely moves in a 2-week test. Netflix uses streaming hours as a proxy validated against retention [12]. Generalized: surrogate indices (12.5.2).

12.5 Long-term metrics, surrogates and holdouts

12.5.1 Measuring long-term effects: Google's "Focusing on the Long-term"

Hohnhold, O'Brien and Tang (Google, KDD 2015) [40] studied how ads-quality changes affect *user learning*. Users exposed to more or worse ads gradually learn to ignore them. That drives ad CTR down over months, an effect invisible in short A/B tests. Their method:

- **Post-period / cookie-cohort design**: expose a cohort to treatment for a long period, then measure that cohort under the *same* (control) system, to isolate the learned behaviour change from the direct treatment effect.
- **Model the long-term effect from short-term metrics**: they fit a model predicting long-term user learning from short-term measurable ad quality and ad load, so future launches can be judged using short tests plus this model.
- **Outcome**: Google *reduced* the number of ads shown on mobile search (by roughly 50% on some surfaces, approx.). The long-term user-learning gains were judged to outweigh the short-term revenue loss.

This is the classic demonstration that **short-term OEC optimization can harm long-term value**, and that an explicit long-term model can fix the decision rule.

12.5.2 Surrogate index

Athey, Chetty, Imbens and Kang [41] formalized the **surrogate index**. Train a model on historical data that predicts the long-term outcome Y (for example 1-year retention) from many short-term surrogates S (engagement in weeks 1 to 4). Under a "surrogacy" assumption (treatment affects Y only through S), the treatment effect on Y equals the treatment effect on E[Y | S]. Using *many* surrogates combined into an index makes surrogacy more plausible than relying on any single proxy.

Recommender teams use this in practice:

- **Netflix** has publicly discussed using surrogate indices to make launch decisions from short tests in place of waiting for retention, including a large-scale evaluation of surrogate-based decisions across a few hundred historical A/B tests (approx.). The finding was that surrogate-index decisions agreed with long-term retention decisions far more often than single-proxy decisions. This is a 2023 Netflix research paper. The author could not verify its URL during compilation, so it is not cited by URL here.

- **Netflix recommendations** [42] (2024 post "Recommending for Long-Term Member Satisfaction at Netflix") treats the recommender as a bandit and argues that the *reward* used for training should itself be a proxy for long-term satisfaction, not the immediate click or play. The proxy is built by "reward engineering" from signals such as completion, thumbs-up and return visits. Its validity is checked against the online A/B results of the north-star (retention-related) metric. Practical complications are **delayed feedback**, where a show may be finished days later, and predicting the missing part of the reward for recent interactions. That ties this chapter to Chapter 11.

```
SURROGATE-INDEX WORKFLOW
historical cohorts: short-term S (wk1-4 engagement) --> long-term Y (retention 6-12 mo)
                   \____ fit f(S) ~ E[Y|S]  (validated on past A/Bs) ____/
new A/B (2-4 wks):   S_treat, S_ctrl  --> f(S_treat) - f(S_ctrl) = estimated effect on Y
safety net:          long-term holdout periodically checks that f still predicts Y
```

12.5.3 Holdouts

A **holdout** is a slice of users kept off a whole class of changes (for example "all recommendation launches this half") for months. It has three uses:

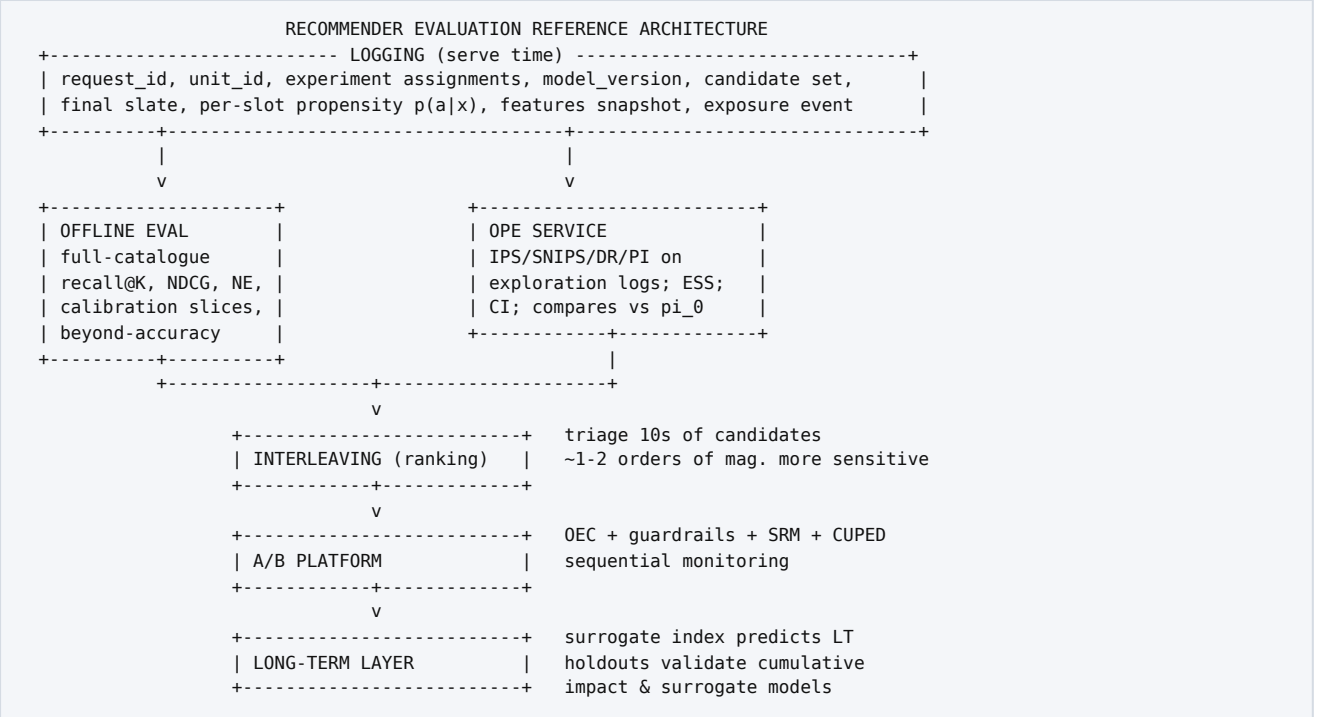
1. **Cumulative impact**: sums of individually measured lifts tend to overstate the combined effect, because of winner's curse, interactions and novelty. A holdout measures the real cumulative effect. Kohavi et al. [26] discuss this, along with "long-term holdouts" and "reverse experiments".
2. **Long-term validation of surrogates/proxies**: the holdout is the ground truth that the surrogate index in 12.5.2 is checked against.
3. **Ecosystem effects** that A/B tests on individuals cannot see, such as creator-side effects or content supply.

Costs are an ethical/product cost (users miss improvements), loss of statistical power for regular tests, and engineering cost (keeping old code paths alive). Common practice described in [26] is a small holdout (for example 1 to 5%) refreshed each period, or *reverse experiments*, where a launched feature is briefly turned off for a small group to re-measure it.

12.5.4 Winner's curse and metric inflation

When many experiments are run and only "significant winners" ship, the reported effects of winners are biased upward. Kohavi et al. [26] recommend shrinkage or holdout-based correction and replication runs for important launches. Airbnb's observation about transient significance [32] is the same phenomenon in time. Sequential designs [37] and pre-registered durations are the platform-level fixes.

12.6 Putting it together: a reference evaluation architecture



Design rules distilled from the sources:

- **Log for the next evaluation, not just the current model.** Propensities [13][18], exposure events [26], model version and candidate sets cost little to log and are impossible to reconstruct later.
- **Budget exploration explicitly.** A uniformly random bucket on a small percentage of traffic made Yahoo!'s offline replay possible [13]. Lighter alternatives are softmax sampling or randomized re-ranking of the top-N.
- **Match the stage to the change type.** Ranking-only change: offline, then interleaving, then A/B. Retrieval or UI change: offline, then A/B. Exploration/RL policy: OPE, then A/B. Ecosystem change: A/B plus holdout, with cluster/switchback designs.
- **Validate every proxy against the stage above.** Netflix validated interleaving against A/B [22], Airbnb did the same [23], Criteo validated OPE against A/B [17], and Netflix validated surrogates against long-term retention.

12.7 Case summaries at a glance

Company / work	Stage	Core technique	Headline result (approx.)	Ref
Yahoo! Today module	OPE	replay on uniform-random logs	offline estimates matched online CTR	[13]
Microsoft/Cornell	OPE (slates)	pseudo-inverse estimator	big variance cut vs slate-IPS	[15]
Criteo	OPE (top-K)	capped/normalized IS	better prediction of A/B outcomes than offline metrics	[17]
YouTube	OPE in training	top-K off-policy corrected REINFORCE	significant online watch-time gain	[18]

Company / work	Stage	Core technique	Headline result (approx.)	Ref
ZOZO	OPE benchmark	Open Bandit Dataset, two logging policies	no estimator dominates	[19]
Netflix	interleaving	team-draft, hours-attributed	~100x fewer users than A/B	[22]
Airbnb	interleaving	competitive pairs, booking attribution	~50x speed-up, high A/B agreement	[23]
Google	A/B infra	overlapping layers	many more concurrent experiments	[24]
Microsoft	A/B stats	CUPED, SRM	~50% variance reduction; SRM in ~6% of expts	[27] [28]
LinkedIn	A/B infra	XLNT, Lix, T-REX	trillions evals/day, 20x faster engine	[34] [35]
Spotify	A/B stats	risk-aware multi-metric decision rules	principled alpha/power across metric types	[37]
Google ads	long-term	user-learning model, post-period	fewer mobile ads, long-term gain	[40]
Netflix	long-term	proxy reward for long-term satisfaction	reward engineering validated by A/B	[42]
Booking.com	offline-online gap	A/B everything	offline gains did not predict business gains	[11]

Key takeaways for system designers

1. **Treat evaluation as a funnel, not a single metric.** Offline metrics generate ideas, OPE and interleaving triage them, A/B tests decide, and holdouts or surrogates protect the long term. Each stage should be validated against the next one.
2. **Compute offline metrics honestly.** Use time-based splits, full-catalogue ranking (not sampled negatives [6]), well-tuned baselines [7][8], and calibration by slice [3][4]. Report beyond-accuracy diagnostics such as genre calibration [5].
3. **Expect the offline-online gap.** Booking.com [11], Airbnb [2] and Netflix [12] all report that offline wins often do not transfer. Budget for online tests and do not over-invest in squeezing offline metrics.
4. **Log propensities and keep an exploration slice.** Without them, IPS, SNIPS, DR and slate estimators [13][14][15][17] cannot be used. With them, the same weights also de-bias training [18].
5. **Prefer DR or SNIPS to raw IPS, and watch the effective sample size.** When the target policy moves far from the logging policy, OPE becomes unreliable and only an online test will settle the question.
6. **Use interleaving for ranking changes.** Netflix (~100x) [22] and Airbnb (~50x) [23] show very large sensitivity gains. Ignore positions where both rankers agree (competitive pairs), attribute the metric you actually care about (hours, bookings), and keep A/B as the final gate.
7. **Build the A/B platform around trust.** Overlapping layers [24], fast in-process assignment [35], SRM checks [27], CUPED [28], sequential and risk-aware decision rules [37], and a curated metric catalogue [33][36] matter more than any single statistical trick.

8. **Recommenders add their own pitfalls.** Watch for shared-training-data leakage between arms, marketplace interference (use switchbacks or clusters [39]), and network effects (use ego/cluster designs [34]).
9. **Optimize for the long term explicitly.** Short-term wins can hurt users over time [40]. Use surrogate indices [41], proxy rewards built for long-term satisfaction [42], and standing holdouts [26] to keep the decision rule aligned with retention and value.

References

1. Järvelin, K., Kekäläinen, J. "Cumulated Gain-Based Evaluation of IR Techniques." ACM TOIS, 2002. https://dl.acm.org/doi/10.1145/582415.582418
2. Haldar, M., et al. "Applying Deep Learning to Airbnb Search." KDD 2019, Airbnb. https://arxiv.org/abs/1810.09591
3. He, X., Pan, J., Jin, O., et al. "Practical Lessons from Predicting Clicks on Ads at Facebook." ADKDD 2014, Facebook. https://dl.acm.org/doi/10.1145/2648584.2648589
4. McMahan, H. B., et al. "Ad Click Prediction: a View from the Trenches." KDD 2013, Google. https://dl.acm.org/doi/10.1145/2487575.2488200
5. Steck, H. "Calibrated Recommendations." RecSys 2018, Netflix. https://dl.acm.org/doi/10.1145/3240323.3240372
6. Krichene, W., Rendle, S. "On Sampled Metrics for Item Recommendation." KDD 2020, Google. https://arxiv.org/abs/1912.02263
7. Ferrari Dacrema, M., Cremonesi, P., Jannach, D. "Are We Really Making Much Progress? A Worrying Analysis of Recent Neural Recommendation Approaches." RecSys 2019. https://arxiv.org/abs/1907.06902
8. Rendle, S., Krichene, W., Zhang, L., Anderson, J. "Neural Collaborative Filtering vs. Matrix Factorization Revisited." RecSys 2020, Google. https://arxiv.org/abs/2005.09683
9. Schnabel, T., Swaminathan, A., Singh, A., Chandak, N., Joachims, T. "Recommendations as Treatments: Debiasing Learning and Evaluation." ICML 2016. https://arxiv.org/abs/1602.05352
10. Joachims, T., Swaminathan, A., Schnabel, T. "Unbiased Learning-to-Rank with Biased Feedback." WSDM 2017. https://arxiv.org/abs/1608.04468
11. Bernardi, L., Mavridis, T., Estevez, P. "150 Successful Machine Learning Models: 6 Lessons Learned at Booking.com." KDD 2019, Booking.com. https://dl.acm.org/doi/10.1145/3292500.3330744
12. Gomez-Uribe, C. A., Hunt, N. "The Netflix Recommender System: Algorithms, Business Value, and Innovation." ACM TMIS, 2015, Netflix. https://dl.acm.org/doi/10.1145/2843948
13. Li, L., Chu, W., Langford, J., Wang, X. "Unbiased Offline Evaluation of Contextual-bandit-based News Article Recommendation Algorithms." WSDM 2011, Yahoo! Research. https://arxiv.org/abs/1003.5956
14. Dudik, M., Langford, J., Li, L. "Doubly Robust Policy Evaluation and Learning." ICML 2011. https://arxiv.org/abs/1103.4601
15. Swaminathan, A., Krishnamurthy, A., Agarwal, A., Dudik, M., Langford, J., Jose, D., Zitouni, I. "Off-policy Evaluation for Slate Recommendation." NeurIPS 2017, Microsoft Research. https://arxiv.org/abs/1605.04812

16. Swaminathan, A., Joachims, T. "Batch Learning from Logged Bandit Feedback through Counterfactual Risk Minimization." JMLR 16, 2015. https://jmlr.org/papers/v16/swaminathan15a.html
17. Gilotte, A., Calauzènes, C., Nedelec, T., Abraham, A., Dollé, S. "Offline A/B Testing for Recommender Systems." WSDM 2018, Criteo. https://arxiv.org/abs/1801.07030
18. Chen, M., Beutel, A., Covington, P., Jain, S., Belletti, F., Chi, E. "Top-K Off-Policy Correction for a REINFORCE Recommender System." WSDM 2019, Google/YouTube. https://arxiv.org/abs/1812.02353
19. Saito, Y., Aihara, S., Matsutani, M., Narita, Y. "Open Bandit Dataset and Pipeline: Towards Realistic and Reproducible Off-Policy Evaluation." NeurIPS Datasets and Benchmarks 2021, ZOZO. https://arxiv.org/abs/2008.07146
20. Radlinski, F., Kurup, M., Joachims, T. "How Does Clickthrough Data Reflect Retrieval Quality?" CIKM 2008. https://dl.acm.org/doi/10.1145/1458082.1458092
21. Chapelle, O., Joachims, T., Radlinski, F., Yue, Y. "Large-Scale Validation and Analysis of Interleaved Search Evaluation." ACM TOIS, 2012. https://dl.acm.org/doi/10.1145/2094072.2094078
22. Parks, J., Aurisset, J., Ramm, M. (Netflix Technology Blog). "Innovating Faster on Personalization Algorithms at Netflix Using Interleaving." 2017, Netflix. https://netflixtechblog.com/interleaving-in-online-experiments-at-netflix-a04ee392ec55
23. Airbnb Engineering. "Beyond A/B Test: Speeding up Airbnb Search Ranking Experimentation through Interleaving." 2022, Airbnb. https://medium.com/airbnb-engineering/beyond-a-b-test-speeding-up-airbnb-search-ranking-experimentation-through-interleaving-7087afa09c8e
24. Tang, D., Agarwal, A., O'Brien, D., Meyer, M. "Overlapping Experiment Infrastructure: More, Better, Faster Experimentation." KDD 2010, Google. https://dl.acm.org/doi/10.1145/1835804.1835810
25. Kohavi, R., Deng, A., Frasca, B., Walker, T., Xu, Y., Pohlmann, N. "Online Controlled Experiments at Large Scale." KDD 2013, Microsoft. https://dl.acm.org/doi/10.1145/2487575.2488217
26. Kohavi, R., Tang, D., Xu, Y. "Trustworthy Online Controlled Experiments: A Practical Guide to A/B Testing." Cambridge University Press, 2020. https://experimentguide.com/
27. Fabijan, A., Gupchup, J., Gupta, S., Omhover, J., Qin, W., Vermeer, L., Dmitriev, P. "Diagnosing Sample Ratio Mismatch in Online Controlled Experiments." KDD 2019, Microsoft. https://dl.acm.org/doi/10.1145/3292500.3330722
28. Deng, A., Xu, Y., Kohavi, R., Walker, T. "Improving the Sensitivity of Online Controlled Experiments by Utilizing Pre-Experiment Data (CUPED)." WSDM 2013, Microsoft. https://dl.acm.org/doi/10.1145/2433396.2433413
29. Netflix Technology Blog. "It's All A/Bout Testing: The Netflix Experimentation Platform." 2016, Netflix. https://netflixtechblog.com/its-all-a-bout-testing-the-netflix-experimentation-platform-4e1ca458c15
30. Netflix Technology Blog. "Reimagining Experimentation Analysis at Netflix." 2019, Netflix. https://netflixtechblog.com/reimagining-experimentation-analysis-at-netflix-71356393af21
31. Netflix Technology Blog. "A Survey of Causal Inference Applications at Netflix." 2022, Netflix. https://netflixtechblog.com/a-survey-of-causal-inference-applications-at-netflix-b62d25175e6f

32. Overgoor, J. "Experiments at Airbnb." 2014, Airbnb Engineering. https://medium.com/airbnb-engineering/experiments-at-airbnb-e2db3abf39e7
33. Parks, J. "Scaling Airbnb's Experimentation Platform." 2017, Airbnb Engineering. https://medium.com/airbnb-engineering/https-medium-com-jonathan-parks-scaling-erf-23fd17c91166
34. Xu, Y., Chen, N., Fernandez, A., Sinno, O., Bhasin, A. "From Infrastructure to Culture: A/B Testing Challenges in Large Scale Social Networks." KDD 2015, LinkedIn. https://dl.acm.org/doi/10.1145/2783258.2788602
35. LinkedIn Engineering. "Making the LinkedIn Experimentation Engine 20x Faster." 2019, LinkedIn. https://engineering.linkedin.com/blog/2019/06/making-the-linkedin-experimentation-engine-20x-faster
36. Spotify Engineering. "Spotify's New Experimentation Platform (Part 1)." 2020, Spotify. https://engineering.atspotify.com/2020/10/spotifys-new-experimentation-platform-part-1/
37. Spotify Engineering. "Risk-Aware Product Decisions in A/B Tests with Multiple Metrics." 2024, Spotify. https://engineering.atspotify.com/2024/03/risk-aware-product-decisions-in-a-b-tests-with-multiple-metrics
38. Spotify Engineering. "Coming Soon: Confidence, an Experimentation Platform from Spotify." 2023, Spotify. https://engineering.atspotify.com/2023/08/coming-soon-confidence-an-experimentation-platform-from-spotify
39. Uber Engineering. "Under the Hood of Uber's Experimentation Platform." 2018, Uber. https://www.uber.com/blog/xp/
40. Hohnhold, H., O'Brien, D., Tang, D. "Focusing on the Long-term: It's Good for Users and Business." KDD 2015, Google. https://dl.acm.org/doi/10.1145/2783258.2788583
41. Athey, S., Chetty, R., Imbens, G., Kang, H. "The Surrogate Index: Combining Short-Term Proxies to Estimate Long-Term Treatment Effects More Rapidly and Precisely." NBER / arXiv, 2016-2019. https://arxiv.org/abs/1603.09326
42. Netflix Technology Blog. "Recommending for Long-Term Member Satisfaction at Netflix." 2024, Netflix. https://netflixtechblog.com/recommending-for-long-term-member-satisfaction-at-netflix-ac15cada49ef

13. Exploration, Bandits and Reinforcement Learning in Recommendation

Most production recommenders are trained on data that their own previous versions chose to collect. A ranking model learns from clicks on the items it ranked highly and never sees the counterfactual: what would have happened with an item it did not show. This chapter covers the designs that deal with that problem directly. Some explore on purpose. Some log the probability of every decision. Some correct their training for the logging policy. Some optimize long-horizon outcomes such as retention instead of the next click.

The material runs in roughly historical order, from the contextual bandits of the early 2010s (Yahoo LinUCB, Thompson sampling at Yahoo and Bing, Microsoft's Decision Service) through bandit products at Netflix, Spotify, Deezer and Amazon, to deep off-policy reinforcement learning (RL) at YouTube, SlateQ, Kuaishou retention optimization, and RL for notifications and ad bidding. The last sections cover off-policy evaluation, which every one of these systems depends on, and give takeaways for system designers.

A note on sourcing. This chapter was assembled when live web access was unavailable, so it relies on the primary papers and engineering posts listed under References, as the author knew them, plus the citations already checked in companion chapters of this Atlas. Numbers are given only where they are well known from the cited source and are marked approximate where exact figures could not be re-checked.

---

1. The problem: closed loops and missing counterfactuals

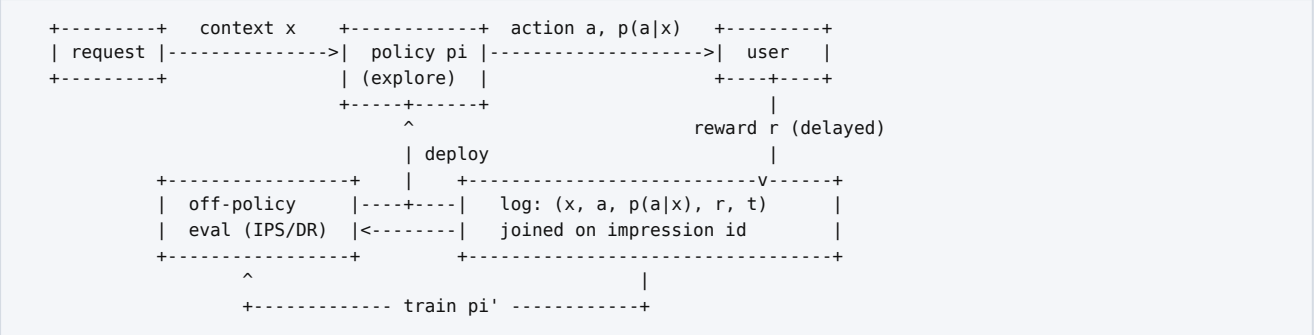
A recommender is an agent that acts (shows items), observes a reward (click, play, dwell, purchase, return visit) and changes its future actions. Three properties make this different from ordinary supervised learning.

1. **Partial feedback.** Rewards are observed only for the items shown. The log is a biased sample, shaped by the policy that produced it.
2. **Feedback loops.** If the model is retrained on its own outputs, items it underrates get no impressions and so can never be corrected. Jiang et al. (DeepMind) formalize these "degenerate feedback loops" and show that random exploration, plus a candidate pool that grows over time, are needed to avoid them [24].
3. **Delayed and long-horizon objectives.** What the business wants (retention, subscription renewal, long-term satisfaction) shows up days or weeks after the action, while the cheap signal (a click) is immediate and often misaligned.

The industry has responded with a sequence of increasingly expressive formulations:

Formulation	State	Action	Reward horizon	Typical use
Multi-armed bandit (MAB)	none	one arm	immediate	layout tests, artwork pool tuning
Contextual bandit	user/request features	one item or slot	immediate	news module, artwork, shelves, carousels
Slate / combinatorial bandit	features	ordered list of K	immediate	carousels, home pages, page layouts
Off-policy policy gradient	sequence (RNN) state	top-K of millions	short return	YouTube candidate generation
Full RL (MDP, Q-learning, actor-critic)	user state over a session or days	slate	long-term (sessions, retention)	SlateQ, Kuaishou RLUR, notifications
Constrained RL / bidding	campaign state	bid or multiplier	episode (budget day)	ad auto-bidding

All of them share one piece of infrastructure: **exploration with logged propensities**, so that the data can later be reweighted, evaluated offline and trained on without bias.



---

2. Yahoo! Today Module: LinUCB and replay evaluation (2010-2011)

Problem

The Yahoo! Front Page "Today Module" featured a small, rapidly changing pool of editor-picked news stories. New stories arrived throughout the day and had no click history, so a pure-exploitation CTR model was always wrong about the newest items. Li, Chu, Langford and Schapire framed article selection as a **contextual bandit** in which each article is an arm and user and article features are the context [1].

Algorithm

LinUCB assumes the expected click-through rate of arm a is linear in a d-dimensional feature vector x: E[r | x, a] = x^T theta_a. For each arm it keeps a ridge-regression state:

- A_a = D_a^T D_a + I (a d x d matrix)
- b_a = D_a^T c_a (the reward-weighted feature sum)
- theta_a = A_a^-1 b_a

At decision time it scores each arm as x^T theta_a + alpha * sqrt(x^T A_a^-1 x) and picks the maximum. The second term is the width of a confidence interval, so arms with little data, or whose data does not cover this kind of user, receive an optimism bonus. The paper also introduces a **hybrid** variant in which some coefficients are shared across all arms. This lets a brand-new article borrow statistical strength from older ones on its first impression [1].

Features were heavily compressed. Raw user features (demographics, geography, behavioral categories) and article features (URL category, editor tags) were reduced through a bilinear "conjoint" model and clustering to a vector of about six dimensions per side [1]. That kept per-arm matrices tiny, so updates were O(d^2) per event and trivially fast in serving.

Scale and results

The evaluation used tens of millions of logged events from the Today Module (the paper reports over 33 million events) [1]. LinUCB achieved a **12.5% click lift** over a standard context-free bandit [1]. The gains were largest when data was sparse, which is exactly the cold-start regime that bandits exist for.

Replay: the first practical offline evaluator

The companion WSDM 2011 paper [2] solved the evaluation problem that blocks every bandit project: how do you compare policies without an A/B test per candidate? Yahoo ran a bucket of traffic with a **uniformly random** logging policy. The **replay** method streams these logs and, for each event, asks the candidate policy what it would have shown. If it matches the logged action, the event is kept and its reward counted, and the candidate's state is updated. Otherwise the event is discarded. Because the logging was uniform, the kept events form an unbiased sample of what the candidate would have experienced online. Li et al. showed that replay estimates matched online bucket results closely [2]. Yahoo also released the R6 dataset from this traffic to the research community, and it became a standard bandit benchmark.

Lessons

- **Keep the context small and the update cheap.** LinUCB was deployed because it was a few matrix operations per event, not because it was the most accurate model.
- **A random bucket is a capital investment.** The replay estimator is only unbiased because someone was willing to pay the regret of uniform exploration on a slice of traffic. Every later system in this chapter has its own version of that bucket.
- **Replay wastes data** in proportion to the number of arms. With K arms you keep about 1/K of the events, which is fine for about 20 news stories and useless for a million-item catalog. Section 11 covers the estimators that replaced it.

---

3. Thompson sampling: Yahoo display ads, news, and Bing adPredictor

Probability matching

Thompson sampling (TS) keeps a posterior over each arm's reward model. At each decision it samples one model from the posterior and acts greedily with respect to the sample. Arms are therefore chosen with exactly the posterior probability that they are optimal. Russo, Van Roy and colleagues give the standard modern treatment, including contextual, linear and non-conjugate versions [4].

Chapelle and Li (2011)

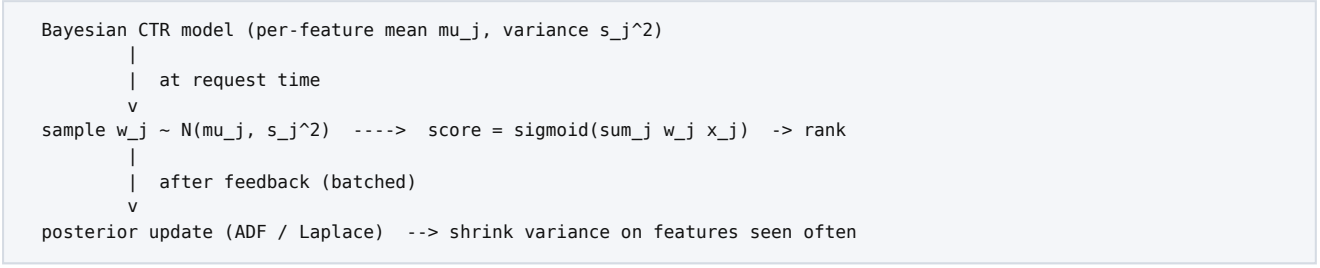
Chapelle and Li's NeurIPS 2011 paper, "An Empirical Evaluation of Thompson Sampling", brought the method back into mainstream use [3]. Using Yahoo data from **display advertising** and **news article recommendation**, they found that TS was competitive with, and often better than, UCB-style methods. They also highlighted two production properties:

- **Robustness to delayed and batched updates.** Production systems update models in batches (for example every few minutes or hours), not after every event. UCB is deterministic between updates, so it shows the same "most uncertain" arm to everyone in the batch. TS randomizes, so it naturally spreads exploration across the batch [3].
- **Easy integration with existing Bayesian CTR models.** For ads they used a Bayesian logistic regression with a Laplace (Gaussian) approximation to the posterior. Sampling the weights from that Gaussian gives TS for free [3]. They also found that shrinking the posterior variance (a tuning knob that makes exploration less aggressive) helped in practice.

Bing adPredictor

Microsoft's adPredictor for Bing sponsored search, described by Graepel, Candela, Borchert and Herbrich at ICML 2010, is a Bayesian probit regression over sparse binary features, trained online with assumed-density filtering [5]. Each weight has a Gaussian mean and variance, so the model always knows how uncertain it is about a new ad or feature. The paper discusses using the posterior for exploration: sampling from the weight distribution gives new ads impressions in proportion to their chance of being good, which is Thompson sampling by another name [5].

Design pattern



The lesson that matters for system design: **if your model already carries uncertainty, exploration is a sampling step at inference, not a separate system.** That is why TS dominates in ads, layout optimization and any setting with batched updates.

4. Microsoft Decision Service and MSN (2016-2017)

Agarwal et al. built the **Decision Service**, described as the first general-purpose system for contextual-bandit learning [6]. It turns exploration into a platform made of four abstractions arranged in a loop: **explore, log, learn, deploy**.

- **Explore**: the client library picks an action using an exploration policy (epsilon-greedy, bagging, softmax or "cover") wrapped around the current model, and records the probability of the chosen action.
- **Log**: the context, action, probability and a unique event id are written immediately. The reward is joined later by event id within a fixed window. Because logging happens at the decision point, the data is "correct" by construction.
- **Learn**: Vowpal Wabbit trains online on the joined stream, and off-policy estimators evaluate candidate policies on the same data.
- **Deploy**: new models are pushed to clients every few minutes.

Reported production results include two content-recommendation deployments with 25-30% click-through improvements and a landing page with an 18% revenue lift [6]. Agarwal and Langford reported that MSN.com clicks rose 26% after the contextual-bandit system went live in January 2016 [7]. The technology shipped as the Custom Decision Service and later as **Azure Personalizer** in Azure Cognitive Services, and the algorithms are open source in Vowpal Wabbit [7].

The key architectural idea is **multiworld testing**. Once every decision is logged with its propensity, you can evaluate many policies offline from one stream of data, instead of running an A/B test per policy. The data needed grows only logarithmically in the number of policies evaluated [6].

---

5. Netflix: artwork personalization with contextual bandits (2016-2017)

Problem

Every Netflix title has several pieces of artwork. The image a member sees is the strongest signal on the home page about whether to press play. In 2016 Netflix described using A/B tests and explore-exploit to choose the single best image per title for everyone [8]. In 2017 it went further: choose the best image **per member, per title** [9].

Formulation

Chandrashekar, Amat, Basilico and Jebara describe the problem as a contextual bandit [9]:

- **Arms**: the candidate images for a title.
- **Context**: member features such as viewing history and genre preferences, plus the title, device, time and other page context. An often-quoted example: a member who

watches many romances sees a romantic-lead image for *Good Will Hunting*, and a comedy fan sees an image featuring a comedian [9].

- **Reward**: whether the member played the title (and watched enough of it) after the impression. Netflix stresses avoiding **clickbait**: an image that attracts plays that are then abandoned quickly is penalized, so the reward is not a raw click.
- **Policy**: a supervised model predicts the probability of a quality play for each (member, title, image) and picks the best one, with an exploration component. The post names several possible exploration strategies for this setting, including simple randomization and Bayesian or upper-confidence approaches [9].

Logging and offline evaluation



Netflix logs every image impression with its context and the probability with which it was chosen. New policies are evaluated with **replay** on the randomized exploration data, using **take fraction** (plays divided by impressions) as the metric, before they go into online A/B tests [9]. The post reports that contextual policies beat non-personalized baselines on the replay metric, with online A/B tests used to confirm wins [9].

Engineering challenges called out

- **Only one image can be shown at a time.** Netflix cannot show a member two images for the same title, so attribution is per impression and exploration has a real cost (regret) [9].
- **Interaction with the rest of the page.** The image choice interacts with the other titles and rows on the page (diversity of images across a row, consistency for a member over time). A member who has seen one image and then sees a different one may be confused, so changes over time are managed.
- **Latency and scale.** Artwork selection runs for many titles per page for every member, so it must fit inside the page-generation latency budget. The post discusses the trade-off between precomputing selections and computing them live (approximate characterization).
- **Artwork supply.** Personalization only helps if there is a diverse set of images. Netflix invested in tooling to generate many candidate images per title.

Netflix later wrote about the broader framing in "Recommending for Long-Term Member Satisfaction" (2024): treat recommendation as a contextual bandit whose reward should reflect long-term satisfaction, engineer **proxy rewards** that predict delayed satisfaction, and accept that the reward function is itself a product decision to be iterated on [10].

---

6. Spotify: BaRT and the home page (2018) and impatient bandits (2023)

BaRT: Bandits for Recommendations as Treatments

McInerney et al. (Spotify, RecSys 2018) describe **BaRT**, the bandit system behind recommendations on Spotify's home page [11]. The home page is a grid of **shelves** (rows) of playlists and albums, and each recommendation can carry an **explanation**, such as "Because you listened to X" or "Your heavy rotation". BaRT treats the (item, explanation) pair as the action and learns which pairing is most likely to lead to engagement for a given user.

Core design [11]:

- **Reward model**: a factorization machine over user, item and explanation features and their interactions, predicting the probability of a stream of meaningful length after the impression.
- **Exploration**: epsilon-greedy. With probability 1 - epsilon show the predicted best, otherwise sample uniformly from the candidates. Epsilon-greedy was chosen deliberately because its propensities are trivially known: (1 - epsilon) + epsilon/K for the greedy arm and epsilon/K for the others.
- **Counterfactual training**: because propensities are logged, BaRT trains its reward model on the logged data with importance weighting, which removes the bias from always showing the greedy choice.
- **Explanations as actions**: the paper shows that explanations change engagement, so the same item with a different explanation is a different treatment.

```
candidates (items) x explanations  --->  FM reward model r_hat(u, i, e)
                                                    |
                                                    v
                                  epsilon-greedy over (i, e) per shelf slot
                                                    |
                  log (u, i, e, p, context) ------+---> home page shelf
                                                    |
                  stream >= threshold seconds? ---+---> reward
                                                    |
                  propensity-weighted retraining <+
```

The paper reports that BaRT improved engagement over non-explorative baselines in both offline counterfactual evaluation and online tests, and that personalized explanations helped [11]. Exact lift numbers are not reproduced here.

Impatient bandits: optimizing for the long term without waiting

Spotify's 2023 KDD paper by Zhang et al. tackles a more recent problem: optimizing **podcast recommendations** for long-term engagement, defined as listening over roughly 60 days, without waiting 60 days to learn [12]. Their approach:

- Build a Bayesian filter that predicts the long-term reward from **progressively revealed** intermediate outcomes (for example engagement in the first days after the first listen).
- Use this progressively updated posterior inside a **Thompson sampling** bandit, so the bandit acts on partial information and sharpens its estimate as more days arrive.
- The paper reports that, on a podcast recommendation problem, this approach substantially outperformed bandits that used either short-term proxies alone or waited for the full delayed reward [12].

This is the bandit-world version of the "proxy reward" idea that Netflix and YouTube also apply.

---

7. Deezer carousels and Amazon page layouts: slate bandits in practice

Deezer: carousel personalization (RecSys 2020)

Bendada, Salha and Bontempelli (Deezer) study the **carousel**: a swipeable row of playlists in which the user sees only the first few cards without swiping [13]. They model it as a contextual **cascade** bandit:

- A user scrolls through the first L visible slots, and the model assumes items beyond the last one the user would have seen get no feedback (no penalty for unseen items).
- To keep training feasible across millions of users, they use **semi-personalization**: users are clustered and a bandit (Thompson sampling or UCB variants) learns per cluster.
- They released a public dataset and simulator derived from Deezer's playlist carousel so the community can reproduce the experiments [13].

The lesson is that the **click model of the surface** (cascade, top-L visible) matters as much as the bandit algorithm. Treating unseen cards as negatives would teach the policy to avoid positions that were never displayed.

Amazon: multivariate layout optimization (KDD 2017)

Hill et al. (Amazon) optimize a page made up of several **widgets** (for example image, headline, button, layout) with many possible values each, giving a combinatorial space of thousands of layouts [14]. They use a Bayesian linear model with pairwise interaction terms, Thompson sampling to choose a layout, and greedy **hill climbing** to find an approximate argmax of the sampled model, because full enumeration is too slow for real-time serving. In a live experiment the method found a layout that improved conversion by about 21% over the median layout [14]. The design pattern is general: **sample a model, then use a cheap combinatorial optimizer to act on it**.

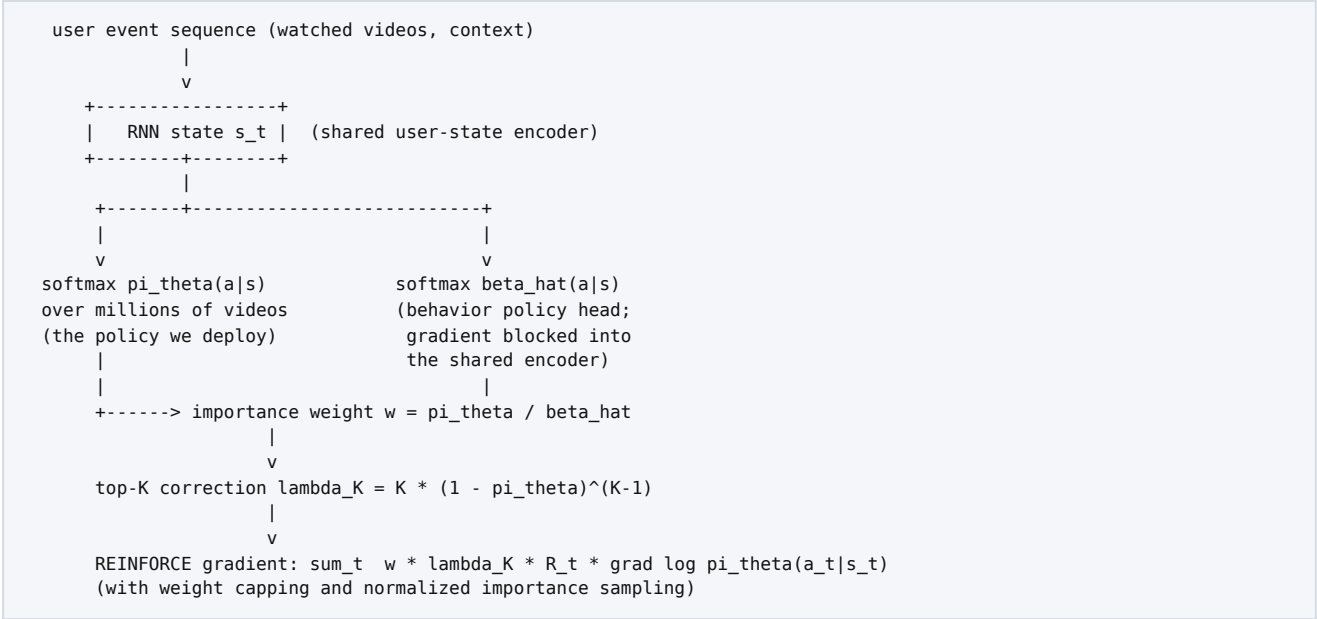
---

8. YouTube: top-K off-policy corrected REINFORCE (2019)

Problem

YouTube's candidate generator chooses a handful of videos from a corpus of millions for each request. It was trained on logs produced by many earlier versions of the recommender and by other systems. Chen, Beutel, Covington, Jain, Belletti and Chi (WSDM 2019) recast it as a **policy** pi(a | s) trained with **REINFORCE**, where the state s is the user's history and the action a is a video [15].

Architecture



The policy is a softmax over a very large action space (millions of videos), trained with sampled softmax [15].

Key tricks

1. **Off-policy correction.** The logs come from a behavior policy beta that is a mixture of historical systems and is unknown. The paper learns beta_hat with a second softmax head on the same state, with its gradient blocked so it does not distort the main encoder [15]. The policy gradient is reweighted by pi_theta / beta_hat.
2. **Top-K off-policy correction.** YouTube shows K items at once, not one. If the policy's probability of including item a in a set of K samples is alpha(a) = 1 - (1 - pi(a))^K, then the gradient picks up an extra multiplier lambda_K = K (1 - pi(a))^(K-1) [15]. When pi(a) is small, lambda_K is close to K, pushing hard to increase the probability. When pi(a) is already large, lambda_K approaches 0, so the policy stops over-concentrating on a single item and spreads mass across a good set of K.
3. **Variance reduction.** Importance weights over millions of actions have enormous variance. The paper uses weight capping, normalized (self-normalized) importance sampling, and a TRPO-style constraint [15].
4. **Exploration at serving.** Serving does not take the argmax. It returns the top K' most likely items and samples the remaining K - K' from the policy, a Boltzmann-style

exploration that also keeps the logged data rich [15].
5. **Reward.** The return combines the immediate reward of the watched item with discounted future rewards within the user's trajectory.

Results

The paper reports live experiments on YouTube in which the top-K off-policy correction gave statistically significant gains in viewing time, roughly 0.8-0.9% in the reported experiment, which is large at YouTube scale [15]. It also reports that the system surfaced more items from the long tail. The authors' simulations show that without correction the learned policy over-concentrates on items that the behavior policy already favored.

Follow-ups

- **Reward shaping for user satisfaction.** Christakopoulou et al. (Google, 2022) extend the REINFORCE recommender with reward shaping that learns from sparse satisfaction signals (such as survey responses) and imputes them onto unlabeled interactions [16].
- **Simulation.** Google released **RecSim**, a configurable simulator for sequential user-recommender interaction with user state, choice models and slates, for testing RL recommenders before they reach users [17].

---

9. SlateQ: tractable value-based RL for slates (Google, 2019)

Problem

Value-based RL (Q-learning) estimates the long-term value Q(s, A) of an action. In recommendation an action is a **slate** of k items from N candidates, so the action space is combinatorial (N choose k, ordered). Q over slates is neither learnable nor optimizable directly.

Decomposition

Ie et al. (Google, IJCAI 2019) show that under two assumptions the slate value decomposes into item-level values [18]:

- **Single choice (SC):** the user consumes at most one item from a slate (or none).
- **Reward and transition depend only on the selected item (RTDS):** the reward and next state depend on the item consumed, not on the rest of the slate.

Under SC and RTDS:

Q(s, A) = sum over i in A of P(i | s, A) * Q_bar(s, i)

where P(i | s, A) is a **user choice model** (for example a conditional logit with a "no click" option) and Q_bar(s, i) is the long-term value of the user consuming item i. Q_bar is learned with ordinary TD (SARSA or Q-learning) on item-level transitions, which is tractable.

Slate optimization

Given Q_bar and the choice model, choosing the best slate is a fractional mixed-integer program, which the paper shows can be solved exactly as a **linear program** for the conditional-logit model. Cheaper greedy and top-k heuristics work almost as well in practice [18].

```
offline logs (s, slate A, clicked i, r, s')
         |
         +--> choice model P(i | s, A)   (trained as a classifier)
         |
         +--> Q_bar(s, i) via SARSA/TD on (s, i, r, s', i')
                        |
serving:  candidates from retrieval (hundreds)
                        |
         score each i by P-weighted Q_bar
                        |
         slate = LP or greedy top-k  ---> user
```

Deployment

SlateQ was tested in simulation (in RecSim, which was released alongside it [17]) and in **live experiments on YouTube**, where it was plugged into an existing ranking stack by replacing the per-item myopic score with the decomposed long-term value. The paper reports improvements in user engagement over the myopic production baseline [18]. A practical point is that SlateQ **reuses the existing pointwise infrastructure**. It adds a value head trained with TD targets and a slate-optimization step, not a new serving system.

---

10. Long-term value, retention and notifications

Microsoft News: DRN (WWW 2018)

Zheng et al. built **DRN** for news recommendation in a Microsoft News app (China) [19]. Its contributions are typical of the early deep-RL wave:

- A Double/Dueling **DQN** whose reward combines click and **user activeness** (how soon the user returns), an early example of optimizing return rather than click.
- **Dueling Bandit Gradient Descent** for exploration: perturb the network weights to create an "explore" network, interleave its results with the current network's, and move the weights toward whichever gets more clicks. This avoids the poor user experience of random item exploration.
- Minor updates every interaction batch, major updates periodically.

JD.com: page-wise RL (RecSys 2018)

Zhao et al. (JD.com) apply actor-critic RL to recommend a **whole 2-D page** of items at once, with a CNN-based actor that generates the page and a critic that values it. They train against an offline simulator built from logs before going online [20].

Alibaba Virtual-Taobao (AAAI 2019)

Shi et al. trained an RL policy for Taobao search in a **learned simulator**. They modeled customers with GAN-based generation (GAN-SD) and multi-agent adversarial imitation learning (MAIL), then trained the ranking policy in the simulator. They reported an online revenue improvement of over 2% compared with the traditional supervised approach [21]. The lesson: **if online exploration is too costly, invest in a simulator, but validate it against real logs.**

Kuaishou: reinforcing user retention at billion scale (WWW 2023)

Cai et al. (Kuaishou) describe **RLUR**, which optimizes **user retention**, measured as the time until the user returns to the app, in a short-video system with hundreds of millions of daily users [22]. Retention is a delayed, sparse, user-level signal that is influenced by many sessions, which makes credit assignment hard.

Design highlights [22]:

- Model the problem as an infinite-horizon MDP in which each request is a step. The reward is defined at the session level: the gap until the next session.
- Learn a **retention critic** with a temporal-difference (TD) objective. Use heuristic immediate rewards (for example watch time) as auxiliary, reward-shaping signals to cut variance.
- Use a **soft regularization** and intrinsic-motivation components to stabilize learning and avoid policy collapse.
- The actor outputs **fusion weights** for the multi-objective ranking formula (how much to weight predicted like, watch time, follow and so on), not item scores directly. This keeps the action space small and low-dimensional, and the existing ranking stack untouched.

The paper reports statistically significant improvements in retention metrics (app return frequency and DAU-related metrics) in production A/B tests [22]. A companion Kuaishou paper, the **Two-Stage Constrained Actor-Critic** (TSCAC), tackles maximizing watch time subject to constraints on auxiliary interactions (likes, follows, shares) [23].

```
request t:  user state s_t (history, profile, time since last visit)
               |
               v
        actor pi(s_t) --> weights w_t = (w_watch, w_like, w_follow, ...)
               |
               v
   existing ranker: score(i) = sum_k w_t,k * prediction_k(i)   ---> feed
               |
 session ends; next visit after gap g  ---> reward r = f(g) (+ shaped terms)
               |
        critic V(s) trained by TD; actor updated by critic
```

The design idea, **RL controls the knobs of the ranker instead of the items**, is now a common production pattern. It keeps the action space continuous and low-dimensional, makes the policy safe to roll back, and reuses all existing predictions.

Notifications: Meta Horizon/ReAgent and Duolingo

Push notifications are a natural RL problem. Each send has an immediate effect (open) and a long-term cost (fatigue, disabling notifications, churn).

- **Meta Horizon (ReAgent).** Gauci et al. describe Horizon, Facebook's open-source applied RL platform, built for batch (offline) RL on logged data. It includes data preprocessing at scale, DQN, parametric-action DQN, and DDPG, and **counterfactual policy evaluation** (doubly robust, sequential DR, MAGIC) run before any deployment [25]. The paper reports production uses including **push notifications** (deciding which notifications to send to optimize long-term value rather than immediate clicks) and choosing when to prefetch content. Horizon was later renamed **ReAgent** [26].
- **Duolingo.** Yancey and Settles (KDD 2020) describe a "sleeping, recovering" bandit for daily practice reminders [27]. Arms are notification templates. Some arms are "asleep" (not eligible) for a given user on a given day. The reward of an arm **recovers** with time since it was last used, which captures novelty decay. The paper reports gains in user engagement (lesson completions and daily active users) in live experiments; the exact magnitudes are not reproduced here.

The common lesson is that a notification system must optimize **sends that do not destroy the channel**. That requires a long-horizon reward (or explicit fatigue modeling), plus counterfactual evaluation, because a bad notification policy damages the user base before an A/B test finishes.

RL for ads: auto-bidding

Advertising platforms use RL most directly in **bidding**, where the state (remaining budget, time of day, spend pace) and the episode (a day's budget) map cleanly onto an MDP.

- **Budget-constrained bidding** (Wu et al., Alibaba, CIKM 2018): model-free RL (DQN) adjusts a bid-scaling factor over the day to maximize value under a budget, with a reward design that avoids myopic overspending [28].
- **Multi-agent real-time bidding** (Jin et al., Alibaba, CIKM 2018): cluster advertisers into agents and train with a distributed multi-agent actor-critic, balancing competition and cooperation among advertisers on the Taobao platform [29].

In both, the RL policy outputs a **scalar control** (a bid multiplier), not an item choice. That is the same pattern as Kuaishou's fusion weights.

---

11. Off-policy evaluation: the shared foundation

Every design above depends on answering "what would policy pi' have earned?" from data logged under policy beta.

Estimator	Formula (informal)	Bias	Variance	Used by
Replay [2]	mean reward on events where pi' agrees with uniform log	unbiased (uniform log)	high; keeps ~1/K events	Yahoo, Netflix artwork
IPS	mean of r * pi'(a	x) / beta(a	x)	unbiased if beta known and > 0
Self-normalized IPS	IPS / mean of weights	small bias	lower	YouTube REINFORCE training
Direct method (DM)	mean of r_hat(x, pi'(x))	biased by model	low	baselines

Estimator	Formula (informal)	Bias	Variance	Used by
Doubly robust (DR) [30]	DM + IPS correction on residual	unbiased if either part is right	medium	Horizon/ReAgent, VW
Marginalized IPS (embeddings) [32]	weight by action-embedding ratio	small bias	far lower for large action sets	large catalogs

- **Doubly robust estimation** (Dudik, Langford and Li, ICML 2011) combines a reward model with importance weighting so that the estimate is unbiased if either the propensities or the reward model are correct [30].
- **Open Bandit Pipeline** (Saito et al., ZOZO and Hanjuku-Kaso, 2020) released a large real dataset collected by both uniform random and Bernoulli Thompson sampling policies on the ZOZOTOWN fashion site, with a Python library for comparing estimators [31]. It is the main public benchmark for checking whether an off-policy estimator would have predicted real online outcomes.
- **Large action spaces.** Saito and Joachims (ICML 2022) show that IPS breaks down with very large action spaces and propose **marginalized IPS** over action embeddings, which reweights by how likely the *embedding* was rather than the exact item [32]. This is the bridge from Yahoo-sized arm sets to YouTube-sized catalogs.

Practical OPE checklist

1. Log the propensity at decision time, from the serving code that made the decision, not from a model reconstructed later.
2. Never let a logged propensity be 0 for an action you may want to evaluate (exploration floor).
3. Join rewards by impression id with an explicit attribution window, and record the window.
4. Report effective sample size alongside the estimate. When weights explode, the estimate is meaningless, whatever the confidence interval says.
5. Validate offline estimators against at least a few online A/B results before trusting them for launch decisions. Netflix [9] and Yahoo [2] both did this explicitly.

---

12. Comparison of production designs

System	Year	Formulation	Action space	Exploration	Reward	Reported outcome
Yahoo LinUCB [1][2]	2010	Contextual bandit, linear	~tens of articles	UCB bonus	click	+12.5% clicks vs context-free bandit
Yahoo TS ads/news [3]	2011	Bayesian logistic + TS	ads, articles	posterior sampling	click	TS competitive with or better than UCB
Bing adPredictor [5]	2010	Bayesian probit, online	ads	posterior sampling	click	production Bing CTR model
MS Decision Service [6][7]	2016	Contextual bandit platform	small per decision	eps-greedy, bagging, cover	click, revenue	+25-30% CTR; MSN +26% clicks
Netflix artwork [9]	2017	Contextual bandit	images per title	randomized exploration	quality play	beats non-personalized on replay and online

System	Year	Formulation	Action space	Exploration	Reward	Reported outcome
Spotify BaRT [11]	2018	Contextual bandit + FM	(item, explanation)	epsilon-greedy	stream >= threshold	engagement lift vs non-explorative
Spotify impatient bandits [12]	2023	TS + progressive Bayesian filter	podcasts	Thompson sampling	~60-day engagement	beats short-term proxy bandits
Deezer carousel [13]	2020	Cascade contextual bandit	playlists, top-L visible	TS / UCB per cluster	stream	public dataset and simulator
Amazon layout [14]	2017	Bayesian linear + TS + hill climb	~thousands of layouts	Thompson sampling	conversion	~+21% vs median layout
YouTube REINFORCE [15]	2019	Off-policy policy gradient	millions of videos	sampling from policy	watch-based return	~+0.8-0.9% viewing time (live)
Google SlateQ [18]	2019	Decomposed Q-learning on slates	slate from candidates	policy-based	long-term engagement	gains in live YouTube tests
Microsoft DRN [19]	2018	DQN + dueling bandit GD	news	weight perturbation	click + return	gains in offline and online tests
Alibaba Virtual-Taobao [21]	2019	RL in learned simulator	ranking policy	in simulator	revenue	>2% online revenue
Kuaishou RLUR [22]	2023	Actor-critic on fusion weights	continuous weight vector	policy noise	return time (retention)	significant retention gains
Meta Horizon/ReAgent [25]	2018	Batch RL + CPE	notifications, prefetch	policy-based	long-term value	production use at Facebook
Duolingo reminders [27]	2020	Sleeping, recovering bandit	message templates	bandit	lesson completion	engagement gains in live tests
Alibaba bidding [28][29]	2018	DQN / multi-agent AC	bid scalar	policy noise	value under budget	online gains on Taobao ads

13. Recurring design patterns

Pattern 1: Exploration bucket plus propensity logging

Reserve a slice of traffic with known, simple randomization (uniform or epsilon-greedy), and log propensities everywhere else as well. Yahoo, Netflix, Spotify, Microsoft and ZOZO all do this. The bucket is what makes unbiased offline evaluation and model training possible.

Pattern 2: Posterior sampling as the default explorer

When the reward model can carry uncertainty (Bayesian linear or logistic regression, bootstrapped ensembles, dropout ensembles), Thompson sampling gives exploration that tolerates batched updates and integrates as a sampling step at inference [3][4][5][14].

Pattern 3: RL on top of, not instead of, the ranker

The most successful deep-RL deployments change little in serving:

- YouTube corrects the training objective of an existing softmax candidate generator [15].
- SlateQ reweights existing pointwise scores by long-term value [18].
- Kuaishou RLUR and ad auto-bidders output a handful of continuous knobs [22][28].

Full end-to-end RL that picks items from scratch is rare in production.

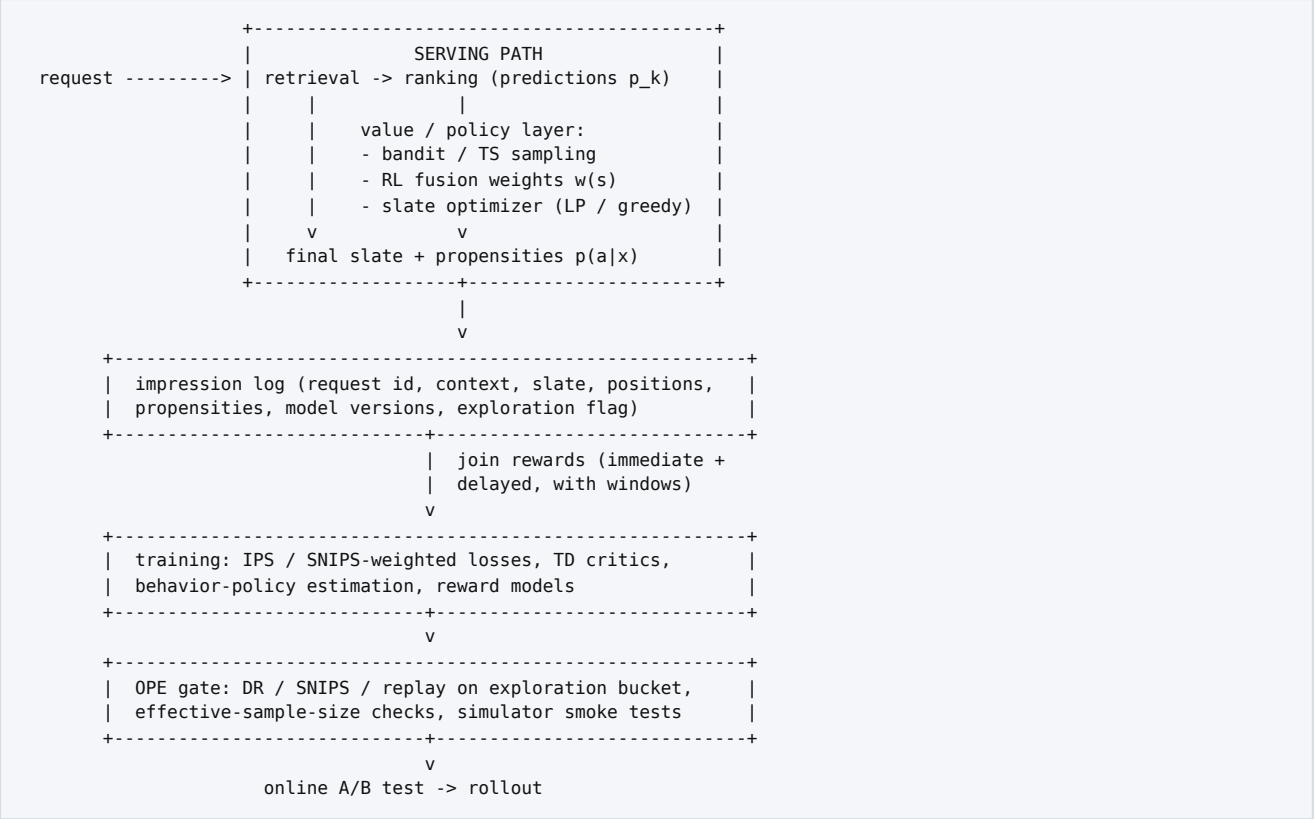
Pattern 4: Proxy and progressive rewards for long horizons

Optimizing retention directly is slow and noisy. The working solutions predict the long-term reward from early signals (Spotify [12], Netflix [10]), shape rewards with immediate signals (Kuaishou [22]), or learn satisfaction imputations (YouTube [16]).

Pattern 5: Simulation before exposure

RecSim [17], Virtual-Taobao [21] and JD's simulator [20] all let teams debug RL policies before exposing real users. Simulators are useful for catching bugs and comparing algorithm families. Their absolute numbers rarely transfer, so they must be validated against held-out logs.

Reference architecture



---

14. Key takeaways for system designers

1. **Treat exploration as infrastructure, not an algorithm choice.** The Decision Service showed that explore, log, learn and deploy belong in a shared platform [6].

The individual bandit algorithm matters less than getting propensities logged correctly at decision time.

2. **The logging schema is the contract.** Context, action, position, propensity, model version and exploration flag must be logged by the serving code at the moment of decision. Reconstructing propensities later is a common and silent source of bias.
3. **Pay for a random bucket.** A small uniformly random (or epsilon-greedy) slice of traffic makes unbiased replay and IPS evaluation possible and breaks degenerate feedback loops [2][9][24].
4. **Prefer Thompson sampling when updates are batched.** It spreads exploration across a batch and reuses existing Bayesian models [3][5].
5. **Use the right click model for the surface.** Cascades, carousels and grids have positions users never see. Do not count them as negatives [13].
6. **For large catalogs, correct, do not replace.** YouTube's top-K off-policy correction shows how to add RL to a softmax retrieval model trained on biased logs, with behavior-policy estimation, weight capping and self-normalization [15].
7. **Decompose slates.** SlateQ's choice-model decomposition makes long-term value learnable and optimizable with existing pointwise infrastructure [18].
8. **Let RL turn the knobs.** Controlling fusion weights or bid multipliers keeps the action space small, the policy safe to roll back, and the rest of the stack intact [22] [28].
9. **Engineer the reward explicitly and treat it as a product decision.** Click rewards produce clickbait; Netflix's quality-play reward [9], Spotify's 60-day objective [12] and Kuaishou's return-time reward [22] all reflect deliberate reward design.
10. **Gate every launch on off-policy evaluation, then A/B test.** Use doubly robust or marginalized estimators, check effective sample size, and validate estimators against past online results [30][31][32].
11. **Simulate to debug, not to decide.** Simulators catch bugs and compare algorithm families. Launch decisions still need real traffic [17][21].
12. **Channels like notifications need long-horizon objectives.** Optimizing opens per send burns out users. Use long-term value and fatigue modeling with counterfactual evaluation before deployment [25][27].

References

1. L. Li, W. Chu, J. Langford, R. E. Schapire. "A Contextual-Bandit Approach to Personalized News Article Recommendation." Yahoo! Labs, WWW 2010. https://arxiv.org/abs/1003.0146
2. L. Li, W. Chu, J. Langford, X. Wang. "Unbiased Offline Evaluation of Contextual-bandit-based News Article Recommendation Algorithms." Yahoo! Labs, WSDM 2011. https://arxiv.org/abs/1003.5956
3. O. Chapelle, L. Li. "An Empirical Evaluation of Thompson Sampling." Yahoo! Research, NeurIPS 2011. https://papers.nips.cc/paper/2011/hash/e53a0a2978c28872a4505bdb51db06dc-Abstract.html
4. D. Russo, B. Van Roy, A. Kazerouni, I. Osband, Z. Wen. "A Tutorial on Thompson Sampling." Stanford / DeepMind, 2017. https://arxiv.org/abs/1707.02038

5. T. Graepel, J. Quiñonero Candela, T. Borchert, R. Herbrich. "Web-Scale Bayesian Click-Through Rate Prediction for Sponsored Search Advertising in Microsoft's Bing Search Engine." Microsoft Research, ICML 2010. https://www.microsoft.com/en-us/research/publication/web-scale-bayesian-click-through-rate-prediction-for-sponsored-search-advertising-in-microsofts-bing-search-engine/
6. A. Agarwal, S. Bird, M. Cozowicz, et al. "A Multiworld Testing Decision Service" (also "Making Contextual Decisions with Low Technical Debt"). Microsoft Research, 2016 (arXiv 1606.03966). https://www.microsoft.com/en-us/research/publication/multiworld-testing-decision-service/
7. A. Agarwal, J. Langford. "Real world interactive learning at cusp of enabling new class of applications." Microsoft Research Blog, 2017. https://www.microsoft.com/en-us/research/blog/real-world-interactive-learning-cusp-enabling-new-class-applications/
8. G. Krishnan. "Selecting the best artwork for videos through A/B testing." Netflix Technology Blog, 2016. https://netflixtechblog.com/selecting-the-best-artwork-for-videos-through-a-b-testing-f6155c4595f6
9. A. Chandrashekar, F. Amat, J. Basilico, T. Jebara. "Artwork Personalization at Netflix." Netflix Technology Blog, 2017. https://netflixtechblog.com/artwork-personalization-c589f074ad76
10. Netflix. "Recommending for Long-Term Member Satisfaction at Netflix." Netflix Technology Blog, 2024. https://netflixtechblog.com/recommending-for-long-term-member-satisfaction-at-netflix-ac15cada49ef
11. J. McInerney, B. Lacker, S. Hansen, K. Higley, H. Bouchard, A. Gruson, R. Mehrotra. "Explore, Exploit, and Explain: Personalizing Explainable Recommendations with Bandits." Spotify, RecSys 2018. https://dl.acm.org/doi/10.1145/3240323.3240354
12. T. M. Zhang, et al. "Impatient Bandits: Optimizing Recommendations for the Long-Term Without Delay." Spotify, KDD 2023. https://arxiv.org/abs/2307.09943
13. W. Bendada, G. Salha, T. Bontempelli. "Carousel Personalization in Music Streaming Apps with Contextual Bandits." Deezer, RecSys 2020. https://arxiv.org/abs/2009.06546
14. D. N. Hill, H. Nassif, Y. Liu, A. Iyer, S. V. N. Vishwanathan. "An Efficient Bandit Algorithm for Realtime Multivariate Optimization." Amazon, KDD 2017. https://arxiv.org/abs/1810.09558
15. M. Chen, A. Beutel, P. Covington, S. Jain, F. Belletti, E. H. Chi. "Top-K Off-Policy Correction for a REINFORCE Recommender System." Google/YouTube, WSDM 2019. https://arxiv.org/abs/1812.02353
16. K. Christakopoulou, C. Xu, S. Zhang, et al. "Reward Shaping for User Satisfaction in a REINFORCE Recommender." Google, 2022. https://arxiv.org/abs/2209.15166
17. E. Ie, C. Hsu, M. Mladenov, et al. "RecSim: A Configurable Simulation Platform for Recommender Systems." Google Research, 2019 (arXiv 1909.04847); repository. https://github.com/google-research/recsim
18. E. Ie, V. Jain, J. Wang, et al. "SlateQ: A Tractable Decomposition for Reinforcement Learning with Recommendation Sets." Google, IJCAI 2019 (extended version arXiv 1905.12767). https://arxiv.org/abs/1905.12767
19. G. Zheng, F. Zhang, Z. Zheng, et al. "DRN: A Deep Reinforcement Learning Framework for News Recommendation." Penn State and Microsoft, WWW 2018. https://dl.acm.org/doi/10.1145/3178876.3185994
20. X. Zhao, L. Xia, L. Zhang, Z. Ding, D. Yin, J. Tang. "Deep Reinforcement Learning for Page-wise Recommendations." JD.com and Michigan State, RecSys 2018. https://arxiv.org/abs/1805.02343

21. J.-C. Shi, Y. Yu, Q. Da, S.-Y. Chen, A.-X. Zeng. "Virtual-Taobao: Virtualizing Real-world Online Retail Environment for Reinforcement Learning." Nanjing University and Alibaba, AAAI 2019. https://arxiv.org/abs/1805.10000
22. Q. Cai, S. Liu, X. Wang, et al. "Reinforcing User Retention in a Billion Scale Short Video Recommender System." Kuaishou, WWW 2023 Companion. https://arxiv.org/abs/2302.01724
23. Q. Cai, Z. Xue, C. Zhang, et al. "Two-Stage Constrained Actor-Critic for Short Video Recommendation." Kuaishou, WWW 2023. https://arxiv.org/abs/2302.01680
24. R. Jiang, S. Chiappa, T. Lattimore, A. György, P. Kohli. "Degenerate Feedback Loops in Recommender Systems." DeepMind, AIES 2019. https://arxiv.org/abs/1902.10730
25. J. Gauci, E. Conti, Y. Liang, et al. "Horizon: Facebook's Open Source Applied Reinforcement Learning Platform." Facebook, 2018. https://arxiv.org/abs/1811.00260
26. Meta. "ReAgent: A platform for reasoning systems (formerly Horizon)." GitHub repository. https://github.com/facebookresearch/ReAgent
27. K. P. Yancey, B. Settles. "A Sleeping, Recovering Bandit Algorithm for Optimizing Recurring Notifications." Duolingo, KDD 2020. https://research.duolingo.com/papers/yancey.kdd20.pdf
28. D. Wu, X. Chen, X. Yang, et al. "Budget Constrained Bidding by Model-free Reinforcement Learning in Display Advertising." Alibaba, CIKM 2018. https://arxiv.org/abs/1802.08365
29. J. Jin, C. Song, H. Li, et al. "Real-Time Bidding with Multi-Agent Reinforcement Learning in Display Advertising." Alibaba, CIKM 2018. https://arxiv.org/abs/1802.09756
30. M. Dudik, J. Langford, L. Li. "Doubly Robust Policy Evaluation and Learning." ICML 2011. https://arxiv.org/abs/1103.4601
31. Y. Saito, S. Aihara, M. Matsutani, Y. Narita. "Open Bandit Dataset and Pipeline: Towards Realistic and Reproducible Off-Policy Evaluation." ZOZO / Hanjuku-Kaso / Yale, NeurIPS 2021 Datasets and Benchmarks. https://arxiv.org/abs/2008.07146
32. Y. Saito, T. Joachims. "Off-Policy Evaluation for Large Action Spaces via Embeddings." Cornell, ICML 2022. https://arxiv.org/abs/2202.06317

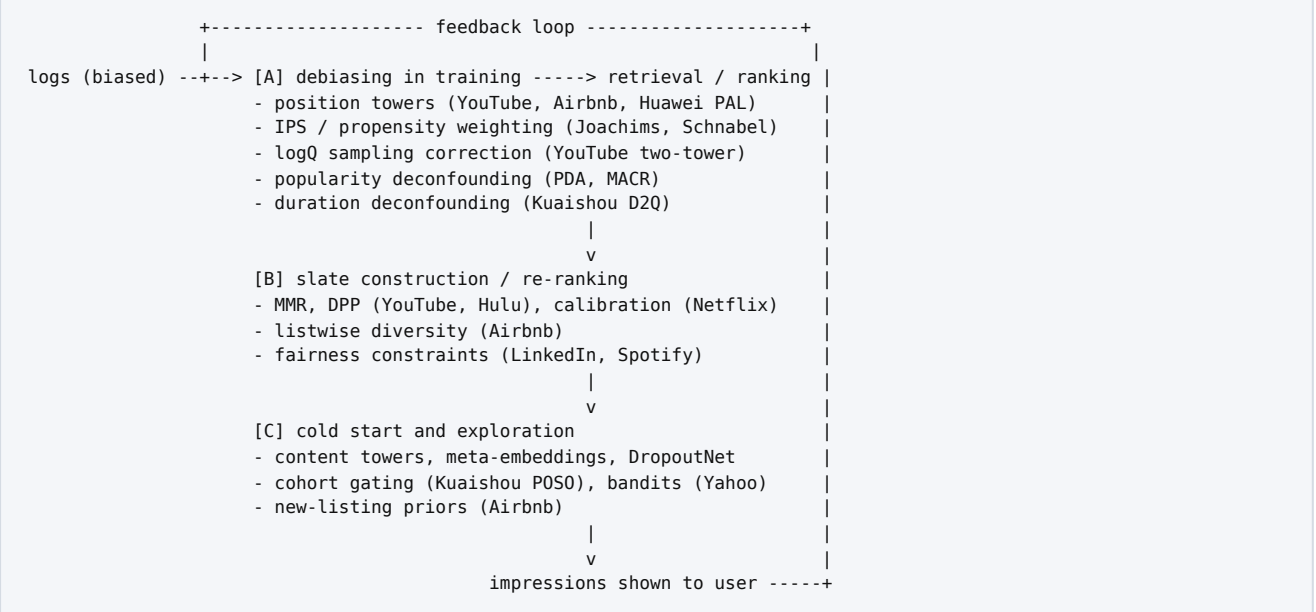
14. Debiasing, Diversity, Fairness and Cold Start: The Corrective Layers of Production Recommenders

> Research note: when this chapter was written, live web search and page fetching were not available in the research environment (the shared search budget was used up and outbound fetches failed DNS resolution). The chapter is therefore built from well-known, widely cited primary papers and engineering write-ups whose content and identifiers the author knows. The claims follow those sources. Any number not checked against the source is marked "approximately" or "reported as". Check figures against the cited originals before quoting them.

Overview

Every production recommender learns from logs that it produced itself. A user can only click what was shown, items shown at the top get more clicks, popular items get more impressions and so look more relevant, and new users, items and creators have almost no data at all. Left alone, the system's feedback loop makes all of these effects stronger. The result is a narrow, popularity-heavy, position-driven product that can be unfair to suppliers.

Large platforms answer this with a set of "corrective layers" that sit around the core retrieval and ranking stack (Chapters 2-4):



This chapter goes through each layer. For each one it covers the problem, the deployed designs, and the lessons for system designers.

14.1 A Taxonomy of Bias in Recommender Logs

The survey by Chen et al. [19] is the standard map of the field. It separates:

Bias	Where it enters	Symptom	Typical fix
Position bias	User clicks on what is ranked high	CTR depends on slot, not relevance	Position feature + dropout, IPS, PAL
Selection bias	Users rate only what they choose	Explicit ratings are MNAR	Propensity-weighted losses [13]
Exposure bias	Users only see a subset of items	Unobserved does not mean negative	Exposure modelling [14], off-policy correction [31]
Popularity bias	Training data dominated by head	Long tail under-recommended	Regularisers [16], deconfounding [17][18], logQ [11]
Conformity	Users imitate the crowd	Labels not individual preference	Causal disentangling
Duration / length bias	Longer videos get more watch time	Watch-time model prefers long content	Duration-bucketed targets [27]
Inductive / feedback-loop bias	Model's own outputs become training data	Homogenisation, filter bubbles	Exploration, randomisation, diversity
Unfairness	Unequal exposure across groups	Creators or sellers starved	Exposure constraints [20][21][22]

Two kinds of fix come up again and again:

1. **Model the bias explicitly as a separate factor** (a "bias tower" or propensity), train it jointly with the relevance model, and remove it at serving time.
2. **Reweight the data** by inverse propensities so that the expected loss matches the loss on unbiased data.

Production systems prefer (1). It is easier to run, has lower variance, and fits into multitask DNN rankers.

14.2 Position Bias

14.2.1 Unbiased learning to rank: IPS and propensity estimation

Problem. A click on position 1 says less about relevance than a click on position 10, because users look at position 1 far more often. Joachims, Swaminathan and Schnabel [8] cast this as counterfactual learning. Under an examination hypothesis,

```
P(click | q, d, k) = P(examined | k) * P(relevant | q, d)
```

and each click is weighted by 1 / P(examined | k). The IPS-weighted ranking loss is then an unbiased estimate of the loss under full-information relevance labels.

Estimating propensities. The original work used randomisation interventions, such as swapping results at rank 1 and rank k, to estimate a propensity curve that roughly follows a power law in rank. Swaps hurt the user experience. Agarwal et al. [29] showed that

propensities can instead be estimated without intrusive interventions, using "intervention harvesting": when several historical ranking functions put the same document at different positions, the logs already contain natural swaps. Google's personal-search team (Wang et al. [28][34]) estimated position bias with regression EM, again avoiding heavy randomisation, and used selection-bias weights in email search ranking.

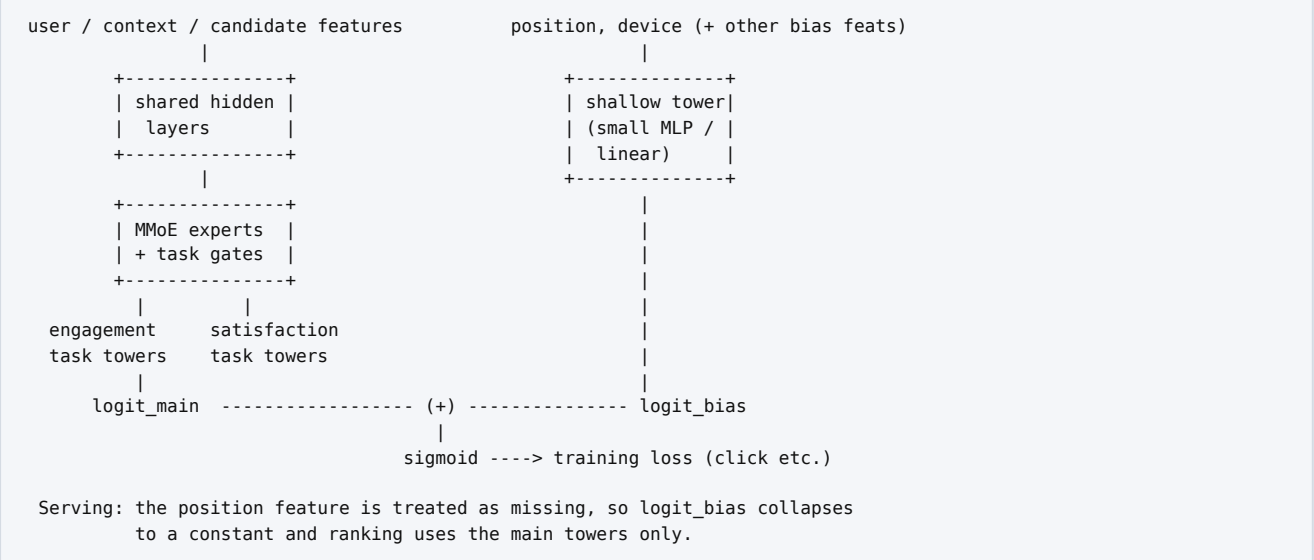
Lessons. IPS is unbiased only if the propensity model is correct. Weights at deep ranks blow up variance, so practitioners clip them. IPS also struggles with features that interact with position, such as device type or presentation layout. That limitation motivated the "bias tower" designs below.

14.2.2 YouTube: the shallow tower in the Watch Next ranker

System. YouTube's "Recommending What Video to Watch Next" ranking system (Zhao et al., RecSys 2019) [1] ranks a few hundred candidates from multiple candidate generators. It uses a Multi-gate Mixture-of-Experts (MMoE) network with engagement objectives (clicks, watch time) and satisfaction objectives (likes, dismissals, survey responses).

Bias problem. Watch Next logs carry strong selection bias. Users click videos near the top of the list partly because of position, not utility. The paper calls this an implicit feedback loop: training on these logs keeps reinforcing the current ranking.

Architecture.



Key tricks.

- The shallow tower gets features that drive bias but not relevance: the position of the item when it was shown, plus device information. Device matters because position effects differ between mobile and desktop.
- The bias logit is **added** to the main model's logit before the sigmoid. This factorises the click probability into a relevance part and a bias part.
- During training the position feature is dropped out at a reported rate of about 10%. This stops the main model from relying on position, and the model learns sensible behaviour when position is missing, which is exactly the serving setting.
- At serving, position is set to missing.

Results. The paper reports that the shallow tower improved the engagement metric in live experiments over baselines, including over using position as a plain input feature. The learned position biases fall monotonically with position, as expected.

Lesson. A "bias-only" tower that is added in logit space is cheap (a few extra parameters), works with any multitask ranker, and has become the de facto industrial pattern. Later work in industry and academia extends it with context-aware bias towers (position crossed with device, UI surface, or session depth).

14.2.3 Huawei PAL: position as a separate probability

Huawei's PAL (Guo et al., RecSys 2019) [9], deployed in the Huawei App Store, factorises CTR multiplicatively:

```
pCTR(click | x, pos) = pSeen(pos) * pCTR(click | x, seen)
```

Two modules, a position module and a CTR module, are trained jointly on the click label. At serving only the CTR module ranks candidates, so the system never needs to know the position before choosing it. That breaks the chicken-and-egg problem of using position as a feature in a ranker that decides positions. In online A/B tests PAL reported gains in CTR and conversion rate over a "position as feature" baseline. The multiplicative form is the probabilistic twin of YouTube's additive-logit tower.

14.2.4 Airbnb: position features with dropout in search ranking

System. Airbnb's search ranking moved from GBDT to deep networks in 2018-2019 [6]. In "Improving Deep Learning for Airbnb Search" (Haldar et al., KDD 2020) [5] the team describes its next round of changes, including handling of position bias.

Problem. Guest bookings depend heavily on where a listing appears. Airbnb is a two-sided marketplace with limited inventory, so a model that learns "top position means good listing" also entrenches incumbents and hurts new hosts.

Design. Airbnb added the listing's search position as a training feature and applied dropout to it during training. At inference the position feature is set to a constant (zero), so all candidates are scored as if they sat in the same position. The paper treats the dropout rate as a regularisation knob. Too little dropout and the model over-relies on position, which hurts offline relevance with position removed. Too much and the model cannot separate position effects. The team tuned the rate to balance the two and reported a statistically significant booking gain online.

Lessons.

- Training with "position as feature" and serving with "position = constant" is the simplest debiasing you can deploy. It needs no propensity estimation.
- In marketplaces, position debiasing is also a fairness and cold-start lever. It lowers the advantage that listings get from having ranked well in the past.

14.2.5 Comparison of position-debiasing designs

Design	Org	Bias model	Combination	Serving	Needs randomisation ?
IPS-LTR [8]	Cornell / academic	Propensity per rank	Loss weight 1/p_k	Unchanged ranker	Yes (or harvested [29])
Regression-EM [28]	Google personal search	Propensity per rank	Loss weight	Unchanged ranker	No
Shallow tower [1]	YouTube	Small net on position + device	Additive logit	Position missing	No
PAL [9]	Huawei App Store	pSeen(pos) module	Multiplicative probability	CTR module only	No
Position feature + dropout [5]	Airbnb search	Position as input feature	Inside main DNN	Position = 0	No

---

14.3 Popularity, Exposure and Sampling Bias

14.3.1 Popularity bias and the long tail

Abdollahpouri et al. [15] show that common collaborative filtering algorithms not only over-recommend popular items but also serve users with niche tastes worse than mainstream users, because their recommendations drift toward the head. Earlier work by the same group [16] added a regulariser to a learning-to-rank objective that penalises lists made only of head items. This made an explicit accuracy versus long-tail-coverage trade-off.

Causal methods treat popularity as a confounder:

- **PDA (Popularity-bias Deconfounding and Adjusting)**, Zhang et al., SIGIR 2021 [17]. Item popularity affects both exposure and the click label. PDA trains a deconfounded model by removing the confounding popularity effect through backdoor adjustment. It then optionally injects *predicted future* popularity at inference. The point is that some popularity is real signal (trending items) and should be kept on purpose, not by accident.
- **MACR**, Wei et al., KDD 2021 [18]. Builds a causal graph with user, item and user-item-matching branches. Training uses multitask learning with auxiliary item-only and user-only modules. At inference, counterfactual reasoning subtracts the direct item (popularity) effect from the prediction.

14.3.2 Sampling bias in two-tower retrieval: logQ correction (YouTube)

In-batch softmax for two-tower retrieval (Chapter 2) treats the other items in a batch as negatives. Popular items show up in batches far more often, so they are over-penalised as negatives. That would bias the model against the head. Yi et al., RecSys 2019 [11], built YouTube's retrieval around **logQ correction**:

```
s_c(x, y_j) = s(x, y_j) - log(p_j)
```

Here p_j is the probability that item j appears in a batch. Because the training data streams, p_j cannot be counted offline. The paper therefore introduces a **streaming**

frequency estimator: a hash array that records the last global step at which each item was seen and keeps a moving average of the gap between sightings (delta). The estimate is p_j ≈ 1 / delta. The design was deployed for YouTube candidate generation over a corpus of (reported) tens of millions of videos. LogQ correction is now standard in almost every in-batch two-tower system.

Design lesson. Popularity bias can enter through the *training procedure* (negative sampling) as well as through the data. Check the sampler before blaming the logs.

14.3.3 Exposure bias and off-policy correction

Implicit feedback mixes "did not like" with "never saw". Liang et al. [14] model exposure as a latent variable (ExpoMF): a missing interaction counts as negative evidence only to the degree that the user was probably exposed to the item. Schnabel et al. [13] frame recommendation as treatments and use propensity-scored estimators for both training and evaluation of rating models under missing-not-at-random data.

At industry scale, YouTube's REINFORCE recommender (Chen et al., WSDM 2019, "Top-K Off-Policy Correction") [31] tackles the exposure bias of training a policy on logs produced by earlier policies:

- A separate head estimates the **behaviour policy** beta(a|s), meaning the probability that the logged system showed item a. Gradients are importance-weighted by pi/beta, with weight capping to control variance.
- A **top-K correction** multiplier, lambda_K = K * (1 - pi)^(K-1), accounts for the fact that the policy recommends a slate of K items, not a single one.
- The paper reports that the off-policy-corrected agent increased viewing time in live experiments, and that the gains came disproportionately from items that the earlier system rarely surfaced. The correction expands exposure toward the tail.

Chapter 13 covers RL and exploration in more depth.

14.3.4 YouTube's "example age" feature: debiasing toward the past

Covington et al. [12] made a simpler but influential observation. A model trained on historical windows learns the *average* popularity of an item over the training window. That under-predicts fresh, viral uploads. YouTube added the **age of the training example** as a feature during training and set it to zero (or slightly negative) at serving. This lets the model represent time-dependent popularity and shows "now" at serving. The same paper also takes a **fixed number of training examples per user**, so that a small set of heavy users does not dominate the loss, which is a form of reweighting against activity bias.

14.3.5 Duration bias in watch time: Kuaishou D2Q

Short-video platforms optimise watch time. Longer videos naturally collect more watch time, so a naive regressor favours length. Kuaishou's D2Q (Zhan et al., KDD 2022) [27] treats video duration as a confounder that affects both exposure and watch time:

```
     duration --------> exposure (system shows long videos more)
        |                  |
        v                  v
              watch time (label)

  D2Q: split training data into duration buckets
       -> within each bucket, convert watch time into a quantile label
       -> fit one model (shared parameters) predicting the within-bucket quantile
       -> serve on quantiles: compares "how well this video holds attention
          relative to similar-length videos"
```

The paper reports online gains on Kuaishou over naive watch-time regression and over value-weighted baselines. Later short-video work builds on the same "normalise by length group" idea.

14.4 Diversity: From MMR to Learned Listwise Rankers

14.4.1 Classic re-ranking: MMR and topic diversification

Maximal Marginal Relevance (Carbonell and Goldstein, SIGIR 1998) [10] builds the list greedily:

```
next = argmax_{i not in S} [ lambda * rel(i) - (1 - lambda) * max_{j in S} sim(i, j) ]
```

MMR is O(N * K) similarity look-ups for K picks from N candidates. It is easy to run on the re-ranking tier and is still the default diversity baseline in many systems, including RAG retrieval today. Ziegler et al. [30] introduced **intra-list similarity** and topic diversification for book recommendations. User studies showed that users liked lists that were diversified even when accuracy metrics dropped slightly. That is an early piece of evidence that offline accuracy is the wrong only-metric.

Limitations of MMR: it only penalises the *maximum* pairwise similarity, it has no probabilistic meaning, and lambda must be tuned per surface.

14.4.2 Determinantal point processes

A DPP over a ground set of items defines P(S) proportional to det(L_S), where L is a positive semi-definite kernel. A common decomposition is

```
L_ij = q_i * q_j * S_ij
```

where q_i is item quality (relevance) and S_ij is similarity. The determinant grows with the qualities and shrinks as items become collinear, so one score covers both relevance and set-level diversity. It rewards whole-set "volume", not only pairwise distance.

YouTube: practical DPP for the home feed

Wilhelm et al., "Practical Diversified Recommendations on YouTube with Determinantal Point Processes" (CIKM 2018) [2], describe how YouTube put a DPP into its mobile home feed.

Problem. The pointwise ranker scores each video on its own, so a feed can fill up with near-duplicate content (for example, several videos on the same topic or from the same creator). Users then scroll past the whole block, and a pointwise ranker never sees that cost.

Architecture.

```
candidate generators --> pointwise ranker (q_i = predicted utility)
                                  |
                                  v  top-N ranked candidates (N ~ a few hundred, approx.)
                   +-------------------------------+
                   | DPP re-ranker                 |
                   |  kernel:                      |
                   |  L_ii = q_i^2                 |
                   |  L_ij = alpha*q_i*q_j*        |
                   |        exp(-D_ij / (2 s^2))   |
                   |  D_ij from video embeddings / |
                   |  topical distance             |
                   |  greedy MAP, window by window |
                   +-------------------------------+
                                  |
                                  v
                  feed served in windows of k items; next window
                  chosen conditioned on items already selected
```

Key tricks.

- **Parameterised kernel with two knobs.** alpha trades relevance against diversity, and sigma sets how fast similarity decays with distance. Early versions found the values by grid search against online metrics. Later the paper describes learning a deep kernel from feed interactions, treating the set of items the user engaged with in a window as a DPP sample.
- **Windowed greedy inference.** The feed is long. Instead of one global MAP, the system greedily picks a window of items (roughly the number visible on a phone screen), fixes it, and moves to the next window. This keeps latency bounded.
- **Engagement as the objective.** The paper reports that DPP-diversified feeds raised long-term user satisfaction and engagement metrics compared with heuristic diversity rules such as per-channel caps, which YouTube had used before.

Hulu: fast greedy MAP inference

Exact DPP MAP is NP-hard, and naive greedy costs O(M * N^3) for selecting N items from M if determinants are recomputed. Chen, Zhang and Zhou (Hulu, NeurIPS 2018) [3] made greedy DPP practical:

- **Incremental Cholesky updates.** Each time an item is added, the marginal gain of every remaining candidate is updated in O(N) work per candidate, giving O(N^2 * M) total. That is fast enough for online re-ranking at hundreds of candidates and tens of selected items.
- **Sliding-window variant.** For long sequences, diversity is enforced only within a window of the last w items. This matches how users perceive a scrolling feed and lowers cost further.
- Experiments showed that the method was orders of magnitude faster than earlier greedy implementations and gave a better relevance-diversity frontier than MMR and

other baselines. The paper also reports an online A/B test in a production movie-recommendation setting.

This algorithm (often called "fast greedy DPP") is the implementation most teams use in practice, including several large short-video and e-commerce platforms that describe DPP-style re-ranking.

14.4.3 Airbnb: learning to rank diversely

Airbnb's search team (Abdool et al., "Learning To Rank Diversely At Airbnb") [7] points out a basic gap in pairwise learning to rank. The loss assumes each listing's utility does not depend on what sits above it. In a marketplace, if the top 5 results are all similar high-priced lofts, a sixth loft adds little.

Design. The team keeps the trained pairwise relevance model and adds a model that scores each listing *in the context of the listings already ranked above it*. It learns a representation of the "query context of results so far" and ranks listing by listing (sequentially), so that a listing similar to those already placed is de-prioritised when the data shows that such redundancy lowers bookings. The paper presents this as moving from pointwise or pairwise to a listwise, context-aware formulation without hand-tuned diversity weights, and reports a statistically significant booking gain in online A/B tests.

Lesson. Diversity does not have to be a heuristic re-ranker bolted on after the model. It can be learned from outcome data once the model sees the slate context. This is the same direction taken by generative, slate-aware rankers (Chapter 6).

14.4.4 Choosing a diversity mechanism

Mechanism	Objective	Cost per request	Tunability	Learned?	Production users (public)
Hard rules (per-creator cap, category quota)	Constraint	O(N)	Very easy	No	Nearly everyone
MMR [10]	Relevance minus max-sim	O(NK)	1 knob	Rarely	Search, RAG, feeds
Fast greedy DPP [3]	log det of quality-similarity kernel	O(K^2 N)	alpha, sigma	Kernel can be learned [2]	YouTube [2], Hulu [3]
Calibration [4]	Match user's genre mix (KL)	O(NK * genres)	lambda	No	Netflix research
Listwise context model [7]	Outcome (bookings) given items above	Model inference per position	Implicit	Yes	Airbnb

---

14.5 Calibrated Recommendations (Netflix)

Problem. Harald Steck (Netflix, RecSys 2018) [4] gives a simple example. A user watched 70% romance and 30% action. An accuracy-optimised top-10 list may be 100% romance, because romance items score highest one by one. The list is accurate but *mis-calibrated*: the user's minority interest disappears. Over time the feedback loop can wipe that interest out of the profile altogether.

Formulation.

- p(g | u): the target distribution over genres g, computed from the user's watch history. Each item's genre mass is spread across its genres, and items can be weighted, for example by recency.
- q(g | u): the genre distribution of the recommended list.
- Mis-calibration is measured with KL divergence, C_KL(p, q) = sum_g p(g|u) log(p(g|u) / q~(g|u)). Here q~ = (1 - a) * q + a * p with a small a (the paper uses a value around 0.01) so that the divergence stays finite when the list misses a genre.

Re-ranking objective.

```
I* = argmax_{I, |I| = N} (1 - lambda) * sum_{i in I} s(u, i) - lambda * C_KL(p, q(I))
```

Steck notes that the objective is close to submodular. A greedy algorithm that adds the item with the largest marginal gain therefore has the usual (1 - 1/e) approximation guarantee and runs fast enough for post-processing. Experiments on MovieLens 20M show that a small lambda removes most of the mis-calibration at a modest accuracy cost. Lists from a standard model trained for accuracy came out *more* mis-calibrated for users with diverse tastes.

Why it matters for system design.

- Calibration is a *personalised* form of diversity. The target mix comes from the user, not from a global "be diverse" prior.
- The same machinery can calibrate along any categorical axis that matters to the product: content type (movies vs series), creator popularity bucket (popular vs emerging), language, or freshness. Many later "calibrated" papers do exactly that.
- It is a cheap re-ranker over the top few hundred candidates and needs no retraining.

14.6 Cold Start

Cold start has three cases with different solutions: **new users**, **new items**, and **new creators/sellers**. The last is item cold start plus a marketplace-health goal.

```
            +-------------------+      +------------------------+
new item -->| content encoders  |----> | ID embedding warm-up   |--> ranker
(meta,      | (text, image,     |      | (meta-embedding [25],  |
 media)     |  audio, creator)  |      |  DropoutNet [24])      |
            +-------------------+      +------------------------+
                      |                             ^
                      v                             |
           exploration traffic (bandits [32], boosted slots, priors [5])
                      |
                      v
           real-time training (minutes) -> ID embedding converges

new user --> onboarding signals + context (geo, device, referrer)
         --> cohort-specific gating (POSO [26]) --> popular-within-cohort fallback
```

14.6.1 Item cold start with content and dropout

DropoutNet (Volkovs et al., Layer 6 AI, NeurIPS 2017) [24] trains a network that takes both preference embeddings (from collaborative filtering) and content features as input. During training it **randomly drops out the preference input** for some users and items. The network is forced to rebuild the warm-start relevance score from content alone, so the same model works for warm, cold and partly cold entities with no separate cold model. This idea, input dropout on ID/CF features, carries over directly to modern two-tower and DLRM systems: drop ID embeddings for some examples so that content and side-feature towers learn to stand alone.

14.6.2 Meta-learned ID embeddings for new ads

Pan et al. (SIGIR 2019, Tencent-related ad data) [25] attack the new-ad problem in CTR models. A new ad's randomly initialised ID embedding is noise until it has gathered enough clicks. **Meta-Embedding** trains a generator that maps an ad's attributes (category, advertiser, and so on) to an initial ID embedding. It uses a two-phase, MAML-style objective: the generated embedding should (a) predict well in the cold phase and (b) adapt fast after a few gradient steps on early data. The paper reports better CTR prediction for new ads in both the cold and warm-up phases on large ad datasets. Production descendants appear across ad systems: initialise from content or neighbours, never from zero.

14.6.3 Airbnb: priors for new listings

The 2020 Airbnb paper [5] describes the cold-start issue for new listings. Engagement features such as historical booking rate are missing or noisy, so the DNN ranks new listings poorly, and fewer impressions mean even less data. Airbnb's fix was to **estimate the missing engagement features for new listings from similar nearby listings** (same geography, capacity and so on) instead of using defaults. The paper reports a large relative booking gain *for new listings* in online tests (reported as approximately double-digit percent) and a smaller but significant overall booking gain. Combined with the position debiasing above (14.2.4), this is a clear example of debiasing and cold start being the same marketplace problem.

14.6.4 New users: Kuaishou POSO

Kuaishou's POSO (Personalized Cold Start Modules, Dai et al., 2021) [26] identifies a structural problem. In a large shared ranking model, signals from new users are swamped by the majority of active users. Even when "is_new_user" is an input feature, the network mostly ignores it because it is one feature among hundreds. POSO adds **user-group-specific gating** (personalised modules) to existing layers such as MLP, MHA and MMoE. A small gating network driven by user-cohort features (new vs regular, activity level) rescales hidden units, so cohort differences shape the whole computation. Kuaishou reports gains in new-user engagement and retention metrics in production, at negligible serving cost.

Pattern. Give minority cohorts a *structural* path through the network (gates, separate experts, separate towers), not just an input feature.

14.6.5 Exploration for cold start: contextual bandits

When features cannot fill the gap, the system must *explore*. Li et al.'s **LinUCB** (Yahoo!, WWW 2010) [32] scored news articles with a linear model plus an upper-confidence bonus alpha * sqrt(x^T A^-1 x). New articles have high uncertainty, so they automatically get exploratory traffic. On the Yahoo! front page the paper reports a click lift (roughly 12.5%) over a context-free bandit, using offline replay evaluation on randomly logged traffic. Chapter 13 covers modern neural bandits and exploration budgets.

14.6.6 Real-time training as a cold-start tool

Infrastructure matters too. ByteDance's **Monolith** [33] (collisionless embedding tables with expiry, plus online training that syncs parameters to serving at minute-level granularity) shortens the time a new item or user spends with an untrained embedding from hours or days to minutes. For fast-churning catalogues such as short video or news, the freshness of the training loop is often the biggest cold-start lever (see Chapter 9).

14.6.7 Cold-start strategy matrix

Entity	Signal available	Primary technique	Secondary	Example
New item	Content, creator, metadata	Content tower / meta-embedding [25]	ID dropout [24], fast online training [33]	Ads CTR, short video
New user	Context, onboarding choices	Cohort gating [26], popular-in-cohort	Bandits [32], onboarding questionnaire	Kuaishou, news
New creator / seller	Content + creator profile	Boosted exploration slots	Priors from similar suppliers [5]	Airbnb hosts, marketplaces
New surface / market	Transfer from other surfaces	Shared embeddings, multi-domain models	Calibration to local mix [4]	Any multi-product platform

---

14.7 Fairness for Creators, Sellers and Users

In multi-sided platforms, "fairness" usually means **fair exposure for suppliers**: creators, artists, hosts, sellers and job candidates. User-side group fairness (for example, equal recommendation quality across demographic groups) is the other main form.

14.7.1 Fairness of exposure: the theory

Singh and Joachims (KDD 2018) [20] define fairness constraints on the *exposure* that ranking policies give to groups. Exposure is the position-weighted attention, using the same position-bias curves as 14.2. Examples are demographic parity of exposure and exposure proportional to merit (relevance). The optimal fair ranking policy is computed as a linear program over doubly stochastic matrices, then decomposed (Birkhoff-von Neumann) into a distribution over rankings. Position bias therefore drives both relevance debiasing *and* fairness: the same propensity curve sets how much exposure each slot is worth.

FA*IR (Zehlike et al.) [35] is a simpler, top-k ranking algorithm. It guarantees that every prefix of the ranking holds at least a minimum share of a protected group, checked with a binomial significance test, while staying as close as possible to the score order.

14.7.2 LinkedIn Recruiter: fairness-aware re-ranking at scale

Problem. In LinkedIn Talent Search, recruiters search for candidates. LinkedIn wanted the gender distribution of top results to match the gender distribution of the qualified candidate pool for that query, a "representative" ranking.

Design (Geyik, Ambler, Kenthapadi, KDD 2019) [21]:

```
query --> candidate retrieval --> ML ranker (relevance scores)
                                        |
                                        v
            desired distribution p_q(attribute) = distribution among candidates
            matching the query (e.g., qualified pool), computed per query
                                        |
                                        v
            fairness re-ranker: per-attribute queues sorted by score;
            fill ranks greedily so that every prefix k has at least
            floor(p_a * k) and at most ceil(p_a * k) of each attribute a
            (DetGreedy / DetCons / DetRelaxed variants)
                                        |
                                        v
                                top-k shown to recruiter
```

Key properties.

- **Post-processing, model-agnostic.** The ranker is untouched. That made the change easy to ship and audit.
- **Per-query targets.** The target is the attribute mix of the *qualified* candidates for that query, not a global quota.
- **Prefix constraints.** Fairness holds at every k the recruiter might look at, not only in aggregate.
- **Results.** The paper reports deployment to all LinkedIn Recruiter users worldwide and a large increase (reported as nearly three-fold) in the share of queries with representative results, with no significant drop in business metrics in A/B tests.

14.7.3 Spotify: relevance, fairness and satisfaction trade-offs

Mehrotra et al. (Spotify, CIKM 2018) [22] studied a music recommender as a two-sided marketplace. Artists differ widely in popularity, and a pure relevance policy pushes exposure to the head. Using **counterfactual (off-policy) evaluation** on logged data, they estimated user satisfaction under policies that mix relevance with a supplier-fairness score (exposure across artist popularity groups). Main findings:

- Pure fairness policies hurt satisfaction. Pure relevance policies concentrate exposure.
- Users differ in how open they are to less-popular artists. **Adaptive policies** that give fairness more weight for users with broader tastes reached better trade-offs than global interpolation.

This links fairness to calibration (14.5). Pushing exposure toward emerging suppliers works best when it follows each user's demonstrated appetite.

14.7.4 Google: pairwise fairness in production recommenders

Beutel et al. (Google, KDD 2019) [23] define **pairwise fairness** for recommenders. Among pairs where one item was engaged with and one was not, the model should rank the

engaged item higher with equal probability whether the item belongs to group A or group B (with intra-group and inter-group variants). Measuring this needs data where both items were actually exposed, which they get from randomised experiments. They add a **pairwise regulariser** that penalises the correlation between group membership and residual pairwise error. They report that, in a large-scale production recommender at Google, the regulariser greatly improved inter-group pairwise fairness with little engagement loss.

14.7.5 User-side fairness

Yao and Huang [36] define unfairness metrics for collaborative filtering across user groups, for example under- or over-estimating ratings for a minority group, and add them as regularisers. Abdollahpouri et al. [15] show that popularity bias is itself a user-fairness issue, because niche-taste users get worse service. In practice, user-side fairness is monitored by slicing evaluation metrics by cohort (Chapter 12) and fixed with structural cohort modelling such as POSO [26].

14.7.6 Fairness design options compared

Approach	Where	Guarantee	Cost	Examples
Prefix-constrained re-ranking	Post-ranking	Hard, per prefix	Low	LinkedIn [21], FA*IR [35]
Exposure LP / stochastic ranking	Post-ranking	Expected exposure	Medium (LP per query)	Singh and Joachims [20]
Regulariser in training	Model	Soft, metric-level	Training only	Google pairwise [23], Yao and Huang [36]
Adaptive mixing per user	Policy	Soft, user-aware	Low	Spotify [22]
Exploration boosts for new suppliers	Serving	Soft	Exploration budget	Marketplaces, creator platforms

---

14.8 Putting It Together: A Reference Corrective Stack

A typical large-scale feed or marketplace stack that combines the designs above:

```
[Training data]
  impressions + engagements + position + device + example age + duration
    |
    |-- reweight: fixed examples per user [12]; IPS for explicit ratings [13]
    v
[Retrieval: two-tower]
  in-batch softmax with logQ correction via streaming frequency estimator [11]
  ID dropout so content towers handle cold items [24]
  meta-embedding initialisation for new IDs [25]
    |
    v  ~ thousands of candidates
[Ranker: multitask DNN / MMoE]
  main towers (engagement, satisfaction)
  + shallow position/device tower, additive logit, position dropout [1][5]
  duration-bucketed quantile targets for watch time [27]
  cohort gates for new users [26]
  example_age = 0 and position = missing at serving [12][1]
    |
    v  ~ hundreds of candidates
[Re-ranking / slate construction]
  fast greedy DPP or MMR within sliding windows [2][3][10]
  calibration to user's genre / creator-popularity mix [4]
  fairness prefix constraints where policy requires [21]
  exploration slots for cold items/creators (bandit bonus) [32]
    |
    v
[Serving + logging]  log true positions, exploration flags, propensities
                     (needed by every debiasing method above)
```

Key Takeaways for System Designers

1. **Log what the debiasers need.** Position, device, surface, exploration flag, sampling probability and (for off-policy work) the serving policy's propensity must be in the impression logs. Without them almost none of this chapter is possible.
2. **Prefer factorised bias towers to raw IPS in industrial rankers.** YouTube's additive shallow tower [1], Huawei's multiplicative PAL [9] and Airbnb's position-feature-with- dropout [5] share one recipe: let a small component absorb the bias in training, then neutralise it at serving. Dropout on the bias feature is what makes neutralisation safe.
3. **Popularity bias hides in the training procedure.** In-batch negatives need logQ correction [11]. Training windows need an example-age feature [12]. Watch-time targets need duration normalisation [27].
4. **Treat diversity as a slate-level property.** Pointwise rankers cannot see redundancy. Use DPP or MMR with windowing for cheap gains [2][3]. Move to learned, context-aware listwise models once you can [7].
5. **Calibrate to the user, not to a global prior.** Netflix's calibration [4] and Spotify's adaptive fairness [22] both show that the best trade-off follows each user's demonstrated breadth of interest.
6. **Cold start is a structural problem, not just a missing feature.** Use content towers with ID dropout [24], meta-learned embedding initialisation [25], cohort-specific gating [26], neighbour-based priors [5], and fast online training [33]. Keep an explicit exploration budget [32].
7. **Fairness of exposure reuses the position-bias curve.** The same propensity model that debiases clicks tells you how much exposure each slot gives. Measure supplier exposure with it [20].

8. **Ship fairness as auditable post-processing first.** LinkedIn's per-query, prefix-constrained re-ranker [21] was model-agnostic, explainable and deployed globally with no measurable business loss.
9. **Evaluate counterfactually and by cohort.** Off-policy estimators [13][22][31] and cohort-sliced metrics (new users, niche users, emerging creators) catch problems that aggregate NDCG hides.
10. **Expect long-term rather than immediate wins.** Diversity, calibration and fair exposure often look neutral or slightly negative on short-term engagement. The reported gains in the YouTube DPP work [2] and the off-policy REINFORCE work [31] show up in satisfaction, retention and tail coverage. Design experiments long enough to measure them.

References

1. Zhe Zhao, Lichan Hong, Li Wei, Jilin Chen, Aniruddh Nath, Shawn Andrews, Aditee Kumthekar, Maheswaran Sathiamoorthy, Xinyang Yi, Ed Chi (Google/YouTube). "Recommending What Video to Watch Next: A Multitask Ranking System." RecSys 2019. https://doi.org/10.1145/3298689.3346997
2. Mark Wilhelm, Ajith Ramanathan, Alexander Bonomo, Sagar Jain, Ed H. Chi, Jennifer Gillenwater (Google/YouTube). "Practical Diversified Recommendations on YouTube with Determinantal Point Processes." CIKM 2018. https://doi.org/10.1145/3269206.3272018
3. Laming Chen, Guoxin Zhang, Hanning Zhou (Hulu). "Fast Greedy MAP Inference for Determinantal Point Process to Improve Recommendation Diversity." NeurIPS 2018. https://arxiv.org/abs/1709.05135
4. Harald Steck (Netflix). "Calibrated Recommendations." RecSys 2018. https://doi.org/10.1145/3240323.3240372
5. Malay Haldar et al. (Airbnb). "Improving Deep Learning For Airbnb Search." KDD 2020. https://arxiv.org/abs/2002.05515
6. Malay Haldar et al. (Airbnb). "Applying Deep Learning To Airbnb Search." KDD 2019. https://arxiv.org/abs/1810.09591
7. Mustafa Abdool et al. (Airbnb). "Learning To Rank Diversely At Airbnb." 2022 (CIKM 2023). https://arxiv.org/abs/2210.07774
8. Thorsten Joachims, Adith Swaminathan, Tobias Schnabel (Cornell). "Unbiased Learning-to-Rank with Biased Feedback." WSDM 2017. https://arxiv.org/abs/1608.04468
9. Huifeng Guo, Jinkai Yu, Qing Liu, Ruiming Tang, Yuzhou Zhang (Huawei). "PAL: A Position-bias Aware Learning Framework for CTR Prediction in Live Recommender Systems." RecSys 2019. https://doi.org/10.1145/3298689.3347033
10. Jaime Carbonell, Jade Goldstein (CMU). "The Use of MMR, Diversity-Based Reranking for Reordering Documents and Producing Summaries." SIGIR 1998. https://doi.org/10.1145/290941.291025
11. Xinyang Yi, Ji Yang, Lichan Hong, Derek Zhiyuan Cheng, Lukasz Heldt, Aditee Kumthekar, Zhe Zhao, Li Wei, Ed Chi (Google/YouTube). "Sampling-Bias-Corrected Neural Modeling for Large Corpus Item Recommendations." RecSys 2019. https://doi.org/10.1145/3298689.3346996

12. Paul Covington, Jay Adams, Emre Sargin (Google/YouTube). "Deep Neural Networks for YouTube Recommendations." RecSys 2016. https://static.googleusercontent.com/media/research.google.com/en//pubs/archive/45530.pdf
13. Tobias Schnabel, Adith Swaminathan, Ashudeep Singh, Navin Chandak, Thorsten Joachims. "Recommendations as Treatments: Debiasing Learning and Evaluation." ICML 2016. https://arxiv.org/abs/1602.05352
14. Dawen Liang, Laurent Charlin, James McInerney, David M. Blei. "Modeling User Exposure in Recommendation." WWW 2016. https://arxiv.org/abs/1510.07025
15. Himan Abdollahpouri, Masoud Mansoury, Robin Burke, Bamshad Mobasher. "The Unfairness of Popularity Bias in Recommendation." RMSE workshop at RecSys 2019. https://arxiv.org/abs/1907.13286
16. Himan Abdollahpouri, Robin Burke, Bamshad Mobasher. "Controlling Popularity Bias in Learning-to-Rank Recommendation." RecSys 2017. https://doi.org/10.1145/3109859.3109912
17. Yang Zhang, Fuli Feng, Xiangnan He, Tianxin Wei, Chonggang Song, Guohui Ling, Yongdong Zhang. "Causal Intervention for Leveraging Popularity Bias in Recommendation." SIGIR 2021. https://arxiv.org/abs/2105.06067
18. Tianxin Wei, Fuli Feng, Jiawei Chen, Ziwei Wu, Jinfeng Yi, Xiangnan He. "Model-Agnostic Counterfactual Reasoning for Eliminating Popularity Bias in Recommender System." KDD 2021. https://arxiv.org/abs/2010.15363
19. Jiawei Chen, Hande Dong, Xiang Wang, Fuli Feng, Meng Wang, Xiangnan He. "Bias and Debias in Recommender System: A Survey and Future Directions." ACM TOIS 2023 (arXiv 2020). https://arxiv.org/abs/2010.03240
20. Ashudeep Singh, Thorsten Joachims. "Fairness of Exposure in Rankings." KDD 2018. https://arxiv.org/abs/1802.07281
21. Sahin Cem Geyik, Stuart Ambler, Krishnaram Kenthapadi (LinkedIn). "Fairness-Aware Ranking in Search and Recommendation Systems with Application to LinkedIn Talent Search." KDD 2019. https://arxiv.org/abs/1905.01989
22. Rishabh Mehrotra, James McInerney, Hugues Bouchard, Mounia Lalmas, Fernando Diaz (Spotify). "Towards a Fair Marketplace: Counterfactual Evaluation of the trade-off between Relevance, Fairness and Satisfaction in Recommendation Systems." CIKM 2018. https://doi.org/10.1145/3269206.3272027
23. Alex Beutel, Jilin Chen, Tulsee Doshi, Hai Qian, Li Wei, Yi Wu, Lukasz Heldt, Zhe Zhao, Lichan Hong, Ed H. Chi, Cristos Goodrow (Google). "Fairness in Recommendation Ranking through Pairwise Comparisons." KDD 2019. https://arxiv.org/abs/1903.00780
24. Maksims Volkovs, Guangwei Yu, Tomi Poutanen (Layer 6 AI). "DropoutNet: Addressing Cold Start in Recommender Systems." NeurIPS 2017. https://papers.nips.cc/paper/7081-dropoutnet-addressing-cold-start-in-recommender-systems
25. Feiyang Pan, Shuokai Li, Xiang Ao, Pingzhong Tang, Qing He. "Warm Up Cold-start Advertisements: Improving CTR Predictions via Learning to Learn ID Embeddings." SIGIR 2019. https://arxiv.org/abs/1904.11547
26. Shangfeng Dai, Haobin Lin, Zhichen Zhao, Jianying Lin, Honghuan Wu, Zhe Wang, Sen Yang, Ji Liu (Kuaishou). "POSO: Personalized Cold Start Modules for Large-scale Recommender Systems." 2021. https://arxiv.org/abs/2108.04690
27. Ruohan Zhan et al. (Kuaishou). "Deconfounding Duration Bias in Watch-time Prediction for Video Recommendation." KDD 2022. https://arxiv.org/abs/2206.06003

28. Xuanhui Wang, Nadav Golbandi, Michael Bendersky, Donald Metzler, Marc Najork (Google). "Position Bias Estimation for Unbiased Learning to Rank in Personal Search." WSDM 2018. https://doi.org/10.1145/3159652.3159732
29. Aman Agarwal, Ivan Zaitsev, Xuanhui Wang, Cheng Li, Marc Najork, Thorsten Joachims. "Estimating Position Bias without Intrusive Interventions." WSDM 2019. https://arxiv.org/abs/1812.05161
30. Cai-Nicolas Ziegler, Sean M. McNee, Joseph A. Konstan, Georg Lausen. "Improving Recommendation Lists Through Topic Diversification." WWW 2005. https://doi.org/10.1145/1060745.1060754
31. Minmin Chen, Alex Beutel, Paul Covington, Sagar Jain, Francois Belletti, Ed H. Chi (Google/YouTube). "Top-K Off-Policy Correction for a REINFORCE Recommender System." WSDM 2019. https://arxiv.org/abs/1812.02353
32. Lihong Li, Wei Chu, John Langford, Robert E. Schapire (Yahoo! Research). "A Contextual-Bandit Approach to Personalized News Article Recommendation." WWW 2010. https://arxiv.org/abs/1003.0146
33. Zhuoran Liu et al. (ByteDance). "Monolith: Real Time Recommendation System With Collisionless Embedding Table." 2022. https://arxiv.org/abs/2209.07663
34. Xuanhui Wang, Michael Bendersky, Donald Metzler, Marc Najork (Google). "Learning to Rank with Selection Bias in Personal Search." SIGIR 2016. https://doi.org/10.1145/2911451.2911537
35. Meike Zehlike, Francesco Bonchi, Carlos Castillo, Sara Hajian, Mohamed Megahed, Ricardo Baeza-Yates. "FA*IR: A Fair Top-k Ranking Algorithm." CIKM 2017. https://arxiv.org/abs/1706.06368
36. Sirui Yao, Bert Huang. "Beyond Parity: Fairness Objectives for Collaborative Filtering." NeurIPS 2017. https://arxiv.org/abs/1705.08804

15. Re-ranking, Blending and Whole-Page Optimization

The last stage of almost every industrial recommender gets the least attention in papers and holds the most product policy in code. After retrieval, pre-ranking and a heavy multi-task ranker have assigned each candidate a score, something still has to turn a scored set into the thing a person sees. That might be a feed of posts mixed with ads and modules, a Netflix homepage of rows, a Taobao result page or a Kuaishou session of six videos. This chapter covers that "last mile":

- **Listwise and slate re-ranking.** Models that score an item in the context of the other items on the list. Examples are DLCM, PRM (Alibaba), SetRank and PEAR, plus generator-evaluator and generative rerankers such as Seq2Slate, GRN/PRS, PIER and NAR4Rec.
- **Diversification.** MMR, determinantal point processes (Hulu, YouTube, X), author decay, calibration.
- **Heterogeneous feed blending.** Mixing content types, sources and ads into one stream (LinkedIn, Pinterest, X/Twitter, Meta).
- **Page-level optimization.** Choosing and ordering rows and shelves (Netflix, Spotify, Deezer, Amazon).
- **The business-rules and constraints layer**, which every published design keeps even as models get larger.

A note on sourcing. The live web-search and fetch budget for this chapter was used up before research could start, and outbound access to arXiv and company blogs was blocked from the writing environment. The chapter therefore relies on two kinds of source. The first is primary sources already fetched and verified by other chapters of this Atlas, such as Meta, X, Pinterest, LinkedIn, Netflix and Kuaishou posts and repositories. The second is a small set of canonical, widely cited papers whose identifiers are well established. Where a number could not be checked again, it is marked approximate. Readers should treat specific online-lift figures as indicative only.

---

1. Why a separate re-ranking stage exists

A pointwise ranker estimates something like P(click | user, item) for each candidate on its own. Then it sorts. That rests on three assumptions that fail in real products:

1. **Independence.** An item's utility does not depend on its neighbours. In reality, the fifth phone case in a row is worth less than the first. A cheap item next to an expensive one changes how both are perceived. Alibaba's work on "mutual influence aware ranking" in Taobao search was motivated directly by this [10].
2. **Single objective.** One score captures everything. In practice the page must also satisfy diversity, freshness, creator exposure, ad revenue, integrity and legal constraints [1][14][15].

3. **Uniform positions.** Every slot is equal apart from a position-bias discount. Real surfaces are two-dimensional (Netflix rows, Spotify shelves) or have strong cascade behaviour (carousels where only the first few cards are seen) [23][24][25].

Google's public recommender course names the three-box pipeline candidate generation, scoring and **re-ranking**. It assigns freshness, diversity and fairness to the last box [2]. Eugene Yan's system-design synthesis likewise ends the online path with a ranking step followed by business logic and filtering [3]. Every public funnel in Chapter 1 has some final pass: Instagram's "final reranking" with integrity, diversity and business rules [14], Facebook Feed's final pass with integrity demotions and diversity [16], X's Home Mixer heuristics [17][18], and Pinterest's Multi-Objective Blender [20].

1.1 Canonical last-mile architecture

```
heavy ranker output: ~10^1-10^3 candidates, each with p_click, p_like, p_hide, ...
       |
       v
+--------------------+   value model: score = sum w_i * p_i (+ offsets)
| 1. score fusion    |   (Ch. 4: Meta value model, X weights, Kuaishou ensemble sort)
+---------+----------+
          v
+--------------------+   hard filters: integrity, blocked/muted, seen, legal,
| 2. eligibility     |   inventory, geo, age, dedup by item / cluster
+---------+----------+
          v
+--------------------+   context-aware rescoring: DLCM / PRM / SetRank / PEAR
| 3. listwise rerank |   or generator + evaluator (Seq2Slate, GRN, PIER, NAR4Rec)
+---------+----------+
          v
+--------------------+   MMR / DPP / author decay / calibration / cold-start lift
| 4. diversification |
+---------+----------+
          v
+--------------------+   interleave content types and sources, insert ads at
| 5. blending / slots|   allowed positions, modules (WTF, stories, carousels)
+---------+----------+
          v
+--------------------+   rows / shelves / modules chosen and ordered;
| 6. page assembly   |   cross-row dedup; device templates
+---------+----------+
          v
    response + impression log (slate, positions, propensities)
```

In real systems these boxes are often merged. X folds diversity, out-of-network discounts and cold-start lifts into one scoring pass before a DPP reranker [18][19]. Netflix folds row selection and title placement into "page generation" [23][24]. The logical split is still useful when designing one.

---

2. Rule-based and heuristic re-ranking

2.1 Maximal Marginal Relevance (MMR)

MMR (Carbonell and Goldstein, SIGIR 1998) is the ancestor of every greedy diversifier. Items are picked one at a time. Each step takes the candidate that maximizes lambda * rel(i) - (1 - lambda) * max_{j in S} sim(i, j), where S is the set already selected [4]. It costs O(N*K) similarity evaluations, needs no training and has one knob. Its weakness is that it only penalizes the nearest already-picked item, not the set as a whole.

2.2 Determinantal point processes (Hulu, YouTube, X)

A DPP scores a set S by det(L_S), where the kernel L_ij = q_i * sim(i, j) * q_j combines per-item quality q_i with pairwise similarity. The determinant is large when items are both good and mutually dissimilar.

- **Hulu (NeurIPS 2018).** Chen, Zhang and Zhou made greedy MAP inference for DPPs practical. They update a Cholesky factor incrementally, so selecting K items from N costs O(N*K^2) instead of a fresh determinant at every step. They also add a sliding-window variant, so diversity is enforced only among nearby positions in a long feed [5]. That window idea matters in feeds, where a user cares that adjacent items differ, not that item 3 differs from item 300.
- **YouTube (CIKM 2018).** Wilhelm et al. describe deploying a DPP on YouTube's mobile home feed. They learn the kernel from user interactions, parametrizing quality from the existing ranker and similarity from embeddings plus a few learned parameters. They apply it in windows over the feed. The paper reports gains in user satisfaction metrics from the deployment. Exact lift figures are not repeated here because they could not be checked again [6].
- **X (2026).** In xAI's open-sourced x-algorithm, a separate `vm-ranker` step reorders the scored candidates with a DPP over embeddings after value-model scoring. It gives up a little score for less similarity between neighbouring posts [18][19].

2.3 Multiplicative decays and lifts (X / Twitter)

The X value-model code is the most detailed public example of a heuristic reranking layer [18][19]:

```
weighted sum of ~25 heads  -> negative-score offset (keeps scores > 0)
   -> + exploration bonus
   -> x author-diversity multiplier: (1 - floor) * decay^k + floor
          (k = number of higher-ranked posts already chosen from that author)
   -> x out-of-network factor (< 1 for unfollowed accounts)
   -> cold-start lift (new author raised to at least a target rank)
   -> DPP reorder (vm-ranker) -> visibility filters -> timeline
```

Two details are worth copying. The **negative-score offset** maps negative raw scores into a small positive band [0, OFFSET) before any multiplier runs. A multiplier below 1 therefore cannot "promote" a negatively scored item by pulling it towards zero. And the **author decay is sequential**: it depends on how many posts by that author already sit above the candidate, so it has to be applied greedily in rank order, not as a single vectorized pass. The 2023 Twitter Home Mixer README lists the same family of heuristics: author diversity, content balance between in-network and out-of-network, feedback fatigue, de-duplication and social proof. Ads, Who-to-Follow and other modules are mixed in afterwards [17].

2.4 Calibration as a reranking objective (Netflix)

Steck's "Calibrated Recommendations" (Netflix, RecSys 2018) re-ranks a list so that its genre distribution q(g) matches the user's historical distribution p(g). It measures miscalibration with a KL divergence C_KL(p, q) and greedily maximizes (1 - lambda) * relevance - lambda * C_KL [7]. The motivation is that accuracy-optimized lists crowd out a user's minority interests. Someone who watches 70% romance and 30% action can get a list

that is 100% romance. Calibration is a *distributional* constraint, not a pairwise similarity penalty, and it is cheap (greedy, submodular-style).

2.5 Round-robin blending (Pinterest)

Pinterest's early home feed ranked each content type with its own specialized model, then "blended with a ratio-based round robin" to produce the final feed [21]. A round robin with fixed ratios is the simplest heterogeneous blender. It guarantees a share of each type, but it cannot trade a mediocre item of type A against an excellent item of type B. That limit motivated Pinterest's later Multi-Objective Blender inside the ranking service [20].

---

3. Listwise contextual re-rankers

These models take the top-K list from the ranker (K is typically tens to low hundreds) and re-score every item *given the whole list*. Output is still a score per item, so serving stays "sort by score". This keeps them compatible with existing infrastructure.

3.1 DLCM: a recurrent list encoder (2018)

Ai et al.'s Deep Listwise Context Model (SIGIR 2018) feeds the top-K documents from a base ranker, in reverse rank order, into a GRU. The final hidden state is a "local context" vector. An attention-style scoring function then re-scores each item against that context. Training uses a listwise attention-rank loss [8]. It established the pattern: **base ranker, then list encoder, then re-scored list**. Its weakness is that the RNN is order-sensitive and processes items sequentially, so distant items interact only weakly.

3.2 Groupwise scoring functions (GSF)

Ai et al. (ICTIR 2019) score small groups of documents jointly with a DNN and aggregate the group scores per document. This makes "multivariate scoring" explicit without a full sequence model [9]. It shows that cross-item features help, but the number of groups grows combinatorially, which pushed later work towards attention.

3.3 Mutual-influence-aware ranking at Taobao (MIDNN)

Zhuang et al. (Alibaba, IJCAI 2018) formalize "globally optimized mutual influence aware ranking" for Taobao search. They add list-level features to each item, such as how an item's price compares with the rest of the list, and train a DNN on them. The final permutation is chosen with beam search under an evaluator [10]. This is an early industrial instance of the **generator-evaluator** pattern described in Section 4.

3.4 PRM: Personalized Re-ranking Model (Alibaba, RecSys 2019)

PRM, by Pei et al., is the most-cited industrial reranker and the template for most later work [11].

Problem. The initial list in Taobao recommendation is ranked pointwise. Item-item interactions on the list, and how those interactions differ per user, are ignored. A user who

cares about price reacts differently to a list of similar items than a user who cares about brand.

Architecture.

```
initial list from ranker: items i_1 .. i_n  (n = list length, tens of items)
      |
      v
per item input  x_i = [ item feature vector  ||  PV_i ]  +  PE_i
      |               PV_i = personalized vector: hidden layer of a pre-trained
      |                      CTR model taking (user, item) -> user-specific item rep
      |               PE_i = learnable position embedding (initial rank)
      v
linear projection to d_model
      v
N x Transformer encoder blocks (multi-head self-attention + FFN)
      |   every item attends to every other item -> O(n^2) but n is small
      v
linear + softmax over the list  -> re-ranked scores
      v
sort -> final list
```

Key ideas.

- *Self-attention instead of RNN.* Any two items interact directly, whatever their distance in the list. Attention is also parallel across positions, which matters for serving latency.
- *Personalized vector (PV).* Rather than feed raw user features into the reranker, PRM pre-trains a pointwise CTR model and uses its hidden-layer output for each (user, item) pair as a per-item personalized embedding. This puts user-specific item representations into the reranker cheaply.
- *Position embedding of the initial rank.* The ranker's ordering is itself a strong feature. Encoding it lets PRM learn when to keep and when to override the ranker.
- *Training objective.* A list-level softmax cross-entropy over clicked items in the displayed list.

Results and deployment. The paper reports offline gains over DLCM and other baselines on a public learning-to-rank dataset and on a Taobao e-commerce recommendation dataset. It reports online A/B gains in page views, clicks and GMV on Taobao's recommendation surface. The exact percentages are not repeated here because they could not be checked again [11]. The authors also released the e-commerce re-ranking dataset.

Lessons. PRM established three conventions most later rerankers keep: (1) re-rank a short list (tens of items) so quadratic attention is affordable; (2) inject personalization through embeddings from upstream models instead of retraining a big user tower; and (3) keep the original rank as an input feature.

3.5 SetRank and permutation invariance

Pang et al.'s SetRank (SIGIR 2020) argues that a reranker should be *permutation-invariant* with respect to its input. The result should not depend on the order in which the base ranker happened to present items. It uses stacked induced multi-head self-attention blocks without positional encodings (or with optional rank encodings) [12]. In practice industrial systems still include the initial rank as a feature, as PRM does, because it carries useful information. SetRank shows that it should be a *feature*, not a structural assumption.

3.6 PEAR: contextualized transformer with history (Huawei, WWW 2022)

PEAR (Li et al.) extends PRM-style models in two ways. It models both feature-level and item-level interactions inside the list. It also lets the list items attend to the user's *historical* interactions, so the reranker sees context from the user's past as well as from the current list. It adds an auxiliary list-level classification task that predicts whether the user engages with the list as a whole [13]. This makes the reranker partly a sequence model. The cost is more attention compute per request.

3.7 The evaluator problem

Every listwise model in this section scores items given the *initial* ordering. But once the reranker moves items, the context it scored them in no longer exists. Re-scoring after re-ranking is circular. This "evaluation-before-reranking" gap is what motivated the generator-evaluator and generative approaches in Section 4.

---

4. Generative and generator-evaluator re-ranking

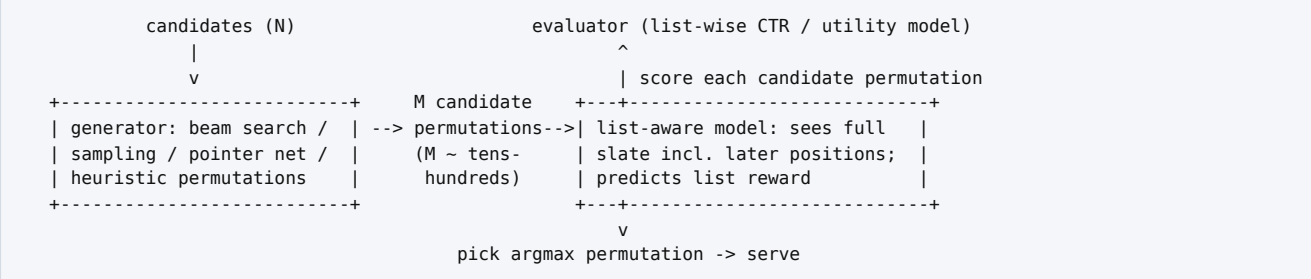
4.1 Seq2Slate (Google, 2018/2019)

Bello et al.'s Seq2Slate frames re-ranking as sequence generation with a **pointer network**. An encoder (RNN in the original) reads the candidate set. A decoder emits one candidate per step, conditioned on what it has already placed, by attending over the inputs [27]. Because each step sees the items already chosen, the model captures "what is good *next* given what is above", which a scoring model cannot do. Training uses click logs. The paper proposes supervised listwise losses that treat clicked items as targets to be placed early, as well as a REINFORCE-style policy-gradient variant. It reports offline gains on learning-to-rank benchmarks and an evaluation on data from a large-scale commercial recommender at Google [27].

Serving cost is K sequential decoding steps over N candidates. That is acceptable for K of around 10-20 and N in the low hundreds, but it is the main obstacle to generative reranking at feed scale.

4.2 Generator-evaluator: Exact-K, PRS, GRN

Several Alibaba and Meituan papers split the problem in two:



- **Exact-K (Alibaba, KDD 2019).** Gong et al. pose "exact-K recommendation", choosing a set of exactly K items that is jointly good, as maximal-clique optimization

on a graph of candidate items. They use a graph-attention encoder plus a pointer-network decoder trained with RL, with a reward from a learned evaluator [28].
- **PRS: "Revisit Recommender System in the Permutation Prospective" (2021).** This paper separates a Permutation-Matching stage, which generates candidate permutations efficiently with beam search, from a Permutation-Ranking stage. The second stage scores each whole permutation with a list-wise evaluator. It reports online gains in a large production feed. The authors are at Alibaba/Taobao; the exact attribution and numbers were not re-checked [29].
- **GRN: Generative Rerank Network (2021).** Feng et al. train a context-wise *evaluator* first, then train a pointer-network *generator* with policy gradient, using the evaluator's predicted list reward as the reward signal [30].

The evaluator is trained on logged slates, so it sees each item in the context it was actually shown in. It is therefore a more faithful judge of a permutation than any pointwise score. The generator only has to propose good candidates.

4.3 PIER (Meituan, WSDM 2023)

PIER (Permutation-level Interest-based End-to-end Re-ranking) addresses the cost of the generator-evaluator pattern at Meituan's scale. A Fine-grained Permutation Selection Module uses SimHash-style hashing of user interest to pick the top permutations cheaply from a very large permutation space. An Omnidirectional Context-aware Prediction Module then scores those permutations. The two modules are trained end to end, instead of with a frozen evaluator [31]. The lesson is that permutation search has to be approximated aggressively. Hashing and retrieval-style tricks from candidate generation carry over to the permutation space.

4.4 NAR4Rec: non-autoregressive generation (Kuaishou, KDD 2024)

Autoregressive generators decode one position at a time, which adds latency in proportion to slate length. Kuaishou's NAR4Rec generates all positions in parallel with a non-autoregressive model. It adds techniques for the sparse-data and position-dependency problems that come with that choice: a matching model for training and contrastive decoding to avoid repeated items. The paper reports deployment in Kuaishou's short-video app, which has hundreds of millions of daily users, with gains in engagement metrics. Specific figures are approximate and were not re-checked [32].

4.5 The generative endpoint: session-wise generation (Kuaishou OneRec)

Kuaishou's OneRec (2025) goes furthest. A single generative model produces a whole *session* of videos, removing the retrieve/rank/rerank cascade for that path. It is then aligned with a reward model via preference optimization [33][34]. In effect the reranker's job of choosing a jointly good list moves into the generator, and the reranker's evaluator becomes a reward model. Chapter 6 covers OneRec in depth. The relevant point here is that even OneRec's reports keep filtering and business-rule logic outside the model.

---

5. Slate RL and long-horizon reranking

Pointwise and listwise rerankers optimize immediate list reward. Two lines of work extend this over time.

- **SlateQ (Google, IJCAI 2019).** Under a single-choice assumption (the user picks at most one item from the slate) and the assumption that rewards and transitions depend only on the item picked, the long-term value of a slate decomposes into item-level Q-values weighted by a user choice model. The slate can then be optimized exactly with a linear program for the conditional-logit choice model, or approximately with greedy top-k. SlateQ was tested in live YouTube experiments by replacing the myopic per-item score with the decomposed long-term value [35].
- **Page-wise RL (JD.com, RecSys 2018).** Zhao et al. treat a whole 2-D page of items as the action. An actor-critic uses a deconvolutional actor to output the page layout, and a critic scores the page given user state and feedback on earlier pages [36].
- **Slate off-policy evaluation (Microsoft, NeurIPS 2017).** Swaminathan et al.'s pseudo-inverse estimator evaluates new slate policies from logged data, assuming the slate reward is additive over (item, position) pairs. Its sample complexity is far lower than naive inverse propensity scoring over the combinatorial slate space [37]. It is the standard tool for deciding whether a new reranker is worth an A/B slot.

Chapter 13 discusses these methods at length. For reranking, the main lesson is that **logging propensities for whole slates** is an architectural prerequisite. Without it, neither OPE nor slate RL works.

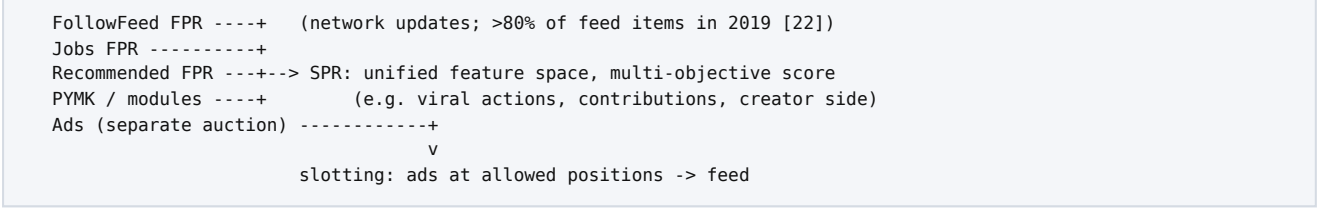
---

6. Heterogeneous feed blending

A single surface often shows several kinds of thing at once. Feed posts, jobs, ads, people-you-may-know cards, stories trays and video carousels come from separate teams with separately trained, separately calibrated models. Blending is the problem of putting them on one comparable scale and one layout.

6.1 LinkedIn: first-pass rankers and a second-pass ranker

LinkedIn's 2019 feed architecture is federated. Multiple **first-pass rankers (FPRs)** each select and score candidates from their own inventory (network updates, jobs, recommended content and so on). A **second-pass ranker (SPR)** merges their outputs into one ranked list. FollowFeed, which serves updates from a member's network, was the dominant FPR, supplying more than 80% of feed updates [22].



The design matches how the organization is split, since each inventory team owns its FPR. But it pushes **cross-source calibration** onto the SPR. If one FPR's scores drift, its share of the feed drifts too. LinkedIn tuned the SPR's multi-objective weights with automated online search instead of by hand (secondary coverage) [38]. The 2019 post's "community-focused" work also changed the objective to account for value to creators and the poster's community, not only to the viewer [22].

6.2 Pinterest: from round-robin to a multi-objective blender

As noted in Section 2.5, Pinterest first blended type-specific rankers with a fixed-ratio round robin [21]. Its WWW 2022 home-feed funnel puts a **Multi-Objective Blender** after the Utility Prediction Scorer inside the Ranking Service [20]. Later work personalizes the utility weights per request: PRL-PUTS, a one-step value-based RL policy that picks a weight vector per request and exposes an empirical Pareto frontier for governance. It reports +0.13% successful sessions online with no added serving latency [39]. Moving from ratio rules, to a single learned utility, to per-request weights is the typical path for a blender.

6.3 X / Twitter: Home Mixer and modules

X's Home Mixer is built on Product Mixer, a framework of pipelines. A product pipeline selects mixer pipelines, which run candidate pipelines, and the mixer interleaves ranked tweets with ads and modules such as Who-to-Follow [17][40]. The 2026 x-algorithm describes the same orchestration as reusable Rust traits named Source, Hydrator, Filter, Scorer, Selector and SideEffect [18]. That vocabulary maps neatly onto the boxes in Section 1.1. The in-network/out-of-network balance, about 50/50 on average in 2023, is a blending decision made through an out-of-network multiplier, not a fixed quota [17][19].

6.4 Meta: Facebook Feed and Instagram

Meta's 2021 Feed post describes ranking in passes. Integrity processes run first. A lightweight model narrows more than 1,000 candidate posts per user to about 500. Multitask neural networks predict several actions. Those predictions are combined into a single value. A final pass applies contextual rules such as diversity, so that a user does not see several posts from the same source in a row [16]. Instagram Explore's 2023 architecture likewise ends with a final reranking stage applying integrity, diversity and business rules after a linear value model over MTML predictions [14]. The 2020 Instagram post explains why some rules live as *constraints*: suggested content in Explore had to pass integrity and quality bars before being eligible. A log-linear value model lets a negative-feedback prediction act as a near-veto [15].

Meta's public posts do not describe in detail how ads are slotted into organic feeds. The architectural pattern visible across the industry is that ads go through their own retrieval, ranking and auction (Chapter 1 summarizes Andromeda and GEM) and are then **inserted** into the organic stream at allowed positions. Insertion is governed by ad-load and spacing constraints, not by direct score comparison with organic items.

6.5 Ads and organic: the allocation problem

A pattern consistent with public designs and with academic literature on feed ad allocation:

```
organic ranked list O = [o1, o2, ...]       ad list A = [a1, a2, ...] (auctioned)
               \                                    /
                v                                  v
          +-------------------------------------------------+
          | allocator                                       |
          |  - candidate slots: fixed (e.g. every k-th) or  |
          |    dynamic                                      |
          |  - constraints: min gap between ads, max ad load|
          |    per session, no ad in slot 1, category rules |
          |  - objective: revenue(a, pos) + lambda *        |
          |    engagement_loss(pos)                         |
          |  - solver: greedy / DP over slots / RL policy   |
          +-------------------------------------------------+
                                v
                    final feed + ad-load telemetry
```

The key design decisions are:

1. **Shared currency.** Organic value (predicted engagement) and ad value (expected revenue) are in different units. Exchange rates (lambda) are set as policy and tuned online against long-term retention, not learned end to end.
2. **Fixed vs dynamic slots.** Fixed slots are predictable and easy to audit. Dynamic slots let a high-value ad move up, or an irrelevant one move down or out, per user and session.
3. **Externalities.** An ad's cost is lost engagement on the organic items it displaces and any session shortening it causes. That makes this a page-level problem, not an item-level one.

Several Chinese platforms (Meituan, ByteDance, Kuaishou) have published RL-based ad-allocation papers with this structure. Because none of them could be checked again for this chapter, they are not cited individually.

---

7. Whole-page optimization

7.1 Netflix: page generation

Netflix's homepage is the canonical two-dimensional recommendation surface. Gomez-Uribe and Hunt (ACM TMIS 2015) describe it as a grid of rows. A typical page had about 40 rows, each with up to about 75 titles (device-dependent). Each row is produced by a different algorithm: the Personalized Video Ranker (PVR) for genre rows, a Top-N ranker for "Top Picks", trending, continue-watching, and video-video similarity for "Because you watched" rows. A **page generation** algorithm selects and orders the rows [23]. The paper notes that a member typically loses interest after about 60-90 seconds of choosing, having looked at perhaps 10-20 titles, which is why page composition matters so much. It also estimates that recommendation and personalization saved Netflix more than 1 billion US dollars a year in reduced churn (approximate, as stated in the paper) [23].

Netflix's 2015 tech-blog post "Learning a Personalized Homepage" (Alvino and Basilico) describes the page-generation problem in more detail [24]:

```
candidate rows: genre rows (PVR), Top Picks, Because-You-Watched, Trending,
                Continue Watching, New Releases, ...  (each row ranks its own titles)
      |
      v
row scoring  (relevance of row to member, given context/device)
      |
      v
stage-wise greedy page construction:
   for slot in 1..R:
      pick row maximizing  score(row | rows already placed)
        - penalize duplicate titles across rows (dedup / diversity)
        - enforce constraints (e.g. Continue Watching near top, device limits,
          min/max of row types)
      |
      v
render: rows top-to-bottom, titles left-to-right (2-D attention)
```

Points from that post:

- **Template vs row-scoring approaches.** A fixed template ("row type X at position Y") is simple but not personal. Scoring rows independently and sorting gives relevance but not diversity. Rows end up redundant and repeat the same titles.
- **Stage-wise (greedy) construction.** The page is built one row at a time. Each candidate row is scored given the rows already chosen, which naturally captures diversity and dedup. It is the row-level analogue of MMR.
- **Page-level metrics.** Because the user scans in two dimensions, Netflix evaluates pages with 2-D extensions of ranking metrics, for example a recall over the top-m rows by top-n columns that a user is likely to see.
- **Balance.** The page has to balance accuracy, diversity, discovery and freshness against constraints that differ by device.

Netflix's later posts add more page-level signals. Artwork is personalized per member with contextual bandits, and the choice of image interacts with the rest of the page [25]. Ranking has been consolidated into fewer, multi-task models serving many rows and canvases [41]. Objectives have moved towards long-term member satisfaction [42]. Netflix also uses interleaving to compare *row* rankers. It cannot measure page-composition effects, so those still need A/B tests [43].

7.2 Spotify home: shelves with explanations

Spotify's BaRT (RecSys 2018) is the bandit system behind the home page, a grid of **shelves** of playlists and albums. Each recommendation can carry an explanation ("Because you listened to X"). BaRT treats the (item, explanation) pair as the action and learns which combination to show in each shelf position. It uses counterfactual training from logged propensities [26]. It is whole-page optimization at the shelf level. The reranker chooses *how a recommendation is framed* as well as which item to show.

7.3 Deezer: carousels and the visibility model

Deezer models its playlist carousel as a contextual **cascade bandit**. Only the first L cards are visible without swiping, and cards beyond the last one the user would have seen get no feedback, so they are neither penalized nor rewarded. Users are clustered so that bandits learn per cluster (semi-personalization) [44]. The lesson for reranking: the **click model of**

the surface has to be part of the objective, otherwise the system learns to avoid positions that were never seen.

7.4 Amazon: combinatorial page layouts

Hill et al. (Amazon, KDD 2017) optimize a page of several widgets with many possible values each, giving a combinatorial space of layouts. They use a Bayesian linear model with pairwise interaction terms, Thompson sampling, and greedy hill climbing to approximately maximize the sampled model in real time. They report about a 21% conversion improvement over the median layout in a live experiment [45]. The pattern is to **sample a model, then use a cheap combinatorial optimizer**. It carries over directly to row ordering and module placement.

7.5 Airbnb: diversity in search ranking

Airbnb's "Learning To Rank Diversely" (2022) addresses a whole-page effect in search. Listings ranked purely by booking probability can become homogeneous, for example all at the same price point or in the same neighbourhood. Users who would book something different then find nothing. The paper changes the training and inference of the neural ranker so that each listing's score depends on what was ranked above it, which is a contextual "similarity to listings above" mechanism. It reports a booking gain in online experiments [46].

---

8. The business-rules and constraints layer

Every public design keeps an explicit, non-learned layer: Netflix, Meta, X, Pinterest, LinkedIn, Kuaishou, even OneRec. Google's "Rules of Machine Learning" recommends keeping heuristics and filtering explicit and outside the model where they are policy rather than prediction [47].

8.1 What lives there

Category	Examples	Hard / soft	Typical implementation
Eligibility	blocked/muted, already seen, out of stock, geo/licensing, age gates	hard	filter before or after rerank [16][17]
Integrity	policy-violating, borderline, low-quality	hard or demotion	integrity models run first; demotion multipliers [14][16]
Dedup	same item, near-duplicate media, same cluster, cross-row duplicates	hard	hash / cluster-id sets; page-level dedup [17][24]
Diversity	author/source caps, category spread, content balance	soft	decay multipliers, DPP, MMR [5][18][19]
Freshness / fatigue	recency boosts, "seen too often" penalties	soft	multipliers, impression-count features [17]
Exposure	cold-start creators, new items, exploration	soft	minimum rank, bonuses, reserved slots [18][19]
Mix / quotas	in-network vs out-of-network, content-type ratios	soft or hard	multipliers or ratio round robin [19][21]

Category	Examples	Hard / soft	Typical implementation
Monetization	ad load, ad spacing, no ad in slot 1	hard	allocator constraints (Section 6.5)
Layout	row count, row-type limits, device templates, pinned rows	hard	page-generation constraints [23][24]

8.2 Design patterns

```
config store (versioned, A/B-able)
     |  rules as data: {id, scope(surface, country), type(filter|cap|boost|slot),
     |                  params, priority, experiment_id}
     v
+-------------------------------------------------------------------------+
| rule engine in the mixer                                                |
|  1. hard filters (set operations; cheap; run early AND re-run at end)   |
|  2. score adjustments (multipliers/offsets; order-preserving sign care) |
|  3. sequential constraints (greedy fill: author cap, gap rules, DPP)    |
|  4. slot constraints (ads, modules, pinned rows)                        |
|  5. backfill if constraints leave holes (fallback sources)              |
+-------------------------------------------------------------------------+
     |
     v  log: which rule touched which item (for debugging and OPE)
```

Practical lessons from public sources:

- **Separate prediction from policy.** X ships its weights and multipliers as configuration (`param.rs`) separate from the model, so policy changes need no retraining [18][19]. Meta tunes value-model weights to trade off online metrics [14].
- **Watch the sign and scale of scores.** Multiplicative rules assume positive, calibrated scores. X's negative-score offset exists because a multiplier below 1 can raise a negative score [19]. Chapter 4 explains why calibration is the contract between the model and this layer.
- **Run filters twice.** Cheap eligibility filters run before expensive models to save compute (Meta runs integrity first [16]). They run again at the end, because later stages such as blending and backfill can reintroduce items.
- **Make rules visible to learning.** Rules change the logged distribution. If the reranker or ranker is trained on post-rule impressions without knowing which rule acted, it learns the rules as if they were user preferences. Logging rule decisions and propensities fixes this [37].
- **Rules accumulate.** The usual failure is hundreds of interacting boosts that nobody can explain. Treat rules as code: give each one an owner and an experiment, and expire it.

---

9. Comparison of re-ranking designs

Design	Org / year	Input size (approx.)	Mechanism	Captures	Serving cost	Production evidence
MMR [4]	CMU, 1998	any	greedy relevance - max sim	pairwise redundancy	O(NK) sims	widely used heuristic
Fast greedy DPP [5]	Hulu, 2018	10^2-10^3	incremental Cholesky MAP, windowed	set diversity	O(NK^2)	Hulu experiments

Design	Org / year	Input size (approx.)	Mechanism	Captures	Serving cost	Production evidence
DPP on feed [6]	YouTube, 2018	feed windows	learned kernel, windowed	set diversity	low	live YouTube home
X value model + DPP [18][19]	X, 2026	~10^3	multipliers, author decay, DPP	policy, diversity	low	open-sourced prod code
Calibration [7]	Netflix, 2018	10^2	greedy KL to user genre mix	interest proportions	low	Netflix research
DLCM [8]	2018	top-K ~10-40	GRU list encoder	local list context	sequential RNN	LTR benchmarks
MIDNN + beam [10]	Alibaba, 2018	tens	list features + beam search	mutual influence	beam x DNN	Taobao search
PRM [11]	Alibaba, 2019	tens	Transformer + PV + rank PE	item-item, personal	O(n^2 d) parallel	Taobao online A/B
SetRank [12]	2020	tens	permutation-invariant attention	set context	O(n^2 d)	LTR benchmarks
PEAR [13]	Huawei, 2022	tens	list + history attention	list + history	higher	reported online
Seq2Slate [27]	Google, 2019	10^2 -> K	pointer-net decoding	sequential context	K decode steps	Google recsys data
Exact-K [28]	Alibaba, 2019	tens -> K	graph attn + pointer, RL	joint set	decode + evaluator	reported online
PRS / GRN [29] [30]	2021	tens	generator + list evaluator	whole permutation	M evaluator calls	reported online
PIER [31]	Meituan, 2023	large perm space	hashed perm selection + eval	permutations	sub-linear search	Meituan online
NAR4Rec [32]	Kuaishou, 2024	tens -> K	non-autoregressive generation	joint slate	1 parallel pass	Kuaishou online
SlateQ [35]	Google, 2019	slate	decomposed long-term Q + LP	long-term value	LP / greedy	live YouTube tests
Page generation [23][24]	Netflix, 2015	~10^2 rows	stage-wise greedy rows	2-D page, dedup	low	Netflix homepage
BaRT [26]	Spotify, 2018	shelves	bandit over item + explanation	framing, explore	low	Spotify home
OneRec [33][34]	Kuaishou, 2025	full corpus	generative session + reward	end-to-end list	heavy generation	Kuaishou online

10. Evaluating re-rankers

Re-rankers are hard to evaluate offline because they change the *context* in which every item is seen. Logged labels were collected under a different ordering.

1. **Pointwise metrics on re-ranked lists** (NDCG@K on logged clicks) are biased. An item moved to a position where it was never shown has no label there.
2. **Evaluator models**, the list-wise CTR models from the generator-evaluator work [29] [30][31], are themselves learned and can be exploited by the generator. They should be validated against online results.

3. **Slate OPE** (the pseudo-inverse estimator [37]) gives unbiased estimates under additivity assumptions, but it needs logged propensities for the production slate policy.
4. **Interleaving** works for comparing two row-level rankers quickly, as at Netflix, but it cannot measure page composition [43].
5. **A/B tests with page-level and long-term metrics** (sessions, retention, diversity of consumption, creator-side metrics) remain the final arbiter. Netflix's shift towards long-term satisfaction [42] and Pinterest's Pareto-frontier governance [39] show that the operating point is a product decision.

---

11. Key takeaways for system designers

1. **Budget a separate last-mile stage.** It turns scores into a page. Even if you start with sort-by-score plus filters, give it its own service or pipeline step, with versioned config and per-rule logging [16][17][18].
2. **Re-rank short lists with context-aware models.** PRM-style Transformers over the top tens of items, with personalized vectors from upstream models and the original rank as a feature, are the proven first step beyond heuristics [11]. Quadratic attention is cheap at n of about 50.
3. **Mind the evaluation-before-reranking gap.** If list context matters, scoring items in the *initial* context is inconsistent. Generator-evaluator designs (PRS, GRN, PIER) or parallel generative models (NAR4Rec) close the gap at higher cost [29][30][31][32].
4. **Use DPP or MMR for diversity, windowed for feeds.** Hulu's fast greedy DPP and YouTube's windowed feed DPP show it scales. X runs it in production after its value model [5][6][18].
5. **Calibrate distributions, not just pairs.** Steck's calibration protects users' minority interests cheaply [7].
6. **Blend on a common, calibrated scale, then apply quotas as soft multipliers.** Fixed ratio round robin is a starting point (early Pinterest) [21]. Federated FPR-to-SPR designs need explicit cross-source calibration (LinkedIn) [22]. Per-request learned weights are where mature blenders end up (Pinterest PRL-PUTS) [39].
7. **Treat ads as an allocation problem with externalities.** Use separate auctions, insertion under ad-load and spacing constraints, and an explicit exchange rate between revenue and engagement, tuned against long-term metrics.
8. **For 2-D surfaces, optimize the page.** Netflix's stage-wise greedy row selection with cross-row dedup and page-level 2-D metrics is the template [23][24]. Spotify and Deezer show that framing (explanations) and visibility (cascade models) belong in the objective [26][44].
9. **Keep business rules explicit, auditable and logged.** Separate hard filters from soft adjustments. Protect score signs before multiplying (X's offset) [19]. Re-run filters at the end. Give every rule an owner and an experiment [47].
10. **Log the slate, not just the click.** Slate-level propensities and rule decisions are what make off-policy evaluation, slate RL and unbiased retraining possible [35][37].
11. **Expect generative models to absorb listwise reranking, not rules.** OneRec-style session generation moves set selection into the model, but published designs still keep integrity, eligibility and policy outside it [33][34].

References

1. Covington, P., Adams, J., Sargin, E. "Deep Neural Networks for YouTube Recommendations." Google, RecSys 2016. https://research.google/pubs/pub45530
2. Google for Developers. "Recommendation systems overview" (Machine Learning: Recommendation Systems course; candidate generation, scoring, re-ranking). Google. https://developers.google.com/machine-learning/recommendation/overview/types
3. Yan, E. "System Design for Recommendations and Search." eugeneyan.com, 2021. https://eugeneyan.com/writing/system-design-for-discovery/
4. Carbonell, J., Goldstein, J. "The Use of MMR, Diversity-Based Reranking for Reordering Documents and Producing Summaries." CMU, SIGIR 1998. https://dl.acm.org/doi/10.1145/290941.291025
5. Chen, L., Zhang, G., Zhou, H. "Fast Greedy MAP Inference for Determinantal Point Process to Improve Recommendation Diversity." Hulu, NeurIPS 2018. https://arxiv.org/abs/1709.05135
6. Wilhelm, M., Ramanathan, A., Bonomo, A., Jain, S., Chi, E. H., Gillenwater, J. "Practical Diversified Recommendations on YouTube with Determinantal Point Processes." Google/YouTube, CIKM 2018. https://dl.acm.org/doi/10.1145/3269206.3272018
7. Steck, H. "Calibrated Recommendations." Netflix, RecSys 2018. https://dl.acm.org/doi/10.1145/3240323.3240372
8. Ai, Q., Bi, K., Guo, J., Croft, W. B. "Learning a Deep Listwise Context Model for Ranking Refinement" (DLCM). SIGIR 2018. https://arxiv.org/abs/1804.05936
9. Ai, Q., Wang, X., Bruch, S., Golbandi, N., Bendersky, M., Najork, M. "Learning Groupwise Multivariate Scoring Functions Using Deep Neural Networks." Google, ICTIR 2019. https://arxiv.org/abs/1811.04415
10. Zhuang, T., Ou, W., Wang, Z. "Globally Optimized Mutual Influence Aware Ranking in E-Commerce Search." Alibaba, IJCAI 2018. https://arxiv.org/abs/1805.08524
11. Pei, C., Zhang, Y., Zhang, Y., Sun, F., Lin, X., Sun, H., Wu, J., Jiang, P., Ou, W. "Personalized Re-ranking for Recommendation" (PRM). Alibaba, RecSys 2019. https://arxiv.org/abs/1904.06813
12. Pang, L., Xu, J., Ai, Q., Lan, Y., Cheng, X., Wen, J. "SetRank: Learning a Permutation-Invariant Ranking Model for Information Retrieval." SIGIR 2020. https://arxiv.org/abs/1912.05891
13. Li, Y., et al. "PEAR: Personalized Re-ranking with Contextualized Transformer for Recommendation." Huawei, WWW 2022 Companion. https://arxiv.org/abs/2203.12267
14. Vorotilov, V., Shugaepov, I. "Scaling the Instagram Explore recommendations system." Meta, Engineering at Meta, 2023. https://engineering.fb.com/2023/08/09/ml-applications/scaling-instagram-explore-recommendations-system/
15. Meta / Instagram. "How Instagram suggests new content" (constrained exploration system). Engineering at Meta, 2020. https://engineering.fb.com/2020/12/10/web/how-instagram-suggests-new-content/
16. Lada, A., Wang, M., Yan, T. "How machine learning powers Facebook's News Feed ranking algorithm." Meta, Engineering at Meta, 2021. https://engineering.fb.com/2021/01/26/ml-applications/news-feed-ranking/
17. Twitter. "Home Mixer README" (the-algorithm). Twitter, 2023. https://github.com/twitter/the-algorithm/blob/main/home-mixer/README.md

18. xAI / X. "x-algorithm" repository README (Home Mixer, Phoenix, Thunder, candidate pipeline traits). xAI / X, 2026. https://github.com/xai-org/x-algorithm
19. xAI / X. "xai-value-model/scoring.rs" and "home-mixer/params/param.rs." xAI / X, 2026. https://github.com/xai-org/x-algorithm/blob/main/xai-value-model/scoring.rs ; https://github.com/xai-org/x-algorithm/blob/main/home-mixer/params/param.rs
20. Pinterest. "Pinterest at WWW 2022" (sponsor slides on the home-feed funnel, incl. Multi-Objective Blender). Pinterest, 2022. https://archives.iw3c2.org/www2022/wp-content/uploads/Sponsors/WWW2022-Pinterest.pdf
21. MLconf. Pinterest Homefeed talk abstract (ratio-based round-robin blending). MLconf, 2016. https://mlconf.com/?p=8909
22. LinkedIn Engineering. "Community-focused Feed optimization." LinkedIn, 2019. https://engineering.linkedin.com/blog/2019/06/community-focused-feed-optimization
23. Gomez-Uribe, C. A., Hunt, N. "The Netflix Recommender System: Algorithms, Business Value, and Innovation." Netflix, ACM TMIS 6(4), 2015. https://dl.acm.org/doi/10.1145/2843948
24. Alvino, C., Basilico, J. "Learning a Personalized Homepage." Netflix Technology Blog, 2015. https://netflixtechblog.com/learning-a-personalized-homepage-aa8ec670359a
25. Chandrashekar, A., Amat, F., Basilico, J., Jebara, T. "Artwork Personalization at Netflix." Netflix Technology Blog, 2017. https://netflixtechblog.com/artwork-personalization-c589f074ad76
26. McInerney, J., Lacker, B., Hansen, S., Higley, K., Bouchard, H., Gruson, A., Mehrotra, R. "Explore, Exploit, and Explain: Personalizing Explainable Recommendations with Bandits" (BaRT). Spotify, RecSys 2018. https://dl.acm.org/doi/10.1145/3240323.3240354
27. Bello, I., Kulkarni, S., Jain, S., Boutilier, C., Chi, E., Eban, E., Luo, X., Mackey, A., Meshi, O. "Seq2Slate: Re-ranking and Slate Optimization with RNNs." Google, 2018/2019. https://arxiv.org/abs/1810.02019
28. Gong, Y., et al. "Exact-K Recommendation via Maximal Clique Optimization." Alibaba, KDD 2019. https://arxiv.org/abs/1905.07089
29. Feng, Y., et al. "Revisit Recommender System in the Permutation Prospective" (PRS). 2021. https://arxiv.org/abs/2102.12057
30. Feng, Y., et al. "GRN: Generative Rerank Network for Context-wise Recommendation." 2021. https://arxiv.org/abs/2104.00860
31. Shi, X., et al. "PIER: Permutation-Level Interest-Based End-to-End Re-ranking Framework in E-commerce." Meituan, WSDM 2023. https://arxiv.org/abs/2302.03487
32. Ren, Y., et al. "Non-autoregressive Generative Models for Reranking Recommendation" (NAR4Rec). Kuaishou, KDD 2024. https://arxiv.org/abs/2402.06871
33. Deng, J., et al. "OneRec: Unifying Retrieve and Rank with Generative Recommender and Iterative Preference Alignment." Kuaishou, 2025. https://arxiv.org/abs/2502.18965
34. Kuaishou OneRec Team. "OneRec Technical Report." Kuaishou, 2025. https://arxiv.org/abs/2506.13695
35. Ie, E., Jain, V., Wang, J., et al. "SlateQ: A Tractable Decomposition for Reinforcement Learning with Recommendation Sets." Google, IJCAI 2019. https://arxiv.org/abs/1905.12767
36. Zhao, X., Xia, L., Zhang, L., Ding, Z., Yin, D., Tang, J. "Deep Reinforcement Learning for Page-wise Recommendations." JD.com / Michigan State, RecSys 2018. https://arxiv.org/abs/1805.02343

37. Swaminathan, A., Krishnamurthy, A., Agarwal, A., Dudik, M., Langford, J., Jose, D., Zitouni, I. "Off-policy Evaluation for Slate Recommendation." Microsoft Research, NeurIPS 2017. https://arxiv.org/abs/1605.04812
38. KDnuggets (LinkedIn). "Autotuning for Multi-Objective Optimization on LinkedIn's Feed Ranking." 2020. https://www.kdnuggets.com/2020/08/autotuning-multi-objective-optimization-linkedin-feed-ranking.html
39. Pinterest. "A Production-Ready RL Framework for Personalized Utility Tuning with Pareto Sweeping in Pinterest Recommender Systems (PRL-PUTS)." Pinterest, 2026. https://arxiv.org/abs/2605.16344
40. Twitter Engineering. "Twitter's Recommendation Algorithm." Twitter, 2023. https://blog.x.com/engineering/en_us/topics/open-source/2023/twitter-recommendation-algorithm
41. Netflix Technology Blog. "Lessons learnt from consolidating ML models in a large scale recommendation system." Netflix, 2023. https://netflixtechblog.medium.com/lessons-learnt-from-consolidating-ml-models-in-a-large-scale-recommendation-system-870c5ea5eb4a
42. Netflix Technology Blog. "Recommending for Long-Term Member Satisfaction at Netflix." Netflix, 2024. https://netflixtechblog.com/recommending-for-long-term-member-satisfaction-at-netflix-ac15cada49ef
43. Parks, J., Aurisset, J., Ramm, M. "Innovating Faster on Personalization Algorithms at Netflix Using Interleaving." Netflix Technology Blog, 2017. https://netflixtechblog.com/interleaving-in-online-experiments-at-netflix-a04ee392ec55
44. Bendada, W., Salha, G., Bontempelli, T. "Carousel Personalization in Music Streaming Apps with Contextual Bandits." Deezer, RecSys 2020. https://arxiv.org/abs/2009.06546
45. Hill, D. N., Nassif, H., Liu, Y., Iyer, A., Vishwanathan, S. V. N. "An Efficient Bandit Algorithm for Realtime Multivariate Optimization." Amazon, KDD 2017. https://arxiv.org/abs/1810.09558
46. Haldar, M., et al. "Learning To Rank Diversely At Airbnb." Airbnb, 2022 (CIKM 2023). https://arxiv.org/abs/2210.07774
47. Zinkevich, M. "Rules of Machine Learning: Best Practices for ML Engineering." Google. https://developers.google.com/machine-learning/guides/rules-of-ml

16. Ads Recommendation Systems: pCTR/pCVR Prediction, Auctions, Pacing, Calibration and the Meta, Google, Pinterest, Snap and Twitter Ads Stacks

> Research note: live web search and page fetching were not available when this chapter was written. The shared search budget for the run was used up, and outbound fetches to company blogs and arXiv failed DNS resolution or were refused by the egress proxy. The chapter is therefore built from two kinds of source. The first is primary sources that other chapters of this Atlas had already fetched and checked: Meta's Andromeda, Lattice and GEM posts and papers, Google's FTRL and Photon papers, Facebook's GBDT+LR paper, Twitter's delayed-feedback paper, Pinterest, Snap and LinkedIn ads write-ups. The second is a small set of canonical auction-theory papers, cited by DOI. Numbers that could not be rechecked are marked "approximately" or "reported". Verify figures against the originals before quoting them.

Overview

An ads recommender is a recommender with a price tag on every slot. It has the same funnel as an organic feed (retrieval, light ranking, heavy ranking, re-ranking), but three things change the design.

1. **The output is money, so predictions must be calibrated probabilities, not just scores.** An auction ranks ads by something like bid x pCTR (or bid x pCTR x pCVR for conversion objectives) and charges a price derived from the next competitor's score. If pCTR is 2x too high, the advertiser is ranked and charged as if the ad were twice as good. Every ads paper since 2013 treats calibration as a first-class output [1][3].
2. **There is a second optimizer: the advertiser.** Advertisers set budgets and targets (cost per click, cost per action, return on ad spend). The platform must spend each budget smoothly over the day (pacing) and, more and more often, bid on the advertiser's behalf (auto-bidding) [39][40].
3. **Ads compete with organic content for the user's attention.** Ad load, spacing and the long-term "ad blindness" users learn are page-level costs. Google showed that cutting mobile ads could raise long-term value [9].

This chapter goes through the stack in the order a request flows, then company by company: Google (FTRL, Photon, DCN), Facebook/Meta (GBDT+LR, DHEN, Wukong, InterFormer, Lattice, Andromeda, GEM), Pinterest, Snap, Twitter/X, LinkedIn, Spotify, and

the 2025-2026 generative-advertising wave (Tencent GPR, Meituan EGA, ByteDance TokenMixer-Large).

```
ad request (user, context, placement)
     |
     v
+-----------------+   eligibility: targeting, budget left, policy, frequency caps
| ad index / ACL  |   10^6 .. 10^7+ eligible ads (Meta: "tens of millions") [10][11]
+--------+--------+
         v
+-----------------+   two-tower / learned index (Andromeda) / GPU brute force
| retrieval       |   -> 10^3 .. 10^4 candidates
+--------+--------+
         v
+-----------------+   cheap model, often distilled from the heavy ranker
| light ranking   |   -> 10^2 candidates
+--------+--------+
         v
+-----------------+   multi-task DNN: pCTR, pCVR (per conversion type), quality,
| heavy ranking   |   negative feedback; calibrated per task
+--------+--------+
         v
+-----------------+   score = bid(or auto-bid) x p(action) + quality terms
| auction         |   allocation + pricing (GSP-like, VCG-like), reserve prices
+--------+--------+
         v
+-----------------+   ad load, spacing, slot choice vs organic items
| allocation      |
+--------+--------+
         v
impression --> click/conversion logs --> joiner --> online trainer (minutes-hours)
         ^                                                  |
         +-------------- pacing / bid controller <----------+ (spend telemetry)
```

1. Foundations: pCTR, pCVR and the expected-value score

1.1 What the ranker predicts

The core predictions are:

- **pCTR** = P(click | impression, user, ad, context).
- **pCVR** = P(conversion | click, ...). It is trained on clicked impressions but used on all impressions, which creates sample-selection bias (Section 4.3) [33].
- **pCTCVR** = pCTR x pCVR = P(click and convert | impression), the quantity a cost-per-action (CPA) advertiser actually pays for [33].
- Quality and negative-feedback predictions (hide, report, low-quality click), which enter the auction as a penalty or a quality term.

The ranking score is an expected value. Under the usual convention, an ad billed per click is scored by bid x pCTR (its expected cost per thousand impressions, eCPM, up to a constant). An ad optimized for conversions is scored by target CPA x pCTR x pCVR. Both Google's and Facebook's ads papers state that the auction multiplies predicted CTR by bids, which is why calibration matters as much as ranking quality [1][3].

1.2 Metrics: normalized entropy, log loss, calibration ratio

Facebook introduced **normalized entropy (NE)**: log loss divided by the entropy of the background CTR. Raw log loss looks deceptively good when the base rate is near 0, and NE makes datasets with different base rates comparable [3]. Google's paper relies on log loss

and AUC and tracks calibration slices as a core monitoring tool [1]. Both papers stress that improvements of a fraction of a percent in log loss or NE matter online [1][3].

Calibration is tracked as a global ratio (sum of predicted / sum of observed) and as reliability curves per slice and per predicted-score bucket [1][3].

2. Google: the linear era and the plumbing that made it work

2.1 Ad Click Prediction: a View from the Trenches (KDD 2013)

Problem. Search ads needed a CTR model trained online on an enormous impression stream, with billions of sparse features, small enough in memory to serve at very high QPS [1].

Model. Logistic regression trained with **FTRL-Proximal**. The algorithm combines the sparsity of L1-regularized updates (exact zeros) with good convergence, and it uses **per-coordinate learning rates**. These require storing the sum of gradients and the sum of squared gradients per feature [1][2].

Memory tricks. Memory, not FLOPs, was the binding constraint [1]:

Trick	Setting	RAM saved	AucLoss cost
Poisson feature inclusion	p = 0.03	~60%	0.020%
Poisson feature inclusion	p = 0.1	~40%	0.006%
Bloom-filter inclusion	admit after n = 2 sightings	~66%	0.008%
16-bit q2.13 weights, randomized rounding	vs 64-bit floats	large (summaries: ~75% for coefficients)	no measurable loss

(Figures as reported in the paper and in summaries of it [1][2].)

The paper also trains many model variants at once with shared feature-index structures, and subsamples non-click queries while keeping all clicks, correcting with importance weights [1]. The companion paper formalizes randomized rounding for low-RAM learning [2].

Negative results. The paper lists ideas that did not pay off in production, for example feature hashing, which needed too large a table to avoid damaging collisions [1]. This "what did not work" section became a model for later industrial papers.

Calibration. A calibration layer sits between the model and the auction, and the paper treats calibration plots across many slices as a primary monitoring tool [1].

2.2 Photon: exactly-once click joins (SIGMOD 2013)

Training labels and billing come from the same join. Photon joins each click to its query event by a unique query ID across several datacenters with exactly-once semantics. The core is the **IdRegistry**, a Paxos-replicated, geo-distributed key-value store of IDs that have

already been joined. A click is never emitted twice even when several datacenters process the same click stream. Clicks that arrive before their query are retried. Throughput is reported in the millions of joined events per minute, with seconds of latency (approximate) [4].

Lesson: in ads, the label pipeline is also the billing pipeline. Duplicate positives inflate CTR, corrupt calibration and overcharge advertisers.

2.3 From crosses to deep crosses: DCN and DCN-V2

Google's Deep & Cross Network (ADKDD 2017) was introduced explicitly for ad click prediction. Each cross layer multiplies the original input with the current layer's output, so the polynomial degree grows by one per layer [6]. DCN-V2 (WWW 2021) replaced the cross vector with a full (optionally low-rank) matrix and reported gains across many web-scale learning-to-rank systems at Google, under very high QPS and strict latency [7]. DCN-V2 became the production baseline at Pinterest ads and LinkedIn (Section 6).

2.4 Long-term ad load: "Focusing on the Long-term" (KDD 2015)

Hohnhold, O'Brien and Tang showed that users exposed to more or worse ads gradually learn to ignore them, which drives ad CTR down over months. Short A/B tests cannot see this. They fit a model that predicts long-term user learning from short-term ad quality and ad load, so that later launches can be judged with short tests plus that model. Google then **reduced** the number of ads on mobile search (reported as roughly 50% on some surfaces, approximate), judging that long-term gains outweighed short-term revenue loss [9].

This is the canonical public evidence that ad load is a long-horizon decision, not a per-request revenue maximization.

2.5 Microsoft Bing adPredictor (ICML 2010)

Bing's sponsored-search CTR model was a Bayesian probit regression over sparse binary features, trained online with assumed-density filtering. Each weight carries a Gaussian mean and variance, so the model knows how uncertain it is about a new ad. Sampling from the posterior gives new ads impressions in proportion to their chance of being good, which is Thompson sampling [5]. It is the earliest well-documented production answer to ad cold-start exploration.

3. Facebook/Meta: from GBDT+LR to foundation models

3.1 Practical Lessons from Predicting Clicks on Ads at Facebook (ADKDD 2014)

Scale at the time. More than 750M daily active users and over 1M active advertisers [3].

Architecture. Boosted decision trees act as a learned feature transform: the leaf index of each tree becomes a one-hot feature, and the concatenation goes into a logistic regression trained online. The hybrid beat either trees or LR alone by more than 3% NE [3].

```
raw features --> boosted trees T1..TK (retrained ~daily)
                    | leaf ids (one-hot)
                    v
              logistic regression (online SGD, near real time)
                    |
                    v
          p (downsampled space) --> q = p / (p + (1 - p) / w) --> auction
```

Online joiner. Impressions and clicks are joined by request ID inside a waiting window. Too short a window mislabels late clicks as negatives. Too long a window delays training and costs memory. The joiner needs alarms that cut the trainer off if the click stream stops [3].

Key lessons [3].

- Historical features (past behavior of user and ad) dominate contextual features.
- Freshness matters: NE worsens as training data falls days behind the serving day.
- **Negative downsampling with recalibration**: keep negatives at rate w, then correct with q = p / (p + (1 - p) / w). This formula became standard practice.
- Uniform subsampling to about 10% of data reportedly costs about 1% NE (approximate).

3.2 Scaling the ranker: DHEN, Wukong, InterFormer

Meta's ranking architecture research since 2022 has mostly been published in ads terms.

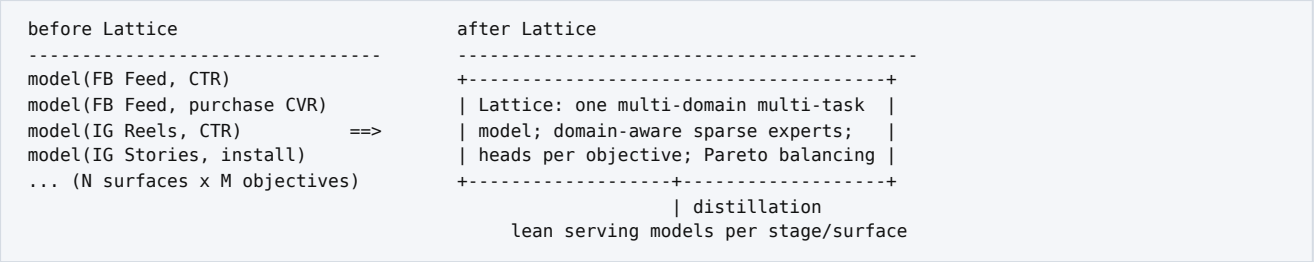
- **DHEN (2022).** Each layer is an ensemble of heterogeneous interaction modules (dot-product, self-attention, convolution, DCN, linear). Layers stack into a hierarchy of increasing interaction order. It improved NE by 0.27% over an AdvancedDLRM baseline on a large CTR dataset and introduced Hybrid Sharded Data Parallel training, reported at up to 1.2x throughput over FSDP [18]. A 2025 paper describes running DHEN for **ad conversion-rate prediction** in production [19], and Meta built AutoML over DHEN-style module search for its ranking models [23].
- **Wukong (ICML 2024).** A stack of identical layers, each with a Factorization Machine Block and a Linear Compress Block. Layer i covers interaction orders 1 to 2^i. On Meta's internal data it held a scaling law across two orders of magnitude of model complexity, beyond 100 GFLOP per example, where earlier architectures saturated [20].
- **InterFormer (2024-2025).** Interleaves an Interaction Arch and a Sequence Arch with bidirectional information flow and a Cross Arch for selective summarization. It reports 0.15% NE and 24% QPS gains on Meta internal data and a **0.6% topline gain in pilot ads launches** [21].
- **HSTU (ICML 2024).** Generative-recommender architecture with reported 1.5-trillion- parameter models and +12.4% online metrics on surfaces with billions of users [22]. It is covered in Chapter 6.

3.3 Lattice: consolidating the ads model fleet (2023, paper 2025)

Problem. Ads ranking had fragmented into many models across surfaces (Feed, Stories, Reels, Audience Network), objectives (click, many conversion types) and funnel stages. That slowed iteration and wasted compute [12].

Design. Lattice is a **multi-domain, multi-task** architecture with sparse activation. It learns from many data sources and handles delayed feedback ranging from seconds to days. Meta's 2023 description cites **trillions of parameters** trained on **hundreds of billions of examples** [12]. Hand-balancing thousands of domains and tens of objectives is impossible, so Meta uses Pareto-optimality-based methods, including Pareto-front feature selection and MetaBalance [12][14]. MetaBalance rescales each auxiliary task's gradient, per network part, toward the magnitude of the target task's gradient so auxiliary tasks neither dominate nor vanish [14].

The 2025 paper ("Model Space Redesign for Cost-Effective Industry-Scale Ads Recommendations") describes four moves: cross-domain knowledge sharing, data consolidation, model unification, and **distillation** of a large consolidated model into lean user-facing models. Reported results: +10% on revenue-driving top-line metrics, +11.5% user satisfaction, +6% conversion rate, and 20% capacity saving (Meta's own figures) [13].



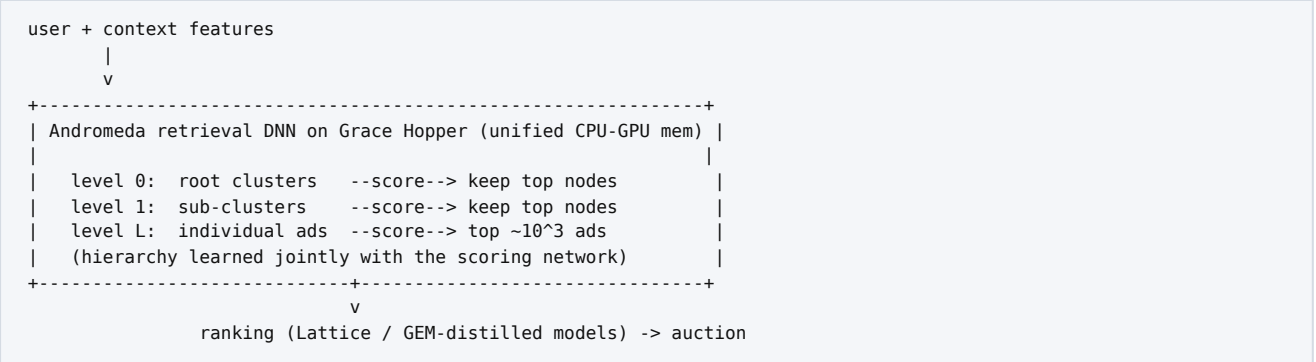
3.4 Andromeda: deep-learning ads retrieval (December 2024)

Problem. Generative-AI creative tools sharply increased the number of eligible ads. Meta said more than a million advertisers created more than 15 million ads in one month with its GenAI tools. Two-tower retrieval with ANN caps how much user-ad interaction the model can express [10].

Architecture [10].

- A highly customized deep neural network with **sub-linear inference cost**, running with heavy parallelism on **NVIDIA Grace Hopper Superchips**, with Meta's **MTIA** accelerator also part of the hardware plan.
- Ads are organized into a **hierarchical index with multiple layers**, so inference only expands the most relevant nodes.
- The **index and the retrieval model are trained jointly**, which Meta says gives better precision and recall than two-tower or ANN approaches.

Results. On Facebook and Instagram: **+6% recall** for the retrieval system and **+8% ads quality** on selected segments [10]. Secondary write-ups cite a roughly **10,000x** increase in retrieval model complexity and a narrowing from **tens of millions** of eligible ads to a **few thousand** candidates per request; treat these as approximate [11].



Lesson. Co-designing model, index and hardware makes heavy computation affordable at the stage that sees the most items, which moves the cascade's quality bottleneck upstream [10].

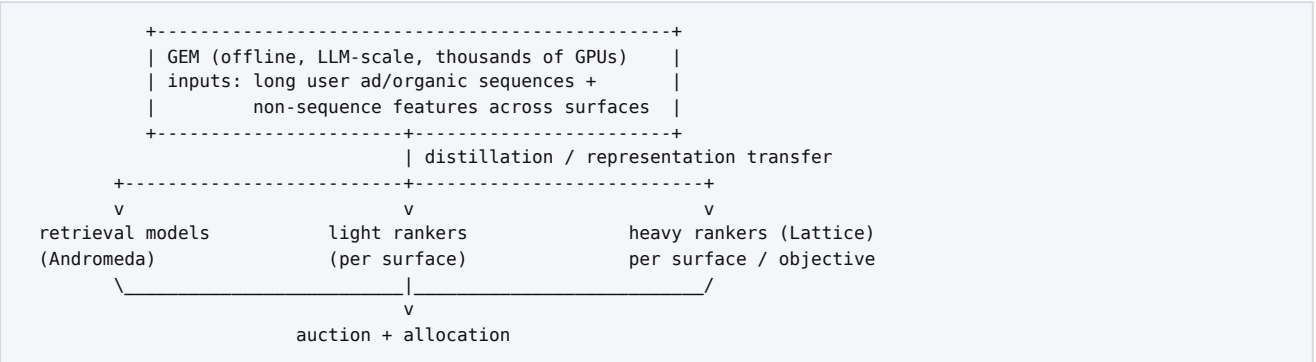
3.5 GEM: the Generative Ads Recommendation Model (November 2025)

Meta describes GEM as "the largest foundation model for recommendation systems (RecSys) in the industry, trained at the scale of large language models" [15]. It is trained across thousands of GPUs with an LLM-inspired paradigm.

Role in the stack. GEM does **not** score every candidate online. It is a central teacher whose learnings are propagated "across the entire ads model fleet" through post-training knowledge transfer (distillation and representation transfer) into the vertical models that run in each funnel stage [15]. Meta frames the work in three pillars: scaling the model architecture, post-training knowledge transfer to downstream models, and training scalability [15]. Secondary coverage links the architecture to InterFormer-style sequence/non-sequence interaction and Wukong-style stacked factorization [17].

Results. +5% ad conversions on Instagram and +3% on Facebook Feed in Q2 2025 [15].

Training (August 2026 follow-up). Meta reports doubling end-to-end training efficiency to **20-25% model FLOPs utilization** while scaling training FLOPs **4x in twelve months**, through co-design of kernels, numeric precision, parallelism, networking and memory [16].



Lesson. When the best model is too expensive to serve, keep it offline as a teacher and pay for it once in training, many times in student quality. This is the same pattern LLM serving uses, and Meta now applies it fleet-wide [15].

3.6 Meta ads evolution at a glance

Year	System	Stage	Key idea	Reported impact
2014	GBDT + LR [3]	ranking	trees as feature transform, online LR	>3% NE vs either alone
2022	DHEN [18]	ranking	hierarchical ensemble of interaction modules	+0.27% NE
2023	Lattice [12]	ranking	multi-domain multi-task consolidation	trillions of params
2024	Wukong [20]	ranking	stacked FMs with scaling law	scaling beyond 100 GFLOP/ex
2024	InterFormer [21]	ranking	bidirectional seq / non-seq interaction	+0.6% topline (pilot ads)
2024	Andromeda [10]	retrieval	learned hierarchical index on GPUs	+6% recall, +8% quality
2025	Lattice paper [13]	fleet	redesign + distillation	+10% topline, -20% capacity
2025	GEM [15]	teacher	LLM-scale foundation model	+5% IG / +3% FB conversions
2026	GEM training [16]	infra	MFU to 20-25%, 4x FLOPs/yr	n/a

4. Conversion modeling: delayed and biased labels

Conversions (purchases, installs, sign-ups) arrive minutes to weeks after the click. This gives ads two label problems that organic feeds mostly avoid.

4.1 Delayed feedback

- **Criteo's joint delay model (Chapelle, KDD 2014).** Each click has a latent "will ever convert" variable and an exponential delay. A click unconverted after elapsed time e is either a real negative or a slow positive. Likelihood terms: converted at delay d gives P(C=1) x lambda x exp(-lambda d); not converted after e gives 1 - P(C=1) + P(C=1) x exp(-lambda e) [30].
- **Twitter's fake negatives (RecSys 2019).** Every impression is ingested immediately as a negative and again as a positive when the engagement arrives. No waiting window, so freshness is maximal. The bias is corrected either by importance weighting (FN-weighted) or at prediction time with p = b / (1 - b), where b is the biased prediction (FN-calibration). Both beat waiting-window baselines and the delayed-feedback model offline and in an online A/B test on Twitter ads [28].
- **Alibaba ES-DFM (AAAI 2021) and DEFER (KDD 2021).** Wait a short elapsed window. Emit early conversions as positives and others as negatives, and re-emit late positives (ES-DFM) [31]. DEFER also re-ingests real negatives once the attribution window closes, so every sample has its final label, with an importance-sampling loss for the duplication. DEFER reports offline and online gains in Alibaba display advertising [32].
- **Lattice** must handle feedback delays from seconds to days in one model [12].

4.2 Streaming training and freshness

ByteDance's Monolith (2022) shows the modern ads/feeds streaming design. A Flink joiner merges serving-time features with user actions from Kafka. Negatives are sampled inside the joiner with a **log-odds correction** at serving time. Sparse parameters sync from training to serving parameter servers at minute granularity, which measurably improves AUC. Embedding tables are collisionless (cuckoo hashing) with frequency filtering and expiry [38].

4.3 Sample-selection bias: ESMM

CVR models trained only on clicked impressions are applied to all impressions. Alibaba's ESMM (SIGIR 2018) trains a CTR tower and a CVR tower with shared embeddings, supervising pCTR (on clicks) and pCTCVR = pCTR x pCVR (on click-and-convert) over the entire impression space. The CVR tower is never directly supervised. It uses the product, not a division, because dividing by small CTR values is numerically unstable [33]. On 8.9 billion Taobao samples, reported gains were roughly +2.18% AUC on CVR and +2.32% on CTCVR (approximate) [33].

```
impression features --> shared embeddings
                            |                 |
                        CTR tower         CVR tower
                            |                 |
                          pCTR ------(x)---- pCVR
                            |                 |
                    loss on click       pCTCVR: loss on click & convert
```

5. Calibration: the contract with the auction

Every step that changes the training distribution moves calibration: negative downsampling, fake-negative duplication, importance weights, new features, and a new architecture [3][28].

Method	Where used	Notes
Downsampling correction q = p / (p + (1-p)/w)	Facebook ads [3]	exact for uniform negative sampling
Log-odds correction at serving	ByteDance Monolith [38]	same idea in logit space
FN-calibration p = b / (1 - b)	Twitter ads [28]	undoes fake-negative duplication
Post-hoc calibration layer + slice plots	Google ads [1]	calibration monitored per slice
Isotonic regression / field-level calibration	general practice [37]	per advertiser or placement
Per-task temperature scaling	Spotify audio ads [36]	one parameter per task; chosen for simplicity
Isotonic layer trained jointly in the DNN	LinkedIn LiRank, Feed/Jobs/Ads CTR [34]	2026 version also debiases position [35]

Spotify states the reason plainly: miscalibration distorts auction ordering and pricing, so each task head of its multi-task audio-ads model gets its own temperature [36]. LinkedIn argues that a single learnable isotonic layer can capture interactions across task, position and platform that a global Platt transform cannot [35].

Operational rule from these sources: recalibrate whenever a head's training distribution changes, and monitor predicted versus observed rates per head, surface, advertiser segment and position bucket. A calibration drift is a pricing change.

6. Pinterest, Snap, Twitter/X, LinkedIn and others

6.1 Pinterest ads ranking

- **Sequence features in ads engagement models.** Pinterest added Transformer-encoded user action sequences to its ads engagement models, with DCN-V2 as the production baseline. It reported DCN-V2 cross-layer overflow problems under float16 training, a practical warning for anyone moving crossing layers to mixed precision [24].
- **Feature interactions under industry constraints (2024).** Pinterest describes how it evaluated interaction modules under latency and cost limits rather than on public benchmarks alone [25].
- **ML-DCN (2026).** Naively scaling DCN-V2 and MaskNet gave rapidly diminishing offline gains. ML-DCN adds an instance-conditioned mask to a low-rank crossing layer and was deployed for ads CTR prediction with neutral serving cost [26].

6.2 Snap ads ranking

Snap's ad ranking predicts several conversion events at once (app installs, purchases, sign-ups) with **MMoE and PLE** multi-task architectures, so sparse conversion types borrow signal from dense ones [27]. Snap has also published extensively on embedding-based retrieval and generative recommendation with semantic IDs (Chapter 6), which is relevant to ad candidate generation.

6.3 Twitter/X ads

- **Continuous training with delayed feedback** (Section 4.1) is Twitter ads' best-known public contribution [28].
- **TwHIN** (KDD 2022) is a heterogeneous-network embedding over users, tweets, advertisers and engagement relations. The same embeddings are reused for candidate generation, ranking features and ads [51].
- In the timeline, ads are ranked by a separate system and then **mixed** into the For You feed by Home Mixer alongside Who-to-Follow modules, after organic ranking and filtering [29].

6.4 LinkedIn

LiRank (KDD 2024) serves Feed, Jobs and **Ads CTR**. It extends DCN-V2 into a Residual DCN with attention and low-rank matrices (nearly 30% fewer parameters) and adds the jointly trained isotonic calibration layer [34]. LinkedIn's LiGNN graph embeddings gave about +2% Ads CTR in A/B tests (approximate) [50].

6.5 Criteo

Criteo ran Field-aware Factorization Machines in production for display-ads CTR and conversion prediction, adding a seeding algorithm and distributed learning to cut training time [8]. Its "Offline A/B testing" work proposed capped and normalized importance-sampling estimators for top-K display ads that predicted the sign and ordering of online A/B results better than standard offline metrics [48].

7. Auctions: allocation and pricing

7.1 GSP, VCG and why ads use "GSP-like" auctions

- **Vickrey (1961)** showed that a sealed-bid second-price auction for a single item makes truthful bidding a dominant strategy [43].
- **Position auctions.** Search and feed ads sell several slots with different click rates. The **Generalized Second-Price (GSP)** auction ranks advertisers by bid (or by bid x quality) and charges each the minimum it needs to keep its slot, roughly the next advertiser's score divided by its own quality. Edelman, Ostrovsky and Schwarz showed that **GSP is not truthful**, but it has a "locally envy-free" equilibrium whose payments equal those of the **VCG** mechanism [41]. Varian independently analyzed the same symmetric Nash equilibria of position auctions and fit them to data [42].
- **VCG** charges each winner the externality it imposes on others. It is truthful but harder to explain to advertisers, and it composes awkwardly with reserve prices and quality scores. Industry practice is "GSP-like ranking by expected value, with quality adjustments", or VCG-style pricing on some surfaces; exact mechanisms are rarely published.

```
rank by  score_i = bid_i x pCTR_i (x pCVR_i) + quality_i         (expected value)
slot k goes to the k-th highest score
GSP price per click for winner k:
    cpc_k = (score_{k+1} - quality_k) / pCTR_k     (+ minimum increment, >= reserve)
=> winner k pays just enough to beat k+1, so pCTR errors move both rank AND price
```

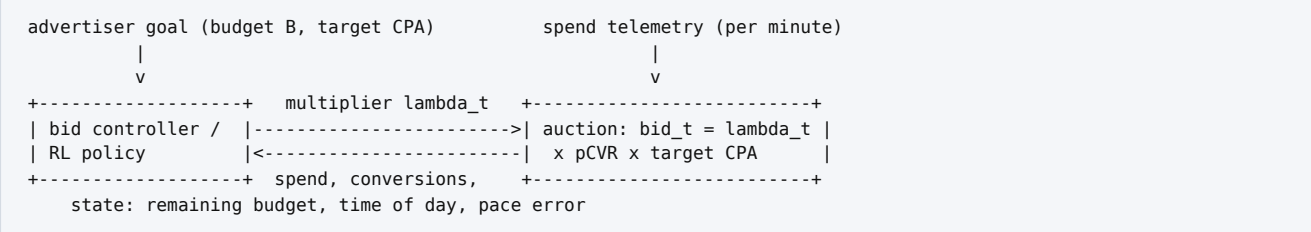
7.2 Generative auctions

The 2025-2026 generative-advertising papers move parts of the auction into the model. Meituan's **EGA-V2** generates ads with RQ-VAE semantic IDs (3 levels x 1024 codes), decomposes generation into P(POI) x P(creative | POI), and adds token-level bids and a **payment network** that guarantees individual rationality (payment never exceeds bid). Incentive compatibility is enforced only approximately, through a Lagrangian on ex-post regret [44]. Tencent's **GPR** for WeChat Channels ads uses four token types (user, organic, environment, ad), RQ-KMeans+ semantic IDs, a heterogeneous hierarchical decoder and value-guided trie beam search. It reports latency falling from 200 ms+ to about 80 ms and five cascade models collapsing into one [45]. Even these designs keep an explicit payment and policy layer: pricing is too consequential to leave implicit.

8. Budgets, pacing and auto-bidding

Advertisers declare budgets and goals, not per-auction bids. Two control loops sit around the auction.

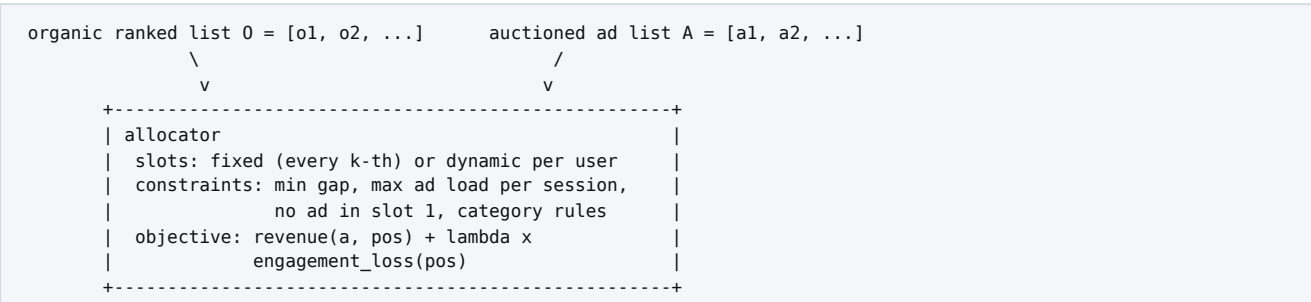
- **Pacing** spreads spend over the campaign. The two common families are probabilistic **throttling** (participate in a fraction of eligible auctions) and **bid modification** (scale the bid by a multiplier driven by a feedback controller on spend versus plan). This is standard practice across platforms; detailed production descriptions are scarce.
- **Auto-bidding** chooses bids to maximize conversions under budget and CPA/ROAS constraints. Alibaba's **budget-constrained bidding** (CIKM 2018) uses model-free RL (DQN) to adjust a bid-scaling factor over the day, with a reward design that avoids myopic overspending [39]. Alibaba's **multi-agent RTB** clusters advertisers into agents trained with a distributed multi-agent actor-critic, balancing competition and cooperation on Taobao [40].



The shared design pattern: the RL policy outputs a **scalar control** (a bid multiplier), not an item choice. That keeps the action space small, the policy easy to roll back, and the rest of the stack unchanged [39][40]. Since bids are multiplied by pCVR, the bidder inherits every calibration error from the ranker. Calibration and pacing are one system.

9. Ad load and allocation against organic content

Public posts from Meta do not describe ad slotting in detail. The pattern visible across the industry, and documented for X's Home Mixer [29], is that ads go through their own retrieval, ranking and auction and are then **inserted** into the organic stream at allowed positions under ad-load and spacing constraints.



Design decisions:

1. **Shared currency.** Organic value (predicted engagement) and ad value (expected revenue) are in different units. The exchange rate lambda is a policy, tuned against long-term retention [9].
2. **Long-horizon measurement.** Ad load changes need long-running holdouts or a model of user learning, because ad blindness develops over months [9].
3. **Generative allocation.** Tencent's GPR feeds organic and ad tokens through one decoder, which is a step toward joint allocation inside the model [45].

10. The 2025-2026 scaling wave in ads ranking

System	Org / year	Stage	Architecture	Reported scale / result
Andromeda [10]	Meta 2024	retrieval	learned hierarchical index, GH200	+6% recall, +8% quality
Lattice [13]	Meta 2025	ranking fleet	multi-domain MTL + distillation	+10% topline, 20% capacity saved
GEM [15][16]	Meta 2025-26	teacher	LLM-scale foundation model	+5% IG / +3% FB conv.; 20-25% MFU
RankMixer [47]	ByteDance 2025	ranking	token mixing + per-token FFN	MFU 4.5% -> 45%; ~100x params
TokenMixer-Large [46]	ByteDance 2026	ranking	RankMixer successor	7B and 15B offline (Douyin ads)
GPR [45]	Tencent 2025	one model	generative, semantic IDs	5 models -> 1; ~80 ms
EGA-V2 [44]	Meituan 2025	one model	generative + payment network	IR guaranteed, approx. IC
ML-DCN [26]	Pinterest 2026	ranking	masked low-rank crossing	neutral serving cost
LiRank [34]	LinkedIn 2024	ranking	Residual DCN + isotonic layer	Feed / Jobs / Ads CTR

Two directions are visible. Meta keeps a classic cascade (Andromeda, Lattice students, the auction) and adds a giant offline teacher. Tencent and Meituan collapse the cascade into one generative model while keeping a small explicit payment and policy module. ByteDance's contribution is hardware efficiency: the jump from about 4.5% to 45% MFU is what makes a 100x parameter increase affordable at constant latency [47].

Key takeaways for system designers

1. **Calibration is the API between ML and money.** Rank and price both depend on pCTR x bid. Correct every sampling choice (downsampling formula, log-odds, FN-calibration), calibrate per task head, and monitor per slice [1][3][28][34][36].
2. **Model the funnel explicitly.** Train CVR over the entire impression space (ESMM), and treat delayed conversions with fake-negative, elapsed-time or real-negative schemes rather than long waits [28][31][32][33].
3. **Freshness pays, so the label join is critical infrastructure.** Mint a unique impression ID, join idempotently (Photon), and train online or at minute granularity. Log features at serving time rather than recomputing them [3][4][38][49].
4. **Push model quality upstream.** Andromeda shows that a deep, hardware-co-designed retrieval stage gains recall that no ranker can recover [10].
5. **Consolidate, then distill.** Many per-surface, per-objective models waste compute and iteration speed. One large multi-domain model (Lattice) or an offline foundation teacher (GEM), distilled into lean serving students, is the pattern at the largest scale [13][15].
6. **Scale only architectures that are known to scale, and measure MFU.** Wukong and RankMixer/TokenMixer-Large show scaling laws, and naively widening DCN or MaskNet does not [20][26][47].

7. **Keep the mechanism explicit and auditable.** Even generative ads systems keep a payment network and policy layer. GSP-like pricing means every model error is also a pricing error [41][44][45].
8. **Let control policies turn knobs.** Pacing and auto-bidding work best as scalar multipliers on top of calibrated predictions [39][40].
9. **Decide ad load on long-term metrics.** Users learn to ignore ads. Use long holdouts or user-learning models, and set the organic-versus-ads exchange rate as policy [9].
10. **Publish your negative results internally.** Google's "what did not work" list saved the industry years. Keep one for your stack [1].

References

1. McMahan, H. B., Holt, G., Sculley, D., et al. "Ad Click Prediction: a View from the Trenches." Google, KDD 2013. https://research.google/pubs/ad-click-prediction-a-view-from-the-trenches/
2. Golovin, D., Sculley, D., McMahan, H. B., Young, M. "Large-Scale Learning with Less RAM via Randomization." Google, ICML 2013. https://arxiv.org/abs/1303.4664 (paper notes on [1]: Qun Le, https://le.qun.ch/en/blog/paper-notes-ad-click-prediction-a-view-from-the-trenches/)
3. He, X., Pan, J., Jin, O., et al. "Practical Lessons from Predicting Clicks on Ads at Facebook." Facebook, ADKDD 2014. https://quinonero.net/Publications/predicting-clicks-facebook.pdf
4. Ananthanarayanan, R., et al. "Photon: Fault-tolerant and Scalable Joining of Continuous Data Streams." Google, SIGMOD 2013. https://research.google/pubs/photon-fault-tolerant-and-scalable-joining-of-continuous-data-streams/
5. Graepel, T., Quiñonero Candela, J., Borchert, T., Herbrich, R. "Web-Scale Bayesian Click-Through Rate Prediction for Sponsored Search Advertising in Microsoft's Bing Search Engine." Microsoft Research, ICML 2010. https://www.microsoft.com/en-us/research/publication/web-scale-bayesian-click-through-rate-prediction-for-sponsored-search-advertising-in-microsofts-bing-search-engine/
6. Wang, R., Fu, B., Fu, G., Wang, M. "Deep & Cross Network for Ad Click Predictions." Google/Stanford, ADKDD 2017. https://arxiv.org/abs/1708.05123
7. Wang, R., et al. "DCN V2: Improved Deep & Cross Network and Practical Lessons for Web-scale Learning to Rank Systems." Google, WWW 2021. https://arxiv.org/abs/2008.13535
8. Juan, Y., Lefortier, D., Chapelle, O. "Field-aware Factorization Machines in a Real-world Online Advertising System." Criteo, WWW 2017 Companion. https://arxiv.org/abs/1701.04099
9. Hohnhold, H., O'Brien, D., Tang, D. "Focusing on the Long-term: It's Good for Users and Business." Google, KDD 2015. https://dl.acm.org/doi/10.1145/2783258.2788583
10. Meta Engineering. "Meta Andromeda: Supercharging Advantage+ automation with the next-gen personalized ads retrieval engine." Meta, December 2024. https://engineering.fb.com/2024/12/02/production-engineering/meta-andromeda-advantage-automation-next-gen-personalized-ads-retrieval-engine/
11. Loomer, J. "Meta Andromeda: What It Means for Your Ad Strategy" (secondary summary of [10]). 2025. https://www.jonloomer.com/meta-andromeda/

12. Meta AI. "Meta Lattice: AI ads performance and efficiency." Meta, 2023. https://ai.meta.com/blog/ai-ads-performance-efficiency-meta-lattice/
13. Meta. "Meta Lattice: Model Space Redesign for Cost-Effective Industry-Scale Ads Recommendations." Meta, 2025. https://arxiv.org/abs/2512.09200
14. He, Y., Feng, X., Cheng, C., Ji, G., Guo, Y., Caverlee, J. "MetaBalance: Improving Multi-Task Recommendations via Adapting Gradient Magnitudes of Auxiliary Tasks." Meta AI / TAMU, WWW 2022. https://arxiv.org/abs/2203.06801
15. Engineering at Meta. "Meta's Generative Ads Model (GEM): The central brain accelerating ads recommendation AI innovation." Meta, November 2025. https://engineering.fb.com/2025/11/10/ml-applications/metas-generative-ads-model-gem-the-central-brain-accelerating-ads-recommendation-ai-innovation/
16. Engineering at Meta. "GEM Training: How Meta Doubled the Efficiency of Its LLM-Scale Ads Foundation Model." Meta, August 2026. https://engineering.fb.com/2026/08/03/ml-applications/training-gem-at-llm-scale-meta-ads-recommendation-foundation-model/
17. ByteByteGo. "How Meta Built a New AI-Powered Ads Model for 5% Better Conversions" (secondary). 2025. https://blog.bytebytego.com/p/how-meta-built-a-new-ai-powered-ads
18. Zhang, B., Luo, L., Liu, X., et al. "DHEN: A Deep and Hierarchical Ensemble Network for Large-Scale Click-Through Rate Prediction." Meta, 2022. https://arxiv.org/abs/2203.11014
19. Meta. "On the Practice of Deep Hierarchical Ensemble Network for Ad Conversion Rate Prediction." Meta, 2025. https://arxiv.org/abs/2504.08169
20. Zhang, B., Luo, L., Chen, Y., et al. "Wukong: Towards a Scaling Law for Large-Scale Recommendation." Meta, ICML 2024. https://arxiv.org/abs/2403.02545
21. Zeng, Z., et al. "InterFormer: Effective Heterogeneous Interaction Learning for Click-Through Rate Prediction." Meta/UIUC, 2024-2025. https://arxiv.org/abs/2411.09852
22. Zhai, J., et al. "Actions Speak Louder than Words: Trillion-Parameter Sequential Transducers for Generative Recommendations." Meta, ICML 2024. https://arxiv.org/abs/2402.17152
23. Meta. "AutoML for Large Capacity Modeling of Meta's Ranking Systems." Meta, 2023. https://arxiv.org/abs/2311.07870
24. Pinterest Engineering. "User Action Sequence Modeling for Pinterest Ads Engagement Modeling." Pinterest, Medium. https://medium.com/pinterest-engineering/user-action-sequence-modeling-for-pinterest-ads-engagement-modeling-21139cab8f4e
25. Pinterest. "Improving feature interactions at Pinterest under industry constraints." Pinterest, CIKM 2024 workshop. https://arxiv.org/abs/2412.01985
26. Li, J., et al. "ML-DCN: Masked Low-Rank Deep Crossing Network Towards Scalable Ads Click-through Rate Prediction at Pinterest." Pinterest, 2026. https://arxiv.org/abs/2602.09194
27. InfoQ. "Snap ad ranking (MMoE and PLE for multiple conversion events)." 2022. https://www.infoq.com/news/2022/10/snap-ad-ranking/
28. Ktena, S. I., Tejani, A., Theis, L., et al. "Addressing Delayed Feedback for Continuous Training with Neural Networks in CTR Prediction." Twitter, RecSys 2019. https://arxiv.org/abs/1907.06558
29. Twitter Engineering. "Twitter's Recommendation Algorithm." Twitter, 2023. https://blog.x.com/engineering/en_us/topics/open-source/2023/twitter-recommendation-algorith

m (Home Mixer: https://github.com/twitter/the-algorithm/blob/main/home-mixer/README.md)
30. Chapelle, O. "Modeling Delayed Feedback in Display Advertising." Criteo, KDD 2014. https://dl.acm.org/doi/10.1145/2623330.2623634
31. Yang, J.-Q., et al. "Capturing Delayed Feedback in Conversion Rate Prediction via Elapsed-Time Sampling." Alibaba, AAAI 2021. https://arxiv.org/abs/2012.03245
32. Gu, S., et al. "Real Negatives Matter: Continuous Training with Real Negatives for Delayed Feedback Modeling." Alibaba, KDD 2021. https://arxiv.org/abs/2104.14121
33. Ma, X., et al. "Entire Space Multi-Task Model: An Effective Approach for Estimating Post-Click Conversion Rate." Alibaba, SIGIR 2018. https://arxiv.org/abs/1804.07931
34. Borisyuk, F., et al. "LiRank: Industrial Large Scale Ranking Models at LinkedIn." LinkedIn, KDD 2024. https://arxiv.org/abs/2402.06859
35. "Isotonic Layer: A Unified Framework for Recommendation Calibration and Debiasing." LinkedIn, 2026. https://arxiv.org/abs/2603.06589
36. "An Audio-centric Multi-task Learning Framework for Streaming Ads Targeting on Spotify." Spotify, 2025. https://arxiv.org/abs/2506.18735
37. "Ranking model calibration in recommender systems." RecSys ML Substack. https://recsysml.substack.com/p/ranking-model-calibration-in-recommender (see also parametric ranking calibration, https://arxiv.org/abs/2112.07428)
38. Liu, Z., et al. "Monolith: Real Time Recommendation System With Collisionless Embedding Table." ByteDance, 2022. https://arxiv.org/abs/2209.07663
39. Wu, D., Chen, X., Yang, X., et al. "Budget Constrained Bidding by Model-free Reinforcement Learning in Display Advertising." Alibaba, CIKM 2018. https://arxiv.org/abs/1802.08365
40. Jin, J., Song, C., Li, H., et al. "Real-Time Bidding with Multi-Agent Reinforcement Learning in Display Advertising." Alibaba, CIKM 2018. https://arxiv.org/abs/1802.09756
41. Edelman, B., Ostrovsky, M., Schwarz, M. "Internet Advertising and the Generalized Second-Price Auction: Selling Billions of Dollars Worth of Keywords." American Economic Review 97(1), 2007. https://doi.org/10.1257/aer.97.1.242
42. Varian, H. R. "Position Auctions." International Journal of Industrial Organization 25(6), 2007. https://doi.org/10.1016/j.ijindorg.2006.10.002
43. Vickrey, W. "Counterspeculation, Auctions, and Competitive Sealed Tenders." Journal of Finance 16(1), 1961. https://doi.org/10.1111/j.1540-6261.1961.tb02789.x
44. "EGA-V2: An End-to-end Generative Framework for Industrial Advertising." Meituan, 2025. https://arxiv.org/abs/2505.17549
45. "GPR: Towards a Generative Pre-trained One-Model Paradigm for Large-Scale Advertising Recommendation." Tencent, 2025. https://arxiv.org/abs/2511.10138
46. ByteDance. "TokenMixer-Large: Scaling Up Large Ranking Models in Industrial Recommenders." ByteDance, 2026. https://arxiv.org/abs/2602.06563
47. ByteDance. "RankMixer: Scaling Up Ranking Models in Industrial Recommenders." ByteDance, 2025. https://arxiv.org/abs/2507.15551
48. Gilotte, A., Calauzènes, C., Nedelec, T., Abraham, A., Dollé, S. "Offline A/B Testing for Recommender Systems." Criteo, WSDM 2018. https://arxiv.org/abs/1801.07030
49. Zinkevich, M. "Rules of Machine Learning: Best Practices for ML Engineering." Google. https://developers.google.com/machine-learning/guides/rules-of-ml
50. Borisyuk, F., et al. "LiGNN: Graph Neural Networks at LinkedIn." LinkedIn, KDD 2024. https://arxiv.org/abs/2402.11139

51. El-Kishky, A., et al. "TwHIN: Embedding the Twitter Heterogeneous Information Network for Personalized Recommendation." Twitter, KDD 2022. https://arxiv.org/abs/2202.05387

17. Google & YouTube Case Studies: From Co-Visitation Graphs to Semantic IDs and LLM-Native Recommenders

> Research note: this chapter was written when the shared live web-search budget for the run had been used up and direct page fetches were failing. It relies on the primary papers and posts listed in the References, using facts already checked against sources for earlier chapters of this Atlas (Chapters 1-4, 6, 11-15) plus the published content of the papers themselves. Numbers that could not be re-checked for this chapter are marked "approx.".

Overview

Google has published more about its production recommenders than almost any other company. The papers form a fairly continuous line, and many techniques that are now standard industry practice started there:

- **2007-2010: graph and counting systems.** Google News used MinHash, PLSI and co-visitation counts [19][20]. YouTube's 2010 recommender built a related-video graph from co-visitation and expanded it from a user's seed videos [1].
- **2016: deep learning arrives.** Wide & Deep for Google Play [15] and the two-stage "Deep Neural Networks for YouTube Recommendations" paper [2] set the retrieval-plus-ranking template that most of the industry still uses.
- **2018-2019: multi-objective, debiased, RL-aware systems.** MMoE [6], the YouTube multitask Watch Next ranker with a shallow position-bias tower [5], two-tower retrieval with streaming logQ correction [4], top-K off-policy REINFORCE [8], SlateQ [9], and DPP diversification [12].
- **2020-2022: feature interaction and representation work.** DCN-V2 [18], mixed negative sampling [16], self-supervised item towers [17], survey-based "valued watch time" objectives [14], and reward shaping [10].
- **2023-2026: tokenized items and LLMs.** TIGER generative retrieval with semantic IDs [23], semantic IDs inside the YouTube ranker [24], LLM-based user-journey modeling [26], and PLUM, which adapts Gemini-family LLMs into industrial generative retrievers for YouTube [25].

The rest of the chapter takes each system in turn: problem, architecture, models, public scale numbers, key tricks and lessons. It closes with a comparison table and takeaways.

```
Google / YouTube recommender lineage (simplified)

2007 Google News  --- MinHash + PLSI + co-visitation (MapReduce/Bigtable)
2010 YouTube      --- co-visitation related graph, seed expansion, heuristic rank
2016 Google Play  --- Wide & Deep (memorization + generalization)
2016 YouTube      --- DNN candidate gen (softmax -> ANN) + DNN ranker (watch time)
2018 MMoE / DPP   --- multi-gate experts; windowed DPP feed diversification
2019 YouTube      --- two-tower + logQ | MMoE + shallow tower | REINFORCE | SlateQ
2020 Google       --- DCN-V2, mixed negative sampling, ScaNN
2021 YouTube      --- "valued watch time" from surveys; self-supervised towers
2023 Google       --- TIGER: semantic IDs + seq2seq generative retrieval
2024 YouTube      --- semantic IDs replace video IDs in ranking embeddings
2025 YouTube      --- PLUM: LLM continued pre-training on SIDs, generative retrieval
```

17.1 Google News personalization (2007, 2010)

Problem

Google News had millions of users and a corpus that turned over within hours. Stories have very short lifetimes, so purely item-based models trained offline go stale, and new stories have no click history at all.

Architecture (Das et al., WWW 2007)

Das, Datar, Garg and Rajaram described a **collaborative filtering system that is a blend of three signals**, each computed with Google's batch and serving infrastructure (MapReduce and Bigtable) [19]:

1. **MinHash clustering.** Users are clustered by the Jaccard similarity of their click histories, approximated with MinHash and locality-sensitive hashing. A story is recommended if users in the same cluster clicked it.
2. **PLSI.** Probabilistic latent semantic indexing over user-story clicks gives soft user-to-latent-class membership. The paper had to build a MapReduce version of the EM algorithm to fit it at Google scale.
3. **Co-visitation.** Two stories are related if the same user clicked both within a time window. These counts can be updated online as clicks arrive, which handles freshness much better than the batch models.

The final score is a **linear combination** of the three. Clusters and PLSI models are recomputed offline, while co-visitation counts and per-cluster click counts update in near-real time [19].

```
Google News CF (2007)

click logs --MapReduce--> MinHash user clusters ---+
           --MapReduce--> PLSI (EM) user classes ---+--> Bigtable stats
live clicks --> co-visitation counts (online) ------+         |
                                                              v
request(user) --> candidate stories --> score = w1*minhash + w2*plsi + w3*covisit
```

Content-plus-trend model (Liu, Dolan and Pedersen, IUI 2010)

A follow-up added a **Bayesian model of each user's news interests by topic**. It separates a user's "genuine" topic interest from the local news trend, which is estimated

from what everyone in the same country is clicking at that moment. This content-based score is combined with the existing CF score. In a live experiment the hybrid reportedly improved click-through on the recommended section by about 30% (approx.) and increased visits [20].

Lessons

- Fast-decaying corpora need at least one signal that updates online (co-visitation, trending counts).
- Blending several cheap scorers linearly was a workable "ensemble" a decade before learned multi-source rankers.
- Modeling the population trend explicitly keeps personalization from fighting the news cycle.

17.2 The YouTube video recommendation system (2010)

Problem

In 2010, YouTube's goal was to show personalized recommendations on the home page. Its corpus was huge, its metadata poor, most videos were short, and interactions were noisy and implicit [1].

Architecture (Davidson et al., RecSys 2010)

```
YouTube 2010 (batch-computed, served from Bigtable)

watch logs (24h sessions) --MapReduce--> co-visitation counts c_ij
                                      --> related(v_i) = top-N by r(v_i, v_j)
                                                r = c_ij / f(v_i, v_j)
user seeds S = {watched, liked, favorited, playlisted videos}
         |
         v
candidates C_n(S) = union of related-graph neighbours up to n hops
         |
         v
rank by linear mix of: video quality | user specificity | diversification
         |
         v
top ~4-60 recs (pre-computed several times a day; served with explanation)
```

- **Related videos.** For each video, co-visitation counts c_ij are computed within a session window (24 hours in the paper, approx.). The relatedness score divides c_ij by a normalization function f(v_i, v_j); the simplest choice is the product of the two videos' global popularity, which keeps very popular videos from being related to everything [1].
- **Candidate generation.** Start from a seed set of videos the user engaged with: watched, liked, favorited, added to playlists. Take the union of their related videos, then expand a limited number of hops through the related-video graph. Candidates further out add breadth (approx. description) [1].
- **Ranking.** Candidates are scored by a combination of video quality (view count, ratings, comments, favorites, shares, upload time), user specificity (how strongly the seed is tied to the user's interests) and diversification (for example limiting recommendations per seed or per channel) [1].

- **Serving.** Recommendations were pre-generated in batch through a series of MapReduce jobs over Bigtable, and refreshed several times a day. Each recommendation came with an explanation naming the seed video ("because you watched ...").

Reported results (approx.)

The paper reported that recommendations accounted for about 60% of video clicks from the home page, and that co-visitation recommendations had a click-through rate around twice the "most viewed" baseline (about 207% of baseline, approx.) [1].

Lessons

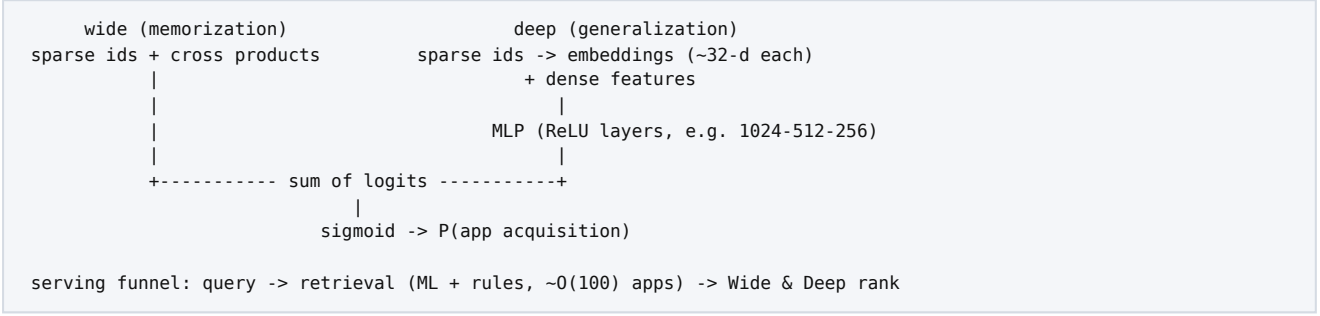
- Item-to-item co-occurrence over sessions is a strong, cheap first recommender. It is still a standard candidate source.
- Normalizing by popularity is required; raw co-counts collapse onto the head.
- Explanations come almost for free when recommendations are derived from seeds.
- Batch precomputation was enough at the time, but it is the opposite of the online freshness the next generation needed.

17.3 Wide & Deep Learning for Google Play (2016)

Problem

Google Play needed app recommendations for over one billion active users and over one million apps [15]. Generalized linear models with hand-crafted cross-product features **memorize** well (for example "user installed Netflix AND impression is Pandora") but generalize poorly to unseen pairs. Embedding-based deep models **generalize** but over-generalize for niche users, recommending irrelevant apps when the user-item matrix is sparse and high-rank.

Architecture



- The wide part is a linear model on raw sparse features plus cross-product transformations, trained with **FTRL with L1 regularization**. The deep part maps categorical features to dense embeddings, concatenates them with continuous features, and passes them through a ReLU MLP, trained with **AdaGrad** [15].
- The two are **jointly trained**: their logits are summed before one sigmoid and one loss. That is different from an ensemble, in which the models are trained separately and combined only at inference.

- The label is **app acquisition** (install) for an impression.
- Retrieval first returns a short list of apps using a mix of learned models and rules. Wide & Deep only ranks that list [15].

Scale and serving

The models were trained on over 500 billion examples. Each new model is **warm-started** from the previous one's embeddings and linear weights, so daily retraining does not start from scratch, and a sanity check runs before a model is pushed to serving. Scoring all candidates single-threaded took about 31 ms; splitting the batch across threads (batch of 50, 4 threads) brought client-side latency down to about 14 ms, against a budget of roughly 10 ms [15].

Results

A three-week live A/B test showed **+3.9% app acquisitions** over a highly optimized wide-only control and about **+1%** over deep-only [15]. The implementation was open-sourced as a TensorFlow Estimator.

Follow-on Google Play work

- **Mixed Negative Sampling (WWW 2020).** Two-tower retrievers trained with in-batch negatives never see items that were never engaged with. MNS adds a stream of negatives sampled uniformly from the whole corpus, shared across the batch. It was evaluated on Google Play app recommendations and beat batch-only sampling [16].
- **Self-supervised item towers (CIKM 2021).** Yao et al. add a contrastive auxiliary task. Two augmented views of an item's features are created with correlated feature masking and dropout, and pushed together in embedding space. This regularizes the item tower for long-tail items. The paper reports gains on Google-scale app recommendation [17].
- **DCN-V2 (WWW 2021, arXiv 2020).** DCN-V2 replaces Wide & Deep's hand-designed crosses with learned explicit crosses: x_{l+1} = x0 * (W_l x_l + b_l) + x_l with full-rank W_l. It then factorizes W into low-rank mixtures for cost. The paper reports offline and online gains "across many web-scale learning to rank systems at Google" [18].

Lessons

Wide & Deep made "memorize with a wide path, generalize with a deep path" the default ranking template. Its weak point was the hand-crafted crosses in the wide part, which DeepFM, DCN and DCN-V2 then learned automatically.

17.4 Deep Neural Networks for YouTube Recommendations (2016)

Problem

Covington, Adams and Sargin describe YouTube as "one of the largest scale and most sophisticated industrial recommendation systems in existence". They name three challenges: **scale** (existing algorithms that work on small problems fail), **freshness** (many hours of video uploaded per second, and new content should be balanced against well-established videos) and **noise** (sparse, implicit feedback with almost no ground-truth satisfaction labels) [2].

Architecture

The system has two neural networks, following the "classic two-stage information retrieval dichotomy" [2]:

```
YouTube 2016

corpus (millions) ---> CANDIDATE GENERATION ---> hundreds ---> RANKING ---> dozens
                       user history + context       |          rich features,
other candidate  ----> -> MLP tower -> user vec u    |           impression data,
sources (earlier       train: sampled softmax        |           weighted LR on
work, co-vis, etc.)    serve: ANN top-N by <u, v_j>  +--------> watch time
                                                                     |
                                                      A/B test on watch time/user
```

Candidate generation

- **Framing.** Recommendation is **extreme multiclass classification**: predict which video, out of millions, the user watches next at time t, given the user U and context C [2].
- **Inputs.** The user's watch history is a variable-length bag of video IDs, each mapped to a learned embedding and **averaged**. Search history is handled the same way, tokenized into unigrams and bigrams. Demographic, geographic, device and gender features are added as dense or embedded inputs. The paper reports vocabularies of about 1M videos and 1M search tokens, embedded in 256 floats each, with up to 50 recent watches and 50 recent searches (approx.) [2][3].
- **Tower.** A "tower" of fully connected ReLU layers whose width halves at each step. Depths up to four layers (for example 2048 -> 1024 -> 512 -> 256) kept improving holdout MAP (approx.) [2].
- **Training. Sampled softmax** with several thousand negatives and importance weighting. Secondary summaries report a speedup of over 100x versus the full softmax [2][3].
- **Serving.** The softmax output layer is replaced by **nearest-neighbor search in dot-product space** over the learned video vectors, using the user vector as the query. The authors note that the choice of ANN algorithm did not much affect A/B results [2]. This "train as a classifier, serve as a MIPS lookup" pattern became the template for later two-tower retrieval (see Chapter 2).

Key tricks in candidate generation

- **Example age.** Models trained on historical data learn the *average* watch likelihood over the training window and under-predict fresh viral videos. Adding the age of the training example as a feature, and setting it to zero (or slightly negative) at serving,

lets the model learn the time-dependence of popularity and reproduce the sharp spike after upload [2].
- **Training on all watches**, including watches of YouTube videos embedded on other sites, not only on watches that came from recommendations. That limits feedback-loop exploitation and lets new content surface [2].
- **A fixed number of examples per user**, so a small group of highly active users does not dominate the loss [2].
- **Withholding information on purpose.** Search queries are fed as an unordered bag of tokens. Otherwise the model learns to recommend the results page of the user's last search, which users find annoying [2].
- **Predict the future, not a random held-out watch.** Consumption is asymmetric (episodes are watched in order; users explore broadly and then narrow). The training label is therefore the *next* watch, using only earlier history as input. Randomly held-out watches leak future information and did much worse in live tests [2].

Ranking

- **Features.** Hundreds of features, most of them categorical and continuous signals about the user's past interaction with the video and similar items: when the user last watched the channel, how many videos from the channel they watched, and how often the video was already shown to them without being watched ("churn"). Impression-level data is available because only hundreds of candidates are scored [2].
- **Embedding sizes.** Each categorical feature gets its own embedding. Dimension grows roughly with the logarithm of the vocabulary size, and very large vocabularies are truncated to the top-N IDs by click frequency. Embeddings of the same ID space are shared across features (for example "impression video ID" and "last watched video ID") [2].
- **Continuous features** are normalized to [0, 1) using their cumulative distribution (quantiles). Powers such as x^2 and sqrt(x) are also fed in, so the network can model super- and sub-linear functions easily [2].
- **Objective: expected watch time, via weighted logistic regression.** Positive (clicked) impressions are weighted by their watch time and negatives get unit weight. The learned odds then approximate E[T](1 + P), which is close to E[T] when CTR P is small. At serving, exp(logit) is used as the predicted watch time [2]. Ranking by click probability promotes clickbait that users do not finish, so watch time is a better proxy for satisfaction.
- **Evaluation.** Offline, the paper uses "weighted, per-user loss" (the fraction of watch time mis-predicted when a watched and an unwatched impression are mis-ordered). Wider and deeper networks lowered this loss, with the best configuration in the paper's table reaching about 34.6% (approx.) [2]. Final decisions were made by live A/B tests on watch time per user.

Scale

The paper and secondary summaries describe a model of roughly one billion parameters trained on hundreds of billions of examples (approx.) [2][3]. It was trained with Google Brain's distributed system (the successor DistBelief infrastructure and TensorFlow).

Lessons

1. The two-stage funnel separates **recall at low cost** from **precision with rich features**, and each stage can have its own objective.
2. **Label and example engineering** (example age, per-user caps, future-watch labels, withholding the search sequence) mattered as much as architecture.
3. Choose the **objective** carefully. Moving from clicks to watch time was the first of several steps toward modeling satisfaction (see 17.8).

17.5 Two-tower retrieval with sampling-bias correction (2019)

Problem

The 2016 candidate generator learned one output vector per video ID. That makes content features and cold start awkward, and the vocabulary must be fixed. Streaming training makes a fixed vocabulary even harder [4].

Design

Yi et al. moved to a **two-tower** model. A query tower encodes user and context features, and a candidate tower encodes video features (ID plus content). Both produce embeddings scored by inner product [4].

```
query tower  q_i = f(user_i, context_i)          batch i = 1..B
item tower   v_j = g(video_j features)
logits       s_ij = <q_i, v_j> / tau  -  log p_j        (logQ correction)
loss         weighted softmax CE over in-batch candidates (weight = reward)
p_j          streaming estimate: hashed arrays of "last step seen" and
             moving average of the gap between occurrences; p_j ~ 1 / gap
```

- **In-batch negatives** are cheap, but popular items appear as negatives more often, so they get over-penalized. Subtracting log p_j from each logit turns the in-batch softmax into an approximately unbiased estimate of the full softmax [4].
- **Streaming frequency estimation** without a fixed vocabulary: hashed arrays on parameter servers store the last global step at which each item was seen and a moving average of the step gap. The estimate of p_j is roughly the inverse gap [4].
- **Reward-weighted loss.** Each training pair can be weighted by a reward, for example watch completion, so that clicks with short watches count less [4].
- **Embedding normalization and temperature** were reported to improve training quality.

Deployment

The model was deployed for YouTube as "Neural Deep Retrieval", retrieving from a corpus of tens of millions of videos, with live A/B gains [4]. Retrieval serving uses Google's quantization-based MIPS (the line of work later released as ScaNN [27]).

17.6 Recommending What Video to Watch Next: the multitask ranker (2019)

Problem

The Watch Next ranker scores a few hundred candidates retrieved for a video the user is currently watching. It faces two problems [5]:

1. **Multiple competing objectives.** Engagement (clicks, watch time) and satisfaction (likes, dismissals, survey-style signals) are correlated but in tension.
2. **Implicit selection bias.** Users click on top-ranked items partly because they are on top. Training on that feedback reinforces the current ranker.

Architecture

The model extends Wide & Deep with two ideas [5][6][7]:

```
YouTube Watch Next ranker (2019)

query video + candidate + user + context features
                |
          shared hidden layer (cheaper than feeding raw inputs to every expert)
                |
     +-------- MMoE: N experts --------+
     |   gate_k(x) = softmax(W_k x)    |    (one gate per objective)
     +--+--------+--------+-----------+
        |        |        |
  engagement  engagement  satisfaction ... task towers
    (click)   (watch)     (like, dismiss, rating)
        |        |        |
        +--------+--------+--> logits
                             + shallow-tower bias logit (training only)
position, device ---> SHALLOW TOWER ------^   (position dropout ~10%, approx.)
                |
serving: position feature missing -> debiased per-task predictions
          -> weighted combination (weights tuned by online experiments) -> rank
```

- **MMoE.** Multi-gate Mixture-of-Experts [6] gives each objective its own softmax gate over a shared set of experts, so each task can choose how much to share. The experts sit on top of a shared hidden layer to keep serving cost down [5]. At the same model complexity, MMoE improved both engagement and satisfaction metrics over shared-bottom, in configurations with 4 and 8 experts (from the paper's tables, approx.) [5].
- **Shallow tower for position bias.** A small network takes features that explain selection bias, such as position and device, and outputs a bias logit that is **added** to each engagement task's logit during training. At serving the position feature is treated as missing, so the main model's prediction is the debiased utility. Position is dropped out during training (about 10%, per a secondary summary) so the model does not over-rely on it [5][7].
- **Fusion.** Per-objective predictions are combined with a weighted function whose weights are tuned manually for the best online engagement-satisfaction trade-off [5] [7].
- **Training** is continuous on fresh logs ("training in sequence"), so the model follows the data distribution.

Lessons

- Put **bias into the model structure**: a separate additive tower absorbs presentation effects that would otherwise contaminate the utility heads.
- Soft parameter sharing (MMoE) is the low-cost upgrade over shared-bottom when objectives conflict. Later work (PLE and others, Chapter 4) adds task-specific experts.
- The paper notes that MMoE gating can become very imbalanced (expert polarization), and that dropout on experts helps.

17.7 Reinforcement learning at YouTube: REINFORCE and SlateQ (2019)

Top-K Off-Policy Correction for a REINFORCE Recommender (WSDM 2019)

Problem. Supervised candidate generators imitate the logs of earlier recommenders, which reinforces their biases. Chen et al. recast YouTube's candidate generator as a **policy** pi(a | s) over millions of videos, trained with REINFORCE to maximize cumulative reward over the user's trajectory [8].

```
user event sequence (watches, context) --> RNN state s_t
                                              |
              +-------------------------------+-------------------+
              v                                                   v
     pi_theta(a|s): softmax over              beta_hat(a|s): behavior-policy head
     millions of videos (deployed)            (gradient blocked into shared state)
              |                                                   |
              +---------- importance weight  pi / beta_hat ------+
                                  |
               lambda_K = K * (1 - pi(a|s))^(K-1)     (top-K correction)
                                  |
         REINFORCE update: sum_t  w_t * lambda_K * R_t * grad log pi(a_t|s_t)
         + weight capping, normalized IS, TRPO-style constraint
```

Key tricks [8]:

1. **Learned behavior policy.** Logs come from a mixture of past systems whose propensities were not logged. A second softmax head estimates beta_hat(a | s) from the same state, with gradients blocked so it does not distort the main encoder.
2. **Top-K correction.** YouTube shows K items at once. The probability that item a is in the slate is 1 - (1 - pi(a))^K, which adds the multiplier lambda_K = K (1 - pi(a))^(K-1) to the gradient. Unlikely items get pushed up strongly; items that are already likely get little extra push, which spreads probability mass across a good set rather than one item.
3. **Variance control.** Weight capping, self-normalized importance sampling, and a TRPO-like trust region.
4. **Exploration.** Serving returns the top K' items and samples the remaining K - K' from the policy (Boltzmann-style exploration), which keeps the logs informative.

Results. Live experiments showed statistically significant gains in viewing time, around 0.8-0.9% (approx.) for the top-K correction, which the authors call one of the biggest single launches in recent years. More long-tail items were also surfaced [8].

Follow-ups. Reward shaping imputes sparse satisfaction signals, such as survey responses, onto unlabeled interactions and uses them as REINFORCE rewards [10]. RecSim is an open-source simulator for testing slate-based RL recommenders before they reach users [11].

SlateQ (IJCAI 2019)

Value-based RL over slates is combinatorial. Ie et al. show that under two assumptions, that the user consumes at most one item and that reward and transition depend only on the consumed item, the slate's Q-value decomposes as Q(s, A) = sum over i in A of P(i | s, A) * Q_bar(s, i). Here P is a user choice model, such as a conditional logit, and Q_bar is an item-level long-term value learned with SARSA/TD. Slate optimization becomes a linear program, or a greedy top-k approximation that works almost as well [9]. SlateQ was tested in live YouTube experiments in which item-level long-term values replaced the myopic scores in the existing ranking, with reported gains in user engagement (approx.) [9].

Lessons

- RL for recommendation became practical by **keeping the deployed architecture** (softmax over items, served by ANN) and changing only the learning signal and corrections.
- Off-policy correction and slate-aware gradients matter more than the choice of RL algorithm.
- Long-term rewards need careful design; reward shaping from surveys is how YouTube tied RL to satisfaction.

17.8 Diversity, fairness and the "valued watch time" objective

DPP diversification in the home feed (CIKM 2018)

Pointwise rankers fill the feed with near-duplicates. Wilhelm et al. re-rank the top candidates with a **determinantal point process** whose kernel is L_ii = q_i^2 and L_ij = alpha * q_i * q_j * exp(-D_ij / (2 sigma^2)), with q the ranker's quality score and D_ij a distance between videos (from embeddings or topics). Greedy MAP inference is run **window by window**, where a window is roughly the items visible on a phone screen, to keep latency bounded. Alpha and sigma were first grid-searched against online metrics; later the paper learns a deep kernel from feed interactions. The DPP beat earlier heuristic rules such as per-channel caps on long-term satisfaction metrics [12].

Pairwise fairness (KDD 2019)

Beutel et al. define a pairwise fairness metric for rankers (does an item from group A that the user engaged with outrank an unengaged item from group B as often as the reverse?), estimated from randomized experiment data. They add a pairwise regularizer to training and report improved fairness with little loss of accuracy in a large-scale production recommender at Google [13].

From clicks to watch time to valued watch time (2021)

YouTube's public explanation of its recommender, written by Cristos Goodrow, VP of Engineering, describes three objective eras [14]:

1. **Clicks.** These encouraged clickbait.
2. **Watch time** (from 2012). A better proxy, but not every minute watched is a good minute.
3. **"Valued watch time"** measured with **user surveys** ("How satisfied were you with this video?", rated 1-5 stars). Only a fraction of views get a survey, so a model is trained to predict survey answers for all watches, and that prediction becomes part of the objective. Likes, dislikes, shares and "not interested" signals are also used.

The same post describes **authoritative-content boosting** for news and information and **demotion of borderline content**, using human raters following published guidelines. It reports that consumption of borderline content from recommendations dropped by about 70% in the US after the 2019 changes (approx.) [14]. The YouTube "How YouTube Works" pages present this as the policy view of the same system [29].

Lesson

Objective design is iterative and partly a policy decision. YouTube treats it as a product of models (survey predictors, quality classifiers) and explicit policy layers, not as a single label.

17.9 TIGER: generative retrieval with semantic IDs (NeurIPS 2023)

Problem

Two-tower retrieval needs one embedding per item and an ANN index. Atomic IDs carry no meaning, cold-start items have no trained vector, and the item-embedding table grows with the corpus [23].

Architecture

```
TIGER (Rajput et al., Google, 2023)

item text/metadata -> Sentence-T5 (768-d, approx.) -> RQ-VAE
    level 1 code c1 (256 options)  -> residual r1
    level 2 code c2 (256 options)  -> residual r2
    level 3 code c3 (256 options)  [+ collision token c4]
Semantic ID(item) = (c1, c2, c3, c4)

user history: SID(i1) SID(i2) ... SID(in)  (+ user id token, hashed)
         |
  Transformer encoder-decoder (T5-style, small)
         |
  beam search decodes the next item's SID tuple -> lookup -> items
```

- **Semantic IDs** are coarse-to-fine: items that share a prefix are semantically similar, so parameters are shared across related items through the shared code embeddings.

Collisions (identical tuples) are resolved by an extra disambiguation token [23].
- **Generative retrieval.** A sequence-to-sequence Transformer reads the user's history as a token sequence and **autoregressively generates the next item's semantic ID**. The model itself acts as the index; no ANN is needed at serving [23].
- **Results.** On Amazon Beauty, Sports and Toys, TIGER beat SASRec, S3-Rec, P5 and other baselines on Recall@K and NDCG@K. The paper reports gains in cold-start retrieval and controllable diversity by sampling at the first code level [23].
- **Caveats.** Beam search can generate invalid IDs (rare in practice). Decoding cost grows with beam width and ID length. Later work identified codebook imbalance and "hourglass" collapse in intermediate levels (Chapter 6).

Semantic IDs inside the YouTube ranker (RecSys 2024)

Singh et al. ask whether semantic IDs can replace video-ID embeddings in a production ranking model. They derive semantic IDs from a frozen content encoder (a VideoBERT-style audio-visual model, approx.) with an RQ-VAE, then compare ways to map them to embeddings: n-gram lookups over code prefixes versus **SentencePiece-style learned subword tokenization** of the codes. SentencePiece-based semantic IDs matched the memorization ability of random-hashed video IDs while generalizing better to new and long-tail videos, with a fixed embedding-table budget [24].

Lessons

- Treat the item tokenizer as infrastructure: content encoder, quantizer, collision handling, and a refresh policy.
- Semantic IDs help even if you never adopt generative retrieval. They are a better feature for ranking embeddings at a fixed table size.

17.10 The LLM / Gemini era: PLUM and user-journey modeling (2023-2026)

LLM user-interest journeys (2023)

Christakopoulou et al. used LLMs to extract and **name user interest journeys** (for example "learning to bake sourdough") from a user's activity, using prompt tuning and fine-tuning on a small set of human-labeled journeys. The paper argues that journey-level interest representations could make recommendations more explainable and better aligned with users' long-term goals. It is research rather than a reported production launch [26].

PLUM: Pre-trained Language Models for industrial-scale generative recommendation (2025)

PLUM is Google's most detailed public description of LLM-based recommendation at YouTube [25]. As described in the paper (details approx.):

```
PLUM (Google / YouTube, 2025)

1. Item tokenization: Semantic IDs v2
   multimodal content embeddings (+ co-engagement signal) -> RQ-style
   multi-level codes; tuned for coverage and low collision at YouTube scale
2. Continued pre-training (CPT) of a pre-trained Gemini-family LLM
   on a mixture of: user watch sequences as SID tokens, SIDs aligned with
   text (titles, topics), so the model "speaks" SIDs and language
3. Task fine-tuning: generative retrieval
   input: user context + history (SIDs + text) -> decode SIDs of next videos
4. Serving: beam decoding of SIDs -> videos, as a retrieval source
   feeding the existing ranking stack
```

- The item vocabulary is semantic IDs rather than atomic video IDs, so a frozen-size vocabulary can cover a corpus of billions of videos and new uploads.
- Continued pre-training aligns the LLM's language knowledge with the SID "language" before fine-tuning for retrieval.
- The paper reports that the generative retriever beat a heavily optimized production large-embedding retrieval model in live experiments on YouTube surfaces, and studies scaling with model size and compute (approx.) [25].
- It also argues that this design uses far fewer item-specific parameters than embedding-table retrieval, at the price of much more compute per request [25].

What changed versus 2016

Dimension	2016 DNN candidate gen [2]	2019 two-tower [4]	2025 PLUM [25]
Item representation	learned per-ID output vector	tower over ID + content features	semantic-ID tokens from content and co-engagement
User representation	averaged watch/search embeddings	query tower over features	LLM context over SID and text tokens
Training	sampled softmax	in-batch softmax + logQ	LLM continued pre-training + fine-tuning
Serving	ANN over item vectors	ANN (quantized MIPS)	beam decoding of SIDs
Cold start	weak (needs ID history)	content features help	content-derived SIDs; shared prefixes
Cost profile	memory-heavy tables, cheap compute	moderate	small tables, heavy compute (TPU)

YouTube Shorts

Public technical detail about the Shorts recommender is limited. YouTube's own help and "How YouTube Works" pages say that the same principles apply: personalization from watch and engagement history, satisfaction signals (likes, skips, "not interested", surveys) and quality and responsibility policies [14][29]. Short-form feeds differ from long-form in ways covered elsewhere in this Atlas: watch time must be normalized by duration (Chapter 11, Kuaishou D2Q), skips are a dense negative label, and the swipe feed is a sequential decision process, which favors the REINFORCE- and slate-style learning in 17.7. The PLUM paper's experiments reportedly include short-form surfaces (approx.) [25]. Beyond that, treat any specific Shorts architecture claim found in secondary sources as unverified.

Google Discover

Google Discover, the feed in the Google app and on mobile google.com, was announced in 2018 as a rebrand of the Google Feed. At that time Google said it reached more than 800 million monthly users (approx.) [21]. Its design as described publicly [21][22]:

- It is a **query-less** feed built from the user's Search and app activity and their declared follows, and it uses the **Knowledge Graph "Topic Layer"** to model interests and how a user's expertise in a topic develops over time.
- It includes evergreen content as well as fresh news, with controls such as "follow topic" and "not interested", which act as explicit feedback.
- The publisher-side documentation says content becomes eligible automatically if it is indexed and meets content policies, and that large, high-quality images (at least 1200 px wide, with max-image-preview:large) increase engagement [22].

No public paper describes Discover's model stack. Its building blocks are presumably those in this chapter, but that is an inference, not a documented fact.

17.11 Infrastructure threads

Several infrastructure papers sit underneath these systems:

- **Large online learners.** Google's 2013 ad-click paper popularized FTRL-Proximal, per-coordinate learning rates and probabilistic feature inclusion for sparse models at very large scale. These ideas are visible in the wide part of Wide & Deep [31][15].
- **Vector search.** ScaNN's anisotropic quantization targets inner-product accuracy for top-k MIPS, the retrieval primitive for every two-tower model above [27].
- **Hardware.** TPU v4 includes SparseCores, dedicated hardware for embedding lookups. Google reports that embeddings are a major share of recommendation workloads and that SparseCores accelerate embedding-heavy models by several times relative to running them on the TensorCores alone (approx.) [28].
- **Engineering practice.** Google's "Rules of Machine Learning" codifies lessons such as logging serving features to avoid training-serving skew and starting with simple heuristics and models [30].

17.12 Comparison table

System (year)	Surface	Stage	Core model	Objective / label	Public scale (approx.)	Signature trick
Google News CF (2007) [19]	News	retrieval + score	MinHash, PLSI, co-visitation	clicks	millions of users	online co-visitation for freshness
YouTube 2010 [1]	Home	full stack	co-visitation graph	clicks	~60% of home clicks from recs	seed-set graph expansion
Wide & Deep (2016) [15]	Play Store	ranking	linear + MLP, joint	app install	1B+ users, 1M+ apps, 500B+ examples	memorization + generalization
YouTube DNN (2016) [2]	Home	retrieval + ranking	MLP tower + sampled softmax; MLP ranker	next watch; watch time	~1B params, 100s of billions of examples	example age; weighted LR

System (year)	Surface	Stage	Core model	Objective / label	Public scale (approx.)	Signature trick
Two-tower NDR (2019) [4]	YouTube	retrieval	two-tower	reward-weighted softmax	tens of millions of videos	streaming logQ correction
Watch Next MTL (2019) [5]	Watch Next	ranking	MMoE + shallow tower	engagement + satisfaction	hundreds of candidates	additive position-bias tower
REINFORCE (2019) [8]	YouTube	retrieval	RNN softmax policy	long-term reward	millions of actions	top-K off-policy correction
SlateQ (2019) [9]	YouTube	ranking	choice model x item Q	long-term engagement	live experiments	slate value decomposition
DPP feed (2018) [12]	Home (mobile)	re-ranking	DPP greedy MAP	long-term satisfaction	windowed feed	learned quality/diversity kernel
DCN-V2 (2020) [18]	many Google LTR systems	ranking	cross network + DNN	task-specific	high QPS	low-rank mixture of crosses
TIGER (2023) [23]	research	retrieval	RQ-VAE + seq2seq	next item	public datasets	semantic IDs
SID ranking (2024) [24]	YouTube	ranking	SID embeddings	ranking tasks	production-scale	SentencePiece over codes
PLUM (2025) [25]	YouTube	retrieval	Gemini-family LLM	next watch (generative)	billions of videos	CPT on SIDs + text

Key takeaways for system designers

1. **The funnel is durable; the components are not.** Google's recommenders kept the same shape from 2016 to 2025: retrieve hundreds from millions or billions, rank with richer features, re-rank for diversity. What changed is each stage's model: co-counts, then DNNs, then two-towers, then RL policies, then LLM-based generative retrieval.
2. **Example and label engineering pays off before architecture.** Example age, per-user caps, predicting the future watch, withholding search order, and reward-weighted losses gave large wins with simple models [2][4].
3. **Choose objectives deliberately and keep revisiting them.** Clicks, then watch time, then survey-predicted valued watch time, plus explicit quality and responsibility policies [14]. Build survey-prediction models so that sparse satisfaction labels can cover all traffic.
4. **Put bias handling in the model.** Use logQ for sampling bias [4], an additive shallow tower for position bias [5], off-policy and top-K corrections for logging-policy bias [8], and uniform negatives for selection bias [16].
5. **Soft sharing for conflicting objectives.** MMoE with per-task gates plus a tunable fusion layer is the baseline multi-objective ranker; watch for gate polarization [5][6].
6. **Make RL compatible with existing serving.** REINFORCE kept the softmax-plus-ANN serving path; SlateQ reused the ranker with long-term values. Both relied on careful variance control and live experimentation [8][9].
7. **Diversity is a set property.** Model it with a set-level re-ranker (DPP), tuned or learned against long-term metrics, not with ad hoc caps [12].

8. **Tokenize items.** Semantic IDs improve generalization within ranking [24], allow generative retrieval [23], and are the bridge that lets pre-trained LLMs work over a catalog of billions of items [25]. Plan for collisions, codebook balance and refresh.
9. **Trade memory for compute knowingly.** Embedding tables (2016-2020) were memory-bound and needed special hardware (SparseCore [28]). LLM-based generative retrieval (2025) is compute-bound. Budget serving cost, latency and caching accordingly.
10. **Ship through A/B tests on long-term metrics.** Nearly every result here was decided online, and Google's long-term experiment methodology warns that short tests can mislead [32].

References

1. James Davidson, Benjamin Liebald, Junning Liu, Palash Nandy, Taylor Van Vleet, Ullas Gargi, Sujoy Gupta, Yu He, Mike Lambert, Blake Livingston, Dasarathi Sampath (Google/YouTube). "The YouTube Video Recommendation System." RecSys 2010. https://dl.acm.org/doi/10.1145/1864708.1864770
2. Paul Covington, Jay Adams, Emre Sargin (Google/YouTube). "Deep Neural Networks for YouTube Recommendations." RecSys 2016. https://research.google/pubs/deep-neural-networks-for-youtube-recommendations/ (PDF: https://static.googleusercontent.com/media/research.google.com/en//pubs/archive/45530.pdf)
3. Adrian Colyer. "Deep neural networks for YouTube recommendations" (paper summary). The Morning Paper, 2016. https://blog.acolyer.org/2016/09/19/deep-neural-networks-for-youtube-recommendations/
4. Xinyang Yi, Ji Yang, Lichan Hong, Derek Zhiyuan Cheng, Lukasz Heldt, Aditee Kumthekar, Zhe Zhao, Li Wei, Ed Chi (Google/YouTube). "Sampling-Bias-Corrected Neural Modeling for Large Corpus Item Recommendations." RecSys 2019. https://research.google/pubs/sampling-bias-corrected-neural-modeling-for-large-corpus-item-recommendations/
5. Zhe Zhao, Lichan Hong, Li Wei, Jilin Chen, Aniruddh Nath, Shawn Andrews, Aditee Kumthekar, Maheswaran Sathiamoorthy, Xinyang Yi, Ed Chi (Google/YouTube). "Recommending What Video to Watch Next: A Multitask Ranking System." RecSys 2019. https://dl.acm.org/doi/10.1145/3298689.3346997
6. Jiaqi Ma, Zhe Zhao, Xinyang Yi, Jilin Chen, Lichan Hong, Ed H. Chi (Google). "Modeling Task Relationships in Multi-task Learning with Multi-gate Mixture-of-Experts." KDD 2018. https://research.google/pubs/modeling-task-relationships-in-multi-task-learning-with-multi-gate-mixture-of-experts/
7. KDnuggets. "How YouTube is Recommending Your Next Video." 2019 (secondary summary of [5]). https://www.kdnuggets.com/2019/10/youtube-recommending-next-video.html
8. Minmin Chen, Alex Beutel, Paul Covington, Sagar Jain, Francois Belletti, Ed H. Chi (Google/YouTube). "Top-K Off-Policy Correction for a REINFORCE Recommender System." WSDM 2019. https://arxiv.org/abs/1812.02353
9. Eugene Ie, Vihan Jain, Jing Wang, Sanmit Narvekar, Ritesh Agarwal, Rui Wu, Heng-Tze Cheng, Tushar Chandra, Craig Boutilier (Google). "SlateQ: A Tractable Decomposition for Reinforcement Learning with Recommendation Sets." IJCAI 2019. https://arxiv.org/abs/1905.12767

10. Konstantina Christakopoulou, Can Xu, Sai Zhang, et al. (Google). "Reward Shaping for User Satisfaction in a REINFORCE Recommender." 2022. https://arxiv.org/abs/2209.15166
11. Eugene Ie, Chih-wei Hsu, Martin Mladenov, et al. (Google Research). "RecSim: A Configurable Simulation Platform for Recommender Systems." 2019; repository. https://github.com/google-research/recsim
12. Mark Wilhelm, Ajith Ramanathan, Alexander Bonomo, Sagar Jain, Ed H. Chi, Jennifer Gillenwater (Google/YouTube). "Practical Diversified Recommendations on YouTube with Determinantal Point Processes." CIKM 2018. https://dl.acm.org/doi/10.1145/3269206.3272018
13. Alex Beutel, Jilin Chen, Tulsee Doshi, Hai Qian, Li Wei, Yi Wu, Lukasz Heldt, Zhe Zhao, Lichan Hong, Ed H. Chi, Cristos Goodrow (Google). "Fairness in Recommendation Ranking through Pairwise Comparisons." KDD 2019. https://arxiv.org/abs/1903.00780
14. Cristos Goodrow (YouTube). "On YouTube's recommendation system." YouTube Official Blog, 2021. https://blog.youtube/inside-youtube/on-youtubes-recommendation-system/
15. Heng-Tze Cheng, Levent Koc, Jeremiah Harmsen, et al. (Google). "Wide & Deep Learning for Recommender Systems." DLRS 2016. https://arxiv.org/abs/1606.07792
16. Ji Yang, Xinyang Yi, Derek Zhiyuan Cheng, et al. (Google). "Mixed Negative Sampling for Learning Two-tower Neural Networks in Recommendations." WWW 2020 Companion. https://research.google/pubs/mixed-negative-sampling-for-learning-two-tower-neural-networks-in-recommendations/
17. Tiansheng Yao, Xinyang Yi, Derek Zhiyuan Cheng, et al. (Google). "Self-supervised Learning for Large-scale Item Recommendations." CIKM 2021. https://arxiv.org/abs/2007.12865
18. Ruoxi Wang, Rakesh Shivanna, Derek Z. Cheng, Sagar Jain, Dong Lin, Lichan Hong, Ed H. Chi (Google). "DCN V2: Improved Deep & Cross Network and Practical Lessons for Web-scale Learning to Rank Systems." WWW 2021. https://arxiv.org/abs/2008.13535
19. Abhinandan Das, Mayur Datar, Ashutosh Garg, Shyam Rajaram (Google). "Google News Personalization: Scalable Online Collaborative Filtering." WWW 2007. https://dl.acm.org/doi/10.1145/1242572.1242610
20. Jiahui Liu, Peter Dolan, Elin Ronby Pedersen (Google). "Personalized News Recommendation Based on Click Behavior." IUI 2010. https://dl.acm.org/doi/10.1145/1719970.1719976
21. Google (Karen Corbin). "Introducing Google Discover." The Keyword (Google blog), 2018. https://blog.google/products/search/introducing-google-discover/
22. Google Search Central. "Google Discover and your website" (documentation). Google, accessed 2026. https://developers.google.com/search/docs/appearance/google-discover
23. Shashank Rajput, Nikhil Mehta, Anima Singh, et al. (Google). "Recommender Systems with Generative Retrieval" (TIGER). NeurIPS 2023. https://arxiv.org/abs/2305.05065
24. Anima Singh, Trung Vu, Nikhil Mehta, et al. (Google/YouTube). "Better Generalization with Semantic IDs: A Case Study in Ranking for Recommendations." RecSys 2024. https://arxiv.org/abs/2306.08121
25. Google / YouTube. "PLUM: Adapting Pre-trained Language Models for Industrial-scale Generative Recommendations." 2025. https://arxiv.org/abs/2510.07784

26. Konstantina Christakopoulou, Alberto Lalama, Cj Adams, et al. (Google). "Large Language Models for User Interest Journeys." 2023. https://arxiv.org/abs/2305.15498
27. Google Research. "Announcing ScaNN: Efficient Vector Similarity Search." Google Research Blog, 2020. https://research.google/blog/announcing-scann-efficient-vector-similarity-search/
28. Norman P. Jouppi, et al. (Google). "TPU v4: An Optically Reconfigurable Supercomputer for Machine Learning with Hardware Support for Embeddings." ISCA 2023. https://arxiv.org/abs/2304.01433
29. YouTube. "How YouTube Works: Recommendations." YouTube, accessed 2026. https://www.youtube.com/howyoutubeworks/product-features/recommendations/
30. Martin Zinkevich (Google). "Rules of Machine Learning: Best Practices for ML Engineering." Google for Developers. https://developers.google.com/machine-learning/guides/rules-of-ml
31. H. Brendan McMahan, Gary Holt, D. Sculley, et al. (Google). "Ad Click Prediction: a View from the Trenches." KDD 2013. https://dl.acm.org/doi/10.1145/2487575.2488200
32. Henning Hohnhold, Deirdre O'Brien, Diane Tang (Google). "Focusing on the Long-term: It's Good for Users and Business." KDD 2015. https://dl.acm.org/doi/10.1145/2783258.2788583

18. Netflix Case Study: From the Netflix Prize to a Foundation Model for Personalization

Netflix is the most thoroughly documented recommender organization in public. It has written about recommendation for close to twenty years: the 2006-2009 Netflix Prize, the 2012 "Beyond the 5 stars" posts, the 2013 online/nearline/offline blueprint, the 2015 ACM TMIS paper on algorithms and business value, the 2015 homepage page-generation post, the 2017 artwork bandits and interleaving posts, the 2019 open-sourcing of Metaflow, the 2024 open-sourcing of Maestro, and the 2025 posts on a single foundation model for personalization. Read in order, these sources describe one long design arc:

1. From **rating prediction** (minimize RMSE on stars) to **ranking for plays** and then to **long-term member satisfaction**.
2. From **many specialized models**, one per row, canvas or surface, to **consolidated multi-task models** and then to **one foundation model** that feeds the rest.
3. From **offline metrics** to a measurement funnel: offline replay, then interleaving, then A/B tests on retention and streaming hours, with proxy rewards checked against the north-star metric.
4. From **ad-hoc batch jobs** to a **human-centric ML platform** (Metaflow) running on a **scalable orchestrator** (Maestro).

This chapter goes through each stage as a system design: the problem, the architecture, the models, the published numbers and the lessons.

A note on sourcing. The live web-search budget for this chapter was used up before research began, and the writing environment's egress proxy blocked direct access to netflixtechblog.com, medium.com, research.netflix.com, arXiv and the ACM Digital Library. The chapter therefore draws on three kinds of source: (a) Netflix primary sources already fetched and summarized by sibling chapters of this Atlas (Chapters 1, 4, 5, 6, 11, 12, 13, 15); (b) canonical Netflix posts and papers whose content is widely reproduced and well established; and (c) a small number of secondary summaries, which are labelled as such. Numbers that could not be re-checked against the primary source in this session are marked "approx." Readers should check exact figures against the linked originals.

---

1. The Netflix Prize (2006-2009) and what Netflix actually took from it

1.1 The competition

In October 2006 Netflix released an anonymized dataset of roughly 100 million movie ratings (1-5 stars) from about 480,000 subscribers on about 17,770 titles. It offered USD 1

million to the first team that beat its in-house algorithm, **Cinematch**, by 10% in root-mean-square error (RMSE) on a held-out "quiz/test" set (approx., as widely reported [1][2]). Progress prizes were paid in 2007 and 2008. The Grand Prize went in September 2009 to "BellKor's Pragmatic Chaos", a merger of the BellKor, BigChaos and Pragmatic Theory teams, which reached about a 10.06% improvement (approx. [2]).

1.2 The winning techniques

The Prize produced the modern latent-factor toolbox [2]:

- **Matrix factorization (MF) with biases.** Rating r_ui is approximated as mu + b_u + b_i + q_i . p_u, where mu is the global mean, b_u and b_i are user and item biases, and p_u and q_i are latent factor vectors learned by regularized SGD or alternating least squares.
- **SVD++.** Adds implicit feedback: the set of items a user rated, whatever the value, contributes to the user representation.
- **timeSVD++.** Lets biases and factors drift over time (for example item popularity changes and per-user rating-scale drift).
- **Neighbourhood models** with learned interpolation weights, **restricted Boltzmann machines (RBMs)**, and finally **large blended ensembles** of hundreds of predictors combined with gradient-boosted blending.

Koren, Bell and Volinsky's IEEE Computer article summarizes the MF family that came out of the competition [2].

1.3 What went into production and what did not

Amatriain and Basilico's "Netflix Recommendations: Beyond the 5 stars (Part 1)" (2012) is the primary source on how the Prize affected production [1]:

- After the 2007 Progress Prize, Netflix took the two strongest single components, **SVD (MF) and RBM**, and put a **linear blend** of the two into production. The post reports that the blend reached an RMSE of about 0.88 (approx. [1]). Netflix had to re-engineer both to scale from the 100M-rating contest dataset to its full rating corpus, which was several billion ratings (approx. [1]), and to retrain as new ratings arrived.
- Netflix **did not deploy the Grand Prize ensemble**. The post says the extra accuracy did not seem to justify the engineering effort of bringing hundreds of blended models into production [1].
- The product had also changed under the competition. Netflix moved from DVD-by-mail to **streaming**. Streaming exposes implicit signals (plays, partial plays, browsing, searches, time of day, device) that matter more than explicit star ratings, and members choose what to watch in seconds, not days ahead [1].

The post states that about **75% of what members watch comes from some form of recommendation** (2012 figure [1]). Part 2 explains the move from rating prediction to **ranking**. A simple ranker combines popularity with predicted rating, for example f(u, v) = w1 * popularity(v) + w2 * predicted_rating(u, v) + b, learned with learning-to-rank methods. Part 2 also describes a richer feature set and the importance of A/B testing over offline metrics [3].

1.4 Lessons

- **Optimizing the wrong objective very well is not enough.** RMSE on stars is a regression proxy. The production goal is choosing the order of titles that members will play and enjoy.
- **Engineering cost is a first-class model metric.** A 10% gain from 100+ blended models lost to a two-model blend that could be retrained and served.
- **Implicit feedback dominates explicit feedback in streaming.** Netflix later replaced five-star ratings with thumbs up/down in 2017. That matches the shift described in [1][3], though the rating change itself is not covered in the sources used here.

---

2. The Netflix recommender as a set of algorithms (TMIS 2015)

Gomez-Uribe and Hunt's "The Netflix Recommender System: Algorithms, Business Value, and Innovation" (ACM TMIS, 2015) is the canonical system-level description [4]. It describes the recommender as a **collection of algorithms**, each serving a different part of the experience:

Algorithm	Purpose	Where it shows up
Personalized Video Ranker (PVR)	Ranks the whole catalog (or a genre subset) per member	Genre rows, e.g. "Suspenseful Movies"
Top-N Video Ranker	Best few titles in the catalog for the member (head of ranking)	"Top Picks" row
Trending Now	Short-term temporal trends blended with personalization	"Trending Now" row
Continue Watching	Ranks recently viewed titles by likelihood of resuming	"Continue Watching" row
Video-Video Similarity ("sims")	Unpersonalized item-item similarity, personalized by choice of anchor	"Because You Watched X" rows, title detail pages
Page Generation	Chooses and orders rows on the homepage	The whole homepage
Evidence selection	Chooses which evidence (synopsis, awards, cast, artwork) to show	Title display
Search	Maps queries to titles, including concepts and unavailable titles	Search page

Key facts from the paper (as summarized in Chapters 12 and 15 [4]):

- About **80% of hours streamed** were influenced by recommendations, and the remaining approx. 20% came from search (approx. [4]).
- A typical homepage had up to about **40 rows**, each with up to about **75 titles**, depending on device (approx. [4]).
- A member usually loses interest after about **60-90 seconds** of browsing, having looked at roughly 10-20 titles. Page composition therefore matters a great deal [4].
- The authors estimate that personalization and recommendations save Netflix **more than USD 1 billion per year** by reducing churn [4].

- **Effective catalog size (ECS)** measures how widely viewing is spread across the catalog. Personalization raises ECS compared with a popularity-only ranking [4].
- The **overall evaluation criterion** for A/B tests is member **retention**. **Streaming hours** serve as a more sensitive engagement proxy that correlates with retention [4].

Design lesson: the "recommender" is not one model but a portfolio. Each algorithm is tied to a UI construct (row, canvas, evidence slot), and a page-level algorithm assembles the portfolio. Section 8 describes how Netflix later collapsed most of this portfolio into fewer models.

---

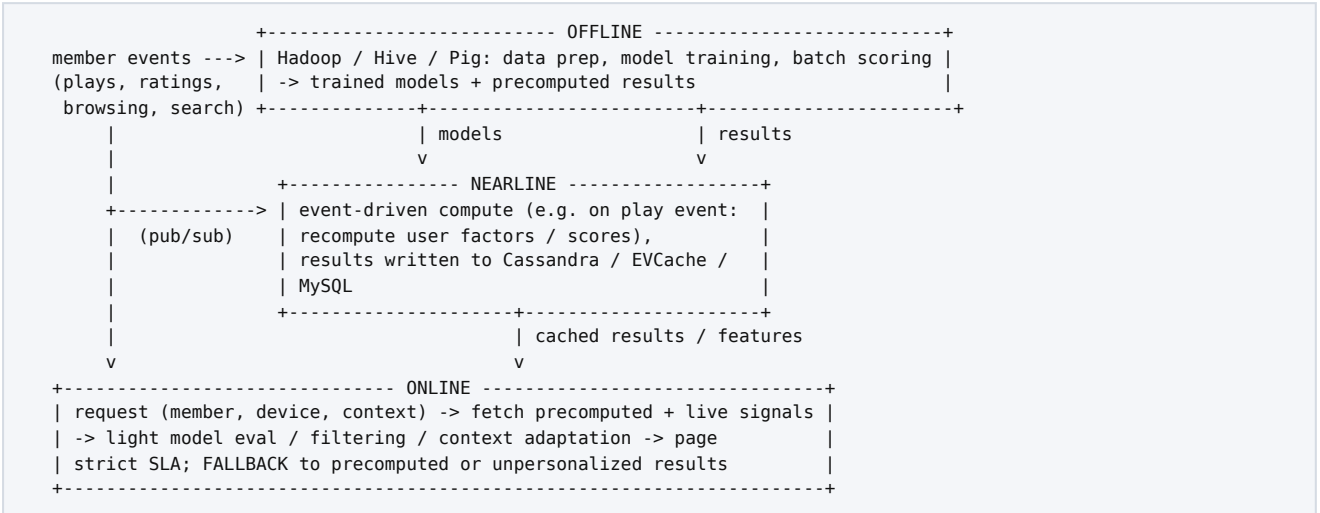
3. System architecture: online, nearline, offline (2013)

3.1 Problem

Netflix needed to combine models that take hours to train on large data (MF, ensembles, later neural networks) with responsiveness to what the member did seconds ago (played a title, added to My List, rated). It also needed to render pages within a strict latency budget and degrade gracefully when components fail.

3.2 The blueprint

Amatriain and Basilico's "System Architectures for Personalization and Recommendation" (March 2013) separates where computation happens from what is computed [5]:



Placement rules from the post [5] (also covered in Chapter 1):

- **Offline** computation has almost no limits on data volume or algorithmic complexity, but its results go stale and cannot react to context. Netflix ran model training and batch pre-computation on Hadoop, with Hive and Pig as query tools.
- **Online** computation responds to the current context and the latest events, but must meet a strict SLA. It needs fallbacks, such as precomputed results, if it times out.
- **Nearline** computation sits in between. It runs in response to events (for example a play) without a request waiting on it, so it can be more complex than online code

while staying fresh. Its results are stored in fast stores (the post names Cassandra, EVCache and MySQL) for online use.
- **Event vs data distinction.** *Events* are small, time-sensitive units that should be processed quickly. The post's internal framework for this was called Manhattan. *Data* is denser information stored for later processing, collected through Chukwa into Hadoop. A publish/subscribe system (Hermes) connects producers and consumers [5].
- **Model training can be split across layers.** Part of a model, such as item factors, can be trained offline, while user-dependent parts are updated nearline or online. Netflix slide decks from the same period describe an MF example: on a play event, the nearline layer re-solves the member's factors, recomputes member-title scores and caches them. The online layer only filters and serves (approx.; described via secondary coverage in Chapter 1).

3.3 Why it lasted

In 2023 Amatriain revisited the blueprint in a "10th anniversary edition" [6]. The split still holds. What changed is that "nearline" grew into streaming feature platforms and near-real-time training, and "offline" grew to include foundation-model pre-training (Section 9).

3.4 Feature generation and training data: DeLorean to Axion

The training data for these models comes from a separate system with its own history (Chapter 11):

- **DeLorean (2016), "time travel".** Netflix periodically snapshotted the online facts that features are computed from (viewing history, My List and similar) for a *sampled* set of contexts (profile, time, device). It called the online microservices through the Prana sidecar and stored the snapshots on S3 as Parquet. Spark jobs then "time travel" to a given timestamp and run the *same* feature encoders that the online system uses. This keeps training and serving consistent and lets new features be evaluated on historical contexts [7].
- **Axion fact store (2022).** Snapshotting was expensive and covered only a sample of contexts. Netflix moved to **logging the facts actually used at compute time**. Axion stores and indexes them in Iceberg tables, with an EVCache layer for low-latency lookups during offline jobs. Offline feature generation reuses the shared encoder library [8].

Lesson: **share feature-encoder code between online and offline, and log what was served.** This is the Netflix answer to training-serving skew.

---

4. Page generation and row selection

4.1 Problem

The Netflix homepage is a two-dimensional grid. Rows run top to bottom, titles run left to right inside a row, and members scan both ways. Each row has a theme ("Because you watched X", "Critically acclaimed dramas", "Trending Now") and an algorithm that ranks

titles inside it. The page-generation problem is: **which rows, in what order, with which titles**, for this member, on this device, now [9].

4.2 Approaches described by Netflix

Alvino and Basilico's "Learning a Personalized Homepage" (2015) [9], as summarized in Chapter 15:

1. **Templates.** Fixed rules such as "row type X at position Y". Simple and predictable, but not personal.
2. **Independent row scoring.** Score every candidate row for relevance and sort. This is personal but produces redundant pages, with similar rows repeating the same titles.
3. **Stage-wise (greedy) construction.** Build the page one row at a time, scoring each candidate row *given the rows already placed*. This captures diversity and deduplication naturally. It is the row-level analogue of maximal marginal relevance (MMR).
4. **Machine-learned page models** that learn row scoring from member interaction data, under business and device constraints.

```
candidate rows (hundreds to thousands of possible row "themes"):
   genre rows (PVR) | Top Picks (Top-N) | Because You Watched (sims)
   Trending | Continue Watching | New Releases | My List | ...
      |      each row independently ranks its own titles
      v
row relevance model  score(row | member, context, device)
      |
      v
stage-wise greedy assembly, for slot k = 1..R:
     choose row maximizing  score(row | rows[1..k-1])
       - dedup titles already shown above (esp. in first visible columns)
       - diversity across row types and genres
       - constraints: e.g. Continue Watching high, row-type min/max,
         device-specific row counts and widths
      |
      v
artwork + evidence selection per (member, title)   -> Section 5
      |
      v
rendered page (top-m rows x top-n columns are what is actually seen)
```

4.3 Page-level evaluation

Because attention is two-dimensional, Netflix described **2-D extensions of ranking metrics**, for example recall over the top-m rows by the top-n columns that a member is likely to see, with discounting by both row and column position [9]. A page that ranks the right titles inside rows can still fail if the right rows sit below the fold.

4.4 Lessons

- **Row selection is a set-function problem.** A row's value depends on what is already on the page, so greedy marginal-gain construction beats independent scoring.
- **Interleaving cannot measure page composition.** It compares two rankers within a row. Changes to page generation still need A/B tests (Section 7) [10].
- **Constraints are product policy.** Device limits and fixed placement of rows such as Continue Watching live in an explicit layer, not inside the learned model.

---

5. Artwork personalization

5.1 From one best image to one best image per member

Each title has several candidate images (artwork). The image is the most visible piece of evidence on the page. Netflix first used A/B tests and explore-exploit to choose a single best image per title for all members (2016) [11]. In 2017 Chandrashekar, Amat, Basilico and Jebara described choosing the image **per member, per title** with **contextual bandits** [12].

5.2 Formulation [12]

- **Arms:** the candidate images for a title.
- **Context:** the member's viewing history and genre preferences, the title, the device, the time and other page context. The post's well-known example is *Good Will Hunting*. A member who watches many romances may see an image of the romantic leads, while a comedy fan may see an image featuring a comedian.
- **Reward:** a **quality play**, meaning the member played the title and watched enough of it, not a raw click. This guards against "clickbait" images that draw plays which are quickly abandoned.
- **Policy:** a model predicts the probability of a quality play for each (member, title, image), with exploration on top. The post discusses several exploration strategies, ranging from simple randomization to Bayesian and upper-confidence approaches.

5.3 Logging and offline evaluation

```
page request -> for each title shown:
   artwork policy (contextual model + exploration)
      -> chosen image_id, propensity p
      -> impression log (member, title, image_id, p, context)
      -> member sees image -> quality play?  -> reward log
offline:
   replay on randomized-exploration logs: keep events where the new
   policy picks the logged image; metric = take fraction (plays/impressions)
online:
   A/B test confirms replay wins
```

New policies are scored offline with **replay** (an unbiased estimator on uniformly explored data) using **take fraction**, then confirmed in A/B tests. The post reports that contextual policies beat non-personalized baselines on replay, and that online tests confirmed the gains [12].

5.4 Engineering issues called out [12]

- **Only one image per title can be shown at a time**, so exploration has a real cost.
- **Page interactions.** The image choice interacts with the other titles and rows. Images should be diverse within a row, and a member should see consistent artwork over time, because a sudden change can confuse.
- **Latency.** Selection runs for every title on every page. Netflix discusses the trade-off between precomputing selections and computing them live (approx.).
- **Supply.** Personalization only works if there is a diverse pool of images per title, which requires creative tooling.

5.5 Generalization: rewards for long-term satisfaction (2024)

The 2024 post "Recommending for Long-Term Member Satisfaction at Netflix" generalizes the bandit view to the whole recommender [13] (Chapters 12 and 13):

- Treat recommendation as a **contextual bandit**. The reward should reflect **long-term member satisfaction**, not the immediate click or play.
- **Reward engineering.** Build a proxy reward from signals such as completion, thumbs up and return visits. Iterate on it as a product decision, and validate it by checking that models trained on it move the north-star (retention-related) metric in online A/B tests.
- **Delayed feedback.** Some signals (for example finishing a series) arrive days later. For recent interactions the missing part of the reward is *predicted*, so the model can still train on fresh data.

---

6. Search

Search accounts for roughly the approx. 20% of streaming not driven by browsing recommendations [4]. The TMIS paper describes Netflix search as a recommendation problem. Search has to handle queries for titles that are not in the catalog, which means recommending related titles that are available, and **concept** queries (for example "French movies"), along with ordinary title and person matching [4].

Netflix's later public work treats search and recommendation as one problem:

- **In-session adaptation.** Netflix research described augmenting search with recommendations adapted to the member's current session. Signals from browsing and partial queries in the session predict intent before or while the member types. This is especially useful on TV devices, where typing is slow (approx.; RecSys 2022 work by Netflix authors [14]).
- **Model consolidation.** The 2023 post "Lessons learnt from consolidating ML models in a large-scale recommendation system" describes merging several related ranking models into **one multi-task model** with the request context as input [15]. Chapter 4 notes that this included search and recommendation tasks. The consolidated model receives a context (which canvas or task, plus the query when there is one) and serves several surfaces. Netflix reports lower maintenance cost and better quality, plus the expected multi-task difficulties such as task interference and shared ownership (Chapter 4; approx.).
- **Foundation model.** The 2025 foundation model (Section 9) is consumed by search as well as by recommendation rows, as one of its listed downstream applications (approx. [16][17]).

Design lesson: when search and browse share the same items, members and intents, a **shared contextual model** with a "query may be empty" input beats two separately engineered stacks.

---

7. Evaluation: offline, interleaving, A/B, and member-level metrics

7.1 The measurement funnel

```
idea / new model
   |
   v
OFFLINE  ranking metrics on held-out plays; replay / OPE on randomized logs
   |      (screening only: offline wins often do not transfer [4][18])
   v
INTERLEAVING (stage 1 online)  team-draft, hours attributed to each ranker;
   |      approx. 100x fewer members than A/B for the same power [10]
   v
A/B TEST (stage 2 online)  ABlaze cells; OEC = retention,
   |      sensitive proxy = streaming hours; proxy rewards validated here
   v
launch -> long-term monitoring; quasi-experiments where no randomization
```

7.2 Offline evaluation and its limits

Gomez-Uribe and Hunt describe offline metrics as screening tools. Final decisions rest on A/B tests, because offline metrics cannot capture the counterfactual "what would the member have played otherwise" [4]. Steck and colleagues' 2021 AI Magazine article, "Deep Learning for Recommender Systems: A Netflix Case Study", reports the same offline-online gap for deep models [18]. It notes that deep learning gave large improvements only once Netflix added many **heterogeneous features** beyond the user-item interaction matrix, and that well-tuned simpler models such as bag-of-items autoencoders stayed strong baselines (approx. [18]). Two examples of Netflix's research on simple models with strong offline results are Steck's **EASE**, a closed-form shallow linear item-item autoencoder [19], and the **calibrated recommendations** reranker, which matches a list's genre distribution to the member's history using KL divergence [20]. Steck et al. also cautioned that the cosine similarity of learned embeddings can be arbitrary because of regularization-induced scale freedom [21]. That matters whenever FM embeddings (Section 9) are used for retrieval.

7.3 Interleaving

Parks, Aurisset and Ramm's 2017 post [10] (Chapter 12):

- **Two-stage process.** Interleaving quickly prunes a large set of candidate rankers to a few, which then go to A/B tests.
- **Team-draft interleaving on rows.** The two rankers "pick" titles in randomized order, so each has an equal chance at each position. This avoids position bias.
- **Attribution by hours streamed** from titles contributed by each ranker, not by clicks.
- **Sensitivity.** In Netflix's simulations, A/B testing needed on the order of **100x more members** than interleaving to reliably detect the same preference (approx., read from a plot [10]).
- **Agreement.** The interleaving preference metric correlated strongly with the A/B engagement metric across the experiments compared, which justified using it as a pruning stage.
- **Limits.** Interleaving measures relative preference between two rankings on one surface. It cannot measure absolute retention effects, page composition or non-

ranking changes.

7.4 A/B testing at member level

- **ABlaze** (2016) is the experimentation front end. Test cells use **batch allocation**, which pre-selects existing members who meet the criteria, or **real-time allocation** at a qualifying action such as sign-up or first use of a device. A test-allocation service tells clients which cell a member is in [22].
- **Unit of analysis.** Randomization, and therefore analysis, is at the member (account/profile) level. Metrics such as retention and streaming hours are aggregated **per member** first and then compared across cells. That keeps heavy viewers from dominating the comparison and matches the business outcome, a member staying or leaving [4][22].
- **Analysis stack** (2019). Netflix rebuilt experimentation analysis as a modular Python/R stack with a shared library of "causal models", such as quantile bootstrapping and regression-based variance reduction, that apply across metrics [23].
- **Beyond randomization.** A 2022 survey describes quasi-experiments, synthetic control and heterogeneous-treatment-effect methods for cases where A/B tests are impossible [24].
- **Long-term proxies.** Retention barely moves within a few weeks, so Netflix uses streaming hours and engineered proxy rewards and checks them against retention [4] [13].

---

8. Consolidation: from many models to few

By the early 2020s Netflix had dozens of ranking models, one per row type and canvas: homepage rows, "More like this", notifications, search and others. Each had its own pipeline, features and owners. The 2023 consolidation post [15] describes the first step toward fewer models:

```
BEFORE                                AFTER
canvas A -> model A (features A)      canvas A --+
canvas B -> model B (features B)      canvas B --+--> one multi-task model
canvas C -> model C (features C)      canvas C --+    input: (member, context,
search   -> model S (features S)      search   --+     task/canvas id, query?)
N pipelines, N on-calls               1 pipeline, shared features, shared infra
```

Reported benefits include less maintenance, knowledge transfer between tasks and faster iteration. Reported costs include multi-task interference, bigger blast radius and harder ownership (Chapter 4; approx. [15]). The foundation model in Section 9 takes this further. It centralizes **member representation learning**, not only ranking.

---

9. Foundation model for personalization (2025)

9.1 Problem

Even after consolidation, many Netflix models were trained separately on overlapping interaction data. Examples include "Continue Watching", "Today's Top Picks", search and notifications. Each had its own feature engineering, so an improvement to one model's understanding of members did not transfer to the others. In March 2025 Hsiao, Feng and Lamkhede described a **foundation model (FM) for personalized recommendation** in the style of large language models: one large, centrally trained model that learns member preferences from complete interaction histories and feeds many downstream systems [16].

9.2 Design (Chapters 5 and 6 summaries of [16], plus secondary sources [25][26])

- **Scale.** An autoregressive Transformer trained on **hundreds of billions of interactions** from **more than 300 million members**. That is roughly the token scale of LLM pre-training [16][25].
- **Interaction tokenization.** Raw events are merged into meaningful tokens. For example, consecutive related actions on the same title are grouped, much as byte-pair encoding (BPE) merges characters. This balances sequence length against detail. Each token carries heterogeneous attributes such as action type, duration, device and time [16].
- **Long context.** Lifetime histories are very long. Training uses **sparse attention** and **sliding-window sampling** over each history. Netflix published the sliding-window method as a RecSys 2024 paper, "Sliding Window Training: Utilizing Historical Recommender Systems Data for Foundation Models" [27]. Inference uses KV caching (approx. [25]).
- **Objectives.** Next-item prediction plus **auxiliary attribute targets** such as genre and original language, and **multi-token prediction** of several future items. Predicting further ahead reduces myopic, short-term behaviour [16].
- **Cold start.** Item representations combine a learnable **ID embedding** with a **metadata/content embedding**. A gate weighted by item "age" lets new titles rely on metadata until they have interactions. The model is retrained incrementally so that new IDs can be added without starting over (approx. [16][26]).
- **Embedding stability.** Downstream consumers need comparable vectors across retrains. Netflix describes aligning successive embedding spaces with a transformation (described as orthogonal) so consumers do not break (approx. [16][26]).
- **Scaling behaviour.** Netflix reports that quality kept improving as it scaled from millions to billions of parameters and as data grew, consistent with scaling laws. It also notes that serving latency and cost limit how large a model can go into production (approx. [16][25]).

```
member event streams (plays, browses, searches, thumbs, My List, ...)
     |
     v
interaction tokenizer: merge adjacent related events; attach attributes
     |  (sequence of tokens per member, lifetime history)
     v
+-------------------- FOUNDATION MODEL (offline, large GPU training) ----+
| item emb = gate(age) * ID emb + (1-gate) * metadata emb                 |
| causal Transformer, sparse attention, sliding-window training           |
| heads: next item | multi-token future items | genre / language aux      |
+-----------+-----------------------+-------------------------+----------+
            |                       |                         |
            v                       v                         v
     (1) direct use           (2) embeddings             (3) fine-tune /
     next-title scores        member + title vectors     subgraph in
     as features / ranker     -> embedding store          downstream models
            |                       |                         |
            +-----------> rows, search, notifications, page gen <------+
```

9.3 Integration patterns (2025 follow-up post [17])

Netflix's follow-up post, "Integrating Netflix's Foundation Model into Personalization Applications", lists three ways downstream teams consume the FM (Chapter 5):

Pattern	How it works	Freshness	Downstream cost	Best for
Embeddings	Precomputed member/title embeddings published to an embedding store and used as features or for retrieval	Batch refresh (can be stale)	Lowest	Many consumers, quick adoption
Subgraph	FM layers embedded in the downstream model and trained or served with it	Fresh at request time	Must build FM inputs (token sequences)	Latency-tolerant rankers that need freshness
Fine-tuning	FM fine-tuned for a specific task or surface	Task-specific	Highest compute and ops	High-value surfaces

9.4 Lessons

- **The gain is partly organizational.** One representation learner with stable embedding contracts replaces duplicated feature engineering across teams.
- **Tokenization is the recsys equivalent of vocabulary design.** Merging events changes what "context length" means and what the model can learn.
- **Cold start and ID churn need structure.** Metadata-gated embeddings and incremental training are required in a catalog where new titles launch every week.
- **Version the embedding space**, not just the model.

---

10. ML infrastructure: Metaflow and Maestro

10.1 Metaflow

Netflix open-sourced **Metaflow** in December 2019 as a "human-centric framework for data science" [28][29]. The design goals were to let data scientists write plain Python while the framework handles infrastructure:

- **Flows as DAGs of steps.** A Python class whose methods are decorated `@step`. Branching (`foreach`) supports fan-out, for example one training job per hyperparameter or per region.
- **Automatic artifact versioning.** Every instance variable assigned in a step is persisted (to S3 in the AWS deployment) and versioned per run. Any past run can be inspected or resumed.
- **Compute decorators.** `@resources` and `@batch` (later also `@kubernetes`) move a step to cloud compute with given CPU, GPU and memory. No code changes are needed between laptop and cluster.
- **Production deployment.** Flows can be exported to production schedulers. Metaflow's documentation lists AWS Step Functions and Argo Workflows, and inside Netflix flows run on Netflix's own orchestrator (Maestro) [29][30].

Netflix's 2024 post "Supporting Diverse ML Systems at Netflix" describes Metaflow as the common layer for ML projects across the company, including recommendation work. It mentions extensions for fast data access, model hosting and orchestration integration (approx.; URL not verified in this session, so not cited).

10.2 Maestro

Maestro is Netflix's horizontally scalable workflow orchestrator. It was open-sourced in 2024 (Apache-2.0) [30]. It schedules both data pipelines and ML workflows. It supports DAG and cyclic workflow patterns, foreach loops, subworkflows, parameterized and event-triggered runs, and several step types (for example containers, notebooks and Metaflow flows) [30]. Within the recommender stack it runs the daily or periodic jobs: feature generation (Axion), model training, FM embedding refresh and publishing, and batch scoring.

```
Maestro (schedule / event trigger)
   -> Metaflow flow: fetch facts (Axion) -> build features (shared encoders)
      -> train (GPU, @resources) -> evaluate offline (replay/ranking metrics)
      -> publish model / embeddings (embedding store, aligned space)
   -> online services pick up new versions -> interleaving -> A/B (ABlaze)
```

10.3 Lessons

- Make the **easy path the right path**. Versioned artifacts and the same code on a laptop and in production remove much accidental complexity.
- **Separate orchestration from authoring.** Data scientists write flows, and the platform team owns a scalable scheduler.

---

11. Comparison of Netflix recommender designs across eras

Era / design	Problem	Core model	Data / signals	Serving placement	Evaluation	Ref
Netflix Prize (2006-09)	Predict star ratings	MF/SVD++, RBM, blends	~100M ratings (contest)	Offline batch	RMSE	[1][2]

Era / design	Problem	Core model	Data / signals	Serving placement	Evaluation	Ref
Production post-Prize (2012)	Rank for plays	SVD+RBM blend, LTR	Billions of ratings + plays	Offline + online	Offline + A/B	[1][3]
Architecture blueprint (2013)	Freshness vs. latency	Any (MF example)	Events + data	Offline / nearline / online	n/a	[5]
Algorithm portfolio (2015)	Many UI constructs	PVR, Top-N, sims, trending	Plays, context	Mostly precomputed rows	Retention OEC	[4]
Page generation (2015)	Choose and order rows	Row scoring + stage-wise greedy	Row interactions	Online assembly	2-D page metrics, A/B	[9]
Artwork (2017)	Image per member-title	Contextual bandit	Impressions + quality plays	Per-title at page time	Replay take fraction, A/B	[12]
Interleaving (2017)	Faster ranker selection	Team-draft	Hours streamed	Online split	~100x sensitivity (approx.)	[10]
Consolidation (2023)	Too many models	Multi-task, context input	Shared features	Shared service	A/B	[15]
Long-term reward (2024)	Myopic objectives	Bandit with proxy reward	Delayed satisfaction signals	n/a	Proxy vs. north star	[13]
Foundation model (2025)	Duplicate representation learning	Autoregressive Transformer	100s of billions of interactions, 300M+ members	Offline FM -> embeddings/subgraph/fine-tune	Downstream A/B	[16] [17] [27]

12. Key takeaways for system designers

1. **Choose the objective before the model.** The Prize optimized RMSE on stars. Production needed ranking for plays, and later long-term satisfaction. Treat reward design as a product decision and validate proxies against the north-star metric [1] [13].
2. **Engineering cost is part of model quality.** Netflix shipped a two-model blend, not the 100+-model Grand Prize ensemble [1]. Prefer designs you can retrain, debug and serve.
3. **Make placement explicit for every component: offline, nearline or online.** Precompute what you can, update on events what must be fresh, and always have an online fallback [5].
4. **Share feature encoders across training and serving, and log the facts you served.** DeLorean's time travel was superseded by Axion's fact logging [7][8].
5. **Optimize the page, not just the list.** Homepage rows interact. Use stage-wise marginal-gain construction with dedup and diversity, evaluate with 2-D metrics, and keep constraints in an explicit policy layer [9].
6. **Personalize the evidence as well as the choice.** The artwork bandit shows that how an item is presented is a recommendation problem in its own right. Log propensities, keep a randomized slice and evaluate with replay before A/B [12].

7. **Build a measurement funnel.** Use offline screening, then interleaving (approx. 100x more sensitive) to prune, then A/B tests on member-level retention and streaming hours as the final decision. Know what each stage cannot measure [4][10][22].
8. **Expect the offline-online gap.** Strong simple baselines (EASE, calibration) and heterogeneous features matter more than architecture novelty [18][19][20].
9. **Consolidate, then centralize.** Merge sibling models into multi-task models, then pre-train a foundation model of member behaviour that feeds many consumers through embeddings, subgraphs or fine-tuning [15][16][17].
10. **Treat embeddings as an API.** Version and align embedding spaces across retrains, handle cold start with metadata-gated item embeddings, and be careful with cosine similarity on learned embeddings [16][21].
11. **Invest in a human-centric ML platform.** Metaflow-style versioned flows on a scalable orchestrator (Maestro) make iteration speed a property of the platform [28][29][30].

References

1. Xavier Amatriain and Justin Basilico (Netflix). "Netflix Recommendations: Beyond the 5 stars (Part 1)." Netflix Technology Blog, 2012. https://netflixtechblog.com/netflix-recommendations-beyond-the-5-stars-part-1-55838468f429
2. Yehuda Koren, Robert Bell, Chris Volinsky. "Matrix Factorization Techniques for Recommender Systems." IEEE Computer 42(8), 2009. https://doi.org/10.1109/MC.2009.263
3. Xavier Amatriain and Justin Basilico (Netflix). "Netflix Recommendations: Beyond the 5 stars (Part 2)." Netflix Technology Blog, 2012. https://netflixtechblog.com/netflix-recommendations-beyond-the-5-stars-part-2-d9b96aa399f5
4. Carlos A. Gomez-Uribe and Neil Hunt (Netflix). "The Netflix Recommender System: Algorithms, Business Value, and Innovation." ACM Transactions on Management Information Systems 6(4), 2015. https://dl.acm.org/doi/10.1145/2843948
5. Xavier Amatriain and Justin Basilico (Netflix). "System Architectures for Personalization and Recommendation." Netflix Technology Blog, 2013. https://netflixtechblog.com/system-architectures-for-personalization-and-recommendation-e081aa94b5d8
6. Xavier Amatriain. "Blueprints for recommender system architectures: 10th anniversary edition." Machine Learning Times, 2023. https://www.predictiveanalyticsworld.com/machinelearningtimes/blueprints-for-recommender-system-architectures-10th-anniversary-edition/12879/
7. Netflix Technology Blog. "Distributed Time Travel for Feature Generation." Netflix, 2016. https://netflixtechblog.com/distributed-time-travel-for-feature-generation-389cccdd3907
8. Netflix Technology Blog. "Evolution of ML Fact Store." Netflix, 2022. https://netflixtechblog.com/evolution-of-ml-fact-store-5941d3231762
9. Chris Alvino and Justin Basilico (Netflix). "Learning a Personalized Homepage." Netflix Technology Blog, 2015. https://netflixtechblog.com/learning-a-personalized-homepage-aa8ec670359a

10. Joshua Parks, Juliette Aurisset, Michael Ramm (Netflix). "Innovating Faster on Personalization Algorithms at Netflix Using Interleaving." Netflix Technology Blog, 2017. https://netflixtechblog.com/interleaving-in-online-experiments-at-netflix-a04ee392ec55
11. Gopal Krishnan (Netflix). "Selecting the best artwork for videos through A/B testing." Netflix Technology Blog, 2016. https://netflixtechblog.com/selecting-the-best-artwork-for-videos-through-a-b-testing-f6155c4595f6
12. Ashok Chandrashekar, Fernando Amat, Justin Basilico, Tony Jebara (Netflix). "Artwork Personalization at Netflix." Netflix Technology Blog, 2017. https://netflixtechblog.com/artwork-personalization-c589f074ad76
13. Netflix Technology Blog. "Recommending for Long-Term Member Satisfaction at Netflix." Netflix, 2024. https://netflixtechblog.com/recommending-for-long-term-member-satisfaction-at-netflix-ac15cada49ef
14. Moumita Bhattacharya, Sudarshan Lamkhede et al. (Netflix). "Augmenting Netflix Search with In-Session Adapted Recommendations." RecSys 2022 (arXiv; identifier not re-verified in this session). https://arxiv.org/abs/2206.02254
15. Netflix Technology Blog. "Lessons learnt from consolidating ML models in a large scale recommendation system." Netflix, 2023. https://netflixtechblog.medium.com/lessons-learnt-from-consolidating-ml-models-in-a-large-scale-recommendation-system-870c5ea5eb4a
16. Ko-Jen Hsiao, Yesu Feng, Sudarshan Lamkhede (Netflix). "Foundation Model for Personalized Recommendation." Netflix Technology Blog, 2025. https://netflixtechblog.com/foundation-model-for-personalized-recommendation-1a0bd8e02d39
17. Netflix Technology Blog. "Integrating Netflix's Foundation Model into Personalization Applications." Netflix, 2025. https://netflixtechblog.medium.com/integrating-netflixs-foundation-model-into-personalization-applications-cf176b5860eb
18. Harald Steck, Linas Baltrunas, Ehtsham Elahi, Dawen Liang, Yves Raimond, Justin Basilico (Netflix). "Deep Learning for Recommender Systems: A Netflix Case Study." AI Magazine 42(3), 2021. https://doi.org/10.1609/aimag.v42i3.18140
19. Harald Steck (Netflix). "Embarrassingly Shallow Autoencoders for Sparse Data." The Web Conference (WWW) 2019. https://arxiv.org/abs/1905.03375
20. Harald Steck (Netflix). "Calibrated Recommendations." RecSys 2018. https://dl.acm.org/doi/10.1145/3240323.3240372
21. Harald Steck, Chaitanya Ekanadham, Nathan Kallus (Netflix). "Is Cosine-Similarity of Embeddings Really About Similarity?" 2024. https://arxiv.org/abs/2403.05440
22. Netflix Technology Blog. "It's All A/Bout Testing: The Netflix Experimentation Platform." Netflix, 2016. https://netflixtechblog.com/its-all-a-bout-testing-the-netflix-experimentation-platform-4e1ca458c15
23. Netflix Technology Blog. "Reimagining Experimentation Analysis at Netflix." Netflix, 2019. https://netflixtechblog.com/reimagining-experimentation-analysis-at-netflix-71356393af21
24. Netflix Technology Blog. "A Survey of Causal Inference Applications at Netflix." Netflix, 2022. https://netflixtechblog.com/a-survey-of-causal-inference-applications-at-netflix-b62d25175e6f
25. ZenML LLMOps Database. "Netflix: Foundation Model for Large-Scale Personalized Recommendation" (secondary summary). 2025. https://www.zenml.io/llmops-database/foundation-model-for-large-scale-personalized-recommendation

26. Madan Mohan Venkatesan. "Foundation Model for Personalized Recommendation (H&M Dataset)" (open reimplementation mapping the Netflix design; secondary). 2026. https://github.com/madanmohan-venkatesan/Recommender-systems-on-HNM-fashion-dataset
27. Netflix authors. "Sliding Window Training: Utilizing Historical Recommender Systems Data for Foundation Models." RecSys 2024. https://arxiv.org/abs/2409.14517
28. Netflix Technology Blog. "Open-Sourcing Metaflow, a Human-Centric Framework for Data Science." Netflix, 2019. https://netflixtechblog.com/open-sourcing-metaflow-a-human-centric-framework-for-data-science-fa72e04a5d9
29. Netflix / Outerbounds. "Metaflow" (source repository and documentation). https://github.com/Netflix/metaflow and https://docs.metaflow.org
30. Netflix. "Maestro: Netflix's workflow orchestrator" (source repository). 2024. https://github.com/Netflix/maestro

19. Meta Case Study: From News Feed Ranking to Andromeda, Lattice, GEM and Generative Recommenders

> **Sourcing note.** This chapter was written when live web search and fetch were not available to the research agent: the shared search budget was used up and the egress proxy refused arXiv and Meta domains. All facts below come from primary and secondary sources that other chapters of this Atlas had already retrieved and checked (Chapters 1-4, 6, 9-11, 15, 16). Their URLs are listed in the References. Topics that could not be checked against a fetched source in this run (Threads feed ranking, Facebook Marketplace, Instagram Reels specifics, the "Scaling User Modeling" paper, MTIA chip internals) are marked as gaps in Section 9 and are not described in detail. Numbers marked "approx." are secondhand or rounded.

Meta runs some of the largest recommender systems in the world: Facebook Feed, Instagram Feed, Explore, Reels and Stories, Threads, Marketplace, and an ads system that sits behind all of these surfaces. Meta has also published more of its recommender architecture than almost any other company. It released DLRM (the reference deep recommendation model for the industry), TorchRec and FBGEMM (the open-source training and serving substrate), a series of ranking-architecture papers (DHEN, Wukong, InterFormer), the generative-recommender line (HSTU and its successors), and engineering posts on Feed, Explore, Lattice, Andromeda and GEM.

Across about a decade (2014-2026), the published record shows five changes:

1. **From features to embeddings.** The 2014 ads system used GBDT leaf features with online logistic regression [16]. DLRM (2019) made sparse-embedding plus MLP models the default, and the cost of embeddings drove the hardware [9][12].
2. **From single objectives to value models.** Feed and Explore predict many actions with multitask networks and combine them in an explicit, tunable value function [1] [7].
3. **From many models to few.** Lattice merged the fragmented ads model fleet into multi-domain, multi-task models [22][23].
4. **From cheap retrieval to deep retrieval.** Andromeda put a deep network with a learned hierarchical index on GPUs at the stage that sees the most items [25].
5. **From discriminative to generative and foundation models.** HSTU recast recommendation as sequence transduction [30], and GEM became an LLM-scale teacher that is distilled into the whole ads fleet [27][28].

```
2014   GBDT + LR ads CTR (online joiner, NE metric)                      [16]
2018   MSI ("meaningful social interactions") Feed value model           [3]
2019   DLRM open-sourced; Instagram Explore 3-pass distillation funnel    [9][5]
2020   Instagram constrained exploration; FB Search embedding retrieval   [6][37]
2021   Feed ranking explainer; survey-weighted objectives; Neo/ZionEX     [1][2][12]
2022   DHEN; DSI data-ingestion paper; TorchRec (RecSys)                  [17][15][13]
2023   Lattice (ads model consolidation); Explore scaled to 4 stages     [22][7]
2024   HSTU generative recs; Wukong; InterFormer; Andromeda   [30][19][20][25]
2025   GEM ads foundation model; Lattice paper; DHEN for CVR              [27][23][18]
2026   GEM trained at LLM scale (MFU 20-25%); ULTRA-HSTU                  [28][34]
```

---

1. Facebook News Feed ranking

1.1 Problem

Feed has to order, for each of more than 2 billion people, the posts from friends, Pages and Groups they follow (and later recommended content), in real time, so that the top of Feed shows what that person will find most valuable [1][4]. An average user has more than 1,000 candidate posts available on a given visit [1].

1.2 Architecture: multi-pass ranking on "predictors"

Meta's January 2021 engineering post describes ranking as a series of passes run in parallel on serving machines that Meta calls *predictors* [1]:

```
inventory: all eligible posts for the viewer (> 1,000 on average)
      |
      v
pass 0: integrity processes + eligibility (remove violating / ineligible content)
      |
      v
pass 1: lightweight model narrows inventory to ~500 most relevant posts
      |
      v
pass 2: heavy multitask neural networks predict P(action_i) per post
        (like, comment, share, hide, "worth your time" survey answer, ...)
        -> combined into one value score per post
      |
      v
pass 3: contextual re-ranking (e.g. content-type diversity, avoid runs of
        similar posts) -> final ordered Feed
                                                  (after Meta 2021 [1][4])
```

The structure is the canonical funnel: cheap eligibility filters first, then a light pass, a heavy pass and a contextual pass. Running integrity first saves heavy-model compute for content that can actually be shown [1].

1.3 Models and the value function

Each post receives several predicted actions from multitask neural networks, and these "all need to somehow aggregate up to a single value" [1]. The 2021 post uses a running example (a user named Juan) to show that different predictions can disagree about which post Juan will respond to most. The tie is broken by weighting each predicted interaction by how *meaningful* it is, based on survey data about what people say they value [1].

Conceptually:

```
value(post) = sum over events i of  w_i * P(event_i | viewer, post, context)
```

The weights are a product policy, set from data analysis and surveys, not learned end to end.

1.4 MSI and survey-calibrated objectives

- **Meaningful Social Interactions (2018).** Internal documents reported by CNN in 2021 describe MSI as a point system. Likes, comments, re-shares, reactions and event RSVPs each had point values, adjusted for the relationship between the people involved. The weights came from surveys (one cited survey covered more than 69,000 people in five countries) together with analysis of user data [3]. A 2019 internal note said MSI was "beneficial for most metrics" in 2018. The same reporting describes how heavy weights on comments, reshares and reaction emojis also amplified outrage-provoking content [3].
- **Survey targets (2019-2021).** Meta added "Is this post worth your time?" surveys. In 2021 it gave more weight to survey-derived predictions of how meaningful a friend, Page or Group is to a person. Classifiers trained to predict survey answers carry the signal to posts nobody was surveyed about [2].
- **Transparency.** Meta publishes a "system card" for the Facebook Feed AI system that lists the main prediction signals and the stages of the pipeline [4].

1.5 Lessons

- Keep prediction (calibrated multitask heads) separate from policy (a small, versioned value function). Product trade-offs then change through configuration, not retraining.
- Engagement-only weights optimize what is easy to measure, with second-order effects on the content ecosystem. Survey-predicted value is a counterweight [2][3].

---

2. Instagram Explore

Explore is Instagram's recommendation surface for content from accounts the user does not follow. Meta has described it publicly three times (2019, 2020, 2023), which makes it one of the best-documented recommender systems anywhere.

2.1 Explore 2019: ig2vec and the three-pass distillation funnel

The 2019 post "Powered by AI: Instagram's Explore recommender system" says the system "extracts 65 billion features and makes 90 million model predictions every second" [5].

Candidate generation. Explore starts from *accounts*, not individual media. ig2vec is a word2vec-style framework that treats the sequence of account IDs a user interacts with in a session as a sentence and learns account embeddings from it [5]. Seed accounts (accounts the user recently engaged with) are expanded by nearest-neighbor lookup in ig2vec space into candidate accounts, and their media become candidates [5].

Ranking funnel.

```
seed accounts --ig2vec KNN--> candidate accounts --> their media (+ other sources)
          |
          v
 ~500 candidates
          |   pass 1: distillation model (minimal features), trained to mimic the
          |           combined output of the later stages
          v
 ~150 candidates
          |   pass 2: lightweight neural network
          v
 ~50 candidates
          |   pass 3: full deep neural network (all dense features)
          v
 ~25 shown first (after integrity and diversity rules)
                               (after Instagram 2019 [5]; counts approx.)
```

Key trick. The first pass is a *distilled* model that imitates the heavy stages with a fraction of the features. Distillation keeps the stages aligned: the cheap stage keeps what the expensive stage would have chosen, rather than what an independent proxy objective prefers [5].

2.2 Explore 2020: constrained exploration and the log-linear value model

The 2020 post "How Instagram suggests new content" describes the multi-label ranking models and how their outputs are combined [6]. The models predict positive engagement (like, save) and negative feedback ("see less", "not interested"). Point-wise heads are trained with cross-entropy, alongside GBDTs and list-wise LambdaRank models. The value model is **log-linear** [6]:

```
Value(post) = P(like) ^ w_like  *  (1 - P(not interested)) ^ w_see_less
```

The weights are tuned with offline replay over user sessions and with online Bayesian optimization, and they are retuned often as the system changes [6]. In a multiplicative form, a high predicted negative probability can veto an item however likely a like is.

2.3 Explore 2023: four stages and "caching everywhere"

The August 2023 post "Scaling the Instagram Explore recommendations system" (Vorotilov and Shugaepov) redraws the system as four stages: **retrieval, first-stage ranking, second-stage ranking and final re-ranking** [7]. Its stated "leitmotif" is the "clever use of caching and pre-computation in different ranking stages". That is what made heavier models affordable at every stage and reduced the use of heuristics [7].

Retrieval. Many sources run in parallel: heuristics (for example trending posts or content from followed authors) and ML sources that capture real-time and pre-generated interests. They are mixed with tunable weights, so different user groups get different blends [7]. Meta's Explore transparency card lists source types: author-based (media from accounts you engaged with), media-based (media similar to media you engaged with), item collaborative filtering, personalized PageRank, and two-tower sparse-network sourcing. Each source picks hundreds of items from a pool of billions [8]. The main ML source is a two-tower network. Because the towers are independent after training, item embeddings are computed offline (typically daily) and indexed. At request time, a fresh user embedding is computed and an ANN search returns similar items [7].

First-stage ranking. This stage reuses the two-tower architecture "because of its cacheability property". It is trained to predict the output of the second-stage ranker, which is a form of knowledge distillation between stages [7]. The transparency card gives the funnel sizes: retrieval returns up to **1,500** media, and the early-stage two-tower ranker passes the top **100** onward [8].

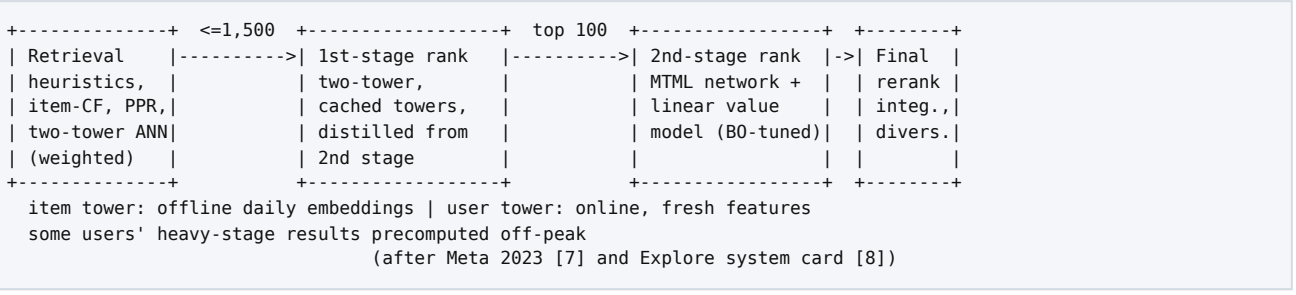
Second-stage ranking. A heavier **multi-task multi-label (MTML)** network with user-item interaction features predicts probabilities of several events. Its outputs are combined by a now explicitly **linear** value model [7]:

```
Expected Value = W_click * P(click) + W_like * P(like) - W_see_less * P(see less) + ...
```

Tuning the weights lets the team "explore different tradeoffs between online engagement metrics" [7]. Because the MTML model is much more expensive than the two-tower models, Meta precomputes some users' recommendations during off-peak hours [7].

Final re-ranking. Integrity filters plus business rules. Examples are diversity rules that penalize repeated posts from the same author or seed account [7].

Weight tuning. The post contrasts offline tuning with Bayesian optimization, in which engineers specify only the parameters, the goal metric and regression thresholds on guardrail metrics [7].



2.4 Explore over time

Aspect	2019 [5]	2020 [6]	2023 [7][8]
Candidate generation	ig2vec account KNN + other sources	(not focus)	weighted mix of heuristic, item-CF, PPR, two-tower ANN
Stage count	sourcing + 3 ranking passes	multi-label rankers	retrieval + 2 ranking stages + rerank
Funnel sizes	~500 -> 150 -> 50 -> 25	n/a	<=1,500 -> 100 -> final
Cheap stage	distilled light model	n/a	two-tower distilled from 2nd stage
Heavy stage	full DNN	multi-label NN + GBDT + LambdaRank	MTML network
Value model	(not detailed)	log-linear, BO-tuned	linear, BO with guardrails
Key infra idea	distillation	offline replay + online BO	caching and precomputation

2.5 Lessons

- **Distil downward.** Both the 2019 and 2023 designs train the cheap stage to imitate the expensive one. Funnel consistency matters more than the cheap stage's own

accuracy.

- **Pick the value-function form on purpose.** Linear is additive and easy to interpret. Log-linear lets one negative signal act as a veto.
- **Precompute what you can.** Daily item towers, cached user towers and off-peak heavy scoring buy model capacity without breaking the latency budget.

---

3. DLRM and the infrastructure it created

3.1 DLRM (2019)

"Deep Learning Recommendation Model for Personalization and Recommendation Systems" (Naumov et al.) defined the reference architecture that most industrial training-systems work assumes [9][10]:

```
dense features            sparse features (multi-hot IDs)
     |                       |        |        |
+---------+             +------+ +------+ +------+
| bottom  |             | EMB  | | EMB  | | EMB  |   <- almost all params,
|  MLP    |             | T1   | | T2   | | Tk   |      model-parallel
+---------+             +------+ +------+ +------+
     |                      | pool   | pool   | pool
     +----------+-----------+--------+--------+
                |  pairwise interaction (dot products / concat)
           +---------+
           | top MLP |   <- data-parallel
           +---------+
                |
            P(click)
```

DLRM deliberately stops at second-order (pairwise dot-product) interactions between embedding vectors, for cost reasons. The open-source code supports synchronous distributed training (gloo, NCCL, MPI) and includes MLPerf benchmark tooling [10].

Why it shaped hardware. Meta's study of production DLRM training found that embedding operators are memory-bound and dominated by communication, and that the variety of table sizes and sparse-feature profiles makes a single hardware configuration a poor fit for all models [11]. Embedding tables, not FLOPs, set the cost.

3.2 Neo and ZionEX (2021-2022): synchronous training of trillion-parameter models

Before about 2020, Meta trained recommendation models asynchronously on CPU parameter-server clusters. Asynchrony hurt quality and reproducibility, and parameter-server traffic limited throughput [11]. Neo/ZionEX moved this to synchronous GPU training [12]:

- **ZionEX nodes:** 8 GPUs (A100-class in the paper) on NVLink, plus a dedicated RoCE scale-out network with several NICs per node, so every GPU has its own path for inter-node all-to-all traffic [12].
- **4D embedding parallelism** chosen per table: table-wise, row-wise, column-wise and data-parallel (replicated) [12].

- **Hybrid kernel fusion** (all tables on a device in one kernel, with the sparse optimizer update fused into backward), a cost-model **sharding planner**, software-managed embedding caching across HBM, DRAM and UVM, and **quantized collectives** for embedding all-to-all traffic [12].
- Reported scale: models of up to **12 trillion parameters**, and time-to-solution speedups of up to roughly 40x over the earlier CPU asynchronous setup (approx., headline figure) [12].

3.3 TorchRec, FBGEMM and the data pipeline

- **TorchRec** productionizes these ideas as a PyTorch library. It provides data-parallel, table-wise, row-wise, column-wise and hybrid sharders, an automatic planner, a pipelined training loop that overlaps the ID all-to-all with dense compute, and C++ inference export. The project says it "powers many production RecSys models at Meta" [13].
- **FBGEMM / FBGEMM_GPU** supplies table-batched embedding (TBE) kernels with fused optimizers, plus low-precision GEMMs with row-wise and outlier-aware quantization for inference [14].
- **Data Storage and Ingestion (DSI, ISCA 2022).** Features and events are logged through Scribe, joined with labels into columnar Hive tables (DWRF) on the Tectonic filesystem, and streamed to trainers by a disaggregated CPU preprocessing service (DPP). Meta found that storage and ingestion can cost as much as, or more than, GPU training [15].

```
serving --> Scribe --> ETL / label join --> Hive (DWRF) on Tectonic
                                            |
                        DPP CPU fleet (read cols, decode, transform)
                                            |  tensors
                                            v
                      GPU trainers (TorchRec: sharded EMB + DP MLP, FBGEMM TBE)
```

3.4 Retrieval building blocks

Two Meta systems underlie much of the industry's retrieval work:

- **Faiss** (2017), the similarity-search library used for ANN retrieval over learned embeddings, including billion-scale GPU search [39].
- **Embedding-based retrieval in Facebook Search** (KDD 2020). It reported that random negatives beat "impressed but not clicked" negatives for retrieval recall, and that offline hard negatives should come from a middle rank band, mixed with a large majority of easy negatives (approx. 100:1) [37].
- **PyTorch-BigGraph** learns shallow graph embeddings over billions of entities and trillions of edges for use as retrieval vectors and ranking features [38].
- **Horizon / ReAgent** (2018) is Facebook's applied batch-RL platform, with counterfactual policy evaluation before launch. It was used for decisions such as notifications [40].

---

4. Ads ranking: from GBDT+LR to DHEN, Wukong, InterFormer and Lattice

4.1 2014: "Practical Lessons from Predicting Clicks on Ads at Facebook"

At more than 750M daily active users and more than 1M advertisers, Facebook used boosted trees as a learned feature transform. The leaf index of each tree became a one-hot feature for an online logistic regression, which beat either model alone by more than 3% normalized entropy (NE) [16]. The paper also introduced practices that are still standard: an online joiner that matches impressions and clicks by request ID within a waiting window; the finding that freshness matters (NE degrades as training data falls days behind); and negative downsampling with recalibration q = p / (p + (1 - p) / w) [16]. NE (log loss normalized by the entropy of the background CTR) is still Meta's headline offline metric.

4.2 Ranking-architecture research, 2022-2025

Model	Year	Core idea	Reported result
DHEN [17]	2022	Each layer is an ensemble of heterogeneous interaction modules (dot, self-attention, conv, DCN, linear); layers stack hierarchically	+0.27% NE over AdvancedDLRM; Hybrid Sharded Data Parallel up to 1.2x throughput over FSDP
DHEN for CVR [18]	2025	DHEN in production ad conversion-rate prediction	production practice paper
AutoML [21]	2023	Neural architecture search over large-capacity ranking modules	(see paper)
Wukong [19]	2024	Stacked Factorization Machine Blocks + Linear Compress Blocks; layer i covers orders 1..2^i	scaling law held over two orders of magnitude, beyond 100 GFLOP per example
InterFormer [20]	2024-25	Interleaved Interaction Arch and Sequence Arch with bidirectional flow; Cross Arch summarizes	+0.15% NE, +24% QPS internal; +0.6% topline in pilot ads launches

The direction is clear. Interaction order rises, sequence features and non-sequence features interact in both directions, and architectures are chosen because quality keeps improving as compute grows. Those properties make GEM possible (Section 6).

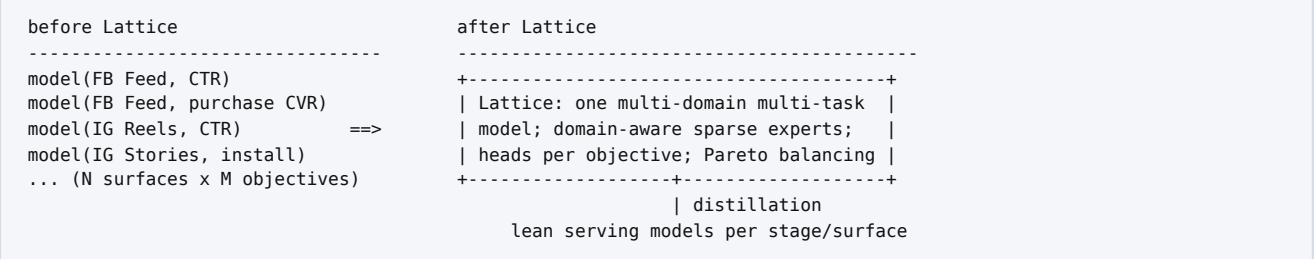
4.3 Meta Lattice (2023 blog, 2025 paper)

Problem. Ads ranking had split into many models across surfaces (Feed, Stories, Reels, Audience Network), objectives (clicks and many conversion types) and funnel stages. That slowed iteration and wasted compute [22].

Design. Lattice is a **multi-domain, multi-task** architecture with sparse activation. It learns from many data sources and handles feedback delays from seconds to days. Meta's 2023 description cites **trillions of parameters** trained on **hundreds of billions of examples** [22]. Thousands of domains and tens of objectives cannot be balanced by hand, so Meta uses Pareto-optimality methods, including Pareto-front feature selection and

MetaBalance [22][24]. MetaBalance rescales each auxiliary task's gradient, per part of the network, toward the magnitude of the target task's gradient, so auxiliary tasks neither dominate nor vanish [24].

The 2025 paper, "Model Space Redesign for Cost-Effective Industry-Scale Ads Recommendations", describes four moves: cross-domain knowledge sharing, data consolidation, model unification, and distillation of the large consolidated model into lean user-facing models. Meta reports +10% on revenue-driving top-line metrics, +11.5% user satisfaction, +6% conversion rate and a 20% capacity saving [23].



Lesson. Consolidation pays when you can (a) balance losses across many tasks automatically and (b) distil the big model into cheap serving models. Without (b), one consolidated model would be too expensive to serve on every surface.

---

5. Andromeda: deep ads retrieval on GPUs (December 2024)

5.1 Problem

Generative-AI creative tools sharply increased the number of eligible ads. Meta said more than a million advertisers had created more than 15 million ads with its GenAI tools in a single month [25]. Retrieval had been limited to lightweight models (two-tower plus ANN, plus heuristics). That limits how much user-ad interaction the model can capture, and the effect is worst at the stage that has to look at the largest pool.

5.2 Architecture

- A heavily customized **deep neural network with sub-linear inference cost**, run with heavy parallelism on **NVIDIA Grace Hopper Superchips**. Meta's **MTIA** accelerator is also part of the hardware plan [25].
- Ads are organized in a **hierarchical index with multiple layers**, so inference only expands the most promising nodes.
- The **index and the retrieval model are trained jointly**. Meta says this gives better precision and recall than two-tower or ANN approaches [25].

```
user + context features
       |
       v
+-------------------------------------------------------------+
| Andromeda retrieval DNN on Grace Hopper (unified CPU-GPU mem)  |
|   level 0:  root clusters   --score--> keep top nodes          |
|   level 1:  sub-clusters    --score--> keep top nodes          |
|   level L:  individual ads  --score--> top ~10^3 ads           |
|   (hierarchy learned jointly with the scoring network)         |
+-----------------------------+-------------------------------+
                              v
         ranking (Lattice / GEM-distilled models) --> auction --> feed insertion
```

5.3 Results and scale

- **+6% recall** for the retrieval system and **+8% ads quality** on selected segments, on Facebook and Instagram [25].
- Secondary write-ups cite about a **10,000x** increase in retrieval model complexity, and a narrowing from **tens of millions** of eligible ads to a **few thousand** candidates per request. Treat these as approximate [26].

5.4 Lessons

Co-designing model, index and hardware makes heavy computation affordable at the stage that sees the most items, which moves the cascade's quality bottleneck upstream. When the dot product becomes the ceiling, a learned hierarchical index is an alternative to ANN over a factorized score.

---

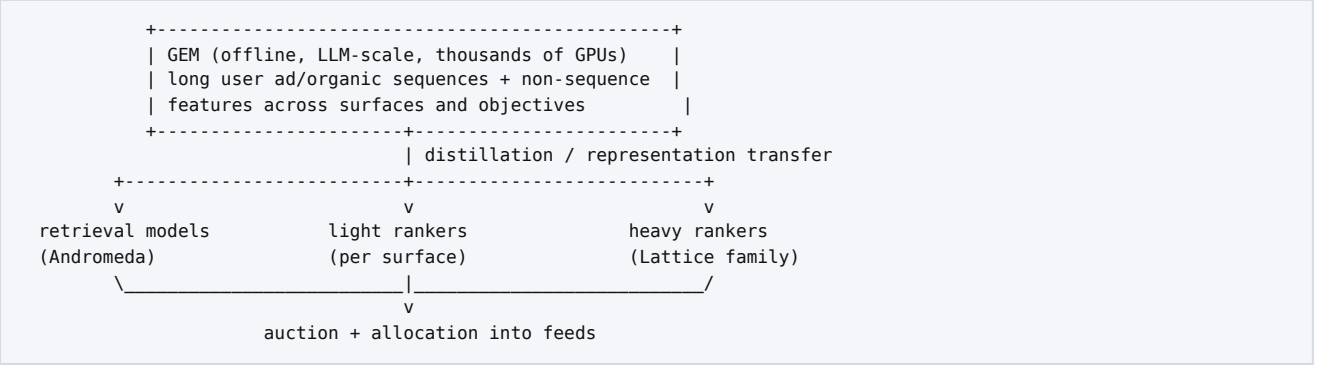
6. GEM: the Generative Ads Recommendation Model (2025-2026)

6.1 What it is

In November 2025 Meta described GEM as "the largest foundation model for recommendation systems (RecSys) in the industry, trained at the scale of large language models", trained on thousands of GPUs with an LLM-inspired paradigm [27]. Meta frames the work in three pillars: **scaling the model architecture**, **post-training knowledge transfer** to downstream models, and **training scalability** [27]. Secondary coverage connects the architecture to InterFormer-style sequence and non-sequence interaction and Wukong-style stacked factorization [29]. The InterFormer and Wukong papers above are the published ingredients.

6.2 Role in the stack: central teacher, not online scorer

GEM does **not** score every candidate online. Its learnings are propagated "across the entire ads model fleet" through distillation and representation transfer into the vertical models that run in each funnel stage [27].



6.3 Results and training efficiency

- **+5% ad conversions on Instagram and +3% on Facebook Feed** in Q2 2025 [27].
- An August 2026 follow-up reports **doubling end-to-end training efficiency to 20-25% model FLOPs utilization** while scaling training FLOPs **4x in twelve months**, through co-design of kernels, numeric precision, parallelism, networking and memory [28].

6.4 Lessons

When the best model is too expensive to serve, keep it offline as a teacher. You pay for it once in training and collect the gains many times through student quality. This is the same pattern as LLM distillation, applied fleet-wide. It also separates two release cycles: the foundation model can be scaled without changing serving latency.

---

7. HSTU and generative recommenders (2024-2026)

7.1 Problem

Meta wanted to know whether recommendation quality scales with compute the way language modeling does, and what architecture would make that possible on high-cardinality, non-stationary streaming data with billions of users [30]. Standard transformers fit poorly. Softmax normalizes away the intensity of user interest, and FFN-heavy blocks use too much activation memory at industrial batch sizes [33].

7.2 Reformulation

"Actions Speak Louder than Words: Trillion-Parameter Sequential Transducers for Generative Recommendations" (Zhai et al., ICML 2024) interleaves content tokens and action tokens into one chronological stream per user [30][33]:

- **Ranking:** p(a_i | Phi_0, a_0, ..., Phi_i). This is target-aware, because the candidate is in the context.
- **Retrieval:** p(Phi_i | Phi_0, a_0, ..., a_{i-1}), next-item generation.

With a causal mask, one forward pass yields a supervision signal at every position. For a user with about 500 interactions, secondary analysis estimates roughly a 500x reduction in training forward passes compared with impression-level DLRM training [33].

7.3 The HSTU block

```
X --> Linear --> SiLU --> split --> U, V, Q, K
                                     |
       SiLU(Q K^T + rab(position, time)) * V     <- no softmax; relative pos+time bias
                                     |
                      LayerNorm(.) * U           <- gating replaces the wide FFN
                                     |
                           Linear --> + X (residual)
```

Secondary notes report per-layer activation memory falling from about 33x the hidden dimension (standard transformer) to about 14x [33].

7.4 Efficiency tricks

- **Stochastic Length.** Long histories are sparsely subsampled during training (80%+ sparsity with almost no quality loss, per secondary notes) [33].
- **Kernels.** HSTU runs 5.3x to 15.2x faster than FlashAttention2-based transformers at sequence length 8,192 [30]. The open-source repo adds Triton kernels, an FA3-based HSTU attention kernel for H100 and row-wise FP8 GEMMs [31].
- **M-FALCON (inference).** Many candidates are appended to the user sequence with a mask, so each candidate attends only to the history. They are processed in micro-batches that share the history's KV cache, which makes target-aware ranking roughly linear in the number of candidates [30][33].

7.5 Scale and results

- **1.5-trillion-parameter** models deployed on multiple surfaces of a billion-user platform, with **+12.4%** on online A/B metrics [30].
- Quality follows a power law of training compute across three orders of magnitude, up to roughly GPT-3/LLaMa-2 training compute [30][32].
- Up to 65.8% NDCG improvement over baselines on public and synthetic data. The repo's reproduction reports HR@10 0.3556 vs 0.2889 for SASRec on ML-20M [30] [31].

7.6 Follow-ups

- **DLRM-v3** (an HSTU-based ranker with an MLCommons LoadGen inference harness) and **HSTU Ultra** (semi-local and target-aware attention, FP8 projections) in the `generative-recommenders` repo [31].
- **ULTRA-HSTU**: "Bending the Scaling Law Curve in Large-Scale Recommendation Systems" (2026) [34], and a study of how well generative recommenders generalize [35].
- **LIGER** (2024) combines semantic-ID generative retrieval with dense retrieval [36].

7.7 Lessons

Most of the gain comes from restructuring training (one causal sequence per user), not only from adding parameters. Softmax is not required: unnormalized SiLU attention with a relative time bias works better for recommendation. And a generative ranker needs its own inference algorithm (M-FALCON) to be affordable.

8. Comparing Meta's systems

System	Year	Stage	Scale (public)	Core technique	Reported impact
FB Feed [1][4]	2021	full funnel	>2B people; >1,000 -> ~500 posts	multipass, multitask NNs, survey-weighted value	n/a
IG Explore [5]	2019	full funnel	65B features/s, 90M predictions/s; 500->150->50->25	ig2vec KNN, distillation	n/a
IG Explore [7][8]	2023	full funnel	<=1,500 -> 100 -> final	two-tower retrieval + 1st stage, MTML, BO-tuned value	n/a
DLRM [9]	2019	ranking	reference model	sparse embeddings + MLPs + pairwise dot	industry standard
Neo/Zion EX [12]	2021-22	training	up to 12T params	4D sharding, fused kernels, RoCE	up to ~40x (approx.)
Lattice [22][23]	2023-25	ads ranking	trillions of params	multi-domain multi-task + distillation	+10% topline, -20% capacity
Andromeda [25]	2024	ads retrieval	10^7 ads -> ~10^3 (approx.)	joint DNN + hierarchical index on GH200/MTIA	+6% recall, +8% quality
HSTU [30]	2024	retrieval + ranking	1.5T params	sequential transduction, M-FALCON	+12.4% online
GEM [27][28]	2025-26	fleet teacher	thousands of GPUs; MFU 20-25%	LLM-scale foundation model + distillation	+5% IG, +3% FB conversions

9. Surfaces and systems not verified in this run

The chapter brief also covers Instagram Reels and Feed ranking, Threads, Facebook Marketplace, "Scaling User Modeling" and MTIA. Live sources could not be retrieved for these, so only verified, cross-referenced facts are given:

- **Per-surface system cards.** Meta's Transparency Center publishes AI system cards per surface. The Facebook Feed and Instagram Explore cards were verified [4][8]. The same Center is the primary public source for the other surfaces' ranking stages and signals, including Reels, Stories, Threads and Marketplace. Readers should consult it directly.
- **Reels and Instagram Feed.** The published patterns (multi-source retrieval, two-tower first stage, MTML second stage, value-model fusion, integrity-first filtering) are documented for Explore and Feed [1][7][8]. Reels-specific numbers were not verified here.
- **Threads and Marketplace.** No architecture details were verified in this run. On the retrieval side, Marketplace's product search sits next to the embedding-retrieval lineage of Facebook Search [37].
- **Scaling User Modeling (SUM).** Meta has a paper titled "Scaling User Modeling: Large-scale Online User Representations for Ads Personalization in Meta" on precomputed user representations shared across many downstream ads models. Its

contents and URL were not re-verified here [41]. The idea of a shared upstream user model feeding many vertical models appears in the verified GEM post [27].

- **MTIA.** The Meta Training and Inference Accelerator is named as part of Andromeda's hardware plan [25]. Chip specifications were not verified in this run.

---

10. Key takeaways for system designers

1. **Run integrity and eligibility first, contextual rules last.** Feed's pass structure spends heavy compute only on showable content and keeps diversity as a separate pass [1].
2. **Separate prediction from policy.** Multitask models predict calibrated event probabilities. A small value function, tuned by surveys or Bayesian optimization with guardrails, encodes the trade-off [1][6][7].
3. **Choose the value-model form deliberately.** Linear sums are interpretable. Log-linear products let negative feedback veto an item [6][7].
4. **Optimize for real value, not only engagement.** MSI's history shows that heavy weights on reactive engagement can amplify harmful content. Survey-predicted objectives are a counterweight [2][3].
5. **Distil downward through the funnel.** Train cheap stages to imitate expensive ones (Explore 2019 and 2023), and distil big consolidated models into lean serving models (Lattice, GEM) [5][7][23][27].
6. **Cache and precompute aggressively.** Daily item towers, cacheable first-stage models and off-peak heavy scoring are what allowed heavier models at every Explore stage [7].
7. **Budget by embeddings and bytes, not FLOPs.** For DLRM-style models, embedding capacity, all-to-all bandwidth and data ingestion set the cost [11][12][15].
8. **Consolidate the model fleet when loss balancing is automated.** Lattice's Pareto methods and MetaBalance make thousands of domain-objective pairs workable [22][24].
9. **Push model quality upstream.** Andromeda shows that retrieval can carry deep models when index, model and hardware are co-designed [25].
10. **Keep the largest model offline.** GEM is an LLM-scale teacher, and the serving fleet inherits its gains through distillation and representation transfer [27][28].
11. **Restructure training before scaling it.** HSTU's one-sequence-per-user causal formulation gives large efficiency gains and a compute scaling law [30].
12. **Design inference with the model.** M-FALCON's candidate-isolated attention with a shared KV cache is what makes target-aware generative ranking affordable [30].

References

1. Lada, A., Wang, M., Yan, T. "How machine learning powers Facebook's News Feed ranking algorithm." Meta, Engineering at Meta, 2021. https://engineering.fb.com/2021/01/26/ml-applications/news-feed-ranking/
2. Meta. "Incorporating More Feedback Into News Feed Ranking." Meta Newsroom, 2021. https://about.fb.com/news/2021/04/incorporating-more-feedback-into-news-feed

-ranking/
3. CNN Business (syndicated). "Likes, anger emojis and RSVPs: the math behind Facebook's News Feed and how it backfired." 2021. https://abc17news.com/news/2021/10/27/likes-anger-emojis-and-rsvps-the-math-behind-facebooks-news-feed-and-how-it-backfired/
4. Meta Transparency Center. "Facebook Feed AI system" (system card). Meta, accessed 2026. https://transparency.meta.com/features/explaining-ranking/fb-feed
5. Medvedev, I., Wu, H., Gordon, T. "Powered by AI: Instagram's Explore recommender system." Instagram Engineering / Facebook AI, 2019. https://instagram-engineering.com/powered-by-ai-instagrams-explore-recommender-system-7ca901d2a882
6. Meta / Instagram. "How Instagram suggests new content" (designing a constrained exploration system). Engineering at Meta, 2020. https://engineering.fb.com/2020/12/10/web/how-instagram-suggests-new-content/
7. Vorotilov, V., Shugaepov, I. "Scaling the Instagram Explore recommendations system." Meta, Engineering at Meta, 2023. https://engineering.fb.com/2023/08/09/ml-applications/scaling-instagram-explore-recommendations-system/
8. Meta Transparency Center. "Instagram Explore AI system" (system card). Meta, accessed 2026. https://transparency.meta.com/features/explaining-ranking/ig-explore/
9. Naumov, M., et al. "Deep Learning Recommendation Model for Personalization and Recommendation Systems." Facebook, 2019. https://arxiv.org/abs/1906.00091
10. Meta Research. "DLRM: Deep Learning Recommendation Model" (GitHub repository). Meta, accessed 2026. https://github.com/facebookresearch/dlrm
11. Acun, B., et al. "Understanding Training Efficiency of Deep Learning Recommendation Models at Scale." Meta, HPCA 2021. https://arxiv.org/abs/2011.05497
12. Mudigere, D., et al. "Software-Hardware Co-design for Fast and Scalable Training of Deep Learning Recommendation Models" (Neo / ZionEX). Meta, ISCA 2022. https://arxiv.org/abs/2104.05158
13. PyTorch / Meta. "TorchRec" (GitHub repository). Accessed 2026. https://github.com/pytorch/torchrec
14. PyTorch / Meta. "FBGEMM / FBGEMM_GPU" (GitHub repository). Accessed 2026. https://github.com/pytorch/FBGEMM
15. Zhao, M., et al. "Understanding Data Storage and Ingestion for Large-Scale Deep Recommendation Model Training." Meta, ISCA 2022. https://arxiv.org/abs/2108.09373
16. He, X., Pan, J., Jin, O., et al. "Practical Lessons from Predicting Clicks on Ads at Facebook." Facebook, ADKDD 2014. https://quinonero.net/Publications/predicting-clicks-facebook.pdf
17. Zhang, B., Luo, L., Liu, X., et al. "DHEN: A Deep and Hierarchical Ensemble Network for Large-Scale Click-Through Rate Prediction." Meta, 2022. https://arxiv.org/abs/2203.11014
18. Meta. "On the Practice of Deep Hierarchical Ensemble Network for Ad Conversion Rate Prediction." Meta, 2025. https://arxiv.org/abs/2504.08169
19. Zhang, B., Luo, L., Chen, Y., et al. "Wukong: Towards a Scaling Law for Large-Scale Recommendation." Meta, ICML 2024. https://arxiv.org/abs/2403.02545
20. Zeng, Z., et al. "InterFormer: Effective Heterogeneous Interaction Learning for Click-Through Rate Prediction." Meta / UIUC, 2024-2025. https://arxiv.org/abs/2411.09852
21. Meta. "AutoML for Large Capacity Modeling of Meta's Ranking Systems." Meta, 2023. https://arxiv.org/abs/2311.07870

22. Meta AI. "Meta Lattice: AI ads performance and efficiency." Meta, 2023. https://ai.meta.com/blog/ai-ads-performance-efficiency-meta-lattice/
23. Meta. "Meta Lattice: Model Space Redesign for Cost-Effective Industry-Scale Ads Recommendations." Meta, 2025. https://arxiv.org/abs/2512.09200
24. He, Y., Feng, X., Cheng, C., Ji, G., Guo, Y., Caverlee, J. "MetaBalance: Improving Multi-Task Recommendations via Adapting Gradient Magnitudes of Auxiliary Tasks." Meta AI / TAMU, WWW 2022. https://arxiv.org/abs/2203.06801
25. Meta Engineering. "Meta Andromeda: Supercharging Advantage+ automation with the next-gen personalized ads retrieval engine." Meta, December 2024. https://engineering.fb.com/2024/12/02/production-engineering/meta-andromeda-advantage-automation-next-gen-personalized-ads-retrieval-engine/
26. Loomer, J. "Meta Andromeda: What It Means for Your Ad Strategy" (secondary summary of [25]). 2025. https://www.jonloomer.com/meta-andromeda/
27. Engineering at Meta. "Meta's Generative Ads Model (GEM): The central brain accelerating ads recommendation AI innovation." Meta, 10 November 2025. https://engineering.fb.com/2025/11/10/ml-applications/metas-generative-ads-model-gem-the-central-brain-accelerating-ads-recommendation-ai-innovation/
28. Engineering at Meta. "GEM Training: How Meta Doubled the Efficiency of Its LLM-Scale Ads Foundation Model." Meta, August 2026. https://engineering.fb.com/2026/08/03/ml-applications/training-gem-at-llm-scale-meta-ads-recommendation-foundation-model/
29. ByteByteGo. "How Meta Built a New AI-Powered Ads Model for 5% Better Conversions" (secondary). 2025. https://blog.bytebytego.com/p/how-meta-built-a-new-ai-powered-ads
30. Zhai, J., et al. "Actions Speak Louder than Words: Trillion-Parameter Sequential Transducers for Generative Recommendations." Meta, ICML 2024. https://arxiv.org/abs/2402.17152
31. meta-recsys. "generative-recommenders" (HSTU, M-FALCON, DLRM-v3, HSTU Ultra code). Meta, 2024-2026. https://github.com/meta-recsys/generative-recommenders
32. Zhai, J., et al. ICML 2024 slides, "Actions Speak Louder than Words." Meta, 2024. https://icml.cc/media/icml-2024/Slides/32684.pdf
33. Xing, H. "HSTU: The Cornerstone of Scaling Laws" (fun-rec-mdbook technical notes, secondary). 2026. https://github.com/Haozhe-Xing/fun-rec-mdbook/blob/main/src/en/part7-scaling/hstu.md
34. Meta. "Bending the Scaling Law Curve in Large-Scale Recommendation Systems" (ULTRA-HSTU). Meta, 2026. https://arxiv.org/abs/2602.16986
35. Meta. "How Well Does Generative Recommendation Generalize?" Meta, 2026. https://arxiv.org/abs/2603.19809
36. Meta. "Unifying Generative and Dense Retrieval for Sequential Recommendation" (LIGER). Meta, 2024. https://arxiv.org/abs/2411.18814
37. Huang, J.-T., et al. "Embedding-based Retrieval in Facebook Search." Facebook, KDD 2020. https://arxiv.org/abs/2006.11632
38. Lerer, A., Wu, L., Shen, J., Lacroix, T., Wehrstedt, L., Bose, A., Peysakhovich, A. "PyTorch-BigGraph: A Large-scale Graph Embedding System." Facebook AI, SysML 2019. https://mlsys.org/Conferences/2019/doc/2019/71.pdf
39. Meta Engineering. "Faiss: A library for efficient similarity search." Facebook, 2017. https://engineering.fb.com/2017/03/29/data-infrastructure/faiss-a-library-for-efficient-similarity-search/

40. Gauci, J., Conti, E., Liang, Y., et al. "Horizon: Facebook's Open Source Applied Reinforcement Learning Platform." Facebook, 2018. https://arxiv.org/abs/1811.00260
41. Meta. "Scaling User Modeling: Large-scale Online User Representations for Ads Personalization in Meta." Meta, 2023-2024 (title cited from prior knowledge; URL not verified in this run, so none is given).

20. TikTok/ByteDance and Kuaishou: Real-Time Short-Video Recommendation, from Monolith to OneRec

Short-video feeds are the hardest everyday recommendation workload. A user may swipe through dozens of items in a few minutes. Every swipe is an implicit label: watch time, completion, replay, skip, like, comment, share, follow. The corpus churns by the hour, and the product has a single full-screen slot, so every exposure has to count. The two companies that defined this category, ByteDance (Douyin in China, TikTok elsewhere) and Kuaishou (Kuaishou and Kuaishou Lite), have published more system detail than almost any other consumer platform. ByteDance's work is mostly about infrastructure and ranking-model scaling. Kuaishou's is mostly about long-sequence modelling, the multi-stage cascade, and, since 2025, replacing that cascade with one generative model.

This chapter covers both companies:

- **ByteDance / TikTok.** What the company has said publicly about the For You feed, the reported "TikTok Algo 101" internal document, Monolith real-time training, Deep Retrieval, LONGER, RankMixer and TokenMixer-Large, OneTrans and HLLM.
- **Kuaishou.** The six-stage cascade (RecFlow), PERSIA training, TWIN and TWIN V2 lifelong sequences, KuaiFormer retrieval, multi-task and fusion work (HoME, Pantheon, RLUR), watch-time debiasing (D2Q), and the OneRec family (V1, technical report, V2, Think, OpenOneRec).

A note on sourcing. The live web-search budget for this run was used up before research could start, and outbound access to arXiv and most company sites failed from the writing environment. This chapter therefore relies on three kinds of source. The first is primary sources that could be fetched (GitHub repositories of ByteDance and Kuaishou). The second is technical notes that summarise the papers in detail (the fun-rec-mdbook chapters). The third is primary-source facts and arXiv identifiers already checked by earlier chapters of this Atlas (Chapters 1, 3, 5, 6, 9, 11, 13, 14 and 15). The two TikTok public disclosures (the 2020 newsroom post and the 2021 New York Times report) could not be re-fetched. They are described from the public record and marked as such. Numbers are as reported, and any that come from secondary notes are marked approximate.

20.1 The short-video recommendation problem

Short-video feeds differ from long-form video (Chapter 17) and photo feeds (Chapter 19) in several ways that drive the designs below:

1. **One item at a time, very many items per session.** Each request returns a small batch, and the feed then prefetches more. Kuaishou's funnel ends with roughly 6 to 10 exposed items per request [23][24]. So request QPS is high relative to the number of users, and the ranking decision is close to a sequential decision problem.
2. **Watch time is the main label, and it is biased.** Longer videos mechanically collect more watch time. Kuaishou's D2Q paper predicts watch-time quantiles within duration buckets to remove that confound [30]. OneRec-V2 log-buckets watch time by duration for the same reason [19][22].
3. **Extreme freshness.** New videos must be learnable within minutes, and interests drift within a session. ByteDance's Monolith exists because daily batch retraining and hashed embedding tables could not keep up [1][2].
4. **Very long histories.** Heavy users produce 10^4 to 10^6 behaviours over their lifetime. Kuaishou reports lifelong sequences beyond 10^5 behaviours, which is what motivates TWIN V2 [14] and OneRec's 100,000-record lifelong pathway [22].
5. **Retention is the business objective.** Both companies optimise immediate engagement as a proxy for long-term return. The reported TikTok internal document names retention and time spent as the ultimate goals [12]. Kuaishou has published RL work that targets return time directly [28].

20.2 ByteDance / TikTok: what the company has disclosed

20.2.1 The 2020 "How TikTok recommends videos #ForYou" post

In June 2020 TikTok published a newsroom post describing the For You feed [11]. (The post could not be re-fetched for this chapter; this summary follows the widely quoted public text.) It describes the feed as a recommendation system that ranks videos on a combination of factors:

- **User interactions.** Videos liked or shared, accounts followed, comments posted, and content created.
- **Video information.** Captions, sounds and hashtags.
- **Device and account settings.** Language preference, country and device type. The post says these get lower weight than the other signals, because users do not actively express them.
- **Signal strength.** Each factor is weighted by its strength as an indicator of interest. The post gives the example that finishing a longer video is a stronger signal than whether the viewer and creator are in the same country.
- **Not direct factors.** Follower count and a creator's previous high-performing videos are not direct ranking factors, though accounts with more followers naturally collect more of the other signals.
- **Diversity rules.** The feed is designed not to show two videos in a row from the same creator or with the same sound. It also inserts videos that do not obviously match a user's stated interests, so that the feed does not become a filter bubble.
- **Cold start.** New users can select interest categories at sign-up, and early interactions quickly replace those defaults.

The design lesson is that TikTok publicly frames the system as a weighted multi-signal ranker with an explicit diversity and exploration layer. This matches the generic funnel of Chapter 1 and the value-model pattern of Chapter 4.

20.2.2 The reported "TikTok Algo 101" document (2021)

In December 2021 the New York Times reported on an internal TikTok document called "TikTok Algo 101", which it said was produced by the Beijing engineering team for non-technical staff [12]. (The article could not be re-fetched; details below follow the widely cited reporting and should be treated as reported, not confirmed by ByteDance.) Key points as reported:

- The company's stated "ultimate goal" is adding daily active users, optimised through two closely related metrics: **retention** and **time spent**.
- The ranking score is described as a simplified weighted sum of predicted engagement:

```
score = P_like * V_like + P_comment * V_comment
      + E_playtime * V_playtime + P_play * V_play
P_*: predicted probability of the action      E_playtime: expected play time
V_*: value weight for that action (set by product policy)
```

- The document reportedly also discussed suppressing repetitive content and preserving creator diversity, since repetitive feeds can bore users.

This is exactly the "value model" pattern documented at Meta, X and Kuaishou in Chapter 4: a multi-task model predicts several engagement probabilities, and hand-set (later learned) weights fuse them into one score. Kuaishou's papers describe the same stage as "ensemble sort" [27].

20.2.3 Regulatory context (2025-2026)

Because of US national-security concerns, TikTok's US recommendation algorithm became a subject of legislation and of the US joint-venture arrangement in 2025-2026. Under that arrangement the algorithm was reported to be retrained on US user data under the oversight of a US partner. No technical design detail about the retrained system had been published in sources available to this chapter, so it is mentioned only as context. Architecturally, the published ByteDance papers below are the best public evidence of how the stack works.

20.3 Monolith: real-time training with collisionless embeddings (ByteDance, 2022)

Problem

ByteDance's feed and ads products have three properties that clash with standard TensorFlow training [1][2]:

- The sparse ID space (users, videos, creators, crosses) is **non-stationary and unbounded**.
- Feature frequency is **long-tailed**.
- **Concept drift** happens within hours.

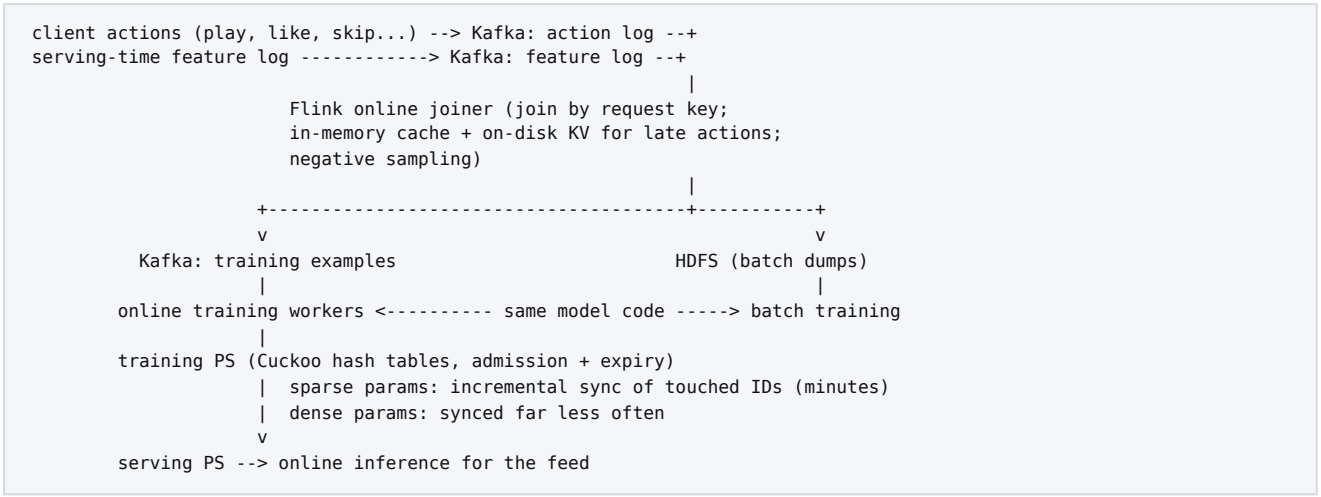
TensorFlow's built-in variables are fixed-size dense tensors. Fitting unbounded IDs into them requires hashing, and hash collisions degrade quality. The paper also argues that conventional frameworks keep batch training and serving "completely separated", so models cannot react to user feedback in real time [1].

Architecture

Collisionless embedding table. Monolith replaces `tf.Variable` tables with a key-value hash table on the parameter servers, built on **Cuckoo hashing** [1]. Cuckoo hashing uses two hash functions and two arrays. On insert, an occupant is evicted to its alternate slot until every key has a place. Lookups are O(1), and distinct IDs never share a row. The repository describes the goal as giving each ID feature "its own unique embedding representation" [2]. Two policies bound memory [1]:

- **Frequency filtering (admission).** An ID gets an embedding only after it has appeared a threshold number of times. A probabilistic filter keeps the counting cost low.
- **Expirable embeddings (eviction).** IDs inactive past an expiry time, such as stale users or deleted videos, are removed.

Online training loop.



How it works [1][2]:

1. **One framework, two modes.** Batch training on historical HDFS data bootstraps the model. Online training then consumes the streaming example queue indefinitely.
2. **Online joiner.** A Flink job joins user actions with the features logged at serving time. Actions can arrive minutes after the impression, so features wait in a memory cache backed by on-disk storage. Negatives are down-sampled, and a log-odds correction at serving time restores calibration.
3. **Asymmetric parameter sync.** Pushing terabytes of parameters to serving every minute is impractical. Sparse embeddings are synced **incrementally**: only IDs

touched since the last sync are sent, at minute-level intervals. Dense weights change slowly under momentum-style optimisers, so they are synced much less often.

4. **Cheap fault tolerance.** The paper finds that daily parameter-server snapshots lose very little quality on a PS failure. Each shard holds a small fraction of IDs, and online training recovers quickly, so frequent snapshots are not worth their I/O cost.

Results and lessons

- Collisionless tables beat hashed (colliding) tables on public and internal data [1].
- Shorter sync intervals give consistently higher online AUC, and real-time training beats batch training in the ads experiments [1].
- The open-source repository is TensorFlow plus Bazel, with demos for distributed asynchronous training on Kubernetes via Primus. It was archived (read-only) by ByteDance in October 2025 [2].

The main lessons: **freshness is a model-quality feature**, the sparse/dense asymmetric sync is what makes minute-level updates affordable, and admission and eviction policy matter as much as the table data structure.

20.4 ByteDance retrieval: Deep Retrieval and LLM-based item/user models

Deep Retrieval (2020)

Deep Retrieval replaces ANN over a fixed embedding space with a **learned discrete structure** [3]. Each item maps to one or more paths through a D-layer lattice with K nodes per layer, so every item gets a code of D tokens from K values. Path assignments are model parameters, learned jointly with the user-to-path model by expectation-maximisation. Serving runs beam search over paths, then scores the items on the retrieved paths. ByteDance reports near brute-force accuracy on public data at sub-linear cost, and online engagement gains over a tuned ANN baseline [3]. This is an early, production-tested ancestor of the semantic-ID generative retrieval that Kuaishou later scaled in OneRec.

```
user features --> layer-1 softmax over K --> layer-2 (cond.) --> ... --> layer-D
                    beam search keeps top-B partial paths at each layer
retrieved paths --> items assigned to those paths --> rerank with a scorer
```

HLLM (2024)

HLLM (Hierarchical Large Language Model) is ByteDance research code that uses two LLMs [8]:

- An **item LLM** reads item text (title, tags, description) and produces an item embedding.
- A **user LLM** consumes the sequence of item embeddings to predict the next item.

Released checkpoints are HLLM-1B (from TinyLlama-1.1B) and HLLM-7B (from Baichuan2-7B). On the Books benchmark with 28,672 negatives, HLLM-1B reaches R@10 9.28 and

HLLM-7B 9.39, against 6.50 for HSTU-large. On Pixel8M, HLLM-1B reaches R@10 6.13 against 5.08 (SASRec) and 4.83 (HSTU) [8]. A follow-up, HLLM-Creator (2025), extends the approach to personalised creative generation [8]. The repository is also archived [8]. Lesson: text-derived item representations help on sparse and cold items, at the cost of LLM-scale compute in the item tower.

20.5 ByteDance ranking: long sequences and hardware-efficient scaling

20.5.1 LONGER: end-to-end long sequences (RecSys 2025)

Problem. Two-stage behaviour retrieval (SIM, TWIN) is lossy: the cheap first stage can drop behaviours that the final model would value. ByteDance wanted end-to-end Transformer modelling of sequences up to about 10,000 behaviours inside the ranker [4].

Design [4][5]:

- **Global tokens** (target item, user summary tokens) attend over the full context and stabilise attention.
- **Token merge.** Adjacent behaviours are grouped into fewer tokens, with a light "InnerTransformer" preserving detail within each group.
- **Hybrid attention.** A small set of query tokens cross-attends over the long sequence, then self-attention runs on the short output, which cuts the quadratic cost.
- **Engineering.** Mixed precision, activation recomputation, and **KV-cache serving**: the user-side sequence is encoded once per request and shared across all candidates.

Results. Consistent offline and online gains in advertising and e-commerce. The paper says LONGER is fully deployed in dozens of ByteDance scenarios serving billions of users [4].

20.5.2 RankMixer (2025) and TokenMixer-Large (2026)

Problem. Years of hand-designed feature-interaction modules left ByteDance's rankers with very poor hardware efficiency. RankMixer reports about **4.5% model FLOPs utilisation (MFU)** on GPUs before the redesign [6]. Technical notes attribute the low MFU to memory-bound embedding lookups (about 40% of time), many small kernels (about 35%, each under 10% MFU) and launch overhead (about 20%) [9] (approximate, from secondary notes).

Architecture [6][9]:

- Features are grouped into T **tokens** of uniform dimension D. Recommendation models have hundreds to thousands of tokens.
- Each RankMixer block has two parts:
  - **Multi-head token mixing.** Each token is split into heads and the heads are recombined across tokens. This is parameter-free, costs O(TD), uses no softmax, and replaces O(T^2 D) self-attention over heterogeneous feature spaces.
  - **Per-token FFN.** Each token (feature subspace) gets its own FFN parameters. This gives T times the parameters of a shared FFN at the same FLOPs.

- A **sparse MoE** variant uses ReLU routing with an L1-style regulariser, and **DTSI-MoE** (dense-training, sparse-inference routers) to scale parameters while keeping per-sample compute fixed.
- Typical depth is 6 to 12 blocks, and the output is mean-pooled into the logits.

```
raw features --> group/project --> T tokens x D
   [ token mixing (heads shuffled across tokens) -> add & norm
     per-token FFN / sparse MoE (separate weights per token) -> add & norm ] x L
--> mean pool --> multi-task heads (finish, like, play time, ...)
```

Results. MFU rose from 4.5% to 45%, and ranking-model parameters grew about 100x at roughly constant inference latency. Full-traffic deployment in **Douyin feed ranking** gave +0.2% active days and +0.5% app duration [6]. The successor **TokenMixer-Large** (2026) reports offline scaling to 7B and 15B parameters for Douyin ads and e-commerce [7].

20.5.3 OneTrans: one Transformer for sequence and features

OneTrans, described in technical notes as a successor in the same line [9][10], folds sequence modelling and feature interaction into a single Transformer [10]:

- **S-tokens** (behaviour events, projected by behaviour-specific MLPs) and **NS-tokens** (non-sequential features, split from one MLP by an "auto-split tokenizer") form one input.
- **Mixed parameterisation.** S-tokens share Q/K/V and FFN weights, while each NS-token has its own.
- **Pyramid stack.** Queries come only from the last L' tokens at each layer, so the sequence shrinks, for example 1190 -> 595 -> 297 -> ... -> 12 tokens.
- **Two-level KV caching.** The S-side keys and values are computed once per request and reused across candidates. Across requests, only newly appended events need new K/V.

The notes describe a model of hundreds of millions of parameters [10]. They do not report deployment numbers, so treat OneTrans's production status as unconfirmed here.

20.5.4 The ByteDance pattern

```
     +-------------------- Monolith-style online training ------------------+
     | Kafka actions + feature logs -> Flink joiner -> streaming trainer    |
     | Cuckoo-hash sparse PS  --(minute-level incremental sync)--> serving  |
     +-----------------------------------------------------------------------+
request --> retrieval (two-tower / Deep Retrieval paths / multi-channel)
        --> pre-rank --> rank: LONGER (10k seq, KV cache) + RankMixer tokens
        --> value-model fusion (P_like*V_like + E_playtime*V_playtime + ...)
        --> diversity rules (no same creator / same sound back to back)
        --> For You feed
         (composite, synthesised from [1][4][6][11][12]; not an official diagram)
```

20.6 Kuaishou: the cascade, quantified

20.6.1 RecFlow and the six-stage funnel

Kuaishou has published the most complete stage-level data of any short-video platform. **RecFlow** (ICLR 2025) logs six stages of the production funnel. It contains about 9.3 million requests and 1.9 billion stage samples, so researchers can study inter-stage effects such as the exposure-only training bias [23]. A 2026 cross-stage-fusion paper on the same cascade gives candidate sizes falling from about 8,000 to about 500 after pre-ranking, and finally to about 6 to 10 exposed items [24]. KuaiFormer describes retrieval feeding pre-rank, then cascade rank, then fine-grained full rank [16].

```
corpus (10^8+ videos)
   | multi-channel retrieval: i2i, u2i two-tower, KuaiFormer, ...
   v
~8,000 merged candidates
   | pre-ranking (lightweight)
   v
~500
   | cascade rank -> full rank (multi-task: pLTR, pWTR, pFTR, watch time, ...)
   v
tens
   | ensemble sort (pXTR fusion) + re-rank (listwise, diversity, business rules)
   v
~6-10 exposed                                  (after [16][23][24]; approximate)
```

20.6.2 PERSIA: training 100-trillion-parameter models

PERSIA (Kuaishou with ETH Zurich, KDD 2022) is Kuaishou's answer to the same sparse-dense split that Monolith addresses [25]. Embedding layers are updated **asynchronously** on CPU parameter servers, because embedding communication is huge and sparse updates tolerate staleness. The dense network is trained **synchronously** on GPUs, because its accuracy suffers under asynchrony. PERSIA reports models of up to **100 trillion parameters** and production use at Kuaishou for applications with hundreds of millions of DAU. It reports several-fold throughput gains (approximately up to 7x) over contemporary systems [25].

20.6.3 Multi-task ranking and ensemble sort

- **HoME** (KDD 2025) fixes MMoE pathologies seen in production: expert collapse (ReLU experts producing mostly zeros), shared experts degrading into single-task experts, and expert underfitting. Its answer is hierarchical, normalised expert groups. It is deployed on short-video services serving about 400 million daily users [26].
- **Ensemble sort.** The ranker emits many pXTRs, and a fusion formula merges them, which is the same value-model idea as the TikTok formula in 20.2.2. **Pantheon** (2025) learns personalised fusion weights jointly with the real-time ranker using Iterative Pareto Policy Optimisation, deployed for about 400 million daily users [27]. **GRADE** (2025) uses GRPO-style group-relative RL over weight vectors with Dirichlet exploration [37].
- **RLUR** (WWW 2023) optimises **user retention**, measured as the time until the next visit, as an infinite-horizon MDP. The actor outputs fusion weights, not item scores, and a TD-trained retention critic is shaped with immediate watch-time rewards [28]. The companion **TSCAC** maximises watch time subject to constraints on likes, follows and shares [29].
- **Re-ranking.** NAR4Rec (KDD 2024) generates the final slate with a non-autoregressive model, filling all positions in one parallel pass instead of decoding position by position. This keeps listwise re-ranking within the latency budget [32].

20.6.4 Label and cold-start engineering

- **D2Q** (KDD 2022) splits videos into duration buckets and predicts watch-time **quantiles** within each bucket. This removes the length bias of raw watch-time regression [30].
- **POSO** (2021) gates model sub-modules by user cohort, such as new versus regular users. Signals from new users are otherwise drowned out by heavy users in shared parameters [31].
- **KuaiRec** is a public dataset with an almost fully observed user-item matrix. The small matrix has 1,411 users, 3,327 items and 99.6% density, and the big matrix has 7,176 users, 10,728 items and 16.3% density. It is useful for unbiased offline evaluation [33].

20.7 Kuaishou lifelong user modelling: TWIN and TWIN V2

20.7.1 TWIN (KDD 2023)

Problem. SIM-style two-stage models (Chapter 5) use a cheap General Search Unit (GSU) to pick relevant behaviours and an Exact Search Unit (ESU) to attend over them. The GSU's relevance metric differs from the ESU's attention, so the GSU drops behaviours that the ESU would rank highly [13].

Key idea. Make the two stages "twins". The **Consistency-Preserved GSU (CP-GSU)** uses the same target-attention metric, architecture and parameters as the ESU. It scores all ~10^4 long-term behaviours for every candidate and keeps the top 100. The ESU then runs multi-head target attention over those 100 [13].

Making it affordable. TWIN splits behaviour features into two kinds:

- **Inherent item features** (video ID, author, tags, duration) are independent of the user. Their key projections are **precomputed and cached** per item, shared across all users.
- **User-item cross features** (when the user watched it, how long they played it) are compressed into a scalar bias on the attention logit.

```
per behaviour j:  K_j = W * inherent_j        (precomputed, cached per item)
                  b_j = w . cross_j            (cheap, user-specific)
per candidate c:  s_j = Q(c) . K_j / sqrt(d) + b_j
CP-GSU: top-100 by s_j over ~10^4 behaviours -> ESU: multi-head target attention
```

Scale. Offline experiments use about 46 billion production samples. Optimised serving infrastructure gave a **99.3% reduction** in the compute bottleneck, which allowed deployment on Kuaishou's main traffic [13]. Secondary summaries report watch-time gains of roughly 1.3% to 6.2% across tabs (approximate) [15].

20.7.2 TWIN V2 (CIKM 2024)

Problem. Full life-cycle histories now exceed 10^5 to 10^6 behaviours, more than even TWIN can scan online [14].

Design [14][15]:

1. **Offline hierarchical clustering** of each user's behaviours into clusters of similar items. Secondary notes describe this as recursive k-means that splits clusters until each leaf holds at most gamma items.
2. Each cluster becomes one **virtual behaviour** with aggregated features.
3. **Cluster-aware target attention** in the GSU and ESU, with attention adjusted by cluster size so that strong interests keep their weight.

Results. Deployed on primary traffic serving hundreds of millions of DAU. Watch-time gains of +0.672%, +0.800% and +0.728% on the Featured-Video, Discovery and Slide tabs. The authors note that a 0.1% gain is significant at Kuaishou [14].

20.8 KuaiFormer: Transformer retrieval (2024)

Problem. Single-vector two-tower retrieval squeezes all of a user's interests into one embedding and handles long sequences poorly [16].

Design [16]:

- **Multiple learnable query tokens** (like BERT's [CLS]) are appended to the behaviour sequence. Each produces one interest vector, which runs its own ANN query, for multi-interest retrieval.
- **Adaptive sequence compression.** The oldest behaviours are merged in groups of 64 and middle-aged ones in groups of 16, each group aggregated by a one-layer bidirectional Transformer. The most recent behaviours are kept as-is. For example, 256 behaviours become 55 tokens.
- A **causal Transformer** over the compressed sequence, trained with smoothed in-batch softmax and logQ correction for popularity bias.

```
[ old 64-groups | mid 16-groups | recent raw ] + [Q1 .. Qk] query tokens
            -> causal Transformer -> k interest vectors -> k ANN lookups -> merge
```

Scale. In production since May 2024 in Kuaishou's short-video recommendation, serving more than 400 million DAU [16]. Lesson: "compress the old, keep the recent exact" is a cheap, robust way to fit long histories into a retrieval latency budget. LONGER's token merge applies the same idea on the ranking side [4].

20.9 OneRec: collapsing the cascade into one generative model

OneRec is the most complete public case of replacing an entire short-video cascade with one generative model at hundreds-of-millions-DAU scale.

20.9.1 OneRec V1 (February 2025 paper, June 2025 technical report)

Problem. In the cascade each stage is optimised separately, compute is fragmented across many small models with low MFU, and earlier stages cap the quality of later ones [17][18]. OneRec generates a **session** (a short list of videos) end to end and aligns it with user preferences through RL.

Item tokenizer [22]:

- Multimodal inputs (title, tags, ASR, OCR, cover image, sampled frames) are encoded by a multimodal LLM (miniCPM-V-8B) into 1,280 tokens. A QFormer compresses these into 4 query vectors.
- An item-pair contrastive loss injects collaborative signal, with title generation as an auxiliary task.
- **RQ-KMeans** over 3 residual levels gives each video a 3-token semantic ID. Chapter 6 reports about 8K codes per level (approximate).

Encoder (four pathways) [22]:

- Static user features through two dense layers.
- Short-term behaviour: the last 20 interactions.
- Positive-feedback behaviour: the last 256 high-engagement interactions.
- Lifelong behaviour: up to 100,000 records. These are compressed by hierarchical k-means into a length-2,000 sequence, then by a QFormer into 128 tokens.
- Total context: 1 + 20 + 256 + 128 = **405 tokens**.

Decoder. A [BOS] token plus semantic-ID tokens, with causal self-attention, cross-attention to the encoder, and top-k **MoE** feed-forward layers, trained with next-token prediction [22].

```
static feats -----+
short seq (20) ---+--> Encoder (Transformer, RMSNorm) --> 405 context tokens
pos-feedback(256)-+                                          | cross-attn
lifelong (1e5) -> hier. k-means (2000) -> QFormer (128) -------+
                                                             v
               [BOS] s1 s2 s3 --> Decoder (causal self-attn + MoE FFN)
                                                             | beam search
                                                             v
               candidate sessions --> reward system:
                   P-Score (SIM-based, one tower per objective: CTR, LTR, VTR, ...)
                   + format reward (legal SIDs only) + SIR (viral down-weighting)
                                                             |
               IPA / DPO (V1 paper)  ->  ECPO = early-clipped GRPO (tech report)
```

Preference alignment.

- The original paper used **Iterative Preference Alignment (IPA)**: a reward model scores sampled sessions, and the best and worst sessions form DPO pairs [17].
- The technical report moved to **ECPO**, an early-clipped GRPO. It pre-clips the policy ratio on negative-advantage samples so their gradients cannot explode [18][22].
- A **format reward** keeps only legal semantic IDs (advantage 1 for legal samples; illegal ones are discarded).
- **SIR** industrial alignment down-weights viral content once its share passes a threshold. It cut viral-content exposure by 9.59% with core metrics stable [22].

Scale and economics (as reported, approximate) [18]:

- Training MFU about 24% and inference MFU about 29%, against single-digit MFU for the legacy rankers.
- Operating expense about 10% of the previous cascade.
- Deployed on the main Kuaishou and Kuaishou Lite feeds, handling about 25% of their traffic.
- App stay-time gains in the 0.5% to 1.2% range.

20.9.2 OneRec-V2 (August 2025)

Problem. In V1 most compute went into the encoder, and only a small fraction trained the generator. RL against a learned reward model was sample-inefficient and risked reward hacking [19][22].

Lazy decoder-only architecture [22]:

- A **context processor** computes key/value pairs from the user context once. They are shared across decoder layers with GQA, and there is no per-layer context projection.
- During training the decoder sees only 3 tokens: [BOS] plus the first two semantic IDs.
- Each layer has lazy cross-attention, causal self-attention, and an FFN, with MoE in the deeper layers.

Architecture (1B params)	GFLOPs	Converged loss
Encoder-decoder (1:1)	296.36	3.28
Lazy decoder-only	18.89	3.27

That is about 94% less compute and about 90% less training resource for the same loss [22].

Scaling law [22]:

- Dense models from 0.1B to 8B fit L(N) = 3.13 + 3660 / N^0.489.
- Loss falls 3.57 (0.1B) -> 3.33 (0.5B) -> 3.27 (1B) -> 3.23 (2B) -> 3.20 (4B) -> 3.19 (8B).
- A 4B-total / 0.5B-active MoE reaches 3.22, better than the 2B dense model.

RL from real user feedback. Rewards come directly from logged watch time, not from a learned reward model [22]:

- Watch time is bucketed logarithmically by duration. The top 25% within a bucket get a positive label, explicit negative feedback gets a negative label, and the rest are ignored.
- **GBPO** (Gradient-Bounded Policy Optimisation) keeps gradients for all samples but bounds the RL gradient by a BCE-style gradient. This avoids the clipping instability of PPO, GRPO and ECPO.

Reported online stay-time gains are again sub-1% (approximate) [19].

20.9.3 OneRec-Think, OpenOneRec and the "One" family

- **OneRec-Think (October 2025)** puts reasoning in the generator. It starts from Qwen3 with a vocabulary extended by itemic tokens and trains in three stages: itemic

alignment, reasoning activation (chain-of-thought SFT), and reasoning enhancement with a reward that accepts multiple valid answers. It reports a further +0.159% app stay time online [20].

- **OpenOneRec (December 2025)** is Kuaishou's open release [21]:
  - **RecIF-Bench**: about 100M interactions from 200K users across short video, ads and products, with 8 tasks in a four-layer capability hierarchy.
  - **Checkpoints**: OneRec-1.7B and OneRec-8B on Qwen3 backbones, trained on about 33B tokens, and "-Pro" variants trained on about 130B tokens, including an industrial Kuaishou corpus.
  - **Training pipeline**: itemic-text alignment and co-pretraining, then multi-task SFT, on-policy distillation and RL.
  - **Results**: OneRec-8B-Pro scores short-video Recall@32 of 0.0369, against 0.0119 for SASRec and 0.0180 for LC-Rec. The README reports a 26.8% average Recall@10 gain over the second-best method on 10 Amazon datasets.
- **MiniOneRec** (October 2025) is a community reproduction with RQ-KMeans semantic IDs, SFT and GRPO with constrained beam search [36].
- **The "One" recipe beyond the feed.** Kuaishou has applied the same recipe to e-commerce search (OneSearch) and other surfaces [35]. Long-sequence generative work continues with GEMs (2026), which uses a multi-stream decoder [34].

20.10 Comparison

System	Org / year	Stage	Core idea	Scale / reported result
For You feed disclosure [11]	TikTok 2020	whole feed	Weighted multi-signal ranking + diversity rules	Public description only
"Algo 101" formula [12]	TikTok (reported) 2021	ranking fusion	Sum of P_action * V_action	Goals: retention, time spent
Monolith [1][2]	ByteDance 2022	training infra	Cuckoo-hash PS, admission/expiry, online training	Minute-level sparse sync
Deep Retrieval [3]	ByteDance 2020	retrieval	Learned D x K path codes, beam search	~Brute-force accuracy
LONGER [4]	ByteDance 2025	ranking	End-to-end 10k sequences, token merge, KV cache	Dozens of scenarios
RankMixer [6]	ByteDance 2025	ranking	Token mixing + per-token FFN/MoE	MFU 4.5% -> 45%; Douyin +0.5% duration
TokenMixer-Large [7]	ByteDance 2026	ranking	RankMixer successor	7B / 15B offline
HLLM [8]	ByteDance 2024	seq. rec	Item LLM + user LLM	1B / 7B checkpoints
PERSIA [25]	Kuaishou+ETH 2022	training infra	Async embeddings, sync dense	Up to 100T params
TWIN [13]	Kuaishou 2023	ranking	Consistent GSU/ESU, cached item keys	99.3% less bottleneck
TWIN V2 [14]	Kuaishou 2024	ranking	Hierarchical clustering of 10^5+ behaviours	+0.67% to +0.80% watch time

System	Org / year	Stage	Core idea	Scale / reported result
KuaiFormer [16]	Kuaishou 2024	retrieval	Multi-query causal Transformer, compression	400M+ DAU
HoME / Pantheon [26][27]	Kuaishou 2024-25	MTL / fusion	Expert hierarchy; learned ensemble sort	~400M daily users
RLUR [28]	Kuaishou 2023	fusion policy	RL on fusion weights for return time	Retention gains
OneRec V1 [17] [18]	Kuaishou 2025	whole cascade	Enc-dec + MoE + SIDs + IPA/ECPO	~25% traffic; OPEX ~10%
OneRec-V2 [19]	Kuaishou 2025	whole cascade	Lazy decoder-only, RL on real watch time	94% less compute at equal loss
OpenOneRec [21]	Kuaishou 2025	foundation	Qwen3 + itemic tokens, open weights	1.7B / 8B

20.11 Cross-cutting design patterns

1. **Freshness through streaming, not just features.** Monolith (online training with incremental sparse sync) and PERSIA (async sparse updates) both separate the fast-moving sparse part of the model from the slow dense part. Kuaishou's Pantheon trains its fusion policy jointly with a real-time ranker [27]. In short video, a model that is a day old is a worse model.
2. **The value model is the product.** TikTok's reported formula, Kuaishou's ensemble sort and RLUR's learned fusion weights all show that the final fusion layer carries the business objective. Over time that layer has become personalised and learned [12] [27][28].
3. **Compute where the user signal is.** TWIN caches item-side projections, KuaiFormer and LONGER merge old tokens, TWIN V2 clusters, and OneRec compresses 100,000 records to 128 tokens. Every system splits the cost into a user-only part computed once per request and a per-candidate part that must be cheap [4][13][14][16][22].
4. **MFU as a first-class metric.** RankMixer (4.5% -> 45%) and OneRec (about 24%/29% training/inference) show that redesigning models for GPU efficiency is a prerequisite for scaling parameters [6][18].
5. **From cascade to generation.** Deep Retrieval's path codes (2020) anticipate semantic-ID generation. OneRec takes it to the whole cascade, keeping a reward model or real-feedback RL in place of the old multi-stage objectives [3][17][19].
6. **Debias the label before you scale.** Duration bias (D2Q, OneRec-V2's bucketed watch time) and viral-content concentration (OneRec SIR) are both handled explicitly in labels and rewards [22][30].

Key takeaways for system designers

- **Design for minute-level freshness from day one.** Use a streaming joiner keyed by a serving-time request ID, an online trainer, and incremental sync of only the touched sparse rows. Snapshot rarely, because online training heals PS losses quickly [1].

- **Use collisionless, dynamic embedding tables with admission and eviction** for unbounded, churning ID spaces such as videos and creators. Hash collisions are a silent quality tax [1][2].
- **Make the cheap stage agree with the expensive one.** TWIN's lesson is that the first-stage selection metric should match the final model's attention. Inconsistent stages waste the expensive model's capacity [13].
- **Compress old history, keep recent history exact.** Hierarchical clustering, group merging and QFormer compression all reach 10^4 to 10^5 behaviours within latency budgets [14][16][22].
- **Treat watch time carefully.** Normalise for duration (quantiles or log-buckets) or the system will learn to promote long videos [22][30].
- **Separate prediction from policy, then learn the policy.** Start with an explicit P_action * V_action fusion that product owners can read, then move to personalised or RL-learned weights. RLUR and Pantheon do this with no change to the ranker [12][27][28].
- **Measure and fix MFU before scaling parameters.** Token-based rankers (RankMixer, OneTrans) and decoder-only generators (OneRec-V2) earn their size by being GPU-friendly [6][10][22].
- **If you go generative, budget compute for the decoder and use real feedback.** OneRec-V2 matched encoder-decoder quality with about 94% less compute and replaced the learned reward model with bucketed real watch-time signals, which reduces reward hacking [22].
- **Keep an explicit rule layer.** TikTok's public description keeps diversity rules (no back-to-back same creator or sound) outside the model, and OneRec adds explicit viral-content control [11][22].
- **Expect small percentages to matter.** At Kuaishou scale a 0.1% watch-time gain is significant. Plan experiments with enough power to detect sub-1% effects [14].

References

1. Zhuoran Liu et al. (ByteDance). "Monolith: Real Time Recommendation System With Collisionless Embedding Table." arXiv 2209.07663, 2022. https://arxiv.org/abs/2209.07663
2. ByteDance. "Monolith: A Lightweight Recommendation System" (GitHub repository, archived October 2025). Accessed 2026. https://github.com/bytedance/monolith
3. Weihao Gao et al. (ByteDance). "Deep Retrieval: Learning A Retrievable Structure for Large-Scale Recommendations." arXiv 2007.07203, 2020. https://arxiv.org/abs/2007.07203v1
4. ByteDance. "LONGER: Scaling Up Long Sequence Modeling in Industrial Recommenders." RecSys 2025, arXiv 2505.04421. https://arxiv.org/abs/2505.04421
5. Machine Learning at Scale. "[RecSys] Part 3: LONGER, scaling up long sequence modelling in industrial recommenders." Substack, 2025. https://machinelearningatscale.substack.com/p/recsys-part-3-longer-scaling-up-long
6. ByteDance. "RankMixer: Scaling Up Ranking Models in Industrial Recommenders." arXiv 2507.15551, 2025. https://arxiv.org/abs/2507.15551

7. ByteDance. "TokenMixer-Large: Scaling Up Large Ranking Models in Industrial Recommenders." arXiv 2602.06563, 2026. https://arxiv.org/abs/2602.06563
8. ByteDance. "HLLM: Enhancing Sequential Recommendations via Hierarchical Large Language Models for Item and User Modeling." arXiv 2409.12740, 2024; code and checkpoints (archived): https://github.com/bytedance/HLLM ; paper: https://arxiv.org/abs/2409.12740
9. Haozhe Xing. "RankMixer: A Hardware-Efficient Architecture" (fun-rec-mdbook technical notes). 2026. https://github.com/Haozhe-Xing/fun-rec-mdbook/blob/main/src/en/part7-scaling/rankmixer.md
10. Haozhe Xing. "OneTrans: A Unified Transformer" (fun-rec-mdbook technical notes). 2026. https://github.com/Haozhe-Xing/fun-rec-mdbook/blob/main/src/en/part7-scaling/onetrans.md
11. TikTok. "How TikTok recommends videos #ForYou." TikTok Newsroom, June 2020 (not re-fetched for this chapter). https://newsroom.tiktok.com/en-us/how-tiktok-recommends-videos-for-you
12. Ben Smith. "How TikTok Reads Your Mind." The New York Times, 5 December 2021 (reporting on the internal "TikTok Algo 101" document; not re-fetched for this chapter). https://www.nytimes.com/2021/12/05/business/media/tiktok-algorithm.html
13. Jianxin Chang et al. (Kuaishou). "TWIN: TWo-stage Interest Network for Lifelong User Behavior Modeling in CTR Prediction at Kuaishou." KDD 2023, arXiv 2302.02352. https://arxiv.org/abs/2302.02352
14. Zihua Si et al. (Kuaishou). "TWIN V2: Scaling Ultra-Long User Behavior Sequence Modeling for Enhanced CTR Prediction at Kuaishou." CIKM 2024, arXiv 2407.16357. https://arxiv.org/abs/2407.16357
15. Machine Learning Frontiers (S. Bhattacharya). "Target Attention Is All You Need: Modeling Extremely Long User Action Sequences in Recommender Systems." Substack, 2024. https://mlfrontiers.substack.com/p/target-attention-is-all-you-need
16. Kuaishou. "KuaiFormer: Transformer-Based Retrieval at Kuaishou." arXiv 2411.10057, 2024. https://arxiv.org/abs/2411.10057
17. Jiaxin Deng et al. (Kuaishou). "OneRec: Unifying Retrieve and Rank with Generative Recommender and Iterative Preference Alignment." arXiv 2502.18965, February 2025. https://arxiv.org/abs/2502.18965
18. Kuaishou OneRec Team. "OneRec Technical Report." arXiv 2506.13695, June 2025. https://arxiv.org/abs/2506.13695
19. Kuaishou OneRec Team. "OneRec-V2 Technical Report." arXiv 2508.20900, August 2025. https://arxiv.org/abs/2508.20900
20. Kuaishou OneRec Team. "OneRec-Think: In-Text Reasoning for Generative Recommendation." arXiv 2510.11639, October 2025. https://arxiv.org/abs/2510.11639
21. Kuaishou OneRec Team. "OpenOneRec Technical Report" (arXiv 2512.24762) and OpenOneRec repository. December 2025. https://arxiv.org/abs/2512.24762 ; https://github.com/Kuaishou-OneRec/OpenOneRec
22. Haozhe Xing. "End-to-End Generative Recommendation" (OneRec / OneRec-V2 technical notes, fun-rec-mdbook). 2026. https://github.com/Haozhe-Xing/fun-rec-mdbook/blob/main/src/en/part8-e2e/e2e-recommendation.md
23. Kuaishou. "RecFlow: An Industrial Full Flow Recommendation Dataset." ICLR 2025, arXiv 2410.20868. https://arxiv.org/html/2410.20868v1
24. "UniRec: Cross-stage Multi-Task Fusion with Preference Alignment for Cascaded Recommender Systems." arXiv 2609.11052, 2026. https://arxiv.org/pdf/2609.11052

25. Xiangru Lian et al. (Kuaishou and ETH Zurich). "Persia: A Hybrid System Scaling Deep Learning Based Recommenders up to 100 Trillion Parameters." KDD 2022, arXiv 2111.05897. https://arxiv.org/abs/2111.05897
26. Xu Wang et al. (Kuaishou). "HoME: Hierarchy of Multi-Gate Experts for Multi-Task Learning at Kuaishou." arXiv 2408.05430, 2024 (KDD 2025). https://arxiv.org/abs/2408.05430
27. Cao et al. (Kuaishou). "Pantheon: Personalized Multi-objective Ensemble Sort via Iterative Pareto Policy Optimization." arXiv 2505.13894, 2025. https://arxiv.org/abs/2505.13894
28. Qingpeng Cai et al. (Kuaishou). "Reinforcing User Retention in a Billion Scale Short Video Recommender System." WWW 2023 Companion, arXiv 2302.01724. https://arxiv.org/abs/2302.01724
29. Qingpeng Cai et al. (Kuaishou). "Two-Stage Constrained Actor-Critic for Short Video Recommendation." WWW 2023, arXiv 2302.01680. https://arxiv.org/abs/2302.01680
30. Ruohan Zhan et al. (Kuaishou). "Deconfounding Duration Bias in Watch-time Prediction for Video Recommendation." KDD 2022, arXiv 2206.06003. https://arxiv.org/abs/2206.06003
31. Shangfeng Dai et al. (Kuaishou). "POSO: Personalized Cold Start Modules for Large-scale Recommender Systems." arXiv 2108.04690, 2021. https://arxiv.org/abs/2108.04690
32. Yuxin Ren et al. (Kuaishou). "Non-autoregressive Generative Models for Reranking Recommendation" (NAR4Rec). KDD 2024, arXiv 2402.06871. https://arxiv.org/abs/2402.06871
33. Chongming Gao et al. "KuaiRec: A Fully-observed Dataset and Insights for Evaluating Recommender Systems." CIKM 2022, arXiv 2202.10842; dataset repository. https://arxiv.org/abs/2202.10842 ; https://github.com/chongminggao/KuaiRec
34. Kuaishou. "GEMs: Long-sequence generative recommendation with a multi-stream decoder." arXiv 2602.13631, 2026. https://arxiv.org/abs/2602.13631
35. Kuaishou. "OneSearch: A Preliminary Exploration of the Unified End-to-End Generative Framework for E-commerce Search." arXiv 2509.03236, 2025. https://arxiv.org/abs/2509.03236
36. Kong et al. "MiniOneRec: An Open-Source Framework for Scaling Generative Recommendation." arXiv 2510.24431, 2025; code: https://github.com/AkaliKong/MiniOneRec ; paper: https://arxiv.org/abs/2510.24431
37. Kuaishou. "GRADE: Personalized Multi-Task Fusion via Group-relative Reinforcement Learning with Adaptive Dirichlet Exploration." arXiv 2510.07919, 2025. https://arxiv.org/abs/2510.07919

21. Twitter/X Case Study: From WTF and GraphJet to the 2023 the-algorithm and the 2026 Phoenix Transformer

Twitter (renamed X in 2023) is the most thoroughly documented large-scale recommender in public. Over about thirteen years the company published or open-sourced each major generation of its stack:

- the Who-to-Follow (WTF) system and its in-memory graph engine Cassovary (WWW 2013) [14];
- GraphJet, a real-time bipartite-graph engine for content recommendation (VLDB 2016) [12][13];
- SimClusters community embeddings (KDD 2020) [8][9] and TwHIN heterogeneous-graph embeddings (KDD 2022) [15][16];
- the March 2023 release of `twitter/the-algorithm` and `twitter/the-algorithm-ml`, which exposed the full For You pipeline: Home Mixer, candidate sources, the Earlybird light ranker, the MaskNet heavy ranker, heuristics and visibility filtering [1][2][3][4][6];
- the January 2026 release of `xai-org/x-algorithm`, a Rust and JAX rebuild. It drops almost all hand-engineered features and ranks with a transformer derived from Grok-1, called Phoenix. Later updates in May, August and September 2026 published production model code, production scoring weights, and the trust-and-safety labeling stack [28][29][30][31].

The 2023 and 2026 systems are close to a controlled experiment in recommender design. The product, the candidate volumes and the multi-action value model barely change between them. The 2023 system is a feature-engineered ensemble of about 6,000 features feeding a parallel MaskNet [4][6]. The 2026 system is an ID- and sequence-based transformer that reads the viewer's raw engagement history [28][29]. This chapter covers both, plus the graph systems that came before them, and ends with lessons for system designers.

A note on sourcing: everything below comes from Twitter/X's own repositories and READMEs, read directly for this chapter. Figures that come only from the 2023 engineering blog post [2] are marked as approximate. That post and the academic papers [9][13][14] [15][17] could not be re-fetched from this research environment, so they are cited by their canonical URLs.

21.1 Pre-history: Who-to-Follow, Cassovary and RealGraph

Problem

Twitter's first large recommendation problem was account recommendation rather than tweet recommendation. A new user follows few accounts, so the home timeline is empty, and the follow graph is what makes the product work. WTF (Gupta, Goel, Lin, Sharma, Wang, Zadeh; WWW 2013) [14] describes the first production system.

Architecture

The main design bet in WTF was to keep the entire follow graph in memory on a single server, using an in-house graph library called Cassovary, instead of partitioning it across a cluster [14]. The authors argued that the graph fit in the RAM of a large commodity machine. Single-machine random-walk algorithms were far simpler and faster than distributed graph processing, and the system could be replicated for throughput.

The core algorithm runs in two stages [14]:

1. **Circle of trust.** For user u, run an egocentric random walk (personalized PageRank) from u and keep the top-k nodes. These are the accounts most "socially near" u.
2. **SALSA on a bipartite graph.** Put the circle of trust on the left ("hubs") and the accounts they follow on the right ("authorities"). Run SALSA, a HITS-like random walk, over this bipartite graph. High-scoring authorities become follow recommendations, and high-scoring hubs are users similar to u.

```
          follow graph (in memory, Cassovary)
                       |
user u --> personalized PageRank --> circle of trust (hubs, top-k)
                       |
          hubs  ==follow edges==>  authorities
                       |
                 SALSA iterations
                       |
   authorities ranked --> "Who to Follow" candidates --> filters --> UI
```

Lessons from WTF

The paper is candid about the system's limits. A single-server design requires periodic graph snapshots, so it is not real-time. Later work moved toward real-time engines; GraphJet is the direct descendant [13]. WTF also established three ideas that every later Twitter system keeps: interaction graphs as the main signal, random walks for candidate generation, and SALSA-style bipartite scoring.

RealGraph and Tweepcred

Two user-level graph models from that period still existed in the 2023 release.

- **RealGraph** [10] predicts how likely user A is to interact with user B. A daily Dataflow (Scio) job aggregates pairwise interactions. These include public ones (favorites, retweets, follows) and private ones (profile views, tweet clicks, and, if the user opted in, address-book membership). A rollup job keeps a decayed sum. A gradient-boosted tree classifier, trained on "edges active in a period, labelled by whether they interact the next day", outputs a pairwise interaction score. In the 2023 For You pipeline, RealGraph decides which followed accounts matter most. It weights in-network candidates and serves as a source signal for CR-Mixer and UTEG [2][20].

- **Tweepcred** [25] is a weighted or unweighted PageRank over the user interaction graph (mentions, retweets and so on), implemented as an iterative Hadoop/Scalding job. It produces a per-user reputation score that the Earlybird light ranker uses as a feature [18][25]. Its 2026 successor, `user-cred-v2/`, still "runs PageRank over the follow graph and engagement edges" [28].

21.2 GraphJet: Real-Time Collaborative Filtering on a Single Server

Problem

Tweets have a very short useful life, often hours. Collaborative filtering over a nightly batch graph is too stale for them. Twitter needed "users who engaged with X also engaged with Y" recommendations computed over the last day or so of activity, with fresh edges visible within seconds.

Architecture

GraphJet (Sharma, Jiang, Bommannavar, Larson, Lin; VLDB 2016) [13] is a Java library that "maintains a full graph index over a sliding time window in memory on a single server" [12]. Its key features:

- **Bipartite graph.** Users are on the left and tweets (or URLs, hashtags) on the right. Edges are interactions such as posts, retweets and likes [12].
- **Temporal partitioning.** The index is a sequence of time-sliced segments. New edges go into the newest segment, and whole segments are dropped as they leave the window. This avoids per-edge deletion and garbage-collection pressure [13].
- **Compact edge encoding and dynamic memory allocation.** These let the engine ingest edges and serve queries concurrently [12].
- **Random-walk algorithms.** SALSA-style personalized walks are the main recommendation primitive, as in WTF [13].

Throughput as stated in the README [12]:

Metric	Value per GraphJet server
Edge ingestion	up to 1,000,000 edges/s
Recommendations (steady state)	up to 500/s
Edge reads	"several million" per second

Production descendants

In the 2023 open-source release, GraphJet powers several services under `src/scala/com/twitter/recos`. The most important for Home is **UTEG (User-Tweet-Entity-Graph)** [11]:

- It is "a Finagle thrift service built on the GraphJet framework" that produces the "XXX Liked" out-of-network tweets on the home timeline [11].

- It takes as input the user's **weighted follow graph** (a list of weighted user IDs, with weights from RealGraph). It traverses the graph, aggregates, and returns tweets ranked by how many weighted followees engaged with them [11].
- It is stateful and fed by Kafka. It keeps **in-memory engagements for the past 24-48 hours** and drops older events [11].
- **Recos-Injector** [26] is the streaming preprocessor for all GraphJet services. It consumes favorites, retweets, follows and client events, filters and combines them, and publishes them to a dedicated Kafka topic per GraphJet service.

```
Event Bus / Kafka (fav, RT, follow, client events)
          |
     recos-injector  (filter, combine, per-service topic)
          |
  +--------+---------------------+
  |                              |
 UTEG (GraphJet, 24-48h window)  other GraphJet recos services
  |
 input: viewer's RealGraph-weighted followees
 traverse user->tweet edges, aggregate engagement weight
  |
 top tweets with social proof "<followee> liked"
```

Lessons from GraphJet

A sliding window kept entirely in RAM is cheap and very fresh for collaborative filtering on short-lived content. The price is that it only sees about the last 1-2 days, which is acceptable because tweets decay quickly. Single-server replication is again preferred to sharding. The same "in-memory recent-content store per server" idea comes back in 2026 as **Thunder** (Section 21.6).

21.3 Representation Layer: SimClusters and TwHIN

Two embedding systems give the 2023 stack its out-of-network reach. They are sparse community vectors (SimClusters) and dense knowledge-graph vectors (TwHIN). Both are served through `representation-manager` and scored by `representation-scorer` [1].

21.3.1 SimClusters (KDD 2020)

Problem. Twitter needed one general-purpose representation of users, tweets, topics and other entities that was interpretable and cheap to compute and update [8][9].

Algorithm (from the official README [8]):

1. Treat the follow graph as a bipartite **producer-consumer** matrix A (consumers follow producers).
2. Compute **producer-producer cosine similarity** over follower sets, and drop edges below a threshold.
3. Run **Metropolis-Hastings sampling-based community detection** on the producer similarity graph with k communities. The result is a maximally sparse **KnownFor** matrix V (n x k), with each producer in at most one community. In production KnownFor covers **the top 20M producers and k is about 145,000 communities**.

4. **InterestedIn** (consumer embedding) U = A x V, followed by noise removal. This captures long-term consumer interest and is "one of our major source[s] for consumer-based tweet recommendations".
5. **Producer embeddings** are the cosine similarity between a producer's follower vector and each community's InterestedIn vector. A producer can belong to many communities.
6. **Tweet embeddings** start empty when the tweet is created. **Each time a user favorites the tweet, that user's InterestedIn vector is added** to the tweet vector. A Heron/Summingbird streaming job does this in real time, and a separate job persists the embeddings from Memcache to Manhattan [8].
7. **Topic embeddings** are built from fav-based Topic-Follow-Graph aggregates with time decay [8].

```
follow graph A (consumers x producers)
       |  cosine sim over followers, threshold
       v
producer-producer graph --MH community detection (k~145K)--> KnownFor V (20M x 145K)
       |
  U = A . V  ---> InterestedIn (consumer, long-term)
       |                        \
       |                         +--> producer embeddings (multi-community)
  fav stream (Heron) --> tweet vec += InterestedIn(faver)   [real-time]
       |
SimClusters ANN: cluster -> top tweets index; user vec -> candidate tweets
```

Serving. SimClusters ANN (SANN) indexes cluster -> top tweets. BigQuery/GCP jobs build engagement-specific indexes, for example a push-open-based index for Notifications and a video-view-based index for Home video [8]. Because the vectors are sparse, retrieval is an inverted lookup: for each of the user's top clusters, fetch that cluster's top tweets, then score them by dot product.

Durability. SimClusters survives into the 2026 X stack. The August 2026 update added `simclusters/` as "an additional source of posts from accounts the viewer does not follow that is called in retrieval alongside Thunder and Phoenix retrieval". It has a 48-hour maximum candidate age and an `OONNsfwSimclustersFilter` [28][30].

21.3.2 TwHIN (KDD 2022)

Problem. SimClusters is built mainly from follow and favorite graphs. Twitter has many more relation types (follows, favorites, retweets, replies, ad clicks) and wanted one dense embedding space covering users, tweets, advertisers and other entities [15][16].

Approach. TwHIN (El-Kishky et al.) [15] trains knowledge-graph embeddings over a heterogeneous information network. Edges are (lhs, relation, rhs) triples, and each relation is modelled as a **translation operator** in the embedding space, in the TransE family. The open-sourced TorchRec version [16] expects parquet edges with `lhs, rel, rhs` columns. Its sample config defines `fav`, `reply`, `retweet` and `magic_recs` relations, all user -> tweet with `operator: translation`, and trains with in-batch negatives. Twitter also released heavily subsampled, anonymized follow and favorite graphs on Hugging Face [16].

Uses. TwHIN embeddings are used "for candidate retrieval and as model features in a variety of recommender system models" [16]. In For You they feed ANN out-of-network candidates through CR-Mixer and are features in the heavy ranker [2]. Later Home Mixer

parameters also show TwHIN used for diversity: `twhin_diversity_rescoring_weight` and `twhin_diversity_rescoring_ratio` [7].

SimClusters vs TwHIN

Property	SimClusters [8]	TwHIN [15][16]
Vector type	Sparse, ~145K dimensions, interpretable communities	Dense, low-dimensional
Input graph	Follow graph (+ fav for tweets/topics)	Multi-relation heterogeneous graph
Training	Community detection + matrix products	Translation-based KG embedding (SGD)
Tweet update	Real time: add InterestedIn of each faver	Batch-trained
Retrieval	Inverted cluster -> top-tweets index	ANN over dense vectors
Still used in 2026?	Yes (`simclusters/` source) [28]	Not in the 2026 public repo

---

21.4 The 2023 For You Pipeline (the-algorithm)

21.4.1 Overview and scale

On 31 March 2023 Twitter published the code for the For You timeline and Recommended Notifications [1][2]. The engineering blog [2] describes the funnel as follows. These numbers are approximate and as reported in the blog:

- About **500 million tweets** posted per day.
- For each request, about **1,500 candidate tweets** are fetched, around **50% in-network and 50% out-of-network** on average (the repo README also says "~50% of posts come from" the Earlybird search index [1]).
- Ranking uses a neural network of about **48M parameters** trained on tweet interactions, with **thousands of features**, predicting **10 labels**.
- The pipeline runs about **5 billion times a day** and completes in **under 1.5 s on average**. One execution takes about **220 s of CPU time**, roughly 57x its wall-clock latency.

The home-mixer README [6] confirms the stages and the feature count: "Fetch the ~6000 features needed for ranking."

21.4.2 Architecture

```
client request (For You)
       |
 HOME MIXER (Scala, Product Mixer framework)
 ForYouProductPipelineConfig
  +-- ForYouScoredTweetsMixerPipeline  (mix tweets + ads + WTF users)
  |     +-- ScoredTweetsRecommendationPipeline
  |     |     candidate pipelines (fetch caps from ScoredTweetsParam):
  |     |       InNetwork (Earlybird)   <= 600
  |     |       TweetMixer / CR-Mixer   <= 400   (SimClusters, TwHIN ANN, ...)
  |     |       UTEG (GraphJet)         <= 300
  |     |       FRS (FutureGraph)       <= 100
  |     |     feature hydration (~6000 features)
  |     |     ScoringPipeline: heavy ranker (MaskNet, Navi)
  |     |     heuristics: author diversity, in/out balance, fatigue, dedup
  |     |     visibility filtering
  |     +-- ConversationService pipeline  (reverse-chron backup on failure)
  |     +-- Ads pipeline
  |     +-- WhoToFollow pipeline (FRS)
  +-- marshal -> timeline (modules, social context, cursors) -> client
```

The candidate caps come from the open-sourced `ScoredTweetsParam.scala` [7]. They are `in_network_max_tweets_to_fetch = 600`, `tweet_mixer_max_tweets_to_fetch = 400`, `uteg_max_tweets_to_fetch = 300`, `frs_max_tweets_to_fetch = 100` and `server_max_results = 50`. Together that is about 1,400 candidates, close to the blog's "~1,500". The result is a page of about 50 posts. The same file has `out_of_network_scale_factor = 0.75`, so out-of-network scores are multiplied by 0.75 [7]. These are code defaults at the last snapshot of the repo and may have been overridden by feature switches in production.

21.4.3 Product Mixer: the framework

Home Mixer is built on **Product Mixer**, a Scala framework of composable pipelines [6][24]:

- **Product Pipelines** choose which Mixer or Recommendation Pipeline to run for a request.
- **Mixer Pipelines** combine heterogeneous candidates (tweets, ads, users).
- **Recommendation Pipelines** score and rank homogeneous candidates through **Scoring Pipelines**.
- **Candidate Pipelines** fetch from Candidate Sources, filter, decorate and hydrate features.

This split between configuration and business logic carries straight into the 2026 Rust `candidate-pipeline` crate (Section 21.6). It is one of the most durable patterns in the codebase.

21.4.4 In-network source: Earlybird and the light ranker

Earlybird [19] is Twitter's Lucene-based real-time search system. It has three clusters:

- a **realtime** cluster with public tweets from roughly the last 7 days;
- a **protected** cluster with protected tweets over the same window;
- an **archive** cluster with all tweets up to about two days ago.

Each cluster is sharded into partitions with a single-writer, multiple-reader inverted index. Roots fan out requests and merge the results. For Home, the query is essentially "recent tweets authored by this list of followees", so retrieval is a user-ID search [19].

Inside Earlybird, the **light ranker** [18] scores candidates before they leave the index:

- It is a **logistic regression** predicting engagement. There are two variants: `recap_earlybird` for in-network and `rectweet_earlybird` for UTEG out-of-network tweets.
- Static features (computed at ingestion) include is-retweet, has-link, has-trend-words, is-reply, a **text quality score** (offensiveness, entropy, "shout" score, length, readability) and **tweepcred**.
- Realtime features (updated through a Heron signal ingester) include like, reply and retweet counts, plus **pToxicity and pBlock** health-model scores.
- The README's own warning reads: "the light ranker is an old part of the stack which we are currently in the process of replacing. The current model was last trained several years ago, and uses some very strange features" [18].

RealGraph scores are used to choose and weight the followee list that Earlybird searches [2].

21.4.5 Out-of-network sources

- **CR-Mixer / Tweet Mixer** [20] is a coordination layer for out-of-network candidate generation. It runs four steps: source-signal extraction (from UserProfileService, RealGraph and similar), candidate generation from compute services (SimClusters ANN, TwHIN ANN, and so on), filtering, and light ranking. Its stated aim is "a 1-stop-shop" that speeds up iteration on candidate sources [20].
- **UTEG** [11] is GraphJet collaborative filtering (Section 21.2). The blog attributes about 15% of For You tweets to social-graph traversal of this kind (approximate) [2].
- **FRS** [21] is the Follow Recommendations Service. Besides Who-to-Follow modules, it supplies "FutureGraph" tweets from accounts the user may follow in future. FRS ranks account candidates by a weighted sum of p(follow | recommendation) and p(positive engagement | follow). Its flow is: candidate generation, light filters, ranking, heavy filters, transforms (social proof such as "followed by X"), then truncation [21].

21.4.6 The heavy ranker (MaskNet)

The heavy ranker is open-sourced in `the-algorithm-ml` (PyTorch/TorchRec) [3][4].

Inputs. Features are documented in FEATURES.md [5]. Most of them are **aggregate features** from the Timelines Aggregation Framework. These are rolling counts over scopes such as user, author and user-author, crossed with engagement types. They are kept over long windows (50 days) and real-time windows (under 3 days, typically 30 minutes), for example `user_aggregate_v2.pair.recap.engagement.is_favorited.....50.days.count`. Other inputs include RealGraph, SimClusters and TwHIN features, author features and tweet content features [5].

Architecture. The backbone is a **parallel MaskNet** (Wang et al., 2021) [17]. From the public config `local_prod.yaml` and `mask_net.py` [36][37]:

- Four MaskBlocks run in parallel (`use_parallel: true`), each with `aggregation_size 1024` and `output_size 1024`.

- Each MaskBlock computes `LayerNorm(W_h (x * MLP_mask(x)))`. Here `MLP_mask` is Linear -> ReLU -> Linear back to the input dimension. This is an instance-guided multiplicative mask over the feature embedding, a learned feature-wise gating.
- The four block outputs (4 x 1024) are concatenated and passed through a shared MLP of width 2048.
- The multi-task type is `share_all`. Each task has its own MLP head `256 -> 128 -> 1` with batch norm, and dropout 0.1 on the negative-feedback head.
- Input featurization uses a `double_norm_log` transform: log, then batch norm and layer norm with magnitude clipping at 5.0.

```
~6000 features (aggregates, graph, embeddings, content)
      |  double-norm-log featurization
      v
+-----------+-----------+-----------+-----------+
| MaskBlock | MaskBlock | MaskBlock | MaskBlock |   parallel
| x*mask(x) | x*mask(x) | x*mask(x) | x*mask(x) |   (1024 each)
+-----------+-----------+-----------+-----------+
      | concat (4096) -> MLP 2048 (shared)
      v
10 task heads: MLP 256-128-1, sigmoid  -> P(engagement_i)
      |
score = sum_i w_i * P_i   (weights from config)
```

Outputs and value model. There are 10 heads, each a probability. The weights published on 5 April 2023 [4] are:

Head (predicted event)	Weight
fav (like)	0.5
retweet	1.0
reply	13.5
good_profile_click (open profile and like/reply)	12.0
video_playback50 (watch >= 50%)	0.005
reply_engaged_by_author	75.0
good_click (open convo and reply/like)	11.0
good_click_v2 (open convo, stay >= 2 min)	10.0
negative_feedback_v2 (show less, block, mute)	-74.0
report	-369.0

The README explains that the weights "were originally set so that, on average, each weighted engagement probability contributes a near-equal amount to the score", and were later tuned against platform metrics [4]. The large weight on a reply that the author engages with (75) shows a deliberate preference for conversation over passive likes.

Serving. Models are served by **Navi**, Twitter's Rust ML-serving server. It is gRPC-compatible with TensorFlow Serving and has TensorFlow and ONNX runtimes; the ONNX runtime is used for "home recommendation with a proprietary BatchPredictRequest format" [23].

21.4.7 Heuristics, filters and mixing

After scoring, the home-mixer README lists the following steps [6]:

- **Author diversity**: demote consecutive tweets by the same author.
- **Content balance**: keep a target in-network vs out-of-network mix (see also `out_of_network_scale_factor`) [7].
- **Feedback fatigue**: demote content similar to things the user gave negative feedback on.
- **Deduplication**: remove previously seen tweets.
- **Social proof**: out-of-network tweets need a second-degree connection (the blog describes requiring that someone you follow engaged with the tweet or follows the author) [2].
- **Visibility filtering**: blocked or muted authors, NSFW settings and similar.
- **Mixing**: ads, Who-to-Follow modules, prompts, conversation modules for replies, edited tweets, and pagination.

Later snapshots of `ScoredTweetsParam.scala` also show newer features [7]:

- "Control AI" show-less and show-more scale factors (0.05 and 20.0) with an embedding-similarity threshold of 0.67;
- content-exploration pipelines for cold posts;
- category and TwHIN diversity rescoring;
- CLIP image clusters and multimodal embeddings as features.

21.4.8 Visibility filtering (VF)

`visibilitylib` [22] is "a centralized rule engine that instructs clients how to alter the display of certain Twitter content on read time". Its concepts are:

- **SafetyLevel**: the product context, such as Timeline or Profile.
- **Features**: safety labels, user flags, user-user relationships and user-content relationships.
- **Conditions** combined into **Rules**, which form a **Policy**. Rules are in priority order, with one policy per SafetyLevel.
- **Actions**: Drop (hard filter), Labels and Interstitials (soft filter), and "coarse-grained downranking".

The 2023 release explicitly omitted part of this code as "being reviewed and rebuilt" [22]. The 2026 release filled that gap (Section 21.7).

21.4.9 Recommended notifications

`pushservice` [27] follows the same funnel: build the target and check eligibility, fetch candidates (including SimClusters ANN indexes built on push opens [8]), hydrate, apply light filters, rank, then take the top candidate that survives heavy filtering. Ranking uses a **light ranker**, which pre-selects from the initial huge pool, and a **multi-task heavy ranker** that predicts open and engagement probabilities. Because the result is a single notification rather than a page, "take step" walks down the ranked list one candidate at a time, running expensive filters until one candidate passes [27].

21.5 Why X rebuilt it: limits of the 2023 design

The 2023 READMEs point to the weaknesses that motivated the 2026 rewrite:

1. **Feature sprawl.** There were about 6,000 features [6], mostly cross-product aggregates [5], spread over many offline jobs, streaming jobs and feature-serving services.
2. **Stale components.** The light ranker was "last trained several years ago" with "very strange features" [18]. `twml` was a "legacy" TensorFlow v1 framework, and `timelineranker` was legacy [1].
3. **Many heterogeneous candidate sources.** Earlybird, CR-Mixer, UTEG, FRS, SimClusters ANN and TwHIN ANN each had its own models and infrastructure.
4. **Heuristics layered on top.** Diversity, balance, fatigue and social-proof rules all altered the model's output.

21.6 The 2026 xAI Rebuild: x-algorithm and Phoenix

21.6.1 Release history

`xai-org/x-algorithm` was first published on **20 January 2026**, with further drops on **15 May**, **13-18 August**, and later notes dated **18 September 2026** (from the repository's git history and README) [28]. The first README called Phoenix "a Grok-based transformer model" and stated: "We have eliminated every single hand-engineered feature and most heuristics from the system. The Grok-based transformer does all the heavy lifting by understanding your engagement history" [28].

The first Phoenix release shipped "a sample transformer ported from the Grok-1 open source release" [34]. The current release ships "the production implementation itself: the real model code, the real training step, and the real Rust serving engine", with synthetic data generators in place of production data [29]. Everything is Apache 2.0 [28].

21.6.2 Request path

Home Mixer is now a Rust service on a generic `candidate-pipeline` crate. The crate's stage types are Source, Hydrator, Filter, Scorer, Selector and SideEffect, and independent stages run in parallel [28].

```
FOR YOU REQUEST
  |
HOME MIXER (Rust) -- Post Pipeline (PhoenixCandidatePipeline)
  1 query hydration: user action sequence (recent engagements = main model input),
    following list, blocks/mutes, muted keywords, seen/served posts, topics
  2 candidate sources (parallel)
      in-network : Thunder (in-memory recent posts of followees)   cap 1200
      out-of-net : Phoenix retrieval (two-tower ANN)               cap 1000
                   SimClusters (cluster similarity, <=48h)
                   Popular posts (shared list)                      cap 500
  3 candidate hydration: text/media, author + labels, quoted post, lang, counts
  4 pre-scoring filters (dedup, 48h age, self, OON replies/RTs, seen/served,
    muted kw, block/mute, subscription, election filter, holdouts ...)
  5 scoring: PhoenixScorer -> RankingScorer (weighted sum, author decay,
    OON discount, new-author boost) -> VMRanker (DPP rerank)
  6 selection: TopKScoreSelector
  7 post-selection: VFCandidateHydrator -> VFFilter -> AncillaryVFFilter
    -> DedupConversationFilter
  |
Blending Pipeline (ForYouCandidatePipeline): ads blender, WTF, prompts
  |
side effects (after response): record served posts, refresh cache, log events
```

Source caps are the defaults in `param.rs`: `PhoenixMaxResults = 1000`, `ThunderMaxResults = 1200`, `PopularPostsMaxResults = 500`, cold-start Phoenix 200, and an optional semantic-ID source of 800. The legacy `TweetMixer` source is still present but disabled (`EnableTweetMixerSource = false`) [30]. The file header says these values are "mirrored from config feature-switch defaults" by a cron job, so the repo tracks primary production values [28][30].

21.6.3 Thunder: the in-network store

Thunder replaces Earlybird for in-network retrieval. It is "an in-memory post store and realtime ingestion pipeline" that consumes post create and delete events from Kafka, serves posts by the viewer's followees, and trims posts older than a retention period. The README promises "sub-millisecond lookups for in-network content without hitting an external database" [28]. It is the GraphJet idea again: a time-windowed in-memory index replaces a general-purpose search engine. Thunder also receives the already-seen list so it can exclude those posts at the source [28][35].

21.6.4 Phoenix retrieval (two-tower)

From the Phoenix README [29]:

- **User tower.** A transformer encodes the user's engagement history plus a single user-features token (country, language and similar) into a normalized embedding. Production retrieval uses **no learned per-user ID embedding** (`use_user_embedding=False`). The user is represented by what they interacted with.
- **Candidate tower.** Normalized post embeddings. Since a "semantic-ID migration", posts are represented by **semantic IDs (SIDs)**: residual-quantized codes, **6 levels x 256 codes**, derived from each post's multimodal embedding, plus hashed author IDs. Same-topic posts share SID prefixes, which helps the model generalize to unseen posts.
- **Index inside the checkpoint.** At every checkpoint save, the trainer runs the candidate tower over the corpus and stores `post_embeddings` in the checkpoint. Serving loads the index from there, so "nothing embeds a corpus at boot time". The

production index (`max_posts`) holds **10.24M posts (28.67M for the combined config)**.
- **Training.** Contrastive, with **favorites as positives**, using in-batch plus **64 sampled global negatives per example with log-Q correction**.
- **Index maintenance.** `phoenix-rankall` keeps the index updated as events arrive. `phoenix-rankall-strato` decides which index a post belongs in, "consulting visibility filtering first" [28].

21.6.5 Phoenix ranking (transformer with candidate isolation)

The ranking model is a transformer over the sequence `[user tokens | history | candidates]` [29]:

- **User tokens** ([B, 2]): hashed IDs plus profile features (country, language, location, gender, age bracket, installed apps).
- **History** ([B, S, D]): each engagement event carries post, author, action type, semantic IDs, and context (timezone, local hour, product surface, post age), plus dwell time.
- **Candidates** ([B, C, D]): post, author, SIDs, context.
- **Attention mask (candidate isolation).** User and history tokens attend to each other bidirectionally. Each candidate attends to user and history tokens and to itself, never to other candidates.
- **Output.** `[B, num_candidates, num_actions]` logits over a **64-action discrete taxonomy**, trained as multi-label targets, plus **8 continuous-action regression slots** (dwell time) [29].

```
keys ->      | user | history (S) | candidates (C) |
query user   |  v   |   v v v v   |   x x x x      |
query hist   |  v   |   v v v v   |   x x x x      |
query cand_1 |  v   |   v v v v   |   v x x x      |  <- self only
query cand_2 |  v   |   v v v v   |   x v x x      |
...
=> score(c) depends only on (user, history, c): batch-invariant, cacheable
```

Published production configs (`xrex/configs`) [29]:

Parameter	Ranking (prod)	Retrieval (prod)
Embedding dim	2560	1024
Transformer layers	8	8
Query / KV heads (GQA)	20 / 4	16 / 4
Attention key size	128	128
Embedding-table width	1024	1024
FFN widening factor	2	2
History length	1022	1023
Candidates per sequence	64	64
User / item / author hash vocab	100M / 100M / 30M	- / 100M / 30M
IP-address vocab	10M	-
Hashes per entity	2	2
Semantic IDs	6 x 256 (input feature)	6 x 256 (candidate identity)
Action taxonomy	64 + 8 dwell slots	64 (positive = favorite)

Parameter	Ranking (prod)	Retrieval (prod)
Per-device batch	512 (GB300) / 256 (H100)	480

Further implementation details:

- **Hash embeddings.** Every entity is looked up with multiple hash functions (2 per entity), so "there is no vocabulary to maintain and a new post is representable immediately" [28][29].
- **Sequence packing.** Production training packs several variable-length sessions per row and uses a variable-length attention kernel [29].
- **Optimizers.** Dense parameters in the flagship ranker use **Muon** ("consistent-RMS scaling with decoupled weight decay"). Embedding tables use **sparse rowwise AdaGrad** with accumulator half-life decay, lazy per-row decay and decoupled weight decay. Steps with non-finite gradients are skipped [32].
- **Parameterization.** µP is used: `emb_size = 512` is the µP base width, and single-GPU "nano" twins share production losses and code paths [29].
- **Serving.** A Rust gRPC engine is used for serving. It links `libibverbs` as "the production embedding transport" (RDMA), with kernels tuned for Hopper and Blackwell GPUs (H100 to GB300) [29].

Compared with the 2023 heavy ranker, this is roughly a move from 48M dense parameters on 6,000 handcrafted features to a 2560-wide, 8-layer transformer. Most of its capacity sits in hashed embedding tables with about 100M rows for each major entity type, and it reads a history of about 1,000 events [2][29].

21.6.6 Value model, adjustments and reranking

`RankingScorer` computes `score = sum_i w_i * P(action_i)`. The weights are in `home-mixer/params/param.rs` and the arithmetic is in `xai-value-model/scoring.rs` [28][30]. Production defaults as of the October 2026 sync [30]:

Action	Weight		Action	Weight
favorite	0.5		share via copy link	20.0
reply	5.0		follow author	4.0
repost	1.0		dwell (binary)	0.05
quote	5.0		continuous dwell time	0.004
click	0.3		continuous click-dwell time	0.4
open link	0.2		not interested	-47.52
photo expand	0.05		mute author	-58.8
video open	0.07		block author	-31.2
share	2.0		report	-234.0
share via DM	5.0		not dwelled	-0.02

The code comments push back on a common misreading of these numbers [30]. "The weights do not multiply raw engagement counts", so it is wrong to say "one report cancels 468 likes". The baseline probability of a report "is more than 1000x lower than a Like", which is why its weight is so large. Mass-reporting by bad actors mostly affects predictions

for users similar to those actors, and only actions on posts served in Home Timeline count [30].

After the weighted sum, three adjustments are applied [28]:

- **Author diversity**: each post after an author's first is multiplied by a decaying factor, down to a floor.
- **Out-of-network discount**: posts from non-followed accounts are multiplied by a factor below 1, and so are replies and reposts from followed accounts. This continues the 2023 `out_of_network_scale_factor = 0.75` [7].
- **New-author boost / cold start**: posts by authors below an impression threshold are lifted toward a target position. The defaults [30] are an impression threshold of 200, a follower cap of 50,000, maximum post age 2 h, boost slots 15-16, and Thompson sampling enabled with Beta prior alpha0 = 0.75 and beta0 = 49.25 (a prior mean of about 1.5%) choosing the top 2.

VMRanker / vm-ranker is a separate service that reorders scored posts with a **determinantal point process (DPP)** over post embeddings, "giving up a little score for less similarity between neighbours" [28]. The service's defaults [33] are theta 0.5, a pool of the top 100 by score, 1024-dimensional fp16 embeddings, and greedy log-determinant selection. This replaces several of 2023's hand-written diversity heuristics with a single principled reranker.

21.6.7 Transparency features unique to the 2026 release

- **Experiment visibility.** X says it aims to make experiments that reach "a notable share of traffic — e.g. 10% or more" visible in the repo, and it syncs production parameter defaults into code [28].
- **Worked algorithm diff.** `docs/BIDIRECTIONAL_BOOST_CHANGE.md` [31] walks through a real launch. A *bidirectional-follow reply boost* adds weight to P(reply) for posts by mutual follows. It was A/B tested from 10 July 2026 with values 5, 10, 15 and 20, rolled out at 20 on 13 July, and cut to 15 on 24 July after feedback, for example that World Cup discussion from non-followed accounts was being crowded out. The current default is `BidirectionalFollowReplyWeightBoost = 15.0`, and a dwell-boost variant was tested but not shipped (`0.0`) [30][31].
- **Under the Hood** gives each account a report of visibility-limiting labels on its account and posts. Since September 2026 it also shows legal withholding by country [28].
- **Deterministic holdouts.** `InventoryHoldoutFilter` and `FavHoldoutFilter` remove a configured share of posts, chosen deterministically per (post, viewer), to measure the effect of inventory [28].

21.7 Visibility and Trust & Safety in 2026

The August 2026 drop published the labeling path that 2023 had kept out [28]:

```
CONTENT UNDERSTANDING (async)            LABELING RULES            SERVE
grox/ (LLM + classifiers: spam, adult,   scarecrow/ (event rules,  visibility-filtering/
  violent media, text/image embeddings)    botmaker rule engine)     per (post, viewer):
media-model-proxy/, clip/, pnsfwmedia/   abuse-enforcement-svc/      ALLOW | NOTICE |
agatha/ (blocks+reports vs favs)           (+ abuse-ledger appeals)  INTERSTITIAL | DROP
bdsm/ (behaviour sequences)              safety-label-user-agg/      |
user-cred-v2/ (PageRank cred)                      |                 VFFilter,
               \_____________ labels -> storage _/                  AncillaryVFFilter
```

Design points [28]:

- **Ranking and visibility are separate systems.** Ranking sets the order. VF decides whether a post can be shown at all, using "different services, different inputs, different rules".
- **VF runs after selection.** `VFCandidateHydrator`, `VFFilter` and `AncillaryVFFilter` run after top-K is fixed. This avoids per-candidate VF calls on the whole pool, although pre-scoring filters already handle blocks, mutes and keywords cheaply. `AncillaryVFFilter` also drops a post when its parent, quoted post or reposted post was dropped.
- **Rule evaluation.** The first rule that answers DROP ends evaluation. A further set of out-of-network-only rules can only drop. For example, high-recall spam classifiers suppress a post as a *recommendation* while still showing it to followers.
- **Withheld rules.** Some rules and the Grox LLM prompts are deliberately not published "to reduce the risk of gaming". The Under the Hood tool is the compensating transparency [28].
- **Jurisdiction filters.** These appear in code, for example `Brazil2026ElectionFilter` [28].

21.8 Side-by-side: 2013 to 2026

Dimension	WTF (2013) [14]	For You 2023 [1][2][4]	For You 2026 [28][29][30]
Language / stack	Scala, Cassovary	Scala (Product Mixer), TF v1 / PyTorch, Navi (Rust)	Rust (candidate-pipeline), JAX, Rust gRPC engine
In-network source	n/a	Earlybird search (Lucene) + LR light ranker	Thunder in-memory store
Out-of-network	PPR + SALSA	CR-Mixer (SimClusters/TwHIN ANN), UTEG (GraphJet), FRS	Phoenix two-tower ANN, SimClusters, popular posts
Candidates / request	n/a	~1,500 (caps 600/400/300/100)	up to ~2,700 (caps 1200/1000/500 + SimClusters)
Ranker	n/a	Parallel MaskNet, ~48M params (approx), ~6,000 features	8-layer transformer, d = 2560, ~1K-event history, hash + SID embeddings
Heads	n/a	10 engagement probabilities	64 discrete actions + 8 dwell regressions
Value model	n/a	weighted sum, 10 weights	weighted sum, ~20 weights + boosts
Diversity	n/a	author diversity, balance, fatigue heuristics	author decay + OON discount + DPP rerank
Safety	n/a	visibilitylib (partially published)	full labeling path + VF + Under the Hood

---

21.9 Key takeaways for system designers

1. **Keep the funnel; change what is inside it.** In both generations about 1-3K candidates per request come from a few sources, are scored by one multi-head model, combined by an explicit weighted value model, diversified, safety-filtered and then mixed with ads. A clean pipeline framework (Product Mixer, then `candidate-pipeline`) lets you swap models without rewriting the product [6][24][28].
2. **Predict many actions, combine them separately.** Both generations predict per-action probabilities and keep the business trade-off in a weight vector held in configuration. This makes the policy auditable, explains why rare negative events need large weights, and lets product teams tune without retraining [4][30].
3. **Calibrate weights to base rates.** The 2023 weights were set so that each term contributes about equally on average [4]. The 2026 comments make the same point: a report is more than 1000x rarer than a like [30]. Weight magnitudes are not exchange rates between raw counts.
4. **In-memory, time-windowed stores win for fresh content.** Cassovary, GraphJet (24-48 h, 1M edges/s per server) and Thunder all trade completeness for latency and freshness on a single machine [11][12][28].
5. **Sequence models can replace feature factories.** X deleted about 6,000 handcrafted features and the aggregate-feature infrastructure behind them. It now feeds a transformer the raw engagement history, with hashed IDs and semantic IDs [6][28][29]. The costs move to GPU serving, RDMA embedding transport and large hash tables.
6. **Candidate isolation makes transformer ranking work in production.** If candidates cannot attend to each other, scores are batch-invariant, cacheable and comparable across requests and sources [28][29].
7. **Semantic IDs plus hashing solve cold start for content.** Hash embeddings need no vocabulary. Residual-quantized SIDs over multimodal embeddings let the model place an unseen post next to similar ones [29].
8. **Put the index in the checkpoint.** Baking the candidate-tower index into each retrieval checkpoint removes the version skew between user tower and item index [29].
9. **Use principled diversity and exploration.** Replace stacks of heuristics with a DPP reranker over embeddings and with Thompson sampling for new authors under an explicit Beta prior [30][33].
10. **Separate relevance from eligibility.** Ranking and visibility filtering are different services with different inputs. Run expensive VF after top-K. Apply stricter rules to out-of-network recommendations than to followed content [28].
11. **Some components last a long time.** SimClusters (2020) remains a 2026 candidate source, and PageRank-based credibility runs from Tweepcred to user-cred-v2 [8][25] [28].
12. **Make open source part of operations.** Syncing production defaults into the repo, publishing experiment diffs such as the bidirectional boost, and pairing code with per-account label reports is a model for transparency in algorithmic feeds [28][31].

References

1. "the-algorithm: Source code for Twitter's Recommendation Algorithm" (README), Twitter/X, 2023. https://github.com/twitter/the-algorithm
2. "Twitter's Recommendation Algorithm", Twitter Engineering Blog, 2023. https://blog.x.com/engineering/en_us/topics/open-source/2023/twitter-recommendation-algorithm
3. "the-algorithm-ml: Source code for Twitter's Recommendation Algorithm (ML)", Twitter/X, 2023. https://github.com/twitter/the-algorithm-ml
4. "Heavy Ranker" README (projects/home/recap), Twitter/X, 2023. https://github.com/twitter/the-algorithm-ml/blob/main/projects/home/recap/README.md
5. "Heavy Ranker FEATURES.md", Twitter/X, 2023. https://github.com/twitter/the-algorithm-ml/blob/main/projects/home/recap/FEATURES.md
6. "Home Mixer" README, Twitter/X, 2023. https://github.com/twitter/the-algorithm/blob/main/home-mixer/README.md
7. "ScoredTweetsParam.scala" (Home Mixer scored-tweets parameters), Twitter/X, 2023-2025. https://github.com/twitter/the-algorithm/blob/main/home-mixer/server/src/main/scala/com/twitter/home_mixer/product/scored_tweets/param/ScoredTweetsParam.scala
8. "SimClusters: Community-based Representations for Heterogeneous Recommendations at Twitter" (README), Twitter/X, 2023. https://github.com/twitter/the-algorithm/blob/main/src/scala/com/twitter/simclusters_v2/README.md
9. Satuluri et al., "SimClusters: Community-Based Representations for Heterogeneous Recommendations at Twitter", KDD 2020 (ADS track). https://www.kdd.org/kdd2020/accepted-papers/view/simclusters-community-based-representations-for-heterogeneous-recommendatio
10. "Real Graph" README (interaction_graph), Twitter/X, 2023. https://github.com/twitter/the-algorithm/blob/main/src/scala/com/twitter/interaction_graph/README.md
11. "UserTweetEntityGraph (UTEG)" README, Twitter/X, 2023. https://github.com/twitter/the-algorithm/blob/main/src/scala/com/twitter/recos/user_tweet_entity_graph/README.md
12. "GraphJet" README, Twitter, 2016. https://github.com/twitter/GraphJet
13. Sharma, Jiang, Bommannavar, Larson, Lin, "GraphJet: Real-Time Content Recommendations at Twitter", PVLDB 9(13), 2016. http://www.vldb.org/pvldb/vol9/p1281-sharma.pdf
14. Gupta, Goel, Lin, Sharma, Wang, Zadeh, "WTF: The Who to Follow Service at Twitter", WWW 2013. https://web.stanford.edu/~rezab/papers/wtf_overview.pdf
15. El-Kishky et al., "TwHIN: Embedding the Twitter Heterogeneous Information Network for Personalized Recommendation", KDD 2022 (arXiv 2202.05387). https://arxiv.org/abs/2202.05387
16. "TwHIN in torchrec" README, Twitter/X, 2023. https://github.com/twitter/the-algorithm-ml/blob/main/projects/twhin/README.md
17. Wang, She, Zhang, "MaskNet: Introducing Feature-Wise Multiplication to CTR Ranking Models by Instance-Guided Mask", arXiv 2102.07619, 2021. https://arxiv.org/abs/2102.07619
18. "Earlybird Light Ranker" README, Twitter/X, 2023. https://github.com/twitter/the-algorithm/blob/main/src/python/twitter/deepbird/projects/timelines/scripts/models/earlybird/README.md
19. "Tweet Search System (Earlybird)" README, Twitter/X, 2023. https://github.com/twitter/the-algorithm/blob/main/src/java/com/twitter/search/README.md ; and Busch et

al., "Earlybird: Real-Time Search at Twitter", ICDE 2012. http://notes.stephenholiday.com/Earlybird.pdf
20. "CR-Mixer" README, Twitter/X, 2023. https://github.com/twitter/the-algorithm/blob/main/cr-mixer/README.md
21. "Follow Recommendations Service" README, Twitter/X, 2023. https://github.com/twitter/the-algorithm/blob/main/follow-recommendations-service/README.md
22. "Visibility Filtering" README (visibilitylib), Twitter/X, 2023. https://github.com/twitter/the-algorithm/blob/main/visibilitylib/README.md
23. "Navi: High-Performance Machine Learning Serving Server in Rust" README, Twitter/X, 2023. https://github.com/twitter/the-algorithm/blob/main/navi/README.md
24. "Product Mixer" README, Twitter/X, 2023. https://github.com/twitter/the-algorithm/blob/main/product-mixer/README.md
25. "Tweepcred" README, Twitter/X, 2023. https://github.com/twitter/the-algorithm/blob/main/src/scala/com/twitter/graph/batch/job/tweepcred/README
26. "Recos-Injector" README, Twitter/X, 2023. https://github.com/twitter/the-algorithm/blob/main/recos-injector/README.md
27. "Pushservice" README, Twitter/X, 2023. https://github.com/twitter/the-algorithm/blob/main/pushservice/README.md
28. "X For You Feed Algorithm" (x-algorithm README and git history; initial release 2026-01-20, updates through 2026-09), xAI / X, 2026. https://github.com/xai-org/x-algorithm
29. "Phoenix: Recommendation System" README, xAI / X, 2026. https://github.com/xai-org/x-algorithm/blob/main/phoenix/README.md
30. "home-mixer/params/param.rs" (production default parameters and scoring weights, synced 2026-10-08), xAI / X, 2026. https://github.com/xai-org/x-algorithm/blob/main/home-mixer/params/param.rs
31. "Example algorithm diff: bidirectional follow boost" (docs/BIDIRECTIONAL_BOOST_CHANGE.md), xAI / X, 2026. https://github.com/xai-org/x-algorithm/blob/main/docs/BIDIRECTIONAL_BOOST_CHANGE.md
32. "Training the Phoenix recsys model" (phoenix/TRAINING.md), xAI / X, 2026. https://github.com/xai-org/x-algorithm/blob/main/phoenix/TRAINING.md
33. "vm-ranker" source (dpp.rs, args.rs), xAI / X, 2026. https://github.com/xai-org/x-algorithm/tree/main/vm-ranker
34. "grok-1" open-source release, xAI, 2024. https://github.com/xai-org/grok-1
35. "Thunder" source (in-memory post store), xAI / X, 2026. https://github.com/xai-org/x-algorithm/tree/main/thunder
36. "local_prod.yaml" (heavy ranker MaskNet config), Twitter/X, 2023. https://github.com/twitter/the-algorithm-ml/blob/main/projects/home/recap/config/local_prod.yaml
37. "mask_net.py" (MaskNet implementation), Twitter/X, 2023. https://github.com/twitter/the-algorithm-ml/blob/main/projects/home/recap/model/mask_net.py

22. Pinterest Case Study: From Pixie Random Walks to PinFM Foundation Models

Pinterest has published one of the most complete public records of how a large recommender evolved. The same company documented a real-time graph-walk retriever (Pixie), the first web-scale graph convolutional network (PinSage), several generations of user representations (PinnerSage, PinnerFormer), real-time and lifelong sequence rankers (TransAct, TransAct V2), a pretrained user-activity foundation model (PinFM), generative retrieval (PinRec, UniPinRec), its own ANN platform (Manas), GPU serving for ranking, and an ads stack built on DCN-style crossing networks. Most of these papers say clearly which production surface they serve and what they replaced. That makes Pinterest a good longitudinal case study: you can watch each layer of the funnel get rebuilt while the others stay in place.

This chapter follows that history layer by layer. It covers the home feed funnel, retrieval (graph, embedding, conditional and generative), item and user representations, ranking (Pinnability, TransAct, TransAct V2, PinFM), utility blending, GPU serving, ads ranking, and Related Pins.

A note on sourcing. The live web-search budget for this run was used up, and arXiv and Medium could not be fetched directly from the writing environment. The facts here come from primary-source abstracts and excerpts, and Pinterest engineering-blog excerpts, that were retrieved earlier in this research effort and cross-checked across several summaries. Figures marked "approximate" or "secondary" come from third-party summaries and should be checked against the cited paper before you quote them. OmniSage, named in this chapter's scope, could not be verified against a primary source in this pass, so it gets only a short pointer (Section 4.4).

---

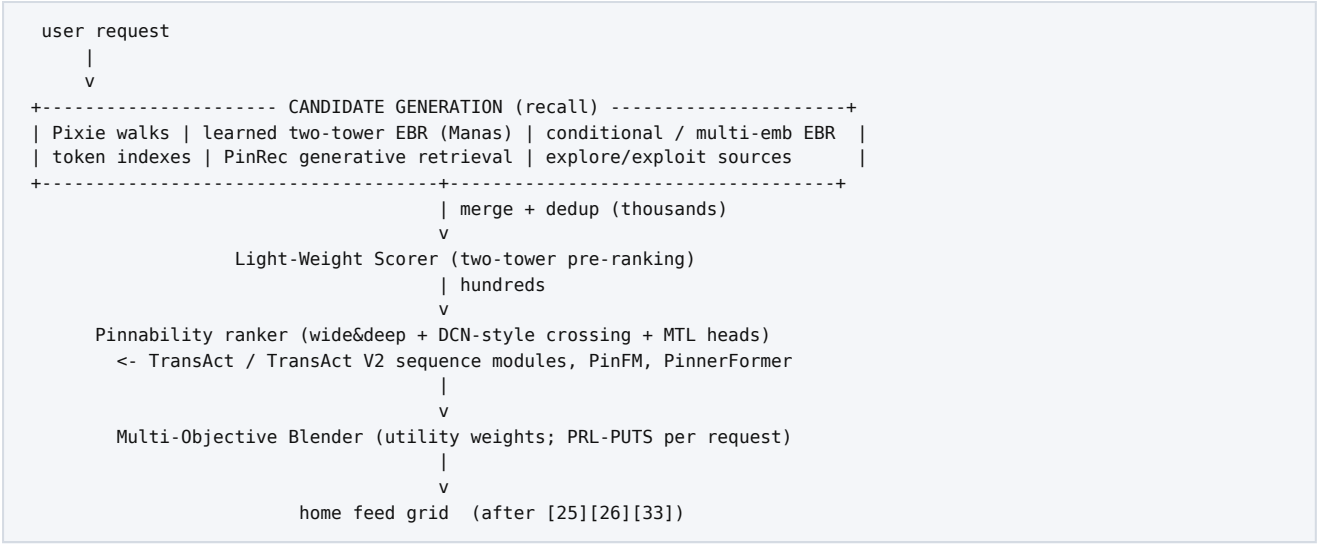
1. The home feed funnel at a glance

Pinterest's WWW 2022 sponsor slides draw the home feed as a classic multi-stage funnel [25]:

- **Candidate generators.** Token-based indexes, HNSW embedding indexes, graph-based random walks (Pixie) and explore/exploit sources.
- **Light-Weight Scorer (LWS).** A pre-ranking stage. Later blog posts describe it as a unified two-tower lightweight scorer [29][30].
- **Ranking Service.** A Utility Prediction Scorer (the multi-task ranker, Pinnability) followed by a Multi-Objective Blender.

An older engineering post describes the same split in terms of services. **Polaris** retrieves, filters and applies lightweight scoring. It returns a list of Pins to the **Feed Generator**,

which sends that list with real-time user signals to **Scorpion** "for a second pass of full scoring" [26]. In the early days, each content type had its own specialised ranking model, and the outputs were "blended with a ratio-based round robin" [27].



Pinterest's Related Pins paper describes how the role of the early stages changed. Once machine-learned ranking arrived, candidate generation was asked to optimise recall rather than precision [5]. That shift is now the norm across the industry, and the rest of this chapter follows it.

---

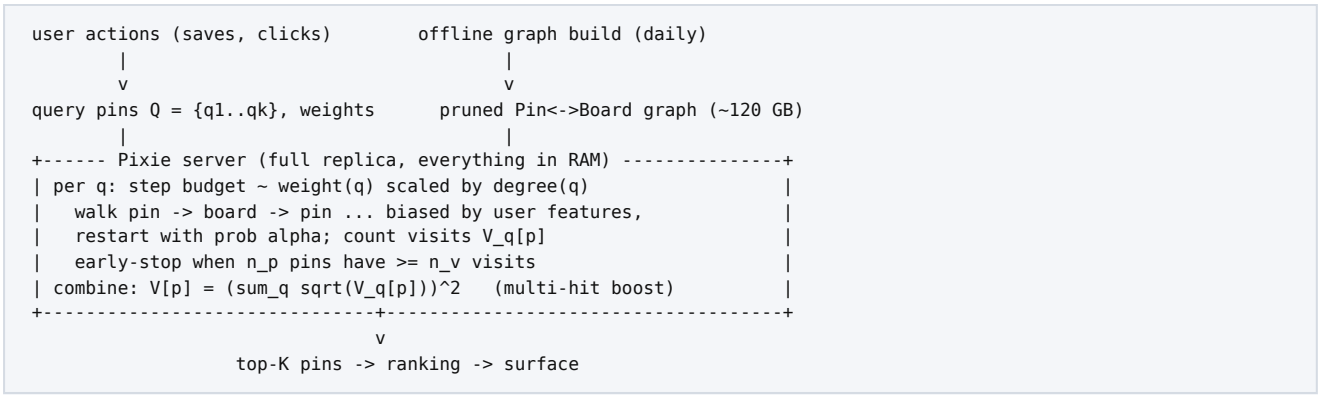
2. Pixie: real-time random walks on a pruned bipartite graph (2017-2018)

Problem

Pinterest needed to recommend from more than 3 billion Pins to more than 200 million users in real time. The recommendations had to react to what the user had just done. The previous Hadoop-based system precomputed results offline and could not react within a session [1].

Architecture

Pixie loads a pruned bipartite **Pin-Board graph** into the RAM of a single machine. A save of a Pin into a board is an edge. The pruned graph has about 3 billion nodes and 17 billion edges and fits into about 120 GB of memory. The unpruned graph had about 7 billion nodes and more than 100 billion edges [1]. The deployment used AWS r3.8xlarge machines with 244 GB of RAM. Each server holds a full replica, so a walk never crosses machines and the service scales by adding replicas [1].



Algorithm and tricks

- **Random walk with restarts.** Many short walks run from the query Pins, and the visit counts approximate personalised PageRank. The walk runs in constant time, independent of graph size [1].
- **Biased walks.** Edge selection is biased toward boards and Pins that match user features such as language and topic, which personalises the walk without per-user graphs [1].
- **Weighted multi-pin queries.** Each query Pin gets a step budget proportional to its weight and adjusted for its degree. The per-pin counts are combined with a sum of square roots, then squared, which rewards candidates reached from several query Pins [1].
- **Boosting small boards.** In a plain walk, low-degree nodes would contribute too little signal, so Pixie boosts the impact of smaller boards [1].
- **Early stopping.** A walk stops once enough Pins have reached a minimum visit count, which cuts latency with little loss in quality [1].
- **Graph pruning as modelling.** Topically diverse boards (high entropy) are removed and very popular Pins keep edges only to their most relevant boards. Pruning improved quality as well as memory [1].

Scale and impact

- 99th-percentile latency under 60 ms [1].
- About 1,200 requests per second per server, and about 100,000 requests per second across the cluster [1].
- Up to 50% higher user engagement than the previous Hadoop-based system [1].
- A summary of the paper reports that Pixie-backed systems contributed more than 80% of all user engagement on Pinterest at the time [2].

Lessons

If the working graph fits in RAM, replicate it rather than partition it, because walks are pointer chasing and network hops would ruin them. Pruning hub nodes and noisy boards is a modelling decision. A walk produces a candidate distribution, and a learned ranker still has to sit on top.

---

3. PinSage: graph convolution at 3 billion nodes (2018)

Problem

Pixie needs graph connectivity at query time and cannot use content features, so new Pins with few saves get weak recommendations. Pinterest wanted one embedding per Pin that combined visual and text features with graph structure, computed for billions of items and served by nearest-neighbour search [3][4].

Architecture and scale

PinSage is a GraphSAGE-style inductive GCN. It was trained on 7.5 billion examples over a graph of 3 billion nodes (Pins and boards) and 18 billion edges [3]. The engineering blog describes the graph as about 10,000x larger than typical GCN applications of the time [4].

```
TRAINING (producer-consumer)
 CPU producers: sample (query, positive) pairs from co-engagement
                -> random-walk neighbourhoods (top-T by visit count)
                -> gather visual + text features            --+
 GPU consumer:  2 conv layers, importance pooling,             |
                max-margin loss, random + curriculum negatives <+
INFERENCE (MapReduce, no redundant computation)
 layer-1 embeddings for all nodes -> join -> layer-2 -> Pin embeddings
                -> ANN index -> Related Pins / Homefeed candidates
```

Key tricks

1. **Random-walk neighbourhoods.** Short random walks from each node pick the top-T most-visited nodes (L1-normalised counts) as the neighbourhood. This keeps the memory footprint per node fixed even though Pinterest degrees can be huge [3].
2. **Importance pooling.** The visit counts become aggregation weights. The paper credits this with a 46% gain in offline metrics [3].
3. **Curriculum training with hard negatives.** Later epochs add harder negatives, Pins ranked moderately high but not top by personalised PageRank. The paper reports a 12% gain from curriculum training [3]. Another chapter of this Atlas notes that the hard negatives roughly doubled the epochs to convergence.
4. **Producer-consumer minibatching** keeps the GPUs fed while CPUs sample and fetch features [3].
5. **MapReduce inference** computes each layer once for all nodes instead of recomputing overlapping neighbourhoods [3].

A secondary summary reports 10-30% repin-rate improvements over earlier embedding systems in Homefeed (approximate) [4].

Why PinSage mattered beyond retrieval

PinSage became Pinterest's shared coordinate system for Pins. PinnerSage clusters users in PinSage space [6]. PinnerFormer uses 256-dimensional PinSage embeddings as frozen per-action features [7]. TransAct V2 stores a 32-dimensional PinSage embedding per lifelong action token and runs its nearest-neighbour selection in that space [10]. Manas benchmarks use a 100-million-vector GraphSage dataset [23]. Graph embeddings that many teams reuse are a platform, not a feature.

4. Multi-graph and multi-entity successors

4.1 MultiBiSage

MultiBiSage extends the PinSage idea from one Pin-Board graph to **multiple bipartite graphs** at Pinterest [41]. The detailed design was not retrieved in this research pass. The title alone shows the direction: one item representation fed by several interaction graphs instead of one.

4.2 OmniSearchSage

OmniSearchSage learns multi-task, multi-entity embeddings for Pinterest Search [42]. Together with the PinSage line, it shows a trend toward one embedding space shared by Pins, queries and products, trained on several tasks at once.

4.3 PinnerSage: multi-modal user embeddings (KDD 2020)

Problem. One user vector averages over unrelated interests such as recipes, tattoos and home decor [6].

Design. Take the user's action Pins from the last 90 days. Cluster their PinSage embeddings with Ward hierarchical clustering. Represent each cluster by its **medoid**, a real Pin from the user's history, which keeps the cluster topically coherent and robust to outliers. Score clusters by importance and sample a few medoids per request as ANN queries [6]. Offline experiments and A/B tests showed it beat single-embedding methods [6]. Because the PinSage item space stays fixed, PinnerSage never retrains the item model.

PinnerSage also became a ranking feature. The PinnerFormer paper says the previous production ranker used a weighted average of a user's top-k PinnerSage embeddings, and that replacing it with one PinnerFormer embedding significantly improved Homefeed repins [7].

4.4 OmniSage (pointer only)

OmniSage, a later Pinterest representation-learning effort over a multi-entity heterogeneous graph, is listed in this chapter's scope. Its primary source could not be retrieved or verified in this pass, so no architecture or numbers are given here. Readers should treat it as the natural successor of PinSage, MultiBiSage and OmniSearchSage, and check the original paper for details.

5. PinnerFormer: a daily batch user embedding trained for the long term (KDD 2022)

Problem

Pinterest wanted a single user embedding, computed in a **daily batch job**, that would work nearly as well as a real-time embedding and predict long-term engagement, not just the next action [7].

Architecture

```
user's recent actions (pin, action type, surface, time features)
    |  each pin: 256-d PinSage embedding (frozen) + learned feature embs
    v
causal Transformer encoder
    v
L2-normalised user embedding, recomputed daily for every user
    |
    +--> ANN retrieval against pin embeddings
    +--> feature in Homefeed ranking and in ads ranking models
```

The dense all-action loss

Instead of predicting only the next action (SASRec style), the embedding is trained to predict **all positive engagements (repins, close-ups, clicks) in a future window of up to 28 days** [7]. Training uses sampled softmax with in-batch and random negatives. A secondary summary reports logQ correction as well [7]. The paper's evaluation uses a 14-day window.

The objective makes the embedding robust to staleness. Moving from real-time to daily batch inference cut Recall@10 by **13.9%** with the next-action objective, but only by **8.3%** with the dense all-action objective [7]. A/B tests showed better engagement and retention than the previous user representation, and the model has been in production since fall 2021 [7]. A secondary source says the team did not scale sequences beyond about 256 actions (approximate).

Ads usage

The paper reports that each primary surface (Homefeed, Related Pins, Search) has a separate ads ordering model, and that adding PinnerFormer as a feature gave significant gains in CTR and long CTR on each surface [7].

Lesson

A user embedding that will be computed offline and reused should forecast a window of future behaviour. The objective, not only the refresh rate, decides how much freshness you need.

---

6. Learned retrieval and the Manas ANN platform (2022-2026)

6.1 Two-tower learned retrieval

Problem. The older candidate generators were heuristic or graph-based. A learned retriever optimises engagement directly [16].

Architecture. A standard two-tower model [16][17]:

- The **Pin tower** encodes visual, text and statistical features. It runs offline, and its embeddings are pushed into Manas.
- The **user tower** encodes engagement sequences and context. It is computed once per Homefeed request.
- Training is contrastive with in-batch negatives (a popularity correction is reported by secondary summaries [44]).

The version-sync problem. Each two-tower model is split into **two artifacts deployed to separate services**. When a model is retrained, the serving versions of both services must be kept in sync [16]. Otherwise user vectors from version k get scored against Pin vectors from version k-1, and retrieval silently degrades.

Results. A secondary summary says the learned retriever became one of the top candidate generators, with the **highest user coverage** and a top-three save rate. It was deployed for Homefeed and notifications [44].

6.2 Advancements: crossing inside towers, ID embeddings, multi-embedding

The 2025 Homefeed EBR post describes several upgrades [17]:

- **Feature crossing within each tower** with MaskNet and DHEN blocks, to win back some of the expressiveness lost to the dot product. A secondary summary reports +0.1-0.2% engagement and more than 1% more Homefeed saves from DHEN (approximate) [44].
- **Pre-trained ID embeddings.** User and Pin ID embeddings are pre-trained contrastively on a cross-surface dataset to reduce overfitting [17].
- **Multi-embedding retrieval.** Several user embeddings per request. Implicit interests come from a differentiable clustering module, and explicit interests from conditional retrieval [18]. Results from each embedding are merged by **round-robin with de-duplication**. Post-filtered conditional retrieval was the best variant, and p90 retrieval latency rose from **150 ms to 205 ms**, which was still within budget [18].
- Reported gains across sessions, saves and clicks of roughly 0.1% to 1.2% (secondary, approximate).

6.3 Conditional learned retrieval

The user tower takes an explicit condition, such as a topic, so one user can issue several conditioned queries grounded in the same behaviour representation [19][20]. A topic-based notification feed built this way gave **+0.26% weekly active users** and beat two-tower models with filters [19]. A 2026 post describes redesigning serving to support many conditions per Home Feed request [20].

6.4 Manas: from in-RAM HNSW to quantized SPANN

- **Realtime HNSW.** Pins are inserted incrementally into the HNSW graph for real-time EBR. Filtering was first done by over-fetching K neighbours and post-filtering [21].

- **Streaming filters.** The inverted index was reused as the filter store, with bitmap and array-backed posting lists replacing skiplists for faster lookups [22].
- **Quantization benchmark (2026)** on a 100-million-embedding GraphSage dataset [23][24]:

Index	Memory	Recall@100
HNSW (baseline)	121 GB	93.72%
HNSW + scalar quantization	50 GB	92.92%
IVF (baseline)	97 GB	91.69%
IVF + scalar quantization	25 GB	95.71%

Product quantization compressed more but lost much more recall [23]. Manas also implemented **SPANN** for SSD-resident indexes, with an in-memory HNSW over the cluster centroids [23]. Pinterest Ads separately describes offline ANN for ad retrieval [40].

Lesson. Scalar quantization is often the best first compression step for recommendation embeddings. Production ANN platforms tend to move from all-in-RAM HNSW toward IVF or SPANN once cost and update rates matter.

6.5 Generative retrieval: PinRec and UniPinRec

PinRec (2025) uses a user-sequence Transformer as a generative retriever. It adds **outcome conditioning**, for example generating for saves versus clicks, and **multi-token generation** [14]. A teardown reports A/B lifts of up to +0.55% time spent and +4.01% Homefeed grid clicks (secondary) [43]. **UniPinRec** (2026) proposes unifying generative retrieval and ranking at Pinterest scale [15]. The same sequence encoder that powers ranking is moving into retrieval.

---

7. Pre-ranking: the Light-Weight Scorer

The LWS sits between thousands of merged candidates and the expensive ranker [25]. Pinterest has written about a **unified two-tower lightweight scorer** for Home Feed [29] and about **modernizing the pre-ranking stage** [30]. On the ads side, **MTMD** is a multi-task, multi-domain framework for unified ad lightweight ranking [39]. The design point is the same in each case. One shared, cheap, decomposable model replaces per-source heuristics, and it is trained to agree with the full ranker.

---

8. Ranking: Pinnability and its sequence modules

8.1 Pinnability

Pinnability was introduced on the engineering blog in 2015 to estimate how likely a Pinner is to interact with a Pin, which turned the home feed from chronological to relevance-ranked [28]. In the TransAct era it is a **point-wise multi-task** model. A wide-and-deep stack feeds feature-interaction layers and MLPs that output per-action probabilities for

click, repin, hide and more, with hide as a negative signal [8]. The "Rethinking Personalized Ranking" paper calls the transformer-learned user sequence embedding one of the most important features in Homefeed ranking [31]. A 2024 workshop paper explains how Pinterest picked a new feature-interaction architecture for Homefeed under latency, GPU-memory and reproducibility constraints rather than purely on public benchmarks [32].

8.2 TransAct: real-time actions inside the ranker (KDD 2023)

Problem. PinnerFormer refreshes daily, so the ranker could not react to what the user did seconds ago [8].

Hybrid design. TransAct encodes the most recent actions in real time inside Pinnability, while PinnerFormer and other batch features provide long-term context. The authors argue this combines real-time responsiveness with the cost-efficiency of batch representations [8].

```
real-time action stream                                  candidate pin
        |                                                      |
latest ~100 actions: PinSage emb + action type + timestamp   |
        |                                                      |
early fusion: concat each action with candidate embedding <--+
        v
small Transformer encoder
        v
output compression (first K tokens + max pooling) -> flatten
        v
concat with PinnerFormer + other features -> DCN v2 crossing -> MLP
        v
multi-task heads (repin, click, hide, ...) -> utility
```

Findings [8]:

- **Positional encodings added little.** Better early fusion and **action-type embeddings** mattered more.
- **Random time-window masking** in training hides recent actions so the model does not over-rely on the last few clicks, which protects diversity (reported by summaries).
- Production problems the paper discusses: high serving complexity, reduced diversity and **engagement decay**, so the model needs frequent retraining.

Serving. The TransAct blog post says that because the Transformer output became much smaller, an explicit DCN v2 crossing layer on the full feature set became affordable. The new model raised CPU latency by more than 20x, so the team moved to **GPU serving** to stay latency-neutral at the same cost [9].

Results. A summary reports +11% Homefeed repin volume and -10% hide volume online (secondary, approximate). TransAct was deployed in Homefeed, Related Pins, Notifications and Search [8].

8.3 TransAct V2: lifelong sequences (2025)

Problem. TransAct saw about 100 recent actions. TransAct V2 extends Homefeed ranking to lifelong histories of up to about **16,000 actions**, roughly 160x more [10][11][12].

```
lifelong seq (~16k, int8)    real-time seq     impression seq
          \                       |                /
           +-- candidate-anchored NN search (dot product in PinSage
               space); most recent real-time actions always kept --+
                                   |
                       short selected subsequence
                                   v
               single-kernel Transformer (Triton)
                                   v
     ranker heads  +  auxiliary Next Action Loss
```

- **Tokens.** Each token holds a timestamp, a multi-hot action type, the surface and a **32-d PinSage embedding**. Lifelong actions are stored losslessly with **int8 quantization** [10][11].
- **Candidate-anchored NN selection** over lifelong, real-time and impression sequences. It is a SIM-style search stage that always keeps the most recent actions [10].
- **Next Action Loss.** An auxiliary task that asks what the user will do next, not only the engagement probability [10][12].
- **Serving kernels** [10]:
  - fusing float16 dequantization and normalisation with NN search cut model latency by 20% versus native PyTorch;
  - a single-kernel Transformer (SKUT) ran 6.6x faster than PyTorch on the forward pass;
  - a broadcast-free NN search cut PCIe transfers for sequences by 8x (secondary);
  - end-to-end inference latency improved **103x (p50), 338x (p90) and 250x (p99)** over the unoptimised baseline.
- **Results.** +6.35% repins, +1.41% time spent and -12.8% hides [12]. Offline, HIT@3/repin improved by 13.31% and HIT@3/hide fell by 11.25% [10][11].

8.4 PinFM: a foundation model for user activity (RecSys 2025)

Problem. Each surface trained its own sequence modules. Pinterest wanted one large pretrained user-activity model shared across applications [13].

Pretraining. A Transformer with **more than 20 billion parameters**, most of them in ID embedding tables, pretrained on about two years of user actions with contrastive next-token, multi-token and future-token losses [13]. A talk summary describes InfoNCE next-Pin prediction for pretraining and BCE fine-tuning on surface-specific tasks (secondary).

Fine-tuning. The pretrained Transformer and embedding tables are plugged into each application's ranking model and fine-tuned with ranking losses. The pretraining losses can be kept to speed adaptation. Switching to bidirectional attention during fine-tuning cost 1.4% in evaluation metrics, so causal attention is kept [13].

DCAT (Deduplicated Cross-Attention Transformer). There are about 1,000 candidates per unique user sequence at serving time, and about 10 in training. DCAT splits the model into a **context part** that runs the Transformer once over the user history and caches its keys and values, and a **crossing part** that scores each candidate against the cached history [13].

```
user history --> context Transformer (once per request) --> cached K,V
                                                                |
candidates c1..c1000 --> crossing attention over cached K,V -+--> features
                                                               -> ranker
```

DCAT raised throughput by **600%** on internal data, and embeddings are **int4-quantized** [13]. Cold-start handling for fresh Pins reportedly uses ID noise, age-dependent dropout and a class token (secondary).

Results. +20% engagement with new items. Deployed in Home Feed and Related Items for more than half a billion users at neutral cost and with a marginal latency increase [13].

8.5 Comparison of Pinterest's user-sequence models

Model (year)	Stage	History length	Refresh	Key mechanism	Reported impact
PinnerSage (2020) [6]	Retrieval	90 days of actions	Daily + real time	Ward clustering, medoids	Beat single embedding
PinnerFormer (2022) [7]	Retrieval + feature	~256 actions (approx.)	Daily batch	Causal Transformer, dense all-action loss	Batch drop 8.3% vs 13.9%
TransAct (2023) [8]	Ranking	~100 real-time actions	Per request	Early fusion, time-window mask	~+11% repins (secondary)
TransAct V2 (2025) [10]	Ranking	up to ~16k actions	Per request	Candidate NN selection, next-action loss	+6.35% repins, -12.8% hides
PinFM (2025) [13]	Ranking (pretrained)	~2 years	Pretrain + fine-tune	20B+ params, DCAT KV reuse	+20% new-item engagement
PinRec (2025) [14]	Retrieval (generative)	User sequence	Per request	Outcome-conditioned, multi-token	+4.01% grid clicks (secondary)

---

9. Utility blending and long-term objectives

The ranker outputs a vector of action probabilities. A **utility-weight vector** turns it into one score, a weighted sum of the predicted objectives [33]. Historically the weights were set by offline analysis and online experiments, applied globally to all requests, and left in place for long periods [33].

- **Round robin to blender.** The 2016 home feed blended per-type rankers with fixed ratios [27]. By 2022 a Multi-Objective Blender sat inside the ranking service [25].
- **Lifecycle-dependent weights.** The Pinner Progression post makes utility weights depend on the user's lifecycle stage and shifts the feed toward retention rather than pure engagement (secondary summary) [35].
- **PRL-PUTS (2026).** Weight selection becomes a one-step, value-based RL problem: for each request an agent picks the weight vector with the highest expected request-level reward. An inference-time scalarization parameter yields a family of policies and an **empirical Pareto frontier**, used to govern the choice between, for example, repins and other actions. It runs alongside ranking with no added latency and no ranker change, is trained on unbiased exploration logs, and gave **+0.13% successful sessions** on Homefeed [33].

- **Downstream rewards (2026).** A model-agnostic method learns longer-horizon downstream rewards on top of immediate engagement. It launched on Homefeed and was extended to other surfaces [34].

---

10. GPU serving for ranking

Pinterest's 2022 post "GPU-accelerated ML Inference at Pinterest" describes moving ranking models from CPU to GPU serving [36]. In outline, per the summary in Chapter 10 of this Atlas (approximate):

- **Motivation.** CPU cost grew roughly linearly with model size, which blocked larger rankers such as transformer sequence modules and bigger embeddings.
- **Outcome.** GPU serving made models with on the order of 100x more compute affordable at comparable cost and latency.
- **Techniques.** Batching candidates per request, CUDA Graphs to cut kernel-launch overhead, merging many small embedding lookups into fewer larger operations, mixed precision, and moving feature preprocessing onto the GPU.

TransAct shows the economic logic in miniature. A 20x CPU latency increase became latency-neutral at the same cost on GPUs [9]. TransAct V2's Triton kernels and PinFM's DCAT and int4 embeddings are the next steps in the same direction: once ranking runs on GPUs, kernel fusion, KV reuse and quantization become first-order design tools [10][13].

```
CPU era                  GPU era (2022+)            kernel era (2025+)
tree/linear + MLP  -->   batched DNN, CUDA graphs -->  fused Triton kernels,
per-candidate CPU        fused embeddings,            int8/int4 sequences,
cost ~ model size        GPU preprocessing            KV dedup (DCAT)
```

---

11. Pinterest ads ranking

Ads run in the same surfaces as organic content (Homefeed, Related Pins, Search), each with its own ads ordering model [7].

- **User sequences in ads engagement models.** A Pinterest post describes adding a Transformer over user action sequences to ads engagement models, with a **DCNv2** production baseline [37]. The team started with a single-layer, single-head Transformer and used position embeddings based on **time deltas**. Adding layers helped, but adding heads did not. Earlier feature interaction between the sequence and user/Pin representations also helped. The post warns that the DCNv2 cross layer sometimes **overflowed in float16 training** [37].
- **PinnerFormer as an ads feature** gave significant CTR and long-CTR gains on each surface [7].
- **ML-DCN (2026).** Scaling DCNv2 and MaskNet, both deployed in Pinterest ads, gave offline AUC gains that flattened quickly at affordable compute. ML-DCN adds an **instance-conditioned mask to a low-rank crossing layer**, reaches higher AUC than DCNv2 and MaskNet at matched FLOPs, and has a better AUC-FLOPs curve as

compute grows. Online it gave significant CTR and click-quality gains and was deployed at neutral serving cost [38].
- **MTMD** unifies ad lightweight ranking across tasks and domains [39].
- **Offline ANN for ad retrieval** is used where online ANN is unnecessary [40].

```
ad candidates (offline/online ANN, targeting)
        v
MTMD lightweight ranking (multi-task, multi-domain)
        v
engagement model: user seq Transformer + PinnerFormer + features
                  -> DCNv2 / ML-DCN crossing -> pCTR, long-CTR heads
        v
auction + blending with organic feed
```

The ads story mirrors organic ranking. Crossing networks scale poorly when made naively bigger, mixed-precision training exposes numeric problems, and sequence features carry most of the new signal.

---

12. Related Pins

Related Pins ("more like this" under a Pin closeup) is Pinterest's oldest recommendation surface. The 2017 paper "Related Pins at Pinterest: The Evolution of a Real-World Recommender System" traces it from heuristic board co-occurrence to a multi-source candidate set followed by learned ranking. It notes that candidate generation moved from optimising precision to optimising recall once machine-learned ranking was introduced [5]. Pixie walks [1], PinSage nearest neighbours [3], TransAct [8] and PinFM [13] were all deployed to Related Pins (Related Items) as well as Homefeed. This shows a reusable-component strategy: one retriever or sequence module serving many surfaces.

---

13. System-level comparison

System	Year	Layer	Data	Scale (public)	Serving
Pixie [1]	2018	Retrieval	Pin-Board graph	3B nodes, 17B edges, 120 GB	RAM replica; p99 < 60 ms; ~100k QPS
PinSage [3]	2018	Item embedding	Graph + content	3B nodes, 18B edges, 7.5B examples	MapReduce inference -> ANN
PinnerSage [6]	2020	User embedding	90-day actions	Per-user medoids	Medoid ANN queries
PinnerFormer [7]	2022	User embedding	Action sequence	256-d PinSage inputs	Daily batch
Learned EBR [16][17]	2023-25	Retrieval	Two-tower	Billions of Pins	Manas; p90 150 -> 205 ms multi-emb
Manas [23]	2019-26	ANN	Embeddings	100M-vector benchmark	HNSW -> IVF/SPANN + SQ
TransAct [8]	2023	Ranking	Real-time seq	~100 actions	GPU
TransAct V2 [10]	2025	Ranking	Lifelong seq	~16k actions	Triton kernels, int8

System	Year	Layer	Data	Scale (public)	Serving
PinFM [13]	2025	Ranking FM	2 yrs actions	20B+ params	DCAT, int4; 500M+ users
PRL-PUTS [33]	2026	Blending	Exploration logs	Per-request weights	No added latency
ML-DCN [38]	2026	Ads ranking	Ads features	Matched-FLOP scaling	Neutral cost

---

Key takeaways for system designers

1. **Rebuild one layer at a time.** Pinterest replaced retrieval, representations, ranking modules and blending at different times while keeping a stable funnel (candidate generators, LWS, ranker, blender) [25]. A stable interface between stages is what makes this incremental evolution possible.
2. **Start with walks, then compile the graph into vectors.** Pixie gave real-time, explainable candidates with no training loop [1]. PinSage turned the same graph plus content into embeddings that any ANN system could serve and that cover cold items [3].
3. **Use random walks for neighbourhood selection.** Importance pooling bounds fan-out and weights neighbours by relevance in one step (+46% offline) [3].
4. **Make a shared embedding space a platform.** PinSage embeddings feed PinnerSage, PinnerFormer, TransAct V2 and Manas benchmarks [6][7][10][23].
5. **Pair batch and real-time user models.** A daily embedding trained on a long-horizon dense loss (PinnerFormer) plus a real-time sequence module in the ranker (TransAct) gives most of the freshness at a fraction of the cost [7][8].
6. **Train for the window you will be stale for.** The dense all-action loss cut the batch-staleness penalty from 13.9% to 8.3% [7].
7. **Action type and early fusion beat positional encoding** for engagement sequences [8]. Time-delta positions worked in ads [37].
8. **Lifelong sequences need retrieval inside the ranker.** Candidate-anchored nearest-neighbour selection with an always-keep-recent rule, plus int8 storage, makes 16k-action histories affordable [10].
9. **Serving engineering is the model.** Moving to GPUs, fused Triton kernels (103-338x latency gains) and DCAT KV reuse (+600% throughput) are what made TransAct, TransAct V2 and PinFM deployable [9][10][13][36].
10. **Pretrain once, fine-tune inside each ranker.** PinFM plugs into existing rankers rather than replacing them, keeps causal attention, and helps new items most [13].
11. **Keep two-tower artifacts version-locked.** Separate user and item services must roll forward together [16].
12. **Multiple user embeddings raise recall at a latency price.** Round-robin merging with dedup kept p90 at 205 ms [18].
13. **Quantize before you shard.** Scalar quantization halved or quartered ANN memory with little or no recall loss [23].
14. **Separate prediction from policy, then personalise the policy.** Global utility weights became per-request RL weights with a Pareto frontier for governance [33].
15. **Do not scale crossing layers naively.** DCNv2 and MaskNet saturate. Low-rank, instance-masked crossing gives a better AUC-FLOPs curve, and float16 training can

overflow cross layers [37][38].

References

1. Eksombatchai, C., Jindal, P., Liu, J. Z., Liu, Y., Sharma, R., Sugnet, C., Ulrich, M., Leskovec, J. "Pixie: A System for Recommending 3+ Billion Items to 200+ Million Users in Real-Time." Pinterest / Stanford, WWW 2018. https://arxiv.org/abs/1711.07601
2. Colyer, A. "Pixie: a system for recommending 3+ billion items to 200+ million users in real-time." The Morning Paper, 2018. https://blog.acolyer.org/2018/05/23/pixie-a-system-for-recommending-3-billion-items-to-200-million-users-in-real-time/
3. Ying, R., He, R., Chen, K., Eksombatchai, P., Hamilton, W. L., Leskovec, J. "Graph Convolutional Neural Networks for Web-Scale Recommender Systems" (PinSage). Pinterest / Stanford, KDD 2018. https://arxiv.org/abs/1806.01973
4. Pinterest Engineering. "PinSage: A new graph convolutional neural network for web-scale recommender systems." Pinterest Engineering Blog, 2018. https://medium.com/pinterest-engineering/pinsage-a-new-graph-convolutional-neural-network-for-web-scale-recommender-systems-88795a107f48
5. Liu, D. C., et al. "Related Pins at Pinterest: The Evolution of a Real-World Recommender System." Pinterest, WWW 2017 companion. https://arxiv.org/abs/1702.07969
6. Pal, A., Eksombatchai, C., Zhou, Y., Zhao, B., Rosenberg, C., Leskovec, J. "PinnerSage: Multi-Modal User Embedding Framework for Recommendations at Pinterest." Pinterest, KDD 2020. https://arxiv.org/abs/2007.03634
7. Pancha, N., Zhai, A., Leskovec, J., Rosenberg, C. "PinnerFormer: Sequence Modeling for User Representation at Pinterest." Pinterest, KDD 2022. https://arxiv.org/abs/2205.04507
8. Xia, X., et al. "TransAct: Transformer-based Realtime User Action Model for Recommendation at Pinterest." Pinterest, KDD 2023. https://arxiv.org/abs/2306.00248
9. Pinterest Engineering. "How Pinterest Leverages Realtime User Actions in Recommendation to Boost Homefeed Engagement Volume." Pinterest Engineering Blog, 2023. https://medium.com/pinterest-engineering/how-pinterest-leverages-realtime-user-actions-in-recommendation-to-boost-homefeed-engagement-volume-165ae2e8cde8
10. Pinterest. "TransAct V2: Lifelong User Action Sequence Modeling on Pinterest Recommendation." 2025. https://arxiv.org/abs/2506.02267
11. Pinterest Engineering. "Next-Level Personalization: How 16k+ Lifelong User Actions Supercharge Pinterest's Recommendations." Pinterest Engineering Blog, 2025. https://medium.com/pinterest-engineering/next-level-personalization-how-16k-lifelong-user-actions-supercharge-pinterests-recommendations-bd5989f8f5d3
12. Social Media Today. "Pinterest Updates Ranking Model With More Data." 2025. https://www.socialmediatoday.com/news/pinterest-updates-ranking-model-with-more-data/750122/
13. Pinterest. "PinFM: Foundation Model for User Activity Sequences at a Billion-scale Visual Discovery Platform." RecSys 2025. https://arxiv.org/abs/2507.12704

14. Pinterest. "PinRec: Outcome-Conditioned, Multi-Token Generative Retrieval for Industry-Scale Recommendation Systems." 2025. https://arxiv.org/abs/2504.10507
15. Pinterest. "UniPinRec: Unifying Generative Retrieval and Ranking at Pinterest Scale." 2026. https://arxiv.org/abs/2606.00422
16. Pinterest Engineering. "Establishing a Large Scale Learned Retrieval System at Pinterest." Pinterest Engineering Blog, 2025. https://medium.com/pinterest-engineering/establishing-a-large-scale-learned-retrieval-system-at-pinterest-eb0eaf7b92c5
17. Pinterest Engineering. "Advancements in Embedding-Based Retrieval at Pinterest Homefeed." Pinterest Engineering Blog, 2025. https://medium.com/pinterest-engineering/advancements-in-embedding-based-retrieval-at-pinterest-homefeed-d7d7971a409e
18. Pinterest. "Synergizing Implicit and Explicit User Interests: A Multi-Embedding Retrieval Framework at Pinterest." KDD 2025. https://arxiv.org/abs/2506.23060
19. Pinterest. "Bootstrapping Conditional Retrieval for User-to-Item Recommendations." RecSys 2024. https://arxiv.org/abs/2508.16793
20. Pinterest Engineering. "Scaling Conditional Learned Retrieval for Pinterest Home Feed." Pinterest Engineering Blog, 2026. https://medium.com/pinterest-engineering/scaling-conditional-learned-retrieval-for-pinterest-home-feed-ecfba7e5a426
21. Pinterest Engineering. "Manas HNSW Realtime: Powering Realtime Embedding-Based Retrieval." Pinterest Engineering Blog. https://medium.com/pinterest-engineering/manas-hnsw-realtime-powering-realtime-embedding-based-retrieval-dc71dfd6afdd
22. Koh, T. "Manas HNSW Streaming Filters." Pinterest Engineering Blog, 2022. https://medium.com/pinterest-engineering/manas-hnsw-streaming-filters-351adf9ac1c4
23. Pinterest Engineering. "Evolving Pinterest's Embedding Retrieval Platform." Pinterest Engineering Blog, 2026. https://medium.com/pinterest-engineering/evolving-pinterests-embedding-retrieval-platform-aede4e831e01
24. InfoQ. "From Memory-Hungry HNSW to Quantized SPANN: The Technical Evolution of Pinterest's Manas Platform." 2026. https://www.infoq.com/news/2026/09/pinterest-search/
25. Pinterest. "Pinterest at WWW 2022" (sponsor slides on the home feed funnel). 2022. https://archives.iw3c2.org/www2022/wp-content/uploads/Sponsors/WWW2022-Pinterest.pdf
26. Pinterest Engineering. "Building a dynamic and responsive Pinterest." Pinterest Engineering Blog. https://medium.com/@Pinterest_Engineering/building-a-dynamic-and-responsive-pinterest-7d410e99f0a9
27. MLconf. Pinterest Homefeed talk abstract (ratio-based round-robin blending). 2016. https://mlconf.com/?p=8909
28. Pinterest Engineering. "Pinnability: Machine learning in the home feed." Pinterest Engineering Blog, 2015. https://medium.com/@Pinterest_Engineering/pinnability-machine-learning-in-the-home-feed-64be2074bf60
29. Pinterest Engineering. "Pinterest Home Feed Unified Lightweight Scoring: A Two-tower Approach." Pinterest Engineering Blog. https://medium.com/pinterest-engineering/pinterest-home-feed-unified-lightweight-scoring-a-two-tower-approach-b3143ac70b55
30. Pinterest Engineering. "Modernizing Home Feed Pre-Ranking Stage." Pinterest Engineering Blog. https://medium.com/pinterest-engineering/modernizing-home-feed-pre-ranking-stage-e636c9cdc36b
31. Pinterest. "Rethinking Personalized Ranking at Pinterest: An End-to-End Approach." 2022. https://arxiv.org/abs/2209.08435

32. Pinterest. "Improving feature interactions at Pinterest under industry constraints." CIKM 2024 workshop (IRS). https://arxiv.org/abs/2412.01985
33. Pinterest. "A Production-Ready RL Framework for Personalized Utility Tuning with Pareto Sweeping in Pinterest Recommender Systems (PRL-PUTS)." 2026. https://arxiv.org/abs/2605.16344
34. Pinterest. "Long-term User Engagement Optimization through Model-agnostic Downstream Rewards Learning." 2026. https://arxiv.org/abs/2607.14192
35. Pinterest Engineering. "Pinner Progression: Better Use-Case Representation Driving Weekly Active User Growth at Pinterest." Pinterest Engineering Blog. https://medium.com/pinterest-engineering/pinner-progression-better-use-case-representation-driving-weekly-active-user-growth-at-pinterest-bd2131ab238a
36. Pinterest Engineering. "GPU-accelerated ML Inference at Pinterest." Pinterest Engineering Blog, 2022. https://medium.com/@Pinterest_Engineering/gpu-accelerated-ml-inference-at-pinterest-ad1b6a03a16d
37. Pinterest Engineering. "User Action Sequence Modeling for Pinterest Ads Engagement Modeling." Pinterest Engineering Blog. https://medium.com/pinterest-engineering/user-action-sequence-modeling-for-pinterest-ads-engagement-modeling-21139cab8f4e
38. Li, J., et al. "ML-DCN: Masked Low-Rank Deep Crossing Network Towards Scalable Ads Click-through Rate Prediction at Pinterest." Pinterest, 2026. https://arxiv.org/abs/2602.09194
39. Pinterest. "MTMD: A Multi-Task Multi-Domain Framework for Unified Ad Lightweight Ranking at Pinterest." 2025. https://arxiv.org/abs/2510.09857
40. Pinterest Engineering. "Unlocking Efficient Ad Retrieval: Offline Approximate Nearest Neighbors in Pinterest Ads." Pinterest Engineering Blog. https://medium.com/pinterest-engineering/unlocking-efficient-ad-retrieval-offline-approximate-nearest-neighbors-in-pinterest-ads-6fccc131ac14
41. Pinterest. "MultiBiSage: A Web-Scale Recommendation System Using Multiple Bipartite Graphs at Pinterest." 2022. https://arxiv.org/abs/2205.10666
42. Pinterest. "OmniSearchSage: Multi-Task Multi-Entity Embeddings for Pinterest Search." 2024. https://arxiv.org/abs/2404.16260
43. Shaped.ai. "Inside PinRec - Pinterest's Production-Ready Generative Retrieval Model." 2025. https://www.shaped.ai/blog/pinrec-teardown-inside-pinterests-production-ready-generative-retrieval-model
44. ZenML LLMOps Database. "Pinterest: Large-Scale Learned Retrieval System with Two-Tower Architecture." https://www.zenml.io/llmops-database/large-scale-learned-retrieval-system-with-two-tower-architecture

23. LinkedIn Case Study: Feed, PYMK, Jobs, LiRank, LiGNN, 360Brew and the ML Platform Beneath Them

LinkedIn is one of the most thoroughly documented recommender organizations in industry. Over about fifteen years it has published production systems at every layer of the stack:

- statistical personalization models (GLMix in Photon ML, then GDMix);
- feature platforms (Feathr, with Venice as the online store);
- search and retrieval infrastructure (Galene, then GPU retrieval with LiNR);
- graph learning (LiGNN);
- large deep ranking models (LiRank);
- a 150B-parameter decoder-only foundation ranker (360Brew);
- LLM-based dual-encoder retrieval for the Feed (2025).

Its products span several recommendation problems with different shapes:

- **Feed:** a heterogeneous, multi-objective stream.
- **People You May Know (PYMK):** link prediction on a social graph.
- **Jobs:** a two-sided marketplace with sparse, high-value labels such as applications and recruiter replies.
- **Ads:** calibrated CTR prediction feeding an auction.
- **Recruiter / Talent Search:** search ranking with fairness constraints.

This chapter follows that history as a single case study. It starts with the statistical era (GLMix and "fixed plus random effects" personalization). It then covers the feature and serving substrate (Feathr, Venice, Galene), the 2019 federated Feed architecture and its multi-objective tuning, the 2024 deep-learning consolidation (LiRank, LiGNN, LiNR), and the 2025-2026 move to LLM-native ranking and retrieval (360Brew, LLM dual encoders). It closes with experimentation and fairness infrastructure (XLNT/T-REX, fairness-aware Recruiter ranking), a comparison table and lessons for designers.

> **Sourcing note.** For this chapter, the live web-search budget was exhausted and outbound access to arxiv.org, linkedin.com, engineering.linkedin.com, dl.acm.org and most blogs was blocked from the research environment. Primary sources read directly in this session: the 360Brew technical report PDF (full text, from the authors' GitHub repository) [1][2][3], and the GitHub repositories and READMEs of Photon ML [6], GDMix [8], Feathr (including its talk slides) [10][11], Venice [12], DeText [14], Liger-Kernel [24] and FastTreeSHAP [25]. Facts about LiRank, LiGNN, LiNR, the 2025 LLM retrieval paper, the Feed architecture, XLNT/T-REX and fairness-aware ranking come from abstracts and primary sources that other chapters of this Atlas fetched and verified earlier (Chapters 1-8, 12, 14-16). Those facts are cross-referenced, and they could not be re-fetched here. Numbers from those sources are marked approximate where the original wording could not be checked again. Readers should check exact figures against the cited papers.

23.1 The shape of LinkedIn's recommendation problems

LinkedIn's recommendation surfaces share one underlying asset, the "economic graph" of members, companies, jobs, skills, schools, posts and groups. Their objectives differ sharply, though:

Surface	Item type	Key labels	Distinctive constraint
Feed	posts, articles, network updates, recommended content, ads	react, comment, share, dwell, click, "viral actions"	many inventory sources; creator-side value; multi-objective
PYMK	members	invitation sent, accepted	graph link prediction; reciprocity; cold-start members
Jobs (recs + search)	job postings	view, apply, dismiss, "qualified application", recruiter hearing back	two-sided marketplace; short-lived items; sparse labels
Ads	sponsored content, text ads	click, conversion	calibrated probabilities for auctions
Recruiter / Talent Search	member profiles	InMail sent, accepted	query-dependent ranking; fairness constraints
LinkedIn Learning	courses	enroll, watch	content cold start

The 360Brew report says this directly. In the traditional design, each of these tasks "is conventionally addressed by dedicated models that have been developed and maintained over multiple years by teams of a similar or larger size" than the foundation-model team [1]. The report describes the resulting stack as "complex, multilayered architectures that ... often incorporate thousands of predictive models" [1]. Much of LinkedIn's published infrastructure attacks the cost of that model sprawl:

- Feathr: shared features.
- GDMix and Photon ML: shared personalization training.
- LiRank: shared ranking architecture.
- LiGNN: shared graph embeddings.
- 360Brew: one model for many tasks.

The canonical retrieval-then-ranking funnel, as drawn in the 360Brew report [1]:

```
members (QPS = member count x interaction frequency)
      |
      v
+-------------------+   large K (item pool >> member pool, e.g. posts),
|  Retrieval layer  |   high recall, two-tower / GPU exhaustive / LLM dual-encoder
+-------------------+
      |  hundreds to ~2,000 candidates
      v
+-------------------+   small K, high precision; DCN-style rankers (LiRank),
|   Ranking layer   |   or a text foundation ranker (360Brew)
+-------------------+
      |
      v
re-ranking / blending / fairness / ads slotting  -->  member-facing list
```

23.2 The statistical era: GLMix, Photon ML and GDMix

23.2.1 Problem

Before deep learning took over, LinkedIn needed per-member personalization for job recommendations, the feed and ads. It had to scale to hundreds of millions of members, each with relatively little data. A single global model underfits individual taste. A separate model per member overfits and cannot share signal.

23.2.2 GLMix and GAME (Photon ML)

GLMix (Generalized Linear Mixed models; Zhang et al., KDD 2016 [7]) solves this with the classical statistical idea of **fixed effects plus random effects**:

- a **global (fixed-effect) model** learns from all data;
- **per-entity random-effect models** learn per-member coefficients (and, for jobs, per-job coefficients) that capture individual deviations from the global trend;
- the score is the sum: global score + member-specific score + item-specific score, passed through a link function (logistic for clicks/applies).

Photon ML is LinkedIn's open-source Spark library for this family [6]:

- Supports GLMs (linear, logistic and Poisson regression) and **GAME (Generalized Additive Mixed Effects)** models, of which GLMix is the supported type.
- Training uses **coordinate descent**. The fixed effect and each random-effect "coordinate" are optimized in turn, while the others are held fixed and contribute as an **offset** (scores take the form x . B + offset).
- Can "scale model training up to hundreds of billions of coefficients" within Spark [6].
- Optimizers: L-BFGS and TRON. Regularization: L1, L2 and elastic net. Workflows: warm start, incremental training, partial retraining with locked coefficients, Avro I/O, and Sobol/Bayesian hyperparameter search (experimental) [6].
- Its example recommender is a fixed-effect model + per-user random effect + per-item random effect + a user-item matrix-factorization term [6].

The Photon ML README lists the LinkedIn products that adopted GLMix: **job search and recommendation, news feed ranking, ads CTR prediction and "People Also Viewed"** [6].

```
GLMix / GAME (coordinate descent with offsets)
-----------------------------------------------
round r:
  1. fixed effect  f(x)       <- fit on ALL rows, offset = sum(random effects)
  2. per-member    g_m(x)     <- fit per member m on its rows, offset = f + h
  3. per-job       h_j(x)     <- fit per job j on its rows,  offset = f + g
repeat until convergence
serve: logit = f(x) + g_member(x) + h_job(x)
```

23.2.3 GDMix: deep fixed effects plus per-member random effects

GDMix (Generalized Deep Mixed model; LinkedIn, 2020 [8][9]) is the successor to Photon ML for deep learning. The design keeps the divide-and-conquer structure but allows the **fixed effect to be a deep model**:

- **Fixed effect:** logistic regression or a **DeText** deep NLP ranking model. Scores from any external model can be plugged in [8].
- **Random effects:** per-entity logistic regressions. There can be **hundreds of millions** of them, one per LinkedIn member [8].
- **Training pipeline** [8]:
    1. Distributed fixed-effect training. LR uses all-reduce gradients and SciPy L-BFGS. DeText uses parameter-server async or Horovod sync training.
    2. Score the training data with the fixed effect.
    3. A Spark job partitions records by entity ID.
    4. Workers train their per-entity LR models independently in parallel.
    5. Iterate (parallel blockwise coordinate descent).
- Runs on Kubernetes with Kubeflow and spark-on-k8s-operator, with shared storage [8].
- The README's worked example is job recommendation. Its MovieLens demo shows how much the random effects add. The LR global model reaches AUC 0.6237. Per-user random effects lift it to 0.7058 and per-movie random effects to 0.7599. A DeText global model gives 0.7090 global and 0.7680 overall [8].

DeText [14] is the deep text-ranking component used as a GDMix fixed effect:

- word embeddings, then a CNN/LSTM/BERT text encoder;
- query-document interaction features (cosine, Hadamard, concatenation);
- a wide-and-deep merge with hand-crafted features in an MLP;
- learning-to-rank heads including TF-Ranking losses.

Its papers (SIGIR 2019, KDD 2019, CIKM 2020, KDD 2020) cover deep NLP for LinkedIn search and recommendation [14].

23.2.4 Lessons from the mixed-model era

- **Per-member models are a structural answer to personalization.** Learned ID embeddings are the alternative. Random-effect coefficients are interpretable, cheap to retrain per entity, and parallelize naturally after a partition by member ID.
- **Offsets make modular stacking possible.** Any upstream score, including a deep model or an external model, can become the "fixed effect", and the personalization layer trains around it. The same idea of "train the residual" returns later in LiRank's calibration layers and 360Brew's in-context personalization.
- **Weakness:** random effects are linear and per-entity, so they need enough data per member. Cold-start members fall back to the global model. 360Brew later attacks this regime explicitly (Section 23.7).

23.3 Feature, storage and search substrate: Feathr, Venice, Galene

23.3.1 Feathr

Problem. Feature pipelines were rebuilt redundantly by every team. Teams had "no common type system, no common feature namespace", definitions were inconsistent, and features were inconsistent between training and serving (Chapter 8 quoting the Feathr community talk [11]). The Feathr talk slides list the problems as: features are not reused and are hard to track for compliance, definitions are inconsistent across teams, getting features into production is hard, offline/online inconsistency degrades models, and feature quality drifts and needs human review [11].

Design [10][11]:

- **Declarative feature definitions** [5] ("anchors" over batch or streaming sources, plus derived features) registered under shared names.
- **Point-in-time correct joins.** For each observation (impression) row, Feathr takes the feature value "closest from the past" for the same key [4]. This prevents label leakage. At billions of rows it relies on optimizations such as **bloom filters and salted joins** [10].
- Window aggregations (for example, 90-day averages), derived features and PySpark/Spark-SQL UDFs.
- A **unified API** where the same transformation runs offline, in streaming and online, plus a registry and UI for search, lineage and access control [10].

History and scale. Development began in 2017. The majority of LinkedIn ML applications were onboarded by 2020. Feathr was open-sourced in 2022 with Azure integration and joined LF AI & Data as a sandbox project in September 2022. The README states production use at LinkedIn for more than 6 years [10]. The Chapter 8 summary of the community talk reports "hundreds of models", "thousands of features", petabyte scale, feature experimentation going "from weeks to days", and running time improved "as much as 50%" [11]. Use cases named in the slides include **"Learning Hiring Preferences: The AI Behind LinkedIn Jobs"**, **"Personalized Recommendations in LinkedIn Learning"** and **"Near real-time features for near real-time personalization"** [11].

23.3.2 Venice: the derived-data online store

Ranking requests fan out to thousands of candidate keys, so the online feature store is read-dominated. LinkedIn's open-source **Venice** [12] is the serving substrate:

- **Ingestion:** batch push (full dataset swap from Hadoop/Spark), incremental push, streaming writes via Samza or an online producer, "write compute" partial updates, and hybrid batch-plus-stream stores with a configurable rewind time.
- **Read paths:** single and batch get, plus **server-side read compute** (dot product, cosine similarity, field projection). This lets embedding similarity run next to the data.
- **Three client tiers with documented latency** [12]:

Client	Hops	Reported latency
Thin client	2 network hops, stateless	< 10 ms
Fast client	1 hop, partition-aware	< 2 ms
Da Vinci client	0 hops, stateful local cache	< 1 ms

- Active-active cross-region replication with **CRDT-based conflict resolution**, multi-tenancy, and change-data-capture streams. The README describes Venice as "particularly suitable as the stateful component backing a Feature Store, such as Feathr" [12].

```
Hadoop/Spark offline features --batch push--\
                                            >--> Venice store (per region,
Kafka -> Samza nearline features --stream---/     active-active, CRDT merge)
                                                      |   read compute:
Feathr defs (anchors, derived, registry)              |   dot / cosine
   |-- offline: PIT join onto impressions --> training data
   `-- online: batch-get features <---------- ranking services
                       (thin <10ms | fast <2ms | Da Vinci in-process <1ms)
```

23.3.3 Galene: search infrastructure as a retrieval engine

Galene is LinkedIn's Lucene-based search architecture. It was announced on the LinkedIn engineering blog in 2014 [13] and powers member (people) search, job search and other verticals. In LinkedIn's design, search and recommendation overlap heavily. Job recommendations, Recruiter and PYMK-adjacent "people search" all retrieve from inverted indexes, and DeText-style models rank the results [13][14]. Based on the 2014 announcement, as summarized in secondary coverage that could not be re-fetched in this session [13], Galene's distinctive features were:

- offline index construction on Hadoop combined with live index updates;
- static rank and early termination;
- a federated broker/searcher tier.

Treat these architectural details as approximate.

Lesson. At LinkedIn the "retrieval layer" for many recommenders is a search engine with structured query rewriting (title, skills, location), not only an ANN index. This matters for Jobs, where hard constraints such as location, seniority and visa requirements are better expressed as index filters. The same insight returns in LiNR's attribute pre-filtering on GPUs (Section 23.6).

23.4 Feed ranking: from federated FPR/SPR to multi-objective autotuning

23.4.1 Architecture (2019)

LinkedIn's 2019 "Community-focused Feed optimization" post [15] describes a **federated funnel**:

- Multiple **first-pass rankers (FPRs)**, one per inventory (network updates, jobs, recommended content, and so on), preselect and score candidates.
- A **second-pass ranker (SPR)** merges them into one list.
- **FollowFeed**, which serves updates from a member's network, was the dominant FPR, supplying **more than 80% of feed updates** [15].

```
FollowFeed FPR ----+   (network updates; >80% of feed items, 2019)
Jobs FPR ----------+
Recommended FPR ---+--> SPR: unified features, multi-objective score
PYMK / modules ----+        sum_k w_k * P(action_k) (+ creator-side terms)
Ads (separate auction) -------------+
                                    v
                    slotting (ads in allowed positions) -> Feed
```

Chapters 1 and 15 discuss the trade-off. The federated design matches the organization, since each inventory team owns its FPR, but it pushes **cross-source calibration** onto the SPR. If one FPR's score distribution drifts, its share of the feed drifts too.

23.4.2 Objectives and the value function

The Feed ranks by a **linear combination of per-objective probabilities**: viral actions (react, comment, share), contributions, clicks, and downstream effects on creators. The 2019 post adds **creator-side value**, meaning the value of timely feedback to the poster and their community, not only the viewer's own engagement [15].

23.4.3 Autotuning the weights (2018-2020)

ML engineers originally set the combination weights by hand, which LinkedIn described as very inefficient [16][17]. The automated approach, from LinkedIn's "Online parameter selection for web-based ranking problems" (KDD 2018, summarized in [17]) and covered in Chapter 4:

1. Treat each online metric as a black-box function of the control parameters (for example the click weight), and model it with a **Gaussian process** fitted on A/B data.
2. Optimize a **primary metric subject to constraints** on others. LinkedIn observed that contributions fall as the click weight rises while normalized clicks rise, so the optimum sits on the constraint boundary.
3. Choose the next parameters with a **Thompson-sampling-style** procedure. Draw functions from the GP posterior, take each draw's argmax, and estimate the probability that each parameter is optimal.
4. Because ramp volume is limited, run many low-traffic experiments in parallel.

Lesson. Separate *prediction* (calibrated per-action models) from *policy* (a small, versioned value function), and tune the policy online with constrained Bayesian optimization. LinkedIn, Meta, X and Pinterest all keep this split (Chapter 4).

23.4.4 Feed retrieval in the LLM era (2024-2026)

Two later systems rebuilt the retrieval side of the Feed:

- **LiNR (2024)** for out-of-network post retrieval on GPUs (Section 23.6).
- **LLM dual-encoder retrieval (2025, AAAI 2026)** [19]. LinkedIn fine-tuned **LLaMA 3** as a dual encoder. Member and post texts are encoded separately and pooled into embeddings. The system retrieves about **2,000 candidates from hundreds of millions of posts** within a few-millisecond budget at several thousand QPS (approximate). Two findings stand out:
    - numerical features written into prompts work better when **quantized into tokens**;

    - the largest online gains went to **newer members** with sparse networks, the cold-start regime where content-based LLM embeddings beat ID and graph signals [19].

Secondary coverage of a 2026 LinkedIn post describes a further consolidation. The multi-source retrieval described above is replaced by a unified LLM retrieval system, followed by a transformer-based sequential "generative recommender" ranker (Chapter 1). These details are from secondary sources and are approximate. A 2026 LinkedIn paper on a **Connected Content Retriever** describes "GPU Retrieval as Ranking": the whole candidate set sits in GPU memory and every candidate is scored with dense matrix operations. It also describes **SilverTorch**, which implements ANN search and filtering as tensor operations with a fused int8 IVF kernel and a GPU bloom-filter index [21].

```
2019:  FPR_1 ... FPR_n (per-inventory)  -->  SPR (GLMix/DNN)  --> blend
2024:  + LiNR GPU out-of-network retriever; LiRank deep SPR; LiGNN embeddings
2025:  LLM (LLaMA-3) dual-encoder retrieval, ~2k cands, quantized numeric tokens
2026:  unified LLM retrieval + sequential generative ranker (approx., secondary)
       GPU "retrieval as ranking" + SilverTorch tensor ANN for pre-ranking
```

23.5 LiRank: industrial large-scale ranking (KDD 2024)

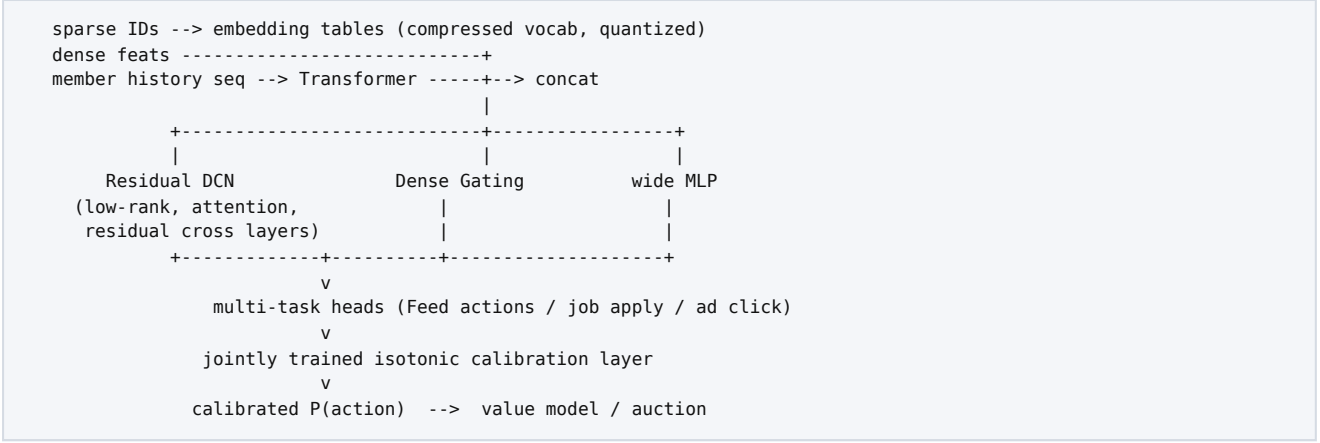
23.5.1 Problem

By 2023 LinkedIn ran separate deep rankers for Feed, Jobs and Ads, each tuned independently. LiRank [18] is a framework that brings "state-of-the-art modeling architectures and optimization methods" into production across all three, along with the engineering needed to train and serve large rankers.

23.5.2 Architecture and key techniques

From the paper's abstract and the summaries verified in Chapters 3, 4 and 16 [18]:

- **Residual DCN.** DCNv2 extended with **attention and residual connections** in the cross network, using **low-rank** cross matrices. The paper reports a reduction of nearly **30% in parameters**.
- **A unified architecture** combining Residual DCN with **Dense Gating**, a large MLP and **Transformers** over member history sequences. The paper reports how to combine and tune these blocks.
- **Isotonic calibration layer trained jointly with the model.** It sits inside the network instead of a post-hoc Platt or isotonic fit. A 2026 follow-up paper generalizes it into one learnable component that also handles position debiasing and multi-task bias correction [20].
- **Deep-learning-based explore/exploit** in production.
- **Model compression for serving:** quantization and **vocabulary compression** of the large ID embedding tables, so large models fit within serving memory and latency budgets.
- **Incremental / warm-start training** to keep models fresh (as described in the paper; details not re-checked).



23.5.3 Reported impact

From the abstract (not re-fetched; approximate): about **+0.5% member sessions** in Feed, about **+1.76% qualified job applications** for Jobs search and recommendations, and about **+4.3% Ads CTR** [18].

23.5.4 Lessons

- **Low-rank cross layers are a large parameter win at small quality cost.** LinkedIn and Pinterest both run low-rank DCN variants in production (Chapter 3).
- **Calibration belongs inside the model** when calibrated probabilities feed a value function or an auction. A jointly trained isotonic layer avoids the drift between a model and a separately fitted calibrator.
- **Compression is part of the modeling work.** Vocabulary compression and quantization decide whether a better model can ship at all.

23.6 Retrieval and graph learning: LiNR and LiGNN

23.6.1 LiNR: model-based neural retrieval on GPUs (2024)

Problem. Out-of-network Feed post retrieval needed attribute filters (language, freshness and so on). Standard KNN with **post-filtering** lost recall when filters removed many results [22].

Design [22]:

- **Item vectors and model weights live in the same model binary.** Index construction becomes part of training and export, so the index cannot fall out of sync with the query tower.
- **Exhaustive GPU scoring with attribute pre-filtering** over a **billion-sized index**. PyTorch and TensorFlow lacked efficient pre-filtering, so LinkedIn built custom indexing and filtering ops.
- Multi-embedding retrieval, cold-start strategies and **quantization** for larger indexes. Secondary reviews describe 1-bit random-projection compression and near-real-time index updates through change-data capture (approximate).
- Result: about **+3% relative professional daily active users** [22].

```
model binary = { query tower weights, item matrix [N x d], item attrs }
request:  q    = tower(member)                      (GPU)
          mask = prefilter(attrs)                  (GPU, exact)
          s    = q . items^T over masked rows      (dense / quantized)
          topk(s) --> ranking
```

LiNR is a step toward merging retrieval and ranking into one GPU model. The 2026 "retrieval as ranking" and SilverTorch work continues in the same direction [21].

23.6.2 LiGNN: production GNNs (KDD 2024)

Problem. LinkedIn's heterogeneous graph covers members, jobs, companies, posts, ads, groups and skills. Static shallow embeddings could not handle cold-start entities (new jobs, new posts), temporal drift, or the cost of training on a graph this large [23].

Design [23] (see Chapter 7 for the full treatment):

- **Temporal graph architectures with long-term losses.** Neighbor sampling respects edge timestamps, so the model learns to predict future engagement from past neighborhoods.
- **Graph densification for cold start.** Low-degree nodes get extra edges to similar entities, so the GNN has neighbors to aggregate.
- **ID embeddings plus multi-hop neighbor sampling**, combining transductive capacity with inductive generalization.
- **Training systems:** about a **7x training speedup** from adaptive neighbor sampling, grouping and slicing of training batches, a specialized shared-memory queue, and local gradient optimization.
- **Nearline inference** keeps embeddings fresh. Embeddings flow into downstream rankers and retrievers as features.

```
heterogeneous graph (member/job/post/ad/company/skill, typed + timestamped edges)
    |  densify low-degree nodes (cold start)
    v
temporal multi-hop sampling --> GNN (ID emb + content feats, long-term losses)
    |  7x faster: adaptive sampling, batch grouping/slicing, shm queue
    v
embeddings (batch + nearline) --> Venice / ANN --> Feed, Jobs, PYMK, Ads models
```

Reported A/B impact (approximate) [23]:

- about +1% relative job-application **hearing-back rate**;
- about **+2% Ads CTR**;
- about +0.5% Feed engaged DAU;
- about +0.2% sessions;
- about +0.1% weekly active users from **people recommendations (PYMK)**.

23.7 People You May Know (PYMK)

PYMK is LinkedIn's oldest graph recommender and the main driver of network growth. Its classical shape is a **triangle-closing** candidate generator (friends of friends, plus co-attributes such as shared company, school or contact-book overlap), followed by a learned

ranker that predicts invitation and acceptance (Chapter 7). The publicly verifiable recent facts in this session's sources are:

- PYMK is one of the four main consumers of **LiGNN** embeddings, with about a +0.1% WAU lift from people recommendations attributed to GNN features [23].
- GLMix was used for the related "**People Also Viewed**" recommender [6].

Detailed PYMK serving numbers, such as candidate counts and latency, were not found in sources reachable for this chapter. They are deliberately left out rather than estimated.

Design notes for a PYMK-style system (synthesis from the graph-recsys patterns in Chapter 7 and LiGNN):

```
member m
  |-- 2-hop graph walk (friends-of-friends, weighted by edge strength)
  |-- co-attribute joins (same company/school/time overlap)
  |-- address-book / imported contacts (where consented)
  |-- GNN embedding ANN (LiGNN-style; covers cold-start members)
  v
candidate union (dedupe, filter existing connections, blocked, pending)
  v
ranker: P(invite) x P(accept | invite)  (reciprocity), plus growth objectives
  v
diversification + frequency caps --> PYMK module (also slotted into Feed)
```

The key modeling point is **reciprocity**. A good PYMK suggestion must be accepted by the *other* member, so the objective is a product of both sides' propensities, unlike a one-sided content recommendation.

23.8 Job recommendations and the Jobs marketplace

Jobs is the surface where LinkedIn's history shows most clearly, because every generation of its stack was applied here:

Generation	Model	Source
2016	GLMix: global + per-member + per-job random effects	[6][7]
2020	GDMix: DeText deep fixed effect + per-member LR	[8][9][14]
2020-22	Feathr features, for example "Learning Hiring Preferences"	[10][11]
2024	LiRank: about +1.76% qualified applications (approx.)	[18]
2024	LiGNN embeddings: about +1% hearing-back rate (approx.)	[23]
2025	360Brew: job apply/view/dismiss as a text prompt	[1][2]

Marketplace considerations. Job recommendations are two-sided. LinkedIn's reported metrics are not raw clicks. They are **"qualified job applications"** [18] and the **"hearing back" rate** [23], meaning applications that a recruiter actually responds to. Optimizing for the employer-side outcome keeps the system from flooding popular postings with poorly matched applications.

Hiring preferences. The Feathr talk cites "Learning Hiring Preferences: The AI Behind LinkedIn Jobs" as a flagship Feathr use case [11]. In that work, recruiter-side signals become features of the member-job ranker. The original post could not be fetched here.

23.9 360Brew: a decoder-only foundation model for ranking (2025)

23.9.1 Problem

The traditional stack relies on "hundreds of thousands of handcrafted features" and on ID-based embedding tables, with three structural weaknesses the report names [1]:

- **Cold start:** ID models cannot generalize to new members and items without frequent retraining.
- **Interactions:** cross features have to be hand-crafted or learned by special architectures such as DCN.
- **Domain generalization:** a model trained on Feed cannot move to Jobs, or from predicting interactions to predicting ratings.

360Brew's stated primary objective is "to enhance AI development productivity across LinkedIn by centralizing common modeling components into a foundational layer". It also enables governance and moderation at that layer [1].

23.9.2 Formulation

Each member's profile and history is treated as a **many-shot in-context learning** problem. The model approximates the joint distribution

P(m, (e1, i1), ..., (eN, iN))

over a member profile m and the member's historical entities e_j with interactions i_j, all verbalized as text. Any task then becomes a conditional marginal: P(i_t, i_t+1, ... | task instruction, m, history, new entities e_t, ...). This is next-token prediction, the same objective decoder LLMs train on [1].

Prompt structure (job recommendation example from the report) [1][2]:

```
Instruction:  member profile + jobs + past interactions (applied / viewed /
              dismissed / no interaction); predict apply/view/dismiss for the
              "Question" job. (May include steering, e.g. "focus on skills,
              location, years of experience"; "30% weight to profile match,
              70% to historical activity")
Member Profile:  current position, company, location
Past job interaction data:
   applied: [Title, Location, Country, Company, Description] ...
   viewed:  [ ... ]
Question:  Will the member apply to the following job: [ ... ]
Answer:    The member will apply
```

For comparison with production binary rankers, prompts are written so the model answers yes or no. The **logits of the answer tokens** are used as scores to compute AUC and similar metrics [1].

23.9.3 Model and data

- **150B parameters**, decoder-only, built on the **Mixtral 8x22B** pre-trained MoE architecture [1].
- Further trained on LinkedIn raw entity data (member profiles, job descriptions, posts) and interaction data (including job applications) across **5+ surfaces**. The data is primarily first-party, from users outside the European Union [1].
- Evaluated on **30+ predictive tasks across 8+ surfaces**, split into T1 (in-domain, with at least a one-month gap between training and test data) and T2 (out-of-domain surfaces and tasks not seen in training) [1].
- Built by a small team over **9 months**. The model is described as "pre-production" (V1.0) [1].
- The report says the language models are used only for ranking and recommendation, not for content creation [1].

23.9.4 Results and scaling observations

- **Quality:** "comparable to or exceeding" current production systems on offline metrics, **without task-specific fine-tuning** [1].
- **Data scaling:** scaling processed data by **over 3x**, across **five releases over nine months**, improved T1 performance significantly over baselines, and "the slope ... does not appear to be flattening" [1].
- **Model scaling:** larger pre-trained architectures performed better [1].
- **History (context) scaling:** performance rises with maximum context length, normalized to an 8K-token baseline. The authors frame this as "replac[ing] the problem of feature engineering with the more tractable challenge of scaling up architecture depth and context length" [1].
- **Out-of-domain:** on 4 T2 tasks across 4 unseen surfaces, performance was similar to or better than production models [1].
- **Cold start:** the gap over production is largest when member history is cut from 100 interactions down to 5 [1].
- **Temporal robustness:** performance degrades less than baselines as test data moves further from the training period. The authors attribute this to in-context adaptation, which could reduce retraining frequency [1].

23.9.5 Lessons and open questions

- The report covers **ranking** only. Using the same model for **retrieval through embeddings** is stated as future work [1].
- Serving a 150B MoE at Feed QPS is not discussed in V1.0. The parallel 2025 Feed retrieval work fine-tunes a much smaller LLaMA 3 dual encoder [19]. That suggests the large model is a teacher or a ranker for high-value surfaces rather than a scorer for every candidate (an inference, not a published fact).
- Training LLM-scale models at LinkedIn is supported by open-source infrastructure such as **Liger-Kernel**. It provides Triton kernels (RMSNorm, RoPE, SwiGLU, fused linear cross-entropy) and reports about **+20% multi-GPU training throughput** and about **60% memory reduction**, with up to 80% memory savings for post-training losses such as DPO and ORPO [24].

23.10 Experimentation and responsible ranking

23.10.1 XLNT, Lix and T-REX

LinkedIn's A/B platform **XLNT** (Xu et al., KDD 2015 [26]) uses the **Lix** targeting DSL over member attributes and supports gradual ramps (for example 1% to 5% to 50%), with hundreds of metrics per test. The **T-REX** engine re-implemented Lix evaluation as compiled, in-process code, about **20x faster**, at a scale of trillions of variant evaluations per day (approximate) [27]. Speed matters because ranking services check experiment membership several times per request. The constrained-BO Feed tuning in Section 23.4.3 runs on top of this infrastructure.

23.10.2 Fairness-aware ranking in Recruiter

For LinkedIn Talent Search, Geyik, Ambler and Kenthapadi (KDD 2019) [28] describe a **post-processing re-ranker**. It makes the gender distribution of every **prefix** of the top-k results match the distribution among *qualified* candidates for that query, using the DetGreedy, DetCons and DetRelaxed algorithms. It shipped to all Recruiter users worldwide, with a reported near three-fold increase in queries with representative results and no significant drop in business metrics (Chapter 14).

23.10.3 Explainability tooling

FastTreeSHAP [25] is LinkedIn's open-source faster TreeSHAP. Variant v1 is about 1.5x faster at equal memory and v2 about 2.5x faster at higher memory, for explaining tree-based models with millions of rows.

23.11 Comparison of LinkedIn's published recommender systems

System (year)	Layer	Core idea	Scale (public)	Reported impact	Ref
GLMix / Photon ML (2016)	ranking	fixed + per-member/per-item random effects, coordinate descent	up to hundreds of billions of coefficients	adopted by Jobs, Feed, Ads CTR, People Also Viewed	[6] [7]
GDMix (2020)	ranking	deep fixed effect (DeText) + per-entity LR	hundreds of millions of random-effect models	MovieLens AUC 0.62 -> 0.77 with random effects	[8] [9]
Feathr (2017; OSS 2022)	features	declarative features, PIT joins, registry	hundreds of models, thousands of features, PB	weeks -> days experimentation; up to 50% faster	[10] [11]
Venice	serving store	batch + stream derived data, read compute	Da Vinci < 1 ms reads	substrate for Feathr	[12]
Feed FPR/SPR (2019)	funnel	federated per-inventory rankers + SPR	FollowFeed > 80% of updates	creator-side objectives	[15]

System (year)	Layer	Core idea	Scale (public)	Reported impact	Ref
Feed autotuning (2018-20)	policy	GP + Thompson sampling, constrained	parallel low-traffic ramps	replaced hand tuning	[16] [17]
LiRank (2024)	ranking	Residual DCN, gating, transformers, isotonic layer	~30% fewer DCN params	Feed +0.5% sessions; Jobs +1.76% qualified apps; Ads +4.3% CTR (approx.)	[18]
LiGNN (2024)	embeddings	temporal GNN, densification, nearline	7x training speedup	Ads +2% CTR; Jobs +1% hearing back (approx.)	[23]
LiNR (2024)	retrieval	GPU exhaustive + pre-filter, index in model	~1B-item index	+3% professional DAU	[22]
LLM dual encoder (2025)	retrieval	LLaMA 3 dual encoder, quantized numeric tokens	~2k of hundreds of millions	biggest gains for new members	[19]
360Brew V1.0 (2025)	ranking FM	150B Mixtral-based decoder, text prompts	30+ tasks, 8+ surfaces	matches/exceeds prod offline, zero-shot OOD	[1] [2]
CCR / SilverTorch (2026)	pre-ranking	GPU retrieval as ranking, tensor ANN	int8 IVF, GPU bloom filter	n/a here	[21]

23.12 Key takeaways for system designers

1. **Model sprawl is the real enemy at scale.** LinkedIn's arc runs from GLMix/GDMix (a shared training paradigm) through Feathr (shared features), LiRank (a shared architecture) and LiGNN (shared embeddings) to 360Brew (one model for 30+ tasks). It is one long campaign against the cost of "thousands of predictive models" [1]. Count the models and features your organization maintains, and budget for that cost.
2. **Fixed plus random effects is still a strong personalization baseline.** Per-member coefficients trained after an ID-based partition scale to hundreds of millions of members on commodity hardware and stack cleanly on any global model through offsets [6][8].
3. **Point-in-time correctness and a single feature definition come first.** Feathr's PIT joins and unified offline/online definitions, plus Venice's tiered read paths (< 10 ms, < 2 ms, < 1 ms), are what make the models trustworthy and fast [10][12].
4. **Federated funnels need explicit calibration.** An FPR/SPR design fits the org chart but makes the SPR responsible for cross-source calibration. Calibrate inside the model (LiRank's isotonic layer) and tune the value function online with constrained Bayesian optimization [15][17][18].
5. **Ship compression together with the model.** Low-rank DCN, vocabulary compression and quantization are what let LiRank serve large models [18]. LiNR's quantized GPU index is what makes billion-item exhaustive retrieval affordable [22].
6. **Pre-filter, don't post-filter.** For filtered retrieval (language, freshness, location), exact pre-filtering on the GPU beats ANN plus post-filtering on recall [22].
7. **Graph learning pays off as a platform.** LiGNN's individual lifts are around 0.1-2%, but they apply to Feed, Jobs, PYMK and Ads at once. Most of the engineering effort

goes into temporal sampling, densification and data-loader throughput, not the convolution [23].

8. **Optimize marketplace outcomes, not clicks.** For Jobs, report qualified applications and hearing-back rates. For PYMK, model reciprocity [18][23].
9. **Text foundation rankers trade compute for engineering effort.** 360Brew's evidence is that more data, more context and larger models keep helping. The gains are largest for cold-start members and out-of-domain surfaces, and the model ages more slowly over time [1]. Expect to pair such a model with smaller, cheaper LLM retrievers and rankers at high-QPS stages [19].
10. **Numbers in prompts need care.** Quantizing numeric features into tokens improved LLM retrieval [19]. 360Brew avoids time-window count features entirely by putting raw history into the context [1].
11. **Ranking ships with experimentation and fairness infrastructure.** A fast in-process assignment engine (T-REX) and auditable post-processing for fairness (Recruiter) are part of the recommender, not add-ons [26][27][28].

References

1. 360Brew Team (Firooz, H., Sanjabi, M., et al.), LinkedIn Foundation AI Technologies. "360Brew: A Decoder-only Foundation Model for Personalized Ranking and Recommendation." arXiv 2501.16450, 2025 (technical report PDF read from the authors' repository). https://arxiv.org/abs/2501.16450
2. LLMPersonalization (LinkedIn authors). "360Brew-V1" repository README and report PDF. GitHub, 2025. https://github.com/LLMPersonalization/360Brew-V1
3. 360Brew Team, LinkedIn. "360Brew: A Decoder-only Foundation Model for Ranking and Recommendation" (V1.0 report PDF). GitHub, 2025. https://github.com/LLMPersonalization/360Brew-V1/blob/main/360brew_v1.pdf
4. Feathr project (LinkedIn). "Point-in-time Correctness and Point-in-time Join" (docs/concepts/point-in-time-join.md). GitHub, 2022. https://github.com/feathr-ai/feathr/blob/main/docs/concepts/point-in-time-join.md
5. Feathr project (LinkedIn). "Feathr Concepts for Beginners" (docs/concepts/feathr-concepts-for-beginners.md). GitHub, 2022. https://github.com/feathr-ai/feathr/blob/main/docs/concepts/feathr-concepts-for-beginners.md
6. LinkedIn. "Photon ML" repository README (GLM, GAME, GLMix on Spark). GitHub, 2016-2020. https://github.com/linkedin/photon-ml
7. Zhang, X., Zhou, Y., Ma, Y., Chen, B.-C., Zhang, L., Agarwal, D. (LinkedIn). "GLMix: Generalized Linear Mixed Models For Large-Scale Response Prediction." KDD 2016. http://www.kdd.org/kdd2016/papers/files/adf0562-zhangA.pdf
8. LinkedIn. "GDMix: Generalized Deep Mixed Model" repository README. GitHub, 2020. https://github.com/linkedin/gdmix
9. Shi, J., et al. (LinkedIn Engineering). "GDMix: A Deep Ranking Personalization Framework." LinkedIn Engineering Blog, 2020 (linked from [8]; not fetched). https://engineering.linkedin.com/blog/2020/gdmix--a-deep-ranking-personalization-framework
10. Feathr project (LinkedIn; LF AI & Data). "Feathr" repository README and docs (point-in-time joins, concepts). GitHub, 2022-2023. https://github.com/feathr-ai/feathr

11. Feathr project (LinkedIn and Microsoft). "Feathr Feature Store Talk" and "Feathr Community Talk - An Enterprise-Grade High Performance Feature Store" slide decks. GitHub, 2022. https://github.com/feathr-ai/feathr/tree/main/docs/talks
12. LinkedIn. "Venice: Derived Data Platform for Planet-Scale Workloads" repository README. GitHub, 2022-2026. https://github.com/linkedin/venice
13. LinkedIn Engineering. "Did you mean 'Galene'?" LinkedIn Engineering Blog, 2014 (not re-fetched; architectural details approximate). https://engineering.linkedin.com/blog/2014/03/did-you-mean-galene
14. LinkedIn. "DeText: A Deep Neural Text Understanding Framework for Ranking and Classification Tasks" repository README (cites Guo et al. SIGIR 2019, KDD 2019, CIKM 2020; Jia et al. KDD 2020). GitHub, 2020. https://github.com/linkedin/detext
15. LinkedIn Engineering. "Community-focused Feed optimization." LinkedIn Engineering Blog, June 2019. https://engineering.linkedin.com/blog/2019/06/community-focused-feed-optimization
16. LinkedIn (via KDnuggets). "Autotuning for Multi-Objective Optimization on LinkedIn's Feed Ranking." 2020. https://www.kdnuggets.com/2020/08/autotuning-multi-objective-optimization-linkedin-feed-ranking.html
17. Colyer, A. "Online parameter selection for web-based ranking problems" (summary of LinkedIn KDD 2018 paper). The Morning Paper, 2018. https://blog.acolyer.org/2018/10/08/online-parameter-selection-for-web-based-ranking-problems/
18. Borisyuk, F., et al. (LinkedIn). "LiRank: Industrial Large Scale Ranking Models at LinkedIn." KDD 2024, arXiv 2402.06859. https://arxiv.org/abs/2402.06859
19. LinkedIn. "Large Scale Retrieval for the LinkedIn Feed using Causal Language Models." arXiv 2510.14223, 2025 (AAAI 2026). https://arxiv.org/abs/2510.14223
20. LinkedIn. "Isotonic Layer: A Unified Framework for Recommendation Calibration and Debiasing." arXiv 2603.06589, 2026. https://arxiv.org/abs/2603.06589
21. LinkedIn. "Connected Content Retriever: Dense Graph Edge Features Powering Pre-Ranking at LinkedIn." arXiv 2609.22441, 2026. https://arxiv.org/pdf/2609.22441
22. Borisyuk, F., et al. (LinkedIn). "LiNR: Model Based Neural Retrieval on GPUs at LinkedIn." arXiv 2407.13218, 2024. https://arxiv.org/pdf/2407.13218
23. Borisyuk, F., et al. (LinkedIn). "LiGNN: Graph Neural Networks at LinkedIn." KDD 2024, arXiv 2402.11139. https://arxiv.org/abs/2402.11139
24. LinkedIn. "Liger Kernel: Efficient Triton Kernels for LLM Training" repository README and technical report (arXiv 2410.10989). GitHub, 2024-2025. https://github.com/linkedin/Liger-Kernel
25. LinkedIn. "FastTreeSHAP" repository README. GitHub, 2022. https://github.com/linkedin/FastTreeSHAP
26. Xu, Y., Chen, N., Fernandez, A., Sinno, O., Bhasin, A. (LinkedIn). "From Infrastructure to Culture: A/B Testing Challenges in Large Scale Social Networks." KDD 2015. https://dl.acm.org/doi/10.1145/2783258.2788602
27. LinkedIn Engineering. "Making the LinkedIn Experimentation Engine 20x Faster." LinkedIn Engineering Blog, 2019. https://engineering.linkedin.com/blog/2019/06/making-the-linkedin-experimentation-engine-20x-faster
28. Geyik, S. C., Ambler, S., Kenthapadi, K. (LinkedIn). "Fairness-Aware Ranking in Search and Recommendation Systems with Application to LinkedIn Talent Search." KDD 2019, arXiv 1905.01989. https://arxiv.org/abs/1905.01989

24. Amazon and E-commerce: From Item-to-Item Collaborative Filtering to Query-Aware Retrieval

E-commerce is where industrial recommendation started. Amazon's item-to-item collaborative filtering went into production in the late 1990s, was patented in 2001 and described publicly in 2003. It is still the clearest example of a recommender design that won because it fitted the system constraints of its time: a catalog of millions of items, tens of millions of customers, and a page that had to render in well under a second. Twenty years on, e-commerce recommenders look different. They use two-tower retrieval, sequence models, graph embeddings and LLM-derived item representations. But the problem has stayed the same, and so has its structure. Commerce recommenders differ from media and social feeds in several ways:

- **The conversion signal is sparse and delayed but very valuable.** One purchase is worth thousands of impressions, so training labels are imbalanced and multi-level (impression, click, add-to-cart, purchase).
- **The catalog is long-tailed and churns.** New SKUs, third-party listings and one-of-a-kind items (eBay, Etsy) arrive constantly. Many items have no interaction history.
- **Item relationships are typed.** "Similar" (substitute) and "goes with" (complement) are different products with different data sources. Mixing them up is the classic failure.
- **Search and recommendation merge.** Most commerce sessions start with a query, so the best recommenders are *query-aware*. Product search is itself a personalized ranking problem.
- **Inventory, price and availability are hard constraints.** An item that is out of stock, undeliverable or (for grocery) unavailable at the shopper's store has to be filtered out at serving time.

This chapter starts with Amazon's original algorithm and its later refinements. It then covers Amazon's deep-learning infrastructure, its product-search and complementary-item research, and Amazon Personalize, the managed service that packages Amazon-style recipes for other companies. After that come the marketplace and retailer designs (eBay, Etsy, Zalando, Alibaba-style graph embeddings, prod2vec) and query-aware recommendation patterns.

A note on sourcing. For this chapter the live web-search budget was used up, and the writing environment could not reach arXiv, amazon.science or AWS documentation directly. The chapter therefore relies on well-established primary sources (papers, patents, official documentation) whose content and URLs are widely cited and stable. Numbers that could not be re-checked are marked *approximate*. Wayfair, Shopify, Target, Walmart and Instacart have all published recommender and search work on their engineering blogs. Their specific architectures and numbers could not be verified against primary sources in this pass, so

Section 9 covers them only at the level of patterns, without specific figures. Readers should consult those companies' engineering blogs directly.

---

1. Amazon item-to-item collaborative filtering (1998-2003)

1.1 The problem

In 2003 Linden, Smith and York described the constraints Amazon faced [1]. The catalog held several million items and there were more than 29 million customers. Recommendations had to be computed in real time, within about half a second, for every page view. They also had to react immediately to new information: an item added to the cart should change the recommendations on the next page. New customers had very little history, and long-time customers could have thousands of purchases.

The paper looked at the three families of methods available at the time and rejected each of them for this setting:

Approach	How it works	Why it failed at Amazon scale [1]
User-based CF	Find similar customers (cosine over M-dim item vectors), recommend what they bought	Worst case O(MN) per request; dimensionality reduction or sampling hurts quality
Cluster models	Assign users to segments, recommend segment favourites	Cheap online, but coarse; recommendations are poor for an individual
Search/content-based	Query the catalog for same author, artist, keywords	Too narrow: recommends what the customer already knows
Item-to-item CF	Precompute similar-items table; at request time look up items in the user's history	Online cost depends only on the user's history size, not on M or N

(M = number of customers, N = number of catalog items.)

1.2 The algorithm

The key inversion is to compute similarity between **items** rather than between users. This can be done entirely offline. The paper's pseudocode is [1]:

```
For each item I1 in catalog:
    For each customer C who purchased I1:
        For each item I2 purchased by customer C:
            record that a customer purchased I1 and I2
    For each item I2:
        compute similarity(I1, I2)      # cosine over customer vectors
```

Each item is a vector over customers, and similarity is the cosine of the angle between two item vectors. The worst-case offline cost is O(N^2 M). In practice it is close to O(NM), because most customers buy very few items. Precomputing on a sample of customers, or ignoring the heaviest buyers, cuts the cost further [1]. The patent filed by Linden, Jacobi and Benson describes the same similarity-mapping design and how it applies to shopping carts and purchase histories [2]. Sarwar et al. published a closely related academic study of item-based CF in the same period [3].

At request time the system works like this:

```
user history / cart / page context
       |
       v
+---------------------+     lookup per item      +--------------------------+
| seed items (k)      | ----------------------> | similar-items table      |
| purchased, rated,   |                          | item -> [(item, score)]  |
| carted, viewed      | <---------------------- | precomputed offline      |
+---------------------+   top-n lists per seed   +--------------------------+
       |
       v
merge + aggregate scores, drop owned/seen items, apply business filters
       |
       v
"Customers who bought this also bought", "Your Recommendations", cart recs
```

Three properties made it a good production design:

1. **Latency is independent of catalog and customer counts.** The online cost is a few table lookups and a merge.
2. **It is reactive.** Because the inputs are the current cart or the page item, the recommendations change immediately without any retraining.
3. **It is explainable.** "Because you purchased X" falls straight out of the algorithm, and that builds trust and helps users debug the recommendations.

Smith and Linden later noted that the 2003 paper was chosen by *IEEE Internet Computing* as the paper from its first 20 years that had best stood the test of time [4]. Amazon's own retrospective also tells the story of how the algorithm spread across the site and beyond [5].

1.3 Refinements: "Two Decades of Recommender Systems at Amazon.com" (2017)

The 2017 follow-up by Smith and Linden is one of the most useful design documents in the field. It explains why naive co-occurrence fails and how Amazon corrected it [4].

The popularity problem. If you rank related items by raw co-purchase count, every item's "customers also bought" list is dominated by bestsellers. The authors' example is a Harry Potter book. To correct this, compute the *expected* number of co-purchases of X and Y under independence and compare the observed count with it. For each customer c who bought X, the probability that c also bought Y by chance is about 1 - (1 - P_Y)^|c|, where P_Y is Y's purchase probability and |c| is the number of items c has bought. Summing over all buyers of X gives the expected count E_XY. A good relatedness score is then N_XY - E_XY, normalised by something like sqrt(E_XY) (a chi-square-style statistic) [4]. In this formulation, heavy buyers contribute less evidence per co-purchase, which is the same intuition as down-weighting very active users in modern CF.

Other lessons from the 2017 paper [4]:

- **Time and direction matter.** Someone who buys a camera wants a memory card next, not another camera. Directional "bought X then Y" statistics work better than symmetric co-occurrence for complements.

- **Use the right signal for the right relation.** Co-views suggest substitutes ("people who viewed this also viewed"). Co-purchases suggest complements or bundle-mates ("frequently bought together").
- **Items are not all equal.** Different product categories need different treatment. Consumables, books in a series and durable goods have different repeat-purchase and complementarity patterns.
- **The vision.** The authors argue that recommendation should become more like a conversation with a knowledgeable friend: every interaction on the site, including search, should be a recommendation. This anticipates today's query-aware and conversational shopping systems.

---

2. Amazon's deep-learning recommendation stack

2.1 DSSTNE: sparse deep learning for recommendations (2016)

In 2016 Amazon open-sourced DSSTNE (Deep Scalable Sparse Tensor Network Engine). It was built for training and serving neural recommenders on very sparse, high-dimensional input such as one-hot purchase histories over a large catalog [6]. Its design points are typical of commerce recommenders of that time:

- Input layers are extremely wide (one unit per item) and very sparse. DSSTNE uses sparse kernels so that it only computes over non-zero inputs.
- **Model parallelism across GPUs.** Wide layers are split across devices, so the model does not have to fit on one GPU. Most frameworks of the day only did data parallelism.
- Models are specified declaratively (JSON network definitions). The typical use case is an autoencoder-style "history in, purchase-probability-over-catalog out" recommender.

DSSTNE's lesson is still relevant: in commerce the *output* side (scoring the whole catalog) and the *embedding* side are the expensive parts. Modern systems solve this with two-tower retrieval and ANN search rather than full-softmax layers.

2.2 Product-to-product relations: substitutes and complements

Academic work using Amazon data formalised the substitute/complement split. McAuley, Pandey and Leskovec (KDD 2015) learned to infer "substitutable" and "complementary" product networks from review text, using Amazon's "also viewed" and "also bought" graphs as supervision [7]. McAuley et al. (SIGIR 2015) showed that visual style alone can predict which items are bought together, using CNN image features projected into a learned "style space" [8]. These datasets (the Amazon review corpora) became a standard benchmark. The 2023 Amazon Reviews release contains on the order of 570 million reviews over about 48 million items (*approximate*). It came with BLaIR, a family of language models pretrained to align review and metadata text with item identity for retrieval and recommendation [9].

2.3 P-Companion: diversified complementary recommendation (CIKM 2020)

P-Companion is an Amazon framework for "frequently bought together"-style complementary recommendation. It deals with three problems that raw co-purchase statistics cannot solve [10]:

1. **Noisy labels.** Co-purchase pairs mix complements with substitutes (people buy two phone cases to compare) and with unrelated items. P-Companion builds *distant supervision*: it keeps co-purchased pairs and drops pairs that are also frequently co-viewed, since co-viewed pairs are likely substitutes.
2. **Cold start.** New products have no co-purchases. Products are represented by a product embedding learned from content (catalog attributes and text) together with behaviour through a graph neural network. So a new item can get complements from its content alone.
3. **Diversity.** A camera should get a lens, a bag *and* a memory card, not five memory cards. The model first predicts a set of complementary **product types** for the query item's type, then retrieves items within each predicted type.

```
query item q
   |
   v
[product embedding (content + GNN over behaviour graph)]
   |
   +--> [complementary type transition]  type(q) -> {t1, t2, ..., tk}
   |                                       (diverse set of complement types)
   v
for each ti: [type-conditioned item embedding] --> ANN within type ti
   |
   v
merge per-type top items -> diversified "frequently bought together" list
```

The paper reports offline gains over co-purchase and embedding baselines, and online A/B improvements in production at Amazon (specific lift figures are *approximate* and should be checked in [10]). The design pattern of **type first, then item** reappears in many later complementary-recommendation systems.

2.4 Session and multi-locale shopping data

In 2023 Amazon released Amazon-M2 for the KDD Cup. It is a multilingual, multi-locale shopping-session dataset (millions of sessions across six locales, *approximate*) with tasks for next-product prediction, cross-locale transfer for low-resource locales, and next-product title generation [11]. The task design shows what Amazon cares about: session-based next-item prediction, transfer from data-rich to data-poor marketplaces, and text generation over product titles as a step toward LLM-based recommenders.

---

3. Amazon product search as a recommender

3.1 Semantic Product Search (KDD 2019)

Nigam et al. describe a neural matching model that Amazon deployed to complement lexical (inverted-index) product search [12]. It is an early, influential commerce two-tower model, and it was built with an unusually explicit eye on production constraints.

Architecture. It is a symmetric two-tower model with a *shared* token embedding table for queries and products. Each side is tokenised into word unigrams, word bigrams and character trigrams. Out-of-vocabulary tokens are **hashed** into a fixed number of bins instead of being dropped. Embeddings are combined by **average pooling**: the authors found that simple pooling beat more complex encoders (RNN/CNN variants) on their data and is much cheaper. Product text comes from the title and other catalog attributes. Relevance is the cosine similarity between query and product vectors, so retrieval works with approximate nearest-neighbour (ANN) search.

```
query text ---tok--> [uni|bi|char-3gram|hashed OOV] --embed--> avg pool --+
                                                                          |
                        shared embedding table                          cos
                                                                          |
product text -tok--> [uni|bi|char-3gram|hashed OOV] --embed--> avg pool --+
(title, attributes)
                     offline: embed catalog -> ANN index; online: embed query -> kNN
```

Loss. A three-part hinge loss uses graded relevance from behaviour. *Purchased* products should score above a high threshold. *Impressed-but-not-purchased* products should fall in a middle band. *Random* negatives should score below a low threshold. The impressed-not-purchased class is important: those items are often relevant but not chosen, and treating them as hard negatives would teach the model to push down relevant items [12].

Lessons from the paper [12]:

- Hashing OOV tokens gave large gains, because product search is full of model numbers, brands and misspellings.
- Average pooling with rich n-gram tokenisation beat heavier encoders, and is also cheaper at billion-product inference.
- Semantic retrieval is used *alongside* lexical matching, not instead of it. Lexical matching stays precise for exact attribute queries, and the semantic model adds recall for vocabulary-gap queries.

3.2 Shopping Queries Dataset (ESCI, 2022)

Amazon's Shopping Queries Dataset contains about 130 thousand difficult search queries and about 2.6 million human-labelled (query, product) judgments in English, Japanese and Spanish (*approximate*). It was released for KDD Cup 2022 [13]. Its label scheme is a useful design artefact in its own right. Each pair is **E**xact, **S**ubstitute, **C**omplement or **I**rrelevant (ESCI). This makes explicit that a search engine for shopping is also a recommender: substitutes and complements are valid, useful results for a query, but they are different from exact matches and should be ranked and presented differently.

3.3 Query-aware recommendation as a design pattern

In e-commerce, a "query-aware" recommender conditions candidates or scores on the shopper's current query (or the query that led to the current page), not only on their long-term profile. The pieces above add up to a recognisable architecture:

```
          query q, user u, context (page, cart, locale, device)
                    |
    +---------------+-----------------+----------------------+
    |                                 |                      |
lexical retrieval              semantic two-tower       item-to-item / complement
(inverted index)               (query emb -> ANN)       expansion from top results
    |                                 |                      |
    +---------------+-----------------+----------------------+
                    v
      candidate union, filters (stock, delivery, region, policy)
                    v
      ranker: f(q, u, item, context) with ESCI-style relevance + behaviour labels
                    v
      slotting: exact matches first; substitutes; complements / "related" widgets
```

The parts that are specific to commerce are: (a) **typed relevance** (ESCI) rather than binary relevance; (b) expanding from the top results through item-to-item graphs, so complements reach the page without being literally retrieved for the query; and (c) treating personalization as a re-ranking signal that must not override relevance. A personalised ranker should never promote an irrelevant item because the user likes the brand.

---

4. Amazon Personalize: Amazon's recommender as a managed service

Amazon Personalize was announced at re:Invent 2018 and became generally available in 2019. It packages Amazon-style recommendation pipelines as "recipes" (algorithm templates) and "recommenders" (use-case-specific, preconfigured endpoints) [14]. Its design shows what a general-purpose commerce recommender has to support.

4.1 Data model

- **Interactions dataset** (required): user_id, item_id, timestamp, event_type (for example click or purchase) and optional event_value and context fields.
- **Items dataset**: item metadata (categorical, numerical and, in later versions, unstructured text fields).
- **Users dataset**: user metadata.
- **Real-time events**: an event tracker with a PutEvents API streams interactions into the deployed solution, so recommendations react within a session without retraining. This is the same "react to the cart now" property that Amazon's original design had.

4.2 Recipe families

Recipe type	Representative recipes	Purpose
USER_PERSONALIZATION	aws-user-personalization, aws-user-personalization-v2; legacy aws-hrnn, aws-hrnn-metadata, aws-hrnn-coldstart	Top-N items for a user
PERSONALIZED_RANKING	aws-personalized-ranking (and v2)	Re-rank a supplied candidate list (search results, curated lists) for a user
RELATED_ITEMS	aws-similar-items, aws-sims, aws-semantic-similarity	Item-to-item ("similar to X")

Recipe type	Representative recipes	Purpose
POPULAR_ITEMS	aws-popularity-count, aws-trending-now	Popularity and trending baselines
USER_SEGMENTATION	aws-item-affinity, aws-item-attribute-affinity	Find users likely to engage with an item or attribute (marketing)
Next best action	aws-next-best-action	Recommend actions (sign up, enrol) rather than items

Sources: AWS Personalize developer guide [14]. The exact set of recipes has changed over time, and some legacy recipes are no longer offered to new customers.

Key design notes:

- **HRNN lineage.** The original user-personalization recipe was a *hierarchical RNN*. It models short-term within-session behaviour with a session-level GRU and carries a user-level state across sessions. This follows the GRU4Rec line of session-based models [15][16]. The "-metadata" and "-coldstart" variants add item and user features and the ability to recommend items with no interactions.
- **aws-sims** is classical item-to-item CF on co-occurrence, the direct descendant of Section 1. **aws-similar-items** combines co-occurrence with item metadata so that cold items get neighbours [14].
- **Exploration is a first-class parameter.** User-Personalization exposes an exploration weight and an item-age cutoff. It reserves part of the slate for new or less-exposed items, which is a bandit-style answer to catalog churn [14].
- **The v2 recipes** (released around 2024) moved to a transformer-based architecture and raised catalog limits, to the order of millions of items (*approximate*; see [14]).
- **Use-case "recommenders"** for e-commerce domain dataset groups wrap these recipes behind business-named endpoints: "Customers who viewed X also viewed", "Frequently bought together", "Best sellers", "Most viewed" and "Recommended for you" [14].
- **Filters and promotions.** Filter expressions exclude or include items by metadata or interaction history (for example out-of-stock items or already-purchased items). Promotions reserve a share of slots for a business-defined subset. Every commerce recommender eventually needs both.

4.3 Architecture of a Personalize-based deployment

```
clickstream / orders ----batch (S3 import)-----> datasets --> solution version (train)
        |                                                          |
        +--real-time PutEvents--> event tracker ------------+     |
                                                            v     v
app request (user, item, ctx) --> campaign / recommender endpoint
                                    - candidate scoring (recipe model)
                                    - filters (stock, owned, category)
                                    - promotions / exploration slots
                                  <-- ranked item ids (+ scores)
batch inference jobs --> S3 --> email, push, offline segments
```

The main lesson of Personalize as a product is that most commerce recommendation needs fit into a small set of tasks: user top-N, re-rank a list, item-to-item, popularity, and user segmentation. These are served with real-time event ingestion, filtering and exploration.

The algorithm inside each task can change (HRNN to transformer) without changing the contract with the application.

---

5. Embedding lineage for product recommendations

Before covering marketplaces, it helps to look at the representation-learning ideas that most commerce systems share.

5.1 prod2vec (Yahoo, KDD 2015)

Grbovic et al. trained word2vec-style skip-gram embeddings on sequences of purchases extracted from e-mail receipts. Products are treated as "words" and a user's purchase sequence as a "sentence" [17]. Variants included **bagged-prod2vec**, which treats each receipt (a basket) as a bag of co-purchased items, and **user2vec**, which embeds users in the same space. The system recommended products in an e-mail inbox setting. The lasting lesson is that *sequential co-purchase context* produces embeddings whose neighbours are good "next purchase" candidates, and they are cheap to train.

5.2 Graph embeddings with side information (Alibaba EGES, KDD 2018)

Alibaba's "billion-scale commodity embedding" builds an item graph from users' session behaviour sequences. It runs weighted random walks over the graph and trains skip-gram on the walks (as in DeepWalk) [18]. To deal with cold start and sparsity, it adds **side information** (category, brand, shop, price bucket). The final item embedding is a learned, attention-weighted average of the item-ID embedding and the side-information embeddings (EGES: Enhanced Graph Embedding with Side information). Items with no interactions can still be embedded from their side information. This "ID embedding + attribute embeddings, gated" pattern is now standard in commerce item towers. Alibaba's systems are covered in more depth in the China e-commerce chapter.

5.3 Zalando Fashion DNA (2016)

Zalando's Fashion DNA work learns a dense item representation for fashion articles from three inputs: curated attributes, product images and sales data [19]. The network is trained to predict customer-article purchase from these inputs. Its intermediate layer becomes the "fashion DNA" vector, which is reused for similarity, search and personalization. Because images and attributes are available on day one, new articles get useful vectors before they have any sales. Fashion is a good example of a domain where most of the catalog turns over every season, so content-derived embeddings are essential.

---

6. eBay: recommendations without a stable catalog

6.1 The problem

eBay's catalog is very different from Amazon's. A large share of listings are unique items or short-lived auctions, often listed by individual sellers with free-text titles. Pure item-to-item

CF works poorly, because most listings never build up enough co-interactions before they expire. eBay's recommendation work has therefore leaned on **content-based and embedding-based** methods that generalise across listings: title text, category, aspects (structured key-value attributes) and images.

6.2 Personalized embedding-based recommendations (2021)

Wang et al. describe eBay's deployment of a personalized embedding-based recommendation system [20]. The design is a two-tower model:

- **Item tower.** It encodes listing content (title text plus categorical features such as category and aspects), so new listings can be embedded immediately without interaction history.
- **User tower.** It encodes the user's recent sequence of interacted listings (views, purchases, watches and similar events).
- **Serving.** Item embeddings are computed offline and in near-real time as listings are created, then indexed for ANN retrieval. User embeddings are computed from the live event stream at request time, so the candidates reflect in-session intent.

```
new listing --> item tower (title tokens, category, aspects) --> item emb --> ANN index
                                                                                ^
user events (recent N) --> user tower (sequence encoder) --> user emb --kNN--+
                                                                             |
                                          candidates --> downstream ranker --> module
```

The paper reports online gains from A/B tests over the previous production recall system (exact figures *approximate*; see [20]). The lesson for designers: **when items are ephemeral, put the generalisation in the item tower** (content features) and keep the user side behaviour-driven.

6.3 Lessons from eBay's setting

- Item-to-item tables keyed on listing IDs go stale fast. Key similarity on content or on canonical product clusters instead.
- Structured aspects (brand, size, colour, condition) are powerful features and filters. In a marketplace they double as a way to normalise messy seller titles.
- Recommendation modules on the view-item page must handle the "already ended" state. If the item a user is looking at has sold, the "similar items" module becomes the main conversion path.

---

7. Etsy: unified embedding-based personalized retrieval

7.1 The problem

Etsy's marketplace has a long tail of handmade and vintage items, and most items are one-of-a-kind or come in small quantities. Search queries are often vague or aesthetic ("boho wedding decor"). Pure lexical retrieval misses many relevant items, and pure behavioural CF fails on the tail.

7.2 Unified Embedding (KDD 2023)

Jha et al. describe Etsy's "Unified Embedding" system for personalized retrieval in search [21]. It brings several representation types into one two-tower model:

- **Product tower.** It combines (a) **transformer-based** text embeddings of the listing, (b) **term-based** embeddings (bag-of-tokens, which keep the precision of lexical matching), and (c) **graph-based** embeddings learned from query-listing interaction graphs.
- **Query/user tower.** It encodes the query and **personalisation features** from the user's recent behaviour, so retrieval itself is personalised rather than only the ranking.
- **Quality-aware retrieval.** Etsy blends a product-quality signal into ANN retrieval, so the nearest-neighbour stage does not surface low-quality listings that the ranker would only demote later.
- **Serving.** Product embeddings are indexed for ANN search. The embedding retrieval path runs alongside the existing lexical retrieval, and the results are unioned before ranking.

```
query + user features --> query/user tower ----------------------------+
                                                                       | cosine
listing --> [transformer text emb] + [term emb] + [graph emb] -> fuse --+
                                        |
                      ANN index (+ quality-aware scoring) --> union with lexical
                                                          --> learning-to-rank
```

Etsy reported site-wide conversion gains from the launch. The paper cites figures of a few percent in search purchase rate and site-wide conversion (*approximate*; see [21]). The paper also discusses practical lessons: negative sampling strategies, the cost of keeping ANN indexes fresh as inventory changes, and how lexical-style token embeddings inside a neural model help recover precision on specific queries.

7.3 Takeaways from Etsy

- **Fuse the three views** (text semantics, exact terms, interaction graph). Each one covers the others' blind spots.
- **Personalise at retrieval, not just at ranking.** For vague queries, the set of candidates is where personalisation has the most effect.
- **Put quality into retrieval.** A purely semantic ANN stage will find relevant but low-quality items, which wastes ranking capacity.

---

8. Comparison of the major e-commerce designs

System	Year	Core idea	Signals	Cold-start strategy	Serving pattern
Amazon item-to-item CF [1] [4]	2003 /2017	Precomputed item similarity, online lookup	Purchases, cart, views	Content fallback; similarity on sparse data	Table lookup + merge

System	Year	Core idea	Signals	Cold-start strategy	Serving pattern
DSSTNE [6]	2016	Sparse wide neural nets, model-parallel	Purchase histories	Not addressed by engine	Batch/online scoring
Semantic Product Search [12]	2019	Shared-vocab two-tower, avg pooling, hinge loss	Purchases, impressions	Text-only product tower	ANN + lexical hybrid
P-Companion [10]	2020	Type-then-item complement retrieval	Co-purchase minus co-view, content	GNN/content product embedding	Per-type ANN
Amazon Personalize [14]	2019 -	Recipe-based managed service	Interactions, items, users, real-time events	Metadata recipes, exploration weight	Campaign/recommender endpoint
prod2vec [17]	2015	Skip-gram on purchase sequences	E-mail receipts	None (ID-based)	kNN on product vectors
Alibaba EGES [18]	2018	Random-walk graph embeddings + side info	Session sequences, attributes	Side-info embeddings	ANN candidate generation
Zalando Fashion DNA [19]	2016	Item vectors from attributes + images + sales	Purchases, images	Image/attribute-driven vectors	Embedding similarity
eBay personalized EBR [20]	2021	Content item tower, sequence user tower	Recent user events, listing text	Content item tower	ANN, real-time user emb
Etsy Unified Embedding [21]	2023	Fused text+term+graph two-tower, personalised	Queries, clicks, purchases	Text/term towers	ANN + lexical union

---

9. Other retailers: Wayfair, Shopify, Walmart, Target, Instacart

These companies have published recommendation and search work through engineering blogs and conference talks. Their specific architectures and numbers could not be checked against primary sources in this pass, so this section describes only the *design problems* that are specific to each business model and that their public work is known to address. Readers should treat it as an orientation and go to each company's engineering blog for details.

- **Wayfair (home goods).** Products are bulky, visually driven and bought rarely, so purchase-based CF signals are very sparse per user. This pushes the design toward visual similarity ("shop the look", visual search), style embeddings and session-based intent rather than long-term purchase history. The same reasoning motivates Zalando's Fashion DNA and Amazon's visual-style work [8][19].
- **Shopify (platform for millions of independent merchants).** The recommender has to serve many small stores, each with little data of its own. This favours models that are pretrained across merchants and use shared content representations (product text and images, product taxonomy), adapted per store. The structure is similar to a managed service like Amazon Personalize, with one important difference: the platform owns the cross-store representation.
- **Walmart and Target (omnichannel general retail).** Recommendations and search have to respect store-level inventory and fulfilment options (ship, pickup, delivery).

The candidate set is filtered by availability at the shopper's location, not just by global stock. Hybrid lexical-plus-semantic retrieval (as in Sections 3 and 7) is the norm for search, and complement recommendations ("goes with") matter in grocery and household categories.

- **Instacart (grocery delivery across many retailers).** Grocery is dominated by **repeat purchase** ("buy it again") and **replacement** (substitutes when an item is out of stock at the chosen store). Availability is uncertain and changes by the hour per store, so availability prediction acts as a hard filter or as a score multiplier. Basket completion (complements) and substitution (substitutes) are the ESCI distinction again, applied to fulfilment.

All five share a pattern: the **business constraint layer** (inventory, fulfilment, merchant boundaries, availability) becomes as important as the relevance model. It has to sit inside the serving path, not in an offline post-filter.

---

10. A reference architecture for e-commerce recommendation

Putting the published designs together gives a typical modern commerce recommender, as sketched here:

```
                       +-------------------------------------------+
events (view, cart,    |  streaming layer: session features,       |
purchase, search) ---->|  real-time user embedding, cart state     |
                       +-------------------------------------------+
                                    |
request(user, page, query?, cart)   v
------------------------------> candidate generation (union)
        - item-to-item tables (co-view: substitutes; co-buy: complements)
        - two-tower ANN (content-rich item tower; sequence user tower)
        - query-conditioned semantic + lexical retrieval
        - complement-type expansion (type -> items)
        - popularity / trending / exploration slots
                                    |
                                    v
             business filters: stock, delivery region, store availability,
             already-owned, policy, merchant/seller constraints
                                    |
                                    v
             ranker: multi-task (click, add-to-cart, purchase, return?)
             features: user x item x query x context; price, deal, rating
                                    |
                                    v
             re-rank: diversity (by product type), dedupe variants,
             promotions, explanations ("because you bought X")
                                    |
                                    v
             widgets: "Frequently bought together", "Customers also viewed",
             "Buy it again", "Recommended for you", search results
```

Design choices that recur across the published systems:

1. **Separate item-relation types.** Substitutes (co-view, same type) and complements (co-buy minus co-view, different type) need different data and different widgets [4][7][10][13].
2. **Correct for popularity.** Use expected-co-occurrence or lift-style normalisation, otherwise bestsellers take over every list [4].

3. **Generalise through content in the item tower.** Churn and one-of-a-kind items make ID-only models fragile [12][18][19][20][21].
4. **Make retrieval hybrid.** Lexical matching for precision, semantic matching for recall, graph or behaviour data for popularity and taste [12][21].
5. **Be reactive.** The cart and current session should change recommendations immediately, through lookups or real-time user embeddings [1][14][20].
6. **Diversify by type.** For complements, predict a diverse set of product types first [10].
7. **Explore.** Reserve slots for new items, because commerce catalogs churn [14].

---

11. Key takeaways for system designers

- **Fit the algorithm to the serving constraints.** Item-to-item CF did not win because it was the most accurate model in 2003. It won because its online cost does not depend on catalog or customer counts, it reacts immediately to the cart, and it explains itself [1].
- **Normalise co-occurrence by its expectation.** Raw co-purchase counts mostly measure popularity. Smith and Linden's expected-count correction is a cheap, high-value fix that still applies to embedding training (popularity-corrected negatives) [4].
- **Model relation types explicitly.** "Similar" and "goes with" are different products. Use co-view for substitutes and co-purchase-minus-co-view for complements, and present them in different widgets. ESCI labels make this distinction explicit in search [10][13].
- **Search is your largest recommender surface.** A query-aware system that combines lexical, semantic and personalised retrieval, and then expands through item graphs, covers more commerce intent than any home-page module [12][13][21].
- **Simple text encoders with good tokenisation go a long way.** N-grams, character trigrams and hashed OOV tokens with average pooling were competitive with heavier encoders for product search at Amazon scale [12].
- **Cold start is the default.** Marketplaces (eBay, Etsy) and seasonal catalogs (fashion) need content-driven item towers or side-information embeddings, so new items are retrievable on day one [18][19][20][21].
- **Personalise retrieval, not just ranking**, especially for vague queries [21].
- **Treat business constraints as first-class serving components.** Inventory, availability, fulfilment and merchant boundaries belong in the serving path, and services like Personalize expose filters and promotions as core APIs [14].
- **Separate the task contract from the model.** Personalize keeps stable task types (user top-N, re-rank, related items) while the model behind them moved from HRNN to transformers. Your internal platform should be built the same way [14].

References

1. Linden, G., Smith, B., York, J. "Amazon.com Recommendations: Item-to-Item Collaborative Filtering." IEEE Internet Computing, 2003. https://www.cs.umd.edu/~samir/498/Amazon-Recommendations.pdf

2. Linden, G. D., Jacobi, J. A., Benson, E. A. "Collaborative recommendations using item-to-item similarity mappings." US Patent 6,266,649, Amazon.com, 2001. https://patents.google.com/patent/US6266649B1/en
3. Sarwar, B., Karypis, G., Konstan, J., Riedl, J. "Item-based Collaborative Filtering Recommendation Algorithms." WWW 2001, University of Minnesota. https://dl.acm.org/doi/10.1145/371920.372071
4. Smith, B., Linden, G. "Two Decades of Recommender Systems at Amazon.com." IEEE Internet Computing, 2017. https://doi.org/10.1109/MIC.2017.72
5. Amazon Science. "The history of Amazon's recommendation algorithm." Amazon Science, 2019. https://www.amazon.science/the-history-of-amazons-recommendation-algorithm
6. Amazon. "DSSTNE: Deep Scalable Sparse Tensor Network Engine." GitHub (amzn/amazon-dsstne), 2016. https://github.com/amzn/amazon-dsstne
7. McAuley, J., Pandey, R., Leskovec, J. "Inferring Networks of Substitutable and Complementary Products." KDD 2015. https://arxiv.org/abs/1506.08839
8. McAuley, J., Targett, C., Shi, Q., van den Hengel, A. "Image-based Recommendations on Styles and Substitutes." SIGIR 2015. https://arxiv.org/abs/1506.04757
9. Hou, Y., Li, J., He, Z., Yan, A., Chen, X., McAuley, J. "Bridging Language and Items for Retrieval and Recommendation" (BLaIR, Amazon Reviews 2023). UCSD, 2024. https://arxiv.org/abs/2403.03952
10. Hao, J., Zhao, T., Li, J., Dong, X. L., Faloutsos, C., Sun, Y., Wang, W. "P-Companion: A Principled Framework for Diversified Complementary Product Recommendation." Amazon, CIKM 2020. https://dl.acm.org/doi/10.1145/3340531.3412732
11. Jin, W., et al. "Amazon-M2: A Multilingual Multi-locale Shopping Session Dataset for Recommendation and Text Generation." Amazon, NeurIPS Datasets and Benchmarks 2023 / KDD Cup 2023. https://arxiv.org/abs/2307.09688
12. Nigam, P., Song, Y., Mohan, V., Lakshman, V., Ding, W., Shingavi, A., Teo, C. H., Gu, H., Yin, B. "Semantic Product Search." Amazon, KDD 2019. https://arxiv.org/abs/1907.00937
13. Reddy, C. K., Marquez, L., Valero, F., Rao, N., Zaragoza, H., Bandyopadhyay, S., Biswas, A., Xing, A., Subbian, K. "Shopping Queries Dataset: A Large-Scale ESCI Benchmark for Improving Product Search." Amazon, 2022. https://arxiv.org/abs/2206.06588
14. Amazon Web Services. "Amazon Personalize Developer Guide: Choosing a recipe / predefined recipes." AWS Documentation, 2019-2025. https://docs.aws.amazon.com/personalize/latest/dg/working-with-predefined-recipes.html
15. Quadrana, M., Karatzoglou, A., Hidasi, B., Cremonesi, P. "Personalizing Session-based Recommendations with Hierarchical Recurrent Neural Networks." RecSys 2017. https://arxiv.org/abs/1706.04148
16. Hidasi, B., Karatzoglou, A., Baltrunas, L., Tikk, D. "Session-based Recommendations with Recurrent Neural Networks" (GRU4Rec). ICLR 2016. https://arxiv.org/abs/1511.06939
17. Grbovic, M., Radosavljevic, V., Djuric, N., Bhamidipati, N., Savla, J., Bhagwan, V., Sharp, D. "E-commerce in Your Inbox: Product Recommendations at Scale." Yahoo Labs, KDD 2015. https://arxiv.org/abs/1606.07154
18. Wang, J., Huang, P., Zhao, H., Zhang, Z., Zhao, B., Lee, D. L. "Billion-scale Commodity Embedding for E-commerce Recommendation in Alibaba." Alibaba, KDD 2018. https://arxiv.org/abs/1803.02349

19. Bracher, C., Heinz, S., Vollgraf, R. "Fashion DNA: Merging Content and Sales Data for Recommendation and Article Mapping." Zalando Research, 2016. https://arxiv.org/abs/1609.02489
20. Wang, T., Brovman, Y. M., Madhvanath, S. "Personalized Embedding-based e-Commerce Recommendations at eBay." eBay, 2021. https://arxiv.org/abs/2102.06156
21. Jha, R., Subramaniyam, S., Benjamin, E., Taula, T. "Unified Embedding Based Personalized Retrieval in Etsy Search." Etsy, KDD 2023 (applied track). https://arxiv.org/abs/2306.04833

25. Spotify and Audio Recommendation: Discover Weekly, BaRT, Podcasts, AI DJ, LLM Playlists, Deezer Flow, Pandora, and the Music Streaming Stack

> Editorial note: the live web tools were unavailable while this chapter was assembled (the search budget was used up and outbound fetches were blocked). The chapter therefore relies on well-known primary publications (papers with stable arXiv/DOI identifiers, official engineering posts, and open-source repositories) that were already established before drafting. Numbers are given only where those sources report them and are marked as approximate where memory of the exact figure is uncertain. Readers should check figures against the cited originals before quoting them.

Music and audio recommendation differs from video or e-commerce recommendation in ways that shape the whole architecture:

- **Consumption is fast and repeated.** A track lasts about 3 minutes, and users listen to the same items hundreds of times. Repeated consumption is the normal case, unlike in most domains.
- **Sessions are sequences.** Playlists, radio, autoplay and "Flow" style endless streams make the unit of recommendation a *sequence* with transitions, not a single item.
- **Implicit feedback dominates.** Streams, skips (often within the first 30 seconds), saves, playlist adds and "ban" actions replace ratings. Skips are a strong, plentiful negative signal that most domains lack.
- **Huge, long-tailed catalog with constant cold start.** Spotify reports a catalog of more than 100 million tracks, with tens of thousands of new uploads per day. Content-based audio models are therefore essential, not optional.
- **Multiple content types.** Music, podcasts and audiobooks share one surface (Home) but have very different consumption patterns and data density.
- **Lean-back and voice surfaces.** Radio, DJ and car/speaker contexts need recommendations that work with almost no explicit input. This drove the shift toward LLM-narrated and LLM-prompted experiences from 2023 onward.

The chapter goes from foundational Spotify components (implicit matrix factorization, content CNNs, ANN search) to the Home page (BaRT bandits, shelves), user representations and sequence models, podcasts and audiobooks (graph neural networks), the LLM era (AI DJ, AI Playlist, generative retrieval), and then other companies: Deezer, Pandora, YouTube Music, Apple Music and SoundCloud.

---

1. Spotify's foundation layer: implicit collaborative filtering and embeddings

1.1 Problem

Spotify's early personalization, from the Echo Nest era through Discover Weekly's 2015 launch, had to turn billions of play events into a "taste space" in which users, tracks, artists and playlists could be compared. Ratings did not exist, so the system had to learn from *implicit* counts.

1.2 Implicit ALS and Logistic MF

The canonical starting point is implicit-feedback matrix factorization (Hu, Koren and Volinsky) [3]. The model treats the play count r_ui as a *confidence* c_ui = 1 + alpha * r_ui on a binary preference p_ui, and solves a weighted least-squares problem with alternating least squares (ALS). ALS parallelizes cleanly on Hadoop/Spark because each user (or item) vector has a closed-form update once the other side is fixed.

Chris Johnson, then at Spotify, published **Logistic Matrix Factorization** [2]. It models the probability that a user listens to an item as a logistic function of the user and item vector dot product plus user and item biases, and weights positives by confidence as in implicit ALS. Johnson's public talks (circa 2014-2015) described Spotify running ALS-style factorization over a matrix with tens of millions of users and tens of millions of tracks on Hadoop. Those numbers have since grown by an order of magnitude.

1.3 Discover Weekly as a pipeline

Discover Weekly (launched 2015) is often described as "collaborative filtering", but public talks by Spotify engineers describe a multi-source pipeline:

```
play logs, playlist co-occurrence (user-curated playlists as "documents")
      |                          |
      v                          v
implicit MF / word2vec-style   playlist-track co-occurrence
track & user vectors           (tracks that appear together)
      |                          |
      +-----------+--------------+
                  v
      candidate tracks near the user's taste vector(s)
      (ANN search: Annoy, later Voyager)
                  |
 filters: not already heard, artist diversity, explicit-content
 settings, cold-start audio-model candidates
                  |
                  v
       ranking + sequencing into a 30-track weekly playlist
                  |
                  v
         batch generation for every user, written to storage,
         served on Monday (precomputed, not real-time)
```

Key design points:

- **Playlists as a supervision source.** Billions of user-created playlists encode human judgments of which tracks "go together". Treating a playlist as a sentence and a track as a word (skip-gram / word2vec style) gives track embeddings that capture context

and mood, not just co-listening. Spotify later released the **Million Playlist Dataset** for the RecSys Challenge 2018 on automatic playlist continuation [13], which shows how central this signal is.
- **Batch, not online.** A weekly playlist is a perfect batch workload. Generation runs offline for every user, so the per-user compute budget can be large.
- **Multiple taste clusters per user.** Users have several distinct tastes (for example workout music and jazz). Practitioner descriptions of Discover Weekly and later Spotify systems describe clustering a user's listening into several sub-tastes and recommending from each one, rather than from a single averaged vector, which would land between them.

1.4 Content-based audio models for cold start

New tracks have no listening data. In 2014, Sander Dieleman (Spotify intern, later DeepMind) described training convolutional networks on mel-spectrograms to *predict the collaborative-filtering latent factors* of a track from its audio [4]. This approach builds on van den Oord, Dieleman and Schrauwen's "Deep content-based music recommendation" (NeurIPS 2013) [5]. The trick is to use the CF embedding as the regression target, so the audio model learns to place a brand-new track where the CF model *would* have placed it. That makes cold-start tracks directly retrievable from the same ANN index.

```
audio (30s clips) -> mel-spectrogram -> conv layers (time-pooled)
                                            |
                                            v
                              predicted latent vector  ~=  CF vector (from ALS/LMF)
                                            |
          loss: MSE / weighted-MSE to the CF factors of tracks with enough plays
          use: embed tracks that have no plays -> same retrieval space
```

The Echo Nest acquisition (2014) brought content analysis (tempo, key, danceability, energy and similar attributes) and text/web mining of artist descriptions. These features were long exposed through Spotify's public Web API "audio features" endpoint. Spotify deprecated that endpoint for new third-party apps in late 2024, but the features remain internal model inputs.

1.5 Nearest-neighbor search: Annoy to Voyager

Every Spotify retrieval stage depends on approximate nearest-neighbor search over embeddings.

- **Annoy** ("Approximate Nearest Neighbors Oh Yeah", Erik Bernhardsson, open-sourced around 2013) [6] builds a forest of random-projection trees. Each split is a hyperplane equidistant between two sampled points. Indexes are static, memory-mapped files, so many processes can share one index and indexes can be built offline and shipped to servers. This fit Spotify's batch-build, read-heavy serving model.
- **Voyager** (open-sourced October 2023) [7][8] replaced Annoy for most use cases. It is based on hnswlib (HNSW graphs), with Python and Java bindings. Spotify reported it as roughly 10x faster than Annoy at the same recall, or up to about 50% more accurate at the same speed, and roughly 4x less memory through E4M3 8-bit float storage (approximate figures from the announcement [7]). It also supports adding items after the index is built, which Annoy cannot do.

Lesson: the ANN library was a *platform* decision. Because every team embedded into vector spaces and served by ANN, one library upgrade improved dozens of products at once.

---

2. Spotify Home: BaRT and explainable bandit recommendations

2.1 Problem

The Home page is a 2-D grid. Rows ("shelves", such as "Jump back in", "Made for you", "Because you listened to X") hold cards (playlists, albums, podcasts, artists). Home must choose *which shelves*, *in which order*, *which cards* and *with which explanation*, for hundreds of millions of users, while avoiding a feedback loop in which only already-popular cards get exposure.

2.2 BaRT: Bandits for Recommendations as Treatments

McInerney et al. (RecSys 2018) [1] introduced **BaRT** (Bandits for Recommendations as Treatments). It is a contextual bandit that jointly personalizes the *item* and the *explanation* attached to it (for example "Because you like jazz" or "Popular right now"). Main ideas:

- **Recommendation as treatment.** Each (item, explanation) pair is an action. The reward is a binary success signal, which on Home means the user played the card's content for at least a minimum duration (on the order of 30 seconds).
- **Model.** A factorization-machine-style reward model predicts the success probability from user, item, explanation and context features, including second-order interactions between explanation and user/item. Explanations matter more to some users than others, and the interaction terms capture that.
- **Exploration.** Epsilon-greedy over the predicted reward: with probability epsilon a random (item, explanation) is shown, and the rest of the time the greedy best. The paper also stresses *propensity logging*. Because exploration probabilities are known, the logs support **counterfactual (off-policy) evaluation and training** with inverse propensity scoring.
- **Results.** Offline counterfactual evaluation and online tests in the paper showed that personalizing explanations together with items improved streaming success over item-only personalization.

```
        user context (time, device, history embeddings)
                          |
   candidate cards  x  candidate explanations ("Because you listened to ...",
   (playlists,          "Your top mixes", "Popular podcasts", ...)
    albums, shows)
                          |
                          v
        reward model  P(stream >= ~30s | user, card, explanation, ctx)
        (factorization machine with explanation interactions)
                          |
             epsilon-greedy policy  ---> log (action, propensity, reward)
                          |                          |
                          v                          v
                rendered shelf            off-policy evaluation and
                                          retraining (IPS-weighted)
```

2.3 Home architecture evolution

Spotify's 2020 engineering post "For Your Ears Only" [9] describes how Home personalization moved from heuristic shelf ordering to ML: offline training on logged interactions, a split between card ranking within shelves and shelf ranking on the page, and a shift from batch-precomputed Home layouts toward request-time ranking so that Home can react to time of day and recent listening. The broader lessons, which echo BaRT:

1. **Log propensities from day one.** Off-policy evaluation is only possible if the serving system records how likely each shown item was to be shown.
2. **Separate the "what" from the "why".** Explanations ("Because you listened to...") are a separate decision with their own measurable effect.
3. **Satisfaction is not clicks.** Related Spotify work by Mehrotra and colleagues on Home models *user intent* (for example, quickly playing something familiar versus exploring) and its relationship to satisfaction. It also studies the trade-off between relevance, artist-side fairness/exposure and user satisfaction using counterfactual estimators [15].

2.4 Diversity and the "generalist vs specialist" finding

Anderson et al. (WWW 2020) [16] analyzed listening by roughly 100 million Spotify users. They found that users with more diverse ("generalist") listening were more likely to retain and convert to paid subscriptions. They also found that *algorithmically driven* listening was, on average, less diverse than *organic* (user-driven) listening, and that users drifted toward organic, more diverse listening over time. The design consequence is that diversity is not just a fairness constraint. It correlates with business outcomes, so it belongs in the objective or in re-ranking constraints.

---

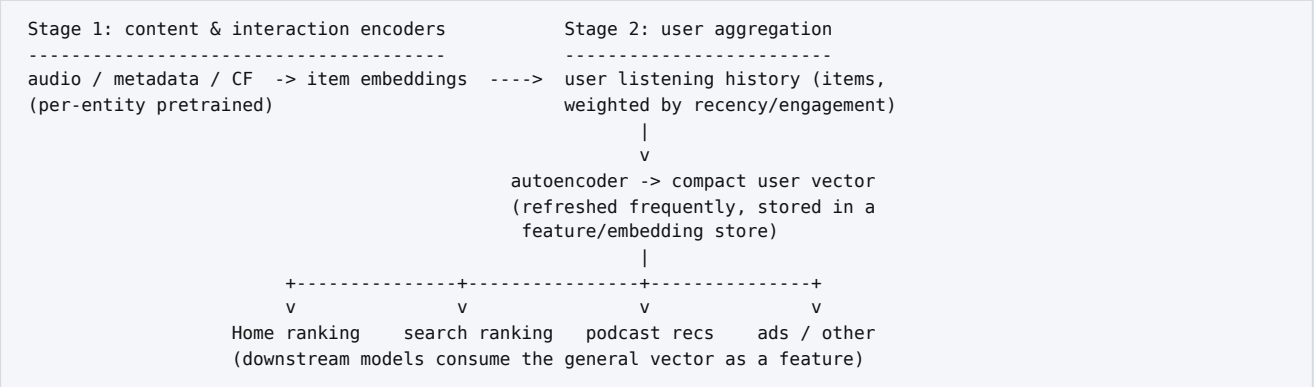
3. User representations and sequence models at Spotify

3.1 CoSeRNN: contextual and sequential user embeddings

Hansen et al. (RecSys 2020) [12] proposed **CoSeRNN**. It models a user as a *sequence of session-level preferences* rather than one static vector. For each new session, an RNN over past sessions plus the current context (time, device, and so on) produces a *session embedding* that lives in the same space as pretrained track embeddings. Candidates are then retrieved by ANN in that space. The paper reports large gains in session-level ranking metrics over static and purely sequential baselines. The design principle is "predict an embedding, then use ANN". The network outputs a point in the item space, which keeps serving cheap and reuses the existing track index.

3.2 Generalized user representations (2024)

Fazelnia et al. (Spotify, 2024) [10] describe a **two-stage user representation framework** used across many Spotify products:



Key points from the paper: one general-purpose user vector is consumed by many downstream models (transfer learning). The representation is updated frequently enough to reflect recent listening. Decoupling representation learning from task models reduced duplicated modeling effort across teams and improved downstream metrics, including cold-start performance. Infrastructure is a large part of the contribution: a shared embedding store, a scheduled refresh, and versioning so downstream consumers do not break when the representation is retrained.

3.3 Skips and sessions as data

Spotify released the **Music Streaming Sessions Dataset** (Brost et al., WWW 2019) [14]: about 160 million listening sessions with skip labels, plus acoustic features for about 3.7 million tracks, used for the WSDM Cup 2019 Sequential Skip Prediction Challenge. The dataset confirms that Spotify treats *skip prediction within a session* as a core modeling task. Skip probability is the natural short-horizon reward for radio, autoplay and DJ.

---

4. Podcasts and audiobooks: cold-start domains with graph models

4.1 Problem

When Spotify added podcasts (from around 2019) and audiobooks (2022-2023), it faced *sparse* domains. Most users had little or no podcast or audiobook history, while music data was abundant. The goal is to transfer taste from music to spoken word.

4.2 Podcast content understanding

For podcasts, content understanding is mostly *text*: transcripts, titles and episode descriptions. Spotify released the **Spotify Podcast Dataset** (Clifton et al., COLING 2020) [17]: about 100,000 episodes with automatically generated transcripts (roughly 50,000 hours of audio). It supported TREC podcast tracks on segment retrieval and summarization. Transcripts feed topic embeddings, which provide a content-based retrieval path for new shows that has the same role as the audio CNN does for music.

4.3 2T-HGNN for audiobooks (WWW 2024)

De Nadai et al. [11] describe the audiobook recommender that Spotify launched at scale. Its architecture is **2T-HGNN**: a heterogeneous graph neural network combined with a two-tower model.

```
heterogeneous graph:  audiobooks -- podcasts -- (co-listened / co-saved edges)
                      nodes carry LLM-based text embeddings of title/description
                             |
                             v
            HGNN (GraphSAGE-style message passing) -> item embeddings
                             |
user features: music + podcast + audiobook  -->  user tower
history, demographics                            |
                             +-------------------+
                             v
             two-tower (2T) model: user vec . item vec
                             |
                ANN retrieval -> downstream ranking on Home / search
```

Design decisions reported in the paper [11]:

- **The graph is item-item only, not user-item.** Including hundreds of millions of user nodes would be costly to train and serve. The HGNN therefore operates over items (audiobooks and podcasts), learning content and co-engagement structure. User personalization is handled by a separate, cheaper user tower. This decoupling is the main scalability trick.
- **Cross-domain transfer.** Podcast co-listening edges carry information into the sparse audiobook domain, and music listening features in the user tower help users with no spoken-word history.
- **Low-latency serving.** Item embeddings are precomputed offline. Online, only the user tower runs, followed by an ANN lookup.
- **Results.** Large online A/B tests (millions of users) showed significant gains in audiobook start rate (approximately +46%) and streaming rate (approximately +23%), along with gains in podcast metrics. These are approximate figures from the paper's abstract.

---

5. The LLM era: AI DJ, AI Playlist and generative retrieval

5.1 Spotify AI DJ (2023)

Launched in February 2023 as a beta in the US and Canada [18], **DJ** is a lean-back radio experience. Spotify's recommender picks and sequences tracks, and a generative layer narrates between segments in a synthetic voice modeled on Spotify's head of cultural partnerships, Xavier "X" Jernigan. Public descriptions from Spotify identify three building blocks [18]:

1. **Personalization tech** (the existing recommender stack) chooses blocks of tracks: recent favorites, older tracks the user has not played for a while, and new discoveries.
2. **Generative AI (OpenAI LLM technology)** writes short commentary scripts. Spotify's music editors supply domain knowledge and style through prompts and curated facts, so the commentary is grounded, not free-form.
3. **Dynamic AI voice** technology (from the Sonantic acquisition, 2022) turns scripts into realistic speech.

```
user history -> recommender -> "segments" (theme + 4-8 tracks)  [precomputable]
                                        |
         editor-curated facts/ ----> LLM script generation (short, templated,
         style guides                checked/filtered before use)
                                        |
                                TTS voice model (Sonantic-derived)
                                        |
              client: [voice intro][track][track]...[voice transition][track]...
              user taps DJ button -> skip whole segment -> new theme
```

Design lessons:

- **The recommender is still the core.** The LLM explains and frames, and the retrieval and ranking stack chooses. That keeps quality controllable and latency predictable.
- **Segment-level feedback.** Tapping the DJ button to switch the vibe is a coarse (segment-level) negative signal, a new kind of feedback compared to track skips.
- **Editorial grounding.** Human editors feed facts and style into the prompt layer to reduce hallucination about artists. The same pattern appears in Netflix and Amazon LLM explanations.
- Later updates (2024-2025) added voice requests to DJ ("play something for a rainy afternoon") and more languages, such as a Spanish-language DJ. Requests make DJ a conversational recommender, not just a narrated radio.

5.2 AI Playlist (2024) and prompt-based playlist generation

AI Playlist (beta from April 2024, first in the UK and Australia, later expanded [19]) lets a user type a prompt such as "indie folk for a cozy autumn evening" and receive a playlist that can be refined with follow-up prompts. Spotify has not published a full system paper on the production feature, but Spotify Research papers show the techniques under study:

- **Prompt-to-attributes plus retrieval.** An LLM parses the prompt into structured intents (genres, moods, eras, activities, and reference artists or tracks). These map to existing retrieval channels (tag/mood classifiers, embedding search, editorial playlists), and the user's taste vector personalizes the result.
- **Generative retrieval. Text2Tracks** (Palumbo et al., Spotify, 2025) [20] fine-tunes an encoder-decoder LLM to *generate track identifiers* directly from a text prompt. The key finding is that **ID design matters**. Generic or random integer IDs, or titles used as IDs, perform worse than **semantic IDs** derived from collaborative-filtering embeddings, such as hierarchical clustering of track embeddings into short code sequences. Generating a short semantic ID is also far cheaper than generating titles and then resolving them, which avoids hallucinated tracks.

```
prompt "upbeat 80s synthpop for running"
    |
    v
LLM (fine-tuned seq2seq) --decode--> semantic IDs  e.g. <c12><c7><c3>  (per track)
    |                                    |  (IDs from clustering CF embeddings)
    |                                    v
    |                        ID -> track lookup (no fuzzy title matching)
    v
personalization re-rank with user vector, dedupe, artist diversity, sequencing
```

5.3 LLM-generated narratives and explanations

Spotify Research has also published work on using LLMs to generate **personalized contextual explanations** ("narratives") for recommendations on Home, for example why a podcast or audiobook fits the user. This continues the BaRT idea that the explanation is part of the treatment. The common production pattern is to precompute LLM text offline for (item, user-segment) pairs, filter it with safety and quality classifiers, and serve it from cache, because per-request LLM generation for hundreds of millions of users is too expensive. Note: the specific paper titles and figures in this sub-area could not be re-verified while drafting this chapter, so they are not cited here.

---

6. Deezer: Flow, cold-start onboarding and carousel bandits

Deezer's research team (Paris) publishes unusually detailed production papers, which makes it one of the best public sources on music recommendation at scale. Deezer reports a catalog of more than 100 million tracks and on the order of 10 million monthly active users.

6.1 Flow

Flow is Deezer's personalized, endless radio and the app's flagship recommendation surface. It mixes favorites with discoveries in real time, and later variants added moods ("Flow Moods": focus, chill, energy, and similar). Architecturally it is a session recommender: candidate generation from user embeddings and track similarity, re-ranked continuously from in-session feedback (skips, likes) and mood constraints.

6.2 Semi-personalized cold start (KDD 2021)

Briand et al. [21] describe Deezer's system for **new users**, in production on the Flow surface:

```
existing users: embeddings u from MF on listening data
        |
        v
k-means over warm users  ->  K clusters (each with precomputed top tracks)
        |
new user registers: demographics (country, age), onboarding answers
(selected artists/genres), first-day interactions
        |
        v
feedforward network predicts the user's embedding u_hat (trained on warm users:
input = their registration-time features, target = their eventual embedding)
        |
        v
assign u_hat to nearest cluster -> serve that cluster's precomputed recs
        (semi-personalized: cheap, cacheable, robust to noisy u_hat)
```

Why it works: predicting the exact embedding of a brand-new user is noisy, so snapping to a cluster *regularizes* the prediction and makes serving a cache lookup. Online A/B tests showed significant improvements in new-user engagement and retention compared with the previous popularity- and onboarding-based approach [21].

6.3 Carousel personalization with contextual bandits (RecSys 2020)

Bendada, Salha and Bontempelli [22] model Deezer's Home **carousels** (swipeable shelves of playlists, about 12 items with only a few visible at once) as a contextual multiple-play bandit. The contributions:

- **Cascade-style position model.** Only the first L cards are seen without swiping, so unseen cards should not be counted as negatives. Their "cascade-based" update treats cards below the last clicked position (or beyond the visible window) as unobserved, not as rejected.
- **Semi-personalization again.** Users are clustered, and bandit parameters are shared within clusters. This speeds learning compared with a fully per-user bandit.
- Thompson sampling and UCB variants were compared on a released dataset of about 1,000 curated playlists, with user features, released for reproducibility.

6.4 Track Mix and new-release discovery

Deezer's **Track Mix** feature (Bendada et al., RecSys 2023) [23] generates a mix seeded by a single track using a Transformer trained on the sequences of millions of user playlists. The model predicts the next tracks given the seed, and constraints are applied for diversity and catalog availability. The paper reports online gains over the earlier similarity-based mix in streams and engagement. Deezer has also published on **fostering the discoverability of new releases** [24], which deals with the cold start of fresh releases on the Home page through content features and exploration.

---

7. Pandora: the Music Genome Project and thumbs-based radio

7.1 Music Genome Project

Pandora (founded as Savage Beast in 2000, now part of SiriusXM) built the **Music Genome Project** [25]. Trained musicologists hand-annotate each song on up to roughly 450 musical attributes ("genes"), such as melody, harmony, rhythm, instrumentation, form, vocal characteristics and lyrics. Annotating a song reportedly takes on the order of 20-30 minutes of expert time (approximate, from public descriptions).

```
seed (artist/song) -> gene vector (expert-annotated, ~hundreds of attributes)
         |
         v
candidate songs by weighted gene-space similarity
         |
+ collaborative signals from thumbs up/down across listeners (added over time)
+ per-station adaptation: thumbs on this station reweight genes/candidates
         |
         v
ensemble ranker -> next song (licensing constraints: DMCA radio rules limit
                  skips and how often an artist/album can repeat)
```

7.2 Design lessons

- **Expert content features remove the item cold start** for annotated songs, but annotation throughput limits catalog coverage. That is why Pandora's catalog has

been far smaller than Spotify's, and why Pandora later combined genome features with collaborative filtering and other machine-learned signals in an ensemble.
- **Thumbs as explicit binary feedback** scoped to a *station*. The same user can like a song on one station and dislike it on another, so feedback is contextual by design. This anticipates modern contextual and session-aware recommenders.
- **Regulatory constraints shape the ranker.** Non-interactive radio licensing restricts skips per hour and artist repetition. These rules are hard constraints in the sequencer, not soft objectives.

---

8. YouTube Music, Apple Music and SoundCloud

8.1 YouTube Music: Transformers over user actions

Google Research described (2024) how YouTube Music's ranking uses a **Transformer over the user's recent actions** to interpret *context-dependent* actions [26]. The motivating example: a skip during a workout session means something different from a skip while relaxing. A Transformer reads the sequence of (track, action type, duration) tokens. Attention learns which past actions are relevant to the current context, and the output is combined with the existing ranking model (it does not replace the whole ranker). Google reported reduced skip rates and increased listening time in live experiments. YouTube Music otherwise inherits YouTube's recommendation stack (candidate generation plus deep ranking, multi-task objectives; see Chapter 17), adapted to music-specific signals such as skips, repeats and the music-versus-video catalog split.

8.2 Apple Music

Apple has published very little system-level detail about Apple Music's recommenders. Public material describes the product (editorially curated playlists, personalized "For You"/"Listen Now" mixes, and station-based radio) but not the models, so this book does not attribute specific architectures to it. The notable *design* stance is strong human editorial curation combined with personalization, and on-device processing of some personalization signals, consistent with Apple's broader privacy approach.

8.3 SoundCloud

SoundCloud's catalog is dominated by user uploads (hundreds of millions of tracks, including remixes, DJ sets and unreleased work). Its recommenders (stations, "Discover" mixes, related tracks) must lean heavily on graph signals (follows, reposts, likes) and content features, because metadata is noisy and much of the catalog is permanently cold. SoundCloud's engineering blog has historically discussed its data and streaming infrastructure more than model details. No production model paper was verified for this chapter, so specifics are omitted.

---

9. Cross-company comparison

System	Primary surface	Core signals	Retrieval	Ranking / policy	Cold start answer
Spotify Discover Weekly	Weekly 30-track playlist	Plays, playlist co-occurrence	MF / word2vec vectors + ANN (Annoy -> Voyager)	Batch scoring, diversity filters	Audio CNN predicts CF vector [4]
Spotify Home (BaRT)	Shelves of cards	Streams >= ~30s, context	Many candidate sources	FM reward model + epsilon-greedy, IPS logging [1]	Exploration budget
Spotify CoSeRNN	Session start	Session sequences, context	Predicted session embedding + ANN	RNN over sessions [12]	Pretrained track embeddings
Spotify audiobooks (2T-HGNN)	Home, search	Podcast/audiobook co-engagement, music history	Two-tower + ANN	HGNN item embeddings [11]	LLM text embeddings, cross-domain edges
Spotify AI DJ	Lean-back radio	All of the above	Existing recsys segments	Recsys + LLM narration + TTS [18]	N/A (uses existing stack)
Spotify Text2Tracks	Prompted playlists	Text prompt, CF embeddings	Generative retrieval of semantic IDs	Personalized re-rank [20]	Semantic IDs share codes with similar tracks
Deezer Flow onboarding	Endless radio	Registration features, onboarding picks	Cluster lookup	Precomputed per-cluster lists [21]	Predict embedding -> snap to cluster
Deezer carousels	Home carousels	Clicks/streams within visible window	Curated playlists pool	Cascade contextual bandit (TS/UCB) [22]	Semi-personalized clusters
Deezer Track Mix	Seeded mix	Playlist sequences	Transformer next-track	Constrained decoding [23]	Seed-track context
Pandora	Stations	Expert genes, thumbs	Gene-space similarity + CF	Ensemble, licensing constraints [25]	Expert annotation
YouTube Music	Home, autoplay	Actions with context (skip, like, duration)	YouTube candidate generation	DNN ranker + action Transformer [26]	YouTube-wide signals

10. Recurring architectural patterns in audio recommendation

10.1 "Predict an embedding, then ANN"

CoSeRNN predicts a session vector [12], the audio CNN predicts a CF vector [4], Deezer predicts a new-user vector [21], and the 2T-HGNN user tower outputs a vector [11]. In each case the heavy model maps its input *into an existing item space*, and serving reduces to an ANN query against a shared, precomputed index. This keeps serving cost almost independent of model complexity.

10.2 Batch versus real-time split

```
BATCH (daily/weekly)                          REAL-TIME (per request / per track)
--------------------                          -----------------------------------
MF / embedding training                       session context, last N tracks, skips
audio & text content embeddings               user vector refresh (frequent)
ANN index builds (Voyager/Annoy)              shelf/card ranking on Home (BaRT-like)
Discover Weekly / Release Radar / Daylists*   radio / autoplay / DJ next-segment
LLM explanation text precompute               bandit exploration + propensity logging
```

(*Daylist, launched 2023, refreshes several times a day by time-of-day "dayparts"; it sits between batch and real time.)

10.3 Skips as the central short-horizon reward

Audio is unusual in having a fast, high-volume, *negative* signal. Spotify's skip dataset [14], YouTube Music's context-dependent skip interpretation [26] and Pandora's licensing-limited skips all treat skip behavior as first-class. Common practice uses a threshold, such as streams of at least ~30 seconds as success, which also matches royalty counting rules.

10.4 Semi-personalization

Deezer's cluster-based cold start [21] and cluster-shared bandits [22], plus Discover Weekly's per-taste-cluster generation, show that *clustering users* is a cheap, robust middle ground between global popularity and full personalization. It is cacheable, robust to noisy features, and quick to learn.

10.5 Explanations and narration as part of the decision

BaRT [1] showed in 2018 that the explanation affects engagement and should be optimized jointly with the item. AI DJ (2023) and LLM-generated narratives extend this: generative AI is used mostly for *framing*, while the recommender keeps *selection*.

10.6 Generative retrieval needs good IDs

Text2Tracks [20] shows that in LLM-based recommendation the item identifier scheme is a modeling decision. Semantic IDs built from CF embeddings beat arbitrary IDs and titles. This matches findings from TIGER-style generative retrieval in other domains (see Chapter 6).

Key takeaways for system designers

1. **Use every free supervision source.** User-built playlists are "human-labeled similarity" at billion scale. Mine them (word2vec-style, Transformers over playlist sequences) before paying for labels.
2. **Solve item cold start by regressing into the CF space.** Train content models (audio CNNs, text encoders, LLM embeddings) to predict collaborative embeddings, so new items land in the same ANN index as everything else.
3. **Make ANN a shared platform.** Annoy and then Voyager served dozens of Spotify teams. Memory-mapped, offline-built indexes suit read-heavy recommendation, and HNSW with compressed floats gives large speed and memory wins.

4. **Log propensities and treat explanations as actions.** BaRT's epsilon-greedy with propensity logging enables off-policy evaluation, and personalizing the "why" improves engagement in its own right.
5. **Model the session, not just the user.** Context (time, device, activity) changes what a skip means. Sequence models over sessions or actions (CoSeRNN, the YouTube Music action Transformer) capture this.
6. **Decouple representation from task.** A general user embedding refreshed regularly and consumed by many downstream models (Spotify 2024) reduces duplicated work and helps sparse products.
7. **For sparse new domains, build item-item graphs and transfer from the dense domain.** 2T-HGNN uses podcast and music signals to launch audiobooks, and keeps users out of the graph for scalability.
8. **Semi-personalize when signals are weak.** Clustering new users and sharing bandit parameters across clusters is cheap, robust and effective (Deezer).
9. **Optimize for diversity deliberately.** Diverse listening correlates with retention and conversion. Algorithmic feeds tend to narrow consumption unless diversity is built into the objective or the re-ranker.
10. **In the LLM era, keep the recommender in charge of selection.** Use LLMs for intent parsing, narration and explanation. If LLMs generate items, have them generate *semantic IDs* grounded in the catalog, not free-text titles.
11. **Respect hard constraints in sequencing.** Licensing (radio skip and repeat rules), explicit-content settings and artist-repetition limits belong in the sequencer as constraints, not as soft penalties.

References

1. James McInerney, Benjamin Lacker, Samantha Hansen, Karl Higley, Hugues Bouchard, Alois Gruson, Rishabh Mehrotra (Spotify). "Explore, Exploit, and Explain: Personalizing Explainable Recommendations with Bandits." ACM RecSys, 2018. https://dl.acm.org/doi/10.1145/3240323.3240354
2. Christopher C. Johnson (Spotify). "Logistic Matrix Factorization for Implicit Feedback Data." NIPS Workshop on Distributed Machine Learning and Matrix Computations, 2014. https://stanford.edu/~rezab/nips2014workshop/submits/logmat.pdf
3. Yifan Hu, Yehuda Koren, Chris Volinsky. "Collaborative Filtering for Implicit Feedback Datasets." IEEE ICDM, 2008. https://doi.org/10.1109/ICDM.2008.22
4. Sander Dieleman. "Recommending music on Spotify with deep learning." Personal blog, 2014. https://benanne.github.io/2014/08/05/spotify-cnns.html
5. Aaron van den Oord, Sander Dieleman, Benjamin Schrauwen. "Deep content-based music recommendation." NeurIPS (NIPS), 2013. https://papers.nips.cc/paper/5004-deep-content-based-music-recommendation
6. Erik Bernhardsson / Spotify. "Annoy: Approximate Nearest Neighbors in C++/Python optimized for memory usage." GitHub repository, 2013-present. https://github.com/spotify/annoy
7. Peter Sobot (Spotify). "Introducing Voyager: Spotify's New Nearest-Neighbor Search Library." Spotify Engineering, 2023. https://engineering.atspotify.com/2023/10/introducing-voyager-spotifys-new-nearest-neighbor-search-library

8. Spotify. "Voyager: an approximate nearest-neighbor search library for Python and Java." GitHub repository, 2023. https://github.com/spotify/voyager
9. Spotify Engineering. "For Your Ears Only: Personalizing Spotify Home with Machine Learning." Spotify Engineering, 2020. https://engineering.atspotify.com/2020/01/for-your-ears-only-personalizing-spotify-home-with-machine-learning
10. Ghazal Fazelnia et al. (Spotify). "Generalized User Representations for Transfer Learning." arXiv, 2024. https://arxiv.org/abs/2403.00584
11. Marco De Nadai, Francesco Fabbri, Paul Gigioli, Alice Wang, Ang Li, Fabrizio Silvestri, Laura Kim, Shawn Lin, Vladan Radosavljevic, Sandeep Ghael, David Nyhan, Hugues Bouchard, Mounia Lalmas-Roelleke, Andreas Damianou (Spotify). "Personalized Audiobook Recommendations at Spotify Through Graph Neural Networks." WWW Companion, 2024. https://arxiv.org/abs/2403.05185
12. Casper Hansen, Christian Hansen, Lucas Maystre, Rishabh Mehrotra, Brian Brost, Federico Tomasi, Mounia Lalmas (Spotify). "Contextual and Sequential User Embeddings for Large-Scale Music Recommendation." ACM RecSys, 2020. https://dl.acm.org/doi/10.1145/3383313.3412248
13. Ching-Wei Chen, Paul Lamere, Markus Schedl, Hamed Zamani. "RecSys Challenge 2018: Automatic Music Playlist Continuation." ACM RecSys, 2018. https://dl.acm.org/doi/10.1145/3240323.3240342
14. Brian Brost, Rishabh Mehrotra, Tristan Jehan (Spotify). "The Music Streaming Sessions Dataset." The Web Conference (WWW), 2019. https://arxiv.org/abs/1901.09851
15. Rishabh Mehrotra, James McInerney, Hugues Bouchard, Mounia Lalmas, Fernando Diaz (Spotify). "Towards a Fair Marketplace: Counterfactual Evaluation of the trade-off between Relevance, Fairness and Satisfaction in Recommendation Systems." ACM CIKM, 2018. https://dl.acm.org/doi/10.1145/3269206.3272027
16. Ashton Anderson, Lucas Maystre, Ian Anderson, Rishabh Mehrotra, Mounia Lalmas (Spotify). "Algorithmic Effects on the Diversity of Consumption on Spotify." The Web Conference (WWW), 2020. https://dl.acm.org/doi/10.1145/3366423.3380281
17. Ann Clifton, Sravana Reddy, Yongze Yu, Aasish Pappu, Rezvaneh Rezapour, Hamed Bonab, Maria Eskevich, Gareth Jones, Jussi Karlgren, Ben Carterette, Rosie Jones (Spotify). "100,000 Podcasts: A Spoken English Document Corpus." COLING, 2020. https://aclanthology.org/2020.coling-main.519/
18. Spotify Newsroom. "Spotify Debuts a New AI DJ, Right in Your Pocket." Spotify, 2023. https://newsroom.spotify.com/2023-02-22/spotify-debuts-a-new-ai-dj-right-in-your-pocket/
19. Spotify Newsroom (AI Playlist beta announcements and expansions), Spotify, 2024-2025. https://newsroom.spotify.com/
20. Enrico Palumbo, Gustavo Penha, Andreas Damianou, Jose Luis Redondo Garcia, Timothy Christopher Heath, Alice Wang, Hugues Bouchard, Mounia Lalmas (Spotify). "Text2Tracks: Prompt-based Music Recommendation via Generative Retrieval." arXiv, 2025. https://arxiv.org/abs/2503.24193
21. Lea Briand, Guillaume Salha-Galvan, Walid Bendada, Mathieu Morlon, Viet-Anh Tran (Deezer). "A Semi-Personalized System for User Cold Start Recommendation on Music Streaming Apps." ACM KDD, 2021. https://arxiv.org/abs/2106.03819
22. Walid Bendada, Guillaume Salha, Theo Bontempelli (Deezer). "Carousel Personalization in Music Streaming Apps with Contextual Bandits." ACM RecSys, 2020. https://arxiv.org/abs/2009.06546

23. Walid Bendada, Theo Bontempelli, Mathieu Morlon, Benjamin Chapus, Thibault Cador, Thomas Bouabca, Guillaume Salha-Galvan (Deezer). "Track Mix Generation on Music Streaming Services using Transformers." ACM RecSys (industry), 2023. https://arxiv.org/abs/2307.03045
24. Lea Briand, Theo Bontempelli, Walid Bendada, Mathieu Morlon, Francois Rigaud, Benjamin Chapus, Thomas Bouabca, Guillaume Salha-Galvan (Deezer). "Let's Get It Started: Fostering the Discoverability of New Releases on Deezer." ECIR (industry), 2024. https://arxiv.org/abs/2401.02827
25. Pandora Media. "About the Music Genome Project." Pandora, accessed 2026. https://www.pandora.com/about/mgp
26. Anushya Subbiah, Vikram Aggarwal (Google Research). "Transformers in music recommendation." Google Research Blog, 2024. https://research.google/blog/transformers-in-music-recommendation/

26. Marketplace and Travel Recommenders: Airbnb, Booking.com, Uber Eats, DoorDash and Others

Marketplace recommenders differ from media recommenders in a few ways that shape the whole design. The "item" is a physical supply unit that someone else owns: a listing, a hotel room, a restaurant, a driver. Supply is limited, tied to a place, and often tied to a date. A recommendation is only useful if it can actually be fulfilled: the listing is free for those dates, the restaurant is open and within delivery range, the host will accept. The label we most care about (a booking or an order) is rare, comes late, and can be undone by cancellations or host rejections. Every ranking change also moves demand between sellers, which creates fairness concerns and makes A/B tests interfere with each other.

This chapter covers the publicly documented designs from these companies. Airbnb has the richest public record: its KDD 2018 listing embeddings, the KDD 2019 and 2020 papers on deep learning for search, the multi-task "journey" ranker, learning to rank diversely, and embedding-based retrieval in 2025-2026. Booking.com's "150 successful models" paper is the best-known statement of what goes wrong between offline metrics and business value. Uber Eats contributes GNN features, a two-tower homefeed retriever and the Michelangelo platform. DoorDash contributes store embeddings, explore/exploit on the homepage and its feature platforms (Riviera, Fabricator, a Redis feature store). We also briefly cover Lyft, Expedia, Zillow, Yelp and Instacart, mostly to describe the patterns they share.

A note on sourcing. This chapter was written in an environment where the web-search budget was used up and outbound fetches to arXiv, Medium and company blogs were blocked by network policy. Facts are therefore taken from (a) sources that were fetched and checked while sibling chapters of this atlas were researched, and (b) well-established primary papers whose content is widely cited. References that could not be re-fetched are marked "not re-verified" in the reference list. Numbers from those sources are given as approximate and should be checked against the original before you quote them.

26.1 What makes marketplace recommenders different

Dimension	Media feed (e.g. video)	Marketplace / travel
Item	Unlimited copies of a piece of content	Unique, capacity-limited supply unit
Validity	Almost always valid	Availability, dates, delivery radius, opening hours
Primary label	Watch time, engagement (dense)	Booking / order (sparse, delayed, reversible)
Session length	Minutes	Days to weeks (travel); minutes (food)
Geo constraint	Weak	Hard (city, market, delivery zone)
Other side	Creators, loosely coupled	Hosts / merchants / drivers who can reject

Dimension	Media feed (e.g. video)	Marketplace / travel
Experiment interference	Low	High: treatment and control compete for supply
Repeat behaviour	Varies	Low for travel; very high for food and grocery

Three consequences come up again and again in the systems below:

1. **Geography partitions the problem.** Retrieval is almost always geo-filtered first, and negatives must be sampled from the same market, or the model only learns geography.
2. **The label is a funnel.** Click, then detail view, then checkout, then booking, then not cancelled. Multi-task and staged models that use the denser upper-funnel signals are standard.
3. **Offline metrics lie more often.** Booking.com and Airbnb both report many cases where offline gains did not show up as business gains. Online experiments, often with interleaving or switchbacks, are the final judge.

26.2 Airbnb search ranking

Airbnb search takes a query (location, dates, number of guests, filters) and returns a ranked list of listings that are available for those dates. The public record covers the system end to end, from the GBDT era to a multi-stage deep-learning stack.

26.2.1 Overall architecture

```
guest query (place, dates, guests, filters)
       |
       v
+-------------------------+     availability, price, geo filters
| Retrieval               |---> (inverted index; since 2025 also
|  - geo + availability   |      two-tower EBR with IVF clusters as filters [8][9])
+-------------------------+
       |  candidates (order of thousands, approximate)
       v
+-------------------------+     features: listing, query, guest, real-time
| First-pass / base ranker |     session features from embeddings [1]
|  DNN (pairwise/listwise) |     (journey ranker: multi-task base module [5])
+-------------------------+
       |
       v
+-------------------------+
| Second pass / context   |     diversity given listings above [6],
| twiddlers, business rules|     cancellation / rejection penalties [5]
+-------------------------+
       |
       v
 ranked results  ---> logs (impressions, clicks, bookings) ---> training
                       \--> interleaving experiments [10]
```

26.2.2 Real-time personalization with listing embeddings (KDD 2018)

Problem. Ranking should react to what the guest is doing in the current session. A guest who clicks on several cheap, modern apartments near the city centre should see more of those, right away, without retraining a model.

Design. Grbovic and Cheng trained word2vec-style skip-gram embeddings on search click sessions [1]. Each session is a sequence of listing IDs that a user clicked, and sessions are split when there is a gap of more than 30 minutes between clicks [1][7]. Figures reported in secondary summaries of the paper (approximate): about 800 million click sessions, about 4.5 million active listings, and 32-dimensional embeddings [1][11].

Two changes to plain skip-gram matter most:

- **Booked listing as global context.** In sessions that end in a booking, the booked listing is added as a context item for every listing in the session, whether or not it falls inside the sliding window. This pulls the embedding towards "what people who looked at this eventually booked", not only towards "what was clicked next to it".
- **Market-aware negatives.** Random negatives almost always come from another city, which makes them trivially easy. Airbnb adds negatives sampled from the same market as the central listing, so the model must learn differences within a market (style, price, neighbourhood), not just geography.

Cold start. A new listing has no clicks. Its embedding is set to the average of the embeddings of about three nearby listings of the same type and similar price, as described in the paper (approximate).

Long-term interest: type embeddings. Bookings are too sparse to learn per-user and per-listing embeddings from booking sequences directly. Airbnb therefore maps users and listings to coarse **types**, built from bucketed attributes such as market, price, room type and capacity for listings, and similar buckets for users. User-type and listing-type embeddings are learned in the same vector space from booking sequences. Host rejections are used as explicit negatives in this training, so the embedding also captures "this kind of guest tends to be rejected by this kind of listing".

How it is used.

- *Similar listings carousel.* K nearest neighbours by cosine similarity, filtered by availability. Secondary summaries report about a 21% CTR increase over the previous similar-listings algorithm [11].
- *Real-time ranking features.* Kafka streams record the user's clicked listings, skipped listings, long clicks, wishlisted listings and contacted listings over roughly the last two weeks. For each candidate, features such as "similarity to clicked listings" and "similarity to skipped listings" are computed online and fed to the GBDT search ranker that was in production at the time. The paper reports a booking gain from these features in an online test, and the embedding similarity features rank high in feature importance.

Lessons. (1) Cheap embeddings and simple online similarity features can give most of the benefit of session personalization, with no online model training. (2) Change the objective to match the business event (booking as global context). (3) Sample negatives from the same constraint set the user actually chooses within.

26.2.3 Moving to deep learning (KDD 2019)

"Applying Deep Learning to Airbnb Search" [2] is unusually candid. It reads as a log of what the team tried, what failed and why.

The trajectory (as described in the paper):

1. A GBDT model with heavy hand-made features had become hard to improve.
2. A simple neural network with one hidden layer (32 ReLU units) reached about parity with GBDT.
3. Training with a **LambdaRank-style pairwise loss** (booked listing versus not-booked listings in the same search, weighted by the NDCG change of a swap) helped.
4. An intermediate step combined GBDT and factorization-machine outputs as inputs to the neural network.
5. The final step was a **deeper network with 2 hidden layers** and about 195 features, trained on the order of 1.7 billion pairs (figures approximate, as reported in [2]). This model let the team stop much of the manual feature engineering.

Things that failed (and the reasons the paper gives):

- **Listing ID embeddings.** They overfit badly. Even a popular listing can be booked at most about 365 times a year, so there is very little data per ID compared with words in a language model.
- **Multi-task learning with long views.** Adding a "long view" (dwell) task made views go up, but bookings did not follow. The model learned to promote listings that were nice to look at, such as expensive or unusual homes, that people did not book.
- **Features that are proxies for the label.** Several offline NDCG wins did not reproduce online, which taught the team to be suspicious of offline gains.

Engineering practices that worked:

- Normalize features so they look roughly normal: log-transform heavy-tailed features such as price, and express geo coordinates relative to the centre of the map viewport rather than as raw latitude and longitude.
- Check feature distributions and plot model output as a function of single features, to catch features that the network uses in surprising ways.
- Accept that the deep model is harder to explain, and invest in debugging tools.

Lesson. Moving from GBDT to DNN was not one model change. It was a journey of many small steps, and each needed an online test. The biggest gains came from scale (more training data, fewer hand-built features) once the basics (normalization, a pairwise loss) were right.

26.2.4 Architecture, bias and cold start (KDD 2020)

"Improving Deep Learning for Airbnb Search" [3] groups its changes under three headings.

Architecture: a two-tower "ideal listing" model. The team found that the DNN had learned a strong "cheaper is better" bias. Many guests have a preference for a price that is not the lowest. The new design has a **query/user tower** that produces an "ideal listing" vector from the query and the guest, and a **listing tower** that produces a vector for each listing. The score depends on the distance between the two. This makes the model favour

listings close to what this guest wants, rather than ones that maximize a single monotone direction such as low price. The paper reports a booking gain online.

Bias: position as a feature, with dropout. Bookings depend strongly on rank position. Airbnb adds the listing's position as an input feature during training and applies dropout to that feature. At inference time the position is set to a constant (zero), so every listing is scored as if shown in the same position. The dropout rate trades off two failures. With too little dropout the model leans on position. With too much it cannot separate the effect of position from the effect of quality. The tuned version gave a statistically significant booking gain [3].

Cold start: estimate engagement for new listings. New listings have missing or noisy engagement features (for example historical booking rate), so the DNN ranks them low, which means they get fewer impressions and stay data-poor. Airbnb replaces the defaults with **estimates computed from similar nearby listings** (same geography, similar capacity and so on). The paper reports a large relative booking gain for new listings (roughly double-digit percent; around 14% as commonly quoted, approximate) and a small but significant overall booking gain [3].

```
query + guest features            listing features (+ position at train time)
        |                                    |
  [ query tower ]                    [ listing tower ]
        |                                    |
    "ideal listing"  ----- distance -----  listing vector
                             |
                   score -> pairwise loss
at serving: position feature := 0 ; new-listing engagement := neighbour estimate
```

26.2.5 Learning to rank diversely (2022, CIKM 2023)

Problem. A ranker that scores each listing on its own puts similar listings next to each other: same price range, same neighbourhood, same style. A guest whose taste differs from the average then finds nothing to book on the first page.

Design. Abdool et al. [6] change training and inference so that a listing's score depends on the listings already ranked above it. In effect, the model learns "what is the chance this listing gets booked, given what the guest has already seen above". Listings that look like the ones above are worth less. Ranking is done greedily, position by position. The paper reports an online booking gain.

Lesson. In a marketplace, diversity is not a nice-to-have. It is a way to cover the spread of guest preferences when the ranker cannot read them from the query.

26.2.6 Journey ranker: multi-task learning along the booking funnel (KDD 2023)

Problem. A booking is the end of a long journey: search, click, view the listing page, go to the payment page, request or instantly book, the host accepts, and the trip is not cancelled. Training only on bookings throws away the denser signals before it. Training on clicks alone optimizes the wrong thing.

Design (Tan et al., "Optimizing Airbnb Search Journey with Multi-task Learning" [5]). The Journey Ranker is built from modules:

- **Base module.** A multi-task network that predicts each step of the positive funnel (click, listing page, payment page, request, booking). The final score combines the conditional probabilities along the funnel, so dense early steps help the model learn while the target is still the booking.
- **Twiddler module.** Predicts negative milestones such as cancellations and host rejections, and uses them to adjust the score. This keeps the ranker from sending guests to listings that tend to fall through.
- **Context module.** Lets the combination weights depend on context, for example how far ahead the trip is or what kind of guest is searching.
- **Monotonic module.** Enforces monotonic relationships for some features (for example, that a better value on a quality signal should never lower the score).

The paper reports online booking gains and says the design was reused for other Airbnb search surfaces (approximate; see [5]).

Lesson. Model the funnel explicitly, including the negative outcomes that only the marketplace sees (rejections, cancellations). A booking that is later cancelled is not a success.

26.2.7 Embedding-based retrieval (2025-2026)

Airbnb's most recent public work adds a learned retrieval stage [8][9].

- **Model.** Two-tower. The listing tower is run offline in a daily batch. The query tower runs online for each search.
- **Training data.** Built from guest search journeys, with **session-based hard negatives** (listings the guest saw in the session but did not book) instead of random negatives.
- **ANN choice.** HNSW gave slightly better recall in evaluation, but Airbnb chose **IVF**. Prices and availability change all the time, and HNSW's memory overhead on frequent updates was a problem. With IVF, a listing's cluster assignment becomes an ordinary filter in the existing search system, so retrieval is "match the query to clusters, then filter by cluster plus the usual availability filters" [8].
- **Result.** The paper reports significant booking gains [9].

Lesson. When items change often and retrieval must respect hard filters, an index that fits into the existing filter engine (IVF clusters) can beat a slightly more accurate graph index (HNSW).

26.2.8 Experimentation for search: interleaving

Airbnb adapted team-draft interleaving to a two-sided marketplace with sparse, delayed bookings [10]. Only "competitive pairs", the positions where the two rankers actually differ, are used for attribution, and searches where both rankers agree are dropped. Airbnb reports roughly a 50x gain in sensitivity compared with A/B tests, and interleaving results agreed in direction with later A/B tests in most paired comparisons (approximate) [10]. Interleaving is used to screen ranking ideas quickly; A/B tests still make the final call.

26.2.9 Data platform: Zipline to Chronon

Airbnb's feature platform Zipline was rebuilt and open-sourced as **Chronon**, which is maintained jointly with Stripe and is described as powering all major ML applications at Airbnb [12]. It lets engineers define features in Python (`GroupBy`, `Join`) once. The same definition is used for point-in-time-correct backfills in Spark and for online serving with streaming updates. This removes the main source of train-serve skew for real-time features such as the session embedding similarities in 26.2.2. Chapter 8 covers Chronon in detail.

26.3 Booking.com: 150 models and 6 lessons

Bernardi, Mavridis and Estevez's KDD 2019 paper [4] is not about one model. It summarizes what Booking.com learned from about 150 customer-facing ML models in production. It is one of the most quoted papers on applied recommender systems.

26.3.1 Model families

The paper groups the models into families (paraphrased):

Family	What it predicts	Example use
Traveller preferences	How flexible the user is about dates or destination	Show alternative dates or places
Traveller context	Trip type (family, business, couple)	Adapt filters and messaging
Item space navigation	Which items or destinations the user will engage with	Recommendations, sorting
User interface optimization	Best layout, font size, number of items	UI parameters
Content curation	Which reviews, photos or descriptions to show	Review highlights
Content augmentation	Derived facts about properties	"Great value today", price trend

Most of these are not "rank the hotels" models. They are small predictions that each shape part of the experience, and their combined effect is the product.

26.3.2 The six lessons

1. **ML projects deliver value.** Across families, models gave measurable business gains in randomized experiments. Even "semantic" models of hidden concepts (such as how flexible a user is) worked as building blocks for many features.
2. **Model performance is not business performance.** Better offline metrics often gave no business gain. Reasons the paper gives: the value is already saturated, the model optimizes a proxy that diverges from conversion, and an "uncanny valley" effect where a prediction that is too accurate feels intrusive to users.
3. **Be on the lookout for the problem to solve.** Framing the problem well (for example as regression versus classification, or picking the right label) mattered more than the learner.

4. **Prediction serving latency matters.** In a controlled experiment, adding about 30% to latency cost roughly 0.5% in conversion (approximate, as reported). Tricks the paper lists: model redundancy (many replicas near the caller), in-house linear and Naive Bayes prediction, sparse models, precomputing and caching, bulking requests, and keeping feature transformation small.
5. **Get early feedback on model quality.** Labels arrive late or never. Booking.com uses **Response Distribution Analysis**: look at the shape of the model's output distribution on live traffic. A healthy binary classifier shows a clear split; a flat or one-peaked histogram is an early warning.
6. **Test business impact with randomized controlled trials.** Everything goes through A/B tests. The paper describes designs for cases where the model only applies to part of the traffic, for example triggering analysis that compares only users who would have been affected, and a three-arm design to separate the effect of the model from the effect of showing any change at all.

26.3.3 Lessons for system designers

- Treat offline metrics as a health check, not as the goal.
- Latency is a business metric: budget it like conversion.
- Build monitoring that works without labels.

26.4 Expedia and the hotel-ranking problem

Expedia's best-known public contribution is a benchmark rather than a production system write-up. The 2013 Kaggle competition "Personalize Expedia Hotel Searches" released search logs with hotel features, user history, price competitiveness, position and click/booking labels [13]. Its structure is still a good model of the hotel-ranking problem:

- Each search returns a list of hotels. The labels are graded (booked > clicked > shown), and the metric is NDCG@38.
- The data contains both "random" and normally ranked result pages, which allowed position bias to be studied.
- Winning solutions were mostly gradient-boosted LambdaMART models with heavy feature engineering: price normalized within the search, hotel historical CTR and booking rate, and the difference between the user's past average price or star rating and the hotel's.

The patterns it shows (graded labels, normalizing features within the query, comparing the item to the user's history) also appear in Airbnb's GBDT era and in Booking.com's work. Expedia Group has published further work on search ranking and travel recommendations on its engineering blog, but those posts could not be verified for this chapter.

26.5 Uber Eats

Uber Eats is a three-sided marketplace (eaters, merchants, couriers) with very strong locality: an eater can only order from restaurants in delivery range. Roughly 95% of users

start their journey on the Homefeed, according to an Uber talk [16], so homefeed ranking is central.

26.5.1 Graph learning for dish and restaurant recommendations (2019)

Problem. Orders are sparse and local. Each eater orders from a small set of nearby restaurants, so standard collaborative filtering has little to work with [14].

Design [14]:

- **Graphs.** Bipartite user-dish and user-restaurant graphs, built **per city**. Delivery range makes edges across cities almost meaningless. Edge weights encode how often the user ordered.
- **Model.** GraphSAGE with sampled neighbourhoods and node features (dish and restaurant descriptors).
- **Loss.** A max-margin ranking loss, extended with "low-rank positives". A dish ordered many times should score above a dish ordered once, which should score above a negative. The edge weights are therefore used in the loss as well as in sampling.
- **Pipeline.** Embeddings are trained and generated offline every day on Uber's ML platform. Similarity between user and dish or restaurant embeddings is fed as **features** into the downstream ranking model and also used for candidate retrieval.

Results (approximate). The blog reports a clear offline AUC gain for the ranker when the graph features were added, with the graph similarity among the most important features, and CTR gains in online tests in the first cities [14].

```
orders (per city) -> weighted bipartite graphs (user-dish, user-restaurant)
     -> GraphSAGE (sampled 2-hop, max-margin + low-rank positives), daily
     -> user / dish / restaurant embeddings
     -> similarity features ---> homefeed ranker ---> carousels and store list
```

Lesson. Splitting the graph by city both scales the training and encodes a real constraint. A GNN does not have to be the recommender: its embeddings used as features in an existing ranker are often the safest way to ship.

26.5.2 Two-tower embeddings for homefeed retrieval

Problem. The earlier first-pass ranker was a Deep Matrix Factorization model trained **per city**. As Uber expanded, this meant thousands of weekly Spark jobs and did not scale [15].

Design. A single global two-tower model [15]:

- The **store tower** runs offline. Store embeddings are indexed in Uber's search platform (SIA) for approximate nearest neighbour search.
- The **eater tower** runs in real time for every Homefeed request, followed by an ANN lookup against the store index, with location filters.
- Cost drops from O(q x M) online inference (q requests, M stores) to O(M) offline plus O(q) online.

A Feature Store Summit talk gives the latency target: find the best stores out of thousands in about **50 ms** [16].

26.5.3 Eats search pipeline: hydration is the hidden cost

A 2026 Uber post on the Eats search pipeline [17] reports that retrieval was returning tens of thousands of candidates, all of which were fully hydrated (all their data loaded) before the ranker discarded most of them. Removing low-yield retrieval paths saved about 120 ms. Splitting hydration into a light phase for ranking and a full phase for the results that are actually shown saved more than 100 ms more. The first-pass ranker has to stay fast "while maintaining a high recall metric" [17].

```
eater request
   |
   v
retrieval paths (lexical, embedding/ANN, geo)  --> tens of thousands of candidates
   |                                                   (before the fix)
   v
light hydration (ranking features only)
   |
   v
first-pass ranker (high recall, low latency)
   |
   v
second-pass ranker (multi-objective)
   |
   v
full hydration (presentation data) only for shown results
```

26.5.4 Michelangelo

Michelangelo, Uber's ML platform [18], is the infrastructure under these models. Its 2017 description covers the full lifecycle: manage data, train, evaluate, deploy, predict and monitor. Two ideas from it are now everywhere:

- A **feature store** with an offline store (Hive) for training and an online store (Cassandra) for serving, fed by batch (Spark) and streaming (Kafka plus Samza, later Flink) pipelines. Features are shared across teams. The 2017 post mentions about 10,000 features in the store (approximate).
- **Transformations inside the model artifact** (a DSL for selecting and transforming features), so the same code runs at training and at serving time.

Michelangelo **Palette** (2019) [19] extended the feature store into a feature engineering platform with a catalog, ownership, and support for batch, near-real-time and RPC feature sources.

26.5.5 Experimentation under marketplace interference

Uber's experimentation platform supports A/B tests, multi-armed bandits, **switchback** experiments and causal inference methods [20]. Switchbacks are the key tool for marketplaces: instead of splitting users, the whole market (a city for a time window) is switched between treatment and control, so treatment and control do not compete for the same couriers or restaurants. The platform runs on the order of 1,000 or more concurrent experiments (approximate) and uses delta-method statistics, sequential tests and CUPED-style variance reduction [20].

26.6 DoorDash

DoorDash's public writing focuses on the homepage store feed, explore/exploit, and the feature and prediction infrastructure behind them.

26.6.1 Store embeddings for the personalized store feed (2018)

DoorDash's early store-feed personalization trained **store2vec**: word2vec-style embeddings where each "sentence" is the sequence of stores a user viewed in a session [21]. A user vector is built by averaging the vectors of stores the user ordered from. The cosine similarity between the user vector and a candidate store becomes a feature in the store ranking model. This is the same pattern as Airbnb's listing embeddings (26.2.2): cheap sequence embeddings, used as features, not as the ranker itself. The post reports gains in click-through and conversion in experiments (details approximate) [21].

26.6.2 Homepage ranking with exploitation and exploration (2022)

Problem. A pure exploitation ranker shows the same familiar stores and cuisines. New or rarely shown merchants never collect enough data to rank well, and users who would like something different never see it.

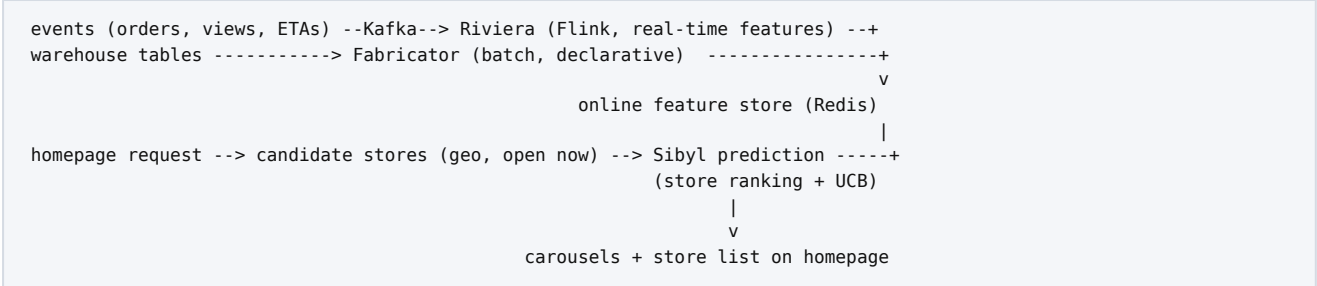
Design [22], as described:

- A personalized ranking model predicts how likely the user is to engage with each store or cuisine (exploitation).
- An **upper-confidence-bound (UCB) style bonus** is added for items with few impressions for this user, so uncertain items get some exposure (exploration).
- The two terms are combined into the final score, with a weight that sets how much to explore.

The post reports engagement gains in A/B tests and better discovery of new merchants (approximate).

26.6.3 Feature and prediction infrastructure

- **Sibyl** is DoorDash's prediction service. It serves many models with batched feature fetches and is built to keep latency predictable [23].
- **Gigascale feature store on Redis (2020).** DoorDash benchmarked Redis against other stores and used binary serialization and compression to store billions of features cheaply, with large savings in memory and CPU (figures approximate) [24].
- **Riviera (2021)** is a declarative framework for **real-time** features on Flink: features are defined in configuration and SQL, and the framework builds the streaming job [25].
- **Fabricator (2022)** is a declarative framework for **batch** feature engineering. A feature definition (sources, SQL, schedule, sinks) generates the pipelines, the backfills and the upload to the online store, which shortens the time from a feature idea to production [26].



26.7 Lyft

Lyft's public ML work is mostly about dispatch, pricing and ETA, not recommendation lists, but its feature serving design is a typical marketplace one. Lyft describes a feature serving system that serves both batch-computed and streaming features with low latency, and that keeps training and serving consistent by logging served values [27]. For marketplace recommenders the lesson is that ranking, pricing and dispatch draw on the same real-time supply and demand features, so a shared feature platform pays off across all of them.

26.8 Zillow, Yelp and Instacart: patterns only

These three companies have published recommender and search work on their engineering blogs. The specific posts could not be fetched for this chapter, so this section covers only the design problems their domains create, without company-specific numbers.

- **Real estate (Zillow).** Each home is unique and sells once. Interaction data per item is very thin, so content features (location, price, size, photos) and user preference profiles (saved searches, favourites) carry most of the weight. Items leave the catalogue quickly when they sell, which favours models that generalize from attributes over models that memorize IDs, and makes freshness of the index important.
- **Local business discovery (Yelp).** Ranking blends text relevance, distance, quality (ratings, review volume) and personal taste, and must avoid letting advertising reduce user trust. Review text is a rich content signal for both search and recommendations.
- **Grocery and retail media (Instacart).** Grocery is dominated by **repeat purchase** ("buy it again") and **replacements** when an item is out of stock. Availability changes by the hour per store, so predicted availability acts as a hard filter or as a score multiplier. Sponsored products are ranked inside organic results, which needs an auction that combines bids with predicted relevance and protects user experience. Chapter 16 covers ads ranking in general.

26.9 Cross-company comparison

System	Stage	Model	Key trick	Reported result	Ref
Airbnb listing embeddings	Features, similar	Skip-gram, d=32	Booked listing as global context; in-market	~21% CTR, similar listings (approx.)	[1] [11]

System	Stage	Model	Key trick	Reported result	Ref
(2018)	listings		negatives		
Airbnb DL search (2019)	Ranking	2-layer DNN, pairwise loss	Feature normalization; drop ID embeddings	Bookings gain vs GBDT	[2]
Airbnb ABA (2020)	Ranking	Two-tower "ideal listing"	Position feature + dropout; new-listing priors	Bookings gain; new listings ~+14% (approx.)	[3]
Airbnb diversity (2022)	Re-ranking	Context-aware DNN	Score depends on listings above	Bookings gain	[6]
Airbnb Journey Ranker (2023)	Ranking	Multi-task modules	Funnel milestones + cancel/reject twiddler	Bookings gain	[5]
Airbnb EBR (2025-26)	Retrieval	Two-tower	Session hard negatives; IVF clusters as filters	Bookings gain	[8] [9]
Booking.com (2019)	Whole product	~150 models	Response distribution analysis; latency budget	~30% latency -> ~0.5% conversion (approx.)	[4]
Uber Eats GNN (2019)	Features, retrieval	GraphSAGE per city	Weighted max-margin with low-rank positives	Offline AUC up; CTR up	[14]
Uber Eats TTE	Retrieval	Global two-tower	Store tower offline; SIA ANN	~50 ms over thousands of stores	[15] [16]
Uber Eats search	Pipeline	Multi-stage	Two-phase hydration	~120 ms + 100+ ms saved	[17]
DoorDash store2vec (2018)	Features	Skip-gram on sessions	User vector = mean of ordered stores	CTR/conversion up (approx.)	[21]
DoorDash homepage (2022)	Ranking	Ranker + UCB	Exploration bonus for low-impression items	Engagement and discovery up (approx.)	[22]

Platform view:

Company	Feature platform	Serving	Experiments
Airbnb	Zipline -> Chronon (OSS, with Stripe) [12]	Search backend with filters + EBR	ERF, interleaving [10]
Uber	Michelangelo, Palette [18][19]	Michelangelo online prediction, SIA ANN	XP: A/B, switchbacks, bandits [20]
DoorDash	Riviera, Fabricator, Redis store [24][25][26]	Sibyl [23]	A/B and switchbacks (general)
Booking.com	In-house; latency-first serving [4]	Replicated, precomputed models	RCTs with triggering, 3-arm designs [4]
Lyft	Feature serving with logged values [27]	LyftLearn (general)	Switchbacks (general)

26.10 Recurring design patterns

Sequence embeddings as features first

Airbnb (listing2vec), DoorDash (store2vec) and Uber Eats (GraphSAGE) all started by learning item embeddings from sessions or order graphs and feeding **similarities as features** into an existing ranker. This is low-risk: the ranker keeps its other signals, and the embedding can be refreshed daily without touching serving code.

Constrained negative sampling

Airbnb's in-market negatives, its session hard negatives in EBR, and Uber's per-city graphs are the same idea: negatives must come from the set the user really chose from. Otherwise the model learns geography or availability, which filters already handle.

Funnel-aware and outcome-aware labels

The booking is sparse, so the systems borrow signal from the funnel above it (clicks, detail views, checkout) and correct for outcomes below it (cancellations, host rejections). Airbnb's Journey Ranker is the clearest example [5]. Airbnb's 2019 failure with long-view multi-task learning [2] is the warning: an upper-funnel task must be tied to the final outcome, or the model will learn to show attractive things that do not convert.

Cold start of supply is a business problem

New hosts and new merchants need impressions to survive. Airbnb estimates engagement from similar nearby listings and removes position from scoring [3]. DoorDash adds UCB exploration [22]. Both protect the supply side of the marketplace as well as improving ranking quality.

Filters before learning

Availability, dates, delivery radius and opening hours are hard constraints. The successful designs keep them as filters in the retrieval engine, and choose ANN structures (Airbnb's IVF) and partitions (Uber's per-city graphs, geo-filtered ANN) that work with those filters.

Measure carefully, because the market is shared

Interleaving (Airbnb [10]), switchbacks (Uber [20]) and triggering and three-arm designs (Booking.com [4]) all address the same issue: marketplace experiments are noisy and biased by interference, and offline metrics are not reliable enough to decide alone.

26.11 Key takeaways for system designers

1. **Model the transaction, not the click.** Use booking or order as the target, the denser funnel steps as auxiliary tasks, and cancellations and rejections as negatives. Check that every auxiliary task actually moves the final outcome online.
2. **Geography and availability come first.** Partition by market or city, keep hard constraints as filters, and sample negatives inside the same constraints.
3. **Start with embeddings as features.** Session or graph embeddings fed into an existing GBDT or DNN ranker are cheap, fast to ship and easy to debug.
4. **Do not memorize IDs when data per item is thin.** Airbnb's listing ID embeddings overfit because each listing can be booked at most a few hundred times a year. Prefer attributes, types and content.
5. **Debias position and help new supply.** Position as a feature with dropout and constant-at-serving is a simple, effective trick. Estimate missing engagement for new items from similar neighbours, and keep an explicit exploration budget.

6. **Diversify the page.** When the query cannot tell you the guest's taste, cover the space. Make each listing's score depend on what is already above it.
7. **Pick the ANN index that fits the update pattern and filter engine,** not only the one with the best recall. Fast-changing prices and availability favoured IVF at Airbnb.
8. **Hydrate late.** Load only ranking features for the large candidate set and fetch presentation data only for the items you will show. Uber saved over 200 ms this way.
9. **Treat latency as a business metric.** Booking.com measured that added latency costs conversion. Precompute, cache, replicate and keep models small where possible.
10. **Trust online experiments, and design them for marketplaces.** Use interleaving to screen, switchbacks or market-level randomization for interference, label-free monitoring (response distribution analysis) for early warnings, and A/B tests for the final decision.
11. **Invest in one feature platform.** Chronon, Michelangelo/Palette and Riviera/Fabricator exist because real-time marketplace features (session clicks, open stores, courier supply) are where train-serve skew is most likely.

References

1. Grbovic, M., Cheng, H. "Real-time Personalization using Embeddings for Search Ranking at Airbnb." KDD 2018 (Applied Data Science best paper), Airbnb, 2018. https://dl.acm.org/doi/10.1145/3219819.3219885
2. Haldar, M., Abdool, M., Ramanathan, P., et al. "Applying Deep Learning to Airbnb Search." KDD 2019, Airbnb, 2018/2019. https://arxiv.org/abs/1810.09591
3. Haldar, M., Ramanathan, P., Sax, T., et al. "Improving Deep Learning for Airbnb Search." KDD 2020, Airbnb, 2020. https://arxiv.org/abs/2002.05515
4. Bernardi, L., Mavridis, T., Estevez, P. "150 Successful Machine Learning Models: 6 Lessons Learned at Booking.com." KDD 2019, Booking.com, 2019. https://dl.acm.org/doi/10.1145/3292500.3330744
5. Tan, C. H., et al. "Optimizing Airbnb Search Journey with Multi-task Learning." KDD 2023, Airbnb, 2023 (not re-verified). https://arxiv.org/abs/2305.18431
6. Abdool, M., Haldar, M., et al. "Learning To Rank Diversely At Airbnb." Airbnb, 2022 (CIKM 2023). https://arxiv.org/abs/2210.07774
7. Airbnb Engineering. "Listing Embeddings in Search Ranking." Airbnb Tech Blog (Medium), 2018 (not re-verified). https://medium.com/airbnb-engineering/listing-embeddings-for-similar-listing-recommendations-and-real-time-personalization-in-search-601172f7603e
8. Airbnb Engineering. "Embedding-Based Retrieval for Airbnb Search." Airbnb Tech Blog, 2025. https://airbnb.tech/ai-ml/embedding-based-retrieval-for-airbnb-search/
9. Airbnb. "Applying Embedding-Based Retrieval to Airbnb Search." arXiv, 2026. https://arxiv.org/pdf/2601.06873
10. Airbnb Engineering. "Beyond A/B Test: Speeding up Airbnb Search Ranking Experimentation through Interleaving." Airbnb Tech Blog, 2022. https://medium.com/airbnb-engineering/beyond-a-b-test-speeding-up-airbnb-search-ranking-experimentation-through-interleaving-7087afa09c8e
11. BestPractice.AI. Case study of "Real-time Personalization using Embeddings for Search Ranking at Airbnb" (secondary summary), n.d. https://bestpractice.ai/ai-use-ca

ses/case-studies/consumer-retail/airbnb-increased-similar-property-listing-recommendations-click-through-rates-by-21-with-machine-learning-embeddings
12. Airbnb and Stripe. "Chronon" (repository and documentation), 2024. https://github.com/airbnb/chronon
13. Expedia and Kaggle. "Personalize Expedia Hotel Searches - ICDM 2013" (competition and dataset), 2013 (not re-verified). https://www.kaggle.com/c/expedia-personalized-sort
14. Jain, A., Liu, I., Sarda, A., Molino, P. "Food Discovery with Uber Eats: Using Graph Learning to Power Recommendations." Uber Engineering Blog, 2019. https://www.uber.com/blog/uber-eats-graph-learning/
15. Uber Engineering. "Innovative Recommendation Applications Using Two Tower Embeddings at Uber." Uber Engineering Blog, 2023 (approx.). https://eng.uber.com/blog/innovative-recommendation-applications-using-two-tower-embeddings/
16. Kurra, D. D. "Uber - Embeddings." Feature Store Summit 2024 talk, Uber, 2024. https://content.hopsworks.ai/hubfs/Feature%20Store%20Summit%202024/FS%20Summit%2024%20-%20Uber%20-%20Embeddings.pdf
17. Uber Engineering. "Uber Eats Search Pipeline." Uber Blog, 2026. https://www.uber.com/us/en/blog/uber-eats-search-pipeline/
18. Hermann, J., Del Balso, M. "Meet Michelangelo: Uber's Machine Learning Platform." Uber Engineering Blog, 2017 (not re-verified). https://eng.uber.com/michelangelo-machine-learning-platform/
19. Uber. "Michelangelo Palette: A Feature Engineering Platform at Uber." InfoQ presentation, 2019 (not re-verified). https://www.infoq.com/presentations/michelangelo-palette-uber/
20. Uber Engineering. "Under the Hood of Uber's Experimentation Platform." Uber Engineering Blog, 2018. https://www.uber.com/blog/xp/
21. DoorDash Engineering. "Personalized Store Feed with Vector Embeddings." DoorDash Engineering Blog, 2018 (not re-verified). https://doordash.engineering/2018/04/02/personalized-store-feed-with-vector-embeddings/
22. DoorDash Engineering. "Homepage Recommendation with Exploitation and Exploration." DoorDash Engineering Blog, 2022 (not re-verified). https://doordash.engineering/2022/10/05/homepage-recommendation-with-exploitation-and-exploration/
23. DoorDash Engineering. "Meet Sibyl - DoorDash's New Prediction Service." DoorDash Engineering Blog, 2020 (not re-verified). https://doordash.engineering/2020/06/29/doordashs-new-prediction-service/
24. DoorDash Engineering. "Building a Gigascale ML Feature Store with Redis, Binary Serialization, String Hashing, and Compression." DoorDash Engineering Blog, 2020 (not re-verified). https://doordash.engineering/2020/11/19/building-a-gigascale-ml-feature-store-with-redis/
25. DoorDash Engineering. "Building Riviera: A Declarative Real-Time Feature Engineering Framework." DoorDash Engineering Blog, 2021 (not re-verified). https://doordash.engineering/2021/03/04/building-a-declarative-real-time-feature-engineering-framework/
26. DoorDash Engineering. "Introducing Fabricator: A Declarative Feature Engineering Framework." DoorDash Engineering Blog, 2022 (not re-verified). https://doordash.engineering/2022/01/11/introducing-fabricator-a-declarative-feature-engineering-framework/

27. Lyft Engineering. "ML Feature Serving Infrastructure at Lyft." Lyft Engineering Blog, 2021 (not re-verified). https://eng.lyft.com/ml-feature-serving-infrastructure-at-lyft-d30bf2d3c32a

27. The Chinese Ecosystem: Alibaba, Tencent, Meituan, Baidu, Huawei, JD, Xiaohongshu and the Architecture of Hyperscale Commerce and Content

Overview

A large share of the industrial recommender-system literature since 2017 comes from Chinese companies. Several reasons explain this. Alibaba, Tencent, Meituan, Kuaishou, ByteDance, Baidu, JD.com and Huawei each run several surfaces with hundreds of millions of daily users. Their advertising teams (Alimama at Alibaba, Tencent Ads, Baidu Phoenix Nest) publish heavily at KDD, SIGIR, CIKM, RecSys and WWW. Many of them also open-source their training stacks: XDL, DeepRec, EasyRec, Graph-Learn, Euler, Angel, embedx and PaddleRec.

This chapter maps that body of work by company. It focuses on the designs that became industry templates:

- **Alibaba / Taobao / Alimama.** Target attention (DIN), interest evolution (DIEN), lifelong sequences (MIMN/UIC, SIM, ETA), entire-space multi-task learning (ESMM), graph item embeddings (EGES), learned tree indexes for retrieval (TDM, JTM), multi-interest retrieval (MIND), cost-aware pre-ranking (COLD), listwise re-ranking (PRM), and the XDL, DeepRec and Graph-Learn platforms.
- **Tencent.** Multi-task learning with PLE, STEM and embedding-collapse research, WeChat look-alike retrieval (RALM), the embedx and Angel platforms, and the generative ads model GPR for WeChat Channels.
- **Meituan.** Long-sequence sampling (SDIM), multi-step conversion modeling (AITM), permutation re-ranking (PIER), and generative ranking (MTGR, EGA).
- **Huawei Noah's Ark.** Feature-interaction models (DeepFM, PNN family, FGCNN, EDCN, FinalMLP), position-bias modeling (PAL), re-ranking (PEAR), and the FuxiCTR/BARS benchmarks.
- **Baidu, JD.com, Xiaohongshu, Sina Weibo** and a note on **Pinduoduo**.

Kuaishou and ByteDance are covered in Chapter 20 and appear here only for comparison.

A note on sourcing. The live web-search budget for this run was used up before research began, and outbound access to arXiv, ACM and most company blogs failed from the writing environment. Facts here come from three kinds of source. The first is primary documents that could be fetched directly: the Alibaba XDL wiki pages for DIN, DIEN, ESMM, TDM, TDM Serving, CrossMedia and Rocket Training; the READMEs of XDL, Blaze, DeepRec, EasyRec, HybridBackend, Euler, Tencent embedx and Angel, PaddleRec, FuxiCTR and DeepCTR; and the DIN and DIEN reference-code READMEs. The second is paper results recorded in other chapters of this Atlas, which were researched from the papers

themselves. The third, used rarely and always flagged, is papers that could not be re-fetched. Treat any number marked approximate as approximate.

```
          THE CHINESE "FULL-LINK" RECOMMENDATION / ADS FUNNEL
  (Alimama's own description: recall -> pre-rank -> rank -> exposure
               -> click -> conversion -> repurchase [12])

corpus 10^7 - 10^9 items
  |  multi-channel recall: item-CF, EGES i2i, MIND multi-interest ANN,
  |  TDM tree beam search, RALM look-alike, graph walks (Graph-Learn / embedx)
  v  ~10^3 - 10^4 candidates
pre-rank ("cu pai")  COLD: small DNN with cross features, compute-aware
  v  ~10^2 - 10^3
rank ("jing pai")    DIN / DIEN / SIM / ETA / BST target attention;
  |                  ESMM / PLE / AITM multi-task heads (CTR, CVR, CTCVR, ...)
  v  ~10^1 - 10^2
re-rank              PRM / PEAR / PIER / generator-evaluator permutations
  v
exposure -> click -> conversion -> repurchase (labels flow back via streaming)
```

1. Alibaba: the Alimama and Taobao line

Alibaba's most influential recommender papers came from the Alimama targeted-advertising algorithm team. The XDL wiki credits DIN, DIEN, ESMM, TDM, CrossMedia and Rocket Training to that team [3][4][12][18][24][25]. All six were open-sourced together as "XDL algorithm solutions" on top of the XDL training engine [22].

1.1 DIN: Deep Interest Network (KDD 2018)

Problem. The standard Embedding&MLP CTR model of 2016 pooled a user's behaviors into one fixed-length vector. The XDL documentation states the problem directly. A Taobao user has many concurrent interests. One fixed vector V_u limits the rank of the model's solution space, and its dimension cannot grow without limit because of compute and generalization. When predicting a click on a down jacket, interest in home appliances is noise [3].

Architecture. An "activation unit" uses the candidate ad to activate the user's historical clicked items. Each behavior gets a relevance weight, and the weighted sum is the user's interest representation for this candidate. That vector is concatenated with user profile, context and ad features and fed to an MLP [3]. The weights are not softmax-normalized, so the magnitude also carries interest intensity [1].

```
candidate ad emb --------------------------+
behavior_1 emb --[act unit(b1, ad)]-- w1 --+
behavior_2 emb --[act unit(b2, ad)]-- w2 --+--> sum_i w_i * b_i  (candidate-specific
...                                        |                       user interest)
behavior_T emb --[act unit(bT, ad)]-- wT --+
       concat(user interest, profile, ad, context) --> MLP (Dice) --> pCTR
```

Training tricks. Two tricks are reported [1][3]:

- **Mini-batch-aware regularization.** L2 is applied only to embedding rows of features present in the mini-batch, because full L2 over billions of parameters per step is infeasible. It added about 0.0031 AUC over dropout on Alibaba data.

- **Dice.** An activation whose rectification point adapts to the mean and variance of its input, instead of the fixed step at zero used by ReLU or PReLU. It added about 0.0015 AUC over PReLU.

Scale and results. The production dataset had more than 2 billion samples. An online A/B test (May to June 2017) showed up to +10.0% CTR and +3.8% RPM, and DIN served the main traffic of Alibaba display advertising [1]. The authors later flagged a BatchNorm bug in the original public demo. Their corrected Amazon-Electronics GAUC table gives PNN 0.8679, DeepFM 0.8683, DIN 0.8698 and DIN with Dice 0.8711, and recommends the refactored DIEN/XDL code instead [5]. That is a useful reminder that public-benchmark gaps between these models are small. BARS found the same thing on a much larger scale (Section 5.4).

Lesson. Make the user representation a function of the candidate. Nearly every later sequence model at Alibaba, Kuaishou, Meituan, Pinterest and Meta inherits this "target attention" idea.

1.2 DIEN: Deep Interest Evolution Network (AAAI 2019)

Problem. DIN treats each behavior as an interest. The DIEN authors argue that interest is latent and behaviors are only its manifestations. Supervision from ad clicks alone cannot teach a sequence model what the user wanted at each step [4].

Architecture. DIEN has two layers [2][4]:

- An **interest-extractor** GRU over the behavior sequence. An **auxiliary loss** asks each hidden state to predict the next clicked item against a sampled non-clicked item. The XDL docs note three benefits of this loss: it adds feedback signal, eases gradient flow on long sequences, and gives the embeddings more semantic supervision.
- An **interest-evolving** layer, **AUGRU**. The GRU update gate is scaled by the attention score between the candidate ad and each extracted interest. Each candidate therefore gets its own evolution path, and interests unrelated to it are suppressed.

Serving. A GRU per candidate is expensive. The paper reports cutting latency from 38.2 ms to 6.6 ms with element-parallel GRU and kernel fusion, batching adjacent requests from different users on GPU, and model compression. The system served more than a million users per second at peak, and the paper reports a 20.7% CTR improvement in Taobao display ads [2]. The XDL docs state that DIEN was deployed across Alimama's targeted-ads products [4]. Serving pressure still limited production DIEN to about 50 behaviors [6].

1.3 Lifelong sequences: MIMN/UIC, SIM, ETA

This line is the clearest example of Chinese system and algorithm co-design.

- **MIMN + UIC (KDD 2019).** A **User Interest Center** keeps a per-user memory state on the serving side. The state is updated *incrementally whenever a behavior event arrives*, not at request time. MIMN is a multi-channel, NTM-style memory network whose memory is exactly the UIC state. Ranking latency is therefore constant in history length, and sequences in the thousands were handled. Online (2019-03-30 to 2019-05-10), MIMN with UIC gave +7.5% CTR and +6% RPM over DIEN [6].

- **SIM (CIKM 2020).** MIMN's user-only memory degraded as histories grew 10x or more. SIM splits the problem into two units [7]:

  - A **General Search Unit (GSU)** picks the top-K relevant behaviors from up to 54,000. Hard search keeps behaviors in the candidate's category. Soft search uses MIPS over embeddings.
  - An **Exact Search Unit (ESU)** runs multi-head target attention on those K behaviors.

  The two searches returned similar lists, so the cheaper hard search went online. Its index is a two-level **User Behavior Tree** (user_id -> category_id -> behaviors), built offline, distributed, about 22 TB. Results: +7.1% CTR and +4.4% RPM, in production since 2019 [7].

- **ETA (2021-2022).** ETA replaces the separate GSU with **SimHash** signatures of the same embeddings the attention uses. Top-k selection by Hamming distance needs only bit operations, and retrieval becomes end-to-end trainable. Reported gains are 3.1% GMV for ETA [8], and +1.8% CTR and +3.1% GMV for ETA-Net on Taobao [9].

```
UIC (event-driven)                    SIM (request-driven, two-stage)
behavior event --> update memory      request(user, candidate c)
                   M_u in UIC           |  GSU: UBT lookup [user][cat(c)]
request --> read M_u (O(1)) --> MIMN    |       (22 TB index, hard search)
                                       v  top-K behaviors
                                      ESU: multi-head target attn(c, top-K)
                                        v
                                      concat with short-term DIN/DIEN -> MLP
```

BST (DLP-KDD 2019) applied a single Transformer layer to the short behavior sequence in Taobao *recommendation* ranking. Its position feature is the time gap between the recommendation and the click. Offline AUC was 0.7894 against 0.7866 for DIN, and online CTR rose +7.57% against +4.55% for DIN [10].

1.4 ESMM: Entire Space Multi-Task Model (SIGIR 2018)

Problem. Alimama's docs decompose GMV as traffic x CTR x CVR x price per order, which makes CVR a direct lever [12]. CVR models trained only on clicked impressions suffer from **sample selection bias (SSB)**, since they score all impressions but train on clicks. They also suffer from **data sparsity (DS)**: in Taobao data CVR samples were about 4% of CTR samples [11]. Delayed feedback is a third challenge [12].

Design. pCVR is never supervised directly. The model predicts pCTR and pCTCVR = pCTR x pCVR over *all impressions* and supervises those two [11][12]:

```
loss = sum_i CE(y_i, pCTR(x_i)) + sum_i CE(y_i AND z_i, pCTR(x_i) * pCVR(x_i))

 impression x --> [shared embedding layer] --+--> CTR tower --> pCTR ----+
                                             |                           (x)--> pCTCVR
                                             +--> CVR tower --> pCVR ----+
```

The product avoids dividing by small pCTR values, which is numerically unstable. The CVR tower shares embeddings with the CTR task, which has orders of magnitude more samples. That transfer is what addresses the sparsity problem [12]. Each tower can be any base

network, so ESMM is a framework rather than a model [12]. The 8.9-billion-sample Taobao dataset was released as a 1% public sample, Ali-CCP [11][12].

Successors. ESM2 decomposes the post-click path into cart and wishlist actions [13]. ESCM2 adds a counterfactual regularizer because ESMM's CVR is itself biased [14]. Meituan's AITM generalizes the idea to learned information transfer across longer funnels (Section 3.2) [60]. Alibaba's delayed-feedback work, ES-DFM and DEFER, handles the third challenge [35].

1.5 EGES: billion-scale commodity embedding (KDD 2018)

Taobao's matching stage connects about a billion users to about two billion items [15]. EGES works in four steps:

1. Build a weighted directed item graph from sessions of about one hour, filtering very short clicks and over-active users.
2. Run DeepWalk-style weighted random walks and train skip-gram on them (BGE).
3. Add side-information embeddings (category, shop, brand, price level), which helps cold-start items (GES).
4. In EGES proper, learn per-item attention weights over those side embeddings.

Embedding nearest neighbors become each item's i2i candidate list, and the paper reports CTR gains over the prior CF-based matching [15]. The lesson: side-information embeddings close the cold-start gap at nearly zero serving cost.

1.6 TDM and JTM: learned tree indexes for retrieval

Problem. Item-CF and inner-product vector retrieval cannot use an expressive deep model over a corpus of tens of millions of items. Scoring every item is O(n) [18].

Design. All items are indexed as leaves of a tree, and each internal node represents a set of items, from coarse to fine. The tree need not be binary. A deep model acts as a per-node "interest discriminator". Retrieval runs **layer-wise beam search**: keep the top K nodes per level and expand their children. The cost of whole-corpus retrieval drops from O(n) to O(log n), and any advanced model can be used because the beam prunes aggressively [16][18]. The open-source model uses only user behaviors, split into time windows, plus node embeddings, with attention over behaviors in the style of DIN [18].

Tree and model jointly. The tree supplies training samples: positives are a positive leaf's ancestors, and negatives come from the same level. It also decides pruning at serving time. TDM therefore alternates two steps. It trains the model, then re-clusters the learned leaf embeddings hierarchically (k-means) to build a new tree, and repeats until both are stable [18]. JTM later learns the tree and model under one objective [17]. The XDL docs claim TDM was applied in video, product and news recommendation as well as Alimama ads [18].

Serving. TDM Serving is a C++ library with three layers [19]:

- an **API** layer for loading data and real-time retrieval;
- an **Index** layer for tree beam search, index management and business-logic filtering;
- a **Model** layer for per-level DNN scoring, executed by the **Blaze** inference engine.

Blaze is Alimama's high-performance inference engine for ads, search and recommendation. It uses its own ULF model format, converted from XDL output [23]. Baidu's PaddleRec also ships a TDM implementation [65].

```
offline (XDL)                                  online (TDM Serving)
item embs --k-means--> tree T_k                request(user behaviors)
    ^                     |                      |
    |      samples: ancestors(+), same-level(-)   v
    +---- train DNN M_k <-+                    level 1: score children of root (Blaze)
         (repeat until stable)                  keep top-K --> expand children
                                                ... level L: top-K leaves = items
export: tree.pb + dense/sparse/item_emb  --->   + business filters in Index layer
```

1.7 MIND: multi-interest retrieval at Tmall

MIND uses capsule dynamic routing to turn a user's behaviors into K interest vectors, trained with label-aware attention. Each vector issues its own ANN query. It served major traffic on the Mobile Tmall homepage [20]. It is the inverse of DIN: DIN builds one user vector per candidate at ranking time, while MIND builds several candidate-independent user vectors so retrieval stays factorized.

1.8 COLD and pre-ranking; Rocket Training and distillation

COLD (2020) rejected the two-tower pre-ranker because it cannot use user-item cross features. COLD uses a small fully connected network with cross features. Feature selection is made under a compute budget, and the model is co-designed with serving (low precision, parallelism) and trained online [21].

Rocket Training is Alimama's co-training compression method. A large "booster" net and a small "light" net share bottom layers such as embeddings and are trained together. A hint loss pulls the light net's logits toward the booster's. A **gradient block** freezes the booster-only parameters on that hint term so the small net cannot drag the large one down. Only the light net serves [25].

1.9 CrossMedia / DICM and the Advanced Model Server

DICM (CIKM 2018) adds images to CTR prediction: the ad image, plus the images of items the user clicked in the past 14 days [24]. Image vectors come from a fixed VGG16 plus a trainable fully connected layer, and user-behavior images are aggregated by sum or attention pooling. Shipping images through a parameter server for billions of samples is too costly. XDL therefore built the **Advanced Model Server (AMS)**, where *servers* run the image-embedding forward and backward passes. Images are stored once on the servers, and duplicate embeddings of the same image within a batch are removed [24]. This is an early form of the "compute near the parameters" pattern.

1.10 Re-ranking, RL and simulators

Taobao contributed several re-ranking and RL designs:

- **MIDNN (IJCAI 2018)**: list-level features plus beam search under an evaluator, for mutual-influence-aware ranking in Taobao search [75].

- **PRM (RecSys 2019)**: a Transformer re-ranker over the top tens of items, with pre-trained "personalized vectors" and the initial rank as an input [31].
- **Exact-K (KDD 2019)**: chooses an exact set of K items jointly [74].
- **Virtual-Taobao (AAAI 2019)**: trains a search-ranking policy in a GAN and imitation-learning simulator, reporting more than 2% online revenue gain [32].
- **PE-LTR (RecSys 2019)**: a Pareto-efficient algorithm for jointly optimizing GMV and CTR [34].

1.11 Platforms: XDL, DeepRec, EasyRec, Graph-Learn, Euler

- **XDL** is Alimama's framework for high-dimensional sparse data. Version 1.2 added 50-100% speedups for large-batch, low-concurrency jobs, automatic global parameter placement with request merging to remove parameter-server hotspots, and full streaming-training features: feature admission, feature eviction, incremental model export and feature counting [22].
- **DeepRec** is a heavily modified TensorFlow 1.15 developed since 2016. It supports Taobao search, recommendation and ads, and dozens of outside companies use it. It claims training at "trillion samples and over ten trillion parameters" [26]. Key features: EmbeddingVariable (dynamic, adaptive and multi-hash), multi-tier HBM/DRAM/PMem/SSD storage, asynchronous parameter-server training (grpc+seastar, StarServer) or synchronous collective training (HybridBackend, SOK), and delta checkpoints [26][73].
- **EasyRec / TorchEasyRec** is the configuration-driven model zoo on Alibaba Cloud (MaxCompute, PAI, EAS). It supports large embeddings and online learning [72].
- **Graph-Learn (formerly AliGraph)** retrains GNNs incrementally every hour [27] [30]. Its **Dynamic Graph Service** pre-samples the graph update stream against pre-registered queries. One node (64 cores, 256 GB) answers 2-hop queries at 20 ms P99 and 20K QPS while ingesting 110 MB/s of updates [28]. **Euler** is the separate Alimama graph framework, with GQL and message passing [29].

1.12 Generative turn: LUM

Alibaba's LUM (2025) pre-trains a large user model generatively, queries it, and feeds its outputs into existing discriminative models. This "three-step paradigm" aims to unlock scaling laws without discarding the existing cascade [33].

2. Tencent: WeChat, Tencent Video and Tencent Ads

2.1 PLE: Progressive Layered Extraction (RecSys 2020 best long paper)

Problem. Tencent's video recommender saw a **seesaw**: MMoE improved one task at the expense of another when task correlations were complex [36].

Design. CGC gives each task its own experts plus a pool of shared experts. Task k's gate mixes only its own experts and the shared ones. **PLE** stacks CGC levels, with a shared gate feeding shared experts at the next level, so specific and shared knowledge separate progressively [36].

```
level 1:  [VTR experts]   [shared experts]   [VCR experts]
             gate_A          gate_shared         gate_B
level 2:  [VTR experts]   [shared experts]   [VCR experts]
             gate_A                              gate_B
          tower_A -> VTR (effective view, AUC)   tower_B -> VCR (completion, MSE)
```

Data and results. The dataset had about 1 billion samples. MMoE barely moved VCR (about +0.0001 MSE improvement), while PLE improved both tasks significantly. Online, view count rose 2.23% and watch time 1.84% over the previous multi-task model [36]. PLE is now a stock component: DeepCTR, FuxiCTR and EasyRec all ship it [53][71][72].

2.2 STEM and embedding collapse

Two Tsinghua/Tencent papers moved the multi-task and scaling debate down to the embedding layer:

- **STEM (AAAI 2024)** keeps one shared embedding table plus one per task. An "All Forward Task-specific Backward" gate lets every expert contribute forward, while stop-gradient keeps a task's gradients out of other tasks' tables [37].
- **"On the Embedding Collapse" (ICML 2024)** shows that embedding matrices collapse into low-dimensional subspaces as they are scaled. Its multi-embedding design keeps improving up to 10x embedding size [38].

2.3 WeChat: RALM look-alike retrieval and embedx

WeChat's article feed ("Kan Yi Kan") published **RALM** (KDD 2019), a real-time attention-based look-alike model for recommending fresh or long-tail content. The idea is to represent an item by the users who engaged with it ("seeds") and match new users against them. PaddleRec ships an implementation [39][65]. (The full paper could not be re-fetched. Treat this description as a summary of its title and framing, not its numbers.)

embedx is a C++ distributed embedding training and inference system. It covers graph models with billions of nodes and hundreds of billions of edges, and deep ranking and retrieval models with tens of billions of samples and features [40]. Its README lists the products it has served:

- recommendation, search and payments at WeChat: Kan Yi Kan, WeChat Channels, WeChat Search and WeChat Pay;
- security: WeChat Security and Tencent's anti-fraud work;
- content and apps: Tencent News, the App Store (Yingyongbao), QQ Music, JOOX and Tencent Classroom.

Its design notes describe four things [40]:

- **Joint graph and ranking models** (graphdeepfm, graphdssm). Training them together avoids the objective mismatch of the usual two-stage use of graph embeddings.
- **A minute-level online-learning loop.** Recall and ranking both train and serve from the real-time stream, and changes take effect end to end within minutes.
- **Incremental parameter export and asynchronous parameter updates** in serving.

- **Importance-sampling correction** for the distribution shift of real-time samples.

The README claims 30-60% machine savings over open-source frameworks [40].

Angel, built by Tencent and Peking University, is the older parameter-server platform in Java/Scala on YARN, with Spark-on-Angel for ML and graph algorithms [41]. Tencent is also a listed contributor to TensorFlow Recommenders-Addons dynamic embeddings [45].

2.4 Tencent Ads: FLEN, meta-embeddings, GPR

- **FLEN (DLP-KDD 2020)** is a field-aware CTR model designed for scalable serving [44].
- **Meta-Embedding (SIGIR 2019)** learns a generator from ad attributes to an initial ID embedding with a MAML-style two-phase objective, which warms up cold-start ads [43].
- **GPR (2025)** serves WeChat Channels ads. It is a decoder over four token types (user, organic, environment, ad). Items get RQ-KMeans+ semantic IDs, and decoding uses value-guided trie beam search. It reportedly collapsed five cascade models into one and cut latency from more than 200 ms to about 80 ms (approximate) [42].

3. Meituan: local services, food delivery and ads

Meituan's recommendation problems are shaped by location, time of day and multi-step conversion: impression, click, order, then delivery.

3.1 SDIM: sampling instead of attention (CIKM 2022)

SDIM hashes the candidate and every behavior with several hash functions and sums the behaviors that share the candidate's signature. The authors prove that this approximates target attention. Hashing runs in a separate **Behavior Sequence Encoding** service, so the CTR model only does a lookup. Meituan search reported +2.98% CTR and +2.69% VBR [59].

3.2 AITM: multi-step conversion (KDD 2021)

AITM targets credit-card customer acquisition, where the funnel is impression, click, application, approval, activation. An Adaptive Information Transfer module learns what to pass from earlier steps to later ones. A Behavioral Expectation Calibrator keeps later-step probabilities consistent with earlier ones. It was deployed in the Meituan app [60].

3.3 PIER: permutation re-ranking (WSDM 2023)

PIER picks top permutations cheaply with SimHash-style hashing of user interest, then scores them with an omnidirectional context-aware module, trained end to end. It makes generator-evaluator re-ranking affordable at Meituan's scale [61].

3.4 MTGR, MTGRBoost and EGA (2025)

Problem. Meituan found that a pure HSTU-style generative ranker, which drops user x item cross features, did worse than a mid-sized DLRM even at the largest size tried [57].

Design. MTGR is a Transformer encoder with HSTU-style user-level sequence compression, trained on discriminative CTR/CTCVR losses [57]:

- **Token groups**: user, long sequence, real-time and candidate. Cross features are kept *only on candidate tokens*.
- **Group LayerNorm**: separate normalization per token group.
- **Dynamic masking**: real-time tokens are causal and time-gated per candidate, and candidates cannot see each other. All m candidates of a user are scored in one pass at O((n+m)^2), against O(m n^2) when scored separately.

Per-sample inference compute was about 65x that of the DLRM baseline (approximate). Meituan called it the largest offline and online gain in about two years. Training infrastructure is described in MTGRBoost [57][58].

EGA-V2 generates ads end to end. It uses RQ-VAE semantic IDs (3 levels x 1024 codes), factors generation into P(POI) x P(creative | POI), and adds a payment network that guarantees individual rationality, with approximate incentive compatibility enforced through a regret Lagrangian [62].

4. Huawei Noah's Ark: feature interactions, bias and benchmarks

Huawei's recommender research is anchored in the **Huawei App Store** (AppGallery) and in open benchmarks.

4.1 DeepFM (IJCAI 2017) and the PNN family

DeepFM replaces Wide&Deep's hand-crafted wide part with an FM component that *shares the embedding layer* with the DNN. It therefore needs no feature engineering beyond raw features. It improved AUC and logloss on Criteo and on a commercial Huawei app-store dataset [46].

PNN (ICDM 2016) inserts a product layer (inner or outer products of field embeddings) before the MLP [47]. It is one of the baselines DIN compared against [5]. A journal extension (the "PIN" product-network-in-network variant) generalizes the pairwise product to small per-pair subnetworks. It could not be re-fetched for this chapter, so its details are not stated here.

Later Huawei CTR models listed in FuxiCTR [53] include:

- **FGCNN (WWW 2019)**: CNN-generated new features [49];
- **EDCN (DLP-KDD 2021)**: information sharing between the parallel explicit and implicit towers [50];
- **FinalMLP (AAAI 2023)**: two MLP streams fused by bilinear interaction, plus FINAL and EulerNet (SIGIR 2023) [51];

- **MIRRN (KDD 2025)**: multi-granularity retrieval and refinement for long behavior sequences [56].

4.2 PAL: position bias as a probability (RecSys 2019)

PAL factorizes pCTR(click | x, pos) = pSeen(pos) x pCTR(click | x, seen). The two modules are trained jointly, and only the CTR module ranks at serving time, so position never has to be known before it is chosen. Huawei App Store A/B tests reported CTR and conversion gains over a "position as feature" baseline [48].

4.3 PEAR and cross-batch negatives

- **PEAR (WWW 2022)** extends PRM-style re-ranking with list-item attention over the user's *history* and an auxiliary list-level engagement task [54].
- **Cross-Batch Negative Sampling (SIGIR 2021)** keeps a FIFO memory of recent item embeddings and reuses them as extra negatives for two-tower training, at close to in-batch cost [55].

4.4 FuxiCTR and BARS: honesty about benchmarks

FuxiCTR is a configurable, reproducible CTR library. Its model zoo labels each model by company, and it is the easiest index of Chinese CTR work [53]. The BARS-CTR benchmark re-ran 24 models in more than 7,000 experiments and more than 12,000 GPU hours. With proper tuning, many deep models differ far less than their papers claim [52]. Read every offline AUC gain in this chapter with that result in mind.

5. Baidu, JD.com, Xiaohongshu, Weibo and Pinduoduo

5.1 Baidu

- **MOBIUS (KDD 2019)** targets Baidu's sponsored-search query-ad matching. It is described as moving the matching layer from pure relevance toward the downstream business objective (CPM), so that matching and ranking are no longer optimized for different targets. The paper uses a relevance "teacher" to keep low-relevance, high-CTR pairs out, plus approximate nearest-neighbor search. (Not re-fetched for this chapter. This summary follows the published abstract as recalled; treat the details as approximate) [66].
- **PaddleRec** is Baidu's open model zoo on PaddlePaddle, with parameter-server and multi-GPU training. It covers matching (DSSM, MultiView-SimNet), recall (TDM, MIND, RALM, YouTube-DNN, DeepWalk, GRU4Rec) and ranking models [65].
- **COBRA (2025)** is Baidu's cascaded sparse-dense generative recommender. It generates a semantic ID and then refines it with a dense vector [64].

5.2 JD.com

JD's best-known public recommender work is **page-wise RL** (RecSys 2018). An actor-critic recommends a whole 2-D page at once, with a deconvolutional actor and a critic that values the page. It is trained first against an offline simulator built from logs [63]. Few JD

production-architecture details (QPS, model sizes) appear in the sources gathered for this chapter.

5.3 Xiaohongshu (RedNote)

- **GenRank (2025)** is generative ranking trained on hundreds of billions of exposure logs [67]:
  - An *action-oriented* sequence makes actions the tokens and folds items in as attributes. This halves sequence length, cuts attention compute by 75% and speeds training by 78.7%.
  - Parameter-free ALiBi-style position and time biases raise the total speedup to 94.8%.
  - Real-time features stayed important, while most historical aggregate features lost value.
- **NoteLLM (WWW 2024)** compresses each note into a special LLM token for note-to-note (item-to-item) recommendation. It trains that token with contrastive learning on co-occurrence signals, alongside hashtag and category generation. (Not re-fetched; summarized from the paper as recalled) [68].

5.4 Sina Weibo

Weibo contributed two widely reused interaction modules [69]:

- **FiBiNET (RecSys 2019)**: SENET-style feature importance plus bilinear interactions.
- **MaskNet (2021)**: instance-guided multiplicative masks. MaskNet later became the backbone of Twitter's heavy ranker.

5.5 Pinduoduo

None of the sources reachable for this chapter documents Pinduoduo's recommender architecture in a primary engineering paper or blog. Pinduoduo is far less public than its peers. Designers should not assume its stack matches the Alibaba templates.

6. Comparison of the major designs

Design	Company / year	Stage	Core idea	Reported scale / result
DIN [1]	Alibaba 2018	Rank	Candidate-aware attention over behaviors	2B+ samples; up to +10% CTR, +3.8% RPM
DIEN [2]	Alibaba 2019	Rank	GRU + aux loss + AUGRU	+20.7% CTR; 38.2 -> 6.6 ms; 1M+ users/s
MIMN+UIC [6]	Alibaba 2019	Rank	Event-driven user memory	+7.5% CTR, +6% RPM over DIEN
SIM [7]	Alibaba 2020	Rank	GSU (category search) + ESU attention	54K behaviors; 22 TB UBT; +7.1% CTR
ESMM [11]	Alibaba 2018	Rank (CVR)	pCTCVR = pCTR x pCVR over all impressions	8.9B samples; Ali-CCP released
EGES [15]	Alibaba 2018	Recall (i2i)	Session-graph walks + side-info attention	~1B users, ~2B items

Design	Company / year	Stage	Core idea	Reported scale / result
TDM/JTM [16][17]	Alibaba 2018-19	Recall	Tree index + deep model, beam search	O(log n); 10^7+ items
MIND [20]	Alibaba 2019	Recall	K interest capsules, K ANN queries	Tmall homepage main traffic
COLD [21]	Alibaba 2020	Pre-rank	Small cross-feature DNN under compute budget	online-trained
PLE [36]	Tencent 2020	Rank (MTL)	Task-specific + shared experts, stacked	~1B samples; +2.23% views
embedx [40]	Tencent (WeChat)	Platform	Graph + sparse DNN, minute-level online learning	10^9 nodes, 10^11 edges
GPR [42]	Tencent 2025	Ads cascade	Generative decoder, semantic IDs	5 models -> 1; ~80 ms
SDIM [59]	Meituan 2022	Rank	Hash sampling replaces attention	+2.98% CTR
AITM [60]	Meituan 2021	Rank (MTL)	Learned transfer across funnel steps	deployed in Meituan app
MTGR [57]	Meituan 2025	Rank	Generative encoder + cross features	~65x DLRM FLOPs/sample
DeepFM [46]	Huawei 2017	Rank	FM + DNN with shared embeddings	Huawei app store data
PAL [48]	Huawei 2019	Rank	pSeen(pos) x pCTR	App Store A/B gains
GenRank [67]	Xiaohongshu 2025	Rank	Action-oriented generative sequence	94.8% training speedup
Page-wise RL [63]	JD 2018	Page	Actor-critic over 2-D pages	simulator then online

7. Architecture patterns characteristic of Chinese hyperscale

Many of these ideas exist elsewhere. Their combination and early adoption are distinctive.

1. **Ads teams drive the frontier.** DIN, DIEN, MIMN, SIM, ESMM, TDM, CrossMedia, Rocket and COLD all came from Alimama's targeted-ads team [3][4][12][18][24][25]. Revenue per point of CTR or CVR justifies GPU serving and custom infrastructure earlier than organic feeds do.
2. **Model the funnel explicitly.** Chinese papers treat recall -> pre-rank -> rank -> exposure -> click -> conversion -> repurchase as one chain [12]. ESMM, ESM2, ESCM2 and AITM encode that chain in the loss. COLD and Rocket make each stage as expressive as its compute budget allows.
3. **Algorithm-system co-design is the norm, not the exception.** UIC moves computation to event time. SIM's 22 TB behavior tree, ETA's SimHash and SDIM's BSE service turn attention into indexing. AMS moves image embedding onto parameter servers. TDM's tree is both a training-sample generator and a serving index. DGS pre-samples graphs. MTGR's masking scores all candidates in one pass. In every case the model is shaped by what the serving system can do cheaply [6][7][8][24][18][28][57][59].

4. **Streaming everything.** Feature admission and eviction, incremental export, delta checkpoints and minute-level online learning appear in XDL, DeepRec and embedx, as in ByteDance's Monolith [22][26][40].
5. **Open-source platforms as recruiting and ecosystem tools.** XDL, DeepRec, EasyRec, Graph-Learn, Euler, embedx, Angel and PaddleRec are released with Chinese-first documentation and cloud integration (Alibaba Cloud PAI/MaxCompute, Baidu AI Studio) [22][26][27][29][40][41][65][72].
6. **Lifelong behavior as a first-class signal.** E-commerce and short-video users accumulate 10^4 to 10^5 behaviors. The SIM -> ETA -> SDIM -> TWIN line (Kuaishou, Chapter 20) exists because Chinese platforms had both the histories and the latency pressure [7][9][59][70].
7. **Fast embrace of generative ranking with pragmatic hybrids.** Kuaishou OneRec, Meituan MTGR and EGA, Tencent GPR, Xiaohongshu GenRank, Baidu COBRA and Alibaba LUM all shipped or reported production results in 2025. Several deliberately keep cross features, discriminative losses or the existing cascade [33][42][57][62][64][67][70].

```
       ALGORITHM-SYSTEM CO-DESIGN: WHERE THE WORK MOVES

request-time compute  -->  moved to ...                 example
--------------------       --------------------------   ---------------------
attend over history        event-time memory update      UIC / MIMN [6]
attend over 54K items      offline category index        SIM UBT, 22 TB [7]
pick top-k behaviors       bit ops on SimHash codes      ETA [8], SDIM [59]
embed images per sample    parameter-server compute      AMS / DICM [24]
traverse graph n hops      write-time pre-sampling       DGS, 20 ms P99 [28]
score 10^7 items           beam search on learned tree   TDM Serving [19]
score m candidates         one pass, masked attention    MTGR [57]
```

Key takeaways for system designers

1. **Make the user representation candidate-aware in ranking and candidate-independent in retrieval.** DIN, DIEN and SIM attend per candidate at ranking time. MIND, EGES and TDM keep retrieval factorized or indexable.
2. **Supervise in the space you serve in.** ESMM's pCTCVR trick removes sample-selection bias and borrows density from the CTR task. Extend it to multi-step funnels (ESM2, AITM) and correct its residual bias (ESCM2).
3. **When attention is too expensive, turn it into retrieval.** Use a two-stage GSU/ESU (SIM), hashing (ETA, SDIM) or event-time memory (UIC). Choose by whether you need candidate-awareness and end-to-end training.
4. **Learned indexes let deep models do retrieval.** TDM's tree, rebuilt from learned embeddings, gives O(log n) whole-corpus scoring with arbitrary models. The price is a serving engine (beam search plus per-level scoring) and a training loop that alternates tree and model.
5. **Separate task-specific from shared capacity when you see a seesaw.** PLE and CGC are the default answer. STEM pushes the separation into embeddings. Watch for embedding collapse when scaling.
6. **Budget compute per stage, and spend it where crosses pay.** COLD's cross-feature pre-ranker and Rocket's booster-light co-training show how to buy quality under a latency cap.

7. **Invest in streaming infrastructure before bigger models.** Feature admission and eviction, incremental export and minute-level online learning (XDL, DeepRec, embedx) are table stakes for fast-churning catalogs.
8. **Do not discard signal to be "purely generative".** MTGR and GenRank show that cross features and real-time features remain decisive. Hybrid generative-discriminative designs are the production norm in 2025-2026.
9. **Discount offline gains.** DIN's corrected GAUC table and the BARS benchmark both show that tuned baselines close most published gaps. Trust online A/B results and production deployment claims more than offline AUC.

References

1. Zhou, G., Song, C., Zhu, X., et al. "Deep Interest Network for Click-Through Rate Prediction." Alibaba, KDD 2018. https://arxiv.org/abs/1706.06978
2. Zhou, G., Mou, N., et al. "Deep Interest Evolution Network for Click-Through Rate Prediction." Alibaba, AAAI 2019. https://arxiv.org/abs/1809.03672
3. Alibaba Alimama. "Deep Interest Network (DIN)" (XDL wiki page). Alibaba, 2018-2019. https://github.com/alibaba/x-deeplearning/wiki/%E6%B7%B1%E5%BA%A6%E5%85%B4%E8%B6%A3%E7%BD%91%E7%BB%9C(DIN)
4. Alibaba Alimama. "Deep Interest Evolution Network (DIEN)" (XDL wiki page). Alibaba, 2018-2019. https://github.com/alibaba/x-deeplearning/wiki/%E7%94%A8%E6%88%B7%E5%85%B4%E8%B6%A3%E6%BC%94%E5%8C%96%E6%A8%A1%E5%9E%8B(DIEN)
5. Zhou, G. "DeepInterestNetwork" reference implementation README (corrected GAUC results). GitHub, 2018. https://github.com/zhougr1993/DeepInterestNetwork
6. Pi, Q., Bian, W., Zhou, G., et al. "Practice on Long Sequential User Behavior Modeling for Click-Through Rate Prediction" (MIMN/UIC). Alibaba, KDD 2019. https://arxiv.org/abs/1905.09248
7. Pi, Q., Zhu, X., Zhou, G., et al. "Search-based User Interest Modeling with Lifelong Sequential Behavior Data for Click-Through Rate Prediction" (SIM). Alibaba, CIKM 2020. https://arxiv.org/abs/2006.05639
8. Chen, Q., Pei, C., et al. "End-to-End User Behavior Retrieval in Click-Through Rate Prediction Model" (ETA). Alibaba, 2021. https://arxiv.org/abs/2108.04468
9. "Efficient Long Sequential User Data Modeling for Click-Through Rate Prediction" (ETA-Net). Alibaba, DLP-KDD 2022. https://arxiv.org/abs/2209.12212
10. Chen, Q., Zhao, H., Li, W., Huang, P., Ou, W. "Behavior Sequence Transformer for E-commerce Recommendation in Alibaba." Alibaba, DLP-KDD 2019. https://arxiv.org/abs/1905.06874
11. Ma, X., Zhao, L., et al. "Entire Space Multi-Task Model: An Effective Approach for Estimating Post-Click Conversion Rate." Alibaba, SIGIR 2018. https://arxiv.org/abs/1804.07931
12. Alibaba Alimama. "Entire Space Multi-task Model (ESMM)" (XDL wiki page). Alibaba, 2018-2019. https://github.com/alibaba/x-deeplearning/wiki/%E5%85%A8%E7%A9%BA%E9%97%B4%E5%A4%9A%E4%BB%BB%E5%8A%A1%E6%A8%A1%E5%9E%8B(ESMM)
13. Wen, H., et al. "Entire Space Multi-Task Modeling via Post-Click Behavior Decomposition for Conversion Rate Prediction" (ESM2). Alibaba, 2019. https://arxiv.or

g/abs/1910.07099
14. Wang, H., et al. "ESCM2: Entire Space Counterfactual Multi-Task Model for Post-Click Conversion Rate Estimation." SIGIR 2022. https://arxiv.org/abs/2204.05125
15. Wang, J., Huang, P., Zhao, H., Zhang, Z., Zhao, B., Lee, D. L. "Billion-scale Commodity Embedding for E-commerce Recommendation in Alibaba" (EGES). Alibaba, KDD 2018. https://arxiv.org/abs/1803.02349
16. Zhu, H., et al. "Learning Tree-based Deep Model for Recommender Systems" (TDM). Alibaba, KDD 2018. https://arxiv.org/abs/1801.02294
17. Zhu, H., et al. "Joint Optimization of Tree-based Index and Deep Model for Recommender Systems" (JTM). Alibaba, NeurIPS 2019. https://arxiv.org/abs/1902.07565
18. Alibaba Alimama. "Tree-based Deep Match (TDM)" (XDL wiki page). Alibaba, 2018-2019. https://github.com/alibaba/x-deeplearning/wiki/%E6%B7%B1%E5%BA%A6%E6%A0%91%E5%8C%B9%E9%85%8D%E6%A8%A1%E5%9E%8B(TDM)
19. Alibaba Alimama. "TDM Serving" (XDL wiki page). Alibaba, 2018-2019. https://github.com/alibaba/x-deeplearning/wiki/TDMServing
20. Li, C., et al. "Multi-Interest Network with Dynamic Routing for Recommendation at Tmall" (MIND). Alibaba, CIKM 2019. https://arxiv.org/abs/1904.08030
21. Wang, Z., Zhao, L., Jiang, B., et al. "COLD: Towards the Next Generation of Pre-Ranking System." Alibaba, DLP-KDD 2020. https://arxiv.org/abs/2007.16122
22. Alibaba Alimama. "X-DeepLearning (XDL)" repository README (v1.2 features). Alibaba, 2018-2019. https://github.com/alibaba/x-deeplearning
23. Alibaba Alimama. "Blaze" inference engine README. Alibaba, 2018-2019. https://github.com/alibaba/x-deeplearning/blob/master/blaze/README.md
24. Ge, T., et al. "Image Matters: Visually Modeling User Behaviors Using Advanced Model Server" (DICM / CrossMedia). Alibaba, CIKM 2018; and XDL wiki page "CrossMedia". https://arxiv.org/abs/1711.06505 ; https://github.com/alibaba/x-deeplearning/wiki/%E5%9B%BE%E5%83%8F%E8%81%94%E5%90%88%E8%AE%AD%E7%BB%83%E7%AE%97%E6%B3%95(CrossMedia)
25. Alibaba Alimama. "Rocket Training: lightweight general model compression" (XDL wiki page). Alibaba, 2018-2019. https://github.com/alibaba/x-deeplearning/wiki/%E8%BD%BB%E9%87%8F%E7%BA%A7%E9%80%9A%E7%94%A8%E6%A8%A1%E5%9E%8B%E5%8E%8B%E7%BC%A9%E7%AE%97%E6%B3%95(Rocket-Training)
26. Alibaba. "DeepRec" repository README. Alibaba / LF AI & Data, accessed 2026. https://github.com/alibaba/DeepRec
27. Alibaba. "Graph-Learn (formerly AliGraph)" repository README. Alibaba, 2020-2023. https://github.com/alibaba/graph-learn
28. Alibaba. "Dynamic Graph Service (DGS) Introduction." Graph-Learn documentation. https://github.com/alibaba/graph-learn/blob/master/docs/en/dgs/intro.md
29. Alibaba Alimama. "Euler" graph learning framework repository README. Alibaba, 2019-2020. https://github.com/alibaba/euler
30. Zhu, R., Zhao, K., Yang, H., et al. "AliGraph: A Comprehensive Graph Neural Network Platform." Alibaba, PVLDB 12(12), 2019. https://arxiv.org/abs/1902.08730
31. Pei, C., Zhang, Y., et al. "Personalized Re-ranking for Recommendation" (PRM). Alibaba, RecSys 2019. https://arxiv.org/abs/1904.06813
32. Shi, J.-C., Yu, Y., Da, Q., Chen, S.-Y., Zeng, A.-X. "Virtual-Taobao: Virtualizing Real-world Online Retail Environment for Reinforcement Learning." Nanjing University / Alibaba, AAAI 2019. https://arxiv.org/abs/1805.10000

33. "Unlocking Scaling Law in Industrial Recommendation Systems with a Three-step Paradigm based Large User Model" (LUM). Alibaba, 2025. https://arxiv.org/abs/2502.08309
34. Lin, X., et al. "A Pareto-Efficient Algorithm for Multiple Objective Optimization in E-Commerce Recommendation" (PE-LTR). Alibaba, RecSys 2019. https://www.doi.org/10.1145/3298689.3346998
35. Yang, J.-Q., et al. "Capturing Delayed Feedback in Conversion Rate Prediction via Elapsed-Time Sampling" (ES-DFM), Alibaba, AAAI 2021, https://arxiv.org/abs/2012.03245 ; Gu, S., et al. "Real Negatives Matter: Continuous Training with Real Negatives for Delayed Feedback Modeling" (DEFER), Alibaba, KDD 2021, https://arxiv.org/abs/2104.14121
36. Tang, H., Liu, J., Zhao, M., Gong, X. "Progressive Layered Extraction (PLE): A Novel Multi-Task Learning (MTL) Model for Personalized Recommendations." Tencent, RecSys 2020. https://dl.acm.org/doi/10.1145/3383313.3412236
37. Su, L., Pan, J., et al. "STEM: Unleashing the Power of Embeddings for Multi-task Recommendation." Tsinghua / Tencent, AAAI 2024. https://arxiv.org/abs/2308.13537
38. Guo, X., et al. "On the Embedding Collapse When Scaling up Recommendation Models." Tsinghua / Tencent, ICML 2024. https://arxiv.org/abs/2310.04400
39. Liu, Y., et al. "Real-time Attention Based Look-alike Model for Recommender System" (RALM). Tencent WeChat, KDD 2019 (not re-fetched; cited via PaddleRec). https://arxiv.org/abs/1906.05022
40. Tencent. "embedx" repository README and "embedx detailed introduction" (docs/intro_embedx.md); Zou, Y., et al. "EmbedX: A Versatile, Efficient and Scalable Platform to Embed Both Graphs and High-Dimensional Sparse Data," PVLDB 16(12), 2023. https://github.com/Tencent/embedx ; https://doi.org/10.14778/3611540.3611546
41. Tencent / Peking University. "Angel" repository README. Angel-ML, accessed 2026. https://github.com/Angel-ML/angel
42. "GPR: Towards a Generative Pre-trained One-Model Paradigm for Large-Scale Advertising Recommendation." Tencent, 2025. https://arxiv.org/abs/2511.10138
43. Pan, F., Li, S., Ao, X., Tang, P., He, Q. "Warm Up Cold-start Advertisements: Improving CTR Predictions via Learning to Learn ID Embeddings." SIGIR 2019. https://arxiv.org/abs/1904.11547
44. Chen, W., et al. "FLEN: Leveraging Field for Scalable CTR Prediction." Tencent, DLP-KDD 2020. https://arxiv.org/abs/1911.04690
45. TensorFlow SIG Recommenders. "TensorFlow Recommenders-Addons" repository README. Accessed 2026. https://github.com/tensorflow/recommenders-addons
46. Guo, H., Tang, R., Ye, Y., Li, Z., He, X. "DeepFM: A Factorization-Machine based Neural Network for CTR Prediction." Huawei Noah's Ark, IJCAI 2017. https://arxiv.org/abs/1703.04247
47. Qu, Y., et al. "Product-based Neural Networks for User Response Prediction" (PNN). ICDM 2016. https://arxiv.org/abs/1611.00144
48. Guo, H., Yu, J., Liu, Q., Tang, R., Zhang, Y. "PAL: A Position-bias Aware Learning Framework for CTR Prediction in Live Recommender Systems." Huawei, RecSys 2019. https://doi.org/10.1145/3298689.3347033
49. Liu, B., et al. "Feature Generation by Convolutional Neural Network for Click-Through Rate Prediction" (FGCNN). Huawei, WWW 2019. https://arxiv.org/abs/1904.04447
50. Chen, B., et al. "Enhancing Explicit and Implicit Feature Interactions via Information Sharing for Parallel Deep CTR Models" (EDCN). Huawei, DLP-KDD 2021. https://dlp-k

dd.github.io/assets/pdf/DLP-KDD_2021_paper_12.pdf
51. Mao, K., et al. "FinalMLP: An Enhanced Two-Stream MLP Model for CTR Prediction." Huawei, AAAI 2023. https://arxiv.org/abs/2304.00902
52. Zhu, J., Liu, J., Yang, S., Zhang, Q., He, X. "BARS-CTR: Open Benchmarking for Click-Through Rate Prediction." Huawei, CIKM 2021. https://arxiv.org/abs/2009.05794
53. RecZoo. "FuxiCTR" repository README and model zoo. Accessed 2026. https://github.com/reczoo/FuxiCTR
54. Li, Y., et al. "PEAR: Personalized Re-ranking with Contextualized Transformer for Recommendation." Huawei, WWW 2022 Companion. https://arxiv.org/abs/2203.12267
55. Wang, J., Zhu, J., He, X. "Cross-Batch Negative Sampling for Training Two-Tower Recommenders." Huawei Noah's Ark, SIGIR 2021. https://arxiv.org/abs/2110.15154
56. "Multi-granularity Interest Retrieval and Refinement Network for Long-Term User Behavior Modeling in CTR Prediction" (MIRRN). Huawei, KDD 2025. https://arxiv.org/abs/2411.15005
57. Han, R., et al. "MTGR: Industrial-Scale Generative Recommendation Framework in Meituan." Meituan, 2025. https://arxiv.org/abs/2505.18654
58. "MTGRBoost: Boosting Large-scale Generative Recommendation Models in Meituan." Meituan, 2025. https://arxiv.org/abs/2505.12663
59. Cao, Y., et al. "Sampling Is All You Need on Modeling Long-Term User Behaviors for CTR Prediction" (SDIM). Meituan, CIKM 2022. https://arxiv.org/abs/2205.10249
60. Xi, D., Chen, Z., et al. "Modeling the Sequential Dependence among Audience Multi-step Conversions with Multi-task Learning in Targeted Display Advertising" (AITM). Meituan, KDD 2021. https://arxiv.org/abs/2105.08489
61. Shi, X., et al. "PIER: Permutation-Level Interest-Based End-to-End Re-ranking Framework in E-commerce." Meituan, WSDM 2023. https://arxiv.org/abs/2302.03487
62. "EGA-V2: An End-to-end Generative Framework for Industrial Advertising." Meituan, 2025. https://arxiv.org/abs/2505.17549
63. Zhao, X., Xia, L., Zhang, L., Ding, Z., Yin, D., Tang, J. "Deep Reinforcement Learning for Page-wise Recommendations." JD.com / Michigan State, RecSys 2018. https://arxiv.org/abs/1805.02343
64. "Sparse Meets Dense: Unified Generative Recommendations with Cascaded Sparse-Dense Representations" (COBRA). Baidu, 2025. https://arxiv.org/abs/2503.02453
65. Baidu PaddlePaddle. "PaddleRec" repository README (model list). Accessed 2026. https://github.com/PaddlePaddle/PaddleRec
66. Fan, M., Guo, J., Zhu, S., et al. "MOBIUS: Towards the Next Generation of Query-Ad Matching in Baidu's Search Engine." Baidu, KDD 2019 (not re-fetched). https://doi.org/10.1145/3292500.3330651
67. "Towards Large-scale Generative Ranking" (GenRank). Xiaohongshu, 2025. https://arxiv.org/abs/2505.04180
68. Zhang, C., et al. "NoteLLM: A Retrievable Large Language Model for Note Recommendation." Xiaohongshu, WWW 2024 (not re-fetched). https://arxiv.org/abs/2403.01744
69. Huang, T., Zhang, Z., Zhang, J. "FiBiNET: Combining Feature Importance and Bilinear Feature Interaction for Click-Through Rate Prediction," Sina Weibo, RecSys 2019, https://arxiv.org/abs/1905.09433 ; Wang, Z., She, Q., Zhang, J. "MaskNet: Introducing Feature-Wise Multiplication to CTR Ranking Models by Instance-Guided Mask," Sina Weibo, 2021, https://arxiv.org/abs/2102.07619

70. Kuaishou cross-references (see Chapter 20): Chang, J., et al. "TWIN," KDD 2023, https://arxiv.org/abs/2302.02352 ; Lian, X., et al. "Persia," KDD 2022, https://arxiv.org/abs/2111.05897 ; Deng, J., et al. "OneRec," 2025, https://arxiv.org/abs/2502.18965
71. Shen, W. "DeepCTR" repository README (model list). GitHub, accessed 2026. https://github.com/shenweichen/DeepCTR
72. Alibaba. "EasyRec" repository README (and TorchEasyRec). Alibaba Cloud PAI, accessed 2026. https://github.com/alibaba/EasyRec
73. Alibaba. "HybridBackend" repository README. Alibaba, accessed 2026. https://github.com/alibaba/HybridBackend
74. Gong, Y., et al. "Exact-K Recommendation via Maximal Clique Optimization." Alibaba, KDD 2019. https://arxiv.org/abs/1905.07089
75. Zhuang, T., Ou, W., Wang, Z. "Globally Optimized Mutual Influence Aware Ranking in E-Commerce Search." Alibaba, IJCAI 2018. https://arxiv.org/abs/1805.08524

28. News, Social and Content Platforms: From Click Co-visitation to Open Feed Protocols

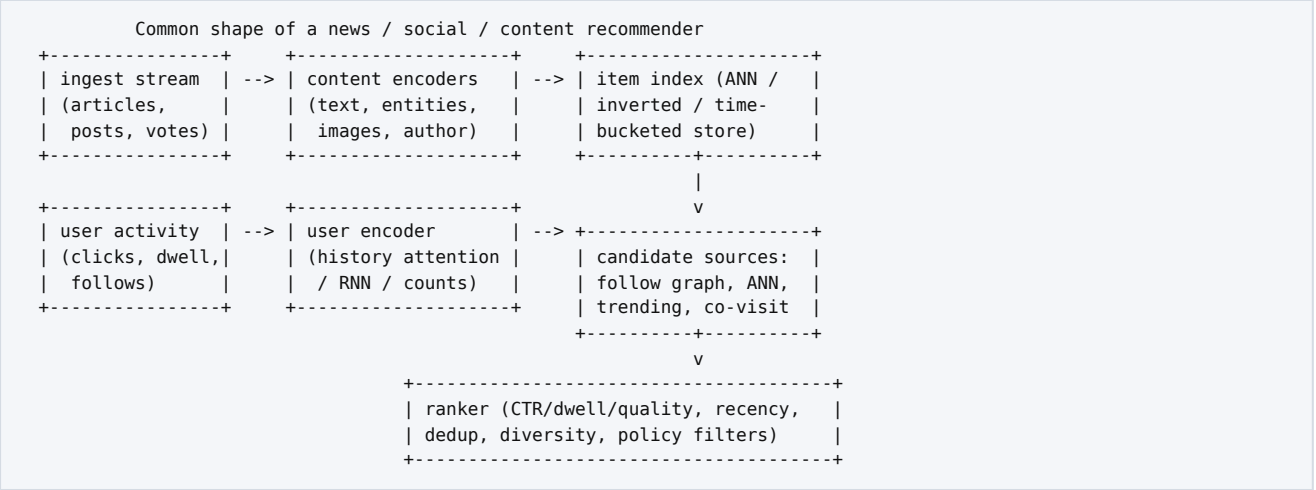
Overview

This chapter covers recommenders for platforms where content is perishable, abundant and often user-generated: news portals (Google News, Yahoo! JAPAN, Microsoft News, The New York Times), discussion and social platforms (Reddit, Quora, Medium, Snapchat), decentralized social networks (Bluesky, Mastodon), and community or streaming services (Substack, Discord, Twitch, Hulu, Disney+, Roku).

These systems share three structural problems that set them apart from the catalog recommenders in Chapters 18 (Netflix) and 24 (Amazon):

1. **Extreme item churn.** A news story or a social post is mostly worthless after hours or days. Item-ID collaborative filtering never collects enough interactions before the item dies. Content encoders such as text, entity and image models therefore do most of the work, and freshness is a primary ranking signal rather than a tie-breaker.
2. **A popularity/recency prior is a strong baseline.** Reddit's "hot" formula and Mastodon's trending scores are closed-form functions of votes and age. They still serve as candidate sources, cold-start fallbacks and sanity baselines, even in platforms with heavy ML.
3. **Governance and control matter.** Mastodon deliberately keeps a reverse-chronological home timeline. Bluesky moves ranking out of the platform into third-party "feed generators". Both are architecture decisions about who owns the ranking function, and they change the system design as much as any model does.

A note on sourcing for this edition. Network access during research was limited to GitHub-hosted material. Every Bluesky, Mastodon, Reddit-ranking and MIND fact below was verified against primary source code or dataset documentation fetched from GitHub [1]-[12]. Facts about older published papers (Google News, Yahoo! JAPAN, NRMS, DKN, NYT, Hulu) come from well-known peer-reviewed publications and are cited at their canonical DOIs or URLs [13]-[20], but were not re-fetched in this session. Those claims are kept to the papers' headline contributions, and any numbers from them are marked approximate. For several platforms (Snapchat Spotlight/Discover, Quora, Medium, Substack, Discord, Twitch, Disney+, Roku), primary engineering detail could not be retrieved. Those sections say so explicitly instead of filling gaps.



28.1 Google News: Scalable Online Collaborative Filtering (2007) and Click-Behavior Profiles (2010)

Problem

Google News aggregates stories from thousands of publishers. Its 2007 personalization paper by Das, Datar, Garg and Rajaram [13] set out the problem that still defines news recommendation: very large numbers of users and items, an item set that changes by the minute, and a need to produce recommendations in real time from implicit click feedback only.

Architecture and models

The paper combines three collaborative-filtering signals into one score per candidate story [13]:

- **MinHash clustering.** Users are represented by the sets of stories they clicked. MinHash, a locality-sensitive hash for Jaccard similarity, puts users with overlapping click sets into the same buckets. Clustering runs as a batch MapReduce job.
- **PLSI (Probabilistic Latent Semantic Indexing).** A latent-class model of user-story co-occurrence, also fitted offline with MapReduce.
- **Co-visitation counts.** These are updated online. Two stories are "co-visited" when the same user clicks both within a time window. The counts give an item-to-item signal that adapts within minutes as new stories gain traffic.

At serving time, the scores for a candidate story from the user's clusters (MinHash and PLSI) and from co-visitation with the user's recent clicks are blended linearly [13]. Cluster memberships are batch artifacts. The click counts per cluster and the co-visitation counts are kept in a large online key-value store (Bigtable in the paper), so new stories get a score as soon as clicks arrive.



Follow-up: content-based click-behavior profiles (2010)

Liu, Dolan and Pedersen [14] analyzed Google News click logs. They found that users' topical interests drift over time and are partly driven by the general news trend in their location. They built a Bayesian model of each user's "genuine" interest, separated from the local trend, and combined it with the existing CF method. The paper reports a substantial live click-through improvement over CF alone (roughly 30%; approximate, from the paper's live experiment) [14].

Lessons

- **Split the pipeline by how fast each part must change.** Slow structure (clusters) is computed in batch. Fast statistics (co-visitation, counts per cluster) are updated online. This is the template for most later news systems.
- **CF alone fails on fresh items and new users.** Even in 2010, Google moved to a hybrid with a content profile [14].

28.2 Yahoo! JAPAN: Embedding-Based News Recommendation for Millions of Users (KDD 2017)

Problem

Yahoo! JAPAN's homepage serves news to very large audiences, and article lifetimes are short. Okura et al. [15] replaced ID-based methods with learned representations, with a strict serving budget in mind.

Architecture

The paper's design has three parts [15]:

1. **Article encoder.** A variant of the denoising autoencoder learns distributed representations of articles from their content. It is trained so that articles in the same category end up closer together. Because the representation comes from content, a brand-new article gets an embedding the moment it is published.
2. **User encoder.** A recurrent neural network (the paper compares RNN variants, with a GRU-based model performing best) reads the user's browsing history as a sequence of article embeddings and outputs a user vector.
3. **Scoring.** Relevance is the inner product of user and article vectors. That makes serving a dot product over fresh candidates, and lets user vectors be computed ahead of time and cached.

```
article text --> [denoising AE] --> a_j  (computed at publish time)
browse history a_1..a_t --> [GRU] --> u  (refreshed as the user reads)
score(u, a_j) = <u, a_j>   -> top-k fresh articles, then business rules
```

Lessons

The paper's main contribution is a reusable split: precompute item vectors from content, update user vectors incrementally, and score with an inner product. That gives cheap serving with no cold start for items. It is the ancestor of the two-tower retrieval used everywhere today (see Chapter 2). The authors report that the deployed system improved engagement metrics over the previous production method in online tests [15]; the exact figures are omitted here.

28.3 Microsoft News and the MIND Ecosystem

Microsoft News (MSN News) is the most openly documented news recommender. That is mostly because Microsoft Research released a production-scale benchmark and a family of open models built on it.

28.3.1 The MIND dataset

MIND (MIcrosoft News Dataset) was collected from anonymized behavior logs of the Microsoft News website. It contains **about 160k English news articles and more than 15 million impression logs generated by 1 million users** [1]. Every article has title, abstract, body, category and entities. Each impression log holds the clicked and non-clicked items plus the user's click history before that impression [1]. The companion paper is Wu et al., "MIND: A Large-scale Dataset for News Recommendation", ACL 2020 [1][16].

Data construction details from the official documentation [2]:

- **Users:** about 1M, sampled at random from users with at least 5 news clicks. **MIND-small** has 50,000 users.
- **Time span:** six weeks, 12 October to 22 November 2019. The test set is the last week. The training set is the fifth week, with click histories built from the first four weeks. Validation is the last day of the fifth week.
- **Files:** `behaviors.tsv` (impression ID, user ID, time, ordered click history, and the impression list with `-1`/`-0` click labels in shuffled order). `news.tsv` (ID, category, subcategory, title, abstract, URL, and title and abstract entities linked to Wikidata). Also 100-dimensional **TransE** entity and relation embeddings from a knowledge graph.

The impression-log format matters for system design. It records **the slate the user actually saw**, not just positives. That allows impression-aware negative sampling, where the negatives are items shown but not clicked, and it lets evaluation use per-impression ranking metrics (AUC, MRR, nDCG@5, nDCG@10) instead of global AUC.

28.3.2 The model family: DKN, NPA, NAML, LSTUR, NRMS

Microsoft's open-source `recommenders` repository ships five news models, all marked as invented or contributed by Microsoft, each with a MIND quick-start notebook [3]:

Model	Core idea (per repo)	User modeling	Item modeling
DKN	Knowledge-graph entities fused with article text	Attention over clicked news	Word + entity (KG) CNN
NPA	Personalized attention	User-ID-conditioned attention	CNN over title words
NAML	Attentive multi-view learning	Attention over clicked news	Title, body, category views
LSTUR	Long- and short-term user representations	GRU (short) + user-ID embedding (long)	CNN over title
NRMS	Multi-head self-attention	Self-attention over clicked news	Self-attention over title words

DKN was originally published for Bing News [17]. NRMS (Wu et al., EMNLP-IJCNLP 2019) [18] became the default baseline in the field.

NRMS architecture (as documented in the reference implementation [4]):

```
title words (<=30) -> GloVe 300d -> multi-head self-attn (20 heads x 20 dim)
                   -> additive attention (hidden 200) -> news vector r_i
clicked news r_1..r_50 -> multi-head self-attn -> additive attention -> user u
candidate news r_c  -> score = <u, r_c>
training: 1 positive + 4 sampled negatives from same impression, softmax CE
```

Reference hyperparameters from the notebook [4]: `title_size=30`, `word_emb_dim=300`, `his_size=50`, `npratio=4`, `head_num=20`, `head_dim=20`, `attention_hidden_dim=200`, `dropout=0.2`, Adam with learning rate 1e-4, batch size 32, 5 epochs. On the small MINDdemo split (5k users), validation group AUC rises from 0.479 untrained to 0.613 after 5 epochs, and nDCG@10 from 0.270 to 0.363 [4]. These are toy-scale numbers. They show the shape of the learning curve, not the model's full-data quality.

Key trick: impression-level "npratio" training. The model is trained as a (1 + K)-way softmax. The K negatives are sampled from the same impression, so they are items the user saw and skipped. This is a listwise approximation that matches the serving task, which is ordering one slate.

28.3.3 Pre-trained language models and the MIND leaderboard

Later Microsoft work replaced GloVe/CNN news encoders with pre-trained language models (the "PLM-NR" line). The public training code (`PLM4NewsRec`) uses a BERT-based news encoder, distributed with Horovod across 4 GPUs, with MIND's `behaviors.tsv` sharded per GPU [5]. Efficient-Transformer user encoders (Fastformer, and "UniUM" universal user modeling) then pushed the public leaderboard.

The live MIND leaderboard (fetched October 2026) [1]:

Rank	Entry	Date	AUC	MRR	nDCG@5	nDCG@10
1	zetik	Jun 2026	0.7326	0.3780	0.4175	0.4735
2	newsbang	May 2026	0.7316	0.3768	0.4162	0.4722

Rank	Entry	Date	AUC	MRR	nDCG@5	nDCG@10
3	UniUM-Fastformer-Pretrain	Oct 2021	0.7304	0.3770	0.4180	0.4718
4	MINER	Sep 2021	0.7275	0.3724	0.4102	0.4661
6	UniUM-Fastformer	Aug 2021	0.7268	0.3745	0.4151	0.4684
29	UNBERT	Feb 2021	0.7207	0.3677	0.4041	0.4602
111	MIND_Organizer	--	0.7012	0.3482	0.3808	0.4377

The leaderboard has plateaued. A 2021 pretrained Fastformer entry is still within 0.003 AUC of the 2026 leaders. On MIND, gains now come from bigger PLMs and pretraining on user behavior, not from new user-encoder architectures.

28.3.4 Lessons from the MIND ecosystem

- **Content encoders are the backbone.** In news, the item tower carries most of the information, and upgrading it (CNN to PLM) gives larger gains than upgrading the user tower.
- **Cache news vectors and compute users online.** News vectors from a heavy PLM are computed once at publish time. Only the lightweight user aggregator runs per request, which keeps PLM-based news recommendation affordable.
- **Log impressions, not just clicks.** The MIND format (shown slate plus click labels plus history) is a good template for any feed's training logs.

28.4 The New York Times: Collaborative Topic Modeling

The New York Times R&D/data team described its recommender evolution in a 2015 engineering post [19]. The early system was content-based, matching keyword and topic tags of articles a reader had read to new articles. Its weakness was over-reliance on tags. The team moved to **Collaborative Topic Modeling (CTM)** (Wang and Blei, KDD 2011). CTM represents each article by LDA topic proportions learned from its text, then adjusts those proportions with offsets learned from readership, so an article's position reflects both what it says and who reads it [19]. Readers are placed in the same topic space according to what they read, and recommendations are articles close to the reader's position.

Design points worth carrying forward:

- **The content prior handles cold start.** A new article starts at its pure text-topic position, so it can be recommended at once. Readership data then moves it.
- **Interpretable latent space.** Topics are human-readable, which matters to an editorial organization that has to explain and audit what the algorithm promotes.

28.5 Reddit: Closed-Form Ranking at the Core

Problem

Reddit ranks posts inside communities (subreddits), on the home feed and in comment threads. Its original ranking functions are open source, and they remain a reference design for vote-driven ranking.

"Hot" ranking for posts

From the open-source `_sorts.pyx` [6]:

```
score   = ups - downs
order   = log10(max(|score|, 1))
sign    = +1 / 0 / -1 by sign of score
seconds = epoch_seconds(post_time) - 1134028003
hot     = round(sign * order + seconds / 45000, 7)
```

Design properties:

- **Log-compressed votes.** The first 10 net votes count as much as the next 90.
- **Time is additive and never decays a post's own score.** Each newer post gets a fixed bonus: 45,000 seconds (12.5 hours) of recency equals a 10x change in net votes. So the score can be computed once per vote and stored, and the sort index stays stable. There is no periodic rescoring job, which is a key systems property.

"Best" (confidence) ranking for comments

Comments use the **lower bound of the Wilson score interval** at about 80% confidence (z = 1.281551565545) on the upvote fraction [6]. A comment with 5 of 5 upvotes does not outrank one with 95 of 100. The ranking therefore gives credit for both quality and evidence, with no time term, since comment threads are short-lived contexts.

"Controversial"

`magnitude ** balance`, where magnitude = ups + downs and balance is the minority/majority vote ratio [6]. Large, evenly split vote totals score highest.

The modern ML home feed

Reddit's later home feed, built on an ML platform with feature stores, candidate generators and learned rankers, has been described in posts on the company's engineering subreddit, r/RedditEng. Those primary posts were not reachable for this edition, so no architecture or scale numbers are given here. The closed-form scores above are still part of the system design story. Votes and age are the strongest features of any learned ranker on Reddit, and a hot-style score is a natural candidate source and fallback.

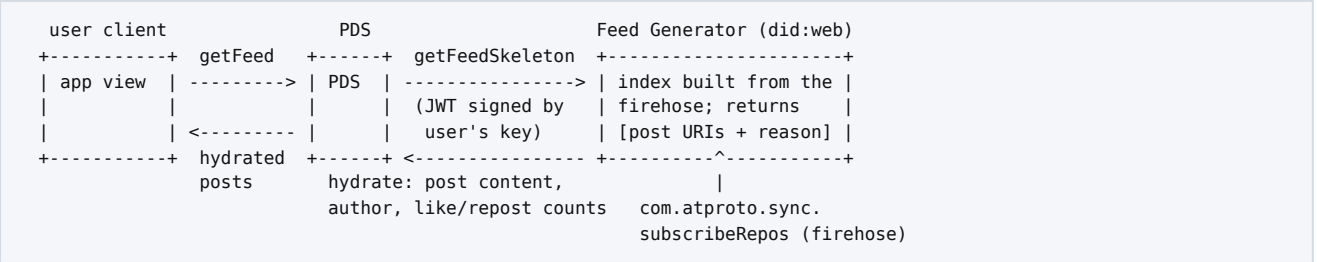
28.6 Bluesky: Recommendation as an Open Protocol Service

Problem

Bluesky, built on the AT Protocol, separates **hosting** (Personal Data Servers, PDS), **indexing** (AppView) and **ranking**. Anyone can publish a **custom feed**: a ranking algorithm users can subscribe to. This is a "marketplace of algorithms" in place of one platform ranker.

Architecture

From the official feed-generator starter kit [7]:



1. A feed generator is a service identified by a **DID**. Each algorithm it hosts is declared by a record in the creator's repo and addressed by that record's **at-uri** [7].
2. The user's PDS resolves the at-uri to the generator's DID document and calls `app.bsky.feed.getFeedSkeleton` at the listed endpoint. The request carries a **JWT signed with the user's repo signing key**, so the generator knows who is asking and can personalize [7].
3. The generator returns only a **skeleton**: a list of post URIs, each with an optional reason. The PDS (and eventually an AppView) **hydrates** the posts with content, author info and aggregate counts [7].
4. Generators build their own index by subscribing to the **firehose** (`com.atproto.sync.subscribeRepos`). They index only what their algorithm needs, and data older than about 48 hours can usually be discarded [7].

Protocol contract details (lexicons)

- `getFeedSkeleton` takes `feed` (at-uri), `limit` (1-100, default 50) and an opaque `cursor`. It returns `feed`, an optional `cursor` and an optional `reqId` (at most 100 chars, unique per request) [8].
- Each `skeletonFeedPost` has `post`, an optional `reason` (a repost reason or a pin reason) and an optional **`feedContext`** string of up to 2,000 characters. The client passes `feedContext` through, and it returns to the generator together with interactions [9].
- **Feedback loop:** `app.bsky.feed.sendInteractions` sends interactions with feed items "back to the feed generator that served them" [10]. Defined events include `requestMore`, `requestLess`, `clickthroughItem`, `clickthroughAuthor`, `clickthroughReposter`, `clickthroughEmbed`, `interactionSeen`, `interactionLike`, `interactionRepost`, `interactionReply`, `interactionQuote` and `interactionShare` [9].
- A generator advertises `acceptsInteractions` and a `contentMode` (`contentModeUnspecified` or `contentModeVideo`) [9].

The `feedContext` plus `reqId` design gives third-party rankers what in-house teams get from request logging. A generator can tag each served item with its own context string (candidate source, model version, experiment arm). It receives that tag back with the user's explicit "show more/less" or implicit "seen/clicked" events, so it can join impressions to outcomes without storing all client state.

Cursor design

The sample algorithm orders by `indexedAt DESC, cid DESC`, encodes the cursor as the millisecond timestamp of the last item, and filters `indexedAt < cursor` on the next page [11].

The docs recommend that cursors be **unique per item**, for example timestamp plus CID [7]. The minimal sample encodes only the timestamp, so posts sharing a millisecond can be skipped across pages. Feed authors are expected to fix this in production.

Lessons

- **Separate ranking from hydration.** Returning IDs only keeps ranking services cheap, stateless toward the client, and swappable.
- **Put the feedback channel in the protocol.** Without `sendInteractions`, third-party rankers would be blind to engagement.
- **Accept fragmentation of the ranking function as a deliberate product choice.** Moderation, spam filtering and quality become the responsibility of the shared hydration and labeling layers, not of each feed.

28.7 Mastodon: Chronological Fan-Out and Trend Detection

Mastodon is the instructive counterexample: a large social network whose **home timeline is not ranked by ML at all**. Its value for system designers is in how it does fan-out, filtering and trend detection.

Home timeline: push fan-out into Redis

From `FeedManager` [12]:

- Each home feed is a Redis sorted set, `feed:home:<account_id>`, of status IDs **scored by status ID**. Status IDs are time-ordered (Snowflake-like), so the sort is chronological.
- `MAX_ITEMS = 800` items per feed. `trim` uses `zremrangebyrank` to keep the newest 800.
- **Fan-out on write:** `push_to_home` inserts a new status into each follower's feed. It skips recipients who have not signed in recently, so storage and work for dormant accounts are not wasted. A streaming update is pushed only if `subscribed:timeline:<id>` shows a connected client.
- **Boost (reblog) dedup:** if the original or a boost of it is already in the newest `REBLOG_FALLOFF = 80` items, later boosts are held back in a per-original set. When the shown boost is deleted, the oldest held-back one is promoted.
- **Rebuild on demand:** `populate_home` rebuilds a feed from scratch. It takes up to 400 of the user's own posts and up to 400 per followed account, and skips accounts whose `last_status_at` is older than the feed's oldest item. `merge_into_home` on a new follow pulls up to 200 of that account's posts [12].
- **Rule-based filtering** (`filter_from_home`) drops posts involving blocks or mutes (author, booster or mentioned accounts), posts in languages the user did not select for that author, replies to people the user does not follow, and boosts from accounts whose boosts are hidden [12].

Federated and local timelines are separate views. The "federated" view shows only what the user's server knows about, because "there is no global shared state between all servers" [21].

Trending: a chi-square-like spike detector

Mastodon's only "algorithmic" surface is Trends. Both scorers use a squared-deviation test that rewards **acceleration** over an expected baseline.

Trending links [22]:

```
expected = accounts sharing the link one day earlier (min 1)
observed = accounts sharing it now
score    = 0                          if observed < 5 or expected > observed
         = (observed - expected)^2 / expected   otherwise
max_score kept with cooldown 2 days; decayed = max_score * 0.5^(age / 8h)
trend if decayed >= 1
```

Trending statuses [23]: `observed` = reblogs + favourites, `expected` = 1, threshold 5, score = (observed - 1)^2, **half-life 1 hour**, kept only while the decayed score is at least 0.3. Eligibility rules exclude replies, sensitive or content-warned posts, non-discoverable and silenced accounts, and invalid locales. Items whose score beats the rank-3 trend in their language, but which have not yet been approved, trigger a **human review request** [22] [23].

Lessons

- **Score against a baseline, not just counts.** Absolute counts favor the largest accounts. A per-item expected value (yesterday's level) catches spikes.
- **Different content needs different half-lives.** Posts use 1 hour and links use 8 hours.
- **Human review gates promotion.** Trends are allowed only after moderator approval or account trust. That is cheap safety for a small operator.

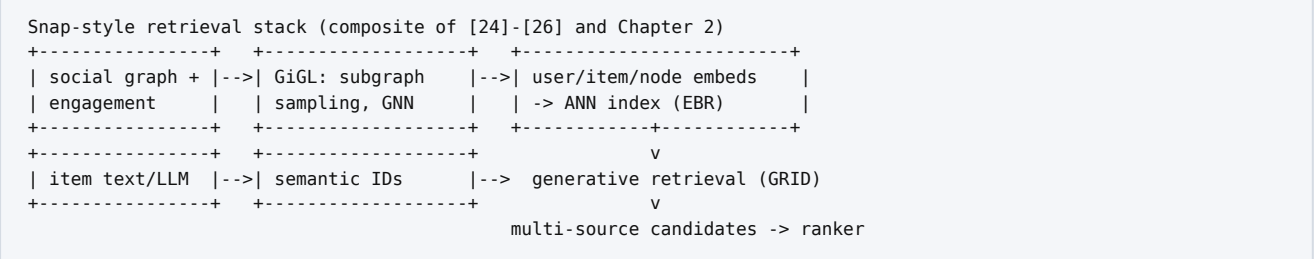
28.8 Snapchat: Embedding-Based Retrieval, GNNs and Generative Recommendation

What is verifiable

Snap's engineering blog describes embedding-based retrieval (EBR) with two-tower models for Spotlight, its short-video feed, and other surfaces. That blog was not reachable for this edition, so no Spotlight-specific numbers are given. Snap's open-source and published work shows the stack that powers its recommendation surfaces:

- **GiGL (Gigantic Graph Learning)** is Snap's open-source library for GNN training and inference at "very large (billion) scale", built for horizontal scaling across preprocessing, training and inference [24]. Its pipeline has six components: Config Populator, Data Preprocessor, Subgraph Sampler, Split Generator, Trainer and Inferencer [24]. It supports node classification and link prediction, runs on Vertex AI/Kubeflow with Dataflow/Dataproc for data work, and ships an example of link prediction with tabularized subgraph sampling and a contrastive loss [25]. The accompanying paper is "GiGL: Large-Scale Graph Neural Networks at Snapchat" (KDD 2025) [24].

- **Friend-recommendation EBR.** Chapter 2 of this Atlas documents Snap's GNN-embedding EBR for friend recommendation and its ANN query expansion. That source reaches beyond friends-of-friends and reports gains in reciprocated communication for more than 800M MAU.
- **GRID** is Snap Research's open framework for generative recommendation with semantic IDs [26]. LLM text embeddings of items are quantized into hierarchical semantic IDs with Residual K-means, RVQ or RQ-VAE, and a TIGER-style transformer generates the next item's ID tokens. It is described in "Generative Recommendation with Semantic IDs: A Practitioner's Handbook" (CIKM 2025) [26].



Lessons

At Snap the social graph is a first-class training signal. GNN embeddings serve as user and item representations across friend, content and possibly ads surfaces, and Snap invests in reusable graph infrastructure (GiGL) instead of building one model per surface.

28.9 Streaming and Long-Tail Content Platforms: Hulu, Disney+, Roku, Twitch

Hulu

Hulu's research group published **CF-NADE** (Zheng et al., ICML 2016), a neural autoregressive collaborative-filtering model [20]. It models a user's ratings as a product of conditionals, shares parameters across rating levels, and reported state-of-the-art results on MovieLens and Netflix benchmarks at the time [20]. In production, Hulu's recommendations have to handle TV series as a unit (episode versus show), live TV, and a mix of subscription tiers. Detailed production architecture was not available from primary sources for this edition.

Disney+, Roku and Twitch

No primary engineering source on the recommender architecture of Disney+, Roku or Twitch could be retrieved for this edition, so no design claims are made about them. From a system-design view, the problem shapes differ:

Platform	Dominant problem shape	Implication for design
Disney+	Small, curated, franchise-heavy catalog	Row/page assembly and editorial control dominate (Ch. 15)
Roku	Cross-app aggregator on a TV device	Recs span third-party channels; few, coarse signals per user
Twitch	Live items exist only while streaming	Candidate set depends on availability; repeat consumption
Hulu	Episodic TV, live TV, mixed tiers	Show-level vs episode-level modeling; continue-watching

This table is analysis from the problem structure. It does not report the companies' disclosed designs.

28.10 Quora, Medium, Substack and Discord

These platforms have described parts of their feeds publicly over the years, including Quora's ML-ranked question feed, Medium's topic- and follow-based feed, Substack's network recommendations between publications, and Discord's server discovery. Their primary engineering posts were not reachable for this edition, so this Atlas gives no architectural specifics for them. Readers should go to each company's engineering blog. The general pattern from the systems above applies: follow-graph candidates, plus content-similarity candidates, plus a learned or rule-based ranker with freshness. That pattern is a reasonable starting hypothesis, not a reported fact.

28.11 Cross-Platform Comparison

System	Year	Item signal	User signal	Candidate generation	Ranking	Freshness handling	Source
Google News	2007	Click sets	Click history	MinHash, PLSI clusters, co-visitation	Linear blend of CF scores	Online counts, co-visit window	[13]
Google News	2010	Topic of content	Bayesian interest vs local trend	CF + content profile	Hybrid score	Trend modeling	[14]
Yahoo! JAPAN	2017	Denoising-AE article vector	GRU over history	Inner product over fresh set	Dot product + rules	Item vectors at publish	[15]
Microsoft News	2019-21	Self-attention / PLM title encoder	Attention over 50 clicks	Dot product	NRMS-style softmax over impression	Cached news vectors	[3][4] [18]
NYT	2015	LDA topics + reader offsets	Topic position of reader	Nearest in topic space	Similarity	Content prior for new items	[19]
Reddit (classic)	2005+	Votes	None (per-community)	Subreddit sets	Closed-form hot / Wilson	Additive time term	[6]
Bluesky	2023+	Anything (generator-defined)	Requester DID via JWT	Generator's own firehose index	Generator-defined	~48h index window typical	[7]-[11]
Mastodon	2016+	None (chronological)	Follow graph	Fan-out on write	Status ID (time)	800-item cap; trends decay	[12] [22] [23]
Snap	2023-25	GNN / LLM / semantic IDs	Graph + engagement	EBR, generative	Learned ranker	Not reported here	[24]-[26]

Key Takeaways for System Designers

1. **Content encoders beat IDs when items are perishable.** Every successful news system here (Google 2010, Yahoo! JAPAN, Microsoft, NYT) moved to content

representations so new items can be scored at publish time. Compute item vectors once, cache them, and keep the per-request user computation light.
2. **Split by rate of change.** Batch-learned structure (clusters, topic models, encoders) plus online counters (co-visitation, vote totals, trend windows) is the durable architecture, from Google News 2007 to Mastodon trends.
3. **Closed-form scores are infrastructure, not legacy.** Reddit's hot score can be computed once per vote with no rescoring jobs. Mastodon's spike detector compares to a baseline with tunable half-lives. Both make cheap candidate sources, fallbacks and features for learned rankers.
4. **Train on impressions.** MIND's format (shown slate, click labels, prior history) enables impression-aware negatives and per-impression metrics. Log what was shown, with request IDs.
5. **Benchmark plateaus point to where gains come from.** On MIND, 2021 pretrained-Fastformer entries are within about 0.3 AUC points of the 2026 leaders. Larger language-model item encoders and behavior pretraining are now the levers, not new attention variants.
6. **Protocol-level ranking needs protocol-level feedback.** Bluesky's `feedContext` + `reqId` + `sendInteractions` gives third-party rankers attribution and feedback loops. Anyone designing an open or multi-tenant ranking system needs the same three pieces: an opaque per-item context, a request ID, and an event return channel.
7. **Fan-out with guardrails.** Mastodon's push model caps storage per feed (800), skips inactive users, deduplicates boosts in the recent window, and rebuilds feeds on demand. That is a fine pattern even beneath an ML ranker, as the in-network source.
8. **Different content types need different decay.** Mastodon decays posts with a 1-hour half-life and links with 8 hours. Reddit's hot score trades 12.5 hours for 10x votes. Tune recency per surface and item type, not globally.
9. **Governance is architecture.** Chronological-only (Mastodon), marketplace-of-feeds (Bluesky), and centralized ML (Snap, Microsoft) are different allocations of control. They change where moderation, spam defense and quality signals must live.
10. **Shared graph infrastructure pays off.** Snap's GiGL shows the value of one billion-scale GNN platform producing embeddings for many retrieval surfaces, instead of one model per product.

References

1. Microsoft. "MIND: MIcrosoft News Dataset" (homepage and leaderboard), 2020-2026. https://msnews.github.io/ (fetched via https://raw.githubusercontent.com/msnews/msnews.github.io/master/index.html)
2. Microsoft. "MIND dataset introduction" (format and construction documentation), 2020. https://github.com/msnews/msnews.github.io/blob/master/assets/doc/introduction.md
3. Microsoft / Recommenders Team (Linux Foundation AI & Data). "Recommenders" repository README (news algorithms DKN, LSTUR, NAML, NPA, NRMS). https://github.com/recommenders-team/recommenders
4. Recommenders Team. "NRMS: Neural News Recommendation with Multi-Head Self-Attention" quick-start notebook on MIND. https://github.com/recommenders-team/recommenders/blob/main/examples/00_quick_start/nrms_MIND.ipynb

5. Wu, C. "PLM4NewsRec" (code for PLM-empowered news recommendation). https://github.com/wuch15/PLM4NewsRec
6. Reddit. `r2/r2/lib/db/_sorts.pyx` (hot, confidence and controversy sorts), reddit open-source archive. https://github.com/reddit-archive/reddit/blob/master/r2/r2/lib/db/_sorts.pyx
7. Bluesky Social. "ATProto Feed Generator" starter kit README. https://github.com/bluesky-social/feed-generator
8. Bluesky Social. Lexicon `app.bsky.feed.getFeedSkeleton`. https://github.com/bluesky-social/atproto/blob/main/lexicons/app/bsky/feed/getFeedSkeleton.json
9. Bluesky Social. Lexicon `app.bsky.feed.defs` (skeletonFeedPost, interaction events, generatorView). https://github.com/bluesky-social/atproto/blob/main/lexicons/app/bsky/feed/defs.json
10. Bluesky Social. Lexicon `app.bsky.feed.sendInteractions`. https://github.com/bluesky-social/atproto/blob/main/lexicons/app/bsky/feed/sendInteractions.json
11. Bluesky Social. Sample feed algorithm `whats-alf.ts`. https://github.com/bluesky-social/feed-generator/blob/main/src/algos/whats-alf.ts
12. Mastodon gGmbH. `app/lib/feed_manager.rb` (home feed fan-out). https://github.com/mastodon/mastodon/blob/main/app/lib/feed_manager.rb
13. Das, A., Datar, M., Garg, A., Rajaram, S. "Google News Personalization: Scalable Online Collaborative Filtering." Google, WWW 2007. https://dl.acm.org/doi/10.1145/1242572.1242610 (not re-fetched this session)
14. Liu, J., Dolan, P., Pedersen, E. R. "Personalized News Recommendation Based on Click Behavior." Google, IUI 2010. https://dl.acm.org/doi/10.1145/1719970.1719976 (not re-fetched this session)
15. Okura, S., Tagami, Y., Ono, S., Tajima, A. "Embedding-based News Recommendation for Millions of Users." Yahoo! JAPAN, KDD 2017. https://dl.acm.org/doi/10.1145/3097983.3098108 (not re-fetched this session)
16. Wu, F., Qiao, Y., Chen, J.-H., Wu, C., Qi, T., Lian, J., Liu, D., Xie, X., Gao, J., Wu, W., Zhou, M. "MIND: A Large-scale Dataset for News Recommendation." Microsoft, ACL 2020. https://aclanthology.org/2020.acl-main.331/
17. Wang, H., Zhang, F., Xie, X., Guo, M. "DKN: Deep Knowledge-Aware Network for News Recommendation." Microsoft Research Asia, WWW 2018. https://arxiv.org/abs/1801.08284 (not re-fetched this session)
18. Wu, C., Wu, F., Ge, S., Qi, T., Huang, Y., Xie, X. "Neural News Recommendation with Multi-Head Self-Attention." Microsoft / Tsinghua, EMNLP-IJCNLP 2019. https://aclanthology.org/D19-1671/ (not re-fetched this session)
19. Spangher, A. "Building the Next New York Times Recommendation Engine." The New York Times (NYT Open blog), 2015. https://open.nytimes.com/ (post index; exact article URL not verified this session, locate by title)
20. Zheng, Y., Tang, B., Ding, W., Zhou, H. "A Neural Autoregressive Approach to Collaborative Filtering." Hulu, ICML 2016. https://arxiv.org/abs/1605.09477 (not re-fetched this session)
21. Mastodon. User documentation, `content/en/user/network.md` (local and federated timelines). https://github.com/mastodon/documentation/blob/main/content/en/user/network.md
22. Mastodon gGmbH. `app/models/trends/links.rb` (trending links scorer). https://github.com/mastodon/mastodon/blob/main/app/models/trends/links.rb

23. Mastodon gGmbH. `app/models/trends/statuses.rb` (trending statuses scorer). https://github.com/mastodon/mastodon/blob/main/app/models/trends/statuses.rb
24. Snap Research. "GiGL: Gigantic Graph Learning" repository and paper "GiGL: Large-Scale Graph Neural Networks at Snapchat," KDD 2025. https://github.com/snap-research/GiGL ; https://arxiv.org/abs/2502.15054
25. Snapchat. GiGL (active development repository). https://github.com/Snapchat/GiGL
26. Ju, C., Collins, L., Neves, L., Kumar, B., Wang, L. Y., Zhao, T., Shah, N. "Generative Recommendation with Semantic IDs: A Practitioner's Handbook" (GRID). Snap Research, CIKM 2025. https://github.com/snap-research/GRID ; https://arxiv.org/abs/2507.22224

29. LLMs in Recommender Systems (2023-2026): Rankers, Features, Agents, Distillation and Production Deployments

Between 2023 and 2026, large language models (LLMs) entered almost every stage of the recommendation stack. The integration did not take a single form, and the mainstream industrial pattern is not "ask ChatGPT what to recommend". Public work splits into five families, each with different costs:

1. **LLM as the recommender.** Recommendation is posed as text-to-text generation, as in P5, TALLRec, LLaRA, LlamaRec and LinkedIn's 360Brew.
2. **LLM as a feature or embedding producer.** The LLM runs offline or nearline to enrich item and user representations, which a conventional model consumes. Examples are KAR, LLMRec, RLMRec, NoteLLM, HLLM, LinkedIn's LLaMA-3 retriever and Spotify's narratives.
3. **LLM-native generative retrieval over item tokens.** Items become semantic IDs and a pre-trained LLM is continued-pre-trained to "speak" them. Examples are TIGER, PLUM at YouTube, Text2Tracks at Spotify, and OneRec-Think and OpenOneRec at Kuaishou.
4. **LLM as the conversational or agentic front end.** The LLM parses intent, plans and calls conventional recommender tools. Examples are InteRecAgent, Chat-REC, RecMind, Spotify AI DJ and AI Playlist.
5. **LLM as teacher.** A large model's knowledge or rankings are distilled into small, servable students. Examples are DLLM2Rec, SLMRec, SLIM and instruction distillation, and the same pattern appears in "teacher foundation model plus distilled students" at Meta and Netflix.

The economic tension is the theme of the chapter. A production ranker scores thousands of candidates per request within tens of milliseconds at tens of thousands of QPS, while a 7B-parameter LLM needs on the order of tens to hundreds of milliseconds for one forward pass over a long prompt. Almost every successful industrial design is therefore an answer to one question: where can the LLM's cost be amortized?

> **Sourcing note.** For this chapter the live web-search budget was exhausted and outbound HTTP to arxiv.org, company blogs and aws.amazon.com was blocked from the research environment. Primary sources read directly in this session were the GitHub repositories (READMEs, code and prompt templates) of P5, OpenP5, TALLRec, LLMRank, LLaRA, LC-Rec, CLLM4Rec, LLMRec and Microsoft RecAI (InteRecAgent, Knowledge Plugin, RecLM-emb), and two curated paper indexes (the TOIS survey list and the LLM4Rec survey list). Industry facts for LinkedIn, YouTube, Spotify, Netflix and Kuaishou come from the companion chapters of this Atlas (Chapters 6, 17, 18, 20, 23 and 25), whose authors read the primary sources. Some

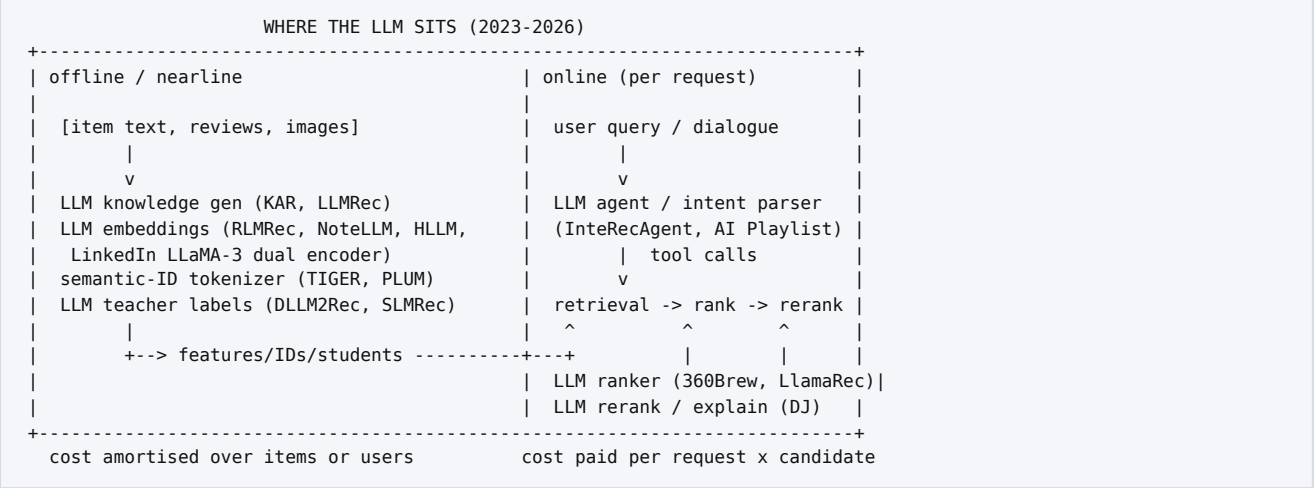
> paper summaries are marked "(as recalled)". For those, the paper's existence, title and URL were verified from a paper index but the full text was not re-read. Amazon Rufus, Instacart and YouTube's Gemini consumer features could not be verified from any primary source in this session, and Section 29.9 treats them accordingly.

29.1 A taxonomy: where and how the LLM is used

The TOIS survey by Lin et al. organises the field along two axes [40]:

- **Where** the LLM sits in the pipeline: feature engineering, feature encoder, scoring or ranking function, user interaction, or pipeline controller.
- **How** it is adapted: whether the LLM is tuned (frozen or fine-tuned), and whether a conventional recommendation model (CRM) is kept at inference time.

The companion index by Wu et al. ("A Survey on Large Language Models for Recommendation") splits the field into discriminative uses (LLM as encoder) and generative uses (LLM as generator) [41]. A third survey focuses on LLM-based generative recommendation [42]. The CHIANGEL index alone lists several hundred papers by 2024 [40].



The single most useful design question is which side of the vertical line the LLM lives on. On the left, cost scales with catalog size and refresh rate. On the right, it scales with traffic multiplied by candidates.

29.2 LLM as the recommender: the 2022-2024 academic foundations

29.2.1 P5: "Recommendation as Language Processing" (RecSys 2022)

Problem. Every recommendation task (rating, sequential next-item, explanation, review summarisation, direct top-k) had its own architecture and loss. P5 asked whether one text-to-text model could learn all of them with a single language-modeling objective [1][2].

Architecture. P5 converts "user-item interactions, item metadata, and user reviews" into natural-language sequences and pre-trains a T5 encoder-decoder (Small and Base checkpoints are released for Amazon Beauty, Toys, Sports and Yelp) on a multi-task mixture of **personalized prompts** [2]. The released template file defines five pre-training prompt families plus a zero-shot family [2]:

Family	Task	Example source template (from the repo)	Metrics
1	Rating	"Which star rating will user_{} give item_{} ? (1 being lowest and 5 being highest)"	RMSE/MAE
2	Sequential	"I find the purchase history list of user_{} : {} I wonder what is the next item to recommend to the user ."	HR, NDCG, MRR
3	Explanation	generate review-style explanation for (user, item)	BLEU/ROUGE
4	Review	summarise or predict rating from review text	ROUGE / RMSE
5	Direct ("traditional")	pick or generate the item for a user, possibly from candidates	HR, NDCG
Z	Zero-shot	unseen prompts / cold items	varies

Users and items appear as tokens such as `user_23` and `item_7391`. Those IDs are split by the SentencePiece tokenizer into sub-tokens, and P5 adds whole-word embeddings so the model knows which sub-tokens form one ID [2].

Lessons.

- P5 established the "instruction plus personalised prompt" interface that almost all later work reuses.
- It also exposed the **item-ID problem**. Arbitrary numeric IDs tokenised as text leak spurious structure: `item_7391` and `item_7392` share sub-tokens without being related. The follow-up "How to Index Item IDs for Recommendation Foundation Models" compared random, sequential, collaborative and semantic indexing [3][61]. OpenP5 turned the setup into a benchmark with 10 datasets and 3 item-indexing methods, on T5 and LLaMA-2 backbones [4].
- P5's scale is tiny by industrial standards. The OpenP5 datasets range from 943 users (ML-100K) to 277,631 users and 112,394 items (Yelp) [4].

29.2.2 TALLRec: aligning an LLM with recommendation via LoRA (RecSys 2023)

Problem. In-context ChatGPT recommendation performed close to random on binary "will the user like this item?" tasks, because a general LLM has never been trained on recommendation data. TALLRec asks how cheaply an LLM can be aligned [5].

Architecture.

- Base model: **LLaMA-7B**, with LoRA adapters. The code builds on alpaca-lora [6].
- Two tuning stages:
    1. **Alpaca instruction tuning**, the general instruction-following LoRA.

2. **Rec-tuning**, a second LoRA on recommendation instructions of the form "user liked these items and disliked these; will the user enjoy target item? Answer Yes/No" [5][6].

- Scoring uses the probability of the "Yes" token, which gives an AUC-ready score.

Results. Few-shot AUC with only 16, 64 or 256 training samples, from the official repository [6]:

Model	Movie 16	Movie 64	Movie 256	Book 16	Book 64	Book 256
GRU4Rec	49.07	49.87	52.89	48.95	49.64	49.86
SASRec	50.43	50.48	52.25	49.48	50.06	50.20
DROS-BERT	50.21	51.71	53.94	50.07	48.98	50.20
TALLRec	**67.24**	**67.48**	**71.98**	**56.36**	**60.39**	**64.38**

Lessons. The LLM's world knowledge yields strong **data efficiency**: ID models are near-random at 256 samples, while TALLRec is well above. A 7B model can be aligned on a single GPU. However, the evaluation is pointwise (binary) with few candidates, so it says nothing about full-corpus retrieval. That gap motivated grounding work such as BIGRec, LlamaRec and LC-Rec.

29.2.3 LLMs as zero-shot rankers (LLMRank, ECIR 2024)

Hou et al. formulated ranking as instruction following. The prompt is built from two natural-language patterns, the user's **sequential history** and the **retrieved candidates**, and the LLM (ChatGPT in the paper) outputs a ranked list [7]. The repository states three findings [7]:

1. **Order blindness.** LLMs "struggle to perceive the order" of interaction histories. Recency-focused prompting and in-context examples trigger order awareness.
2. **Biases.** LLM rankers show **position bias**, where the candidate's slot in the prompt changes its rank, and **popularity bias**. Prompt design and **bootstrapping** reduce both. Bootstrapping means shuffling candidate order across several calls and aggregating.
3. **Zero-shot ability is real but conditional.** It is strongest on candidates retrieved by multiple candidate generators with different strategies.

Early evaluations of ChatGPT as a recommender, "Is ChatGPT a Good Recommender?" [14] and "Uncovering ChatGPT's Capabilities in Recommender Systems" [15], reached similar conclusions. InstructRec instead fine-tuned FLAN-T5-XL (3B) on recommendation instructions [13]. Zero-shot quality trails tuned ID models on accuracy, while explanation and dialogue are strengths. FaiRLLM showed that LLM recommendations vary with sensitive user attributes stated in the prompt, which makes fairness evaluation necessary [16].

Design implication. Bootstrapping multiplies cost by the number of permutations. Position bias also means that a listwise LLM reranker is **not permutation-invariant**, which complicates logging and counterfactual evaluation (Chapter 12).

29.2.4 Grounding: from "generate a title" to "pick a real item"

A generative LLM can name items that do not exist. Four solutions recur:

Approach	Mechanism	Example
Post-hoc grounding	Generate text, then map it to the nearest real item embedding	BIGRec (LLaMA-7B, LoRA) [11] (as recalled)
Candidate verbalizer	Retrieve about 20 candidates with a sequential model, label them with letters, read the LLM's next-token distribution over the letter tokens in one forward pass	LlamaRec (LLaMA-2 7B, QLoRA) [10] (as recalled)
Collaborative tokens in vocabulary	Add user and item ID tokens and train their embeddings while freezing the LM	CLLM4Rec (LinkedIn + UVA, WWW 2024) [9]
Learned semantic indices plus alignment tasks	Vector-quantize item embeddings into non-conflicting indices, then teach the LLM via alignment tasks	LC-Rec (LLaMA-7B) [8]
Hybrid prompts with CF embeddings	Project SASRec/GRU4Rec item embeddings into the LLM token space and mix them with text tokens; curriculum learning	LLaRA (LLaMA-2 7B, SIGIR 2024) [12]

The CLLM4Rec repository describes its tokenizer extension concretely. With original vocabulary size N, the token `user_i` gets ID N+i and `item_j` gets ID N+I+j, with user and item tokens treated as atomic words. Only the user and item embeddings are updated during pre-training, while the transformer and the original token embeddings stay frozen. A recommendation-oriented fine-tuning stage then generates multiple items over the **whole item space** "without hallucination" [9].

LLaRA's updated numbers show the effect of the injected CF signal [12]:

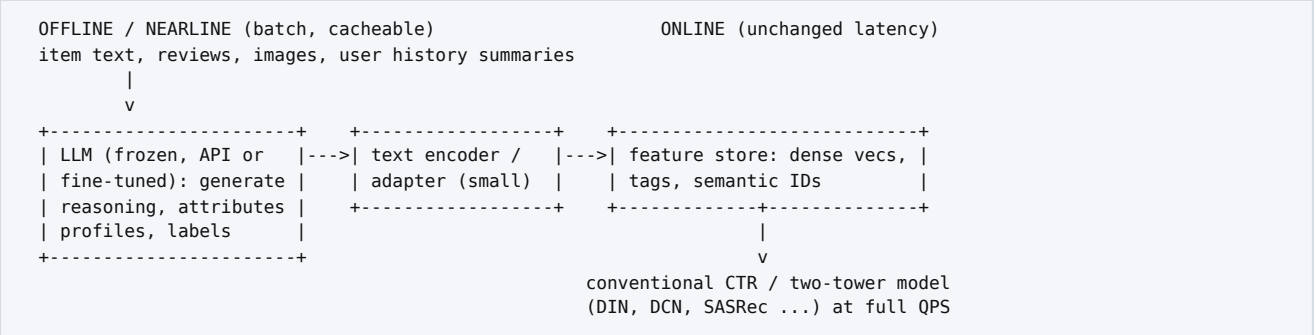
LLaRA variant (CF encoder)	MovieLens HR@1	Steam HR@1	LastFM HR@1
LLaRA (GRU4Rec)	0.4000	0.4916	0.4918
LLaRA (Caser)	0.4211	0.4621	0.4836
LLaRA (SASRec)	0.4526	0.5051	0.5246

LLaRA also reports a **ValidRatio** of about 0.95-0.996, the fraction of outputs that name a valid candidate [12]. Production designs need this metric: every LLM ranker should log the share of malformed or out-of-catalog outputs.

LC-Rec's README states the two key ideas directly [8]. The first is a "learning-based vector quantization method with uniform semantic mapping, which can assign meaningful and non-conflicting IDs". The second is tuning tasks that make the LLM "deeply integrate language and collaborative semantics". This is the academic precursor of the semantic-ID tokenizers used at YouTube and Kuaishou (Section 29.4).

29.3 LLM-generated features, knowledge and embeddings (the most deployed pattern)

The pattern that ships most easily keeps the production model and adds LLM outputs as features. Cost scales with the number of items (or user-refresh events), not with traffic.



29.3.1 KAR: open-world knowledge augmentation (Huawei, 2023)

KAR ("Towards Open-World Recommendation with Knowledge Augmentation from Large Language Models") prompts a frozen LLM (ChatGPT in the paper listing [17]) for two kinds of knowledge. **Reasoning knowledge** covers user preferences, for example "what factors does this user care about in movies?". **Factual knowledge** covers items. KAR uses "factorization prompting", which first enumerates the scenario-specific factors that matter and then asks per factor. A small **hybrid-expert adaptor** compresses the generated text encodings into augmentation vectors that any CTR model can consume. Because the knowledge is pre-computed and stored, online inference cost is unchanged (as recalled). The follow-up "Efficient and Deployable Knowledge Infusion for Open-World Recommendations via LLMs" [37] and a decoding-acceleration paper for "industrial deployable LLM-based recommender systems" [36] focus on making that offline generation cheap enough to refresh. Rec-GPT4V extends knowledge generation to item images with large vision-language models [39]. Both titles appear in [40]. The acceleration work speeds up long-form knowledge generation for very large user and item populations (as recalled).

29.3.2 LLMRec: LLM-based graph augmentation (WSDM 2024)

LLMRec (HKU with Baidu) uses an LLM to augment the user-item interaction graph in three ways, as the repository states [18]:

1. **Reinforcing user-item edges**, where the LLM samples plausible positive and negative items from candidates.
2. **Enhancing item node attributes**, where the LLM generates missing attributes.
3. **Profiling user nodes**, where the LLM writes user profiles from history.

The released artifacts show the cost pattern. Augmented text was generated once with `gpt-3.5-turbo-0613` and embedded with `text-embedding-ada-002`, and the recommender was then trained on the augmented data in a separate stage [18]. A denoising step prunes unreliable augmented edges.

29.3.3 Representation alignment: RLMRec, ONCE, LLM-CF

- **RLMRec** (WWW 2024) aligns CF embeddings with LLM-generated user and item profile representations through contrastive and generative alignment, as a plug-in for existing recommenders [19] (as recalled).
- **ONCE** (WSDM 2024) combines an open-source LLM fine-tuned as a content encoder with a closed-source LLM used as a prompt-based data enricher, for content-based (news) recommendation [20] (as recalled).

- **LLM-CF**, "Large Language Models Enhanced Collaborative Filtering" [60], and "Breaking the Barrier", an LLM-built inferential knowledge graph for industrial recommendation [38], follow the same "generate offline, consume online" structure.

29.3.4 LLM item and user encoders in production-scale research

- **NoteLLM (Xiaohongshu, WWW 2024)** compresses each note into a special token from the LLM's last layer. That embedding is trained contrastively on co-occurrence pairs and used for item-to-item retrieval, with hashtag and category generation as auxiliary tasks [56].
- **HLLM (ByteDance, 2024)** uses an **item LLM** to read item text and produce an item embedding, and a **user LLM** to model the sequence of item embeddings and predict the next one. The released HLLM-1B is initialised from TinyLlama-1.1B and HLLM-7B from Baichuan2-7B. On the Books benchmark, HLLM-1B reaches R@10 9.28 and HLLM-7B 9.39, versus 6.50 for HSTU-large [55] (Chapter 20).
- **Microsoft RecLM-emb** fine-tunes LMs for "versatile text-based item retrieval". It trains on ten query types: user history, item-to-item, user summary, full or sparse attributes, misspelled names, vague conditions, negative attributes, and history plus explicit query. The point is that general text-embedding models "often obtain suboptimal performance" for item retrieval [21][23].
- **Kuaishou LEARN** ("Knowledge Adaptation from Large Language Model to Recommendation for Practical Industrial Application") [35] freezes an LLM as an item content encoder and trains a twin-tower adaptor on collaborative signals. This avoids catastrophic forgetting from fine-tuning the LLM on ID data (as recalled).

29.3.5 LinkedIn: LLaMA-3 dual-encoder retrieval for the Feed (2025)

This is one of the clearest public descriptions of an LLM inside a high-QPS production retrieval stage [44] (Chapter 23):

- **Model.** A fine-tuned **LLaMA 3** decoder used as a **dual encoder**. Member text (profile plus activity) and post text are encoded separately and pooled into embeddings.
- **Scale.** About **2,000 candidates from a pool of hundreds of millions of posts**, in a budget of a few milliseconds at several thousand QPS (approximate).
- **Trick 1: numbers in prompts.** Numeric features, such as counts and recency, worked better when **quantized into discrete tokens** than when written as raw digits.
- **Trick 2: who benefits.** The biggest online gains went to **new members** with sparse networks. This is the cold-start regime, where text understanding beats ID and graph signals.

```
member profile + recent activity text --> LLaMA-3 (fine-tuned) --pool--> member vec
                                                                            |
post text (nearline, on create/update) --> LLaMA-3 (shared or twin) --pool--> post vec
                                                                            |
                        ANN / GPU exhaustive search over hundreds of millions  v
                        ---------------------------------------> ~2,000 candidates
                                                                  -> Feed ranker
```

The cost works because post embeddings are computed once per post, nearline, while member embeddings are refreshed asynchronously. The online path is a vector search.

29.4 LLM-native generative retrieval: semantic IDs meet pre-trained LLMs

Chapter 6 covers semantic IDs in depth. Here the focus is the step where a **pre-trained language model** is adapted to an item-token vocabulary.

29.4.1 YouTube PLUM (2025)

PLUM ("Adapting Pre-trained Language Models for Industrial-scale Generative Recommendations") is Google's most detailed public account of LLM-based recommendation at YouTube [46] (Chapter 17):

```
1. Item tokenization: Semantic IDs v2 (multimodal content emb + co-engagement)
      -> RQ-style multi-level codes, tuned for coverage and low collision
2. Continued pre-training (CPT) of a Gemini-family LLM on:
      user watch sequences as SID tokens + SID<->text alignment (titles, topics)
3. Task fine-tuning for generative retrieval:
      user context + history (SIDs + text) -> decode SIDs of next videos
4. Serving: beam decoding of SIDs -> videos, as one retrieval source
      feeding the existing ranking stack
```

- The paper reports that the generative retriever beat a heavily optimised production large-embedding retrieval model in live YouTube experiments. It also studies scaling with model size and compute (approx.) [46].
- **Cost profile shift.** PLUM uses far fewer item-specific parameters than embedding-table retrieval, but much more compute per request. Memory-bound becomes compute-bound (TPU) [46].
- PLUM builds on TIGER's RQ-VAE semantic IDs [47] and on YouTube's finding that semantic IDs improve ranking generalisation at a fixed table size (Chapter 17).

Earlier Google work used LLMs to **name user interest journeys** (for example "learning to bake sourdough") from activity, with prompt tuning on a small set of human-labelled journeys. This was framed as research toward explainable, goal-aligned recommendations, not as a launch [48].

29.4.2 Spotify Text2Tracks (2025)

Text2Tracks fine-tunes an encoder-decoder LLM to **generate track identifiers directly from a text prompt**, such as "upbeat 80s synthpop for running" [50]. The key finding is about the **ID scheme**. Random integer IDs, or titles used as IDs, perform worse than **semantic IDs built by hierarchical clustering of collaborative-filtering embeddings**. Short IDs are also far cheaper to decode than titles, and they avoid hallucinated tracks (Chapter 25).

29.4.3 Kuaishou OneRec-Think and OpenOneRec (2025)

- **OneRec-Think** is built on **Qwen3**, with the vocabulary extended by semantic-ID tokens. Training has three stages: **itemic alignment** (mapping item tokens into text space), **reasoning activation** (chain-of-thought SFT), and **reasoning enhancement**

(RL with a reward that allows multiple valid answers). It reports a further **+0.159% app stay time** online [52] (Chapter 6).

- **OpenOneRec** released OneRec-1.7B and 8B on Qwen3 backbones. These were trained on about 33B tokens, and "-Pro" variants on about 130B tokens including industrial data. Post-training runs multi-task SFT, then **on-policy distillation to restore general reasoning**, then RL. The release also includes **RecIF-Bench**, 8 tasks over 100M interactions from 200K users [53].
- **MiniOneRec** is a community reproduction. It builds RQ-VAE / RQ-KMeans semantic IDs from text embeddings, runs SFT with language alignment and GRPO with a rank-aware reward, and decodes with **constrained beam search** through a logits processor so that only valid IDs are emitted [54].
- **Snap GRID** is an open framework that quantizes LLM text embeddings of items into hierarchical semantic IDs, using residual k-means, RVQ or RQ-VAE, for TIGER-style generation [57].

Lesson. Kuaishou's "restore general reasoning" step is the giveaway. Fine-tuning an LLM heavily on item tokens erodes its language ability, which defeats the reason for starting from an LLM. Alignment (SID-to-text) and anti-forgetting steps are now standard.

29.5 LLM as the ranker in production: LinkedIn 360Brew (2025)

360Brew is the most aggressive public "LLM is the ranker" system [43] (Chapter 23).

- **Problem.** Remove "hundreds of thousands of handcrafted features" and many per-surface rankers. Fix cold start, hand-crafted interactions and poor domain transfer.
- **Formulation.** Each member's profile and history are verbalized as text. Ranking becomes **many-shot in-context learning**: P(interaction on new entity | instruction, profile, history). For comparison with binary rankers, the model answers yes or no, and the **logits of the answer tokens** are the score.
- **Model. 150B-parameter decoder-only**, based on the **Mixtral 8x22B** MoE, further trained on LinkedIn entity and interaction data across 5+ surfaces. It is evaluated on **30+ tasks across 8+ surfaces**, built by a small team over **9 months**, and labelled "pre-production" (V1.0).
- **Results.** Comparable to or better than production models offline **without task-specific fine-tuning**. Performance improved with 3x more data over five releases, with larger models, and with longer context (normalised to an 8K-token baseline). The largest gains are in cold start (history cut from 100 to 5 interactions) and on unseen surfaces. Quality also degrades more slowly as test data ages.
- **Prompt steering.** Instructions can include steering such as "30% weight to profile match, 70% to historical activity". This form of product control is impossible in a feature-based ranker.

```
Instruction: predict apply/view/dismiss for the Question job (optional steering)
Member Profile: position, company, location
Past job interactions:  applied: [title, location, company, description] ...
                        viewed:  [...]   dismissed: [...]
Question: Will the member apply to: [job text]
Answer:   "The member will apply"   <-- score = logit(answer tokens)
```

Cost. The V1.0 report does not discuss serving a 150B MoE at Feed QPS. LinkedIn's parallel retrieval work uses a much smaller LLaMA-3 dual encoder [44]. A reasonable inference, not a published fact, is that 360Brew acts as a teacher or as a ranker for high-value, lower-QPS surfaces. LinkedIn open-sourced **Liger-Kernel** (Triton RMSNorm, RoPE, SwiGLU, fused linear cross-entropy). It reports about +20% multi-GPU training throughput and about 60% memory reduction, which helps make LLM-scale training affordable [45].

29.6 Conversational and agentic recommenders

29.6.1 Why conversational recommendation changed in 2023

Pre-LLM conversational recommender systems (CRS) such as UniCRS (KDD 2022, DialoGPT-small with knowledge-enhanced prompts) [29] were hard to build and evaluated on small dialogue datasets. LLMs made the dialogue layer nearly free. "Rethinking the Evaluation for Conversational Recommendation in the Era of LLMs" (EMNLP 2023) argued that static turn-level metrics undercount LLM ability. It proposed **interactive evaluation with LLM-simulated users** (iEvaLM) [28] (as recalled). LLM-simulated users became a standard evaluation tool.

29.6.2 InteRecAgent (Microsoft RecAI): LLM brain, recommender tools

InteRecAgent is the reference design for "LLM orchestrates, recommender executes" [21] [22]. From the repository [59]:

- **Components.**
  1. An LLM as the conversational agent.
  2. An **item profile table** (id, title, tags, description, price, release date, visit counts, and so on).
  3. A **query module**, a SQL tool over the item table.
  4. A **retrieval module** with at least two tools: a **SQL tool** for hard conditions ("popular sports games") and an **item-similarity tool** for implicit intent ("games similar to Call of Duty").
  5. A **ranking module**, which can be a traditional recommender, that orders candidates by a schema (popularity, similarity or preference) using the user profile of `prefer` and `unwanted` items.
- **Principle.** "Neither the LLM nor the recommendation tools will be updated or modified." Integration is purely through tool interfaces [22].
- Paper details (as recalled): a **candidate memory bus** passes candidate sets between tools without putting thousands of item IDs in the prompt, a **plan-first execution** strategy makes one plan call rather than many ReAct rounds, and **RecLlama** is a 7B model fine-tuned to imitate GPT-4 tool plans for cheaper serving [22].

```
user: "something like Hollow Knight but co-op, under $20"
  |
  v
LLM planner --(1 plan call)--> [SQL: price<20 AND tag~co-op] -> candidate bus
                               [item-sim: Hollow Knight]     -> candidate bus
                               [rank: preference(user profile)] -> top-k
  |                                                              |
  +<------------------------ results (IDs -> titles) ---------------+
  v
LLM writes the answer + explanation; profile memory updated (prefer/unwanted)
```

The RecAI repository also covers **Knowledge Plugins** (DOKE), which inject collaborative item-to-item and user-to-item knowledge extracted from MF and SASRec embeddings into prompts without fine-tuning [58]. It further includes **RecExplainer**, which uses LLMs as surrogate models to explain an embedding-based recommender, and **RecLM-eval**, an evaluation service [21].

29.6.3 Other agentic designs

System	Year	Role of LLM	Key idea	Source
Chat-REC	2023	ChatGPT with recommender candidates injected into the prompt	Interactive, explainable rec; conversation turns become queries for a CRM	[24]
RecMind	2023 (NAACL 2024)	LLM agent with planning over tools	"Self-inspiring" planning that keeps alternative reasoning paths	[25] (as recalled)
InteRecAgent	2023	Planner and dialogue; tools do retrieval and rank	Candidate bus, plan-first, RecLlama	[22]
Agent4Rec	2023	LLM agents *simulate users*	Generative user agents with profile, memory and action modules for offline evaluation	[26] (as recalled)
AgentCF	2023 (WWW 2024)	Both users and items as agents	Agents "collaboratively learn" by reflecting on mismatched choices	[27]
Spotify AI DJ	2023	Narration only; recommender selects	LLM scripts grounded by editors, plus TTS	[49]
Spotify AI Playlist	2024	Prompt -> intents -> retrieval	Refinable via follow-up prompts	[49][50]

29.6.4 Spotify AI DJ and AI Playlist: LLM at the edge of a mature stack

Spotify's DJ (beta from February 2023, US and Canada) is a useful counterexample to "LLM as recommender" (Chapter 25):

1. The **existing personalization stack** chooses segments of tracks: recent favourites, rediscoveries and new music.
2. **Generative AI (OpenAI LLM technology)** writes short commentary. Spotify's music editors supply facts and style through prompts, which grounds the output.
3. A **dynamic AI voice** (from the Sonantic acquisition) speaks the script [49].

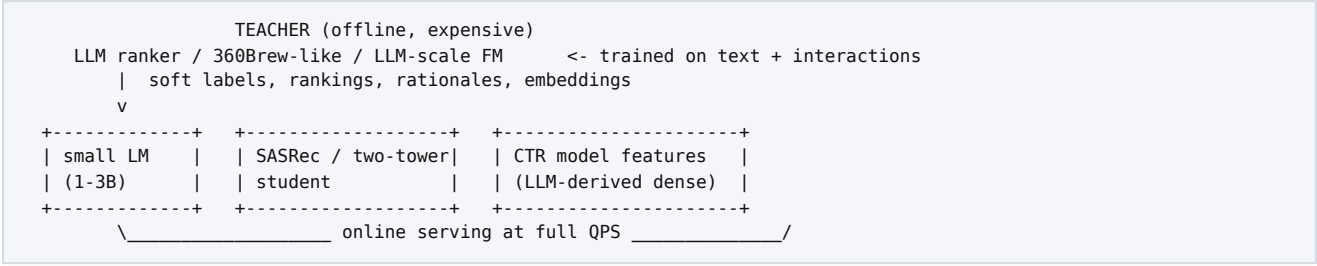
The LLM explains and frames, while the recommender selects. This keeps quality controllable and latency predictable, and it adds a new feedback signal: tapping DJ to

"switch the vibe" is a segment-level negative. AI Playlist (2024) lets the user type a prompt. Spotify Research's Text2Tracks [50] shows one way to map prompts to tracks generatively, though Spotify has not published the production system's internals.

29.7 Knowledge distillation from LLMs

Distillation is the bridge from LLM quality to recommender latency. Three flavours appear:

1. **Ranking distillation into conventional sequential models.**
   - "Distillation Matters: Empowering Sequential Recommenders to Match the Performance of LLMs" (DLLM2Rec) [30] distills an LLM recommender's rankings and embeddings into SASRec/GRU-style students, using importance-aware ranking distillation and collaborative embedding distillation. The student keeps conventional inference cost (as recalled).
   - "Can Small Language Models be Good Reasoners for Sequential Recommendation?" (SLIM, WWW 2024) [32] distills step-by-step reasoning from a large teacher into a small LM, whose rationales are then encoded as features (as recalled).
2. **Model compression of the LLM recommender itself.**
   - SLMRec [31] studies how much of a large LLM-based sequential recommender's depth is needed, and distills into a much smaller model (as recalled).
   - "Instruction Distillation Makes LLMs Efficient Zero-shot Rankers" [33] distills a pairwise or listwise ranking teacher into a pointwise FLAN-T5-XL (3B) student.
   - POD ("Prompt Distillation for Efficient LLM-based Recommendation", CIKM 2023) distills discrete prompts into continuous prompt vectors on a T5-small (60M) backbone, which shortens inputs [34].
3. **Foundation teacher, many students (industry).** Meta's GEM is described as trained at LLM scale and is used offline as a teacher whose knowledge is transferred to serving models (Chapters 16 and 19) [62]. Netflix's foundation model is a large, centrally trained model whose embeddings and fine-tuned variants feed many downstream systems [51]. Neither is a text LLM. Both borrow the LLM operating model of pre-training once and distilling or fine-tuning many times.



29.8 Industry deployments: what is publicly documented

Company	System	LLM role	Stage	Model / scale (public)	Cost strategy	Ref
LinkedIn	Feed LLM retrieval (2025)	Dual-encoder embeddings	Retrieval	LLaMA 3; ~2k of hundreds of millions; few ms; thousands QPS (approx.)	Nearline item embedding, ANN	[44]
LinkedIn	360Brew (2025)	Text ranker / foundation	Ranking	150B MoE (Mixtral 8x22B); 30+ tasks	Pre-production; likely teacher (inference)	[43]
LinkedIn + UVA	CLLM4Rec (2024)	Collaborative LLM	Gen. rec	GPT-2 with user and item tokens	Freeze LM, train ID tokens	[9]
YouTube	PLUM (2025)	SID-speaking LLM	Retrieval	Gemini-family LLM, CPT + FT	Beam decode SIDs; small item tables	[46]
Spotify	AI DJ (2023)	Narration	Presentation	OpenAI LLM + Sonantic TTS	Recommender selects; LLM frames	[49]
Spotify	Text2Tracks (2025)	Prompt -> track IDs	Retrieval (research)	Fine-tuned enc-dec LLM; CF semantic IDs	Short IDs, no title resolution	[50]
Netflix	Foundation model (2025)	LLM-style FM (not text LLM)	Representation	Hundreds of billions of interactions; 300M+ members	Central training, downstream reuse	[51]
Kuaishou	OneRec-Think / OpenOneRec (2025)	LLM backbone (Qwen3) + SIDs	End-to-end gen.	1.7B / 8B; ~33B-130B tokens	RL post-training; reasoning restore	[52] [53]
ByteDance	HLLM (2024)	Item and user LLMs	Seq. rec	1B / 7B	Item embeddings cacheable	[55]
Xiaohongshu	NoteLLM (2024)	Item embedding LLM	I2I retrieval	LLM compressed-token embedding	Offline item embeddings	[56]
Huawei (as recalled)	KAR (2023)	Knowledge generation	Features	ChatGPT-class LLM + adaptor	Precompute and cache knowledge	[17]
Microsoft	RecAI / InteRecAgent	Agent and tools	Conversational	GPT-4 class or RecLlama 7B (as recalled)	Plan-first; candidate bus	[21] [22]

29.8.1 Lessons visible across deployments

- **LLM compute lives where it can be cached.** LinkedIn embeds posts nearline, Spotify keeps the LLM at narration, KAR precomputes knowledge, and PLUM spends compute on retrieval but replaces large embedding tables. The most expensive design (360Brew) is the one not yet described as serving at Feed scale.
- **Cold start is the most consistent win.** It shows up in LinkedIn's retrieval (new members), 360Brew (5-interaction histories), TALLRec (16-256 samples) and Netflix (metadata gating for new titles).
- **IDs remain essential.** Systems that drop collaborative signals entirely underperform. Successful designs inject CF through semantic IDs (PLUM, Text2Tracks, LC-Rec), CF embeddings (LLaRA, CLLM4Rec) or CF knowledge in prompts (Knowledge Plugin).

29.9 Deployments we could not verify in this session

The topic brief lists **Amazon Rufus** (Amazon's generative shopping assistant), **Instacart** (LLM use in search and discovery), and **YouTube's Gemini-powered consumer features**. Primary sources for these could not be fetched from the research environment, and no companion chapter of this Atlas documents them. To avoid stating unverified numbers, this chapter gives no architecture or scale figures for them. Structurally, public descriptions of shopping assistants match the agentic pattern of Section 29.6. A conversational LLM grounds its answers in catalog, review and Q&A data through retrieval, and product suggestions come from the existing search and recommendation stack. The serving challenge is LLM inference at peak shopping traffic, not model quality alone. Readers should consult the companies' engineering blogs directly. For YouTube, PLUM [46] is the verified primary source on Gemini-family models in recommendation.

29.10 Cost and latency trade-offs

29.10.1 Where the cost lands

```
cost per day  ~=  (#requests) x (#candidates scored by LLM) x (tokens per candidate)
                  x (cost per token)                          <- per-request LLM ranker
              vs. (#items + #users refreshed) x (tokens) x (cost per token)
                                                              <- offline feature/embedding
```

Pattern	Scales with	Typical latency impact	Freshness	Main risk
Offline LLM features (KAR, LLMRec)	catalog size x refresh rate	none online	hours to days	stale or hallucinated knowledge
Nearline LLM embeddings (LinkedIn, NoteLLM)	new items + user updates	ANN only	minutes	embedding drift across model versions
LLM generative retrieval (PLUM, OneRec)	requests x beam x decode steps	high; needs TPU/GPU serving	per request	invalid IDs; compute cost
LLM pointwise ranker (360Brew, TALLRec)	requests x candidates x prompt length	very high	per request	QPS infeasible beyond small candidate sets
LLM listwise reranker (LLMRank, LlamaRec)	requests x (prompt with ~20 candidates)	high; one call	per request	position bias; non-determinism
LLM agent (InteRecAgent, DJ)	conversational turns	seconds acceptable	per turn	tool errors; prompt injection
Distilled student (DLLM2Rec, SLIM)	training only	none extra	retrain cadence	teacher bias inherited

29.10.2 Engineering tricks that recur

1. **Score with logits, not generation.** TALLRec ("Yes" probability), 360Brew (answer-token logits) and LlamaRec (letter-index verbalizer) all turn ranking into one forward pass with no autoregressive decoding [6][43][10].

2. **Shrink the candidate set first.** LLM rankers run on about 10-100 candidates from a conventional retriever. LLMRank found zero-shot ability strongest on candidates from multiple diverse retrievers [7].
3. **Share the user prefix.** When every candidate shares the user history prefix, KV caching of the prefix amortizes it. This is the same idea as the DCAT and M-FALCON-style tricks in Chapter 6, and as KV-cache serving of user sequences at ByteDance (Chapter 20).
4. **Constrain decoding.** Prefix tries or logits processors over valid semantic IDs prevent invalid items and reduce beam waste [54]. Track a ValidRatio-style metric [12].
5. **Quantize numbers into tokens.** Raw digits are poorly modelled, and bucketed tokens work better [44].
6. **Parameter-efficient tuning.** LoRA (TALLRec, LLaRA) or QLoRA (LlamaRec) keeps per-domain adapters cheap. CLLM4Rec freezes the LM and trains only ID embeddings [9].
7. **Plan once.** Agents that plan once and execute tools avoid multi-round ReAct latency. Fine-tuned small planners (RecLlama-style) replace GPT-4-class planners [22].
8. **Restore general ability after domain tuning.** OpenOneRec's on-policy distillation step [53] keeps the LLM useful for explanation and dialogue.
9. **Precompute explanations.** Generate LLM text per (item, segment) offline, filter it with quality and safety classifiers, and serve it from cache (Spotify pattern, Chapter 25).
10. **Keep a guardrail layer.** LLM outputs pass through the same business-rule, diversity and safety re-rankers as any model (Chapter 15). FaiRLLM-style audits check sensitivity to user attributes [16].

29.11 Key takeaways for system designers

1. **Decide which side of the cache line the LLM lives on.** Offline and nearline LLM use (features, embeddings, semantic IDs, teacher labels) is cheap to adopt and leaves online latency unchanged. Per-request LLM ranking needs a very small candidate set or a high-value, low-QPS surface.
2. **Start with LLM features and embeddings, not LLM ranking.** KAR, LLMRec, NoteLLM and LinkedIn's LLaMA-3 retriever show measurable gains with no change to the serving stack.
3. **Inject collaborative signal.** Pure-text LLMs underperform ID models on warm items. Use semantic IDs, CF embeddings in prompts, or added ID tokens.
4. **Invest in the item tokenizer.** PLUM, Text2Tracks, LC-Rec and OneRec all depend on good, collision-free, refreshable item codes. Treat the tokenizer as infrastructure with versioning.
5. **Exploit cold start.** The most consistent LLM wins are for new users, new items and new surfaces. Route traffic accordingly, for example LLM-heavy paths for sparse-history users.
6. **Use the LLM's strengths at the edges.** Intent parsing, conversation, explanation and narration (AI DJ, InteRecAgent) add product value while the recommender keeps control of selection.

7. **Distill.** Train the expensive model once and serve students: sequential models (DLLM2Rec), small LMs (SLIM, SLMRec) or feature producers. This is the same pattern as industrial foundation models at Meta and Netflix.
8. **Measure LLM-specific failure modes.** Track position bias, popularity bias, invalid-output rate, attribute sensitivity (fairness) and non-determinism across calls. Bootstrapped permutations cost money, so budget for them.
9. **Plan for evaluation with simulated users, but do not trust it alone.** LLM-simulated users (iEvaLM, Agent4Rec) are good for conversational iteration. Online A/B tests remain the arbiter (Chapter 12).
10. **Expect convergence.** By 2025-2026, generative recommenders (Chapter 6) and LLMs are merging: Qwen3-backbone OneRec, Gemini-backbone PLUM and Mixtral-backbone 360Brew. The open problems are serving economics and freshness, not whether LLMs can recommend.

References

1. Shijie Geng, Shuchang Liu, Zuohui Fu, Yingqiang Ge, Yongfeng Zhang (Rutgers). "Recommendation as Language Processing (RLP): A Unified Pretrain, Personalized Prompt & Predict Paradigm (P5)." RecSys 2022. https://arxiv.org/abs/2203.13366
2. Shijie Geng et al. "P5" official repository (README, prompt templates `src/all_amazon_templates.py`, checkpoints). GitHub, 2022. https://github.com/jeykigung/P5
3. Wenyue Hua, Shuyuan Xu, Yingqiang Ge, Yongfeng Zhang. "How to Index Item IDs for Recommendation Foundation Models." 2023. https://arxiv.org/pdf/2305.06569.pdf
4. Shuyuan Xu, Wenyue Hua, Yongfeng Zhang (agiresearch). "OpenP5: An Open-Source Platform for Developing, Training, and Evaluating LLM-based Recommender Systems." Paper and repository, 2023-2024. https://arxiv.org/pdf/2306.11134.pdf ; https://github.com/agiresearch/OpenP5
5. Keqin Bao, Jizhi Zhang, Yang Zhang, Wenjie Wang, Fuli Feng, Xiangnan He. "TALLRec: An Effective and Efficient Tuning Framework to Align Large Language Model with Recommendation." RecSys 2023. https://arxiv.org/abs/2305.00447
6. TALLRec official repository (README, main results table, training scripts). GitHub, 2023. https://github.com/SAI990323/TALLRec
7. Yupeng Hou, Junjie Zhang, Zihan Lin, Hongyu Lu, Ruobing Xie, Julian McAuley, Wayne Xin Zhao. "Large Language Models are Zero-Shot Rankers for Recommender Systems." ECIR 2024. https://arxiv.org/abs/2305.08845 ; repository: https://github.com/RUCAIBox/LLMRank
8. Bowen Zheng et al. (RUC AI Box). "Adapting Large Language Models by Integrating Collaborative Semantics for Recommendation" (LC-Rec). ICDE 2024. https://ieeexplore.ieee.org/abstract/document/10597986/ ; repository: https://github.com/RUCAIBox/LC-Rec
9. Yaochen Zhu, Liang Wu, Qi Guo, Liangjie Hong, Jundong Li (UVA and LinkedIn). "Collaborative Large Language Model for Recommender Systems" (CLLM4Rec). WWW 2024. https://arxiv.org/abs/2311.01343 ; repository: https://github.com/yaochenzhu/LLM4Rec

10. Zhenrui Yue et al. "LlamaRec: Two-Stage Recommendation using Large Language Models for Ranking." PGAI@CIKM 2023. https://arxiv.org/abs/2311.02089
11. Keqin Bao et al. "A Bi-Step Grounding Paradigm for Large Language Models in Recommendation Systems" (BIGRec). 2023. https://arxiv.org/abs/2308.08434
12. Jiayi Liao et al. "LLaRA: Large Language-Recommendation Assistant." SIGIR 2024. https://arxiv.org/abs/2312.02445 ; repository (updated results, 2024.7): https://github.com/ljy0ustc/LLaRA
13. Junjie Zhang et al. "Recommendation as Instruction Following: A Large Language Model Empowered Recommendation Approach" (InstructRec). 2023. https://arxiv.org/abs/2305.07001
14. Junling Liu et al. "Is ChatGPT a Good Recommender? A Preliminary Study." 2023. https://arxiv.org/abs/2304.10149
15. Sunhao Dai et al. "Uncovering ChatGPT's Capabilities in Recommender Systems." RecSys 2023. https://arxiv.org/abs/2305.02182
16. Jizhi Zhang et al. "Is ChatGPT Fair for Recommendation? Evaluating Fairness in Large Language Model Recommendation" (FaiRLLM). RecSys 2023. https://arxiv.org/abs/2305.07609
17. Yunjia Xi et al. (SJTU and Huawei). "Towards Open-World Recommendation with Knowledge Augmentation from Large Language Models" (KAR). 2023. https://arxiv.org/abs/2306.10933
18. Wei Wei, Xubin Ren, Jiabin Tang, Qingyong Wang, Lixin Su, Suqi Cheng, Junfeng Wang, Dawei Yin, Chao Huang (HKU and Baidu). "LLMRec: Large Language Models with Graph Augmentation for Recommendation." WSDM 2024. https://arxiv.org/abs/2311.00423 ; repository: https://github.com/HKUDS/LLMRec
19. Xubin Ren et al. "Representation Learning with Large Language Models for Recommendation" (RLMRec). WWW 2024. https://arxiv.org/abs/2310.15950
20. Qijiong Liu et al. "ONCE: Boosting Content-based Recommendation with Both Open- and Closed-source Large Language Models." WSDM 2024. https://arxiv.org/abs/2305.06566
21. Microsoft. "RecAI: Leveraging Large Language Models for Next-Generation Recommender Systems" (InteRecAgent, Knowledge Plugin, RecLM-emb, RecLM-gen, RecExplainer, RecLM-eval). GitHub, 2023-2024. https://github.com/microsoft/RecAI
22. Xu Huang, Jianxun Lian, et al. (Microsoft). "Recommender AI Agent: Integrating Large Language Models for Interactive Recommendations" (InteRecAgent). 2023. https://arxiv.org/abs/2308.16505
23. Yuxuan Lei, Jianxun Lian, et al. (Microsoft). "Aligning Language Models for Versatile Text-based Item Retrieval" (RecLM-emb). 2024. https://arxiv.org/abs/2402.18899
24. Yunfan Gao et al. "Chat-REC: Towards Interactive and Explainable LLMs-Augmented Recommender System." 2023. https://arxiv.org/abs/2303.14524
25. Yancheng Wang et al. "RecMind: Large Language Model Powered Agent For Recommendation." NAACL 2024 Findings. https://arxiv.org/abs/2308.14296
26. An Zhang et al. "On Generative Agents in Recommendation" (Agent4Rec). 2023. https://arxiv.org/abs/2310.10108
27. Junjie Zhang et al. "AgentCF: Collaborative Learning with Autonomous Language Agents for Recommender Systems." WWW 2024. https://arxiv.org/abs/2310.09233
28. Xiaolei Wang et al. "Rethinking the Evaluation for Conversational Recommendation in the Era of Large Language Models." EMNLP 2023. https://arxiv.org/abs/2305.13112

29. Xiaolei Wang et al. "Towards Unified Conversational Recommender Systems via Knowledge-Enhanced Prompt Learning" (UniCRS). KDD 2022. https://arxiv.org/abs/2206.09363
30. Yu Cui et al. "Distillation Matters: Empowering Sequential Recommenders to Match the Performance of Large Language Model" (DLLM2Rec). 2024. https://arxiv.org/abs/2405.00338
31. Wujiang Xu et al. "SLMRec: Empowering Small Language Models for Sequential Recommendation." 2024. https://arxiv.org/abs/2405.17890
32. Yuling Wang et al. "Can Small Language Models be Good Reasoners for Sequential Recommendation?" (SLIM). WWW 2024. https://arxiv.org/abs/2403.04260
33. Weiwei Sun et al. "Instruction Distillation Makes Large Language Models Efficient Zero-shot Rankers." 2023. https://arxiv.org/abs/2311.01555
34. Lei Li, Yongfeng Zhang, Li Chen. "Prompt Distillation for Efficient LLM-based Recommendation" (POD). CIKM 2023. https://lileipisces.github.io/files/CIKM23-POD-paper.pdf
35. Jian Jia et al. (Kuaishou). "Knowledge Adaptation from Large Language Model to Recommendation for Practical Industrial Application" (LEARN). 2024. https://arxiv.org/abs/2405.03988
36. "A Decoding Acceleration Framework for Industrial Deployable LLM-based Recommender Systems." 2024. https://arxiv.org/abs/2408.05676
37. "Efficient and Deployable Knowledge Infusion for Open-World Recommendations via Large Language Models." 2024. https://www.arxiv.org/abs/2408.10520
38. "Breaking the Barrier: Utilizing Large Language Models for Industrial Recommendation Systems through an Inferential Knowledge Graph." 2024. https://arxiv.org/abs/2402.13750
39. "Rec-GPT4V: Multimodal Recommendation with Large Vision-Language Models." 2024. https://arxiv.org/abs/2402.08670
40. Jianghao Lin et al. "How Can Recommender Systems Benefit from Large Language Models: A Survey." ACM TOIS, 2024; paper index repository. https://dl.acm.org/doi/10.1145/3678004 ; https://github.com/CHIANGEL/Awesome-LLM-for-RecSys
41. Likang Wu et al. "A Survey on Large Language Models for Recommendation." 2023; paper index repository. https://arxiv.org/abs/2305.19860 ; https://github.com/WLiK/LLM4Rec-Awesome-Papers
42. Lei Li et al. "Large Language Models for Generative Recommendation: A Survey and Visionary Discussions." 2023. https://arxiv.org/abs/2309.01157
43. 360Brew Team (H. Firooz, M. Sanjabi, et al.), LinkedIn. "360Brew: A Decoder-only Foundation Model for Personalized Ranking and Recommendation." 2025. https://arxiv.org/abs/2501.16450 ; report and README: https://github.com/LLMPersonalization/360Brew-V1
44. LinkedIn. "Large Scale Retrieval for the LinkedIn Feed using Causal Language Models." arXiv 2510.14223, 2025 (AAAI 2026). https://arxiv.org/abs/2510.14223
45. LinkedIn. "Liger Kernel: Efficient Triton Kernels for LLM Training." GitHub, 2024-2025. https://github.com/linkedin/Liger-Kernel
46. Google / YouTube. "PLUM: Adapting Pre-trained Language Models for Industrial-scale Generative Recommendations." 2025. https://arxiv.org/abs/2510.07784
47. Shashank Rajput, Nikhil Mehta, Anima Singh, et al. (Google). "Recommender Systems with Generative Retrieval" (TIGER). NeurIPS 2023. https://arxiv.org/abs/2305.05065

48. Konstantina Christakopoulou, Alberto Lalama, Cj Adams, et al. (Google). "Large Language Models for User Interest Journeys." 2023. https://arxiv.org/abs/2305.15498
49. Spotify Newsroom. "Spotify Debuts a New AI DJ, Right in Your Pocket." Spotify, 2023. https://newsroom.spotify.com/2023-02-22/spotify-debuts-a-new-ai-dj-right-in-your-pocket/ (AI Playlist announcements: https://newsroom.spotify.com/)
50. Enrico Palumbo, Gustavo Penha, Andreas Damianou, et al. (Spotify). "Text2Tracks: Prompt-based Music Recommendation via Generative Retrieval." 2025. https://arxiv.org/abs/2503.24193
51. Ko-Jen Hsiao, Yesu Feng, Sudarshan Lamkhede (Netflix). "Foundation Model for Personalized Recommendation." Netflix Technology Blog, 2025. https://netflixtechblog.com/foundation-model-for-personalized-recommendation-1a0bd8e02d39
52. Kuaishou OneRec Team. "OneRec-Think: In-Text Reasoning for Generative Recommendation." 2025. https://arxiv.org/abs/2510.11639
53. Kuaishou OneRec Team. "OpenOneRec Technical Report" and repository. 2025. https://arxiv.org/abs/2512.24762 ; https://github.com/Kuaishou-OneRec/OpenOneRec
54. Kong et al. "MiniOneRec" (open-source generative recommendation framework). 2025. https://arxiv.org/abs/2510.24431 ; https://github.com/AkaliKong/MiniOneRec
55. ByteDance. "HLLM: Enhancing Sequential Recommendations via Hierarchical Large Language Models for Item and User Modeling." 2024. https://arxiv.org/abs/2409.12740 ; https://github.com/bytedance/HLLM
56. Chao Zhang et al. (Xiaohongshu). "NoteLLM: A Retrievable Large Language Model for Note Recommendation." WWW 2024. https://arxiv.org/abs/2403.01744
57. Ju, C. et al. (Snap). "Generative Recommendation with Semantic IDs: A Practitioner's Handbook" (GRID). CIKM 2025. https://arxiv.org/abs/2507.22224 ; https://github.com/snap-research/GRID
58. Microsoft RecAI. "Knowledge Plugins: Enhancing Large Language Models for Domain-Specific Recommendations" (DOKE) repository. GitHub, 2023-2024. https://github.com/microsoft/RecAI/tree/main/Knowledge_Plugin
59. Microsoft RecAI. "InteRecAgent" repository README. GitHub, 2023-2024. https://github.com/microsoft/RecAI/tree/main/InteRecAgent
60. "Large Language Models Enhanced Collaborative Filtering" (LLM-CF). 2024. https://arxiv.org/abs/2403.17688
61. Wenyue Hua et al. "LLM-RecSys-ID" repository (code for item-ID indexing in recommendation foundation models). GitHub, 2023. https://github.com/Wenyueh/LLM-RecSys-ID
62. Engineering at Meta. "GEM Training: How Meta Doubled the Efficiency of Its LLM-Scale Ads Foundation Model." Meta, August 2026. https://engineering.fb.com/2026/08/03/ml-applications/training-gem-at-llm-scale-meta-ads-recommendation-foundation-model/

30. Scaling Laws and Hardware for Recommenders: Compute, Embeddings, Accelerators and Cost

> Source note. This chapter was written while live web search and fetching were unavailable. The shared search budget for the run was used up and the egress proxy refused arXiv and Meta domains. Facts tagged with references that other chapters of this Atlas fetched and checked (HSTU, Wukong, GEM, Andromeda, Lattice, RankMixer, OneRec, TPU v4, Neo/ZionEX, TorchRec, HugeCTR, PERSIA, DeepRecSys, Pinterest GPU serving) are reused from those chapters. A few primary papers could not be re-fetched in this run: Ardalani et al. on DLRM scaling laws, Climber, the MTIA posts, Gupta et al. HPCA 2020, Sustainable AI and RecNMP. Their figures come from the author's prior reading and are marked "(approx., not re-fetched)". Check them against the linked primary source before you quote them.

Overview

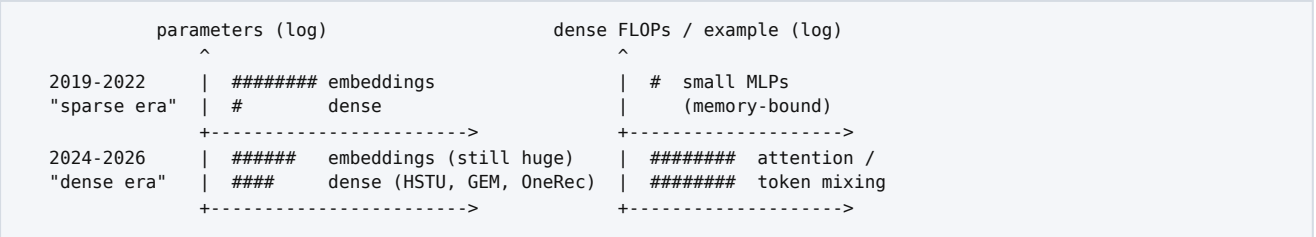
For about a decade, industrial recommenders scaled in a way that differed sharply from vision and language models. Parameter counts grew enormously, into the trillions and eventually a claimed 100 trillion [4][26]. Almost all of those parameters sat in sparse embedding tables, though. Each example touched only a few kilobytes of them and did very little arithmetic on what it read. The models were memory-capacity-bound and memory-bandwidth-bound, not FLOP-bound. Hardware utilization was poor, and secondary analyses put DLRM model FLOPs utilization (MFU) at roughly 4-5% [11][31]. Adding embedding rows or dimensions also stopped improving quality early [5][14].

Three shifts between 2022 and 2026 changed this picture:

1. **Scaling laws for dense compute.** Wukong [6], HSTU [7], Climber [10], RankMixer [11], OneRec-V2 [12] and ULTRA-HSTU [9] showed that recommendation quality follows predictable power laws when the scaled axis is dense interaction or sequence compute, not table size.
2. **Hardware co-design.** Embedding traffic received dedicated silicon and fabrics: Google TPU SparseCore [19], Meta ZionEX's RoCE scale-out network [4], Grace Hopper's unified memory used by Meta Andromeda [18], and Meta's own MTIA accelerators [20][21].
3. **MFU as the north-star metric.** Architectures are now judged partly by how well they use GPUs. RankMixer raised ranking MFU from 4.5% to 45% [11], OneRec reported about 24%/29% training/inference MFU [13], and GEM about 20-25% [16]. That headroom is what makes LLM-scale recommenders affordable.

This chapter covers the scaling-law papers, the embedding-versus-dense compute trade-off, the accelerators (TPU SparseCore, MTIA, GPUs and CPUs), and what is publicly known

about power and cost in recommendation datacenters.



---

1. Why recommender compute is different

1.1 The DLRM workload profile

DLRM [1] is the reference model. Dense features pass through a bottom MLP. Each sparse feature indexes an embedding table, and the looked-up rows are pooled (sum or mean). Pairwise dot-product interactions are then concatenated and fed to a top MLP. Its compute characteristics come from this structure:

- **Embedding lookups** are gathers of random rows from tables that can reach many GB or TB. They do almost no arithmetic per byte moved, so they are limited by memory bandwidth and, once tables are sharded, by all-to-all network bandwidth [3] [4].
- **MLPs and interactions** are ordinary GEMMs. In classic DLRMs they were small, so they could not saturate an accelerator's math units.

Meta's characterization "Understanding Training Efficiency of Deep Learning Recommendation Models at Scale" [3] found three things. Embedding operators are memory-bound and dominated by communication. Production models differ widely in sparse-feature profile, so no single hardware configuration fits all of them. CPU parameter-server trainers also spent much of each step waiting on embedding pulls and pushes. Gupta et al. (HPCA 2020) made a similar case for inference [2]. They reported that recommendation models consumed the large majority of AI inference cycles in Facebook datacenters, about 79% (approx., not re-fetched). They also showed that model classes split into embedding-dominated and MLP-dominated types with very different bottlenecks.

1.2 A rough roofline for the two halves

Component	Typical arithmetic intensity	Bound by	Scales with
Embedding bag (gather + pool)	very low (well under 1 FLOP/byte, approx.)	HBM/DRAM bandwidth, all-to-all	lookups x dim x bytes
Feature interaction (dot/FM/DCN)	moderate	mixed	features^2 x dim
MLP / token-mixing / FFN	high at large batch	tensor-core FLOPs	width^2 x tokens
Self-attention over sequences	high (with fused kernels)	FLOPs and HBM for KV	seq^2 x dim

The practical consequence is that in a DLRM, buying more FLOPs barely helps. You need memory capacity, memory bandwidth and network bisection bandwidth. Generative and

sequential models (Section 3) move most of the cost into the bottom two rows, where GPUs and TPUs work well.

---

2. Scaling laws for classic DLRMs: data, parameters, compute

2.1 Understanding Scaling Laws for Recommendation Models (Meta, 2022)

Ardalani et al. [5] (approx., not re-fetched) ran the first systematic scaling study of production-style DLRMs on Meta click-through-rate data. They looked at three axes: data, embedding parameters and dense (MLP) parameters. They fit power laws plus an irreducible constant. Their main findings, as commonly summarized, were:

- **Data scaling** gave the most reliable gains for a given compute budget.
- **Parameter scaling**, especially of embedding tables, was "running out of steam": loss improvements flattened quickly as tables grew.
- Under a compute budget, the best mix favored more data over larger tables. This motivated efforts to find architectures whose dense compute could scale usefully.

2.2 Embedding collapse: why bigger tables do not help

Guo et al. (Tsinghua and Tencent, ICML 2024) [14] give a mechanistic explanation. When embedding size is scaled up, the embedding matrices tend to **collapse** into a low-dimensional subspace, so the extra dimensions carry little information. Feature interaction modules make the collapse worse even as they limit overfitting. Their multi-embedding design uses several independent embedding sets, each with its own interaction module. It kept improving quality up to 10x embedding size [14]. The lesson for system designers is not to buy memory capacity on the assumption that wider embeddings will pay off. Scale through compute in the interaction and sequence layers instead.

---

3. Dense-compute scaling laws (2024-2026)

3.1 Wukong (Meta, ICML 2024)

Problem. Existing interaction architectures (DLRM, DCNv2, AutoInt, and others) stopped improving as they were scaled up in compute. A recommender with a predictable scaling law, like an LLM's, needs an interaction backbone whose quality keeps rising with FLOPs [6].

Architecture. Wukong is a stack of identical layers. Each layer has two parallel blocks whose outputs are concatenated:

- A **Factorization Machine Block (FMB)**, which computes FM-style pairwise interactions of the previous layer's embeddings and projects them with an MLP.

- A **Linear Compress Block (LCB)**, which linearly recombines the embeddings and preserves lower-order terms.

Each layer interacts the outputs of the previous one, so layer i covers interaction orders 1 to 2^i. Order grows exponentially with depth, whereas DCN-style crosses grow linearly. Scaling knobs are the number of layers, layer width, number of FM-projected embeddings and MLP sizes [6].

```
embeddings X0 (n x d)
     |
+----v-----------------------------------------+
| Wukong layer i                               |
|   FMB: X_{i-1} X_{i-1}^T -> flatten -> MLP --+  |
|   LCB: W * X_{i-1} (linear recombine) -------+--> concat -> LN + residual
+----------------------------------------------+
     |  (repeat L times: orders 1 .. 2^L)
     v
top MLP -> task heads
```

Scale. On an internal Meta dataset, Wukong held a scaling law across about two orders of magnitude of model complexity, beyond about 100 GFLOP per example. Prior architectures had stopped improving well before that point. Wukong also beat state-of-the-art models on six public datasets [6]. Secondary coverage connects Wukong-style stacked factorization to Meta's GEM ads foundation model [15].

Lesson. The interaction operator determines whether extra FLOPs turn into quality. Designs that raise interaction order cheaply, and that run as dense GEMMs, are the ones that scale.

3.2 HSTU and Generative Recommenders (Meta, ICML 2024)

Problem. DLRMs depend on heterogeneous, hand-built features and fail to scale with compute. Meta recast retrieval and ranking as **sequential transduction** over a user's chronological stream of (item, action) tokens [7].

Architecture. The HSTU block uses pointwise SiLU attention without a softmax, plus a relative position and time bias. A gating term `LayerNorm(attn) * U` replaces the wide FFN. Per-layer activation memory falls from about 33x hidden dimension to about 14x (secondary estimate) [7][31]. Training uses **Stochastic Length**, sparsely subsampling long histories. Inference uses **M-FALCON**: candidates are appended to the user sequence with a mask that lets them attend only to history, and they are processed in micro-batches that share the history's KV cache [7].

Scale numbers (as reported).

- Models of about **1.5 trillion parameters** were deployed on multiple surfaces of a billion-user platform, with **+12.4%** on online A/B metrics [7].
- HSTU runs **5.3x-15.2x faster** than FlashAttention2-based transformers at sequence length 8,192 [7].
- Quality follows a **power law in training compute across three orders of magnitude**, up to roughly GPT-3 or LLaMa-2 scale training compute [7]. Secondary notes give the fits as HR@100 = 0.15 + 0.0195 ln C for retrieval and NE = 0.549 -

0.0053 ln C for ranking. The sweep covered sequence lengths 512-8192, hidden dimensions 256-1024 and up to 24 layers [31].
- The open-source repository adds Triton kernels, an FA3-based HSTU attention kernel for H100, row-wise FP8 GEMMs, a "DLRM-v3" HSTU ranker with an MLCommons LoadGen harness, and "HSTU Ultra" [8].

Lesson. Restructuring training data, from one example per impression to one causal sequence per user, amortizes user-side compute across all of that user's events. This efficiency gain arrives before any parameters are added, and it is what makes the scaling law economical.

3.3 ULTRA-HSTU: "Bending the Scaling Law Curve" (Meta, 2026)

Meta's 2026 follow-up [9] targets the slope and offset of the curve rather than just moving along it. The repository's HSTU Ultra variant [8] combines three techniques:

- semi-local attention (local windows plus selected global context),
- target-aware attention,
- FP8 projections, reported at 1.13-1.25x end-to-end speedups.

The framing is that once a scaling law exists, architecture and systems work should aim to get the same quality for fewer FLOPs, which shifts the whole curve.

3.4 Climber (NetEase Cloud Music, 2025)

Climber [10] (approx., not re-fetched) targets the cost of HSTU-style scaling in a mid-sized company. Its main components, as summarized from the abstract:

- **ASTRO** (Adaptive Scalable Transformer for RecOmmendation). It uses multi-scale sequence partitioning: the user sequence is split into sub-sequences that are processed separately, which cuts attention complexity from O(n^2 d) to roughly O(n^2 d / Nb) for Nb blocks. It also uses dynamic temperature modulation to adapt attention distributions across scenarios and behaviors.
- **TURBO**, an acceleration framework that co-designs training and serving, including "encode once, decode many" style shared computation and fused operators.

The paper reports that this design gives more favorable scaling curves than HSTU-style baselines. It also reports roughly a 5x training-throughput improvement (approx.) and an online metric gain of about 12% in NetEase Cloud Music deployment (approx.). Treat the exact numbers as unverified.

Lesson. Scaling laws are only actionable if their compute cost is affordable. Block-wise sequence partitioning and serving-side KV reuse are the usual levers.

3.5 RankMixer and TokenMixer-Large (ByteDance, 2025-2026)

Problem. Douyin's industrial rankers had accumulated many handcrafted interaction modules that ran at about **4.5% MFU** [11].

Architecture. Features are grouped into tokens. Each block applies **multi-head token mixing**, a parameter-free recombination of token heads that replaces quadratic self-

attention over heterogeneous feature spaces. It then applies a **per-token FFN** with separate parameters per feature subspace, optionally extended to a sparse MoE that is dense in training and sparse at inference [11].

Results.

- MFU rose from **4.5% to 45%**.
- Ranking-model parameters scaled about **100x** at roughly constant inference latency.
- Full Douyin deployment gave +0.2% active days and +0.5% app duration [11].
- TokenMixer-Large reports offline scaling to 7B and 15B parameters [33].

Lesson. Hardware efficiency is itself an architecture objective. Making the model "GEMM-shaped" let ByteDance scale parameters 100x within the same latency envelope.

3.6 OneRec and OneRec-V2 (Kuaishou, 2025)

OneRec replaces the retrieval-and-ranking cascade with a generative model. Its technical report claims about **24% training MFU and 29% inference MFU** and operating cost of about **10%** of the previous cascade (approx.) [13].

OneRec-V2 introduced a lazy decoder-only design. At 1B parameters it matched encoder-decoder quality (loss 3.27 vs 3.28) with **18.89 vs 296.36 GFLOPs**, about 94% less compute [12]. Its dense scaling law, fit from 0.1B to 8B parameters, is L(N) = 3.13 + 3660 / N^0.489. A 4B-total, 0.5B-active MoE reached loss 3.22, better than a 2B dense model [12].

3.7 Other scaling studies

- **Large sequential recommendation scaling** (RUC, 2023-2024) [30] (approx., not re-fetched). Decoder-only ID-based sequential recommenders scaled to about 0.8B parameters followed power laws. The gains were largest for cold-start and long-tail users.
- **MTGR** (Meituan, 2025) [32]. Pure generative ranking without cross features underperformed. The hybrid design reached about 65x FLOPs per sample versus DLRM (approx.). The point is that scaling laws hold within an architecture family, but feature information can matter more than a 10x compute change.
- **LUM** (Alibaba, 2025) [34]. A generative pre-trained user model is queried by discriminative downstream models to "unlock" scaling in industrial settings.

3.8 Scaling-law comparison

Work (org, year)	Scaled axis	Reported law / range	Hardware-efficiency note
Ardalani et al. (Meta, 2022) [5]	data, emb params, MLP params	power law + constant; emb scaling saturates (approx.)	DLRM, memory-bound
Embedding collapse (Tsinghua/Tencent, 2024) [14]	embedding size	gains stall unless multi-embedding	n/a
Wukong (Meta, 2024) [6]	interaction layers/width	~2 orders of magnitude, >100 GFLOP/example	dense FM GEMMs
HSTU (Meta, 2024) [7]	seq len, depth, width	power law over 3 orders of compute; 1.5T params	5.3-15.2x vs FA2

Work (org, year)	Scaled axis	Reported law / range	Hardware-efficiency note
Climber (NetEase, 2025) [10]	seq blocks, depth	improved curve vs HSTU (approx.)	~5x training speedup (approx.)
RankMixer (ByteDance, 2025) [11]	params (dense + MoE)	~100x params at constant latency	MFU 4.5% -> 45%
OneRec-V2 (Kuaishou, 2025) [12]	params 0.1B-8B	L = 3.13 + 3660/N^0.489	94% fewer FLOPs than enc-dec
GEM (Meta, 2025-26) [15][16]	training FLOPs	4x FLOPs in 12 months	MFU 20-25%
ULTRA-HSTU (Meta, 2026) [9]	architecture efficiency	"bends" the curve	FP8, semi-local attention

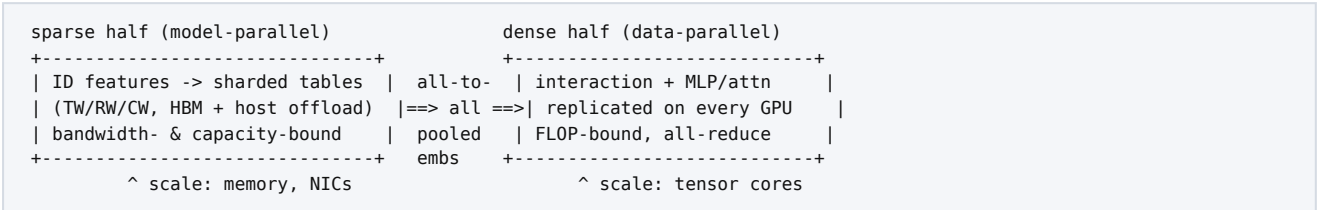
---

4. Embedding compute versus dense compute: systems consequences

4.1 The sparse side: sharding, caching, offload

When embeddings dominate, the training system has to be designed around them.

- **Neo / ZionEX (Meta)** [4]. This is a fully synchronous GPU training system. Each node has 8 GPUs on NVLink, and each node also has a **dedicated RoCE scale-out network**, with several NICs per node so every GPU has its own path for all-to-all embedding traffic. "4D parallelism" chooses table-wise, row-wise, column-wise or data-parallel placement per table using a cost-model planner. The paper reports training models of up to **12 trillion parameters** and roughly order-of-magnitude (up to about 40x) speedups over CPU-based asynchronous training [4].
- **TorchRec + FBGEMM_GPU** [24] is the open-source production version of these ideas. It provides sharded embedding collections, table-batched embedding (TBE) kernels that run many tables in one launch, quantized tables, and pipelined input distribution.
- **PERSIA (Kuaishou/ETH)** [26] keeps embeddings on a CPU parameter server trained asynchronously and the dense network on GPUs trained synchronously. It reports models of up to **100 trillion parameters**.
- **HugeCTR (NVIDIA)** [25] keeps the whole loop on GPU. Its documentation states that two 8x A100-80GB nodes can hold models of about **1 TB** entirely in GPU memory.
- **Check-N-Run (Meta)** [28] checkpoints only modified rows and quantizes them. It reports roughly 6-17x lower checkpoint write bandwidth and 2.5-8x lower storage.



4.2 The dense side: MFU becomes the metric

Once dense compute is the scaled axis, the important question is what fraction of peak FLOPs the model actually uses:

System	Reported MFU	Main tricks
Typical legacy DLRM ranker (per ByteDance) [11]	~4.5%	(baseline)
RankMixer [11]	~45%	token mixing, per-token FFN, MoE
OneRec V1 [13]	~24% train / ~29% infer (approx.)	MoE decoder, compute in generator
GEM training [16]	20-25%	kernels, precision, parallelism, networking, memory co-design
LLM pre-training (for context, secondary) [31]	~40-60%	dense transformer

The HSTU kernels [7][8] and fused attention for jagged sequences show what it takes to close the gap to LLMs. Recommendation data comes in variable-length ("jagged") batches, many tables and small per-feature tensors. Without padding-free kernels and op fusion, GPUs sit idle.

4.3 Where the parameters versus the FLOPs live

Model family	Params mostly in	FLOPs mostly in	Hardware fit
Classic DLRM/DCN (2019-2022)	embeddings (often >90%) [1][31]	small MLPs	memory/network-bound; CPUs and SparseCore
Wukong / RankMixer (2024-25)	embeddings + large dense stacks	dense interaction GEMMs	GPU-friendly if fused
HSTU / GEM / OneRec (2024-26)	embeddings + transformer blocks	attention + FFN over long sequences	GPU/TPU, LLM-like

---

5. Google TPU SparseCore: embeddings in silicon

Problem. In Google's fleet, embedding-heavy models (DLRMs for ads, search and YouTube) are a large workload. On TensorCores alone, embedding lookups and the all-to-all traffic around them run poorly [19].

Design (TPU v4, ISCA 2023). Each TPU v4 chip includes **SparseCores**. These are dataflow processors dedicated to embedding lookups, which are gathers with variable-length pooling and scatter-add updates, and to the related all-to-all communication over the inter-chip interconnect. Embedding tables are sharded across the HBM of many chips in a slice. SparseCores perform the lookup, the reduction and the cross-chip exchange while the TensorCores run the dense layers [19].

```
TPU v4 chip
+-------------------------------------------------------------+
|  TensorCores (MXU systolic arrays) <-- dense MLP / attention |
|  SparseCores (dataflow tiles)      <-- embedding gather,     |
|        |                                pool, scatter-add   |
|        +--> ICI (3D torus, OCS-reconfigurable) all-to-all     |
|  HBM: shard of every large embedding table                   |
+-------------------------------------------------------------+
  x 4,096 chips per pod (optically switched)
```

Reported numbers [19].

- SparseCores accelerate embedding-heavy models by about **5-7x**.
- They use only about **5% of die area and power** (approx., from the abstract).
- At the system level, TPU v4 pods use **optical circuit switches** to reconfigure the interconnect topology.
- The paper compares TPU v4 with contemporary GPUs and reports better performance and performance per watt on its benchmarks (approx.).

Later generations. Google has said that later TPUs continue the SparseCore line. Trillium (TPU v6e) is described as having a third-generation SparseCore for "ultra-large embeddings" in ranking and recommendation workloads [37] (approx., not re-fetched).

Lesson. About 5% of the silicon budget, spent on the operator that dominates recommendation runtime, bought several-fold speedups. That is the clearest public case for specialized sparse hardware. YouTube's later move toward semantic-ID generative retrieval (see Chapter 17) shifts the balance back toward dense compute, so a chip needs both kinds of engine.

---

6. Meta MTIA: a recommendation-first accelerator

> Figures in this section are from Meta's MTIA posts and ISCA papers as previously read. They could not be re-fetched in this run and are marked approximate.

Problem. Ranking and recommendation inference is a large share of Meta's AI fleet [2]. GPUs were seen as over-provisioned in FLOPs and expensive in power for these memory-intensive, small-batch models. Meta wanted better performance per watt and per TCO dollar for its own workloads [20][21].

6.1 MTIA v1 (announced May 2023)

Reported design (approx.) [20]:

- TSMC 7nm at 800 MHz, with about 102 TOPS INT8 and 51 TFLOPS FP16/BF16 at about a 25 W TDP.
- A grid of 64 processing elements (8x8), each with RISC-V cores, plus fixed-function units for matrix multiply, SIMD and data movement.
- About 128 MB of on-chip SRAM and 64 GB of off-chip LPDDR5.

The design philosophy was to keep hot embeddings and activations in large on-chip SRAM and to use cheap, high-capacity LPDDR instead of HBM for the long tail. The co-designed

software stack is PyTorch-native and integrates with FBGEMM-style embedding kernels.

6.2 MTIA v2 ("next-generation MTIA", April 2024)

Reported design (approx.) [21]:

- TSMC 5nm at 1.35 GHz, with about 354 TOPS INT8 dense (about 708 with sparsity) at about a 90 W TDP.
- About 256 MB of on-chip SRAM and 128 GB of LPDDR5.
- A rack design holding 72 accelerators (three chassis of 12 boards with two chips each).

Meta reported roughly 3x performance over the first generation on four key models and roughly 6x model-serving throughput at the platform level. It also reported roughly 1.5x performance per watt. The chip was deployed for ranking and recommendation ads models in production.

6.3 Andromeda and the mixed fleet

Meta's Andromeda ads retrieval engine (December 2024) is co-designed with **NVIDIA Grace Hopper Superchips**, and the post names **MTIA** as part of the hardware plan [18]. Andromeda replaced lightweight retrieval with a deep network over a jointly learned hierarchical index. It reported **+6% recall** and **+8% ads quality** on selected segments. Secondary coverage cites about a **10,000x** increase in retrieval model complexity (approx.) [18]. Grace Hopper's unified CPU-GPU memory over NVLink-C2C lets large embedding and index structures sit in CPU-attached memory while the GPU computes on them. This directly addresses the capacity problem from Section 4.

```
Meta recsys fleet (as publicly described, 2024-2026)
  training: GPU clusters (ZionEX lineage -> large GPU clusters for GEM)
  retrieval: Andromeda DNN on Grace Hopper (unified memory) [18]
  ranking:  Lattice / GEM-distilled students on GPUs and MTIA [17][15][21]
  teacher:  GEM, LLM-scale, thousands of GPUs, MFU 20-25% [15][16]
```

Lesson. Hyperscalers run heterogeneous fleets. Custom ASICs (MTIA, SparseCore) win on perf/W for stable, high-volume operators. GPUs are used for the fast-moving frontier (generative, LLM-scale teachers) and for retrieval stages co-designed with unified memory.

---

7. GPU versus CPU (and accelerator) trade-offs

7.1 The historical CPU case

Until roughly 2020, most industrial recommendation inference ran on CPUs, and much training ran on CPU parameter-server clusters [3][4]. CPUs had three advantages. First, large and cheap DRAM holds TB-scale tables. Second, request handling is flexible, with variable candidate counts and many small ops. Third, the fleet already existed. DeepRecSys (Harvard/Facebook, ISCA 2020) [22] showed that query sizes (candidates per request) vary widely. Its scheduler, DeepRecSched, picks batch sizes and splits queries between CPU cores and accelerators: small queries stay on CPU and large ones go to the accelerator.

This gave about 2x latency-bounded throughput and over 30% latency reduction in production [22]. Near-memory processing proposals such as RecNMP [36] (approx., not re-fetched) went further and put embedding pooling in DIMM-side logic.

7.2 The GPU transition

- **Training.** Meta moved from asynchronous CPU training to synchronous GPU training with Neo/ZionEX and TorchRec. It reported up to about 40x faster time-to-solution and better reproducibility and quality from synchronous updates [4][24]. NVIDIA HugeCTR and Merlin make the same case [25].
- **Inference.** Pinterest's 2022 post describes moving ranking from CPU to GPU serving because CPU cost grew about linearly with model size. GPUs let it serve models with on the order of 100x more compute at comparable or lower cost and latency [23]. The techniques were batching all candidates of a request, CUDA Graphs, merging embedding lookups, mixed precision, and moving feature preprocessing onto the GPU [23].

7.3 Decision table

Criterion	CPU	GPU	Custom ASIC (MTIA, TPU+SparseCore)
Embedding capacity	best (TB DRAM, cheap)	limited HBM; needs sharding, caching, or Grace Hopper	large SRAM + LPDDR (MTIA); sharded HBM (TPU)
Embedding bandwidth	moderate	high HBM; NVLink/RoCE all-to-all	dedicated (SparseCore)
Dense FLOPs	poor	best	good
Small/variable batches	good	needs batching, CUDA graphs	designed for it (MTIA)
Perf/W on stable DLRMs	low	medium	best (vendor claims)
Flexibility for new archs	high	high	lower; compiler-dependent
Best fit	light rankers, feature logic, retrieval glue	heavy ranking, generative models, training	high-volume stable ranking inference

7.4 Precision as a cost lever

Low precision is applied unevenly:

- **Embeddings** are quantized first, to FP16/INT8/INT4 rows with row-wise scales, for serving and checkpoints [24][28].
- **Dense layers** follow. HSTU Ultra uses row-wise FP8 GEMMs [8], and GEM training efficiency gains cite numeric-precision co-design [16].

Most of the bytes are embedding bytes, so cutting embedding precision first gives the largest memory and bandwidth savings per unit of quality risk.

---

8. Power, cost and datacenter footprint

Public, precise numbers on recommendation power are rare. What is documented:

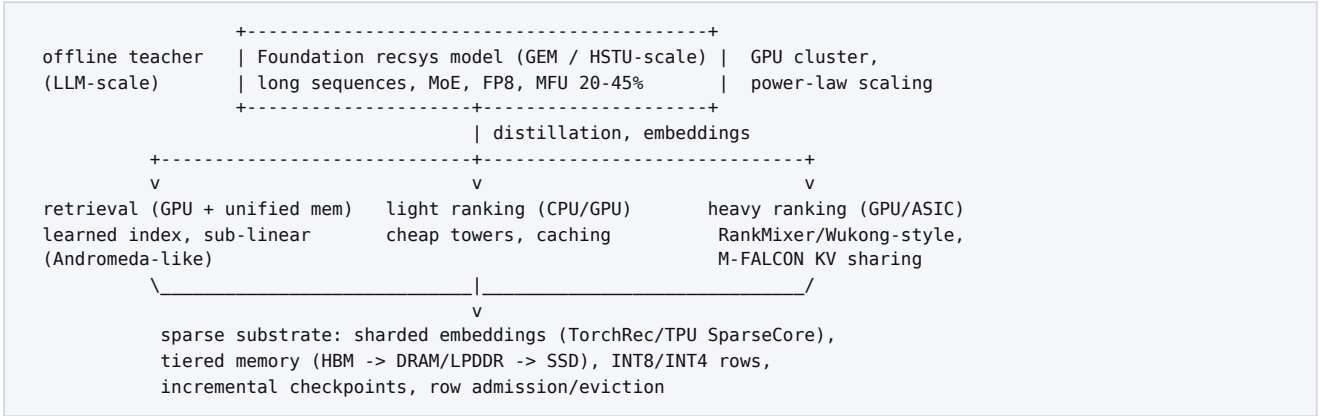
- **Share of cycles.** Gupta et al. [2] (approx., not re-fetched) reported that recommendation models accounted for most AI inference cycles in Facebook's datacenters, on the order of 79%.
- **Sustainable AI (Wu et al., Meta, MLSys 2022)** [27] (approx., not re-fetched) characterized the carbon footprint of Meta's production models, including recommendation models (RM1-RM5). It reported that power capacity for AI was split roughly 10:20:70 across experimentation, training and inference. Recommendation data and model sizes grew several times over in about two years. Most of the gain came from efficiency work: hardware-software co-design, caching and quantization.
- **TPU v4 SparseCore** [19] delivered about 5-7x embedding speedups for about 5% of die power, so for Google's embedding-heavy models the efficiency gain is several-fold.
- **MTIA v2** [21] targets better perf/W and TCO than GPUs for ranking models. Meta reported roughly 1.5x perf/W over MTIA v1 (approx.).
- **Model consolidation as a cost tool.** Meta Lattice merged many ads models into one multi-domain, multi-task model. It reported **+10% topline** and **about 20% capacity savings** [17]. OneRec reported operating cost of about 10% of its prior cascade (approx.) [13]. GEM is kept offline as a teacher and distilled into cheap serving students [15].

```
cost per served recommendation  ~=  (training $ / requests served over model life)
                                  + (serving FLOPs x $/FLOP at achieved MFU)
                                  + (embedding bytes x $/GB-month of the memory tier)
                                  + (network all-to-all at training scale)
levers: MFU up | precision down | distill teacher->student | consolidate models |
        tier embeddings (SRAM/HBM -> DRAM/LPDDR -> SSD) | cache hot rows
```

Lesson. The biggest documented cost wins came from structural changes, not from low-level tuning alone. Examples are consolidating model fleets, distilling large teachers, replacing cascades, and moving to architectures that use accelerators well.

---

9. Putting it together: a reference scaling stack (2026)



Key takeaways for system designers

1. **Scale dense compute, not table size.** The evidence of saturating embedding scaling [5], embedding collapse [14], and dense-compute power laws (Wukong, HSTU, RankMixer, OneRec-V2) [6][7][11][12] all point the same way. Budget extra FLOPs into interaction and sequence modules.
2. **Expect logarithmic returns.** Reported laws are linear in ln(compute) [7][31] or power laws in N [12]. Each fixed quality step costs about 10x compute, so efficiency work that bends the curve [9][10] is worth as much as raw scale.
3. **Track MFU as a first-class metric.** Going from about 5% to 20-45% MFU [11][13] [16] is an order of magnitude of free model capacity. Make architectures GEMM-shaped, use padding-free jagged kernels, and fuse operators.
4. **Restructure training data before scaling.** One causal sequence per user (HSTU) and encode-once/score-many inference (M-FALCON, Climber's TURBO) amortize user compute [7][10].
5. **Embeddings are a memory-hierarchy and network problem.** Size clusters by table bytes and all-to-all bandwidth first. Shard per table with a cost-model planner. Cache the hot head, and quantize and offload the tail [4][24][25].
6. **Specialized sparse hardware pays off.** SparseCore's about 5-7x for about 5% of die area [19] and ZionEX's dedicated RoCE fabric [4] show that embedding traffic justifies its own silicon and network.
7. **Run a heterogeneous fleet deliberately.** Use CPUs for feature logic and light stages, GPUs for frontier models and training, and ASICs (MTIA, TPU) for stable, high-volume inference. Grace Hopper-style unified memory changes what retrieval can afford [18][21].
8. **Train big, serve small.** LLM-scale teachers distilled into fleet students (GEM) and model consolidation (Lattice) were the largest documented cost and quality levers [15][17].
9. **Quantize embeddings before dense weights.** Most bytes are in tables, so INT8/INT4 rows and quantized incremental checkpoints save the most for the least risk [24][28].
10. **Treat vendor and paper numbers as approximate.** Scaling-law constants are dataset-specific and do not transfer between companies. Re-fit them on your own data at small scale before committing a hardware budget.

References

1. Naumov, M. et al. "Deep Learning Recommendation Model for Personalization and Recommendation Systems" (DLRM). Meta (Facebook), 2019. https://arxiv.org/abs/1906.00091
2. Gupta, U. et al. "The Architectural Implications of Facebook's DNN-based Personalized Recommendation." Facebook / Harvard, HPCA 2020 (not re-fetched in this run). https://arxiv.org/abs/1906.03109
3. Acun, B. et al. "Understanding Training Efficiency of Deep Learning Recommendation Models at Scale." Meta, HPCA 2021. https://arxiv.org/abs/2011.05497
4. Mudigere, D. et al. "Software-Hardware Co-design for Fast and Scalable Training of Deep Learning Recommendation Models" (Neo / ZionEX). Meta, ISCA 2022. https://arxiv.org/abs/2104.05158

5. Ardalani, N. et al. "Understanding Scaling Laws for Recommendation Models." Meta, 2022 (not re-fetched in this run). https://arxiv.org/abs/2208.08489
6. Zhang, B., Luo, L., Chen, Y. et al. "Wukong: Towards a Scaling Law for Large-Scale Recommendation." Meta, ICML 2024. https://arxiv.org/abs/2403.02545
7. Zhai, J. et al. "Actions Speak Louder than Words: Trillion-Parameter Sequential Transducers for Generative Recommendations" (HSTU). Meta, ICML 2024. https://arxiv.org/abs/2402.17152
8. meta-recsys. "generative-recommenders" repository (HSTU, M-FALCON, DLRM-v3, HSTU Ultra, Triton/FP8 kernels). Meta, 2024-2026. https://github.com/meta-recsys/generative-recommenders
9. Meta. "Bending the Scaling Law Curve in Large-Scale Recommendation Systems" (ULTRA-HSTU). Meta, 2026. https://arxiv.org/abs/2602.16986
10. NetEase Cloud Music. "Climber: Toward Efficient Scaling Laws for Large Recommendation Models." 2025 (not re-fetched in this run). https://arxiv.org/abs/2502.09888
11. ByteDance. "RankMixer: Scaling Up Ranking Models in Industrial Recommenders." 2025. https://arxiv.org/abs/2507.15551
12. Kuaishou OneRec Team. "OneRec-V2 Technical Report." Kuaishou, 2025. https://arxiv.org/abs/2508.20900
13. Kuaishou OneRec Team. "OneRec Technical Report." Kuaishou, 2025. https://arxiv.org/abs/2506.13695
14. Guo, X. et al. "On the Embedding Collapse When Scaling up Recommendation Models." Tsinghua / Tencent, ICML 2024. https://arxiv.org/abs/2310.04400
15. Engineering at Meta. "Meta's Generative Ads Model (GEM): The central brain accelerating ads recommendation AI innovation." Meta, November 2025. https://engineering.fb.com/2025/11/10/ml-applications/metas-generative-ads-model-gem-the-central-brain-accelerating-ads-recommendation-ai-innovation/
16. Engineering at Meta. "GEM Training: How Meta Doubled the Efficiency of Its LLM-Scale Ads Foundation Model." Meta, August 2026. https://engineering.fb.com/2026/08/03/ml-applications/training-gem-at-llm-scale-meta-ads-recommendation-foundation-model/
17. Meta. "Meta Lattice: Model Space Redesign for Cost-Effective Industry-Scale Ads Recommendations." Meta, 2025. https://arxiv.org/abs/2512.09200
18. Engineering at Meta. "Meta Andromeda: Supercharging Advantage+ automation with the next-gen personalized ads retrieval engine." Meta, December 2024. https://engineering.fb.com/2024/12/02/production-engineering/meta-andromeda-advantage-automation-next-gen-personalized-ads-retrieval-engine/
19. Jouppi, N. et al. "TPU v4: An Optically Reconfigurable Supercomputer for Machine Learning with Hardware Support for Embeddings." Google, ISCA 2023. https://arxiv.org/abs/2304.01433
20. Meta AI. "MTIA v1: Meta's first-generation AI inference accelerator." Meta, May 2023 (not re-fetched in this run). https://ai.meta.com/blog/meta-training-inference-accelerator-AI-MTIA/
21. Meta AI. "Our next-generation Meta Training and Inference Accelerator." Meta, April 2024 (not re-fetched in this run). https://ai.meta.com/blog/next-generation-meta-training-inference-accelerator-AI-MTIA/
22. Gupta, U. et al. "DeepRecSys: A System for Optimizing End-To-End At-scale Neural Recommendation Inference." Harvard / Facebook, ISCA 2020. http://vlsiarch.eecs.har

vard.edu/wp-content/uploads/2020/05/DeepRecSys_Gupta_ISCA2020.pdf
23. Pinterest Engineering. "GPU-accelerated ML Inference at Pinterest." Pinterest, 2022. https://medium.com/@Pinterest_Engineering/gpu-accelerated-ml-inference-at-pinterest-ad1b6a03a16d
24. PyTorch / Meta. "TorchRec" and "FBGEMM / FBGEMM_GPU" repositories. Accessed 2026. https://github.com/pytorch/torchrec ; https://github.com/pytorch/FBGEMM
25. Wei, Y. et al. "Merlin HugeCTR: GPU-accelerated Recommender System Training and Inference." NVIDIA, RecSys 2022. https://arxiv.org/abs/2210.08803
26. Lian, X. et al. "Persia: A Hybrid System Scaling Deep Learning Based Recommenders up to 100 Trillion Parameters." Kuaishou / ETH Zurich, KDD 2022. https://arxiv.org/abs/2111.05897
27. Wu, C.-J. et al. "Sustainable AI: Environmental Implications, Challenges and Opportunities." Meta, MLSys 2022 (not re-fetched in this run). https://arxiv.org/abs/2111.00364
28. Eisenman, A. et al. "Check-N-Run: A Checkpointing System for Training Deep Learning Recommendation Models." Meta, NSDI 2022. https://arxiv.org/abs/2010.08679
29. Liu, Z. et al. "Monolith: Real Time Recommendation System With Collisionless Embedding Table." ByteDance, 2022 (background reading). https://arxiv.org/abs/2209.07663
30. Zhang, G. et al. "Scaling Law of Large Sequential Recommendation Models." Renmin University of China, 2023 / RecSys 2024 (not re-fetched in this run). https://arxiv.org/abs/2311.11351
31. Xing, H. "HSTU: The Cornerstone of Scaling Laws" (fun-rec-mdbook technical notes, secondary). 2026. https://github.com/Haozhe-Xing/fun-rec-mdbook/blob/main/src/en/part7-scaling/hstu.md
32. Han, R. et al. "MTGR: Industrial-Scale Generative Recommendation Framework in Meituan." Meituan, 2025. https://arxiv.org/abs/2505.18654
33. ByteDance. "TokenMixer-Large: Scaling Up Large Ranking Models in Industrial Recommenders." 2026. https://arxiv.org/abs/2602.06563
34. Alibaba. "Unlocking Scaling Law in Industrial Recommendation Systems with a Three-step Paradigm based Large User Model" (LUM). 2025. https://arxiv.org/abs/2502.08309
35. Meta Research. "DLRM" repository (reference implementation and MLPerf DLRM lineage). Accessed 2026. https://github.com/facebookresearch/dlrm
36. Ke, L. et al. "RecNMP: Accelerating Personalized Recommendation with Near-Memory Processing." Facebook / Harvard, ISCA 2020 (not re-fetched in this run). https://arxiv.org/abs/1912.12953
37. Google Cloud. "Introducing Trillium, sixth-generation TPUs." Google, May 2024 (not re-fetched in this run). https://cloud.google.com/blog/products/compute/introducing-trillium-6th-gen-tpus

31. Search Meets Recommendation: Learning-to-Rank, Personalized and Semantic Search, Hybrid Retrieval and Unified Models

Search and recommendation are usually built by different teams, but they solve the same problem. Given a context, return a short ranked list of items from a very large corpus. In search the context contains an explicit query. In recommendation the query is implicit: who the user is, what they did recently, and the page they are on. Eugene Yan's widely cited "System Design for Discovery" draws both as one two-stage system, with offline training and indexing and online retrieval followed by ranking [40]. Amazon's research line goes further and treats product search as a personalized ranking problem, where "every interaction on the site is a recommendation" [9][10].

This chapter follows the parts of the stack where the two fields meet:

1. **Learning-to-rank (LTR).** RankNet, LambdaRank and LambdaMART, the losses that most production search rankers still descend from, and the unbiased-LTR corrections for click logs [1]-[6].
2. **Query understanding and semantic matching.** From DSSM's letter-trigram towers to BERT-class query encoders [7][41][42].
3. **Embedding-based retrieval (EBR) in production search.** Facebook, Amazon, Etsy, Taobao, JD, Airbnb and Pinterest [8]-[21][38][39].
4. **Hybrid lexical plus vector retrieval.** Boolean filters inside ANN indexes, score fusion (RRF, normalization), learned sparse models, late interaction [8][17][23]-[29].
5. **Personalized search.** Session and long-term signals in the ranker, from Bing to Airbnb [12]-[16][30].
6. **Unified search and recommendation.** Shared embedding spaces, joint behavior models and single generative models that serve both tasks (Pinterest, Kuaishou, Spotify, Amazon) [20][31]-[37].

A note on sourcing. This chapter was written in an environment where the live web-search budget had already been used up and most company blogs and arXiv could not be reached directly. Primary sources that could be fetched (Microsoft Research pages, the Burges LambdaMART report, GitHub repositories and documentation for LightGBM, TF-Ranking, Vespa, ColBERT, SPLADE, SPTAG, DiskANN, the Elasticsearch LTR plugin, Amazon's ESCI dataset, UniSAR and a curated generative-recommendation paper list) were read directly. Facts about Airbnb, Facebook, Etsy, Amazon, Pinterest, Uber and Kuaishou come from primary-source excerpts that were retrieved and cross-checked for sibling chapters of this atlas (Chapters 2, 6, 22, 24 and 26). References marked "not re-fetched" are well-known papers cited from their stable arXiv or publisher URLs. Numbers marked approximate should be checked against the original paper before reuse.

31.1 The common skeleton

Every system in this chapter fits the same skeleton. The differences are what goes into the "query" box and which retrieval paths exist.

```
explicit query (text, filters)     implicit context (user, session, page, geo)
          \                                  /
           v                                v
 +-------------------- QUERY UNDERSTANDING --------------------+
 | normalize, spell-correct, segment, classify (category,      |
 | intent, locale), extract attributes, rewrite / expand,      |
 | encode -> dense vector and/or sparse terms                  |
 +-------------------------------------------------------------+
           |                |                    |
           v                v                    v
 lexical / boolean     dense EBR (ANN)      behavior / graph sources
 (inverted index,      (two-tower, IVF,     (co-click, item-to-item,
  BM25, filters)        HNSW, SPANN)         history, trending)
           \                |                    /
            v               v                   v
 +--------- MERGE: union / RRF / quotas / dedup (10^3 - 10^4) -----+
                            |
                            v
 first-pass ranker (GBDT/LambdaMART or small DNN, in-index or near data)
                            |
                            v
 second-pass ranker (deep multi-task model, personalization, cross-features)
                            |
                            v
 re-rank: diversity, business rules, ads blending -> page
                            |
                            v
 logs: impressions, clicks, add-to-cart, bookings -> labels, propensities
```

Two observations hold across all the case studies.

- **Search is precision-constrained; recommendation is not.** A query such as "iphone 15 case" defines a relevance floor that personalization must not break. A home feed has no such floor. This is why search systems keep lexical retrieval and hard filters even after EBR works, while feed systems can be purely embedding- or graph-based [8][9][11].
- **Recommendation is search with an empty or implicit query.** Once the query is just another feature (text embedding, category, attributes), the same retrieval and ranking stack can serve both. The unified models in Section 31.7 make this explicit.

31.2 Learning-to-rank: RankNet, LambdaRank, LambdaMART

31.2.1 RankNet

Burges' report "From RankNet to LambdaRank to LambdaMART: An Overview" (Microsoft Research, MSR-TR-2010-82) is the canonical self-contained description [1]. RankNet works on pairs of documents for the same query. The model (any differentiable scorer; Burges notes neural nets and also boosted trees) produces scores s_i and s_j. The probability that document i should rank above j is a sigmoid of the score difference:

- P_ij = 1 / (1 + exp(-sigma * (s_i - s_j)))

The cost is cross-entropy against the target probability. With S_ij in {-1, 0, +1} for the label order, this becomes

- C = 0.5 * (1 - S_ij) * sigma * (s_i - s_j) + log(1 + exp(-sigma * (s_i - s_j)))

Two properties matter in practice. Equal scores for differently labelled documents still cost log 2, so the loss contains a margin. The cost is linear for badly ordered pairs and goes to zero for correct ones [1].

Factorization trick. The gradient for each pair can be written as a scalar "lambda" times the difference of score gradients. Summing the lambdas per document before back-propagating turns the training cost from roughly quadratic in pairs to close to linear in documents. Burges notes this speed-up "results from the way the problem factorizes, not from using mini-batch alone" and that it laid the groundwork for LambdaRank [1].

31.2.2 LambdaRank

IR metrics such as NDCG and ERR depend only on sorted order, so they are flat or discontinuous as functions of the scores. LambdaRank sidesteps this: instead of defining a smooth cost, it **defines the gradient directly**. The RankNet lambda for each pair is multiplied by |delta NDCG|, the change in NDCG that swapping the two documents would cause. Burges reports that models trained this way empirically optimize NDCG directly, and that any other IR metric can be targeted by swapping delta NDCG for the change in that metric [1].

31.2.3 LambdaMART

MART (gradient-boosted regression trees) fits each new tree to the gradients of the current ensemble. LambdaRank specifies exactly those gradients. LambdaMART is "the marriage of the two": each boosting round fits a regression tree to the lambdas, and leaf values are set with a Newton step that uses second derivatives [1]. An ensemble of LambdaMART rankers won Track 1 of the 2010 Yahoo! Learning to Rank Challenge [1].

```
per query q, documents d1..dn with graded labels l_i
     |
     v
current ensemble F_m(x) --> scores s_i
     |
     v
for each pair (i, j) with l_i > l_j:
      rho_ij   = 1 / (1 + exp(sigma * (s_i - s_j)))
      lambda_ij = sigma * |delta NDCG(i <-> j)| * rho_ij
      lambda_i += lambda_ij ;  lambda_j -= lambda_ij
     |
     v
fit regression tree m+1 to {(x_i, lambda_i)}
leaf value = sum(lambda) / sum(second derivative)   (Newton step)
     |
     v
F_{m+1} = F_m + eta * tree_{m+1}
```

31.2.4 LTR in today's libraries

LightGBM exposes the same ideas as configuration [2]:

- `objective=lambdarank` with a default `label_gain` of 0, 1, 3, 7, 15, ... (2^label - 1), so grades map to exponential NDCG gains.
- `lambdarank_truncation_level` (default 30) limits the pairs considered to the top of the list. The documentation advises setting it slightly above the target NDCG@k cutoff (for example k + 3).
- `lambdarank_norm` (default true) normalizes lambdas per query, which helps when queries have very different numbers of results.
- `rank_xendcg` (XE-NDCG, Bruch 2019) is documented as faster than lambdarank with similar quality [2][43].

TF-Ranking (Google, archived in 2025) brought the same family to neural networks: pointwise, pairwise and listwise losses, LambdaLoss for direct metric optimization, groupwise scoring functions, and unbiased LTR from biased feedback [3]. In search engines, the OpenSource Connections Elasticsearch LTR plugin stores features as query templates, logs feature values to build training sets, and stores RankLib, XGBoost or linear models that re-rank the top results inside the engine [6].

31.2.5 From graded judgments to clicks: unbiased LTR

LTR was born on human relevance judgments (graded 0-4). Production search mostly trains on clicks, which are biased by position. Google's "Learning to Rank with Selection Bias in Personal Search" (Wang et al., SIGIR 2016) addressed this for email and personal search, where editorial judgments are impossible for privacy reasons [4]. Joachims et al. (WSDM 2017) formalized **inverse propensity scoring** for LTR: weight each click by 1 / P(examined at position k), with propensities estimated from randomized interventions [5] (not re-fetched).

Airbnb took a simpler route in its deep ranker: **position as an input feature with dropout**, set to a constant at inference time. Too little dropout lets the model lean on position. Too much stops it from separating position from quality. The tuned version gave a significant booking gain [14].

31.2.6 Airbnb: a LambdaRank-style DNN in production

Airbnb's "Applying Deep Learning to Airbnb Search" is the best public log of moving a LambdaMART-era ranker to a neural network [13]:

- A GBDT ranker with heavy hand-made features had plateaued.
- A simple network with one hidden layer of 32 ReLU units reached about parity.
- Switching to a **pairwise LambdaRank-style loss** (booked versus not-booked listings in the same search, weighted by the NDCG change of a swap) helped.
- The final model had 2 hidden layers, about 195 features and was trained on the order of 1.7 billion pairs (approximate).
- Listing-ID embeddings overfit (a listing can be booked at most about 365 times per year), and a multi-task "long view" head raised views but not bookings.
- Offline NDCG wins that did not reproduce online taught the team to distrust offline gains [13].

Lesson. The loss family (pairwise, metric-weighted) survived the move from trees to networks. What changed was feature engineering effort and the amount of data the model could use.

31.3 Query understanding and semantic matching

31.3.1 What query understanding does

Daniel Tunkelang's "Query Understanding" series frames it as everything that happens to the query before retrieval: language identification, tokenization, spelling correction, segmentation, entity and attribute recognition, query classification (category, intent), relaxation, expansion and rewriting [42] (not re-fetched). In commerce search, the most valuable outputs are usually:

- **Category or product-type prediction**, used both as a retrieval filter and as a ranking feature.
- **Attribute extraction** (brand, color, size, model number), turned into structured filters.
- **Rewrites and relaxations** for null or low-recall queries.
- **A dense query embedding**, which is the input to EBR.

31.3.2 DSSM: the first two-tower search model

"Learning Deep Structured Semantic Models for Web Search using Clickthrough Data" (Huang, He, Gao, Deng, Acero and Heck, Microsoft, CIKM 2013) maps queries and documents into a shared low-dimensional space and measures relevance as the distance between them. It is trained to maximize the likelihood of clicked documents given a query, and uses **word hashing** to scale to web-size vocabularies (letter-trigram hashing in the paper). The best model beat earlier latent semantic models on a real web ranking dataset [7]. DSSM is the direct ancestor of every two-tower model in this atlas.

31.3.3 Transformer query encoders

Google announced in 2019 that BERT was being used in Search ranking and featured snippets, helping with roughly 1 in 10 US English queries, especially longer, conversational queries where prepositions change the meaning [41] (not re-fetched). BERT-class cross-encoders are too slow to score thousands of candidates, so production systems use them in three ways: as **teachers** distilled into two-tower or small rankers, as **late-stage re-rankers** over tens of documents, or as **offline labelers** for relevance data.

Two research lines try to keep transformer quality at retrieval speed:

- **Late interaction (ColBERT).** Each passage is encoded into a matrix of token embeddings. At query time, each query token takes the MaxSim over passage tokens, and the sums give the score. ColBERTv2 adds residual compression (2-bit examples in the repository), and the PLAID engine adds centroid-based pruning with knobs such as `ncells`, `centroid_score_threshold` and `ndocs` [26].

- **Learned sparse retrieval (SPLADE).** The BERT MLM head produces sparse expansions of queries and documents, with a sparsity regularizer. Because the output is sparse, it runs on an ordinary inverted index (Anserini, PISA). Later work reports latency close to BM25, and the authors report better out-of-domain generalization on BEIR [27].

Design implication. Learned sparse models let a team get "semantic" recall without a separate vector stack, at the cost of a bigger posting-list index. Late interaction gives the best quality per query, at the cost of much larger storage.

31.4 Embedding-based retrieval in production search

31.4.1 Facebook Search: EBR inside the inverted index (KDD 2020)

Problem. Facebook Search ran on Boolean term matching in Unicorn, its inverted-index engine. Social search also needs the searcher's own context (location, social graph), which text matching cannot express [8].

Architecture. A **unified embedding** two-tower model. The query side encodes query text plus searcher context. The document side encodes text plus document context. Documents are quantized with FAISS, and the result is folded back into Unicorn: the coarse IVF cluster ID becomes an index **term** and the PQ-coded residual becomes the posting **payload**. One Unicorn query can then mix Boolean constraints with nearest-neighbor matching [8].

```
query text + searcher ctx --> query tower --> q (e.g. 128-d)
                                                |
     Unicorn:  (nn q :radius r)  AND  (term:location ...)  OR  (lexical terms)
                                                |
index:  term = IVF centroid id ; payload = PQ(residual)
                                                |
hybrid candidate set --> ranking (embedding similarity also a ranking feature)
```

Training lessons [8]:

- Impressed-but-not-clicked results are **bad negatives** for retrieval. Using them caused about a 55% absolute recall regression for the people model versus random negatives.
- **Online hard negatives** (the most similar other positives in the batch) gave +8.38% recall for people, +7% for groups and +5.33% for events.
- Unified (text + context) embeddings beat text-only embeddings by +18% recall for events and +16% for groups.

ANN lessons. PQ size around d/4 bytes, re-tune nprobe and cluster counts on every major model change, and choose final settings online. In one 128-d test, PCA+PQ16 reached 51.62% 1-Recall@10 versus 74.81% for a flat index [8].

Lesson. The cheapest way to add semantic retrieval to an existing search engine is to make vectors look like terms. Filters, sharding and serving then come for free.

31.4.2 Amazon Semantic Product Search (KDD 2019)

Nigam et al. built a neural matcher to complement Amazon's lexical product search [9]. Key choices (via the sibling Chapter 24 summary):

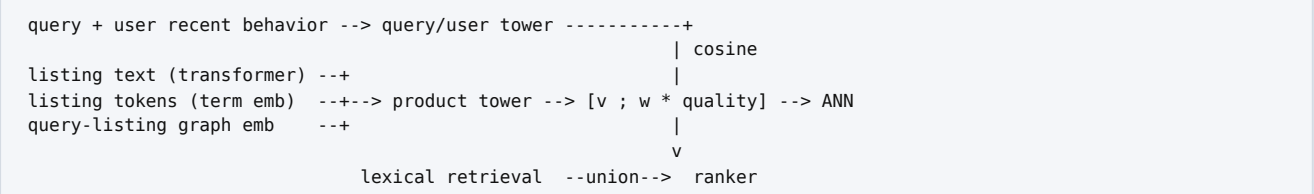
- A symmetric two-tower model with a **shared token embedding table** for queries and products.
- Tokens are word unigrams, bigrams and character trigrams. Out-of-vocabulary tokens are **hashed** into extra buckets, which mattered because product queries are full of model numbers, brands and misspellings.
- Simple **average pooling** was competitive with heavier encoders.
- A hinge-style loss separates purchased, impressed-not-purchased and random products.
- Semantic retrieval runs **alongside** lexical matching. Lexical stays precise for exact-attribute queries; the semantic path adds recall for vocabulary-gap queries.

Evaluation data. Amazon later released the Shopping Queries (ESCI) dataset. The larger version has 130,652 queries and about 2.62 million judgments in English, Spanish and Japanese. The reduced version has 48,300 queries and 1,118,011 judgments. Each (query, product) pair is labelled Exact, Substitute, Complement or Irrelevant, and the KDD Cup 2022 tasks were query-product ranking, 4-way classification and substitute identification [10]. Graded, typed relevance is the commerce analogue of the 0-4 web judgments that LambdaMART was built for.

31.4.3 Etsy Unified Embedding (KDD 2023)

Etsy's long tail of one-of-a-kind items and vague aesthetic queries ("boho wedding decor") defeats pure lexical retrieval. Jha et al. describe a two-tower **Unified Embedding** model [11]:

- **Product tower** fuses transformer text embeddings, term-based (bag-of-tokens) embeddings that keep lexical precision, and graph embeddings from query-product interaction data.
- **Joint query-user tower** encodes the query plus personalization features from recent behavior, so **retrieval itself is personalized**, not just ranking.
- **ANN-based boosting.** Quality signals such as price and seller quality are appended to the vectors as extra dimensions. The inner product then adds a tuned quality boost inside the ANN search. The weights were tuned with black-box optimization.
- Results are unioned with lexical retrieval before ranking.
- Online A/B: **+5.58% search purchase rate** and **+2.63% site-wide conversion** [11].



Lesson. Folding quality into the vector is a neat way to stop ANN from returning relevant but low-quality items, which a post-filter cannot fix once they have used up the top-K.

31.4.4 Taobao and JD: semantic retrieval with relevance control

Two Chinese e-commerce papers describe the same pattern at very large scale (both not re-fetched; see also Chapter 27):

- **Taobao, "Embedding-based Product Retrieval in Taobao Search" (Li et al., KDD 2021)** [38]. A multi-grained query tower combines several granularities of query tokens with the user's real-time, short-term and long-term behavior. Training uses a softmax over a large candidate set with a temperature. A key point is **relevance control**: because personalized EBR can drift away from the literal query, Boolean term constraints are applied to EBR results so that they still match key query terms. The design is a direct statement of the search-specific "precision floor".
- **JD, "Towards Personalized and Semantic Retrieval: An End-to-End Solution for E-commerce Search via Embedding Learning" (DPSR, Zhang et al., SIGIR 2020)** [39]. A query tower with multiple heads, each producing a different embedding, captures different intents and personalization. The paper covers the full system: offline training, an online indexing and serving layer, and its integration as a retrieval path next to the lexical engine.

31.4.5 Airbnb: from embeddings-as-features to EBR

Airbnb used embeddings in search for seven years before using them for retrieval.

1. **2018, embeddings as ranking features** [12]. Skip-gram listing embeddings were trained on about 800 million click sessions (approximate), 32 dimensions. Two tricks: the **booked listing is a global context** for every listing in a booking session, and **negatives are sampled from the same market**. Real-time Kafka streams of clicked, skipped, wishlisted and contacted listings produce "similarity to clicked/skipped listings" features for the then-GBDT ranker.
2. **2020, "ideal listing" two-tower ranker** [14]. A query/guest tower produces an "ideal listing" vector and the score depends on distance to each listing vector. This removed a learned "cheaper is always better" bias.
3. **2023, Journey Ranker** [15]. Multi-task heads along the booking funnel (click, listing page, payment page, request, booking), a twiddler for negative outcomes (cancellations, host rejections), context-dependent weights and monotonic constraints.
4. **2022-2023, learning to rank diversely** [16]. A listing's score depends on the listings ranked above it, applied greedily.
5. **2025-2026, EBR** [17][18]. A two-tower model whose listing tower runs in a daily batch and whose query tower runs per search. It is trained with **session-based hard negatives** (seen but not booked). HNSW gave slightly better recall, but Airbnb chose **IVF**, because frequent price and availability updates made HNSW's memory overhead painful. With IVF, a listing's cluster becomes an ordinary filter in the existing search system. Significant booking gains are reported [18].

```
guest query (place, dates, guests, filters) + guest context
       |
       v
retrieval: geo + availability filters  AND  IVF cluster ids of query vector [17]
       |  (order of thousands of listings, approximate)
       v
base ranker: two-tower / multi-task journey DNN, pairwise loss [14][15]
       |
       v
second pass: diversity given listings above [16], twiddlers [15]
       |
       v
interleaving for fast screening (~50x sensitivity vs A/B, approximate) [19]
```

Lesson. Facebook and Airbnb both reached the same conclusion: in search, the ANN index should fit into the filter engine. For Facebook that meant cluster IDs as terms; for Airbnb, IVF clusters as filters.

31.4.6 Pinterest OmniSearchSage (WWW 2024)

Pinterest search must match a text query to Pins, products and other entities. OmniSearchSage learns a **single query embedding** that is trained jointly against several entity types (Pins and products) and several tasks, in the same space as Pinterest's existing entity embeddings [20]. Item-side text is enriched with signals such as board titles, engaged queries and generated captions (reported in the abstract; details not re-fetched). Pinterest reports gains in offline relevance and online search engagement; check [20] for the exact figures.

The serving side runs on Manas, Pinterest's in-house ANN platform [21]. Its 2026 evolution post benchmarks a 100-million-embedding set: HNSW 121 GB at 93.72% Recall@100, HNSW with scalar quantization 50 GB at 92.92%, IVF 97 GB at 91.69%, and IVF with scalar quantization 25 GB at 95.71%. PQ compressed more but lost much more recall. Manas also implements SPANN for SSD-resident indexes, with an in-memory HNSW over centroids [21]. Manas' streaming filters reuse inverted-index posting lists (bitmap and array-backed) as filter stores, so lexical-style constraints apply during ANN search.

Lesson. One query embedding that lives in the same space as the item embeddings used by recommendation turns search and related-item retrieval into the same ANN problem over the same index.

31.5 Hybrid lexical plus vector retrieval

31.5.1 Why hybrid wins

All the production systems above kept lexical retrieval [8][9][11][17][38]. The reasons are consistent:

- **Exact-match queries** (SKUs, model numbers, names, rare tokens) are where dense models are weakest and lexical is near perfect.
- **Hard filters** (availability, price, location, category, permissions) are native to inverted indexes.
- **Explainability and debuggability.** A lexical match shows why a result was returned.

- **Cold start.** A new item is lexically searchable the moment it is indexed, before any embedding refresh.

Dense retrieval adds recall for vocabulary-gap queries, synonyms, multilingual matches, and personalization through user features in the query tower.

31.5.2 Five integration patterns

Pattern	How it works	Example	Strengths	Weaknesses
Parallel union	Run lexical and ANN separately, union, rank	Etsy [11], Amazon [9]	Simple; independent scaling	Duplicate infra; ranker must calibrate both
Vectors as terms	IVF cluster ID as an index term, PQ residual as payload	Facebook Unicorn [8]	One engine, one query language, filters free	Recall limited by IVF and PQ settings
Clusters as filters	IVF assignment stored as a filterable attribute	Airbnb [17]	Handles frequent updates; fits existing filters	Coarse; more candidates to re-score
In-engine hybrid ranking	BM25 and closeness both computed per hit; fused in rank phases	Vespa [23][24], Elasticsearch LTR [6]	Score fusion and LTR near data	Engine lock-in; tuning per dataset
Learned sparse	One sparse model gives lexical + expansion	SPLADE [27]	One inverted index	Bigger postings; less personalizable

31.5.3 Score fusion

When two retrieval paths return ranked lists, they must be combined before (or as input to) the ranker.

- **Reciprocal rank fusion (RRF).** Cormack, Clarke and Buettcher (SIGIR 2009) score each document as the sum over lists of 1 / (k + rank), with k = 60 [25] (not re-fetched). RRF needs no score calibration, which makes it a safe default.
- **Normalized linear combination.** Map each score to [0, 1] (min-max over the candidate set, or a squashing function such as arctan for BM25) and add with weights.

Vespa's hybrid-search tutorial is a useful reality check [23]. On NFCorpus test queries (nDCG@10): BM25 alone 0.3210, dense (a 384-d embedding model) 0.3077, closeness x (1 + BM25) 0.3330, raw sum 0.3244 (BM25's unbounded scale dominates), atan-normalized BM25 plus cosine 0.3410, RRF over the top 100 in a global phase 0.3233, and linear normalization in a global phase 0.3423. The tutorial's conclusion is that "the effectiveness of the hybrid model depends on the dataset", so the fusion method must be tested on your own data [23]. The tutorial also shows a cost trick: use the `rank()` operator to retrieve with one operator and compute the other only as a ranking feature, which avoids building an HNSW index at all [23].

Vespa's phased ranking documents the in-engine cascade that makes this cheap: a first phase on every matched document on content nodes, a second phase on the top hits per node, and a global phase on the merged top hits in the container. Global normalization and RRF need the global phase, because they need the full candidate set across nodes [24].

```
query --> [text() OR nearestNeighbor()]  on each content node
                |
          first-phase: cheap expression (e.g. bm25 + closeness) on all hits
                |
          second-phase: GBDT / small NN on top-N per node
                |
 container: global-phase over merged top-K: RRF or normalize, cross-encoder
                |
              results
```

31.5.4 Filtered ANN

Hybrid search almost always means ANN under predicates. The options seen in production:

- **Post-filtering** with over-fetch (early Manas): simple, but recall collapses for selective filters [21].
- **Pre-filter through index structure**: cluster IDs as terms or filters (Facebook, Airbnb) [8][17].
- **Filter-aware ANN**: Microsoft's DiskANN library now exposes attribute-filter hooks, range filters and diversity-aware top-k alongside real-time updates (IP-DiskANN, Fresh-DiskANN), and its legacy branch implements Filtered-DiskANN [29]. SPTAG, released by Microsoft Research and Bing, combines balanced k-means or kd trees with a relative neighborhood graph and supports online insertion and deletion; its SPANN and SPFresh papers target billion-scale, SSD-resident and updatable indexes [28].

31.5.5 Uber Eats: the hidden cost of hybrid retrieval

Uber's 2026 Eats search post shows what happens when many retrieval paths (lexical, embedding/ANN, geo) all feed one ranker [22]. Retrieval returned tens of thousands of candidates, and all of them were fully hydrated before the first-pass ranker discarded most. Removing low-yield retrieval paths saved about 120 ms. Splitting hydration into a light phase (ranking features only) and a full phase (presentation data for shown results) saved more than 100 ms more. The first-pass ranker must stay fast "while maintaining a high recall metric" [22].

Lesson. Each extra retrieval path costs more than its own latency: it inflates hydration and first-pass ranking downstream. Measure each path's marginal contribution to final results and prune.

31.6 Personalized search

31.6.1 Short-term versus long-term signals

Bennett, White, Chu, Dumais, Bailey, Borisyuk and Cui (Microsoft/Bing, SIGIR 2012) studied how within-session and historical behavior each help web search personalization [30]. Their findings still shape feature design:

- Historical (long-term) behavior helps most at the **start** of a session.
- Within-session behavior accounts for most gains in **longer** sessions.

- Combining both beats either alone, and their relative value shifts during the session [30].

Airbnb's 2018 real-time features are a direct industrial version: two-week Kafka streams of clicks and skips produce per-candidate similarity features, while booking-trained "type" embeddings capture long-term taste [12].

31.6.2 Where to personalize: retrieval or ranking

System	Personalization in retrieval	Personalization in ranking	Guardrail for relevance
Facebook Search [8]	Searcher context in query tower	Embedding similarity as feature	Boolean terms in same Unicorn query
Etsy [11]	Joint query-user tower	Downstream ranker	Union with lexical; quality boost dims
Taobao [38]	Real-time + long-term behavior in query tower	Downstream ranker	Boolean relevance control on EBR output
JD DPSR [39]	Multi-head personalized query tower	Downstream ranker	Lexical engine in parallel
Airbnb [12] [14][17]	Query tower over guest + query (EBR)	Session embedding features, ideal-listing tower	Hard availability and geo filters
Bing (study) [30]	n/a	Session + history re-ranking features	Base relevance ranker

The pattern: **personalize retrieval only with a relevance guardrail**. A purely personalized query tower drifts toward "what this user likes" and away from "what this user asked for", which is acceptable in a feed but is a bug in search.

31.6.3 Practical feature families for personalized search rankers

From the case studies above, the features that recur are:

- **Query-item relevance**: BM25 and field matches, semantic similarity (two-tower cosine), category-match probability.
- **User-item affinity**: similarity of the candidate to recently clicked or skipped items [12], to purchased categories, to long-term type embeddings.
- **User-query context**: is this a refinement of the previous query; time since session start [30].
- **Item quality and marketplace signals**: ratings, price normalized within the result set (as in Expedia-style LTR), availability, seller quality [11].
- **Position and presentation**: used at training time only, with dropout or propensity weighting [4][5][14].

31.7 Unified search and recommendation

There are four levels of unification, in increasing order of ambition.

```
L1 shared infra      same ANN platform, feature store, ranking service
                     (Manas, Vespa, Unicorn)
L2 shared space      one embedding space for queries, users and items
                     (OmniSearchSage, Etsy, Facebook unified embedding)
L3 joint behavior    search history feeds recommendation and rec history feeds
                     search (UniSAR, Bing short/long-term)
L4 one model         a single generative model serves both tasks
                     (Spotify GR study, Amazon SynerGen, Kuaishou OneSearch)
```

31.7.1 L2: shared embedding spaces

Facebook's unified embedding [8], Etsy's joint query-user tower [11] and Pinterest's OmniSearchSage [20] share one idea: the query vector lives in the same space as item vectors that recommendation surfaces also use. Benefits: one index per entity type, transfer of engagement signal from recommendation into search (and vice versa), and cheap "search-like" recommendation modules (for example, related items for the current query).

31.7.2 L3: modeling transitions between search and recommendation

UniSAR ("Modeling User Transition Behaviors between Search and Recommendation", Shi et al., SIGIR 2024) models the user's interleaved stream of search and recommendation actions. Its official PyTorch implementation trains on the Amazon dataset and **KuaiSAR**, a Kuaishou dataset that logs both search and recommendation behavior for the same users [31]. The paper's premise is that what a user searched for is a strong signal of recommendation intent, and what they watched from recommendations disambiguates their next query. In industry this shows up as **cross-surface features**: recent queries as features in the feed ranker, recent feed engagement as features in the search ranker. The Bing study shows the same session-dynamics effect inside search alone [30].

31.7.3 L4: one generative model for both tasks

Generative retrieval represents each item as a short sequence of discrete **semantic ID** tokens (Chapter 6) and trains a sequence model to emit them. A query, a user history, or both can be the input, so search and recommendation become the same decoding problem.

Spotify: does one task help the other? Penha et al. (Spotify, RecSys 2024) trained generative retrieval models jointly on search (query to item) and recommendation (history to item) and asked whether multi-task training helps [32] (not re-fetched). The paper reports that the joint model can outperform task-specific models, and attributes gains to two effects: joint training regularizes item popularity estimates, and it regularizes item latent representations through signals from both tasks. Gains depend on how much the two tasks' item distributions overlap.

Amazon: SynerGen (2025). "SynerGen: Contextualized Generative Recommender for Unified Search and Recommendation" is a single generative model that conditions on context and serves both search and recommendation [36] (listed in [37]; details not re-fetched).

Kuaishou: OneSearch (2025) and OneSearch-V2 (2026). OneSearch applies Kuaishou's "One" end-to-end generative recipe (Chapter 6's OneRec) to e-commerce search [33]. It replaces the cascade (lexical/semantic recall, pre-rank, rank) with a model that takes the query and user behavior and decodes item semantic IDs. The tokenizer uses **3**

residual-quantization layers (4096/1024/512 codes) plus 2 OPQ codes (2 x 256) to separate near-duplicate products (via the Chapter 6 summary of [33]). The paper also adds keyword-aware item encoding so that tokens respect query terms, multi-view user behavior inputs, and a preference-aware reward for post-training (abstract-level; details not re-fetched). OneSearch-V2 (March 2026) adds latent reasoning and self-distillation [34]. The same team built **OneSug** (2025) for query suggestion, an end-to-end generative model that unifies the multi-stage suggestion cascade [35].

```
OneSearch-style unified generative search (simplified)

query text --> keyword-aware encoding --+
user behavior (search + rec history) --+--> encoder
                                       |
                                       v
                              decoder (beam search over
                              valid semantic-ID prefixes)
                                       |
           SID tokens:  [RQ1 4096][RQ2 1024][RQ3 512][OPQ 256][OPQ 256]
                                       |
                                       v
                  items  --> optional light re-rank --> results
training: SFT on (query, user) -> purchased/clicked item SIDs,
          then preference / reward alignment
```

31.7.4 What unification buys and costs

Dimension	Separate stacks	Shared space (L2)	Joint behavior (L3)	One generative model (L4)
Infra cost	2x indexes, rankers	1 index per entity	2 stacks, shared features	1 model, 1 serving path
Cold-start items	Lexical covers search	Content towers help both	Same as base	SID from content helps
Relevance guarantee	Strong (lexical)	Needs lexical union	Strong	Weakest; needs constrained decoding and filters
Cross-task transfer	None	Through embeddings	Through features	Through all parameters [32]
Debuggability	High	Medium	Medium	Low
Public scale evidence	Everywhere	Facebook, Etsy, Pinterest [8][11][20]	Research + features [30][31]	Kuaishou, Amazon, Spotify [32] [33][36]

The main risk of L4 in search is the precision floor. A generative model can emit a fluent but wrong item, and SKU-like exact queries are where semantic IDs are weakest. Production designs therefore keep constrained decoding over valid ID prefixes, filter checks after decoding, and often a lexical fallback path.

31.8 Reference architecture: a unified discovery stack

Combining the lessons, a modern design that serves search, related items and home feed from one platform looks like this:

```
                    +------------------- shared feature store -------------------+
                    | user long/short-term seqs (search + rec), item stats,      |
                    | query stats, real-time session events                      |
                    +------------------------------------------------------------+
request(surface, query?, user, ctx)
      |
      v
query understanding (only if query): spell, classify, attributes, rewrite, encode
      |
      v
retrieval portfolio (per surface quotas)
 - lexical / learned sparse (search only; also relevance guardrail)
 - unified two-tower ANN (query+user tower; empty query for feed)
 - item-to-item / graph (related items; expansion from top search hits)
 - generative SID decoding (optional, constrained)
      |
      v
merge: RRF or learned blend; dedup; filter (availability, policy)
      |
      v
first pass (in-index or near data): GBDT LambdaMART / small DNN
      |
      v
second pass: multi-task DNN (click, purchase/booking, negative outcomes),
             surface-specific heads, position debiasing
      |
      v
re-rank: diversity, ads, business rules  --> page
      |
      v
logs + interleaving + A/B
```

Design rules embedded in this diagram:

1. The **query tower accepts an empty query**. Feed requests and search requests use one model, and the surface is a feature.
2. **Lexical retrieval is a guardrail as well as a source.** Its matches can gate or boost semantic results for search [38].
3. **Per-surface heads** let search optimize relevance-constrained conversion while feed optimizes engagement.
4. **One feature store** guarantees that search history is visible to recommendation and vice versa [30][31].

31.9 Key takeaways for system designers

1. **LambdaMART is still the baseline to beat.** It is fast, robust and directly targets NDCG. Start with LightGBM `lambdarank`, a sensible `lambdarank_truncation_level` and exponential label gains, then move to neural rankers when features and data justify it [1][2]. Airbnb kept the pairwise, metric-weighted loss when it switched to a DNN [13].
2. **Fix the labels before the model.** Click logs are position-biased. Use propensity weighting or position-as-feature-with-dropout [4][5][14]. Typed relevance labels (ESCI) beat binary clicks for commerce [10].
3. **Keep lexical retrieval.** Every production search system here kept it for exact matches, filters and cold start. Add EBR next to it, not instead of it [8][9][11][17].
4. **Make the ANN index speak the filter engine's language.** Cluster IDs as terms (Facebook) or as filters (Airbnb), or streaming filters over posting lists (Pinterest), beat post-filtering [8][17][21].

5. **Negatives define retrieval quality.** Random plus in-batch hard negatives work; impressed-not-clicked results are poor retrieval negatives (about 55% recall loss at Facebook). Session-based hard negatives work for Airbnb because they are true alternatives within the same intent [8][17][18].
6. **Personalize retrieval only with a relevance guardrail.** Boolean constraints, lexical union, or quality and relevance dimensions in the vector [11][38].
7. **Fusion is dataset-dependent.** RRF is the safe default. Normalized linear fusion was best on Vespa's benchmark, but by small margins. Always measure [23][25].
8. **Count the downstream cost of each retrieval path.** Uber saved 120 ms by removing low-yield paths and over 100 ms more with two-phase hydration [22].
9. **Use session and long-term signals together.** Long-term history matters early in a session; in-session behavior dominates later [30].
10. **Unify in steps.** Shared infrastructure and a shared embedding space are proven at scale. Joint behavior features are cheap. A single generative model for search and recommendation is now in production at Kuaishou and studied at Spotify and Amazon, but it needs constrained decoding and a precision guardrail for search [20][32][33][36].
11. **Use interleaving to iterate on rankers.** Airbnb reports about 50x sensitivity over A/B for search ranking changes (approximate), with A/B as the final gate [19].

References

1. Burges, C. J. C. "From RankNet to LambdaRank to LambdaMART: An Overview." Microsoft Research Technical Report MSR-TR-2010-82, 2010. https://www.microsoft.com/en-us/research/publication/from-ranknet-to-lambdarank-to-lambdamart-an-overview/
2. Microsoft. "LightGBM Parameters" (lambdarank, rank_xendcg, lambdarank_truncation_level, label_gain, lambdarank_norm). LightGBM documentation, accessed 2026. https://github.com/microsoft/LightGBM/blob/master/docs/Parameters.rst
3. Google. "TF-Ranking: Learning-to-Rank in TensorFlow" (repository, archived 2025). https://github.com/tensorflow/ranking
4. Wang, X., Bendersky, M., Metzler, D., Najork, M. "Learning to Rank with Selection Bias in Personal Search." Google, SIGIR 2016. https://research.google/pubs/learning-to-rank-with-selection-bias-in-personal-search/
5. Joachims, T., Swaminathan, A., Schnabel, T. "Unbiased Learning-to-Rank with Biased Feedback." Cornell, WSDM 2017 (not re-fetched). https://arxiv.org/abs/1608.04468
6. OpenSource Connections. "Elasticsearch Learning to Rank" plugin (repository and README). https://github.com/o19s/elasticsearch-learning-to-rank
7. Huang, P.-S., He, X., Gao, J., Deng, L., Acero, A., Heck, L. "Learning Deep Structured Semantic Models for Web Search using Clickthrough Data." Microsoft Research, CIKM 2013. https://www.microsoft.com/en-us/research/publication/learning-deep-structured-semantic-models-for-web-search-using-clickthrough-data/
8. Huang, J.-T., et al. "Embedding-based Retrieval in Facebook Search." Facebook, KDD 2020. https://arxiv.org/abs/2006.11632

9. Nigam, P., Song, Y., Mohan, V., et al. "Semantic Product Search." Amazon, KDD 2019. https://arxiv.org/abs/1907.00937
10. Reddy, C. K., et al. "Shopping Queries Dataset: A Large-Scale ESCI Benchmark for Improving Product Search." Amazon, 2022. Dataset repository: https://github.com/amazon-science/esci-data ; paper: https://arxiv.org/abs/2206.06588
11. Jha, R., Subramaniyam, S., Benjamin, E., Taula, T. "Unified Embedding Based Personalized Retrieval in Etsy Search." Etsy, KDD 2023. https://arxiv.org/abs/2306.04833
12. Grbovic, M., Cheng, H. "Real-time Personalization using Embeddings for Search Ranking at Airbnb." Airbnb, KDD 2018. https://dl.acm.org/doi/10.1145/3219819.3219885
13. Haldar, M., Abdool, M., Ramanathan, P., et al. "Applying Deep Learning to Airbnb Search." Airbnb, KDD 2019. https://arxiv.org/abs/1810.09591
14. Haldar, M., Ramanathan, P., Sax, T., et al. "Improving Deep Learning for Airbnb Search." Airbnb, KDD 2020. https://arxiv.org/abs/2002.05515
15. Tan, C. H., et al. "Optimizing Airbnb Search Journey with Multi-task Learning." Airbnb, KDD 2023 (not re-fetched). https://arxiv.org/abs/2305.18431
16. Abdool, M., Haldar, M., et al. "Learning To Rank Diversely At Airbnb." Airbnb, 2022 (CIKM 2023). https://arxiv.org/abs/2210.07774
17. Airbnb Engineering. "Embedding-Based Retrieval for Airbnb Search." Airbnb Tech Blog, 2025. https://airbnb.tech/ai-ml/embedding-based-retrieval-for-airbnb-search/
18. Airbnb. "Applying Embedding-Based Retrieval to Airbnb Search." arXiv, 2026. https://arxiv.org/pdf/2601.06873
19. Airbnb Engineering. "Beyond A/B Test: Speeding up Airbnb Search Ranking Experimentation through Interleaving." Airbnb Tech Blog, 2022. https://medium.com/airbnb-engineering/beyond-a-b-test-speeding-up-airbnb-search-ranking-experimentation-through-interleaving-7087afa09c8e
20. Agarwal, P., et al. "OmniSearchSage: Multi-Task Multi-Entity Embeddings for Pinterest Search." Pinterest, WWW 2024 companion. https://arxiv.org/abs/2404.16260
21. Pinterest Engineering. "Evolving Pinterest's Embedding Retrieval Platform." Pinterest Engineering Blog, 2026. https://medium.com/pinterest-engineering/evolving-pinterests-embedding-retrieval-platform-aede4e831e01 (summary: InfoQ, "From Memory-Hungry HNSW to Quantized SPANN", 2026, https://www.infoq.com/news/2026/09/pinterest-search/ ; streaming filters: https://medium.com/pinterest-engineering/manas-hnsw-streaming-filters-351adf9ac1c4)
22. Uber Engineering. "Uber Eats Search Pipeline." Uber Blog, 2026. https://www.uber.com/us/en/blog/uber-eats-search-pipeline/
23. Vespa.ai. "Hybrid Text Search Tutorial." Vespa documentation (source on GitHub), accessed 2026. https://github.com/vespa-engine/documentation/blob/master/en/learn/tutorials/hybrid-search.md
24. Vespa.ai. "Phased Ranking." Vespa documentation, 2025-2026. https://docs.vespa.ai/en/ranking/phased-ranking.html
25. Cormack, G. V., Clarke, C. L. A., Buettcher, S. "Reciprocal Rank Fusion Outperforms Condorcet and Individual Rank Learning Methods." University of Waterloo, SIGIR 2009 (not re-fetched). https://plg.uwaterloo.ca/~gvcormac/cormacksigir09-rrf.pdf
26. Khattab, O., Zaharia, M., Santhanam, K., et al. "ColBERT / ColBERTv2 / PLAID" (repository; papers arXiv 2004.12832, 2112.01488, 2205.09707). Stanford, 2020-2022. https://github.com/stanford-futuredata/ColBERT

27. Formal, T., Lassance, C., Piwowarski, B., Clinchant, S. "SPLADE: sparse neural search" (repository; papers arXiv 2107.05720, 2109.10086, 2205.04733). NAVER Labs Europe, 2021-2022. https://github.com/naver/splade
28. Microsoft Research and Bing. "SPTAG: A library for fast approximate nearest neighbor search" (repository; SPANN NeurIPS 2021, SPFresh SOSP 2023). https://github.com/microsoft/SPTAG
29. Microsoft. "DiskANN" (repository: DiskANN3, Fresh/IP-DiskANN, Filtered-DiskANN). Accessed 2026. https://github.com/microsoft/DiskANN
30. Bennett, P. N., White, R. W., Chu, W., Dumais, S. T., Bailey, P., Borisyuk, F., Cui, X. "Modeling the Impact of Short- and Long-Term Behavior on Search Personalization." Microsoft, SIGIR 2012. https://www.microsoft.com/en-us/research/publication/modeling-the-impact-of-short-and-long-term-behavior-on-search-personalization/
31. Shi, T., et al. "UniSAR: Modeling User Transition Behaviors between Search and Recommendation." Renmin University of China / Kuaishou data, SIGIR 2024. Official implementation: https://github.com/TengShi-RUC/UniSAR
32. Penha, G., et al. "Bridging Search and Recommendation in Generative Retrieval: Does One Task Help the Other?" Spotify, RecSys 2024 (not re-fetched). https://arxiv.org/abs/2410.16823
33. Kuaishou. "OneSearch: A Preliminary Exploration of the Unified End-to-End Generative Framework for E-commerce Search." 2025. https://arxiv.org/abs/2509.03236
34. Kuaishou. "OneSearch-V2: The Latent Reasoning Enhanced Self-distillation Generative Search Framework." 2026. https://arxiv.org/abs/2603.24422
35. Kuaishou. "OneSug: The Unified End-to-End Generative Framework for E-commerce Query Suggestion." 2025. https://arxiv.org/abs/2506.06913
36. Amazon. "SynerGen: Contextualized Generative Recommender for Unified Search and Recommendation." 2025. https://arxiv.org/abs/2509.21777
37. Huang, Y., et al. "Awesome-Generative-Recommendation" (curated industrial paper list). 2025-2026. https://github.com/uestc-huangyw/Awesome-Generative-Recommendation
38. Li, S., et al. "Embedding-based Product Retrieval in Taobao Search." Alibaba, KDD 2021 (not re-fetched). https://arxiv.org/abs/2106.09297
39. Zhang, H., et al. "Towards Personalized and Semantic Retrieval: An End-to-End Solution for E-commerce Search via Embedding Learning." JD.com, SIGIR 2020 (not re-fetched). https://arxiv.org/abs/2006.02282
40. Yan, E. "System Design for Recommendations and Search." eugeneyan.com, 2021. https://eugeneyan.com/writing/system-design-for-discovery/
41. Nayak, P. "Understanding searches better than ever before." Google, The Keyword blog, 2019 (not re-fetched). https://blog.google/products/search/search-language-understanding-bert/
42. Tunkelang, D. "Query Understanding" (essay series). 2017 onward (not re-fetched). https://queryunderstanding.com/
43. Bruch, S. "An Alternative Cross Entropy Loss for Learning-to-Rank" (XE-NDCG). 2019 (linked from LightGBM docs; not re-fetched). https://arxiv.org/abs/1911.09798

32. Notification, Push and Email Recommendation: Volume, Fatigue and Timing

> Research note for this chapter: the session's shared web-search quota ran out before this chapter's research began, and most company blog and paper hosts (linkedin.com, medium.com, arxiv.org, research.duolingo.com) did not resolve from the research sandbox. The only sources fetched and read in this session are the open-sourced X/Twitter `pushservice` code and READMEs [1][2][3][4][5]. Every other section summarizes peer-reviewed papers the author already knew. Their headline results are given qualitatively or marked "approximately", and their URLs (mostly DOIs and arXiv IDs) should be checked again before print. Read those sections as a guide to the literature, not as freshly verified numbers.

Overview

Notification recommendation is the "push" side of recommender systems. A feed ranker answers "what should this user see now that they are here?" A notification system has to answer three harder questions together:

1. **Whether** to interrupt the user at all (send or don't send).
2. **What** to send (which candidate item, template or channel).
3. **When** and **how often** to send (timing, volume, fatigue).

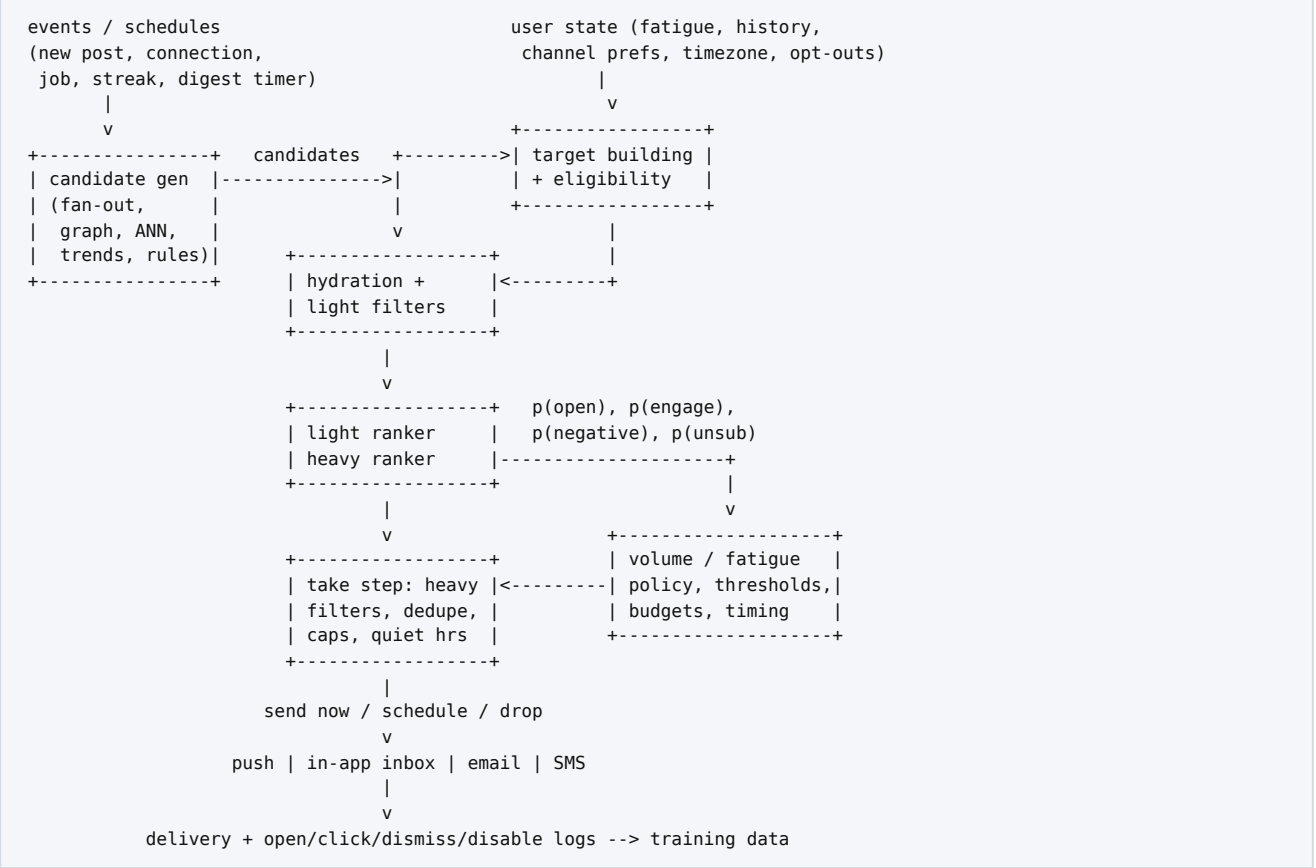
These differences shape the architecture:

- **The user is not present.** The system starts the interaction, so there is no request context. Systems are either event-driven (a friend posted, a job matched) or schedule-driven (a daily digest, a streak reminder).
- **Every action has a hidden cost.** A badly targeted push can lead to a disabled notification channel, an email unsubscribe or an app uninstall. These are near-permanent losses of the channel. Click-through rate (CTR) maximization alone pushes volume upward, and the cost shows up weeks later as churn. That is why most mature systems optimize a *long-term* objective, such as sessions or daily/weekly active users (DAU/WAU) over a horizon, against a negative-feedback penalty.
- **Strong counterfactual confounding.** Users who get more notifications are already more active. Observational CTR data overstates how much notifications cause activity, so production systems depend on randomized holdouts, volume-perturbation experiments and causal or uplift modeling.
- **Several channels share one attention budget.** Push, in-app badge/inbox ("NTab" in X's code), email and SMS all compete for it. A send on one channel changes the response on the others.

The chapter covers the open-sourced X (Twitter) push stack in depth (the only system here whose code was read directly). It then covers the published designs from LinkedIn (Concourse, Air Traffic Controller, survival-model timing, offline reinforcement learning), Pinterest (notification volume control), Duolingo (sleeping, recovering bandits), Twitter's "should I send?" model-the-future work, and the Meta (Facebook and Instagram) notification setting. It ends with the shared patterns for fatigue modeling and send-time optimization.

32.1 The canonical notification recommendation pipeline

Nearly every published system breaks down into the same stages. They differ mainly in where the "send or not" decision sits and which objective trains it.



The design decisions that separate systems are:

Decision	Options seen in practice
Trigger	event-driven near-real-time (LinkedIn Concourse), scheduled batch (digests), hybrid
Send decision	fixed threshold on p(open) or p(engage); per-user threshold from a volume model; RL policy
Objective	open/click CTR; sessions or DAU over a horizon; multi-objective with unsub/disable penalty
Volume control	global caps; per-user budgets from causal response curves (Pinterest); learned thresholds
Timing	send immediately; delay to the predicted best hour; survival/hazard model (LinkedIn)
Template choice	A/B tests; multi-armed bandit with novelty decay (Duolingo)
Fatigue features	recent send counts, time since last send, recent dismissals, "show less" feedback

32.2 X / Twitter: `pushservice` (open-sourced 2023)

Problem

X sends recommended pushes. Some are in-network (activity from accounts the user follows). Many are out-of-network (OON): popular or relevant posts, trends, Spaces, topics and "magic fanout" events such as news and sports. The service has to decide, per user, whether any candidate is good enough to deliver as a push plus an in-app notification-tab entry.

Architecture (from the open-source code)

The `pushservice` README describes two entry points [2]:

RefreshForPushHandler is the recommendation path. For a given user ID it:

1. Builds a *target* object (the recipient) and applies target-level eligibility filters.
2. Queries several candidate sources using the target.
3. Hydrates candidates with batched downstream calls.
4. Applies lightweight, RPC-based "light" pre-rank filters.
5. Hydrates features for candidates and target, then runs a **light ranker** and a **heavy ranker**.
6. Runs a **take step**. It walks the ranked list from the top and applies "heavy" filters one candidate at a time until **one** candidate passes.
7. Sends that candidate as a push and in-app notification.

SendHandler is the direct-send path for a candidate supplied by the caller. It builds the target, hydrates, runs heavy filtering and validation, and sends. It skips candidate generation and ranking.

```
user id --> Target(eligibility) --> candidate sources --> hydrate
       --> light filters --> LIGHT RANKER (MLP) --> HEAVY RANKER (ClemNet MTL)
       --> TAKE: for c in ranked: if heavy_predicates(c): send(c); break
```

The take step is a key design choice. Expensive checks (quality-model predicates, fatigue and caret-feedback checks, health checks, engagement-ratio checks) run *after* ranking and only until the first pass. Their cost therefore scales with the rejection rate, not the candidate count.

Candidate types

The `model` package lists the push candidate types [5]. They include `F1FirstdegreeTweet` (in-network posts), `OutOfNetworkTweetPushCandidate`, `TopTweetImpressionsPushCandidate`, `TrendTweetPushCandidate`, `TripTweetPushCandidate`, `TopicProofTweetPushCandidate`, `SubscribedSearchTweetPushCandidate`, `ListRecommendationPushCandidate`, `DiscoverTwitter`, scheduled-Space speaker and subscriber candidates, and a family of "MagicFanout" candidates (news events, sports, product launches, creator events). They also include social-proof actions such as `TweetFavorite` and `TweetRetweet`.

Ranking models

- **Light ranker.** It sits between candidate generation and heavy ranking and "pre-selects highly-relevant candidates from the initial huge candidate pool". The model is defined in `model_pools_mlp.py` [3]. The file name suggests a multilayer perceptron (MLP).
- **Heavy ranker.** The README calls it a multi-task model that estimates the probability that targets will open and engage with sent notifications [1][2]. The architecture is **ClemNet** [4]: a stack of blocks, each made of a `ChannelWiseDense` layer, batch norm (momentum 0.9999), an activation, then a 1-D convolution, batch norm and activation, with an optional residual connection. The output is flattened into a logits layer.
- **Default configuration** (from `params.py`, as read in this session): sparse features hashed to 2^18 buckets (`bits = 18`) with embedding size 50; four blocks with dense output sizes 1024, 512, 256 and 128; ReLU activations; conv kernel size 3 with 5 filters; residual off by default.
- **Tasks and score weights.** Two default tasks:
  - `OONC` on `label`, weight 0.9. The acronym is not expanded in the files read; it is commonly read as "open or NTab click", which fits the open-focused task description.
  - `Engagement` on `label.engagement`, weight 0.1.

  The final score is therefore a weighted mix that leans heavily toward the open/click probability.
- **Negative-feedback label.** The feature config always adds `label.ntabDislike` [4]. "Dislike" feedback in the notification tab is a first-class training label, the raw material for fatigue- and annoyance-aware scoring.

Predicates: where fatigue and quality live

The `predicate` package [5] shows how much of the "send or not" logic sits in rule and model predicates outside the ranker:

- **Fatigue:** `FatiguePredicate`, `TargetNtabCaretClickFatiguePredicate`, `CaretFeedbackHistoryFilter`, and an `ntab_caret_fatigue` subpackage. The "caret" is the per-notification "show less / see fewer like this" menu; recent caret clicks suppress similar future pushes.
- **Quality and health:** `BqmlQualityModelPredicates`, `JointDauAndQualityModelPredicate`, `OutOfNetworkCandidatesQualityPredicates`, `BqmlHealthModelPredicates`, `HealthPredicates`, and `PNegMultimodalPredicates` (a predicted-negative model). `JointDauAndQualityModelPredicate` suggests that gating combines a DAU-impact model with a quality model rather than relying on CTR alone.
- **Spread and exploration:** `OONSpreadControlPredicate` limits how widely an out-of-network post fans out. `BigFilteringEpsilonGreedyExplorationPredicate` adds epsilon-greedy exploration to filtering, so the training data includes some sends the filters would have blocked. `MlModelsHoldbackExperimentPredicate` keeps an ML holdback for causal measurement.

- **Content safety and other gates:** withheld content, language, engagement ratio, negative-feedback-based OON filtering, CAS locks (to stop concurrent duplicate sends), and decider (feature-flag) gates.

Lessons

- One candidate per refresh. The take step turns "rank many, send at most one" into the basic unit of work.
- Annoyance is modeled explicitly, both as a label (`ntabDislike`) and as stateful fatigue predicates.
- Exploration and holdbacks are built into the serving path, because notification training data is heavily biased by past send policies.

32.3 Twitter: "Should I send this notification?" (modeling the future)

Twitter researchers published work (circa 2022) on framing the send decision as a sequential decision rather than a one-shot CTR threshold [6]. The core argument is that the value of sending now depends on what the user will do later. A user who would open the app anyway gains little from a push. A push sent now may also displace a more valuable one later or speed up fatigue. The paper models the future with long-horizon user-activity targets and uses those predictions to set the send policy, reporting online gains over a myopic open-probability policy. The exact metrics were not re-verified in this session.

Design lesson: train the send/no-send decision on *incremental* long-term engagement (with send minus without send). Training on the probability of an open conditional on sending is not enough.

32.4 LinkedIn

LinkedIn has published one of the most complete series of notification papers. Its notifications span activity-based notifications ("X shared a post", "Y mentioned you"), network updates, job alerts, and email digests.

32.4.1 Concourse: near-real-time activity-based notifications

Problem. When a member's connection or followed entity creates content, LinkedIn must decide within minutes which of that creator's followers (possibly millions) should get a notification. Batch, offline scoring was too stale for timely content.

Architecture (from the KDD 2018 paper [7]; details from prior knowledge):

```
content creation event (Kafka)
         |
         v
+------------------+     follower / connection lists
| fan-out          |<-------------------------------------+
| (Samza stream    |                                      |
|  processors)     |---> per-(creator,content,recipient) candidates
+------------------+
         |
         v
+------------------+  features: recipient profile, affinity to creator,
| scoring (online  |  content features, notification history, fatigue
|  logistic/GLMix) |
+------------------+
         |
         v
+------------------+     decision events
| Air Traffic      |----------------------> push / in-app / email channels
| Controller (ATC): |
| volume caps,     |
| dedupe, channel, |
| timing           |
+------------------+
```

- The fan-out is stream-based (Apache Samza over Kafka). Each (content, recipient) pair is scored with a response-prediction model. Following the GLMix/Photon-ML lineage, the model has global and per-member components.
- The decision is a threshold on a utility that combines the predicted click/engagement probability with penalties for volume and for negative actions such as disables.
- **Air Traffic Controller (ATC)** is the centralized downstream "member-first" arbitration layer. It applies frequency caps, merges or deduplicates related notifications, picks the channel, and can delay delivery. Every upstream producer (activity notifications, jobs, recommendations) goes through ATC, so the global volume budget is enforced in one place.

Lessons. Separate per-item relevance (in the producers) from per-member volume control (centralized in ATC). Scoring in near real time made notifications for fresh content more relevant; the paper reports engagement lifts over the earlier offline pipeline.

32.4.2 Volume optimization as constrained multi-objective optimization

LinkedIn's notification and email volume work [7][8] frames the problem as maximizing sessions/engagement subject to an upper limit on negative outcomes such as unsubscribes, disables and complaints, or the reverse framing. Each send gets a utility like:

```
utility(send) = p(click) * w_click + E[delta sessions]
                - lambda * p(disable or unsubscribe)
send iff utility(send) > threshold_m        (threshold per member segment)
```

The Lagrangian dual of the constraint gives the multiplier lambda. It and the thresholds are tuned on randomized volume experiments that send more or fewer notifications to random groups of members, so the response curve can be estimated causally.

32.4.3 Timing: a state-transition / survival model

LinkedIn published a state-transition model of mobile notifications based on survival analysis (WSDM 2019) [9]. A member moves between states: notification received, visit, idle, and so on. Hazard models estimate the time to the next visit with and without a

notification. The send decision, including whether to delay a notification, uses the predicted *change in time to visit*. If the member is likely to visit organically soon, the notification can be held, folded into the in-app inbox, or dropped. This is one of the clearest public examples of **send-time optimization as a causal hazard problem**, not a "best hour of day" lookup.

32.4.4 Offline reinforcement learning for notifications

Later LinkedIn work (KDD 2022) [10] treats the notification decision as a Markov decision process. The state is member activity plus notification history, the actions are send/no-send (and threshold choices), and the reward is long-term sessions minus negative feedback. A policy is learned with **offline RL** (a double-DQN-style value-based method) on logged data, because online exploration on real members is expensive. The paper reports that the RL policy beat the hand-tuned multi-objective threshold system in online A/B tests.

Lesson: once the multi-objective threshold system is mature, the next step is to make the threshold state-dependent and forward-looking. RL does this by learning the value of "saving" a member's attention for a later, better notification.

32.5 Pinterest: notification volume control and optimization

Problem

Pinterest sends large volumes of email and push: boards and Pins you might like, activity on your Pins, digests. Its 2018 KDD paper [11] describes a central finding. Optimizing per-notification CTR, and sending whenever CTR is above a threshold, did not maximize long-term engagement. More notifications raised short-term activity but also raised unsubscribes and "notification blindness".

Design



- **Model the long-term effect of volume, not single sends.** Pinterest trained models that predict a user's activity over a time window as a function of the number of

notifications they receive. These were trained on data from randomized volume experiments in which users were randomly assigned different volume levels.
- **Budget then rank.** A per-user weekly budget is set first by marginal analysis: send more to users whose activity curve is still rising and fewer to users who are saturated or at risk of unsubscribing. The per-item ranker then fills the budget.
- **Unsubscribe as a hard constraint.** Unsubscribing removes the channel permanently, so it enters as a constraint or a heavy penalty, not a small negative weight.
- **Results** (approximate, from prior knowledge): the paper reports gains in weekly active users together with *fewer* notifications sent and fewer unsubscribes than the earlier threshold system. This is the main reason "send less, better" became industry consensus.

Pinterest has also written in its engineering blog about multi-channel notification infrastructure and email ranking (for example, digest composition and ranking of Pins inside emails). Those posts could not be retrieved in this session, so no figures from them appear here.

32.6 Duolingo: sleeping, recovering bandits for practice reminders

Problem

Duolingo sends a daily practice reminder ("Hi! It's Duo…") to keep learners' streaks going. The *content*, meaning which of many templates to use, strongly affects whether the learner comes back. Two properties break standard bandits:

1. **Recovering arms.** A template's effect wears off when it is repeated: novelty decays. An arm's reward depends on how recently that user last saw it, and it recovers after a rest period.
2. **Sleeping arms.** Not every template is eligible for every user on every day. Some apply only to users with a streak, to certain languages, or to certain contexts. The available arm set changes from round to round.

Algorithm

Yancey and Settles (KDD 2020) [12] propose a **recovering difference softmax** bandit:

- Each arm's value is estimated as the *difference* between the reward when the arm is chosen and the reward when it is eligible but not chosen. This is a built-in counterfactual baseline that removes the effect of eligibility-correlated user populations.
- A **recency penalty** lowers an arm's score when the user saw it recently. The penalty decays as the time since the last exposure grows, which models recovery.
- Arms are sampled with a **softmax** over the adjusted scores. This gives continual exploration and avoids sending the same "best" template every day, which would destroy its novelty.

- The reward is whether the learner completes a lesson within a short window, about two hours, after the notification.

Scale and results

The paper describes deployment to Duolingo's whole notification population, then hundreds of millions of reminders. It reports statistically significant gains in daily active users and new-user retention over the previous random or round-robin template policy (gains in the low single-digit percent range, approximate). The authors also released a large dataset of logged practice-reminder notifications for bandit research. Duolingo's later public talks and blog posts describe the same bandit infrastructure as the base for its notification experimentation; those posts could not be fetched in this session.

Lessons

- In repeated-message channels the *novelty* of an item is part of its value. Model per-user exposure recency explicitly.
- Under eligibility constraints, compare chosen against eligible-but-not-chosen to avoid selection bias.
- Softmax/Thompson-style stochastic policies are a feature here, not a compromise, because variety itself raises reward.

32.7 Meta: Facebook and Instagram notifications

Facebook and Instagram notifications are among the highest-volume recommendation surfaces in the world, but primary technical sources specific to notifications could not be fetched in this session. From Meta's public practice and published ranking material (prior knowledge, unverified here), the general shape is:

- Notifications are ranked by multi-task models predicting click/open, downstream engagement after the open, and negative signals (turning off a notification type, "hide"). A value model combines these predictions, as in feed ranking.
- Notification candidates (friend activity, group posts, Reels and creator updates, "memories", birthdays) come from many producers into a central ranking and filtering layer with per-type and global volume caps. This is architecturally the same split as LinkedIn's producers-plus-ATC.
- In-app notification tabs (the "jewel" or activity tab) get the same ranking with a looser volume constraint, because they do not interrupt the user.
- Meta publishes "system cards" or ranking explainers for some surfaces, which name the main prediction targets. Readers should check the current transparency-center pages for Facebook and Instagram notifications for the authoritative list.

Treat this section as a structural sketch. The chapter has no verified scale numbers for Meta notifications.

32.8 Fatigue modeling: patterns across systems

Across the designs above, fatigue appears in four complementary forms:

Mechanism	Example	Strength	Weakness
Hard frequency caps	per-day/per-week caps in ATC [7]; X `FatiguePredicate` [5]	simple, safe, auditable	ignores heterogeneity
Fatigue features in ranker	recent sends, time since last open, recent dismissals	personalized	still myopic per send
Negative labels / penalties	X `label.ntabDislike` [4]; LinkedIn disable penalty [8]	trains against annoyance directly	rare, noisy labels
Volume response curves / RL	Pinterest per-user budgets [11]; LinkedIn offline RL [10]	optimizes long-term causal effect	needs randomized volume data

Practical recipe:

1. Log *every* decision, including drops and their reasons, so that "not sent" exists in the data.
2. Keep a long-running **notification holdback**, a small random population with reduced or zero notifications. It is the only way to measure the true incremental value of the channel.
3. Run **volume perturbation** experiments (randomized higher or lower thresholds) to estimate response curves. This is the training data for budget or RL policies.
4. Treat channel-loss events (push disable, email unsubscribe, uninstall) as constraint terms with their own dual multipliers.
5. Add stateful predicates for explicit feedback ("show less", the X caret) that suppress similar candidates for a cooling period.

32.9 Send-time optimization (STO)

Send-time optimization chooses *when* within a window to deliver. Published approaches, from simple to sophisticated:

- **Historical best hour.** Per user, pick the hour of day with the highest historical open rate, smoothed with segment priors. Widely used in email platforms. Cheap, but confounded by past send times.
- **Activity-aligned delivery.** Send shortly before the user's predicted next active period. In Duolingo's setting the reminder is tied to the time the learner usually practises [12].
- **Hazard / survival models.** LinkedIn's state-transition model [9] predicts time to the next visit with and without a notification and sends when the *difference* is largest. This is the causally correct version: avoid notifying users who were about to come back anyway.
- **Sequential decision.** Twitter's model-the-future approach [6] and LinkedIn's offline RL [10] fold timing into a policy that can "wait" as an action.

Timing interacts with volume. Delaying a notification often lets the system batch several events into one richer notification (digest-ification), which lowers volume and raises per-notification value. LinkedIn's ATC arbitration and email digests make this trade-off explicitly [7].

32.10 Email recommendation specifics

Email differs from push in several ways that matter to the designer:

- **Composite items.** An email (a digest) contains several recommended items. The problem is slate ranking plus layout, and the "send" decision covers the whole slate.
- **Delayed and weak open signals.** Privacy features such as image-proxy prefetching inflate measured opens, so clicks and downstream sessions are more reliable labels.
- **Deliverability is part of the objective.** Spam complaints and low engagement damage sender reputation for *all* users, a cross-user externality that per-user optimizers miss. Pinterest's and LinkedIn's volume systems [8][11] effectively add a global constraint for this.
- **Unsubscribe is close to permanent.** This favors conservative budgets for low-activity users, where the marginal gain is small and the unsubscribe risk is high.

32.11 Comparison of published designs

System	Decision unit	Model(s)	Objective	Volume / fatigue control	Timing
X pushservice [1-5]	best candidate per refresh	MLP light ranker; ClemNet MTL heavy ranker (OONC 0.9, engagement 0.1)	open/NTab click + engagement; dislike label	fatigue, caret, quality, health predicates; epsilon-greedy exploration	refresh-driven
Twitter "should I send" [6]	send/no-send	long-horizon activity models	incremental future activity	decision-theoretic threshold	implicit
LinkedIn Concourse + ATC [7]	(content, recipient) pair	GLMix-style response models, streaming (Samza)	engagement with negative penalties	central ATC caps, dedupe, channel choice	near real time, can delay
LinkedIn MOO [8]	per send	response + negative-outcome models	sessions s.t. disable/unsub limits	Lagrangian thresholds	n/a
LinkedIn survival [9]	send/delay	hazard (time-to-visit) models	reduce time to visit causally	implicit via incrementality	core focus
LinkedIn offline RL [10]	per-step policy	double-DQN-style offline RL	long-term sessions minus negatives	learned, state-dependent	wait as action
Pinterest [11]	weekly per-user budget	volume response curves	long-term weekly activity	budget s.t. unsubscribes	scheduler
Duolingo [12]	template per reminder	recovering difference softmax bandit	lesson completion within about 2h	recency penalty (novelty)	daily, user-habit-aligned

Key takeaways for system designers

1. **Optimize incremental, long-term value, not CTR.** Every mature system (Pinterest, LinkedIn, Twitter) moved from "send if p(open) > t" to models of *causal* lift in activity over a horizon.

2. **Separate relevance from arbitration.** Producers score items. A central arbiter (LinkedIn ATC, X's take step and predicates) enforces per-user budgets, deduplication, quiet hours and channel choice. This keeps global volume coherent as teams add new notification types.
3. **Make "don't send" a first-class outcome.** Log drops, keep holdbacks, add exploration (X's epsilon-greedy filtering predicate) and run randomized volume experiments. Without them the training data reflects only the old policy.
4. **Model annoyance explicitly.** Use dislike, caret and "show less", disable and unsubscribe as labels or constraints. Channel loss is near-permanent, so price it high.
5. **Rank many, send one.** A light/heavy cascade followed by a sequential take step keeps expensive safety and quality checks cheap.
6. **Novelty decays.** For recurring messages, put exposure recency in the policy (Duolingo's recovering bandit) and keep stochastic exploration.
7. **Timing is a causal question.** The best time to notify is when the notification changes behavior most, not when the user is most likely to open anyway. Survival models and RL policies with a "wait" action capture this.
8. **Budget across channels.** Push, in-app, email and SMS share one attention budget. Coordinate them centrally and use digests to turn volume into value.

References

1. "the-algorithm" README (Recommended Notifications: pushservice, light and heavy rankers), X Corp (Twitter), 2023. https://github.com/twitter/the-algorithm
2. "pushservice" README (RefreshForPushHandler and SendHandler), X Corp (Twitter), 2023. https://github.com/twitter/the-algorithm/tree/main/pushservice
3. "Light ranking" model README for pushservice, X Corp (Twitter), 2023. https://github.com/twitter/the-algorithm/tree/main/pushservice/src/main/python/models/light_ranking
4. "Heavy ranking" model (ClemNet; `model.py`, `params.py`, `features.py`), X Corp (Twitter), 2023. https://github.com/twitter/the-algorithm/tree/main/pushservice/src/main/python/models/heavy_ranking ; https://github.com/twitter/the-algorithm/blob/main/pushservice/src/main/python/models/heavy_ranking/params.py
5. pushservice Scala packages (`predicate`, `model`), X Corp (Twitter), 2023. https://github.com/twitter/the-algorithm/tree/main/pushservice/src/main/scala/com/twitter/frigate/pushservice/predicate ; https://github.com/twitter/the-algorithm/tree/main/pushservice/src/main/scala/com/twitter/frigate/pushservice/model
6. C. O'Brien et al., "Should I send this notification? Optimizing push notifications decision making by modeling the future", Twitter, 2022. https://arxiv.org/abs/2202.08812 (not re-verified in this session)
7. Y. Gao et al., "Near Real-time Optimization of Activity-based Notifications", LinkedIn, KDD 2018. https://dl.acm.org/doi/10.1145/3219819.3219880 (DOI not re-verified in this session)
8. LinkedIn notification/email volume optimization work (multi-objective optimization of sessions vs. unsubscribes/disables), as summarized in [7] and [10]; LinkedIn Engineering Blog, https://www.linkedin.com/blog/engineering (index page; specific post URL not retrievable in this session)

9. Y. Yuan, J. Zhang, S. Chatterjee, S. Yu, R. Rosales, "A State Transition Model for Mobile Notifications via Survival Analysis", LinkedIn, WSDM 2019. https://dl.acm.org/doi/10.1145/3289600.3290981 (DOI not re-verified in this session)
10. P. Prabhakar, Y. Yuan, G. Yang, W. Sun, A. Muralidharan, "Multi-objective Optimization of Notifications Using Offline Reinforcement Learning", LinkedIn, KDD 2022. https://arxiv.org/abs/2207.03029 (not re-verified in this session)
11. B. Zhao, K. Narita, B. Orten, J. Egan, "Notification Volume Control and Optimization System at Pinterest", Pinterest, KDD 2018. https://dl.acm.org/doi/10.1145/3219819.3219906 (DOI not re-verified in this session)
12. K. P. Yancey, B. Settles, "A Sleeping, Recovering Bandit Algorithm for Optimizing Recurring Notifications", Duolingo, KDD 2020. https://research.duolingo.com/papers/yancey.kdd20.pdf (host unreachable from this session's sandbox)

33. Content Understanding and Integrity in Recommender Systems: Multimodal Item Embeddings, Integrity Filters, Borderline Demotion, Spam and the Policy Layer

Most of this Atlas is about predicting what a person will engage with. This chapter covers the two layers that decide what an item *is* and whether it *should* be recommended at all.

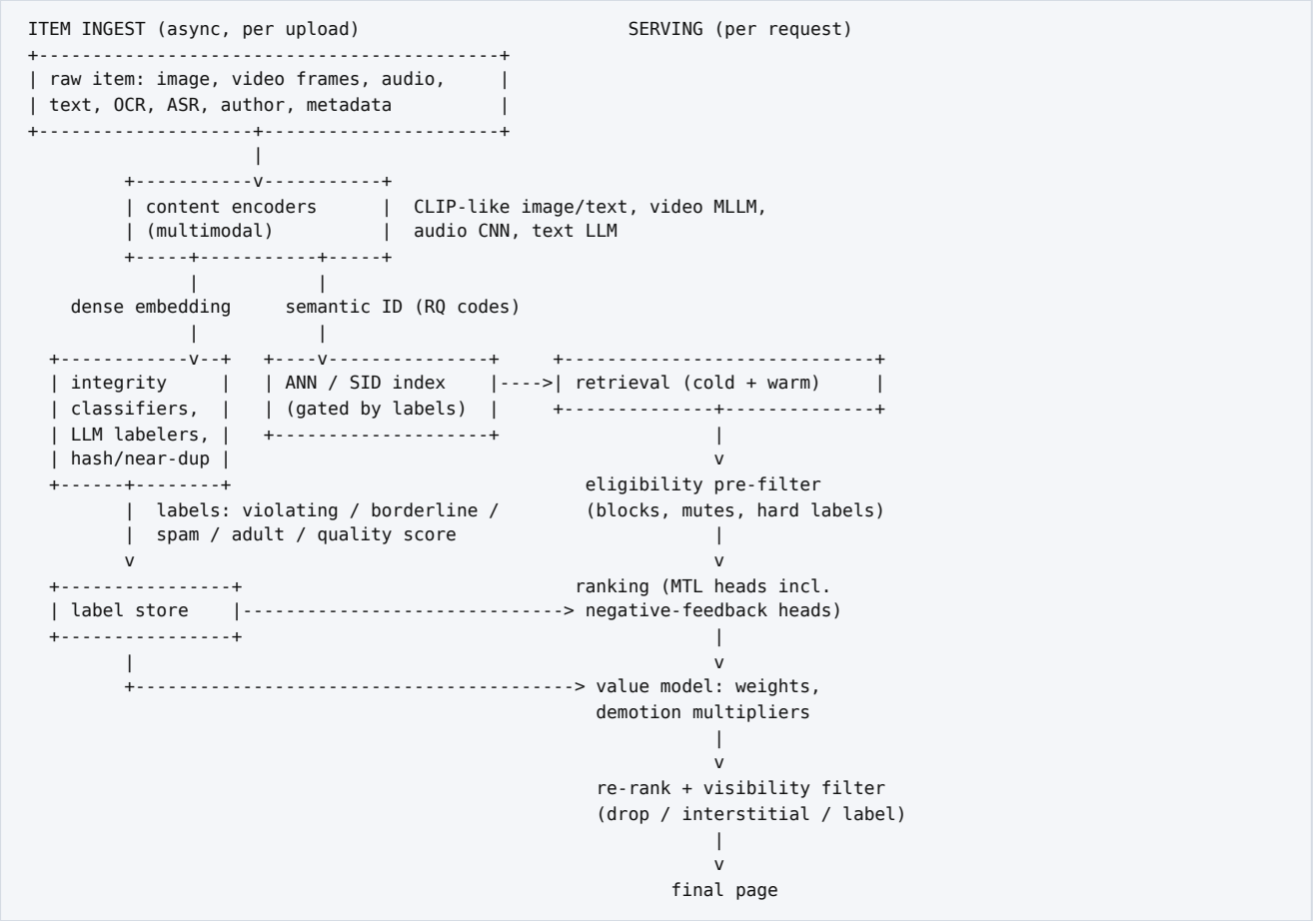
1. **Content understanding.** Models turn raw pixels, text, audio and video into dense vectors or discrete codes. Those representations feed retrieval (cold start), ranking features, deduplication, and the integrity classifiers themselves.
2. **Integrity and policy.** Classifiers, rule engines and value-model terms remove violating content, demote borderline content, suppress spam and manipulation, and apply legal and product policy. These layers are kept explicit and separate from the engagement model.

The two layers share infrastructure. The same image or post embedding that places a new Pin near similar Pins also lets a safety classifier spot a near-duplicate of known violating media. The same semantic ID that lets a generative recommender decode a cold item is checked against visibility filtering before the item enters an index [13].

A note on sources. The live web was not reachable for this chapter's research pass. Facts are taken from primary sources that other chapters of this Atlas fetched and verified (Meta, Instagram, X/Twitter open-source code, YouTube, Pinterest, Kuaishou, Spotify, Netflix, Etsy, Google). A small number of well-known primary sources are cited from prior knowledge and are marked "(not re-fetched)" in the references. Numbers from those sources are given only where they are widely reported, and are marked approximate.

33.1 Where content understanding and integrity sit in the stack

Every public design in this Atlas keeps an explicit, non-learned layer for eligibility and policy alongside the learned funnel (Chapter 15). Google's "Rules of Machine Learning" recommends keeping heuristics and filtering outside the model when they express policy rather than prediction [40]. The composite picture below is drawn from Meta Feed [20], Instagram Explore [26], X's 2023 and 2026 open-source releases [13][16][17] and YouTube's public descriptions [30][33].



Three properties recur across companies:

- **Label once, enforce many times.** Content understanding runs asynchronously at upload or first impression. Labels go to a store that retrieval, ranking and serving-time filters all read. X's 2026 release shows this explicitly: content-understanding services write labels to storage, and a separate visibility-filtering service reads them per (post, viewer) [13].
- **Enforce at several points.** Cheap eligibility checks run before the expensive models to save compute. Meta Feed runs "integrity processes" before the first ranking pass [20]. Stricter or costlier checks run again at the end, because blending and backfill can reintroduce items [13][26].
- **Prefer demotion to deletion for grey areas.** Content that violates policy is removed. Content near the line is kept but not amplified [29][30][31]. That distinction needs a soft lever (multipliers or value-model terms) as well as a hard filter.

33.2 Multimodal item embeddings

Content embeddings serve three purposes in recommendation:

1. **Cold start.** A new item can be scored before it has any interactions.
2. **Similarity and deduplication.** Near-duplicate detection, "more like this", and diversity kernels all need a content distance.
3. **Integrity reuse.** Safety classifiers can sit on top of general-purpose embeddings rather than each training its own backbone.

33.2.1 CLIP-style contrastive pretraining

Design. OpenAI's CLIP trains an image encoder and a text encoder jointly with a contrastive objective. Matching (image, caption) pairs in a batch are pulled together and all other pairs are pushed apart. The model was trained on roughly 400 million image-text pairs collected from the web, and it showed strong zero-shot transfer to many classification tasks (approximate; not re-fetched) [1].

Why recommenders care. A CLIP-like model gives one space for images and text. A text query, a product title and a photo can be compared directly. Classifiers can be bootstrapped from text prompts before labeled data exists. In recommendation stacks it most often appears as:

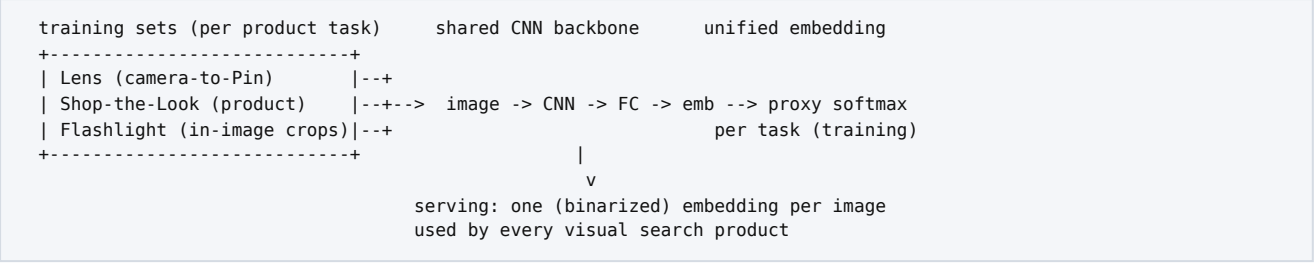
- a frozen feature extractor whose embedding is fed to rankers;
- a clustering basis, as in X's Home Mixer parameters, which list "CLIP image clusters and multimodal embeddings as features" [15];
- an input to safety services. X's 2026 release lists `clip/` and `pnsfwmedia/` (a predicted-NSFW media model) among the content-understanding services that write safety labels [13].

Lesson. A general vision-language embedding is a platform asset. Engagement ranking, near-duplicate detection and safety classification can all reuse it, which justifies centralizing its training and serving.

33.2.2 Pinterest: unified visual embeddings

Problem. Pinterest ran several visual products (Lens camera search, Flashlight in-image search, Shop the Look), each with its own specialized visual embedding. Maintaining many embeddings multiplies training, storage and backfill costs, and each improvement had to be repeated per product.

Design. Zhai et al. (KDD 2019) describe a single **unified visual embedding** trained with **multi-task metric learning**: one shared convolutional backbone, several task-specific training datasets, and proxy-based classification losses, so that one embedding serves all the visual search applications. The paper also reports binarizing the embedding to cut storage and serving cost for billions of images (details approximate; not re-fetched) [2].



Lessons. (1) Multi-task training with one embedding beat maintaining specialized embeddings, operationally and in quality, according to the paper [2]. (2) Compression (binarization) is part of the design at billion-image scale, not an afterthought.

33.2.3 Pinterest: from PinSage to ItemSage and OmniSearchSage

Pinterest's representation stack layers content on top of graph signals (Chapter 22).

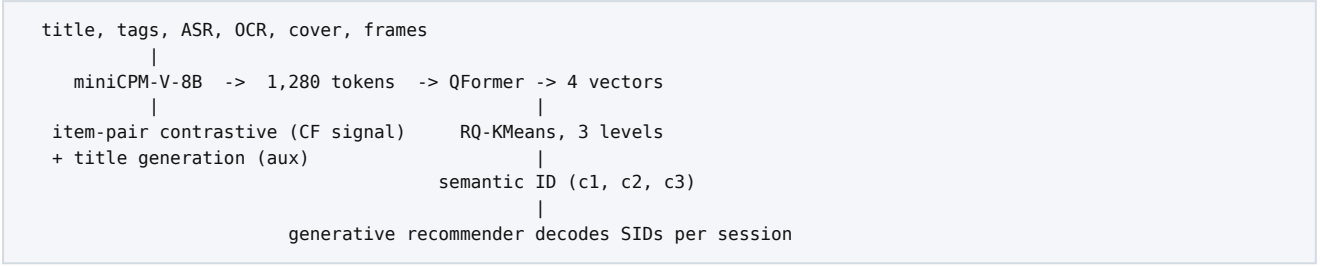
- **PinSage (KDD 2018).** A GraphSAGE-style inductive GCN that aggregates **visual and text features** of each Pin with those of its random-walk neighbours. It was trained on 7.5 billion examples over a graph of 3 billion nodes and 18 billion edges [3]. Because the input is content features, new Pins can be embedded inductively. PinSage embeddings became Pinterest's shared coordinate system: PinnerSage clusters users in PinSage space [6], and PinnerFormer uses 256-dimensional PinSage embeddings as frozen per-action features [7].
- **ItemSage (KDD 2022).** Baltescu et al. describe product (shopping) embeddings that combine image and text features with a transformer, are trained on several engagement tasks at once, and are made compatible with PinSage space so that product and Pin embeddings can be compared across surfaces (details approximate; not re-fetched) [5].
- **OmniSearchSage (2024).** Multi-task, multi-entity embeddings for Pinterest Search: queries, Pins and products in one space trained on several tasks [4].

Lesson. The trend at Pinterest is from one embedding per product, to one embedding per entity type, to one shared multi-entity space. Content features keep that space usable for cold items, and graph signals keep it aligned with engagement.

33.2.4 Kuaishou OneRec: multimodal LLM item tokenizer

Kuaishou's OneRec (2025) replaces the retrieval-and-ranking cascade with one generative model, and its item representation is entirely content-derived [8][9]:

- Inputs per video: **title, tags, ASR, OCR, cover image and sampled frames**.
- A multimodal LLM (**miniCPM-V-8B**) encodes them into **1,280 tokens**. A **QFormer** compresses these into **4 query vectors**.
- An **item-pair contrastive loss** injects collaborative signal, with title generation as an auxiliary task.
- **RQ-KMeans** over 3 residual levels turns each video into a **3-token semantic ID** (about 8K codes per level, approximate).



Lesson. Once the generator only emits item codes, the **tokenizer is the content-understanding system**. Its quality, collision rate and refresh policy determine cold-start behaviour. It is also the natural place to attach integrity labels: an item without a vetted code cannot be generated.

33.2.5 YouTube and Google: semantic IDs from content

- **TIGER (NeurIPS 2023)** quantizes content embeddings into hierarchical semantic IDs with an RQ-VAE, and a seq2seq transformer decodes the next item's IDs. Similar items share prefixes, which helps cold items [12].
- **Semantic IDs in ranking (RecSys 2024)** showed that replacing random item-ID hashing with content-derived semantic IDs improves generalization at a fixed embedding-table size [11].
- **PLUM (2025)** builds "Semantic IDs v2" from **multimodal content embeddings plus a co-engagement signal**, continues pre-training a Gemini-family LLM on watch sequences written as SIDs, and serves generative retrieval as a source feeding YouTube's ranking stack (approximate) [10].

33.2.6 X Phoenix: semantic IDs over multimodal post embeddings

X's 2026 Phoenix retrieval model represents posts by **semantic IDs: residual-quantized codes, 6 levels x 256 codes, derived from each post's multimodal embedding**, plus hashed author IDs. Posts on the same topic share SID prefixes, which helps the model generalize to unseen posts [14]. The production index holds 10.24M posts (28.67M in the combined configuration) [14]. Integrity is coupled to indexing: the `phoenix-rankall-strato` service decides which index a post belongs in by "consulting visibility filtering first" [13].

33.2.7 Audio and text: Spotify and Xiaohongshu

- **Spotify audio CNN.** A convolutional network over mel-spectrograms of 30-second clips is trained to **regress the collaborative-filtering latent vector** of tracks that have enough plays. A brand-new track is then embedded where the CF model would have placed it, in the same ANN index [36][37]. This "regress into the CF space" trick generalizes to any modality.
- **NoteLLM (Xiaohongshu, WWW 2024)** compresses each note into a special token from an LLM's last layer. That embedding is trained contrastively on co-occurrence pairs for item-to-item retrieval, with hashtag and category generation as auxiliary tasks [38].

33.2.8 Comparison of content-embedding designs

System	Year	Modalities	Training signal	Output	Main use in recs	Ref
CLIP	2021	image + text	image-caption contrastive (~400M pairs, approx.)	dense vectors	features, clusters, zero-shot labels	[1]
Pinterest unified visual	2019	image	multi-task proxy metric learning	one (binarized) vector	all visual search products	[2]
PinSage	2018	image + text + graph	co-engagement, hard negatives	256-d (as used downstream)	retrieval, user models	[3] [7]
ItemSage	2022	image + text (products)	multi-task engagement	vector compatible with PinSage	shopping surfaces	[5]
OmniSearch Sage	2024	query, Pin, product	multi-task, multi-entity	shared vector space	search retrieval	[4]
Spotify audio CNN	2014	audio	regress CF factors	vector in CF space	cold-start tracks	[36]

System	Year	Modalities	Training signal	Output	Main use in recs	Ref
NoteLLM	2024	text (notes)	co-occurrence contrastive + generation	LLM token embedding	I2I retrieval	[38]
OneRec tokenizer	2025	video, image, text, ASR, OCR	MLLM + item-pair contrastive	3-level semantic ID	generative rec	[8] [9]
YouTube PLUM SIDs v2	2025	multimodal + co-engagement	RQ-style quantization	multi-level SID	LLM generative retrieval	[10]
X Phoenix SIDs	2026	multimodal post embedding	RQ, 6 x 256 codes	SID + hashed author	retrieval and ranking	[14]

33.3 Integrity filters inside the funnel

33.3.1 Meta Feed: integrity first

Meta's 2021 Feed ranking post describes the funnel as passes over an inventory of more than 1,000 eligible posts per viewer, run in parallel on serving machines called predictors [20]:

```
inventory (> 1,000 posts)
   |
pass 0: integrity processes + eligibility   <- violating / ineligible content removed
   |
pass 1: lightweight model -> ~500 posts
   |
pass 2: multitask NNs -> P(like), P(comment), P(hide), P("worth your time"), ...
        -> one value per post
   |
pass 3: contextual re-rank (diversity, avoid runs of similar posts)
```

Running integrity before the heavy pass means the expensive models are only spent on content that can be shown [20]. Meta also publishes a "system card" for Feed that lists the main prediction signals and stages [23].

33.3.2 Instagram Explore: negative heads and final integrity filters

Instagram Explore shows three integrity mechanisms at different stages.

1. **Final re-ranking with integrity filters.** The 2019 funnel goes from about 500 candidates through three ranking passes to about 25 shown "after integrity and diversity rules" [24]. In the 2023 four-stage design, final re-ranking applies "integrity filters plus business rules" such as author-repetition penalties [26].

2. **Negative-feedback heads in the value model.** The 2020 design predicts "see less" and "not interested" alongside likes and saves, and combines them **log-linearly** [25]:

   ```
   Value(post) = P(like) ^ w_like * (1 - P(not interested)) ^ w_see_less
   ```

   In a multiplicative form, a high predicted negative probability can veto an item however likely a like is. The 2023 design uses a linear value model with a negative term `- W_see_less * P(see less)` [26].

3. **Guardrail-constrained tuning.** Value-model weights are tuned with Bayesian optimization in which engineers specify the parameters, the goal metric and regression thresholds on guardrail metrics [26].

33.3.3 X / Twitter: visibility filtering as a separate rule engine

X has published the most detailed integrity layer of any recommender. It is useful as a reference design.

2023: `visibilitylib`. The README calls it "a centralized rule engine that instructs clients how to alter the display of certain Twitter content on read time" [16]. Its model is:

- **SafetyLevel**: the product context (Timeline, Profile, and so on). Each SafetyLevel has one policy.
- **Features**: safety labels, user flags, user-user relationships, user-content relationships.
- **Conditions -> Rules -> Policy.** Conditions combine into rules. Rules are evaluated in priority order.
- **Actions**: **Drop** (hard filter), **Labels and Interstitials** (soft filter), and **coarse-grained downranking**.

Part of the code was withheld in 2023 as "being reviewed and rebuilt" [16].

2026: the full labeling path. The `xai-org/x-algorithm` release added the content-understanding and labeling services [13]:

```
CONTENT UNDERSTANDING (async)          LABELING RULES              SERVE
grox/  LLM + classifiers: spam,        scarecrow/ event rules,     visibility-filtering/
       adult, violent media,           botmaker rule engine        per (post, viewer):
       text/image embeddings           abuse-enforcement-svc/        ALLOW | NOTICE |
media-model-proxy/, clip/,              (+ abuse-ledger appeals)     INTERSTITIAL | DROP
  pnsfwmedia/                          safety-label-user-agg/            |
agatha/ (blocks+reports vs favs)                |                     VFFilter,
bdsm/   (behaviour sequences)                   |                     AncillaryVFFilter
user-cred-v2/ (PageRank credibility)            |
            \____________ labels -> storage ___/
```

Design points from the release [13]:

- **Ranking and visibility are separate systems** with "different services, different inputs, different rules". Ranking sets order; VF decides whether a post can be shown at all.
- **VF runs after selection.** `VFCandidateHydrator`, `VFFilter` and `AncillaryVFFilter` run after top-K is fixed. This avoids per-candidate VF calls on the whole pool. Cheap pre-scoring filters already handle blocks, mutes and muted keywords. `AncillaryVFFilter` also drops a post when its parent, quoted post or reposted post was dropped.
- **First DROP wins.** Evaluation ends at the first rule that answers DROP.
- **Stricter rules for out-of-network recommendations.** An additional set of out-of-network-only rules can only drop. High-recall spam classifiers suppress a post *as a recommendation* while still showing it to followers.
- **Withheld rules.** Some rules and the Grox LLM prompts are deliberately unpublished "to reduce the risk of gaming". An "Under the Hood" tool gives each account a report of the visibility-limiting labels on its account and posts, and since September 2026 also shows legal withholding by country.

- **Jurisdiction filters** appear in code, for example `Brazil2026ElectionFilter`.
- **Index gating.** As noted in 33.2.6, retrieval indexing consults VF before a post enters an index.

Lessons. (1) Model visibility as `(item, viewer, surface) -> action`, not as a property of the item alone. (2) Separate recall-oriented recommendation rules from precision-oriented removal rules: a false positive in "do not recommend" costs much less than a false positive in "remove". (3) Run expensive checks on the final top-K, and cheap ones early.

33.3.4 Health signals inside ranking and notifications

Integrity does not live only in filters. X's 2023 heavy ranker consumed realtime **pToxicity and pBlock** health-model scores as features [17]. The For You value model put large negative weights on reports, blocks and mutes, calibrated to base rates. The 2026 parameters note that a report is more than 1,000x rarer than a like (Chapter 21) [13].

Notifications use the same idea with stricter gating, because a push interrupts the user. X's `pushservice` predicate package includes `BqmlHealthModelPredicates`, `HealthPredicates`, `OutOfNetworkCandidatesQualityPredicates`, `JointDauAndQualityModelPredicate` and `PNegMultimodalPredicates` (a predicted-negative model), plus gates for withheld content and negative-feedback-based out-of-network filtering [19].

33.4 Borderline content demotion

33.4.1 The engagement curve argument

In his November 2018 note "A Blueprint for Content Governance and Enforcement", Mark Zuckerberg argued that, left alone, people engage more with content the closer it gets to the policy line, even when they later say they dislike it. Meta's stated response was to **reduce the distribution of borderline content**, so that engagement falls as content approaches the line instead of rising (approximate; not re-fetched) [29]. Meta's Transparency Center now documents the classes of content whose distribution it reduces as "Content Distribution Guidelines"; the exact page was not verified for this chapter.

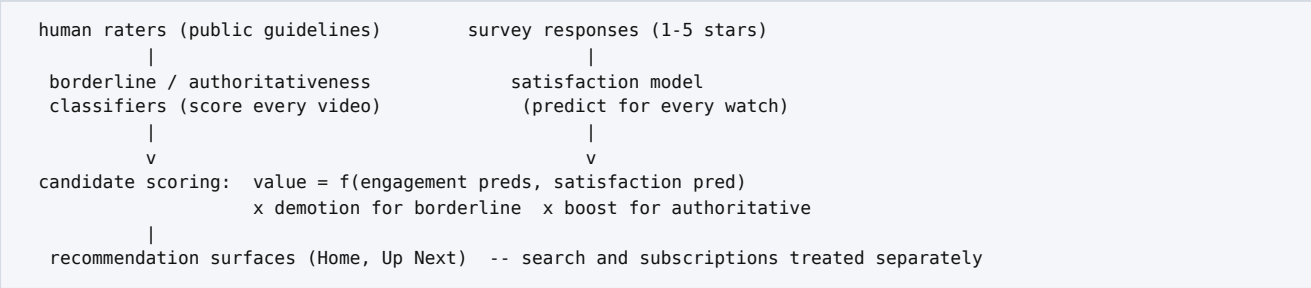
This argument explains why a pure engagement optimizer drifts toward borderline content, and why a separate counter-term is needed. Reporting on Meta's 2018 **Meaningful Social Interactions (MSI)** value model gives an empirical example. MSI gave point values to likes, comments, re-shares, reactions and RSVPs, with weights derived from surveys (one cited survey covered more than 69,000 people in five countries) and data analysis. Heavy weights on comments, re-shares and reaction emojis were reported to amplify outrage-provoking content [22]. Meta later added "Is this post worth your time?" surveys and gave more weight to survey-predicted value [21].

33.4.2 YouTube: raters, borderline classifiers and authoritative boosts

YouTube's January 2019 announcement said it would begin reducing recommendations of borderline content: content that comes close to violating Community Guidelines, and content that could misinform users in harmful ways. It estimated such content at well under

1% of what is watched on YouTube (approximate; not re-fetched) [31]. The signals come from **human evaluators** trained on public guidelines, whose ratings train classifiers that score videos at scale [31][32].

In December 2019 YouTube reported that, after these changes, watch time of borderline content coming from recommendations to non-subscribed channels fell by about 70% on average in the US (approximate; not re-fetched) [32]. Cristos Goodrow's 2021 post repeats the roughly 70% figure and describes the companion lever: **raising authoritative sources** for news and information queries and recommendations [30]. The same post places this in the evolution of YouTube's objective: clicks (which encouraged clickbait), then watch time, then "valued watch time" predicted from user satisfaction surveys [30]. The "How YouTube Works" site presents the same system from the policy side [33].



Lessons.

- **Borderline is a ranking problem, not a removal problem.** The item stays available, especially to subscribers who seek it out, but is not amplified to people who did not ask for it. This matches X's out-of-network-only rules [13].
- **Rater programs are infrastructure.** Guidelines, rater training, agreement measurement and classifier retraining form a pipeline that needs its own owners.
- **Pair demotion with promotion.** Demoting harmful content in a topic area leaves a vacuum that the ranker fills with whatever is next. Boosting authoritative sources fills it deliberately [30].

33.4.3 Remove, reduce, inform

Across Meta, YouTube and X, the public designs converge on three graded responses:

Response	Typical trigger	Mechanism	Example implementations
Remove	clear policy violation	hard filter at ingest or serving; takedown	Meta pass-0 integrity [20]; X DROP action [13][16]
Reduce	borderline, low quality, spam-like, unoriginal	demotion multiplier, value-model penalty, out-of-network ineligibility	YouTube borderline demotion [30][32]; X OON-only rules [13]; Meta borderline reduction [29]
Inform / interstitial	sensitive but allowed; disputed	label, warning screen, context panel	X Labels and Interstitials, NOTICE / INTERSTITIAL [13][16]
Raise	authoritative or high-quality	boost multiplier	YouTube authoritative sources [30]

33.5 Harmful-content classifiers that serve recommendation

33.5.1 Meta Few-Shot Learner

Policies change faster than labeled datasets can be collected. Meta's December 2021 post on its **Few-Shot Learner (FSL)** describes a large multilingual model pre-trained on very large amounts of general and integrity-related text, then adapted with zero or few labeled examples. Policies are expressed as natural-language descriptions, and classification is framed as entailment between the post and the policy text. Meta reported that it covered more than 100 languages, worked on text and images, and cut the time to deploy a new classifier from months to weeks (approximate; not re-fetched) [28].

Why it matters for recsys. Borderline and emerging-harm categories are exactly the ones where labels are scarce and policy definitions shift. A few-shot classifier lets the "reduce" lever track new policy quickly, without waiting for a supervised dataset.

33.5.2 Shared embeddings as integrity features

Twitter's TwHIN paper reports that the same heterogeneous-graph embeddings are reused for candidate generation, as heavy-ranker features, in ads, and for **harmful-content detection** [18]. Behavioural embeddings catch coordinated accounts and spam rings that content-only models miss, because those accounts look alike in who they interact with rather than in what they post.

33.5.3 LLM labelers

X's 2026 `grox/` service combines an LLM with classifiers for spam, adult content and violent media, using text and image embeddings, and writes labels consumed by visibility filtering [13]. The prompts are withheld to limit gaming [13]. The pattern of an LLM labeling offline, with classifiers or distilled models serving at scale, mirrors how LLMs enter recommendation stacks generally (Chapter 29): put the LLM on the asynchronous side of the cache line.

33.6 Spam, fraud and manipulation in recommendations

Recommenders create an incentive to manipulate whatever signal they reward. Public designs show defenses at several layers.

33.6.1 Reward design against clickbait

- **YouTube** moved from clicks to watch time in 2012 because clicks encouraged clickbait, and later to survey-predicted "valued watch time" [30]. The 2016 deep ranking model predicts expected watch time with weighted logistic regression for the same reason [34].
- **Netflix** artwork personalization rewards a **quality play** (a play followed by enough viewing), not a raw click, so a misleading thumbnail that draws abandoned plays is

penalized [35].
- **Twitter/X** uses dwell-qualified "good click" labels and large negative weights on report, block and mute (Chapters 11 and 21) [17].

33.6.2 Credibility and behaviour signals

X's labeling path includes services aimed at manipulation rather than content [13]:

- `user-cred-v2/`: PageRank-based user credibility, a descendant of the earlier Tweepcred reputation score.
- `agatha/`: compares blocks and reports with favorites, a ratio that separates content people dislike from content that is merely unpopular.
- `bdsm/`: models behaviour sequences, which catches automated or coordinated activity.
- `safety-label-user-agg/`: aggregates post-level labels to the author level, so repeat offenders lose distribution across all their posts.

33.6.3 Quality-aware retrieval

Etsy blends a product-quality signal into ANN retrieval, so the nearest-neighbour stage does not surface low-quality listings that the ranker would only demote later [39]. X gates its retrieval index on visibility filtering for the same reason [13]. Filtering inside retrieval protects recall: if a selective policy filter runs only after ANN, most of the retrieved budget can be wasted on ineligible items (Chapter 2).

33.6.4 Platform eligibility rules

TikTok's 2020 explainer said that some content, such as spam, duplicated content and content that may be shocking to a general audience, may not be eligible for the For You feed, even when it remains on the platform (approximate; not re-fetched) [41]. This is the same "allowed but not recommended" tier as YouTube's borderline policy and X's out-of-network rules.

33.6.5 Exploration under adversaries

Integrity filters also bias the training data. X's notification stack includes `BigFilteringEpsilonGreedyExplorationPredicate`, which lets a small share of candidates through filters that would have blocked them, so the logs contain outcomes for filtered items [19]. Without such exploration, a filter's mistakes cannot be measured from the data it shapes. X's 2026 For You pipeline uses deterministic per-(post, viewer) holdout filters to measure inventory effects [13].

33.7 Responsible recommendation and the policy layer

33.7.1 Separate prediction from policy

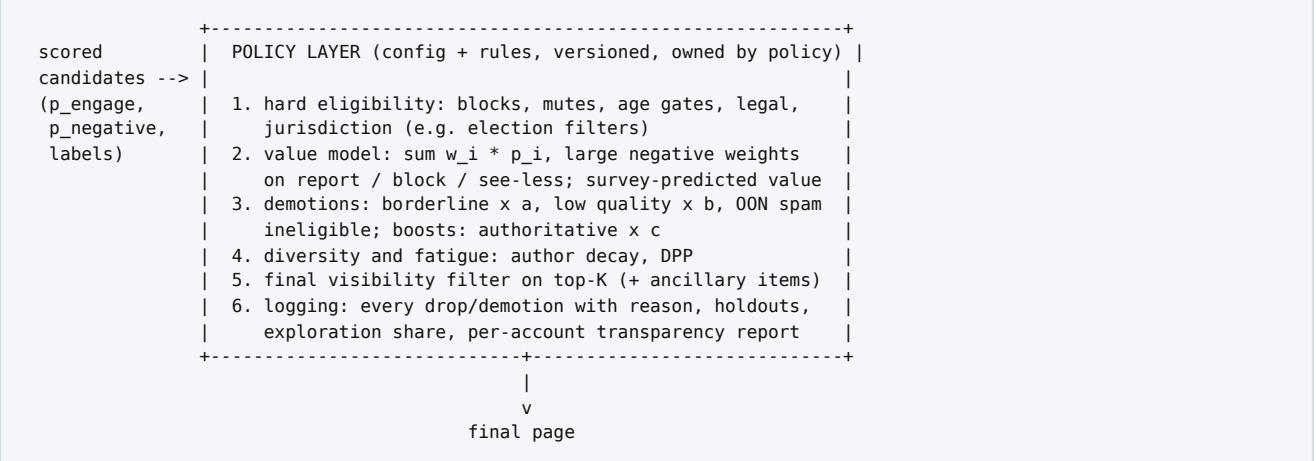
The most consistent pattern in this Atlas is that engagement models predict calibrated probabilities of concrete actions, and a small, versioned **value function** turns them into a

score (Chapter 4). Meta Feed [20], Instagram Explore [25][26] and X [13][17] all keep this split. For integrity it has three benefits:

1. **Auditable trade-offs.** A weight on `P(report)` or a borderline multiplier is a reviewable line of configuration.
2. **Fast policy changes.** New demotions ship as configuration, with no retraining.
3. **Clean attribution.** When harmful content rises, one can tell whether a predictor drifted or a policy weight changed.

The cost is calibration. A value model that adds `w_i * p_i` is only meaningful if each `p_i` is calibrated; a mis-calibrated `P(report)` silently changes policy (Chapter 4).

33.7.2 A reference policy layer



Every stage maps to a public example: jurisdiction filters and post-selection VF [13], value-model negative weights [13][26], borderline demotion and authoritative boosts [30][32], log-linear vetoes [25], filter exploration and holdouts [13][19], and per-account label reports [13].

33.7.3 Transparency as a design requirement

Transparency artifacts are becoming part of the architecture rather than PR:

- **Meta** publishes system cards for Facebook Feed and Instagram Explore that list stages and main prediction signals [23][27].
- **YouTube** maintains "How YouTube Works" pages for recommendations [33].
- **X** publishes production ranking code and parameters, and gives each account an "Under the Hood" report of labels limiting its visibility, including legal withholding by country [13].
- **Bluesky** moves ranking out of the platform into third-party feed generators that return only post URIs, which the platform hydrates. Moderation and spam filtering then sit in shared hydration and labeling layers rather than inside each feed [42] (Chapter 28).

The tension is gaming. X publishes code but withholds some rules and LLM prompts "to reduce the risk of gaming", and compensates with per-account reports [13]. Designers

should decide explicitly which parts of the integrity layer are public, which are disclosed per account, and which stay private.

33.7.4 Objectives beyond engagement

Responsible-recommendation work in production is mostly about objectives. YouTube's valued watch time [30], Meta's survey-weighted Feed value [21] and Netflix's quality-play reward [35] all replace a raw engagement label with a measure of satisfaction or value. The MSI history [22] is the cautionary counterexample: weights chosen from surveys still amplified divisive content when they rewarded reactive engagement. Objective changes need long-term holdouts and integrity metrics (prevalence of borderline content in recommendations), not only engagement metrics.

33.8 Side-by-side: integrity mechanisms by company

Company / system	Where integrity runs	Hard actions	Soft actions	Signals	Transparency	Ref
Meta Facebook Feed	pass 0 before ranking; value model	remove ineligible	borderline reduction; survey-weighted value	integrity classifiers, surveys, few-shot classifiers	Feed system card	[20][21] [23][28] [29]
Instagram Explore	value model; final re-rank	integrity filters	log-linear veto on P(not interested); see-less penalty	negative-feedback heads	Explore system card	[24][25] [26][27]
YouTube	candidate scoring	removal of violations	borderline demotion; authoritative boost	rater-trained classifiers, survey models	How YouTube Works	[30][31] [32][33]
X 2023	features; post-selection VF	Drop	labels, interstitials, coarse downranking; pToxicity/pBlock	safety labels, health models	open code (partial VF)	[16][17]
X 2026	async labeling; index gating; VF on top-K	DROP (first wins), jurisdiction filters	NOTICE, INTERSTITIAL; OON-only spam rules	Grox LLM, CLIP, pNSFW, cred, behaviour	code + Under the Hood	[13][14]
X notifications	predicates in take step	withheld-content gates	health / quality / pNeg predicates; OON spread control	BQML health and quality models	open code	[19]
Etsy search	inside ANN retrieval	n/a	quality-aware retrieval	product quality score	n/a	[39]
TikTok For You	eligibility	not eligible for FYF	n/a (details not public)	spam, duplicate, shocking content	newsroom explainer	[41]

Key takeaways for system designers

1. **Build one content-understanding platform and reuse it.** A shared multimodal embedding (CLIP-like, Pinterest unified visual, PinSage) serves cold-start retrieval, ranking features, deduplication and safety classifiers. Pinterest's move from per-product embeddings to one unified embedding shows the operational payoff [2][3].
2. **Regress content into the collaborative space.** Spotify's audio CNN and OneRec's item-pair contrastive loss both align content representations with engagement, so cold items land where warm ones would [8][36].
3. **In generative recommenders, the tokenizer is the content and integrity boundary.** Semantic IDs from multimodal embeddings (YouTube, Kuaishou, X) decide what can be generated. Gate index and codebook membership on integrity labels, as X does by consulting VF before indexing [10][13][14].
4. **Label asynchronously, enforce at several points.** Write labels once to a store. Apply cheap filters before ranking, demotions in the value model, and a final visibility filter on top-K, including dependent items such as quoted or parent posts [13][20].
5. **Model visibility as (item, viewer, surface) -> action.** X's SafetyLevel, rule priority and action set (drop, label, interstitial, downrank) is a reusable schema [16].
6. **Use graded responses.** Remove violations, reduce borderline and spammy content, inform with labels, and raise authoritative sources. Apply stricter rules to recommendations of unfollowed content than to content users chose to follow [13][30][32].
7. **Expect engagement optimizers to drift toward the line.** Borderline content attracts engagement. Counter it with explicit demotion and with satisfaction-based objectives (surveys, quality plays), and monitor prevalence, not only engagement [22][29][30][35].
8. **Give negative signals real weight.** Negative-feedback heads, large report and block weights calibrated to base rates, and multiplicative vetoes are the cheapest safety mechanisms in the stack [13][25].
9. **Plan for policy churn.** Few-shot and LLM-based classifiers let integrity levers follow new policy in weeks rather than months [13][28].
10. **Defend against manipulation with behaviour, not just content.** Credibility scores, block-versus-favorite ratios, behaviour-sequence models and author-level label aggregation catch coordinated abuse that content classifiers miss [13][18].
11. **Measure the filters.** Log every drop and demotion with its reason, keep holdouts, and explore a small share of filtered candidates so filter errors are observable [13][19].
12. **Decide transparency deliberately.** System cards, policy explainer sites, open code and per-account label reports each disclose a different layer. Withhold only what enables gaming, and compensate with per-account disclosure [13][23][33].

References

Sources marked "(not re-fetched)" are well-known primary sources cited from prior knowledge because the live web was unreachable during this chapter's research pass. Their figures are marked approximate in the text. All other sources were fetched and verified by other chapters of this Atlas.

1. Radford, A., Kim, J. W., Hallacy, C., et al. "Learning Transferable Visual Models From Natural Language Supervision" (CLIP). OpenAI, ICML 2021. https://arxiv.org/abs/2103.00020 (not re-fetched)
2. Zhai, A., Wu, H.-Y., Tzeng, E., Park, D. H., Rosenberg, C. "Learning a Unified Embedding for Visual Search at Pinterest." Pinterest, KDD 2019. https://arxiv.org/abs/1908.01707 (not re-fetched)
3. Ying, R., He, R., Chen, K., Eksombatchai, P., Hamilton, W. L., Leskovec, J. "Graph Convolutional Neural Networks for Web-Scale Recommender Systems" (PinSage). Pinterest / Stanford, KDD 2018. https://arxiv.org/abs/1806.01973
4. Pinterest. "OmniSearchSage: Multi-Task Multi-Entity Embeddings for Pinterest Search." 2024. https://arxiv.org/abs/2404.16260
5. Baltescu, P., Chen, H., Pancha, N., Zhai, A., Leskovec, J., Rosenberg, C. "ItemSage: Learning Product Embeddings for Shopping Recommendations at Pinterest." Pinterest, KDD 2022. https://arxiv.org/abs/2205.11728 (not re-fetched)
6. Pal, A., Eksombatchai, C., Zhou, Y., Zhao, B., Rosenberg, C., Leskovec, J. "PinnerSage: Multi-Modal User Embedding Framework for Recommendations at Pinterest." Pinterest, KDD 2020. https://arxiv.org/abs/2007.03634
7. Pancha, N., Zhai, A., Leskovec, J., Rosenberg, C. "PinnerFormer: Sequence Modeling for User Representation at Pinterest." Pinterest, KDD 2022. https://arxiv.org/abs/2205.04507
8. Kuaishou OneRec Team. "OneRec Technical Report." Kuaishou, 2025. https://arxiv.org/abs/2506.13695
9. Xing, H. "End-to-End Generative Recommendation" (OneRec / OneRec-V2 technical notes, fun-rec-mdbook; secondary). 2026. https://github.com/Haozhe-Xing/fun-rec-mdbook/blob/main/src/en/part8-e2e/e2e-recommendation.md
10. Google / YouTube. "PLUM: Adapting Pre-trained Language Models for Industrial-scale Generative Recommendations." 2025. https://arxiv.org/abs/2510.07784
11. Singh, A., Vu, T., Mehta, N., et al. "Better Generalization with Semantic IDs: A Case Study in Ranking for Recommendations." Google / YouTube, RecSys 2024. https://arxiv.org/abs/2306.08121
12. Rajput, S., Mehta, N., Singh, A., et al. "Recommender Systems with Generative Retrieval" (TIGER). Google, NeurIPS 2023. https://arxiv.org/abs/2305.05065
13. xAI / X. "X For You Feed Algorithm" (x-algorithm README, code and history; initial release 2026-01-20, updates through 2026-09). 2026. https://github.com/xai-org/x-algorithm
14. xAI / X. "Phoenix: Recommendation System" README. 2026. https://github.com/xai-org/x-algorithm/blob/main/phoenix/README.md
15. Twitter / X. "ScoredTweetsParam.scala" (Home Mixer scored-tweets parameters). 2023-2025. https://github.com/twitter/the-algorithm/blob/main/home-mixer/server/src/main/scala/com/twitter/home_mixer/product/scored_tweets/param/ScoredTweetsParam.scala
16. Twitter / X. "Visibility Filtering" README (visibilitylib). 2023. https://github.com/twitter/the-algorithm/blob/main/visibilitylib/README.md
17. Twitter Engineering. "Twitter's Recommendation Algorithm." Twitter Engineering Blog, 2023. https://blog.x.com/engineering/en_us/topics/open-source/2023/twitter-recommendation-algorithm
18. El-Kishky, A., Markovich, T., Park, S., et al. "TwHIN: Embedding the Twitter Heterogeneous Information Network for Personalized Recommendation." Twitter,

KDD 2022. https://arxiv.org/abs/2202.05387
19. Twitter / X. pushservice Scala `predicate` package. 2023. https://github.com/twitter/the-algorithm/tree/main/pushservice/src/main/scala/com/twitter/frigate/pushservice/predicate
20. Lada, A., Wang, M., Yan, T. "How machine learning powers Facebook's News Feed ranking algorithm." Engineering at Meta, 2021. https://engineering.fb.com/2021/01/26/ml-applications/news-feed-ranking/
21. Meta. "Incorporating More Feedback Into News Feed Ranking." Meta Newsroom, 2021. https://about.fb.com/news/2021/04/incorporating-more-feedback-into-news-feed-ranking/
22. CNN Business (syndicated). "Likes, anger emojis and RSVPs: the math behind Facebook's News Feed and how it backfired." 2021. https://abc17news.com/news/2021/10/27/likes-anger-emojis-and-rsvps-the-math-behind-facebooks-news-feed-and-how-it-backfired/
23. Meta Transparency Center. "Facebook Feed AI system" (system card). Accessed 2026. https://transparency.meta.com/features/explaining-ranking/fb-feed
24. Medvedev, I., Wu, H., Gordon, T. "Powered by AI: Instagram's Explore recommender system." Instagram Engineering, 2019. https://instagram-engineering.com/powered-by-ai-instagrams-explore-recommender-system-7ca901d2a882
25. Meta / Instagram. "How Instagram suggests new content." Engineering at Meta, 2020. https://engineering.fb.com/2020/12/10/web/how-instagram-suggests-new-content/
26. Vorotilov, V., Shugaepov, I. "Scaling the Instagram Explore recommendations system." Engineering at Meta, 2023. https://engineering.fb.com/2023/08/09/ml-applications/scaling-instagram-explore-recommendations-system/
27. Meta Transparency Center. "Instagram Explore AI system" (system card). Accessed 2026. https://transparency.meta.com/features/explaining-ranking/ig-explore/
28. Meta AI. "Harmful content can evolve quickly. Our new AI system adapts to tackle it" (Few-Shot Learner). Meta, 2021. https://ai.meta.com/blog/harmful-content-can-evolve-quickly-our-new-ai-system-adapts-to-tackle-it/ (not re-fetched)
29. Zuckerberg, M. "A Blueprint for Content Governance and Enforcement." Facebook, 2018. https://www.facebook.com/notes/mark-zuckerberg/a-blueprint-for-content-governance-and-enforcement/10156443129621634/ (not re-fetched)
30. Goodrow, C. "On YouTube's recommendation system." YouTube Official Blog, 2021. https://blog.youtube/inside-youtube/on-youtubes-recommendation-system/
31. YouTube. "Continuing our work to improve recommendations on YouTube." YouTube Official Blog, January 2019. https://blog.youtube/news-and-events/continuing-our-work-to-improve/ (not re-fetched)
32. YouTube. "The Four Rs of Responsibility, Part 2: Raising authoritative content and reducing borderline content and harmful misinformation." YouTube Official Blog, December 2019. https://blog.youtube/inside-youtube/the-four-rs-of-responsibility-raise-and-reduce/ (not re-fetched)
33. YouTube. "How YouTube Works: Recommendations." Accessed 2026. https://www.youtube.com/howyoutubeworks/product-features/recommendations/
34. Covington, P., Adams, J., Sargin, E. "Deep Neural Networks for YouTube Recommendations." Google / YouTube, RecSys 2016. https://research.google/pubs/deep-neural-networks-for-youtube-recommendations/

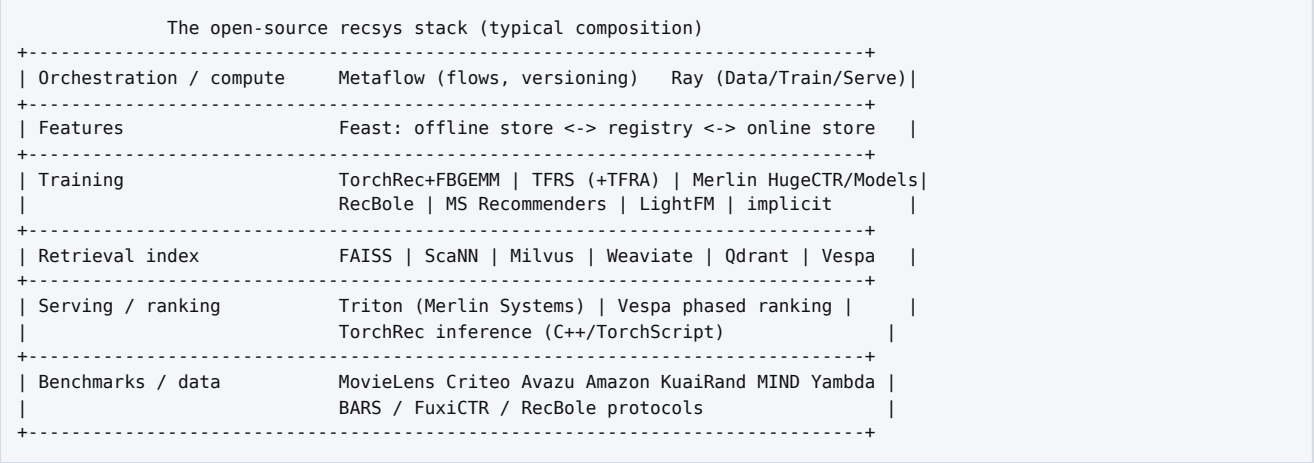
35. Chandrashekar, A., Amat, F., Basilico, J., Jebara, T. "Artwork Personalization at Netflix." Netflix Technology Blog, 2017. https://netflixtechblog.com/artwork-personalization-c589f074ad76
36. Dieleman, S. "Recommending music on Spotify with deep learning." Personal blog, 2014. https://benanne.github.io/2014/08/05/spotify-cnns.html
37. van den Oord, A., Dieleman, S., Schrauwen, B. "Deep content-based music recommendation." NeurIPS 2013. https://papers.nips.cc/paper/5004-deep-content-based-music-recommendation
38. Zhang, C., et al. "NoteLLM: A Retrievable Large Language Model for Note Recommendation." Xiaohongshu, WWW 2024. https://arxiv.org/abs/2403.01744
39. Jha, R., Subramaniyam, S., Benjamin, E., Taula, T. "Unified Embedding Based Personalized Retrieval in Etsy Search." Etsy, KDD 2023. https://arxiv.org/abs/2306.04833
40. Zinkevich, M. "Rules of Machine Learning: Best Practices for ML Engineering." Google for Developers. https://developers.google.com/machine-learning/guides/rules-of-ml
41. TikTok. "How TikTok recommends videos #ForYou." TikTok Newsroom, 2020. https://newsroom.tiktok.com/en-us/how-tiktok-recommends-videos-for-you (not re-fetched)
42. Bluesky Social. "ATProto Feed Generator" starter kit README. https://github.com/bluesky-social/feed-generator

34. Open-Source Frameworks, Tooling, Benchmarks and Public Datasets

Most of the systems in this atlas are proprietary. Almost all of them, though, are built from a small set of open-source parts, or have open-source versions of their core ideas. Meta's sharded embedding training comes out as TorchRec and FBGEMM. Google's two-tower retrieval and quantized maximum-inner-product search come out as TensorFlow Recommenders and ScaNN. NVIDIA's GPU-resident pipelines come out as Merlin. Academic groups maintain RecBole, Microsoft Recommenders, LightFM and implicit as testbeds. The serving layer is served by search engines and vector databases such as Vespa, Milvus, Weaviate, Qdrant and FAISS. Feature stores (Feast) and workflow and compute frameworks (Ray, Metaflow) hold the system together.

This chapter maps that ecosystem layer by layer. It then covers the public datasets and benchmarks that nearly every paper in the field reports on: MovieLens, Criteo, Avazu, Amazon Reviews, KuaiRand, MIND and Yambda. For each one it gives the problem it targets, its architecture, the scale numbers the maintainers publish, and the lessons a system designer should take away.

A note on sourcing. Live web search was unavailable while this chapter was being researched, and several hosts (arxiv.org, grouplens.org, huggingface.co, kuairand.com, docs.vespa.ai) could not be fetched. The chapter therefore relies mainly on primary GitHub repositories, READMEs, source files and notebooks, which were read directly. Facts that come from other chapters of this atlas, or from the author's prior knowledge of well-known papers, are marked **(approximate / not re-verified)**. Treat those figures as pointers to check, not as authoritative.



34.1 Industrial-scale training frameworks

34.1.1 TorchRec and FBGEMM (Meta / PyTorch)

Problem. Deep learning recommendation models (DLRMs) are dominated by sparse embedding tables. These can reach terabytes, far beyond one GPU's HBM. The dense MLPs on top are comparatively small. Plain data parallelism (replicating everything) fails on memory, and plain model parallelism leaves dense compute underused. Meta built TorchRec as "a PyTorch domain library" to make hybrid parallelism a first-class primitive [1][2].

Architecture. According to the README and the RecSys 2022 paper [1][2], TorchRec provides:

- **Modeling primitives:** embedding bags and jagged tensors (the `KeyedJaggedTensor` abstraction holds variable-length ID lists per feature), plus common recsys modules and an end-to-end DLRM example [1][2].
- **Sharding types:** data-parallel, table-wise, row-wise, table-wise-row-wise, column-wise and table-wise-column-wise sharding of embedding tables [1].
- **A planner** that "automatically generates optimized sharding plans" [1]. It estimates compute, memory and communication cost and assigns a scheme per table.
- **Kernels from FBGEMM** [1][3]. FBGEMM is a low-precision, high-performance GEMM library that supports row-wise and outlier-aware quantization. FBGEMM_GPU adds recsys GPU operators for training and inference, and a newer FBGEMM_GPU GenAI package adds FP8 row-wise quantization and collectives [3].
- **Train pipelining** that overlaps data loading, host-to-device copy, inter-device communication (ID and embedding all-to-all) and compute [1].
- **Reduced-precision training and inference**, and export to C++ inference [1]. The TorchRec inference library captures the model with `torch.fx` and runs it as TorchScript in a C++ gRPC server. Its README calls it "a simple reference … not a fully scaled out solution" [5].

```
      TorchRec sparse/dense hybrid step (per batch, conceptual)
GPU0..GPUn each hold: replicated dense MLPs (DDP) + a SHARD of embedding tables
1. input KJT (ids per feature) --all-to-all--> owner shard of each table/row range
2. owner shard: fused table-batched lookup (FBGEMM TBE) -> pooled embeddings
3. pooled embeddings --all-to-all--> back to the sample's GPU
4. dense forward/backward (interaction + MLPs), allreduce dense grads
5. embedding grads --all-to-all--> owner shard; fused optimizer update in-kernel
Pipeline: step 1 of batch i+1 overlaps steps 2-5 of batch i
```

Scale. TorchRec "powers many production RecSys models at Meta" [1]. Its paper abstract describes Meta's move from CPU asynchronous training to accelerator-based synchronous training [2]. The README links a follow-up on "Toward 100TB model with Embedding Offloading" [1]. The Neo/ZionEX system that TorchRec productionizes reported models of up to 12 trillion parameters **(approximate, from the Neo paper as summarized in Chapter 9)** [37]. The repository ships loaders for Criteo 1TB (24 days, 13 integer and 26 categorical features, 4,195,197,692 training rows across days 0-22, with day 23 for validation and test) and for MovieLens 20M/25M [4].

Key tricks and lessons.

- Choose sharding *per table*. Tiny tables go data-parallel, most tables go table-wise, and huge tables go row-wise or column-wise. A cost-model planner beats hand placement.
- Fuse many small embedding lookups into one table-batched kernel, and fuse the sparse optimizer into the backward pass so dense gradient tensors for sparse tables

are never materialized [3].
- Quantize both storage (row-wise INT8/INT4) and communication [3].
- Treat the C++ inference path as a reference. Production adds request batching, feature pipelines and sharded serving [5].

34.1.2 TensorFlow Recommenders (Google)

Problem. TFRS packages the Google two-tower and ranking recipes as a Keras-native library. It covers "data preparation, model formulation, training, evaluation, and deployment" [6].

Architecture. Models subclass `tfrs.Model` and implement `compute_loss`. Two task objects carry most of the logic [6]:

- `tfrs.tasks.Retrieval` computes an in-batch softmax loss for a query tower and a candidate tower. The `tfrs.metrics.FactorizedTopK` metric computes top-K recall against the full candidate corpus.
- `tfrs.tasks.Ranking` handles pointwise or listwise scoring of the retrieved candidates (cross layers such as DCN are offered as layers, **not re-verified in this pass**).

For serving, TFRS offers `factorized_top_k.BruteForce` and `factorized_top_k.ScaNN` layers with the same interface. Both can be exported as a SavedModel [7].

Scale numbers (tutorial). The efficient-serving tutorial [7] starts from MovieLens 100K (fewer than 1,000 movies). It pads the corpus with perturbed dummy embeddings to about 1 million candidates. Brute-force latency "grows linearly" with corpus size and is projected at about 250 ms per query at 10 million candidates. At about 1M candidates, ScaNN returns results in about 2 ms, roughly 15x faster than brute force, with top-100 accuracy "almost the same." Tuning `num_leaves` and `num_leaves_to_search` (for example 1000/100 rather than the 100/10 default), or adding reordering candidates (for example 70 leaves searched and 400 reordering candidates), trades latency for about 3 points of absolute recall [7].

Lessons. TFRS is the most readable reference for two-tower training with in-batch negatives. It is also a good teaching tool for the shift in evaluation from brute force to ANN. Its activity is modest (about 2k stars, 389 commits) [6]. Large-scale dynamic embeddings in the TensorFlow world live in the separate TensorFlow Recommenders-Addons (TFRA) project **(see Chapter 9)**.

34.1.3 NVIDIA Merlin (NVTabular, HugeCTR, Models, Transformers4Rec, Systems)

Problem. NVIDIA's goal is to keep the entire recommender lifecycle on GPUs: feature engineering, training and inference. Merlin is an umbrella of libraries with a shared schema [8].

Components [8][9][12][13][14][15]:

Component	Role	Notes
NVTabular	GPU ETL for tabular data	Handles TB-scale data beyond GPU and CPU memory
HugeCTR	CUDA/C++ training framework	Model-parallel embeddings, data-parallel dense layers, multi-node
Sparse Operation Kit (SOK)	GPU sparse ops for TF users	For teams who cannot adopt HugeCTR directly
HierarchicalKV	Header-only GPU hash table for embeddings	HBM + host memory, score-based eviction
Merlin Models	Reference retrieval and ranking models	MF, two-tower, YouTube DNN, DLRM, DCN-v2, DeepFM; TF first
Transformers4Rec	Session-based and sequential recommendation	Wraps 64+ Hugging Face architectures
Merlin Systems	Serving DAGs on Triton	Feast, FAISS and model operators in one ensemble
Merlin Core	Dataset abstraction, schema, DAG API	Shared by the above

HugeCTR training. Embedding layers are model-parallel across GPUs and nodes, and the dense model is data-parallel. Communication uses NCCL with GPUDirect RDMA. Embedding variants include localized-slot (each slot on one GPU) and distributed-slot (keys spread across GPUs) [11]. The RecSys 2022 paper describes a **hybrid embedding**: hot categories are replicated and cold categories are sharded, so frequent IDs avoid all-to-all. This was a key optimization in NVIDIA's MLPerf DLRM submissions [10]. HugeCTR documentation states that two nodes of 8x A100 80 GB can train models of about 1 TB entirely in GPU memory [11]. The repository's quick-start is a DCN model on Criteo-style input (26 sparse slots, 13 dense features, embedding size 16) [9].

Deprecations. The Hierarchical Parameter Server (HPS) and Embedding Cache are listed as deprecated as of v25.03 [9]. HierarchicalKV is the surviving general-purpose piece. It keeps keys in HBM, lets values spill to host memory, and evicts only when a bucket is full. Eviction is driven by a per-key score under LRU, LFU, EpochLRU, EpochLFU or custom policies [12]. Chapter 9 summarizes HKV's published A100 benchmarks: about 1-2.5 billion KV/s for pure-HBM operations, dropping by an order of magnitude or more in hybrid HBM+host mode **(approximate)**.

Merlin Systems expresses an end-to-end recommender as a Triton ensemble [14]:

```
request(user_id)
   |
QueryFeast(user features) --> PredictTensorflow(query tower) --> QueryFaiss(top-100)
   |                                                                 |
   +--> FilterCandidates(remove already-seen) <--------------------+
            |
        QueryFeast(item features) -> UnrollFeatures -> PredictTensorflow(ranker)
            |
        SoftmaxSampling(temperature, top-10)  --> response
Ensemble(...).export(dir);  tritonserver --model-repository=dir
```

This is the canonical four-stage design (retrieval, filtering, ranking, ordering) shown in Chapter 1, packaged as code. Softmax sampling at the ordering stage adds explicit exploration [14].

Transformers4Rec adapts Hugging Face transformer architectures, which natively take only token-ID sequences, to sequences of multiple tabular features. It supports next-item prediction, classification and regression heads [15]. Transformer-based approaches from the team won the 2021 WSDM WebTour (Booking.com) and SIGIR eCommerce (Coveo) session-based challenges [15].

Lessons. Merlin's main contribution is architectural: one schema flows from ETL through training to the serving DAG, so training/serving skew is reduced by construction. Its maintenance signals are mixed, though. The Merlin meta-repository shows about 200 open issues, Merlin Models states its PyTorch API is "initiated, but incomplete", and HPS is deprecated [8][9][13]. Adopters should treat it as a set of components to cherry-pick (NVTabular, HKV, Triton ensembles) rather than as a single supported platform.

34.2 Research and prototyping libraries

34.2.1 RecBole (RUC AI Box)

RecBole is the de facto unified academic benchmark library in PyTorch. Its README currently lists **94 algorithms** across four families (general/collaborative filtering, sequential, context-aware/CTR, and knowledge-based). It also cites 44 benchmark datasets in the introduction, while its comparison table lists 43 [16]. All datasets are converted to a single atomic-file format (`.inter`, `.user`, `.item`, `.kg`). Features include multi-GPU and mixed-precision training, three hyperparameter search methods, standardized evaluation protocols, and time and memory benchmarks [16]. Releases run from v0.1.1 (November 2020) to v1.2.1 (February 2025). The project is described in three papers: RecBole 1.0 (CIKM 2021), RecBole 2.0 (CIKM 2022, eight extension packages) and a SIGIR 2023 resource paper [16].

Design lesson. RecBole separates the data, model, trainer and evaluator modules behind a configuration system. A new model therefore only implements `calculate_loss` and `predict`/`full_sort_predict`. That separation is what makes fair comparison possible: same splits, same negative sampling, same full-ranking metrics.

34.2.2 Microsoft Recommenders (Linux Foundation AI & Data)

Microsoft Recommenders, now under the Linux Foundation AI & Data, is a best-practices repository. It passed 20,000 GitHub stars in April 2025 [17]. It is organized as notebooks for five tasks (prepare data, model, evaluate, model select and optimize, operationalize), plus a `recommenders` Python package with data loaders, splitters (random, chronological, stratified) and metrics (MAP, nDCG@k, Precision@k, Recall@k, RMSE, MAE, R2, explained variance) [17].

The algorithm list spans classic and deep CF: ALS, BPR (via Cornac), BiVAE, LightGCN, NCF, SAR, SASRec, xDeepFM, Wide & Deep, Multinomial VAE, RBM and more. It also includes Microsoft's news models (DKN, LSTUR, NAML, NPA, NRMS), which are evaluated on MIND [17]. SARplus provides a Spark implementation of the Simple Algorithm for Recommendation (SAR) [17].

Its MovieLens 100K benchmark notebook (a single 75/25 split, explicitly "illustrative") is a useful sanity table [18]:

Algorithm	Train time (s)	MAP@10	nDCG@10	Precision@10	Recall@10
ALS (Spark)	13.4	0.011	0.037	0.043	0.015
SAR	0.6	0.258	0.394	0.341	0.185
NCF	70.3	0.277	0.411	0.360	0.189
BPR (Cornac)	3.8	0.229	0.362	0.285	0.252
BiVAE	31.5	0.330	0.472	0.411	0.224
LightGCN	28.9	0.281	0.420	0.362	0.193

The lesson is clear. SAR, a co-occurrence item-similarity method that trains in under a second, comes within a few points of nDCG of much heavier neural models on small data [18]. The ALS row is low because it optimizes rating RMSE rather than ranking. A simple, well-tuned baseline belongs in every system's offline evaluation.

34.2.3 LightFM (Lyst)

LightFM implements Maciej Kula's hybrid matrix factorization [19]. Each user and item is represented "as the sum of the latent representations of their features." Identity features recover pure MF, and metadata features give cold-start coverage for new items and new users [19]. It ships efficient BPR and WARP ranking losses with multithreaded (Hogwild-style) fitting [19]. Its WARP loss, which samples negatives until one violates the margin and weights the update by the estimated rank, is a widely copied recipe for optimizing precision at the top of the list. LightFM is a good production baseline for small and medium catalogs with rich metadata and frequent cold start. It is not designed for large-scale distributed training.

34.2.4 implicit (Ben Frederickson)

`implicit` is the pragmatic workhorse for implicit-feedback CF on a single machine [20]. It implements ALS (Hu-Koren-Volinsky confidence weighting, with a conjugate-gradient solver), BPR, logistic MF, and item-item nearest neighbours (cosine, TF-IDF, BM25). ALS and BPR have custom CUDA kernels, and pip wheels include GPU support on Linux [20]. Recommendation generation can use Annoy, NMSLIB or FAISS for approximate top-K [20]. Two operational notes from the README matter in practice. Set `OPENBLAS_NUM_THREADS=1` / `MKL_NUM_THREADS=1` to avoid oversubscription with the library's own OpenMP threading, which gives "substantial speedups." The project also benchmarks ALS fitting against Spark MLlib [20].

Design lesson. For a first-version recommender, or a candidate generator in a larger cascade, `implicit` ALS plus an ANN index over item factors is often the strongest cost/quality point. A GPU ALS sweep over millions of users completes in minutes on one machine.

34.2.5 CTR benchmarking libraries: FuxiCTR and BARS

Huawei Noah's Ark and collaborators maintain **FuxiCTR**, a configurable CTR library with a model zoo of 57 entries. The entries span feature interaction (DeepFM, DCN, AutoInt and others), behavior sequence, long sequence, dynamic weight and multi-task models, mostly in PyTorch with some TensorFlow ports [44]. **BARS** ("open BenchmArking for Recommender Systems", SIGIR 2022) builds standardized benchmarks on top of it: BARS-CTR and BARS-Match, with BARS-Rerank and BARS-MTL in progress [43]. The project's stated motivation is that without a recognized benchmark, studies are "hard to reproduce" and results conflict across papers [43]. The companion `reczoo/Datasets` repository publishes frozen splits with versioned IDs (Criteo_x1/x2/x4, Avazu_x1/x2/x4, KKBox_x1, Frappe_x1, MIND_small_x1, AmazonBooks_m1, Yelp18_m1, Gowalla_m1 and more) [42]. A paper can then report "Avazu_x4" and mean exactly one preprocessing and split.

34.3 Retrieval indexes: ANN libraries and vector databases

Every two-tower retrieval stage needs an index that returns approximate top-K by inner product or cosine over millions to billions of item vectors. The open-source options fall into **libraries** (FAISS, ScaNN), which give you a data structure, and **engines** (Milvus, Weaviate, Qdrant, Vespa), which add storage, replication, filtering and APIs.

34.3.1 FAISS (Meta FAIR)

FAISS is the reference ANN library [27][28]. Its index families are [27]:

- Exact (flat) search.
- Compressed-code indexes (product quantization and its variants, binary codes), which do not need the original vectors and "can scale to billions of vectors in main memory on a single server."
- Graph indexes on raw vectors (HNSW, NSG).

Inverted-file (IVF) coarse partitioning composes with these. Index factory strings such as `IVF65536,PQ32` combine coarse partition and code. Metrics are L2, inner product, and cosine (inner product on normalized vectors) [27]. The GPU implementation can replace CPU indexes directly (for example `GpuIndexFlatL2`), supports multi-GPU, runs on CUDA or ROCm, and can optionally use NVIDIA cuVS as a backend [27]. The GPU paper (Johnson, Douze, Jégou) claimed the fastest exact and approximate k-NN for high-dimensional vectors as of 2017 [29]. The 2024 "The Faiss library" paper is now the recommended citation [28].

34.3.2 ScaNN (Google Research)

ScaNN targets maximum inner product search (MIPS), which is what dot-product two-tower models need, and it is optimized for x86 AVX/FMA (and ARM NEON) [30]. Search runs in three phases [31]:

```
query q
  |
  v
[1] partition (tree, num_leaves ~ sqrt(n)) -> pick num_leaves_to_search leaves
  |
  v
[2] score candidates with asymmetric hashing (AH), anisotropic quantization,
    dimensions_per_block = 2   (or brute force, optionally int8-quantized)
  |
  v
[3] rescore top k' (reordering_num_neighbors > k) with exact / int8 dot product
  |
  v
top-k
```

The library's sizing guidance [31] is: brute force below about 20k points; AH plus rescoring below about 100k; and partitioning plus AH plus rescoring above that, with `num_leaves` around sqrt(n). Int8 brute-force scoring cuts memory 4x and helps bandwidth-bound workloads, with "generally negligible" accuracy loss [31]. The core algorithmic contribution is **anisotropic vector quantization** (ICML 2020). It penalizes quantization error parallel to the data point more than orthogonal error, because parallel error dominates inner-product ranking errors [32]. **SOAR** (NeurIPS 2023) adds spilled assignments with orthogonality-amplified residuals for better partition recall [33]. ScaNN claims state-of-the-art results on ann-benchmarks (glove-100-angular) [30]. It is integrated into TFRS [7], and Milvus offers it as an index type [24].

34.3.3 Milvus, Weaviate, Qdrant

Engine	Language	Index types (as documented)	Distinctive features for recsys	Scale claim (vendor)
Milvus [24]	Go/C++	HNSW, IVF, FLAT, SCANN, DiskANN, quantized variants, mmap; GPU (CAGRA)	Compute/storage disaggregation; query nodes scale for reads, data nodes for writes; Kubernetes-native; streaming ingest	"tens of thousands of search queries on billions of vectors"
Weaviate [25]	Go	HNSW (plus others, not re-verified)	Hybrid BM25 + vector in one call; built-in vectorizer modules; multi-tenancy; quantization and multi-vector	"billions of vectors in milliseconds"
Qdrant [26]	Rust	HNSW	Rich payload filters (keyword, full-text, range, geo; must/should/must_not); **recommend API with positive and negative examples**; Discovery; sparse vectors; quantization claimed to cut RAM up to 97%	Horizontal sharding and replication, zero-downtime resizing

All three scale claims are vendor statements without benchmarks in the READMEs [24][25][26].

Recsys-relevant observations.

- **Filtering is the hard part.** Recommendation queries almost always carry constraints such as in-stock, region, language, already-seen and safety. Pre-filtering versus post-filtering versus filter-aware graph traversal determines recall under selective filters. Qdrant's payload filter model and Milvus's metadata filtering exist for this reason [24][26].

- **Qdrant's recommend API** turns "more like these, less like those" into a single query using positive and negative example points [26]. That is effectively a lightweight session-based recommender without a trained query tower.
- **Freshness.** Milvus's separation of data nodes from query nodes [24] is a direct answer to the recsys need for continuous item insertion (new videos, new listings) without rebuilding the index.

34.3.4 Vespa

Vespa (Apache 2.0, originally from Yahoo) is not just a vector database. It is a serving engine that runs **retrieval and machine-learned ranking next to the data** [21]. Content nodes hold documents, inverted indexes and HNSW tensors, and evaluate first- and second-phase ranking expressions locally. Container nodes scatter queries, merge results and run a global phase. Models (ONNX, GBDT, tensor expressions) execute in the ranking phases [22]. The README states that Vespa is used by large internet services serving "hundreds of thousands of queries" per second, typically within about 100 ms [21]. The sample-apps repository includes album-recommendation and news-recommendation (MIND-based) tutorials that implement two-tower retrieval with `nearestNeighbor` plus ranking profiles [23].

```
query ---> container (stateless)
             | scatter
   +---------+---------+
   v         v         v
content   content   content       per node: retrieve (WAND / HNSW / filters)
node 1    node 2    node N          -> first-phase (cheap expr, all hits)
   |         |         |            -> second-phase (ML model, top rerank-count)
   +---------+---------+
             | gather top hits
             v
      global-phase rerank (e.g. ONNX cross-model on GPU) -> response
```

Lesson. Moving compute to the data avoids shipping thousands of candidates' features over the network. That is the same insight as Twitter Earlybird's index-side light ranker (Chapter 10). Vespa is the main open-source system where retrieval, filtering, feature storage (attributes) and multi-phase ranking all live in one engine.

34.4 Feature stores and ML platforms

34.4.1 Feast

Feast is the most widely used open-source feature store [34]. Its architecture has four parts:

- An **offline store** for historical features (Snowflake, BigQuery, Redshift, Spark, DuckDB, Postgres and community plugins such as Hive and Trino).
- An **online store** for low-latency lookup (Redis, DynamoDB, Bigtable, Cassandra/AstraDB, Postgres, SQLite).
- A **registry** of feature definitions, populated by `feast apply`.
- **Feature servers**: Python, with Go and Java servers in alpha [34].

Two operations carry the recsys value [34]:

1. **Point-in-time joins** (`get_historical_features`). Each training row gets feature values as of its event timestamp, which prevents future-data leakage into training labels.
2. **Materialization** (`feast materialize` / `materialize-incremental`) copies the latest values to the online store for serving.

```
event log (user_id, item_id, ts, label) --+
                                          |  point-in-time join
feature tables (entity, ts, values) -----+--> training set (no leakage)
          |
          | materialize(-incremental)
          v
online store (Redis/Dynamo/...) <-- get_online_features(entity rows) <-- ranker
```

Feast also lists vector search (alpha), with Milvus, Qdrant and FAISS as online-store backends [34]. This blurs the line between feature store and retrieval index. Merlin Systems' `QueryFeast` operator shows the standard integration in a serving DAG [14].

Lesson. Feast does not compute streaming features itself. It stores and serves them. Teams that need sub-minute counters (clicks in the last 5 minutes) pair it with a stream processor, or choose a computation-oriented platform (Chronon, Tecton, Feathr; see Chapter 8).

34.4.2 Ray

Ray is a general distributed runtime. **Core** provides tasks (stateless functions), actors (stateful workers) and an immutable object store. On top sit **Data** (scalable datasets for ML), **Train** (distributed training), **Tune** (hyperparameter search), **Serve** (programmable model serving) and **RLlib** (reinforcement learning) [35]. In recsys stacks, Ray typically does one of three jobs:

- Ray Data streams preprocessing to GPU trainers (replacing Spark-to-files-to-trainer hops).
- Ray Tune sweeps embedding dimensions and learning rates.
- Ray Serve composes multi-model inference graphs (retrieval, ranking and business-logic stages as separate deployments with independent autoscaling).

RLlib is a natural home for bandit and RL slate experiments (Chapter 13).

34.4.3 Metaflow (Netflix, now Outerbounds)

Metaflow came out of Netflix's ML platform. It models ML work as a DAG of Python steps with automatic artifact versioning, local-to-cloud scaling on CPUs and GPUs, gang-scheduled distributed jobs, dependency management, and deployment to production orchestrators with event-triggered ("reactive") flows [36]. Netflix reports more than 3,000 AI/ML projects on it, "hundreds of millions of compute jobs", petabytes of data processed and tens of petabytes of models and artifacts [36]. Other named users include Amazon, DoorDash, Dyson, Goldman Sachs and Ramp [36]. For recommender teams, its value is reproducible daily retraining DAGs (extract, features, train, evaluate, publish embeddings, refresh index) with every intermediate artifact addressable for debugging.

34.4.4 Framework comparison

Tool	Layer	Primary language / runtime	Best fit	Scale evidence (public)
TorchRec + FBGEMM [1][3]	Training / inference	PyTorch, CUDA	Terabyte embedding DLRMs, sync GPU training	Meta production; 100TB offloading work
TFRS [6]	Training	TF / Keras	Two-tower and ranking prototypes, education	Tutorials to ~1M candidates
Merlin HugeCTR / HKV [9][12]	Training / embeddings	CUDA / C++	GPU-resident CTR training, GPU hash tables	~1 TB on 2x8 A100
Merlin Systems [14]	Serving	Triton	Multi-stage serving DAGs	Reference examples
RecBole [16]	Research	PyTorch	Reproducible comparison of 94 models	44 datasets
MS Recommenders [17]	Research / best practices	Python, Spark	Notebooks, metrics, splitters, MIND models	20k+ stars
LightFM [19]	Model	Cython	Hybrid cold-start MF	Single machine
implicit [20]	Model	Cython / CUDA	Fast implicit ALS/BPR candidate generation	Single machine, GPU
FAISS [27]	ANN library	C++ / CUDA	Billion-scale ANN on CPU or GPU	Billions per server
ScaNN [30]	ANN library	C++ (AVX/NEON)	MIPS for dot-product towers	ann-benchmarks SOTA claim
Milvus / Weaviate / Qdrant [24-26]	Vector DB	Go-C++ / Go / Rust	Managed ANN with filters and updates	Vendor claims to billions
Vespa [21]	Search + ranking engine	C++ / Java	Retrieval and ML ranking on content nodes	Hundreds of thousands of QPS
Feast [34]	Feature store	Python (+Go/Java)	Point-in-time training sets, online lookup	Many backends
Ray [35]	Compute	Python	Data, train, tune, serve pipelines	General
Metaflow [36]	Workflow	Python	Versioned retraining DAGs	3,000+ projects at Netflix

34.5 Public datasets

A dataset is a design decision. It fixes the feedback type (explicit ratings versus implicit events versus impressions), whether negatives are observed, whether the exposure policy is known, the time span, and the split protocol. Those choices decide which conclusions transfer to production.

34.5.1 MovieLens (GroupLens, University of Minnesota)

MovieLens is the most-cited recommendation dataset family. It consists of explicit 1-5 star ratings (half stars in later versions) from the MovieLens research site, with tags and genre metadata. The history and appropriate uses are documented by Harper and Konstan (ACM TiiS 2015) [39]. Common variants **(approximate / not re-verified; GroupLens could not be fetched)**: ML-100K (about 100k ratings, 943 users, 1,682 movies), ML-1M (about 1M ratings, about 6k users), ML-20M, ML-25M (about 25M ratings, about 162k users, about

62k movies), and the 2024 ML-32M release. TorchRec ships loaders for 20M and 25M with columns `userId, movieId, rating, timestamp` [4]. TFRS and Microsoft Recommenders use 100K in their tutorials and benchmarks [6][18].

Caveats for designers. Ratings are explicit and filtered (every user has at least 20 ratings). Users rate in bursts at sign-up, so timestamps poorly reflect consumption order. The data comes from a recommender-mediated site with no exposure logs. MovieLens is good for debugging pipelines and teaching. It is weak as evidence that a sequential or implicit-feedback model will win in production.

34.5.2 Criteo (display advertising CTR)

- **Criteo Kaggle / Display Advertising Challenge.** About one week of display-ad click data, with 13 numeric and 26 categorical (hashed) features. The BARS Criteo_x1 version has 45,840,617 samples, split randomly 7:2:1 into 33,003,326 train, 8,250,124 validation and 4,587,167 test rows [40].
- **Criteo 1TB Click Logs.** 24 days with the same 13+26 schema. TorchRec counts 4,195,197,692 training rows across days 0-22 and uses day 23 for validation and test, with a frequency threshold of 3 for remapping rare IDs [4]. This is the dataset behind the DLRM reference implementation and MLPerf recommendation training [37][38]. A multi-hot synthetic extension is used for the newer DLRM-DCNv2 MLPerf benchmark **(approximate / not re-verified)**.

The original DLRM repository [37] documents the pipeline: raw `train.txt`, then a preprocessed `.npz`, then training with `--memory-map` and sub-sampling options for the Terabyte data. DLRM itself is a bottom MLP for dense features plus embedding lookups for sparse features, combined by a dot, sum or concatenation interaction, followed by a top MLP and a sigmoid [37][38].

Caveats. Features are anonymized and hashed, so no feature engineering transfers. Criteo AUC gains of 0.001 are conventionally treated as significant, which makes split and preprocessing standardization (BARS) essential [43].

34.5.3 Avazu (mobile ads CTR)

Avazu comes from the Kaggle Avazu CTR Prediction competition. It covers about 10 days of mobile-ad click logs with 22 fields (ad, site, app, device attributes and anonymized categorical features). The BARS Avazu_x1 version has 40,428,967 samples, split randomly 7:1:2 into 28,300,276 train, 4,042,897 validation and 8,085,794 test rows [41]. Avazu and Criteo are the standard pair for feature-interaction models (FM, DeepFM, DCN, xDeepFM, AutoInt, FinalMLP) in FuxiCTR [44].

34.5.4 Amazon Reviews (McAuley lab, UCSD)

The Amazon reviews corpora have been released in several versions (2013, 2014, 2018, and the 2023 release accompanying BLaIR). They provide user-item-rating-timestamp-review data with rich item metadata (title, description, categories, price, images), split by product category [45]. The 2023 release is reported at roughly 570M reviews and about 48M items across 33 categories, spanning 1996 to 2023 **(approximate / not re-verified)**. Its

repository provides scripts that build "standard train/validation/test splits" for benchmarking [45]. The 2023 data is the substrate for text-based and LLM-based recommenders (BLaIR pairs language and items for retrieval) [45]. Older category subsets (Beauty, Sports, Toys, Books, CDs) remain the standard sequential-recommendation benchmarks, and they appear as AmazonBooks_m1 and others in reczoo [42].

34.5.5 KuaiRand (Kuaishou, CIKM 2022)

KuaiRand is the key public dataset for **unbiased and off-policy** recommendation research [46]. It is built from Kuaishou short-video logs in which each recommended video had an equal chance of being replaced by a random video from a candidate pool. About 0.37% of final interactions are these random exposures, giving "millions of intervened interactions" [46].

Version	Users	Items	Interactions	Size
KuaiRand-27K	27,285	~32 million	322,278,385	~46 GB
KuaiRand-1K	1,000 (sampled)	~4.37 million	11,713,045	~4.3 GB
KuaiRand-Pure	27,285	7,551	1,436,609	~194 MB

Each interaction carries **12 feedback signals** (click, like, follow, comment, forward, hate, long view, view time and others), and the logs span 15 recommendation policies [46]. That makes the dataset suitable for multi-task learning, debiasing (random exposures act as an unbiased test set), and off-policy evaluation. Its sibling KuaiRec provides a nearly fully observed user-item matrix **(not re-verified in this pass)**.

34.5.6 MIND (Microsoft News Dataset, ACL 2020)

MIND was collected from anonymized Microsoft News logs over six weeks (12 October to 22 November 2019) [47]. It covers about 1M users (sampled from users with at least 5 news clicks), about 160k English articles and more than 15M impression logs [47][48]. MIND-small has 50,000 users and train/validation splits only. The full dataset trains on week five (with histories from weeks one to four), validates on the last day of week five, and tests on week six [47]. Each article has title, abstract, body, category and Wikidata-linked entities, with 100-dimensional TransE entity embeddings [47].

The design-relevant property is the **impression log**. Each record lists exactly what was shown and which items were clicked, so models can train on true shown-but-not-clicked negatives and be evaluated with per-impression AUC, MRR and nDCG@5/10 [47][48]. Microsoft Recommenders ships DKN, NPA, NAML, LSTUR and NRMS on MIND [17], and Vespa's news tutorial uses it [23].

34.5.7 Yambda (Yandex, 2025)

Yambda ("Yandex Music Billion-interactions Dataset") is, as of 2025-2026, the largest open recommendation dataset. All figures below are **approximate / not re-verified**, because Hugging Face and arXiv could not be fetched [49]:

- **Size and versions.** About 4.79 billion user-item interactions from roughly 1M Yandex Music users over about 9.4M tracks, released in 50M, 500M and 5B-event versions.

- **Feedback.** Implicit listens (with played-ratio information) and explicit likes, dislikes, unlikes and undislikes.
- **Flags and content.** An `is_organic` flag distinguishes organic listens from recommendation-driven ones, and precomputed audio embeddings are included for many tracks.
- **Evaluation.** A *global temporal split* (train strictly before a time cutoff, test after), rather than leave-one-out, to mimic production deployment.
- **Baselines.** Popularity variants, ItemKNN, iALS, BPR, SANSA and SASRec.

Yambda matters because it closes the scale gap. Most academic results are on datasets with 10^5-10^8 events, while production systems train on 10^10 or more. The organic/recommended split is a cheap exposure signal similar in spirit to KuaiRand's random-exposure slice. It lets researchers separate "what users chose" from "what the system pushed."

34.5.8 Dataset comparison

Dataset	Domain	Feedback	Negatives / exposure	Scale (approx.)	Typical use
MovieLens [39]	Movies	Explicit ratings	None	100K-32M ratings	Teaching, CF baselines
Criteo Kaggle / 1TB [4][40]	Display ads	Click label	Shown, non-clicked rows	45.8M / 4.2B rows	CTR, DLRM, MLPerf
Avazu [41]	Mobile ads	Click label	Shown, non-clicked rows	40.4M rows	CTR feature interaction
Amazon Reviews [45]	E-commerce	Ratings + text	None	Up to hundreds of millions of reviews	Sequential, text/LLM recsys
KuaiRand [46]	Short video	12 signals	Random-exposure slice	322M interactions	Debiasing, OPE, MTL
MIND [47][48]	News	Clicks	Impression slates	15M+ impressions	News recsys, NLP encoders
Yambda [49]	Music	Listens + likes/dislikes	Organic flag	~4.8B events	Scale, temporal-split evaluation

---

34.6 Benchmarking practice

Tooling has converged on a few protocol choices that strongly affect reported numbers:

1. **Split.** Random splits (the BARS Criteo/Avazu convention [40][41]) leak future information. Leave-one-out (common in sequential recsys) ignores global time. Global temporal splits (MIND [47], Yambda [49]) best match deployment. Report which one you used.
2. **Candidate set at evaluation.** Full ranking over all items (RecBole's `full_sort_predict` [16], TFRS `FactorizedTopK` [6]) versus ranking against sampled negatives. Sampled metrics can reorder models, so prefer full ranking or report both.
3. **Frozen preprocessing.** Frequency thresholds (TorchRec uses 3 for Criteo [4]), hashing, and the definition of a "positive" (rating >= 4, play ratio above a threshold)

all move metrics. BARS dataset IDs exist to freeze them [42][43].

4. **Strong simple baselines.** The Microsoft Recommenders benchmark shows SAR and BPR within reach of neural models on MovieLens 100K at a fraction of the training time [18]. iALS and ItemKNN sit in the Yambda baseline set for the same reason [49].
5. **Systems metrics.** Throughput, latency at a recall target (ScaNN/TFRS tuning curves [7][31]) and memory footprint belong next to quality metrics. A 0.1% AUC gain that doubles embedding memory is rarely worth it.

```
Reproducible offline benchmark loop
raw logs -> frozen preprocessing (versioned ID, e.g. Avazu_x4) -> temporal split
       -> library (RecBole / FuxiCTR / TorchRec) with fixed config + seeds
       -> full-ranking metrics (nDCG@k, Recall@k, AUC/logloss) + latency/memory
       -> compare vs simple baselines (popularity, ItemKNN, iALS, SAR)
       -> only then: online A/B (Chapter 12)
```

34.7 Key takeaways for system designers

1. **Assemble, don't invent.** A credible production stack can be built entirely from open source: Feast for features, `implicit` or a TFRS/TorchRec two-tower for candidates, FAISS/ScaNN or a vector DB for the index, TorchRec/HugeCTR for the ranker, Triton or Vespa for serving, and Metaflow or Ray for orchestration. The proprietary edge is in data, features and objectives, not in the plumbing.
2. **Embedding tables are the system.** TorchRec's sharding planner, FBGEMM's fused table-batched kernels, HugeCTR's hybrid hot/cold embedding and HierarchicalKV's score-based eviction all solve the same problem: skewed, huge, sparse parameters. Choose a framework by how it shards, caches and evicts embeddings [1][3][10][12].
3. **Pick the ANN layer by filtering and freshness needs, not peak QPS.** Libraries (FAISS, ScaNN) are fastest when you control rebuilds. Engines (Milvus, Qdrant, Weaviate, Vespa) earn their cost when you need filtered queries, streaming inserts and replication [24]-[27][30].
4. **Use MIPS-aware quantization for dot-product towers.** ScaNN's anisotropic quantization targets inner-product ranking error directly. Always add a rescoring stage over a few hundred candidates [31][32].
5. **Compute near the data when candidate sets are large.** Vespa's phased ranking on content nodes shows the payoff of not shipping thousands of candidate features across the network [21][22].
6. **Point-in-time correctness is non-negotiable.** Use a feature store's point-in-time join (or an equivalent) for every training set, and materialize the same definitions to the online store [34].
7. **Package the four-stage pipeline as one versioned DAG.** Merlin Systems' retrieval, filter, rank and sample ensemble is a concrete template that keeps exploration (softmax sampling) explicit [14].
8. **Match the dataset to the claim.** MovieLens for debugging. Criteo/Avazu for CTR feature interaction. KuaiRand for debiasing and off-policy work. MIND for impression-aware news ranking. Amazon 2023 for text/LLM methods. Yambda when scale and temporal realism matter.
9. **Freeze protocols and report systems costs.** Use versioned splits (BARS), full-ranking metrics, temporal splits and strong simple baselines. Report latency and

memory alongside accuracy [16][18][43].

10. **Watch maintenance signals.** Several well-known projects have deprecated components (HugeCTR HPS), incomplete APIs (Merlin Models PyTorch) or modest activity (TFRS). Check commit activity and deprecation notes before you build on a component [6][9][13].

References

1. PyTorch / Meta. "TorchRec" GitHub repository and README. Accessed 2026. https://github.com/pytorch/torchrec
2. Ivchenko, D., Van Der Staay, D., Taylor, C., Liu, X., Feng, W., Kindi, R., Sudarshan, A., Sefati, S. "TorchRec: a PyTorch Domain Library for Recommendation Systems." Meta, RecSys 2022. https://doi.org/10.1145/3523227.3547387
3. PyTorch / Meta. "FBGEMM and FBGEMM_GPU" GitHub repository. Accessed 2026. https://github.com/pytorch/FBGEMM
4. PyTorch / Meta. TorchRec dataset loaders `criteo.py` and `movielens.py`. Accessed 2026. https://github.com/pytorch/torchrec/blob/main/torchrec/datasets/criteo.py ; https://github.com/pytorch/torchrec/blob/main/torchrec/datasets/movielens.py
5. PyTorch / Meta. "TorchRec Inference Library" README. Accessed 2026. https://github.com/pytorch/torchrec/blob/main/torchrec/inference/README.md
6. TensorFlow. "TensorFlow Recommenders" GitHub repository. Google, accessed 2026. https://github.com/tensorflow/recommenders
7. TensorFlow. "Efficient serving" tutorial notebook (BruteForce vs ScaNN). Google, accessed 2026. https://github.com/tensorflow/recommenders/blob/main/docs/examples/efficient_serving.ipynb
8. NVIDIA. "NVIDIA Merlin" GitHub repository. Accessed 2026. https://github.com/NVIDIA-Merlin/Merlin
9. NVIDIA. "Merlin HugeCTR" GitHub repository and README (v25.03 deprecation notes). Accessed 2026. https://github.com/NVIDIA-Merlin/HugeCTR
10. Wei, Y. et al. "Merlin HugeCTR: GPU-accelerated Recommender System Training and Inference." NVIDIA, RecSys 2022. https://arxiv.org/abs/2210.08803
11. NVIDIA. "HugeCTR Core Features" documentation. Accessed 2026. https://github.com/NVIDIA-Merlin/HugeCTR/blob/main/docs/source/hugectr_core_features.md
12. NVIDIA. "HierarchicalKV" GitHub repository. Accessed 2026. https://github.com/NVIDIA-Merlin/HierarchicalKV
13. NVIDIA. "Merlin Models" GitHub repository. Accessed 2026. https://github.com/NVIDIA-Merlin/models
14. NVIDIA. "Merlin Systems" GitHub repository. Accessed 2026. https://github.com/NVIDIA-Merlin/systems
15. NVIDIA. "Transformers4Rec" GitHub repository; Moreira, G. de S. P. et al., "Transformers4Rec: Bridging the Gap between NLP and Sequential / Session-Based Recommendation," RecSys 2021. https://github.com/NVIDIA-Merlin/Transformers4Rec ; https://dl.acm.org/doi/10.1145/3460231.3474255
16. RUCAIBox. "RecBole" GitHub repository (RecBole 1.0 CIKM 2021, 2.0 CIKM 2022, SIGIR 2023). Accessed 2026. https://github.com/RUCAIBox/RecBole

17. Recommenders team (Microsoft, Linux Foundation AI & Data). "Recommenders" GitHub repository. Accessed 2026. https://github.com/recommenders-team/recommenders
18. Recommenders team. "MovieLens benchmark" notebook (examples/06_benchmarks). Accessed 2026. https://github.com/recommenders-team/recommenders/blob/main/examples/06_benchmarks/movielens.ipynb
19. Lyst / Kula, M. "LightFM" GitHub repository; Kula, M., "Metadata Embeddings for User and Item Cold-start Recommendations," CBRecSys workshop at RecSys 2015. https://github.com/lyst/lightfm ; https://arxiv.org/abs/1507.08439
20. Frederickson, B. "implicit: Fast Python Collaborative Filtering for Implicit Datasets" GitHub repository. Accessed 2026. https://github.com/benfred/implicit
21. Vespa.ai. "Vespa" GitHub repository and README. Accessed 2026. https://github.com/vespa-engine/vespa
22. Vespa.ai. "Phased Ranking" documentation. 2025-2026. https://docs.vespa.ai/en/ranking/phased-ranking.html
23. Vespa.ai. "Vespa sample applications" (album recommendation, news recommendation on MIND). Accessed 2026. https://github.com/vespa-engine/sample-apps
24. Zilliz / LF AI & Data. "Milvus" GitHub repository. Accessed 2026. https://github.com/milvus-io/milvus
25. Weaviate. "Weaviate" GitHub repository. Accessed 2026. https://github.com/weaviate/weaviate
26. Qdrant. "Qdrant" GitHub repository. Accessed 2026. https://github.com/qdrant/qdrant
27. Meta FAIR. "Faiss" GitHub repository. Accessed 2026. https://github.com/facebookresearch/faiss
28. Douze, M. et al. "The Faiss library." Meta FAIR, 2024. https://arxiv.org/abs/2401.08281
29. Johnson, J., Douze, M., Jégou, H. "Billion-scale similarity search with GPUs." Facebook AI Research, 2017 (IEEE Transactions on Big Data, 2019). https://arxiv.org/abs/1702.08734
30. Google Research. "ScaNN: Scalable Nearest Neighbors" README. Accessed 2026. https://github.com/google-research/google-research/tree/master/scann
31. Google Research. "ScaNN algorithms and configuration" (docs/algorithms.md). Accessed 2026. https://github.com/google-research/google-research/blob/master/scann/docs/algorithms.md
32. Guo, R., Sun, P., Lindgren, E., Geng, Q., Simcha, D., Chern, F., Kumar, S. "Accelerating Large-Scale Inference with Anisotropic Vector Quantization." Google, ICML 2020. https://arxiv.org/abs/1908.10396
33. Sun, P. et al. "SOAR: Improved Indexing for Approximate Nearest Neighbor Search." Google, NeurIPS 2023. https://arxiv.org/abs/2404.00774
34. Feast project. "Feast: Feature Store for Machine Learning" GitHub repository. Accessed 2026. https://github.com/feast-dev/feast
35. Anyscale / Ray project. "Ray" GitHub repository. Accessed 2026. https://github.com/ray-project/ray
36. Netflix / Outerbounds. "Metaflow" GitHub repository. Accessed 2026. https://github.com/Netflix/metaflow
37. Meta (Facebook Research). "DLRM: Deep Learning Recommendation Model" GitHub repository. Accessed 2026. https://github.com/facebookresearch/dlrm ; Mudigere, D. et al., "Software-Hardware Co-design for Fast and Scalable Training of Deep Learning Recommendation Models," ISCA 2022: https://arxiv.org/abs/2104.05158

38. Naumov, M. et al. "Deep Learning Recommendation Model for Personalization and Recommendation Systems." Facebook, 2019. https://arxiv.org/abs/1906.00091
39. Harper, F. M., Konstan, J. A. "The MovieLens Datasets: History and Context." GroupLens, ACM TiiS, 2015. https://doi.org/10.1145/2827872
40. reczoo. "Criteo_x1" dataset card. Accessed 2026. https://github.com/reczoo/Datasets/tree/main/Criteo/Criteo_x1
41. reczoo. "Avazu_x1" dataset card. Accessed 2026. https://github.com/reczoo/Datasets/tree/main/Avazu/Avazu_x1
42. reczoo. "Datasets for recommender systems benchmarking." Accessed 2026. https://github.com/reczoo/Datasets
43. Zhu, J., Dai, Q., Su, L., Ma, R., Liu, J., Cai, G., Xiao, X., Zhang, R. "BARS: Towards Open Benchmarking for Recommender Systems." Huawei Noah's Ark et al., SIGIR 2022. https://github.com/reczoo/BARS ; https://arxiv.org/abs/2205.09626
44. reczoo. "FuxiCTR: A configurable, tunable, and reproducible library for CTR prediction." Accessed 2026. https://github.com/reczoo/FuxiCTR
45. Hou, Y., Li, J., He, Z., Yan, A., Chen, X., McAuley, J. "Bridging Language and Items for Retrieval and Recommendation" (BLaIR; Amazon Reviews 2023). UCSD, 2024. https://github.com/hyp1231/AmazonReviews2023 ; https://arxiv.org/abs/2403.03952
46. Gao, C. et al. "KuaiRand: An Unbiased Sequential Recommendation Dataset with Randomly Exposed Videos." Kuaishou / USTC, CIKM 2022. https://github.com/chongminggao/KuaiRand ; https://arxiv.org/abs/2208.08696
47. Microsoft. "MIND: MIcrosoft News Dataset" homepage and introduction document. 2020. https://msnews.github.io/ ; https://github.com/msnews/msnews.github.io/blob/master/assets/doc/introduction.md
48. Wu, F. et al. "MIND: A Large-scale Dataset for News Recommendation." Microsoft, ACL 2020. https://aclanthology.org/2020.acl-main.331/
49. Yandex. "Yambda-5B: A Large-Scale Multi-modal Dataset for Ranking and Retrieval" (dataset card and paper). 2025. https://huggingface.co/datasets/yandex/yambda ; https://arxiv.org/abs/2505.22238

35. The Recommender System Design Interview Playbook: Feeds, PYMK, Ads, E-commerce and Notifications

> Research note for this chapter: the shared web-search quota for this run was used up before this chapter started, and most blog hosts (eugeneyan.com, huyenchip.com, engineering.fb.com, research.google) did not resolve from the research sandbox. The sources read directly for this chapter are GitHub-hosted: the ByteByteGo "Machine Learning System Design Interview" reference repository [6], Chip Huyen's booklet and book repositories [3][4], Eugene Yan's design-doc template and applied-ml list [13][56], the community interview guide by Alireza Dirafzoon and contributors [7]-[12], and the open-sourced X/Twitter code [22][23]. Production facts (funnel sizes, parameter counts, latencies) are taken from the company-specific chapters of this Atlas (Chapters 1, 2, 16, 17, 19, 21, 23, 24, 32), whose authors read the primary sources. Those sources are cited directly here. All back-of-envelope figures are this chapter's own arithmetic, labelled as illustrative, and are not claims about any company.

Overview

The "design a recommender" question is the most common ML system design interview prompt. The ByteByteGo / Aminian-and-Xu book *Machine Learning System Design Interview* gives six of its ten case studies to recommendation-shaped problems: video recommendation, event recommendation, ad click prediction on social platforms, similar listings on vacation-rental platforms, personalized news feed and People You May Know [6]. The widely used open-source guide by Dirafzoon et al. lists the same families (video/movie recs, PYMK, newsfeed ranking, ads serving and click prediction) as the core of its question bank [7]. Chip Huyen's ML systems design booklet ends with 27 open-ended design questions and organizes the answer around project setup, data pipeline, modeling and serving [3].

This chapter turns those resources and the production designs documented elsewhere in this Atlas into one playbook. It covers:

1. A reusable answer framework, with the industry references behind each step.
2. Back-of-envelope arithmetic for QPS, candidate counts, embedding memory and logs.
3. Six worked designs: YouTube/TikTok video feed, social news feed, People You May Know, ad click prediction, e-commerce recommendations, and notifications.
4. A comparison table, the follow-up questions interviewers ask most often, and the answers production systems give.

The guiding observation comes from Eugene Yan's survey of discovery systems: nearly every industrial recommender and search system fits a 2x2 grid of offline versus online and

candidate retrieval versus ranking [1][2]. A good interview answer is a tailored instance of that grid, with numbers.

35.1 The answer framework

35.1.1 Nine steps, mapped to time

The Dirafzoon guide breaks an ML design answer into nine steps [7]. Yan's design-doc template [13] and Huyen's four-stage booklet [3] cover the same ground in different words. The table below aligns them and suggests a 45-minute time split (the timings are this chapter's suggestion, not from a source).

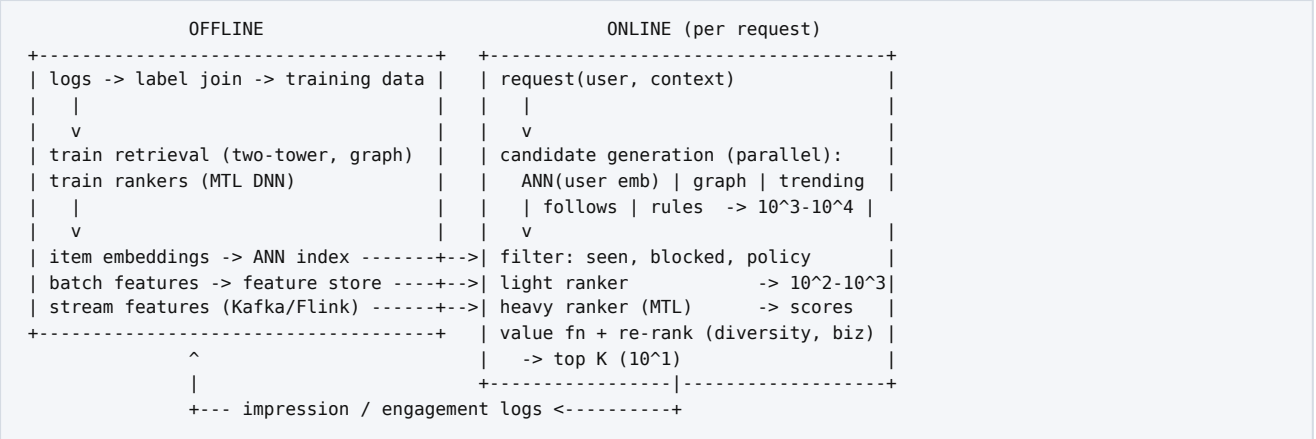
Step	Guide [7]	Design doc [13]	Huyen [3]	Minutes
1	Problem formulation, clarify scope	Overview, motivation, requirements	Project setup	5
2	Metrics (offline and online)	Success metrics, guardrails	Project setup	4
3	Architecture (MVP)	High-level design	Serving	6
4	Data and labels	Methodology: data	Data pipeline	5
5	Features	Methodology: techniques	Data pipeline	5
6	Model development, offline eval	Techniques, validation	Modeling	8
7	Prediction service	Performance, infra	Serving	5
8	Online testing, deployment	Experimentation and validation	Serving	3
9	Scaling, monitoring, updates	Monitoring, risks	Serving	4

Practical points from the sources:

- **Translate the business goal into an ML objective.** The guide's examples: input user_id, output a ranked list; "relevance" must be defined, for example a like or playing half of the video [8]. Huyen notes that recommender labels are *natural* (the user's reaction) but delayed, and the delay ("feedback loop length") drives how fast you can retrain [4].
- **Start simple and iterate**: heuristics, then logistic regression, then GBDT, then neural networks [7]. This mirrors Google's Rules of ML, which tells teams to launch a simple model and get the pipeline right first [42].
- **Batch, online or hybrid prediction** is an explicit decision, with Netflix's mix of batch-computed titles and online-composed rows as the stock example [7]. Huyen frames the trade-off as responsiveness to changing preferences versus inference latency [4].
- **Write down what you are not doing.** Yan's template has an explicit scope section and an "alternatives considered" appendix [13]. Saying "I'm deferring ads blending and creator-side fairness" is a strong signal in an interview.

35.1.2 The canonical architecture to draw first

Every case in this chapter starts from the same skeleton. Production systems (YouTube [14], Instagram [17][18], Facebook Feed [20], X [21]) differ in the number of stages and the model at each stage, not in the shape.



Funnel sizes from public systems are a useful anchor. Google's course describes candidate generation that "reduces billions of videos down to hundreds or thousands" [43]. Instagram Explore retrieves up to 1,500 media, keeps the top 100 after early-stage ranking, then runs late-stage ranking [19]. Facebook Feed starts from more than 1,000 candidate posts per visit and prunes to about 500 before heavy ranking [20]. X's 2023 pipeline draws about 1,500 candidates (approximate) from about 500 million daily tweets [21]. Chapter 1 summarises the pattern: retrieval leaves 10^3-10^4 candidates, pre-ranking 10^2-10^3, and ranking 10^1-10^2.

35.2 Back-of-envelope numbers

Interviewers want sizing that drives decisions, not precision. The numbers below are illustrative arithmetic, with the assumptions shown.

35.2.1 Traffic

- Seconds per day: about 86,400, so 1 billion requests per day is about 11.6K QPS on average. Use 2-3x for peak (an assumption, not a sourced constant).
- The Dirafzoon newsfeed case assumes 2B DAU, each refreshing twice a day, with a 200 ms latency target [9]. That is 4B feed requests per day, or about 46K QPS average and roughly 100K+ at peak.
- The video case assumes 100M DAU and 100M videos [8]. At 10 feed loads per user per day that is 1B requests per day, about 12K QPS.
- X's 2023 blog reports its For You pipeline runs about 5 billion times a day and completes in under 1.5 s on average [21]. That is about 58K pipeline runs per second on average.

35.2.2 Scoring load

- Heavy-ranker load = QPS x candidates per request. At 12K QPS and 500 candidates per request, that is 6M item scorings per second. This explains why heavy rankers see only hundreds of items and why a light ranker exists.
- Instagram reported in 2019 that Explore "extracts 65 billion features and makes 90 million model predictions every second" [17]. That is a real anchor for "how big can this get."

- X's 2023 heavy ranker was a neural network of about 48M parameters using about 6,000 features and predicting about 10 engagement labels [21][22]. A single pipeline execution consumed about 220 s of CPU time [21], so feature hydration and model inference, not ANN search, dominate cost.

35.2.3 Memory

- Item embeddings: 100M items x 128 dims x 4 bytes (fp32) is about 51 GB. fp16 halves it to about 26 GB and int8 to about 13 GB. A single large host can hold that index in memory, but a sharded ANN service is needed once you add replicas and freshness.
- User embeddings computed on request (two-tower user tower) need no storage. Precomputed user vectors for 1B users at 128 dims fp16 come to about 256 GB, which belongs in a key-value store.
- Sparse ID embedding tables dominate ranking-model size. Google's 2013 ads system found memory, not FLOPs, was the binding constraint. It used Poisson and Bloom-filter feature admission and 16-bit weights to save roughly 40-75% of RAM at negligible AucLoss [24].

35.2.4 Logs and training data

- Impressions per day = requests x items shown. 1B requests x 20 items gives 20B impressions per day. At about 1 KB per logged impression with features, that is about 20 TB per day.
- Negative downsampling is standard. Facebook's ads paper keeps negatives at rate w and recalibrates with $q = p / (p + (1 - p) / w)$ [25].
- YouTube's 2016 ranker trained on hundreds of billions of examples with roughly one billion parameters (approximate figures from the paper and summaries) [14].

35.2.5 Latency budget

A common split of a 200-300 ms p99 budget (a heuristic from Chapter 10, not one company's numbers):

Stage	Share of p99	Dominant cost
User/query hydration	5-15%	KV lookups, user sequence fetch
Candidate generation (parallel)	15-25%	ANN, slowest source
Feature hydration	20-35%	fan-out RPCs, tail latency
Light ranking	5-10%	CPU, small model, many items
Heavy ranking	20-35%	GPU compute, batching delay
Re-rank, filters, mixing	5-10%	rules, diversity, ad auction

Uber Eats found that hydration was the real hotspot. Retrieval returned tens of thousands of candidates that were fully hydrated before ranking discarded most of them. Removing low-yield retrieval paths saved about 120 ms [50]. This makes a good interview point: candidate *count* is a latency lever, not just a recall lever.

35.3 Design 1: YouTube / TikTok video recommendation feed

Problem and clarifying questions

Homepage or For You feed recommendations, personalised per user, worldwide and multilingual, aimed at engagement [8]. Ask about: home feed versus watch-next, long-form versus short-form, whether search history is available, the latency target (the guide suggests about 200 ms), and whether creator-side goals matter [8].

Metrics

- Offline: precision@k, mAP and diversity for the list. Recall@k for retrieval. AUC or log loss per head for the ranker [8].
- Online: CTR, completed videos, total watch time, explicit feedback [8]. Guardrails: dislikes, "not interested", reports, session abandonment.
- Long-term: Google found short-term ad experiments mislead without long-term holdouts [47]. Netflix similarly writes about recommending for long-term member satisfaction rather than immediate engagement [55]. Mentioning user-level retention or satisfaction surveys sets a senior answer apart.

Architecture

```
user_id + context
     |
     v
+------------ CANDIDATE GENERATION (parallel, ~1-2K total) ------------+
| two-tower ANN (recent watches) | co-watch / item-to-item | subs      |
| trending / fresh (explore)     | search-history sources  | multi-int |
+-----------------------------------------------------------------------+
     |
     v   filter: watched, blocked, region/age policy, dedupe
light ranker (distilled two-tower or small DNN)  ->  ~100-500
     |
     v
heavy ranker: multi-task DNN (MMoE) -> p(click), E[watch time], p(like), p(share),
              p(dislike), p(survey satisfied); shallow position-bias tower
     |
     v
value function  sum_i w_i * pred_i  ->  re-rank: diversity (DPP / MMR), freshness,
                                        creator caps, policy  ->  top ~10-20
```

Models and the key tricks

- **Candidate generation as extreme multiclass classification.** YouTube 2016 averages embeddings of watched videos and search tokens, feeds them into an MLP, and trains a sampled softmax over the corpus. At serving time it uses nearest-neighbour search on the softmax output vectors [14]. Two refinements matter in interviews. The **example age** feature lets the model learn a freshness preference. Training uses watches from everywhere (not just recommendations) with a fixed number of examples per user, so heavy users do not dominate [14].
- **Two-tower retrieval with in-batch negatives and logQ correction.** In-batch softmax over-penalises popular items. Subtracting log p_j (estimated with streaming

frequency counts) corrects the bias. Google deployed this on a YouTube corpus of tens of millions of videos [15].

- **Ranking on watch time, not clicks.** YouTube trained weighted logistic regression with positives weighted by watch time, so the odds approximate expected watch time [14]. This answers "how do you avoid clickbait?"
- **Multi-task ranking with position debiasing.** The 2019 Watch Next ranker uses MMoE for engagement and satisfaction objectives. A shallow tower takes position and device features and adds its logit, which absorbs selection bias. At serving time position is set to missing [16].
- **Diversity.** Google's course lists freshness, diversity and fairness as re-ranking goals [38]. YouTube re-ranks with a determinantal point process over quality scores and embedding distances [48].
- **Beyond supervised imitation.** A REINFORCE candidate generator with top-K off-policy correction was one of YouTube's largest single launches in its period, with roughly 0.8-0.9% more viewing time (approximate) [49].
- **Short-video specifics (TikTok, Kuaishou).** Signals arrive in seconds and interests shift within a session, so real-time training matters. ByteDance's Monolith uses collisionless, expirable hash embedding tables and minute-level parameter sync from online training to serving [44]. Kuaishou's OneRec goes further and replaces the retrieve-then-rank cascade with one generative model [45]. In an interview, treat it as a "what's next" follow-up, not the baseline.

Cold start and exploration

New users get profile and context features through the user tower. New videos get a small, randomised exposure budget, then fine-tuning on the collected data, with bandits for explore/exploit [8]. Content embeddings (title, thumbnail, audio) let the item tower place a new video near similar ones before any interaction data exists.

Lessons

Optimise watch time or satisfaction rather than clicks. Correct sampling and position bias explicitly. Keep the item corpus fresh through example age, streaming updates and exploration slots. Most of the gain sits in retrieval coverage plus ranker objectives, not in model depth.

35.4 Design 2: Social news feed (Facebook, LinkedIn, X)

Problem

Show unseen recent posts from connections and (increasingly) recommended accounts, ranked by predicted value, within about 200 ms. The guide's sizing is 2B DAU, two refreshes a day each, with privacy constraints [9].

Architecture: Facebook's four passes

Meta's 2021 engineering post describes Feed ranking as passes run on serving machines it calls *predictors* [20]:

```
inventory: all eligible posts for viewer (> 1,000 on average)
  -> pass 0: integrity + eligibility filters
  -> pass 1: lightweight model -> ~500 posts
  -> pass 2: multitask NNs predict P(like), P(comment), P(share), P(hide),
             P("worth your time" survey) ... -> one value score per post
  -> pass 3: contextual re-ranking (content-type diversity, no runs of similar posts)
```

The value function is a weighted sum, value(post) = sum_i w_i * P(event_i). The weights are a product decision informed by surveys about what people find meaningful, not a learned output [20]. The guide's interview version ranks weights as share > comment > like > click and adds heads for dwell time and P(skip) to serve passive users [9].

X's 2023 open-sourced pipeline

Twitter had moved timeline ranking to deep learning by 2017 [53]. Its 2023 Home Mixer is the most detailed public feed implementation [21][22]:

- Candidate sourcing: about 1,500 candidates (approximate), roughly half in-network (Earlybird search index, scored by a light ranker) and half out-of-network (social graph walks via UTEG/GraphJet, SimClusters community embeddings, TwHIN embeddings) [21].
- Heavy ranker: about 48M-parameter multi-task network (MaskNet), about 6,000 features, about 10 engagement labels, combined with configured weights [21][22].
- Heuristics and filters: author diversity, content balance between in- and out-of-network, feedback fatigue, visibility filtering [21].
- Mixing: ads, who-to-follow modules and prompts are inserted into the timeline [22].
- Resilience: Home Mixer keeps a reverse-chronological backup pipeline in case scored tweets fail (Chapter 10, [22]).

Key design decisions to discuss

1. **Fan-out on write versus fan-out on read** for candidate inventory. Precomputed per-user inboxes make reads cheap but are expensive for celebrity authors. Pull-time retrieval from an index (Earlybird) handles high-fanout authors. A hybrid is usual. (This is the classic systems-design half of the question.)
2. **Separate prediction from policy.** Calibrated multi-task heads plus a small, versioned weight vector let product change trade-offs without retraining [20][21].
3. **Dwell time and passive users.** LinkedIn's dwell-time work is a standard reference for adding implicit signals beyond clicks [52]. The guide's skip head does the same [9].
4. **Integrity first.** Filter violating content before heavy ranking, to save compute and avoid amplifying it [20].
5. **Freshness.** Feature pipelines are split: static features computed in batch (daily or weekly) into a feature store, and dynamic features (recent clicks, post engagement velocity) streamed [9].

Follow-ups specific to feeds

Viral and celebrity posts, cold-start users, position bias, calibration, data leakage and catastrophic forgetting during continual fine-tuning are all listed as expected follow-ups [9]. Have one sentence ready for each.

35.5 Design 3: People You May Know (friend / follow / connection recommendation)

Problem

Recommend users to connect with, to maximise connections formed. The guide's sizing is 1B users with an average of 1,000 connections each (its text reads "10000", but its own candidate arithmetic of 1,000 x 1,000 = 1M friends-of-friends implies 1,000) [10]. The objective is two-sided: a suggestion is only good if the *other* person accepts.

Architecture

```
OFFLINE (daily / hourly)                          ONLINE
graph snapshot at t ----------------------+      request(user)
  -> FoF expansion (2-hop) ~10^5-10^6     |        -> read precomputed PYMK list
  -> + EBR: GNN embeddings -> ANN         |        -> filter: already connected,
  -> + co-attributes: school, company,    |           blocked, pending, privacy
       contacts, location                 |        -> light re-score with fresh
  -> ranker: p(send) * p(accept)          |           signals (recent views)
  -> top ~100 per user -> KV store -------+------> -> diversity, frequency caps
```

Candidate generation

- **Triangle closing / friends-of-friends** is the classic generator. It reduces 1B users to about 1M FoF candidates per user in the guide's arithmetic [10]. In practice you keep the top FoF by weighted mutual-friend counts.
- **Random walks.** Graph walks scale well: Pinterest's Pixie serves random-walk recommendations over 3+ billion items in real time [37]. X's Follow Recommendations Service uses SALSA-style personalised random walks with restarts over a local bipartite graph built from address book, mutual follows and recent follows. It ranks by a weighted sum of p(follow | rec) and p(positive engagement | follow) (Chapter 7, [22]).
- **Embedding-based retrieval beyond two hops.** Snap added GNN-embedding EBR for friend recommendation across billions of users. It reaches candidates that FoF misses, especially for new users in sparse markets, and improved friendships made in A/B tests [33]. A follow-up issuing several ANN queries per user (query expansion) gave an 11% relative gain in reciprocated-communication metrics at more than 800M MAU [34].

Models

- Pairwise classifier on (user_i, user_j) with features such as mutual connections (time-discounted), shared school or company, profile similarity and interaction history; or a GNN that produces node embeddings and scores edges by similarity [10].
- Training data: take a graph snapshot at time t and label pairs that connect by t+1 as positive [10]. This temporal split avoids leakage.
- LinkedIn reports its LiGNN graph embeddings feed PYMK among other surfaces, with about +0.1% weekly active users attributed to people recommendations [36].

Serving

PYMK is the best example of **batch prediction**. The friend graph changes slowly, so precompute ranked lists for active users and store them in a key-value store [10], with a light online re-score for freshness. Fresh signals (a just-accepted connection opens new triangles) can trigger incremental recomputation for that user.

Follow-ups

- **Delayed feedback**: invitations are accepted days later, so labels need waiting windows or delayed-feedback correction [10][27].
- **Reciprocity and fairness**: optimise p(send) x p(accept), and avoid rich-get-richer. LinkedIn has written about optimising PYMK for equity in network creation [35].
- **Privacy**: do not reveal sensitive edges, for example contact-book-derived suggestions [10].

35.6 Design 4: Ad click prediction (pCTR) and ads ranking

Problem

Predict p(click | user, ad, context) for ads shown on a social feed, to maximise revenue while protecting user experience [11]. It is pointwise learning-to-rank over <user, ad> pairs. A click within t seconds of the impression is a positive [11].

What makes ads different

1. **The output feeds an auction**, so predictions must be **calibrated probabilities**, not just well ordered. Ranking typically uses expected value, bid x pCTR (x pCVR for conversion objectives) plus quality terms. Every ads paper since 2013 treats calibration as a first-class output [24][25].
2. **Metrics**: normalized entropy (NE), which is log loss divided by the entropy of the background CTR, plus calibration ratio (sum predicted / sum observed) per slice [24] [25]. The guide lists cross-entropy and normalized cross-entropy offline, and CTR, conversion rate, revenue lift and hide rate online [11].
3. **Freshness is worth money.** Facebook found NE degrades as training data falls days behind serving [25]. This motivates online or continual training.

Architecture

```
ad request (user, placement)
  -> eligibility: targeting (age, geo, interests), budget/pacing, policy
  -> retrieval: tens of millions of eligible ads -> a few thousand      (Andromeda-like)
  -> light ranker -> hundreds
  -> heavy ranker: pCTR, pCVR, p(hide) heads; calibration layer
  -> auction: rank by bid * pCTR * pCVR + quality; price (GSP/VCG-style)
  -> blend with organic feed (ad load, spacing)
logs -> online joiner (impression + click within window) -> streaming trainer
```

Secondary coverage of Meta's Andromeda post reports that ads retrieval narrows tens of millions of eligible ads to a few thousand candidates per impression before ranking and auction [28]. In 2025 Meta described GEM, an LLM-scale ads foundation model used to transfer knowledge into the served models rather than to score every candidate inline [29].

Models: the expected progression

The guide recommends starting with logistic regression and moving to DCN and DeepFM [11]. The ByteByteGo chapter's references cover the same path: GBDT feature extraction, DCN and DCN-V2, Deep Crossing, FM and DeepFM, FFM, and delayed-feedback correction [6].

Model	Why it appears	Weakness raised in interviews
LR + hand crosses (FTRL) [24]	sparse, fast, online-updatable, calibrated	manual feature crosses
GBDT + LR [25]	trees as learned feature transform; >3% NE gain	trees retrain slowly; no embeddings
FM / FFM [6]	pairwise interactions on sparse IDs	only second order
Wide & Deep [54], DeepFM [6]	memorisation + generalisation	heavier serving
DCN-V2 [26]	explicit learned crosses, low-rank for cost	tuning cross depth
DIN-style target attention [31]	user-history attention to candidate ad	sequence fetch latency
Foundation model + distillation [29]	scale beyond per-request budget	infra cost, transfer design

GBDT is interpretable but inefficient for continual training and cannot learn embeddings. Plain DNNs struggle with very sparse features without explicit interaction structure [11].

Key tricks

- **Online joiner and label windows.** Join impressions and clicks by request ID inside a waiting window. Too short mislabels late clicks as negatives, too long delays training [25]. Twitter's delayed-feedback work trains continuously with fake negatives and corrects with an importance-weighted or FN-calibrated loss [27].
- **Negative downsampling with recalibration** (formula in 35.2.4) [25].
- **Memory tricks** for huge sparse feature spaces: probabilistic feature admission and low-precision weights [24].
- **Conversion prediction (pCVR)** suffers sample-selection bias because conversions are only observed after clicks. ESMM models p(click) x p(conversion | click) over the entire impression space [32].

Follow-ups

Calibration after downsampling; leakage from post-impression features; catastrophic forgetting under continual training; ad fatigue and negative feedback (hide/block); budget pacing; cold-start ads (use advertiser and campaign-level priors and content embeddings) [11].

35.7 Design 5: E-commerce recommendations

Problem variants

E-commerce interviews usually narrow to one surface:

- "Customers who viewed/bought X also bought" (item-to-item, product page).
- Personalised homepage ("recommended for you").
- Complementary products / cart add-ons.
- Similar listings (Airbnb-style), which ByteByteGo treats as its own chapter [6].

Business metrics are conversion rate, GMV or revenue per session and add-to-cart rate. CTR alone is a weak proxy.

Item-to-item collaborative filtering (the baseline that still wins)

Amazon's 2003 design computes item-item cosine similarity offline over customer purchase vectors. At request time it only looks up similar items for the user's recent items, so online cost depends on the size of the user's history, not on catalogue or customer count [30]. Worst-case offline cost is O(N^2 M), but sparsity makes it close to O(NM) in practice (Chapter 24).

```
OFFLINE: orders/views -> item-item co-occurrence/cosine -> similar-items table (KV)
ONLINE : user history (viewed, carted, bought) -> lookup neighbours -> merge, dedupe,
         remove purchased -> rank (personal DNN, price, availability) -> slate
```

Session embeddings and similar listings

Airbnb trained listing embeddings word2vec-style on click sessions. It used booked listings as global context and in-market negatives, and applied them to similar-listing recommendations and real-time search personalisation [39]. This is ByteByteGo's reference design for the similar-listings chapter [6]. Interview points: sessions as "sentences", in-market negative sampling so the model learns within-city distinctions, and the cold-start rule of averaging embeddings of nearby similar listings.

Personalised ranking

- **Target attention over behaviour sequences.** Alibaba's DIN weights each past behaviour by relevance to the candidate item instead of averaging history [31]. This is the standard answer to "users have diverse interests."
- **Multi-objective**: p(click), p(add-to-cart), p(purchase), with ESMM-style modelling to handle the click-then-buy funnel [32].
- **Business re-ranking**: in-stock and deliverable filters, price-band diversity, sponsored-product slots, and suppressing items already bought (except consumables).

Follow-ups

Substitutes versus complements (do not recommend a second TV after a TV purchase); cold start for new SKUs (content embeddings from title, image and attributes); seasonality; popularity bias; and real-time session intent versus long-term profile.

35.8 Design 6: Notification and push recommendation

Problem

Decide **whether**, **what**, **when** and **how often** to notify a user (Chapter 32). The user is not present, so there is no request. The system is event-driven (a friend posted) or schedule-driven (a digest). Every send has a hidden cost: a bad push can lead to disabled notifications or an uninstall, which are near-permanent channel losses.

Architecture (from X's open-sourced pushservice)

X's `pushservice` README describes a RefreshForPushHandler flow [23]:

```
user id -> Target (eligibility, settings) -> query candidate sources
  -> hydrate candidates -> light filters
  -> LIGHT RANKER (pre-selects from the large pool)
  -> HEAVY RANKER (multi-task: p(open), p(engage), negative-feedback risk)
  -> take step: run expensive predicates (quality, fatigue) one candidate at a time
     until ONE passes -> send push + in-app notification
```

The "take" step is the design point worth copying. Expensive checks, such as quality predicates and fatigue predicates, run lazily on the ranked list until one candidate passes, which bounds cost per user (Chapter 32, [23]).

Models and policy

- **Ranker**: multi-task p(open), p(downstream session), p(disable or unsubscribe).
- **Send policy**: send if value(candidate) - cost(fatigue, disable risk) > threshold. The threshold can be per user. Pinterest's notification volume control and LinkedIn's near-real-time activity-notification optimisation are the standard references [40][41] (DOIs not re-verified in this run; see Chapter 32).
- **Volume and timing**: per-user daily budgets, quiet hours, time-zone-aware send-time optimisation, and dedup across channels (push, email, in-app).

Metrics and experimentation

Optimise long-term metrics (DAU/WAU, sessions over weeks) with disables and unsubscribes as guardrails. Notification effects are heavily confounded, because active users get more notifications, so randomised volume holdouts are required (Chapter 32). Short-term open rate is a trap, in the same way short-term ad CTR misleads [47].

35.9 Cross-design comparison

Design	Primary objective	Candidate gen	Ranker	Serving mode	Main pitfall
Video feed [8][14][16]	watch time, satisfaction	two-tower ANN, co-watch, fresh	MMoE MTL + position tower	online, real-time features	clickbait, popularity bias
News feed [9][20][21]	weighted engagement value	in-network index + OON graph/emb	MTL heads + value fn	online, fan-out hybrid	engagement-only weights
PYMK [10][33]	connections formed (both sides)	FoF, random walks, GNN EBR	pairwise / GNN edge score	batch precompute + refresh	delayed accept, rich-get-richer
Ad click [11][24][25]	revenue (calibrated pCTR)	targeting + ANN ad retrieval	LR/GBDT+LR -> DCN/DIN	online, continual	miscalibration, label delay

Design	Primary objective	Candidate gen	Ranker	Serving mode	Main pitfall
				training	
E-commerce [30][31][39]	conversion, GMV	item-to-item, session emb	DIN-style, multi-objective	hybrid: batch tables + online	substitutes vs complements
Notifications [23]	long-term DAU minus disables	event/schedule sources	light + heavy MTL + policy	async batch/stream	fatigue, confounding
Events [12]	registrations / tickets	rules (location, time)	GBDT -> NN	online	perpetual cold start

35.10 Common follow-up questions and strong answers

Follow-up	Strong answer (with source)
How do you handle cold-start items?	Content features in the item tower; exploration budget; bandits [8]; example-age feature [14]
How do you handle position bias?	Shallow position tower added to logits, position missing at serving [16]; randomised slots; IPS weighting [46]
Your two-tower favours popular items. Why?	In-batch negatives oversample popular items; apply logQ correction [15]; mix in random negatives
How do you combine multiple objectives?	Calibrated MTL heads + configured weights [20][21]; tune weights online
Model is stale; what now?	Measure NE vs data age [25]; streaming features; continual or online training [44]; Huyen's continual learning [5]
How do you prevent feedback loops?	Exploration traffic, off-policy correction [49], diversity re-ranking [48]; Huyen names degenerate feedback loops as a system design issue [4]
Offline gain, online loss?	Check train-serving skew and leakage [9][11]; log features at serving time [1]; use A/B tests with guardrails
How do you ensure diversity?	DPP or MMR re-rank [48]; author/creator caps [21]
Labels arrive late (conversions, accepts)?	Waiting windows, fake-negative correction [27]
Scale to 10x traffic?	Cut candidates (hydration dominates) [50]; distil heavy ranker; cache user embeddings; quantise ANN index
What would you do with LLMs?	Content embeddings and semantic IDs for cold start; generative retrieval and one-model ranking as the frontier [45][51]

35.11 Anti-patterns interviewers flag

- **Jumping to a model before defining the label.** "Relevance" must be operationalised: watched more than half, registered, accepted [8][10][12].
- **One-stage design for a large corpus.** Scoring 100M items with a heavy model per request is infeasible. Derive the funnel from the latency budget [43].
- **Optimising CTR only.** Every production source above moves to watch time, satisfaction surveys or weighted value functions [14][16][20].
- **Random train/test splits on time-ordered data.** Use time-based splits [7][10].
- **Ignoring calibration** in ads or in any value function that sums probabilities [24][25].

- **Ignoring the data pipeline.** Yan's 2x2 grid puts as much weight on offline data, embedding and index jobs as on the online path [1].

Key takeaways for system designers

1. **Draw the funnel first, then size it.** Corpus size and latency budget set the number of stages. Public systems cluster at about 1,000-1,500 candidates entering ranking (Instagram [19], X [21], Facebook [20]).
2. **Do the arithmetic out loud.** Requests per day / 86,400 = QPS; QPS x candidates = scoring load; items x dims x bytes = index memory; impressions x bytes = log volume. Each number should justify a design choice.
3. **Separate prediction from policy.** Calibrated multi-task heads plus a configured value function is the dominant production pattern for feeds, ads and notifications [20][21][23].
4. **Choose serving mode per problem.** PYMK and item-to-item tables are batch-friendly [10][30]. Feeds and ads need online scoring with streaming features [9][25]. Notifications are asynchronous decision problems [23].
5. **Name the biases and fix them.** Sampling bias (logQ) [15], position bias (shallow tower) [16], selection bias in conversions (ESMM) [32] and feedback loops (exploration, off-policy correction) [49].
6. **Plan for freshness and delayed labels.** Online joiners, continual training and delayed-feedback correction are core, not extras [25][27][44].
7. **Measure the long term.** Use holdouts and long-term metrics, because short-term CTR and open-rate wins can hurt retention [47][55].
8. **Know the frontier but lead with the baseline.** Generative recommenders (HSTU, OneRec) are strong follow-up material [45][51]. The interview answer should still start from retrieval + ranking + re-ranking.

References

1. Eugene Yan. "System Design for Recommendations and Search." eugeneyan.com, 2021. https://eugeneyan.com/writing/system-design-for-discovery/
2. Eugene Yan. "System Design for Discovery (RecSys and Search)." ApplyingML. https://applyingml.com/resources/discovery-system-design/
3. Chip Huyen. "Machine Learning Systems Design" (booklet source; four steps: project setup, data pipeline, modeling, serving; 27 open-ended questions). GitHub, 2019-2020. https://github.com/chiphuyen/machine-learning-systems-design
4. Chip Huyen. "Designing Machine Learning Systems" (book repository, chapter summary). O'Reilly / GitHub, 2022. https://github.com/chiphuyen/dmls-book
5. Chip Huyen. "Real-time machine learning: challenges and solutions." huyenchip.com, 2022 (cited from [6]). https://huyenchip.com/2022/01/02/real-time-machine-learning-challenges-and-solutions.html
6. ByteByteGo (Ali Aminian and Alex Xu). "Machine Learning System Design Interview" chapter reference list. GitHub, ByteByteGoHq/ml-bytebytego, 2023. https://github.com/ByteByteGoHq/ml-bytebytego

7. Alireza Dirafzoon et al. "ML System Design" (9-step framework and question bank), Machine-Learning-Interviews. GitHub. https://github.com/alirezadir/Machine-Learning-Interviews/blob/main/src/MLSD/ml-system-design.md
8. Alireza Dirafzoon et al. "Video Recommendation System" (mlsd-video-recom). GitHub. https://github.com/alirezadir/Machine-Learning-Interviews/blob/main/src/MLSD/mlsd-video-recom.md
9. Alireza Dirafzoon et al. "Newsfeed ranking" (mlsd-newsfeed). GitHub. https://github.com/alirezadir/Machine-Learning-Interviews/blob/main/src/MLSD/mlsd-newsfeed.md
10. Alireza Dirafzoon et al. "People You May Know" (mlsd-pymk). GitHub. https://github.com/alirezadir/Machine-Learning-Interviews/blob/main/src/MLSD/mlsd-pymk.md
11. Alireza Dirafzoon et al. "Ads click prediction / ads ranking" (mlsd-ads-ranking). GitHub. https://github.com/alirezadir/Machine-Learning-Interviews/blob/main/src/MLSD/mlsd-ads-ranking.md
12. Alireza Dirafzoon et al. "Event Recommendation System" (mlsd-event-recom). GitHub. https://github.com/alirezadir/Machine-Learning-Interviews/blob/main/src/MLSD/mlsd-event-recom.md
13. Eugene Yan. "ml-design-docs: A template for ML design docs." GitHub. https://github.com/eugeneyan/ml-design-docs
14. Paul Covington, Jay Adams, Emre Sargin (Google). "Deep Neural Networks for YouTube Recommendations." RecSys 2016. https://research.google/pubs/deep-neural-networks-for-youtube-recommendations/
15. Xinyang Yi et al. (Google). "Sampling-Bias-Corrected Neural Modeling for Large Corpus Item Recommendations." RecSys 2019. https://research.google/pubs/sampling-bias-corrected-neural-modeling-for-large-corpus-item-recommendations/
16. Zhe Zhao et al. (Google). "Recommending What Video to Watch Next: A Multitask Ranking System." RecSys 2019. https://dl.acm.org/doi/10.1145/3298689.3346997
17. Ivan Medvedev, Haotian Wu, Taylor Gordon (Instagram). "Powered by AI: Instagram's Explore recommender system." Instagram Engineering, 2019. https://instagram-engineering.com/powered-by-ai-instagrams-explore-recommender-system-7ca901d2a882
18. Vladislav Vorotilov, Ilnur Shugaepov (Meta). "Scaling the Instagram Explore recommendations system." Engineering at Meta, 2023. https://engineering.fb.com/2023/08/09/ml-applications/scaling-instagram-explore-recommendations-system/
19. Meta Transparency Center. "Instagram Explore AI system." Accessed 2026. https://transparency.meta.com/features/explaining-ranking/ig-explore/
20. Akos Lada, Meihong Wang, Tak Yan (Meta). "How machine learning powers Facebook's News Feed ranking algorithm." Engineering at Meta, 2021. https://engineering.fb.com/2021/01/26/ml-applications/news-feed-ranking/
21. Twitter Engineering. "Twitter's Recommendation Algorithm." Twitter (X) Engineering Blog, 2023. https://blog.x.com/engineering/en_us/topics/open-source/2023/twitter-recommendation-algorithm
22. Twitter. "the-algorithm" repository (home-mixer, follow-recommendations-service READMEs). GitHub, 2023. https://github.com/twitter/the-algorithm
23. Twitter. "pushservice" README (RefreshForPushHandler, light and heavy rankers). GitHub, 2023. https://github.com/twitter/the-algorithm/tree/main/pushservice
24. H. Brendan McMahan et al. (Google). "Ad Click Prediction: a View from the Trenches." KDD 2013. https://research.google/pubs/ad-click-prediction-a-view-from-the-trenches/

25. Xinran He et al. (Facebook). "Practical Lessons from Predicting Clicks on Ads at Facebook." ADKDD 2014. https://quinonero.net/Publications/predicting-clicks-facebook.pdf
26. Ruoxi Wang et al. (Google). "DCN V2: Improved Deep & Cross Network and Practical Lessons for Web-scale Learning to Rank Systems." WWW 2021. https://arxiv.org/abs/2008.13535
27. Sofia Ira Ktena et al. (Twitter). "Addressing Delayed Feedback for Continuous Training with Neural Networks in CTR Prediction." RecSys 2019. https://arxiv.org/abs/1907.06558
28. Meta Engineering. "Meta Andromeda: Supercharging Advantage+ automation with the next-gen personalized ads retrieval engine." 2024 (figures via secondary coverage, https://www.jonloomer.com/meta-andromeda/). https://engineering.fb.com/2024/12/02/production-engineering/meta-andromeda-advantage-automation-next-gen-personalized-ads-retrieval-engine/
29. Engineering at Meta. "Meta's Generative Ads Model (GEM): The central brain accelerating ads recommendation AI innovation." 2025. https://engineering.fb.com/2025/11/10/ml-applications/metas-generative-ads-model-gem-the-central-brain-accelerating-ads-recommendation-ai-innovation/
30. Greg Linden, Brent Smith, Jeremy York (Amazon). "Amazon.com Recommendations: Item-to-Item Collaborative Filtering." IEEE Internet Computing, 2003. https://www.cs.umd.edu/~samir/498/Amazon-Recommendations.pdf
31. Guorui Zhou et al. (Alibaba). "Deep Interest Network for Click-Through Rate Prediction." KDD 2018. https://arxiv.org/abs/1706.06978
32. Xiao Ma et al. (Alibaba). "Entire Space Multi-Task Model: An Effective Approach for Estimating Post-Click Conversion Rate." SIGIR 2018. https://arxiv.org/abs/1804.07931
33. Snap Research. "Embedding Based Retrieval in Friend Recommendation." SIGIR 2023. https://research.snap.com/publications/embedding-based-retrieval-in-friend-recommendation.html
34. Snap Research. "Improving Embedding-Based Retrieval in Friend Recommendation with ANN Query Expansion." https://research.snap.com/publications/improving-embedding-based-retrieval-in-friend-recommendation-with-ann-query-expansion.html
35. LinkedIn Engineering. "Optimizing PYMK for equity in network creation." LinkedIn Engineering Blog, 2021 (cited via [6]). https://engineering.linkedin.com/blog/2021/optimizing-pymk-for-equity-in-network-creation
36. Fedor Borisyuk et al. (LinkedIn). "LiGNN: Graph Neural Networks at LinkedIn." KDD 2024. https://arxiv.org/abs/2402.11139
37. Chantat Eksombatchai et al. (Pinterest). "Pixie: A System for Recommending 3+ Billion Items to 200+ Million Users in Real-Time." WWW 2018. https://arxiv.org/pdf/1711.07601
38. Google for Developers. "Re-ranking: freshness, diversity and fairness" (Recommendation Systems course, cited via [6]). https://developers.google.com/machine-learning/recommendation/dnn/re-ranking
39. Mihajlo Grbovic et al. (Airbnb). "Listing Embeddings in Search Ranking." Airbnb Engineering, 2018 (cited via [6]). https://medium.com/airbnb-engineering/listing-embeddings-for-similar-listing-recommendations-and-real-time-personalization-in-search-601172f7603e
40. Yan Gao et al. (LinkedIn). "Near Real-time Optimization of Activity-based Notifications." KDD 2018 (DOI not re-verified). https://dl.acm.org/doi/10.1145/321981

9.3219880
41. Bo Zhao et al. (Pinterest). "Notification Volume Control and Optimization System at Pinterest." KDD 2018 (DOI not re-verified). https://dl.acm.org/doi/10.1145/3219819.3219906
42. Martin Zinkevich (Google). "Rules of Machine Learning: Best Practices for ML Engineering." https://developers.google.com/machine-learning/guides/rules-of-ml
43. Google for Developers. "Recommendation systems overview." Machine Learning: Recommendation Systems course. https://developers.google.com/machine-learning/recommendation/overview/types
44. Zhuoran Liu et al. (ByteDance). "Monolith: Real Time Recommendation System With Collisionless Embedding Table." 2022. https://arxiv.org/abs/2209.07663
45. Kuaishou OneRec Team. "OneRec Technical Report." 2025. https://arxiv.org/abs/2506.13695
46. Eugene Yan. "How to Measure and Mitigate Position Bias." eugeneyan.com (cited via [6]). https://eugeneyan.com/writing/position-bias/
47. Henning Hohnhold, Deirdre O'Brien, Diane Tang (Google). "Focusing on the Long-term: It's Good for Users and Business." KDD 2015. https://dl.acm.org/doi/10.1145/2783258.2788583
48. Mark Wilhelm et al. (Google/YouTube). "Practical Diversified Recommendations on YouTube with Determinantal Point Processes." CIKM 2018. https://dl.acm.org/doi/10.1145/3269206.3272018
49. Minmin Chen et al. (Google/YouTube). "Top-K Off-Policy Correction for a REINFORCE Recommender System." WSDM 2019. https://arxiv.org/abs/1812.02353
50. Uber Engineering. "Uber Eats search pipeline" (retrieval, hydration and ranking latency). Uber Blog, 2026. https://www.uber.com/us/en/blog/uber-eats-search-pipeline/
51. Jiaqi Zhai et al. (Meta). "Actions Speak Louder than Words: Trillion-Parameter Sequential Transducers for Generative Recommendations." ICML 2024. https://arxiv.org/abs/2402.17152
52. LinkedIn Engineering. "Understanding dwell time to improve LinkedIn feed ranking." 2020 (cited via [6]). https://engineering.linkedin.com/blog/2020/understanding-feed-dwell-time
53. Twitter Engineering. "Using Deep Learning at Scale in Twitter's Timelines." 2017 (cited via [6]). https://blog.twitter.com/engineering/en_us/topics/insights/2017/using-deep-learning-at-scale-in-twitters-timelines
54. Heng-Tze Cheng et al. (Google). "Wide & Deep Learning for Recommender Systems." DLRS 2016. https://arxiv.org/abs/1606.07792
55. Netflix Technology Blog. "Recommending for Long-Term Member Satisfaction at Netflix." 2024. https://netflixtechblog.com/recommending-for-long-term-member-satisfaction-at-netflix-ac15cada49ef
56. Eugene Yan et al. "applied-ml: Papers and tech blogs by companies sharing their work on data science and machine learning in production." GitHub. https://github.com/eugeneyan/applied-ml

36. Classic Algorithms Foundations: Neighborhoods, Factorization, Linear Autoencoders, Embeddings, and Why Baselines Still Win

Overview

Every modern recommender stack, from two-tower retrieval to generative sequence models, sits on top of a small set of ideas that were mostly worked out between 1994 and 2019:

- **Neighborhood collaborative filtering (CF).** User-kNN, and later the item-to-item CF that Amazon made famous [1][2].
- **Latent-factor matrix factorization (MF).** Popularised by the Netflix Prize (Funk-SVD, SVD++, timeSVD++) [3][4], then adapted to implicit feedback with confidence-weighted ALS (Hu, Koren and Volinsky) [5]. It was later industrialised on Spark and Giraph [6][7].
- **Pairwise ranking objectives.** BPR [8], plus the WARP and BPR losses in hybrid factorization libraries such as LightFM [9].
- **Sparse and dense linear item-item models.** SLIM [10] and its closed-form descendant EASE [11].
- **Skip-gram item embeddings.** item2vec and prod2vec, later Airbnb's listing embeddings [12][13][14].
- **Content-based and hybrid models** [9][15].

These models matter for more than history. Between 2019 and 2022 a series of reproducibility studies showed a pattern [16][17][18][19][20]. Many neural recommenders published at top venues could not be reproduced. Of those that could, most were beaten by properly tuned item-kNN, graph random walks (P3alpha/RP3beta), SLIM, EASE or iALS. This chapter covers each classic family as a system design: what problem it solves, what it computes and how it scales. It then covers what the reproducibility crisis means for anyone building or judging a recommender in 2026.

A note on sourcing. Live web search was unavailable for most of this chapter's research session, and the paper hosts (arXiv, ACM, company blogs) could not be reached. The project repositories on GitHub were read directly: Dacrema et al. evaluation code [16], Google Research iALS and NCF-vs-MF code [18][19], LightFM [9], SLIM [10], an EASE implementation [11b], Microsoft Recommenders/SAR [15][21], implicit [22], Elliot [23], RecBole [24] and Anelli et al. [20]. Numbers that come from the canonical papers but could not be re-checked in this session are marked "approximately".

```
           The classic recommender family tree (simplified)

 Explicit ratings (Netflix Prize era)       Implicit feedback (clicks, plays, buys)
 -----------------------------------        ---------------------------------------
 user-kNN (GroupLens, 1994)                 item-to-item CF (Amazon, 1998-2003)
       |                                          |
 item-kNN (Sarwar et al., 2001) ---------------+  +--> SAR (co-occurrence + decay)
       |                                       |  |
 Funk-SVD / biased MF (2006)                   |  +--> SLIM (2011) --> EASE (2019)
       |                                       |              \        /
 SVD++ / timeSVD++ (Koren 2008-09)             |              linear autoencoders
       |                                       |
       +--> WALS / iALS (Hu-Koren-Volinsky 2008) --> iALS++ / revisited iALS (2021-22)
       +--> BPR-MF (Rendle 2009) --> WARP / LightFM hybrid (2015)
                                          |
 word2vec (2013) --> item2vec / prod2vec (2015-16) --> Airbnb listing emb. (2018)
                                          |
                      two-tower / sequential / generative models (later chapters)
```

1. Neighborhood collaborative filtering

1.1 User-based kNN

Problem. Predict a user's interest in an item from the behaviour of similar users.

Mechanism. Build user-user similarities from co-rated items, usually with Pearson correlation or cosine similarity. Pick the k most similar neighbours who interacted with the target item. Predict a similarity-weighted average of their (mean-centred) ratings.

Why it fell out of favour at scale. The user space is far larger and changes far faster than the item space. A user's neighbourhood changes with every new interaction, so the similarity matrix has to be recomputed constantly. Sparse overlap between users also makes similarities noisy. Amazon's engineers gave exactly these reasons for moving away from user-based CF [1][2].

1.2 Item-to-item CF (Amazon)

Problem. Amazon needed recommendations for tens of millions of customers and millions of products. Results had to be computed in real time on every page view, and had to react to what the customer had just put in the cart [1].

Architecture. The key move was to swap the roles of users and items. Item-item relatedness is much more stable than user-user similarity, so it can be computed offline. The online step is then a cheap lookup and merge:

```
OFFLINE (batch, periodic)                        ONLINE (per request, ms)
-------------------------------------------      ------------------------------
for each item i1 in catalog:                      user context:
  for each customer c who bought i1:                recent purchases / cart / views
    for each item i2 bought by c:                        |
       record co-purchase(i1, i2)                        v
  for each i2: sim(i1,i2) = cosine over customers    lookup top-N similar items
store top-N similar items per item  ------------>    for each seed item
  (item -> [(item, score), ...])                             |
                                                             v
                                                    merge, de-dup, remove owned,
                                                    re-rank (business rules)
```

Scale properties. The worst-case offline cost is O(N^2 M), where N is the number of items and M the number of customers. In practice it is closer to O(NM), because most customers buy very few items. Online cost depends only on how many items the user has interacted with, not on catalogue size or user count [1]. The 2003 IEEE Internet Computing paper later received the journal's "test of time" recognition. Its 2017 retrospective, "Two Decades of Recommender Systems at Amazon.com", describes how the basic item-to-item idea was refined: correcting relatedness for purchase base rates, handling time and context, and using it far beyond the "customers who bought" widget [2].

Academic formalisation. Sarwar, Karypis, Konstan and Riedl (WWW 2001) studied item-based CF systematically. They compared cosine, adjusted-cosine and correlation similarities, and showed that item-based models could be precomputed and gave better quality than user-based kNN on sparse data [25].

1.3 SAR: item-kNN with time decay as a production recipe

Microsoft's open-source Recommenders repository ships **SAR (Simple Algorithm for Recommendation)**, a cleaned-up item-kNN that works as a strong first baseline [15][21]:

1. **Co-occurrence matrix C (m x m).** c_ij counts the users who interacted with both item i and item j.
2. **Similarity S by rescaling C.** Jaccard is s_ij = c_ij / (c_ii + c_jj - c_ij), a "compromise between popularity and discovery". Lift is s_ij = c_ij / (c_ii * c_jj), which favours niche and serendipitous items. Raw counts favour popular items.
3. **Affinity matrix A (n x m) with event weights and exponential time decay.** a_ij = sum over events k of w_k * (1/2)^((t0 - t_k)/T), where T is a half-life.
4. **Scores = A x S.** Previously seen items can optionally be filtered out [21].

The example notebook uses Jaccard similarity, a 30-day half-life and top-10 on MovieLens. It reports NDCG@10 of about 0.385 and Precision@10 of about 0.33 [21]. The notebook also says plainly that the single-node implementation is not meant for very large data, because the dense item-item work becomes expensive [21]. That trade-off holds for every item-item model: quality is strong and the model is simple, but memory grows as O(m^2) unless similarities are sparsified to top-k per item.

1.4 Graph random walks: P3alpha and RP3beta

The Dacrema et al. evaluation suite includes two graph baselines next to kNN: **P3alpha** and **RP3beta** [16]. Both score items by the probability of a three-step random walk user -> item -> user -> item on the bipartite interaction graph. P3alpha raises transition probabilities to a power alpha. RP3beta also divides by item popularity raised to beta, which pushes the ranking towards the long tail. They are item-item models in disguise: the walk collapses into a sparse item-item matrix, so serving is the same as kNN. In the reproducibility studies these cheap models often matched or beat neural baselines [16][17].

2. Matrix factorization for explicit feedback: Funk-SVD, SVD++, timeSVD++

2.1 The Netflix Prize context

The Netflix Prize (2006-2009) offered USD 1M for a 10% RMSE improvement over Cinematch. It released about 100M ratings from about 480K users on about 17,770 movies. Latent-factor models became the mainstream approach as a result [3][4]. The summary of the winning techniques by Koren, Bell and Volinsky, "Matrix Factorization Techniques for Recommender Systems" (IEEE Computer, 2009), is the standard reference [3].

Biased MF (Funk-SVD). Predict r_ui = mu + b_u + b_i + q_i^T p_u. Here mu is the global mean, b_u and b_i are user and item biases, and p_u and q_i are f-dimensional factors. Fit by stochastic gradient descent (SGD) on observed ratings only, with L2 regularisation. The bias terms explain much of the variance in ratings. Leaving them out is still one of the most common reasons a "baseline" looks weak [3][26].

SVD++. In "Factorization Meets the Neighborhood" (KDD 2008), Koren added implicit information: the set N(u) of items the user rated, whatever the ratings were. The user representation becomes p_u + |N(u)|^(-1/2) * sum over j in N(u) of y_j. The fact that a user chose to rate an item carries signal. The same paper combined factor and neighbourhood models in one objective [4].

timeSVD++. This variant lets biases and user factors drift over time, because user taste and item popularity are not stationary. It was one of the strongest single models in the winning ensemble [3].

Lesson from production. The Netflix Prize also showed the cost of ensembles. The grand-prize blend combined on the order of a hundred models. Netflix said publicly that it adopted the core MF and RBM components, but did not put the full blend into production, because the extra accuracy did not justify the engineering cost. The company was also moving from DVD ratings to streaming implicit signals [3]. That shift, from rating prediction (RMSE) to ranking over implicit feedback, is what the rest of this chapter covers.

3. Implicit-feedback MF: confidence-weighted ALS (Hu, Koren, Volinsky)

3.1 Problem

Most real systems never see ratings. They see views, plays, purchases and dwell time. Such data has several properties that break rating-style MF [5]:

- There is no negative feedback. A missing entry may mean dislike, or may mean the user never saw the item.
- The data is noisy. A watched show may have been playing in the background.
- The numeric value shows confidence, not preference. Watching a show many times makes it more likely the user likes it, but the count is not a rating.
- Evaluation must be ranking-based.

3.2 Model

Hu, Koren and Volinsky (ICDM 2008) built the method on TV-viewing data from a digital set-top box service: roughly 300K households and about 17K programmes [5]. They split each observation r_ui into:

- **Preference** p_ui = 1 if r_ui > 0, else 0.
- **Confidence** c_ui = 1 + alpha * r_ui. A log variant, c_ui = 1 + alpha * log(1 + r_ui / epsilon), is also proposed. The paper reports alpha of about 40 working well on their data.

The objective covers **all** user-item pairs, observed or not:

```
min over x_*, y_*:  sum_{u,i} c_ui (p_ui - x_u^T y_i)^2
                  + lambda ( sum_u ||x_u||^2 + sum_i ||y_i||^2 )
```

Summing over all m*n cells is the point of the method: unobserved cells act as low-confidence negatives. The engineering problem is that this dense sum is intractable if done naively.

3.3 The ALS trick that makes it scale

With item factors Y fixed, each user's factor has a closed form:

```
x_u = (Y^T C^u Y + lambda I)^(-1) Y^T C^u p(u)
    where  Y^T C^u Y = Y^T Y  +  Y^T (C^u - I) Y
                       ^^^^^     ^^^^^^^^^^^^^^^^
                  precompute     only n_u nonzero rows (items u touched)
                  once, O(f^2 n)
```

Y^T Y is shared by all users. The correction term only touches the n_u items the user interacted with, so one sweep over users costs O(f^2 N + f^3 m). N is the number of non-zeros, f the number of factors and m the number of users. The cost is linear in the data, not in the full matrix [5]. Items are handled the same way. Every user (and every item) solves independently, so the problem is embarrassingly parallel. That is why ALS rather than SGD became the default for distributed implicit MF.

The same paper also gives **explanations**. Because the solve is linear, a user's score for an item can be written as a weighted sum over the items they consumed. "Because you watched X" falls out of the algebra [5].

3.4 Industrial implementations

Implementation	Algorithm(s)	Execution model	Notes
Spark MLlib ALS [6]	explicit ALS, implicit ALS (Hu et al.)	distributed, block-partitioned	parameters rank, regParam, alpha, implicitPrefs; coldStartStrategy to drop NaN predictions
Facebook Giraph CF [7]	SGD and ALS on Apache Giraph	graph-parallel "rotational" scheme	reported on graphs of roughly 100B ratings (approx.)
benfred/implicit [22]	iALS (incl. conjugate gradient), BPR, logistic MF, item-kNN (cosine/TF-IDF/BM25)	multi-threaded Cython/OpenMP, CUDA kernels for ALS and BPR	integrates Annoy, NMSLIB and Faiss for ANN; advises disabling BLAS threading for big speedups

Implementation	Algorithm(s)	Execution model	Notes
Google iALS / iALS++ [18]	iALS, block (subspace) iALS++	single-machine research code	embedding dims up to 2048 (ML-20M) and 8192 (MSD)
TensorFlow WALS (historical)	weighted ALS	distributed TF	basis of earlier Google Cloud recommendation tutorials (approx.)

Facebook's rotational hybrid. Facebook's 2015 engineering post "Recommending items to more than a billion people" describes running MF on Giraph [7]. Workers hold partitions of users, and item factors rotate between workers in a ring. This avoids broadcasting the full item matrix, which standard Giraph and Spark ALS would do. Facebook reported that the approach handled datasets of around 100 billion ratings, which was much larger than the open-source alternatives of the time could handle. The post also covers approximate top-k serving by grouping items. Figures are approximate here because the post could not be re-read during this session.

The implicit library's practical tips. Set OPENBLAS_NUM_THREADS=1 or MKL_NUM_THREADS=1 so BLAS threading does not fight the library's own OpenMP parallelism. Use GPU kernels for ALS and BPR. Serve with an ANN index (Annoy, NMSLIB, Faiss) rather than brute-force scoring [22].

3.5 iALS revisited (2021-2022)

Rendle, Krichene, Zhang and Koren re-ran iALS on the benchmarks that neural papers use. These were the NCF setups (ML-1M, Pinterest) and the Mult-VAE setups (ML-20M, Million Song Dataset) [18]. The released configurations show what "properly tuned" means [18]:

Benchmark	Dim	Reg	Unobserved weight	Epochs
ML-1M (NCF protocol)	192	0.007	0.3	12
Pinterest (NCF protocol)	192	0.02	0.007	16
ML-20M (VAE protocol)	256 (2048 in headline table)	0.003	0.1	16
MSD (VAE protocol)	256 (8192 in headline table)	0.002	0.02	16

Three details carried most of the gain [18]:

1. A **separate weight on unobserved entries**, rather than the implicit weight of 1 in the original formulation.
2. **Frequency-scaled regularisation** (the "reg exp" exponent).
3. **Much larger embedding dimensions than papers usually allow**, up to 2048 and 8192.

The companion **iALS++** paper uses block (subspace) coordinate descent so that these large dimensions are cheap enough to train [18]. The conclusion of the revisiting paper was that a well-tuned iALS is competitive with the strongest published results on these benchmarks, including Mult-VAE and EASE. The numbers are in the paper's Tables 3 and 4 and are not reproduced here.

4. Pairwise learning to rank: BPR and WARP

4.1 BPR (Rendle et al., UAI 2009)

Problem. Pointwise regression on 0/1 implicit data optimises the wrong thing. What matters is that observed items rank above unobserved ones for each user.

Model. BPR assumes that for user u, an observed item i should be preferred to an unobserved item j. It maximises the posterior

```
sum over (u,i,j) in D_S of  ln sigma( xhat_uij )  -  lambda ||Theta||^2
  with xhat_uij = xhat_ui - xhat_uj
```

This is a smooth surrogate for AUC [8]. BPR is a **criterion**, not a model. xhat can come from MF (BPR-MF), item-kNN (BPR-kNN) or anything differentiable.

Training. The full set of (u, i, j) triples is huge, so BPR uses **LearnBPR**: SGD with bootstrap sampling of triples, drawn uniformly with replacement. The paper argues this converges much faster than user-wise sweeps [8].

Lessons from later practice.

- Uniform negative sampling gives mostly easy negatives late in training, so gradients vanish. This led to popularity-aware, adaptive and hard negative sampling, and to WARP.
- BPR-MF is in every evaluation suite. In Dacrema et al., BPRMF sits next to iALS, PureSVD, FunkSVD and SLIM-BPR [16].
- It needs careful tuning: learning rate, regularisation and the number of sampled negatives all matter a lot.

4.2 WARP and LightFM

WARP (Weighted Approximate-Rank Pairwise). For each positive, sample negatives until one violates the margin. The number of draws needed estimates the positive's rank, and the update is weighted by a function of that rank. This focuses learning on the top of the list. LightFM implements both BPR and WARP losses, and its quickstart uses LightFM(loss='warp') [9].

5. Sparse and dense linear item-item models: SLIM and EASE

5.1 SLIM (Ning and Karypis, ICDM 2011)

Idea. Learn the item-item weight matrix W directly rather than computing it from a similarity heuristic:

```
min_W  1/2 ||A - A W||_F^2 + beta/2 ||W||_F^2 + lambda ||W||_1
s.t.   W >= 0,  diag(W) = 0
```

A is the user-item matrix. The L1 + L2 (elastic net) penalty makes W sparse, so serving is as cheap as item-kNN. The zero-diagonal constraint stops the trivial solution of each item predicting itself. The non-negativity keeps W interpretable. Each column of W is its own regression, so training splits by item.

System notes. The KarypisLab SLIM library is a multi-threaded C implementation with command-line tools (slim_learn, slim_predict, slim_mselect for model selection) and a Python interface [10]. It offers coordinate descent and an ADMM solver (which needs Intel MKL). The README says ADMM usually wins when there are few items relative to users and many non-zeros [10]. In the Dacrema et al. suite, **SLIM ElasticNet** and **SLIM BPR** are among the strongest baselines [16].

5.2 EASE (Steck, WWW 2019)

Idea. Drop the L1 penalty and the non-negativity constraint from SLIM and keep only L2 and the zero diagonal. The problem then has a **closed-form solution**:

```
min_B ||X - X B||_F^2 + lambda ||B||_F^2   s.t. diag(B) = 0

P   = (X^T X + lambda I)^(-1)                # item-item, m x m
B_ij = - P_ij / P_jj   for i != j ,  B_jj = 0
scores(u) = x_u B                            # dense item-item "autoencoder"
```

An open-source implementation [11b] follows exactly this recipe: build the sparse user-item matrix, form the regularised Gram matrix X^T X + lambda I, invert it, take B with a zeroed diagonal, score with X B, filter seen items and take the top-k.

Why it matters. EASE is a linear autoencoder with no hidden layer, which is why the title calls it "embarrassingly shallow". It matched or beat deep autoencoders such as Mult-VAE on the standard ML-20M, Netflix and MSD benchmarks [11]. Training is one m x m inversion, with cost roughly O(m^3) in the number of items. That cost does not depend on the number of users, and m in the low tens of thousands fits comfortably in memory. Its two limits are the m^2 memory of a dense B and the cubic inversion. Beyond about 10^5 items you need item pruning, sparse approximations or blockwise methods. Steck also observed that the learned weights include many **negative** item-item weights, which carry useful information. SLIM's non-negativity constraint throws that information away [11].

Follow-ups. The main deep model EASE was measured against was Mult-VAE [27]. Steck's later work extended the linear family, with higher-order relations and denoising views of the linear autoencoder (not re-verified this session). It also pointed out pitfalls of using cosine similarity on learned embeddings [28]. In 2026 EASE is still the default "is your fancy model actually better?" check in the major benchmarking frameworks.

```
Linear item-item models: what each one learns

            similarity source           constraints            training cost
item-kNN    cosine / Jaccard heuristic  top-k sparsify         O(nnz * avg deg)
SAR         co-occurrence + rescale     time-decayed A         O(nnz + m^2)
RP3beta     3-step random walk          popularity penalty     sparse mat-mults
SLIM        learned (elastic net)       W >= 0, diag 0         m regressions
EASE        learned (ridge)             diag 0                 one m x m inverse
```

6. Word2vec-style item embeddings: item2vec, prod2vec, Airbnb

6.1 item2vec (Barkan and Koenigstein, 2016)

Apply skip-gram with negative sampling (SGNS) to item sets, such as the items in one user's basket or listening list. Each set is treated as a "sentence" in which every pair of items is context, because order is ignored. The learned vectors give item-item similarity that is competitive with SVD-based item similarity, and it was evaluated on Microsoft Store and music data [12]. The appeal for systems is that SGNS is cheap, scales linearly with the number of co-occurrences, and gives dense vectors that drop straight into an ANN index.

6.2 prod2vec and bagged-prod2vec (Yahoo, KDD 2015)

Grbovic et al. ("E-commerce in Your Inbox") learned product embeddings from purchase sequences mined from e-mail receipts. Receipts were ordered in time per user, and products in the same receipt formed context. They added user embeddings (user2vec), and a "bagged" variant that treats each receipt as an unordered bag. This was one of the first web-scale industrial uses of word2vec for recommendation, serving product ads in Yahoo Mail. It reported clear CTR gains over popularity-based targeting (exact figures not re-verified here) [13].

6.3 Airbnb listing embeddings (KDD 2018 best paper)

Airbnb trained listing embeddings on **click sessions**, roughly 800 million of them. A session is split when consecutive clicks are more than 30 minutes apart. Approximate details from the KDD paper and the Airbnb engineering post [14]:

- 32-dimensional embeddings for about 4.5M active listings.
- **Booked listing as global context.** In booked sessions, the booked listing is kept in every context window. Every click in the session is then pulled towards the conversion.
- **Market-aware negatives.** Extra negatives are sampled from the same market, so embeddings separate listings within a city instead of just learning geography.
- **Cold start.** A new listing's embedding is the average of its three nearest listings by type, price and location (approx.).
- **User-type and listing-type embeddings** trained on bookings to handle sparsity for long-horizon personalisation.
- Production use: similar-listing carousels, and real-time personalisation features in search ranking. They reported a meaningful CTR lift on similar-listing recommendations (approx. 21%, not re-verified this session).

```
Airbnb-style session embedding pipeline
---------------------------------------
 click logs --> sessionize (30-min gap) --> filter (dwell >= threshold)
      |
      v
 SGNS training (window, negatives = random + same-market, global ctx = booking)
      |
      v
 listing vectors (d=32) --> ANN index --> "similar listings" candidates
      |
      +--> real-time features: sim(candidate, recently clicked / skipped)
                                --> fed to GBDT/NN search ranker
```

Lesson. Each of these word2vec variants is the same algorithm. What made them work was domain-specific sessionisation, context design and negative sampling. Treat embedding quality as a data-engineering problem first.

7. Content-based and hybrid recommenders

7.1 Content-based filtering

Content-based recommenders represent items by features: text, tags, categories, audio or image embeddings. They represent users by aggregating the features of items they liked, then score candidates by similarity or a learned classifier. Microsoft Recommenders includes a TF-IDF content baseline for text datasets [15]. The Dacrema et al. suite includes CBF-kNN and a CF+CBF hybrid kNN [16]. Strengths: cold start for new items, transparency, and no need for other users' data. Weaknesses: over-specialisation (the "more of the same" filter bubble), and quality capped by feature quality. In 2026 that cap has been raised a lot by pretrained text and image encoders.

7.2 Hybrids: LightFM as the canonical design

LightFM (Kula, RecSys 2015 workshop, built at Lyst) represents each user and item as **the sum of the latent vectors of its features** [9]. Feature sets usually include a per-entity indicator feature plus metadata such as tags, brand and category. The model:

```
user u  -> features f_u (one-hot id + metadata)  -> e_u = sum_{j in f_u} E_U[j]
item i  -> features f_i (one-hot id + metadata)  -> e_i = sum_{j in f_i} E_I[j]
score(u,i) = e_u . e_i + b_u + b_i     trained with BPR or WARP (or logistic)
```

With only indicator features, LightFM is plain MF. With only metadata features, it is a content model. With both, a new item with no interactions still gets a meaningful embedding from its tags [9]. This structure is conceptually a two-tower model with linear, bag-of-features towers. It is the direct ancestor of the neural two-tower retrieval in Chapter 2. Factorization machines (xLearn FM/FFM in Microsoft Recommenders [15]) generalise the same idea to arbitrary feature crosses.

7.3 Hybridisation patterns in production

- **Feature augmentation.** CF embeddings (iALS, item2vec) become features in a GBDT or neural ranker. Airbnb does this [14].
- **Switching.** Use content and popularity for cold users and items, and CF once there are enough interactions.

- **Candidate-source union.** Item-kNN, MF, content and trending sources each supply candidates. A learned ranker merges them. This is the "classic models as retrieval sources" pattern that most large systems still use (see Chapter 1).

8. The reproducibility crisis: why classic baselines still matter

8.1 Dacrema, Cremonesi and Jannach (RecSys 2019, best paper)

"Are We Really Making Much Progress? A Worrying Analysis of Recent Neural Recommendation Approaches" examined 18 top-N recommendation algorithms. All were neural and published at major venues (KDD, SIGIR, WWW, RecSys) in the preceding years. The findings [16][17]:

- Only **7 of 18** could be reproduced with reasonable effort, using the authors' code and data.
- Of those 7, **6 were often outperformed by comparatively simple heuristics**: nearest-neighbour or graph methods tuned properly.
- The remaining one beat the simple baselines but **did not consistently beat a well-tuned non-neural linear ranking method**.

8.2 The TOIS extension (2021)

"A Troubling Analysis of Reproducibility and Progress in Recommender Systems Research" widened the study [17]. Roughly **12 of 26** examined neural methods could be reproduced, and **11 of those 12** were outperformed by simple, well-tuned baselines (approximate counts as stated in the paper's abstract). The accompanying code base [16] ported the neural methods:

- CollaborativeDL, CollaborativeVAE, MCRec, DELF, DMF, ConvNCF, CoupledCF, NeuRec, CFM, NeuMF, Mult-VAE, SpectralCF and CMN.

It evaluated them against 17 baselines, all with systematic hyperparameter search:

- Random and TopPop.
- UserKNN and ItemKNN.
- CBF and CF+CBF kNN.
- P3alpha and RP3beta.
- PureSVD, NMF, iALS, BPRMF and FunkSVD.
- EASE-R, SLIM-BPR and SLIM-ElasticNet.

A 2021 maintenance note fixed an HR/NDCG implementation issue that affected a few NDCG numbers, and stated that "the analysis and conclusions of the study are not affected" [16].

A companion CIKM 2020 paper went after a specific architectural claim. It showed that the reported benefit of applying CNNs to the outer-product "embedding map" of user and item vectors (ConvNCF and relatives) did not come from the convolutions [16].

8.3 Converging evidence from other groups

- **Rendle, Krichene, Zhang and Anderson (RecSys 2020), "Neural Collaborative Filtering vs. Matrix Factorization Revisited."** On NCF's own datasets and protocol (ML-1M, Pinterest; embedding sizes swept from 16 to 192, 8 repeats), a properly set-up dot-product MF beat NeuMF. A second experiment showed that an MLP needs very large capacity and data to learn even a plain dot product [19]. The systems corollary: dot-product scoring is also what makes ANN retrieval possible, which an MLP scoring function rules out.
- **Rendle, Zhang and Koren (2019), "On the Difficulty of Evaluating Baselines."** On MovieLens-10M and the Netflix Prize data, carefully tuned classic methods (Bayesian timeSVD++ flipped and similar) beat many recently published results. The conclusion was that the reported baseline numbers had been far too weak [26].
- **Rendle et al., iALS revisited (2021-2022).** Tuned iALS matched EASE and Mult-VAE on ML-20M and MSD [18].
- **Anelli et al. (UMAP 2022), "Top-N Recommendation Algorithms: A Quest for the State of the Art."** A large Elliot-based comparison on MovieLens-1M (about 1M interactions), Amazon Digital Music (about 1.58M interactions, 840K users) and Epinions (about 300K interactions) [20]. It again found classic linear and neighbourhood models highly competitive. It also examined beyond-accuracy dimensions such as novelty, diversity and popularity bias, where rankings differ.
- **Krichene and Rendle (KDD 2020), "On Sampled Metrics for Item Recommendation."** Evaluating with 1 positive vs. 100 sampled negatives, the protocol inherited from NCF, can change the ranking of algorithms compared with full ranking [29]. That puts many published comparisons in doubt.
- **Session-based recommendation.** Ludewig, Jannach and colleagues found that session-kNN variants are often competitive with GRU4Rec and later neural session models [30].

8.4 Root causes, and the fixes the community adopted

Failure mode	Typical symptom	Fix adopted by benchmark frameworks
Untuned baselines	baseline uses defaults; proposed model tuned for weeks	equal hyperparameter budget (Bayesian or random search) for all models [16][23]
Weak baseline choice	only BPR-MF / NeuMF as baselines	always include ItemKNN, RP3beta, SLIM/EASE, iALS, TopPop [16][20]
Sampled metrics	1 positive + 100 negatives	full-ranking evaluation [29]
Data leakage, inconsistent splits	test info used for early stopping or tuning	explicit train/validation/test splits; temporal splits
Missing code or data	cannot reproduce	released configs and seeds (Elliot, RecBole, Dacrema repo) [16][23][24]
Single-metric reporting	only HR@10	accuracy + beyond-accuracy + significance tests (Elliot has Wilcoxon and paired t-test) [23]

Frameworks now encode these practices:

- **Elliot (SIGIR 2021)** runs the whole pipeline, from loading and splitting through hyperparameter optimisation to statistical testing. It covers BPRMF, FunkSVD, SVD++, kNN, SLIM, VAE, NeuMF and more [23].

- **RecBole** offers 94 algorithms across general, sequential, context-aware and knowledge-based tasks, with BPR as its quick-start default [24].
- **Microsoft Recommenders** wraps ALS (PySpark), BPR (Cornac), SAR, LightFM, Surprise SVD, xLearn FM/FFM and TF-IDF next to neural models [15].

9. Comparison: classic algorithms as a system designer sees them

Algorithm	Feedback	Training cost (approx.)	Serving pattern	Cold start	Typical role in 2026 stacks
User-kNN	explicit or implicit	O(n^2) user sims or on-the-fly	neighbour lookup per request	poor	rarely used at scale
Item-kNN / item-to-item [1] [25]	implicit	O(nnz * deg), offline batch	precomputed top-N per item, merge	poor for new items	"similar items" widgets, candidate source
SAR [21]	implicit + time	co-occurrence O(m^2) memory	A x S, or top-N per item	poor	quick strong baseline
P3alpha / RP3beta [16]	implicit	sparse matrix products	precomputed item-item	poor	strong baseline, long-tail boost
Funk-SVD / SVD++ [3][4]	explicit ratings	SGD, O(nnz * f) per epoch	dot product	poor	rating prediction, legacy
iALS [5][18]	implicit	O(f^2 nnz + f^3 (n+m)) per sweep	dot product + ANN	poor (fold-in possible)	retrieval source, embedding features
BPR-MF [8]	implicit	SGD over sampled triples	dot product + ANN	poor	baseline; sampling research
SLIM [10]	implicit	m elastic-net regressions	sparse x_u W	poor	strong accuracy, sparse serving
EASE [11]	implicit	one m x m inverse, O(m^3)	dense x_u B	poor	strongest simple baseline for m < ~1e5
item2vec / prod2vec [12] [13]	sequences / baskets	SGNS, linear in co-occurrences	ANN on item vectors	poor (fixable via averaging)	similar items, session features
Content-based (TF-IDF etc.) [15]	item features	feature extraction	similarity search	good	cold start, explanations
LightFM hybrid [9]	implicit/explicit + metadata	SGD (BPR/WARP)	dot product + ANN	good	small/mid-scale hybrid, cold-start retrieval

10. Reference architecture: classic models in a modern pipeline

```
                           +---------------------------+
  interaction logs -------> | batch training (daily)    |
  (views, plays, buys)      |  - item-kNN / RP3beta     |----> item->topN table (KV)
                            |  - iALS / BPR (Spark/GPU) |----> item & user factors
                            |  - EASE (if m small)      |----> B matrix / top-N
                            |  - item2vec on sessions   |----> item vectors
                            +-------------+-------------+
                                          |
                                ANN index build (Faiss/ScaNN/Annoy)
                                          |
request --> context (recent items, user id) v
        --> candidate generation: union of
              [kNN lookups of recent items] + [ANN on user factor]
              + [content / trending for cold users]          (~100s-1000s)
        --> feature join (incl. CF scores, embedding sims as features)
        --> learned ranker (GBDT or DNN)                          (~10s)
        --> business rules, diversity, dedup  --> response
```

Here the classic models do three jobs:

1. **Candidate sources.** They are cheap and high-recall.
2. **Features for the ranker.** Examples: an iALS dot product, an item2vec similarity to the last clicked item, a co-occurrence count.
3. **Baselines and guardrails.** A new model has to beat them offline on full-ranking metrics, and online in A/B tests, before it ships.

Key takeaways for system designers

1. **Tune baselines before building new models.** In the reproducibility studies, ItemKNN, RP3beta, SLIM, EASE and iALS beat most published neural recommenders once they were tuned [16][17][18][19]. Give every baseline the same hyperparameter-search budget as the new model.
2. **Pick item-item over user-user for online serving.** Item relationships are stable and can be precomputed. Online cost then depends on the user's history length, not on catalogue size [1][2].
3. **For implicit data, weight by confidence and count the unobserved entries.** The Hu-Koren-Volinsky formulation, made tractable by the Y^T Y precompute trick, is still the right mental model. Modern tuning adds a separate weight for unobserved entries, frequency-scaled regularisation and large dimensions [5][18].
4. **Prefer dot products.** They are as accurate as MLP scorers in fair comparisons, and they are what makes ANN retrieval and caching possible [19].
5. **EASE is the cheapest strong baseline when the catalogue is small enough.** One matrix inverse, no SGD and no sampling. Move to SLIM (sparse), item-kNN, iALS or blockwise approximations as m grows past about 1e5 [10][11].
6. **Embedding quality depends on the training data you construct.** item2vec-style models succeed or fail on sessionisation, context windows and negative sampling, as with Airbnb's booked-listing global context and same-market negatives [14].
7. **Hybrids solve cold start.** Building entity embeddings as sums of feature embeddings (LightFM) is the simplest way to embed new items. Today's two-tower models are the neural version of the same idea [9].
8. **Evaluate with full ranking, fixed splits and significance tests.** Sampled metrics can reorder algorithms [29]. Use a framework such as Elliot, RecBole or the Dacrema code base, and release configs [16][23][24].

9. **Retire classic models with care.** Many production stacks still use kNN and MF as candidate sources and ranker features even after adding deep models, because they are cheap, interpretable and robust.

References

1. G. Linden, B. Smith, J. York. "Amazon.com Recommendations: Item-to-Item Collaborative Filtering." IEEE Internet Computing, 2003. https://doi.org/10.1109/MIC.2003.1167344
2. B. Smith, G. Linden. "Two Decades of Recommender Systems at Amazon.com." IEEE Internet Computing, 2017. https://doi.org/10.1109/MIC.2017.72
3. Y. Koren, R. Bell, C. Volinsky. "Matrix Factorization Techniques for Recommender Systems." IEEE Computer, 2009. https://doi.org/10.1109/MC.2009.263
4. Y. Koren. "Factorization Meets the Neighborhood: a Multifaceted Collaborative Filtering Model." KDD, 2008. https://doi.org/10.1145/1401890.1401944
5. Y. Hu, Y. Koren, C. Volinsky. "Collaborative Filtering for Implicit Feedback Datasets." IEEE ICDM, 2008. https://doi.org/10.1109/ICDM.2008.22
6. Apache Spark. "Collaborative Filtering (ALS) - MLlib Guide." Apache Software Foundation. https://spark.apache.org/docs/latest/ml-collaborative-filtering.html
7. M. Kabiljo, A. Ilic (Facebook Engineering). "Recommending items to more than a billion people." 2015. https://engineering.fb.com/2015/06/02/core-infra/recommending-items-to-more-than-a-billion-people/
8. S. Rendle, C. Freudenthaler, Z. Gantner, L. Schmidt-Thieme. "BPR: Bayesian Personalized Ranking from Implicit Feedback." UAI, 2009. https://arxiv.org/abs/1205.2618
9. M. Kula (Lyst). "Metadata Embeddings for User and Item Cold-start Recommendations" (LightFM). CBRecSys@RecSys, 2015. Code: https://github.com/lyst/lightfm ; paper: https://arxiv.org/abs/1507.08439
10. X. Ning, G. Karypis. "SLIM: Sparse Linear Methods for Top-N Recommender Systems." ICDM, 2011; SLIM library (Ning, Nikolakopoulos, Shui, Sharma, Karypis, 2019). https://github.com/KarypisLab/SLIM
11. H. Steck. "Embarrassingly Shallow Autoencoders for Sparse Data." WWW, 2019. https://arxiv.org/abs/1905.03375 (11b) Open-source EASE implementation (Darel13712/ease_rec). https://github.com/Darel13712/ease_rec
12. O. Barkan, N. Koenigstein. "Item2Vec: Neural Item Embedding for Collaborative Filtering." IEEE MLSP, 2016. https://arxiv.org/abs/1603.04259
13. M. Grbovic et al. (Yahoo Labs). "E-commerce in Your Inbox: Product Recommendations at Scale." KDD, 2015. https://arxiv.org/abs/1606.07154
14. M. Grbovic, H. Cheng (Airbnb). "Real-time Personalization using Embeddings for Search Ranking at Airbnb." KDD, 2018. https://doi.org/10.1145/3219819.3219885
15. Microsoft / Recommenders team (LF AI & Data). "Recommenders: Best Practices on Recommendation Systems." GitHub. https://github.com/recommenders-team/recommenders
16. M. Ferrari Dacrema, P. Cremonesi, D. Jannach. "Are We Really Making Much Progress? A Worrying Analysis of Recent Neural Recommendation Approaches." RecSys, 2019; code and TOIS/CIKM/IJCAI material. Paper: https://arxiv.org/abs/1907.06902 ; code: https://github.com/MaurizioFD/RecSys2019_DeepLearning_Evaluation

17. M. Ferrari Dacrema, S. Boglio, P. Cremonesi, D. Jannach. "A Troubling Analysis of Reproducibility and Progress in Recommender Systems Research." ACM TOIS, 2021. https://arxiv.org/abs/1911.07698
18. S. Rendle, W. Krichene, L. Zhang, Y. Koren. "Revisiting the Performance of iALS on Item Recommendation Benchmarks" (RecSys 2022) and "iALS++: Speeding up Matrix Factorization with Subspace Optimization" (2021). Google Research code: https://github.com/google-research/google-research/tree/master/ials ; paper: https://arxiv.org/abs/2110.14037
19. S. Rendle, W. Krichene, L. Zhang, J. Anderson. "Neural Collaborative Filtering vs. Matrix Factorization Revisited." RecSys, 2020. https://arxiv.org/abs/2005.09683 ; code: https://github.com/google-research/google-research/tree/master/dot_vs_learned_similarity
20. V. W. Anelli, A. Bellogin, T. Di Noia, D. Jannach, C. Pomo. "Top-N Recommendation Algorithms: A Quest for the State-of-the-Art." UMAP, 2022. https://github.com/sisinflab/Top-N-Recommendation-Algorithms-A-Quest-for-the-State-of-the-Art
21. Recommenders team. "SAR Single Node on MovieLens (Python, CPU) - deep dive notebook." GitHub. https://github.com/recommenders-team/recommenders/blob/main/examples/02_model_collaborative_filtering/sar_deep_dive.ipynb
22. B. Frederickson. "implicit: Fast Python Collaborative Filtering for Implicit Feedback Datasets." GitHub. https://github.com/benfred/implicit
23. V. W. Anelli et al. "Elliot: A Comprehensive and Rigorous Framework for Reproducible Recommender Systems Evaluation." SIGIR, 2021. https://github.com/sisinflab/elliot ; https://arxiv.org/abs/2103.02590
24. W. X. Zhao et al. (RUCAIBox). "RecBole: A Unified, Comprehensive and Efficient Recommendation Library." GitHub. https://github.com/RUCAIBox/RecBole
25. B. Sarwar, G. Karypis, J. Konstan, J. Riedl. "Item-based Collaborative Filtering Recommendation Algorithms." WWW, 2001. https://doi.org/10.1145/371920.372071
26. S. Rendle, L. Zhang, Y. Koren. "On the Difficulty of Evaluating Baselines: A Study on Recommender Systems." arXiv, 2019. https://arxiv.org/abs/1905.01395
27. D. Liang, R. G. Krishnan, M. D. Hoffman, T. Jebara. "Variational Autoencoders for Collaborative Filtering" (Mult-VAE). WWW, 2018. https://arxiv.org/abs/1802.05814 ; code: https://github.com/dawenl/vae_cf
28. H. Steck, C. Ekanadham, N. Kallus. "Is Cosine-Similarity of Embeddings Really About Similarity?" WWW Companion, 2024. https://arxiv.org/abs/2403.05440
29. W. Krichene, S. Rendle. "On Sampled Metrics for Item Recommendation." KDD, 2020. https://doi.org/10.1145/3394486.3403226
30. M. Ludewig, N. Mauro, S. Latifi, D. Jannach. "Performance Comparison of Neural and Non-Neural Approaches to Session-based Recommendation." RecSys, 2019. https://arxiv.org/abs/1910.12781

37. Real-Time and Session-Based Recommendation: Streaming Updates, Online Learning, Nearline Pipelines and In-Session Intent

A recommender reacts to user behaviour on several clocks. The **item corpus** changes by the minute: new videos, posts, listings and ads appear constantly. **User intent** changes within a session: a user who has just saved three kitchen Pins wants kitchens now, whatever their 90-day profile says. **Model parameters** go stale as the world drifts: Facebook showed in 2014 that prediction quality degrades measurably for every day the training data falls behind the serving day [3]. Real-time recommendation is the set of designs that shorten these clocks without breaking cost, latency or stability budgets.

This chapter surveys the public designs along four axes:

1. **Fresh features.** Streaming counters, real-time action sequences and session embeddings computed from event streams (Kafka plus Flink, Heron or Samza) and served from a low-latency store. Examples: Twitter's aggregation framework, Airbnb's session embeddings, Chronon's tiled Flink aggregations, DoorDash Riviera.
2. **Fresh candidates.** Indexes and graphs that absorb new edges and items within seconds. Examples: GraphJet, Pixie, Manas realtime HNSW and X's Thunder.
3. **Fresh user representations inside the model.** Real-time sequence encoders that read the last N actions at request time. Examples: TransAct, TransAct V2, SIM/TWIN-style search over long histories, and the academic session-based line GRU4Rec, SASRec and BERT4Rec.
4. **Fresh parameters.** Online or continual training with streaming label joins and incremental parameter sync. Examples: Facebook's online joiner, Google Photon, ByteDance Monolith, Kuaishou PERSIA, and delayed-feedback corrections from Twitter and Alibaba.

The recurring tension is **freshness versus stability**. Fresher means more infrastructure, more ways to diverge between training and serving, more exposure to feedback loops and noise, and less diversity. The best designs pair a cheap, stable long-term component (a batch embedding, a daily model, a lifelong history) with a small, fast real-time component, and they measure the decay curve before buying freshness.

Sourcing note. The sandbox egress proxy blocked direct fetches of arXiv, Medium and company blogs. The chapter therefore draws on two kinds of material. The first is primary sources (papers, engineering posts and open-source READMEs) that sibling chapters of this Atlas had already fetched and summarized (Chapters 5, 8, 9, 11, 18, 19, 20, 21, 22, 26 and 28). The second is a later research pass of about 25 web searches, whose result abstracts and snippets added references [53]-[63]. Figures marked "approx." come from secondary summaries or could not be re-verified here.

37.1 The landscape: where freshness lives

37.1.1 Offline, nearline, online: the Netflix blueprint (2013)

Netflix's "System Architectures for Personalization and Recommendation" by Amatriain and Basilico is still the clearest vocabulary for placing computation [27]. It separates **where** computation runs from **what** is computed:

```
member events (plays, ratings, browsing, search)
       |
       +--> OFFLINE  (Hadoop/Hive/Pig): training, batch scoring; no data or
       |             complexity limit, but stale and context-blind
       |
       +--> NEARLINE (event-driven, no request waiting): e.g. on a play event
       |             recompute member factors and scores; results written to
       |             Cassandra / EVCache / MySQL
       |
       v
    ONLINE (per request, strict SLA): fetch precomputed + live signals,
           light model eval, filtering, context; FALLBACK to precomputed
           or unpersonalized results on timeout
```

Key rules from the post [27]:

- **Nearline** is the middle tier. It reacts to events within seconds, can be more complex than online code because no request waits on it, and writes its results to fast stores.
- **Events vs data.** Events are small, time-sensitive messages that must be processed fast and flow over a pub/sub system (Hermes in the post). Data is the denser record collected into Hadoop for later processing.
- **A model can be split across tiers.** Item factors are trained offline, while user-dependent parts are re-solved nearline when the member acts. Netflix slides from the period describe re-solving a member's MF factors on a play event (approx.).

Amatriain's 2023 "10th anniversary" revisit says the split still holds, but "nearline" has grown into streaming feature platforms and near-real-time training [28]. Almost every system in this chapter can be placed in one of the three tiers.

Netflix's later stack shows how far the nearline tier has grown. Third-party summaries of Netflix talks describe Keystone, a managed Kafka routing service with Flink on top, running on the order of 20,000 Flink jobs. One stated use case is replacing day-old batch jobs for personalization signals such as "take fraction" (plays divided by impressions), which previously had a delay of hours to days (approx.) [62].

Eugene Yan's survey of real-time recommendations frames the trade-off from the operator's side [61]:

- Batch recommendations are computed once a day and cached, so a failed compute step has no customer-facing impact.
- Real-time recommendations aggregate streamed events and generate results on demand. They lose batch's economies of scale and are considerably trickier to run in production.

37.1.2 Four kinds of freshness

Freshness kind	Typical latency	Mechanism	Representative systems
Feature freshness	ms to seconds	Stream processor -> KV store; request-time sequence fetch	Twitter RTA [24], Chronon tiles [32], Airbnb [30], TransAct [9]
Candidate freshness	seconds	In-memory graphs, incremental ANN inserts	GraphJet [18], Pixie [15], Manas realtime [16], Thunder [26]
Representation freshness	per request	Real-time sequence encoder in the ranker	TransAct [9], TransAct V2 [11], TWIN [46]
Parameter freshness	minutes to hours	Online joiner + streaming trainer + incremental sync	FB 2014 [3], Monolith [1], PERSIA [47]

These are complementary, not alternatives. A short-video feed typically uses all four. A marketplace like Airbnb can get most of the benefit from feature freshness alone [30].

37.2 Event backbone: a canonical action stream

Every real-time design starts with a reliable, low-latency stream of user actions with stable keys. Two public examples:

Twitter/X Unified User Actions (UUA). UUA is described as "a centralized, real-time stream of user actions". It merges client-side and server-side events into one Kafka stream, which is replicated to HDFS, Pub/Sub, GCS and BigQuery [22]. The **User Signal Service** then standardizes explicit and implicit signals (likes, retweets, follows, video views and so on) so that retrieval and ranking consume one schema [23].

Recos-Injector. For Twitter's GraphJet-based services, Recos-Injector is the streaming preprocessor. It consumes favorites, retweets, follows and client events, filters and combines them, and publishes them to a **dedicated Kafka topic per GraphJet service** [21].

Design rules that recur across sources:

- **Mint an impression or request ID at serving time and carry it into every downstream event.** Facebook's online joiner, Google Photon and Monolith all depend on such a key for the label join [1][3][4].
- **Log features at serving time** rather than recomputing them later. This is the only robust defence against training/serving skew for real-time features, because the value served at time t is hard to reconstruct offline [1][34].
- **Per-consumer topics.** Recos-Injector's per-service topics let each consumer get a filtered, compact stream and scale independently [21].

Pinterest real-time user signal serving. Pinterest's log path runs from the Singer logging agent on serving hosts into Kafka. The Merced archiver copies Kafka to S3 for long retention, and a unified Flink source concatenates archived and live data behind one API, which the team calls "an unlimited log" [54]. On top of this, the real-time user signal service works in three steps [53]:

1. It consumes trimmed engagement events (user ID, Pin ID, action type).
2. A materialization step enriches each event with Pin and query features.
3. It writes the result to a time-series store that rankers query online.

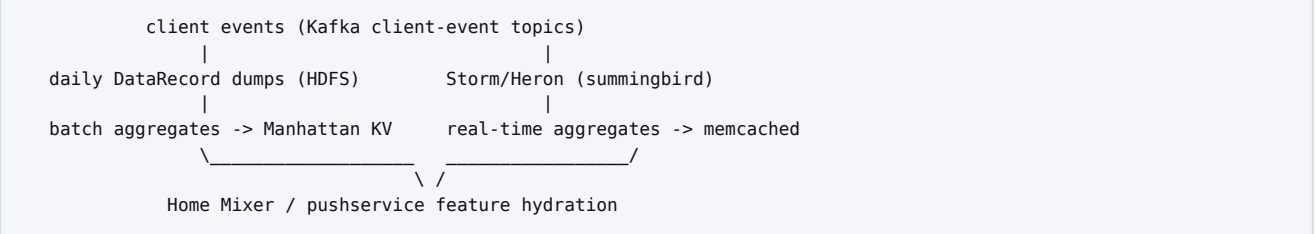
The post reports about **1.2 million events/s** and a **p99 of about 10 seconds** from engagement to servable signal [53]. Services such as TransAct read their "last N actions" from this kind of store (Section 37.5.1).

37.3 Streaming features: counters, windows and tiles

37.3.1 Twitter's aggregation framework: one definition, two engines

Problem. The Home ranker, push notifications and email recommendations all need user, author, user x author and tweet engagement aggregates, conditioned on content type and engagement label, both over long horizons and over the last few minutes [24].

Design. An `AggregateGroup` declares keys (for example `USER_ID` or `(USER_ID, AUTHOR_ID)`), features, labels, metrics and **half-lives** (exponential decay rather than hard windows). The same definition drives a daily summingbird-scalding batch job, whose output is uploaded to Manhattan, and a summingbird-storm/Heron **real-time** job backed by memcached [24].



Scale. The 2023 open-source release describes ranking features kept over long windows (50 days) and real-time windows (under 3 days, typically 30 minutes), with roughly 6,000 features hydrated per ranked candidate for the For You heavy ranker (approx.) [24][25].

Tricks. Decayed counters are monoid-friendly: an old value plus elapsed time is enough to roll forward, so each key stores O(1) state instead of a window of events. The real-time path also updates "realtime features" such as like, reply and retweet counts on tweets for Earlybird-based retrieval [25].

37.3.2 Airbnb/Stripe Chronon: tiled Flink aggregation

Chronon (open-sourced by Airbnb and co-maintained with Stripe) makes the freshness model an explicit part of the feature definition.

- A `GroupBy` with `accuracy = SNAPSHOT` is refreshed daily and backfilled as of the previous midnight. With `TEMPORAL` (the default when a Kafka `topic` is set) it is updated in real time online and backfilled with millisecond point-in-time accuracy offline [33].
- **Untiled vs tiled serving.** In the untiled design the streaming job writes individual events and the fetcher folds them at request time. With 10 events/s for one key and a 12-hour window, one request has to fold 432,000 events. In the tiled design a stateful **Flink** job pre-aggregates events into tiles of intermediate representations (for

example sum and count for an average), and the fetcher reads 12 or 13 one-hour tiles. Stripe reported a **33% reduction in feature-serving latency** after the initial tiled rollout [32].
- The Flink job evaluates the same Spark SQL expressions as the batch engine by embedding Catalyst inside Flink, so batch and streaming share one expression language [35].
- Chronon logs served values and runs a daily job comparing them with backfilled values. It names the inherent sources of skew: Kafka plus KV write latency makes streaming features lag by milliseconds to seconds, and batch features are inconsistent "centered around midnight" until the bulk upload finishes [34].

```
Kafka -> Flink [source] -> [Spark SQL eval] -> [tiling: IRs per tile] -> KV
fetcher: get(key, tiles in [now - W, now)) -> merge IRs -> finalize -> feature
batch upload holds hop-aligned partials; stream covers everything after batch end
```

37.3.3 Marketplace variants: DoorDash Riviera, Uber, Lyft

DoorDash's **Riviera** (2021) is a declarative framework for real-time features on Flink: features are defined in configuration and SQL, and the framework builds the streaming job. Its output lands in a Redis-based online feature store next to batch features from Fabricator [36]. Uber's Michelangelo used the same dual-path pattern: Kafka plus streaming jobs (Samza, later Flink) for near-real-time features such as a restaurant's recent average prep time, written to the same Cassandra store as batch features (approx.; see Chapter 8).

Instacart's account of moving to real-time ML shows the same shape. Its pipeline reads raw service events from Kafka, transforms them into features with Flink, and sinks them into a feature store for on-demand access. The motivation was that batch-only features made predictions stale during a shopping journey. The post also names an organizational trap: different teams had adopted three different streaming tools, which made cross-team event features hard to build [57]. A secondary case study of Instacart's Griffin ML platform lists session-based recommendations among the applications this streaming path enabled (approx.) [58].

Lesson. The streaming feature problem is solved. The remaining design work is (1) one declarative definition for batch, stream and backfill, (2) pre-aggregation on the write path for hot keys, and (3) logging served values to measure and remove skew.

37.4 Real-time candidate generation

37.4.1 GraphJet (Twitter, VLDB 2016)

Problem. Tweets have a useful life of hours. Collaborative filtering on a nightly batch graph is too stale, so Twitter needed "users who engaged with X also engaged with Y" over roughly the last day, with new edges visible within seconds [18].

Architecture. GraphJet "maintains a full graph index over a sliding time window in memory on a single server" [19]. The bipartite graph has users on one side and tweets (or URLs, hashtags) on the other. The index is a sequence of **time-sliced segments**: new

edges go into the newest segment and whole segments are dropped when they leave the window, which avoids per-edge deletion and garbage-collection pressure. Recommendations come from SALSA-style personalized random walks [18].

Metric (per server, from README [19])	Value
Edge ingestion	up to 1,000,000 edges/s
Recommendations, steady state	up to 500/s
Edge reads	"several million" per second

Production descendant: UTEG. The User-Tweet-Entity-Graph service is "a Finagle thrift service built on the GraphJet framework". It produces the "XXX liked" out-of-network tweets on the home timeline. It is fed by Kafka through Recos-Injector, keeps **in-memory engagements for the past 24-48 hours**, takes the viewer's RealGraph-weighted followees as input, and returns tweets ranked by weighted followee engagement [20][21]. Twitter's 2023 blog attributes roughly 15% of For You tweets to this kind of social-graph traversal (approx.) [25].

```
Kafka (fav, RT, follow, client events)
       |
 recos-injector (filter, combine, per-service topic)
       |
 UTEG / GraphJet (24-48 h window, all in RAM, full replica per server)
       |  input: viewer's weighted followees
       v
 tweets ranked by followee engagement, with social proof
```

37.4.2 Thunder (X, 2026)

In the 2026 open-source X algorithm, in-network retrieval moved from the Lucene-based Earlybird search cluster to **Thunder**, "an in-memory post store and realtime ingestion pipeline". It consumes post create and delete events from Kafka, serves the posts of the viewer's followees, trims posts older than a retention period, and promises "sub-millisecond lookups for in-network content without hitting an external database". It also receives the already-seen list so it can exclude those posts at the source [26]. Architecturally this is GraphJet again: a time-windowed in-memory index replacing a general-purpose search engine for very fresh content.

37.4.3 Pixie (Pinterest, WWW 2018)

Problem. Recommend from more than 3 billion Pins to more than 200 million users and react to what the user just did. The previous Hadoop system precomputed results and could not react within a session [15].

Design. A pruned Pin-Board bipartite graph of about 3 billion nodes and 17 billion edges, about 120 GB, is loaded into RAM on each server. Every server holds a full replica, so walks never cross machines. The graph is rebuilt offline. The **query** is real time: the user's most recent Pins, each weighted, seed many short random walks with restarts. Per-Pin visit counts are combined as V[p] = (sum over q of sqrt(V_q[p]))^2, which rewards candidates reached from several query Pins. Walks stop early once enough Pins pass a visit threshold [15].

Scale. p99 latency under 60 ms, about 1,200 requests/s per server and about 100,000 requests/s across the cluster, with up to 50% higher engagement than the Hadoop system [15].

Lesson. Freshness can come from the **query**, not the index. A daily graph with real-time seed actions already gives in-session responsiveness.

37.4.4 Manas realtime HNSW (Pinterest)

Pinterest's ANN platform Manas added a **realtime HNSW** mode in which new Pins are inserted incrementally into the graph index, so embedding-based retrieval can surface fresh content without waiting for a full rebuild. Filtering was first done by over-fetching K neighbours and post-filtering [16]. **Streaming filters** later reused the inverted index as the filter store, with bitmap and array-backed posting lists replacing skiplists for faster lookups [17].

37.4.5 Google News co-visitation (2007)

The earliest large-scale example of the split is Google News personalization [49]. MinHash user clusters and PLSI classes are computed in batch with MapReduce. **Co-visitation counts** (two stories clicked by the same user within a time window) and per-cluster click counts are updated online in Bigtable, so a story gets a score as soon as clicks arrive. Serving blends the three signals linearly. The pattern of batch structure plus streaming counts is the same one Twitter and Pinterest later used.

37.5 Real-time user intent inside the ranker

37.5.1 Pinterest TransAct (KDD 2023)

Problem. Pinterest's user embedding PinnerFormer is computed in a **daily batch**. It was explicitly trained to tolerate staleness: moving from real-time to daily inference cut Recall@10 by 13.9% with a next-action objective but only by 8.3% with the dense all-action objective [13]. Even so, a daily embedding cannot react to what the user did seconds ago [9].

Hybrid design. TransAct encodes the user's most recent actions (on the order of 100) **in real time inside the Homefeed ranker (Pinnability)**, while PinnerFormer and other batch features provide long-term context. The authors present this as combining real-time responsiveness with the cost efficiency of batch representations [9].

```
real-time action stream                                  candidate Pin
        |                                                       |
latest ~100 actions: PinSage emb + action type + timestamp     |
        |                                                       |
early fusion: concat each action with candidate embedding <---+
        v
small Transformer encoder
        v
compress (first K tokens + max pooling) -> flatten
        v
concat with PinnerFormer + other features -> DCN v2 -> MLP
        v
multi-task heads (repin, click, hide, ...) -> utility
```

Findings and tricks [9][10]:

- **Early fusion** of each action with the candidate makes the sequence encoder target-aware.
- **Action-type embeddings** mattered more than positional encodings.
- **Random time-window masking** during training hides the most recent actions for some examples, so the model does not over-rely on the last few clicks. This protects diversity (as reported in summaries).
- The paper lists production problems: serving complexity, reduced diversity, and **engagement decay**, which means the model needs frequent retraining.
- **Serving.** The new model raised CPU latency by more than 20x, so Pinterest moved Homefeed ranking to **GPU serving** to stay latency-neutral at the same cost [10].
- **Results.** Secondary summaries report about +11% Homefeed repin volume and -10% hides (approx.). TransAct was deployed in Homefeed, Related Pins, Notifications and Search [9].

37.5.2 TransAct V2 (2025): real-time plus lifelong

TransAct V2 extends Homefeed ranking from about 100 real-time actions to lifelong histories of up to about **16,000 actions** [11][12]. It keeps three sequences, lifelong, real-time and impressions, and selects a short subsequence by **candidate-anchored nearest-neighbour search** in PinSage space. The **most recent real-time actions are always kept**, so in-session intent is never lost to the long-history search [11].

- Each token carries a timestamp, a multi-hot action type, the surface and a 32-d PinSage embedding. Lifelong actions are stored with int8 quantization [11].
- An auxiliary **Next Action Loss** predicts what the user does next [11].
- Serving kernels (fused dequantization plus NN search, a single-kernel Triton Transformer) improved end-to-end inference latency by 103x (p50), 338x (p90) and 250x (p99) over the unoptimised baseline [11].
- Reported online results: +6.35% repins, +1.41% time spent, -12.8% hides [12].
- A literature review of the paper reports offline gains over the real-time-only TransAct baseline: +13.31% HIT@3 on repins and -11.25% HIT@3 on hides. It also notes that only the nearest-neighbour features are logged at request time, and the NN search is recomputed offline for training (approx.) [63].

37.5.3 Long histories with real-time recency: SIM and TWIN

Alibaba's **SIM** (CIKM 2020) introduced the two-stage pattern for lifelong histories. A General Search Unit (GSU) retrieves behaviours relevant to the candidate. In production this is "hard search" by category over a **User Behavior Tree** keyed by user ID then category ID, about 22 TB in size. An Exact Search Unit (ESU) then runs target attention over the top-K. The output is concatenated with **short-term, real-time DIN features**. SIM modelled up to 54,000 behaviours and reported +7.1% CTR and +4.4% RPM in display advertising [45].

Kuaishou's **TWIN** (KDD 2023) makes the GSU use the same target-attention relevance as the ESU. It keeps it affordable by precomputing and caching linear projections of item-inherent features per item, and compressing user-item cross features into a scalar attention bias. The CP-GSU scores about 10^4 behaviours, keeps the top 100, and the authors report a 99.3% reduction of the compute bottleneck, enough to deploy on main traffic [46].

The real-time lesson: **lifelong and real-time are separate inputs with separate freshness**. The long history is indexed or cached and can lag. The last few minutes of actions are streamed and always included.

37.5.4 Airbnb: session embeddings as real-time features (KDD 2018)

Problem. Search ranking should react to what the guest is doing in this session, for example clicking several cheap, modern apartments near the centre, without retraining a model [30].

Design. Grbovic and Cheng trained skip-gram listing embeddings on click sessions, splitting sessions at gaps of more than 30 minutes. They added two changes: the **booked listing as a global context** for every listing in a booking session, and **same-market negatives**. Secondary summaries report about 800 million click sessions, about 4.5 million active listings and 32-dimensional embeddings (approx.) [30][31].

Real-time use. Kafka streams record the user's clicked, skipped, long-clicked, wishlisted and contacted listings over roughly the last two weeks. For each candidate the ranker computes features such as "similarity to clicked listings" and "similarity to skipped listings" online and feeds them to the GBDT ranker in production at the time. The paper reports an online booking gain, and the embedding similarity features ranked high in importance [30].

Lesson. Freshness can come from **cheap online similarity features over a streamed short history**, with no online training at all. Skipped items are a valuable real-time negative signal.

37.5.5 Netflix: in-session adaptation for search

Netflix research described augmenting search with recommendations adapted to the member's **current session**. Browsing and partial-query signals in the session predict intent before or while the member types, which matters most on TV devices where typing is slow (approx.) [29].

37.5.6 Instagram Explore: online user tower, offline item tower

Instagram Explore's main ML retrieval source is a two-tower network. **Item embeddings are computed offline (typically daily) and indexed**, and **a fresh user embedding is computed at request time** from online features before ANN search. The first-stage ranker reuses two towers for cacheability, and some users' heavy-stage results are precomputed off-peak [37]. This asymmetry, where the user side is fresh and the item side is batched, is the standard compromise for two-tower retrieval.

37.6 Session-based models: the academic line

Session-based recommendation predicts the next item from a short, often anonymous, sequence. It is the modelling counterpart of real-time intent.

Model	Year / org	Core idea	Notable result or caveat
GRU4Rec [38]	2015, Gravity R&D / Telefonica	GRU over session clicks; session-parallel mini-batches; in-batch negatives; BPR/TOP1 ranking losses	Beat item-kNN in the paper
Session-kNN [39]	2017, Jannach & Ludewig	Nearest-neighbour over sessions	Beat GRU4Rec in most tested setups; hybrids best
HRNN [44]	2017	Hierarchical RNN: session RNN initialized from a user-level RNN	Personalizes sessions for logged-in users
SASRec [40]	2018, UCSD	Causal self-attention for next item	About 10x faster than CNN/RNN baselines
BERT4Rec [41]	2019, Alibaba	Bidirectional Cloze training	Not consistently better than SASRec on replication [42]
gSASRec [42]	2023, Glasgow	SASRec + gBCE to fix negative-sampling overconfidence	+9.47% NDCG vs BERT4Rec on ML-1M; ~73% faster training
Transformers4Rec [43]	2021, NVIDIA	Library bridging NLP Transformers and session recs	Used in competitions and industry pipelines

Key details:

- **GRU4Rec** made RNNs practical for sessions with **session-parallel mini-batches**: many sessions run in parallel, and when one ends the next takes its slot and its hidden state is reset. Scoring is restricted to items in the mini-batch, an early form of in-batch negatives [38].
- **Strong simple baselines.** Jannach and Ludewig showed that session-based nearest neighbours often beat GRU4Rec, and the best results came from combining the two [39]. Co-visitation-style item-to-item methods (as in Google News [49]) remain hard to beat for very short sessions.
- **Loss and sampling matter more than architecture.** gSASRec traces most of the BERT4Rec vs SASRec gap to overconfidence caused by negative sampling [42].

From academia to production. Industrial systems rarely deploy a pure session model. Instead the session encoder becomes one component: a real-time sequence feature in a ranker (TransAct [9]), a user tower in retrieval (Instagram [37]), or the short-term branch next to a lifelong search unit (SIM [45], TWIN [46]). Meta's HSTU goes further, treating recommendation as sequential transduction over a single chronological stream of content

and action tokens per user. The paper motivates the design explicitly by high-cardinality, **non-stationary streaming data** [48].

37.7 Online learning: streaming labels and continual training

37.7.1 Facebook ads online joiner (2014)

He et al. describe an online data joiner. Impressions and clicks are keyed by request ID. Impressions wait in a window: a click within the **waiting window** makes a positive, and otherwise a negative is emitted when the window closes. The labeled stream feeds an online trainer for the logistic-regression layer on top of boosted-tree feature transforms, and models are pushed back to the ranker [3].

- **Waiting-window trade-off.** Too short and late clicks become false negatives, biasing CTR down. Too long and examples arrive late and need more buffer memory.
- **Safety.** The joiner needs anomaly protection, for example alarms and cutting off the trainer if the click stream stops [3].
- **Freshness evidence.** Normalized entropy worsens as training data falls days behind evaluation, which motivates daily or online training [3].
- **Negative downsampling with recalibration** $q = p / (p + (1 - p)/w)$ became the standard correction [3].

37.7.2 Google Photon (2013): exactly-once joins

Photon joins clicks to queries across datacenters with exactly-once semantics, because each joined click is billed and used for training. A Paxos-replicated **IdRegistry** records joined event IDs, so duplicate processing in several datacenters never emits a click twice. Clicks that arrive before their query are retried. The paper reports millions of joined events per minute with end-to-end latency of seconds (approx.) [4]. The lesson for recommenders is that **the join must be idempotent**: duplicate positives inflate CTR and break calibration.

37.7.3 ByteDance Monolith (2022): minute-level training at TikTok scale

Problem. ByteDance's feed and ads models face an unbounded, non-stationary sparse ID space, long-tailed feature frequency and concept drift within hours. Standard TensorFlow keeps batch training and serving "completely separated", and fixed-size hashed tables cause collisions [1].

Architecture.

```
client actions --> Kafka: actions ---+
serving-time feature log --> Kafka --+
                                     v
          Flink online joiner (join on request key; in-memory cache +
          on-disk KV for late actions; negative sampling)
                  |                                    |
         Kafka: training examples              HDFS (batch dumps)
                  |                                    |
         online training workers <--- same code ---> batch training
                  |
         training PS: Cuckoo-hash tables (frequency admission + expiry)
                  |  sparse: incremental sync of touched IDs (minute-level)
                  |  dense : synced far less often
                  v
         serving PS -> online inference
```

Key tricks [1][2]:

- **Collisionless embeddings.** Cuckoo-hash key-value tables on the parameter servers give every ID its own row. **Frequency filtering** admits an ID only after it has been seen a threshold number of times, and **expirable embeddings** evict inactive IDs. Together they bound memory.
- **One codebase, two modes.** Batch training on HDFS bootstraps the model, and online training then consumes the Kafka example stream indefinitely.
- **Asymmetric parameter sync.** Pushing terabytes every minute is impractical. Only sparse rows touched since the last sync are pushed, at minute-level intervals, while dense weights, which change slowly, are synced much less often.
- **Cheap fault tolerance.** Daily PS snapshots lose little quality on failure, because each shard holds a small fraction of IDs and online training heals quickly.

Results. Collisionless tables beat hashed ones. Shorter sync intervals (from hours down toward about 30 minutes in the experiments, approx.) gave consistently higher online AUC, and real-time training beat batch training for ads [1]. The repository is TensorFlow plus Bazel and was archived by ByteDance in October 2025 [2].

37.7.4 ShareChat: batch-to-real-time training with Kafka and Flink (2024)

ShareChat's OARS 2024 workshop paper is one of the few public accounts of migrating an existing recommender from batch to real-time training [55]:

- **Stack.** The team moved event ingestion from Google Pub/Sub to Kafka. It used Flink for streaming joins of feature and label events, with RocksDB state and checkpointing.
- **Semantics.** The paper works through causal dependencies between events, event time versus processing time, and exactly-once versus at-least-once delivery.
- **Cost.** Avro schemas plus compression cut event throughput by about 85%, and the redesign cut cost by about 40%.

A companion ShareChat study analyses embeddings on a short-video app with more than 180 million users [56]. Embeddings trained in real time converge toward their final vectors faster than batch-trained ones, needing about one fifth as many views to mature. Batch-trained embeddings end up with markedly higher norms, which the author links to

popularity bias. This is quantitative evidence that real-time training changes **what** is learned, not only **when**.

37.7.5 Kuaishou PERSIA (2022)

PERSIA splits training the same way. Embedding layers are updated **asynchronously** on CPU parameter servers, because sparse updates tolerate staleness, and the dense network is trained **synchronously** on GPUs, because its accuracy suffers under asynchrony. It reports models up to 100 trillion parameters in production at Kuaishou [47]. Monolith and PERSIA agree: **the sparse part is fast-moving and staleness-tolerant, and the dense part is slow-moving and sensitive**.

37.7.6 Delayed feedback in streaming training

Online training on conversions faces labels that arrive minutes to weeks late.

Strategy	Freshness	Bias handling	Source
Waiting window	Delayed by window	Late positives become false negatives	Facebook [3]
Delay model (exponential delay)	Medium	Joint conversion + delay likelihood	Criteo, Chapelle [8]
Fake negatives, no wait	Best	FN-weighted or FN-calibrated loss	Twitter [5]
Elapsed-time sampling	Good	Short wait + re-emit late positives + importance weights	Alibaba ES-DFM [6]
Real negatives re-ingest	Good	Also re-ingest real negatives after window	Alibaba DEFER [7]

Twitter's design (Ktena et al., RecSys 2019) ingests every eligible impression **immediately as a negative** and again as a positive when the engagement arrives. FN-weighted and FN-calibrated losses undo the bias, and they beat waiting-window and delay-model baselines offline and in an online A/B test [5]. For continuous training this is the freshest option and has become a common default.

37.7.7 Online learning for the policy layer

Freshness also applies to exploration and fusion. Yahoo's LinUCB updates a per-article ridge regression online for a small, fast-changing pool of news stories [50].

YouTube Online Matching (RecSys 2023). Batch-trained recommenders are slow to find good new videos. YouTube therefore added a real-time bandit as an extra retrieval source in candidate generation [59]. Its parts:

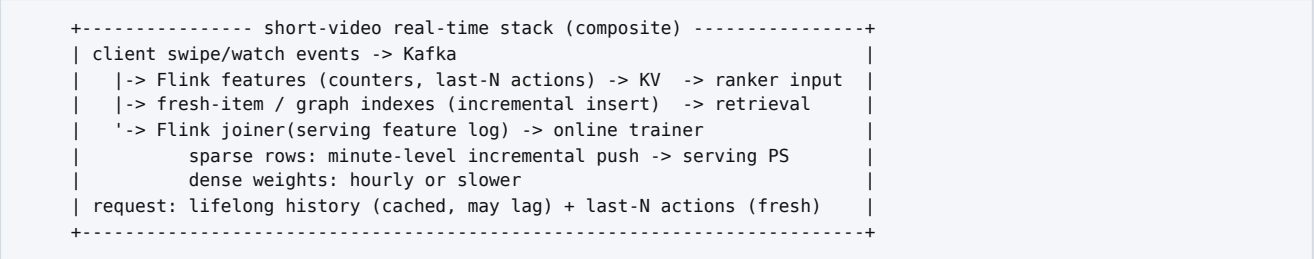
- An **offline** pipeline builds a sparse bipartite graph between users (or user clusters) and items.
- An **online agent** aggregates bandit parameters in real time from the feedback stream.
- **Diag-LinUCB**, an extension of LinUCB, makes the parameter updates distributable, which lets the system serve billions of users and millions of items.

The paper reports a policy-update latency of about **45 minutes at p50 and 74 minutes at p95**. Live experiments on two use cases, fresh-content discovery and corpus exploration, showed significant top-line gains [59]. The design pattern is a hybrid: the expensive structure (the graph) is offline, and only a small set of bandit statistics is real time. Kuaishou's **Pantheon** learns personalized fusion weights jointly with its real-time ranker, deployed for about 400 million daily users [52]. When the candidate pool churns faster than a model can be retrained, a bandit or online-updated policy layer on top of a slower ranker is a cheap way to stay fresh.

37.8 TikTok and the short-video case

TikTok's 2020 newsroom post describes the For You feed as ranking on user interactions (likes, shares, follows, comments, content created), video information and, with lower weight, device and account settings. Signals are weighted by strength: finishing a longer video is a stronger signal than shared country. It also describes diversity rules (no two consecutive videos from the same creator or with the same sound) and cold start through interest selection, which "early interactions quickly replace" [51].

Short video stresses every clock at once. A user may swipe through dozens of items in minutes, each swipe is a label, and the corpus churns hourly. ByteDance's published answer is the Monolith loop: streaming joins, online training, and minute-level sparse sync [1]. Kuaishou's is PERSIA plus real-time sequence features and TWIN-style lifelong search [46] [47].



37.9 Freshness versus stability

Freshness has costs, and the sources name them explicitly.

1. **Feedback loops and over-reaction.** A model that reacts to the last click can collapse diversity. TransAct reports reduced diversity and uses random time-window masking to stop the model from over-weighting the most recent actions [9]. TikTok describes explicit diversity rules on top of the ranker [51].
2. **Train/serve skew in real-time features.** Streaming features lag by milliseconds to seconds, and batch features have a "midnight" inconsistency until upload finishes [34]. Logging served values and backfilling with the same freshness semantics (SNAPSHOT vs TEMPORAL) is the fix [33][34].
3. **Label bias in fast training.** Short waiting windows create false negatives, and fake-negative pipelines duplicate positives. Both need loss corrections [3][5][6].

4. **Operational risk.** An online trainer can be poisoned by a broken stream. Facebook recommends alarms and automatic trainer cutoff [3]. Monolith keeps a batch path and snapshots for recovery [1].
5. **Cost.** TransAct's real-time encoder raised CPU latency more than 20x and forced a move to GPU serving [10]. Untiled streaming aggregation can force 432,000-event folds per request [32].
6. **Engagement decay.** Real-time models degrade quickly if not retrained. TransAct names engagement decay as a reason for frequent retraining [9].

How designs balance it.

- **Pair slow and fast components.** PinnerFormer (daily, trained to be robust to staleness) plus TransAct (real-time) [9][13]. SIM/TWIN lifelong search plus short-term DIN features [45][46]. Instagram's offline item tower plus online user tower [37].
- **Split by parameter type.** Sparse embeddings fast, dense weights slow [1][47].
- **Bound the window.** GraphJet and UTEG keep 24-48 hours, and Thunder trims by retention [20][26].
- **Measure the decay curve first.** Facebook's NE-versus-delay curve [3] and PinnerFormer's real-time-versus-daily Recall@10 gap (13.9% vs 8.3%) [13] are examples of quantifying the value of freshness before paying for it.

37.10 Comparison of public real-time designs

System	Org / year	Freshness layer	Stream stack	Window / latency	Scale (public)
Online joiner [3]	Facebook 2014	Parameters	Custom joiner	Waiting window	750M DAU, 1M advertisers
Photon [4]	Google 2013	Labels	Paxos IdRegistry	Seconds (approx.)	Millions of events/min
Google News [49]	Google 2007	Candidates	Bigtable counts	Minutes	News scale
Netflix blueprint [27]	Netflix 2013	Nearline compute	Pub/sub (Hermes)	Seconds	n/a
GraphJet / UTEG [18][20]	Twitter 2016-23	Candidates	Kafka + recos-injector	24-48 h in RAM	1M edges/s per server
Aggregation fw [24]	Twitter 2023	Features	Heron + memcached	~30 min RT windows	~6k features/candidate
Thunder [26]	X 2026	Candidates	Kafka	Sub-ms lookups	n/a
Pixie [15]	Pinterest 2018	Query-time walks	Real-time seeds	p99 < 60 ms	100k QPS, 17B edges
Manas realtime [16]	Pinterest	ANN index	Incremental HNSW	Seconds (approx.)	Billions of Pins
TransAct [9]	Pinterest 2023	Representation	Real-time action seq	~100 actions	GPU serving
TransAct V2 [11]	Pinterest 2025	Representation	RT + lifelong	~16k actions	103-338x kernel gains
Airbnb [30]	Airbnb 2018	Features	Kafka	~2 weeks of actions	~4.5M listings (approx.)

System	Org / year	Freshness layer	Stream stack	Window / latency	Scale (public)
Chronon [32]	Airbnb/Stripe	Features	Kafka + Flink tiles	ms-s lag	-33% serving latency
Monolith [1]	ByteDance 2022	Parameters	Kafka + Flink joiner	Minute-level sparse sync	TB-scale tables
PERSIA [47]	Kuaishou 2022	Parameters	Async sparse PS	n/a	Up to 100T params
TWIN [46]	Kuaishou 2023	Representation	Cached item keys	~10^4 behaviours	99.3% compute cut

---

37.11 Key takeaways for system designers

1. **Decide which clock you are shortening.** Feature, candidate, representation and parameter freshness have different costs. Many products (Airbnb [30], Pixie [15]) get most of the value from fresh features or fresh query seeds without online training.
2. **Build one canonical action stream first.** A deduplicated Kafka stream with stable request and impression IDs (UUA [22], Recos-Injector [21]) is the foundation for features, graphs and label joins.
3. **Log features at serving time and join by request ID.** This is how Facebook, Photon and Monolith make streaming labels trustworthy [1][3][4]. Make the join idempotent.
4. **Declare features once and compile to batch and stream.** Twitter's AggregateGroup and Chronon's GroupBy are the templates [24][33]. Pre-aggregate into tiles on the write path for hot keys [32].
5. **Use decayed counters and bounded windows.** Exponential half-lives give O(1) state per key [24]. Sliding-window in-memory graphs (GraphJet, UTEG, Thunder) keep fresh candidates cheap [18][20][26].
6. **Put a real-time sequence encoder in the ranker, next to a batch user embedding.** TransAct plus PinnerFormer is the reference hybrid [9][13]. Always include the most recent actions even when you also search a lifelong history [11][45].
7. **Train the stable component to be robust to staleness.** PinnerFormer's dense all-action loss shrank the real-time-vs-daily gap from 13.9% to 8.3% [13].
8. **For online training, sync sparse fast and dense slow.** Use collisionless dynamic tables with admission and expiry for churning ID spaces [1][47].
9. **Handle delayed labels explicitly.** Prefer fake-negative ingestion with FN-weighted or calibrated loss, or elapsed-time sampling, over long waiting windows [5][6][7].
10. **Guard stability.** Use time-window masking or recency dropout to avoid over-reaction [9], diversity rules on the final slate [51], stream anomaly alarms with trainer cutoff [3], and a batch fallback path [1][27].
11. **Respect simple session baselines.** Session kNN and co-visitation counts remain strong for short anonymous sessions [39][49]. Fix losses and negative sampling before adding architecture [42].
12. **Budget the serving cost of freshness.** Real-time encoders can multiply latency (20x at Pinterest) and may require GPUs and fused kernels to ship [10][11].

References

1. Zhuoran Liu et al. (ByteDance). "Monolith: Real Time Recommendation System With Collisionless Embedding Table." arXiv 2209.07663, 2022. https://arxiv.org/abs/2209.07663
2. ByteDance. "Monolith: A Lightweight Recommendation System" (GitHub repository, archived October 2025). Accessed 2026. https://github.com/bytedance/monolith
3. He, X., Pan, J., Jin, O., et al. (Facebook). "Practical Lessons from Predicting Clicks on Ads at Facebook." ADKDD 2014. https://dl.acm.org/doi/10.1145/2648584.2648589
4. Ananthanarayanan, R., et al. (Google). "Photon: Fault-tolerant and Scalable Joining of Continuous Data Streams." SIGMOD 2013. https://research.google/pubs/photon-fault-tolerant-and-scalable-joining-of-continuous-data-streams/
5. Ktena, S. I., Tejani, A., Theis, L., et al. (Twitter). "Addressing Delayed Feedback for Continuous Training with Neural Networks in CTR Prediction." RecSys 2019. https://arxiv.org/abs/1907.06558
6. Yang, J.-Q., et al. (Alibaba). "Capturing Delayed Feedback in Conversion Rate Prediction via Elapsed-Time Sampling." AAAI 2021. https://arxiv.org/abs/2012.03245
7. Gu, S., et al. (Alibaba). "Real Negatives Matter: Continuous Training with Real Negatives for Delayed Feedback Modeling." KDD 2021. https://arxiv.org/abs/2104.14121
8. Chapelle, O. (Criteo). "Modeling Delayed Feedback in Display Advertising." KDD 2014. https://dl.acm.org/doi/10.1145/2623330.2623634
9. Xia, X., et al. (Pinterest). "TransAct: Transformer-based Realtime User Action Model for Recommendation at Pinterest." KDD 2023. https://arxiv.org/abs/2306.00248
10. Pinterest Engineering. "How Pinterest Leverages Realtime User Actions in Recommendation to Boost Homefeed Engagement Volume." Pinterest Engineering Blog, 2023. https://medium.com/pinterest-engineering/how-pinterest-leverages-realtime-user-actions-in-recommendation-to-boost-homefeed-engagement-volume-165ae2e8cde8
11. Pinterest. "TransAct V2: Lifelong User Action Sequence Modeling on Pinterest Recommendation." 2025. https://arxiv.org/abs/2506.02267
12. Pinterest Engineering. "Next-Level Personalization: How 16k+ Lifelong User Actions Supercharge Pinterest's Recommendations." Pinterest Engineering Blog, 2025. https://medium.com/pinterest-engineering/next-level-personalization-how-16k-lifelong-user-actions-supercharge-pinterests-recommendations-bd5989f8f5d3
13. Pancha, N., Zhai, A., Leskovec, J., Rosenberg, C. (Pinterest). "PinnerFormer: Sequence Modeling for User Representation at Pinterest." KDD 2022. https://arxiv.org/abs/2205.04507
14. Pal, A., Eksombatchai, C., Zhou, Y., Zhao, B., Rosenberg, C., Leskovec, J. (Pinterest). "PinnerSage: Multi-Modal User Embedding Framework for Recommendations at Pinterest." KDD 2020. https://arxiv.org/abs/2007.03634
15. Eksombatchai, C., Jindal, P., Liu, J. Z., et al. (Pinterest / Stanford). "Pixie: A System for Recommending 3+ Billion Items to 200+ Million Users in Real-Time." WWW 2018. https://arxiv.org/abs/1711.07601
16. Pinterest Engineering. "Manas HNSW Realtime: Powering Realtime Embedding-Based Retrieval." Pinterest Engineering Blog. https://medium.com/pinterest-engineering/manas-hnsw-realtime-powering-realtime-embedding-based-retrieval-dc71dfd6afdd

17. Koh, T. (Pinterest). "Manas HNSW Streaming Filters." Pinterest Engineering Blog, 2022. https://medium.com/pinterest-engineering/manas-hnsw-streaming-filters-351adf9ac1c4
18. Sharma, A., Jiang, J., Bommannavar, P., Larson, B., Lin, J. (Twitter). "GraphJet: Real-Time Content Recommendations at Twitter." PVLDB 9(13), 2016. http://www.vldb.org/pvldb/vol9/p1281-sharma.pdf
19. Twitter. "GraphJet" README. GitHub, 2016. https://github.com/twitter/GraphJet
20. Twitter/X. "UserTweetEntityGraph (UTEG)" README, the-algorithm, 2023. https://github.com/twitter/the-algorithm/blob/main/src/scala/com/twitter/recos/user_tweet_entity_graph/README.md
21. Twitter/X. "Recos-Injector" README, the-algorithm, 2023. https://github.com/twitter/the-algorithm/blob/main/recos-injector/README.md
22. Twitter/X. "Unified User Actions (UUA)" README, the-algorithm, 2023. https://github.com/twitter/the-algorithm/blob/main/unified_user_actions/README.md
23. Twitter/X. "User Signal Service" README, the-algorithm, 2023. https://github.com/twitter/the-algorithm/blob/main/user-signal-service/README.md
24. Twitter/X. Aggregation framework docs (aggregation.rst, batch.rst, real-time.rst, joining.rst), the-algorithm, 2023. https://github.com/twitter/the-algorithm/tree/main/timelines/data_processing/ml_util/aggregation_framework/docs
25. Twitter Engineering. "Twitter's Recommendation Algorithm." Twitter (X) Engineering Blog, 31 March 2023. https://blog.x.com/engineering/en_us/topics/open-source/2023/twitter-recommendation-algorithm
26. xAI / X. "Thunder" source (in-memory post store), x-algorithm repository, 2026. https://github.com/xai-org/x-algorithm/tree/main/thunder
27. Amatriain, X., Basilico, J. (Netflix). "System Architectures for Personalization and Recommendation." Netflix Technology Blog, 2013. https://netflixtechblog.com/system-architectures-for-personalization-and-recommendation-e081aa94b5d8
28. Amatriain, X. "Blueprints for recommender system architectures: 10th anniversary edition." Machine Learning Times, 2023. https://www.predictiveanalyticsworld.com/machinelearningtimes/blueprints-for-recommender-system-architectures-10th-anniversary-edition/12879/
29. Bhattacharya, M., Lamkhede, S., et al. (Netflix). "Augmenting Netflix Search with In-Session Adapted Recommendations." RecSys 2022 (identifier not re-verified). https://arxiv.org/abs/2206.02254
30. Grbovic, M., Cheng, H. (Airbnb). "Real-time Personalization using Embeddings for Search Ranking at Airbnb." KDD 2018. https://dl.acm.org/doi/10.1145/3219819.3219885
31. Airbnb Engineering. "Listing Embeddings for Similar Listing Recommendations and Real-time Personalization in Search." Airbnb Engineering Blog, 2018. https://medium.com/airbnb-engineering/listing-embeddings-for-similar-listing-recommendations-and-real-time-personalization-in-search-601172f7603e
32. Airbnb / Stripe, Chronon project. "The Tiled Architecture." 2024-2025. https://github.com/airbnb/chronon/blob/main/docs/source/Tiled_Architecture.md
33. Airbnb, Chronon project. "GroupBy" documentation. 2024. https://github.com/airbnb/chronon/blob/main/docs/source/authoring_features/GroupBy.md
34. Airbnb, Chronon project. "Online Offline Consistency." 2024. https://github.com/airbnb/chronon/blob/main/docs/source/test_deploy_serve/Online_Offline_Consistency.md

35. Airbnb, Chronon project. "Chronon on Flink." 2024-2025. https://github.com/airbnb/chronon/blob/main/docs/source/setup/Flink.md
36. DoorDash Engineering. "Building Riviera: A Declarative Real-Time Feature Engineering Framework." DoorDash Engineering Blog, 2021. https://doordash.engineering/2021/03/04/building-a-declarative-real-time-feature-engineering-framework/
37. Vorotilov, V., Shugaepov, I. (Meta). "Scaling the Instagram Explore recommendations system." Engineering at Meta, 2023. https://engineering.fb.com/2023/08/09/ml-applications/scaling-instagram-explore-recommendations-system/
38. Hidasi, B., Karatzoglou, A., Baltrunas, L., Tikk, D. "Session-based Recommendations with Recurrent Neural Networks" (GRU4Rec). ICLR 2016. https://arxiv.org/abs/1511.06939
39. Jannach, D., Ludewig, M. "When Recurrent Neural Networks meet the Neighborhood for Session-Based Recommendation." RecSys 2017. https://web-ainf.aau.at/pub/jannach/files/Conference_RecSys_2017.pdf
40. Kang, W.-C., McAuley, J. (UC San Diego). "Self-Attentive Sequential Recommendation." ICDM 2018. https://arxiv.org/abs/1808.09781
41. Sun, F., et al. (Alibaba). "BERT4Rec: Sequential Recommendation with Bidirectional Encoder Representations from Transformer." CIKM 2019. https://arxiv.org/abs/1904.06690
42. Petrov, A., Macdonald, C. (University of Glasgow). "gSASRec: Reducing Overconfidence in Sequential Recommendation Trained with Negative Sampling." RecSys 2023. https://arxiv.org/abs/2308.07192
43. Moreira, G. de S. P., et al. (NVIDIA). "Transformers4Rec: Bridging the Gap between NLP and Sequential / Session-Based Recommendation." RecSys 2021; repository. https://github.com/NVIDIA-Merlin/Transformers4Rec
44. Quadrana, M., Karatzoglou, A., Hidasi, B., Cremonesi, P. "Personalizing Session-based Recommendations with Hierarchical Recurrent Neural Networks." RecSys 2017. https://arxiv.org/abs/1706.04148
45. Pi, Q., Zhu, X., Zhou, G., et al. (Alibaba). "Search-based User Interest Modeling with Lifelong Sequential Behavior Data for Click-Through Rate Prediction" (SIM). CIKM 2020. https://arxiv.org/abs/2006.05639
46. Chang, J., et al. (Kuaishou). "TWIN: TWo-stage Interest Network for Lifelong User Behavior Modeling in CTR Prediction at Kuaishou." KDD 2023. https://arxiv.org/abs/2302.02352
47. Lian, X., et al. (Kuaishou and ETH Zurich). "Persia: A Hybrid System Scaling Deep Learning Based Recommenders up to 100 Trillion Parameters." KDD 2022. https://arxiv.org/abs/2111.05897
48. Zhai, J., et al. (Meta). "Actions Speak Louder than Words: Trillion-Parameter Sequential Transducers for Generative Recommendations" (HSTU). ICML 2024. https://arxiv.org/abs/2402.17152
49. Das, A., Datar, M., Garg, A., Rajaram, S. (Google). "Google News Personalization: Scalable Online Collaborative Filtering." WWW 2007. https://dl.acm.org/doi/10.1145/1242572.1242610
50. Li, L., Chu, W., Langford, J., Schapire, R. E. (Yahoo! Labs). "A Contextual-Bandit Approach to Personalized News Article Recommendation." WWW 2010. https://arxiv.org/abs/1003.0146
51. TikTok. "How TikTok recommends videos #ForYou." TikTok Newsroom, June 2020. https://newsroom.tiktok.com/en-us/how-tiktok-recommends-videos-for-you

52. Cao, et al. (Kuaishou). "Pantheon: Personalized Multi-objective Ensemble Sort via Iterative Pareto Policy Optimization." arXiv 2505.13894, 2025. https://arxiv.org/abs/2505.13894